

Interview Preparation Pack

Java · Spring Boot · Angular · MongoDB · AWS
Full Stack Developer

Priyanka Ramachandran

47 documents

Built 3 September 2026

Priyanka R — Interview Preparation Pack

A complete, resume-tailored interview study kit for a **Full Stack Developer (Angular + Java/Spring Boot, 5 years)** — covering technical + managerial + HR rounds.

Prepared from your resume: Angular, Ionic, RxJS, NgRx, Angular Formly, TypeScript, JavaScript, HTML/CSS, Java, Spring Boot, Microservices, AWS (basics), MongoDB/MySQL/Postgres/Neo4j — and your projects **RoboGebra**, **Subsea**, **Backoffice Migration**, **EasyVisa**.

● **NEW — "Easy Version" rewrite in progress (Sept 2026).** The Java files are being rewritten in a simple-words, easy-to-memorise format. Every question now follows the same shape: **Easiest way to remember** → **simple explanation** → **real-world example** → **ASCII diagram** → **interview trap** → **interview-ready answer** → **Easy memory box**, and each file ends with a **Quick Revision Sheet** to read the night before.

✓ **All Java files converted:** **05 Core Java** · **06 Spring Boot** · **12 Java 17** · **17 SOLID & Design Patterns** · **22 Java Streams** · **23 Predict the Output** · **24 ORM/JPA/Hibernate** · **31 HashMap Internals** · **32 Multithreading** · **33 Interface vs Abstract Class**.

✓ **ALL FILES DONE — plus 2 new ones.** Every file in the pack has been updated (except 27/28/29 .NET, excluded by request), and two new files were added: **39 RoboGebra Code Examples** and **40 Java 21 Features**.

NEW — 40 Java 21. "And what's new in Java 21?" is now the standard follow-up to every Java 17 question, and **virtual threads** come up constantly. Covers the four finalised features (**virtual threads** ★★, pattern matching for **switch**, record patterns, **sequenced collections** — which weren't documented anywhere in the pack), the preview ones, and the honest script: "we're on 17 — what I'd migrate for is virtual threads for our push fan-out, and the risk is **synchronized** pinning in hot paths." **LTS line:** 8 → 11 → 17 → 21 → 25.

● **Last fixes:** **37 Altimetrik** still said "result pending" while **26** said not cleared — now consistent, plus a **"what a DEPTH round actually asks"** page (anyone can drive a car; this round asked what happens when you press the clutch). **14 Berribot** said 4.8 years and called MongoDB a gap — both corrected. **16**

Codeboard now flags that it's the best **rehearsal template** for the IQVIA-shaped round. **19 JS Variables** gained the three-picture frame (rooms · guest list · fixed address).

STUDY-PLAN rewritten for **IQVIA** (it was still July/Berribot and mentioned Wipro). 7 days, front-loading **Ionic — your differentiator**, with the four "do not claim" items on the interview-day checklist. **MongoDB flashcards** — the old card said *"my production DB experience is MySQL/Postgres/Neo4j"*; replaced with the confident version plus **9 new cards from your own code** (N+1, @DBRef cost, index prefix rule, Mongoose hooks).

◆ **38 IQVIA** now has a **"YOUR EVIDENCE" table** — a real file from your own codebase for **every one of their 10 areas**, plus an **honesty guard on signals** (you have 0 usages) and your three rehearsed stories: the **Ionic mobile app** (it is the job description), the **N+1 aggregation fix**, and the **Angular 16 migration**. **18 re-audited 3 Sept** — every version confirmed unchanged, counts refreshed, and the seven "do not claim" items proven by count.

● **THE THREE THINGS TO NEVER GET WRONG** (now enforced across every converted file): **(1)** RoboGebra is **MongoDB**, not MySQL. **(2)** It is a **modular monolith** — no Eureka, Feign, circuit breaker or Kafka. **(3)** **NgRx and Formly are EasyVisa**, not RoboGebra — and **signals/standalone are not adopted** (0 and 2 usages). Each of those is a one-follow-up trap; the honest version is the stronger answer every time.

▲ **11 HR** was **hardcoded to Wipro** — every mention is now **[COMPANY]**, and Q2 ("why us") and Q22 ("what do you know about us") now have a **fill-in template plus a worked IQVIA example**. Those two answers must be rewritten for every interview; a generic answer there undoes a good technical round.

● **TWO MORE POSITIONING FIXES. (1)** **07 AWS** treated AWS as theory — but RoboGebra runs on it: **S3 with presigned URLs, Cognito with three user pools + JWKS validation, LocalStack** for local dev, and **Jenkins CI/CD**. Say *"I use S3 and Cognito in production"*, not *"I know AWS basics"*. **(2)** **08 Microservices** — your backend has **no Eureka, no Feign, no circuit breaker, no Kafka**. It is a **modular monolith**. Claiming microservices dies on the first follow-up; the honest "monolith first, and here's why" answer is **stronger**.

● **RESUME SYNC (Sept 2026)** — **stale facts corrected in 7 files**. Checked against **recent-resumes/Updated-Priyanka-R-Resume.docx** :

(1) experience is **5 years**, not "4+" or "over four" — fixed in 00, 11, 29, 30, 35 and this README. (2) RoboGebra's database is **MongoDB**, not MySQL — fixed in 10 and 30 (00 and 18 were already right). (3) the closing line said **"...which is why I'm excited about Wipro"** — now a **[COMPANY]** blank you fill per interview. (4) the intro now leads with **"I built the Ionic mobile app for iOS and Android"**, which is on your new resume and is your strongest single line.

▲ **Two factual fixes made from your real code:** (1) **13 MongoDB** said Mongo was a *gap* on your resume — it isn't; RoboGebra runs on it (**104 @Document** , **85 repositories**, **27 @Aggregation pipelines**, plus Mongoose in the CRM). Stop apologising for it. (2) **15 Ionic** listed camera/geolocation plugins you don't use — replaced with your real **19 Capacitor plugins**.

● **NEW — 39 — RoboGebra Code Examples.** Every interview topic mapped to **real code in your own workspace** (`/Users/safi/workspace/robogebra-workspace`) — the **AsyncConfig** thread pool, the **@Aggregation** N+1 fix, the auth interceptor, the role-based route guard, the **BehaviorSubject** state services, the 19 Capacitor plugins. Converted topic files now carry a ● **In your RoboGebra code** block linking here. **This is what turns a textbook answer into your answer.**

● **NEW — 41 Capgemini — the round you just attended (Sept 2026), all 10 questions answered.** Java 8 features · HashMap vs LinkedHashMap · constructor vs setter injection · **@Component** vs **@Bean** · how microservices load values · monolith vs microservices · Angular components · **ng g c** · **ngOnChanges** vs **ngDoCheck** — and **the program you didn't get to write:** *second non-repeating character from the end of "Priyanka"* → **n**, solved in the **simplest 3-step way** — count each character → keep the ones that come once → take **size - 2** — explained line by line and verified to run. **7 of the 10 were repeats**, and two of them had been asked at Altimetrik four days earlier — the logged question bank is paying off. Round log updated in **26** (now **144 questions across 9 companies**).

★ **NEW — 42 Java Coding + Streams: Final Easy-Memory Sheet.** All **32 questions on one page** — Question → Logic Snippet → What to Think → ♦ Easy Memory — then the snippet for each one. **#1–#3 use the normal Java loop**, the rest use the shortest useful stream logic. Ends with the **FINAL MEMORY block** and **the 9 families** that collapse 32 questions into nine ideas. Read this last before a Java round.

● **NEXT INTERVIEW — IQVIA · Angular / Ionic Technical Lead · coming days.**
Go straight to 38 — IQVIA Full Prep Pack. Built directly from the 10 areas they told you to be comfortable with: Angular (incl. change detection, **Signals**, performance, memory leaks) · TypeScript/JavaScript · RxJS/NgRx · **Ionic + Capacitor** · REST + Security · MongoDB + SQL · Microservices · Git/CI-CD/AWS · **Technical Lead questions (STAR) · AI-assisted development.** Plus a 5-day plan, a one-page cheat sheet and the questions to ask them.

✗ **Altimetrik (31 Aug, 2nd attempt) — not cleared.** All 22 questions are answered in 37: it opened on **record**, then pattern matching, then ten Spring-wiring questions.

● **Result pending: Deloitte (USI) — Fullstack Java + Angular, 25 Aug 2026.**
Prep and the 25 ranked questions: 35.

● **Result pending: Tech Mahindra — Angular Developer.** All 15 questions they asked are answered in 34 — **Tech Mahindra Angular Round.**

◆ **Virtusa Round 2 (not cleared)** drilled Java core: HashMap internals → 31, multithreading → 32, interface vs abstract class → 33.

● **Mphasis L1 cleared. Level 2 + client technical round scheduled → 30 — Mphasis L2 + Client Round.** Start with Part A (the re-probes of the 10 questions L1 already asked) and Part C (multithreading — the thread they've already opened twice).

SQL now comes up in every round — Deloitte's rubric has a SQL row and Mphasis L1 opened the thread. Everything is in one place: 36 — **SQL: The Complete Interview File** — starting with the two questions you were actually asked (headcount + average salary per department, second-highest salary), then joins, **GROUP BY** vs **HAVING**, the whole salary-question family, window functions, NULL traps, indexing and "make this query faster", the **join row-count puzzle** (**A = 1,1,1,1** joined to **B = 1,1,1,1** → **16 rows, for every join type**), plus 23 predict-the-output programs and 18 drills.

Before ANY interview, read 26 — Companies: Questions Actually Asked. It logs every question really asked at **Virtusa (assessment + Round 2)**, **Infosys**, **Altimetrik (2 rounds + the 31 Aug fullstack round)**, **Photon (2 rounds)**, **Codeboard**, **Mphasis**, **Tech Mahindra** and **Deloitte** — **134 questions** with answers. The same ~25 topics repeat across all of them (**lifecycle hooks have**

now come up in 5 separate rounds, interceptors in 4). Add every new round to it the same day.

▲ **Lesson from the last interview (Aug 2026):** the Java round was **output-based programs**, not theory — "*what does this print?*". That's now covered in **23 — Java "Predict the Output"**, which opens with the two exact programs you were given (overloading with `null`, and an override that adds `throws IOException`) and adds 58 more of the same type. **Do this file before any Java round from now on.**

● **Interview scheduled: Codeboard Technology — "Java with Angular Ionic — Level 1 — Virtual Interview".** HR confirmed this is the **ONLY** round — clear it and you go straight to onboarding. Go to **16 — Codeboard Technology Level-1 Guide** first, then **15 — Ionic Level 1**. The guide covers the 60–90 min single-round time split, 25 most-likely questions, self-intro script, RoboGebra walkthrough, how to *close* the call, and the onboarding/CTC/documents checklist.

◆ **Targeting the Berribot assessment?** Start with **14 — Berribot Assessment Guide** (Senior Full Stack: Java 17, Spring Boot, Angular 16, MongoDB, MySQL, Microservices, AWS S3). It maps the whole stack to these files, flags your gaps (MongoDB & Java 17), and gives a focused 5-day plan.

Contents

#	File	What's inside
00	Self Introduction	Ready-to-memorize "Tell me about yourself" (30s / 60s / 2-min) + cheat sheet
01	JavaScript	Closures, async/event loop, ES6+, debounce/throttle, output-based Qs
02	HTML & CSS	Semantics, box model, Flexbox/Grid, responsive, specificity

#	File	What's inside
03	TypeScript	Types vs interfaces, generics, utility types, decorators
04	Angular	Your core skill — lifecycle, RxJS, NgRx, forms, DI, routing, change detection
05	Core Java	OOP, collections, HashMap internals, Java 8 streams, multithreading
06	Spring & Spring Boot	IoC/DI, REST, JPA, auto-config, exception handling, security basics
07	AWS (Basics)	EC2, S3, Lambda, RDS, CloudWatch — fundamentals
08	Microservices (Basics)	Monolith vs micro, Spring Cloud, patterns, communication
09	Coding Problems	35+ problems with JS & Java solutions + complexity
10	Projects Deep-Dive	Tough "project difficulty" Q&A on your 4 real projects
11	HR & Behavioral	Why join, strengths/weaknesses, STAR answers, salary, notice
12	Java 17 Features	★ Records, sealed classes, pattern matching, switch expressions, text blocks
13	MongoDB	★ Document model, CRUD, aggregation, indexing, Spring Data MongoDB (<i>your gap</i>)
14	Berribot Assessment Guide	◆ Stack map, gap analysis, full-stack S3 scenario, 5-day plan

#	File	What's inside
15	Ionic — Level 1	★ 43 Q&A — Ionic vs Angular, components, page lifecycle, Capacitor, theming, build & deploy + RoboGebra answers
16	Codeboard Technology — Level 1 Guide	● SINGLE ROUND → onboarding. Time split, 25 likely Java/Spring/Angular/Ionic Qs, self-intro, project walkthrough, closing script, CTC + documents checklist, 2-day plan
17	SOLID & Design Patterns	★ All 5 SOLID principles with bad→good code, GoF creational/structural/behavioural patterns, patterns in JDK + Spring + Angular, "spot the violation" scenarios, rapid-fire table
18	RoboGebra — Technical Stack & Versions	★ Real versions read from your actual repos — Java 17, Spring Boot 3.2.0, Angular 16.2/18.2, Ionic 8.7.5, Capacitor 6.2.1, MongoDB, React/Vite CRM. 8-repo architecture, master version table, 10 traps in your own stack, answer scripts
19	JS Variables Made Easy	★ Beginner-friendly deep dive on <code>var</code> / <code>let</code> / <code>const</code> — scope, hoisting, TDZ explained step by step, the famous loop problem with 3 fixes, 5 real Angular bugs, 8 practice problems with hidden answers, 18 output drills, cheat sheet

#	File	What's inside
20	RxJS & Operators	★ Full operator guide — Observables, all 4 Subjects, creation/transformation/filtering/combination/error/utility operators with ASCII marble diagrams, switchMap vs mergeMap vs concatMap vs exhaustMap, forkJoin vs combineLatest vs zip, memory leaks & unsubscribe patterns, 7 real Angular pipelines, 9 anti-patterns, RxJS 7 changes, 8 drills
21	NgRx	★ Store, Actions, Reducers, Selectors, Effects — data-flow diagram, memoization, effect operator choice, @ngrx/entity, facade pattern, full EasyVisa worked example with tests, ComponentStore & SignalStore, when NOT to use NgRx, 9 anti-patterns. ▲ Includes your honesty position: NgRx = EasyVisa, not RoboGebra
22	Java Streams — 20 Coding Problems	★ The 20 most-asked Stream problems with answers — char count, first non-repeated, duplicates, groupingBy, partitioningBy, toMap, flatMap, anagrams, second highest + bonus problems, API quick reference, 12 traps

#	File	What's inside
23	Java "Predict the Output" — 60 Tricky Programs	● THE GAP THAT COST YOU THE LAST INTERVIEW. Starts with the exact 2 programs you were asked (<code>methodTest(null)</code> overloading, override + <code>IOException</code>) fully broken down with follow-ups, then 58 more: overloading resolution, static/field hiding, init order, String pool, Integer cache, <code>i = i++</code> , <code>finally</code> vs <code>return</code> , <code>remove(int)</code> vs <code>remove(Object)</code> , CME, threads + rapid-fire table + 20 hidden-answer drills

#	File	What's inside
24	ORM — JPA, Hibernate & ODM	★ "Have you used ORM?" answered honestly. Opens with your real position — RoboGebra is MongoDB = ODM, not ORM — then full JPA depth: entity mapping, the 4 entity states + persistence context + dirty checking, <code>persist</code> / <code>merge</code> / <code>save</code>, relationships & owning side, N+1 with 4 fixes, <code>LazyInitializationException</code>, <code>@Transactional</code> (propagation, rollback rules, self-invocation trap), L1/L2 caching, optimistic vs pessimistic locking, the Lombok <code>@Data</code> entity disaster, 20 predict-the-output ORM programs, Spring Data MongoDB + Mongoose mapping, ORM vs ODM table, rapid-fire + 12 drills + 6 answer scripts

#	File	What's inside
25	Angular — Data Binding & Forms	★ The two things every Angular round drills. All 4 binding types, property vs <code>[attr.]</code>, how <code>[(x)]</code> desugars + custom two-way binding, <code>@Input</code> setters vs <code>ngOnChanges</code>, <code>ExpressionChangedAfterItHasBeenCheckedError</code>; then reactive forms in full — TD vs Reactive table, <code>FormControl</code> / <code>FormGroup</code> / <code>FormArray</code> / <code>FormBuilder</code>, typed forms + <code>nonNullable</code>, <code>setValue</code> vs <code>patchValue</code>, <code>value</code> vs <code>getRawValue()</code>, control state, custom/cross-field/async validators, <code>FormArray</code> dynamic rows, Formly (EasyVisa), <code>ControlValueAccessor</code> full implementation, 18 "what happens?" traps, testing, rapid-fire + 12 drills + 6 answer scripts

#	File	What's inside
27	C# / .NET "Predict the Output" — 75 Programs	★ The .NET twin of file 23. Opens with Section A — the 8 programs where C# prints something different from Java (non-virtual by default, derived field initializers before the base ctor, no <code>return</code> in <code>finally</code> , structs copy, constants overflow at compile time, <code>for</code> vs <code>foreach</code> capture, optional-parameter defaults, <code>dynamic</code>), then 67 more: boxing & defensive struct copies, <code>override</code> / <code>new</code> / interface re-implementation, init order, string interning, <code>checked</code> / <code>unchecked</code> , lifted nullable operators that are <i>both</i> false, exception filters & <code>throw</code> vs <code>throw ex</code> , deferred LINQ, <code>async</code> ordering / <code>.Result</code> deadlock / <code>async void</code> , <code>record</code> equality + rapid-fire table + 20 hidden-answer drills

#	File	What's inside
28	ASP.NET MVC — 62 Tricky "What Happens?" Programs	★ Output-based MVC. Core-first with ** In MVC 5 contrast boxes throughout. Routing & action selection, model-binding source priority + over-posting, [Required] on an int , ViewBag/ViewData/ TempData lifetimes, PRG & Response.Redirect , full filter order + what exception filters miss, ▲ the TextBoxFor -reads- ModelState-not-your-model trap, Razor @ parsing & layout execution order, per-request controllers, DI lifetimes & captive dependencies, middleware/ UseRouting order + rapid-fire table, 18 hidden-answer drills, and a Spring→ASP.NET Core mapping table** so your Java experience counts

#	File	What's inside
29	ASP.NET MVC — 72 Interview Questions	★ The spoken Q&A round — do this BEFORE file 28. Opens with \$0 the honesty position: how to answer "how much .NET have you worked on?" — lead with Subsea, your real .NET Web API project, with the accurate boundary (Web API, not Razor MVC), plus a Spring→ASP.NET Core mapping table (HandlerInterceptor →action filter, @ControllerAdvice →exception filter, servlet Filter →middleware). Then the ★15 most-likely questions, MVC fundamentals, the full request lifecycle, routing, action results, Razor/layouts/view components, ViewBag vs ViewData vs TempData, model binding & validation, the 5 filter types & their order, DI lifetimes & middleware order, CSRF/XSS/CORS/auth, EF Core + N+1 + caching + testing — with ** In MVC 5** contrast boxes throughout, 4 answer scripts (self-intro for a .NET role, project walkthrough translated, "I don't know", questions to ask them) and a 72-row rapid-fire table

● Mphasis — Level 2 + Client
Technical Round

● **ACTIVE** — you cleared **Mphasis L1 (6 Aug 2026) and L2 + a client round are scheduled.** HR's brief decoded: "*Java only mostly; Angular = what you used in recent projects; basics not mandatory.*" **Part A is the highest-yield section in the pack** — the expected L2 re-probe of each of the 10 questions L1 already asked (Saga with a real compensation, "is a Spring singleton a Java singleton?", diagnosing a hung production app, "give me a number" on API perf). Then Java core depth (HashMap internals, collections, immutability, Streams), concurrency beyond deadlock (`ExecutorService`, `CompletableFuture`, `volatile` vs atomic), Spring depth (bean lifecycle, the 3 `@Transactional` traps, auto-configuration, REST design), JPA (N+1, `LazyInitializationException`, locking), microservices (Feign vs WebClient, gateway, JWT), **a full SQL section for the gap that opened at L1**, live-coding list, a system-design skeleton rehearsed on *your own* reminder feature, **Angular as 4 project narrations rather than theory**, and the **client**

#	File	What's inside
		technical round — a different round with a different objective, incl. red flags and the questions you ask them. Ends with a 48-hour plan and a one-page cheat sheet
31	● HashMap Internals	★ The exact Virtusa Round-2 questions. <code>Node[] table</code>, the <code>h ^ (h>>>16)</code> spread, <code>(n-1) & hash</code>, what a bucket actually is, the internal structure of a <code>Node</code>, linked list → red-black tree at 8 (table ≥ 64), untreeify at 6 and why the gap exists, load factor 0.75 + the lo/hi resize split, <code>equals</code> / <code>hashCode</code> contract + the mutable-key leak, Java 7 vs 8 and the famous infinite loop, HashMap vs Hashtable vs LinkedHashMap vs TreeMap vs ConcurrentHashMap, 8 predict-the-output programs, 20-row rapid-fire, 7 drills

#	File	What's inside
32	● Multithreading & Concurrency	★ The full concurrency file. Atomicity/visibility/ordering as three separate problems, thread creation + lifecycle, <code>start()</code> vs <code>run()</code>, <code>sleep</code> vs <code>wait</code>, race conditions with 3 fixes, <code>synchronized</code> vs <code>volatile</code> vs atomics + CAS, <code>ReentrantLock</code> / <code>ReadWriteLock</code> /latches/ semaphores, wait-notify & <code>BlockingQueue</code>, deadlock: 4 conditions, prevention by lock ordering, detection with <code>jstack</code>, <code>ExecutorService</code> sizing & why <code>Executors</code> factories are a trap, <code>CompletableFuture</code>, concurrent collections, <code>ThreadLocal</code> leaks, thread-safe singleton 5 ways + why DCL needs <code>volatile</code>, live-coding programs, <code>@Async</code> traps in Spring, virtual threads, 8 output programs, 28-row rapid-fire

#	File	What's inside
33	● Interface vs Abstract Class	★ Asked at Virtusa R2. The 60-second answer, the full comparison table, what Java 8/9 changed (<code>default</code> , <code>static</code> , <code>private</code> methods — the pre-2014 answer is now a wrong answer), abstract classes in depth (constructors, no-abstract-method classes, the private/static/final contradictions), when to choose which + template method, the diamond problem & <code>A.super.m()</code> , "class wins", functional & marker interfaces, JDK/ Spring/your-project examples, abstraction vs encapsulation, sealed types, 7 output programs, 19-row rapid-fire, 6 drills, plus the C# differences

#	File	What's inside
34	● Tech Mahindra — Angular Round	● RESULT PENDING — all 15 questions they asked, answered. State management as a 4-level ladder, NgRx + every feature with the effect <code>catchError</code> trap, TypeScript range answer, HTTP interceptors (functional + class-based) and their benefits, string interpolation, Observables vs Promises (4th round running), HttpClient features, callback hell → Promises → async/await → RxJS, event capturing vs bubbling + delegation, ES5 vs ES6, arrow functions & lexical <code>this</code>, OOP in TypeScript (<code>private</code> vs <code>#private</code>, structural typing), modules (ESM/CJS and NgModule + standalone), unknown vs never vs any vs void with the exhaustiveness check, HTML/CSS essentials

#	File	What's inside
35	▲ TOMORROW — Deloitte: Fullstack Java + Angular (25 Aug)	● THE ACTIVE FILE. The requisition decoded (Deloitte USI = rubric-scored breadth, never go silent; EaaS = client-staffed, so they screen for client-readiness; AI&E = your RoboGebra AI engine is directly on-theme — lead with it), Deloitte's full interview process, an hour-by-hour plan for tonight and tomorrow morning, the 25 most-likely questions ranked from your own six-round log , your 60-second opening script, the three habits that decide the round (the second sentence, clean "I don't know", honest scale), 5 questions to ask them, the Deloitte behavioural/STAR layer incl. "Why Deloitte?", a one-page cheat sheet for 14:45 and a night-before checklist

★ **SQL, end to end — the gap that opened at Mphasis L1 and is now on every rubric.** Opens with **the two questions you were actually asked** (total employees + average salary per department; second-highest salary) answered to mid-level depth with the three sentences that upgrade the answer, then a **4-step script for answering any SQL question**. A fixed 3-table schema with deliberate landmines (an empty department, a NULL salary, a NULL `dept_id`, a salary tie) runs through the whole file so every result is predictable. Covers **logical execution order** (the idea behind `WHERE` vs `HAVING`, aliases, and half the traps), all six joins with row-by-row worked output, **`ON` vs `WHERE` in an outer join**, **\$3.6 join cardinality — the "A has 1,1,1,1 and B has 1,1,1,1, how many rows does each join return?" puzzle** (answer: 16 for all five) with the counting formula, six worked cases including NULL keys and an empty table, the `COUNT(*)` vs `COUNT(b, id)` follow-up, the self-join `<` trick, and the real-world `SUM` double-count bug it causes, the three anti-joins, `COUNT(*)` vs

COUNT(col) vs **COUNT(DISTINCT)** ,
GROUP BY / HAVING /
 aggregate NULL behaviour,
the salary questions in every variant (Nth-highest 5 ways with trade-offs, per-department max, top-N per group, above-department-average, salary bands, median), **window functions in full** (**RANK / DENSE_RANK / ROW_NUMBER / NTILE / LAG / LEAD** , frames, the **LAST_VALUE** trap, why they can't go in **WHERE**), **IN** vs **EXISTS** vs **JOIN** , the **NOT IN NULL bomb** and three-valued logic, duplicates (find/delete/prevent), set operations, **CTEs + the recursive employee-hierarchy query**, self joins, dates/strings/ **CASE** + pivoting by conditional aggregation, **indexes & "make this query faster"** (leftmost-prefix, covering, sargability, reading **EXPLAIN**), transactions/ACID/isolation levels/locking/DB deadlock, normalization & keys, **DELETE** vs **TRUNCATE** vs **DROP** , views/procedures/triggers, **SQL from the Spring side** (N+1 and its 4 fixes, JPQL vs native, SQL injection, **@Transactional** traps) — then **23 predict-the-output SQL programs**, a **65-row**

#	File	What's inside
		rapid-fire table, 18 drills with hidden answers , and a one-page cheat sheet

● Altimetrik — Fullstack Java
+ Angular (31 Aug 2026)

★ **The round you attended on 31 Aug — all 22 questions, answered.** Your **second** Altimetrik attempt, and a Spring *depth* round rather than a breadth sweep: **why Spring Boot** (DI · auto-config · starters · embedded server), **Java 17/21 pattern matching** (`instanceof` patterns, `switch` + `when` guards, record patterns, sealed-type exhaustiveness), Spring Boot annotations answered **in groups**, `@Component` vs `@Service` (and why `@Repository` is the only one with real behaviour), **component scanning across multiple packages** (`scanBasePackages` , refactor-safe `scanBasePackageClasses` , filters, `@EntityScan` + `@EnableJpaRepositories` , auto-configuration for a shared jar), `@Bean` vs stereotypes + the bean lifecycle and the `proxyBeanMethods` trap, `@Autowired` / `@Qualifier` / `@Primary` **precedence**, the full `CrudRepository` → `PagingAndSortingRepository` → `JpaRepository` hierarchy, **pagination methods and every class involved** (`Pageable` ,

PageRequest , **Sort** ,
Page / PageImpl , **Slice** ,
Window) with **Page = 2**
queries vs Slice = 1 and
the keyset-paging + **JOIN**
FETCH traps, **microservice**
communication (sync vs
async, Feign/WebClient/
RestClient, gateway,
discovery), **error cascading**
(timeout → circuit breaker →
retry+backoff → bulkhead →
fallback), and the sharpest
question of the round — *two*
services, a network failure,
how do you inform B →
idempotency keys, the
transactional outbox, DLQs
and compensating
transactions; then **dev/QA/**
prod configuration (profiles,
one jar many environments,
config server, secrets, **Flyway**
+ **ddl-auto: validate**),
and the Angular half:
ngOnInit vs constructor
(5th round running),
decorators, **interceptors** (4th
round), **token expiry** →
silent refresh + toast +
returnUrl redirect, with
the refresh-stampede fix,
the five ways to redirect (+
the **pathMatch: 'full'**
infinite-loop trap), **data**
binding, lazy loading and its
disadvantages — closing
with the live-coding **highest-**
paid employee per
department in six variants
(incl. ties, **TreeMap** , pre-

#	File	What's inside
		Java-8 and the SQL <code>RANK()</code> form)

● **THE ACTIVE FILE** — built from the 10 areas IQVIA told you to prepare. Opens with **who IQVIA is** (healthcare/ clinical data → PHI, HIPAA/ GDPR, 21 CFR Part 11 audit trails, offline tablets in clinics) and the three habits that decide the round. **Angular:** components, the full lifecycle order + `ExpressionChangedAfterItHasBeenChecked`, data binding, DI & the injector hierarchy, routing with all five **guard** types + **resolvers** (and why they block navigation), lazy loading + `@defer`, interceptors, reactive forms, decorators, **change detection** (`OnPush` + the immutable-update trap, `runOutsideAngular`), **Signals** (`signal` / `computed` / `effect` / `input` / `model` / `toSignal`, zoneless), **10 performance levers** and **the five sources of memory leaks + how to diagnose them in DevTools**. **TS/JS:** `var/let/const` + the loop-closure output, arrow functions & lexical `this`, the **event loop** (microtask vs macrotask), `Promise.all/allSettled/race/any`, bubbling/capturing + delegation, destructuring/ spread (**shallow copy**), interfaces vs types, OOP +

structural typing, generics + utility types, **any** / **unknown** / **never** / **void** with the exhaustiveness **check**, modules (ESM + NgModule). **RxJS/NgRx**: Observable vs Promise, all four Subject types, hot/cold + the **shareReplay** leak, **switchMap** / **mergeMap** / **concatMap** / **exhaustMap** with the right use case **each** (▲ never **switchMap** a POST), **forkJoin** + isolating one failure, error handling (**catchError** inside the inner pipe), unsubscribe strategies, and the full **Store/Action/Reducer/Effect/Selector** flow with the effect- **catchError** trap and *when not to use NgRx*. **Ionic + Capacitor**: architecture, **ngOnInit** vs **ionViewDidEnter** (the 'list doesn't refresh on back' bug story), navigation/ **NavController** /modals/ hardware back, plugins + the **cap sync** workflow, **permissions**, **storage** chosen for **PHI** (encrypted SQLite + Keychain, never **localStorage**), **offline-first sync** with client-generated **UUIDs** and **conflict policy**, mobile performance. **REST + Security**: methods/ idempotency, the status

codes that matter (**401 vs 403**), authN vs authZ, JWT/ access/refresh + storage trade-offs, the **complete expired-token flow incl. the refresh stampede, CORS** (browser-enforced, preflight, credentials, Capacitor origins), and API error handling with **ProblemDetail**. **Data: WHERE vs HAVING**, execution order, joins + the **COUNT(e.id)** trap, **Nth-highest salary four ways** and **RANK / DENSE_RANK / ROW_NUMBER**, **highest salary per department in SQL and Streams**, subqueries/CTEs, indexes (leftmost prefix, covering, sargability, **EXPLAIN**), ACID + isolation levels, keyset pagination, and MongoDB (CRUD, **aggregation pipeline, embedding vs referencing**, the 16 MB limit, transactions since 4.0). **Microservices**: REST vs messaging, discovery, gateway/BFF, timeout→breaker→retry→bulkhead→fallback with **Resilience4j, Kafka** (partitions, keys, consumer groups, at-least-once, DLQ), **network failures + idempotency keys + the transactional outbox**, Saga with a concrete compensation, tracing (**never**

log PHI), database-per-service. **Git/CI-CD/Cloud:** merge vs rebase + the team policy, branching & conflict prevention, a full **PR/code-review process**, pipeline stages + blue-green/canary/feature flags (and the **GxP approval-gate caveat**), AWS fundamentals + a real deployment topology, and one-artefact-many-environments config (▲ `environment.ts` is public). **Technical Lead:** nine STAR-shaped answers with blanks for your real numbers — code reviews, mentoring, architecture decisions + **ADRs**, estimation and missing a sprint, **production incidents (mitigate first, blameless post-mortem)**, technical debt (**strangler, not rewrite**), disagreements (**timeboxed spike**), stakeholder communication, and the full code-quality mechanism stack. **AI-assisted development:** Copilot vs Cursor vs Claude per use case, where they help and where they're risky, a **9-point review checklist for generated code** (hallucinated APIs, slopsquatting, version drift, edge cases), **PHI/privacy rules and enterprise vs consumer deployment**, testing generated code, and

#	File	What's inside
		two example skeletons to fill in. Ends with a 5-day study plan , a one-page cheat sheet , and 10 questions to ask them
39	● RoboGebra Code Examples	Every topic mapped to real code in your own workspace — thread pools, aggregation pipelines, interceptors, guards, Capacitor plugins
40	Java 21 Features	★ The current LTS. Virtual threads (unmount on I/O, don't pool, the <code>synchronized</code> pinning trap, <code>spring.threads.virtual.enabled=true</code>), pattern matching for <code>switch</code> (type patterns, <code>case null</code>, <code>when</code> guards, and sealed + switch = compiler-checked exhaustiveness), record patterns (destructuring, nesting, algebraic data types), sequenced collections (<code>getFirst</code> / <code>getLast</code> / <code>reversed</code> — the one nobody knows), structured concurrency + scoped values (preview), generational ZGC, the String Templates withdrawal, a Java 17 vs 21 table, and the honest "we're on 17 — here's what I'd migrate for and what it would cost" script

★ **The round you just attended — all 10 questions, answered.** A breadth sweep: **Java 8 features** in five groups (L.S.I.O.D.) with the lazy/single-use/parallel traps, **HashMap vs LinkedHashMap vs TreeMap** (bucket → treeify at 8, load factor 0.75, and the 6-line **LRU cache** with `accessOrder` + `removeEldestEntry`), **constructor vs setter injection** (why fields should be `final`, why field injection is out, and why a circular dependency *should* fail at startup — Boot 2.6+ disallows it), **@Component vs @Bean** (class vs method, who constructs it, the only way to register a third-party class, + the `proxyBeanMethods` trap), **how microservices load values** (`@Value` vs type-safe `@ConfigurationProperties`, the **full precedence order**, relaxed binding, Config Server + `@RefreshScope` + Cloud Bus, ConfigMaps, secrets in Vault), **monolith vs microservices** (the cost table, the *distributed monolith* anti-pattern, and the ● **RoboGebra is a modular monolith** honesty guard),

#	File	What's inside
		Angular components (metadata, the tree, the four communication routes), ng g c and the 4th step everyone forgets (it registers the class in declarations) with the 'app-x' is not a known element triage, ngOnChanges vs ngDoCheck (Angular compares inputs with === — that one fact answers the whole question) — closing with the live-coding problem: ● String name = "Priyanka" → 2nd non-repeating character from the END solved in the simplest possible way — 3 steps with the Stream API: ① count each character (groupingBy + counting() into a LinkedHashMap — that is the whole trick) → ② keep the ones whose count is 1 → ③ take size - 2 — with a line-by-line plain-English table, the reverse() + skip() variant, the Nth version, a no-streams version, and 6 pre-answered follow-ups. Answer: n

#	File	What's inside
42	★ Java Coding + Streams — Final Easy-Memory Sheet	★ The revision sheet — read it in the last ten minutes before a Java round. All 32 most-asked coding questions in one table: Question → Logic Snippet → What to Think → ◆ Easy Memory (<i>BACKWARD</i> → <i>ADD, REVERSE</i> → <i>COMPARE, SORT BOTH</i> → <i>EQUALS, TARGET</i> – <i>CURRENT = NEED, DISTINCT</i> → <i>DESC</i> → <i>SKIP 1</i> → <i>FIRST, !seen.add = DUPLICATE</i>). Then the logic snippet for every one, with the plain-English meaning and an arrow diagram — #1–#3 kept as the normal Java loop (not streams), the non-repeating family solved by <code>indexOf(c) ==</code> <code>lastIndexOf(c)</code> with <code>findFirst</code> → <code>skip(1)</code> → <code>skip(2)</code> for first/second/third, 12 & 17, 13 & 16, 23/24/25, 26 & 27 and 28 & 29 each taught as <i>one family, not separate programs</i>, and #32 broken into its three parts (KEY = department, VALUE = employee, merge = keep higher salary). Closes with the FINAL MEMORY block to revise before the interview and the 9 families that reduce all 32 questions to nine ideas.

#	File	What's inside
26	Companies — Questions Actually Asked	● YOUR RECALL LOG — read before every interview. Every question actually asked across Virtusa · Infosys · Altimetrik · Photon (2 rounds) · Codeboard · Mphasis — 93 questions, all now answered. Track record table, most-repeated questions (lifecycle hooks asked in 4 rounds, component communication in 4), and a pattern analysis of where the rounds are being lost. Append every new round the same day — template at the bottom

Study plans

- **2-Week Plan (Berribot-focused)** — gap-first schedule with dates, weekly checkpoints, and a compressed 1-week track. *Recommended*.
- Quick 7-day version below.

Rapid-recall flashcards (drill your gaps daily)

- **MongoDB flashcards** — 26 cards (▲ = high-yield for Berribot)
- **Java 17 flashcards** — 21 cards (▲ = high-yield for Berribot)

Cover the answer, say it out loud, then check. Clear each set in <10 min as part of your daily habit.

Suggested 7-day study plan

Day	Focus
1	Self-intro (00) + Projects deep-dive (10) — these are asked in every round
2	JavaScript (01) + TypeScript (03)
3	Angular (04) — spend the most time here
4	Core Java (05) + Spring Boot (06)
5	Coding problems (09) — practice writing on paper / whiteboard
6	AWS (07) + Microservices (08) + HTML/CSS (02)
7	HR/behavioral (11) + revise weak areas + mock interview

✓ Interview-day checklist

- [] Can deliver the 60-second intro smoothly without notes
- [] Can draw the architecture of **RoboGebra** and **EasyVisa** on a whiteboard
- [] Have **one strong "challenge I solved"** story ready per project (STAR format)
- [] Can explain RxJS `switchMap` vs `mergeMap` and the `takeUntil` unsubscribe pattern
- [] Can explain Spring Boot auto-configuration and `@SpringBootApplication`
- [] Can code FizzBuzz, palindrome, two-sum, and reverse-a-string from memory
- [] Can answer the **output-based Java programs** in 23 — especially `methodTest(null)` and the `throws IOException` override
- [] Can say the one-liner: "Overloading = compiler decides by the reference type. Overriding = JVM decides by the object type."

- [] Can explain `setValue` vs `patchValue`, `value` vs `getRawValue()`, and write a **cross-field + async validator** (25)
 - [] Can implement `ControlValueAccessor` and desugar `[(ngModel)]` on a whiteboard
 - [] Can answer **"Have you used ORM?"** honestly — *RoboGebra is MongoDB, so it's an ODM, not an ORM* — and still explain JPA concepts (persistence context, N+1, `@Transactional`) confidently (24)
 - [] Have 2–3 thoughtful questions ready to ask the interviewer
 - [] Be honest about **AWS & Microservices = basic/working knowledge** — don't oversell
-

◆ How to use this pack

1. **Read actively** — say answers out loud, don't just skim.
2. **For project questions, replace any generic phrasing with your real details** — interviewers can tell the difference.
3. **Practice the coding problems by typing them yourself**, then check against the solution.
4. **Time your self-introduction** — aim for under 90 seconds.

▲ The technical answers are accurate study material. The project answers are written to match your resume — review them and adjust any specifics (team size, exact metrics) to your real experience before the interview.

Good luck, Priyanka!

Study Plan — IQVIA (Angular / Ionic Technical Lead)

▲ **This plan replaces the old July/Berribot one.** Rewritten **3 Sept 2026** for your actual next interview.

Target: IQVIA — Angular / Ionic **Technical Lead**

Their checklist: ~**75% frontend** (Angular · TS/JS · RxJS/NgRx · Ionic+Capacitor) + REST/Security · MongoDB/SQL · Microservices · Git/CI-CD/AWS · **Tech Lead** · AI-assisted dev

Main file: **38 — IQVIA Full Prep Pack** — everything below feeds into it.

◆ The one thing that decides this round

This role is 75% YOUR STRONGEST GROUND. ★

Altimetrik was lost on Spring depth. IQVIA is Angular + Ionic — which is what you build every day. Do NOT over-prepare backend at the cost of the frontend lead answers. ★

The three habits — apply them to every answer

1. DEFINITION → then ONE thing from YOUR code → then ONE trade-off ★
"OnPush skips checking unless an input reference changes.
We use it on the progress lists — not globally, because on a mature codebase that ships silent bugs wherever an array is mutated."
2. Say the NUMBER, not the adjective ★
✗ "we handle subscriptions carefully"
✓ "over four hundred takeUntil calls" ★
3. When you haven't used it, say the MIGRATION PLAN ★
✗ "yes we use signals"
✓ "studied, not shipped — I'd start with signal inputs on leaf components"

The plan — 7 days (compress or stretch to your window)

Day 1 — Your story + your evidence ★

- [00 Self-Intro] — the 60-second version, out loud, 5 times ★
- [39 RoboGebra Code Examples] — the whole file
- [38 IQVIA] \$"YOUR EVIDENCE" table — one real file per area ★
- Fill in the [COMPANY] blanks: "Why IQVIA?" + "What do you know about IQVIA?"

GOAL: the 60-second intro with NO notes, ending on
"I built the Ionic mobile app for iOS and Android" ★

Day 2 — Angular core (their biggest area)

- [04 Angular] — lifecycle · DI · routing/guards · change detection ★
- [38 IQVIA] Part 1 (all of it — 1.1 → 1.13)
- Re-read the SIGNALS honesty guard ★ (you have 0 usages)

GOAL: lifecycle order cold · OnPush's 4 triggers · 5 sources of memory leaks
Say the guard answer for signals out loud ★

Day 3 — RxJS + state

- [20 RxJS] — the four flattening operators + WHERE catchError goes ★
- [21 NgRx] — the bank-ledger model + "why we chose NOT to use it" ★
- [38 IQVIA] Part 3

GOAL: draw the 4 marble diagrams from memory ★
write the typeahead pipeline (debounceTime → filter → switchMap)
cold ★
say the "BehaviorSubject not NgRx, deliberately" answer ★

Day 4 — Ionic + Capacitor ★ YOUR DIFFERENTIATOR

- [15 Ionic] – lifecycle ★ · CSS variables/Shadow DOM · storage · plugins
- [38 IQVIA] Part 4
- Memorise your 19 plugins and the 3 with a story ★

GOAL: ngOnInit vs ionViewWillEnter, with the STALE LIST bug ★
"Preferences over localStorage – the OS can evict WebView storage"

★

Rehearse the mobile-app story until it's smooth ★ ★

★ **Most Angular candidates cannot answer this section at all. It's on their list. This is where you win.**

Day 5 — TypeScript + REST/Security + data

- [03 TypeScript] – generics ★ · type guards · utility types · strict flags
- [38 IQVIA] Parts 2, 5, 6
- [13 MongoDB] – the N+1 → aggregation story ★
- [36 SQL] – the 3 master ideas + joins + GROUP BY/HAVING

GOAL: explain `PageResponse<T>` + `fromJson<U>` ★
the JWT/Cognito/JWKS answer ★
the N+1 aggregation story, three questions deep ★

Day 6 — Tech Lead + architecture ★

- [38 IQVIA] Part 9 (Technical Lead) + Part 10 (AI-assisted dev) ★
- [10 Projects] – the CAPS frame + your 3 hard-problem stories
- [08 Microservices] – the "modular monolith, monolith first" answer ★
- [07 AWS] – S3 presigned URLs · Cognito · LocalStack · Jenkins ★

GOAL: draw YOUR architecture from memory (4 clients → Spring Boot → MongoDB/S3/Cognito/Firebase/Razorpay) ★
The Angular 16 migration as your LEADERSHIP story ★
Code review, mentoring, tech-debt and estimation answers ready

Day 7 — Mock + the honesty pass ★

- Full dry run, out loud: intro → Angular → RxJS → Ionic → lead → questions
- Re-read the FOUR "do not claim" items ★
- [11 HR] — the STAR stories + your questions for them
- Re-read [18] the version numbers — say them without pausing ★

GOAL: nothing surprises you. You know what you'll say when you DON'T know.

▲ The four things to never claim — read this the morning of

- ✗ SIGNALS → 0 usages → "studied, not shipped; I'd start with signal inputs" ★
- ✗ STANDALONE → 2 only → "new code is standalone-first; legacy converts when touched" ★
- ✗ NgRx on RoboGebra → 0 usages → "that was EasyVisa; here we chose BehaviorSubject deliberately" ★
- ✗ MICROSERVICES → monolith → "modular monolith. Monolith first." ★

And the stack facts:

RoboGebra = MONGODB (not MySQL) ★
Angular 16.2 web · 18.2 mobile · Ionic 8.7.5 · Capacitor 6.2.1
Java 17 · Spring Boot 3.2.0

★ **Every one of these is stronger as an honest answer.** For a *lead* role they are testing judgement, not vocabulary — "we chose not to, and here's the trade-off" beats the buzzword every time.

◆ Interview-day checklist

- 60-second intro, no notes, ending on the IONIC MOBILE APP ★
- "Why IQVIA?" and "What do you know about IQVIA?" – SPECIFIC, not generic ★
- Three stories rehearsed:
 - Ionic mobile app ★ ★ | N+1 aggregation fix ★ ★ | Angular 16 migration ★
- Can draw YOUR architecture on a whiteboard ★
- Version numbers without pausing ★
- The healthcare line, used once early:
 - "I'd treat auditability and PHI handling as functional requirements, not afterthoughts" ★
- 2–3 questions ready for them ★
- The four "do not claim" items firmly in mind ★

If you only have ONE day

MORNING [00] intro out loud ★ · [39] your evidence ★ · [38] the EVIDENCE table

MIDDAY [38] Part 1 Angular · Part 3 RxJS · Part 4 IONIC ★ ★

AFTERNOON [38] Part 9 Tech Lead ★ · the 3 stories rehearsed

EVENING the four "do not claim" items · version numbers · your questions ★

After the interview — do this the same day

Add the round to **26 — Companies: Questions Actually Asked** while it's fresh:

- Every question they asked, in their words
- What you answered — and what you WISH you'd answered ★
- Anything you couldn't answer → that's your next study item ★
- The format (rounds, duration, who was on the call)

★ That log is why this pack keeps getting sharper. **The same ~25 topics repeat across companies** — lifecycle hooks have now come up in five rounds, interceptors in four.

Related files: [38 — IQVIA Prep ★](#) · [39 — RoboGebra Code Examples](#) · [00 — Self-Introduction](#) · [26 — Companies Asked](#) · [18 — RoboGebra Versions](#)

Self-Introduction — "Tell Me About Yourself" (Easy Version)

✓ **Rewritten from your latest resume** — `recent-resumes/Updated-Priyanka-R-Resume.docx` — and checked against your real code.

Three things were out of date and have been fixed:

x "over four years" → ✓ 5 YEARS ★ (Oct 2021 – now) x RoboGebra uses "MySQL" → ✓ MONGODB ★ (verified: 104 @Document classes) x closing line "...Wipro" → ✓ a BLANK you fill with the actual company ★

Everything below uses **short, simple sentences you can actually say out loud** — not written English.

Contents

1. The shape of the answer
 2. 30-second version
 3. 60-second version ★ (learn this one)
 4. 2-minute version
 5. Your anchor facts
 6. How to say it
 7. The follow-ups that always come next
-

The shape of the answer

The easiest way to remember:

NOW → BEFORE → NEXT ★

1. NOW – who I am today (role + years + stack + current project)
2. BEFORE – where I came from (2–3 projects, quickly)
3. NEXT – what I want now (and why THIS company)

NOW

"I'm Priyanka,
5 years,
Angular +
Java..."
↑ 20 sec

BEFORE

"Before RoboGebra I
worked on Subsea
and EasyVisa..."
↑ 25 sec

NEXT

"Now I'm looking for
_____, which is why
I'm interested in
[COMPANY]."
↑ 15 sec

★ **Why this order works:** the interviewer decides in the first ten seconds whether you sound senior. Give them role + years + stack **immediately**, then earn it with the projects.

Real-world idea: a **film trailer**. The title and the star come first, then the story, then "in cinemas soon." Nobody starts a trailer with the production history.

Easy memory

NOW → BEFORE → NEXT ★

20 sec | 25 sec | 15 sec = 60 seconds ★

First sentence = ROLE + YEARS + STACK ★ (they judge you in 10 seconds)

30-Second Version

(For when they say "briefly" — or on a phone screen.)

"Hi, I'm Priyanka. I'm a Full Stack Developer with **5 years of experience**, working mainly with **Angular on the front end and Java with Spring Boot on the back end**.

I'm a Software Engineer at **Provility Software Solutions** in Chennai. My current project is **RoboGebra**, an AI-based maths learning platform. I built the learning

dashboard, the Study List module and the quiz module — and I built the **Ionic mobile app** for iOS and Android from our existing Angular code.

I'm now looking for a bigger platform and more variety, which is what interests me about **[COMPANY]**."

Word count $\approx$ 90 $\rightarrow$ about 30 seconds at a calm pace ★

60-Second Version ★ (learn this one)

(This is your default. Practise until it's smooth, not memorised.)

"Hi, I'm Priyanka. I'm a Full Stack Developer with **5 years of experience**. I work mainly with **Angular on the front end and Java with Spring Boot on the back end**.

I'm a Software Engineer at **Provility Software Solutions** in Chennai. My current project — and my favourite — is **RoboGebra**. It's an AI-based maths learning platform built with **Angular, Ionic, Java and MongoDB**.

On RoboGebra I did four main things. I built the **personalised learning dashboard** with progress analytics and learning timelines. I built the **Study List module**, where students create and track their goals, with real-time sync across the app. I built the **quiz module**, with instant evaluation and solution walkthroughs. And I worked on the **AI explanation engine**, which gives step-by-step maths reasoning in both **Tamil and English**.

I also **built our Ionic mobile app for iOS and Android** from the existing Angular codebase — that's the piece I'm proudest of, because it took the product to mobile without a rewrite.

Before RoboGebra I worked on **Subsea**, a marine operations platform using Angular and .NET, where I built the Schedule-Manager module. I also **migrated four repositories to Angular 16**. And I started on **EasyVisa**, a visa management system for attorneys, where I built the document portal and a lot of reusable form components.

I'm now looking for a role with more scale and variety — which is exactly why I'm interested in **[COMPANY]**."

≈ 190 words → about 60–70 seconds ★

★ Notice what this version does:

It names **FOUR** concrete modules, not "I worked on features" ★

It has **ONE** proudest thing (the mobile app) — a natural hook for the interviewer's next question ★

It goes present → past, so the strongest work is heard **FIRST** ★

2-Minute Version

(For "walk me through your background" or a manager round.)

"Sure. I'm Priyanka. I'm a Full Stack Developer with **5 years of experience**. I did my **B.Tech in Information Technology at Anna University, IRTT Erode**, finishing in 2020, and I've been with **Provlity Software Solutions** in Chennai as a Software Engineer since **October 2021**.

Across those five years I've worked on both sides of the stack. On the front end that's **Angular, Ionic, TypeScript, RxJS and NgRx**, and some React. On the back end it's **Java and Spring Boot**, plus **Grails/Groovy** and **ASP.NET Core** on older projects. For databases I've used **MongoDB, MySQL, Postgres and Neo4j**, and I've worked with **AWS** and **Azure**.

Let me walk through the projects.

I started on EasyVisa, a visa management platform for attorneys, built with Angular, Grails, Groovy, Postgres and Neo4j. I built the full **document portal** — the accordion panels for uploading documents — and a lot of reusable UI: custom **Angular Formly** components, deeply nested reactive forms, and a customised image cropper. I also wrote the unit tests for the components and services I owned. That project is where I really became a front-end engineer.

Then I led an Angular migration, upgrading **four repositories to Angular 16**.

That taught me a lot about the framework internals and about managing

dependencies, because the hard part is never Angular itself — it's the third-party libraries.

After that I moved to Subsea, a marine equipment and operations platform on Angular and .NET, running in Docker on Azure. I built a new module called **Schedule-Manager**, including the .NET APIs behind it, used **Kendo UI Grid** for the complex table work, and built the dynamic category and sub-category management.

Since late 2024 I've been on RoboGebra, an AI-based maths learning platform on **Angular, Ionic, Java and MongoDB**. I built the personalised learning dashboard, the Study List module with real-time sync, and the quiz module. I worked on the AI explanation engine that produces step-by-step maths reasoning in **Tamil and English**. And I **built the Ionic mobile app for iOS and Android** from our existing Angular code, which took the product cross-platform without a rewrite.

On the backend side of RoboGebra I've worked in **Java 17 and Spring Boot 3** with **Spring Data MongoDB** — including the aggregation pipelines we use where a simple query isn't enough.

Outside the code, I work in agile teams using **Jira, Git, GitHub and Bitbucket**. What I enjoy most is owning a feature from the API all the way to the UI.

I'm now looking for a role with more scale and broader exposure, which is what interests me about **[COMPANY]**."

≈ 430 words → about 2 minutes ★

Your Anchor Facts

Memorise this table. Every version above is built from it.

Name / role	Priyanka R — Full Stack Developer (Angular & Java)
Experience	5 years ★

Current job	Software Engineer, Provility Software Solutions , Chennai (Oct 2021 – Present)
Contact	priyankaiirt@gmail.com · 8778015851
Education	B.Tech Information Technology — Anna University, IRTT Erode — 2020, 77.1%
Languages	English (professional) · Tamil (native)

The four projects, in order

1. EasyVisa	Nov 2021 – Jan 2024	Angular, Grails/Groovy, Postgres, Neo4j → document portal, Angular Formly, nested reactive forms
2. Backoffice	Feb 2024 – Mar 2024	Angular → migrated 4 repos to Angular 16 ★
3. Subsea	Apr 2024 – Sep 2024	Angular, .NET, Docker, Azure → Schedule-Manager module, Kendo UI Grid
4. RoboGebra	Oct 2024 – Present	Angular, Ionic, Java, MongoDB ★ → dashboard, Study List, quiz, AI explanations, Ionic mobile app (iOS/Android) ★

Your stack, in the order you should say it

FRONTEND	Angular ★ · Ionic ★ · TypeScript · JavaScript · RxJS · NgRx Angular Formly · React/Formik · HTML/CSS · Bootstrap · Kendo UI
BACKEND	Java ★ · Spring Boot ★ · Spring Reactive · Groovy/Grails · ASP.NET Core
DATABASE	MongoDB ★ · MySQL · Postgres · Neo4j
CLOUD	AWS · Azure · Microservices
TOOLS	Jira · GitHub · Bitbucket · IntelliJ IDEA · Visual Studio

● Real numbers you can quote (from your own code)

★ Nobody invents these, so they land hard:

RoboGebra backend – Java 17, Spring Boot 3.2, MongoDB
~110 REST controllers · ~200 services · 104 @Document
classes
27 aggregation pipelines

RoboGebra mobile – Angular 18.2, Ionic 8.7, Capacitor 6.2
163 components · 89 services · 19 native plugins ★

Full detail: [39 – RoboGebra Code Examples](#).

How to say it

The five rules

1. FIRST SENTENCE = role + years + stack ★
"Full Stack Developer, 5 years, Angular and Java with Spring Boot."
→ they judge your level in 10 seconds. Don't make them wait.
2. Say "I BUILT", not "I was involved in" ★
✗ "I helped with the quiz module"
✓ "I built the quiz module"
3. Name the MODULE, not the technology ★
✗ "I used Angular and Java"
✓ "I built the Study List module with real-time sync"
4. ONE flagship, the rest in a line each ★
Spend half your words on RoboGebra. Don't give four projects
equal time – it flattens the story.
5. PAUSE after the first sentence ★
Let the headline land. Then continue.

Speaking tips

- Speak SLOWER than feels natural – nerves speed you up by ~20%
- Short sentences. Full stops. This is speech, not an essay ★
- Smile before you start – it changes your voice, even on a call
- Don't recite the resume line by line – they already have it ★
- If you go past 75 seconds on the 60-second version, you're over-explaining

▲ Three things NOT to say

- ✗ "I'm a fresher-level..." or "I only worked on small parts..."
→ you have 5 years and four real projects. Own them ★
- ✗ "RoboGebra uses MySQL"
→ it's MONGODB ★ (this was wrong in the old notes)
- ✗ Naming a technology you can't discuss for two minutes
→ if you say NgRx, be ready for "how do effects work?"
Say what you've USED, and be honest about the rest ★

The follow-ups that always come next

Your intro **plants** these. Have a 30-second answer ready for each.

1. "Tell me more about RoboGebra." ★ (almost guaranteed)

"It's an AI-based maths learning platform. A student picks an exercise, and the platform gives a step-by-step solution — in Tamil or English — with the reasoning and formula breakdown, not just the answer. The backend is Java 17 and Spring Boot 3 on MongoDB, and there are three front ends: an Angular web app, an Ionic mobile app I built for iOS and Android, and an admin portal."

2. "Which part are you proudest of?" ★

"The Ionic mobile app. We had a working Angular web app, and the ask was to reach mobile without rebuilding. I built it with Ionic and Capacitor, reusing our existing services and models, and added about nineteen native plugins — push

notifications through Firebase, native Razorpay checkout, offline detection. The genuinely new part for me was the **Ionic page lifecycle**: Ionic keeps pages alive in the navigation stack, so `ngOnInit` runs only once and any data that must be fresh has to be loaded in `ionViewWillEnter`."

★ That last sentence is a real technical detail. It turns a project story into evidence.

3. "What was the hardest problem you solved?" ★

"An N+1 query problem. We were loading a chapter's exercises and then fetching each exercise's items one at a time — one query plus N. I replaced it with a single MongoDB aggregation pipeline that matches on all the exercise ids at once and sorts them, so it became one round trip instead of fifty. What I took from it is that switching to a document database doesn't remove the N+1 problem — it just changes what it looks like."

4. "Why are you looking to change?" ★

Keep it **forward-looking**. Never criticise your current employer.

"I've had good ownership at Provility — I've built features end to end across four products. What I want next is more scale: a larger user base, a bigger engineering team to learn from, and more structured processes around code review and architecture. That's what interests me about [COMPANY]."

- ✓ What you want MORE of ★
- ✗ What you're escaping FROM ▲

5. "Why this company?"

▲ **Prepare this per company.** A generic answer is worse than a short one.

Name ONE specific thing:
their product, their domain, the scale, the role's stack,
or something in the JD you genuinely want to work on ★

Quick Revision Sheet — read this in the waiting room

STRUCTURE ★ NOW → BEFORE → NEXT (20s | 25s | 15s)

OPENING LINE ★

"I'm Priyanka. Full Stack Developer, 5 years, mainly Angular on the front end and Java with Spring Boot on the back end."

CURRENT Software Engineer, Provility Software Solutions, Chennai, Oct 2021 – now

FLAGSHIP RoboGebra ★ – AI maths learning platform
Angular · Ionic · Java · MONGODB ★ (NOT MySQL!)
→ dashboard · Study List · quiz module · AI Tamil/English explanations
→ and I BUILT THE IONIC MOBILE APP (iOS/Android) ★ ← the proudest thing

BEFORE Subsea (Angular/.NET, Schedule-Manager) → Angular 16 migration, 4 repos
→ EasyVisa (document portal, Angular Formly, nested reactive forms)

CLOSING "More scale and variety – which is why I'm interested in [COMPANY]."

RULES ★ "I BUILT" not "I helped" | name the MODULE not the tech
pause after the first sentence | speak SLOWER than feels natural

NUMBERS 5 years · 4 projects · B.Tech IT 2020, 77.1%
RoboGebra: 163 components · 89 services · 19 Capacitor plugins

★

Related files: [11 — HR & Behavioural](#) · [10 — Projects Deep Dive](#) · [39 — RoboGebra Code Examples](#) · [38 — IQVIA Prep](#)

JavaScript — Interview Q&A (Easy Version)

Every question follows the same shape — **Easiest way to remember** → **simple explanation** → **real-world example** → **diagram** → **interview trap** → **Easy memory box**.

If `var` / `let` / `const`, hoisting or the loop problem feel shaky, read **19 — JavaScript Variables Made Easy** first — it teaches those from zero.

Contents

- 1. [Fundamentals](#)
- 2. [Functions & Closures](#)
- 3. [Objects & Prototypes](#)
- 4. [Asynchronous JavaScript](#)
- 5. [ES6+ Features](#)
- 6. [Advanced](#)
- 7. [Tricky Output-Based Questions](#)
- 8. [Quick Revision Sheet](#)

Fundamentals

Q: `var` vs `let` vs `const` ?

The easiest way to remember:

<code>var</code>	→ FUNCTION-scoped, hoisted as undefined, re-declarable	▲	old
<code>let</code>	→ BLOCK-scoped, can be reassigned	✓	
<code>const</code>	→ BLOCK-scoped, CANNOT be reassigned	✓	default
choice	★		

	var	let	const
Scope	function ▲	block {}	block {}
Re-declare in the same scope	✓	✗	✗
Re-assign	✓	✓	✗
Hoisted	✓ as undefined ▲	✓ but in the TDZ ★	✓ but in the TDZ
Adds to window	✓ ▲	✗	✗

```

if (true) {
  var a = 1;
  let b = 2;
}
console.log(a); // 1 ▲ var LEAKED out of the block
console.log(b); // ✗ ReferenceError – let stayed inside ✓

```

★ The **const** trap everyone falls into:

```

const user = { name: 'Priya' };
user.name = 'Ravi'; // ✓ ALLOWED – the CONTENTS changed
user = { name: 'Ravi' }; // ✗ TypeError – the BINDING is locked

```

const locks the BOX, not what's INSIDE it. ★

Real-world idea: your **house address is fixed** (**const**), but you can rearrange the furniture inside.

The rule to state: " **const** by default, **let** when the value genuinely changes, **var** never. "

Easy memory

```

var → FUNCTION scope, leaks, hoisted as undefined ▲ never use
let → block scope, reassignable
const ★ → block scope, binding LOCKED – but objects/arrays are still mutable ★

```

const = a fixed ADDRESS , movable furniture

Q: What is hoisting?

The easiest way to remember:

Before running your code, JavaScript SCANS it and moves every DECLARATION to the top of its scope. The ASSIGNMENT stays put. ★

```
console.log(x);    // undefined    ← not an error! ▲  
var x = 5;  
  
console.log(y);    // ✗ ReferenceError: Cannot access 'y' before  
initialization  
let y = 5;
```

What JavaScript actually does:

YOU WROTE	WHAT RUNS
<code>console.log(x);</code>	<code>var x;</code> ← DECLARATION hoisted ★
<code>var x = 5;</code>	<code>console.log(x);</code> → undefined
	<code>x = 5;</code> ← ASSIGNMENT stays ★

`var` → hoisted AND initialised to undefined → prints undefined ▲
`let/const` → hoisted but NOT initialised → the TEMPORAL DEAD ZONE ★
→ a ReferenceError, which is much better than a silent undefined

Function declarations are hoisted **completely**:

```
greet();                // ✓ works!  
function greet() { return 'hi'; }  
  
greet2();               // ✗ TypeError: greet2 is not a function  
var greet2 = function () {};
```

// only `var greet2` was hoisted ▲

Real-world idea: a **guest list read out before the party**. Every name is registered up front, but the guests only arrive at their own time. `var` guests are registered as "unknown person"; `let` guests are registered but barred from entering until their moment.

Easy memory

Hoisting = DECLARATIONS move up, ASSIGNMENTS stay ★

var → undefined (silent, dangerous) ▲

let/const → TDZ → ReferenceError ★ (better – it fails loudly)

function declaration → FULLY hoisted, callable before its definition ★

function EXPRESSION → not callable early

Q: Explain scope in JavaScript.

GLOBAL → visible everywhere

FUNCTION → visible inside that function (what `var` respects) ★

BLOCK → visible inside { } (what let/const respect) ★

LEXICAL → an inner function can see the OUTER function's variables ★

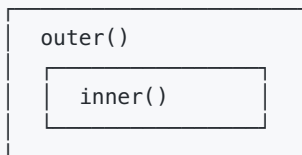
```
const global = 'g';

function outer() {
  const outerVar = 'o';

  function inner() {
    const innerVar = 'i';
    console.log(global, outerVar, innerVar); // ✓ can see ALL THREE ★
  }

  console.log(innerVar); // ✗ ReferenceError – can't look INWARD
}
```

GLOBAL



inner looks OUT ↑ ✓

outer looks IN ↓ ✗

★ **Lexical scope means "decided by where the code is WRITTEN, not where it's CALLED."** That single sentence is also the foundation of closures.

Real-world idea: **rooms in a house**. From inside a bedroom you can see the corridor and the front door. From the corridor you cannot see inside the bedroom.

Easy memory

```
global → function ★ (var) → block ★ (let/const)
LEXICAL scope = decided by WHERE THE CODE IS WRITTEN ★
Inner sees OUT ↑ | outer cannot see IN ↓
→ this is what makes closures possible ★
```

Q: What are the data types in JavaScript?

```
PRIMITIVES (7) – stored BY VALUE, immutable
  string · number · boolean · null · undefined · symbol · bigint

REFERENCE (1) – stored BY REFERENCE
  object (which includes arrays, functions, dates, Map, Set) ★
```

```
// PRIMITIVE – copied by value
let a = 1; let b = a; b = 2;
console.log(a);          // 1 ✓ unaffected

// REFERENCE – copied by reference ★
let x = { n: 1 }; let y = x; y.n = 2;
console.log(x.n);        // 2 ▲ they point at the SAME object
```

```
PRIMITIVE: a = [1]   b = [1]       two separate boxes ✓
REFERENCE:  x ┌       ──> { n: 2 }   one object, two labels ★
             y └
```

★ null vs undefined – the classic follow-up:

```
undefined → JavaScript's "nothing" – a variable declared but never
assigned
null       → YOUR "nothing" – you deliberately set it to empty ★
```

```
typeof undefined // "undefined"
typeof null      // "object"  ▲ a legendary language BUG, kept for
compatibility ★
null == undefined // true    (loose)
null === undefined // false  (strict) ★
```

Easy memory

7 PRIMITIVES (by value) + object (by reference) ★
string number boolean null undefined symbol bigint

undefined = JS didn't set it | null = YOU set it to empty ★
typeof null === "object" ▲ a famous bug ★

Q: == vs === ?

The easiest way to remember:

== → compares AFTER converting the types (loose) ▲
=== → compares the VALUE **and** the TYPE (strict) ✓ always use this ★

```
5 == '5'      // true  ▲ '5' is converted to 5
5 === '5'     // false ✓ different types
```

```
0 == false    // true  ▲
0 === false   // false ✓
```

```
null == undefined // true  ▲
null === undefined // false ✓
```

```
[] == false    // true  ▲ ▲ (!)
```

== runs a CONVERSION first → surprising results ▲
=== compares as-is → predictable ★

★ The one legitimate use of ==:

```
if (x == null) { } // true for BOTH null and undefined – a deliberate
idiom ★
```

Easy memory

`==` → converts types first ▲ | `===` → value AND type ✓ ★
ALWAYS use `===` (and `!==`)
The one exception: `x == null` catches `null` AND `undefined` ★

Q: What is type coercion?

Coercion = JavaScript automatically converting one type into another. ★

```
'5' + 3      // "53"      ▲ + with a string means CONCATENATION ★
'5' - 3      // 2         ★ - has no string meaning → both become numbers
'5' * '2'    // 10
1 + true     // 2         (true → 1)
'abc' - 1    // NaN
[] + {}      // "[object Object]"
```

★ The rule that explains almost all of it:

``+`` is the ONLY operator with a string meaning.
→ if EITHER side is a string, ``+`` CONCATENATES ★

Every other arithmetic operator converts both sides to NUMBERS ★

```
'5' + 3 → one side is a string → CONCATENATE → "53"
'5' - 3 → `-` has no string meaning → both to numbers → 2 ★
```

Real-world idea: `+` is a word that means two things — "add" and "join". The other operators only mean one thing, so there's no ambiguity.

How to avoid it: always use `===`, and convert explicitly with `Number(x)` and `String(x)`.

Easy memory

Coercion = automatic type conversion ★

'5' + 3 = "53" ← '+' CONCATENATES if either side is a string ★

'5' - 3 = 2 ← every OTHER operator converts to NUMBER ★

Avoid it: === , Number(x), String(x)

Q: What are truthy and falsy values?

★ There are exactly **8 falsy values** — memorise the list, and everything else is truthy.

```
false · 0 · -0 · 0n · "" · null · undefined · NaN
```

```
if ('')      {} // ✓ TRUTHY - a non-empty string ▲
if ([])      {} // ✓ TRUTHY - an empty array is truthy! ▲ ★
if ({})      {} // ✓ TRUTHY - an empty object too ▲
if ('false') {} // ✓ TRUTHY - a non-empty string ▲
```

★ **The [] and {} case is the trap** — people expect an empty array to be falsy.

```
if (arr) { }           // ▲ ALWAYS true – even for []
if (arr.length) { }    // ✓ what you actually meant ★
```

And the || vs ?? consequence:

```
const port = input || 3000; // ▲ 0 and "" fall through to 3000 ▲
const port = input ?? 3000; // ✓ only null/undefined do ★
```

Easy memory

8 FALSY: false 0 -0 0n "" null undefined NaN ★
EVERYTHING else is truthy – including [] and {} ▲ ★

Check an array with arr.length, not arr ★

|| replaces 0 and "" ▲ | ?? only replaces null/undefined ★

Functions & Closures

Q: What is a closure?

★ **The most-asked JavaScript question. Have this memorised word for word.**

The easiest way to remember:

A closure = a function that REMEMBERS the variables from where it was CREATED, even after that outer function has finished. ★

```
function counter() {  
  let count = 0;           // ★ this survives!  
  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const inc = counter();      // counter() has FINISHED and returned  
inc();    // 1  
inc();    // 2           ★ `count` is still alive  
inc();    // 3
```

counter() runs and RETURNS
↓
Normally `count` would be garbage collected
↓
BUT the returned function still REFERENCES it
↓
so JavaScript keeps `count` alive, in a private "backpack" ★

Real-world idea: a **backpack**. The inner function carries a backpack containing the variables it needs. Wherever you take the function, the backpack goes with it.

Another: a **locker key**. The room (`counter()`) is closed, but you still hold the key to one locker inside it, so its contents remain yours — and nobody else's.

Why closures matter — three real uses

```
// 1. PRIVATE data – there is no other way to make a truly private
variable ★
function bankAccount(initial) {
  let balance = initial;                                // ★ nobody outside can
touch it
  return {
    deposit: (amt) => (balance += amt),
    getBalance: () => balance
  };
}
const acc = bankAccount(100);
acc.balance;      // undefined ✓ genuinely inaccessible
acc.getBalance(); // 100

// 2. Function factories
const multiply = (x) => (y) => x * y;
const double = multiply(2);   double(5);   // 10 ★

// 3. What makes DEBOUNCE and THROTTLE work ★
```

▲ The memory-leak caveat to volunteer:

A closure KEEPS its captured variables alive.
Capture a huge array or a DOM node in a long-lived callback
→ it can never be garbage collected ▲

Interview-ready answer

A closure is a function that retains access to variables from the scope in which it was created, even after that outer function has returned. JavaScript keeps those variables alive because the inner function still references them. The main uses are data privacy — a variable inside a closure genuinely cannot be reached from outside, which is how the module pattern works — function factories, and utilities like debounce and throttle that need to remember state between calls. The trade-off is memory: a closure keeps whatever it captured alive, so capturing a large object in a long-lived handler can prevent it being garbage collected.

Easy memory

Closure = a function + the VARIABLES it was born with (a backpack) ★

Uses: PRIVATE data ★ | function factories | debounce/throttle

Cost: captured variables can never be garbage collected ▲

"The inner function remembers, even after the outer one has finished." ★

Q: What is an IIFE?

IIFE = Immediately Invoked Function Expression –
a function that runs the instant it's defined. ★

```
(function () { ... })();
```

```
(function () {  
  const secret = 'hidden';    // ★ never touches the global scope  
  console.log('runs immediately');  
})();
```

```
((() => { })());           // arrow version
```

Why it existed: before ES6 modules and `let`, this was the ONLY way to avoid polluting the global scope. ★

▲ Be honest about relevance:

Modern code uses ES MODULES (each file has its own scope) and `let`/`const`.

IIFEs are mostly historical – you'll meet them in older libraries and bundles. ★

Easy memory

IIFE = (function(){ ... })(); → runs immediately ★

Purpose: private scope BEFORE modules and let/const existed

Today: ES modules do the same job – IIFEs are mostly legacy ★

Q: What are higher-order functions?

A higher-order function TAKES a function as an argument, or RETURNS a function. ★

```
// takes a function
[1, 2, 3].map(n => n * 2);           // ★ map, filter, reduce, forEach,
sort                                sort

// returns a function
const multiply = (x) => (y) => x * y;
```

The four you must be able to write cold:

```
const nums = [1, 2, 3, 4, 5];

nums.map(n => n * 2);                // [2,4,6,8,10]  -
TRANSFORM ★
nums.filter(n => n % 2 === 0);        // [2,4]        - KEEP
some ★
nums.reduce((acc, n) => acc + n, 0);   // 15           - COMBINE
★
nums.forEach(n => console.log(n));     // undefined    - side
effects only
```

```
map    → same LENGTH, different values ★
filter → same VALUES, fewer of them ★
reduce → MANY values → ONE value ★
forEach → returns NOTHING (can't be chained) ▲
```

★ **reduce** traced step by step, since that's the one people fumble:

```
[1,2,3,4,5]  acc starts at 0
0+1=1 → 1+2=3 → 3+3=6 → 6+4=10 → 10+5=15 ✓
```

Easy memory

```
HOF = takes and/or returns a FUNCTION ★
map (transform) | filter (keep) | reduce (combine to one) ★ | forEach
(side effect)
forEach returns undefined → you cannot chain it ▲
```

Q: What is currying?

Currying = turning $f(a, b, c)$ into $f(a)(b)(c)$ – one argument at a time. ★

```
// normal
const add = (a, b, c) => a + b + c;
add(1, 2, 3);           // 6

// curried ★
const addC = (a) => (b) => (c) => a + b + c;
addC(1)(2)(3);         // 6
```

Why it's useful — **partial application**:

```
const multiply = (x) => (y) => x * y;

const double = multiply(2);      // ★ pre-fill the first argument
const triple = multiply(3);

double(5);    // 10
triple(5);    // 15
```

You've already used this shape:

RxJS operators! `map(fn)` returns a function that takes the source ★

Real-world idea: a **coffee order taken in stages** — size, then milk, then sugar. At each step you have a partially completed order you can reuse.

Easy memory

Currying: $f(a,b,c) \rightarrow f(a)(b)(c)$ ★ – one argument at a time
Benefit: PARTIAL APPLICATION $\rightarrow$ `multiply(2)` becomes ``double`` ★
You already use it: every RxJS operator is curried ★

Q: Explain `call`, `apply` and `bind`.

The easiest way to remember:

call → invoke NOW, arguments as a LIST f.call(obj, a, b) ★
apply → invoke NOW, arguments as an ARRAY f.apply(obj, [a, b]) ★
bind → do NOT invoke; return a NEW function f.bind(obj) ★

★ The memory hook: **"C for Comma, A for Array, B for Bind (later)."**

```
const person = { name: 'Priya' };  
function greet(greeting, punctuation) {  
  return `${greeting}, ${this.name}${punctuation}`;  
}  
  
greet.call(person, 'Hello', '!');            // "Hello, Priya!"    - comma-  
separated ★  
greet.apply(person, ['Hello', '!']);        // "Hello, Priya!"    - an array  
★  
const bound = greet.bind(person);            //  
★ returns a function, doesn't run it  
bound('Hello', '!');                        // "Hello, Priya!"
```

All three do the same thing: **set what `this` means inside the function.**

★ Where `bind` actually matters — the classic callback bug:

```
class Timer {  
  constructor() { this.seconds = 0; }  
  start() {  
    setInterval(function () {  
      this.seconds++;            // ✗ `this` is NOT the Timer ▲  
    }, 1000);  
  }  
}
```

```
setInterval(this.tick.bind(this), 1000);    // ✓ fix 1  
setInterval(() => this.seconds++, 1000);    // ✓ fix 2 – an arrow  
function ★
```

Easy memory

call → NOW, Comma-separated args ★
apply → NOW, Array of args ★
bind → LATER, returns a bound function ★

All three set `this`.
Modern fix for callbacks: an ARROW FUNCTION (no own `this`) ★

Q: Arrow functions vs normal functions?

★ The one difference that matters: `this`.

Normal function → gets its OWN `this`, decided by HOW IT'S CALLED ★
Arrow function → has NO own `this`; it inherits from where it was WRITTEN ★

```
const obj = {  
  name: 'Priya',  
  regular() { return this.name; },           // ✓ "Priya" – `this` is obj  
  arrow: () => this.name,                   // ✗ undefined – `this` is the  
  outer scope ★  
};
```

	Normal	Arrow
own <code>this</code>	✓ (dynamic)	✗ inherits (lexical) ★
<code>arguments</code> object	✓	✗ (use rest <code>...args</code>)
Usable as a constructor (<code>new</code>)	✓	✗
Can be hoisted	✓ (declarations)	✗
Best for	object methods, constructors	callbacks ★

Use an ARROW for callbacks (it keeps the surrounding `this`) ★
Use a NORMAL function for object methods (it needs its own `this`) ★

Real-world idea: a normal function is a **contractor** who works for whoever hired them (`this` depends on the caller). An arrow function is a **family member** who always belongs to the same household, wherever they go.

Easy memory

Arrow = NO own ``this`` – it inherits from where it was WRITTEN ★
No ``arguments``, no ``new``, not hoisted

CALLBACKS → arrow ★ | OBJECT METHODS → normal function ★

Q: How is `this` determined?

★ Check the **call site**, in this order:

- | | | |
|--|--|-------------------------|
| 1. <code>`new Foo()`</code> | → <code>`this`</code> = the NEW object | (new binding) |
| 2. <code>f.call/apply/bind</code> | → <code>`this`</code> = what you passed ★ | (explicit) |
| 3. <code>obj.method()</code>
the DOT) | → <code>`this`</code> = obj ★ | (implicit – |
| 4. plain <code>f()</code> | → <code>`this`</code> = undefined (strict) / window (sloppy) ▲ | |
| 5. arrow function | → <code>`this`</code> = the ENCLOSING scope ★ | (lexical – no own this) |

```
function show() { return this; }
```

```
show();           // undefined (strict) / window ▲  
obj.show();       // obj ★ – the DOT decides  
show.call({ a: 1 }); // { a: 1 } ★  
new show();       // the new object
```

★ The "lost `this`" bug – the one that appears in real code:

```
const obj = { name: 'Priya', greet() { return this.name; } };  
  
const fn = obj.greet;    // ▲ detached from the object  
fn();                   // undefined ▲ – no dot, no `this`
```

"NO DOT, NO THIS." ★

Fix: `obj.greet.bind(obj)` or `() => obj.greet()`.

Easy memory

Check the CALL SITE, in order:

new ★ → call/apply/bind ★ → obj.method() (the DOT ★) → plain call (undefined)

Arrow → the ENCLOSING scope, always ★

"NO DOT, NO THIS." ★ (detaching a method loses `this`)

Objects & Prototypes

Q: What is prototypal inheritance?

Every object has a hidden link to another object – its PROTOTYPE.

If a property isn't found, JavaScript looks UP that chain. ★

```
const animal = { eats: true };
const dog = Object.create(animal);    // dog's prototype is animal ★
dog.barks = true;
```

```
dog.barks;    // true  – found on dog
dog.eats;     // true  ★ – NOT on dog; found on the prototype
```

```
dog { barks: true }
  | __proto__
  ▼
animal { eats: true }
  | __proto__
  ▼
Object.prototype (toString, hasOwnProperty...)
  | __proto__
  ▼
null ← the end of the chain ★
```

Real-world idea: **asking your family for something**. You check your own pocket, then your parent, then your grandparent — until someone has it, or nobody does.

★ The point that distinguishes JavaScript from Java:

Java → classes are BLUEPRINTS; objects are built from them
JavaScript → objects inherit DIRECTLY from other OBJECTS ★
`class` is just SYNTACTIC SUGAR over prototypes ★

Easy memory

Every object has a `__proto__` link → JS looks UP the chain ★
`Object.create(parent)` sets it explicitly
Chain ends at `Object.prototype` → null ★

`class` in JS is SUGAR over prototypes – not real class inheritance ★

Q: What is the prototype chain?

The prototype chain = the lookup path JavaScript walks when it can't find a property on the object itself. ★

```
const arr = [1, 2, 3];  
arr.map(...)      // not on arr → found on Array.prototype ★  
arr.toString()    // not on Array.prototype → found on Object.prototype ★  
arr.foo           // walked to null → undefined ★
```

`arr` → `Array.prototype` → `Object.prototype` → null

★ This is why it matters practically:

1. Every array shares ONE copy of `map/filter/reduce` ★
→ not 1,000 copies for 1,000 arrays
2. A LONG chain means a slower lookup
3. It's why `hasOwnProperty` exists ★

```
const obj = { a: 1 };  
'toString' in obj;                // true ▲ – inherited from the  
prototype  
obj.hasOwnProperty('toString');    // false ✓ – not its OWN property  
★  
Object.hasOwn(obj, 'toString');    // false ✓ modern form
```


Easy memory

Property lookup walks UP: object → prototype → ... → null ★
Methods live ONCE on the prototype, shared by every instance ★
`in` includes INHERITED properties ▲ | hasOwnProperty checks OWN only ★

Q: What does `Object.create` do?

`Object.create(proto)` makes a NEW object whose prototype is `proto`. ★

```
const parent = { greet() { return 'hello'; } };  
const child = Object.create(parent);  
child.greet(); // "hello" ★ inherited  
  
Object.create(null); // ★ an object with NO prototype at all
```

★ `Object.create(null)` is genuinely useful:

```
const map = {}; // ▲ inherits toString, constructor...  
map['toString']; // a function! not undefined ▲  
  
const safeMap = Object.create(null); // ★ a truly empty dictionary  
safeMap['toString']; // undefined ✓
```

Use it whenever an object is being used as a **pure dictionary** with user-supplied keys.

Easy memory

`Object.create(proto)` → a new object with that PROTOTYPE ★
`Object.create(null)` → NO prototype → a safe dictionary ★
(`{}` inherits `toString` and can collide with user keys ▲)

Q: Shallow copy vs deep copy?

SHALLOW → copies the TOP level; nested objects are still SHARED ▲
DEEP → copies EVERYTHING, recursively ✓ ★

```
const original = { name: 'Priya', address: { city: 'Chennai' } };

// SHALLOW
const shallow = { ...original };
shallow.name = 'Ravi'; // ✓ original.name unchanged
shallow.address.city = 'Mumbai'; // "Mumbai" ▲ SHARED! ★
original.address.city;

// DEEP ★
const deep = structuredClone(original); // modern, built-in ★
deep.address.city = 'Delhi';
original.address.city; // "Mumbai" ✓ unaffected
```

SHALLOW copy

```
shallow.name — its own ✓
shallow.address —┐
                  └─→ the SAME nested object ▲
original.address —┐
```

The deep-copy options:

```
structuredClone(obj)      ★ built-in, handles Date/Map/Set/circular
refs
JSON.parse(JSON.stringify(x)) ▲ loses Date, undefined, functions; breaks
on circular refs
lodash cloneDeep          ✓ if you already have lodash
```

▲ The **JSON** trick is the one people quote — know its limits:

```
JSON.parse(JSON.stringify({ d: new Date(), u: undefined, f: () => {} }));
// → { d: "2026-09-02T..." } ▲ Date became a STRING, u and f VANISHED
▲
```

Easy memory

```
SHALLOW: { ...obj } or Object.assign → nested objects are SHARED ▲ ★
DEEP: structuredClone(obj) ★ (built-in)

▲ JSON.parse(JSON.stringify()) loses Date/undefined/functions + circular
refs ▲
Immutable updates in Angular/NgRx need a NEW reference ★
```

Asynchronous JavaScript

Q: What is a callback and what is callback hell?

Callback = a function passed into another function, to be called LATER. ★
Callback hell = callbacks nested inside callbacks until the code slides off the right of the screen. ●

```
getUser(1, (user) => {  
  getOrders(user.id, (orders) => {  
    getOrderDetails(orders[0].id, (details) => {  
      getShipping(details.id, (shipping) => {  
        console.log(shipping);          // ▲ the "pyramid of doom"  
      });  
    });  
  });  
});
```

Problems:
unreadable (the pyramid ●)
error handling repeated at EVERY level ▲
impossible to run steps in parallel

Fixed with promises / async-await:

```
const user = await getUser(1);  
const orders = await getOrders(user.id);  
const details = await getOrderDetails(orders[0].id);  // ★ flat and readable
```

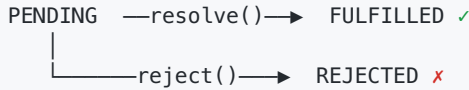
Easy memory

Callback = "call me back when you're done" ★
Callback hell = nesting → the PYRAMID OF DOOM ● + repeated error handling
Fix: Promises → async/await ★ (flat, one try/catch)

Q: What is a Promise and what are its states?

A Promise = an object representing a value that will exist LATER. ★

THREE states, and once settled it NEVER changes again ★



Once settled → IMMUTABLE ★ (it can never flip back)

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => resolve('done'), 1000);
});

promise
  .then(v => console.log(v))           // fulfilled
  .catch(e => console.error(e))        // rejected
  .finally(() => console.log('always')); // ★ runs either way
```

Real-world idea: an **online order**.

PENDING → the order is placed, still shipping

FULFILLED → it arrived ✓

REJECTED → it was cancelled ✗

Either way, it's DECIDED. It can't un-arrive. ★

★ Promises are **eager** — the executor runs the moment you create it (unlike an RxJS Observable, which is lazy). That contrast is a very common follow-up. → [20](#)
— RxJS

Easy memory

3 states: PENDING → FULFILLED ✓ / REJECTED ✗ — settled is FINAL ★
.then / .catch / .finally ★

Promise = EAGER (runs immediately) | Observable = LAZY ★

Q: What is promise chaining?

Each `.then()` RETURNS a new promise, so they can be chained flat instead of nested. ★

```
getUser(1)
  .then(user => getOrders(user.id))           // ★ RETURN the promise
  .then(orders => getDetails(orders[0]))
  .then(details => console.log(details))
  .catch(err => console.error(err));           // ★ ONE catch for the whole
chain
```

▲ The #1 chaining bug — forgetting `return`:

```
.then(user => { getOrders(user.id); })         // ✗ nothing is returned
.then(orders => console.log(orders));           // → undefined ▲
```

No ``return`` → the next `.then()` receives undefined ★
(An arrow function with braces needs an explicit ``return``.)

★ And the big win: a single `.catch()` at the end handles an error from any step — no repeated error handling.

Easy memory

`.then()` returns a NEW promise → chain flat, not nested ★
ONE `.catch()` at the end covers EVERY step ★

▲ Forgetting ``return`` inside a `.then()` → the next step gets undefined ▲

Q: What is `async` / `await`?

`async/await` = SYNTAX SUGAR over promises.
It makes `async` code READ like synchronous code. ★

```

async function loadData() {
  try {
    const user    = await getUser(1);           // ★ "pause here"
    const orders  = await getOrders(user.id);
    return orders;
  } catch (err) {                               // ★ ONE try/catch for
all of it
    console.error(err);
  }
}

```

async → the function ALWAYS returns a PROMISE ★
 await → pause until the promise settles (only inside an async function)
 try/catch → replaces .catch() ★

★ The performance trap that gets asked:

```

// ✗ SEQUENTIAL – 3 seconds
const a = await getA();    // 1s
const b = await getB();    // 1s
const c = await getC();    // 1s

// ✓ PARALLEL – 1 second ★
const [a, b, c] = await Promise.all([getA(), getB(), getC()]);

```

await in a loop / one after another = the SUM of the times
 Promise.all = the SLOWEST of them ▲ ★

Only `await` sequentially when the next call genuinely needs the previous result.

Easy memory

async → always returns a PROMISE ★ | await → pause until settled
 try/catch replaces .catch() ★

- ▲ Sequential awaits = the SUM of the times
- ✓ Independent calls → Promise.all → the SLOWEST one ▲ ★

Q: `Promise.all` vs `race` vs `allSettled` vs `any`?

`all` → ALL succeed → an array of results. ONE fails → the WHOLE thing fails ▲
`allSettled` → waits for ALL, reports each outcome. NEVER rejects ★
`race` → the FIRST to SETTLE wins (success OR failure) ★
`any` → the first to SUCCEED (ignores failures)

```
Promise.all([a, b, c]);           // ▲ one rejection kills everything
Promise.allSettled([a, b, c]);    // ★ [{status:'fulfilled',value},
                                   {status:'rejected',reason}]
Promise.race([fetchData, timeout(5000)]); // ★ a TIMEOUT pattern
Promise.any([mirror1, mirror2]);   // the first that works
```

`all` → needs EVERY result (a dashboard where all data is required)
`allSettled` → show what you CAN, report what failed ★ (partial dashboard)
`race` → TIMEOUTS ★
`any` → the first working mirror/CDN

★ `race` for a timeout is the pattern worth showing:

```
const timeout = (ms) => new Promise( (_, rej) => setTimeout(() =>
  rej('timeout'), ms));
await Promise.race([fetch('/api/slow'), timeout(5000)]); // ★
```

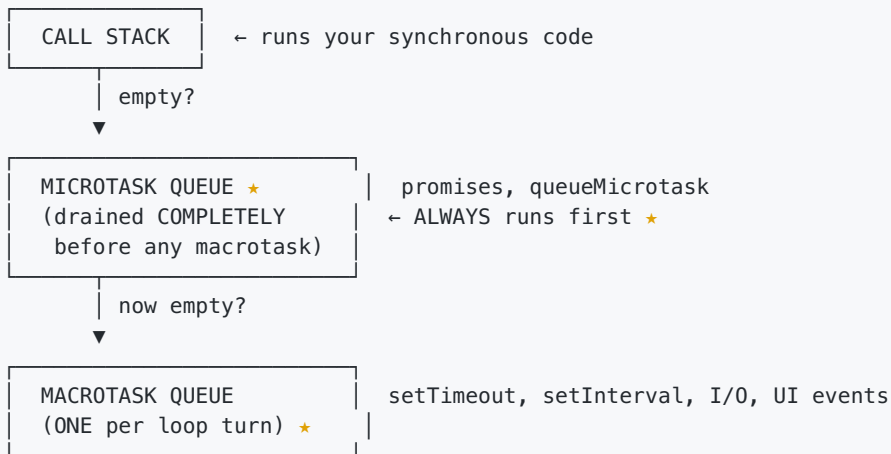
Easy memory

`all` ★ → all-or-nothing (one failure kills it ▲)
`allSettled` ★ → never rejects → partial results
`race` ★ → the first to SETTLE → TIMEOUT pattern
`any` → the first to SUCCEED

Q: Explain the event loop.

★ The most-asked "senior JavaScript" question.

JavaScript is SINGLE-THREADED – one call stack, one thing at a time.
The event loop is what lets it still handle async work. ★



The rule in one line:

Run all SYNC code → drain ALL microtasks ★ → take ONE macrotask → repeat

```
console.log(1); // sync
setTimeout(() => console.log(2), 0); // MACROtask
Promise.resolve().then(() => console.log(3)); // MICROtask ★
console.log(4); // sync

// Output: 1 4 3 2 ★
```

```
1, 4 → synchronous, straight down the stack
3 → microtask – drained BEFORE any timer ★
2 → macrotask – last, even with a 0 ms delay ★
```

Real-world idea: a **doctor's clinic**.

```
Current patient → the call stack
EMERGENCY queue ★ → microtasks: cleared COMPLETELY before anyone else
Normal appointments → macrotasks: ONE at a time, then check emergencies again ★
```

▲ And the consequence to volunteer:

An infinite microtask chain STARVES the macrotask queue — timers and UI events never run, and the page freezes ▲

Interview-ready answer

JavaScript is single-threaded with one call stack, so the event loop is what allows it to handle asynchronous work. Synchronous code runs on the stack first. When the stack is empty, the event loop drains the entire microtask queue — promise callbacks and `queueMicrotask` — and only then takes a single task from the macrotask queue, such as a `setTimeout` callback or an I/O event, before checking microtasks again. That ordering is why a promise callback always runs before a `setTimeout(..., 0)`. It also means a long synchronous function or an endless chain of microtasks blocks rendering entirely.

Easy memory

SYNC → drain ALL microtasks ★ → ONE macrotask → repeat

MICROtask ★ = promises, `queueMicrotask`, `MutationObserver` (the EMERGENCY queue)

MACROtask = `setTimeout`, `setInterval`, I/O, UI events

1 4 3 2 ★ — memorise that output

`setTimeout(fn, 0)` still runs AFTER every promise ★

Q: Microtask vs macrotask queues?

MICROTASK ★ → promise `.then/.catch/.finally`, `await`, `queueMicrotask`, `MutationObserver`

→ the queue is drained COMPLETELY every turn

MACROTASK → `setTimeout`, `setInterval`, `setImmediate`, I/O, UI events

→ exactly ONE is taken per loop turn ★

The whole difference in one line:

microtasks are drained ALL AT ONCE ★

macrotasks are taken ONE AT A TIME ★

```

setTimeout(() => console.log('macro 1'), 0);
Promise.resolve().then(() => {
  console.log('micro 1');
  Promise.resolve().then(() => console.log('micro 2')); // ★ added
  DURING the drain
});
setTimeout(() => console.log('macro 2'), 0);

// micro 1, micro 2, macro 1, macro 2 ★
// note: micro 2 was queued during the drain and STILL ran before any
timer ★

```

Easy memory

Microtasks ★ drained COMPLETELY (even ones added mid-drain)
 Macrotasks – ONE per turn ★

 Promises ALWAYS beat setTimeout, even setTimeout(fn, 0) ★
 Infinite microtasks → the UI freezes ▲

Q: setTimeout vs setInterval ?

setTimeout → run ONCE after a delay
 setInterval → run REPEATEDLY every interval ▲

```

const id = setTimeout(fn, 1000); clearTimeout(id);
const id2 = setInterval(fn, 1000); clearInterval(id2); // ★ MUST be
cleared

```

▲ The setInterval problem, and the better pattern:

setInterval fires on schedule regardless of whether the previous run has FINISHED.

↓

An HTTP call that takes 3s on a 1s interval → calls PILE UP ▲

```
// ✓ a self-scheduling timeout – the next run starts only after this one ends ★
function poll() {
  fetchData().finally(() => setTimeout(poll, 1000));
}
```

Also: the delay is a **minimum, not a guarantee** — a busy stack delays the callback.

Easy memory

```
setTimeout = ONCE | setInterval = REPEAT (always clearInterval! ▲ )
▲ setInterval doesn't wait for the previous run → overlapping calls ▲
✓ Self-scheduling setTimeout for polling ★
The delay is a MINIMUM, not a guarantee ★
```

ES6+ Features

Q: What is destructuring?

Destructuring = pull values OUT of an object or array into variables. ★

```
// OBJECT – matched by KEY NAME ★
const { name, age, city = 'Chennai' } = user;           // with a default
const { name: fullName } = user;                       // renamed ★
const { address: { city } } = user;                   // nested

// ARRAY – matched by POSITION ★
const [first, second] = [1, 2, 3];
const [a, , c] = [1, 2, 3];                             // skip one
const [head, ...tail] = [1, 2, 3];                     // rest ★

// swap with no temp variable ★
let x = 1, y = 2;
[x, y] = [y, x];

// in a function signature ★
function greet({ name, age = 0 }) { }
```

OBJECT → by NAME ★ (order doesn't matter)
ARRAY → by POSITION ★ (order is everything)

▲ The trap:

```
const { name } = null;          // ▲ TypeError: Cannot destructure property
                                // of null
const { name } = obj ?? {};    // ✓ safe ★
```

Easy memory

Object → by NAME ★ { name, age = 0 } | rename: { name: fullName }
Array → by POSITION ★ [a, , c] | rest: [head, ...tail]
Swap: [x, y] = [y, x] ★
▲ Destructuring null throws → use `?? {}` ★

Q: Spread vs rest?

★ **Same three dots, opposite jobs — decided by WHERE they appear.**

SPREAD → EXPANDS one thing into many (used where VALUES go) ★
REST → COLLECTS many into one array (used where NAMES go) ★

```
// SPREAD — expand
const arr2 = [...arr1, 4];           // copy + add
const merged = { ...obj1, ...obj2 }; // merge (later wins ★ )
Math.max(...numbers);               // array → arguments

// REST — collect
function sum(...nums) { }           // ★ in a SIGNATURE
const [first, ...others] = arr;      // ★ in a PATTERN
const { name, ...rest } = obj;       // everything else ★
```

In a CALL or a LITERAL → SPREAD (expanding) ★
In a SIGNATURE or PATTERN → REST (collecting) ★

★ Spread is the standard **immutable update**, which is exactly what **NgRx** reducers and **OnPush** need:

```
const updated = { ...state, loading: true }; // ★ a NEW object, new reference
```

▲ But it's a **shallow** copy — nested objects are still shared.

Easy memory

```
SPREAD ★ expands: [...arr] { ...obj } fn(...args)
REST ★ collects: function f(...args) | const [a, ...rest] |
{ x, ...rest }
```

Position decides which it is ★

Spread = the immutable-update tool (NgRx, OnPush) — but SHALLOW ▲

Q: What are template literals?

```
const name = 'Priya';
const greeting = `Hello, ${name}!`; // ★ interpolation

const multiline = `line 1
line 2`; // ★ real
newlines

const expr = `Total: ${price * qty}`; // any expression
```

Backticks ` → interpolation \${} + real multi-line strings ★

Tagged templates (rarer, but worth recognising):

```
sql`SELECT * FROM users WHERE id = ${id}`; // the tag function can
escape the value ★
```

Easy memory

```
Backticks ` + ${expression} ★ + genuine multi-line
Tagged templates: tag`text ${v}` → the function controls the interpolation
```

Q: What are default parameters?

```
function greet(name = 'Guest', greeting = 'Hello') {  
  return `${greeting}, ${name}`;  
}  
greet(); // "Hello, Guest"  
greet('Priya'); // "Hello, Priya"  
greet(undefined, 'Hi'); // "Hi, Guest" ★ undefined triggers the  
default  
greet(null); // "Hello, null" ▲ null does NOT ★
```

Only `undefined` triggers a default. `null` does not. ★

★ Defaults can reference earlier parameters:

```
function makeUser(name, role = `${name}_user`) { } // ★ evaluated at  
CALL time
```

Easy memory

function f(a = 1) – only `undefined` triggers it ▲ null does NOT ★
Defaults are evaluated at CALL time and can use earlier parameters ★

Q: How do ES modules work?

```
// export  
export const name = 'Priya';  
export function greet() {}  
export default class User {} // ★ ONE default per file  
  
// import  
import User, { name, greet } from './user'; // default + named  
import * as utils from './utils'; // namespace  
const mod = await import('./heavy'); // ★ DYNAMIC import  
(lazy!)
```

NAMED exports → many per file → braces on import → the name must MATCH ★
DEFAULT export → ONE per file → no braces → you may RENAME it freely ★

★ Two properties that matter:

1. Each module has its OWN SCOPE – no global pollution ★
2. Imports are STATIC → bundlers can TREE-SHAKE unused exports ★
→ which is why ``import { x }`` beats importing a whole namespace

`import()` returns a promise — this is exactly what Angular's `loadChildren` uses for lazy routes.

Easy memory

export / export default (ONE) / import { } / import X / import * as ★
Own scope per file ★ | STATIC imports → TREE-SHAKING ★
Dynamic import() → a Promise → Angular's lazy loading ★

Q: Map / Set vs objects/arrays?

Map → keys of ANY type ★ , keeps INSERTION ORDER, has `.size` ★
Set → UNIQUE values ★

```
const map = new Map();
map.set('a', 1);
map.set(42, 'num');           // ★ a number key
map.set(objKey, 'obj');       // ★ an OBJECT as a key – impossible with
{} ★
map.get(objKey); map.size; map.has('a');

const set = new Set([1, 2, 2, 3]);
set.size;                     // 3 ★ duplicates removed
[...new Set(arr)];            // ★ the one-line dedupe
```

	Object	Map
Key types	string / symbol only ▲	anything ★
Order	mostly insertion (integer keys reorder ▲)	guaranteed insertion ★
Size	<code>Object.keys(o).length</code>	<code>.size</code> ★
Prototype keys	inherits <code>toString</code> etc. ▲	clean ★
Frequent add/delete	slower	optimised ★

★ Use a **Map** when keys aren't strings, or when you add and remove often. Use an **object** for plain records and JSON.

Easy memory

```
Map ★ → ANY key type, insertion order, .size, no prototype keys
Set ★ → unique values → [...new Set(arr)] dedupes in one line ★

Object → string keys only, inherits prototype keys ▲ , JSON-friendly
```

Q: What is optional chaining (?.)?

```
user?.address?.city           // undefined instead of a TypeError ★
user?.getName?.()             // safe METHOD call ★
arr?.[0]                       // safe INDEX access ★
```

```
Before: user && user.address && user.address.city
After:  user?.address?.city      ✓ ★
```

▲ The nuance:

```
?. short-circuits ONLY on null or undefined.
It does NOT protect against 0, "" or false. ★
```

Easy memory

```
?. → returns undefined instead of throwing ★
Forms: a?.b | a?.[0] | a?.() ★
Short-circuits on null/undefined ONLY ★
```

Q: What is nullish coalescing (??)?

```
?? → a fallback ONLY for null or undefined ★
|| → a fallback for ANY falsy value (0, "", false, NaN too) ▲
```



```
const count = 0;
count || 10;      // 10  ▲ WRONG – 0 is a valid count!  ▲
count ?? 10;      // 0   ✓  ★

const name = '';
name || 'Guest';  // "Guest"  ▲
name ?? 'Guest';  // ""      ✓  ★
```

★ This is the single most useful small feature in modern JavaScript — it fixes a bug class that `||` silently created for years.

Also:

```
config.retries ??= 3;      // ★ assign only if null/undefined
```

Easy memory

```
?? ★ → only null/undefined | || ▲ → any falsy (0 "" false)
```

```
count = 0 → count || 10 gives 10  ▲ | count ?? 10 gives 0  ✓  ★
??= assigns only when nullish
```

Q: What are generators?

A generator can PAUSE and RESUME. ``yield`` hands a value out and freezes the function until you ask for the next one. ★

```
function* idGenerator() {
  let id = 1;
  while (true) {
    yield id++;           // ★ pause here, hand out a value
  }
}

const gen = idGenerator();
gen.next().value;  // 1
gen.next().value;  // 2      ★ resumed exactly where it stopped
```

Normal function → runs to the end, once ►

Generator → runs → PAUSES at yield " → resumes on next() ★

Real-world idea: a **ticket dispenser**. It gives you one ticket and waits; it doesn't print the whole roll.

Where they're actually used:

```
Infinite sequences without infinite memory ★
Lazy iteration over a huge or paged dataset ★
Redux-Saga (the async model NgRx Effects replaces)
```

▲ Be honest: generators are rare in day-to-day Angular work — `async/await` covers most of what they were used for.

Easy memory

```
function* + yield ★ → pause ⏸ and resume ▶ (a ticket dispenser )
gen.next() → { value, done }
Uses: infinite/lazy sequences, Redux-Saga
Rare in Angular – async/await covers most cases ★
```

Advanced

Q: Debounce vs throttle?

★ **Asked constantly, and confused constantly.**

```
DEBOUNCE → wait for a PAUSE, then run ONCE ★ → search, resize, autosave
THROTTLE → run at most ONCE per interval ★ → scroll, mousemove
```

```
Events:  x x x x x          x x x          x

DEBOUNCE: .....R  .....R  ....R
           (runs only after the events STOP) ★

THROTTLE: R....R....R.... R....R...  R
           (runs on a fixed rhythm, ignoring the rest) ★
```

```
function debounce(fn, delay) {
  let timer; // ★ a CLOSURE holds the
  timer
  return (...args) => {
    clearTimeout(timer); // ★ cancel the previous
    timer = setTimeout(() => fn(...args), delay);
  };
}

function throttle(fn, limit) {
  let waiting = false;
  return (...args) => {
    if (waiting) return; // ★ ignore
    fn(...args);
    waiting = true;
    setTimeout(() => (waiting = false), limit);
  };
}
```

Real-world ideas:

DEBOUNCE → a LIFT: it waits until nobody else gets in, THEN closes ★
 THROTTLE → a BUS: it leaves every 10 minutes regardless of who arrives ★

Search box → DEBOUNCE ★ (don't search per keystroke)
 Scroll handler → THROTTLE ★ (must respond, but not 100×/second)

★ Note both are built on **closures** — the timer variable survives between calls.
 That ties two interview questions together.

Easy memory

DEBOUNCE ★ = wait for the PAUSE, run ONCE (a lift) → SEARCH, resize, autosave
 THROTTLE ★ = at most once per interval (a bus) → SCROLL, mousemove

Both use a CLOSURE to hold the timer ★
 RxJS: debounceTime / throttleTime ★

Q: What is memoization?

Memoization = CACHE a function's result, keyed by its arguments, so the same input never recomputes. ★

```
function memoize(fn) {
  const cache = new Map(); //
  ★ a closure holds the cache
  return (...args) => {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key); // ★ a HIT – no work
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}

const slowSquare = (n) => { /* expensive */ return n * n; };
const fast = memoize(slowSquare);
fast(5); // computed
fast(5); // ★ instant – from the cache
```

Only safe for PURE functions ★
(same input → same output, no side effects)
An impure function would serve a stale result ▲

★ Where you already meet it: **NgRx createSelector** is memoisation → 21 — **NgRx**, and Angular's **pure pipes** are the same idea.

▲ The cache grows forever unless you bound it (an LRU).

Easy memory

Memoize = cache results by ARGUMENTS ★ (a closure holds the Map)
Only for PURE functions ★ | the cache grows forever ▲ (bound it)
You already use it: NgRx createSelector + Angular pure pipes ★

Q: Implement a deep clone.

```
function deepClone(obj, seen = new WeakMap()) {
  if (obj === null || typeof obj !== 'object') return obj; // primitives
  if (obj instanceof Date) return new Date(obj); // ★
  special types
  if (obj instanceof Map) return new Map([...obj].map(([k, v]) => [k,
    deepClone(v, seen)]));
  if (seen.has(obj)) return seen.get(obj); // ★
  CIRCULAR refs

  const clone = Array.isArray(obj) ? [] : {};
  seen.set(obj, clone);
  for (const key of Object.keys(obj)) clone[key] = deepClone(obj[key],
    seen);
  return clone;
}
```

★ The three things that make this a *good* answer rather than a naive one:

1. Handling Date / Map / Set – a naive clone turns a Date into {} ▲
2. A WeakMap for CIRCULAR references – otherwise infinite recursion ▲
3. Saying "in production I'd use structuredClone()" ★

```
const copy = structuredClone(obj); //
★ built-in, handles all of the above
```

Easy memory

Recurse + handle Date/Map/Set + a WeakMap for CIRCULAR refs ★
Production: structuredClone(obj) ★
▲ JSON.parse(JSON.stringify()) loses Date/undefined/functions ▲

Q: Implement a generic curry function.

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) return fn(...args); // ★
    enough args → run
    return (...next) => curried(...args, ...next); // ★ else
    collect more
  };
}

const add = (a, b, c) => a + b + c;
const c = curry(add);
c(1)(2)(3); // 6
c(1, 2)(3); // 6 ★ any grouping works
c(1)(2, 3); // 6
```

`fn.length` = the number of DECLARED parameters ★ – that's the trick ★

Easy memory

curry: collect args until `args.length >= fn.length` ★
`fn.length` = the declared parameter count ★
`c(1)(2)(3)` and `c(1,2)(3)` both work

Q: What is event delegation?

Event delegation = attach ONE listener to a PARENT instead of one per child, and use `event.target`. ★

```
// ✗ 1,000 listeners
document.querySelectorAll('.item').forEach(el =>
  el.addEventListener('click', handler));

// ✓ ONE listener ★
document.querySelector('#list').addEventListener('click', (e) => {
  const item = e.target.closest('.item'); // ★ closest() handles
  inner elements
  if (item) handleClick(item.dataset.id);
});
```

- ✓ ONE listener instead of 1,000 → far less memory ★
- ✓ Works for elements added LATER ★ ★ (no re-binding after a render)

★ That second point is the real reason: dynamically added rows work automatically.

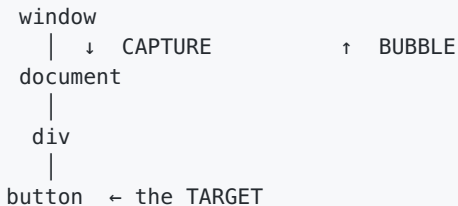
It relies on **bubbling** — the click travels up from the child to the parent.

Easy memory

- ONE listener on the PARENT + `e.target.closest()` ★
- ✓ less memory | ✓ works for elements added LATER ★ ★
- Relies on BUBBLING ★

Q: Explain event bubbling and capturing.

- CAPTURING → the event travels DOWN, window → target (phase 1)
- TARGET → it reaches the element (phase 2)
- BUBBLING → it travels UP, target → window ★ (phase 3, the DEFAULT)



- `el.addEventListener('click', fn);` // BUBBLING (default) ★
- `el.addEventListener('click', fn, true);` // CAPTURING ★

- `e.stopPropagation()` → stop it travelling further ★
- `e.preventDefault()` → stop the DEFAULT behaviour (form submit, link navigation) ★

▲ Those two are **different things** and get confused constantly:

stopPropagation → the event stops moving, but the default STILL happens ★
preventDefault → the default is cancelled, but the event KEEPS bubbling ★

Easy memory

CAPTURE ↓ → TARGET → BUBBLE ↑ (the default) ★
addEventListener(type, fn, true) → capture phase ★

stopPropagation = stop TRAVELLING ★ | preventDefault = stop the DEFAULT ACTION ★
Bubbling is what makes event delegation possible ★

Tricky Output-Based Questions

★ Your rounds have been output-based — treat these as the JavaScript equivalent of [23 — Java Predict the Output](#).

Q1

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

Output: 3 3 3

`var` is FUNCTION-scoped → all three callbacks share ONE `i`
The loop finishes (i = 3) BEFORE any timer fires ★
Fix: `let` → block-scoped → a NEW binding per iteration → 0 1 2 ★

Q2

```
console.log(1);  
setTimeout(() => console.log(2), 0);  
Promise.resolve().then(() => console.log(3));  
console.log(4);
```

Output: 1 4 3 2 ★


```
1, 4 → synchronous
3 → Microtask – drained before any macrotask ★
2 → Macrotask – last, even at 0 ms ★
```

Q3

```
console.log(typeof null);
console.log(0.1 + 0.2 === 0.3);
console.log([] + []);
console.log(!!"false");
```

Output: "object", false, "", true

```
typeof null === "object" → a legacy language bug ★
0.1 + 0.2 = 0.30000000000000004 → IEEE-754 binary fractions ★
[] + [] → both coerce to "" → "" ★
!!"false" → a NON-EMPTY string is truthy ★
```

Q4

```
const obj = {
  name: "Priya",
  regular: function () { return this.name; },
  arrow: () => this.name,
};
console.log(obj.regular());
console.log(obj.arrow());
```

Output: "Priya", undefined

```
regular → called with a DOT → `this` is obj ★
arrow → NO own `this` → inherits the module/global scope → undefined ★
```

Q5

```
let a = { n: 1 };
let b = a;
a.n = 2;
a = { n: 3 };
console.log(b.n);
```

Output: 2

```
b points at the ORIGINAL object  
a.n = 2 → MUTATES the shared object → b.n is 2 ★  
a = {n:3} → REASSIGNS `a` only → b still points at the first object ★
```

★ Exactly the same idea as Java's pass-by-value → [05 — Core Java](#).

Quick Revision Sheet

FUNDAMENTALS

var (function scope, leaks ▲) | let | const ★ (locks the BINDING, not contents)

Hoisting: declarations move up; var → undefined ▲, let/const → TDZ ★

LEXICAL scope = where the code is WRITTEN ★ → the basis of closures

7 primitives (by value) + object (by reference) ★

typeof null === "object" ▲ | == converts ▲ | === always ★

8 FALSYS: false 0 -0 0n "" null undefined NaN ★ ([and {} are TRUTHY!

▲)

FUNCTIONS

CLOSURE ★★ = a function + the variables it was born with

→ private data, factories, debounce/throttle | cost: they stay in memory

HOF: map (transform) filter (keep) reduce (combine) ★

Currying: f(a)(b)(c) → partial application ★

call (Comma) | apply (Array) | bind (later) ★

ARROW: no own `this` ★, no arguments, no new → use for CALLBACKS

`this`: new → call/bind → obj.method() (the DOT) → undefined | arrow = lexical

"NO DOT, NO THIS" ★

PROTOTYPES

Lookup walks UP: obj → prototype → ... → null ★

`class` is SUGAR over prototypes ★ | Object.create(null) = a safe dictionary ★

SHALLOW { ...obj } shares nested objects ▲ | DEEP: structuredClone ★

ASYNC

Promise: PENDING → FULFILLED/REJECTED, settled is FINAL ★ | EAGER (Observable = lazy)

Chaining: forgetting `return` gives undefined

▲ | ONE .catch covers the chain ★

async/await: sequential awaits = the SUM → Promise.all = the SLOWEST

▲ ★

all (all-or-nothing) | allSettled ★ | race (TIMEOUT ★) | any

EVENT LOOP ★★ : sync → ALL microtasks → ONE macrotask → repeat

→ "1 4 3 2" ★ | setTimeout(fn,0) still loses to a promise ★

ES6+

Destructure: object by NAME ★, array by POSITION ★

SPREAD expands / REST collects – position decides ★

Map (any key ★) | Set (unique ★ → [...new Set(arr)])

?. safe access ★ | ?? only null/undefined ★ (|| also catches 0 and ""

▲)

Modules: own scope + STATIC imports → tree-shaking ★ | import() = lazy

ADVANCED

DEBOUNCE ★ wait for the pause (search) | THROTTLE ★ fixed rhythm (scroll)

Memoize = cache by args ★ (= NgRx createSelector)

Event delegation ★ = ONE parent listener → works for elements added

LATER ★ ★

Bubbling ↑ (default) | stopPropagation ≠ preventDefault ★

Related files: [19 — JavaScript Variables Made Easy](#) · [03 — TypeScript](#) · [04 — Angular](#) · [20 — RxJS](#) · [09 — Coding Problems](#)

HTML & CSS — Interview Q&A (Easy Version)

Every question follows the same shape — **Easiest way to remember** → **simple explanation** → **real-world example** → **diagram** → **interview trap** → **Easy memory box**.

● Your Angular/Ionic work uses **Ionic CSS variables + Shadow DOM**, **PrimeNG** and **Angular Material** — those specifics are flagged where they matter.

Contents

1. [HTML](#)
 2. [CSS](#)
 3. [Layout](#)
 4. [Responsive](#)
 5. [Modern / Advanced](#)
 6. [Quick Revision Sheet](#)
-

HTML

Q: What is semantic HTML and why does it matter?

The easiest way to remember:

Semantic HTML = tags that describe the **MEANING** of the content,
not just how it looks. ★

```

<!-- ✗ NON-semantic – a screen reader sees nothing but boxes -->
<div class="header">
  <div class="nav">...</div>
</div>
<div class="main">
  <div class="article">...</div>
</div>

<!-- ✓ SEMANTIC -->
<header>
  <nav>...</nav>
</header>
<main>
  <article>...</article>
</main>
<footer>...</footer>

```

```

<header> <nav> <main> <article> <section> <aside> <footer>
<figure> <figcaption> <time> <mark> <details> <summary>

```

Why it matters — three reasons

1. ACCESSIBILITY ★ – a screen reader can jump straight to `<main>` or list the `<nav>` landmarks. With `<div>` soup it cannot ★
2. SEO – search engines weight `<h1>` and `<article>` content
3. MAINTAINABILITY – `<footer>` tells the next developer what it is; `<div class="fp-b">` does not

Real-world idea: **labelled boxes when you move house.** "KITCHEN" and "BOOKS" versus twenty identical unlabelled boxes. The contents are the same; only one version is usable.

★ The strongest single point: `<main>` **gives a keyboard user a "skip to content" target.** Without it, they tab through the entire navigation on every page.

Easy memory

Semantic = tags with MEANING ★ (labelled moving boxes)
 header nav main article section aside footer

Why: ACCESSIBILITY ★ (screen-reader landmarks) + SEO + readability
`<div class="header">` works visually – and tells assistive tech NOTHING ★

Q: Block vs inline elements?

BLOCK → starts on a NEW LINE, takes the FULL width ★
INLINE → flows within the text, only as wide as its content ★
INLINE-BLOCK → flows inline, but ACCEPTS width/height ★

BLOCK	INLINE
<div>	text text [span] text text ↑ no line break, sits in the flow
<p>	

BLOCK : div p h1-h6 ul ol li section article form
INLINE : span a strong em img label input code

▲ The trap that gets asked:

An INLINE element IGNORES width, height, and vertical margins ★

`<span style="width: 200px">` → the width does NOTHING ▲
Fix: `display: inline-block` ★ (or `block`, or `flex`)

Real-world idea: **paragraphs vs words**. A paragraph occupies its own line; a word sits inside the sentence.

Easy memory

BLOCK → new line, full width ★ (a paragraph)
INLINE → sits in the flow, content-sized ★ (a word)
INLINE-BLOCK → in the flow BUT accepts width/height ★

▲ Inline IGNORES width/height and vertical margins ▲ → use `inline-block`

Q: `<div>` vs `<span>` ?

`<div>` → a BLOCK container → layout, sections ★
`<span>` → an INLINE container → styling part of a line ★

```
<div>A whole section</div>
<p>Only <span class="highlight">this word</span> is highlighted.</p>
```

Both are SEMANTICALLY EMPTY ★ – they carry no meaning.
Prefer a semantic tag whenever one exists ★

Easy memory

div = BLOCK box (layout) | span = INLINE box (part of a line) ★
Both mean NOTHING semantically → prefer header/nav/article when they fit
★

Q: What are `data-*` attributes?

`data-*` = your own custom data on an element, without breaking HTML validity ★

```
<button data-user-id="42" data-role="admin">Delete</button>
```

```
btn.dataset.userId;      // "42"    ★ data-user-id → dataset.userId
                           (camelCase)
btn.dataset.role;         // "admin"
```

```
button[data-role="admin"] { border: 2px solid red; } /*
★ styleable too */
```

★ Where they earn their place: **event delegation** — one listener on the parent reads `e.target.dataset.id`. → [01 — JavaScript](#)

▲ In Angular you rarely need them — you have property binding and template variables. Say that:

"In Angular I'd bind a property or use a template reference instead; `data-*` matters most in plain DOM code and for hooks that tools like Cypress can target."

Easy memory

`data-user-id="42" → element.dataset.userId` ★ (kebab → camelCase)
Valid HTML, styleable with `[data-x="y"]` ★
Main use: event delegation + test hooks. In Angular, prefer bindings ★

Q: What are meta tags and which matter?

```
<meta charset="UTF-8">                                <!-- ★
always first -->
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<!-- ★ responsive -->
<meta name="description" content="...">              <!-- SEO
snippet -->
<meta http-equiv="Content-Security-Policy" content="..."> <!--
security -->
<meta property="og:title" content="...">             <!--
★ link previews -->
```

`charset` ★ → without it, non-English characters break (mojibake) ▲
`viewport` ★ → WITHOUT IT, a mobile browser renders a 980px desktop page
and zooms out → your media queries never fire ▲
`description` → the snippet Google shows
`og:*` ★ → the card WhatsApp/LinkedIn shows when the link is shared

★ **The viewport tag is the one to emphasise** — it's the single most common reason "my responsive site doesn't work on mobile."

Easy memory

`charset` ★ (UTF-8, first) | `viewport` ★ (responsive – without it, NOTHING works)
`description` (SEO) | `og:*` ★ (WhatsApp/LinkedIn previews)

Q: **async** vs **defer** on a script tag?

The easiest way to remember:

(neither) → download BLOCKS parsing, then executes

async → downloads in parallel, executes THE MOMENT it's ready ▲
(order NOT guaranteed)

defer → downloads in parallel, executes AFTER parsing, IN ORDER ★

NORMAL parse —■■■■ download ■■■■ execute — parse blocks

async parse ————— ▲ execute — parse

▲ interrupts, any order

defer parse ————— parse — execute ★ in order
(downloading in parallel the whole time)

async → INDEPENDENT scripts: analytics, ads ★

defer → scripts that depend on the DOM or on each other ★ (the safe default)

▲ **async** with two scripts where B depends on A is a race condition — whichever downloads first runs first.

Easy memory

none → blocks parsing

async → runs ASAP ▲ order NOT guaranteed → analytics

defer ★ → runs after parsing, IN ORDER → the safe default

Both download in PARALLEL ★

Q: **localStorage** vs **sessionStorage** vs cookies?

	localStorage	sessionStorage	cookies
Lifetime	forever until cleared ★	until the tab closes ★	an expiry you set
Size	~5–10 MB	~5–10 MB	~4 KB ▲
Sent to the server	✗	✗	✓ on every request ▲

	localStorage	sessionStorage	cookies
Scope	per origin	per tab ★	per domain/path
Accessible to JS	✓	✓	✓ unless HttpOnly ★

localStorage → a user preference, a theme, a cached list ★
 sessionStorage → per-tab state (a wizard step, a draft) ★
 cookies → things the SERVER must see ★ (session id, auth)

★ The security answer to give:

An access token in localStorage is readable by ANY JavaScript on the page, so one XSS bug leaks it. ▲

An HttpOnly + Secure + SameSite cookie CANNOT be read by JavaScript ★
 → the safer place for a session, at the cost of CSRF protection work

● In your RoboGebra mobile app

@capacitor/preferences is used instead of localStorage ★
 → because the OS can EVICT WebView storage under memory pressure, silently logging the user out ▲

→ 15 — Ionic

Easy memory

localStorage ★ forever | sessionStorage ★ per TAB | cookies ★ sent to the SERVER (4 KB)

▲ A token in localStorage is XSS-readable ▲ → HttpOnly cookie is safer ★
 On native, use Capacitor Preferences — the OS can wipe WebView storage ★

Q: Common HTML5 input types?

```
<input type="email">      <input type="tel">      <input type="url">
<input type="number">    <input type="date">    <input type="time">
<input type="range">    <input type="color">  <input type="search">
<input type="file" accept="image/*">
```

★ The real benefit on mobile — worth saying:

```
type="email" → the keyboard shows @ ★
type="tel" → a NUMERIC keypad ★
type="number" → a numeric keypad + spinners
↓
Using type="text" for a phone number makes the user hunt for digits ▲
```

▲ Never trust client validation:

```
type="email" is a convenience, not a security control.
Anyone can bypass it with devtools or curl → ALWAYS validate on the
server ★
```

Easy memory

```
email tel url number date time range color file search
The real win: the right MOBILE KEYBOARD ★ (tel → numeric keypad)
▲ Client validation is UX only — always validate server-side ★
```

Q: What are the basics of accessibility (ARIA)?

```
ARIA = extra attributes that tell assistive technology what an element
IS and what STATE it's in — when the HTML alone can't. ★
```

```
<button aria-label="Close dialog">x</button>           <!-- ★ "x" alone
means nothing -->
<div role="alert">Save failed</div>                     <!-- announced
immediately -->
<input aria-describedby="hint"> <span id="hint">8+ characters</span>
<button aria-expanded="false" aria-controls="menu">Menu</button> <!--
★ state -->
       <!-- ★ describe,
don't label -->
```

★ The first rule of ARIA — say this, it's the answer that shows real knowledge:

"The best ARIA is NO ARIA." ★

Use `<button>` instead of `<div role="button" tabindex="0" @keydown...>`.
A real `<button>` is focusable, keyboard-activatable and announced –
for free. ARIA is a patch for when semantic HTML can't express it. ★

The checklist worth naming:

- ✓ Semantic tags first ★
- ✓ alt text that DESCRIBES (or `alt=""` for decorative images ★)
- ✓ Keyboard reachable – everything usable with Tab and Enter ★
- ✓ Visible focus outline (never `outline: none` with no replacement ▲)
- ✓ Colour contrast $\geq 4.5:1$ for body text
- ✓ Labels tied to inputs (`<label for>` or `aria-label`)

Easy memory

"The best ARIA is NO ARIA" ★ – use a real `<button>`, not `div role="button"`
`aria-label (name) | role (what it is) | aria-expanded (state) ★`
`alt describes ★ | alt=""` for decorative
Keyboard + visible focus + 4.5:1 contrast ★

Q: What is `<!DOCTYPE html>` ?

It tells the browser to use STANDARDS MODE. ★
Leave it out and you get QUIRKS MODE – 1990s box-model behaviour. ▲

Standards mode → width = the CONTENT width (the real box model) ★
Quirks mode → width INCLUDES padding and border (the old IE model) ▲
→ every layout shifts

It is not an HTML tag and has no attributes — just the first line of the document.

Easy memory

`<!DOCTYPE html>` → STANDARDS mode ★
Missing it → QUIRKS mode → the OLD box model → layouts break ▲

Q: What is an iframe and what are its concerns?

```
<iframe src="https://x.com" title="Report" sandbox loading="lazy"></iframe>
```

- ✓ Embed third-party content (maps, videos, payment forms)
- ✗ SECURITY – clickjacking, and the framed site can attempt things ▲
- ✗ Performance – a whole second document to load
- ✗ SEO – the content isn't part of your page
- ✗ Accessibility – needs a `title` ★

sandbox → strip permissions from the framed page ★
X-Frame-Options / CSP frame-ancestors → stop OTHERS framing YOU ★ (anti-clickjacking)
loading="lazy" → don't load it until it's near the viewport

★ **Clickjacking** is the concern to name: an attacker frames your page invisibly over a decoy button, so the user's click lands on *your* "Confirm payment".

Easy memory

iframe = an embedded document
▲ CLICKJACKING ★ → defend with X-Frame-Options / CSP frame-ancestors
sandbox to restrict it | title for accessibility ★ | loading="lazy"

Q: On-page SEO basics?

- ✓ ONE <h1> per page, then a logical h2/h3 hierarchy ★
- ✓ <title> and <meta name="description">
- ✓ Semantic structure (<main>, <article>) ★
- ✓ alt text on images
- ✓ Clean, readable URLs
- ✓ Fast load + mobile-friendly (Core Web Vitals) ★
- ✓ Structured data (schema.org JSON-LD)

▲ **The SPA problem — worth raising:**

An Angular SPA ships an EMPTY `<div id="app"></div>`.
A crawler that doesn't execute JavaScript sees NOTHING ▲
↓
Fix: Angular Universal (SSR) or prerendering ★

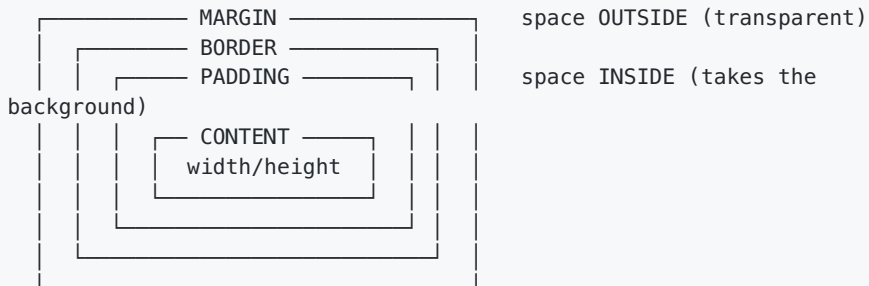
Easy memory

One `<h1>` ★ | title + description | semantic tags | alt | fast + mobile
▲ An SPA renders empty HTML → crawlers see nothing → SSR/prerender ★

CSS

Q: Explain the CSS box model.

Every element is FOUR nested boxes: ★



★ The question they actually want — **box-sizing** :

```
/* DEFAULT: content-box */
width: 200px; padding: 20px; border: 5px;
→ ACTUAL width = 200 + 40 + 10 = 250px ▲ ★

/* border-box ★ */
box-sizing: border-box;
width: 200px; padding: 20px; border: 5px;
→ ACTUAL width = 200px ✓ (padding and border grow INWARD)
```



```
*, ::before, ::after { box-sizing: border-box; } /* ★ every modern reset does this */
```

Real-world idea: **a gift box**. **content-box** measures only the item, so wrapping paper makes the parcel bigger. **border-box** measures the *finished parcel*.

Easy memory

content → padding → border → margin ★

content-box (default) → width = CONTENT only → padding ADDS to it ▲
border-box ★ → width = the WHOLE box → padding grows INWARD ✓

Every reset starts with: `* { box-sizing: border-box; } ★`

Q: What is margin collapsing?

Two VERTICAL margins that touch MERGE into one – the LARGER wins, they don't add up. ★

```
<div style="margin-bottom: 30px">A</div>
<div style="margin-top: 20px">B</div>

<!-- gap = 30px, NOT 50px ★ -->
```

Only VERTICAL margins collapse. Horizontal ones NEVER do ★

It happens in three places:

1. Between ADJACENT siblings ★
2. Between a PARENT and its FIRST/LAST child (the margin "escapes" the parent) ▲
3. On an EMPTY element (its own top and bottom margins collapse together)

★ Case 2 is the confusing one:

```
<div class="parent">
  <div style="margin-top: 50px">child</div>
</div>
<!-- the PARENT moves down 50px, not the child ▲ -->
```

Fixes: give the parent **padding**, a **border**, **overflow: hidden**, or make it a **flex/grid container** — margins never collapse inside flex or grid ★.

Easy memory

Adjacent VERTICAL margins MERGE → the LARGER wins ($30 + 20 = 30$ ★, not 50)

Horizontal margins NEVER collapse ★

▲ A child's margin can ESCAPE and move the PARENT ▲

Fix: padding/border on the parent, or use FLEX/GRID (no collapsing there) ★

Q: Explain CSS **position** values.

static → the default; in the normal flow; top/left do NOTHING ★

relative → in the flow, but nudged from its own position ★

absolute → REMOVED from the flow, positioned against the nearest POSITIONED ancestor ★

fixed → removed; positioned against the VIEWPORT (stays on scroll)

sticky → relative until it hits a threshold, then fixed ★

NORMAL FLOW

static, relative
(still occupies space)

REMOVED FROM FLOW

absolute, fixed
(leaves a gap behind it) ★

★ The rule that makes **absolute** click:

`absolute` positions itself against the nearest ancestor with position OTHER than static ★

No positioned ancestor? → it jumps to the whole PAGE ▲

↓

Which is why the pattern is ALWAYS:

```
parent { position: relative }
child { position: absolute; top: 0; right: 0 } ★
```

sticky needs a threshold or it does nothing:

```
position: sticky; top: 0; /* ★ `top` is REQUIRED */
```

▲ And **sticky** silently fails if any ancestor has **overflow: hidden** — a classic waste of an afternoon.

Easy memory

static (default) | relative (nudge, keeps its space) ★
absolute (out of flow, vs the nearest POSITIONED ancestor ★)
fixed (vs the VIEWPORT) | sticky (relative → fixed at a threshold ★)

THE PATTERN: parent relative + child absolute ★

▲ sticky needs `top:` and breaks under an ancestor's overflow:hidden ▲

Q: What are the main **display** types?

block	→ new line, full width
inline	→ in the flow, ignores width/height ▲
inline-block	→ in the flow, ACCEPTS width/height ★
flex	→ 1-DIMENSIONAL layout (a row OR a column) ★
grid	→ 2-DIMENSIONAL layout (rows AND columns) ★
none	→ REMOVED from the page entirely (no space kept) ★
contents	→ the box disappears; the children stay (useful in grid)

Easy memory

block | inline ▲ | inline-block ★ | flex (1-D ★) | grid (2-D ★) | none (gone ★)

Q: How is CSS specificity calculated?

Count FOUR numbers: (inline, IDs, classes, elements) ★
Compare left to right – a higher number always wins.

inline style	1,0,0,0	★	beats everything except !important
#id	0,1,0,0	★	
.class / [attr] / :hover	0,0,1,0		
element / ::before	0,0,0,1		
*	0,0,0,0		

#nav .item a	→	0,1,1,1	★	WINS
.nav .item a.link	→	0,0,3,1		
a	→	0,0,0,1		

★ The rule that surprises people:

ONE id beats ANY number of classes. ★
#nav (0,1,0,0) > .a.b.c.d.e.f.g.h.i.j (0,0,10,0)

The columns are compared LEFT TO RIGHT, never added up ★

Tie-break: **the last rule in the source order wins.**

Real-world idea: **military rank**. One General outranks any number of Sergeants – you don't add sergeants together.

Easy memory

(inline, ID, class, element) – compare LEFT to RIGHT ★
inline 1,0,0,0 | #id 0,1,0,0 | .class 0,0,1,0 | tag 0,0,0,1

ONE id beats TEN classes ★ (military rank – you don't add up sergeants)
Tie → the LAST rule wins

Q: What does !important do?

!important overrides EVERYTHING except another !important (and inline !important). ★

The cascade, highest first:

1. !important on an inline style
2. !important in a stylesheet ★
3. inline style
4. #id → .class → element

▲ Why it's discouraged:

It breaks the cascade, so the ONLY way to override it is ANOTHER !important

↓

an escalating war of !important across the codebase ▲

✓ Legitimate uses: overriding a third-party library you can't edit ★ , utility classes (.hidden { display: none !important }), print stylesheets

● Relevant to your stack

Overriding **PrimeNG** or **Angular Material** internals is exactly the case where **!important** (or **::ng-deep**, which is deprecated) shows up — worth naming as the honest exception.

Easy memory

!important beats the cascade ★ → the only counter is another !important

▲ Legit: third-party overrides ★ , utility classes, print styles
Otherwise it's a specificity war

Q: Inline vs internal vs external CSS?

INLINE → style="" on the element	→ highest specificity ▲ ,
unreusable	
INTERNAL → <style> in the <head>	→ one page only
EXTERNAL → <link> to a .css file	→ reusable + CACHED ★

EXTERNAL wins because the browser CACHES it ★
→ the second page load costs nothing

★ The exception worth naming: **critical CSS is deliberately inlined** in the `<head>` so the first paint doesn't wait for a network round trip.

Easy memory

external ★ (reusable + CACHED) > internal > inline ▲
Exception: CRITICAL CSS is inlined on purpose, for the first paint ★

Q: Pseudo-classes vs pseudo-elements?

PSEUDO-CLASS : one colon → a STATE of an existing element ★
PSEUDO-ELEMENT :: two colons → a NEW element that isn't in the HTML ★

```
/* pseudo-CLASS – a state */
a:hover input:focus li:first-child li:nth-child(2n) input:disabled
:not(.active) li:last-child

/* pseudo-ELEMENT – creates something */
p::before { content: "→"; } /* ★ `content` is REQUIRED */
p::after
p::first-line p::first-letter ::placeholder ::selection
```

★ The memory hook: **one colon = a condition, two colons = a creation.**

▲ `::before` and `::after` **do nothing without `content`** — even `content: ""`.

Easy memory

:hover ONE colon = a STATE ★ (a condition)
::before TWO colons = a NEW element ★ (a creation)
▲ `::before/::after` need ``content:`` — even `content: ""` ★

Q: em vs rem vs px vs %?

px → a fixed, absolute size ★
em → relative to the PARENT's font size ▲ (it COMPOUNDS)
rem → relative to the ROOT font size ★ (predictable – the default choice)
% → relative to the parent's corresponding property

▲ The **em** compounding trap:

```
<div style="font-size: 2em">          <!-- 32px (root 16) -->
  <div style="font-size: 2em">        <!-- 64px ▲ not 32 -->
    <div style="font-size: 2em">      <!-- 128px ▲ ▲ -->
```

em MULTIPLIES down the tree ★
rem always measures from the ROOT → no surprises ★

Use rem
★ → font sizes, spacing (it also respects the user's browser font setting ★)
Use em → padding that should scale WITH this element's own text ★
Use px → borders, hairlines, precise 1px details
Use % → widths in fluid layouts

★ The accessibility point: **a user who sets a larger default font size gets it with rem, and is ignored by px.**

Easy memory

px = fixed | em = vs the PARENT ▲ COMPOUNDS | rem ★ = vs the ROOT | % = vs the parent

rem for font sizes and spacing ★ (respects the user's browser setting)
em for padding that scales with its own text

Q: What is **z-index** and a stacking context?

z-index = the front-to-back order – but ONLY within a stacking context ★
And it only works on a POSITIONED element (not `static`) ▲

★ The trap that produces "my z-index: 9999 doesn't work":

A new STACKING CONTEXT is created by:
position + a z-index other than auto ★
opacity < 1 ★ ★ (the sneaky one)
transform, filter, will-change ★
position: fixed / sticky

```
Parent A (z-index: 1)      Parent B (z-index: 2)
└─ child (z-index: 9999)  └─ child (z-index: 1)
```

Parent B's child WINS ★ – the children are compared INSIDE their parents, and the PARENTS are compared first ▲

Real-world idea: **floors in a building**. A person standing on a chair on the 1st floor is still below someone lying on the 2nd floor. **z-index** only orders you *within your own floor*.

▲ The one that wastes hours: adding **opacity: 0.99** or a **transform** for an animation **creates a stacking context** and silently traps every child behind a sibling.

Easy memory

z-index works ONLY on POSITIONED elements ▲
It's scoped to the STACKING CONTEXT ★ (floors in a building)

Created by: position+z-index | opacity < 1 ★ ★ | transform | filter | fixed/sticky
"z-index: 9999 doesn't work" → the PARENT's context is losing ★

Layout

Q: Explain Flexbox and its main properties.

Flexbox = ONE-DIMENSIONAL layout – a row OR a column. ★

```
.container {
  display: flex;
  flex-direction: row | column;           /* ★ sets the MAIN axis */
  justify-content: center | space-between; /* ★ along the MAIN axis */
  align-items: center | stretch;         /* ★ across the CROSS axis */
  /*
  flex-wrap: wrap;
  gap: 16px;                             /* ★ modern spacing */
}
.item { flex: 1; }                       /* grow | shrink | basis */
```


★ The one thing that makes flexbox click:

justify-content → the MAIN axis (the direction you set) ★
align-items → the CROSS axis (the other one) ★

flex-direction: row → main = horizontal ↔, cross = vertical †
flex-direction: column → main = VERTICAL †, cross = HORIZONTAL ↔ ★ (they SWAP!)

That swap is why "justify-content: center didn't centre it horizontally"
– because the direction was `column` ▲ ★

row: [1][2][3] → justify = ↔ align = †
column: [1]
[2] → justify = † align = ↔ ★ swapped
[3]

flex: 1 explained:

flex: 1 = flex-grow: 1; flex-shrink: 1; flex-basis: 0 ★
→ "take an equal share of the free space"

Easy memory

Flexbox = ONE dimension (row OR column) ★
justify-content = MAIN axis ★ | align-items = CROSS axis ★
flex-direction: column SWAPS them ▲ ★ – the #1 confusion
flex: 1 = grow 1, shrink 1, basis 0 ★ | gap for spacing

Q: Explain CSS Grid basics.

Grid = TWO-DIMENSIONAL layout – rows AND columns at the same time. ★

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);      /* ★ 3 equal columns */
  grid-template-columns: 200px 1fr auto;      /* mixed */
  grid-template-rows: auto 1fr auto;
  gap: 16px;
}
.item {
  grid-column: 1 / 3;          /* span columns 1-2 ★ */
  grid-area: header;
}
```

★ Two things worth knowing beyond the basics:

1. `fr` = a fraction of the FREE space ★ – it accounts for gaps, which percentages don't ▲
2. RESPONSIVE WITH NO MEDIA QUERY ★ ★
`grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));`
 → as many columns as fit, each at least 250px, wrapping automatically

That second line is the single most impressive thing to write on a whiteboard for a layout question.

Named areas — very readable:

```
grid-template-areas:
  "header header"
  "sidebar main"
  "footer footer";
```

Easy memory

Grid = TWO dimensions ★ (rows AND columns)
`repeat(3, 1fr)` | fr = a share of the FREE space ★ (percentages ignore gaps ▲)

★ ★ `repeat(auto-fit, minmax(250px, 1fr))` = responsive with NO media query
`grid-template-areas` = a readable ASCII layout ★

Q: Flexbox vs Grid — when to use which?

FLEX → ONE direction, content-driven ★ (a nav bar, a toolbar, a card's footer)
GRID → TWO directions, layout-driven ★ (a page skeleton, a photo gallery)

FLEX: "put these items in a line and space them"
GRID: "here is the page structure; put things in the cells" ★

★ The line to say: *"Flexbox is content-first — the items decide the layout. Grid is layout-first — the container decides, and the items fill it."*

And they compose: a **grid** page skeleton with **flex** inside each cell is the standard combination.

Easy memory

FLEX = 1-D, content-first ★ (navbar, toolbar, button row)
GRID = 2-D, layout-first ★ (page skeleton, gallery, dashboard)
Use BOTH: grid for the page, flex inside each cell ★

Q: How do you centre a div?

★ The interviewer wants **more than one** answer.

```
/* 1. FLEX — the modern default ★ */
.parent { display: flex; justify-content: center; align-items: center; }

/* 2. GRID — the shortest of all ★ */
.parent { display: grid; place-items: center; }

/* 3. ABSOLUTE + transform — works without touching the parent's layout
★ */
.child { position: absolute; top: 50%; left: 50%; transform:
translate(-50%, -50%); }

/* 4. HORIZONTAL only — a block with a known width */
.child { margin: 0 auto; width: 300px; }
```

- ★ Why ``translate(-50%, -50%)`` is needed in #3:
 top: 50% puts the child's TOP EDGE at the middle.
`translate(-50%, -50%)` pulls it back by HALF ITS OWN SIZE ★
 → and % on transform refers to the ELEMENT's own size, which is
 why it works without knowing the width

Easy memory

```
place-items: center ★ (grid – the shortest)
flex + justify-content + align-items ★
absolute + top/left 50% + translate(-50%, -50%) ★ (half its OWN size)
margin: 0 auto (horizontal only, needs a width)
```

Responsive

Q: What are media queries?

```
@media (min-width: 768px) { } /* ★ mobile-first: styles ADD as it grows */
@media (max-width: 767px) { } /* desktop-first */
@media (orientation: landscape) { }
@media (prefers-color-scheme: dark) { } /* ★ dark mode */
@media (prefers-reduced-motion: reduce) { } /* ★ accessibility */
@media print { }
```

- ★ The last two are the ones that make you sound current:

```
prefers-color-scheme → respect the user's OS dark mode ★
prefers-reduced-motion → users with vestibular disorders can be made
                        physically unwell by animation ★ – disable it
```

Easy memory

```
@media (min-width: 768px) ★ mobile-first | max-width = desktop-first
prefers-color-scheme ★ (dark mode) | prefers-reduced-motion ★
(accessibility)
```

Q: What is mobile-first design?

Mobile-first = write the MOBILE styles as the base,
then ADD complexity with min-width. ★

```
/* base = MOBILE ★ */
.container { padding: 16px; }

@media (min-width: 768px) { .container { padding: 32px; } } /* tablet
*/
@media (min-width: 1024px) { .container { padding: 48px; } } /* desktop
*/
```

MOBILE-FIRST ★	DESKTOP-FIRST
base = simple	base = complex
ADD as it grows ★	REMOVE as it shrinks
phone downloads less ★	phone parses desktop CSS it will never use ▲

★ The performance argument is the real one: the constrained device gets the **simplest** stylesheet.

Easy memory

Mobile-first ★ = base is MOBILE, then min-width ADDS ★
Why: the weakest device does the LEAST work ★
Desktop-first = max-width, subtracting – the phone parses styles it never uses ▲

Q: Why is the viewport meta tag needed?

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

WITHOUT it: a mobile browser pretends to be ~980px wide, renders the
desktop layout, and ZOOMS OUT ▲
→ your media queries NEVER fire ★

WITH it: the viewport = the real device width → media queries work ✓

▲ Never add `user-scalable=no` — it blocks pinch-zoom, which is an accessibility failure.

Easy memory

```
width=device-width, initial-scale=1 ★  
Without it → the phone fakes 980px and zooms out → media queries never  
fire ▲  
▲ Never user-scalable=no (accessibility) ★
```

Q: What are responsive units?

```
%      → relative to the parent  
vw/vh  → 1% of the viewport WIDTH / HEIGHT ★  
vmin/vmax → the smaller / larger of the two  
rem     → relative to the root font size ★  
ch      → the width of a "0" — great for line length ★  
clamp(min, preferred, max) ★ ★ → fluid, with hard limits
```

```
font-size: clamp(1rem, 2.5vw, 2rem);      /* ★ scales, but never too  
small or huge */  
max-width: 65ch;                          /* ★ the readable line length  
*/
```

★ `clamp()` is the modern answer to "how do you make typography responsive?"
— one line, no media queries.

▲ The `vh` mobile trap:

```
100vh on mobile includes the area BEHIND the browser's address bar  
→ content gets cut off ▲  
Fix: 100dvh (dynamic viewport height) ★
```

Easy memory

```
% | vw/vh ★ | rem ★ | ch (line length ★) | clamp(min, fluid, max) ★ ★  
clamp(1rem, 2.5vw, 2rem) = responsive type with NO media query ★  
▲ 100vh is broken on mobile (the address bar) → use 100dvh ★
```

Q: How does the Bootstrap grid work?

12 COLUMNS ★ , inside .container → .row → .col-*

```
<div class="container">
  <div class="row">
    <div class="col-12 col-md-6 col-lg-4">A</div>    <!-- ★ full → half →
third -->
    <div class="col-12 col-md-6 col-lg-4">B</div>
  </div>
</div>
```

Breakpoints: sm 576 | md 768 | lg 992 | xl 1200 | xxl 1400
Mobile-first ★ – col-md-6 applies from 768px UPWARD ★
The numbers in a row should total 12; beyond that it WRAPS ★

▲ Be honest about your stack:

"My projects use PrimeNG and Angular Material rather than Bootstrap, and Ionic has its own grid — but it's the same 12-column, mobile-first idea."

Easy memory

12 columns ★ | container → row → col-
sm 576 | md 768 | lg 992 | xl 1200
col-md-6 applies from 768 UPWARD ★ (mobile-first)
Your stack: PrimeNG / Material / Ionic grid – same 12-column idea ★

Modern / Advanced

Q: What are CSS variables (custom properties)?

```
:root {
  --primary: #3880ff;
  --spacing: 16px;
}
.button {
  background: var(--primary);
  padding: var(--spacing, 8px);          /* ★ with a fallback */
}
```

★ The two things that make them different from Sass variables:

1. They are LIVE at RUNTIME ★
element.style.setProperty('--primary', 'red'); → changes instantly
(a Sass variable is compiled away and cannot change)
2. They CASCADE and INHERIT ★
.dark-theme { --primary: #fff; }
→ every child using var(--primary) updates automatically ★

That is exactly how DARK MODE is implemented with no duplicate stylesheet
★

● In your RoboGebra / Ionic work — this is *the* answer ★

Ionic components use SHADOW DOM.
↓
Your normal CSS CANNOT reach inside them ▲
↓
Ionic exposes CSS VARIABLES as the styling API ★

```
ion-button { --background: #3880ff; --border-radius: 8px; }
:root { --ion-color-primary: #3880ff; }
```

■ "Theming an Ionic app is done entirely through CSS custom properties, because the components use Shadow DOM — a normal selector can't cross that boundary. That was one of the genuinely new things to learn moving from Angular to Ionic:

you style through the variables the component exposes, or `::part()`, not by targeting its internals."

Easy memory

```
--name in :root, read with var(--name, fallback) ★  
LIVE at runtime ★ + CASCADES ★ (unlike Sass variables)  
→ dark mode with no duplicate stylesheet ★
```

- Ionic: Shadow DOM means CSS variables ARE the styling API ★

Q: Transitions vs animations?

TRANSITION → A → B, needs a TRIGGER (hover, a class change) ★
ANIMATION → keyframes, runs BY ITSELF, can LOOP ★

```
/* TRANSITION – needs a trigger */  
.btn { transition: background 0.3s ease; }  
.btn:hover { background: blue; }  
  
/* ANIMATION – self-running */  
@keyframes spin { from { transform: rotate(0); } to { transform: rotate(360deg); } }  
.loader { animation: spin 1s linear infinite; } /*  
★ loops forever */
```

transition → 2 states, triggered ★
animation → MANY keyframes, automatic, loopable ★

★ The performance rule that gets asked:

Animate ONLY ``transform`` and ``opacity`` ★
→ the GPU handles them (the compositor), 60fps ▲

Animating width/height/top/left triggers LAYOUT on every frame ▲

```
/* ✗ janky */ transition: left 0.3s;  
/* ✓ smooth */ transition: transform 0.3s;
```

Easy memory

TRANSITION ★ = A→B, needs a TRIGGER | ANIMATION ★ = keyframes, self-running, loops

▲ Animate ONLY transform + opacity ★ (GPU, 60fps)
width/height/top/left → LAYOUT every frame → junk ▲

Q: What is the **transform** property?

```
transform: translate(10px, 20px);
transform: rotate(45deg);
transform: scale(1.5);
transform: skew(10deg);
transform: translate(-50%, -50%) rotate(45deg); /* ★ applied RIGHT to LEFT */
```

★ Three points worth knowing:

1. transform does NOT affect layout ★ – surrounding elements don't move, which is exactly why it's cheap to animate
2. Percentages are relative to the ELEMENT'S OWN size ★
→ which is why translate(-50%, -50%) centres without knowing the width
3. Multiple transforms apply RIGHT TO LEFT ★

▲ And the side effect: **transform** creates a stacking context, which can silently break a nearby **z-index**.

Easy memory

translate | rotate | scale | skew ★
Does NOT affect layout ★ → cheap to animate (GPU)
% is relative to the ELEMENT'S OWN size ★ → translate(-50%, -50%) centres
Applied RIGHT to LEFT ★ | ▲ creates a stacking context (z-index surprises)

Q: opacity: 0 vs visibility: hidden vs display: none ?

	opacity: 0	visibility: hidden	display: none
Takes up space	✓	✓	✗ removed ★
Clickable	✓ YES ▲	✗	✗
Animatable	✓ ★	✗ (it's discrete)	✗
Read by a screen reader	✓ ▲	✗	✗

★ The two traps:

1. opacity: 0 is STILL CLICKABLE ★
 - an invisible button intercepting clicks ▲
 - also still read aloud by a screen reader ▲
2. display: none cannot be TRANSITIONED ★
 - the standard fade-out is:
opacity 0 + visibility hidden together ★

```
.fade-out { opacity: 0; visibility: hidden; transition: opacity .3s, visibility .3s; }
```

Easy memory

opacity: 0 → invisible, KEEPS space, STILL CLICKABLE ▲ ★ , animatable ★
visibility: hidden → invisible, keeps space, not clickable
display: none → GONE, no space ★ , cannot be animated ★
Fade-out = opacity + visibility together ★

Q: What is BEM naming?

BEM = Block __ Element -- Modifier ★

```
.card { }                /* BLOCK – a standalone component */
.card__title { }         /* ELEMENT – a part of the block ★ */
.card--featured { }      /* MODIFIER – a variant ★ */
.card__title--large { }
```

Why: FLAT selectors → specificity stays at 0,0,1,0 ★
 → no nesting wars, no !important escalation
 → and the class name tells you exactly where it belongs ★

▲ Honest relevance to your stack:

Angular has COMPONENT-SCOPED styles (ViewEncapsulation) ★
 → the styles are already isolated, so BEM matters far less.
 It's still valuable for a SHARED global stylesheet.

Easy memory

```
.block__element--modifier ★ (.card__title--large)
Why: FLAT specificity ★ + self-documenting names
▲ Angular already scopes styles per component → BEM matters less ★
```

Q: What causes reflow and repaint?

```
REFLOW (layout) → recalculating GEOMETRY – EXPENSIVE ★
REPAINT (paint) → redrawing pixels, same geometry – cheaper
COMPOSITE       → just moving existing layers – CHEAPEST ▲ ★
```

```
REFLOW    ← width, height, padding, margin, font-size, position,
            adding/removing DOM, reading offsetHeight ★
REPAINT    ← color, background, visibility, box-shadow
COMPOSITE  ← transform, opacity ★ ★ (the GPU – no layout, no paint)
```

★ The killer detail — layout thrashing:

```
for (const el of items) {
  el.style.height = el.offsetHeight + 10 + 'px';    //
★ READ then WRITE, in a loop
}
```

Reading `offsetHeight` FORCES a synchronous reflow, because the browser must flush your pending writes to give you an accurate number.

↓

Read-write-read-write = a reflow PER ITERATION ▲ ("layout thrashing")

↓

Fix: READ everything first, THEN write everything ★

Easy memory

REFLOW (geometry) > REPAINT (pixels) > COMPOSITE (transform/opacity) ▲ ★

Animate transform + opacity ONLY ★

▲ LAYOUT THRASHING: reading `offsetHeight` in a write loop forces a reflow every iteration ▲ → batch READS, then WRITES ★

Q: What is the critical rendering path?

HTML → DOM
CSS → CSSOM

} → RENDER TREE → LAYOUT → PAINT → COMPOSITE ★

- ★ CSS is RENDER-BLOCKING – nothing paints until the CSSOM is built
- ★ A plain `<script>` is PARSER-BLOCKING – it stops DOM construction dead

How to make it fast:

- ✓ Inline the CRITICAL CSS, load the rest asynchronously ★
- ✓ defer / async on scripts ★
- ✓ Minify and compress; use HTTP/2
- ✓ Preload key fonts; `font-display: swap` ★ (avoid invisible text)
- ✓ Lazy-load below-the-fold images (`loading="lazy"`)

★ Tie it to Core Web Vitals — that's the modern framing:

LCP → the largest element painted (loading speed)
INP → responsiveness to input (replaced FID in 2024) ★
CLS → layout shift (set width/height on images! ★)

Easy memory

HTML→DOM + CSS→CSSOM → RENDER TREE → LAYOUT → PAINT → COMPOSITE ★

CSS is RENDER-blocking ★ | a plain <script> is PARSER-blocking ★

Fix: critical CSS inline + defer scripts + font-display: swap ★

Core Web Vitals: LCP | INP ★ (replaced FID) | CLS (size your images! ★)

Quick Revision Sheet

HTML

Semantic tags ★ = accessibility landmarks + SEO (labelled boxes)
block (new line) | inline (▲ ignores width/height) | inline-block ★
async (any order ▲) vs defer ★ (in order, after parsing)
localStorage (forever) | sessionStorage (per TAB) | cookie (sent to the server, 4KB)
▲ a token in localStorage is XSS-readable → HttpOnly cookie ★
"The best ARIA is NO ARIA" ★ – use a real <button>
<!DOCTYPE html> → standards mode (else the OLD box model ▲)

CSS

box-sizing: border-box ★ (else padding ADDS to the width ▲)
Vertical margins COLLAPSE → the larger wins ★ (never horizontal)
position: parent RELATIVE + child ABSOLUTE ★ | sticky needs `top:` ★
Specificity (inline, id, class, tag) – ONE id beats TEN classes ★
:hover ONE colon = state | ::before TWO = a new element (needs `content`) ★
rem ★ (root, respects user settings) | em ▲ COMPOUNDS | px | %
z-index only on POSITIONED elements; scoped to the STACKING CONTEXT ★
created by opacity < 1 ★★ and transform ▲

LAYOUT

FLEX = 1-D ★ | GRID = 2-D ★
justify-content = MAIN axis | align-items = CROSS axis
▲ flex-direction: column SWAPS them ▲ ★
repeat(auto-fit, minmax(250px, 1fr)) ★★ = responsive, no media query
Centre: place-items: center ★ | flex | absolute + translate(-50%,-50%) ★
★

RESPONSIVE

Mobile-first
★ = base mobile + min-width (the weakest device does least work)
viewport meta ★ – without it, media queries NEVER fire ▲
clamp(1rem, 2.5vw, 2rem) ★★ | ▲ 100vh is broken on mobile → 100dvh ★

MODERN

CSS variables: LIVE + CASCADE ★ → dark mode
● Ionic Shadow DOM → variables ARE the styling API ★
Animate ONLY transform + opacity ★ (GPU) – never width/top ▲
opacity: 0 is STILL CLICKABLE ▲★ | display: none can't be animated ★
Reflow > repaint > composite ▲ | layout thrashing = read/write in a loop ▲
CSS is render-blocking ★ | Core Web Vitals: LCP, INP ★ , CLS

Related files: [01 — JavaScript](#) · [04 — Angular](#) · [15 — Ionic \(CSS variables + Shadow DOM\)](#) · [25 — Angular Binding & Forms](#)

TypeScript — Interview Q&A (Easy Version)

Every question follows the same shape — **Easiest way to remember** → **simple explanation** → **real-world example** → **diagram** → **interview trap** → **Interview-ready answer** → **Easy memory box**.

● In your RoboGebra code blocks point at real files. Full index: [39](#) — [RoboGebra Code Examples](#).

Your real TypeScript versions:

robogebra-mobile	→ TypeScript 5.4	(Angular 18)
robogebra-web	→ TypeScript 5.1	(Angular 16)
robogebra-backoffice	→ TypeScript 4.9	(Angular 16)
robogebra-crm-web	→ TypeScript 5.8	(React 18)

Contents

1. [Basics](#)
 2. [Functions](#)
 3. [OOP](#)
 4. [Advanced Types](#)
 5. [Decorators, Declaration Files, Modules](#)
 6. [Practical / Angular](#)
 7. [Quick Revision Sheet](#)
-

Basics

Q: What is TypeScript and what are its advantages over JavaScript?

The easiest way to remember:

TypeScript = JavaScript + TYPES, checked at COMPILE time.

It finds your bugs BEFORE the user does. ★

```
// JavaScript – the bug reaches production ▲  
function getArea(w, h) { return w * h; }  
getArea("5", 10);           // "5" * 10 = 50 ... but getArea("a", 10) = NaN,  
silently  
  
// TypeScript – the bug is caught while you type ★  
function getArea(w: number, h: number): number { return w * h; }  
getArea("5", 10);           // ✗ Argument of type 'string' is not  
assignable to 'number'
```

JAVASCRIPT	TYPESCRIPT
write → ship → user hits it	write → ✗ RED SQUIGGLE → fix it ★
▲ 3am call	✓ never shipped

Real-world idea: **a spell-checker while you write** versus discovering the typo after the book is printed.

The advantages worth naming

- ✓ Errors at COMPILE time, not runtime ★
- ✓ IntelliSense / autocomplete – the IDE knows every property
- ✓ SAFE REFACTORING – rename a field and every usage is found ★
- ✓ Self-documenting – the signature says what it needs
- ✓ Modern JS features, compiled down for old browsers
- ✓ Interfaces, generics, enums, access modifiers

★ Two things to be precise about, because interviewers probe them:

1. TypeScript is a SUPERSET – every valid .js file is valid .ts ★
2. Types are ERASED at compile time. There is NO type checking at runtime. ★
 - the compiled output is plain JavaScript

That second point explains a lot later (why an interface can't be a DI token, why `instanceof` doesn't work on an interface).

Interview-ready answer

TypeScript is a strongly typed superset of JavaScript developed by Microsoft that compiles down to plain JavaScript. Its main advantage is catching type errors at compile time instead of at runtime, which removes a whole class of production bugs. It also gives much better tooling — accurate autocomplete and safe refactoring, because the compiler can find every usage of a renamed symbol. It adds interfaces, generics, enums and access modifiers, which makes large codebases far easier to maintain. The key thing to remember is that types are erased at compile time — there's no runtime type checking, so the output is ordinary JavaScript.

Easy memory

TypeScript = JavaScript + TYPES at COMPILE time

★ (a spell-checker as you write)

✓ compile-time errors | IntelliSense | SAFE REFACTORING ★ | self-documenting

Superset: all valid JS is valid TS ★

Types are ERASED at compile time — NO runtime checking ★

Q: How does TypeScript compile / how does it run?

```
.ts file
  ↓ tsc (the TypeScript compiler)
type CHECKING happens here ★ — then the types are THROWN AWAY
  ↓
.js file ← plain JavaScript, no types at all
  ↓
browser / Node runs it
```

```
// input.ts
let name: string = "Priya";

// output.js ← the type annotation is GONE
var name = "Priya";
```

★ The consequence to state:

The browser NEVER sees TypeScript. It only runs the compiled JavaScript.

So a type error CANNOT crash your app at runtime –
but a wrong value from an API CAN, because nothing checks it at runtime.

★

That last clause matters: `response as User` is a *promise you make to the compiler*, not a validation.

Easy memory

`.ts` → `tsc` → type check ★ → types ERASED → plain `.js` → the browser runs that

The browser never sees TypeScript.

``as User`` on an API response is a PROMISE, not a validation ★

Q: What are the key `tsconfig.json` options?

```
{
  "compilerOptions": {
    "target": "ES2022",           // which JS version to output
    "module": "ES2022",         // module system
    "strict": true,              // ★ turns on ALL the strict checks
    "noImplicitAny": true,       // an untyped parameter is an error
    "strictNullChecks": true,    // ★ null/undefined must be handled
    "sourceMap": true,           // debug .ts in the browser
    "outDir": "./dist",
    "esModuleInterop": true,
    "experimentalDecorators": true, // required by Angular
    "paths": { "@app/*": ["src/app/*"] } // ★ import aliases
  }
}
```

★ The two that actually change your day:

<code>strict: true</code>	→ the master switch; turns on ~8 checks at once
<code>strictNullChecks</code>	→ the single most valuable one ★

```
// WITHOUT strictNullChecks
let name: string = null;           // allowed → a runtime NPE waiting to
                                    // happen

// WITH strictNullChecks ★
let name: string = null;           // ✗ Type 'null' is not assignable to
                                    // type 'string'
let name: string | null = null;    // ✓ you must be EXPLICIT
```

● In your RoboGebra code — and the honest answer ★

robogeбра-mobile/tsconfig.json:

```
"strict": true,
"noImplicitAny": false,      ▲
"strictNullChecks": false,   ▲
"strictPropertyInitialization": false, ▲
"strictTemplates": true      ✓
```

■ "*strict*" is on but three of the sub-flags are switched back off — *strictNullChecks*, *noImplicitAny* and *strictPropertyInitialization*. That's a legacy decision: the app started on Angular 12 and turning *strictNullChecks* on today produces thousands of errors at once. *strictTemplates* is on, which catches the template-level type errors. If I were tightening it, I'd enable *strictNullChecks* folder by folder using a path-scoped config rather than flipping it globally, because it's the flag that actually prevents runtime null errors."

★ That is exactly the right answer: **name the gap, explain the reason, and give the migration strategy.** Pretending it's fully strict invites one follow-up you can't survive.

Easy memory

```
target | module | strict ★ | strictNullChecks ★ | paths (aliases) |
sourceMap
experimentalDecorators → required by Angular
```

strictNullChecks is the one that matters ★

Your answer: "strict is on, but strictNullChecks is off for legacy reasons
–
I'd enable it folder by folder, not globally." ★

Q: **type** vs **interface** — what's the difference?

The easiest way to remember:

```
interface → for OBJECT SHAPES, and it can be REOPENED (declaration
merging) ★
type      → for ANYTHING (unions, primitives, tuples), but it CANNOT be
reopened ★
```

```
interface User { id: number; name: string; }
type UserT    = { id: number; name: string; };    // the same thing here
```

The differences that actually matter:

```
// 1. UNIONS – only `type` can do this ★
type Status = 'active' | 'inactive' | 'pending';
type ID = string | number;
// interface Status = ... ✗ impossible

// 2. DECLARATION MERGING – only `interface` ★
interface Window { myApp: string; }
interface Window { version: number; }
// → Window now has BOTH ✓ (this is how you extend third-party types)

type A = { x: number };
type A = { y: number };    // ✗ Duplicate identifier

// 3. Extending
interface Admin extends User { role: string; }    // interface
type AdminT = UserT & { role: string; }           // type –
intersection
```

	interface	type
Object shapes	✓	✓
Unions / primitives / tuples	✗	✓ ★
Declaration merging	✓ ★	✗
Extends	extends	& intersection
Error messages	usually clearer	can get noisy

The rule to state

Use `INTERFACE` for object shapes and public API contracts (mergeable, clearer errors)
 Use `TYPE` for unions, intersections, tuples, and mapped/conditional types
 ★
 Be **CONSISTENT** within a codebase – that matters more than the choice.

Easy memory

interface → object shapes + **MERGEABLE** ★ (extend third-party types)
 type → unions, tuples, primitives, mapped/conditional ★ (not mergeable)

 type Status = 'a' | 'b' ← ONLY type can do this ★
 interface Window {...} ← declaration merging: ONLY interface ★

Q: Explain any vs unknown vs never vs void.

The easiest way to remember:

any → "I give up on type checking" ▲ DANGEROUS
 unknown → "I don't know YET – check me first" ✓ the SAFE any ★
 void → "returns NOTHING"
 never → "NEVER returns at all" (throws or loops forever)

```
// any – switches OFF all checking ▲
let a: any = "hello";
a.toFixed(2); // ✓ compiles... ▲ crashes at runtime

// unknown – you MUST narrow it first ★
let u: unknown = "hello";
u.toFixed(2); // ✗ 'u' is of type 'unknown'
if (typeof u === 'number') { u.toFixed(2); } // ✓ now it's safe

// void – no return value
function log(msg: string): void { console.log(msg); }

// never – this function NEVER finishes normally ★
function fail(msg: string): never { throw new Error(msg); }
function loop(): never { while (true) {} }
```

any → the compiler LOOKS AWAY → bugs walk straight through
 unknown → the compiler BLOCKS you until you PROVE the type ✓ ★
 void → a function that returns nothing
 never → a function that never returns at all

Real-world idea:

any → a security guard who waves everyone through
 unknown → a guard who checks every ID before letting you in ✓
 void → the meeting ends with no decision
 never → the meeting never ends

★ **The interview point:** `unknown` is the type-safe replacement for `any`. If you must accept anything (a JSON parse, a third-party callback), use `unknown` and narrow it.

`never` also has a clever use — **exhaustiveness checking**:


```

type Status = 'active' | 'inactive';

function label(s: Status): string {
  switch (s) {
    case 'active':    return 'Active';
    case 'inactive': return 'Inactive';
    default:
      const _exhaustive: never = s;    //
    ★ adding a 3rd status = COMPILE ERROR here
      return _exhaustive;
  }
}

```

Easy memory

any → no checking ▲ (the guard waves you through)
 unknown → must NARROW first ✓ the safe `any` ★
 void → returns nothing
 never → never returns (throw / infinite loop) – also EXHAUSTIVENESS
 checking ★

Rule: never use `any` when `unknown` will do ★

Q: What is type inference?

Type inference = TypeScript works out the type for you, so you don't have to annotate everything. ★

```

let name = "Priya";           // inferred: string ★
name = 42;                    // ✗ error – it's locked to string

const age = 30;               // inferred: 30 (a LITERAL type, because it's
const) ★                      const)
let age2 = 30;                // inferred: number

function add(a: number, b: number) { return a + b; } // return type
inferred: number ★

```

let → widened to the general type ("Priya" → string)
 const → narrowed to the LITERAL type ("Priya" → "Priya") ★

★ The style rule: **annotate the inputs, let the compiler infer the outputs.**

```
// ✗ noisy – the compiler already knew
const names: string[] = ['a', 'b'].map((s: string): string =>
  s.toUpperCase());

// ✓ annotate only what's needed
const names = ['a', 'b'].map(s => s.toUpperCase());
```

▲ One place you **should** annotate: a function's public return type, so an accidental change to the body becomes a compile error instead of silently changing the API.

Easy memory

Inference = TS works out the type from the value ★
let → general type (string) | const → LITERAL type ("Priya") ★
Annotate INPUTS, let it infer OUTPUTS – but annotate PUBLIC return types
★

Q: What are literal types?

A literal type restricts a value to EXACT values, not just "a string". ★

```
let status: 'active' | 'inactive' | 'pending';
status = 'active';      // ✓
status = 'deleted';     // ✗ not assignable

type Dice = 1 | 2 | 3 | 4 | 5 | 6;
type Method = 'GET' | 'POST' | 'PUT' | 'DELETE';
```

string → ANY string, including typos ("active") ▲
'active'|'idle' → only these two ✓ + autocomplete in the IDE ★

★ This is often better than an enum, because it compiles to **nothing** — no runtime object is generated.

Real-world idea: a **dropdown vs a free-text box**. The dropdown makes a typo impossible.

Easy memory

Literal type = only these EXACT values ★
type Method = 'GET' | 'POST' → typo becomes a compile error + autocomplete
Zero runtime cost (unlike an enum) ★

Q: What are union and intersection types?

UNION A | B → "EITHER A or B" ★ (OR)
INTERSECTION A & B → "BOTH A and B" ★ (AND)

```
// UNION – either
type ID = string | number;
function print(id: ID) {
  if (typeof id === 'string') id.toUpperCase(); // ★ narrowing needed
  else id.toFixed(0);
}

// INTERSECTION – both, merged
interface Person { name: string; }
interface Employee { salary: number; }
type Staff = Person & Employee; // has BOTH ★

const s: Staff = { name: 'Priya', salary: 50000 }; // both required
```

Union A | B → you may only use what BOTH have, until you NARROW ★
Intersection A & B → you get EVERYTHING from both

▲ The counter-intuitive bit worth knowing:

A UNION makes the usable surface SMALLER (only the shared members)
An INTERSECTION makes it BIGGER (all members of both) ★

Easy memory

| = OR (union) → narrow it before using type-specific members ★
& = AND (intersection) → gets everything from both
Union = FEWER usable members | Intersection = MORE ★

Q: What are tuples?

Tuple = an array with a FIXED LENGTH and a FIXED TYPE PER POSITION. ★

```
let pair: [string, number] = ['Priya', 30];  
pair = [30, 'Priya']; // ✗ wrong order  
  
let point: [number, number, number?] = [1, 2]; // optional third  
let flexible: [string, ...number[]] = ['a', 1, 2, 3]; // rest element  
  
// named tuple members (TS 4.0+) – much more readable ★  
type Coord = [x: number, y: number];
```

Array number[] → any length, ALL the same type
Tuple [string, number] → EXACTLY 2, in THAT order ★

Where you meet them in real code:

```
const [value, setValue] = useState(0); // React – a tuple ★  
Object.entries(obj); // [string, T][] – a tuple  
array  
combineLatest([a$, b$]).subscribe(([a, b]) => ...); // RxJS ★
```

Easy memory

Tuple = fixed LENGTH + fixed TYPE PER POSITION ★ → [string, number]
Named members: [x: number, y: number]
You've used them: useState, Object.entries, combineLatest ★

Q: What are enums?

Enum = a set of NAMED CONSTANTS. It exists at RUNTIME (unlike a type). ★

```
enum Status { ACTIVE, INACTIVE, PENDING } // numeric: 0, 1, 2 ▲
enum Role   { ADMIN = 'ADMIN', USER = 'USER' } // string enum ✓
preferred

const enum Direction { Up, Down } // inlined, zero runtime
cost ★
```

▲ The numeric-enum trap — the same bug as JPA's `@Enumerated(ORDINAL)` :

```
enum Status { ACTIVE, INACTIVE } // stored in the DB as 0, 1

// six months later, someone inserts a value:
enum Status { ACTIVE, PENDING, INACTIVE }
//           0           1           2
```

A row saved as 1 used to mean INACTIVE.
 It now means PENDING. ▲
 Every existing record silently changed meaning. No error.

Always use string enums. → the same rule as 24 — JPA `@Enumerated(STRING)` .

● In your RoboGebra code — enums done right ★

File: `core/model/http-error.model.ts`

```

export enum HttpStatus {
    UNAUTHORIZED = 401,           // ★ EXPLICIT values, not positional
    FORBIDDEN = 403,
    TOO_MANY_REQUEST = 429
}

export enum ErrorCode {
    TOKEN_EXPIRED = 30000,
    TOKEN_INVALID = 30001,
    REFRESH_TOKEN_INVALID = 30002,
    SOLUTION_TRIAL_LIMIT_EXCEEDED = 50001,
    MULTI_USER_LOGIN_EXCEEDED = 60001,
    SESSION_FORCEFULLY_REMOVED = 60002
}

export const MultiUserErrorCodes: ErrorCode[] = [    //
    ★ a typed grouping constant
    ErrorCode.MULTI_USER_LOGIN_EXCEEDED,
    ErrorCode.SESSION_FORCEFULLY_REMOVED
];

```

■ "Our error codes are enums with explicit numeric values agreed with the backend — never positional, so inserting a new code can't shift the meaning of existing ones. They're also banded by domain: 30000s for tokens, 50000s for trial limits, 60000s for multi-user sessions, which makes an unknown code readable at a glance. And *MultiUserErrorCodes* is a typed constant array so the interceptor can test membership instead of listing conditions."

Also *AppEndpoints* and *AppResolver* are enums — centralising every URL and resolver key so a rename is a compile error rather than a broken link.

Easy memory

```

Enum = named constants that EXIST at runtime ★
▲ Numeric enums are POSITIONAL — inserting a value shifts every meaning
▲
    (the same bug as JPA @Enumerated(ORDINAL))
→ ALWAYS use explicit values or string enums ★
Alternative with zero runtime cost: a union of literals

```

Functions

Q: How do you type functions?

```
// parameter types + return type
function add(a: number, b: number): number { return a + b; }

// arrow function
const multiply = (a: number, b: number): number => a * b;

// a FUNCTION TYPE ★
type MathOp = (a: number, b: number) => number;
const divide: MathOp = (a, b) => a / b;    // params inferred from the
type ★

// a callback parameter
function process(items: number[], cb: (item: number) => void): void {
    items.forEach(cb);
}
```

★ Note the two different arrows:

Type position: (a: number) => number	← a function TYPE
Value position: (a) => a * 2	← an arrow FUNCTION

Easy memory

```
function f(a: T, b: T): R
type MathOp = (a: number, b: number) => number ★ - assign it and params
are inferred
=> in a TYPE position means "returns"; in a VALUE position it's an arrow
function
```

Q: How do optional and default parameters work?

```
function greet(name: string, greeting?: string): string {           //
  OPTIONAL
  return `${greeting ?? 'Hello'}, ${name}`;
}

function greet2(name: string, greeting: string = 'Hello'): string { //
  DEFAULT ★
  return `${greeting}, ${name}`;
}
```

optional ? → the type becomes `string | undefined` ★
default = x → the type stays `string`, and it's optional at the call site ★

▲ Optional parameters must come last:

```
function bad(a?: string, b: string) { }    // ✗ required cannot follow optional
```

★ And the ?? vs || detail that gets asked:

```
greeting ?? 'Hello'    // only when null/undefined ★
greeting || 'Hello'    // ALSO replaces "" and 0 and false ▲
```

Easy memory

```
name?: string → optional → type is `string | undefined` ★
name = 'x'    → default → type stays `string`
Optional params must come LAST ▲
?? (null/undefined only) ★ vs || (also "" 0 false) ▲
```


Q: What are rest parameters?

```
function sum(...numbers: number[]): number {  
    return numbers.reduce((t, n) => t + n, 0);  
}  
sum(1, 2, 3, 4); // 10  
  
function log(prefix: string, ...args: unknown[]): void { } // rest must  
be LAST ★
```

Rest ...x → collects MANY arguments into ONE array (in a signature)
Spread ...x→ expands ONE array into MANY values (at a call site)

```
const nums = [1, 2, 3];  
sum(...nums); // SPREAD – same syntax, opposite  
job ★
```

Easy memory

...args in a SIGNATURE = REST (collect) – must be LAST ★
...args at a CALL SITE = SPREAD (expand)
Same three dots, opposite directions ★

Q: What is function overloading?

Overloading = several SIGNATURES for ONE implementation, so the caller
gets an exact return type instead of a union. ★

```
function parse(value: string): string[]; // overload 1  
function parse(value: number): number[]; // overload 2  
function parse(value: string | number): any { // ★ the IMPLEMENTATION  
(not callable)  
    return typeof value === 'string' ? value.split('') : [value];  
}  
  
const a = parse('abc'); // typed string[] ★  
const b = parse(123); // typed number[] ★
```

WITHOUT overloads → the return type is ``string[] | number[]``
→ the caller has to narrow it every time
WITH overloads → the return type is exact ★

▲ Two rules:

1. The implementation signature is NOT callable – only the overloads are
2. The implementation must be COMPATIBLE with every overload

★ Unlike Java, TypeScript overloading is **purely a type-level trick** — the compiled JavaScript has exactly one function.

Easy memory

Several SIGNATURES + ONE implementation ★
The implementation signature is NOT callable
Purpose: an exact return type instead of a union
Unlike Java – it's compile-time only; JS has one function ★

OOP

Q: How do you define a class in TypeScript?

```
class Employee {  
  private id: number;  
  public name: string;  
  
  constructor(id: number, name: string) {  
    this.id = id;  
    this.name = name;  
  }  
  
  getDetails(): string { return `${this.id} - ${this.name}`; }  
}
```

★ The **parameter property** shorthand — this is what real Angular code uses:

```
class Employee {
  constructor(
    private id: number,           // ★ declares AND assigns the field
    public name: string,
    private readonly dept: string
  ) {}
}
```

An access modifier on a constructor parameter

- TypeScript creates the field AND assigns it ★
- which is exactly why Angular DI looks so clean:

```
constructor(private http: HttpClient) {} ← declares this.http in one line ★
```

Easy memory

```
constructor(private http: HttpClient) {}
★ a PARAMETER PROPERTY – declares the field AND assigns it
→ this is why Angular DI is a one-liner
```

Q: Explain the access modifiers.

```
public    → anywhere (the DEFAULT)
private   → this class ONLY
protected → this class + SUBCLASSES ★
readonly  → set once (in the constructor), never reassigned ★
```

```
class Account {
  public name: string;
  private balance: number;           // only inside Account
  protected type: string;           // Account and its subclasses
  readonly id: number;               // ★ assign once, then locked

  constructor(id: number) { this.id = id; }
  changeId() { this.id = 5; }         // ✗ cannot assign to a read-only property
}
```

▲ The critical caveat — **private** is **COMPILE-TIME** only:

```
const a = new Account(1);
a.balance;           // ✗ TypeScript error
(a as any).balance;  // ✓ works at runtime – the field is just there
▲
```

TypeScript `private` → erased at compile time → NOT real privacy ★
 JavaScript `#field` → TRUE runtime privacy ★

```
class Account {
  #balance = 0;           // ★ genuinely private, enforced by the JS
  runtime
}
```

★ That distinction is a very good thing to volunteer.

And `readonly` has the same shallow-lock behaviour as Java's `final`:

```
readonly items: string[] = [];
this.items.push('x');      // ✓ allowed – the CONTENTS aren't locked ★
this.items = [];           // ✗ the REFERENCE is locked
```

Easy memory

```
public (default) | private (this class) | protected (+ subclasses) |
readonly (set once)
```

▲ TS `private` is COMPILE-TIME only → `(obj as any).x` reaches it ▲
 JS `#field` is REAL runtime privacy ★
 readonly locks the REFERENCE, not the contents (like Java's final) ★

Q: What are abstract classes?

```
abstract class = a base class you CANNOT instantiate;
                 it can hold shared code AND unimplemented methods. ★
```

```

abstract class Shape {
    constructor(protected name: string) {}           // ★ constructors
allowed

    abstract area(): number;                         // children MUST
implement
    describe(): string { return `${this.name}: ${this.area()}`; } //
shared code ★
}

class Circle extends Shape {
    constructor(private r: number) { super('Circle'); }
    area(): number { return Math.PI * this.r ** 2; }
}

new Shape('x');    // ✗ Cannot create an instance of an abstract class
new Circle(5);    // ✓

```

Real-world idea: a **house foundation** — you can't live in it alone, but every house is built on one.

Easy memory

```

abstract class = cannot be instantiated; has shared CODE + shared STATE ★
abstract method → the child MUST implement it
Constructors and fields ARE allowed (unlike an interface)

```

Q: Interface vs abstract class — when to use which?

```

interface      → a CONTRACT only; ERASED at compile time ★ (zero runtime
cost)
abstract class → shared CODE + shared STATE; EXISTS at runtime ★

```

	interface	abstract class
Runtime existence	✗ erased ★	✓ exists
Implementation	✗ (contract only)	✓ concrete methods
Fields / state	✗	✓
Constructor	✗	✓

	interface	abstract class
Multiple	✓ implements A, B	✗ one extends
Access modifiers	✗ (all public)	✓

★ The TypeScript-specific consequence of erasure:

```
interface Logger { log(m: string): void; }
```

```
x instanceof Logger           // ✗ IMPOSSIBLE – interfaces don't exist at
runtime ★
providers: [Logger]           // ✗ can't be an Angular DI token ★
```

→ which is exactly why Angular needs InjectionToken for interface-typed dependencies, and why an abstract class is sometimes used AS a DI token ★

```
abstract class Logger { abstract log(m: string): void; }           // ★
usable as a token
providers: [{ provide: Logger, useClass: ConsoleLogger }];       // ✓
works
```

Deeper comparison (in Java terms): [33 — Interface vs Abstract Class](#).

Easy memory

```
interface → contract, ERASED ★ → no instanceof, NOT a DI token
abstract class → shared code + state, EXISTS at runtime → CAN be a DI
token ★
```

That erasure is WHY Angular has InjectionToken ★

Q: implements vs extends ?

```
extends → INHERIT the implementation (only ONE class) ★
implements → PROMISE to match a shape (as MANY as you like) ★
```

```

class Animal { move() {} }
interface Swimmable { swim(): void; }
interface Flyable { fly(): void; }

class Duck extends Animal implements Swimmable, Flyable {
  swim() {}
  fly() {}
}

```

extends → you GET the code
 implements → you PROMISE the shape, and must write it yourself ★

★ TypeScript nuance: you can **implements** a **class**, which takes only its *shape*, not its implementation:

```

class B implements A { } //
✓ B must have A's members, but inherits NO code ★

```

Easy memory

extends = INHERIT code (ONE) ★ | implements = PROMISE a shape (MANY) ★
 You may `implements` a class – you get its SHAPE, not its code ★

Q: How do getters and setters work?

```

class Temperature {
  private _celsius = 0;

  get celsius(): number { return this._celsius; }

  set celsius(value: number) {
    if (value < -273) throw new Error('Below absolute zero'); // ★
  }
  VALIDATION
  this._celsius = value;
}

get fahrenheit(): number { return this._celsius * 9 / 5 + 32; } // ★
COMPUTED
}

```

```
const t = new Temperature();
t.celsius = 25;           // calls the SETTER (looks like a field ★)
console.log(t.fahrenheit); // calls the GETTER – computed, never stored
```

Used like a PROPERTY, behaves like a METHOD ★
 → validation on write, computed values on read, without changing the call site

▲ In an Angular template, a getter runs on **every change detection cycle** — so keep it cheap, exactly like a method call in a template.

Easy memory

```
get x() / set x(v) → used like a FIELD, runs like a METHOD ★
Use for: validation on write, COMPUTED values on read
▲ A getter in an Angular template runs on every CD cycle – keep it cheap
★
```

Q: What are static members?

static = belongs to the CLASS, not to any instance. ★

```
class MathUtil {
  static readonly PI = 3.14159;
  static square(x: number): number { return x * x; }

  private static instance: MathUtil;
  static getInstance(): MathUtil {                      // ★ singleton
    factory
    return this.instance ??= new MathUtil();
  }
}

MathUtil.square(5);    // ✓ no object needed
```

● In your RoboGebra code — a static generic factory ★

File: `core/model/page-response.ts`


```

export class PageResponse<T> {
    content: T[];
    totalElements: number;
    totalPages: number;

    static fromJSON<U>(data: any, itemMapper: (item: any) => U):
    PageResponse<U> { // ★
        const content = (data.content || []).map(itemMapper);
        return new PageResponse<U>(content, data.totalElements ?? 0, ...);
    }

    static empty<T>(size = 20, number = 0): PageResponse<T> {
        return new PageResponse<T>([], 0, 0, size, number);
    }
}

```

■ *"**PageResponse<T>** wraps every paginated API response. The static **fromJSON** is a generic factory that takes a mapper function, so one class handles pages of books, exercises or study materials with full type safety. **empty()** gives components a safe initial value so they never have to null-check the list before the first load."*

★ That's **static + generics + a factory method + defensive defaults** in one small file.

Easy memory

static = belongs to the CLASS → MathUtil.square(5) ★
 Common uses: constants, utilities, FACTORY methods, singletons
 Your example: PageResponse<T>.fromJSON<U>(data, mapper) ★

Advanced Types

Q: What are generics?

The easiest way to remember:

Generics = a TYPE PLACEHOLDER, so ONE piece of code works with MANY types WITHOUT losing type safety. ★

The problem they solve

```
// ✗ any – works with everything, and loses everything
function first(arr: any[]): any { return arr[0]; }
const n = first([1, 2, 3]);
n.toUpperCase();           // ✓ compiles... ▲ crashes at runtime

// ✓ generic – works with everything, KEEPS the type ★
function first<T>(arr: T[]): T { return arr[0]; }
const n = first([1, 2, 3]);    // typed number ★
n.toUpperCase();             // ✗ caught at compile time ✓
```

any → the type is THROWN AWAY ▲
<T> → the type is CARRIED THROUGH ★

Real-world idea: a **shipping box**. The same box carries books or clothes — and the **label** tells you which, so you never open it expecting the wrong thing.

```
// generic interface
interface ApiResponse<T> { data: T; status: number; message: string; }
const r: ApiResponse<User[]> = await http.get('/users');
r.data[0].name;           // ★ fully typed all the way down

// generic class
class Box<T> {
  constructor(private value: T) {}
  get(): T { return this.value; }
}

// multiple type parameters
function pair<K, V>(key: K, value: V): [K, V] { return [key, value]; }
```

● In your RoboGebra code

File: `core/model/page-response.ts` — `PageResponse<T>` wraps every paginated response.

File: `core/model/public/public-list-item.model.ts` — `PublicListItem<T>` with `fromJSON<T>(row, mapFn)`.

■ "Every paginated endpoint returns `PageResponse<T>` — content, totalElements, totalPages, size and number. One generic class covers pages of books, exercises and study materials, and the static `fromJSON<U>` takes a mapper function so the item type is preserved end to end. Without generics we'd either duplicate the class per type or fall back to `any` and lose the safety at exactly the point where the data enters the app."

Easy memory

Generics = a type PLACEHOLDER `<T>` → one implementation, MANY types, type-safe ★
any THROWS the type away ▲ | `<T>` CARRIES it through ★
Shipping box — same box, labelled contents

Your example: `PageResponse<T>` + `static fromJSON<U>(data, mapper)` ★

Q: What are generic constraints?

A constraint says: "T can be anything — AS LONG AS it has this." ★
`<T extends SomeShape>`

```
// ✗ unconstrained — T might not have .length
function longest<T>(a: T, b: T): T {
  return a.length > b.length ? a : b;    // ✗ Property 'length' does
  not exist on type 'T'
}

// ✓ constrained ★
function longest<T extends { length: number }>(a: T, b: T): T {
  return a.length > b.length ? a : b;
}

longest('abc', 'de');    // ✓ strings have length
longest([1,2], [3]);    // ✓ arrays have length
longest(1, 2);          // ✗ numbers don't
```

The `keyof` constraint — the one interviewers love:

```
function getProp<T, K extends keyof T>(obj: T, key: K): T[K] { // ★
  return obj[key];
}

const user = { name: 'Priya', age: 30 };
getProp(user, 'name'); // typed string ★
getProp(user, 'email'); // ✗ 'email' is not a key of the object ★
```

K extends keyof T → the key must actually EXIST on the object ★
 T[K] → the return type is the type of THAT property ★

★ That is genuinely impressive to write on a whiteboard: a typo in a property name becomes a **compile error**.

Easy memory

```
<T extends Shape> = "T can be anything, as long as it HAS this" ★
<T, K extends keyof T>(obj: T, key: K): T[K] ← type-safe property access
★
→ a typo in the key name becomes a COMPILE ERROR
```

Q: Explain the common utility types.

The easiest way to remember them **by what they do to an existing type**:

Partial<T>	→ make every property OPTIONAL	★ (PATCH updates)
Required<T>	→ make every property REQUIRED	
Readonly<T>	→ make every property READ-ONLY	
Pick<T, K>	→ KEEP only these keys	★
Omit<T, K>	→ REMOVE these keys	★ (drop the password!)
Record<K, V>	→ build an object type from keys→values	★

```

interface User { id: number; name: string; email: string; password:
string; }

type UserUpdate = Partial<User>; // every field optional
★
type UserPreview = Pick<User, 'id' | 'name'>; // just those two ★
type SafeUser = Omit<User, 'password'>; // everything except
password ★
type Frozen = Readonly<User>;
type ById = Record<number, User>; // { [key: number]:
User } ★
type Flags = Record<'dev' | 'prod', boolean>;

```

Where each one actually earns its place:

```

// PATCH endpoint – only the changed fields
updateUser(id: number, changes: Partial<User>) { } ★

// API response – never leak the password hash
toDto(u: User): Omit<User, 'password'> { } ★

// a lookup map built from an array
const byId: Record<number, User> = {}; ★

```

Others worth naming:

```

ReturnType<typeof fn> → the return type of a function ★
Parameters<typeof fn> → its parameter types as a tuple
NonNullable<T> → strips null | undefined ★
Awaited<T> → unwraps a Promise
Exclude<T,U> / Extract<T,U> → filter a union

```

★ **Omit<User, 'password'>** is the strongest one to mention — it connects directly to the DTO-vs-entity answer in [06 — Spring Boot](#) and [24 — JPA](#).

Easy memory

```

Partial ★ (all optional – PATCH) | Required | Readonly
Pick ★ (keep these) | Omit ★ (drop these – the password!) | Record<K,V>
★
ReturnType | Parameters | NonNullable | Awaited

Omit<User, 'password'> = the DTO answer, in the type system ★

```

Q: What are mapped types?

A mapped type LOOPS OVER the keys of a type and transforms each one. ★

```
type Optional<T> = { [K in keyof T]?: T[K] };           // this IS
Partial<T> ★
type Frozen<T>   = { readonly [K in keyof T]: T[K] }; // this IS
Readonly<T>
type Nullable<T> = { [K in keyof T]: T[K] | null };

// with a KEY REMAP (TS 4.1+) – generate getters ★
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K]
};

type UserGetters = Getters<{ name: string; age: number }>;
// → { getName: () => string; getAge: () => number } ★
```

```
[K in keyof T] → "for every key K in T..." ★
as `get${...}` → rename the key while mapping
```

★ The line to say: "*Partial*, *Required* and *Readonly* aren't magic — they're one-line mapped types in the standard library."

Easy memory

```
Mapped type = a LOOP over keys ★ → { [K in keyof T]: ... }
Partial/Required/Readonly are all one-line mapped types ★
Key remapping: [K in keyof T as `get${Capitalize<K>}`]
```

Q: What are conditional types?

A conditional type is a TYPE-LEVEL if/else. ★
T extends U ? X : Y

```

type IsString<T> = T extends string ? true : false;
type A = IsString<'hello'>;    // true
type B = IsString<42>;        // false

// with `infer` – EXTRACT a type from inside another ★
type Unwrap<T> = T extends Promise<infer U> ? U : T;
type C = Unwrap<Promise<string>>;    // string ★

type ElementOf<T> = T extends (infer E)[] ? E : never;
type D = ElementOf<number[]>;        // number ★

```

extends here means "is assignable to", NOT class inheritance ★
 infer U = "capture whatever type sits in that slot" ★

★ You will rarely write these yourself, but the library types you use every day are built from them — `ReturnType`, `Awaited`, `NonNullable`. Saying *that* is the right level of answer.

Easy memory

T extends U ? X : Y = a type-level IF ★ (extends = "assignable to")
 infer U = capture the inner type ★ → `Unwrap<Promise<string>>` = `string`
 You rarely write them; `ReturnType` / `Awaited` / `NonNullable` ARE them ★

Q: What are type guards?

A type guard NARROWS a broad type down to a specific one,
 so the compiler lets you use type-specific members. ★

```

// 1. typeof – for primitives
function print(v: string | number) {
  if (typeof v === 'string') v.toUpperCase();    // ★ narrowed to string
  else v.toFixed(2);                            // ★ narrowed to number
}

// 2. instanceof – for classes
if (err instanceof HttpResponse) { err.status; }

// 3. `in` – for object shapes ★
type Dog = { bark(): void }; type Cat = { meow(): void };
function speak(a: Dog | Cat) {
  if ('bark' in a) a.bark(); else a.meow();      // ★
}

// 4. a CUSTOM type guard – `x is T` ★ ★
function isUser(obj: unknown): obj is User {
  return typeof obj === 'object' && obj !== null && 'id' in obj && 'name'
  in obj;
}

if (isUser(data)) { data.name; }                // ★ narrowed to User

// 5. a DISCRIMINATED UNION – the cleanest of all ★ ★
type Result =
  | { status: 'success'; data: User }
  | { status: 'error'; message: string };

function handle(r: Result) {
  if (r.status === 'success') r.data;           // ★ TS knows which
  branch
  else r.message;
}

```

The `obj is User` return type is what makes a custom guard work – without it the function just returns a boolean and narrows NOTHING ★

Real-world idea: **airport security narrowing "a passenger" down to "a business-class passenger with hand luggage only"** — once verified, different rules apply.

★ **Discriminated unions are the answer to give for API responses** — a shared literal field (`status` , `kind` , `type`) lets the compiler pick the branch for you, with no casts anywhere.

Easy memory

```
typeof (primitives) | instanceof (classes) | in (shapes)
CUSTOM: function isUser(x): x is User ★ - the `is` is what does the
narrowing
DISCRIMINATED UNION ★ ★ - a shared literal field picks the branch, no
casts
```

Q: Explain `typeof`, `keyof` and `in`.

```
typeof → take the TYPE of an existing VALUE ★ (value → type)
keyof → take the KEYS of a type as a union ★ (type → keys)
in → loop over keys (mapped types), or check a property at runtime
```

```
const config = { url: 'https://api.x', retries: 3 };
type Config = typeof config;           // ★ { url: string; retries:
number }

type Keys = keyof Config;              // ★ 'url' | 'retries'

type Copy = { [K in keyof Config]: Config[K] }; // `in` - a mapped type

if ('url' in config) { }               // `in` - a runtime check
```

★ The combination is the classic idiom:

```
function get<T, K extends keyof T>(obj: T, key: K): T[K] { return
obj[key]; }
```

▲ Careful: `typeof` in a type position is not the same as JavaScript's runtime `typeof`.

```
typeof config           // TYPE position → the object's type ★
typeof x === 'string'   // VALUE position → the JS operator
```

Easy memory

```
typeof → VALUE → TYPE ★ (type Config = typeof config)
keyof → TYPE → its KEYS ★ (type Keys = keyof Config)
in → loop over keys (mapped type) OR a runtime property check
```

Same word `typeof`, two different meanings by position ★

Q: What is type assertion (`as`)?

```
`as` = "trust me, compiler, I know what this is."
```

It CHANGES NOTHING at runtime. It only silences the checker. ★

```
const el = document.getElementById('x') as HTMLInputElement;
el.value;                                     // ✓ compiles

const data = JSON.parse(json) as User;        // ▲ a PROMISE, not a
validation ★
```

``as`` performs NO conversion and NO check.
If the value isn't really a User, it crashes LATER, somewhere else ▲

★ The right framing: "`as` moves the risk from the compiler to me."

```
// ✗ dangerous – a lie the compiler believes
const user = apiResponse as User;

// ✓ verify first ★
if (isUser(apiResponse)) { /* now genuinely a User */ }
```

Also worth knowing:

```
const config = { url: 'x' } as const; //
★ `as const` – deep readonly + LITERAL types
// url is typed 'x', not string
```

Easy memory

```
`as` = "trust me" → NO runtime conversion, NO check ★  
JSON.parse(x) as User is a PROMISE, not a validation ▲  
Prefer a type GUARD over an assertion ★  
`as const` → literal types + deep readonly
```

Q: What is the non-null assertion operator (!)?

```
value! = "I promise this is not null or undefined."  
Again: NO runtime check. ★
```

```
const el = document.getElementById('x')!; // "definitely exists"  
el.focus(); //  
▲ if it doesn't, crash at runtime  
  
@ViewChild('input') input!: ElementRef; //  
★ "Angular WILL assign this"
```

```
! → silences strictNullChecks  
?. → SAFELY returns undefined instead ★
```

```
user!.address.city // ▲ crashes if user is null  
user?.address?.city // ✓ undefined, no crash ★  
user?.address?.city ?? 'Unknown' // ✓ with a fallback ★
```

★ The one legitimate use is **definite assignment** — `@ViewChild`, `@Input`, and fields Angular fills in later. Everywhere else, prefer `?.` and `??`.

Easy memory

```
! = "trust me, not null" → NO runtime check ★  
?. = optional chaining → returns undefined safely ★  
?? = nullish coalescing → a fallback for null/undefined only
```

```
Legit use of !: @ViewChild / @Input definite assignment ★  
Everywhere else: ?. and ??
```

Q: What are index signatures?

An index signature says: "this object can have ANY key of this type, and every value is of that type." ★

```
interface StringMap { [key: string]: string; }

const headers: StringMap = { 'Content-Type': 'application/json' };
headers['Authorization'] = 'Bearer x';           // ✓ any string key

// mixing a known key with an index signature
interface Config {
  name: string;
  [key: string]: string | number;    //
  ▲ every declared property must FIT this type ★
}
```

▲ The trade-off:

Index signature → flexible, but you LOSE key checking:

headers['Contnet-Type'] ← a typo compiles fine ▲

Record<'a' | 'b', string> → only the known keys ✓ + autocomplete ★

★ The rule: use an index signature only when the keys are **genuinely unknown at compile time** (HTTP headers, a translation dictionary). Otherwise prefer `Record` with a literal union.

Easy memory

```
[key: string]: T = ANY key of that type ★
  ▲ You lose key checking – typos compile ▲
Known keys? → Record<'a' | 'b', T> instead ★
Every declared property must fit the index signature's value type ▲
```

Decorators, Declaration Files, Modules

Q: What are decorators? How does Angular use them?

A decorator is a FUNCTION that attaches METADATA to a class, method, property or parameter. ★

```
@Component({ selector: 'app-x', template: '...' }) // class decorator
export class XComponent {
  @Input() data!: string; // property decorator
  @Output() changed = new EventEmitter<string>();
  @ViewChild('ref') ref!: ElementRef;
  @HostListener('click') onClick() {} // method decorator

  constructor(@Inject(TOKEN) private cfg: Config) {} // parameter decorator
}
```

The decorator STORES metadata that Angular reads at runtime
→ "this class is a component, its selector is app-x,
its template is ..., and `data` is an input" ★

Enabling them:

```
"experimentalDecorators": true // ★ required by Angular
"emitDecoratorMetadata": true // emits type info for DI
```

▲ Angular's decorators are the **legacy/experimental** proposal, not the TC39 Stage-3 decorators standardised in TypeScript 5. Knowing that distinction is a good detail.

Writing your own:

```
function LogTime(target: any, key: string, descriptor: PropertyDescriptor)
{
  const original = descriptor.value;
  descriptor.value = function (...args: any[]) {
    const t = performance.now();
    const r = original.apply(this, args);
    console.log(`${key} took ${performance.now() - t}ms`);
    return r;
  };
  return descriptor;
}
```

★ That is exactly the **Decorator pattern** from [17 — Design Patterns](#) — wrap and delegate.

Easy memory

```
Decorator = a function attaching METADATA ★
@Component @Directive @Pipe @Injectable @NgModule | @Input @Output
@ViewChild @HostListener
Needs experimentalDecorators: true ★
Angular uses the LEGACY proposal, not TS 5 standard decorators ▲
Writing one = the Decorator PATTERN (wrap + delegate) ★
```

Q: What are declaration files (**.d.ts**)?

A **.d.ts** file contains TYPES ONLY — no implementation.
It teaches TypeScript about JavaScript code. ★

```
// my-lib.d.ts
declare module 'my-lib' {
  export function greet(name: string): string;
}

// globals
declare const APP_VERSION: string;
declare global {
  interface Window { dataLayer: any[]; } // ★ typing a global you
  didn't create
}
```

A JS library with no types → every import is `any` ▲



npm install --save-dev @types/lodash ← community types from DefinitelyTyped ★



full IntelliSense and checking ✓

★ Three facts worth stating:

1. @types/* packages come from DefinitelyTyped ★
2. Modern libraries SHIP their own .d.ts ("types" field in package.json)
3. Angular generates .d.ts when you build a library, so consumers get types ★

Easy memory

.d.ts = types WITHOUT implementation ★ → teaches TS about JS
npm i -D @types/lodash (DefinitelyTyped) ★
declare global { interface Window { ... } } → type a global ★

Q: Namespaces vs modules?

NAMESPACE → the OLD way (an internal object wrapper) → avoid in new code

▲
MODULE → the STANDARD way (one file = one module, import/export) ★

```
// ✗ namespace – legacy
namespace Utils {
  export function format(d: Date): string { return d.toISOString(); }
}
Utils.format(new Date());

// ✓ module – standard ES modules ★
// utils.ts
export function format(d: Date): string { return d.toISOString(); }
// consumer.ts
import { format } from './utils';
```

Namespace → one global object, NOT tree-shakable ▲

Module → per-file scope, TREE-SHAKABLE ★, works with bundlers

★ **Tree-shaking is the deciding argument:** a bundler can remove an unused module export, but it cannot safely remove a property from a namespace object.

Easy memory

Namespace = legacy internal module ▲ NOT tree-shakable
Module ★ = one file, import/export, tree-shakable, standard ES

Angular uses MODULES everywhere. Namespaces only appear in old .d.ts files.

Practical / Angular

Q: How does TypeScript specifically help in Angular development?

Answer it **feature by feature** — a generic "types are good" answer scores nothing:

1. DECORATORS – @Component/@Injectable are TypeScript decorators ★
2. DI by TYPE – constructor(private http: HttpClient) works because the TYPE
is emitted as metadata ★ ★
3. Typed HTTP – http.get<User[]>() types the whole chain
4. Typed reactive forms (v14+) – form.value is typed, not `any` ★
5. strictTemplates – type errors in the TEMPLATE become build errors ★
6. Safe refactoring – rename a model field and every template + class is found ★
7. Interfaces as API contracts between frontend and backend

★ Point 2 is the one that impresses:

```
constructor(private http: HttpClient) {}
```

Angular needs to know WHICH class to inject.

`emitDecoratorMetadata` writes the parameter's TYPE into the compiled output.

↓

Without TypeScript, Angular DI would need explicit @Inject() everywhere ★

That is also exactly why an **interface can't be a DI token** — it's erased, so there's no metadata to emit.

● In your RoboGebra code

```
strictTemplates: true ✓ → template type errors caught at build
strictNullChecks: false ▲ → the legacy gap (see the tsconfig question
above)
89 services, 163 components, all with typed constructor DI
```

Easy memory

Decorators | DI by TYPE ★ ★ (emitDecoratorMetadata) | typed HttpClient
typed forms (v14+) | strictTemplates ★ | safe refactoring | API contracts

"Interfaces are erased → which is exactly why they can't be DI tokens." ★

Q: What are common TypeScript compiler errors and how do you fix them?

"Object is possibly 'null'"	→ ?. or a guard, or ! if genuinely safe
"Property 'x' does not exist on type 'y'"	→ wrong type, or narrow it first
"Type 'string' is not assignable to 'number'"	→ a real bug – fix the type, don't cast
"Cannot find module 'x'"	→ npm i -D @types/x, or a .d.ts
"Parameter implicitly has an 'any' type" (noImplicitAny)	→ annotate it
"Property has no initializer" (assignment) or a default	→ use ! (definite)
"Argument of type 'X undefined'..."	→ narrow it, or ?? a fallback

★ The habit to describe:

An error is the compiler DOING ITS JOB. ★
The fix is almost never `as any` – that just moves the crash to runtime.

```
// ✗ the reflex that hides a real bug
const x = something as any;

// ✓ narrow, guard, or fix the model ★
if (something) { ... }
```

Easy memory

```
"possibly null" → ?. / a guard | "does not exist" → narrow it
"not assignable" → a REAL bug, fix the type ★
"cannot find module" → @types/x
"no initializer" → ! or a default value

▲ `as any` doesn't fix an error – it MOVES it to runtime ★
```

Q: How do you type an HTTP response and handle optional API fields?

```
export interface User {
  id: number;
  name: string;
  email?: string;           // ★ optional – the API may omit it
  address?: { city?: string };
}

// typed request
getUser(id: number): Observable<User> {
  return this.http.get<User>(`/api/users/${id}`); // ★ typed
}

// safe consumption ★
const city = user.address?.city ?? 'Unknown';
```

▲ The honest caveat every interviewer likes to hear:

```
http.get<User>() does NOT validate anything at runtime. ★

It's a PROMISE to the compiler. If the backend changes a field,
TypeScript will happily hand you an object that doesn't match. ▲
```

Options, in order of rigour:

1. Optional fields (?) + ?. + ?? ← the pragmatic default ★
2. A mapper/factory that builds the model ← your `PageResponse.fromJSON` pattern ★
3. A runtime validator (zod, io-ts) ← real validation at the boundary

● In your RoboGebra code — option 2, done properly ★

```
static fromJSON<U>(data: any, itemMapper: (item: any) => U):
PageResponse<U> {
    const content = (data.content || []).map(itemMapper);    // ★
    defensive default
    return new PageResponse<U>(
        content,
        data.totalElements ?? 0,                            // ★ ??
    fallbacks
        data.totalPages ?? 0,
        data.size ?? 0,
        data.number ?? 0
    );
}
```

■ "We don't trust the raw JSON. Every paginated response goes through *PageResponse.fromJSON*, which takes a mapper for the item type and applies ?? 0 defaults for the paging fields. So a missing or renamed field degrades to a safe default at one boundary, instead of throwing somewhere deep in a template. That's the practical middle ground between blindly casting with *as* and pulling in a full runtime validator."

★ **That is a very strong answer** — it shows you know `as` is a lie *and* you did something pragmatic about it.

Easy memory

`http.get<User>(url) → typed`, but NO runtime validation ★
Optional fields: `email?: string` → use `?.` and `??` ★

▲ ``as User`` is a PROMISE, not a check ▲
Your pattern: a `fromJSON MAPPER` with `??` defaults at the boundary ★
Stricter: `zod` / `io-ts` for real runtime validation

Quick Revision Sheet

BASICS

TS = JS + types at COMPILE time; types are ERASED ★ (no runtime checking)

strict ★ | strictNullChecks ★ (your app: OFF – be honest, give the migration plan)

interface → object shapes + MERGEABLE ★ | type → unions/tuples ★

any (no checking ▲) | unknown (must narrow ✓ ★) | void | never (+ exhaustiveness ★)

let → general type | const → LITERAL type ★

Union | = OR (narrow it) | Intersection & = AND

Tuple = fixed length + type per position

▲ NUMERIC enums are positional → inserting a value shifts meanings

▲ → use strings ★

FUNCTIONS

name?: T → `T | undefined` | name = x → default | optional comes LAST

?? (null/undefined) ★ vs || (also "" 0 false) ▲

...args = REST in a signature, SPREAD at a call site

Overloads = many signatures, ONE implementation (compile-time only) ★

OOP

constructor(private http: HttpClient) ★ = parameter property → Angular DI in one line

▲ TS `private` is compile-time only → (x as any).field reaches it ▲ | JS #field is real ★

readonly locks the REFERENCE, not the contents

interface ERASED ★ → no instanceof, NOT a DI token → hence

InjectionToken ★

abstract class EXISTS at runtime → CAN be a DI token ★

extends = inherit code (ONE) | implements = promise a shape (MANY)

ADVANCED

Generics <T> = one implementation, many types, type-safe ★

(PageResponse<T>)

<T, K extends keyof T>(obj: T, key: K): T[K] ★ → typo = compile error

Partial ★ | Pick | Omit ★ (drop the password) | Record<K,V> ★ |

Readonly

Mapped type = a loop over keys ★ | Conditional = T extends U ? X : Y + infer ★

Type guards: typeof | instanceof | in | `x is T` ★ | DISCRIMINATED UNION ★ ★

typeof → value→type ★ | keyof → type→keys ★

`as` and `!` do NO runtime check ★ – prefer a guard, ?. and ??

ANGULAR

DI works because emitDecoratorMetadata writes the parameter TYPE ★ ★

strictTemplates catches template type errors at BUILD time ★

```
http.get<User>() types the chain but validates NOTHING at runtime ★  
→ your fromJSON mapper with ?? defaults is the pragmatic fix ★
```

Related files: [04 — Angular](#) · [01 — JavaScript](#) · [33 — Interface vs Abstract Class](#)
· [17 — SOLID & Design Patterns](#) · [39 — RoboGebra Code Examples](#)

Angular — Interview Q&A (Easy Version)

Your core skill. Every question follows the same shape —

Easiest way to remember → **simple explanation** → **real-world example** → **diagram** → **interview trap** → **Interview-ready answer** → **Easy memory box.**

- Wherever a topic appears in your own codebase, there's an **"In your RoboGebra code"** block with the real file. Full index: [39 — RoboGebra Code Examples](#).

Your real Angular stack:

```
robogebra-mobile      → Angular 18.2 + Ionic 8.7 + Capacitor 6.2 + RxJS 7.8
+ TS 5.4
robogebra-web         → Angular 16.2 + Material 16 + PrimeNG 16 + TS 5.1
robogebra-backoffice → Angular 16.2 + PrimeNG 16.9 + TS 4.9
```

Contents

1. [Core Concepts](#)
2. [Components & Templates](#)
3. [Component Communication](#)
4. [Services & Dependency Injection](#)
5. [Routing](#)
6. [Forms](#)
7. [RxJS & Observables](#)
8. [State Management \(NgRx\)](#)
9. [HTTP](#)
10. [Pipes](#)
11. [Performance & Change Detection](#)
12. [Testing](#)
13. [Migration & Versioning](#)

Core Concepts

Q: What is Angular and how is it different from AngularJS?

The easiest way to remember:

AngularJS (1.x) → JavaScript, \$scope, two-way digest cycle → SLOW, rewritten

Angular (2+) → TypeScript, components, one-way data flow → a COMPLETE rewrite ★

They are **not** versions of the same thing. Angular 2 was written from scratch in 2016.

	AngularJS (1.x)	Angular (2+)
Language	JavaScript	TypeScript
Building block	controllers + \$scope	components
Data flow	two-way everywhere (digest cycle)	one-way down, events up ★
DI	string-based, minification-fragile	hierarchical, type-based
Mobile	not designed for it	first-class (Ionic/Capacitor)
Performance	digest loop re-checks everything	Ivy + change detection strategies

Real-world idea: **a house rebuilt on the same plot**. Same address, entirely new structure — you can't "upgrade" one into the other room by room.

Interview-ready answer

Angular is a TypeScript-based front-end framework for building single-page applications, and it is a complete rewrite of AngularJS rather than a new version of it. AngularJS used controllers and \$scope with a two-way digest cycle that re-

checked everything and became a performance bottleneck. Angular is component-based with one-way data flow down and events up, a hierarchical type-based dependency injection system, and the Ivy compiler. It also ships with routing, HTTP, forms and testing built in, whereas React needs those assembled from separate libraries.

Easy memory

AngularJS = JS + \$scope + digest cycle (slow)
Angular = TypeScript + COMPONENTS + one-way flow ★ – a REWRITE, not an upgrade

Q: What is a Single Page Application (SPA)?

The easiest way to remember:

SPA = the browser loads ONE HTML page, and JavaScript swaps the content.
The page NEVER fully reloads. ★

TRADITIONAL (MPA)	SPA
click a link	click a link
↓	↓
FULL page request	only a JSON API call
↓	↓
white flash, full reload	the router swaps a component ▲
↓	↓
everything re-downloads	only DATA moves

Real-world idea: a **flip-book vs a whiteboard**.

MPA → tear out the page and put in a whole new one every time
SPA → wipe one section of the whiteboard and rewrite it ✓

Trade-off worth naming:

- ✓ Fast navigation, app-like feel, less bandwidth after the first load
- ✗ A big FIRST load, and SEO needs SSR (Angular Universal)

Easy memory

SPA = ONE page, JS swaps the content, no full reload ★
✓ fast after load | ✗ heavy first load + SEO needs SSR

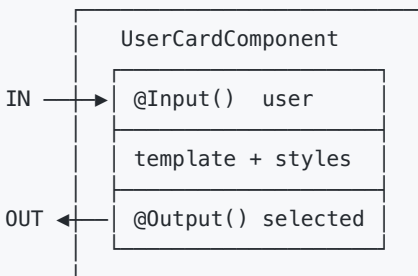
Q: What is a Component in Angular?

The easiest way to remember:

Component = TEMPLATE (HTML) + CLASS (TypeScript) + STYLES (CSS)
→ one self-contained piece of the screen.

```
@Component({
  selector: 'app-user-card',           // the HTML tag you write
  templateUrl: './user-card.component.html',
  styleUrls: ['./user-card.component.scss']
})
export class UserCardComponent {
  @Input() user!: User;                // data IN ↓
  @Output() selected = new EventEmitter<User>(); // event OUT ↑
}
```

```
<app-user-card [user]="currentUser" (selected)="onSelect($event)"></app-user-card>
```



Real-world idea: a **LEGO brick**. It has a fixed shape, plugs into others in defined ways, and can be reused anywhere.

● In your RoboGebra code

```
robogebra-mobile → 163 components
@Input() → 252 usages
@Output() → 69 usages
```

Easy memory

```
Component = template + class + styles → one UI piece
@Input() = data DOWN ↓ [prop]="value"
@Output() = events UP ↑ (event)="handler($event)"
```

Q: What is an NgModule?

The easiest way to remember:

```
NgModule = a BOX that groups related components, directives, pipes and
services,
           and tells Angular how they fit together.
```

```
@NgModule({
  declarations: [AppComponent, UserCardComponent], // what BELONGS to
  this module
  imports:      [BrowserModule, HttpClientModule], // other modules I
NEED
  providers:    [UserService],                      // services available
here
  exports:      [UserCardComponent],                // what OTHERS may
use
  bootstrap:    [AppComponent]                      // the root component
  (root module only)
})
export class AppModule { }
```

★ The four words, in one line each:

```
declarations → MINE      (components/directives/pipes I own)
imports      → BORROWED  (other modules whose exports I use)
providers     → SERVICES  (registered with the injector)
exports       → SHARED   (what I let other modules use)
```

Real-world idea: a **department in a company**.

```
declarations → the staff who work in this department
imports      → other departments you collaborate with
exports      → the staff you lend out to other departments
providers    → the shared equipment
```

▲ The #1 NgModule error:

```
"'app-user-card' is not a known element"
```

↓

The component is declared, but NOT EXPORTED from its module,
or its module isn't IMPORTED where you're using it. ★

Easy memory

```
declarations = MINE | imports = BORROWED | exports = SHARED | providers = SERVICES
```

"not a known element" → you forgot to EXPORT it, or to IMPORT its module
★

Q: What are standalone components and why do they matter?

The easiest way to remember:

```
Standalone component = a component that declares its OWN imports
                        and needs NO NgModule. ★
```

```
@Component({
  standalone: true,                                // ★
  selector: 'app-user-card',
  imports: [CommonModule, RouterLink],             // its OWN
  dependencies
    template: `<a [routerLink]="['/user', user.id]">{{ user.name }}</a>`
})
export class UserCardComponent { }
```

BEFORE (NgModule)

```
component.ts
imports ✓
+ module.ts      ← boilerplate
+ remember to declare it
+ remember to export it
+ remember to import the module
```

AFTER (standalone)

```
component.ts ← declares its own
```

Bootstrapping without any module at all:

```
bootstrapApplication(AppComponent, {
  providers: [provideRouter(routes), provideHttpClient()]
});
```

Real-world idea: **a freelancer vs an employee**. A freelancer brings their own laptop and tools and can be dropped into any project; an employee needs the department to provide everything.

● In your RoboGebra code — be honest here ★

```
robogebra-mobile is Angular 18.2, but still largely NgModule-based:
standalone: true      → 2 components (migration started)
inject()              → 30 usages    (adopted ✓ )
takeUntilDestroyed() → 5 usages    (adopted ✓ )
signal()              → 0           ▲ not adopted
```

■ "We're on Angular 18 but the app is still mostly NgModule-based — it grew from Angular 12, and a big-bang standalone migration was never worth the regression risk on a shipping product. We've adopted the pieces that are cheap and safe: *inject()* in new code, *takeUntilDestroyed()* for teardown, and standalone for new components. New work is standalone-first; the legacy modules get converted when we're already touching them."

★ **That is the right answer.** Claiming full adoption invites one follow-up you can't back up. Describing a *staged migration policy* is what a lead is expected to have.

Easy memory

Standalone = no NgModule; the component declares its OWN imports ★
bootstrapApplication() + provideRouter() + provideHttpClient()
Default for new components since Angular 17; the DEFAULT in v19

Your answer: "New code is standalone-first; legacy modules convert when touched." ★

Q: What are decorators in Angular?

The easiest way to remember:

A decorator = a function starting with @ that attaches METADATA to a class,
telling Angular what that class IS and how to use it.

@Component	→ "this is a UI piece"	(has a template)
@Directive	→ "this changes an element"	(no template)
@Pipe	→ "this transforms a value"	{{ x myPipe }}
@Injectable	→ "this can be injected"	(a service)
@NgModule	→ "this groups things"	
@Input / @Output	→ property metadata	

```
@Component({ selector: 'app-x', template: '...' })  
class X { }  
// ↑ without the decorator this is just a plain TypeScript class –  
//   Angular would have no idea what to do with it
```

Real-world idea: a **label on a box**. The box is the same; the label ("FRAGILE", "PERISHABLE") tells the handler how to treat it.

Easy memory

Decorator = @metadata attached to a class
@Component (has a template) | @Directive (no template) | @Pipe |
@Injectable | @NgModule
Without it, Angular sees only a plain class.

Q: What is the Angular CLI and which commands do you use most?

```
ng new my-app --standalone --routing --style=scss
ng serve                                     # dev server with live reload
ng generate component user                  # or: ng g c user
ng g s core/services/auth                  # service
ng g guard core/guard/auth                 # guard
ng g pipe shared/pipes/safe                # pipe
ng build --configuration production
ng test                                     # Karma + Jasmine
ng update @angular/core @angular/cli      # ★ runs migration SCHEMATICS
ng add @angular/material                   # installs AND configures
```

★ The two worth explaining:

```
ng update → not just a version bump; it runs SCHEMATICS that
           AUTOMATICALLY REWRITE your code for breaking changes ★
ng add    → not just npm install; it also runs the library's setup
schematic
           (imports the module, adds styles, updates angular.json)
```

Easy memory

```
ng new | ng serve | ng g c | ng build --configuration production | ng test
ng update ★ = version bump + automatic code MIGRATIONS (schematics)
ng add     ★ = install + configure (not just npm install)
```

Q: Describe a typical Angular project structure.

The structure your own app uses — quote it, it's a good one:

```

src/app/
├── core/                ← singletons, loaded ONCE
│   ├── guard/          ← route guards
│   ├── interceptors/   ← HTTP interceptors
│   ├── resolvers/      ← route data pre-fetch
│   ├── services/       ← app-wide services
│   ├── model/          ← interfaces / DTOs
│   └── config/
├── features/           ← one folder PER FEATURE (lazy-loadable)
│   └── pages/
│       ├── auth/  books/  quiz/  study-list/  settings/ ...
└── shared/            ← reusable dumb components, pipes, directives
    ├── components/  header/  footer/  modal/  side-menu/ ...

```

```

core/      → things you want EXACTLY ONE of      (services, interceptors,
guards)
features/ → things a USER navigates to          (lazy loaded)
shared/    → things you REUSE everywhere          (dumb components, pipes)

```

★ The rule that keeps it clean: **core** is imported once by the root module; **shared** is imported by many feature modules. If a service ends up in **shared** and gets provided per-module, you accidentally create multiple instances.

● In your RoboGebra code

This is exactly `robogebra-mobile/src/app/` — with 89 services, 163 components, 27 resolvers, 4 guards and 3 interceptors.

Easy memory

```

core/ = ONCE (services, guards, interceptors, resolvers)
features/ = per-feature, LAZY loaded
shared/ = reused dumb components + pipes

```

▲ Never provide a singleton service from `shared` — you'll get multiple instances.

Components & Templates

Q: Explain all the Angular lifecycle hooks in order.

★ **This has now come up in 5 separate rounds (26 — Companies).** Know the order cold.

The easiest way to remember — **the order is the life of the component:**

constructor

ngOnChanges

every change)

ngOnInit

ngDoCheck

ngAfterContentInit

ngAfterContentChecked

ngAfterViewInit

ngAfterViewChecked

ngOnDestroy

→ the class is created (NO inputs yet ▲)

→ an @Input changed (runs BEFORE ngOnInit, and on

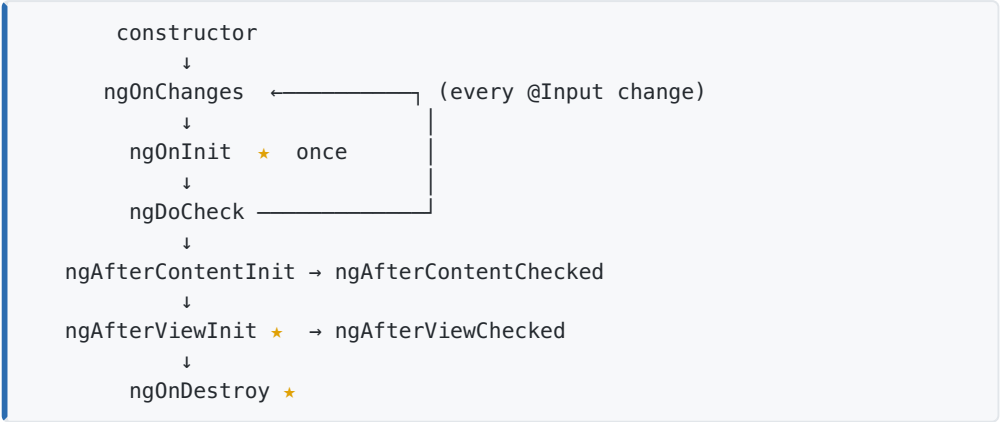
→ inputs are ready → DO YOUR SETUP HERE ★

→ every change detection run (expensive ▲)

→ projected <ng-content> is ready

→ the template + @ViewChild are ready ★

→ CLEAN UP HERE ★ (unsubscribe!)



Hook	When	Use it for
ngOnChanges	an @Input changes	react to input changes
ngOnInit ★	once, after the first ngOnChanges	initial data fetch, subscriptions
ngDoCheck	every CD cycle	custom change detection (rare, costly)

Hook	When	Use it for
<code>ngAfterContentInit</code>	after <code><ng-content></code> projection	access <code>@ContentChild</code>
<code>ngAfterViewInit</code> ★	after the view + children render	access <code>@ViewChild</code> , DOM measurement
<code>ngOnDestroy</code> ★	just before destruction	unsubscribe, clear timers, remove listeners

Real-world idea: **a person's life.**

```

constructor      → born
ngOnInit         → school starts – now you can actually do things ★
ngAfterViewInit  → you can see and use everything around you
ngOnDestroy      → handing back your ID card on the last day ★

```

The three most-asked follow-ups

1. Why `ngOnInit` and not the constructor?

```

constructor → the class exists, but @Input values are NOT set yet ▲
              → this.user is undefined ▲

ngOnInit    → Angular has set every @Input ✓ → safe to use them

```

Also: the constructor's job is dependency injection only. Doing HTTP there makes the class impossible to test without side effects.

2. `ngAfterViewInit` + updating a value = `ExpressionChangedAfterItHasBeenCheckedError`

```

Angular already checked the view.
You change a bound value in ngAfterViewInit.
    → in dev mode Angular re-checks and sees a DIFFERENT value → throws
★
Fix: setTimeout(), or cdr.detectChanges()

```

3. What must go in `ngOnDestroy` ?

- ✓ unsubscribe manual subscriptions (or use takeUntil / async pipe)
- ✓ clearInterval / clearTimeout
- ✓ remove window / document event listeners
- ✓ complete any Subjects you own

● In your RoboGebra code

ngOnDestroy → 100 components
takeUntil(\$destroy) → 426 usages ★

Easy memory

constructor → ngOnChanges → ngOnInit ★ → ngDoCheck
→ AfterContentInit/Checked → AfterViewInit ★ /Checked → ngOnDestroy ★

@Inputs are NOT ready in the constructor → setup goes in ngOnInit ★
@ViewChild is only ready in ngAfterViewInit ★
Cleanup ALWAYS in ngOnDestroy ★
Changing a bound value in AfterViewInit →
ExpressionChangedAfterItHasBeenChecked ▲

Q: What are the types of data binding in Angular?

The easiest way to remember — **by the brackets:**

{{ value }}	INTERPOLATION	→ class → template
[prop]="value"	PROPERTY binding	→ class → template
(event)="handler()"	EVENT binding	→ template → class
[(ngModel)]="value"	TWO-WAY	→ both directions ★

CLASS		TEMPLATE
{{ }} and []	→	(data flows DOWN)
←	←	(events flow UP)
() = []		
[()] = both = "banana in a box"		

<code><h1>{{ title }}</h1></code>	<code><!-- interpolation --></code>
<code></code>	<code><!-- property --></code>
<code><button (click)="save()">Save</button></code>	<code><!-- event --></code>
<code><input [(ngModel)]="name"></code>	<code><!-- two-way --></code>
<code><div [class.active]="isActive"></code>	<code><!-- class binding --></code>
<code><div [style.color]="color"></code>	<code><!-- style binding --></code>
<code><app-child [user]="user" (selected)="onSel(\$event)"></code>	<code><!-- component I/O --></code>

★ The memory hook for `[(ngModel)]` : **"banana in a box"** — the parentheses look like a banana inside the square box.

Easy memory

<code>{{ }}</code>	interpolation	↓	
<code>[]</code>	property	↓	
<code>()</code>	event	↑	
<code>[()]</code>	two-way	↓ ↑	"banana in a box"

Q: How does two-way binding actually work under the hood?

The easiest way to remember:

`[(ngModel)]` is NOT magic. It's just `[ngModel]` + `(ngModelChange)` in one shorthand. ★

<code><!-- what you write --></code>
<code><input [(ngModel)]="name"></code>
 <code><!-- what it actually means --></code>
<code><input [ngModel]="name" (ngModelChange)="name = \$event"></code>

name	—[ngModel]→	the input's value	(down)
name	←(ngModelChange)—	the user types	(up)

★ And it works on **your own components** too — the naming convention is the trick:

```
@Input() value!: string;
@Output() valueChange = new EventEmitter<string>(); // ★ MUST be
<input-name> + "Change"
```

```
<app-counter [(value)]="count"></app-counter>    <!-- now works! -->
```

The rule: an `@Input` called `x` plus an `@Output` called `xChange` gives you `[(x)]` for free.

Easy memory

`[(ngModel)] = [ngModel] + (ngModelChange)` – just a shorthand ★

Custom two-way: `@Input() value + @Output() valueChange`
→ the "Change" suffix is what makes `[(value)]` work ★
Needs `FormsModule` for `ngModel`.

Q: What are template reference variables?

`#name` = a variable that points at an `ELEMENT` or `COMPONENT` in the template.

```
<input #phoneInput type="text">
<button (click)="call(phoneInput.value)">Call</button>

<app-child #child></app-child>
<button (click)="child.reset()">Reset</button>    <!-- call a child's
method ★ -->

<video #player></video>
<button (click)="player.play()">Play</button>
```

`#var` on an HTML element → the DOM element
`#var` on a component → the COMPONENT INSTANCE ★

▲ Scope: a template reference variable is only visible **inside its own template**, and not inside a different `*ngIf` / `*ngFor` embedded view.

Easy memory

```
#name → element (or COMPONENT INSTANCE) reference, usable in the template only  
<input #x> then x.value | <app-child #c> then c.method() ★
```

Q: @ViewChild vs @ContentChild?

The easiest way to remember:

```
@ViewChild    → something in MY OWN template          ← ready in  
ngAfterViewInit ★  
@ContentChild → something PROJECTED INTO me via <ng-content> ←  
ngAfterContentInit ★
```

```
<!-- parent.component.html -->  
<app-card>  
  <app-badge></app-badge>      ← PROJECTED content → @ContentChild in  
AppCard  
</app-card>  
  
<!-- card.component.html -->  
<div class="card">  
  <h2 #title></h2>              ← CardComponent's OWN view → @ViewChild  
  <ng-content></ng-content>  
</div>
```

```
@ViewChild('title') title!: ElementRef;           // my own template  
@ContentChild(BadgeComponent) badge!: BadgeComponent; // projected in  
  
@ViewChildren(ItemComponent) items!: QueryList<ItemComponent>; // many
```

```
VIEW    = what I wrote in MY template      → ngAfterViewInit  
CONTENT = what the PARENT put inside my tags → ngAfterContentInit ★  
          (content is checked FIRST, then view)
```

▲ { static: true } VS { static: false }:

```
static: true → available in ngOnInit, but ONLY if the element is not
inside *ngIf ★
static: false → (default) available in ngAfterViewInit
```

Easy memory

```
@ViewChild → MY template → ngAfterViewInit ★
@ContentChild → PROJECTED in → ngAfterContentInit ★
Content is resolved BEFORE view.
static: true → usable in ngOnInit (only if not inside *ngIf)
```

Q: What is **ng-content** and content projection?

Content projection = let the PARENT decide what goes INSIDE your component.

(React calls this `children`.)

```
<!-- card.component.html -->
<div class="card">
  <div class="card-header"><ng-content select="[header]"></ng-content></div>
  <div class="card-body"> <ng-content></ng-content></div>
  <div class="card-footer"><ng-content select="[footer]"></ng-content></div>
</div>
```

```
<!-- usage -->
<app-card>
  <h2 header>Title</h2>
  <p>Any body content the parent wants</p>
  <button footer>OK</button>
</app-card>
```

Without projection → the card must know about EVERY possible body
With projection → the card owns the FRAME; the parent owns the CONTENT

✓

Real-world idea: a **picture frame**. The frame doesn't care what picture goes in it.

Easy memory

```
<ng-content> = a SLOT the parent fills  
select="[header]" → multi-slot projection  
Projected content is queried with @ContentChild, ready in  
ngAfterContentInit ★
```

Q: `ng-template` vs `ng-container` ?

```
ng-template → a template that renders NOTHING until something uses it ★  
ng-container → an invisible GROUPING tag; renders its children, but no  
element
```

```
<!-- ng-template – NOT rendered by itself -->  
<ng-template #loading>  
  <app-spinner></app-spinner>  
</ng-template>  
  
<div *ngIf="data; else loading">{{ data }}</div>      <!-- ★ used via  
'else' -->
```

```
<!-- ng-container – avoids a useless wrapper <div> -->  
<ng-container *ngIf="user">  
  <h2>{{ user.name }}</h2>  
  <p>{{ user.email }}</p>  
</ng-container>  
<!-- output: just the h2 and p – NO extra div ★ -->
```

★ The other big use — **two structural directives on one element is illegal:**

```
<div *ngIf="show" *ngFor="let x of list">      <!-- ✗ compile error -->  
  
<ng-container *ngIf="show">                    <!-- ✓ -->  
  <div *ngFor="let x of list">{{ x }}</div>  
</ng-container>
```


Easy memory

ng-template → defined but NOT rendered until referenced (*ngIf...else) ★
ng-container → groups elements with NO extra DOM node ★
→ also how you combine *ngIf and *ngFor

Q: Structural vs attribute directives?

STRUCTURAL → changes the DOM STRUCTURE (adds/removes elements) → prefixed with *
ATTRIBUTE → changes the APPEARANCE or BEHAVIOUR of an existing element

Structural: *ngIf *ngFor *ngSwitchCase ← the * is sugar for <ng-template>
Attribute : ngClass ngStyle ngModel ← no *

```
<!-- what you write -->
<div *ngIf="show">Hello</div>

<!-- what Angular actually creates ★ -->
<ng-template [ngIf]="show">
  <div>Hello</div>
</ng-template>
```

★ That desugaring explains why you can't put two structural directives on one element — there'd be two competing `<ng-template>` wrappers.

Custom attribute directive:

```
@Directive({ selector: '[appHighlight]' })
export class HighlightDirective {
  @HostListener('mouseenter') onEnter() {
    this.el.nativeElement.style.background = 'yellow';
  }
  constructor(private el: ElementRef) {}
}
```

Easy memory

STRUCTURAL (*) → adds/removes DOM → *ngIf *ngFor *ngSwitchCase
the * is SUGAR for <ng-template> ★ → which is why two on one element fails
ATTRIBUTE → changes look/behaviour → ngClass ngStyle ngModel

Q: Explain *ngIf, *ngFor and trackBy.

```
<div *ngIf="user; else noUser">{{ user.name }}</div>
<ng-template #noUser>No user</ng-template>

<li *ngFor="let item of items; let i = index; trackBy: trackById">
  {{ i }} - {{ item.name }}
</li>
```

```
trackById(index: number, item: Item): number {
  return item.id; // ★ a STABLE identity
}
```

Why trackBy matters — the whole point ★

WITHOUT trackBy:
the array reference changes → Angular assumes EVERYTHING is new
→ it DESTROYS every DOM node and rebuilds them ▲
→ you lose focus, scroll position, and animations, on every refresh

WITH trackBy:
Angular matches items by your key → reuses the unchanged DOM nodes ✓
→ only the genuinely changed rows re-render ▲

1000-row list refreshed by polling:
without trackBy → 1000 DOM nodes destroyed + recreated, every poll
with trackBy → 0 nodes touched if nothing changed ▲

*ngFor local variables worth knowing:

index, first, last, even, odd

● In your RoboGebra code

trackBy → 55 usages in robogeбра-mobile ★

■ "We use **trackBy** on every list that refreshes — the chapter progress list and the study-material results especially. Without it, a poll or a filter change re-creates every row, which loses scroll position and makes the list flicker."

Easy memory

```
*ngIf="x; else tpl" + <ng-template #tpl>  
*ngFor="let i of list; let idx = index; trackBy: trackById"
```

trackBy = give each row a STABLE id → Angular REUSES DOM nodes ★
Without it: the whole list is destroyed and rebuilt on every change ▲

Q: ngClass vs ngStyle ?

ngClass → applies CSS CLASSES
ngStyle → applies INLINE STYLES

```
<div [ngClass]="{ active: isActive, disabled: isDisabled }"></div>  
<div [ngStyle]="{ color: textColor, 'font-size.px': size }"></div>  
  
<!-- single-value shortcuts (preferred – faster) ★ -->  
<div [class.active]="isActive"></div>  
<div [style.color]="textColor"></div>
```

★ Prefer classes over inline styles: classes are cacheable, themeable and CSP-friendly.

Easy memory

ngClass → CSS classes | [class.active]="x" (single)
ngStyle → inline style | [style.color]="x" (single)
Prefer CLASSES – themeable and cacheable ★

Q: In Angular 17+, what are the new control flow blocks?

```
*ngIf    → @if / @else if / @else
*ngFor    → @for (...; track ...)    ← `track` is MANDATORY ★
*ngSwitch → @switch / @case / @default
new       → @defer                  ← LAZY-LOAD part of a template ★
```

```
@if (user) {
  <p>{{ user.name }}</p>
} @else {
  <p>Loading...</p>
}

@for (item of items; track item.id) {          <!-- track is REQUIRED ★
  -->
  <li>{{ item.name }}</li>
} @empty {
  <li>No items</li>                            <!-- ★ built-in empty
state -->
}

@defer (on viewport) {                          <!-- loads only when
  scrolled to ★ -->
  <app-heavy-chart />
} @placeholder {
  <div class="skeleton"></div>
}
```

Why it's better:

- ✓ No import needed (CommonModule isn't required)
- ✓ `track` is MANDATORY → the trackBy performance bug becomes impossible
- ★
- ✓ Built-in @empty
- ✓ Better type narrowing
- ✓ @defer gives you lazy loading at the TEMPLATE level, not just the route level ★

▲ Be honest: your apps are Angular 16/18 and mostly still use `*ngIf` / `*ngFor`. Say you know the new syntax and would adopt it in new templates.

Easy memory

```
@if / @else | @for (...; track x) ★ MANDATORY track | @switch | @empty
@defer (on viewport | on interaction | on idle) ★ = lazy load PART of a
template
No CommonModule import needed.
```

Component Communication

Q: How do parent and child components communicate?

The easiest way to remember:

```
Parent → Child : @Input()   ↓   data down
Child → Parent : @Output()  ↑   events up
```

```
// child
@Input() user!: User;
@Output() selected = new EventEmitter<User>();

select() { this.selected.emit(this.user); }
```

```
<!-- parent -->
<app-child [user]="currentUser" (selected)="onSelected($event)"></app-
child>
```

```

PARENT
[user] | ▲ (selected)
data ▼ | event
CHILD
```

Other (weaker) options:

```
@ViewChild → the parent calls a child's method directly (tight coupling
▲ )
a SERVICE → for anything not directly related ★ (see the next question)
```

★ The rule: **data down, events up**. A child should never mutate an object the parent owns — emit and let the parent decide.

Easy memory

```
@Input() ↓ data down | @Output() + EventEmitter ↑ events up
"Data down, events up" ★
@ViewChild for direct access (tighter coupling)
```

Q: How do sibling components communicate?

Siblings CANNOT talk directly.

Option 1 → through the shared PARENT (@Output up, then @Input down)
Option 2 → through a SHARED SERVICE with a Subject ★ (the real answer)

```
@Injectable({ providedIn: 'root' })
export class MessageService {
  private messageSubject$ = new BehaviorSubject<string>('');
  message$ = this.messageSubject$.asObservable(); // ★ read-only
  outside

  send(msg: string) { this.messageSubject$.next(msg); }
}
```



Real-world idea: two colleagues who don't share a manager use a **shared noticeboard**, not a chain of messages up and back down.

● In your RoboGebra code

This is the dominant pattern in your app — **122 BehaviorSubject** usages.

File: `core/services/user-summary.service.ts`

```

@Injectables({ providedIn: 'root' })
export class UserSummaryService {
  private userSummarySubject$ = new
  BehaviorSubject<UserSummaryGroupDTO>(null);

  getUserSummarySubject(): Observable<UserSummaryGroupDTO> {
    return this.userSummarySubject$.asObservable(); // ★ expose
  }
  READ-ONLY
  getUserSummary(): UserSummaryGroupDTO {
    return this.userSummarySubject$.getValue(); // ★
  }
  synchronous read
}

```

■ "The subject is kept private and only `asObservable()` is exposed, so no component can call `.next()` and mutate global state from anywhere. We also expose a synchronous `getValue()` accessor, because the institute-code HTTP interceptor needs the value without subscribing."

Easy memory

Siblings → a SHARED SERVICE with a BehaviorSubject ★
 Keep the subject PRIVATE; expose `asObservable()` ★
 BehaviorSubject because a late subscriber still needs the CURRENT value.

Services & Dependency Injection

Q: What is dependency injection in Angular?

DI = you DECLARE what you need; Angular CREATES it and HANDS it to you.
 You never write ``new``. ★

```

@Injectable({ providedIn: 'root' })
export class UserService {
  constructor(private http: HttpClient) {} // ★ injected, not
  `new`
}

@Component({ ... })
export class UserComponent {
  constructor(private userService: UserService) {} // ★ injected
}

// modern alternative (Angular 14+)
private userService = inject(UserService);

```

Angular's INJECTOR

```

  | creates HttpClient
  | creates UserService(HttpClient)
  | creates UserComponent(UserService)
  ↓
Everything wired automatically ✓

```

Why it matters:

- ✓ Testability – inject a mock instead of the real service
- ✓ Singletons – one HTTP client, one auth state
- ✓ Loose coupling – swap the implementation without touching the consumer

● In your RoboGebra code

`inject()` is used **30 times** in `robogebra-mobile` – modern DI adopted alongside constructor injection.

Easy memory

DI = declare it in the constructor (or `inject()`), Angular provides it ★
`@Injectable({ providedIn: 'root' })` = an app-wide, tree-shakable singleton
 Benefit: testability (inject mocks) + singletons + loose coupling

Q: `providedIn: 'root'` vs module vs component?

`providedIn: 'root'` → ONE instance app-wide ★ + TREE-SHAKABLE (removed if unused)
`providers: []` in a module → one instance per module injector
`providers: []` in a component → a NEW instance per component instance ★

```
@Injectable({ providedIn: 'root' })    // singleton, tree-shakable ★
export class AuthService {}

@Component({
  providers: [DrawerStateService]      // ★ each component gets its OWN
})
export class DrawerComponent {}
```

root → 1 instance for the whole app (auth, config, cache)
component → 1 instance PER COMPONENT INSTANCE (per-widget state)

▲ The lazy-loading trap:

A service provided in a LAZY-LOADED module gets its OWN instance, separate from the root one ▲
→ two "singletons", two different states
Fix: `providedIn: 'root'` ★

Easy memory

`providedIn: 'root'` ★ → app-wide singleton + tree-shakable → the default choice
component providers → a fresh instance per component (per-widget state)
▲ a service provided in a LAZY module = a SECOND instance ▲

Q: What are hierarchical injectors?

Angular has a TREE of injectors that mirrors the component tree.
A lookup walks UP the tree until it finds a provider. ★

Root injector	(providedIn: 'root')
↑	
Module injector	(module providers)
↑	
Component injector	(component providers)
↑	
Child component	← the lookup starts HERE and walks UP

The NEAREST provider wins ★

Modifiers worth naming:

```
@Optional() → don't throw if it's missing (inject null)
@Self()     → look ONLY in this component's own injector
@SkipSelf() → skip mine, start at the parent
@Host()     → stop at the host component
```

Easy memory

Injectors form a TREE mirroring the components; lookup walks UP; NEAREST wins ★
 @Optional / @Self / @SkipSelf / @Host modify the search.

Q: What is an injection token, and when do you use it?

An interface disappears at runtime (TypeScript types are erased), so it CANNOT be a DI key.

InjectionToken gives you a runtime key for non-class values. ★

```
export const API_URL = new InjectionToken<string>('api.url');

providers: [{ provide: API_URL, useValue: 'https://api.robogebra.com' }]

constructor(@Inject(API_URL) private apiUrl: string) {}
// or: private apiUrl = inject(API_URL);
```

Use a token for: config strings, feature flags, an interface-typed dependency,
or a third-party object you don't own ★

Easy memory

InjectionToken = a DI key for non-class values (strings, config, interfaces) ★
Because interfaces DON'T EXIST at runtime.
{ provide: TOKEN, useValue: ... } + @Inject(TOKEN) or inject(TOKEN)

Routing

Q: How do you configure routes in Angular?

```
const routes: Routes = [  
  { path: '', component: HomeComponent },  
  { path: 'books', component: BookListComponent, canActivate:  
    [AuthGuard] },  
  { path: 'books/:id', component: BookDetailComponent, resolve: { book:  
    BookResolver } },  
  { path: 'admin', loadChildren: () => import('./admin/  
    admin.module').then(m => m.AdminModule) },  
  { path: '**', component: PageNotFoundComponent } // ★ MUST  
    be last  
];
```

▲ Order matters — the router takes the FIRST match:

{ path: '**' } placed FIRST → every route becomes 404 ▲
Always put the wildcard LAST ★

● In your RoboGebra code

`core/AppRoutes.ts` holds route path constants, referenced by every guard and redirect — so a renamed route is a compile error, not a silent broken link. Good practice to mention.

Easy memory

path | component | canActivate | resolve | loadChildren | ** (LAST ★)
The router takes the FIRST match → order matters
Keep route paths in a constants file → renames become compile errors ★

Q: routerLink and programmatic navigation?

```
<a routerLink="/books" routerLinkActive="active">Books</a>  
<a [routerLink]="['/books', book.id]" [queryParams]="{ page: 2 }">Detail</a>
```

```
this.router.navigate(['books', id], { queryParams: { page: 2 } });  
this.router.navigateByUrl('/books/5');  
this.router.navigate(['../'], { relativeTo: this.route }); // relative  
★
```

▲ Never use href for internal links:

href → a FULL page reload → the whole SPA restarts ▲
routerLink → the router swaps the component ✓

Easy memory

routerLink (template) | router.navigate(...)) (code)
routerLinkActive="active" for styling
▲ href on an internal link = FULL RELOAD ▲

Q: Route params vs query params?

/books/5 → ROUTE param → identifies WHICH resource ★
/books?page=2&sort=name → QUERY param → optional filters/options ★

```
// snapshot – fine if the component is RECREATED each time
const id = this.route.snapshot.paramMap.get('id');

// observable – REQUIRED if you navigate between /books/5 → /books/6
//               without leaving the component ★
this.route.paramMap.pipe(takeUntil(this.destroy$)).subscribe(params => {
  this.loadBook(params.get('id'));
});

this.route.queryParamMap.subscribe(q => this.page = +q.get('page'));
```

▲ The classic bug:

Using `.snapshot`, then navigating `/books/5 → /books/6`
 → Angular REUSES the component, `ngOnInit` does NOT run again
 → the page still shows book 5 ▲
 Fix: subscribe to `paramMap` ★

Easy memory

`/books/5 → paramMap (WHICH resource) | ?page=2 → queryParamMap (options)`
`snapshot = read ONCE | paramMap.subscribe = reacts to param changes ★`
 ▲ `snapshot + navigating between ids of the same route = stale page ▲`

Q: What are route guards?

<code>CanActivate</code>	→ may they ENTER this route?	★ (auth)
<code>CanActivateChild</code>	→ may they enter the CHILDREN?	
<code>CanDeactivate</code>	→ may they LEAVE?	★ (unsaved changes)
<code>Resolve</code>	→ PRE-FETCH data before the route activates	★
<code>CanMatch</code>	→ should this route even be CONSIDERED?	(replaces
<code>CanLoad</code>		)

Navigation requested

↓

`CanMatch → CanActivate → CanActivateChild → Resolve → component renders ✓`

↓ any returns false / a `UrlTree`

navigation is CANCELLED

Modern functional style (Angular 15+):

```
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.isLoggedIn() ? true : router.createUrlTree(['/login']);
};
```

● In your RoboGebra code — the real guard ★

File: `core/guard/auth-guard.service.ts` (4 guards, 27 resolvers)

```
canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot):
Observable<boolean> {
  return this.authService.isLoggedInObservable().pipe(
    tap(isLoggedIn => {
      if (!isLoggedIn) {
        this.authService.setRedirectUrl(state.url); // ★
        remember the destination
        this.router.navigate(['/'] + AppRoutes.LOGIN);
      }
    }),
    map(isLoggedIn => {
      if (!isLoggedIn) return false;
      if (this.authService.isLoggedInAsParent()) { // ★ ROLE-
        based routing
        const targetPath = state.url.split('?')[0].replace(/^\/\//,
        '');
        const isAllowed = PARENT_ALLOWED_ROUTES.some(r =>
          targetPath === r || targetPath.startsWith(r + '/'));
        if (!isAllowed) { this.router.navigate(['/'] +
        AppRoutes.PARENT_DASHBOARD)]; return false; }
      }
      return true;
    })
  );
}
```

■ "The guard returns an `Observable<boolean>` rather than a plain boolean, because login state is itself async — the router waits for it to emit. It stores the requested URL before redirecting, so after login the user lands where they originally wanted instead of on a generic home page. On top of authentication we do role-based routing: a parent account is only allowed into the parent routes and gets bounced to the dashboard otherwise."

Your other guards: `login-guard` (keeps a logged-in user off `/login`), `public-preview-guard`, `anonymous-solution-guard`.

★ **Resolvers:** 27 of them, including `chapter-access-ruler.resolver.ts` — *"the access-ruler resolvers decide what a user may see before the route activates, so the component never renders a 'no access' flash."*

Easy memory

```
CanActivate (enter) ★ | CanDeactivate (leave – unsaved changes) ★  
Resolve (pre-fetch) ★ | CanMatch (should this route be considered)
```

```
Return true | false | a UrlTree (redirect) | or an Observable of those ★  
Modern: functional guards with inject()
```

Q: What is lazy loading and why use it?

Lazy loading = don't download a feature's code until the user navigates to it. ★

```
// module-based  
{ path: 'admin', loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule) }  
  
// standalone (Angular 14+)  
{ path: 'admin', loadComponent: () => import('./admin/admin.component').then(m => m.AdminComponent) }
```

EAGER (everything in main.js)

```
main.js = 4 MB  
fast)  
↓  
visit /admin)  
3s to first paint
```

LAZY

```
main.js = 1 MB ▲ (first paint is  
admin.js = 1 MB (only if you  
books.js = 1 MB
```

Real-world idea: a **restaurant menu**. They don't cook every dish the moment you sit down.

Preloading strategies:

```
RouterModule.forRoot(routes, { preloadingStrategy: PreloadAllModules })
// downloads lazy chunks in the BACKGROUND after the app boots ★
// → fast first paint AND instant navigation
```

Easy memory

loadChildren (module) / loadComponent (standalone) → separate chunk per feature ★
 Benefit: smaller initial bundle → faster first paint
 PreloadAllModules = fast start + instant later navigation ★

Forms

Q: Template-driven vs reactive forms?

The easiest way to remember:

TEMPLATE-DRIVEN → the logic lives in the HTML → ngModel → simple forms
 REACTIVE → the logic lives in the CLASS → FormGroup → everything else ★

	Template-driven	Reactive ★
Module	FormsModule	ReactiveFormsModule
Where the model lives	the template	the class
Setup	[(ngModel)]	FormGroup / FormControl
Validation	directives in HTML	functions in TypeScript
Async / dynamic	painful	easy ★
Testable without the DOM	✗	✓ ★
Best for	login, a 2-field search	everything real


```
// REACTIVE
this.form = this.fb.group({
  name: ['', [Validators.required, Validators.minLength(3)]],
  email: ['', [Validators.required, Validators.email]]
});
```

```
<form [formGroup]="form" (ngSubmit)="onSubmit()">
  <input formControlName="name">
  <div *ngIf="form.get('name')?.touched && form.get('name')?.errors?.
['required']">
    Name is required
  </div>
</form>
```

★ The deciding line: **"reactive forms are testable without rendering the template."**

● In your RoboGebra code

```
Validators.* → 67 usages
FormBuilder → 2 usages (most forms build FormGroup explicitly)
```

File: `features/pages/settings/profile/parent-info/parent-info.component.ts` — an async availability check with `debounceTime` + `distinctUntilChanged` before validating.

Easy memory

```
Template-driven → ngModel → logic in HTML → simple forms
REACTIVE ★      → FormGroup/FormControl → logic in the CLASS → testable,
dynamic
```

"Reactive forms are testable without rendering the template." ★

Q: Explain `FormControl`, `FormGroup` and `FormArray`.

```
FormControl → ONE field
FormGroup   → a fixed OBJECT of controls
FormArray   → a DYNAMIC LIST of controls (add/remove at runtime) ★
```

```

this.form = new FormGroup({
  name: new FormControl(''),
  address: new FormGroup({           // nested group
    city: new FormControl(''),
    zip: new FormControl('')
  }),
  phones: new FormArray([           // ★ dynamic
    new FormControl('')
  ])
});

get phones() { return this.form.get('phones') as FormArray; }
addPhone()   { this.phones.push(new FormControl('')); }
removePhone(i: number) { this.phones.removeAt(i); }

```

```

FormGroup (form)
├─ FormControl  name
├─ FormGroup    address
│   └─ FormControl city
│       └─ FormControl zip
└─ FormArray    phones ← grows and shrinks at runtime ★
    └─ FormControl [0]
        └─ FormControl [1]

```

The control state flags you should be able to name:

```
valid / invalid | touched / untouched | dirty / pristine | pending (async)
```

★ **touched** vs **dirty**: **touched** = they focused and left it; **dirty** = they changed the value. Error messages usually use **touched**.

Easy memory

```

FormControl = 1 field | FormGroup = an object | FormArray = a dynamic
LIST ★
State: valid | touched (focused+left) | dirty (value changed) | pending
Show errors on `touched`, not on `dirty` ★

```

Q: What is FormBuilder?

```
FormBuilder = a shorthand so you stop typing `new FormControl` everywhere.
```

```
// without
this.form = new FormGroup({
  name: new FormControl('', [Validators.required])
});

// with FormBuilder ✓
this.form = this.fb.group({
  name: ['', [Validators.required]],           // [initial value, sync
  validators, async validators]
  email: ['', [Validators.required, Validators.email]]
});
```

★ Angular 14+ adds `NonNullableFormBuilder`, so `reset()` returns to the initial value rather than `null` — a nice detail:

```
constructor(private fb: NonNullableFormBuilder) {}
```

Easy memory

```
fb.group({ name: ['', [Validators.required]] })
Array = [initialValue, syncValidators, asyncValidators] ★
NonNullableFormBuilder → reset() restores the initial value, not null
```

Q: How do validators work — built-in, custom and async?

```
// BUILT-IN
Validators.required | minLength(3) | maxLength(50) | email |
pattern(/.../) | min | max

// CUSTOM (sync) — return null when VALID ★
export function noSpaces(control: AbstractControl): ValidationErrors |
null {
  return control.value?.includes(' ') ? { noSpaces: true } : null;
}

// CROSS-FIELD — goes on the GROUP, not the control ★
export function passwordsMatch(group: AbstractControl): ValidationErrors |
null {
  return group.get('password')?.value === group.get('confirm')?.value
    ? null : { mismatch: true };
}

this.form = this.fb.group({ password: [''], confirm: [''] }, { validators:
passwordsMatch });

// ASYNC — returns an Observable ★
export class EmailValidator implements AsyncValidator {
  validate(control: AbstractControl): Observable<ValidationErrors | null>
  {
    return this.http.get(`/api/check-email?e=${control.value}`).pipe(
      map(taken => (taken ? { emailTaken: true } : null)),
      catchError(() => of(null))
    );
  }
}
```

★ **The rule everyone forgets: return `null` when the value is VALID.** Returning an object means "there is an error".

```
null          → VALID ✓
{ errorKey: ... } → INVALID ✗
```

▲ Async validators run **only after** all sync validators pass, and the control's state is **PENDING** in between — so disable the submit button on `form.pending` too.

Easy memory

Built-in: required, minLength, email, pattern
Custom sync: (control) => null (VALID ★) | { key: true } (invalid)
Cross-field: put the validator on the GROUP ★
Async: returns an Observable → state is PENDING ★ → disable submit on pending too

Q: How do you build dynamic forms? (Angular Formly)

Dynamic form = the fields come from CONFIG (often JSON from the backend), not from hard-coded HTML. ★

```
fields: FormlyFieldConfig[] = [
  { key: 'name', type: 'input', props: { label: 'Name', required: true } },
  { key: 'email', type: 'input', props: { label: 'Email', type: 'email' } },
  { key: 'dob', type: 'datepicker', props: { label: 'Date of birth' } }
];
```

```
<form [formGroup]="form"><formly-form [form]="form" [fields]="fields"
[model]="model"></formly-form></form>
```

JSON config → Formly looks up the `type` in a REGISTRY → renders that component

↓

This is the STRATEGY / FACTORY pattern ★ → [17 – Design Patterns](./17-solid-design-patterns.md)

■ **Your EasyVisa answer:** *"Formly is essentially a strategy/factory pattern — a JSON config picks the field type and a registry maps that type to a component. In EasyVisa I registered custom field types for our nested reactive forms, which meant a new form field was a config change rather than a template change — the backend could add a field without a front-end release."*

★ That last clause is the business value, and it's what makes the answer land.

Easy memory

Dynamic forms = fields from CONFIG, not hard-coded HTML

Formerly: key + type + props → a registry maps type → component

= the STRATEGY/FACTORY pattern ★

Value: a new field is a CONFIG change, not a release ★

RxJS & Observables

Deep dive on operators: [20 — RxJS Operators](#).

Q: Observable vs Promise?

The easiest way to remember:

Promise → ONE value, eager, cannot be cancelled

Observable → MANY values, lazy, CAN be cancelled ★

	Promise	Observable
Values	exactly one	0..∞ (a stream)
Execution	eager — runs the moment it's created	lazy — nothing runs until <code>subscribe()</code> ★
Cancellable	✗	✓ <code>unsubscribe()</code> ★
Operators	<code>.then</code> chains	100+ operators (<code>map</code> , <code>switchMap</code> , <code>debounceTime</code> ...)
Retry	manual	<code>retry()</code> , <code>retryWhen()</code>

Promise —————[value]————→| one delivery, already dispatched

Observable —[v1]—[v2]—[v3]————→| a subscription, cancellable ★

Real-world idea:

Promise → ordering ONE pizza. Once you order, it's coming.
Observable → a MAGAZINE subscription: many issues, and you can cancel ★

★ The cancellation point is the one that matters in Angular: a user navigating away should cancel the in-flight HTTP request, and only Observables can do that.

Easy memory

Promise = 1 value, EAGER, not cancellable
Observable = many values, LAZY, CANCELLABLE ★
"Nothing happens until you subscribe."

Q: Subject vs BehaviorSubject vs ReplaySubject ?

The easiest way to remember — **what does a LATE subscriber get?**

Subject	→ NOTHING (only future values)	→ an event/ notification
BehaviorSubject	→ the CURRENT value ★	→ state (needs an initial value)
ReplaySubject	→ the last N values	→ history
AsyncSubject	→ only the FINAL value, on complete	→ rare

Subject:	A—B—[you subscribe]—C—D	you get: C, D
BehaviorSubject:	A—B—[you subscribe]—C—D	you get: B ★, C, D
ReplaySubject(2):	A—B—[you subscribe]—C—D	you get: A, B, C, D

```
const s = new Subject<string>(); // no initial value
const bs = new BehaviorSubject<string>('init'); // ★ REQUIRES an initial value
const rs = new ReplaySubject<string>(2); // buffers the last 2
```

Real-world idea:

Subject	→ a LIVE broadcast: join late, you miss what happened
BehaviorSubject	→ a WhatsApp group's pinned message: you see it on joining ★
ReplaySubject	→ the last 2 messages replayed when you join

switchMap → changing the TV channel: the old one stops instantly
mergeMap → several taps filling several buckets at once
concatMap → a queue at a counter: one at a time, in order
exhaustMap → a lift: pressing the button again while the doors close does nothing

```
// SEARCH – cancel the stale request ★
searchTerm$.pipe(debounceTime(300), distinctUntilChanged(),
                 switchMap(term => this.api.search(term)));

// SAVE – order matters, never lose one ★
saveClicks$.pipe(concatMap(data => this.api.save(data)));

// LOGIN – ignore double clicks ★
loginClicks$.pipe(exhaustMap(creds => this.auth.login(creds)));
```

★ The **why** behind `switchMap` for search (say this): *"it prevents the out-of-order-response bug, where a slow query for 'ma' lands after a fast one for 'maths' and overwrites the correct results."*

● In your RoboGebra code

switchMap → 246 usages | map → 496 | filter → 485
mergeMap / concatMap / exhaustMap → 0 ★

■ Be honest and precise: *"We use `switchMap` heavily — in the auth interceptor and in every search — because latest-wins is almost always what you want for reads. We haven't needed `concatMap` or `exhaustMap` yet, but I know where they belong: `concatMap` for ordered writes, `exhaustMap` to stop a double-submitted login."*

Easy memory

switchMap → CANCEL previous ★ (TV channel) → search, latest wins
mergeMap → PARALLEL (many taps) → independent work
concatMap → QUEUE in order ★ (a ticket queue) → ordered writes
exhaustMap → IGNORE new ★ (a lift) → double-click protection

Q: How do you subscribe and unsubscribe? Why does it matter?

An unsubscribed Observable keeps its subscription – and the component – ALIVE.

That's a MEMORY LEAK. ▲

Component destroyed

↓ but the subscription is still active

the callback still runs → it references `this` → the component can't be GC'd

↓

100 navigations later → 100 dead components in memory → the app crawls ▲

The four ways to prevent it, best first:

1. `async` pipe ★ ★ – Angular subscribes AND unsubscribes for you
2. `takeUntilDestroyed()` ★ (Angular 16+) – no boilerplate
3. `takeUntil(this.destroy$)` ★ – the classic pattern
4. `subscription.unsubscribe()` in `ngOnDestroy` – manual, easy to forget

★ Which observables do NOT need unsubscribing?

`HttpClient` calls → they COMPLETE after one emission ✓
route params on a component destroyed with the route ✓

Everything long-lived DOES: a Subject, an interval, `valueChanges`,
a router event stream, a window event ★

That nuance (" `HttpClient` completes, so it self-cleans") is a good thing to volunteer.

Easy memory

Not unsubscribing = a MEMORY LEAK (the component can't be garbage collected) ▲

BEST → `async` pipe ★ ★ (subscribes AND unsubscribes)
then → `takeUntilDestroyed()` (v16+) → `takeUntil(destroy$)` → manual
`unsubscribe`

`HttpClient` COMPLETES → no leak. Subjects/intervals/`valueChanges` DO leak ★

Q: What is the `async` pipe and why prefer it?

`async` pipe = subscribe + unsubscribe + trigger change detection,
all handled by Angular. ★

```
// component – no subscribe, no ngOnDestroy ★  
users$ = this.userService.getUsers();
```

```
<div *ngFor="let user of users$ | async">{{ user.name }}</div>  
  
<!-- with loading and error states -->  
<ng-container *ngIf="users$ | async as users; else loading">  
  <div *ngFor="let user of users">{{ user.name }}</div>  
</ng-container>  
<ng-template #loading><app-spinner/></ng-template>
```

- | | |
|-----------------------|---------------------------------------|
| ✓ No manual subscribe | ✓ No ngOnDestroy needed |
| ✓ No leak possible | ✓ Marks OnPush components for check ★ |

▲ **The trap:** using `| async` twice on the same observable creates **two subscriptions** → two HTTP calls.

```
<!-- ✗ TWO subscriptions, TWO HTTP calls ▲ -->  
<div>{{ (user$ | async)?.name }}</div>  
<div>{{ (user$ | async)?.email }}</div>  
  
<!-- ✓ subscribe ONCE with `as` -->  
<ng-container *ngIf="user$ | async as user">  
  <div>{{ user.name }}</div>  
  <div>{{ user.email }}</div>  
</ng-container>
```

(Or add `shareReplay(1)` to the stream.)

● In your RoboGebra code

`async` appears **311 times** in `robogebra-mobile` templates and code.

Easy memory

```
async pipe = subscribe + unsubscribe + mark for check ★ – the DEFAULT choice
▲ Two `| async` on the same stream = TWO subscriptions = TWO HTTP calls
▲ Fix: `*ngIf="x$ | async as x"` or shareReplay(1) ★
```

Q: Explain the **takeUntil** pattern.

```
export class MyComponent implements OnDestroy {
  private readonly destroy$ = new Subject<void>();

  ngOnInit() {
    this.service.data$.pipe(
      takeUntil(this.destroy$)           // ★ MUST BE LAST in the pipe
    ).subscribe(data => this.data = data);
  }

  ngOnDestroy() {
    this.destroy$.next();                // emit → every takeUntil
    completes
    this.destroy$.complete();            // ★ clean up the Subject itself
  }
}
```

```
destroy$.next()
  ↓
EVERY takeUntil(destroy$) in the component completes at once ✓
one line of cleanup, however many subscriptions you have ★
```

▲ **takeUntil** must be the LAST operator:

```
.pipe(takeUntil(this.destroy$), switchMap(...)) // ✗ switchMap's inner
subscription leaks
.pipe(switchMap(...), takeUntil(this.destroy$)) // ✓
```

Angular 16+ removes the boilerplate entirely:

```
this.service.data$.pipe(takeUntilDestroyed()).subscribe(...); // ★ in
an injection context
```

● In your RoboGebra code

```
takeUntil          → 426 usages ★  
takeUntilDestroyed → 5 usages (the newer code)  
ngOnDestroy        → 100 components
```

Even the HTTP interceptor uses it — `cancelRequests$` lets logout cancel every in-flight request at once:

```
return next.handle(req).pipe(  
  takeUntil(this.cancelRequests$),    // ★ mass-cancel on logout  
  catchError(error => this.handleUnAuthResponse(error, req, next))  
);
```

Easy memory

```
private destroy$ = new Subject<void>();  
.pipe(..., takeUntil(this.destroy$))      ← LAST operator ★  
ngOnDestroy() { destroy$.next(); destroy$.complete(); }
```

One emit cancels EVERY subscription in the component ★
Angular 16+: `takeUntilDestroyed()` – no boilerplate

Q: How do you handle errors in RxJS?

```
catchError → CATCH and RECOVER (return a fallback observable) ★  
retry(n)    → resubscribe n times  
retryWhen   → conditional / delayed retry  
finalize    → ALWAYS runs, success or error ★ (hide the spinner)
```

```
this.http.get<User[]>('/api/users').pipe(  
  retry(2),                                // try twice more  
  catchError(err => {  
    this.notify.error('Could not load users');  
    return of([]);                          // ★ RECOVER with a  
  fallback  
  }),  
  finalize(() => this.loading = false)      // ★ always  
) .subscribe(users => this.users = users);
```

▲ The critical rule:

An ERROR TERMINATES the observable. Permanently. ★

So if the error escapes to the outer stream, the stream is DEAD – a search box would stop working after the first failed request ▲

```
// ✗ one failure kills the search forever
searchTerm$.pipe(switchMap(t => this.api.search(t)), catchError(() =>
of([])))

// ✓ catch INSIDE the inner observable → the outer stream survives ★
searchTerm$.pipe(switchMap(t => this.api.search(t).pipe(catchError(() =>
of([])))))
```

★ **Where to put `catchError` is the real question here** — inside the inner observable, not outside it.

● In your RoboGebra code

`catchError` → 338 usages | `finalize` → 254 usages ★

`finalize` at 254 usages is the spinner-hiding pattern — mention that: "*`finalize` guarantees the busy spinner is hidden whether the call succeeded or failed.*"

Easy memory

`catchError` → recover with a fallback: `return of([])` ★
`retry(n)` | `retryWhen` | `finalize` → ALWAYS runs (hide the spinner) ★

▲ An error KILLS the stream permanently.
Put `catchError` INSIDE the `switchMap`'s inner observable ★
or one failure ends the search box forever ▲

State Management (NgRx)

Deep dive: [21 — NgRx](#).

Q: What is NgRx and what are its building blocks?

NgRx = Redux for Angular – ONE immutable store, changed only by dispatched actions.



Action → "what happened" (an event, past tense: [Books] Load Success)
Reducer → a PURE function (state, action) => newState – no side effects ★
Store → the single source of truth, immutable
Selector → a MEMOISED read of a slice ★
Effect → where the impure stuff lives: HTTP, router, storage ★

```
export const loadBooks      = createAction('[Books] Load');
export const loadBooksSuccess = createAction('[Books] Load Success',
  props<{ books: Book[] }>());

export const booksReducer = createReducer(initialState,
  on(loadBooks,      state => ({ ...state, loading: true })), //
  ★ new object, never mutate
  on(loadBooksSuccess, (state, { books }) => ({ ...state, books, loading:
    false })))
);

loadBooks$ = createEffect(() => this.actions$.pipe(
  ofType(loadBooks),
  switchMap(() => this.api.getBooks().pipe(
    map(books => loadBooksSuccess({ books })),
    catchError(err => of(loadBooksFailure({ err }))) // ★ never let
    an effect die
  ))
));
```

Real-world idea: a **bank ledger**. You never edit a past entry; you add a new transaction, and the balance is derived.

Easy memory

Action (what happened) → Reducer (PURE, returns a NEW state) → Store → Selector (memoised)
Effects = side effects (HTTP, router) → dispatch another action ★
NEVER mutate state – always spread into a new object ★

Q: What are effects and why memoise selectors?

EFFECT → keeps side effects OUT of reducers, so reducers stay pure and testable ★
SELECTOR → memoised: if the input slice hasn't changed, it returns the CACHED result
without recomputing ★

```
export const selectBooks = createSelector(selectBooksState, s => s.books);  
  
export const selectExpensiveBooks = createSelector(  
  selectBooks,  
  books => books.filter(b => b.price > 500).sort(...) // recomputed ONLY  
  if books changed ★  
);
```

Any unrelated state change → the selector returns the CACHED array (same reference)

↓

An OnPush component bound to it does NOT re-render ★

★ That link — **memoised selector** → **stable reference** → **OnPush** skips the **re-render** — is the answer that shows you understand *why* it matters.

Easy memory

Effects = side effects out of reducers → reducers stay PURE and testable ★
Selectors are MEMOISED → same input, same reference → OnPush skips re-render ★

Q: **NgRx vs a service with a BehaviorSubject ?**

Service + BehaviorSubject → simple, little code → MOST apps ★
NgRx → many features share state, you need traceability

Use a service + BehaviorSubject	Use NgRx
state used by a few related components	state shared across many unrelated features
a small/medium app	a large app, several teams
you want minimal boilerplate	you need time-travel debugging + a strict audit trail
simple CRUD	complex flows, optimistic updates, undo

● **In your RoboGebra code — the honest answer ★**

robogeбра-mobile: NgRx → NOT used. BehaviorSubject services → 122 usages.

■ "We deliberately use *BehaviorSubject*-backed services rather than NgRx. *BehaviorSubject* gives the two things we actually needed: a current value on subscribe, and a synchronous *getValue()* for callers that can't subscribe — the institute-code HTTP interceptor, for instance. NgRx would have added actions, reducers, effects and selectors for state that only a handful of components read. I'd reach for NgRx when state is shared across many unrelated features and I need time-travel debugging or a strict audit trail — I've worked with it, but on this app it would have been ceremony."

★ **Do not pretend to use NgRx.** "I chose not to, and here's the trade-off" is a *stronger* answer than "yes we use it".

Easy memory

Service + BehaviorSubject ★ → most apps, minimal boilerplate
NgRx → many features + traceability + time-travel debugging

Your answer: "We chose BehaviorSubject services deliberately — NgRx would have been ceremony for state a handful of components read." ★

HTTP

Q: How do you use `HttpClient` ?

```
@Injectable({ providedIn: 'root' })
export class UserService {
  private readonly base = '/api/users';
  constructor(private http: HttpClient) {}

  getAll(page = 0): Observable<User[]> {
    const params = new HttpParams().set('page', page).set('size', 20);
    return this.http.get<User[]>(this.base, { params }); // ★ typed
    generic
  }
  create(user: User): Observable<User> { return
this.http.post<User>(this.base, user); }
  update(id: number, user: User) { return this.http.put<User>(`$
{this.base}/${id}`, user); }
  delete(id: number) { return this.http.delete<void>(`$
{this.base}/${id}`); }
}
```

- ✓ Returns an OBSERVABLE (cancellable, retryable, composable)
- ✓ Typed with a generic → `get<User[]>()`
- ✓ JSON parsed automatically
- ✓ Interceptors apply to every call ★
- ✓ LAZY – nothing is sent until you subscribe ★

▲ The classic beginner bug:

```
this.http.get('/api/users'); // ✗ NO REQUEST IS SENT – nobody
subscribed ▲
this.http.get('/api/users').subscribe(...) // ✓
```

Easy memory

```
http.get<T>(url, { params, headers }) → an OBSERVABLE, lazy ★
No subscribe = NO REQUEST ▲
Typed generics + automatic JSON + interceptors on every call
```

Q: What are HTTP interceptors?

Interceptor = middleware for EVERY HTTP request and response.
One place for auth tokens, errors, logging and retries. ★

Request → [auth] → [logging] → [error] → server
Response ← [auth] ← [logging] ← [error] ← server
(they run in REGISTRATION ORDER for requests,
and in REVERSE for responses ★)

● In your RoboGebra code — your real interceptor ★

File: `core/interceptors/header-authorization.interceptor.ts`

This one file demonstrates **six** interview topics at once:

```

intercept(req: HttpRequest<any>, next: HttpHandler):
Observable<HttpEvent<any>> {
    if (this.isAuthEndpoint(req.url)) {                // 1. don't attach a
token to /auth
        return this.invokeHttpCall(req, next);
    }

    let updatedHeader: HttpHeaders = req.headers;
    if (!req.headers.has('Content-Type') && !(req.body instanceof
FormData)) {
        updatedHeader = updatedHeader.set("Content-Type", 'application/
json');    // 2. FormData guard ★
    }

    return this.authService.getAuthDetailsObservable().pipe(
        switchMap((authDetails: AuthDetails) =>
{
            // 3. the token is ASYNC
            if (authDetails != null) {
                updatedHeader = updatedHeader.set("Authorization", `Bearer
${authDetails.token}`);
            }
            const cloned = req.clone({ headers:
updatedHeader });
            // 4. IMMUTABLE clone ★
            return this.invokeHttpCall(cloned, next);
        })
    );
}

private invokeHttpCall(req, next) {
    return next.handle(req).pipe(

takeUntil(this.cancelRequests$),                        /
// 5. mass-cancel on logout ★
        catchError(err => this.handleUnAuthResponse(err, req,
next))
        // 6. 401 → refresh → retry
    );
}

```

The six things to say about it:

1. `HttpRequest` is IMMUTABLE → you MUST `req.clone()`; mutating it silently does nothing ★
2. `FormData` guard – setting `Content-Type` manually BREAKS multipart file uploads,
because the BROWSER has to generate the multipart boundary itself ★ ★
3. `switchMap` because the token itself arrives asynchronously (from native storage)
4. Auth endpoints are skipped, or login would need a token to get a token
5. `takeUntil(cancelRequests$)` lets LOGOUT cancel every in-flight request at once ★
6. `catchError` centralises 401 → refresh-token → retry in ONE place, instead of
in every one of the 89 services

★ **Point 2 is the one that makes interviewers sit up** — it's a real bug most people have shipped and never diagnosed.

Modern functional style (Angular 15+):

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {  
  const token = inject(AuthService).getToken();  
  return next(token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req);  
};  
provideHttpClient(withInterceptors([authInterceptor]));
```

Easy memory

Interceptor = middleware for EVERY request/response ★
Uses: auth token | error handling | logging | retry | loading spinner

- ▲ `HttpRequest` is IMMUTABLE → `req.clone()` ★
 - ▲ Don't set `Content-Type` on `FormData` → it breaks file uploads ★ ★
 - ▲ Skip the `/auth` endpoints, or login needs a token to get a token
- Requests run in registration order; responses in REVERSE ★

Q: How do you handle HTTP errors gracefully?

```
// in an error interceptor – ONE place for the whole app ★
catchError((error: HttpResponse) => {
  if (error.status === 0)          this.notify.error('Network error – are
you offline?');
  else if (error.status === 401) this.auth.logout();           // token
expired
  else if (error.status === 403) this.router.navigate(['/forbidden']);
  else if (error.status === 404) this.notify.error('Not found');
  else if (error.status >= 500)  this.notify.error('Server error, please
try again');
  return throwError(() => error);    // ★ rethrow so the caller can
also react
})
```

★ `status === 0` is the detail worth knowing:

```
status 0 = the request never reached the server
        → offline, DNS failure, CORS block, or a cancelled request ★
```

That is especially relevant on mobile — pair it with `@capacitor/network`.

Easy memory

```
Handle centrally in an ERROR INTERCEPTOR, not in 89 services ★
0 = network/CORS ★ | 401 = logout | 403 = forbidden | 404 | 5xx = retry
message
Rethrow with throwError(() => error) so callers can still react ★
```

Pipes

Q: What are pipes? Name the common built-in ones.

A pipe TRANSFORMS a value in the template. It never changes the source data. ★

```

{{ name | uppercase }}
{{ price | currency:'INR' }}
{{ date | date:'dd/MM/yyyy' }}
{{ obj | json }}
{{ items | slice:0:5 }}
{{ ratio | percent }}
{{ n | number:'1.2-2' }}
{{ data$ | async }}
{{ name | uppercase | slice:0:10 }}
-->

```

★ the important one
<!-- chained, left to right

Easy memory

uppercase | lowercase | titlecase | date | currency | number | percent
 json | slice | keyvalue | async ★
 Pipes TRANSFORM for display; they never mutate the source ★

Q: Pure vs impure pipes?

PURE (default) → re-runs ONLY when the input REFERENCE changes ▲
 IMPURE → re-runs on EVERY change detection cycle ▲

```
@Pipe({ name: 'filter', pure: false }) // ▲ runs on every CD cycle
```

Pure pipe on a list: push an item (SAME array reference)
 → the pipe does NOT re-run → the UI looks stale ▲
 Fix: create a NEW array (immutability), don't make the pipe impure ★

`async` and `keyvalue` are IMPURE by necessity.
 Everything you write should be PURE. ★

▲ An impure pipe over a 1,000-item list can run thousands of times per second. If you need filtering, do it in the component or with a memoised selector.

Easy memory

PURE ★ → only when the INPUT REFERENCE changes → fast
IMPURE → EVERY change detection cycle → slow, use only if unavoidable

Mutating an array won't re-run a pure pipe → create a NEW array ★
async and keyvalue are impure by design.

Q: How do you create a custom pipe?

```
@Pipe({ name: 'truncate', standalone: true })
export class TruncatePipe implements PipeTransform {
  transform(value: string, limit = 20, suffix = '...'): string {
    if (!value) return '';
    return value.length > limit ? value.slice(0, limit) + suffix : value;
  }
}
```

```
{{ description | truncate:50 }}
{{ description | truncate:50:' [more]' }}
```

▲ The security pipe every Angular dev writes — and the warning to give with it:

```
@Pipe({ name: 'safeHtml' })
export class SafeHtmlPipe implements PipeTransform {
  constructor(private sanitizer: DomSanitizer) {}
  transform(html: string): SafeHtml {
    return this.sanitizer.bypassSecurityTrustHtml(html); // ▲ bypasses
    XSS protection!
  }
}
```

■ "I'd only use *bypassSecurityTrust** on content we control or have sanitised server-side — the name is a deliberate warning. Angular escapes interpolation by default, and that default is what prevents XSS."

● In your RoboGebra code

There is a `sanitization.service.ts` in `core/services/` — mention it as the place that centralises this decision rather than scattering `bypassSecurityTrust` calls.

Easy memory

```
@Pipe({ name: 'x' }) + implements PipeTransform +  
transform(value, ...args)  
Extra pipe args after the colon: {{ x | truncate:50:'...' }}  
▲ bypassSecurityTrustHtml DISABLES XSS protection – only for trusted  
content ★
```

Performance & Change Detection

Q: What is change detection? Default vs `OnPush` ?

Change detection = how Angular keeps the DOM in sync with your data.

DEFAULT → after ANY async event, check the ENTIRE component tree

OnPush → check this component ONLY when something it depends on changes

▲

DEFAULT: one click anywhere

↓

Angular checks EVERY component in the tree
(1000 components = 1000 checks, most pointless)

OnPush: this component is re-checked ONLY when

1. an `@Input` REFERENCE changes ★
2. an event fires INSIDE it (click, etc.)
3. an ``async`` pipe in its template emits ★
4. you call `cdr.markForCheck()` manually

```
@Component({ changeDetection: ChangeDetectionStrategy.OnPush, ... })
```

▲ The trap that catches everyone:

```
// ✗ MUTATING – the reference is unchanged → OnPush does NOT re-render ▲
this.items.push(newItem);

// ✓ NEW reference → OnPush sees the change
this.items = [...this.items, newItem];
```

OnPush + mutation = a UI that silently doesn't update ★
 → which is exactly why OnPush pairs with IMMUTABLE data and the async pipe

● In your RoboGebra code — selective, not global ★

ChangeDetectionStrategy.OnPush → 6 components:
 learning-progress/chapter-progress-list/ ← a long list
 learning-progress/chapter-detail-modal/
 settings/payments/
 books/exercise-solution/.../solution-step-ai-assistant-editor/
 shared/video-splash-screen/

■ "OnPush is applied selectively rather than globally — on the long progress lists and the AI-assistant editor, where default change detection was re-checking on every unrelated event. Turning it on everywhere at once on a mature codebase is a good way to ship silent UI bugs, because any component that mutates an input array stops updating."

★ That last sentence is a genuinely experienced answer.

Easy memory

DEFAULT → checks the WHOLE tree on any async event
 OnPush
 ★ → only on: @Input REFERENCE change | internal event | async pipe | markForCheck

▲ MUTATION doesn't change the reference → OnPush won't update ▲
 this.items.push(x) ✗
 this.items = [...x] ✓

Pair OnPush with IMMUTABLE data + the async pipe ★

Q: What is zone.js and its role?

zone.js MONKEY-PATCHES every async API – setTimeout, addEventListener, XHR/fetch, promises – so Angular knows "something might have changed" and can run change detection AUTOMATICALLY. ★

You click a button

↓

zone.js notices the patched addEventListener fired

↓

Angular runs change detection → the DOM updates ✓
(you never call anything yourself – that's the magic zone.js provides)

★ The cost, and where Angular is going:

zone.js patches EVERYTHING, so change detection runs even for events that changed nothing → wasted work.

Angular 16+ → SIGNALS: the framework knows EXACTLY what changed
Angular 18+ → experimental ZONELESS change detection ★

Escaping the zone for hot code:

```
this.ngZone.runOutsideAngular(() => {  
  // e.g. a mousemove or animation loop – no change detection per frame ★  
});
```

Easy memory

zone.js monkey-patches async APIs → Angular auto-runs change detection ★
Cost: it fires even when nothing changed
Future: signals (v16+) → zoneless (v18 experimental)
runOutsideAngular() for high-frequency events (mousemove, animation) ★

Q: What are signals? (Angular 16+)

A signal = a value that KNOWS who is reading it,
so Angular can update EXACTLY those places – no tree-walking.

★

```
count = signal(0); // writable
double = computed(() => this.count() * 2); // ★ auto-recalculates
constructor() { effect(() => console.log(this.count())); } // ★ runs on
change

increment() { this.count.update(v => v + 1); }
```

```
{{ count() }} <!--
★ note the parentheses – a signal is a FUNCTION -->
```

```
zone.js → "something happened somewhere – re-check the whole tree"
signals → "count changed → update exactly these 3 bindings" ▲
```

Real-world idea: **a newspaper subscription vs shouting in the street**. Signals deliver to exactly the people who asked.

● In your RoboGebra code — be honest ★

```
signal() / computed() / effect() → 0 usages ▲
```

■ *"Signals I've studied but haven't shipped — **robogebra-mobile** is on Angular 18 but grew from Angular 12, and the change-detection model is still zone-based. The migration path I'd take is signal inputs on leaf components first, since those are the lowest-risk and give the clearest win, then work upward. What signals really unlock is zoneless change detection, and that's not something you turn on halfway."*

★ Naming a **migration order** is what a technical lead is expected to have. Claiming adoption invites one follow-up you cannot answer.

Easy memory

```
signal(0) | computed(() => ...) | effect(() => ...)
Read with PARENTHESES: count() ★
.set() | .update(fn)

Why: fine-grained updates instead of walking the tree → the path to
ZONELESS ★
Your honest line: "studied, not shipped – I'd migrate leaf components
first." ★
```

Q: AOT vs JIT compilation?

```
JIT (Just-in-Time) → compile templates in the BROWSER, at runtime → dev
only, historically
AOT (Ahead-of-Time) → compile at BUILD time
★ → the default since Angular 9 (Ivy)
```

| | JIT | AOT ★ |
|-----------------|-----------------------------|--------------------------|
| When | in the browser | at build time |
| Bundle size | larger (ships the compiler) | smaller ★ |
| Startup | slower | faster ★ |
| Template errors | found at runtime ▲ | found at build time ★ |
| Security | uses eval | no eval — CSP-friendly ★ |

★ The best reason: **template type errors become build failures instead of a blank page in production.**

Easy memory

```
AOT ★ = compiled at BUILD time → smaller, faster, template errors caught
at build,
        no eval (CSP-safe). The DEFAULT since Angular 9 (Ivy).
JIT = compiled in the browser → legacy.
```

Q: What techniques do you use to optimise an Angular app?

Group them — a list in random order sounds memorised:

BUNDLE SIZE

- ✓ Lazy-load feature routes (loadChildren / loadComponent) ★
- ✓ AOT + production build (tree-shaking, minification) – the default
- ✓ Analyse with source-map-explorer; @defer for heavy widgets (v17+)
- ✓ Preload strategies for instant later navigation

RENDERING

- ✓ OnPush + immutable data + the async pipe ★
- ✓ trackBy on every list ★
- ✓ Virtual scrolling (CDK) for long lists
- ✓ Pure pipes instead of method calls in templates ★

MEMORY / NETWORK

- ✓ Unsubscribe – async pipe or takeUntil ★
- ✓ debounceTime + switchMap on search ★
- ✓ shareReplay(1) for data fetched by several components
- ✓ Pagination instead of loading everything

▲ The one people never mention — **method calls in templates**:

```
{{ getTotal() }}      <!-- ✗ runs on EVERY change detection cycle ▲ -->
{{ total }}           <!-- ✓ a field -->
{{ items | totalPipe }} <!-- ✓ a PURE pipe – cached ★ -->
```

● In your RoboGebra code

```
trackBy 55 | OnPush 6 (selective) | debounceTime 13 | takeUntil 426 |
async 311
```

Easy memory

```
BUNDLE   → lazy routes ★ | AOT | @defer | source-map-explorer
RENDER   → OnPush + immutable + async ★ | trackBy ★ | virtual scroll |
pure pipes
MEMORY   → unsubscribe ★ | debounce+switchMap ★ | shareReplay(1) |
paginate
```

▲ NEVER call a method in a template – it runs on every CD cycle ▲

Testing

Q: What tools are used for Angular unit testing?

Jasmine → the FRAMEWORK (describe / it / expect / spyOn)
Karma → the RUNNER (executes the tests in a real browser)
TestBed → Angular's utility for building a testing module ★

Many teams now swap Karma for **Jest** (faster, no browser, better watch mode).

● In your RoboGebra code

```
karma + jasmine configured in all three Angular apps
*.spec.ts alongside services and guards
(auth.service.spec.ts, auth-guard.service.spec.ts,
header-authorization.interceptor.spec.ts ...)
```

★ Note *what* is tested: services, guards and the interceptor — the logic-heavy, non-visual parts. That's the right instinct, and worth saying.

Q: What is **TestBed** ?

TestBed = a miniature NgModule for tests — you declare the component and swap real dependencies for mocks. ★

```
describe('UserCardComponent', () => {
  let fixture: ComponentFixture<UserCardComponent>;

  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [UserCardComponent], // standalone
      providers: [{ provide: UserService, useValue:
mockUserService }] // ★ inject a mock
    });
    fixture = TestBed.createComponent(UserCardComponent);
  });

  it('should create', () => {
    expect(fixture.componentInstance).toBeTruthy();
  });
});
```

| | |
|---------------------------|--|
| fixture.detectChanges() | → run change detection ★ (or the template never renders) |
| fixture.nativeElement | → the DOM |
| fixture.componentInstance | → the class |

▲ Forgetting `fixture.detectChanges()` is the #1 reason a template assertion fails for no visible reason.

Q: How do you test a service that uses `HttpClient` ?

```
TestBed.configureTestingModule({ imports: [HttpClientTestingModule] });
const httpMock = TestBed.inject(HttpTestingController);

service.getUsers().subscribe(users => expect(users.length).toBe(2));

httpMock.expectOne('/api/users').flush([{}], {}]); // ★ assert AND
respond
afterEach(() => httpMock.verify()); // ★ no unexpected
calls
```

| | |
|----------------|--|
| expectOne(url) | → asserts EXACTLY ONE request was made to that URL ★ |
| flush(data) | → supplies the fake response |
| verify() | → fails the test if any request went unanswered ★ |

Easy memory

```
Jasmine (framework) + Karma (runner) + TestBed (module) ★  
fixture.detectChanges() or the template never renders ▲  
HTTP: HttpClientTestingModule + HttpTestingController  
expectOne(url).flush(data) + verify() ★
```

Migration & Versioning

Q: How do you upgrade Angular versions?

ONE major version at a time. Never jump. ★

```
ng update @angular/core @angular/cli # runs SCHEMATICS that rewrite  
your code ★
```

Your process, as a sequence:

1. Check `update.angular.io` for the breaking changes between the two versions
2. `ng update ONE` major at a time → build → run the test suite after each step ★
3. Update third-party libraries to compatible versions
(Ionic, PrimeNG, Material, Formly, NgRx – often the real blocker ▲)
4. Fix failing tests and deprecation warnings
5. THEN adopt new features incrementally
(standalone, functional guards/interceptors, `inject()`, `takeUntilDestroyed`)

★ **The sentence that matters: "migrate first, modernise second."** Get it green on the new version *before* changing any code style.

▲ **The real blocker is almost never Angular itself** — it's a third-party library that hasn't released a compatible version yet. Saying that shows you've actually done one.

Q: What changed across recent Angular versions?

14 → standalone (preview), TYPED reactive forms ★ , inject(), functional guards
15 → standalone stabilised, functional HttpClient providers, NgOptimizedImage
16 → SIGNALS (preview) ★ , takeUntilDestroyed ★ , required inputs, esbuild dev server
17 → new control flow @if/@for ★ , @defer ★ , Vite + esbuild by default
18 → zoneless change detection (experimental) ★ , signal APIs maturing, Material 3
19 → standalone is the DEFAULT, incremental hydration

The through-line to state:

Angular is moving AWAY from zone.js and NgModules, toward SIGNALS + STANDALONE. Everything from 14 onward is a step on that path. ★

● In your RoboGebra code

robogebra-mobile → Angular 18.2 (Ionic 8.7, Capacitor 6.2)
robogebra-web → Angular 16.2
robogebra-backoffice → Angular 16.2 (TypeScript 4.9)

■ *"The mobile app is on Angular 18 and the two web apps are on 16. We upgrade one major at a time and the real constraint is always the ecosystem — Ionic and PrimeNG dictate the pace more than Angular does."*

Easy memory

14 typed forms + inject() | 15 standalone stable | 16 SIGNALS + takeUntilDestroyed ★
17 @if/@for + @defer ★ | 18 zoneless (experimental) | 19 standalone by DEFAULT

Direction: away from zone.js and NgModules → toward SIGNALS + STANDALONE
★
"Migrate first, modernise second."
★ The blocker is usually a third-party lib.

Quick-Fire Bonus

Q: `declarations` vs `imports` vs `providers`?

```
declarations → MINE      (components/directives/pipes this module owns)
imports      → BORROWED  (other modules whose EXPORTS I use)
providers    → SERVICES  (registered with the injector)
exports      → SHARED    (what I let other modules use)
```

Q: `forRoot` vs `forChild`?

```
RouterModule.forRoot(routes) → ONCE, in the root module
                                → creates the Router SERVICE + the root
                                routes ★
RouterModule.forChild(routes) → in feature/lazy modules
                                → adds routes WITHOUT recreating the
                                router ★
```

▲ Calling `forRoot` twice creates a second Router service and breaks navigation.

Q: What is the `inject()` function?

```
private userService = inject(UserService);    // ★ no constructor
needed
```

Works in: field initialisers, functional guards, functional interceptors, factories

Must run in an INJECTION CONTEXT ▲ (not inside `ngOnInit` or a callback)

● 30 usages in `robogebra-mobile`.

Q: What are environment files for?

```
environment.ts    → dev    (API URL, feature flags)
environment.prod.ts → prod
```

The CLI swaps them via `fileReplacements` in `angular.json` at build time.

▲ **Never put a secret in an environment file** — it ships to the browser and anyone can read it.

Q: How does Ionic relate to Angular in your experience?

■ "Ionic is a UI and native-bridge layer on top of Angular — its components, routing and lifecycle integrate with Angular's, so services, reactive forms and RxJS are all reused. On RoboGebra the mobile app is Ionic 8 with Capacitor 6 on Angular 18, with about nineteen native plugins including push notifications via Firebase, native Razorpay checkout, network detection and native **Preferences** storage. Ionic adds its own lifecycle events on top of Angular's — **ionViewWillEnter** fires on every entry, whereas **ngOnInit** only runs once because Ionic keeps pages alive in the navigation stack."

★ That last sentence — **ionViewWillEnter** vs **ngOnInit** — is the Ionic + Angular question. → [15 — Ionic](#)

Quick Revision Sheet

CORE

Angular = TypeScript + COMPONENTS + one-way flow (a REWRITE of AngularJS)

NgModule: declarations MINE | imports BORROWED | exports SHARED | providers SERVICES

Standalone = no NgModule; your app: 2 usages ▲ "new code is standalone-first" ★

core/ (once) | features/ (lazy) | shared/ (reusable)

TEMPLATES

Lifecycle: constructor → ngOnChanges → ngOnInit ★ → ngDoCheck
→ AfterContentInit → AfterViewInit ★ → ngOnDestroy ★

@Inputs NOT ready in the constructor ★ | @ViewChild only in

AfterViewInit ★

Binding: {{ }} | [prop] | (event) | [(banana in a box)]

[(ngModel)] = [ngModel] + (ngModelChange); custom = @Input x + @Output
xChange ★

ng-template (not rendered until used) | ng-container (no extra DOM) ★

trackBy ★ = stable id → REUSE DOM nodes (55 usages in your app)

v17: @if / @for (track MANDATORY ★) / @defer

DI

providedIn:'root' ★ = singleton + tree-shakable

▲ a service provided in a LAZY module = a SECOND instance ▲

InjectionToken for non-class values (interfaces don't exist at runtime)

★

ROUTING

** LAST ★ | paramMap.subscribe, not snapshot, when reusing a component

★

Guards: CanActivate ★ / CanDeactivate / Resolve; return true|false|

UrlTree

Your guard: async Observable + setRedirectUrl + ROLE-based routing ★

Lazy: loadChildren / loadComponent + PreloadAllModules

FORMS

Reactive ★ = logic in the CLASS = testable without the DOM

FormControl | FormGroup | FormArray (dynamic) ★

Validator returns NULL when VALID ★ | cross-field goes on the GROUP ★

Formly = strategy/factory → a new field is CONFIG, not a release ★

RxJS

Promise = 1, eager, uncancelable | Observable = many, LAZY,
cancelable ★

Subject (nothing) | BehaviorSubject (CURRENT ★) | ReplaySubject (last
n)

switchMap CANCEL ★ | mergeMap parallel | concatMap ORDER ★ |

exhaustMap IGNORE ★
 async pipe ★ ★ > takeUntilDestroyed > takeUntil(destroy\$) ★ (LAST operator!)
 catchError INSIDE the inner observable ★ or one error kills the stream
▲
 finalize = always runs (hide the spinner)

STATE

BehaviorSubject services ★ (your app: 122; NgRx: 0 – a deliberate choice) ★
NgRx: Action → Reducer (PURE) → Store → Selector (MEMOISED) + Effects

HTTP

Lazy – no subscribe, NO request ▲
Interceptor: req.clone() (IMMUTABLE ★) | FormData Content-Type trap ★
★
 skip /auth | takeUntil mass-cancel | 401 → refresh
status 0 = network/CORS ★

PERFORMANCE

OnPush ★ = @Input REFERENCE change | internal event | async pipe | markForCheck
▲ MUTATION doesn't change the reference → the UI silently won't update
▲
 zone.js patches async APIs → signals → zoneless (your app: 0 signals – be honest ★)
 AOT = build-time, errors caught at build, no eval
▲ NEVER call a method in a template ★

TESTING

Jasmine + Karma + TestBed | fixture.detectChanges() ▲
HttpClientTestingModule + expectOne().flush() + verify() ★

MIGRATION

ng update ONE major at a time ★ | "migrate first, modernise second" ★
The blocker is usually a third-party library, not Angular.

Related files: [03 – TypeScript](#) · [20 – RxJS Operators](#) · [21 – NgRx](#) · [25 – Angular Binding & Forms](#) · [15 – Ionic](#) · [39 – RoboGebra Code Examples](#) · [38 – IQVIA Prep](#)

Core Java — Interview Q&A (Easy Version)

How to use this file: every question follows the same shape —

Easiest way to remember → **Simple explanation** → **Real-world example** → **Diagram** → **Interview trap** → **Interview-ready answer** → **Easy memory box.**

Read the "Easy memory" box the night before the interview. Read the full answer during study time.

● About the "In your RoboGebra project" blocks

Throughout this file, after an interview answer you'll see a ● **In your RoboGebra project** block containing **real code from your own repositories** (`/Users/safi/workspace/robogebra-workspace`).

Use them like this ★ 1. Give the DEFINITION (the interview answer above) 2. Then "in our codebase..." ← the ● block ★ 3. Then ONE trade-off or trap

Three sentences. That is the difference between someone who read the docs and someone who has shipped it. Full index: [39 — RoboGebra Code Examples](#).

Contents

1. [OOP Concepts](#)
2. [Core Java](#)
3. [Collections](#)
4. [Exceptions](#)
5. [Java 8+ Features](#)
6. [Multithreading](#)
7. [Memory & Garbage Collection](#)

OOP Concepts

Q: What are the four pillars of OOP? Give a short example of each.

The easiest way to remember them is the word **A PIE**:

```
A → Abstraction
P → Polymorphism
I → Inheritance
E → Encapsulation
```

Think of an **ATM machine**. That one machine explains all four pillars.

1. Encapsulation — "Hide the data, give a safe door"

Encapsulation means keeping the data **private** and giving controlled access through methods.

```
class Account {
    private double balance;           // hidden – nobody can touch directly

    public double getBalance() {      // safe door to READ
        return balance;
    }

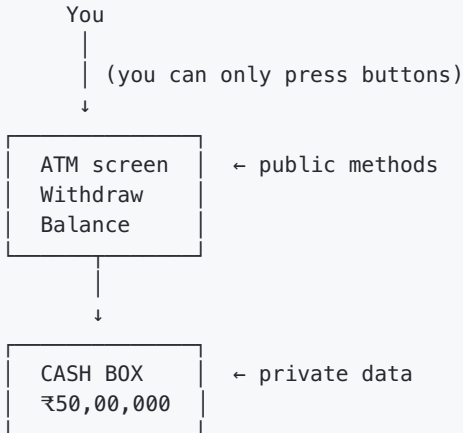
    public void deposit(double amount) { // safe door to WRITE
        if (amount > 0) {             // rule protects the data
            balance = balance + amount;
        }
    }
}
```

Why is this good?

```
Account a = new Account();

a.balance = -5000;    // ✗ Not allowed – field is private
a.deposit(-5000);    // ✓ Allowed to call, but our rule rejects it
```

Real-world idea: **ATM machine**



You can never open the cash box directly. You must use the buttons.

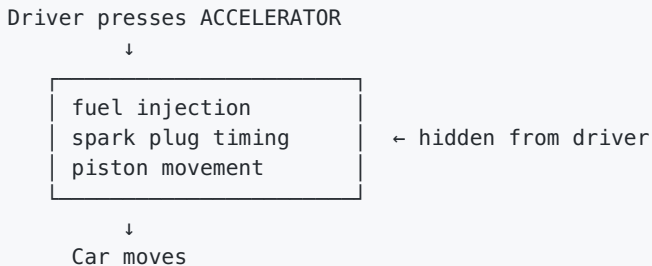
Encapsulation = private data + public methods

2. Abstraction — "Show what it does, hide how it does it"

```
interface Payment {  
    void pay(double amount);  
}
```

The caller only knows `pay()`. They don't know if it is UPI, card, or net banking.

Real-world idea: **Driving a car**



The driver only needs the pedal. They don't need engine knowledge.

Abstraction = hide complexity, show only the necessary part

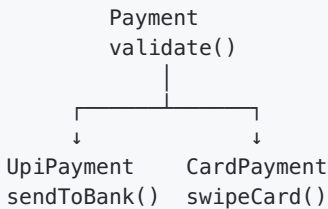
3. Inheritance — "Child gets parent's property"

```
class Payment {  
    void validate() {  
        System.out.println("Common validation");  
    }  
}  
  
class UpiPayment extends Payment {  
    void sendToBank() {  
        System.out.println("UPI request sent");  
    }  
}
```

Now:

```
UPIPayment u = new UPIPayment();  
  
u.validate();    // came free from parent  
u.sendToBank(); // its own method
```

Diagram:



Real-world idea: You inherit your family surname. You didn't create it, you received it.

Inheritance = reuse the parent's code

4. Polymorphism — "One name, many behaviours"

```
Payment p;  
  
p = new UpiPayment();  
p.pay(500);           // UPI logic runs  
  
p = new CardPayment();  
p.pay(500);           // Card logic runs
```

Same line `p.pay(500)` gives different behaviour.

Real-world idea: The word **"run"**

```
"run" → a person runs    (legs)  
"run" → a program runs  (CPU)  
"run" → a nose runs     (cold )
```

Same word, different meaning based on context.

Polymorphism = same method call, different behaviour

All four together

```
// ABSTRACTION – contract only
interface Payment {
    void pay(double amount);
}

// ENCAPSULATION – private data + safe access
class Wallet {
    private double balance;
    public double getBalance() { return balance; }
}

// INHERITANCE + POLYMORPHISM
class UpiPayment implements Payment {
    public void pay(double amount) { System.out.println("UPI paid " +
amount); }
}

class CardPayment implements Payment {
    public void pay(double amount) { System.out.println("Card paid " +
amount); }
}

Payment p = new UpiPayment();    // polymorphic reference
p.pay(500);
```

Interview-ready answer

The four pillars of OOP are Encapsulation, Abstraction, Inheritance and Polymorphism. Encapsulation means hiding the internal data using private fields and exposing controlled access through getters and setters — like an ATM where you can only use the buttons, not the cash box. Abstraction means showing only the required behaviour and hiding the implementation, which we achieve using interfaces and abstract classes. Inheritance lets a child class reuse the parent class's code. Polymorphism means the same method call behaves differently depending on the actual object — for example a `Payment` reference can point to `UPIPayment` or `CardPayment` and `pay()` behaves accordingly.

● In your RoboGebra project — all four pillars in ONE example ★

File: `common/service/event/EventTrackingService.java` + its two implementations

```
// ABSTRACTION – the contract. Callers see only WHAT, never HOW ★
public interface EventTrackingService {
    void trackUserLogin(User user);
    void trackPaymentEvent(SubscriptionInstanceWithItems subscription);
    void trackSolutionEvent(String userId, ExerciseSolutionDTO dto);
}

// POLYMORPHISM – two implementations, chosen by ENVIRONMENT ★
@Service
@Profile({"local", "dev"})
@ConditionalOnProperty(prefix = "posthog", name = "enabled",
    havingValue = "false", matchIfMissing = true)
public class LoggingEventTrackingService implements EventTrackingService
{ ... }

@Service
@Profile({"prod"})
@ConditionalOnProperty(prefix = "posthog", name = "enabled", havingValue =
    "true")
public class PostHogEventTrackingService implements EventTrackingService
{ ... }
```

```
// ENCAPSULATION – private state, controlled access
@Document(collection = "exercise_solution")
public class ExerciseSolutionEntity {
    @Id private String id; // private ★
    @Field("step_output_json") private Object stepOutputJson;
    // Lombok @Getter/@Setter generate the controlled accessors
}

// INHERITANCE – a shared base with three subclasses
public abstract class AbstractServiceAuthenticationFilter extends
    OncePerRequestFilter { ... }
public class CrmServiceAuthenticationFilter extends
    AbstractServiceAuthenticationFilter { ... }
public class WebhookAuthorizationFilter extends
    AbstractServiceAuthenticationFilter { ... }
```

■ **Say it like this:** *"Our analytics layer is the clearest example.*

***EventTrackingService** is the abstraction — services depend on the interface and never know where events go. There are two implementations chosen by profile: in dev it just logs, in production it posts to PostHog. That's polymorphism*

doing real work — the calling code is identical in both environments, and nobody needs a PostHog key on their laptop."

★ Note it's `@Profile` **plus** `@ConditionalOnProperty` — so it's environment-driven *and* individually switchable. That's a nice detail to add.

Easy memory

A PIE

| | | |
|---------------|-------------------|----------------|
| Abstraction | → hide HOW | (car pedal) |
| Polymorphism | → many FORMS | (word "run") |
| Inheritance | → get from PARENT | (surname) |
| Encapsulation | → hide DATA | (ATM cash box) |

Q: What is the difference between abstraction and encapsulation?

The easiest way to remember:

| | | |
|---------------|--------------------|----------------|
| Abstraction | → hides COMPLEXITY | (design level) |
| Encapsulation | → hides DATA | (code level) |

People confuse these two constantly, so use one single example — a **mobile phone**.

Abstraction

You tap the "Call" button. You don't know about towers, signals, or network protocols.

You tap "Call"



tower handshake

signal encoding

network routing

← hidden complexity



Call connects

In code:

```
interface Phone {  
    void call(String number);    // WHAT it does  
}
```

Nobody sees HOW.

Encapsulation

Inside the phone, your contacts are stored privately. Apps can't directly open that storage — they must ask permission.

```
class Phone {  
  
    private String pin = "1234";    // DATA is hidden  
  
    public boolean unlock(String input) {  
        return pin.equals(input);  
    }  
}
```

Nobody can read the `pin` directly.

Side-by-side

| | Abstraction | Encapsulation |
|---------------------|---|--|
| Hides | Complexity / implementation | Data / state |
| Level | Design level | Implementation level |
| Achieved by | interface, abstract class | private fields + getters/setters |
| Question it answers | WHAT does it do? | HOW is data protected? |
| Example | <code>interface Payment
{ pay(); }</code> | <code>private double
balance;</code> |

Interview-ready answer

Abstraction is about hiding complexity and exposing only the essential behaviour — it is solved at the design level using interfaces and abstract classes.
Encapsulation is about hiding the data itself by making fields private and giving

controlled access through getters and setters — it is solved at the implementation level. In short, abstraction hides complexity, encapsulation hides data.

● In your RoboGebra project

ABSTRACTION → `EventTrackingService` — a service calls `trackUserLogin(user)` and has NO IDEA whether that logs to a file or posts to

`PostHog` ★

ENCAPSULATION → `ExerciseSolutionEntity` — every field is private; the cached ``hash``, the ids and the AI payload are only reachable

through

accessors, so nothing outside can put the object in a bad

state ★

■ *"In our codebase the analytics interface is abstraction — it hides the entire implementation behind three method names. The entities are encapsulation — private fields with controlled accessors. One hides complexity, the other hides data."*

Easy memory

Abstraction → "WHAT it does" → interface

Encapsulation → "Data is safe" → private + getter/setter

Q: Abstract class vs interface — when do you use which?

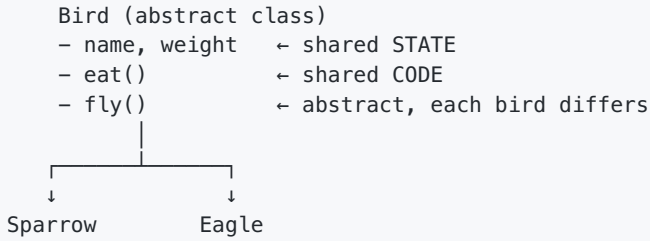
The easiest way to remember:

Abstract class → "IS-A" relationship → common CODE + common STATE

Interface → "CAN-DO" ability → only a CONTRACT

Simple story

Think about **birds**.



Every bird IS-A bird, and every bird eats the same way — so **Bird** is an **abstract class**.

Now, a bird **can swim**, and a fish **can swim** too — but a fish is not a bird. Swimming is an **ability**, not a family.

```
interface Swimmable → Duck, Fish, Human (unrelated classes)
```

That is an **interface**.

Code

```
abstract class Bird {  
  
    String name;                                // ✓ state allowed  
  
    Bird(String name) {                          // ✓ constructor allowed  
        this.name = name;  
    }  
  
    void eat() {                                  // ✓ shared code  
        System.out.println(name + " is eating");  
    }  
  
    abstract void fly();                        // each child decides  
}  
  
interface Swimmable {                            // pure ability  
    void swim();  
}  
  
class Duck extends Bird implements Swimmable {  
  
    Duck() { super("Duck"); }  
  
    void fly() { System.out.println("Duck flies short distance"); }  
    public void swim() { System.out.println("Duck swims"); }  
}
```

Note: **one** `extends`, but **many** `implements`.

```
Duck  
├── extends Bird          (only ONE parent allowed)  
└── implements Swimmable, Quackable, Sellable (MANY allowed)
```

Full comparison

| Aspect | Abstract class | Interface |
|----------------------|----------------|---|
| Multiple inheritance | ✗ only one | ✓ many |
| Instance fields | ✓ allowed | ✗ only <code>public static final</code> constants |
| Constructor | ✓ yes | ✗ no |

| Aspect | Abstract class | Interface |
|------------------|--|---|
| Method types | concrete + abstract | abstract + <code>default</code> + <code>static</code> (Java 8), <code>private</code> (Java 9) |
| Access modifiers | any (<code>private</code> , <code>protected</code> ...) | methods implicitly <code>public</code> |
| Keyword to use | <code>extends</code> | <code>implements</code> |
| Best for | shared code + shared state | contract / capability |

Interview-ready answer

Use an abstract class when the classes share common state and common code and there is a clear "is-a" relationship — for example `Bird` with a shared `eat()` method and a `name` field. Use an interface when you only want to define a contract or a capability that unrelated classes can implement, such as `Swimmable` or `Comparable`. A class can extend only one abstract class but can implement many interfaces, so interfaces also solve the multiple-inheritance need. Since Java 8, interfaces can have default and static methods, but they still cannot hold instance state.

- In your RoboGebra project — you use BOTH, for exactly the right reasons
★★

Abstract class: `security/AbstractServiceAuthenticationFilter.java`

```

public abstract class AbstractServiceAuthenticationFilter extends
OncePerRequestFilter {

    protected final RequestMappingHandlerMapping handlerMapping;    // ★
    SHARED STATE
    protected final ExceptionResponseWriter responseWriter;        //
    (impossible in an interface)
    protected final UserDetailsService userDetailsService;

    protected AbstractServiceAuthenticationFilter(                  // ★
    CONSTRUCTOR
        RequestMappingHandlerMapping handlerMapping,
        ExceptionResponseWriter responseWriter,
        UserDetailsService userDetailsService) { ... }

    // ===== subclasses MUST implement =====
    protected abstract Class<? extends Annotation> getAnnotationClass();
    protected abstract String getHeaderName();
    protected abstract String getExpectedToken();
    protected abstract String getFilterName();
    protected abstract UserRole getUserRole();

    // ===== optional HOOKS with a default ★ =====
    protected String extractToken(HttpServletRequest request) {
        return request.getHeader(getHeaderName());                // sensible
    default
    }
    protected void additionalValidation(HttpServletRequest request) {
        // default: nothing
    }
}

```

Subclass:

```

@Component
@Order(3)
public class CrmServiceAuthenticationFilter extends
AbstractServiceAuthenticationFilter {

    private final CrmApiProperties crmApiProperties;

    public CrmServiceAuthenticationFilter(..., CrmApiProperties
crmApiProperties) {
        super(handlerMapping, responseWriter, userDetailsService); //
★ super(...)
        this.crmApiProperties = crmApiProperties;
    }

    @Override protected Class<? extends Annotation> getAnnotationClass()
{ return CrmServiceAPI.class; }
    @Override protected String getHeaderName() { return
ApplicationConstants.CRM_SERVICE_TOKEN; }
    @Override protected String getExpectedToken() { return
crmApiProperties.getCrmServiceToken(); }
    @Override protected String getFilterName() { return "CRM-
Service"; }
}

```

Interface: `EventTrackingService` — a pure contract, two implementations, no shared state.

■ **The answer that shows judgement:** "We use both, and the reason is exactly the textbook one. The service-authentication filters share state and behaviour — the handler mapping, the response writer, and the whole 'read the header, compare the token, build the security context' algorithm. That's an abstract class, and it's a template method: `doFilterInternal` is written once, and each subclass supplies five small pieces — which annotation to look for, which header, which expected token. CRM auth and webhook auth are two subclasses of about thirty lines each. The analytics tracker, by contrast, shares nothing — the logging version and the PostHog version have no common code — so that's an interface."

★ **The rule, from your own code:** "If I catch myself wanting a field or a constructor, that's an abstract class. If it's just a contract, it's an interface."

★ And note the **optional hooks** — `extractToken()` and `additionalValidation()` have defaults that subclasses *may* override. That's another thing an interface couldn't give you before Java 8 default methods.

Easy memory

```
Abstract class → IS-A → Duck IS-A Bird      → extends (only 1)
Interface      → CAN-DO → Duck CAN swim     → implements (many)

Need shared FIELDS or a CONSTRUCTOR? → abstract class
Need MULTIPLE types / pure contract? → interface
```

Q: What are default and static methods in interfaces?

The easiest way to remember:

```
default method → interface gives a READY-MADE body that children inherit
static method  → utility that belongs to the INTERFACE itself
```

Why were they added? (This is the real interview point)

Imagine you wrote an interface used by 500 classes in production:

```
interface Payment {
    void pay(double amount);
}
```

Now you want to add `refund()`.

```
interface Payment {
    void pay(double amount);
    void refund(double amount);    // x 500 classes break instantly
}
```

Every implementing class fails to compile. This is exactly what happened to Java 8 — they wanted to add `stream()` to the `Collection` interface without breaking the whole world.

Solution: **default method**.

```
interface Payment {

    void pay(double amount);

    default void refund(double amount) {           // ✓ nothing breaks
        System.out.println("Refund not supported");
    }
}
```

Before Java 8

add method



all children break ✗

After Java 8

add DEFAULT method



children inherit body ✓

default method — example

```
interface Payment {

    void pay(double amount);

    default void receipt() {
        System.out.println("Payment receipt generated");
    }
}

class UpiPayment implements Payment {
    public void pay(double amount) {
        System.out.println("UPI paid " + amount);
    }
    // receipt() comes free
}
```

```
UPIPayment u = new UPIPayment();
u.pay(500);
u.receipt();           // Payment receipt generated
```

A child can also **override** it if it wants its own behaviour.

static method — example

```
interface Payment {  
  
    void pay(double amount);  
  
    static boolean isValid(double amount) {        // helper utility  
        return amount > 0;  
    }  
}
```

Called on the **interface name**, not on the object:

```
Payment.isValid(500);    // ✓  
u.isValid(500);         // ✗ compile error
```

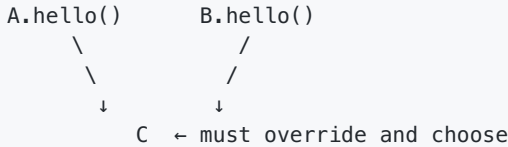
```
default method → belongs to the OBJECT → obj.method()  
static method → belongs to the INTERFACE → Interface.method()
```

The diamond problem trap

```
interface A {  
    default void hello() { System.out.println("A"); }  
}  
  
interface B {  
    default void hello() { System.out.println("B"); }  
}  
  
class C implements A, B {        // ✗ which hello() should C use?  
}
```

Java cannot decide, so it forces **you** to decide:

```
class C implements A, B {  
  
    public void hello() {  
        A.super.hello();        // explicitly pick one  
    }  
}
```



Interview-ready answer

Default and static methods were introduced in Java 8 so that interfaces could evolve without breaking existing implementations. A default method has a body inside the interface, so all implementing classes automatically inherit it and can optionally override it — this is how `stream()` was added to the `Collection` interface without breaking existing code. A static method is a utility that belongs to the interface itself and is called using the interface name. If a class implements two interfaces that have the same default method, it causes an ambiguity, and the class must override the method and choose one explicitly using `InterfaceName.super.method()`.

Easy memory

default → gives a FREE body → `obj.method()` → can override
static → gives a HELPER utility → `Interface.method()` → cannot override

Reason they exist: add new methods WITHOUT breaking old classes.

Q: Overloading vs overriding?

The easiest way to remember:

Overloading = Same method name, DIFFERENT parameters
Overriding = Child class CHANGES the parent's implementation

1. Method Overloading

Suppose you have a payment application where customers can pay in different ways.

```
class Payment {  
  
    void pay(int amount) {  
        System.out.println("Pay amount");  
    }  
  
    void pay(int amount, String currency) {  
        System.out.println("Pay with currency");  
    }  
  
    void pay(int amount, String currency, String coupon) {  
        System.out.println("Pay with coupon");  
    }  
}
```

All methods have the **same name**, but **different parameters**:

```
pay(int)  
pay(int, String)  
pay(int, String, String)
```

You can call:

```
Payment p = new Payment();  
  
p.pay(1000);  
p.pay(1000, "INR");  
p.pay(1000, "INR", "SAVE10");
```

Real-world idea — a **calculator**:

```
add(10, 20)  
add(10, 20, 30)  
add(10.5, 20.5)
```

Same operation, different inputs.

Overloading = same name + different parameters

2. Method Overriding

```
class Payment {  
    void pay() {  
        System.out.println("Processing payment");  
    }  
}  
  
class UpiPayment extends Payment {  
    @Override  
    void pay() {  
        System.out.println("Processing UPI payment");  
    }  
}  
  
class CardPayment extends Payment {  
    @Override  
    void pay() {  
        System.out.println("Processing Card payment");  
    }  
}
```

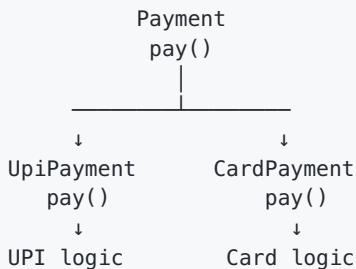
Now:

```
Payment p = new UpiPayment();  
p.pay();
```

Output:

```
Processing UPI payment
```

The child class **replaced** the parent's implementation.



Overriding = same method + child provides different behaviour

Main difference

| Overloading | Overriding |
|-----------------------------------|---------------------------------------|
| Same method name | Same method signature |
| Different parameters | Same parameters |
| Usually within the same class | Parent–child relationship |
| Inheritance not required | Inheritance required |
| Compile-time polymorphism | Runtime polymorphism |
| Return type alone cannot overload | Return type must be same or covariant |
| @Override not used | @Override recommended |

Very common interview trap

Can we overload only by changing the return type?

No. ✗

```
int calculate() {
    return 10;
}

double calculate() {           // ✗ compile error
    return 10.5;
}
```

Because both have `calculate()` with exactly the same parameter list. Java decides which method to call by looking at the **arguments you pass**, not the return type — so it cannot tell them apart.

Overriding rules to remember

- Same signature ✓ must
- Return type same or covariant ✓ (child return type allowed)
- Access modifier NOT more restrictive
 - public → public ✓
 - public → protected ✗
- Cannot throw BROADER checked exceptions
- private / static / final methods ✗ cannot be overridden

Interview-ready answer

Method overloading means having multiple methods with the same name but different parameter lists. It is resolved at compile time, so it is compile-time polymorphism — for example `pay(amount)` and `pay(amount, currency)`. Method overriding happens when a child class provides its own implementation of a method already defined in the parent class with the same signature. It is resolved at runtime based on the actual object, so it is runtime polymorphism — for example `UpiPayment` and `CardPayment` both overriding `pay()` from `Payment`. We cannot overload a method by changing only the return type, because Java resolves overloads using the parameter list.

● In your RoboGebra project — overriding, and why `@Override` matters ★

File: `security/CrmServiceAuthenticationFilter.java`

```
public class CrmServiceAuthenticationFilter extends
AbstractServiceAuthenticationFilter {

    @Override protected Class<? extends Annotation> getAnnotationClass()
{ return CrmServiceAPI.class; }
    @Override protected String getHeaderName()    { return
ApplicationConstants.CRM_SERVICE_TOKEN; }
    @Override protected String getExpectedToken() { return
crmApiProperties.getCrmServiceToken(); }
    @Override protected String getFilterName()    { return "CRM-
Service"; }
}
```

The PARENT calls `getHeaderName()` — but at RUNTIME it resolves to the CRM subclass's version, or the Webhook subclass's version. ★

That is RUNTIME polymorphism doing the actual work:
one algorithm in the base class, five decisions per subclass ★

■ "Every one of those is an override, and `@Override` is doing real work — if someone renames `getHeaderName` in the abstract class, the subclass fails to compile instead of silently becoming a new unused method."

▲ Note these are `protected`, not `public` — an override **cannot reduce visibility**, so a subclass could widen them to `public`, but never narrow them to `private`.

Easy memory

```
OverLOADing → LOAD different parameters → same class    → compile time
OverRIDing  → RIDE over the parent       → child class   → runtime
```

Q: What do `super` and `this` mean?

The easiest way to remember:

```
this → THIS object   (current class)
super → SUPERior      (parent class)
```

`this` — points to the current object

The most common use is when the parameter name and the field name are the same:

```
class Employee {
    private String name;

    Employee(String name) {
        this.name = name;    // this.name = field, name = parameter
    }
}
```

Without `this`:

```
Employee(String name) {
    name = name;    // ✗ assigning parameter to itself – field stays
    null
}
```

```
Constructor parameter: name = "Priya"
Object field:          this.name = null

this.name = name;
  ↓         ↓
  field    parameter
```

Other uses of **this**:

```
class Employee {

    private String name;
    private int age;

    Employee() {
        this("Unknown", 0);    // 1 call another constructor
    }

    Employee(String name, int age) {
        this.name = name;      // 2 refer to a field
        this.age = age;
    }

    Employee getSelf() {
        return this;           // 3 return the current object
    }
}
```

super — points to the parent object

```
class Payment {

    void pay() {
        System.out.println("Common validation done");
    }
}

class UpiPayment extends Payment {

    @Override
    void pay() {
        super.pay();            // run parent logic first
        System.out.println("UPI payment done");
    }
}
```


Output:

```
Common validation done
UPI payment done
```

Diagram:

```
UpiPayment.pay()
    |
    | super.pay()
    ↓
Payment.pay() ← parent code runs
    |
    ↓ back
UPI specific code runs
```

Real-world idea: You are cooking your own recipe, but first you follow your mother's base gravy — `super.makeGravy()` — then add your own twist.

`super()` in constructors

```
class Payment {
    Payment() {
        System.out.println("Payment constructor");
    }
}

class UpiPayment extends Payment {
    UpiPayment() {
        super(); // implicit – Java adds it for you
        System.out.println("UPI constructor");
    }
}
```

Output:

```
Payment constructor
UPI constructor
```

Parent is **always** built first — you cannot build the first floor before the ground floor.

```
new UpiPayment()  
  ↓  
Payment() ← parent constructor runs FIRST  
  ↓  
UpiPayment()
```

Interview trap

```
class Child extends Parent {  
    Child() {  
        System.out.println("hi");  
        super();           // x compile error  
    }  
}
```

`this()` and `super()` **must be the very first statement** in a constructor, and you cannot use both in the same constructor.

Interview-ready answer

`this` refers to the current object. We use it to distinguish a field from a parameter with the same name, to call another constructor of the same class using `this(...)`, and to return the current instance. `super` refers to the immediate parent class. We use `super.method()` to call the parent's version of an overridden method, `super.field` to access a parent field, and `super(...)` to call the parent constructor. Both `this(...)` and `super(...)` must be the first statement in a constructor, and the parent constructor always runs before the child constructor.

● In your RoboGebra project — `super(...)` in a real constructor chain ★

File: `security/CrmServiceAuthenticationFilter.java`

```

public CrmServiceAuthenticationFilter(
    RequestMappingHandlerMapping handlerMapping,
    ExceptionResponseWriter responseWriter,
    UserDetailsService userDetailsService,
    CrmApiProperties crmApiProperties) {

    super(handlerMapping, responseWriter, userDetailsService); // ★
    FIRST statement
    this.crmApiProperties = crmApiProperties; // ★
    then my own field
}

```

```

new CrmServiceAuthenticationFilter(...)
    ↓
super(...) → AbstractServiceAuthenticationFilter's constructor runs FIRST
★
    ↓      (sets handlerMapping, responseWriter, userDetailsService)
    ↓
back here → this.crmApiProperties = ...

```

★ This is also the **Spring dependency-injection** pattern: the subclass takes *everything* the parent needs **plus** its own dependency, and passes the parent's up through `super(...)`.

■ *"The base filter needs three collaborators and each subclass needs one of its own, so the subclass constructor takes all four and passes three up with `super(...)` — which has to be the first statement, because the parent must be fully built before the child touches anything."*

Easy memory

| | |
|---|--------------------------|
| this → current object → this.name = name; | → this(...) same class |
| ctor | |
| super → parent object → super.pay(); | → super(...) parent ctor |

Both must be the FIRST line in a constructor.
 Parent constructor ALWAYS runs first.

Q: Explain constructors and constructor chaining.

The easiest way to remember:

```
Constructor = the method that runs automatically when you say `new`  
Its job      = set the object's starting values
```

Rules of a constructor

1. Name must be EXACTLY the class name
2. NO return type (not even void)
3. Runs automatically on `new`
4. If you write none, Java gives a free default constructor

```
class Employee {  
    String name;  
  
    Employee() {                                // constructor  
        System.out.println("Object created");  
    }  
}  
  
Employee e = new Employee();                    // prints: Object created
```

If it had a return type it would just be a normal method:

```
void Employee() { }    // ▲ this is a METHOD named Employee, not a  
constructor
```

Types of constructors

```
class Employee {  
  
    String name;  
    int age;  
  
    Employee() {                                // 1 no-arg / default  
        name = "Unknown";  
        age = 0;  
    }  
  
    Employee(String name, int age) {           // 2 parameterised  
        this.name = name;  
        this.age = age;  
    }  
  
    Employee(Employee other) {                 // 3 copy constructor  
        this.name = other.name;  
        this.age = other.age;  
    }  
}
```

The default constructor trap ★

```
class Employee {  
    Employee(String name) { }                // you wrote a parameterised one  
}  
  
Employee e = new Employee();                 // ✗ compile error
```

Java gives a free no-arg constructor **only if you write no constructor at all**. The moment you write one, the free gift is withdrawn.

No constructor written → Java adds `Employee()` for free ✓
Any constructor written → Java adds NOTHING ✗

This is exactly why Hibernate/JPA entities need an explicit no-arg constructor.

Constructor chaining

Constructor chaining means **one constructor calls another**, so you write the setup logic only once.

Two kinds:

```
this(...) → chain WITHIN the same class  
super(...) → chain to the PARENT class
```

Chaining with `this()` :

```
class Employee {  
  
    String name;  
    int age;  
    String dept;  
  
    Employee() {  
        this("Unknown", 0, "General");    // calls the 3-arg one  
    }  
  
    Employee(String name) {  
        this(name, 0, "General");    // calls the 3-arg one  
    }  
  
    Employee(String name, int age, String dept) {    // the REAL work  
happens here  
        this.name = name;  
        this.age = age;  
        this.dept = dept;  
    }  
}
```

Flow:

```
new Employee()  
  ↓  
this("Unknown", 0, "General")  
  ↓  
Employee(String, int, String) ← actual assignment happens once
```

Benefit: if tomorrow you add validation, you add it in **one** place, not three.

Chaining with `super()` :

```

class Person {
    Person() { System.out.println("1. Person"); }
}

class Employee extends Person {
    Employee() { System.out.println("2. Employee"); }
}

class Manager extends Employee {
    Manager() { System.out.println("3. Manager"); }
}

new Manager();

```

Output:

```

1. Person
2. Employee
3. Manager

```

```

new Manager()
  ↓
Manager() → calls super() implicitly
  ↓
Employee() → calls super() implicitly
  ↓
Person() → calls super() → Object()
  ↓
Now bodies run BOTTOM-UP: Person → Employee → Manager

```

Real-world idea: building a house — foundation → ground floor → first floor. You can never build the first floor first.

Interview trap

```

Employee() {
    System.out.println("hi");
    this("a", 1);      // x must be the FIRST statement
}

```

Also, you cannot call `this()` and `super()` in the same constructor.

Interview-ready answer

A constructor is a special block that has the same name as the class, has no return type, and runs automatically when an object is created using `new`. Its job is to initialise the object's state. If we do not write any constructor, Java provides a default no-argument constructor, but once we write any constructor ourselves, that default is no longer provided. Constructor chaining means one constructor calls another — `this(...)` chains to another constructor in the same class to avoid duplicate initialisation code, and `super(...)` chains to the parent constructor. Both must be the first statement in the constructor, and the parent constructor always executes before the child constructor.

● In your RoboGebra project — constructor overloading, done well ★

File: `common/exception/ResourceNotFoundException.java`

```
public class ResourceNotFoundException extends RuntimeException {  
  
    // 1 the CONVENIENT one – builds a consistent message ★  
    public ResourceNotFoundException(String entityType, Object id) {  
        super(MessageFormat.format("The {0} with {1} not found.",  
entityType, id));  
    }  
  
    // 2 the FLEXIBLE one – a free-form message  
    public ResourceNotFoundException(String message) {  
        super(message);  
    }  
}
```

```
throw new ResourceNotFoundException("Exercise", exerciseId);  
    // → "The Exercise with 64f3a1b2 not found." ★ consistent, every  
time
```

File: `common/exception/APIException.java` — carrying extra state on the exception:


```

@Getter
public class APIException extends RuntimeException {
    private final HttpStatus status;           // ★ extra business data
    private final String errorMessage;

    public APIException(HttpStatus status, String errorMessage) {
        super(errorMessage);                 // ★ always pass it to the
parent
        this.status = status;
        this.errorMessage = errorMessage;
    }
}

```

■ "Two constructors on the same exception — one takes an entity type and an id and formats a consistent message, which is what almost every call site uses; the other takes a free-form message for the odd case. *APIException* goes further and carries an *HttpStatus*, so the exception itself knows what status code it should become — the advice just reads it rather than mapping by type."

★ **12 custom exceptions** in the codebase: *ResourceNotFoundException*, *APIException*, *CognitoException*, *DatabaseException*, *ExternalApiException*, *SolutionExecutionException*, *QuizConflictException*, *EmailSendingException*, *RestrictedAPIAccessException*, *FirebaseMessageException*, *JsonException*, *FileNotFoundException* — all extending *RuntimeException*, i.e. unchecked, the Spring style.

Easy memory

```

Constructor = class name + no return type + runs on `new`
Free default = ONLY if you wrote zero constructors
this(...) = same class → avoid duplicate code
super(...) = parent class → parent built first
Must be the FIRST line.

```

Q: Static vs instance members?

The easiest way to remember:

static → belongs to the CLASS → ONE copy shared by everyone
instance → belongs to the OBJECT → EVERY object gets its own copy

Real-world example — a school

School name : "St. Mary's" ← same for ALL students → static
Student name: "Priya", "Ravi" ← different per student → instance

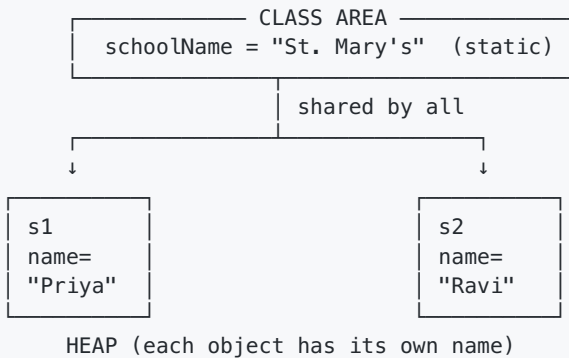
```
class Student {  
  
    static String schoolName = "St. Mary's";    // ONE copy for all  
    String name;                                // one copy PER object  
  
    Student(String name) {  
        this.name = name;  
    }  
}
```

```
Student s1 = new Student("Priya");  
Student s2 = new Student("Ravi");  
  
System.out.println(s1.name);           // Priya  
System.out.println(s2.name);           // Ravi  
System.out.println(Student.schoolName); // St. Mary's
```

Now change the school name once:

```
Student.schoolName = "St. Xavier's";  
  
System.out.println(s1.schoolName);    // St. Xavier's  
System.out.println(s2.schoolName);    // St. Xavier's ← both changed!
```

Diagram:



A practical counter example

```
class Counter {

    static int totalVisitors = 0;    // shared
    int myId;                       // personal

    Counter() {
        totalVisitors++;
        myId = totalVisitors;
    }
}

new Counter(); new Counter(); new Counter();

System.out.println(Counter.totalVisitors);    // 3
```

This is exactly how a website visitor counter works.

Static methods

```
class MathUtil {
    static int square(int x) {
        return x * x;
    }
}

MathUtil.square(5);    // no object needed ✓
```

That is why `main` is static — the JVM must call it **before** any object exists.

```
public static void main(String[] args)
```

The #1 interview trap ★

Can a static method access an instance variable?

No. ✗

```
class Test {  
  
    int a = 10;           // instance  
    static int b = 20;    // static  
  
    static void show() {  
        System.out.println(b); // ✓ fine  
        System.out.println(a); // ✗ compile error  
    }  
}
```

Why? Because a static method can run when **no object exists**, so there is no `a` to read.

static method runs → object may NOT exist → whose ``a`` should it print? → error

But the reverse works fine:

```
void display() {           // instance method  
    System.out.println(a); // ✓  
    System.out.println(b); // ✓ static is always available  
}
```

instance method → can access BOTH ✓
static method → can access ONLY static ✗

Static block

Runs **once**, when the class is first loaded — before any object or `main`.

```
class Config {  
  
    static String url;  
  
    static {  
        url = "jdbc:mysql://localhost/db";  
        System.out.println("Static block runs once");  
    }  
}
```

Order of execution:

```
Class loaded  
  ↓  
static block  
  ↓  
object created  
  ↓  
instance block  
  ↓  
constructor
```

Interview-ready answer

Static members belong to the class, so only one copy exists and it is shared by all objects — for example a school name shared by all student objects, or a visitor counter. Instance members belong to each object, so every object gets its own copy. Static members are accessed using the class name and are loaded when the class is loaded, which is why `main` is static — the JVM calls it before any object exists. A static method cannot directly access instance variables because the object may not exist yet, but an instance method can access both. A static block runs only once when the class is loaded and is generally used for one-time initialisation.

● In your RoboGebra project — `static final` constants and enums ★

```
// A static constant used as a compile-time qualifier ★
public class AsyncConfig {
    /** @Async needs a COMPILE-TIME CONSTANT, so this must be `static
    final`. */
    public static final String NOTIFICATION_TASK_EXECUTOR =
    "notificationTaskExecutor";
}

@Async(AsyncConfig.NOTIFICATION_TASK_EXECUTOR)    // ★ only a constant
is legal here
public void sendPush(...) { }
```

★ **That's a genuinely good `static final` example** — the annotation *requires* a compile-time constant, so it physically cannot be an instance field.

```
// enum = a fixed set of constants, with state and behaviour ★
@AllArgsConstructor
@Getter
public enum Feature {
    SOLUTION("solution"),
    AI_ASSISTANT("ai-assistant"),
    EXPLORE("explore");

    final String collectionName;    // ★ each constant carries data
}
```

```
// a static grouping constant, used by the interceptor for a membership
test ★
public static final ErrorCode[] MultiUserErrorCodes = {
    ErrorCode.MULTI_USER_LOGIN_EXCEEDED,
    ErrorCode.SESSION_FORCEFULLY_REMOVED
};
```

■ " `NOTIFICATION_TASK_EXECUTOR` has to be `static final` because `@Async` takes a compile-time constant — that's a case where the language forces the decision rather than style. And our enums carry data: `Feature` holds the collection name for each feature, so the mapping lives with the constant instead of in a switch somewhere."

Easy memory

```
static    → CLASS    → one copy    → School name    → ClassName.member  
instance → OBJECT → many copies → Student name → object.member
```

```
static method → can use ONLY static    ✗    instance  
instance method → can use BOTH        ✓
```

Q: Difference between final, finally, and finalize?

The easiest way to remember:

```
final    → a KEYWORD    → "cannot change"  
finally  → a BLOCK      → "always runs"  
finalize → a METHOD      → "before garbage collection" (deprecated)
```

They sound similar but have nothing to do with each other — that's why interviewers love this question.

1. **final** — cannot be changed

It works in three places:

```
final int MAX = 100;           // 1 variable → value cannot change  
MAX = 200;                     // ✗ compile error
```

```
class Payment {  
    final void validate() { }    // 2 method → cannot be overridden  
}  
  
class UpiPayment extends Payment {  
    void validate() { }          // ✗ compile error  
}
```

```
final class Constants { }       // 3 class → cannot be extended  
  
class MyConstants extends Constants { }    // ✗ compile error
```

Real-world examples in the JDK:

```
String → final class (that is why nobody can break String's behaviour)
Integer → final class
```

The famous final trap ★

```
final List<String> list = new ArrayList<>();

list.add("Priya"); // ✓ ALLOWED – we changed the CONTENT
list = new ArrayList<>(); // ✗ NOT allowed – we changed the REFERENCE
```

```
final list —————> [ ArrayList ]
      ↑               ↑
      |               |
reference LOCKED    contents FREE to change
```

Real-world idea: your house address is fixed (final), but you can still change the furniture inside.

2. **finally** — always runs

Used with try-catch. It runs whether an exception happens or not.

```
try {
    int result = 10 / 0; // exception!
} catch (ArithmeticException e) {
    System.out.println("Error caught");
} finally {
    System.out.println("Connection closed");
}
```

Output:

```
Error caught
Connection closed
```

Even with a **return** :


```
int test() {
    try {
        return 1;
    } finally {
        System.out.println("finally runs");    // still prints!
    }
}
```

```
try → runs
    ↓
catch → runs only IF exception
    ↓
finally → ALWAYS runs ✓
```

Real-world idea: You go to a restaurant. You may enjoy the food (try) or find a problem (catch), but you **must pay the bill** (finally) either way.

The only cases where `finally` doesn't run: `System.exit(0)`, JVM crash, or power failure.

3. `finalize()` — before garbage collection

An old method the GC called before destroying an object.

```
protected void finalize() throws Throwable {
    System.out.println("Object is being destroyed");
}
```

Why it is **deprecated since Java 9**:

- No guarantee it will ever run
- No guarantee WHEN it will run
- Slows down garbage collection
- Can accidentally resurrect the object

Modern replacement:

```
try (Connection con = getConnection()) {
    // use it
} // auto-closed here – reliable ✓
```

Comparison table

| | final | finally | finalize() |
|--------------|---------------------------|-----------------|-------------------------|
| Type | keyword | block | method |
| Used with | variable / method / class | try-catch | object cleanup |
| Purpose | prevent change | always execute | pre-GC cleanup |
| When it runs | compile time check | after try/catch | before GC (maybe never) |
| Status | actively used | actively used | deprecated (Java 9+) |

Interview-ready answer

final is a keyword. A final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be extended — **String** is a good example of a final class. **finally** is a block used with try-catch that always executes, whether an exception occurs or not, so it is used to release resources like connections. **finalize()** was a method called by the garbage collector before destroying an object, but it is deprecated since Java 9 because there is no guarantee it runs or when it runs. Today we use try-with-resources or **Cleaner** instead. A common trap is that a final reference to a collection can still have its contents modified — only the reference is locked, not the object's state.

Easy memory

| | | | |
|----------|-----------|-------------------|---------------------------|
| final | → KEYWORD | → "cannot change" | → final int MAX = 10; |
| finally | → BLOCK | → "always runs" | → pay the restaurant bill |
| finalize | → METHOD | → "before GC" | → deprecated, don't use |

final List → add() ✓ | list = new ✓ ✗ (not allowed)

Core Java

Q: JDK vs JRE vs JVM?

The easiest way to remember:

```
JDK → for DEVELOPERS  (write + compile + run)
JRE → for USERS       (only run)
JVM → the ENGINE       (actually executes)
```

They sit inside each other like boxes:

```
graph TD
    subgraph JDK [JDK (Java Development Kit)]
        javac
        javadoc
        jar
        debugger
        subgraph JRE [JRE (Java Runtime Environment)]
            library_classes["library classes: String, List..."]
            subgraph JVM [JVM (Java Virtual Machine)]
                class_loader
                memory_areas
                interpreter_JIT["interpreter + JIT"]
            end
        end
    end
```

JDK (Java Development Kit)
javac, javadoc, jar, debugger

JRE (Java Runtime Environment)
library classes: String, List...

JVM (Java Virtual Machine)
class loader
memory areas
interpreter + JIT

Real-world idea — a **restaurant**:

```
JVM → the CHEF          (actually cooks the food)
JRE → the KITCHEN       (chef + gas + utensils + ingredients)
JDK → the FULL SETUP    (kitchen + recipe books + knives to create new
dishes)
```

A customer only needs the kitchen to produce food. A chef who invents new recipes needs the full setup.

What each one does

JVM — runs the `.class` bytecode.

Loads the class → verifies bytecode → allocates memory → executes → garbage collects

JRE = JVM + the standard library (`java.lang` , `java.util` ...). If you only want to *run* a Java app, JRE is enough.

JDK = JRE + development tools:

| | |
|----------------------|-----------------|
| <code>javac</code> | → compiler |
| <code>java</code> | → launcher |
| <code>jar</code> | → packaging |
| <code>javadoc</code> | → documentation |
| <code>jdb</code> | → debugger |

Practical check

| | |
|-------------------------------|------------------------|
| <code>javac Hello.java</code> | # needs JDK (compiler) |
| <code>java Hello</code> | # needs JRE (runtime) |

If you only installed the JRE, `javac` gives "command not found" — a very common real-life mistake.

Interview-ready answer

The JVM is the engine that actually executes Java bytecode — it loads classes, verifies them, manages memory and performs garbage collection, and it is platform-specific. The JRE is the JVM plus the standard Java class libraries, and it is what you need to simply run a Java application. The JDK is the JRE plus development tools such as the `javac` compiler, `jar` and `javadoc` , and it is what a developer installs. So JDK contains JRE, and JRE contains JVM.

Easy memory

| | |
|----------------------------|----------------------|
| JDK = JRE + compiler tools | → developer |
| JRE = JVM + libraries | → user who only runs |
| JVM = the executing engine | → chef |

JDK ⊃ JRE ⊃ JVM

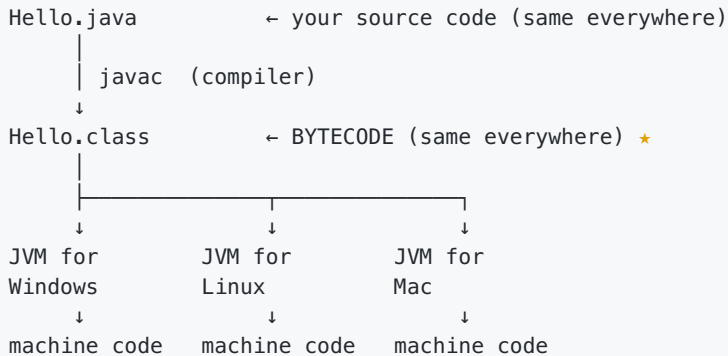
Q: How does Java achieve platform independence?

The easiest way to remember:

"Write Once, Run Anywhere"

Java code → BYTECODE (same everywhere) → JVM (different for each OS)

The full flow



The **bytecode is the same everywhere**. Only the JVM is different for each operating system.

Compare with C

C program
↓
compiled on Windows → Windows .exe → runs ONLY on Windows ✗

Java program
↓
compiled anywhere → .class bytecode → runs on ANY OS with a JVM ✓

Real-world idea: A **movie file** (**.mp4**) is the same everywhere. You just need the right **player** for your device — VLC on Windows, VLC on Mac. The file doesn't change; the player does.

The important nuance interviewers look for

```
Java LANGUAGE → platform INDEPENDENT ✓  
JVM           → platform DEPENDENT   ✓ (you download a different JVM per OS)
```

So Java is platform independent *because* the JVM is platform dependent.

Interview-ready answer

Java achieves platform independence through bytecode. The `javac` compiler does not produce machine code — it produces `.class` bytecode, which is identical on every platform. That bytecode is then executed by the JVM, and the JVM itself is platform-specific, so there is a separate JVM implementation for Windows, Linux and Mac. This is what "Write Once, Run Anywhere" means. So the Java program is platform independent precisely because the JVM is platform dependent.

Easy memory

```
.java → javac → .class BYTECODE (same everywhere) → JVM (different per OS)  
→ runs
```

```
Movie file = same      |      Player = different per device
```

Q: String vs StringBuilder vs StringBuffer?

The easiest way to remember:

```
String          → IMMUTABLE → cannot change  
StringBuilder → MUTABLE   → fast, NOT thread-safe ★ use this normally  
StringBuffer  → MUTABLE   → slow, thread-safe
```

The core problem with String

```
String s = "Hello";  
s = s + " World";
```

You think you changed `s`. You didn't — Java created a **brand new object**.

```
"Hello"      ← old object, now garbage
"Hello World" ← new object
  ↑
  s now points here
```

Now imagine a loop:

```
String s = "";
for (int i = 0; i < 10000; i++) {
    s = s + i;          // ✗ creates 10,000 objects!
}
```

```
Loop 1 → new object
Loop 2 → new object
Loop 3 → new object
...
Loop 10000 → new object

Memory: ▲    Speed:
```

With `StringBuilder`:

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) {
    sb.append(i);      // ✓ ONE object, modified in place
}
String result = sb.toString();
```

```
ONE object → appended 10,000 times

Memory:    Speed:
```

Real-world idea: **String** is like writing on stone — to fix a mistake you carve a whole new stone. **StringBuilder** is like writing on a whiteboard — you just keep adding.

StringBuilder vs StringBuffer

Both are mutable. The only difference is thread safety.

```
// StringBuffer – every method is synchronized
public synchronized StringBuffer append(String s) { ... }

// StringBuilder – no synchronization
public StringBuilder append(String s) { ... }
```

StringBuffer → LOCKS the object → safe with many threads → slower
 StringBuilder → no lock → single thread only → faster

Real-world idea:

StringBuffer → a bank locker: one person at a time, safe but slow
 StringBuilder → your own diary: only you use it, so no lock needed

99% of real code uses **StringBuilder** — string building usually happens inside one method, on one thread.

Comparison table

| | String | StringBuilder | StringBuffer |
|--------------|-----------------------------|-----------------------------|---------------------------------|
| Mutable? | ✗ No | ✓ Yes | ✓ Yes |
| Thread-safe? | ✓ (because immutable) | ✗ No | ✓ Yes |
| Speed | slow in loops | fastest | slower |
| Since | 1.0 | Java 5 | 1.0 |
| Stored in | String pool (literals) | heap | heap |
| Use when | fixed text, keys, constants | building text in one thread | building text shared by threads |

Interview trap

```
String a = "Hello";
String b = "Hello";
System.out.println(a == b);           // true → same pooled object

String c = new String("Hello");
System.out.println(a == c);           // false → new object in heap
System.out.println(a.equals(c));       // true → same content
```

Interview-ready answer

`String` is immutable, so every modification creates a new object — concatenating inside a loop creates thousands of objects and hurts performance. `StringBuilder` and `StringBuffer` are mutable, so they modify the same object using an internal character array. The difference between them is that `StringBuffer`'s methods are synchronized, which makes it thread-safe but slower, while `StringBuilder` is not synchronized and is therefore faster. In practice we use `StringBuilder` for string building inside a single thread, `StringBuffer` only when the same buffer is shared across threads, and `String` for fixed values like constants and map keys.

Easy memory

```
String          → stone carving → immutable → new object every change
StringBuilder → whiteboard      → mutable   → FAST   → single thread ★
StringBuffer   → bank locker    → mutable   → SAFE   → multi thread

Loop + concatenation? → ALWAYS StringBuilder
```

Q: Why are Strings immutable and what is the string pool?

The easiest way to remember:

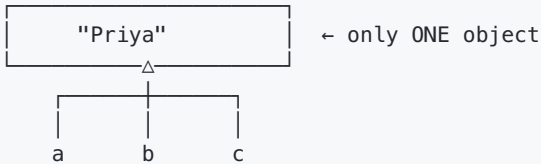
```
Immutable = once created, the value can NEVER change
String pool = a special area in the heap that REUSES string literals
```

What is the String pool?

```
String a = "Priya";  
String b = "Priya";  
String c = "Priya";
```

Java does **not** create three objects.

STRING POOL (inside heap)



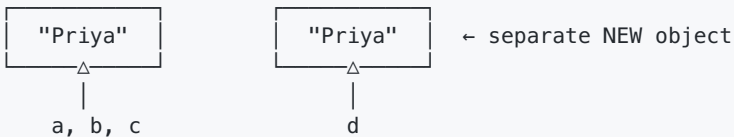
```
System.out.println(a == b);    // true – same object
```

But with **new**:

```
String d = new String("Priya");
```

STRING POOL

HEAP



```
System.out.println(a == d);    // false – different objects  
System.out.println(a.equals(d)); // true – same content
```

You can push it into the pool manually:

```
String e = d.intern();  
System.out.println(a == e);    // true
```

Why must String be immutable? (4 solid reasons)

Reason 1 — the pool would be dangerous otherwise

Because **a**, **b** and **c** share one object, if **b** could change the value, **a** and **c** would silently change too:

```
b = "Ravi";           // if String were mutable...
System.out.println(a); // ...a would also become "Ravi" ▲
```

Immutability makes sharing safe.

Reason 2 — security ★ (say this one in the interview)

```
void connect(String url) {
    checkPermission(url);    // security check passes
    openConnection(url);    // if String were mutable, someone could
}                           // change url between these two lines!
```

That is exactly why usernames, URLs, file paths and class names are Strings.

Reason 3 — HashMap keys stay valid

```
map.put(name, "Developer");
```

The key's **hashCode** is calculated once. If the String could change, the hash would change, and the entry would be lost forever in the wrong bucket.

```
"Priya" → hash 500 → bucket 500 → stored

if it changed to "Ravi" → hash 900
    ↓
map.get() looks in bucket 900 → NOT FOUND ▲
```

That is why String is the most common HashMap key — its hash never changes.

Reason 4 — thread safety for free

Nothing can change, so many threads can share one String with no locks.

Interview trap

```
String s = "Hello";
s.concat(" World");
System.out.println(s);    // Hello ← NOT "Hello World"!
```

`concat` returns a **new** String. You must assign it:

```
s = s.concat(" World");  
System.out.println(s);           // Hello World ✓
```

Same trap with `replace()`, `toUpperCase()`, `trim()` — they all return new Strings.

Interview-ready answer

Strings are immutable in Java, meaning once a String object is created its value can never be changed — any modification method like `concat` or `toUpperCase` returns a new String. The string pool is a special area in the heap where string literals are stored and reused, so `String a = "Priya"` and `String b = "Priya"` point to the same object, which saves a lot of memory. Immutability is required for this pooling to be safe, and it also gives security — a String used for a URL or file path cannot be modified after a security check. It also keeps HashMap keys valid because the hash code never changes, and it makes Strings thread-safe by default. When we use `new String("Priya")`, a separate object is created in the heap outside the pool, and `intern()` can push it into the pool.

Easy memory

```
Literal "Priya"      → String POOL → reused → == is true  
new String("Priya") → HEAP object → new    → == is false
```

Immutable because:

1. Pool sharing must be safe
2. SECURITY (url can't change after the check) ★
3. HashMap key hash stays valid
4. Thread-safe for free

Trap: `s.concat(" World");` ← does nothing unless you assign it back

Q: `==` vs `equals()` ?

The easiest way to remember:

`==` → compares the ADDRESS (are they the SAME object?)
`equals()` → compares the CONTENT (do they LOOK the same?)

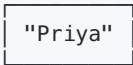
Real-world idea — twins

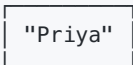
Two identical twins:

`==` → "Are these the same person?" → NO (two people)
`equals()` → "Do they look the same?" → YES (same face)

Simple example

```
String a = new String("Priya");  
String b = new String("Priya");  
  
System.out.println(a == b);      // false  
System.out.println(a.equals(b)); // true
```

a →  address: 100

b →  address: 200

`==` compares 100 vs 200 → false
`equals()` compares "Priya" vs "Priya" → true

For primitives, `==` is correct

```
int a = 10;  
int b = 10;  
System.out.println(a == b);      // true ✓ - primitives have no address
```

Primitives store the actual value, so `==` compares values. There is no `equals()` for primitives.

Primitive → use `==`
Object → use `equals()`

The default `equals()` trap ★

```
class Employee {
    int id;
    Employee(int id) { this.id = id; }
}

Employee e1 = new Employee(101);
Employee e2 = new Employee(101);

System.out.println(e1.equals(e2));    // false ▲
```

Why false? Because `Object`'s default `equals()` is literally:

```
public boolean equals(Object obj) {
    return (this == obj);    // it just does ==
}
```

So unless **you** override `equals()`, it behaves exactly like `==`.

Fix it:

```
class Employee {

    int id;

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Employee other)) return false;
        return this.id == other.id;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

Now `e1.equals(e2)` is `true`. ✓

`String` already overrides `equals()` — that is why `"a".equals("a")` works out of the box.

The Integer cache trap ★★

```
Integer a = 100, b = 100;
System.out.println(a == b);           // true

Integer c = 200, d = 200;
System.out.println(c == d);           // false ▲
```

Java caches Integer objects from **-128 to 127**. Inside that range the same object is reused; outside it, new objects are created.

```
-128 ————— 127   → cached → == true
128 and above       → new    → == false
```

So never use `==` on wrapper objects. Use `.equals()` or `Objects.equals()`.

Null safety

```
String s = null;

s.equals("Priya");           // ✗ NullPointerException
"Priya".equals(s);           // ✓ false – constant first
Objects.equals(s, "Priya");  // ✓ false – safest
```

Interview-ready answer

`==` compares references, meaning it checks whether two variables point to the same object in memory, and for primitives it compares the actual values. `equals()` compares content, but only if the class overrides it — the default implementation in `Object` simply does a `==` check, which is why two `Employee` objects with the same id return false unless we override `equals()`. Classes like `String` and the wrapper classes already override `equals()`. A common trap is comparing `Integer` objects with `==`: values from -128 to 127 come from the Integer cache and compare as equal, while larger values do not. So the rule is: use `==` for primitives and `equals()` for objects, preferably `Objects.equals()` for null safety.

Easy memory

```
==          → same OBJECT?   → address   → twins: same person?  
equals() → same CONTENT? → value      → twins: same face?
```

```
primitive → ==  
object    → equals()
```

```
Default equals() in Object == == → must OVERRIDE it  
Integer cache: -128 to 127 → == true (trap!)  
"literal".equals(var) → null-safe
```

Q: Explain the equals() and hashCode() contract.

The main rule you need to remember is:

```
If two objects are EQUAL by equals(), they MUST have the SAME hashCode.  
  
But the same hashCode does NOT mean the objects are equal.
```

Simple example

```
Employee e1 = new Employee(101, "Priya");  
Employee e2 = new Employee(101, "Priya");
```

They are two different objects in memory:

```
e1 → Employee(101, "Priya")  
e2 → Employee(101, "Priya")
```

But logically we consider them equal, because the id and name match.

```
e1.equals(e2);      // true
```

And if `equals()` says true, the hash codes must match:

```
e1.hashCode() == e2.hashCode();    // MUST be true
```


Why do we need both? (HashMap is the answer)

```
Map<Employee, String> map = new HashMap<>();  
map.put(e1, "Developer");
```

What HashMap does when storing:

```
e1  
↓  
hashCode()  
↓  
find the BUCKET  
↓  
store e1 there
```

What it does when reading:

```
map.get(e2);
```

```
e2  
↓  
hashCode()      ← STEP 1: WHERE to look  
↓  
find the bucket  
↓  
equals()        ← STEP 2: WHICH entry is the match  
↓  
return the value
```

So the easy way to remember is:

```
hashCode() → WHERE to search   (which bucket / which street)  
equals()   → WHICH one matches (which house on that street)
```

Real-world idea: a **postal address**.

```
PIN code    → tells the postman which AREA      → hashCode()  
House name  → tells him WHICH house in that area → equals()
```

Many houses share a PIN code (collision), but the house name identifies the exact one.

What happens if I override only equals()? ▲

```
e1.equals(e2);      // true

// but hashCode() was not overridden:
e1.hashCode();      // 100
e2.hashCode();      // 500
```

HashMap

Bucket 100

↓

e1 ← the entry actually lives here

Bucket 500

↓

HashMap searches here using e2 → EMPTY → returns null ▲

The postman goes to the wrong PIN code, so he never finds the house — even though the house name is correct.

This is why: if you override `equals()`, you MUST override `hashCode()`.

Correct implementation

```
class Employee {  
    private int id;  
    private String name;  
  
    @Override  
    public boolean equals(Object obj) {  
        if (this == obj) return true; // same object  
  
        if (!(obj instanceof Employee other)) return false; // null + type  
check  
  
        return id == other.id  
            && Objects.equals(name, other.name); // compare  
fields  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(id, name); // SAME fields  
    }  
}
```

The golden rule is that both methods must use the **same fields**:

```
equals()    → id + name  
hashCode() → id + name    ← must match
```

The contract rules

For `equals()`:

```
Reflexive   → a.equals(a) is true  
Symmetric   → a.equals(b) true ⇒ b.equals(a) true  
Transitive  → a=b and b=c ⇒ a=c  
Consistent → same data → same result every time  
Null        → a.equals(null) is always false
```

For `hashCode()`:

```

a.equals(b) == true
  ↓
a.hashCode() == b.hashCode()      MUST be true ✓

Same hashCode
  ↓
equals() true?                    NOT guaranteed ✗ (hash collision)

```

A hash collision simply means two different objects landed in the same bucket. That is normal and `HashMap` handles it using `equals()`.

Practical warning — mutable keys

```

Employee e = new Employee(101, "Priya");
map.put(e, "Developer");

e.setId(102);                // hash changed!

map.get(e);                  // null ▲ — it's in the OLD bucket

```

This is why immutable objects (especially `String`) make the best map keys.

Interview-ready answer

The contract states that if two objects are equal according to `equals()`, they must return the same hash code. However, two objects having the same hash code do not have to be equal, because hash collisions are possible. This matters for `HashMap` and `HashSet`: `hashCode()` decides which bucket to search, and `equals()` identifies the exact matching entry within that bucket. If we override `equals()` without overriding `hashCode()`, two logically equal objects can produce different hash codes, so `HashMap` looks in the wrong bucket and fails to find the entry. Therefore whenever we override `equals()`, we must also override `hashCode()` using the same fields. We should also avoid using mutable objects as keys, because changing a field changes the hash code and the entry becomes unreachable.

Easy memory

hashCode() → WHERE? → find the bucket → PIN code
equals() → WHICH? → find the object → house name

Golden rule:

equals() TRUE → hashCode SAME ✓ (guaranteed)
hashCode SAME → equals() TRUE? ✗ (not guaranteed – collision)

Override equals() → ALWAYS override hashCode()
Use the SAME fields in both.

Q: What is autoboxing and unboxing?

The easiest way to remember:

Autoboxing = Primitive → Wrapper Object
Unboxing = Wrapper Object → Primitive

First — what are wrapper classes?

| Primitive | Wrapper class |
|-----------|---------------|
| int | Integer |
| long | Long |
| double | Double |
| float | Float |
| boolean | Boolean |
| char | Character |
| byte | Byte |
| short | Short |

```
int number = 10;           // primitive – just a value
Integer obj = 10;          // wrapper object – a real object
```

1. Autoboxing

Java automatically converts a primitive into its wrapper object.

```
int a = 10;  
Integer b = a; // autoboxing
```

Behind the scenes Java writes:

```
Integer b = Integer.valueOf(a);
```

```
int  
↓  
10  
↓  
AUTOBOXING  
↓  
Integer object
```

Where you see it every day — Collections

```
List<int> numbers; // ✗ not allowed  
List<Integer> numbers = new ArrayList<>(); // ✓
```

Collections can only store **objects**, not primitives. So:

```
numbers.add(10);
```

```
10 → int  
↓  
AUTOBOXING  
↓  
Integer object  
↓  
stored in ArrayList
```

2. Unboxing

The opposite direction.

```
Integer a = 10;  
int b = a;                // unboxing
```

Behind the scenes:

```
int b = a.intValue();
```

```
Integer  
  ↓  
  10  
  ↓  
UNBOXING  
  ↓  
  int
```

Both together

```
int a = 100;  
  
Integer b = a;    // AUTOBOXING  
int c = b;        // UNBOXING
```

```
int a = 100  
  ↓  
  AUTOBOXING  
  ↓  
Integer b = 100  
  ↓  
  UNBOXING  
  ↓  
int c = 100
```

Real-world idea: a **gift box**.

```
A chocolate (primitive)  
  ↓ put it in a gift box  
Boxed chocolate (wrapper)    ← autoboxing  
  ↓ open the box  
Chocolate again (primitive)  ← unboxing
```

Why box it at all? Because the "courier service" (collections, generics) only accepts boxed items.

Interview problem 1 — NullPointerException ★

```
Integer number = null;
int value = number;           // ▲ NullPointerException
```

Why? Unboxing internally calls:

```
number.intValue();
```

but `number` is `null`, so it is effectively:

```
null.intValue();           // ✗
```

Real-life version of this bug:

```
Integer count = map.get("missing-key"); // returns null
int total = count + 1;                  // ▲ NPE
```

Always null-check before unboxing:

```
int total = (count != null) ? count + 1 : 1; // ✓
```

Interview problem 2 — the `==` cache trap ★

```
Integer a = 100, b = 100;
System.out.println(a == b);           // true

Integer a2 = 200, b2 = 200;
System.out.println(a2 == b2);          // false
```

`Integer.valueOf()` uses a cache guaranteed for **-128 to 127**, so those values reuse the same object.

```
-128 ————— 127      → same cached object → == true
outside that range      → brand new objects  → == false
```

Never compare wrapper values with `==`:


```
a.equals(b);           // ✓
Objects.equals(a, b);  // ✓ null-safe
```

Performance note

```
Long sum = 0L;           // ✗ wrapper in a loop
for (long i = 0; i < 1_000_000; i++) {
    sum += i;             // box + unbox one MILLION times
}
```

Use the primitive `long` instead — this is a classic performance bug.

Interview-ready answer

Autoboxing is the automatic conversion of a primitive into its corresponding wrapper class, for example `int` to `Integer`, and internally it calls `Integer.valueOf()`. Unboxing is the reverse conversion from a wrapper object back to a primitive, and it internally calls `intValue()`. Java performs these conversions automatically, and we see them constantly when working with collections, because collections store objects and not primitives. The two things to be careful about are unboxing a null wrapper, which throws a `NullPointerException`, and comparing wrappers with `==`, because only values from -128 to 127 come from the Integer cache. We should also avoid wrapper types inside hot loops because the repeated boxing hurts performance.

Easy memory

```
Primitive → AUTOBOXING → Wrapper Object    (chocolate → gift box)
Wrapper   → UNBOXING   → Primitive          (gift box → chocolate)
```

```
int a = 10;
Integer b = a;    // Autoboxing
int c = b;        // Unboxing
```

- ▲ `Integer x = null; int y = x;` → `NullPointerException`
- ▲ `Integer == Integer` → only -128..127 is true

Q: Is Java pass-by-value or pass-by-reference?

Java is ALWAYS pass-by-value. ✓

This is a favourite interview question because objects make Java *look* like pass-by-reference, but technically it is not.

The easiest way to remember:

```
Primitive → a copy of the VALUE is passed
Object    → a copy of the REFERENCE is passed
```

1. Primitive example

```
public class Test {

    static void change(int x) {
        x = 50;
    }

    public static void main(String[] args) {
        int a = 10;
        change(a);
        System.out.println(a);
    }
}
```

Output:

```
10
```

Why?

```
main()
a = 10
|
| copy the value
↓
change()
x = 10 → x = 50    (only the copy changed)

a is still 10
```

2. What happens with objects?

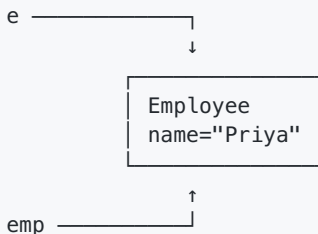
```
class Employee {  
    String name;  
}  
  
static void changeName(Employee emp) {  
    emp.name = "Ravi";  
}  
  
public static void main(String[] args) {  
    Employee e = new Employee();  
    e.name = "Priya";  
  
    changeName(e);  
    System.out.println(e.name);  
}
```

Output:

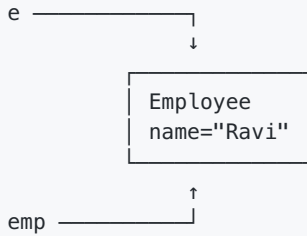
Ravi

You might think: "The original object changed, so Java must be pass-by-reference." **No — it is still pass-by-value.**

What actually happened: Java copied the **reference value**, so both variables point to the same object.



So `emp.name = "Ravi"` modifies that one shared object:



That is why `e.name` becomes `"Ravi"` .

3. The proof that Java is pass-by-value ★

Change the **reference itself** inside the method:

```
static void changeEmployee(Employee emp) {
    emp = new Employee();           // point the COPY somewhere else
    emp.name = "Ravi";
}

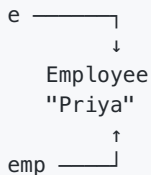
public static void main(String[] args) {
    Employee e = new Employee();
    e.name = "Priya";

    changeEmployee(e);
    System.out.println(e.name);
}
```

Output:

Priya

Before:



After `emp = new Employee()` :

```

e  —————> Employee
                "Priya"      ← original untouched

emp —————> Employee
                "Ravi"       ← only the copy moved

```

If Java were pass-by-reference, `e` would now point to the new object and print "Ravi". It prints "Priya", which proves only a **copy** of the reference was passed.

Real-world idea — a house key

```

You have a house key.
You give your FRIEND A DUPLICATE key (a copy of the reference).

Friend paints the house      → you see the change ✓ (same house)
Friend throws away his key and
buys a different house      → YOUR house is unchanged ✓

```

The two cases to remember

1. MODIFY the object `emp.name = "Ravi";`
 → caller sees the change ✓
2. REASSIGN the parameter `emp = new Employee();`
 → caller sees nothing ✗

Interview trap

If the interviewer asks: "Are objects passed by reference in Java?" — do not say yes.

Say:

No. Java is always pass-by-value. When passing an object, Java passes a copy of the object's reference value. Therefore we can modify the object through that copied reference, but reassigning the parameter does not change the caller's reference.

Interview-ready answer

Java is always pass-by-value. For primitive types, a copy of the value is passed, so changes inside the method do not affect the caller. For objects, a copy of the reference value is passed. Because the original reference and the copied reference point to the same object, changes to the object's state are visible to the caller — for example setting `emp.name`. However, if we reassign the parameter to a new object inside the method, the caller's reference is unaffected, which proves that Java passed a copy of the reference rather than the reference itself.

Easy memory

PRIMITIVE

`int a = 10` → copy of 10 → method

OBJECT

Employee e → Object
| copy of the reference
↓
emp → SAME Object

Modify the object → caller sees it ✓

Reassign the param → caller sees nothing ✗

One line: "Java is always pass-by-value; for objects, the value copied is the reference."

Q: What are wrapper classes and why are they needed?

The easiest way to remember:

Wrapper class = an OBJECT version of a primitive

Reason = Java collections and generics only work with OBJECTS

The 8 wrappers

```
byte    → Byte
short   → Short
int      → Integer    ★
long     → Long
float    → Float
double   → Double
char     → Character
boolean  → Boolean
```

Why do we need them? (4 reasons)

1. Collections only store objects

```
List<int> list;                // ✗ not allowed
List<Integer> list = new ArrayList<>(); // ✓
```

2. A primitive cannot be null

```
int    age1 = null;    // ✗ compile error
Integer age2 = null;    // ✓ allowed
```

This matters hugely in databases:

```
DB column `age` is NULL
    ↓
int    → cannot represent it (0 would be a WRONG value)
Integer → null means "not provided" ✓
```

That is why JPA entities use `Integer`, `Long`, `Double` — not primitives.

```
@Entity
class Employee {
    @Id
    private Long id;        // ✓ null before saving
    private Integer age;    // ✓ null allowed
}
```

If `id` were `long`, a new unsaved entity would have id `0` instead of "no id yet".

3. Useful helper methods

```

Integer.parseInt("100");           // String → int
Integer.MAX_VALUE;                 // 2147483647
Integer.MIN_VALUE;
Integer.toBinaryString(10);        // "1010"
Integer.valueOf("100");
Double.parseDouble("10.5");
Boolean.parseBoolean("true");
Character.isDigit('5');

```

A primitive `int` has no methods at all — `10.parseInt()` is meaningless.

4. Generics need objects

```

Map<String, Integer> scores = new HashMap<>(); // ✓
Map<String, int> scores;                       // ✗

```

Primitive vs wrapper

| | Primitive | Wrapper |
|---------------------|-------------------------------------|--------------------------------|
| Stores | actual value | object reference |
| Memory | stack (as a local) | heap |
| Default value | <code>0</code> , <code>false</code> | <code>null</code> |
| Can be null | ✗ | ✓ |
| Size | small (4 bytes for int) | larger (object header + value) |
| Methods | none | many |
| Used in collections | ✗ | ✓ |
| Speed | faster | slower |

When to use which

```

Use PRIMITIVE → loop counters, calculations, performance-critical code
Use WRAPPER   → collections, generics, JPA entities, DTO fields that can
                be null

```


Interview trap

```
Integer count = null;
if (count > 0) { }           // ▲ NullPointerException (unboxing null)
```

Always null-check wrapper fields coming from a database or an API.

Interview-ready answer

Wrapper classes are the object representations of the eight primitive types — `Integer` for `int`, `Double` for `double` and so on. They are needed because Java collections and generics only work with objects, so we cannot write `List<int>`. They also allow null, which is important when mapping database columns or API fields that may be missing — that is why JPA entities use `Integer` and `Long` rather than primitives. In addition, wrappers provide useful utility methods like `Integer.parseInt()` and constants like `Integer.MAX_VALUE`. Java converts between them automatically through autoboxing and unboxing, but we must be careful with null wrappers because unboxing null throws a `NullPointerException`.

Easy memory

Wrapper = primitive wearing an OBJECT costume

Needed for:

| | |
|-------------------------|--------------------------------------|
| 1. Collections/Generics | (<code>List<Integer></code>) |
| 2. null support | (DB / JPA) |
| 3. Utility methods | (<code>Integer.parseInt()</code>) |

`int` → fast, cannot be null

`Integer` → object, CAN be null → NPE risk when unboxing

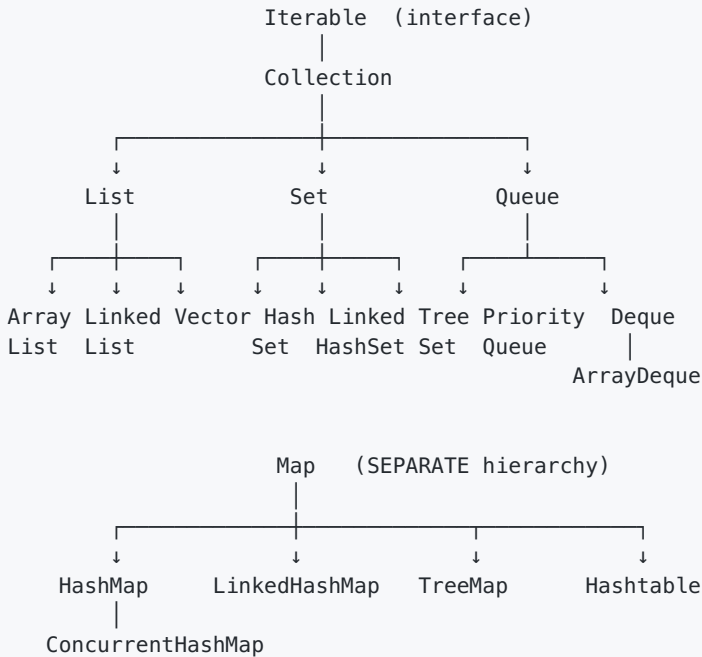
Collections

Q: Describe the Collections framework hierarchy.

The easiest way to remember:

Collection → stores SINGLE values (List, Set, Queue)
Map → stores KEY-VALUE pairs (NOT part of Collection!)

The full picture



★ **Map does NOT extend Collection.** This is a favourite interview question. The reason is simple: `Collection` methods like `add(E e)` take a single element, but a `Map` needs `put(K key, V value)` — two values. The shapes don't match, so `Map` was kept separate.

What each one is for

List → ORDERED, allows DUPLICATES, index access
→ shopping cart (2 kg rice twice is fine)

Set → NO duplicates
→ Aadhaar numbers, email IDs

Queue → FIFO processing
→ ticket counter line

Map → key → value lookup
→ dictionary: word → meaning

Quick code

```
List<String> list = new ArrayList<>();
list.add("Apple");
list.add("Apple");           // ✓ duplicate allowed
System.out.println(list);    // [Apple, Apple]

Set<String> set = new HashSet<>();
set.add("Apple");
set.add("Apple");           // ignored
System.out.println(set);     // [Apple]

Map<String, Integer> map = new HashMap<>();
map.put("Apple", 100);
map.put("Apple", 200);      // replaces the old value
System.out.println(map);    // {Apple=200}

Queue<String> q = new LinkedList<>();
q.offer("A"); q.offer("B");
System.out.println(q.poll()); // A (first in, first out)
```

Interview-ready answer

The Collections framework starts with the **Iterable** interface, which **Collection** extends. **Collection** has three main branches — **List** for ordered collections that allow duplicates, **Set** for unique elements, and **Queue** for FIFO processing. Common implementations are **ArrayList** and **LinkedList** for List, **HashSet**, **LinkedHashSet** and **TreeSet** for Set, and **PriorityQueue** and **ArrayDeque** for Queue. **Map** is a separate hierarchy and does not extend **Collection**, because a Map stores key-value pairs while **Collection** is

designed around single elements. `HashMap`, `LinkedHashMap`, `TreeMap` and `ConcurrentHashMap` are the main Map implementations.

Easy memory

Iterable → Collection → List / Set / Queue
Map → SEPARATE (put needs TWO values, add needs ONE)

List → duplicates OK, ordered → shopping cart
Set → unique only → Aadhaar numbers
Queue → FIFO → ticket line
Map → key → value → dictionary

Q: List vs Set vs Map?

The easiest way to remember with one real-world example — a **classroom**:

List → attendance sheet in roll-number order (order matters, a name can repeat)
Set → the set of unique subjects offered (no repeats)
Map → roll number → student name (lookup by key)

Comparison

| | List | Set | Map |
|--------------|-----------------------|--------------------|------------------------|
| Duplicates | ✓ allowed | ✗ not allowed | keys ✗, values ✓ |
| Order | ✓ insertion order | depends on type | depends on type |
| Index access | ✓ <code>get(0)</code> | ✗ | ✗ (get by key) |
| Null | many nulls OK | one null (HashSet) | one null key (HashMap) |
| Main use | ordered items | uniqueness | key-based lookup |

Code showing the difference clearly

```
// LIST – keeps everything, in order
List<String> list = new ArrayList<>();
list.add("Priya");
list.add("Ravi");
list.add("Priya");
System.out.println(list);           // [Priya, Ravi, Priya]
System.out.println(list.get(0));    // Priya (index access)

// SET – silently drops duplicates
Set<String> set = new HashSet<>();
set.add("Priya");
set.add("Ravi");
set.add("Priya");                  // ignored
System.out.println(set);           // [Priya, Ravi]
System.out.println(set.size());    // 2

// MAP – key is unique, value can repeat
Map<Integer, String> map = new HashMap<>();
map.put(101, "Priya");
map.put(102, "Ravi");
map.put(101, "Meena");             // REPLACES Priya
System.out.println(map);           // {101=Meena, 102=Ravi}
System.out.println(map.get(101));  // Meena
```

The classic real use — removing duplicates

```
List<String> names = List.of("Priya", "Ravi", "Priya", "Meena");

Set<String> unique = new HashSet<>(names);
System.out.println(unique);        // [Priya, Ravi, Meena]
```

One line, duplicates gone.

Which one do I pick?

```
Do I need to keep the ORDER and allow repeats? → List
Do I only care about UNIQUE values?             → Set
Do I need to look something up BY A KEY?        → Map
```

Interview-ready answer

A **List** is an ordered collection that allows duplicates and supports index-based access, so we use it when the order matters — for example items in a shopping cart. A **Set** does not allow duplicates and is used when we care only about uniqueness, such as a set of email IDs; it also has no index access. A **Map** stores key-value pairs where keys must be unique but values can repeat, and it is used for lookups such as employee id to employee name. Internally **HashSet** is backed by a **HashMap**, and **Map** is not part of the **Collection** hierarchy.

Easy memory

| | | | |
|------|-------------------------|----------------------|------------|
| List | → [Priya, Ravi, Priya] | → order + duplicates | → get(0) |
| Set | → [Priya, Ravi] | → unique only | → no index |
| Map | → {101=Priya, 102=Ravi} | → key → value | → get(101) |

Remove duplicates instantly: `new HashSet<>(list)`

Q: ArrayList vs LinkedList?

The easiest way to remember:

| | | | |
|------------|------------|-------------------------|----------------|
| ArrayList | → an ARRAY | → FAST to READ | (get by index) |
| LinkedList | → a CHAIN | → FAST to INSERT/DELETE | in the middle |

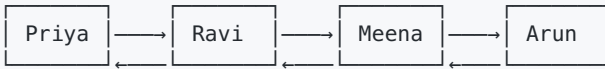
How they store data

ArrayList — one continuous block, like seats in a cinema hall:

| index: | 0 | 1 | 2 | 3 |
|--------|-------|------|-------|------|
| | Priya | Ravi | Meena | Arun |

get(2) → jump straight to seat 2 → 0(1) ▲

LinkedList — separate nodes joined by links, like a train:



get(2) → walk: Priya → Ravi → Meena → 0(n)

Insertion in the middle

ArrayList must shift everything to the right:

Insert "Kumar" at index 1

Before: [Priya] [Ravi] [Meena] [Arun]

← ← ← shift all of them

After: [Priya] [Kumar] [Ravi] [Meena] [Arun]

Cost: $O(n)$

Real-world idea: in a cinema hall, inserting one person in the middle of a row means everyone slides over.

LinkedList just rewires two links:

Priya → Ravi

Kumar

Priya → Kumar → Ravi (only 2 links changed)

Cost: $O(1)$ once you're at the position ▲

Real-world idea: adding a coach to a train — you just unhook and rehook.

ArrayList resizing

```
List<String> list = new ArrayList<>(); // capacity 10
```

When it fills up:

```

Old array (10) FULL
    ↓
create a NEW array of 15  (roughly 1.5×)
    ↓
copy all elements
    ↓
old array becomes garbage

```

If you know the size in advance, avoid the copies:

```
new ArrayList<>(1000);    // ✓ pre-sized
```

Big-O table

| Operation | ArrayList | LinkedList |
|---------------------------------|-----------------|---------------------------|
| <code>get(index)</code> | O(1) ▲ | O(n) |
| <code>add()</code> at end | O(1) amortised | O(1) |
| <code>add()</code> at middle | O(n) (shifting) | O(1)* |
| <code>remove()</code> at middle | O(n) (shifting) | O(1)* |
| Memory per element | low | higher (2 extra pointers) |

*after you have reached the node — reaching it is O(n).

The honest real-world answer ★

In practice, **ArrayList wins almost always.**

Even for middle insertion, **LinkedList** must first *walk* to the position, which is O(n), and ArrayList's shifting uses `System.arraycopy()`, which is extremely fast at the CPU level. ArrayList is also cache-friendly because its data sits together in memory.

```

Use ArrayList → 95% of the time (default choice)
Use LinkedList → when you need a Queue/Deque
                  (addFirst / removeFirst) – heavy work at the ENDS

```


Interview-ready answer

`ArrayList` is backed by a dynamic array, so index-based access is $O(1)$, but inserting or removing in the middle is $O(n)$ because elements have to be shifted, and it also resizes by creating a larger array and copying. `LinkedList` is a doubly-linked list, so adding or removing at a known position is $O(1)$ since we only rewire pointers, but accessing an element by index is $O(n)$ because it must traverse from the start, and each node needs extra memory for two pointers. In practice I use `ArrayList` as the default because reads dominate in most applications and it is cache-friendly; I use `LinkedList` mainly when I need `Deque` or queue behaviour with frequent additions and removals at the ends.

Easy memory

`ArrayList` → cinema seats → `get(2)` instantly ▲ → shifting is costly
`LinkedList` → train coaches → walk to find → rewiring is cheap

Read a lot? → `ArrayList` (default ★)
Add/remove at the ends? → `LinkedList` (as a `Deque`)

Q: How does HashMap work internally?

The easiest way to remember:

`put()` → `hashCode()` decides the BUCKET → `equals()` checks for a duplicate key
`get()` → `hashCode()` finds the BUCKET → `equals()` finds the exact entry

A much deeper version of this topic (with resizing, treeify, load factor and Java 7 vs 8 differences) is in [31-hashmap-internals.md](#).

The structure

A `HashMap` is an **array of buckets**. Each bucket holds a linked list (or a tree) of entries.

```

Node[] table                (default size 16)

index 0 → null
index 1 → null
index 2 → [ "Priya" | 500 ] → null
index 3 → null
...
index 5 → [ "Ravi" | 700 ] → [ "Kumar" | 800 ] → null
                                ↑
                                two keys landed in the
                                same bucket = COLLISION
...
index 15 → null

```

What happens on `put("Priya", 500)`

1. `hash = key.hashCode()` → e.g. 78453
2. spread the bits: `hash ^ (hash >>> 16)` → better distribution
3. `index = hash & (n - 1)` → same as `hash % 16`, but faster
4. Is that bucket empty?
 - YES → store the node ✓
 - NO → collision:
 - compare keys with `equals()`
 - same key → REPLACE the value
 - different → append to the list/tree

Why `hash & (n-1)` instead of `%`? Because the table size is always a power of 2, so bitwise AND gives the same result and is much faster.

What happens on `get("Priya")`

```

"Priya"
  ↓
hashCode() → find the bucket      ← STEP 1: WHERE
  ↓
walk that bucket
  ↓
equals()    → find the exact key  ← STEP 2: WHICH
  ↓
return 500

```

Real-world idea — a **library**:

```
Book's subject → tells you which SHELF → hashCode()  
Book's title → finds the exact BOOK → equals()
```

Collision handling — Java 8 improvement ★

```
Java 7 : collisions → LINKED LIST only → worst case  $O(n)$   
Java 8 : collisions → linked list, and once a  
          bucket has MORE THAN 8 entries and the  
          table is  $\geq 64$ , it converts to a  
          RED-BLACK TREE → worst case  $O(\log n)$  ▲
```

```
Bucket 5: A → B → C → D → E → F → G → H → I (list, 9 entries)  
          ↓ treeify  
          balanced tree →  $O(\log n)$ 
```

If the bucket shrinks back to 6 entries, it un-trees back to a list.

Load factor and resizing

```
Default capacity = 16  
Default load factor = 0.75  
  
Resize threshold =  $16 \times 0.75 = 12$  entries
```

When the 13th entry arrives:

```
capacity 16 → 32 (doubles)  
↓  
ALL entries are REHASHED into the new table
```

Resizing is expensive, so if you know the size in advance:

```
new HashMap<>(1000); // avoids repeated resizing ✓
```

Why 0.75? It is the balance point — a smaller value wastes memory, a larger value causes too many collisions.

Time complexity

```
Best / average case → O(1)      ▲ (good hashCode, few collisions)
Worst case Java 7   → O(n)      (all keys in one bucket)
Worst case Java 8   → O(log n)  (treeified bucket)
```

Interview traps

1. Mutable key

```
Employee e = new Employee(101);
map.put(e, "Developer");

e.setId(102);           // hash changed
map.get(e);             // null ▲ – the entry is in the old bucket
```

Use immutable keys — this is why `String` is the most common key.

2. Null key

```
map.put(null, "value"); // ✓ allowed – always stored in bucket 0
```

`HashMap` allows **one** null key and many null values. `Hashtable` and `ConcurrentHashMap` allow none.

3. Not thread-safe

Two threads resizing at the same time can corrupt the map (in Java 7 it could even cause an infinite loop). Use `ConcurrentHashMap` for concurrent access.

Interview-ready answer

Internally a `HashMap` is an array of buckets, default size 16. When we call `put`, it takes the key's `hashCode()`, applies a spreading function to mix the high bits, and computes the bucket index using `hash & (n-1)`. If the bucket is empty the entry is stored directly; if not, it is a collision, and it uses `equals()` to check whether the same key already exists — if yes it replaces the value, otherwise it appends the entry to that bucket. In Java 8, once a bucket has more than 8 entries and the table size is at least 64, the bucket converts from a linked list to a red-black tree, improving the worst case from $O(n)$ to $O(\log n)$. The default load factor is 0.75, so at 12 entries the table doubles to 32 and everything is rehashed.

Lookups are $O(1)$ on average. Keys should be immutable, because changing a field after insertion changes the hash code and makes the entry unreachable.

Easy memory

Array of buckets (16) + load factor 0.75 → resize at 12 → double to 32

put: hashCode → bucket → equals → replace or append

get: hashCode → WHERE → equals → WHICH

Library: subject = shelf (hashCode) | title = book (equals)

Java 8: bucket > 8 entries (table ≥ 64) → red-black TREE → $O(\log n)$

One null key allowed. Not thread-safe → use ConcurrentHashMap.

Q: HashMap vs Hashtable vs ConcurrentHashMap?

The easiest way to remember:

| | | |
|-------------------|-----------------------------|---------------------|
| HashMap | → FAST, not thread-safe | → single thread ★ |
| Hashtable | → SLOW, locks the WHOLE map | → legacy, don't use |
| ConcurrentHashMap | → FAST + thread-safe | → multi-thread ★ |

The locking difference — this is the whole answer

Hashtable locks the entire map:

Thread 1 wants bucket 2
Thread 2 wants bucket 9
Thread 3 wants bucket 14

ALL wait for ONE lock

◆ ENTIRE MAP LOCKED

Only ONE thread works at a time → very slow

Real-world idea: a bank with 10 counters but only **one** key for the whole building — everyone queues.

ConcurrentHashMap locks only the bucket being written:

```
Thread 1 → bucket 2  ♦  works ✓  
Thread 2 → bucket 9  ♦  works ✓    (at the same time!)  
Thread 3 → bucket 14 ♦  works ✓
```

Different buckets → no waiting → fast ▲

Real-world idea: the same bank, but each counter has its own lock. Only two people wanting the *same* counter have to wait.

```
Java 7 → segment locking (16 segments)  
Java 8 → per-bucket CAS + synchronized on the first node (even finer)
```

Reads in `ConcurrentHashMap` need **no lock at all**.

Comparison table

| | HashMap | Hashtable | ConcurrentHashMap |
|-------------|--------------------|------------------------|-------------------------------|
| Thread-safe | ✗ No | ✓ Yes | ✓ Yes |
| Locking | none | whole map | per bucket |
| Performance | fastest (1 thread) | slowest | fast (many threads) |
| null key | ✓ one | ✗ NPE | ✗ NPE |
| null value | ✓ many | ✗ NPE | ✗ NPE |
| Iterator | fail-fast | fail-fast (enumerator) | fail-safe (weakly consistent) |
| Since | 1.2 | 1.0 (legacy) | 1.5 |
| Use it? | ✓ single thread | ✗ never | ✓ multi thread |

Why does ConcurrentHashMap not allow null?

```
map.get("key");    // returns null
```

In a concurrent map that is ambiguous:

```
null means → the key is absent?  
null means → the key exists with a null value?
```

In a single-threaded `HashMap` you can resolve it with `containsKey()`, but in a concurrent map another thread could change the answer between the two calls. So nulls were simply banned.

What about `Collections.synchronizedMap()` ?

```
Map<String, String> m = Collections.synchronizedMap(new HashMap<>());
```

This just wraps every method in a lock on the whole map — the same problem as `Hashtable`. Use `ConcurrentHashMap` instead.

Interview-ready answer

`HashMap` is not synchronized, so it is the fastest option but unsafe if multiple threads modify it. `Hashtable` is the legacy thread-safe version, but it synchronizes every method on the entire map, so only one thread can operate at a time and it becomes a bottleneck. `ConcurrentHashMap` is the modern thread-safe option: in Java 8 it locks only the individual bucket being written using CAS and synchronization on the first node, and reads require no locking at all, so many threads can work in parallel. Also, `HashMap` allows one null key and multiple null values, while `Hashtable` and `ConcurrentHashMap` do not allow nulls at all, because in a concurrent map a null return value would be ambiguous. In real projects I use `HashMap` for single-threaded code and `ConcurrentHashMap` for shared caches or counters.

Easy memory

| | | | |
|--------------------------------|----------------|-----------|-------------------|
| <code>HashMap</code> | → no lock | → fastest | → single thread ★ |
| <code>Hashtable</code> | → ONE big lock | → slow | → legacy ✗ |
| <code>ConcurrentHashMap</code> | → bucket lock | → fast | → multi thread ★ |

Bank: one key for the whole bank (`Hashtable`)
vs one lock per counter (`ConcurrentHashMap`)

null: `HashMap` ✓ | `Hashtable` ✗ | `ConcurrentHashMap` ✗

Q: HashSet vs TreeSet?

The easiest way to remember:

| | | | | |
|---------------|-------------------|----------|---------------|-----------|
| HashSet | → NO order | → FAST | → $O(1)$ | ★ default |
| TreeSet | → SORTED order | → slower | → $O(\log n)$ | |
| LinkedHashSet | → INSERTION order | | → $O(1)$ | |

See it immediately

```
Set<String> hashSet = new HashSet<>();
hashSet.add("Zebra");
hashSet.add("Apple");
hashSet.add("Mango");
System.out.println(hashSet);      // [Apple, Zebra, Mango] ← random-
looking

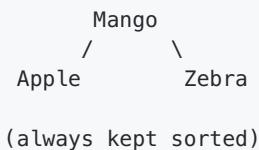
Set<String> treeSet = new TreeSet<>();
treeSet.add("Zebra");
treeSet.add("Apple");
treeSet.add("Mango");
System.out.println(treeSet);      // [Apple, Mango, Zebra] ← SORTED ✓

Set<String> linked = new LinkedHashSet<>();
linked.add("Zebra");
linked.add("Apple");
linked.add("Mango");
System.out.println(linked);      // [Zebra, Apple, Mango] ← insertion
order
```

How they store data

HashSet → backed by a HashMap → buckets → no order

TreeSet → backed by a TreeMap → red-black TREE



Real-world idea:

HashSet → clothes thrown into a cupboard: finding one is instant if you know it,
but there is no order
TreeSet → a dictionary: always alphabetical, but inserting a new word means
finding the right place

Comparison table

| | HashSet | LinkedHashSet | TreeSet |
|------------------|---------|-----------------------|--|
| Order | none | insertion order | sorted |
| Speed | O(1) ▲ | O(1) | O(log n) |
| Internally | HashMap | HashMap + linked list | TreeMap (red-black tree) |
| null allowed | ✓ one | ✓ one | ✗ NPE |
| Needs Comparable | ✗ | ✗ | ✓ yes |
| Extra methods | — | — | first(),
last(),
headSet(),
tailSet(),
ceiling() |

TreeSet's bonus methods

```
TreeSet<Integer> marks = new TreeSet<>(List.of(45, 78, 92, 60));

marks.first();           // 45  (lowest)
marks.last();            // 92  (highest)
marks.headSet(70);       // [45, 60] → below 70
marks.tailSet(70);       // [78, 92] → 70 and above
marks.ceiling(70);       // 78 → smallest value ≥ 70
marks.floor(70);         // 60 → largest value ≤ 70
```

Very useful for range queries — "all students who scored above 70".

TreeSet traps ★

1. No null

```
TreeSet<String> ts = new TreeSet<>();
ts.add(null);          // ▲ NullPointerException
```

Because it must **compare** elements, and it can't compare null.

2. Custom objects must be comparable

```
TreeSet<Employee> set = new TreeSet<>();
set.add(new Employee(101, "Priya"));    // ▲ ClassCastException
```

Fix it either way:

```
// Option A – implement Comparable
class Employee implements Comparable<Employee> {
    public int compareTo(Employee o) {
        return Integer.compare(this.id, o.id);
    }
}

// Option B – pass a Comparator
TreeSet<Employee> set =
    new TreeSet<>(Comparator.comparing(Employee::getName));
```

3. TreeSet uses **compareTo()**, not **equals()**, to decide duplicates

```
// if compareTo() returns 0, TreeSet treats them as the SAME element
```

Interview-ready answer

HashSet is backed by a **HashMap** and stores elements with no ordering, giving $O(1)$ add, remove and contains, so it is the default choice for uniqueness.

TreeSet is backed by a **TreeMap** which is a red-black tree, so elements are always kept in sorted order, but operations are $O(\log n)$. **TreeSet** also does not allow null and requires elements to be comparable, either by implementing **Comparable** or by passing a **Comparator**. In return it gives useful navigation methods like **first()**, **last()**, **headSet()** and **ceiling()** for range queries. If I only need uniqueness I use **HashSet**; if I need sorted output I use **TreeSet**; and if I need to preserve insertion order I use **LinkedHashSet**.

Easy memory

HashSet → cupboard → no order → $O(1)$ ★
LinkedHashSet → same, but remembers the order it went in
TreeSet → dictionary → SORTED → $O(\log n)$

TreeSet: no null ✖, needs Comparable/Comparator, gives first()/last()/ceiling()

Q: Comparable vs Comparator?

The easiest way to remember:

Comparable → the class sorts ITSELF → ONE natural order → compareTo()
Comparator → an OUTSIDE sorter → MANY orders → compare()

The story

You have an `Employee` class. The "natural" order is by id — that belongs inside the class.

```
class Employee implements Comparable<Employee> {  
  
    int id;  
    String name;  
    double salary;  
  
    @Override  
    public int compareTo(Employee other) {  
        return Integer.compare(this.id, other.id);    // natural order = id  
    }  
}
```

```
Collections.sort(list);    // uses compareTo() automatically
```

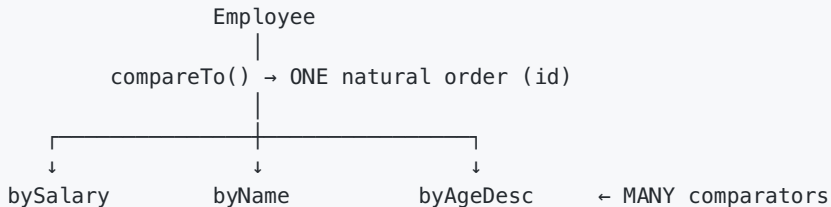
But tomorrow the manager wants sorting by salary, and HR wants sorting by name. You can't have three `compareTo()` methods — so you write **Comparators**.

```

Comparator<Employee> bySalary = Comparator.comparingDouble(e -> e.salary);
Comparator<Employee> byName    = Comparator.comparing(e -> e.name);

list.sort(bySalary);
list.sort(byName);

```



Real-world idea: **Amazon product listing.**

```

Default listing order      → Comparable (built into the product)
"Sort by: Price / Rating / → Comparator (you choose at runtime)
  Newest / Popularity"

```

Modern Comparator syntax (use this in the interview) ★

```

// single field
list.sort(Comparator.comparing(Employee::getName));

// descending
list.sort(Comparator.comparing(Employee::getSalary).reversed());

// multi-level: department, then salary descending
list.sort(Comparator.comparing(Employee::getDept)
    .thenComparing(Comparator.comparing(Employee::getSalary).reversed()));

// null-safe
list.sort(Comparator.nullsFirst(Comparator.comparing(Employee::getName)));

```

This `thenComparing` chain is a strong thing to show — it means "sort by department; if the department is the same, sort by salary descending".

What does the return value mean?

```
compareTo(other)

negative → this comes BEFORE other
zero     → they are equal
positive → this comes AFTER other
```

Easy way to remember ascending order:

```
return this.id - other.id;           // ▲ works, but can overflow
return Integer.compare(this.id, other.id); // ✓ safe
```

For descending, just flip it:

```
return Integer.compare(other.id, this.id);
```

Comparison table

| | Comparable | Comparator |
|-----------------------|----------------------------|--|
| Package | java.lang | java.util |
| Method | compareTo(T o) | compare(T a, T b) |
| Where the logic lives | inside the class | outside the class |
| Number of orders | one (natural) | many |
| Modifies the class? | ✓ yes | ✗ no |
| Sort call | Collections.sort(list) | list.sort(comparator) |
| Use when | there is one obvious order | you need several orders, or you can't edit the class |

The key practical point

If the class is from a library and you **cannot edit it**, `Comparable` is impossible — `Comparator` is your only option. That is a great point to mention.

Interview-ready answer

`Comparable` is implemented by the class itself and defines its single natural ordering through `compareTo()` — for example an `Employee` sorted by id by default. `Comparator` is a separate object that defines an alternative ordering through `compare()`, so we can have many of them, such as sorting employees by salary, by name or by joining date. `Comparable` requires modifying the class, so if the class comes from a third-party library, `Comparator` is the only option. Since Java 8 we usually write comparators using `Comparator.comparing()` with method references, and chain them with `thenComparing()` and `reversed()` for multi-level sorting.

Easy memory

`Comparable` → "I can compare MYSELF" → `compareTo()` → 1 natural order → `java.lang`
`Comparator` → "I compare OTHERS" → `compare(a,b)` → many orders → `java.util`

Amazon: default listing = `Comparable` | "Sort by price" = `Comparator`

```
Comparator.comparing(Employee::getSalary).reversed()
            .thenComparing(Employee::getName)
```

Q: Fail-fast vs fail-safe iterators?

The easiest way to remember:

Fail-fast → works on the ORIGINAL → modification → throws `ConcurrentModificationException` ▲
Fail-safe → works on a COPY → modification → no exception, but you may see stale data

The famous crash

```
List<String> list = new ArrayList<>(List.of("A", "B", "C"));

for (String s : list) {
    if (s.equals("B")) {
        list.remove(s);          // ▲ ConcurrentModificationException
    }
}
```

Why does it happen? — **modCount**

Every ArrayList/HashMap keeps a counter of structural changes:

```
list created           → modCount = 0
iterator created       → expectedModCount = 0

list.remove("B")       → modCount = 1

next loop step → iterator checks:
    modCount (1) != expectedModCount (0)
        ↓
    throw ConcurrentModificationException ▲
```

Real-world idea: You are reading a book. Someone tears out a page while you are reading. The iterator refuses to continue with a book that changed underneath it — better to fail loudly than to give wrong results.

The correct fixes

```
// 1. Use the iterator's own remove() ★
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("B")) {
        it.remove();                // ✓ updates expectedModCount too
    }
}

// 2. removeIf() – cleanest, Java 8
list.removeIf(s -> s.equals("B")); // ✓

// 3. Collect into a new list
List<String> result = list.stream()
    .filter(s -> !s.equals("B"))
    .toList(); // ✓

// 4. Use a concurrent collection
List<String> safe = new CopyOnWriteArrayList<>(list);
```

Fail-safe iterators

```
Map<String, String> map = new ConcurrentHashMap<>();
map.put("A", "1");
map.put("B", "2");

for (String key : map.keySet()) {
    map.put("C", "3"); // ✓ no exception
}
```

CopyOnWriteArrayList literally copies the array on every write:

```
Write happens
  ↓
create a NEW copy of the array
  ↓
the running iterator keeps reading the OLD snapshot → safe, but slightly stale
```

That is why **CopyOnWriteArrayList** is used for read-heavy, write-rare data (like a listener list or a config cache) — writes are expensive.

`ConcurrentHashMap` 's iterator is "weakly consistent": it never throws, and it may or may not show changes made after it started.

Comparison

| | Fail-fast | Fail-safe |
|-----------------|--|--|
| Works on | the original collection | a copy / snapshot |
| On modification | <code>ConcurrentModificationException</code> | no exception |
| Memory | low | higher (copies) |
| Data freshness | always current | may be stale |
| Examples | <code>ArrayList</code> , <code>HashMap</code> , <code>HashSet</code> , <code>Vector</code> | <code>CopyOnWriteArrayList</code> , <code>ConcurrentHashMap</code> |

Important nuance ★

Fail-fast is **best effort** — it is not guaranteed. It is a debugging aid, not a synchronization mechanism. Never write code that depends on catching `ConcurrentModificationException` .

Interview-ready answer

Fail-fast iterators operate directly on the collection and keep an internal `modCount` . If the collection is structurally modified while iterating, the count no longer matches what the iterator expected and it throws a `ConcurrentModificationException` — this is the behaviour of `ArrayList` , `HashMap` and `HashSet` . Fail-safe iterators work on a copy or a snapshot of the data, so modification during iteration does not throw, but the iterator may not reflect the latest changes — `CopyOnWriteArrayList` and `ConcurrentHashMap` behave this way. To remove elements safely while iterating I use `iterator.remove()` or, more commonly, `removeIf()` . It is also worth noting that fail-fast behaviour is best-effort and should not be relied on for correctness.

Easy memory

Fail-fast → ORIGINAL → modCount mismatch → ▲
ConcurrentModificationException
(ArrayList, HashMap, HashSet)

Fail-safe → COPY → no crash, maybe stale data
(CopyOnWriteArrayList, ConcurrentHashMap)

Fix: `it.remove()` | `list.removeIf(...)` ★
Someone tears a page out of the book you're reading → fail-fast complains

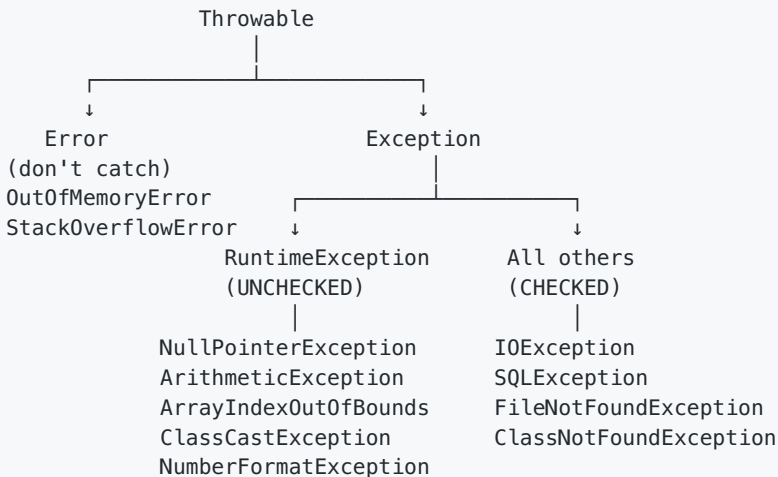
Exceptions

Q: Checked vs unchecked exceptions?

The easiest way to remember:

Checked → the COMPILER forces you to handle it → external problems
Unchecked → the compiler does NOT force you → programming mistakes

The hierarchy first



★ One line rule: **everything under `RuntimeException` (and `Error`) is unchecked; everything else under `Exception` is checked.**

Checked — the compiler blocks you

```
FileReader fr = new FileReader("data.txt");    // x won't compile
```

You must either handle it:

```
try {
    FileReader fr = new FileReader("data.txt");
} catch (FileNotFoundException e) {
    System.out.println("File missing");
}
```

or declare it:

```
void readFile() throws FileNotFoundException {
    FileReader fr = new FileReader("data.txt");
}
```

Why does Java force this? Because the file genuinely **might** be missing — it is an *expected* external failure and your program should have a plan for it.

Unchecked — the compiler stays silent

```
String s = null;
s.length();           // compiles fine, crashes at runtime ▲
```

```
int[] arr = new int[3];
arr[5] = 10;          // compiles fine, crashes at runtime ▲
```

The compiler doesn't force handling because these are **bugs in your code**, not external conditions. The fix is not a try-catch — the fix is a null check or a correct index.

Real-world idea

CHECKED → going out in the monsoon
You KNOW it may rain → carry an umbrella (compiler insists)
→ server down, file missing, network failure

UNCHECKED → you trip over your own feet
No umbrella helps → just walk properly (fix the code)
→ null pointer, wrong index, divide by zero

Comparison

| | Checked | Unchecked |
|--------------|-------------------------------|---|
| Checked at | compile time | runtime |
| Must handle? | ✓ yes | ✗ no |
| Parent | Exception | RuntimeException |
| Cause | external / environment | programming bug |
| Examples | IOException ,
SQLException | NullPointerException ,
ArithmeticException |
| Recoverable? | usually yes | usually means fix the code |

Errors — never catch these

```
try {  
    recursion();           // infinite recursion  
} catch (StackOverflowError e) { }    // ✗ bad practice
```

Error means the JVM itself is in trouble (OutOfMemoryError , StackOverflowError). You cannot meaningfully recover, so let it crash.

Modern practice ★ (great point for a lead-level interview)

Spring and most modern frameworks prefer **unchecked** exceptions:

```
public class ResourceNotFoundException extends RuntimeException {    // unchecked  
    public ResourceNotFoundException(String msg) { super(msg); }  
}
```

Why? Checked exceptions force every caller in the chain to either catch or declare, which pollutes method signatures:

```
void a() throws SQLException { b(); }  
void b() throws SQLException { c(); }  
void c() throws SQLException { ... }      // noise all the way up
```

That is why Spring wraps `SQLException` into the unchecked `DataAccessException`.

Interview-ready answer

Checked exceptions are checked at compile time, and the compiler forces us to either handle them with try-catch or declare them with `throws`. They extend `Exception` but not `RuntimeException`, and they represent expected external problems such as a missing file or a database failure — `IOException` and `SQLException` are examples. Unchecked exceptions extend `RuntimeException` and are not verified at compile time; they usually indicate programming errors such as `NullPointerException` or `ArrayIndexOutOfBoundsException`, and the right fix is to correct the code rather than catch the exception. `Error` is a third category representing serious JVM problems like `OutOfMemoryError`, which we should not catch. In modern applications, especially with Spring, we generally prefer custom unchecked exceptions because checked exceptions force every layer to declare them and clutter the API.

● In your RoboGebra project — all 12 exceptions are UNCHECKED ★

```
common/exception/  
  APIException           ExternalApiException  
QuizConflictException  
  ResourceNotFoundException CognitoException  
EmailSendingException  
  DatabaseException       SolutionExecutionException  
RestrictedAPIAccessException  
  FirebaseMessageException JsonException  
FileNotFoundException
```

★ EVERY ONE of them extends `RuntimeException` — none is checked.

```
public class ResourceNotFoundException extends RuntimeException
{ ... }    // unchecked ★

@Getter
public class APIException extends RuntimeException {
    private final HttpStatus status;    // ★ carries the status it
    should become
}
```

■ "Every custom exception in our codebase is unchecked. That's deliberate and it's the Spring convention — a checked exception forces every layer in between to either catch it or declare it, which clutters signatures for something the controller advice is going to handle centrally anyway. **APIException** even carries its own **HttpStatus**, so the advice reads the status off the exception instead of mapping by type."

★ Note we also **wrap** third-party checked exceptions rather than propagate them — **JsonException** and **CognitoException** exist so a Jackson or AWS failure doesn't leak its own exception type into our service layer.

Easy memory

CHECKED → compiler FORCES you → outside problem → IOException, SQLException
(monsoon → carry an umbrella)

UNCHECKED → compiler is silent → your bug → NullPointerException, ArrayIndex
(you tripped → fix your walking)

ERROR → JVM is dying → don't catch → OutOfMemory, StackOverflow

Rule: under RuntimeException = unchecked. Everything else = checked.

Q: throw vs throws ?

The easiest way to remember:

`throw` → ACTUALLY throws one exception → inside the method body → one object
`throws` → WARNS that it may throw → in the method signature → many types

Note the `s` : `throws` is a **warning** (like a signboard), `throw` is the **action**.

`throw` — the action

```
void withdraw(double amount) {  
    if (amount > balance) {  
        throw new IllegalArgumentException("Insufficient balance");    //  
ACTION  
    }  
  
    balance -= amount;  
}
```

`throw` + ONE object + inside the body

`throws` — the declaration

```
void readFile() throws IOException, SQLException {    // WARNING  
    // ...  
}
```

`throws` + one or MORE class names + in the signature

It tells the caller: "be prepared, this can fail."

Both together

```
void validateAge(int age) throws InvalidAgeException {    // ← declaration  
    if (age < 18) {  
        throw new InvalidAgeException("Age must be 18+");    // ← action  
    }  
}
```

Caller side:

```
try {
    validateAge(15);
} catch (InvalidAgeException e) {
    System.out.println(e.getMessage());
}
```

Flow:

```
validateAge(15)
    ↓
    throw new InvalidAgeException ← thrown here
    ↓
    method stops immediately
    ↓
    caller's catch block           ← handled here
```

Real-world idea — a medicine bottle

throws → the WARNING LABEL on the bottle: "may cause drowsiness"
 throw → you actually FEEL drowsy

The label warns; the effect is the action.

Comparison

| | throw | throws |
|-------------|---|--|
| Purpose | actually throw it | declare the possibility |
| Where | inside the method body | in the method signature |
| Followed by | an exception object | exception class names |
| How many | exactly one | one or many (comma-separated) |
| Example | <code>throw new
IOException();</code> | <code>void m() throws
IOException</code> |

Interview traps

1. You can throw only a Throwable


```
throw new String("error");    // ✗ compile error
```

2. Unchecked exceptions do not need **throws**

```
void m() {  
    throw new RuntimeException("boom");    // ✓ no `throws` required  
}
```

3. Code after **throw** is unreachable

```
throw new RuntimeException("x");  
System.out.println("hi");    // ✗ unreachable statement – compile error
```

4. An overriding method cannot declare broader checked exceptions

```
class Parent { void m() throws IOException { } }  
  
class Child extends Parent {  
    void m() throws Exception { }    // ✗ Exception is broader than  
    IOException  
}
```

Interview-ready answer

throw is used inside a method body to actually throw a single exception object — for example `throw new IllegalArgumentException("Insufficient balance")`. **throws** is used in the method signature to declare that the method may throw one or more exception types, so the caller knows it must handle or propagate them. **throw** takes an instance, **throws** takes class names. Only checked exceptions need to be declared with **throws**; unchecked exceptions can be thrown without declaring them. Also, an overriding method cannot declare broader checked exceptions than the parent method.

Easy memory

throw → ACTION → throw new IOException(); → 1 object → in body
throws → WARNING (s) → void m() throws IOException, SQLException → in signature

Medicine bottle: LABEL = throws | actual drowsiness = throw

Q: What is try-with-resources?

The easiest way to remember:

try-with-resources = Java CLOSES the resource for you, automatically

Introduced in Java 7. It replaced the ugly old **finally** cleanup code.

The old painful way

```
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader("data.txt"));
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
} finally {
    if (br != null) {           // null check
        try {
            br.close();         // close can ITSELF throw!
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Eleven lines of noise just to close one file — and everyone forgets the null check.

The new way

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
// br.close() is called AUTOMATICALLY ✓
```

```
try ( resource )
    ↓
    use it
    ↓
block ends (normally OR by exception)
    ↓
close() is called automatically ✓
```

Real-world idea

Old way → you must remember to switch off the tap after washing your hands
New way → a sensor tap that closes by itself the moment you walk away

Multiple resources

```
try (Connection con = getConnection();
    PreparedStatement ps = con.prepareStatement(SQL);
    ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {
        System.out.println(rs.getString("name"));
    }
}
```

Closing order is **reverse** of creation — exactly what you want:

```
open : Connection → Statement → ResultSet
close: ResultSet → Statement → Connection ← reverse ✓
```

The requirement — **AutoCloseable**

Any class works as long as it implements **AutoCloseable** :

```
class MyResource implements AutoCloseable {

    void use() { System.out.println("using resource"); }

    @Override
    public void close() {
        System.out.println("resource closed automatically");
    }
}
```

```
try (MyResource r = new MyResource()) {
    r.use();
}
```

Output:

```
using resource
resource closed automatically
```

Suppressed exceptions ★ (a strong point to mention)

What if the body throws *and* `close()` throws?

| | | |
|----------------|------------------|------------------------------------|
| body throws | → "Read failed" | ← this one is reported (primary) |
| close() throws | → "Close failed" | ← this one is SUPPRESSED, not lost |

```
catch (Exception e) {
    System.out.println(e.getMessage());                // Read failed
    for (Throwable t : e.getSuppressed()) {
        System.out.println("Suppressed: " + t.getMessage()); // Close
    }
}
```

In the old `finally` style, the close exception would **overwrite** the real one and you would lose the actual cause — a classic debugging nightmare.

Java 9 improvement

```
BufferedReader br = new BufferedReader(new FileReader("data.txt"));

try (br) {           // ✓ Java 9+: use an existing effectively-final
    variable
    System.out.println(br.readLine());
}
```

Interview-ready answer

Try-with-resources was introduced in Java 7 and automatically closes any resource declared in the try parentheses, as long as the resource implements `AutoCloseable`. It replaces the old pattern of closing in a `finally` block with null checks and nested try-catch. Multiple resources can be declared, and they are closed in reverse order of creation. Another important benefit is suppressed exceptions: if the try block throws and `close()` also throws, the original exception is reported and the close exception is attached as suppressed, so the real cause is never lost — in the old style the close exception would hide the actual error. Since Java 9 we can also use an already-declared effectively-final variable inside the parentheses.

● In your RoboGebra project — try-with-resources, with TWO resources ★

File: `common/importer/ClassPathFileReader.java`

```
try (InputStream inputStream = getClass().getResourceAsStream(filePath);
    BufferedReader reader = new BufferedReader(new
InputStreamReader(inputStream))) {    // ★ two

    if (inputStream.available() <= 0) {
        throw new RuntimeException(filePath + " Not Available ");
    }
    String fileContents = readAllLines(reader);
    return new CSVReader(new StringReader(fileContents));

} catch (Exception e) {
    throw new RuntimeException(e);    // ★ the CAUSE is preserved
}
```

Both resources are closed AUTOMATICALLY, in REVERSE order: ★

```
open : InputStream → BufferedReader  
close: BufferedReader → InputStream
```

Also in `common/service/FileService.java`:

```
try (InputStream inputStream = resource.getInputStream()) { ... }  
try (OutputStream os = new FileOutputStream(tempPdfFile)) { ... }
```

■ "The CSV importer opens two resources in one try-with-resources — the stream and the reader — and both close automatically in reverse order. The old *finally* version needed a null check and a nested try just to close them, and it would have masked the real exception if *close()* also threw."

Easy memory

```
try (Resource r = ...) { } → close() runs AUTOMATICALLY ✓
```

Needs: implements `AutoCloseable`

Multiple resources → closed in REVERSE order

Body + close both throw → body wins, close becomes SUPPRESSED (not lost)

Sensor tap: closes by itself.

Q: How do you create a custom exception?

The easiest way to remember:

```
extends RuntimeException → UNCHECKED → no forced handling ★ modern  
choice  
extends Exception      → CHECKED   → caller MUST handle
```

Why create one at all?

Compare these two:

```
throw new Exception("error"); // ✗ meaningless  
throw new InsufficientBalanceException(  
    "Balance 500, requested 2000"); // ✓ tells the full  
story
```

A custom exception makes your logs and your API responses readable, and lets callers catch **exactly** the failure they care about.

Unchecked custom exception (most common)

```
public class ResourceNotFoundException extends RuntimeException {  
  
    public ResourceNotFoundException(String message) {  
        super(message);  
    }  
  
    public ResourceNotFoundException(String message, Throwable cause) {  
        super(message, cause);    // ★ keeps the original cause  
    }  
  
}
```

Usage:

```
public Employee getEmployee(Long id) {  
    return repo.findById(id)  
        .orElseThrow(() ->  
            new ResourceNotFoundException("Employee not found with id " +  
id));  
}
```

Checked custom exception

```
public class InsufficientBalanceException extends Exception {  
  
    private final double balance;  
    private final double requested;  
  
    public InsufficientBalanceException(double balance, double requested)  
    {  
        super("Balance is " + balance + " but " + requested + " was  
requested");  
        this.balance = balance;  
        this.requested = requested;  
    }  
  
    public double getShortfall() {    // extra business info ★  
        return requested - balance;  
    }  
  
}
```

The nice part: you can carry **extra data** on the exception, not just a message.

```
catch (InsufficientBalanceException e) {
    System.out.println("Add " + e.getShortfall() + " more");
}
```

Always keep the cause ★

```
try {
    jdbc.query(...);
} catch (SQLException e) {
    throw new DataAccessException("Failed to load employee " + id, e);
    //                                     ↑
    //                                     never drop this!
}
```

With cause:

DataAccessException: Failed to load employee 101
Caused by: SQLException: Connection timed out ← the REAL problem ✓

Without cause:

DataAccessException: Failed to load employee 101 ← debugging blind ▲

Real Spring Boot usage — the pattern interviewers like

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse>
    handleNotFound(ResourceNotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse(404,
e.getMessage()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAll(Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse(500, "Something went
wrong"));
    }
}
```


Service throws `ResourceNotFoundException`

↓

`@RestControllerAdvice` catches it

↓

Client receives a clean 404 JSON, not a Java stack trace ✓

Checklist for a good custom exception

- ✓ Name ends with "Exception"
- ✓ Name says WHAT went wrong (`ResourceNotFoundException`, not `MyException`)
- ✓ Provide a `(String message)` constructor
- ✓ Provide a `(String message, Throwable cause)` constructor
- ✓ Add business fields if they help the caller
- ✓ Unchecked unless the caller can genuinely recover

Interview-ready answer

A custom exception is created by extending `RuntimeException` for an unchecked exception or `Exception` for a checked one. I usually extend `RuntimeException`, because forcing every layer to declare a checked exception clutters the API — this is also the approach Spring takes. I always provide two constructors: one taking a message and one taking a message and a cause, because preserving the cause is essential for debugging. I may also add business fields, for example an `InsufficientBalanceException` carrying the balance and the requested amount. In a Spring Boot application these custom exceptions are then handled centrally in a `@RestControllerAdvice` class so the client receives a clean error response with the correct HTTP status instead of a stack trace.

● In your RoboGebra project — the real global handler ★

File: `common/exception/handler/ExceptionHandlerAdvice.java`

```

@ControllerAdvice
@Slf4j
public class ExceptionControllerAdvice {

    @ExceptionHandler(MethodArgumentNotValidException.class)           //
    ★ @Valid failures
        public ResponseEntity<Object>
handleValidationException(MethodArgumentNotValidException ex,
                                                                    WebRequest
request) {
    log.error("Validation exception: {}", ex.getMessage(), ex);    //
    ★ LOG the real cause
        String message = findErrorMessageForValidation(ex);
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                                .body(createErrorMessage(HttpStatus.BAD_REQUE
ST, message, request));
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorMessageModel>
handleResourceNotFoundException(...) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(...);
    }

    @ExceptionHandler(CognitoException.class)                        //
    ★ AWS failures, translated
    @ExceptionHandler(NoHandlerFoundException.class)                //
    ★ unmapped URLs → a clean 404
        // + MaxUploadSizeExceededException, ConstraintViolationException,
        //   MongoWriteException, InvalidOTPException,
CouponValidationException ...
    }
}

```

Every handler returns the SAME ErrorMessageModel shape ★
 → the client parses one error format, never a Java stack trace

★ And the second layer most people don't know about:

@ControllerAdvice only covers exceptions thrown INSIDE the MVC dispatch.

Anything thrown in a SECURITY FILTER happens BEFORE the DispatcherServlet,
 so the advice never sees it and the client gets a bare 500 ▲

↓

which is why we ALSO have security/FilterChainExceptionHandler ★

■ "One `@ControllerAdvice` handles the whole application and every handler returns the same error model, so clients parse one shape. It also translates third-party failures — a Cognito `UserNotFoundException` or a `MongoWriteException` becomes our error format rather than leaking AWS or Mongo types to the client. And because the advice only covers the MVC dispatch, we have a separate filter-chain handler for exceptions thrown before the `DispatcherServlet`."

Easy memory

extends `RuntimeException` → unchecked ★ (Spring style)
extends `Exception` → checked (caller must handle)

ALWAYS: `super(message)` and `super(message, cause)` ← keep the cause!
Name it after the PROBLEM: `ResourceNotFoundException`

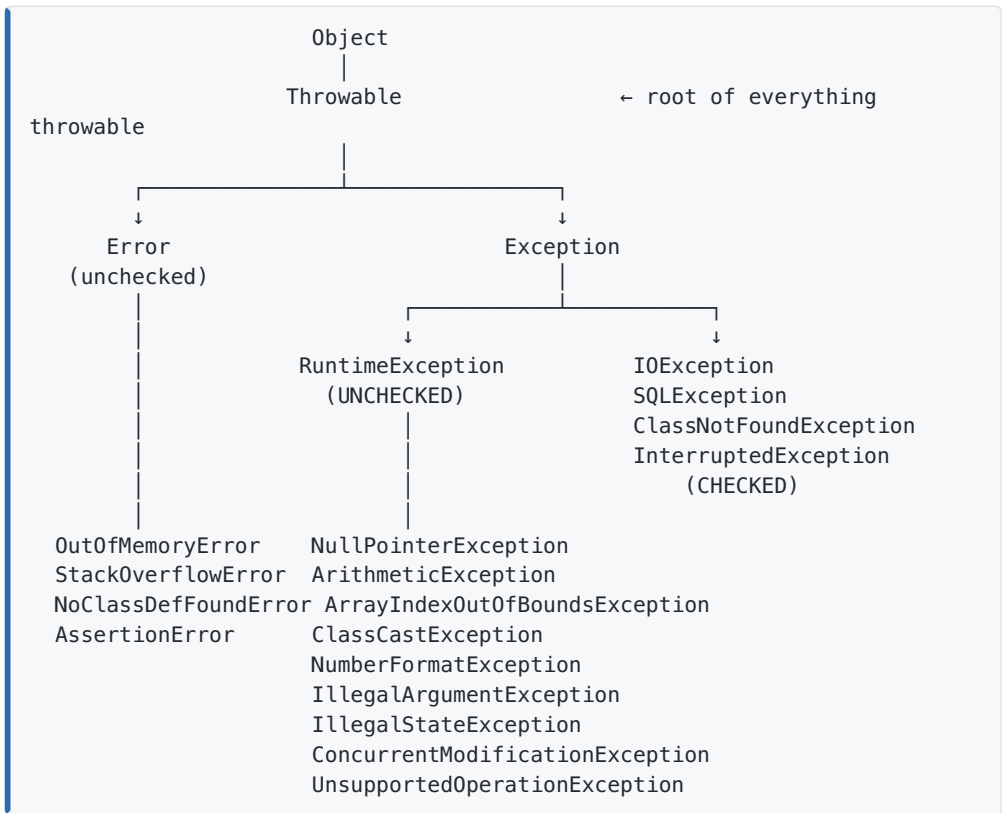
Spring: throw in service → `@RestControllerAdvice` → clean JSON error

Q: Describe the exception hierarchy.

The easiest way to remember:

```
Throwable
├── Error      → JVM problems → DON'T catch
├── Exception  → app problems → DO handle
│   ├── RuntimeException → UNCHECKED (your bug)
│   └── everything else → CHECKED (external problem)
```

Full diagram



The three groups in plain words

ERROR

- the JVM itself is broken
- `OutOfMemoryError`, `StackOverflowError`
- you cannot fix it in code → let it crash

CHECKED EXCEPTION

- the outside world failed
- file missing, DB down, network dropped
- compiler FORCES you to plan for it

UNCHECKED (RuntimeException)

- your code has a bug
- null, bad index, bad cast
- compiler stays silent → fix the code

Real-world idea — a **restaurant**:

| | | |
|-------------------|---------------------------------|----------------------|
| ERROR | → the building is on fire ▲ | → evacuate, don't |
| "handle" it | | |
| CHECKED EXCEPTION | → an ingredient is out of stock | → have a backup plan |
| UNCHECKED | → the chef forgot the salt | → fix the chef, not |
| the recipe | | |

Catch order matters ★

```
try {  
    // ...  
} catch (Exception e) {           // ✗ too broad, FIRST  
} catch (IOException e) {         // ✗ unreachable → compile error  
}
```

Always go **specific** → **general**:

```
try {  
    // ...  
} catch (FileNotFoundException e) { // most specific  
} catch (IOException e) {          // broader  
} catch (Exception e) {           // most general – last  
}
```

| | | |
|----------|---|-------------|
| Specific | ↑ | catch order |
| | | |
| General | ↓ | |

Multi-catch (Java 7)

```
try {  
    // ...  
} catch (IOException | SQLException e) { // same handling for both  
    log.error("Data access failed", e);  
}
```

The variable `e` is implicitly `final` in multi-catch.

Useful Throwable methods

```
e.getMessage();           // "File not found"
e.getCause();              // the underlying exception
e.printStackTrace();       // full trace (use a logger in production!)
e.getStackTrace();         // array of stack frames
e.getSuppressed();         // exceptions suppressed by try-with-resources
```

Interview-ready answer

Throwable is the root of the hierarchy and has two direct children, **Error** and **Exception**. **Error** represents serious JVM-level problems such as **OutOfMemoryError** and **StackOverflowError**, which an application cannot meaningfully recover from, so we should not catch them. **Exception** splits into two groups: **RuntimeException** and its subclasses, which are unchecked and usually indicate programming bugs like **NullPointerException** or **ArrayIndexOutOfBoundsException**, and all other exceptions, which are checked and represent external failures like **IOException** and **SQLException**. When writing catch blocks we must order them from the most specific to the most general, otherwise the code will not compile, and since Java 7 we can combine unrelated exception types in a single multi-catch block.

Easy memory

```
Throwable
├── Error      → JVM dying      → don't catch   → building on fire ▲
└── Exception
    ├── RuntimeException → UNCHECKED → your bug      → chef forgot salt
    └── others          → CHECKED   → outside      → out of stock
```

catch order: SPECIFIC first, GENERAL last (else compile error)

multi-catch: `catch (IOException | SQLException e)`

Q: What happens with finally and a return statement?

The easiest way to remember:

finally ALWAYS runs – even after return.
And if finally has its OWN return, it WINS (and swallows exceptions). ▲

Case 1 — return in try, print in finally

```
static int test() {  
    try {  
        return 1;  
    } finally {  
        System.out.println("finally runs");  
    }  
}
```

Output:

```
finally runs  
1
```

The return value **1** is computed and **held**, then **finally** runs, then the value is returned.

```
return 1  
  ↓  
value 1 is saved aside  
  ↓  
finally block runs  
  ↓  
NOW 1 is actually returned
```

Case 2 — return in BOTH ▲

```
static int test() {  
    try {  
        return 1;  
    } finally {  
        return 2;           // ▲ overrides!  
    }  
}
```

Output:

```
2
```

The **finally** return completely replaces the try's return.

```
try    → return 1  (discarded)
finally → return 2  ← WINS
```

Case 3 — the dangerous one: finally swallows the exception ▲

```
static int test() {
    try {
        throw new RuntimeException("Something failed!");
    } finally {
        return 99;        // ▲ exception DISAPPEARS
    }
}

System.out.println(test());    // 99 – no exception at all!
```

The exception is silently destroyed. The caller never learns anything went wrong. This is why **you should never put return inside finally** — most linters flag it as a bug.

```
try    → throws Exception ▲
finally → return 99
        ↓
        Exception is SWALLOWED – nobody ever sees it x
```

Case 4 — modifying a primitive in finally does nothing

```
static int test() {
    int x = 1;
    try {
        return x;        // the VALUE 1 is copied out right here
    } finally {
        x = 99;          // too late – the return value was already
fixed
    }
}
```

Output:

```
1
```

Because the return value was already copied. But with an **object**, mutation IS visible:


```
static List<String> test() {
    List<String> list = new ArrayList<>();
    try {
        return list;           // the REFERENCE is copied
    } finally {
        list.add("added");     // ✓ same object → visible!
    }
}

System.out.println(test());    // [added]
```

| | |
|---|-----------------------|
| Primitive → the VALUE is already copied | → change is invisible |
| Object → the REFERENCE is copied | → mutation IS visible |

This is the same "pass-by-value" idea again.

When does **finally** NOT run?

| | |
|---------------------------|----------------------|
| System.exit(0) | → JVM shuts down |
| JVM crash | → power cut, kill -9 |
| Infinite loop in try | |
| Thread killed at OS level | |

```
try {
    System.exit(0);
} finally {
    System.out.println("never printed");    // ✗
}
```

Interview-ready answer

finally always executes, even when the **try** block has a return statement — the return value is evaluated and held first, then **finally** runs, and then the value is returned. If **finally** itself contains a return statement, it overrides the try block's return, and worse, it silently swallows any exception thrown in the try block, so the caller never sees the failure. For that reason returning from **finally** is considered a bug and should be avoided. Also, modifying a primitive local inside **finally** does not change the already-captured return value, whereas mutating an object is visible because the reference points to the same object. The only

cases where **finally** does not run are **System.exit()** , a JVM crash, or the thread being killed.

Easy memory

| | | |
|---------------------------------|---|---|
| return in try, print in finally | → finally runs FIRST, then returns | ✓ |
| return in BOTH | → finally's return WINS | ▲ |
| exception + return in finally | → exception SWALLOWED | ▲ |
| never do this | | |
| primitive changed in finally | → return value unchanged (already copied) | |
| object mutated in finally | → change IS visible (same reference) | |

finally skipped only by: **System.exit()**, JVM crash

Java 8+ Features

Q: What are lambda expressions?

The easiest way to remember:

Lambda = a method WITHOUT a name, written in one line

(parameters) -> { body }

The before-and-after that explains everything

Before Java 8 — 5 lines just to say "print hello":

```
Runnable r = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Hello");  
    }  
};
```

With a lambda — 1 line:

```
Runnable r = () -> System.out.println("Hello");
```

All the ceremony (`new Runnable()`, `@Override`, `public void run()`) was noise. The only real logic was one line.

```
Anonymous class → 5 lines of boilerplate + 1 line of logic
Lambda           → 1 line of logic ✓
```

Another everyday example — sorting

```
// Before
Collections.sort(names, new Comparator<String>() {
    public int compare(String a, String b) {
        return a.compareTo(b);
    }
});

// Lambda
Collections.sort(names, (a, b) -> a.compareTo(b));

// Method reference (even shorter)
Collections.sort(names, String::compareTo);
```

Lambda syntax variations

| | |
|---|---|
| <code>() -> System.out.println("hi")</code> | <code>// no parameter</code> |
| <code>x -> x * 2</code>
<code>optional)</code> | <code>// one parameter (brackets</code> |
| <code>(x, y) -> x + y</code> | <code>// two parameters</code> |
| <code>(int x, int y) -> x + y</code>
<code>needed)</code> | <code>// explicit types (rarely</code> |
| <code>(x, y) -> {</code>
<code>return</code>
<code> int sum = x + y;</code>
<code> return sum;</code>
<code>}</code> | <code>// multi-line needs braces +</code> |

```
Single expression → no braces, no `return` (it returns automatically)
Multiple lines    → braces + explicit `return`
```

Where can a lambda be used? — functional interfaces only ★

A lambda can only be assigned to an interface that has **exactly one abstract method**.

```
@FunctionalInterface
interface Calculator {
    int operate(int a, int b);           // exactly ONE abstract method
}
```

```
Calculator add      = (a, b) -> a + b;
Calculator multiply = (a, b) -> a * b;

System.out.println(add.operate(10, 5));    // 15
System.out.println(multiply.operate(10, 5)); // 50
```

The `@FunctionalInterface` annotation is optional but useful — the compiler will error if someone later adds a second abstract method.

```
Interface with 1 abstract method → lambda works ✓
Interface with 2 abstract methods → lambda impossible ✗
```

Real-world usage you will actually write

```
// Filtering a list
employees.stream()
    .filter(e -> e.getSalary() > 50000)
    .forEach(e -> System.out.println(e.getName()));

// Sorting
employees.sort((a, b) -> a.getName().compareTo(b.getName()));

// Iterating a map
map.forEach((key, value) -> System.out.println(key + " = " + value));

// Starting a thread
new Thread(() -> System.out.println("running")).start();

// Removing elements
list.removeIf(name -> name.startsWith("A"));
```

Interview trap — variable capture

A lambda can only use local variables that are **final or effectively final**:

```
int count = 0;
Runnable r = () -> System.out.println(count);    // ✓ count is never
reassigned

count = 5;                                       // ✗ now the lambda
won't compile
```

Why? The lambda may run later, on another thread, after the method has already ended — so Java copies the value and requires it to be stable.

Instance fields have no such restriction:

```
class Test {
    int count = 0;                                // field, not a local
    Runnable r = () -> System.out.println(count);    // ✓ fine
}
```

this inside a lambda

```
class Test {
    void m() {
        Runnable anon = new Runnable() {
            public void run() {
                System.out.println(this);    // the ANONYMOUS class object
            }
        };

        Runnable lam = () -> System.out.println(this);    // the OUTER Test
        object ★
    }
}
```

A lambda does not create its own **this** — another point that shows real understanding.

Interview-ready answer

A lambda expression is an anonymous function introduced in Java 8, written as parameters, an arrow, and a body. It lets us pass behaviour as an argument

instead of writing verbose anonymous inner classes — for example `Runnable r = () -> System.out.println("Hello")` replaces five lines with one. A lambda can only be used where a functional interface is expected, meaning an interface with exactly one abstract method, such as `Runnable`, `Comparator`, `Predicate` or `Function`. Lambdas can only capture local variables that are final or effectively final, because the lambda may execute later, possibly on another thread. Unlike an anonymous inner class, a lambda does not have its own `this` — `this` refers to the enclosing instance.

Easy memory

Lambda = nameless method → (params) → body

```
() -> print("hi")           no param
x -> x * 2                   one param
(x, y) -> x + y              two params
(x, y) -> { ...; return; }   multi-line
```

Works ONLY with a FUNCTIONAL INTERFACE (exactly 1 abstract method)
Captured locals must be final / effectively final

Q: What are the built-in functional interfaces (Predicate / Function / Consumer / Supplier)?

The easiest way to remember by **what goes in and what comes out**:

| | | | | |
|----------------------------------|--------------|---------------|------------------|------------------|
| <code>Predicate<T></code> | → T in | → boolean out | → "is it true?" | → filter |
| <code>Function<T,R></code> | → T in | → R out | → "transform it" | → map |
| <code>Consumer<T></code> | → T in | → NOTHING out | → "just use it" | → forEach |
| <code>Supplier<T></code> | → nothing in | → T out | → "give me one" | → factory / lazy |

Picture it:

| | |
|------------|------------------------------|
| Predicate: | [T] → (test) → true/false |
| Function: | [T] → (apply) → [R] |
| Consumer: | [T] → (accept) → x nothing |
| Supplier: | x → (get) → [T] |

1. Predicate — asks a yes/no question

```
Predicate<Integer> isEven = n -> n % 2 == 0;

isEven.test(10);    // true
isEven.test(7);     // false
```

Used by **filter**:

```
List<Integer> evens = numbers.stream()
    .filter(n -> n % 2 == 0)
    .toList();
```

It can be combined:

```
Predicate<String> notEmpty = s -> !s.isEmpty();
Predicate<String> shortStr = s -> s.length() < 10;

Predicate<String> valid = notEmpty.and(shortStr);    // AND
Predicate<String> either = notEmpty.or(shortStr);    // OR
Predicate<String> invalid = notEmpty.negate();        // NOT
```

Real-world idea: a **security guard** — "Do you have an ID card?" Yes or no.

2. Function — converts one thing into another

```
Function<String, Integer> length = s -> s.length();

length.apply("Priya");    // 5
```

Used by **map**:

```
List<String> names = employees.stream()
    .map(e -> e.getName())    // Employee →
String
    .toList();
```

Chaining:

```
Function<Integer, Integer> doubleIt = x -> x * 2;
Function<Integer, Integer> addTen = x -> x + 10;

doubleIt.andThen(addTen).apply(5);    // (5*2)+10 = 20   → double FIRST
doubleIt.compose(addTen).apply(5);    // (5+10)*2 = 30   → addTen FIRST
```

Real-world idea: a **juicer** — apples go in, juice comes out.

3. Consumer — takes it and returns nothing

```
Consumer<String> print = s -> System.out.println(s);

print.accept("Hello");    // Hello
```

Used by `forEach`:

```
list.forEach(name -> System.out.println(name));

map.forEach((k, v) -> System.out.println(k + "=" + v));    // BiConsumer
```

Real-world idea: a **dustbin** — you put things in, nothing comes back out.

4. Supplier — gives you something, asks for nothing

```
Supplier<String> greeting = () -> "Hello World";

greeting.get();    // Hello World
```

The important real use is **lazy evaluation**:

```
// ✗ getDefault() runs even when the value EXISTS
optional.orElse(getDefault());

// ✓ getDefault() runs ONLY when the value is missing
optional.orElseGet(() -> getDefault());
```

```
orElse    → always computes the fallback (eager)
orElseGet → computes it only if needed (lazy) ★
```

Also used for exceptions:


```
repo.findById(id)
    .orElseThrow(() -> new ResourceNotFoundException("Not found"));
```

Real-world idea: a **water tap** — you don't give it anything, you just get water when you open it.

Summary table

| Interface | Method | Input | Output | Stream usage |
|--------------------------------------|-----------------------|-------|---------|---|
| <code>Predicate<T></code> | <code>test()</code> | T | boolean | <code>filter()</code> |
| <code>Function<T,R></code> | <code>apply()</code> | T | R | <code>map()</code> |
| <code>Consumer<T></code> | <code>accept()</code> | T | void | <code>forEach()</code> |
| <code>Supplier<T></code> | <code>get()</code> | — | T | <code>orElseGet()</code> ,
<code>generate()</code> |
| <code>BiFunction<T,U,R></code> | <code>apply()</code> | T, U | R | <code>reduce()</code> ,
<code>merge()</code> |
| <code>UnaryOperator<T></code> | <code>apply()</code> | T | T | <code>replaceAll()</code> |
| <code>BinaryOperator<T></code> | <code>apply()</code> | T, T | T | <code>reduce()</code> |

All four in one stream

```
List<String> result = employees.stream()
    .filter(e -> e.getSalary() > 50000)    // Predicate
    .map(e -> e.getName())                 // Function
    .peek(n -> System.out.println(n))      // Consumer
    .toList();

String value = Optional.ofNullable(input)
    .orElseGet(() -> "default");           // Supplier
```

Interview-ready answer

Java 8 added a set of standard functional interfaces in `java.util.function` so we don't have to define our own for common cases. `Predicate<T>` takes a value

and returns a boolean, and it is what `filter()` uses. `Function<T,R>` takes one type and returns another, which is what `map()` uses. `Consumer<T>` takes a value and returns nothing, used by `forEach()`. `Supplier<T>` takes nothing and returns a value, used for lazy evaluation such as `orElseGet()` and `orElseThrow()`. There are also `BiFunction` for two inputs, and `UnaryOperator` and `BinaryOperator` where the input and output types are the same. Predicates and Functions can also be combined using `and`, `or`, `negate`, `andThen` and `compose`.

Easy memory

```
Predicate → true/false → filter() → security guard: "ID card?" ✓ /✗
Function → convert → map() → juicer: apple → juice
Consumer → take only → forEach() → dustbin: goes in, nothing out
Supplier → give only → orElseGet() → tap: nothing in, water out

orElse = eager (always computes)
orElseGet = lazy (computes only when needed) ★
```

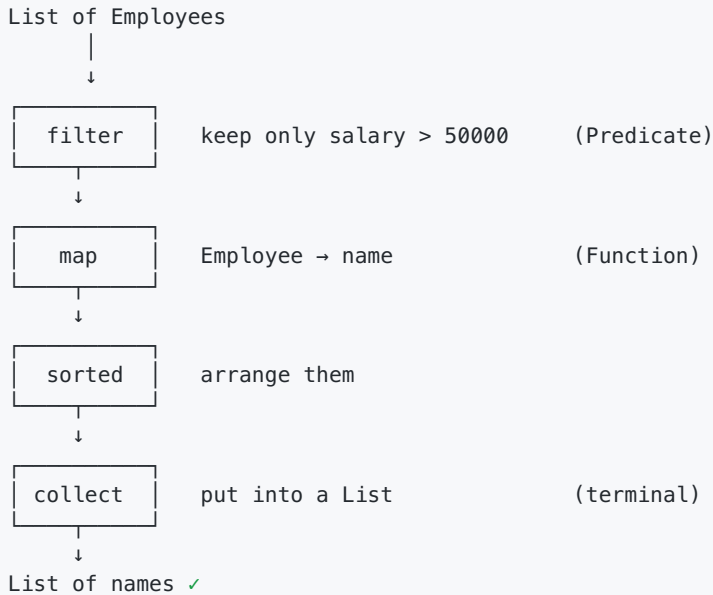
Q: Explain the Stream API with filter / map / collect / reduce examples.

The easiest way to remember:

```
Stream = a PIPELINE that processes a collection step by step

source → filter (keep some) → map (change each) → collect/reduce (final result)
```

The picture



Real-world idea: a **fruit juice factory conveyor belt**.

Basket of fruits

- filter: throw away rotten ones
- map: squeeze each into juice
- collect: fill the bottles

Nothing moves on the belt until you switch the machine on — that switch is the **terminal operation**.

Before vs after

```
// Old way
List<String> names = new ArrayList<>();
for (Employee e : employees) {
    if (e.getSalary() > 50000) {
        names.add(e.getName());
    }
}
Collections.sort(names);

// Stream way
List<String> names = employees.stream()
    .filter(e -> e.getSalary() > 50000)
    .map(Employee::getName)
    .sorted()
    .toList();
```

Same result, but the stream version reads like the requirement sentence.

filter — keep what you want

```
List<Integer> evens = numbers.stream()
    .filter(n -> n % 2 == 0)
    .toList();

// numbers = [1,2,3,4,5,6] → [2,4,6]
```

map — transform every element

```
List<String> upper = names.stream()
    .map(String::toUpperCase)
    .toList();

// [priya, ravi] → [PRIYA, RAVI]
```

flatMap flattens nested structures:

```
List<List<String>> nested = List.of(List.of("a","b"), List.of("c","d"));

List<String> flat = nested.stream()
    .flatMap(List::stream)
    .toList();           // [a, b, c, d]
```

```
map      → 1 element → 1 element
flatMap → 1 element → MANY elements, flattened into one stream
```

collect — build the final result

```
// to a List
List<String> list = stream.collect(Collectors.toList());
List<String> list2 = stream.toList(); // Java 16+ ★

// to a Set
Set<String> set = stream.collect(Collectors.toSet());

// to a Map
Map<Integer, String> map = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Employee::getName));

// joining into one String
String joined = names.stream().collect(Collectors.joining(", "));
// "Priya, Ravi, Meena"

// GROUPING — very common in interviews ★
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept));

// counting per group
Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
    Collectors.counting()));

// average salary per department
Map<String, Double> avgByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
    Collectors.averagingDouble(Employee::getSalary)));

// partition into true / false groups
Map<Boolean, List<Employee>> highLow = employees.stream()
    .collect(Collectors.partitioningBy(e -> e.getSalary() > 50000));
```

reduce — squeeze everything into ONE value

```
// sum
int sum = numbers.stream().reduce(0, (a, b) -> a + b);
int sum2 = numbers.stream().mapToInt(Integer::intValue).sum();    //
simpler ✓

// maximum
Optional<Integer> max = numbers.stream().reduce(Integer::max);

// total salary
double total = employees.stream()
                        .mapToDouble(Employee::getSalary)
                        .sum();
```

How reduce works step by step:

```
numbers = [1, 2, 3, 4]    identity = 0

0 + 1 = 1
1 + 2 = 3
3 + 3 = 6
6 + 4 = 10    ← final result
```

Other useful operations

```
.distinct()                // remove duplicates
.sorted()                  // natural order
.sorted(Comparator.comparing(Employee::getSalary).reversed())
.limit(5)                  // first 5
.skip(2)                   // skip the first 2
.count()                   // how many
.anyMatch(e -> e.getAge() > 60) // boolean
.allMatch(...) .noneMatch(...)
.findFirst()               // Optional
.max(Comparator.comparing(Employee::getSalary))
```

A realistic combined example

```
// Top 3 highest-paid employees in the IT department, as names
List<String> top3 = employees.stream()
    .filter(e -> "IT".equals(e.getDept()))
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .limit(3)
    .map(Employee::getName)
    .toList();
```

Read it top to bottom — it is literally the sentence above.

Important behaviour — lazy evaluation ★

```
employees.stream()
    .filter(e -> {
        System.out.println("filtering " + e.getName());
        return e.getSalary() > 50000;
    });
// prints NOTHING – no terminal operation!
```

Nothing runs until a terminal operation (`collect` , `forEach` , `count` , `reduce` ...) is called.

Two more rules

1. A stream can be consumed only ONCE

```
Stream<String> s = list.stream();
s.forEach(...);
s.forEach(...);           // ▲ IllegalStateException
```
2. Streams do NOT modify the source collection — they produce a new result

Interview-ready answer

The Stream API, introduced in Java 8, lets us process collections in a declarative pipeline instead of writing explicit loops. A pipeline has a source, zero or more intermediate operations like `filter` , `map` , `sorted` , `distinct` and `limit` , and one terminal operation such as `collect` , `reduce` , `forEach` or `count` . Intermediate operations are lazy — nothing executes until the terminal operation runs, which allows optimisations like short-circuiting. `filter` takes a `Predicate` and keeps matching elements, `map` takes a `Function` and transforms each

element, `collect` gathers the result into a collection, and `reduce` combines everything into a single value. `Collectors.groupingBy` is especially useful, for example grouping employees by department and counting or averaging within each group. A stream can only be consumed once and it never modifies the source collection.

● In your RoboGebra project — `groupingBy` in production ★

File: `common/service/scheduler/CleanupExpiredSessionsJob.java`

```
List<UserActiveSessionEntity> allSessions =
    userActiveSessionRepository.findAll();

// group sessions by user, so each user's sessions can be processed as a
// batch ★
Map<String, List<UserActiveSessionEntity>> sessionsByCognitoId =
    allSessions.stream()
        .collect(Collectors.groupingBy(UserActiveSessionEntity::getCognito
            Id));

for (Map.Entry<String, List<UserActiveSessionEntity>> entry :
    sessionsByCognitoId.entrySet()) {
    ...
}
```

File: `domain/institutionlearning/service/`

`InstitutionChapterLessonService.java` — the **three-argument** form:

```
Map<String, List<InstitutionChapterLessonEntity>> byLesson = rows.stream()
    .collect(Collectors.groupingBy(
        InstitutionChapterLessonEntity::getInstitutionLessonId,
        LinkedHashMap::new,                // ★ PRESERVE insertion
        order
        Collectors.toList()));
```

```
List<String> klassIds = lessonRows.stream()
    .map(InstitutionChapterLessonEntity::getKlassId)
    .filter(Objects::nonNull)                // ★ null-safe before
collecting
    ...
```


★ The `LinkedHashMap::new` is the detail worth pointing at — a plain `groupingBy` returns a `HashMap` with no ordering, so the lessons would come back in an arbitrary order and the UI would look random on every request.

■ "We use `groupingBy` a lot — the session-cleanup job groups sessions by user so each user is processed as a batch instead of one row at a time. Where the order matters we use the three-argument form with `LinkedHashMap::new`, because the default `HashMap` would scramble the lesson order between requests."

★ 847 stream usages across the backend.

Easy memory

source → filter (keep) → map (change) → sorted → collect / reduce (result)

filter → Predicate → keeps some

map → Function → changes each

flatMap → flattens nested lists

collect → toList / toSet / toMap / joining / groupingBy ★

reduce → many values → ONE value

LAZY: nothing runs until the terminal operation.

A stream is used ONCE only.

Q: What are method references?

The easiest way to remember:

Method reference = an even SHORTER lambda,
when the lambda does nothing but call one existing method.

`ClassName::methodName`

The idea in one comparison

```
list.forEach(s -> System.out.println(s));    // lambda
list.forEach(System.out::println);          // method reference ✓
```

The lambda's whole body was just "call `println` with `s`". So why repeat `s` twice?
Just point at the method.

```
s -> System.out.println(s)
      ↓ same thing
System.out::println
```

The 4 types

1. Static method → `ClassName::staticMethod`

```
list.stream().map(s -> Integer.parseInt(s)); // lambda
list.stream().map(Integer::parseInt);        // reference ✓
```

2. Instance method of a particular object → `object::method`

```
Printer p = new Printer();

list.forEach(s -> p.print(s)); // lambda
list.forEach(p::print);        // reference ✓
```

3. Instance method of an arbitrary object of a type →

`ClassName::instanceMethod` ★

This is the confusing one, so look carefully:

```
list.stream().map(s -> s.toUpperCase()); // lambda
list.stream().map(String::toUpperCase); // reference ✓
```

Here `String` is the **type**, and the method is called **on each element**.

```
s -> s.toUpperCase()
    ↑
    the parameter itself becomes the receiver

String::toUpperCase
```

Same idea with your own class:

```
employees.stream().map(Employee::getName); // e -> e.getName()
employees.sort(Comparator.comparing(Employee::getSalary));
```

4. Constructor → `ClassName::new`

```
Supplier<ArrayList<String>> s = ArrayList::new;    // () -> new
ArrayList<>()

list.stream()
    .map(Employee::new)                          // name -> new
Employee(name)
    .toList();

stream.collect(Collectors.toCollection(TreeSet::new));
```

Cheat sheet

| Type | Syntax | Lambda equivalent |
|-------------------------------|----------------------------------|--|
| Static | <code>Integer::parseInt</code> | <code>s -> Integer.parseInt(s)</code> |
| Instance of a specific object | <code>System.out::println</code> | <code>s -> System.out.println(s)</code> |
| Instance of the parameter | <code>String::toUpperCase</code> | <code>s -> s.toUpperCase()</code> |
| Constructor | <code>Employee::new</code> | <code>n -> new Employee(n)</code> |

When you CANNOT use it

```
// ✗ extra logic → keep the lambda
list.stream().map(s -> s.toUpperCase() + "!");

// ✗ multiple statements
list.forEach(s -> { validate(s); save(s); });

// ✗ argument order changed
list.stream().map(s -> compare(b, s));
```

Rule: if the lambda body is **exactly one existing method call with the same arguments**, use a method reference. Otherwise keep the lambda.

Interview-ready answer

A method reference is shorthand for a lambda whose body only calls one existing method. There are four kinds: a reference to a static method like `Integer::parseInt`, a reference to an instance method of a particular object like

`System.out::println`, a reference to an instance method of an arbitrary object of a type like `String::toUpperCase` where the stream element becomes the receiver, and a constructor reference like `Employee::new`. They make stream pipelines more readable — for example `map(Employee::getName)` instead of `map(e -> e.getName())`. If the lambda does anything more than a single method call with the same arguments, we keep the lambda instead.

Easy memory

Lambda body = ONE existing method call? → use a method reference

| | |
|----------------------------------|--|
| <code>Integer::parseInt</code> | static |
| <code>System.out::println</code> | specific object |
| <code>String::toUpperCase</code> | the parameter itself is the receiver ★ |
| <code>Employee::new</code> | constructor |

```
s -> System.out.println(s)    ≡    System.out::println
e -> e.getName()              ≡    Employee::getName
```

Q: What is Optional and why use it?

The easiest way to remember:

Optional = a BOX that may contain a value, or may be empty.

It forces the caller to think about "what if there is nothing?"

The problem it solves

```
Employee e = repo.findById(101);
System.out.println(e.getName());           // ▲ NullPointerException if not
found
```

`NullPointerException` is famously called the "billion dollar mistake". The method signature `Employee findById(...)` tells you **nothing** about the possibility of null — so callers forget to check.

```
Optional<Employee> e = repo.findById(101); // ★ the signature itself
warns you
```

Now the compiler makes you deal with the empty case.

Real-world idea: a **gift box**.



`Optional.of(x)`



`Optional.empty()`

Either way, you receive a BOX – you must open it to find out.

Creating an Optional

```
Optional<String> a = Optional.of("Priya");           // never null, else NPE
Optional<String> b = Optional.ofNullable(maybeNull); // null → empty ✓
Optional<String> c = Optional.empty();               // deliberately empty
```

Using it – the RIGHT way ★

```
// 1. default value
String name = optional.orElse("Unknown");

// 2. lazy default (computed only when empty)
String name = optional.orElseGet(() → loadDefault());

// 3. throw a meaningful exception ← most common in Spring
Employee e = repo.findById(id)
    .orElseThrow(() → new ResourceNotFoundException("Employee " + id + "
not found"));

// 4. do something only if present
optional.ifPresent(emp → System.out.println(emp.getName()));

// 5. if present ... else ... (Java 9)
optional.ifPresentOrElse(
    emp → System.out.println(emp.getName()),
    () → System.out.println("Not found"));

// 6. transform safely
String name = optional.map(Employee::getName).orElse("Unknown");

// 7. filter
optional.filter(emp → emp.getSalary() > 50000)
    .ifPresent(emp → System.out.println("High earner"));
```

The `map` chain is the real power — no null checks anywhere:

```
String city = Optional.ofNullable(employee)
    .map(Employee::getAddress)
    .map(Address::getCity)
    .orElse("Unknown");
```

Compare with the old code:

```
String city = "Unknown";
if (employee != null) {
    if (employee.getAddress() != null) {
        if (employee.getAddress().getCity() != null) {
            city = employee.getAddress().getCity();
        }
    }
}
```

Three nested ifs become one readable chain.

The **WRONG** way ❌

```
if (optional.isPresent()) {
    System.out.println(optional.get());    // ❌ you just recreated the
    null check
}
```

This gains nothing over `if (x != null)`. Use `ifPresent`, `map`, `orElse`, `orElseThrow` instead.

```
optional.get();    // ❌ never call this without checking – throws
NoSuchElementException
```

Where NOT to use Optional ★

```
class Employee {  
    private Optional<String> name;           // ✗ never as a field (not  
    serializable, wasteful)  
}  
  
void process(Optional<String> name) {       // ✗ never as a method parameter  
}  
  
Optional<List<String>> getNames();          // ✗ return an EMPTY LIST instead
```

Correct usage:

- ✓ As a RETURN TYPE when a value may legitimately be absent
- ✗ As a field
- ✗ As a method parameter
- ✗ For collections – return an empty list instead

Real Spring Data usage

```
public interface EmployeeRepository extends JpaRepository<Employee, Long>  
{  
    Optional<Employee> findByEmail(String email);  
}  
  
public Employee getByEmail(String email) {  
    return repo.findByEmail(email)  
        .orElseThrow(() -> new ResourceNotFoundException("No user:  
" + email));  
}
```

Interview-ready answer

Optional is a container introduced in Java 8 that may or may not hold a non-null value. Its purpose is to make the absence of a value explicit in the method signature so callers cannot silently forget a null check — this is why Spring Data repositories return **Optional<Employee>** from **findById**. We create it with **Optional.of**, **ofNullable** or **empty**, and consume it with **orElse**, **orElseGet**, **orElseThrow**, **ifPresent** and **map**. The **map** chain is especially useful for nested objects, replacing several nested null checks with one readable chain. The anti-pattern is calling **isPresent()** followed by **get()**, because that

is just a null check again. Optional is intended as a return type, not as a field or a method parameter, and for collections we return an empty collection rather than an Optional.

● In your RoboGebra project — `orElseThrow` everywhere ★

```
// domain/solutionstepfeedback/service/SolutionStepFeedbackService.java
SolutionStepFeedback feedback =
repository.findById(request.getSolutionStepFeedbackId())
    .orElseThrow(() -> new ResourceNotFoundException(
        "Solution Step Feedback",
request.getSolutionStepFeedbackId())); // ★

// security/GoogleAuthSuccessHandler.java
User user = userRepository.findById(googleId)
    .orElseThrow(() -> new RuntimeException("User not found after
OAuth: " + googleId));

// common/service/FileService.java
String extension = extractExtension(fileName)
    .orElseThrow(() -> new IllegalArgumentException("Invalid file name
or extension."));
```

The pattern, everywhere:

```
repository.findById(id)          → returns Optional<T> ★ (Spring Data)
    .orElseThrow(() -> new ResourceNotFoundException("Exercise", id))
                                ↓
    caught by @ControllerAdvice → a clean 404 with our error model ★
```

■ "Spring Data repositories return *Optional*, and we always resolve it with `orElseThrow` and a domain exception rather than `.get()` or an `isPresent()` check. That gives one line at the call site and a correct 404 from the controller advice — the absence of a record is handled once, at the boundary, instead of null-checked in every service."

★ Note the **lazy** supplier form: `orElseThrow(() -> new ...)` builds the exception **only when the value is missing**. `orElseThrow(new ...)` would construct it every single time.

Easy memory

Optional = a gift BOX: may hold a value, may be empty

CREATE : of() | ofNullable() ★ | empty()

USE : orElse() | orElseGet() | orElseThrow() ★ | ifPresent() | map()

- ✗ isPresent() + get() → that's just a null check again
- ✗ field / parameter → RETURN TYPE only
- ✗ Optional<List> → return an empty list instead

Q: Why were default methods added in Java 8?

The easiest way to remember:

To add NEW methods to old interfaces WITHOUT breaking the millions of classes that already implement them.

The exact problem Java faced

Java 8 wanted to add `stream()` to every collection:

```
list.stream()...  
set.stream()...
```

`stream()` had to go into the `Collection` interface. But:

```
interface Collection<E> {  
    Stream<E> stream();           // ✗ new ABSTRACT method  
}
```

Collection is implemented by:

ArrayList, LinkedList, HashSet, TreeSet, Vector, ...
+ thousands of custom classes in every company's codebase
↓
ALL of them would fail to compile ▲

That is called "breaking backward compatibility" — completely unacceptable for Java.

The solution

```
interface Collection<E> {  
    default Stream<E> stream() {                // ✓ has a BODY  
        return StreamSupport.stream(spliterator(), false);  
    }  
}
```

Old classes → inherit the default implementation → still compile ✓
New classes → can override it if they want ✓

That single feature is what made the whole Stream API possible.

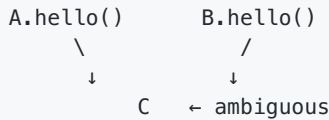
Simple demonstration

```
interface Payment {  
    void pay(double amount);                // abstract  
  
    default void receipt() {                // NEW method, safe to add  
        System.out.println("Receipt generated");  
    }  
}  
  
class UpiPayment implements Payment {  
    public void pay(double amount) {  
        System.out.println("UPI " + amount);  
    }  
    // did not write receipt() – inherits it for free ✓  
}
```

Real-world idea: a **mobile network operator** adds a new free service (caller tunes). Existing customers get it automatically without changing their SIM card. Anyone who wants a custom version can configure their own.

The diamond problem it creates

```
interface A { default void hello() { System.out.println("A"); } }  
interface B { default void hello() { System.out.println("B"); } }  
  
class C implements A, B { }                // ✗ compile error: which hello()?
```



You must resolve it:

```
class C implements A, B {
    public void hello() {
        A.super.hello();    // pick one explicitly ★
    }
}
```

Resolution rules (nice to know)

1. The CLASS wins over any interface
2. The more SPECIFIC interface wins over its parent interface
3. Otherwise → you must override and choose

Does this mean Java now has multiple inheritance?

| | | |
|-----------------------------------|-----|---|
| Multiple inheritance of BEHAVIOUR | → ✓ | yes (default methods have bodies) |
| Multiple inheritance of STATE | → ✗ | no (interfaces still have no instance fields) |

That is exactly why the classic diamond problem is still avoided — there is no duplicated state, only a method-choice question that the compiler makes you answer.

Interview-ready answer

Default methods were added in Java 8 to allow interfaces to evolve without breaking existing implementations. Java wanted to add `stream()` to the `Collection` interface, but adding an abstract method would have broken every class implementing `Collection`, including thousands of custom classes. By giving the method a default implementation in the interface, existing classes inherit it automatically and still compile, while new classes can override it. The trade-off is the diamond problem: if a class implements two interfaces that declare the same default method, the compiler reports an ambiguity and the class must override the method and pick one using `InterfaceName.super.method()`. Java allows

multiple inheritance of behaviour this way, but still not of state, since interfaces cannot have instance fields.

Easy memory

Reason: add `stream()` to `Collection` WITHOUT breaking existing classes

New ABSTRACT method → all implementers break ▲

New DEFAULT method → all implementers inherit it ✓

Diamond: `A.hello()` + `B.hello()` → `C` must override → `A.super.hello()`

Behaviour inheritance ✓ | State inheritance ✗

Q: Intermediate vs terminal stream operations?

The easiest way to remember:

Intermediate → returns a `STREAM` → LAZY → nothing happens yet

Terminal → returns a `RESULT` → EAGER → the whole pipeline RUNS

The picture

```
list.stream()
  .filter(...)    ← intermediate (just recorded)
  .map(...)       ← intermediate (just recorded)
  .sorted()       ← intermediate (just recorded)
  .collect(...)   ← TERMINAL ▲ everything actually executes NOW
```

Real-world idea: a **washing machine**.

```
Put in clothes      }
Add detergent       } ← setting knobs (intermediate)
Select the wash cycle }
Press START         ← terminal – only now does anything happen ▲
```

You can turn the knobs all day; without `START` nothing washes.

Proof of laziness

```
List<String> names = List.of("Priya", "Ravi", "Meena");

names.stream()
    .filter(n -> {
        System.out.println("filtering " + n);
        return n.startsWith("P");
    });

System.out.println("Done");
```

Output:

```
Done
```

Not a single "filtering" line printed — there is no terminal operation.

Now add one:

```
names.stream()
    .filter(n -> { System.out.println("filtering " + n); return
n.startsWith("P"); })
    .toList();
```

Output:

```
filtering Priya
filtering Ravi
filtering Meena
```

Short-circuiting — why laziness is powerful ★

```
List<String> names = List.of("Priya", "Ravi", "Meena", "Arun");

Optional<String> first = names.stream()
    .filter(n -> { System.out.println("checking " + n); return
n.startsWith("R"); })
    .findFirst();
```

Output:

checking Priya
checking Ravi

← STOPS here! Meena and Arun are never checked ✓

It processes **element by element**, not stage by stage. A **for** loop with a **break** does the same thing, but the stream does it automatically.

Priya → filter → no
Ravi → filter → YES → findFirst is satisfied → STOP ▲
Meena → never touched
Arun → never touched

The lists

Intermediate (lazy, return a Stream):

| | |
|---------------------------|-------------------------------------|
| filter() | keep matching elements |
| map() | transform each element |
| flatMap() | flatten nested structures |
| distinct() | remove duplicates |
| sorted() | sort |
| peek() | look at elements (debugging) |
| limit(n) | keep the first n ← short-circuiting |
| skip(n) | drop the first n |
| takeWhile() / dropWhile() | (Java 9) |

Terminal (eager, return a result):

| | |
|---------------------------------------|------------------------|
| collect() | build a collection |
| toList() | Java 16+ |
| forEach() | do something with each |
| reduce() | combine into one value |
| count() | how many |
| min() / max() | Optional |
| findFirst() / findAny() | ← short-circuiting |
| anyMatch() / allMatch() / noneMatch() | ← short-circuiting |
| sum() / average() | (on IntStream etc.) |

The "stream is used once" rule

```
Stream<String> s = names.stream();

s.forEach(System.out::println);    // ✓ works
s.forEach(System.out::println);    // ▲ IllegalStateException:
                                   //   stream has already been
                                   //   operated upon
```

After the terminal operation the stream is closed. Create a fresh one if you need it again.

Interview-ready answer

Intermediate operations such as `filter`, `map`, `sorted`, `distinct` and `limit` return another stream, so they can be chained, and they are lazy — they simply build the pipeline and do not execute anything. Terminal operations such as `collect`, `forEach`, `reduce`, `count`, `findFirst` and `anyMatch` return a result rather than a stream and trigger the execution of the entire pipeline. Because of laziness, streams process elements one at a time through the whole chain and can short-circuit: `findFirst` stops as soon as a match is found, so the remaining elements are never evaluated. A stream can only be consumed once — calling a second terminal operation on the same stream throws `IllegalStateException`.

Easy memory

Intermediate → returns a STREAM → LAZY → `filter`, `map`, `sorted`, `distinct`, `limit`
Terminal → returns a RESULT → RUNS → `collect`, `forEach`, `count`, `reduce`, `findFirst`

Washing machine: turning the knobs = intermediate | pressing START = terminal ▲

Short-circuit: `findFirst` / `anyMatch` / `limit` stop early
One stream = ONE terminal operation only

Multithreading

A much deeper version of this section (thread pools, `CompletableFuture`, atomic classes, producer-consumer) is in [32-multithreading.md](#).

Q: Thread vs Runnable — which is better?

The easiest way to remember:

Runnable is better ✓

extends `Thread` → uses up your ONE inheritance slot
implements `Runnable` → keeps it free, and separates the TASK from the WORKER

Two ways to create a thread

```
// Way 1 – extend Thread
class MyThread extends Thread {
    public void run() {
        System.out.println("Running: " +
            Thread.currentThread().getName());
    }
}
new MyThread().start();

// Way 2 – implement Runnable ★
class MyTask implements Runnable {
    public void run() {
        System.out.println("Running: " +
            Thread.currentThread().getName());
    }
}
new Thread(new MyTask()).start();

// Way 2 with a lambda (Runnable is a functional interface)
new Thread(() -> System.out.println("Running")).start();
```

Why is Runnable better? (3 reasons)

Reason 1 — Java allows only ONE parent class


```
class MyThread extends Thread { }           // slot used up

class MyService extends BaseService
    extends Thread { }                       // ✗ impossible
```

But:

```
class MyService extends BaseService implements Runnable { } // ✓ both!
```

Reason 2 — separation of concerns

```
Thread    = the WORKER (who runs it)
Runnable  = the TASK   (what to run)
```

Real-world idea: a **delivery company**.

```
extends Thread    → every parcel is its own delivery boy (wasteful)
implements Runnable → parcels (tasks) are given to a pool of delivery boys
(threads)
```

Reason 3 — reusable with thread pools ★ (the strongest reason)

```
ExecutorService pool = Executors.newFixedThreadPool(5);

pool.submit(new MyTask());           // ✓ pools accept Runnable/Callable
pool.submit(() -> doWork());
```

Thread pools cannot accept a **Thread** subclass as a task. In real Spring Boot applications you almost never write `new Thread()` — you submit tasks to an executor. So **Runnable** is the only practical choice.

The #1 interview trap — **start()** vs **run()** ★★

```
MyThread t = new MyThread();

t.run();           // ✗ just a normal method call — runs on the MAIN thread!
t.start();         // ✓ creates a NEW thread, which then calls run()
```

```

public static void main(String[] args) {
    Thread t = new Thread() ->
        System.out.println("Inside: " +
            Thread.currentThread().getName());

    t.run();    // Inside: main        ← no new thread at all
    t.start();  // Inside: Thread-0    ← real thread ✓
}

```

```

start()
↓
JVM creates a NEW thread stack
↓
the new thread calls run()

run()
↓
just a plain method call on the CURRENT thread ✗

```

Second trap — calling `start()` twice

```

t.start();
t.start();    // ▲ IllegalArgumentException

```

A thread cannot be restarted once it has finished.

Comparison

| | extends Thread | implements Runnable |
|-------------------------|-------------------------|-----------------------------|
| Inheritance slot | used up ✗ | free ✓ |
| Multiple interfaces | limited | ✓ |
| Works with thread pools | ✗ | ✓ |
| Lambda-friendly | ✗ | ✓ |
| Design | task = thread (coupled) | task separate from thread ✓ |
| Recommended | ✗ | ✓ |

Interview-ready answer

There are two ways to create a thread: extending the `Thread` class or implementing `Runnable`. Implementing `Runnable` is preferred for three reasons. First, Java supports only single inheritance, so extending `Thread` consumes the one available parent class, whereas a class can implement `Runnable` along with extending something else. Second, it separates the task from the worker — `Runnable` represents what to do and `Thread` represents who runs it. Third and most importantly, `Runnable` works with `ExecutorService` and thread pools, which is how threading is actually done in production, since creating raw threads is expensive. Also, `Runnable` is a functional interface so it can be written as a lambda. An important point is that we must call `start()` and not `run()` — calling `run()` directly executes the code on the current thread without creating a new one.

Easy memory

```
implements Runnable ★ → inheritance slot FREE + works with thread pools
+ lambda
extends Thread      ✗ → wastes the single-inheritance slot

start() → NEW thread ✓          run() → same thread ✗
start() twice → IllegalStateException ▲

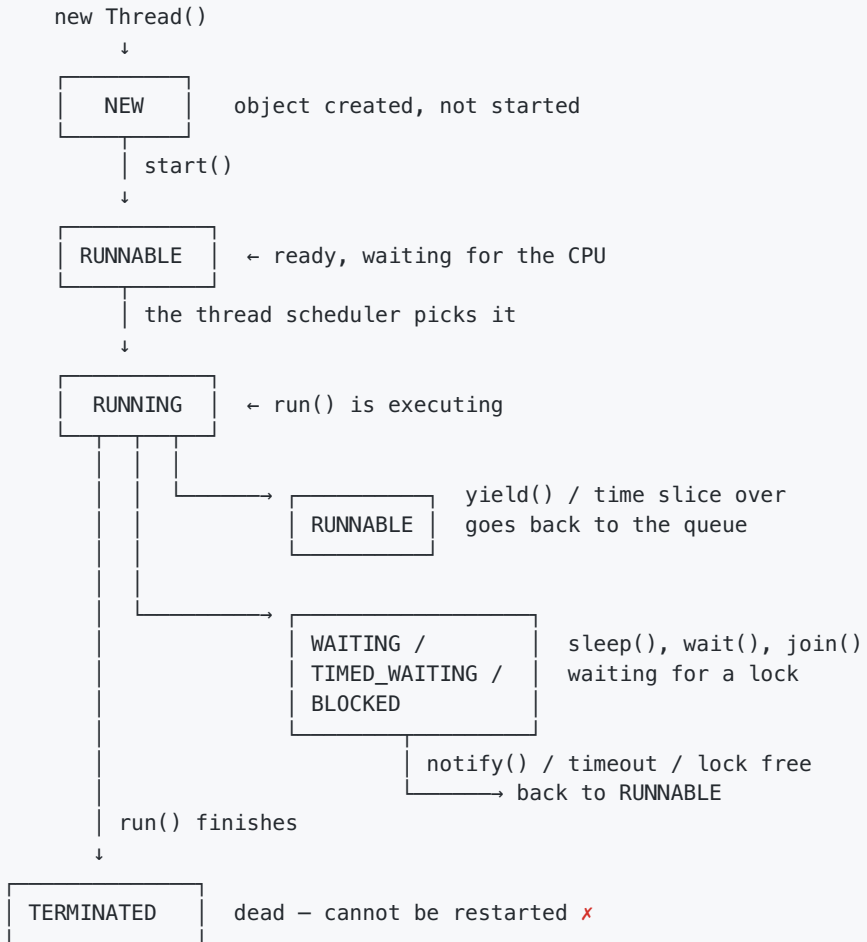
Thread = worker | Runnable = task
```

Q: What is the thread lifecycle?

The easiest way to remember the 5 states:

```
NEW → RUNNABLE → RUNNING → WAITING/BLOCKED → TERMINATED
```

The diagram



The states in plain words

| | |
|---------------|---|
| NEW | → <code>Thread t = new Thread();</code> (created, not started) |
| RUNNABLE | → <code>t.start()</code> called; ready and waiting for CPU time |
| RUNNING | → the CPU picked it; <code>run()</code> is executing |
| BLOCKED | → waiting to ENTER a synchronized block (lock held by someone else) |
| WAITING | → <code>wait()</code> , <code>join()</code> with no timeout – waits indefinitely |
| TIMED_WAITING | → <code>sleep(1000)</code> , <code>wait(1000)</code> , <code>join(1000)</code> – waits for a fixed time |
| TERMINATED | → <code>run()</code> finished, or an exception killed it |

Note: officially the JVM has six `Thread.State` values — `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, `TERMINATED`. "RUNNING" is a sub-state of `RUNNABLE` — the JVM does not distinguish "ready" from "actually on the CPU". Mentioning this scores points.

Real-world idea — a doctor's clinic

| | |
|----------------------------|--|
| <code>NEW</code> | → you decide to visit the doctor (not left home yet) |
| <code>RUNNABLE</code> | → you're in the waiting room with a token |
| <code>RUNNING</code> | → you're inside, being examined |
| <code>BLOCKED</code> | → the room is occupied by another patient |
| <code>TIMED_WAITING</code> | → nurse says "come back in 10 minutes" |
| <code>WAITING</code> | → "wait until we call your name" (no fixed time) |
| <code>TERMINATED</code> | → consultation over, you go home (you can't reuse the token) |

Seeing states in code

```
Thread t = new Thread(() -> {
    try { Thread.sleep(1000); } catch (InterruptedException e) { }
});

System.out.println(t.getState());    // NEW

t.start();
System.out.println(t.getState());    // RUNNABLE

Thread.sleep(100);
System.out.println(t.getState());    // TIMED_WAITING

t.join();
System.out.println(t.getState());    // TERMINATED
```

BLOCKED vs WAITING ★ (favourite follow-up)

`BLOCKED` → wants to ENTER a synchronized block, but another thread holds the lock

- wakes up automatically when the lock is released
- the thread did NOT choose this

`WAITING` → called `wait()` / `join()` and voluntarily gave up the lock

- needs `notify()` / `notifyAll()` to wake up
- the thread CHOSE to wait

sleep() vs wait() — another classic ★

| | sleep() | wait() |
|------------------------------|-------------------------|-------------------------------------|
| Class | Thread | Object |
| Releases the lock? | ✗ NO (keeps holding it) | ✓ YES |
| Wakes up by | timeout | notify() /
notifyAll() / timeout |
| Must be inside synchronized? | ✗ no | ✓ yes |
| Used for | pausing | inter-thread communication |

The key point: **sleep()** keeps the lock, **wait()** releases it.

Interview-ready answer

A thread starts in the NEW state when the object is created. Calling **start()** moves it to RUNNABLE, where it waits for the thread scheduler to give it CPU time; when scheduled it is effectively running. From there it can move to BLOCKED if it is waiting to acquire a lock held by another thread, to WAITING if it calls **wait()** or **join()** with no timeout, or to TIMED_WAITING if it calls **sleep()** or a timed **wait()**. When those conditions end it returns to RUNNABLE. Once **run()** completes or an uncaught exception occurs, the thread becomes TERMINATED and cannot be restarted — calling **start()** again throws **IllegalThreadStateException**. Technically the JVM defines six states, with RUNNING being part of RUNNABLE.

Easy memory

NEW → RUNNABLE → RUNNING → (BLOCKED / WAITING / TIMED_WAITING) → TERMINATED

BLOCKED → waiting for a LOCK (forced)

WAITING → called wait()/join() (chosen)

sleep() → keeps the lock ◆

wait() → releases it ◆

TERMINATED is final – a thread can never be restarted.

Q: What does **synchronized** do?

The easiest way to remember:

synchronized = only ONE thread at a time can be inside.

It gives MUTUAL EXCLUSION (one at a time)
and VISIBILITY (changes become visible to other threads).

The problem it solves — race condition

```
class Counter {  
    int count = 0;  
  
    void increment() {  
        count++;           // NOT atomic!  
    }  
}
```

Run it with 1000 threads and you expect 1000. You get 987, or 993 — a different wrong number every time.

Why? **count++** is actually **three** operations:

1. READ count from memory (say 5)
2. ADD 1 (6)
3. WRITE back to memory (6)

Two threads can interleave:

```
Thread A: reads 5  
Thread B: reads 5            ← both read the SAME value  
Thread A: writes 6  
Thread B: writes 6           ← one increment is LOST ▲  
  
Expected 7, got 6
```

The fix

```
class Counter {  
    private int count = 0;  
  
    synchronized void increment() {        // one thread at a time ✓  
        count++;  
    }  
}
```

Thread A → ♦ acquires the lock → count 5→6 → ♦ releases
Thread B → waits... → ♦ → count 6→7 → ♦

Result is always correct ✓

Real-world idea: a **single toilet with one key**.

Person A takes the key ♦ → goes in → comes out → returns the key
Person B waits outside → then takes the key

Nobody can be inside at the same time.

Three ways to use it

1. Synchronized method — locks the whole object (`this`):

```
synchronized void increment() {  
    count++;  
}
```

2. Synchronized block — locks only what you need ★ better:

```
void process() {  
    readConfig();                // no lock needed – stays parallel  
  
    synchronized (this) {        // lock ONLY the critical part  
        count++;  
    }  
  
    sendEmail();                 // no lock needed  
}
```


Method-level → locks the ENTIRE method → slower
Block-level → locks only the risky lines → faster ▲

3. Static synchronized — locks the CLASS, not the object:

```
static synchronized void update() {    // lock is Counter.class
    staticCount++;
}
```

Important difference:

```
Counter c1 = new Counter();
Counter c2 = new Counter();

c1.increment();    // locks c1
c2.increment();    // locks c2 → these run in PARALLEL (different locks!)

Counter.update(); // locks the CLASS → all objects share this one lock
```

The two guarantees ★

1. MUTUAL EXCLUSION → only one thread inside at a time
2. VISIBILITY → on release, changes are flushed to main memory, so the next thread sees the latest values

The second one is often forgotten but is just as important.

Common mistakes

```
// ✗ Locking on a String literal – it's pooled and SHARED globally!
synchronized ("lock") { }

// ✗ Locking on a changing reference
private Integer lock = 0;
synchronized (lock) { lock++; }    // the lock object CHANGES → broken

// ✓ Correct – a dedicated, final, private lock object
private final Object lock = new Object();
synchronized (lock) { }
```

Performance note

```
// ✗ Whole method locked for 5 seconds
synchronized void process() {
    callSlowApi();           // 5 seconds
    count++;                 // 1 microsecond
}

// ✓ Lock only the microsecond
void process() {
    callSlowApi();
    synchronized (lock) { count++; }
}
```

Modern alternatives:

```
AtomicInteger count = new AtomicInteger();
count.incrementAndGet();           // lock-free, uses CAS – faster ▲

ConcurrentHashMap           // instead of synchronized HashMap
ReentrantLock               // when you need tryLock / timeout /
fairness
```

Interview-ready answer

synchronized ensures that only one thread at a time can execute a block of code guarded by a particular lock. It solves race conditions — for example **count++** is not atomic because it is a read, an increment and a write, so two threads can read the same value and one update is lost. **synchronized** gives two guarantees: mutual exclusion, so only one thread is inside at a time, and visibility, so when a thread releases the lock its changes are flushed to main memory and become visible to the next thread. A synchronized instance method locks the object, a static synchronized method locks the class object, and a synchronized block lets us lock only the critical section, which performs much better. We should always lock on a private final object, never on a String literal or a changing reference. For simple counters **AtomicInteger** is a faster lock-free alternative, and **ReentrantLock** is used when we need **tryLock** or timeouts.

Easy memory

synchronized = ONE key to ONE toilet ♦ → one person at a time

Gives: 1. MUTUAL EXCLUSION 2. VISIBILITY

instance method → locks THIS object

static method → locks the CLASS

block → locks only the critical lines ★ (fastest)

count++ is NOT atomic → read + add + write

Lock on: private final Object lock = new Object(); ✓

Never on: a String literal or a changing reference ✗

Q: What is the **volatile** keyword?

The easiest way to remember:

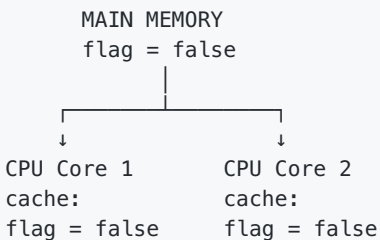
volatile = "always read from MAIN memory, never from the CPU cache"

It guarantees VISIBILITY.

It does NOT guarantee ATOMICITY. ▲

The problem — CPU caching

Each CPU core has its own cache. A thread may read a stale copy forever.



Thread A sets flag = true → writes to ITS cache

Thread B keeps reading its OWN cache → still sees false → INFINITE LOOP ▲

The famous broken code:

```

class Task implements Runnable {

    private boolean running = true;           // ✗ not volatile

    public void run() {
        while (running) {                     // may loop FOREVER
            // work
        }
        System.out.println("Stopped");
    }

    public void stop() {
        running = false;                      // another thread may never see
this
    }
}

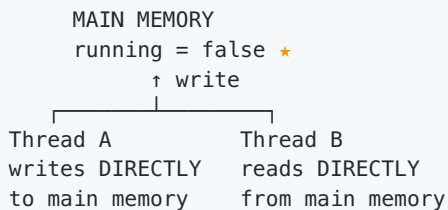
```

The fix is one word:

```

private volatile boolean running = true;     // ✓

```



No caching → the change is seen immediately ✓

Real-world idea: a **shared office whiteboard**.

Without volatile → everyone keeps a personal notepad copy → they go stale
 With volatile → everyone must look at the whiteboard every time ✓

The big trap — volatile does NOT make things atomic ★★

```

volatile int count = 0;

void increment() {
    count++;           // ✗ STILL BROKEN with 1000 threads
}

```

Why? `count++` is read + add + write — three steps. `volatile` guarantees each read and each write sees main memory, but another thread can still slip in between the read and the write.

```
Thread A: reads 5 (from main memory ✓ )
Thread B: reads 5 (from main memory ✓ )
Thread A: writes 6
Thread B: writes 6           ← one increment still LOST ▲
```

```
volatile → fixes VISIBILITY ✓
volatile → does NOT fix ATOMICITY ✗
```

For a counter, use:

```
AtomicInteger count = new AtomicInteger();
count.incrementAndGet();           // ✓ atomic AND visible
```

or `synchronized` .

When IS volatile enough?

- ✓ A simple flag written by one thread, read by others (running = false)
- ✓ A reference published after being fully constructed
- ✓ Where only visibility matters, not read-modify-write
- ✗ Counters, or anything of the form `x = x + 1`
- ✗ Anything needing several fields updated together

volatile vs synchronized

| | volatile | synchronized |
|-----------------|----------------|--------------------|
| Visibility | ✓ | ✓ |
| Atomicity | ✗ | ✓ |
| Blocks threads? | ✗ never | ✓ yes |
| Performance | very fast | slower |
| Works on | variables only | methods and blocks |
| Use for | flags | compound actions |

Double-checked locking — the classic use

```
class Singleton {  
  
    private static volatile Singleton instance;    // volatile is  
    ESSENTIAL  
  
    static Singleton getInstance() {  
        if (instance == null) {                    // check 1 (no lock)  
            synchronized (Singleton.class) {  
                if (instance == null) {            // check 2 (with lock)  
                    instance = new Singleton();  
                }  
            }  
        }  
        return instance;  
    }  
}
```

Without `volatile`, another thread could see a **partially constructed** object, because `new Singleton()` involves allocate → construct → assign, and the JVM is allowed to reorder those steps. `volatile` forbids that reordering.

Interview-ready answer

`volatile` guarantees visibility: a volatile variable is always read from and written to main memory instead of a thread's local CPU cache, so a change made by one thread is immediately visible to all others. The classic use is a boolean stop flag — without `volatile` a worker thread may cache the flag and loop forever. However, `volatile` does not provide atomicity, so `count++` is still unsafe because it is a read-modify-write sequence and two threads can interleave; for that we need `AtomicInteger` or `synchronized`. `volatile` also prevents instruction reordering, which is why it is required in the double-checked locking singleton — without it another thread could observe a partially constructed object. So the rule is: use `volatile` for simple flags, and `synchronized` or atomic classes for compound updates.

Easy memory

volatile = read/write DIRECTLY from MAIN memory (skip the CPU cache)

- ✓ VISIBILITY → the stop flag is seen immediately
- ✗ ATOMICITY → count++ is STILL broken ▲

Whiteboard (volatile) vs personal notepad (cached)

- Flag → volatile ✓
- Counter → AtomicInteger ✓
- Multi-step → synchronized ✓

Also needed in double-checked-locking singletons (prevents reordering).

Q: Explain `wait()` and `notify()`.

The easiest way to remember:

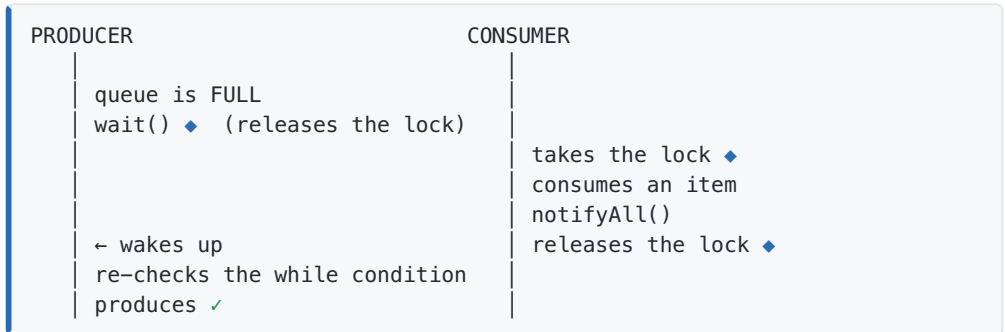
- `wait()` → "I'll pause and RELEASE the lock; wake me when things change"
- `notify()` → "Hey, something changed – one of you can wake up"
- `notifyAll()` → "Everyone wake up and re-check" ★ safer

These are how two threads **talk to each other**.

The classic example — producer / consumer

```
class SharedQueue {  
  
    private final Queue<Integer> queue = new LinkedList<>();  
    private final int CAPACITY = 5;  
  
    public synchronized void produce(int item) throws InterruptedException  
    {  
  
        while (queue.size() == CAPACITY) {    // while, NOT if ★  
            wait();                            // box full → pause +  
release lock  
        }  
  
        queue.add(item);  
        System.out.println("Produced " + item);  
  
        notifyAll();                          // tell consumers there is  
data  
    }  
  
    public synchronized int consume() throws InterruptedException {  
  
        while (queue.isEmpty()) {  
            wait();                            // box empty → pause +  
release lock  
        }  
  
        int item = queue.poll();  
        System.out.println("Consumed " + item);  
  
        notifyAll();                          // tell producers there is  
space  
        return item;  
    }  
}
```


The flow



Real-world idea: a **restaurant kitchen counter**.

Chef (producer) → puts a plate on the counter → rings the bell (notify)
Waiter (consumer) → waiting near the counter (wait)
→ hears the bell → picks up the plate

Counter full? → chef waits
Counter empty? → waiter waits

Three rules you MUST state ★

Rule 1 — they must be called inside a `synchronized` block

```
void m() {  
    wait();           // ▲ IllegalMonitorStateException  
}  
  
synchronized void m() throws InterruptedException {  
    wait();           // ✓  
}
```

Because `wait()` releases the lock — you cannot release a lock you never held.

Rule 2 — always use `while`, never `if` ★★

```
if (queue.isEmpty()) { wait(); }    // ✗ dangerous  
while (queue.isEmpty()) { wait(); } // ✓ correct
```

Why? Two reasons:

1. SPURIOUS WAKEUP – the JVM is allowed to wake a thread with no notify at all
2. notifyAll() wakes EVERY waiting thread, but by the time this thread gets the lock, another thread may already have taken the item

With `if`, the thread continues believing the condition is still true and crashes.

With `while`, it re-checks and goes back to waiting.

Rule 3 — prefer `notifyAll()` over `notify()`

```
notify()    → wakes ONE random waiting thread
              → it might wake the WRONG kind of thread (another producer
                instead of a consumer) → everyone sleeps forever =
deadlock ▲

notifyAll() → wakes all of them; each re-checks its condition → safe ✓
```

wait() vs sleep() ★ (asked constantly)

| | <code>wait()</code> | <code>sleep()</code> |
|------------------------|---|----------------------|
| Defined in | <code>Object</code> | <code>Thread</code> |
| Releases the lock? | ✓ YES | ✗ NO |
| Wakes up on | <code>notify()</code> /
<code>notifyAll()</code> / timeout | timeout only |
| Requires synchronized? | ✓ yes | ✗ no |
| Purpose | thread communication | pause execution |

The one line to remember: `wait()` releases the lock, `sleep()` holds on to it.

The modern replacement

In real code you rarely write `wait` / `notify` today:

```
BlockingQueue<Integer> queue = new LinkedBlockingQueue<>(5);

queue.put(item);           // blocks automatically when full ✓
int item = queue.take();   // blocks automatically when empty ✓
```

`BlockingQueue` does all of the above internally. Mentioning this shows practical maturity.

Interview-ready answer

`wait()`, `notify()` and `notifyAll()` are defined on `Object` and are used for communication between threads. `wait()` makes the current thread pause and release the lock it holds, so another thread can proceed; `notify()` wakes one waiting thread and `notifyAll()` wakes all of them. They must be called inside a synchronized block, otherwise we get `IllegalMonitorStateException`, because a thread can only release a monitor it owns. The condition must always be checked in a `while` loop rather than an `if`, because of spurious wakeups and because another thread may consume the condition before this thread re-acquires the lock. I generally prefer `notifyAll()` since `notify()` can wake the wrong thread and cause the application to hang. The key difference from `sleep()` is that `wait()` releases the lock while `sleep()` keeps it. In modern code I would use `BlockingQueue` or the `Condition` interface instead of writing wait/notify manually.

Easy memory

```
wait()      → pause + RELEASE the lock ♦
notify()    → wake ONE (risky)
notifyAll() → wake ALL ★ safer
```

Kitchen: chef puts a plate → rings the bell (notify) → waiter picks it up

3 RULES:

1. Must be inside synchronized (else `IllegalMonitorStateException`)
2. Use WHILE, never IF (spurious wakeups)
3. Prefer `notifyAll()`

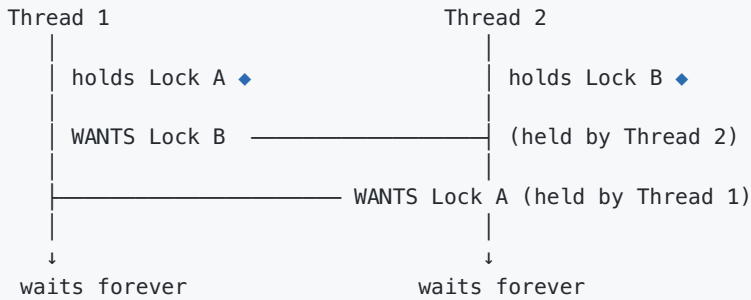
```
wait() releases the lock ♦ | sleep() keeps it ♦
Modern code: use BlockingQueue instead.
```

Q: What is a deadlock and how do you avoid it?

The easiest way to remember:

Deadlock = two threads each holding a lock the other one needs,
so both wait FOREVER. ♦ ↔♦

The picture



Real-world idea — **two people, one plate of food.**

Person A grabs the SPOON and waits for the FORK
Person B grabs the FORK and waits for the SPOON

Neither will let go. Nobody eats. Forever.

Code that deadlocks

```
class DeadlockDemo {  
  
    private final Object lockA = new Object();  
    private final Object lockB = new Object();  
  
    public void method1() {  
        synchronized (lockA) {                // takes A  
            System.out.println("Thread 1: holding A");  
            sleep(100);  
            synchronized (lockB) {                // wants B ← waits forever  
                System.out.println("Thread 1: holding A and B");  
            }  
        }  
    }  
  
    public void method2() {  
        synchronized (lockB) {                // takes B  
            System.out.println("Thread 2: holding B");  
            sleep(100);  
            synchronized (lockA) {                // wants A ← waits forever  
                System.out.println("Thread 2: holding B and A");  
            }  
        }  
    }  
}
```

The bug is simply that the two methods take the locks in **opposite order**.

The 4 conditions (all must hold)

1. Mutual exclusion → the resource can be held by only one thread
2. Hold and wait → a thread holds one lock while asking for another
3. No preemption → a lock cannot be forcibly taken away
4. Circular wait → T1 waits for T2, T2 waits for T1 ← easiest to break ★

Break any one and the deadlock disappears.

Solution 1 — LOCK ORDERING ★ (the main answer)

Make every thread take the locks in the **same order**.

```

public void method1() {
    synchronized (lockA) {           // A first
        synchronized (lockB) { }     // then B
    }
}

public void method2() {
    synchronized (lockA) {           // A first ← was B before ✓
        synchronized (lockB) { }     // then B
    }
}

```

Both threads: A → B (never B → A)
 Circular wait is impossible ✓

Back to the dinner table: agree that **everyone picks up the spoon first, then the fork**. Now the second person simply waits for the spoon and nobody is stuck holding half a set.

For dynamic objects (like two bank accounts), order by a unique id:

```

void transfer(Account from, Account to, double amount) {

    Account first = from.getId() < to.getId() ? from : to;
    Account second = from.getId() < to.getId() ? to : from;

    synchronized (first) {
        synchronized (second) {
            from.debit(amount);
            to.credit(amount);
        }
    }
}

```

This is the classic bank-transfer deadlock question.

Solution 2 — `tryLock()` with a timeout

```
ReentrantLock lockA = new ReentrantLock();
ReentrantLock lockB = new ReentrantLock();

if (lockA.tryLock(1, TimeUnit.SECONDS)) {
    try {
        if (lockB.tryLock(1, TimeUnit.SECONDS)) {
            try {
                // safe work
            } finally { lockB.unlock(); }
        } else {
            // couldn't get B → give up and RELEASE A ✓
        }
    } finally { lockA.unlock(); }
}
```

This breaks the "hold and wait" condition — the thread refuses to wait forever.

Solution 3 — avoid nested locks entirely

```
// ✗ nested
synchronized (a) { synchronized (b) { } }

// ✓ one at a time
synchronized (a) { doPartOne(); }
synchronized (b) { doPartTwo(); }
```

Also: never call an unknown/external method while holding a lock — it might take another lock.

How do you DETECT a deadlock?

```
jps                # find the process id
jstack <pid>       # thread dump → prints "Found one Java-level deadlock"
```

Or use JConsole / VisualVM → Threads → **Detect Deadlock**.

Programmatically:

```
ThreadMXBean bean = ManagementFactory.getThreadMXBean();
long[] ids = bean.findDeadlockedThreads();
```

Mentioning `jstack` is a strong practical point.

Interview-ready answer

A deadlock happens when two or more threads each hold a lock the other needs, so all of them wait forever. It requires four conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait. The most practical fix is to break the circular wait by enforcing a consistent global lock ordering — if every thread acquires lock A before lock B, a cycle can never form. For dynamic resources such as two bank accounts, we order the locks by a unique identifier like the account id. Another approach is to use `ReentrantLock.tryLock()` with a timeout, so a thread that cannot get the second lock releases the first instead of waiting forever. We should also avoid nested locks where possible and never call external code while holding a lock. In production, deadlocks are diagnosed with a thread dump using `jstack`, which explicitly reports Java-level deadlocks.

Easy memory

Deadlock = T1 holds A wants B | T2 holds B wants A → both wait FOREVER
(spoon & fork: neither person lets go)

4 conditions: mutual exclusion + hold-and-wait + no preemption + CIRCULAR WAIT

FIX #1 ★ SAME LOCK ORDER everywhere (A then B, always)

FIX #2 `tryLock(timeout)` → give up instead of waiting

FIX #3 avoid nested locks

DETECT: `jstack <pid>` → "Found one Java-level deadlock"

Q: What is `ExecutorService` and why use thread pools?

The easiest way to remember:

Thread pool = a fixed team of REUSABLE threads.

Creating a new thread per task is expensive → reuse them instead.

The problem with `new Thread()`

```
for (int i = 0; i < 10000; i++) {  
    new Thread() -> handleRequest().start();    // ✗ 10,000 threads!  
}
```

Each thread costs:

- ~1 MB of stack memory
- OS-level creation time
- CPU time for context switching

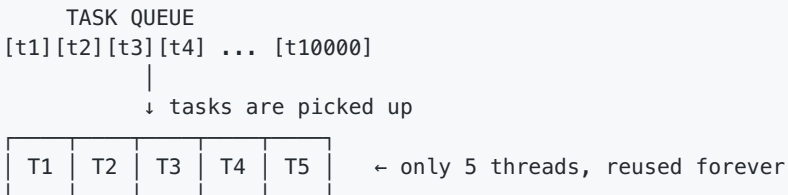
10,000 threads → ~10 GB of memory → `OutOfMemoryError` ▲

Real-world idea: a **restaurant**.

Without a pool → hire a NEW waiter for every customer, fire him after ✗
With a pool → keep 5 waiters; each serves customer after customer ✓

The solution

```
ExecutorService pool = Executors.newFixedThreadPool(5);  
  
for (int i = 0; i < 10000; i++) {  
    pool.submit() -> handleRequest();    // 10,000 tasks, only 5  
    threads  
}  
  
pool.shutdown();
```



The types of pools

```
// 1. Fixed – N threads, extra tasks queue up ★ most common
ExecutorService fixed = Executors.newFixedThreadPool(10);

// 2. Cached – creates threads as needed, reuses idle ones, kills after
60s
ExecutorService cached = Executors.newCachedThreadPool();

// 3. Single – exactly one thread, tasks run in order
ExecutorService single = Executors.newSingleThreadExecutor();

// 4. Scheduled – for delayed / repeating tasks
ScheduledExecutorService sch = Executors.newScheduledThreadPool(3);
sch.scheduleAtFixedRate(() -> cleanup(), 0, 1, TimeUnit.HOURS);

// 5. Work-stealing (Java 8) – uses ForkJoinPool
ExecutorService ws = Executors.newWorkStealingPool();
```

submit() vs execute()

```
pool.execute(() -> doWork()); // Runnable, returns
nothing

Future<String> f = pool.submit(() -> "result"); // Callable, returns a
Future ★
String value = f.get(); // blocks until the
result is ready
```

```
execute() → fire and forget → an exception is just printed
submit() → gives you a Future → the exception is stored and rethrown by
get()
```

▲ A trap: with `submit()`, if you never call `get()`, exceptions are **silently swallowed**.

Shutting down properly ★

```
pool.shutdown();                // no new tasks; finish the current
ones

if (!pool.awaitTermination(60, TimeUnit.SECONDS)) {
    pool.shutdownNow();          // interrupt running tasks
}
```

shutdown() → polite: finish what's running, accept nothing new ✓
shutdownNow() → forceful: interrupt everything, return pending tasks
no shutdown → the JVM never exits ▲ (non-daemon threads keep it alive)

Production-grade configuration ★

`Executors.newFixedThreadPool()` uses an **unbounded** queue, which can silently grow until you run out of memory. Serious code uses the constructor directly:

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(
    5,                                // core pool size
    10,                               // maximum pool size
    60L, TimeUnit.SECONDS,            // idle keep-alive
    new ArrayBlockingQueue<>(100),    // BOUNDED queue ✓
    new ThreadPoolExecutor.CallerRunsPolicy()); // back-pressure
when full
);
```

How many threads?

CPU-bound work → threads ≈ number of cores
I/O-bound work → many more (they spend time waiting on the network/DB)

In Spring Boot

```
@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean
    public Executor taskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(10);
        executor.setQueueCapacity(100);
        executor.setThreadNamePrefix("async-");
        executor.initialize();
        return executor;
    }
}

@Async
public void sendEmail(String to) { ... }    // runs on the pool ✓
```

Interview-ready answer

ExecutorService is the high-level API for managing threads through a thread pool. Creating a thread per task is expensive — each thread costs around a megabyte of stack plus OS-level creation and context-switching overhead — so with thousands of tasks the application would run out of memory. A thread pool keeps a fixed set of reusable threads and queues the tasks, so resources stay bounded. The common types are **newFixedThreadPool** for a fixed number of workers, **newCachedThreadPool** for many short-lived tasks, **newSingleThreadExecutor** for sequential execution, and **newScheduledThreadPool** for periodic work. **execute()** accepts a **Runnable** and returns nothing, while **submit()** accepts a **Callable** and returns a **Future** we can use to get the result or the exception. We must always call **shutdown()**, otherwise the JVM will not exit. In production I prefer configuring **ThreadPoolExecutor** directly with a bounded queue and a rejection policy, because the **Executors** factory methods use unbounded queues that can hide memory problems. In Spring Boot this is configured as a **ThreadPoolTaskExecutor** bean and used with **@Async**.

Easy memory

new Thread() per task → 1 MB each → OutOfMemory ▲
Thread pool → 5 reusable waiters serving 10,000 customers ✓

newFixedThreadPool(n) ★ most common
newCachedThreadPool() many short tasks
newSingleThreadExecutor() sequential
newScheduledThreadPool() timers

execute() → Runnable → nothing back
submit() → Callable → Future.get() ★ (exceptions hide without get(!))

ALWAYS shutdown() – else the JVM never exits.
Production: ThreadPoolExecutor + BOUNDED queue + rejection policy.

Q: Callable vs Runnable?

The easiest way to remember:

| | |
|----------------------------|------------------------------------|
| Runnable → returns NOTHING | → cannot throw a checked exception |
| Callable → RETURNS a value | → CAN throw a checked exception ★ |

The interfaces side by side

```
@FunctionalInterface
interface Runnable {
    void run();
}
// no return, no throws

@FunctionalInterface
interface Callable<V> {
    V call() throws Exception;
}
// returns V, and CAN throw ★
```

Runnable — fire and forget

```
Runnable task = () -> System.out.println("Sending email...");

pool.execute(task);           // nothing comes back
```

Use it for: logging, sending a notification, cleanup — anything where you don't need an answer.

Callable — gives you an answer

```
Callable<Integer> task = () -> {
    Thread.sleep(1000);
    return 42;                                // ✓ returns a value
};

Future<Integer> future = pool.submit(task);

// ... do other work meanwhile ...

Integer result = future.get();                // blocks until ready
System.out.println(result);                  // 42
```

```
submit(Callable)
    ↓
returns a Future immediately (a "receipt") ← doesn't block ✓
    ↓
you keep working
    ↓
future.get() ← NOW it blocks until the answer is ready
```

Real-world idea: a **dry cleaner**.

```
Runnable → you drop off clothes and walk away, expecting nothing back
Callable → you drop off clothes and get a TOKEN (Future)
           → later you show the token and collect the clothes
(future.get())
```

Exception handling — the important difference

```
Runnable r = () -> {
    throw new IOException();                // ✗ compile error – checked exception
};

Callable<String> c = () -> {
    throw new IOException();                // ✓ allowed
};
```

With `Callable`, the exception is stored and rethrown when you call `get()`:

```
try {
    future.get();
} catch (ExecutionException e) {
    System.out.println("Task failed: " + e.getCause()); // the real
    exception ★
}
```

Note it is wrapped in `ExecutionException` — use `getCause()` to reach the original.

Useful Future methods

```
future.get(); // blocks forever until done
future.get(5, TimeUnit.SECONDS); // blocks with a timeout ★ safer
future.isDone(); // finished? (true even if it failed)
future.cancel(true); // try to cancel / interrupt
future.isCancelled();
```

Always prefer the timeout version in production — a plain `get()` can hang your thread forever.

Running many tasks at once

```
List<Callable<Integer>> tasks = List.of(
    () -> compute(1),
    () -> compute(2),
    () -> compute(3));

List<Future<Integer>> results = pool.invokeAll(tasks); // waits for ALL

for (Future<Integer> f : results) {
    System.out.println(f.get());
}

Integer fastest = pool.invokeAny(tasks); // returns the FIRST completed
result
```

The modern upgrade — `CompletableFuture` ★

`Future.get()` blocks, which wastes a thread. `CompletableFuture` (Java 8) is non-blocking and chainable:

```

CompletableFuture.supplyAsync(() -> fetchUser(id))
    .thenApply(user -> user.getName())
    .thenAccept(name -> System.out.println(name))
    .exceptionally(e -> { log.error("failed", e); return
null; });

```

You can also combine several calls:

```

CompletableFuture<String> a = CompletableFuture.supplyAsync(() ->
callServiceA());
CompletableFuture<String> b = CompletableFuture.supplyAsync(() ->
callServiceB());

CompletableFuture.allOf(a, b).join();    // both run in PARALLEL ▲

```

This is very common in microservices — calling three services in parallel instead of one after another.

Comparison

| | Runnable | Callable |
|--------------------|---|----------------------------|
| Since | Java 1.0 | Java 5 |
| Method | <code>run()</code> | <code>call()</code> |
| Returns | void | a value <code>V</code> |
| Checked exceptions | ✗ | ✓ |
| Submitted with | <code>execute()</code> or <code>submit()</code> | <code>submit()</code> only |
| Result via | — | <code>Future</code> |

Interview-ready answer

Runnable has a `run()` method that returns nothing and cannot throw checked exceptions, so it is used for fire-and-forget tasks. **Callable** was added in Java 5; its `call()` method returns a value and is allowed to throw checked exceptions. When we submit a **Callable** to an **ExecutorService** we get back a **Future**, which acts like a receipt — the submission returns immediately and we call `future.get()` later to retrieve the result, which blocks until the task completes. If the task threw an exception, `get()` rethrows it wrapped in an

`ExecutionException` , and we use `getCause()` to see the original. In production I always use the timeout version of `get()` so a hung task cannot block the caller indefinitely. For more complex flows I prefer `CompletableFuture` , which is non-blocking and lets me chain and combine asynchronous calls — very useful when calling multiple microservices in parallel.

Easy memory

```
Runnable → run() → void → no checked exceptions → execute()
Callable → call() → VALUE → CAN throw → submit() → Future ★

Dry cleaner: Runnable = drop and leave | Callable = get a TOKEN (Future)

future.get() → blocks forever
future.get(5, SECONDS) → safer ★
exception → ExecutionException → use e.getCause()

Modern: CompletableFuture → non-blocking, chainable, parallel service calls
```

Q: `synchronized` vs `Lock` (`ReentrantLock`)?

The easiest way to remember:

```
synchronized → simple, automatic, but you CANNOT give up waiting
ReentrantLock → more code, but gives tryLock, timeout, fairness,
interruptibility ★
```

The core limitation of `synchronized`

```
synchronized (lock) {
    // if another thread holds this lock, I wait FOREVER.
    // I cannot time out. I cannot check. I cannot be interrupted.
}
```

`ReentrantLock` removes all of those limitations.

Basic ReentrantLock usage

```
private final ReentrantLock lock = new ReentrantLock();

public void increment() {
    lock.lock();                // acquire
    try {
        count++;
    } finally {
        lock.unlock();         // ▲ MUST be in finally
    }
}
```

▲ The biggest danger: forgetting `unlock()`. With `synchronized` the JVM releases the lock automatically, even if an exception is thrown. With `ReentrantLock`, if you forget the `finally`, the lock is **never released** and the application freezes.

```
synchronized → lock released automatically ✓ (even on exception)
ReentrantLock → YOU must unlock() in a finally block ▲
```

Feature 1 — `tryLock()` : don't wait if it's busy

```
if (lock.tryLock()) {           // returns immediately
    try {
        doWork();
    } finally { lock.unlock(); }
} else {
    System.out.println("Busy, I'll do something else"); // ✓ no waiting
}
```

With a timeout:

```
if (lock.tryLock(2, TimeUnit.SECONDS)) { // wait max 2 seconds
    try { doWork(); } finally { lock.unlock(); }
} else {
    log.warn("Could not acquire lock in 2s");
}
```

This is the main tool for **avoiding deadlocks**.

Real-world idea: a **restaurant table**.

```
synchronized → you stand at the door waiting for a table, forever  
tryLock()      → you ask "is a table free?" → if not, you go to another  
restaurant ✓
```

Feature 2 — interruptible waiting

```
lock.lockInterruptibly();    // another thread can interrupt this waiting
```

With `synchronized`, a waiting thread cannot be interrupted at all — you cannot cancel it.

Feature 3 — fairness

```
ReentrantLock fair = new ReentrantLock(true);    // FIFO order
```

```
Unfair (default) → whoever grabs it first wins → faster, but a thread can  
starve  
Fair (true)      → longest-waiting thread wins → no starvation, but slower
```

Real-world idea: a proper queue at a ticket counter (fair) versus a crowd pushing in (unfair).

Feature 4 — multiple conditions ★

`synchronized` has only one wait-set. `ReentrantLock` can have several:

```

private final ReentrantLock lock = new ReentrantLock();
private final Condition notFull = lock.newCondition();
private final Condition notEmpty = lock.newCondition();

public void produce(int item) throws InterruptedException {
    lock.lock();
    try {
        while (queue.size() == CAPACITY) {
            notFull.await();           // producers wait HERE
        }
        queue.add(item);
        notEmpty.signalAll();         // wake only the CONSUMERS ★
    } finally { lock.unlock(); }
}

public int consume() throws InterruptedException {
    lock.lock();
    try {
        while (queue.isEmpty()) {
            notEmpty.await();         // consumers wait HERE
        }
        int item = queue.poll();
        notFull.signalAll();         // wake only the PRODUCERS ★
        return item;
    } finally { lock.unlock(); }
}

```

synchronized + notifyAll() → wakes EVERYONE, including the wrong threads
 Lock + 2 Conditions → wakes exactly the right group ✓ much more efficient

Bonus — ReadWriteLock

```

ReadWriteLock rw = new ReentrantReadWriteLock();

rw.readLock().lock();    // MANY readers at the same time ✓
rw.writeLock().lock();    // only ONE writer, and no readers

```

Perfect for a cache that is read constantly and written rarely.

Comparison

| | synchronized | ReentrantLock |
|-----------------------|----------------------|-----------------------------------|
| Since | 1.0 | Java 5 |
| Unlock | automatic ✓ | manual, in <code>finally</code> ▲ |
| tryLock / timeout | ✗ | ✓ |
| Interruptible | ✗ | ✓ |
| Fairness option | ✗ | ✓ |
| Multiple conditions | ✗ (one wait-set) | ✓ |
| Read/write separation | ✗ | ✓ (<code>ReadWriteLock</code>) |
| Code readability | simpler ✓ | more verbose |
| Performance | equal in modern JVMs | equal |

Note: since Java 6, `synchronized` is heavily optimised (biased locking, lock coarsening), so performance is **not** a reason to choose one over the other. Choose based on **features**.

Which one do I use?

Default → `synchronized` (simple, safe, less code) ★
Switch to `ReentrantLock` ONLY when you need:
 `tryLock` / `timeout` (deadlock avoidance)
 `interruptible lock`
 `fairness`
 `multiple conditions`
 `read-write separation`

Interview-ready answer

`synchronized` is the built-in keyword: the JVM acquires and releases the monitor automatically, so the lock is always released even if an exception is thrown, and the code stays simple. `ReentrantLock`, added in Java 5, is an explicit lock that must be released manually in a `finally` block, but it offers capabilities `synchronized` does not have. It supports `tryLock()` with an optional timeout, which lets a thread give up instead of waiting forever and is a standard way to

avoid deadlocks. It supports interruptible locking, an optional fairness policy that prevents thread starvation, and multiple **Condition** objects, so in a producer-consumer scenario we can signal only the consumers instead of waking every waiting thread. There is also **ReentrantReadWriteLock**, which allows many concurrent readers but exclusive writers, which suits read-heavy caches. Since Java 6 both perform similarly, so I default to **synchronized** for simplicity and use **ReentrantLock** only when I need one of those extra features.

Easy memory

synchronized → automatic unlock ✓ → simple → DEFAULT ★
ReentrantLock → manual unlock in finally ▲ → but gives:

tryLock() → don't wait → deadlock avoidance ★
lockInterruptibly() → can be cancelled
new ReentrantLock(true) → fairness, no starvation
newCondition() → wake only the right group
ReadWriteLock → many readers, one writer

Restaurant: **synchronized** = wait at the door forever
tryLock() = "no table? I'll go elsewhere" ✓

▲ `lock.lock(); try { ... } finally { lock.unlock(); }`

Memory & Garbage Collection

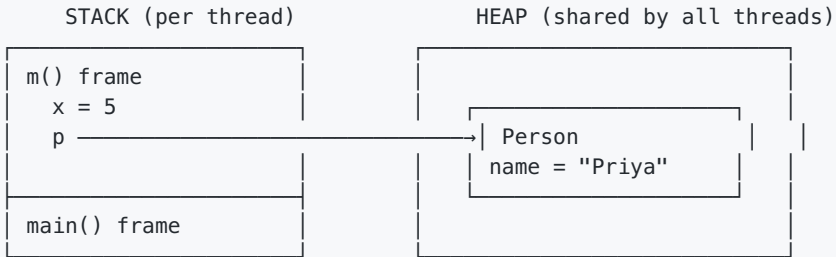
Q: Stack vs heap memory?

The easiest way to remember:

STACK → per THREAD → method calls + local variables + references → auto-cleaned
HEAP → SHARED → all OBJECTS → cleaned by GC

The picture

```
void m() {  
    int x = 5;  
    Person p = new Person("Priya");  
}
```



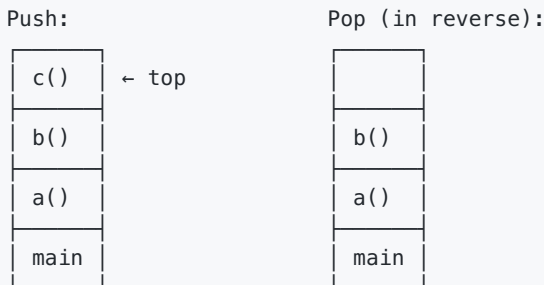
the VALUE 5 and the REFERENCE p
live on the stack

the OBJECT itself
lives on the heap

★ The key sentence: **the reference is on the stack, the object is on the heap.**

How the stack works — LIFO

```
void a() { b(); }  
void b() { c(); }  
void c() { System.out.println("done"); }
```



When a method returns, its whole frame is popped — instantly, with no GC involved.

Real-world idea: a **stack of plates**. The last plate placed is the first one removed.

Comparison

| | Stack | Heap |
|-----------------|-----------------------------------|--------------------------------------|
| Stores | method frames, locals, references | all objects, instance fields, arrays |
| Shared? | ✗ one per thread | ✓ shared by all threads |
| Cleanup | automatic on method return | garbage collector |
| Speed | very fast (just move a pointer) | slower (allocation + GC) |
| Size | small (~512 KB–1 MB per thread) | large (megabytes to gigabytes) |
| Thread-safe? | ✓ naturally (private) | ✗ needs synchronization |
| Error when full | <code>StackOverflowError</code> | <code>OutOfMemoryError</code> |
| Order | LIFO | no order |

The two errors ★

StackOverflowError — infinite recursion

```
void recurse() {  
    recurse();           // never returns  
}
```

```
recurse()  
recurse()  
recurse()  
...  
← frames pile up  
▲ StackOverflowError
```

OutOfMemoryError — too many live objects

```
List<int[]> list = new ArrayList<>();  
while (true) {  
    list.add(new int[1_000_000]);    // the list keeps them REACHABLE  
}  
// ▲ OutOfMemoryError: Java heap space
```


Note the important detail: the GC cannot free them **because the list still references them**. That is exactly what a memory leak is.

Where do static and String literals live?

| | |
|--------------------|---|
| Java 7 and earlier | → PermGen |
| Java 8+ | → METASPACE (class metadata, outside the heap, grows dynamically) |
| Static variables | → in the heap, with the class object |
| String pool | → in the HEAP since Java 7 ★ (was PermGen before) |

That is why `-XX:MaxPermSize` no longer exists — Java 8 replaced PermGen with Metaspace and removed a whole class of `OutOfMemoryError: PermGen space` failures.

Useful JVM flags

| | |
|--|-------------------------|
| <code>-Xms512m</code> | # initial heap size |
| <code>-Xmx2g</code> | # maximum heap size |
| <code>-Xss1m</code> | # stack size per thread |
| <code>-XX:+HeapDumpOnOutOfMemoryError</code> | # dump for analysis ★ |

Interview-ready answer

The stack is per-thread memory that stores method frames, local primitive variables and object references. It works in LIFO order, and when a method returns its frame is popped automatically, so it is very fast and needs no garbage collection. It is also naturally thread-safe because each thread has its own stack. The heap is shared across all threads and stores every object and its instance fields; it is managed by the garbage collector and requires synchronization when objects are shared. The key point is that a reference lives on the stack while the object it points to lives on the heap. Deep or infinite recursion causes `StackOverflowError`, while too many reachable objects cause `OutOfMemoryError`. Since Java 8, class metadata lives in Metaspace instead of PermGen, and the String pool has been part of the heap since Java 7.

Easy memory

STACK → per THREAD → locals + references → LIFO → auto-cleaned → `StackOverflowError`

HEAP → SHARED → OBJECTS → GC → slower → `OutOfMemoryError`

```
Person p = new Person();  
    ↑           ↑  
  STACK (reference)  HEAP (object)
```

Stack of plates: last in, first out.

Java 8: PermGen → METASPACE. String pool → heap (since Java 7).

Q: How does garbage collection work?

The easiest way to remember:

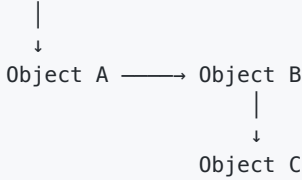
GC automatically deletes objects that are NO LONGER REACHABLE from a GC root.

"Not reachable" – not "not used". ★

What counts as reachable?

- GC ROOTS:
- local variables on any thread's stack
 - static fields
 - active threads
 - JNI references

GC Root (a local variable)



← all three are REACHABLE ✓

Object X ↔ Object Y
root

← they reference each other,
but NOTHING points to them from a
root

→ both are GARBAGE ✓ collected

★ That last point is worth saying: Java uses **reachability**, not reference counting, so circular references are collected correctly. (Older languages using reference counting leaked memory here.)

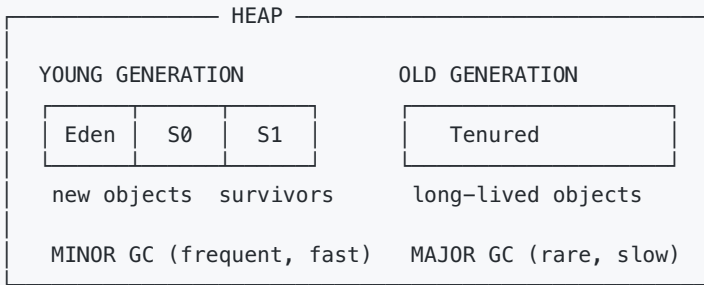
Example

```
Person p = new Person("Priya");    // reachable
p = null;                          // now unreachable → eligible for GC
✓

Person a = new Person("A");
Person b = new Person("B");
a.friend = b;
b.friend = a;                      // circular
a = null;
b = null;                          // BOTH are collected ✓ (unreachable)
```

Generational GC — "most objects die young"

Studies showed that the vast majority of objects become garbage almost immediately (temporary variables, DTOs, strings inside a loop). So the heap is split:



METASPACE (outside the heap) → class metadata

The life of an object

1. `new Object()` → born in EDEN
2. Eden is full → MINOR GC runs
 - living objects → moved to Survivor S0
 - dead objects → deleted ✓
3. Next Minor GC → S0 survivors → moved to S1 (age +1)
4. After surviving ~15 rounds → PROMOTED to the OLD generation
5. Old generation fills up → MAJOR / FULL GC (slow, "stop the world")

Real-world idea: an **office probation system**.

Eden → new joiners (many leave quickly)
 Survivor → those who survive the first review
 Old gen → permanent employees (rarely reviewed, but reviewing them takes long)

Mark and Sweep

MARK → walk from every GC root and mark everything reachable
 SWEEP → delete everything not marked
 COMPACT → slide the survivors together to remove fragmentation

Before: [A][][B][][][C][] ← fragmented
 After : [A][B][C][][][][] ← compacted, allocation is fast again

The collectors

```
Serial GC      → one thread → tiny apps
Parallel GC    → multiple threads → throughput-focused (Java 8 default)
CMS            → concurrent, low pause → deprecated
G1 GC          → region-based, predictable pauses → DEFAULT since Java 9
★
ZGC / Shenandoah → pauses under 10 ms even with huge heaps (Java 11+)
```

```
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200
```

Can I force garbage collection?

```
System.gc();           // ✗ only a REQUEST – the JVM may ignore it
                        completely
```

Never rely on it. Calling it in production is considered a bug, because it can trigger an expensive Full GC.

Memory leaks in Java ★ (great practical point)

Java has a GC, but leaks still happen — whenever you keep an unwanted reference:

```
// 1. A static collection that only ever grows
static List<Session> sessions = new ArrayList<>(); // nothing is ever
removed ▲

// 2. Unclosed resources
Connection con = getConnection(); // never closed → use try-with-
resources ✓

// 3. Listeners never unregistered
button.addListener(listener); // never removed

// 4. Mutable keys in a HashMap
map.put(employee, data);
employee.setId(99); // the entry becomes unreachable but
never freed
```

How to investigate:

```
jmap -dump:live,format=b,file=heap.hprof <pid>    # take a heap dump
# then analyse in Eclipse MAT or VisualVM
```

Interview-ready answer

Garbage collection automatically reclaims heap memory occupied by objects that are no longer reachable from GC roots such as thread stacks, static fields and active threads. It uses reachability rather than reference counting, which is why objects that reference each other in a cycle are still collected once nothing points to them from a root. The heap is generational, based on the observation that most objects die young: new objects are allocated in Eden, and a Minor GC moves survivors into the survivor spaces; objects that survive enough cycles are promoted to the old generation, which is cleaned by a much slower Major or Full GC. The algorithm is mark-sweep-compact — mark everything reachable, sweep the rest, and compact to avoid fragmentation. G1 is the default collector since Java 9, and ZGC and Shenandoah offer very low pause times for large heaps. We cannot force collection — `System.gc()` is only a hint. Java can still have memory leaks when we hold unwanted references, for example in a static collection that is never cleared or an unclosed resource, so I use try-with-resources and analyse heap dumps with a tool like Eclipse MAT when memory grows unexpectedly.

Easy memory

GC deletes what is UNREACHABLE from a GC root (not "unused") ★
Circular references ARE collected (reachability, not counting)

HEAP: [Eden | S0 | S1] → [Old Generation]
Minor GC (fast) Major/Full GC (slow, stop-the-world)

Object life: Eden → Survivor → (survive ~15 rounds) → Old gen
Algorithm : MARK → SWEEP → COMPACT
Default GC : G1 (Java 9+) | ZGC/Shenandoah for very low pause

`System.gc()` = only a REQUEST, never rely on it

LEAKS: static collections, unclosed resources, listeners, mutable map keys
Debug : jmap heap dump → Eclipse MAT

Quick Revision Sheet — read this the night before

OOP

A PIE: Abstraction, Polymorphism, Inheritance, Encapsulation
Overloading = same name, different params → compile time
Overriding = child changes parent's body → runtime
Abstract class = IS-A + shared state | Interface = CAN-DO + contract
final (can't change) | finally (always runs) | finalize (deprecated)

CORE

JDK > JRE > JVM

String immutable → pool, security, hash-key safety, thread-safe
StringBuilder (fast, 1 thread) | StringBuffer (safe, many threads)
== address | equals() content → default equals() IS ==
equals() true → hashCode MUST match. hashCode same → equals NOT

guaranteed.

Autoboxing int→Integer | Unboxing Integer→int (null → NPE!)

Java is ALWAYS pass-by-value (for objects, the value is the reference)

COLLECTIONS

List (dup, ordered) | Set (unique) | Map (key→value, NOT a Collection)
ArrayList = read fast | LinkedList = a Deque
HashMap: hashCode→bucket, equals→entry; load factor 0.75; treeify > 8
HashMap (1 thread) | ConcurrentHashMap (many) | Hashtable (never)
Comparable = 1 natural order | Comparator = many orders
Fail-fast → ConcurrentModificationException → use removeIf()

EXCEPTIONS

Checked = compiler forces (IOException) | Unchecked = your bug (NPE)
throw = action | throws = declaration

try-with-resources → AutoCloseable → reverse close order → suppressed exceptions

Never return from finally (it swallows exceptions)

JAVA 8

Lambda → functional interface (1 abstract method)

Predicate(filter) Function(map) Consumer(forEach) Supplier(orElseGet)

Stream: intermediate = LAZY | terminal = RUNS

Optional → return type only; orElseThrow ★ ; never isPresent()+get()

default methods → added stream() to Collection without breaking anyone

THREADS

implements Runnable > extends Thread

start() = new thread | run() = same thread ✗

synchronized → mutual exclusion + visibility

volatile → visibility ONLY (count++ still broken)

wait() releases the lock ♦ | sleep() keeps it ♦ | always use while + notifyAll

Deadlock → fix with consistent LOCK ORDERING; detect with jstack

ExecutorService → reuse threads; always shutdown()

Callable → returns a value via Future

MEMORY

Stack = per thread, references, StackOverflowError

Heap = shared, objects, OutOfMemoryError

GC = reachability from GC roots; Eden→Survivor→Old; G1 by default

System.gc() is only a hint

Related files

- [31-hashmap-internals.md](#) — HashMap deep dive
- [32-multithreading.md](#) — concurrency deep dive
- [33-interface-vs-abstract-class.md](#) — with real design examples
- [12-java17-features.md](#) — records, sealed classes, pattern matching
- [40-java21-features.md](#) — ★ **virtual threads**, pattern matching for `switch`, record patterns, sequenced collections
- [22-java-streams-coding-problems.md](#) — stream coding questions
- [23-java-output-tricky-questions.md](#) — "what is the output?" traps
- [17-solid-design-patterns.md](#) — SOLID and design patterns

Spring & Spring Boot — Interview Q&A (Easy Version)

How to use this file: every question follows the same shape —
Easiest way to remember → **simple explanation** → **real-world example** →
diagram → **interview trap** → **Interview-ready answer** → **Easy memory box**.
The **Quick Revision Sheet** at the end is what you read the night before.

● About the "In your RoboGebra project" blocks

After an interview answer you'll see a ● **In your RoboGebra project** block with **real code from your own repositories**.

DEFINITION → "in our codebase..." ★ → **ONE trade-off**

Three sentences. That's the difference between reading the docs and shipping it.
Index: **39 — RoboGebra Code Examples**.

Contents

1. [Spring Core](#)
 2. [Spring Boot](#)
 3. [REST APIs](#)
 4. [Spring Data JPA](#)
 5. [Security & Misc](#)
 6. [Reactive \(Spring WebFlux\)](#)
 7. [Quick Revision Sheet](#)
-

Spring Core

Q: What is the Spring Framework?

The easiest way to remember:

Spring = a framework that CREATES and CONNECTS your objects for you, so your classes stop saying `new`.

Real-world idea: **a wedding planner.**

Without a planner → YOU book the caterer, the decorator, the photographer, and you must know each one's phone number

With a planner → you just say "I need a wedding".
The planner arranges everyone and hands it to you ✓

Spring is that planner. It is called the **IoC container**.

What Spring gives you

| | |
|---------------------------|--|
| DI (Dependency Injection) | → objects are handed to you |
| AOP | → logging/security/transactions without touching business code |
| Transactions | → @Transactional |
| Data access | → JDBC, JPA, MongoDB |
| MVC | → REST controllers |
| Security | → authentication + authorisation |

Key benefits: loose coupling, testability (easy mocking), declarative transactions and security, and a huge ecosystem.

Interview-ready answer

Spring is a lightweight open-source Java framework for enterprise applications. Its core idea is Inversion of Control — the framework manages object creation and wiring instead of our code doing it with `new`. On top of that it provides modules for dependency injection, AOP, declarative transactions, data access, MVC and security. The practical benefits are loose coupling and testability: because a class

receives its dependencies instead of creating them, we can swap the implementation or pass a mock in a unit test without changing the class.

Easy memory

Spring = the WEDDING PLANNER for your objects
It creates them, connects them, and hands them over.
Core = IoC + DI. Everything else (AOP, MVC, Data, Security) is built on that.

Q: What is IoC (Inversion of Control) and DI (Dependency Injection)?

The easiest way to remember:

IoC = the IDEA → "don't call us, we'll call you" – the container is in charge
DI = the METHOD → the container INJECTS your dependencies

DI is *how* Spring achieves IoC. People use the two words interchangeably, but that one line separates them cleanly.

The code that shows it

```
// ✗ Without DI – TIGHT coupling
class OrderService {
    private PaymentService payment = new PaymentService(); // hard-coded
}
```

Problems:

- OrderService is stuck with THIS PaymentService forever
- You cannot swap it for a test mock
- Changing PaymentService's constructor breaks OrderService

```
// ✓ With DI – the container injects it
@Service
class OrderService {
    private final PaymentService payment;

    OrderService(PaymentService payment) {        // injected ✓
        this.payment = payment;
    }
}
```

```
Spring container
    | creates PaymentService
    | creates OrderService, PASSING the PaymentService in
    ↓
OrderService doesn't know or care HOW PaymentService was built ✓
```

Real-world idea: **a restaurant kitchen.**

Without DI → the chef grows his own vegetables (tightly coupled to one farm)

With DI → vegetables are DELIVERED to the kitchen.
Change the supplier and the chef's recipe never changes ✓

Interview-ready answer

Inversion of Control means the control over object creation and wiring is inverted — instead of a class creating its own dependencies with **new**, the Spring container creates them and hands them over. Dependency Injection is the pattern Spring uses to implement IoC: the container injects an object's dependencies through the constructor, a setter or a field. The benefit is loose coupling — **OrderService** doesn't know how **PaymentService** is built, so we can swap the implementation or inject a mock in tests without modifying **OrderService** at all.

Easy memory

IoC = the PRINCIPLE (container is in charge)
DI = the TECHNIQUE (container injects the dependencies)

new PaymentService() → tight coupling ✗
constructor parameter → injected, swappable, testable ✓

Q: ApplicationContext vs BeanFactory?

The easiest way to remember:

BeanFactory = the BASIC container (bare minimum)
ApplicationContext = BeanFactory + all the enterprise features ★ (what you actually use)

| BeanFactory | ApplicationContext |
|---------------------------------------|--|
| basic container, lazy init by default | superset of BeanFactory |
| no enterprise features | eager singleton init, events, i18n, AOP, annotations |
| rarely used directly | the standard choice in real apps |

ApplicationContext
└ extends BeanFactory
 + event publishing
 + internationalisation (i18n)
 + annotation support
 + AOP integration
 + eager singleton creation ★

Real-world idea:

BeanFactory → a basic phone: calls only
ApplicationContext → a smartphone: calls + camera + apps + internet ✓

The **eager vs lazy** difference matters practically:

BeanFactory → creates a bean only when first requested (lazy)
ApplicationContext → creates all singletons at STARTUP (eager) ★

Eager is better: a misconfigured bean fails at **startup**, not at 2 a.m. when the first user hits that endpoint.

Interview-ready answer

Both are IoC containers, but **BeanFactory** is the basic, low-level one with lazy initialisation and no enterprise features. **ApplicationContext** is a superset — it

adds event publishing, internationalisation, annotation support, AOP integration, and it eagerly instantiates singleton beans at startup, so configuration errors surface immediately rather than at first use. In practice we always use `ApplicationContext`, and Spring Boot creates one for us automatically; `BeanFactory` is really just the underlying foundation.

Easy memory

`BeanFactory` → basic phone → lazy
`ApplicationContext` → smartphone → EAGER singletons ★ → what you always use

Eager is better: broken config fails at STARTUP, not in production.

Q: What are the bean scopes in Spring?

The easiest way to remember:

singleton (DEFAULT) → ONE object for the whole application ★
prototype → a NEW object every time you ask
request / session → web only

| Scope | How many instances |
|----------------------------|--|
| singleton (default) | one shared instance per container ★ |
| prototype | a new instance on every request for the bean |
| request | one per HTTP request (web only) |
| session | one per HTTP session (web only) |
| application | one per ServletContext |
| websocket | one per WebSocket session |

```
@Service
@Scope("prototype")
public class ReportBuilder { }
```

Real-world idea:

singleton → the office PRINTER: one machine, everybody shares it
prototype → a sheet of PAPER: everyone gets their own, fresh ✓
session → your shopping CART: one per logged-in user
request → the BILL for one purchase: created and thrown away

The trap you must mention ★

A singleton bean is shared by EVERY concurrent HTTP request.

So a mutable instance field on a @Service is a THREAD-SAFETY BUG. ▲

```
@Service
public class OrderService {
    private int counter;           // ▲ shared across all requests!
    private String currentUser;   // ▲ request A can see request
    B's user
}
```

Tomcat threads: req1 req2 req3 req4



↓

ONE OrderService object
private String currentUser; ← whoever wrote last wins ▲

Rule: keep @Service beans stateless. Pass state as method parameters.

Interview-ready answer

Spring has six scopes. The default is singleton — one shared instance per application context, not per JVM. Prototype creates a new instance every time the bean is requested. Then there are the web scopes: request, session, application and websocket. The important practical point is that because singleton beans are shared across all concurrent HTTP requests, any mutable instance field on a @Service is shared state and therefore a thread-safety bug — so I keep service beans stateless and pass state as method parameters.

Easy memory

singleton ★ default → ONE for everyone → printer
prototype → NEW each time → sheet of paper
request / session → web only

▲ Singleton + mutable field = thread-safety bug. Keep @Service STATELESS.

Q: Explain the Spring bean lifecycle.

The easiest way to remember:

CREATE → INJECT → INITIALISE → USE → DESTROY

1. Container INSTANTIATES the bean (calls the constructor)
2. Populates DEPENDENCIES (DI happens here)
3. Aware interfaces called (BeanNameAware, ApplicationContextAware)
4. @PostConstruct / afterPropertiesSet() ← your init hook ★
5. Bean is READY and used by the application
6. On shutdown: @PreDestroy / destroy() ← your cleanup hook ★

```
@Component
public class CacheManager {

    @PostConstruct
    public void init()    { /* warm up the cache – runs AFTER injection
★ */ }

    @PreDestroy
    public void cleanup() { /* flush the cache before shutdown */ }
}
```

Real-world idea: a **new employee**.

1. Hired → constructor
2. Given laptop + ID card → dependencies injected
3. Induction / training → @PostConstruct ★
4. Doing the actual job → bean in use
5. Exit formalities, hand back → @PreDestroy

Why `@PostConstruct` and not the constructor?

Constructor → runs BEFORE the dependencies are injected
(field injection isn't done yet → `NullPointerException`
▲)

`@PostConstruct` → runs AFTER everything is injected ✓

That is exactly why the hook exists.

Interview-ready answer

The container first instantiates the bean by calling its constructor, then performs dependency injection, then calls any Aware interfaces, then the initialisation callback — `@PostConstruct`, or `afterPropertiesSet()` if the bean implements `InitializingBean`. At that point the bean is ready and used by the application. On context shutdown the destruction callback runs — `@PreDestroy` or `destroy()`. The reason we use `@PostConstruct` rather than the constructor for initialisation logic is that the constructor runs before dependency injection completes, so injected fields would still be null.

Easy memory

CREATE → INJECT → `@PostConstruct` ★ → USE → `@PreDestroy`

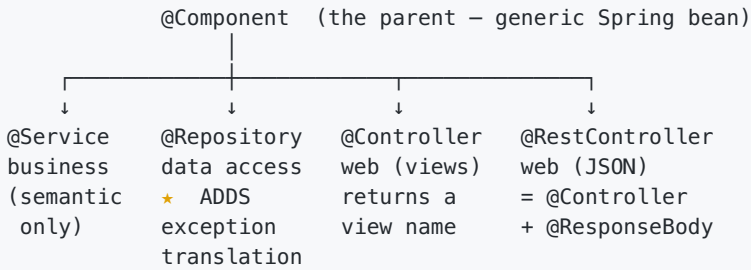
New employee: hired → laptop → induction → work → exit formalities

`@PostConstruct` runs AFTER injection (the constructor runs BEFORE it).

Q: `@Component` vs `@Service` vs `@Repository` vs `@Controller`?

The easiest way to remember:

They are all `@Component` underneath.
The difference is INTENT — plus ONE real behaviour difference.



The one that actually does something ★

```
@Repository
public class UserDao { }
```

@Repository adds **exception translation**: it converts vendor-specific JDBC/JPA exceptions into Spring's own **DataAccessException** hierarchy.

```
SQLException / HibernateException (vendor-specific, checked)
    ↓ @Repository translates
DataAccessException (Spring's own, unchecked) ✓
```

Why that matters: your service layer doesn't have to change if you switch from Hibernate to JDBC.

The others (**@Service**, **@Controller**) are **semantic only** — they behave exactly like **@Component**.

Real-world idea: **job titles in a company**. Manager, Engineer and Accountant are all *employees*; the title tells you what they do. But the Accountant also has a real extra duty nobody else has.

Interview-ready answer

All four are stereotype annotations and all are specialisations of **@Component**, so all of them make the class a Spring-managed bean. **@Component** is generic, **@Service** marks the business layer, **@Repository** marks the data-access layer, and **@Controller** marks a web MVC controller returning view names, with **@RestController** being **@Controller** plus **@ResponseBody**. The only one that adds real behaviour is **@Repository**, which enables exception translation — it

converts JDBC or Hibernate exceptions into Spring's unchecked `DataAccessException` hierarchy, so the service layer isn't coupled to a specific persistence technology. The others are semantic, but using the specific one makes the layering obvious and lets AOP target a layer precisely.

Easy memory

All four = @Component underneath.

@Service → business layer (semantic only)
@Repository → data layer ★ ADDS exception translation → `DataAccessException`
@Controller → returns a VIEW
@RestController → @Controller + @ResponseBody → returns JSON ★

Q: @Autowired — field vs constructor vs setter injection? Which is best?

The easiest way to remember:

CONSTRUCTOR injection is the best. ★

Because it lets the field be `final`, makes dependencies obvious, and makes the class testable without Spring.

```
// ✗ FIELD injection – concise but bad
@Service
public class OrderService {
    @Autowired private PaymentService payment;    // cannot be final,
    hard to test
}

// ● SETTER injection – fine for OPTIONAL dependencies
@Autowired
public void setPayment(PaymentService payment) { this.payment = payment; }

// ✓ CONSTRUCTOR injection – the recommended one
@Service
public class OrderService {
    private final PaymentService payment;    // final ✓
    private final InventoryService inventory;

    // @Autowired is optional for a SINGLE constructor (Spring 4.3+) ★
    public OrderService(PaymentService payment, InventoryService
inventory) {
        this.payment = payment;
        this.inventory = inventory;
    }
}
```

Why constructor injection wins — four reasons

1. The field can be FINAL → immutable, cannot be reassigned by accident
2. The object is FULLY BUILT the moment it exists → no half-initialised bean
3. TESTABLE without Spring:


```
new OrderService(mockPayment, mockInventory);    ✓
```

 With field injection you need reflection or `@SpringBootTest`
4. It EXPOSES bad design: a constructor with 8 parameters screams "this class does too much" – field injection HIDES that ★

That fourth point is the one that impresses.

Real-world idea:

```
Constructor injection → a car built WITH its engine already inside ✓
Field injection       → a car rolled off the line, with someone
                        pushing the engine in through the window later
```

Interview-ready answer

There are three types — field, setter and constructor injection. Constructor injection is the recommended one for several reasons: the dependency can be declared `final`, so the object is immutable and fully initialised the moment it exists; the dependencies are explicit in the signature; and it makes unit testing trivial because we can just call the constructor with mocks, without starting a Spring context. Field injection is concise but hides dependencies, prevents `final`, and requires reflection to test. Setter injection is reasonable for genuinely optional dependencies. Since Spring 4.3, `@Autowired` is not even required when a class has a single constructor. A practical benefit is that constructor injection makes a class with too many dependencies visibly ugly, which is a useful design signal that field injection hides.

● In your RoboGebra project — constructor injection, without writing a constructor ★

File: `domain/robochat/controller/RoboChatController.java`

```
@RestController
@RequestMapping("/api/robochat")
@AllArgsConstructor                                     // ★ Lombok generates the
constructor
public class RoboChatController {

    private final RoboChatService roboChatService;    // ★ final →
immutable, injected
}
```

`@AllArgsConstructor` + ``final`` fields = CONSTRUCTOR INJECTION ★

- no `@Autowired` anywhere (a single constructor doesn't need it, Spring 4.3+)
- the field is ``final``, so it can never be reassigned
- the class is testable with ``new RoboChatController(mockService)`` ★

■ "We use constructor injection everywhere, but written as Lombok's `@AllArgsConstructor` with `final` fields — so you get immutability and testability without the boilerplate. There's no `@Autowired` in the codebase on a constructor, because with a single constructor Spring doesn't need it."

★ 207 `@Service` classes and 110 `@RestController` s, all wired this way.

▲ The trade-off to name: `@AllArgsConstructor` injects **every** field, so adding a field silently changes the constructor signature. On a class where that matters, write the constructor by hand — or use `@RequiredArgsConstructor`, which only takes the `final` ones.

Easy memory

```
CONSTRUCTOR injection ★
    final ✓ | fully built ✓ | testable without Spring ✓ | exposes too
many deps ✓
    @Autowired NOT needed for a single constructor (4.3+)

Field injection ✗ can't be final, needs reflection to test
Setter injection ● optional dependencies only
```

Q: What is `@Qualifier` ?

The easiest way to remember:

```
Two beans of the SAME type → Spring can't choose → @Qualifier NAMES the
one you want.
```

```
@Component("smsSender") class SmsSender implements MessageSender { }
@Component("emailSender") class EmailSender implements MessageSender { }
```

```
@Service
public class NotificationService {

    private final MessageSender sender;

    public NotificationService(@Qualifier("smsSender") MessageSender
sender) {
        this.sender = sender;                                // ✓ no
ambiguity
    }
}
```

Without it:

```
NoUniqueBeanDefinitionException:  
    expected single matching bean but found 2: smsSender, emailSender ▲
```

```
MessageSender  
├── smsSender  
└── emailSender  
    ↑  
    @Qualifier("smsSender") points at exactly one ✓
```

Real-world idea: you ask for "**a driver**" and two drivers step forward. `@Qualifier` is you saying "**Ravi**, specifically."

Easy memory

```
2+ beans of one type → NoUniqueBeanDefinitionException ▲  
Fix: @Qualifier("beanName") ← names the exact bean
```

Q: What is `@Primary`?

The easiest way to remember:

```
@Primary = "if nobody says otherwise, use ME."
```

```
@Primary → the DEFAULT  
@Qualifier → the EXCEPTION
```

```
@Bean  
@Primary  
public MessageSender emailSender() { return new EmailSender(); }  
  
@Bean  
public MessageSender smsSender() { return new SmsSender(); }
```

```
// gets emailSender automatically ✓  
public NotificationService(MessageSender sender) { }  
  
// explicitly asks for the other one  
public AlertService(@Qualifier("smsSender") MessageSender sender) { }
```



```
@Primary    → set the default once, in ONE place
@Qualifier → override it where needed
```

Real-world idea: `@Primary` is your **default payment card**. `@Qualifier` is choosing a different card at checkout.

▲ If `@Primary` and `@Qualifier` are both present, `@Qualifier` **wins** — the explicit request always beats the default.

Easy memory

```
@Primary    → the default bean (used when nothing is specified)
@Qualifier → the explicit choice → BEATS @Primary ★
```

Default card vs "no, use this other card today"

Q: `@Configuration` and `@Bean`?

The easiest way to remember:

```
@Component → "Spring, YOU create this class"      (your own classes)
@Bean       → "I'LL create it, you just manage it" (third-party
classes ★ )
```

You cannot put `@Component` on `RestTemplate` — it's not your code. So:

```

@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate(); // I build it, Spring manages
it ✓
    }

    @Bean
    public ModelMapper modelMapper() {
        ModelMapper m = new ModelMapper();
        m.getConfiguration().setSkipNullEnabled(true); // custom setup
        return m;
    }
}

```

Your own class → @Component / @Service / @Repository
 Third-party class → @Bean inside a @Configuration class ★
 Needs custom setup → @Bean (you control the construction)

Real-world idea: `@Component` is hiring a full-time employee. `@Bean` is bringing in a machine you bought elsewhere and registering it as company property.

The `@Configuration` bonus detail ★

```

@Configuration
class AppConfig {
    @Bean A a() { return new A(b()); }
    @Bean B b() { return new B(); } // called twice?
}

```

`@Configuration` classes are **CGLIB-proxied**, so calling `b()` inside `a()` returns the **same singleton**, not a new object. With plain `@Component` on the config class (a "lite" mode), you would get two different `B` instances. That is a nice detail to know.

Easy memory

@Configuration = a class that DEFINES beans
@Bean = a method that RETURNS a bean

Use @Bean for: third-party classes (RestTemplate, ModelMapper) + custom construction
@Configuration is CGLIB-proxied → calling another @Bean method returns the SAME singleton ★

Q: What is AOP (Aspect-Oriented Programming)?

The easiest way to remember:

AOP = pull the REPEATED code (logging, security, transactions) OUT of every method, and apply it from OUTSIDE.

The problem

```
public void createOrder() {  
    log.info("start");           // repeated  
    checkSecurity();             // repeated  
    beginTransaction();          // repeated  
  
    // ...the 3 lines that are actually the business logic...  
  
    commit();                    // repeated  
    log.info("end");             // repeated  
}
```

That noise appears in **every single method** of the application. These are called **cross-cutting concerns** — they cut across all layers.

| | |
|------------|---------------------------------------|
| Controller | |
| Service | ← logging cuts across ALL of them |
| Repository | ← security cuts across ALL of them |
| | ← transactions cut across ALL of them |

The AOP solution

```
@Aspect
@Component
public class LoggingAspect {

    @Around("execution(* com.app.service.*(..))")
    public Object logTime(ProceedingJoinPoint jp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = jp.proceed();           // run the real
method
        log.info("{} took {} ms", jp.getSignature(),
System.currentTimeMillis() - start);
        return result;
    }
}
```

Now **every** service method is timed, and no business method contains a single line of logging code.

```
Caller
↓
PROXY ← @Around advice runs here (before)
↓
Real method (pure business logic ✓ )
↓
PROXY ← @Around advice runs here (after)
↓
Caller
```

Real-world idea: **CCTV cameras in a shop.**

```
Without AOP → every employee must also write down who entered
With AOP    → cameras at the doors record everything.
              Employees just do their job ✓
```

The vocabulary

```
Aspect    → the module (the LoggingAspect class)
Advice    → the action: @Before, @After, @AfterReturning, @AfterThrowing,
@Around
Pointcut  → WHERE it applies: execution(* com.app.service.*(..))
JoinPoint → the actual point in execution (the method being called)
```

★ **@Transactional** is itself implemented with AOP — that is the sentence that connects this to everything else, and it explains the self-invocation trap below.

The self-invocation trap ★

```
@Service
public class OrderService {

    public void a() {
        this.b();          // ✗ AOP does NOT apply – bypasses the proxy!
    }

    @Transactional
    public void b() { }
}
```

Because AOP works via a **proxy**, an internal `this.` call never leaves the object, so the proxy never sees it. The same trap applies to **@Async** and **@Cacheable**.

Interview-ready answer

AOP lets us modularise cross-cutting concerns — logging, security, transactions, metrics — that would otherwise be repeated in every class. Instead of putting that code inside every method, we write an Aspect containing Advice, and a Pointcut expression that says where it applies. Spring implements this with proxies: when a bean is advised, Spring wraps it in a proxy that runs the advice before or after the real method. **@Transactional** and **@Async** are themselves built on AOP. The important consequence of the proxy model is self-invocation: if a method calls another method of the same class using `this`, the call doesn't pass through the proxy, so the annotation has no effect.

Easy memory

AOP = CCTV cameras – watch everything without changing the shop staff

Aspect (the class) | Advice (@Before/@After/@Around) | Pointcut (where) | JoinPoint

@Transactional and **@Async** ARE AOP ★

▲ SELF-INVOCATION: `this.method()` bypasses the proxy → the annotation is ignored

Spring Boot

Q: What is Spring Boot and what are its advantages over Spring?

The easiest way to remember:

Spring

= a box of parts you must assemble yourself

Spring Boot

= the same parts, PRE-ASSEMBLED, with a working engine ✓

Spring Boot is an **opinionated** layer on top of Spring: it makes sensible default choices so you don't configure everything by hand.

| | Spring | Spring Boot |
|------------------------|-------------------------------------|--|
| Configuration | lots of XML / Java config | auto-configuration ✓ |
| Dependencies | pick every jar and version yourself | starters — curated bundles ✓ |
| Server | deploy a WAR to an external Tomcat | embedded server, <code>java -jar</code> ✓ |
| Time to first endpoint | hours | minutes |
| Monitoring | build your own | Actuator built in ✓ |

Real-world idea:

Spring

→ buying flour, sugar, eggs and baking the cake yourself

Spring Boot

→ a ready cake mix: add water and bake ✓
(and you can still add your own ingredients)

That last part matters: Boot never *stops* you overriding a default.

Interview-ready answer

Spring Boot is an opinionated layer on top of the Spring Framework that removes boilerplate configuration and lets us build stand-alone, production-ready applications quickly. Its main features are auto-configuration, which configures beans based on what is on the classpath; starter dependencies, which bundle

compatible libraries so we don't manage versions; an embedded server, so the application runs as an executable jar with `java -jar` instead of being deployed to an external Tomcat; and Actuator, which provides health and metrics endpoints out of the box. Importantly it is convention over configuration, not a restriction — if we define our own bean, Boot backs off and ours wins.

Easy memory

Spring Boot = ready CAKE MIX

Auto-configuration | Starters | Embedded server | No XML | Actuator
Opinionated, but YOUR bean always overrides the default ★

Q: How does auto-configuration work?

The easiest way to remember:

Boot LOOKS at your classpath and asks:
"Is this library present? Did the developer already define this bean?"
If yes / no → it configures it FOR you.

Application starts

↓

@SpringBootApplication → @EnableAutoConfiguration

↓

reads META-INF/spring/...AutoConfiguration.imports (the list of candidates)

↓

for each candidate, checks the @Conditional annotations:

 @ConditionalOnClass → is this class on the classpath?

 @ConditionalOnMissingBean → did the developer NOT define one? ★

 @ConditionalOnProperty → is this property set?

↓

if all conditions pass → the bean is created automatically ✓

A concrete example:

You add `spring-boot-starter-data-jpa` + the H2 driver

↓

`@ConditionalOnClass(DataSource.class)` ✓ present

`@ConditionalOnMissingBean(DataSource.class)` ✓ you defined none

↓

Boot creates a `DataSource`, an `EntityManagerFactory` and a `TransactionManager` for you

★ `@ConditionalOnMissingBean` is the key to the whole design: **your bean always wins**. Define your own `DataSource` and Boot silently steps aside.

Real-world idea: **a smart home**.

It detects you own a TV → sets up the remote

It detects you own an AC → sets up the thermostat

You already installed your own → it leaves yours alone ✓

How to debug it (a great practical point)

```
debug=true
```

This prints the **auto-configuration report** at startup: every candidate, and whether it matched or not and *why*. Mentioning this shows real experience.

Interview-ready answer

Auto-configuration is driven by `@EnableAutoConfiguration`, which is part of `@SpringBootApplication`. At startup, Spring Boot reads a list of auto-configuration classes from `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` and applies each one conditionally using annotations such as `@ConditionalOnClass`, `@ConditionalOnMissingBean` and `@ConditionalOnProperty`. So if the JPA starter and a database driver are on the classpath and we have not defined a `DataSource` ourselves, Boot configures one. The `@ConditionalOnMissingBean` condition is what makes it safe — our own beans always take precedence. If I need to see what was applied or why something did not activate, I run with `debug=true`, which prints the auto-configuration report.

Easy memory

@EnableAutoConfiguration → reads AutoConfiguration.imports → applies @Conditional checks

@ConditionalOnClass → library present?

@ConditionalOnMissingBean → you didn't define one? ★ (YOUR bean always wins)

@ConditionalOnProperty → property set?

Debug it with: debug=true → prints the auto-configuration report ★

Q: What are Spring Boot starters?

The easiest way to remember:

A starter = ONE dependency that pulls in a whole working set of libraries, with COMPATIBLE versions already chosen for you.

```
<!-- ONE line... -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

```
...brings in:
  spring-web + spring-webmvc
  embedded Tomcat
  Jackson (JSON)
  validation
  logging
  - all at versions tested together ✓
```

Without starters you would list ~15 dependencies and hand-check every version for conflicts.

The ones to be able to name:

spring-boot-starter-web	REST + embedded Tomcat + Jackson
spring-boot-starter-data-jpa	Hibernate + Spring Data + JDBC
spring-boot-starter-security	authentication + authorisation
spring-boot-starter-validation	@Valid, @NotNull, @Email
spring-boot-starter-test	JUnit 5 + Mockito + AssertJ + MockMvc
spring-boot-starter-actuator	health, metrics, info endpoints
spring-boot-starter-webflux	reactive stack

Real-world idea: a **thali / combo meal**. You order one thing and get rice, dal, curry, curd and sweet — already chosen to go together.

Easy memory

Starter = a COMBO MEAL – one dependency, a whole working stack, versions matched

web | data-jpa | security | validation | test | actuator | webflux

Q: What does `@SpringBootApplication` consist of?

The easiest way to remember:

`@SpringBootApplication` = 3 annotations in one:

`@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`

```
@SpringBootApplication
|
├── @Configuration           → this class can define beans
├── @EnableAutoConfiguration → turn on auto-configuration ★
└── @ComponentScan          → scan THIS package and below for
components
```

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

The trap ★ — package placement

```
com.app                ← put App.java HERE
├── controller         ✓ scanned
├── service             ✓ scanned
└── repository         ✓ scanned

com.other.util        ✗ NOT scanned → "no qualifying bean of type..."
▲
```

`@ComponentScan` starts at the **main class's own package** and works downwards. This is the single most common "why isn't my bean found?" bug. Fix it by moving the class, or by using `@ComponentScan(basePackages = "...")`.

Interview-ready answer

`@SpringBootApplication` is a meta-annotation combining three: `@Configuration`, which allows the class to define beans; `@EnableAutoConfiguration`, which turns on Spring Boot's auto-configuration; and `@ComponentScan`, which scans the main class's package and its sub-packages for components. A practical consequence is that the main class should sit in the root package of the application — if a component lives outside that package tree it will not be picked up, which is the most common cause of a "no qualifying bean" error at startup.

Easy memory

```
@SpringBootApplication = @Configuration + @EnableAutoConfiguration +
@ComponentScan
```

▲ Main class must be in the ROOT package — scanning goes DOWNWARDS only

Q: `application.properties` vs `application.yml` ?

The easiest way to remember:

```
Same job, different format.
.properties → FLAT key=value
.yml        → INDENTED, hierarchical (less repetition)
```

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/shop
spring.datasource.username=root
```

```
server:
  port: 8081
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/shop
    username: root
```

.properties → you repeat "spring.datasource." on every line
.yaml → you write it once and nest ✓

▲ Two things to remember about YAML:

1. INDENTATION MATTERS – and you must use SPACES, never tabs ▲
2. If BOTH files exist, .properties WINS for the same key

Easy memory

.properties → flat, no indentation risk
.yaml → nested, cleaner for deep config, but tabs BREAK it ▲

Both present → .properties wins ★

Q: What are Spring profiles?

The easiest way to remember:

Profiles = different SETTINGS for different ENVIRONMENTS
(dev / test / prod) – with no code change.

application.properties	← common settings, always loaded
application-dev.properties	← loaded only when the dev profile is active
application-prod.properties	← loaded only when the prod profile is active

```
# application-dev.properties
spring.datasource.url=jdbc:h2:mem:testdb
logging.level.root=DEBUG

# application-prod.properties
spring.datasource.url=jdbc:postgresql://prod-db:5432/shop
logging.level.root=WARN
```

Activate it:

```
java -jar app.jar --spring.profiles.active=prod
# or an env var: SPRING_PROFILES_ACTIVE=prod
```

Profiles on beans too ★

```
@Service
@Profile("dev")
public class MockPaymentService implements PaymentService { }

@Service
@Profile("prod")
public class RealPaymentService implements PaymentService { }
```

```
dev → the mock is loaded, no real money moves ✓
prod → the real one is loaded ✓
```

Real-world idea: the **same play, different stages**. Same script; the lighting and props change for the rehearsal hall versus the real theatre.

Easy memory

application-{profile}.properties	→ per-environment settings
--spring.profiles.active=prod	→ activate
@Profile("dev") on a bean	→ load that bean only in dev ★

Common config in application.properties + the profile file layered on top

Q: How does the embedded server work, and how do you change it?

The easiest way to remember:

The server is **INSIDE** your jar.

Old way → build a WAR, install Tomcat, deploy into it

Boot → java -jar app.jar ← done ✓

```
app.jar
├─ your classes
├─ your dependencies
└─ Tomcat itself ★
```

That is what makes Spring Boot suitable for containers and microservices — the image just runs the jar.

Switching to Jetty or Undertow means **excluding Tomcat and adding the other starter**:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions><exclusion>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
  </exclusion></exclusions>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

Auto-configuration then detects Jetty on the classpath and configures it instead.

Easy memory

Server is packaged **INSIDE** the jar → java -jar app.jar ✓

Default = Tomcat

Change it = EXCLUDE spring-boot-starter-tomcat + ADD jetty/undertow starter

Why it matters: Docker/microservices — no external server to install

Q: How do you change the server port?

```
server.port=8082
```

```
java -jar app.jar --server.port=9090    # command line wins over the  
file ★
```

```
server.port=0    # a RANDOM free port – very useful in integration tests
```

★ Worth knowing the **precedence order** (highest first):

```
command-line arguments > environment variables > application-  
{profile}.properties  
                                                                >  
application.properties
```

Easy memory

server.port=8082	in the file
--server.port=9090	on the command line → WINS ★
server.port=0	random free port (great for tests)

● In your RoboGebra code — auto-configuration back-off, for real

File: `robogebra-portal/.../config/AsyncConfig.java`

The best possible proof that you understand `@ConditionalOnMissingBean`:

```

/**
 * Boot's own applicationTaskExecutor, declared BY HAND.
 * It has to be. TaskExecutorConfiguration is
 * @ConditionalOnMissingBean(Executor.class),
 * so declaring ANY executor backs the auto-configured one out entirely –
 * and then
 * every unqualified @Async silently resolves to the notification pool.
 */
@Bean(name = {"applicationTaskExecutor", "taskExecutor"})
@Lazy
public ThreadPoolTaskExecutor
applicationTaskExecutor(ThreadPoolTaskExecutorBuilder builder) {
    return builder.build();
}

```

■ "Auto-configuration backing off isn't just theory — it bit us. The moment we declared a second executor bean, Boot's *applicationTaskExecutor* disappeared and every unqualified *@Async* in the app silently started using our notification pool. We had to rebuild Boot's default from its own builder to keep the *spring.task.execution.** properties working."

See [39 — RoboGebra Code Examples](#) for the whole story.

Q: What is Spring Boot Actuator?

The easiest way to remember:

Actuator = the DASHBOARD of your running application.

/actuator/health	→ is the app alive? is the DB reachable? ★
/actuator/metrics	→ memory, CPU, request counts, response times
/actuator/info	→ build version, git commit
/actuator/env	→ all configuration properties
/actuator/loggers	→ view AND CHANGE log levels at runtime ★
/actuator/beans	→ every bean in the context
/actuator/mappings	→ every URL mapping


```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

```
management.endpoints.web.exposure.include=health,info,metrics
```

▲ Only **health** and **info** are exposed over HTTP by default — the rest must be opted in, and should be secured, because **/actuator/env** reveals configuration.

Real-world idea: the **dashboard of a car** — fuel, temperature, warning lights. You don't open the bonnet to check whether the engine is running.

Where it's actually used:

```
Kubernetes / load balancer → hits /actuator/health to decide if the pod is
alive ★
Prometheus + Grafana          → scrape /actuator/prometheus for graphs
Production incident           → change a log level to DEBUG WITHOUT a
restart ★
```

Easy memory

Actuator = the app's DASHBOARD

```
/health ★ (used by k8s / load balancers) | /metrics | /info | /loggers
★ | /env
```

Only health + info exposed by default → opt in with
management.endpoints.web.exposure.include=...

▲ SECURE it — /env leaks configuration

Q: What is **CommandLineRunner** ?

The easiest way to remember:

```
CommandLineRunner = "run this code ONCE, right after the app has started."
```

```

@Component
public class DataSeeder implements CommandLineRunner {

    private final UserRepository repo;
    public DataSeeder(UserRepository repo) { this.repo = repo; }

    @Override
    public void run(String... args) {
        repo.save(new User("admin")); // runs once, after startup ✓
    }
}

```

```

SpringApplication.run(...)
    ↓
context created, all beans ready
    ↓
CommandLineRunner.run() ★ your code here
    ↓
application is serving requests

```

```

CommandLineRunner → run(String... args)           → raw arguments
ApplicationRunner → run(ApplicationArguments args) → parsed arguments
★

```

Used for: seeding test data, warming a cache, printing startup banners, running a one-off migration.

Real-world idea: **switching on the lights and unlocking the door** just before the shop opens.

Easy memory

```

CommandLineRunner → run() executes ONCE after the context is ready
ApplicationRunner → the same, but with PARSED arguments

```

Use for: seed data, cache warm-up, startup checks

REST APIs

Q: @RestController vs @Controller?

The easiest way to remember:

```
@Controller      → returns a VIEW NAME (an HTML page)      → JSP / Thymeleaf
@RestController → returns DATA (JSON)                      → REST API ★

@RestController = @Controller + @ResponseBody
```

```
@Controller
public class PageController {
    @GetMapping("/home")
    public String home() {
        return "home";           // → looks for home.html / home.jsp
    }
}

@RestController
public class UserController {
    @GetMapping("/api/users/1")
    public User get() {
        return new User(1L, "Priya"); // → {"id":1,"name":"Priya"} ✓
    }
}
```

```
@Controller      → "home" → the view resolver finds a template → HTML page
@RestController → object → Jackson converts it → JSON in the response
body
```

▲ The classic bug: using @Controller for an API and getting a **404 for a view named after your data**, because Spring tried to find a template called "Priya".

Real-world idea:

```
@Controller      → a waiter who brings you a plated dish (a full page)
@RestController → a takeaway counter handing you a packed box (raw data)
```

Easy memory

@RestController = @Controller + @ResponseBody ★ → JSON
@Controller alone → view name → HTML page

Using @Controller for an API → it tries to find a TEMPLATE → 404 ▲

Q: What are the request mapping annotations?

The easiest way to remember:

GET → READ
POST → CREATE
PUT → FULL update
PATCH → PARTIAL update
DELETE → REMOVE

```
@RestController
@RequestMapping("/api/users")           // base path for the whole class
★
public class UserController {

    @GetMapping("/{id}")    public User get(@PathVariable Long id)
    { ... }
    @GetMapping             public List<User> all()
    { ... }
    @PostMapping            public User create(@RequestBody User u)
    { ... }
    @PutMapping("/{id}")    public User update(@PathVariable Long id,
                                                @RequestBody User u)
    { ... }
    @PatchMapping("/{id}")  public User patch(...)
    { ... }
    @DeleteMapping("/{id}") public void delete(@PathVariable Long id)
    { ... }
}
```

```
@RequestMapping(method = RequestMethod.GET) ← the old, verbose way
@GetMapping                                           ← the shortcut ✓ (Spring
4.3+)
```

PUT vs PATCH — the follow-up ★

PUT → send the WHOLE object; missing fields get wiped
{ "name": "Priya" } → email becomes NULL ▲

PATCH → send ONLY what changed; everything else is untouched
{ "name": "Priya" } → only the name changes ✓

● In your RoboGebra project — a real controller, annotation by annotation ★

File: `domain/robochat/controller/RoboChatController.java`

```
@RestController //
★ = @Controller + @ResponseBody
@RequestMapping("/api/robochat") //
★ base path for the class
@AllArgsConstructor // constructor
injection
@Tag(name = "RoboChat", description = "RoboChat AI Assistant") // ★
OpenAPI/Swagger
public class RoboChatController {

    private final RoboChatService roboChatService;

    @PostMapping(value = "/chat", produces =
MediaType.TEXT_EVENT_STREAM_VALUE) // ★ SSE
    @PermissionContext(userPermission =
{Permission.ROBOCHAT_WRITE}) // ★ custom
    public Flux<ServerSentEvent<String>> chat(
        @Valid @RequestBody RoboChatMessageRequestDTO
dto, // ★ validated body
        @RequestAttribute(ACTIVE_USER_ID_ATTRIBUTE) String
activeUserId, // ★ set by a FILTER
        @RequestAttribute(LOGGED_USER_ATTRIBUTE) JwtAuth loggedUser) {
        return roboChatService.streamChat(activeUserId, loggedUser, dto);
    }

    @GetMapping("/conversations/{conversationId}")
    @PermissionContext(userPermission = {Permission.ROBOCHAT_READ})
    public Object getConversation(@PathVariable String
conversationId, ...) { ... }

    @DeleteMapping("/conversations/{conversationId}")
    public Object deleteConversation(@PathVariable String
conversationId, ...) { ... }
}
```

★ Four things in there worth pointing at:

1. `@RequestAttribute` ★ – the user id is NOT a path or query param.
A security filter validated the JWT and put it on the request, so the controller can NEVER be tricked into acting on someone else's id ▲
→ a `userId` in the URL would be an authorisation bug waiting to happen
★
2. `@PermissionContext` ★ – a CUSTOM annotation. Permissions are declared on the endpoint and enforced centrally, not with an if-check in the method.
3. `produces = TEXT_EVENT_STREAM_VALUE + Flux<ServerSentEvent<String>>` ★
The AI chat STREAMS tokens back as they're generated, so the user sees the answer appear rather than waiting for the whole response.
(This is the one place we use WebFlux inside an otherwise MVC app.)
4. `@Valid @RequestBody` ★ – validation happens before the method body runs;
a failure becomes `MethodArgumentNotValidException` → the `ControllerAdvice` → 400

The DTO it validates:

```
@Data
public class RoboChatMessageRequestDTO {
    @NotBlank // ★ not null, not empty, not
    whitespace
    private String message;
    private String conversationId; // optional – a new conversation if
    absent
    private String attachmentId;
}
```

■ "The detail I'd point at is `@RequestAttribute`. The active user id comes from a security filter that validated the JWT, not from the URL — so a caller can't pass someone else's id and read their conversations. And the chat endpoint returns a `Flux<ServerSentEvent>` because the AI response streams token by token; that's the only place we use WebFlux in an otherwise servlet-based app."

Easy memory

@GetMapping | @PostMapping | @PutMapping | @PatchMapping | @DeleteMapping
@RequestMapping("/api/users") on the CLASS = the base path ★

PUT = replace the WHOLE thing (missing fields are wiped)
PATCH = update only the fields you send

Q: @PathVariable vs @RequestParam vs @RequestBody ?

The easiest way to remember by **where the data sits in the request**:

/users/42 ← @PathVariable (part of the PATH)
/users?status=new ← @RequestParam (after the ? – QUERY string)
{ "name": "Priya" } ← @RequestBody (the JSON BODY)

GET /api/users/42?detailed=true
 ↑ ↑
 @PathVariable @RequestParam

POST /api/users
Body: {"name": "Priya", "email": "p@x.com"}
 ↑
 @RequestBody

```
@GetMapping("/users/{id}")
public User get(@PathVariable Long id,
               @RequestParam(defaultValue = "false") boolean detailed)
{ ... }

@PostMapping("/users")
public User create(@RequestBody @Valid UserDto dto) { ... }
```

When to use which

@PathVariable → identifies WHICH resource → /users/42, /orders/99/items/3
@RequestParam → filters / options / paging → ?status=active&page=2&size=20
@RequestBody → the data being sent → POST / PUT payloads

Real-world idea: a **postal address**.

House number (path) → WHICH house → @PathVariable
"deliver in the evening" → an INSTRUCTION → @RequestParam
The parcel contents → the actual DATA → @RequestBody

▲ Two traps:

@RequestParam is REQUIRED by default → a missing param = 400 Bad Request
Fix: @RequestParam(required = false) or defaultValue = "..."

Only ONE @RequestBody per method – a request has only one body.

Easy memory

/users/42 → @PathVariable → WHICH resource
?status=active → @RequestParam → filter/option (required by default
▲)
JSON body → @RequestBody → the data (only ONE per method)

Q: What is **ResponseEntity** and why use it?

The easiest way to remember:

Returning an object → you only control the BODY (status is always 200)
ResponseEntity → you control STATUS + HEADERS + BODY ★


```
// Plain return → always 200 OK
@PostMapping("/users")
public User create(@RequestBody UserDto dto) {
    return service.create(dto);           // 200 – but it should be 201!
}

// ResponseEntity → full control ✓
@PostMapping("/users")
public ResponseEntity<User> create(@RequestBody @Valid UserDto dto) {
    User saved = service.create(dto);
    return ResponseEntity.status(HttpStatus.CREATED)           // 201 ✓
        .header("Location", "/api/users/" +
saved.getId())
        .body(saved);
}
```

```
ResponseEntity
├── STATUS → 200 / 201 / 204 / 404 ...
├── HEADERS → Location, Cache-Control, custom headers
└── BODY → the object
```

Common shortcuts:

```
ResponseEntity.ok(user);           // 200 with a body
ResponseEntity.status(HttpStatus.CREATED).body(user); // 201
ResponseEntity.noContent().build(); // 204, no body
ResponseEntity.notFound().build(); // 404
ResponseEntity.badRequest().body(error); // 400
```

A very common real pattern:

```
return repo.findById(id)
    .map(ResponseEntity::ok)           // found → 200 +
body
    .orElse(ResponseEntity.notFound().build()); // absent → 404
✓
```

Real-world idea: a plain return is handing over the parcel. `ResponseEntity` is the **full courier slip** — parcel + delivery status + tracking details.

Easy memory

```
ResponseEntity<T> = STATUS + HEADERS + BODY ★  
  
ok() | status(CREATED).body() | noContent() | notFound() | badRequest()  
  
POST that creates something should return 201 + a Location header – not 200.
```

Q: Which HTTP status codes should a REST API use?

The easiest way to remember by **the first digit**:

```
2xx → SUCCESS      "here you go"  
3xx → REDIRECT     "look over there"  
4xx → CLIENT error  "YOUR mistake" ★  
5xx → SERVER error  "MY mistake" ★
```

The ones you must know:

Code	Meaning	When
200 OK	success	successful GET / PUT
201 Created	resource created	successful POST ★
204 No Content	success, no body	successful DELETE
400 Bad Request	invalid input	validation failure
401 Unauthorized	not logged in	missing/invalid token ★
403 Forbidden	logged in, not allowed	wrong role ★
404 Not Found	resource missing	bad id
409 Conflict	duplicate / version clash	email already exists
500 Internal Server Error	unexpected crash	an unhandled exception

The 401 vs 403 trap ★ (asked constantly)

```
401 UNAUTHORIZED → "I don't know WHO you are"      → log in first  
403 FORBIDDEN    → "I know who you are, and NO"      → you lack the role
```

Real-world idea: a **nightclub**.

401 → you have no ID → the bouncer can't identify you
403 → your ID is valid, but you're not on the VIP list ✗

Easy memory

2xx success | 4xx YOUR fault | 5xx MY fault

200 OK | 201 Created (POST) ★ | 204 No Content (DELETE)
400 bad input | 401 NOT LOGGED IN | 403 LOGGED IN BUT NOT ALLOWED ★
404 missing | 409 duplicate | 500 crash

401 = who are you? | 403 = I know you, still no.

Q: How do you handle exceptions globally?

The easiest way to remember:

@RestControllerAdvice = ONE place that catches exceptions from EVERY controller.

The problem

```
// ✗ try/catch repeated in every single controller method
@GetMapping("/{id}")
public ResponseEntity<?> get(@PathVariable Long id) {
    try {
        return ResponseEntity.ok(service.find(id));
    } catch (ResourceNotFoundException e) {
        return ResponseEntity.status(404).body(e.getMessage());
    } catch (Exception e) {
        return ResponseEntity.status(500).body("error");
    }
}
```

The solution

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError>
    handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiError(404, ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError>
    handleValidation(MethodArgumentNotValidException ex) {
        String msg =
        ex.getBindingResult().getFieldError().getDefaultMessage();
        return ResponseEntity.badRequest().body(new ApiError(400, msg));
    }

    @ExceptionHandler(Exception.class)           // the safety net ★
    public ResponseEntity<ApiError> handleAll(Exception ex) {
        log.error("Unexpected error", ex);         // LOG the real cause
        return ResponseEntity.status(500)
            .body(new ApiError(500, "Something went wrong")); //
    }
    hide internals
}
}
```

Now the controller is clean:

```
@GetMapping("/{id}")
public User get(@PathVariable Long id) {
    return service.find(id);           // just throw – the advice handles it
}
✓
}
```

Controller throws `ResourceNotFoundException`

↓

`@RestControllerAdvice` catches it

↓

Client receives: `404 { "code": 404, "message": "User 42 not found" }` ✓
(not a Java stack trace)

Real-world idea: a **hospital reception**. Instead of every doctor handling paperwork for every complaint, one reception desk handles all of them, consistently.

★ Two points that score:

1. NEVER leak a stack trace to the client – it exposes class names, framework versions and sometimes SQL. Log it, return a generic message.
2. Handlers are matched MOST SPECIFIC first, so a catch-all `@ExceptionHandler(Exception.class)` is a safe last resort.

Easy memory

`@RestControllerAdvice + @ExceptionHandler(X.class) = ONE global error handler` ★

Service THROWS → advice CATCHES → clean JSON + correct status code
Always add a catch-all `@ExceptionHandler(Exception.class)` → log it, return a generic 500
Never send a stack trace to the client.

● In your RoboGebra code — exception handling has a second layer ★

Files: `common/exception/handler/ExceptionHandlerAdvice.java` and `security/FilterChainExceptionHandler.java`

```
Request
↓
[ SECURITY FILTER CHAIN ] ← an exception thrown HERE never reaches
@ExceptionHandler ▲
↓
[ DispatcherServlet ]
↓
[ Controller ]           ← @ExceptionHandler only covers from HERE
onwards
```

■ "`@ExceptionHandler` only catches exceptions thrown inside the MVC dispatch. Anything thrown in a security filter happens before the `DispatcherServlet`, so the advice never sees it and the client gets a bare 500 with no JSON body. We added

a dedicated *FilterChainExceptionHandler* in the filter chain for exactly that gap."

★ Most candidates have never thought about this. It is a very strong thing to volunteer.

Q: How do you do request validation?

The easiest way to remember:

Annotate the DTO + put `@Valid` on the controller parameter.
Spring does the rest.

```
public class UserDto {
    @NotBlank(message = "Name is required")
    private String name;

    @Email(message = "Invalid email")
    private String email;

    @Min(value = 18, message = "Must be 18 or older")
    private int age;
}
```

```
@PostMapping("/users")
public User create(@RequestBody @Valid UserDto dto) { ... }
//                               ↑ WITHOUT this, the annotations do
NOTHING ▲
```

```
Request arrives
↓
@Valid triggers the Bean Validation checks
↓
FAIL → MethodArgumentNotValidException thrown
      ↓ caught by @RestControllerAdvice
      400 Bad Request + a readable message ✓

PASS → your method body runs
```

The annotations worth naming:

```
@NotNull    → not null (but "" is allowed)
@NotEmpty   → not null AND not empty
@NotBlank   → not null, not empty, not just whitespace ★ (use this for
Strings)
@Size(min, max) | @Min | @Max | @Email | @Pattern(regexp)
@Past | @Future (dates)
@Valid on a nested object → validates it too ★
```

▲ Requires `spring-boot-starter-validation` — it stopped being included automatically in Boot 2.3+, which catches people out.

The `@NotNull` vs `@NotBlank` trap ★

<code>@NotNull</code>	→ ""	passes	✗	(it's not null!)
<code>@NotEmpty</code>	→ " "	passes	✗	(a space is not empty!)
<code>@NotBlank</code>	→ " "	FAILS	✓	← what you actually want for Strings

● In your RoboGebra project — validation, end to end ★

```
// 1. the DTO declares the rules
@Data
public class RoboChatMessageRequestDTO {
    @NotBlank // ★ jakarta.validation – NOT javax
    (Boot 3) ★
    private String message;
    private String conversationId;
}

// 2. the controller TRIGGERS them
@PostMapping("/chat")
public Flux<ServerSentEvent<String>> chat(@Valid @RequestBody
RoboChatMessageRequestDTO dto, ...) { }
//                                     † without @Valid, the
annotations do NOTHING ▲

// 3. a failure is handled ONCE, centrally
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Object>
handleValidationException(MethodArgumentNotValidException ex,
                                                                    WebRequest
request) {
    log.error("Validation exception: {}", ex.getMessage(), ex);
    String message = findErrorMessageForValidation(ex); // ★ pull
out the field error
    return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                                                                    .body(createErrorMessage(HttpStatus.BAD_REQUEST,
message, request));
}
```

92 × @Valid across the controllers ★
jakarta.validation-api 3.0.2 – jakarta, NOT javax ★ (Spring Boot 3)

■ "Validation is declared on the DTO and triggered with `@Valid` on the parameter — both are needed; the annotations do nothing on their own. The failure surfaces as `MethodArgumentNotValidException`, which the controller advice turns into a 400 with our standard error model, so no controller contains validation plumbing. And on Boot 3 the imports are `jakarta.validation`, not `javax` — that rename was the biggest cost of our Boot 2 to 3 migration."

Easy memory

DTO gets the annotations + controller parameter gets `@Valid` ★ (both needed!)

Failure → `MethodArgumentNotValidException` → handle it in `@RestControllerAdvice` → 400

For Strings use `@NotBlank` (not `@NotNull` – "" would pass)
Needs the `spring-boot-starter-validation` dependency ▲

● In your RoboGebra code — validation at scale

92 × `@Valid` across the controllers
`spring-boot-starter-validation` + `jakarta.validation-api 3.0.2`

▲ Note **jakarta**, not **javax** — the portal is Spring Boot **3.2.0** on **Java 17**. That package rename is the biggest cost of a Boot 2 → 3 migration. → [12 — Java 17](#)

Q: What is idempotency in REST?

The easiest way to remember:

Idempotent = doing it ONCE or TEN times gives the SAME final result.

GET	→ idempotent ✓	reading twice changes nothing
PUT	→ idempotent ✓	setting name="Priya" twice = still "Priya"
DELETE	→ idempotent ✓	deleting an already-deleted record = still deleted
POST	→ NOT idempotent ✗	each call creates a NEW record

POST `/orders` called 3 times → 3 ORDERS created ▲
PUT `/users/1` called 3 times → 1 user, same final state ✓

Real-world idea: a **light switch**.

Pressing "switch OFF" ten times → the light is still just off ✓
(idempotent)
Pressing "buy a ticket" ten times → ten tickets ▲ (not idempotent)

Why this matters in real life ★

User clicks "Pay" → the network times out → they click again
↓
The first request DID succeed
↓
The customer is charged TWICE ▲

The fix is an **idempotency key**:

```
POST /payments
Idempotency-Key: 7f3e9a2b-...
```

Server: have I seen this key before?
YES → return the ORIGINAL response, don't charge again ✓
NO → process it, and store the key with the result

This is exactly what Stripe and Razorpay do.

Easy memory

Idempotent = same result whether called once or many times

GET ✓ | PUT ✓ | DELETE ✓ | POST ✗

Make POST safe with an Idempotency-Key header ★ (payments!)
Light switch: "off" ten times = still off.

Q: What are some REST API best practices?

1. NOUNS, plural, for resources
 - ✓ `/api/users` `/api/users/42/orders`
 - ✗ `/api/getUser` `/api/deleteUserById`
2. Let the HTTP METHOD be the verb – never put an action in the URL
 - ✓ `DELETE /users/42`
 - ✗ `GET /users/deleteUser?id=42`
3. Correct STATUS CODES + a consistent error body
4. PAGINATION, filtering and sorting on every collection
 - `/api/users?page=0&size=20&sort=name,asc`
 - (returning 100,000 rows in one response is a real production incident)
5. VERSION the API `→ /api/v1/users`
6. STATELESS – no server session; use a token on every request
7. Use DTOs – NEVER expose entities directly ★
8. HTTPS everywhere

The one they probe: **why not expose entities?**

Entity exposed directly:

- leaks your DB schema to the world
- a client could `POST {"role":"ADMIN"}` and change a field you never intended to expose (over-posting) ▲
- lazy collections blow up during JSON serialisation
- any DB column rename becomes a BREAKING API change

Easy memory

Nouns + plural | HTTP verb = the action | correct status codes
Pagination ★ | versioning | stateless | DTOs not entities ★ | HTTPS

✗ `/api/getUserById/5` ✓ `GET /api/v1/users/5`

Q: How do you version a REST API?

1. URI versioning

→ /api/v1/users

★ most common, most visible
2. Header versioning

→ Accept: application/vnd.app.v1+json
3. Request parameter

→ /api/users?version=1

```
@RestController
@RequestMapping("/api/v1/users")
public class UserV1Controller { }

@RestController
@RequestMapping("/api/v2/users")
public class UserV2Controller { }
```

	URI	Header
Visible in the browser	✓	✗
Easy to test / curl	✓	harder
Cacheable separately	✓	trickier
"Purity"	mixes version into the resource identity	cleaner in theory

Why version at all? Because you cannot force every mobile app on every phone to update at the same moment. v1 must keep working while v2 rolls out.

Easy memory

```
/api/v1/users ★ URI versioning – most common, visible, easy to test
Alternatives: an Accept header, or a query parameter

Reason: old mobile clients cannot be force-updated. v1 must keep working.
```

Spring Data JPA

Q: What is Spring Data JPA?

The easiest way to remember:

You write an **INTERFACE**. Spring writes the **IMPLEMENTATION** at runtime. ★

```
public interface UserRepository extends JpaRepository<User, Long> { }
```

That is the whole file — and you already have:

```
save() saveAll() findById() findAll() findAllById()  
delete() deleteById() count() existsById() flush()  
+ pagination and sorting
```

Old DAO (JDBC)

open connection
write SQL
set parameters
execute
loop the ResultSet
map to objects
close everything
~50 lines PER TABLE

Spring Data JPA

```
public interface UserRepository  
    extends JpaRepository<User, Long> { }
```

← that's it ✓

Your interface

↓

Spring creates a **PROXY** at startup

↓

the proxy implements every method for you ✓

Real-world idea: you write the **menu**; the kitchen appears and cooks it.

Easy memory

Declare an **INTERFACE** → Spring generates the implementation (a runtime proxy) ★

You get **CRUD** + **paging** + **sorting** + **derived queries** for free.

Q: JpaRepository vs CrudRepository vs PagingAndSortingRepository ?

The easiest way to remember — **each one adds to the previous:**

```

Repository                (marker, nothing in it)
  ↑
CrudRepository            save, findById, findAll, delete, count
  ↑
PagingAndSortingRepository + Pageable, Sort
  ↑
JpaRepository             + flush(), saveAndFlush(), batch
deletes,                  + flush(), saveAndFlush(), batch
                           findAll() returns List (not Iterable)
★

```

	CrudRepository	PagingAndSorting	JpaRepository ★
Basic CRUD	✓	✓	✓
Pagination / sorting	✗	✓	✓
flush(), batch delete	✗	✗	✓
findAll() returns	Iterable<T>	Iterable<T>	List<T> ✓

```

public interface UserRepository extends JpaRepository<User, Long> { }    //
just use this ★

```

★ The small detail that shows you've used it: `JpaRepository.findAll()` returns a `List`, while `CrudRepository.findAll()` returns an `Iterable` — which you cannot `.stream()` or `.size()` directly.

Easy memory

```

Crud → PagingAndSorting → JpaRepository (each ADDS to the one before)
Just extend JpaRepository ★
JpaRepository.findAll() → List | CrudRepository.findAll() → Iterable

```

Q: Explain @Entity, @Id and @GeneratedValue.

The easiest way to remember:

@Entity → this CLASS = a TABLE
@Id → this FIELD = the PRIMARY KEY
@GeneratedValue → who creates the id value

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;
}
```

class User	→ table `users`
field id	→ column id (PRIMARY KEY, AUTO_INCREMENT)
field name	→ column name NOT NULL VARCHAR(100)

The generation strategies

IDENTITY	→ the DATABASE auto-increments	→ MySQL ★
SEQUENCE	→ a database SEQUENCE object	→ PostgreSQL / Oracle ★
AUTO	→ the provider picks	→ default
TABLE	→ a separate table holds counters	→ slow, avoid

★ The detail worth knowing:

IDENTITY disables JDBC BATCH INSERTS.

Hibernate must ask the DB for the id after EVERY insert, so it cannot batch 1,000 inserts into one round trip. SEQUENCE can (it pre-fetches ids).

That is why high-volume systems on PostgreSQL prefer **SEQUENCE**.

Entity requirements

- ✓ a NO-ARG constructor (Hibernate needs it to instantiate via reflection) ★
- ✓ not final; fields not final
- ✓ an @Id

That no-arg constructor requirement is exactly **why a record cannot be a JPA entity**.

Easy memory

@Entity = table | @Id = primary key | @GeneratedValue = who makes the id

IDENTITY → MySQL auto-increment (▲ blocks batch inserts)

SEQUENCE → Postgres/Oracle (batch-friendly ★)

Entities need a NO-ARG constructor → so a record can NEVER be an @Entity

★

Q: What are derived query methods?

The easiest way to remember:

Spring READS THE METHOD NAME and writes the SQL for you.

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    List<User> findByStatus(String status);  
    User      findByEmail(String email);  
    List<User> findByAgeGreaterThanAndStatus(int age, String status);  
    List<User> findByNameContainingIgnoreCase(String name);  
    long      countByStatus(String status);  
    boolean   existsByEmail(String email);  
    List<User> findTop5ByOrderBySalaryDesc();  
}
```

How the name is parsed:

findBy	Age	GreaterThan	And	Status
verb	field	operator	join	field
	↓			

SELECT * FROM users WHERE age > ? AND status = ?

The keywords worth remembering:

And / Or
 GreaterThan / LessThan / Between
 Like / Containing / StartingWith / EndingWith
 IgnoreCase
 IsNull / IsNotNull
 In / NotIn
 OrderBy...Asc / Desc
 Top / First → findTop5By...
 Distinct

Real-world idea: you write the **request in plain English** and it becomes SQL.

▲ Two traps:

1. A TYPO in a field name fails at STARTUP, not at runtime ✓
 "No property 'nmae' found for type User" – actually a GOOD thing: you find out immediately, not in production.
2. Long names become unreadable:
 findByStatusAndAgeGreaterThanAndDepartmentInOrderByNameAsc(...)
 → at that point switch to @Query. ★

Easy memory

The METHOD NAME becomes the query ★

findBy | countBy | existsBy | deleteBy
 And/Or | GreaterThan | Between | Containing | IgnoreCase | OrderBy | Top5

 Typo → fails at STARTUP (good!)
 Name too long → switch to @Query

Q: When and how do you use @Query (JPQL vs native)?

The easiest way to remember:

JPQL → works on ENTITY names and FIELD names (User, u.status)
Native → works on TABLE names and COLUMN names (users, status)

```
// JPQL – uses the ENTITY name and its fields
@Query("SELECT u FROM User u WHERE u.status = :status")
List<User> findActive(@Param("status") String status);

// NATIVE SQL – uses the real table and columns
@Query(value = "SELECT * FROM users WHERE age > ?1", nativeQuery = true)
List<User> findOlderThan(int age);

// MODIFYING query – needs BOTH annotations ★
@Modifying
@Transactional
@Query("UPDATE User u SET u.status = :s WHERE u.id = :id")
int updateStatus(@Param("id") Long id, @Param("s") String s);
```

JPQL: SELECT u FROM User u ← "User" is the CLASS, "u.status" is the FIELD
Native: SELECT * FROM users ← "users" is the TABLE, "status" is the COLUMN

	JPQL	Native SQL
Works on	entities and fields	tables and columns
Database portable	✓	✗ tied to one DB
DB-specific features	✗	✓ (window functions, hints)
Returns entities	✓ automatically	needs mapping

▲ The @Modifying trap:

Writing an UPDATE/DELETE @Query WITHOUT @Modifying
→ "Not supported for DML operations" ▲

And @Modifying queries BYPASS the persistence context, so entities already loaded in memory keep the OLD values.
Fix: @Modifying(clearAutomatically = true) ★

Easy memory

```
@Query("SELECT u FROM User u ...")           → JPQL (entity + fields)
@Query(value = "SELECT * FROM users", nativeQuery=true) → real SQL

UPDATE/DELETE → needs @Modifying + @Transactional ★
@Modifying bypasses the persistence context → clearAutomatically = true
```

Q: Explain JPA relationships.

The easiest way to remember:

```
@OneToOne    → 1 ↔ 1      User ↔ Profile
@OneToMany    → 1 → many  Department → Employees
@ManyToOne    → many → 1  Employee → Department  ★ owns the FK
@ManyToMany   → many ↔ many Students ↔ Courses (needs a join table)
```

```
@Entity
public class Department {
    @Id @GeneratedValue private Long id;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    private List<Employee> employees = new ArrayList<>();
}

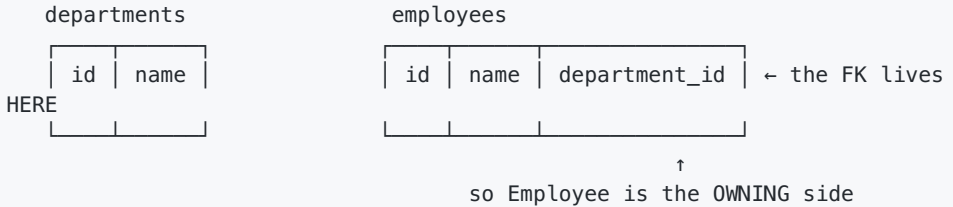
@Entity
public class Employee {
    @Id @GeneratedValue private Long id;

    @ManyToOne(fetch = FetchType.LAZY)           // ★ always set LAZY here
    @JoinColumn(name = "department_id")         // this side holds the
    FK ★ private Department department;
}
```

The concept that decides everything — the OWNING side ★

The side with the FOREIGN KEY COLUMN is the OWNER.
Only the OWNER's changes are written to the database.

@JoinColumn → "I own it, I hold the FK" ← Employee (many side)
mappedBy → "I'm the mirror, HE owns it" ← Department (one side)



The classic bug this causes:

```
department.getEmployees().add(employee); // only touched the NON-owning
side
save(department);
// → nothing is saved ▲ the FK column is never set
```

```
employee.setDepartment(department); // ✓ set the OWNING side
save(employee);
```

Best practice — set **both** sides with a helper method:

```
public void addEmployee(Employee e) {
    employees.add(e);
    e.setDepartment(this); // ★ keeps the object graph consistent
}
```

Easy memory

@ManyToOne = the MANY side = holds the FK = the OWNING side ★
@JoinColumn
@OneToMany = the ONE side = mappedBy = the mirror

Only the OWNING side's changes are persisted.
Always set BOTH sides with a helper method.

Q: Lazy vs Eager loading?

The easiest way to remember:

```
EAGER → load it NOW, along with the parent
LAZY → load it ONLY when the code actually touches it ★
```

Defaults (worth memorising – they're asked directly):

```
@ManyToOne → EAGER ▲ (change this to LAZY!)
@OneToOne → EAGER ▲
@OneToMany → LAZY ✓
@ManyToMany → LAZY ✓
```

★ The memory hook: **"To-One is Eager, To-Many is Lazy."**

```
@ManyToOne(fetch = FetchType.LAZY) // ★ almost always override the
default
private Department department;
```

Why EAGER hurts

```
SELECT * FROM employees (you asked for employees)
    ↓ @ManyToOne is EAGER
SELECT * FROM departments WHERE id = 1 ← extra query you never asked for
SELECT * FROM departments WHERE id = 2
... ← the N+1 problem ▲
```

And EAGER is **contagious**: if `Department` eagerly loads `Company`, and `Company` eagerly loads `Address`, one `findById` can drag in half the database.

But LAZY has its own trap ★

```
@Transactional
public Employee get(Long id) { return repo.findById(id).orElseThrow(); }

// later, OUTSIDE the transaction:
employee.getDepartment().getName(); // ▲ LazyInitializationException
```

```
Inside @Transactional → the session is open → the lazy proxy can load ✓
After the method ends → the session is CLOSED → the proxy cannot load ▲
```

Real-world idea:

EAGER → ordering the entire restaurant menu when you sat down
LAZY → ordering each dish when you actually want it ✓
(but the kitchen closes at 10pm – order after that and you get nothing ▲)

The correct fixes:

✓ JOIN FETCH or @EntityGraph → fetch what you need WITHIN the transaction
✓ Map to a DTO inside the transactional service method
✗ spring.jpa.open-in-view=true → hides the problem and causes N+1 in production

Easy memory

"To-One is EAGER, To-Many is LAZY" ★ → always override @ManyToOne to LAZY

EAGER → extra queries you never asked for (and it's contagious)

LAZY → LazyInitializationException outside the transaction ▲

Fix: JOIN FETCH / @EntityGraph, and map to a DTO inside the service.

Q: What is the N+1 query problem and how do you fix it?

The easiest way to remember:

1 query to load the parents
+ N queries, one per parent, to load its children
= N + 1 queries ▲

```
List<Department> depts = repo.findAll();           // 1 query  
  
for (Department d : depts) {  
    d.getEmployees().size();                       // 1 query EACH ▲  
}
```

```

SELECT * FROM departments          ← 1 query
SELECT * FROM employees WHERE dept_id = 1 ← query 2
SELECT * FROM employees WHERE dept_id = 2 ← query 3
SELECT * FROM employees WHERE dept_id = 3 ← query 4
...
100 departments → 101 QUERIES ▲

```

Real-world idea: going to the market and coming home **101 times** — once for the shopping list, then once for each item — instead of buying everything in one trip.

The fixes

```

// 1. JOIN FETCH — one query, explicit ★
@Query("SELECT DISTINCT d FROM Department d JOIN FETCH d.employees")
List<Department> findAllWithEmployees();

// 2. @EntityGraph — declarative, works with derived queries ★
@EntityGraph(attributePaths = "employees")
List<Department> findAll();

// 3. Batch fetching — turns N queries into N/size queries
@BatchSize(size = 25)
private List<Employee> employees;
// or globally:
spring.jpa.properties.hibernate.default_batch_fetch_size=25

```

Before: 1 + 100 queries ▲
 After : 1 query with a JOIN ✓

▲ Note the **DISTINCT** in the JOIN FETCH — a join multiplies parent rows by the number of children, so without it you get duplicate **Department** objects.

How to detect it

```

spring.jpa.show-sql=true
logging.level.org.hibernate.SQL=DEBUG

```

Then watch the log during one request. Seeing 101 SELECTs is unmistakable. Mentioning that you *look* for it is a strong practical signal.

Interview-ready answer

The N+1 problem happens when we load N parent entities with one query, and then accessing a lazy association triggers one additional query per parent — so 100 departments becomes 101 queries. I fix it by fetching the association up front: either **JOIN FETCH** in a JPQL query, or **@EntityGraph** on the repository method, which works with derived queries too. For cases where a join isn't practical, Hibernate's batch fetching with **@BatchSize** reduces N queries to N divided by the batch size. I detect it by enabling SQL logging and watching the query count for a single request. With **JOIN FETCH** on a collection I also use **DISTINCT**, because the join duplicates the parent rows.

Easy memory

$N+1 = 1 \text{ query for parents} + 1 \text{ query PER parent for children} \blacktriangle$
(going to the market 101 times)

FIX ★ **JOIN FETCH** (use **DISTINCT**!) | **@EntityGraph** | **@BatchSize**

Detect with: `spring.jpa.show-sql=true`

Q: What is **@Transactional** and how does propagation work?

The easiest way to remember:

@Transactional = "ALL of this succeeds, or NONE of it does."

Success → COMMIT

Exception → ROLLBACK

```
@Transactional
public void transfer(Long from, Long to, BigDecimal amt) {
    accountRepo.debit(from, amt);    // step 1
    accountRepo.credit(to, amt);     // step 2
}
```

// both commit, or both roll back

✓

Without @Transactional:

debit succeeds -1000
credit CRASHES ▲
→ the money has vanished ▲

With @Transactional:

debit succeeds
credit crashes → EVERYTHING rolls back → the money is safe ✓

Real-world idea: **a bank transfer**. It cannot be half done.

Propagation — what happens when a transaction calls another

REQUIRED (default) ★ → join the existing one, or create a new one
REQUIRES_NEW → ALWAYS a new one; suspend the current
SUPPORTS → join if one exists, otherwise run without
MANDATORY → must already be inside one, else throw
NESTED → a savepoint inside the current transaction
NEVER / NOT_SUPPORTED → refuse / suspend

The one that matters in practice:

```
methodA() @Transactional
    ↓ calls
methodB() @Transactional(REQUIRED)    → SAME transaction
        B fails → A rolls back too ✓

methodB() @Transactional(REQUIRES_NEW) → SEPARATE transaction
        B commits independently → an audit log survives even if A fails
★
```

That is the classic use of **REQUIRES_NEW**: **audit logging** you want to keep even when the business operation fails.

The three traps ★

1. Rollback happens only on unchecked exceptions

```

@Transactional
public void save() throws IOException {
    repo.save(x);
    throw new IOException();           // ✗ CHECKED → NO ROLLBACK, it
    commits! ▲
}

@Transactional(rollbackFor = Exception.class)    // ✓ the fix

```

2. Self-invocation bypasses the proxy (the same AOP rule as `@Async`)

```

public void a() {
    this.b();           // ✗ @Transactional on b() is IGNORED
}

@Transactional
public void b() { }

```

3. It must be `public` — a proxy cannot intercept a private or protected method.

Interview-ready answer

`@Transactional` declaratively wraps a method in a database transaction — it commits when the method returns normally and rolls back when a runtime exception propagates. It is implemented with an AOP proxy. Propagation controls what happens when a transactional method calls another one: `REQUIRED` is the default and joins the existing transaction, while `REQUIRES_NEW` suspends the current transaction and starts an independent one, which is useful for audit logging that should survive a business rollback. There are three traps worth knowing: by default only unchecked exceptions trigger a rollback, so a checked exception commits unless we specify `rollbackFor`; self-invocation through `this` bypasses the proxy so the annotation has no effect; and the method must be public.

Easy memory

@Transactional = all-or-nothing (a bank transfer) ★

REQUIRED ★ default → join the existing transaction

REQUIRES_NEW → independent transaction (audit logs) ★

- ▲ CHECKED exceptions do NOT roll back → rollbackFor = Exception.class
- ▲ this.method() → the proxy is bypassed → the annotation is ignored
- ▲ must be public

Q: How do you implement pagination?

The easiest way to remember:

Pass a Pageable in → get a Page back.

```
public interface UserRepository extends JpaRepository<User, Long> {  
    Page<User> findByStatus(String status, Pageable pageable);  
}
```

```
Pageable pageable = PageRequest.of(0, 20, Sort.by("name").ascending());  
//                                     ↑page ↑size ← page is 0-BASED ▲
```

```
Page<User> page = userRepository.findByStatus("active", pageable);
```

```
List<User> content = page.getContent();  
long total        = page.getTotalElements();  
int  totalPages   = page.getTotalPages();  
boolean hasNext   = page.hasNext();
```

Spring generates:

```
SELECT * FROM users WHERE status='active' ORDER BY name LIMIT 20 OFFSET  
0  
SELECT COUNT(*) FROM users WHERE status='active' ← the second  
query ★
```

That second COUNT query is what gives you `getTotalElements()`.

Page VS Slice VS List ★

```
Page<T> → content + TOTAL COUNT → 2 queries → needed for "Page 3 of 47"  
Slice<T> → content + hasNext only → 1 query → perfect for INFINITE  
SCROLL ▲  
List<T> → just the content → 1 query → no paging metadata
```

If your UI is infinite scroll, **Slice** skips an expensive **COUNT(*)** on a large table.

From a controller, Spring can build it automatically:

```
@GetMapping("/users")  
public Page<User> list(Pageable pageable) { // ?  
    page=0&size=20&sort=name,asc  
    return repo.findAll(pageable);  
}
```

Real-world idea: **Google search results**. You never receive all 4 million results — you get 10, plus "next".

Easy memory

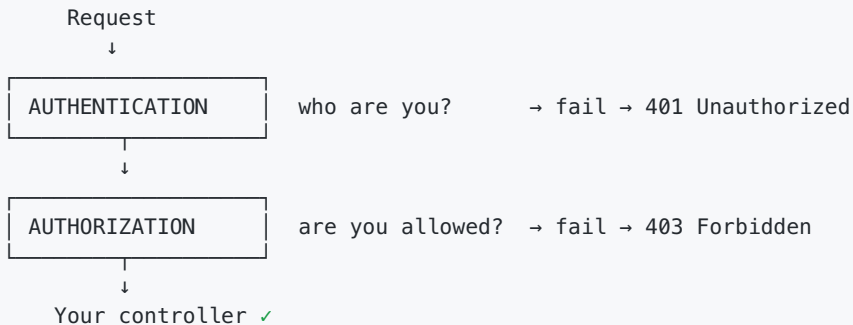
```
PageRequest.of(page, size, Sort.by("name")) ← page is 0-BASED ▲  
Page<T> = content + total (2 queries: data + COUNT)  
Slice<T> = content + hasNext (1 query) → infinite scroll ★  
  
Controllers accept Pageable directly: ?page=0&size=20&sort=name,asc
```

Security & Misc

Q: Spring Security basics — authentication vs authorization?

The easiest way to remember:

```
AuthentiCation → WHO are you? → login → 401 if it fails  
AuthoriZation → WHAT may you do? → roles → 403 if it fails
```



```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{
    http.csrf(c -> c.disable()) // stateless API ->
    CSRF not needed
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/public/**").permitAll()
        .requestMatchers("/api/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated()) // ★ always end
    with this
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
  
```

Spring Security is a **chain of filters** sitting in front of your application:

Request → [CORS] → [CSRF] → [Auth filter] → [Authorization] → Controller

▲ Order matters: rules are matched **top to bottom**, first match wins. Put `anyRequest()` **last**, or it swallows everything below it.

Real-world idea: an **office building**.

Authentication → your ID card opens the front door (who you are)
 Authorization → that card does NOT open the server room (what you may do)

Easy memory

AuthentiCation = who? → login → 401

AuthoriZation = what? → roles → 403 ★

SecurityFilterChain bean → permitAll / hasRole / authenticated

Rules match TOP-DOWN → put anyRequest() LAST ▲

Q: Explain the JWT authentication flow.

The easiest way to remember:

JWT = a signed ID card the client carries on EVERY request.

The server keeps NO session. ★

1. POST /login { username, password }
- ↓
2. Server validates → creates a SIGNED token → returns it
- ↓
3. Client stores it and sends it every time:
Authorization: Bearer eyJhbGci...
- ↓
4. A filter validates the SIGNATURE + EXPIRY on each request
- ↓
5. Sets the SecurityContext → the controller runs ✓
(no server-side session at all)

The three parts of a JWT

eyJhbGciOiJIUzI1NiJ9	. eyJzdWIiOiIxMjMifQ	. SflKxwRJSMeKKF2QT4f
HEADER	PAYLOAD	SIGNATURE
algorithm	claims: user id, roles, expiry	proves it wasn't tampered with ★

▲ The point everyone gets wrong:

The payload is BASE64-ENCODED, not ENCRYPTED.

Anyone can decode and READ it.

→ NEVER put a password or sensitive data in a JWT.

→ The signature only proves it wasn't CHANGED, not that it's secret.

Why JWT for microservices ★

Session-based: every server must share the session store
→ sticky sessions, or Redis, or you get logged out randomly

JWT: the token carries the identity itself
→ ANY server instance can validate it independently ✓
→ scales horizontally with no shared state

Real-world idea: a **cinema ticket**. The staff don't phone the box office — the ticket itself is proof, and it's stamped so it can't be forged.

The trade-off to mention:

- ✓ Stateless, scalable, works across services
- ✗ You cannot easily REVOKE one before it expires
→ so keep access tokens SHORT-lived (15 min) + use a refresh token ★

Easy memory

login → signed JWT → client sends "Authorization: Bearer <token>" every request
→ filter checks signature + expiry → SecurityContext → controller

header . payload . signature ← payload is READABLE (base64, not encrypted) ▲

Stateless ★ any instance can validate it — but you can't revoke it early
→ short-lived access token + refresh token

Q: DTO vs Entity — why separate them?

The easiest way to remember:

Entity = the DATABASE shape
DTO = the API shape

Never let the outside world see your database. ★

```

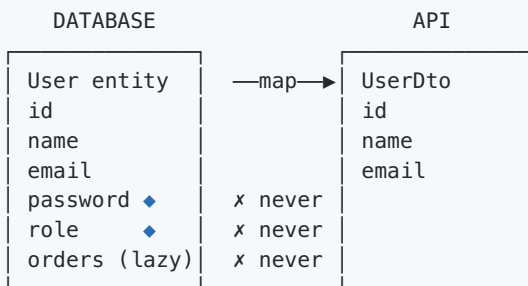
@Entity
public class User {
    @Id private Long id;
    private String name;
    private String email;
    private String password;           // ▲ NEVER send this out
    private String role;              // ▲ a client must not be able to
    set this
    @OneToMany private List<Order> orders;    // ▲ lazy → serialisation
    blows up
}

public record UserDto(Long id, String name, String email) { }    // exactly
what the API needs ✓

```

The five reasons — say two or three

- | | |
|-----------------|---|
| 1. SECURITY | → password, internal flags and salary never leave the server |
| 2. OVER-POSTING | → a client POSTs {"role":"ADMIN"} and privilege-escalates ▲ |
| 3. DECOUPLING | → renaming a DB column shouldn't break every mobile client |
| 4. LAZY LOADING | → serialising an entity touches lazy collections
→ LazyInitializationException, or an accidental N+1 |
| 5. SHAPING | → the API can combine or flatten data the tables don't match |



Map them manually, or with MapStruct:

```

UserDto toDto(User u) {
    return new UserDto(u.getId(), u.getName(), u.getEmail());
}

```


Real-world idea: a **restaurant menu**. The customer sees dish names and prices — not the supplier invoices, the stock levels or the staff salaries.

Easy memory

Entity = DB shape | DTO = API shape ★

Why: security (no password) | over-posting ("role":"ADMIN") ▲ |
decoupling | no lazy-loading blowups | API shaping

Records make perfect DTOs. Map with MapStruct or a simple method.

Q: How do you handle CORS in Spring Boot?

The easiest way to remember:

CORS = the BROWSER refusing to let one website call a different domain's API,
unless that API explicitly allows it.

Angular app at http://localhost:4200
Spring API at http://localhost:8080 ← a DIFFERENT origin
↓
Browser: "Blocked by CORS policy" ▲

★ The point that impresses: **CORS is a browser rule, not a server rule**. Postman and curl work perfectly, which is exactly why the error only ever appears in the browser and confuses people.

```
// Per controller
@CrossOrigin(origins = "https://app.example.com")
@RestController
public class UserController { }

// Global – the better way ★
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("https://app.example.com")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("*")
            .allowCredentials(true);
    }
}
```

The preflight request ★

For anything beyond a simple GET, the browser first sends:

```
OPTIONS /api/users
Origin: http://localhost:4200
Access-Control-Request-Method: POST
↓
Server replies with the Access-Control-Allow-* headers
↓
ONLY THEN does the browser send the real POST ✓
```

That is why you sometimes see **two** requests in the network tab for one call.

▲ With Spring Security you must **also** enable CORS there, or the security filter chain blocks the preflight before your MVC config is ever reached:

```
http.cors(Customizer.withDefaults()); // ★ easily forgotten
```

Also: `allowedOrigins("*")` **cannot** be combined with `allowCredentials(true)` — use `allowedOriginPatterns` instead.

Easy memory

CORS = a BROWSER rule ★ (Postman/curl are unaffected)

@CrossOrigin on a controller, or a global WebMvcConfigurer ★

Preflight OPTIONS request happens first for non-simple requests

▲ With Spring Security also add: http.cors(...)

▲ "*" + allowCredentials(true) is illegal

Reactive (Spring WebFlux)

Q: Spring WebFlux vs Spring MVC?

The easiest way to remember:

Spring MVC → ONE THREAD per request, and it BLOCKS while waiting

Spring WebFlux → a FEW threads handle THOUSANDS of requests, never blocking

SPRING MVC (thread per request)

req1 → thread1 → waits 200ms for the DB (thread is stuck, doing nothing)

req2 → thread2 → waits

...

req201 → NO THREAD LEFT → queued ▲ (Tomcat default is ~200 threads)

SPRING WEBFLUX (event loop)

req1 → starts the DB call → thread is RELEASED ▲

req2 → starts the DB call → same thread reused

...

the response arrives → a thread picks it up and finishes ✓

	Spring MVC	Spring WebFlux
Model	thread per request, blocking	event loop, non-blocking
Built on	Servlet API	Project Reactor
Default server	Tomcat	Netty

	Spring MVC	Spring WebFlux
Return types	<code>User</code> , <code>List<User></code>	<code>Mono<User></code> , <code>Flux<User></code>
Learning curve	easy ✓	steep
Debugging	normal stack traces	painful stack traces ▲
Best for	most applications ★	very high concurrency, streaming

▲ The honest, senior answer:

WebFlux only helps if the WHOLE chain is non-blocking.

One blocking JDBC call inside a WebFlux app makes it SLOWER than MVC – you've added complexity and blocked the small event-loop pool. ▲

(Real reactive data access needs R2DBC or reactive MongoDB, not JPA.)

Real-world idea: a **restaurant**.

MVC → each waiter stands at ONE table until that meal is finished

WebFlux → waiters take an order, move to the next table, and return when the food is ready ▲ – far fewer waiters serve far more tables

Interview-ready answer

Spring MVC uses a thread-per-request, blocking model on the Servlet API, so a thread is occupied for the whole request including the time it spends waiting on the database. Spring WebFlux is asynchronous and non-blocking, built on Project Reactor and running on Netty by default, so a small number of event-loop threads can handle a very large number of concurrent connections. WebFlux is worth it for high-concurrency, I/O-bound or streaming workloads, but only if the entire chain is non-blocking — a blocking JDBC call inside a WebFlux application will block the event loop and perform worse than MVC. For most CRUD applications, including mine, MVC is simpler and the right choice.

Easy memory

MVC → thread per request, BLOCKS → Tomcat → simple ★ (default choice)
WebFlux → event loop, NON-blocking → Netty → Mono / Flux

Only worth it if EVERYTHING is non-blocking (R2DBC, not JPA) ▲
One blocking call ruins it.

Q: Mono vs Flux ?

The easiest way to remember:

Mono<T> → 0 or 1 item ("one result, or none")
Flux<T> → 0 to N items ("a stream of results")



```
@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping("/{id}")
    public Mono<User> getById(@PathVariable String id) {
        return userService.findById(id); // 0 or 1
    }

    @GetMapping
    public Flux<User> getAll() {
        return userService.findAll(); // 0..N, and it can STREAM
    }
}
```

Blocking equivalent		Reactive equivalent
User	→	Mono<User>
List<User>	→	Flux<User>
void	→	Mono<Void>
Optional<User>	→	Mono<User> ★ (empty Mono = not found)

★ The critical property: **they are LAZY.**

```
Mono<User> mono = service.findById("1");    // NOTHING has happened yet  
mono.subscribe(...);                      // NOW it runs ▲
```

"Nothing happens until you subscribe."

In a Spring controller, the **framework subscribes for you** — which is why returning a **Mono** just works.

Real-world idea:

Mono → ordering ONE coffee

Flux → a subscription that delivers a coffee every morning

Both are just the ORDER. Nothing is brewed until you actually place it (subscribe).

Easy memory

Mono<T> → 0 or 1 (a single result)

← replaces User / Optional<User>

Flux<T> → 0 to N (a stream)

← replaces List<User>

LAZY ★ — nothing happens until subscribe()

In a controller, Spring subscribes for you.

Quick Revision Sheet

Read this the night before.

SPRING CORE

IoC = the principle | DI = the technique (container injects)
ApplicationContext ⊃ BeanFactory (eager singletons at STARTUP ★)
Scopes: singleton (default) ★ | prototype | request | session
▲ singleton + mutable field = thread-safety bug → keep @Service

STATELESS

Lifecycle: construct → inject → @PostConstruct ★ → use → @PreDestroy
@Component/@Service/@Controller = semantic
@Repository ★ ADDS exception translation → DataAccessException
CONSTRUCTOR injection ★ (final, testable, exposes too many deps)
@Qualifier names one bean | @Primary = default | @Qualifier BEATS
@Primary
@Bean = for third-party classes; @Configuration classes are CGLIB-
proxied
AOP = CCTV | Aspect/Advice/Pointcut/JoinPoint
▲ SELF-INVOCATION (this.method()) bypasses the proxy – kills
@Transactional/@Async

SPRING BOOT

@SpringBootApplication = @Configuration + @EnableAutoConfiguration +
@ComponentScan
▲ main class must be in the ROOT package (scanning goes DOWN only)
Auto-config = @ConditionalOnClass / OnMissingBean ★ (YOUR bean always
wins)
debug=true → prints the auto-configuration report ★
Starters = combo meals | embedded Tomcat → java -jar
Profiles: application-{profile}.properties + --
spring.profiles.active=prod
Actuator: /health ★ /metrics /loggers – only health+info exposed by
default
CommandLineRunner → runs once after startup

REST

@RestController = @Controller + @ResponseBody → JSON
/users/42 = @PathVariable | ?x= = @RequestParam | JSON = @RequestBody
ResponseEntity = STATUS + HEADERS + BODY ★ (POST should return 201 +
Location)
401 = who are you? | 403 = I know you, still NO ★
@RestControllerAdvice + @ExceptionHandler = one global handler ★
Validation: DTO annotations + @Valid on the parameter (BOTH needed ▲)
@NotBlank for Strings (@NotNull lets "" through)
Idempotent: GET/PUT/DELETE ✓ POST ✗ → use an Idempotency-Key for
payments
Version with /api/v1/... | DTOs, never entities ★

JPA

Declare an interface → Spring generates the implementation

```

    extend JpaRepository ★ (findAll returns List)
    @Entity/@Id/@GeneratedValue: IDENTITY (MySQL) | SEQUENCE (Postgres,
batch-friendly)
    Entities need a NO-ARG constructor → a record can NEVER be an
@Entity ★
    Derived queries: findByStatusAndAgeGreaterThan... (typo fails at
STARTUP)
    @Query JPQL (entities) vs nativeQuery (tables); UPDATE needs @Modifying
★
    @ManyToOne = the FK side = the OWNING side ★ | @OneToMany = mappedBy =
mirror
    "To-One is EAGER, To-Many is LAZY" ★ → always make @ManyToOne LAZY
N+1 ★ → JOIN FETCH (with DISTINCT) | @EntityGraph | @BatchSize
    @Transactional = all-or-nothing; REQUIRED default, REQUIRES_NEW for
audit logs
    ▲ CHECKED exceptions do NOT roll back → rollbackFor
    ▲ self-invocation + must be public
    Pagination: PageRequest.of(page, size, sort) – page is 0-BASED
    Page (2 queries, has total) vs Slice (1 query, infinite scroll) ★

```

SECURITY

```

    Authentication = who (401) | Authorization = what (403) ★
    JWT: header.payload.signature – payload is READABLE (base64, not
encrypted) ▲
        stateless ★ but cannot be revoked → short-lived + refresh token
    CORS is a BROWSER rule ★ (Postman is fine); preflight OPTIONS;
        with Spring Security also add http.cors(...)

```

WEBFLUX

```

    MVC = thread per request, blocking ★ (default choice)
    WebFlux = event loop, non-blocking, Netty – only if EVERYTHING is non-
blocking
    Mono = 0..1 | Flux = 0..N | LAZY – nothing happens until subscribe

```

Related files

- [05-java.md](#) — Core Java
- [24-orm-jpa-hibernate.md](#) — JPA/Hibernate deep dive
- [08-microservices-basics.md](#) — microservices patterns
- [12-java17-features.md](#) — records for DTOs, text blocks for queries
- [32-multithreading.md](#) — `@Async`, thread pools, stateless beans

AWS (Basics) — Interview Q&A (Easy Version)

✓ POSITIONING — read this first

The old version of this file treated AWS as theory. **It isn't — RoboGebra runs on it.** Verified in your code:

'''

robogebra-portal/src/main/java/com/robogebra/cms/
config/AwsConfig.java → S3 client wiring
config/CognitoConfig.java → AWS COGNITO authentication ★
common/properties/S3Properties.java, AwsProperties.java,
AwsUserCognitoProperties.java,
AwsInstituteCognitoProperties.java ← THREE user pools ★
common/model/S3UploadResult.java

Real APIs in use: PutObjectRequest · GeneratePresignedUrlRequest ★
AWSCognitoIdentityProvider · AuthenticationResultType
pom.xml: aws-java-sdk-s3 · aws-java-sdk-cognitoidp · lambda
docker/: LOCALSTACK scripts ★ (AWS emulated locally for dev)
jenkins/: Jenkinsfile-CI + Jenkinsfile-CD ★

'''

Say "I use S3 and Cognito in production", not "I know AWS basics". ★

Every question follows the same shape — **Easiest way to remember** → **simple explanation** → **real-world example** → **Easy memory box** — with ● blocks where it's in your code.

◆ The easiest way to remember AWS

Think of it as **renting a building instead of constructing one.**

EC2	→ the COMPUTER you rent	(a rented office)
S3	→ the FILING CABINET	(infinite storage)
RDS	→ the managed DATABASE	(someone else does backups)
Lambda	→ a worker you pay PER TASK	▲ (no office at all)
IAM	→ the SECURITY DESK	◆ (who may open which door)
CloudWatch	→ the CCTV + alarm system	
Cognito	→ the RECEPTION that checks IDs	★ (what YOU use)

The one line that ties it together:

EC2 = you manage the server ★

Lambda = you manage NOTHING but the code ★

Everything in between is "how much do you want to manage?"

Easy memory

EC2 compute | S3 storage | RDS database | Lambda ▲ functions
IAM ◆ permissions | CloudWatch monitoring | Cognito auth ★

"Rent, don't build." Managed ↔ control is the whole trade-off ★

Cloud & AWS Fundamentals

Q: What is cloud computing?

On-demand delivery of computing resources (servers, storage, databases, networking) over the internet, with pay-as-you-go pricing. Instead of buying and maintaining physical hardware, you rent resources from a provider like AWS and scale up or down as needed.

Q: What are the benefits of cloud computing?

- **No upfront cost** — pay only for what you use.
- **Elasticity / scalability** — scale resources up or down on demand.
- **High availability & reliability** — data centers across the globe.
- **Speed & agility** — provision resources in minutes.
- **Managed services** — AWS handles patching, backups, hardware.

Q: What is IaaS, PaaS, and SaaS?

- **IaaS (Infrastructure as a Service):** you manage OS, runtime, app — provider manages hardware. *Example: EC2.*
- **PaaS (Platform as a Service):** provider manages OS & runtime, you deploy code. *Example: Elastic Beanstalk, AWS Lambda (close to it).*
- **SaaS (Software as a Service):** fully managed software you just use. *Example: Gmail, Salesforce.*

Q: What is AWS?

Amazon Web Services — the world's most widely used cloud platform, offering 200+ services for compute, storage, databases, networking, ML, and more, on a pay-as-you-go model.

Q: What is a Region and an Availability Zone (AZ)?

- **Region:** a geographic area (e.g., `ap-south-1` Mumbai) containing multiple data centers.
- **Availability Zone:** one or more isolated data centers within a Region, with independent power/networking.
- You deploy across **multiple AZs** for high availability — if one AZ fails, the others keep running.

Q: What is the AWS Free Tier?

A set of services free for 12 months or always-free within limits — e.g., 750 hours/month of a `t2.micro` EC2 instance, 5 GB S3 storage, and 1M Lambda requests/month. Great for learning and small projects.

Q: What is the AWS Shared Responsibility Model?

- **AWS is responsible for security of the cloud** — physical hardware, networking, data centers.
- **You are responsible for security in the cloud** — your data, access management (IAM), OS patching (on EC2), encryption, and app security.

IAM (Identity & Access Management)

Q: What is IAM?

A service to securely control **who** (authentication) can do **what** (authorization) in your AWS account. It manages users, groups, roles, and permission policies — and it's free.

Q: What are IAM users, roles, and policies?

- **User:** a person or app with long-term credentials.
- **Group:** a collection of users sharing permissions.
- **Role:** temporary credentials assumed by a user, service, or app (e.g., an EC2 instance assumes a role to access S3 — no hardcoded keys).
- **Policy:** a JSON document defining allowed/denied actions on resources.

Q: What is the principle of least privilege?

Grant only the **minimum permissions** needed to do a task — nothing more. It limits the blast radius if credentials are compromised.

● In your RoboGebra code — Cognito is your identity provider ★

Files: `config/CognitoConfig.java` · `AwsUserCognitoProperties` · `AwsInstituteCognitoProperties` · `AwsEmailUserCognitoProperties`

THREE separate Cognito user pools ★
users · institutes · email-based users

```
AWSCognitoIdentityProvider // the client
AuthenticationResultType   // the tokens Cognito returns
UserNotFoundException, LimitExceededException // ★ real error handling
```

■ "Authentication runs through AWS Cognito rather than a hand-rolled user table. Cognito issues the JWT, and our API validates it against Cognito's JWKS endpoint using `jwks-rsa` — so the service verifies tokens without ever holding a signing

secret. We run three separate user pools, because institute accounts and individual users have different attributes and policies."

★ Two strong points in there: **JWKS validation instead of a shared secret**, and **separate pools for separate identity domains**.

● LocalStack — the detail that makes you sound like you've actually shipped on AWS ★

Files: `docker/localstack-create-pool.sh` · `localstack-delete-user.sh` · `docker-compose.dev.yml`

LocalStack = AWS, emulated on your own machine ★

- S3 buckets and Cognito pools exist **LOCALLY**
- developers don't need real AWS credentials
- no shared dev account for people to break
- tests run offline, and cost nothing ★

■ "For local development we run LocalStack in Docker, so S3 and Cognito are emulated on the developer's machine. There are scripts to create and tear down the Cognito pool, which means a new developer gets a working environment without real AWS credentials, and nobody is sharing a dev account they can break."

★ Almost nobody mentions LocalStack. It signals real day-to-day AWS work rather than tutorial knowledge.

● CI/CD — Jenkins ★

<code>jenkins/Jenkinsfile-CI</code>	→ build + test on every push
<code>jenkins/Jenkinsfile-CD</code>	→ deploy
<code>docker/</code>	→ containerised, so the runtime is identical
<code>everywhere</code>	

■ "CI and CD are Jenkins pipelines, with the app containerised via Docker — so what runs in dev is the same image that runs deployed. That's also why LocalStack

matters: the container needs an S3 and a Cognito to talk to, whether it's on my laptop or in the cloud."

EC2 (Elastic Compute Cloud)

Q: What is EC2?

A service providing **resizable virtual servers (instances)** in the cloud. You choose the OS, CPU, RAM, and storage, and you fully control the instance — ideal for hosting applications like a Spring Boot backend.

Q: What is an AMI?

Amazon Machine Image — a template containing the OS, software, and configuration used to launch an EC2 instance. You can use AWS-provided AMIs or create your own.

Q: What are EC2 instance types?

Families optimized for different workloads:

- **General purpose** (t3, m5) — balanced, for web/app servers.
- **Compute optimized** (c5) — CPU-heavy work.
- **Memory optimized** (r5) — in-memory databases, caching.
- **Storage optimized** (i3) — high disk I/O.

Q: What is a Security Group?

A **virtual firewall** at the instance level controlling inbound and outbound traffic by port, protocol, and IP. It's **stateful** — if inbound is allowed, the response is automatically allowed out. *Example: allow port 22 (SSH) from your IP and 8080 for your app.*

Q: Security Group vs NACL?

- **Security Group:** instance-level, **stateful**, allow-rules only.

- **NACL (Network ACL):** subnet-level, **stateless**, supports allow *and* deny rules.

Q: What is a key pair?

A public/private key used to securely SSH into a Linux EC2 instance. AWS stores the public key; you keep the private `.pem` file.

Q: What is an Elastic IP?

A **static, public IPv4 address** you can attach to an instance. Unlike the default public IP (which changes on stop/start), an Elastic IP stays the same.

Q: When would you use EC2 vs Lambda?

- **EC2:** long-running apps, full control over the server, predictable/steady load.
Example: a Spring Boot API running 24/7.
- **Lambda:** short, event-driven tasks, no server management, spiky/occasional workloads. *Example: resizing an image when uploaded to S3.*

S3 (Simple Storage Service)

Q: What is S3?

Object storage for any amount of data — files (objects) stored in **buckets**. Highly durable (99.999999999% — "11 nines"), scalable, and accessed via HTTP/REST APIs. Common for images, backups, logs, and static websites.

Q: What are buckets and objects?

- **Bucket:** a globally-unique-named container for storing objects.
- **Object:** the actual file plus metadata, identified by a **key** (its name/path).

Q: What are S3 storage classes?

- **S3 Standard** — frequent access.

- **S3 Standard-IA / One Zone-IA** — infrequent access, cheaper.
- **S3 Glacier / Glacier Deep Archive** — archival, lowest cost, slower retrieval.
- **S3 Intelligent-Tiering** — auto-moves data between tiers based on usage.

Q: Can S3 host a website?

Yes — **static website hosting** serves HTML/CSS/JS directly from a bucket. *This is exactly how you'd deploy an Angular build* — upload the `dist/` folder to S3, often fronted by CloudFront (CDN) for speed and HTTPS.

Q: What is S3 versioning?

Keeps **multiple versions** of an object so you can recover from accidental deletes/overwrites. Once enabled, deleting an object just adds a "delete marker."

● In your RoboGebra code — S3, for real ★

Files: `config/AwsConfig.java` · `common/properties/S3Properties.java` · `common/model/S3UploadResult.java`

```
// the two S3 APIs actually in your codebase:
PutObjectRequest           // uploading files
GeneratePresignedUrlRequest // ★ time-limited download links
```

■ "We use S3 for learning assets and user uploads. Files are uploaded with `PutObjectRequest` from the Spring service, and downloads go through presigned URLs rather than making the bucket public or proxying the bytes through our API. That means the client fetches straight from S3 with a URL that expires, so the bucket stays private and our server never becomes a file-serving bottleneck."

★ **That is the single best S3 answer** — it shows you understand *why* presigned URLs exist, not just that they do.

Q: What is a presigned URL?

A time-limited URL that grants temporary access to a private S3 object without making the bucket public — useful for secure downloads/uploads.

Q: S3 vs EBS vs EFS?

- **S3**: object storage, accessed over HTTP, unlimited scale.
 - **EBS (Elastic Block Store)**: a virtual hard disk attached to **one** EC2 instance.
 - **EFS (Elastic File System)**: a shared file system mountable by **many** EC2 instances.
-

Lambda (Serverless)

Q: What is AWS Lambda?

A **serverless compute** service that runs your code in response to events — no servers to provision or manage. You upload a function, AWS runs and scales it automatically, and you pay only for execution time.

Q: How does Lambda work / what triggers it?

An **event source** invokes the function — e.g., an S3 upload, an API Gateway HTTP request, a DynamoDB change, or a scheduled CloudWatch event. Lambda spins up, runs your handler, and shuts down.

Q: What is a cold start?

The delay when Lambda initializes a new execution environment for the first request (or after idle). Subsequent "warm" invocations are faster. Choosing a lighter runtime and smaller package reduces it.

Q: What are typical Lambda use cases?

Image/file processing on upload, lightweight REST APIs (with API Gateway), scheduled jobs, real-time stream processing, and backend automation.

Q: What are Lambda's limitations?

- Max execution timeout (15 minutes).
- Limited memory/temp storage (configurable, with caps).

- Stateless — no persistent local state between invocations.
 - Cold starts for latency-sensitive apps.
-

RDS (Relational Database Service)

Q: What is RDS?

A **managed relational database** service. AWS handles provisioning, patching, backups, and scaling, so you focus on your data. Supports **MySQL**, **PostgreSQL** (both in your stack), MariaDB, Oracle, SQL Server, and Amazon Aurora.

Q: RDS vs running a database on EC2?

- **RDS:** AWS manages backups, patching, failover, monitoring — less operational work.
- **DB on EC2:** full control and customization, but **you** manage everything (backups, patching, HA). Use only when you need special configs RDS doesn't support.

Q: What is Multi-AZ in RDS?

A **standby replica in another AZ** for **high availability**. AWS automatically fails over to the standby if the primary fails. It's for **resilience**, **not read scaling**.

Q: What is a Read Replica?

A **read-only copy** of the database used to **scale read traffic** (e.g., reporting/dashboards). It's for **performance**, not automatic failover.

Q: Multi-AZ vs Read Replica — what's the difference?

- **Multi-AZ:** disaster recovery / high availability (automatic failover, synchronous).
- **Read Replica:** scaling reads (asynchronous, can serve read queries).

Q: What about RDS backups?

RDS supports **automated daily backups** (with point-in-time recovery) and **manual snapshots** you trigger yourself.

CloudWatch

Q: What is CloudWatch?

AWS's **monitoring and observability** service. It collects **metrics, logs, and events** from AWS resources and your apps, and lets you set **alarms** and dashboards.

Q: Metrics vs Logs vs Alarms?

- **Metrics:** numeric data over time (CPU %, memory, request count).
- **Logs:** text output from apps/services (e.g., your Spring Boot logs via CloudWatch Logs).
- **Alarms:** trigger actions when a metric crosses a threshold (e.g., email/SNS when CPU > 80%, or auto-scale).

Q: How would you monitor an application with CloudWatch?

Send app logs to **CloudWatch Logs**, watch key **metrics** (CPU, latency, error rate), set **alarms** for thresholds, and visualize everything on a **dashboard**. Alarms can notify via SNS or trigger auto-scaling.

Q: CloudWatch vs CloudTrail?

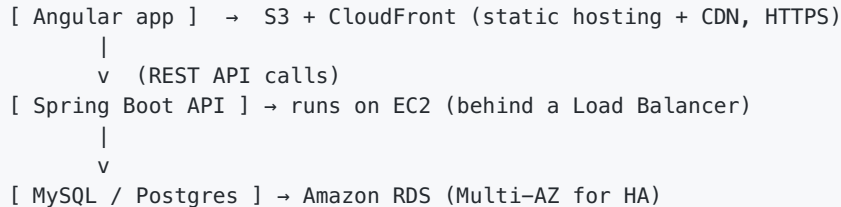
- **CloudWatch:** monitors **performance & health** (what's happening *in* resources).
 - **CloudTrail:** records **API calls / who did what** (auditing & governance).
-

Putting It Together

Q: How do these services fit a typical web application?

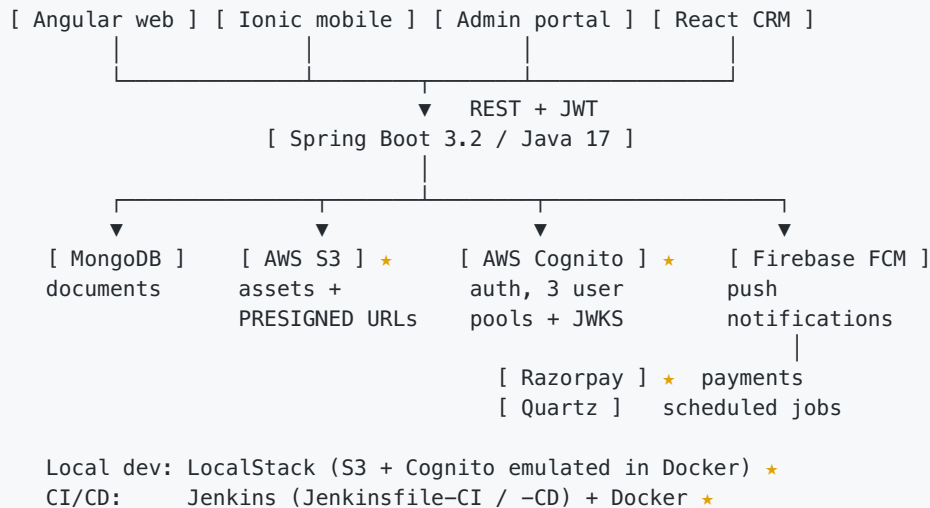
A simple architecture for your stack:

GENERIC reference architecture (the one to draw on a whiteboard):



Monitoring/logs across all → CloudWatch (metrics, logs, alarms)
Occasional tasks (e.g. image processing) → Lambda triggered by S3
Access control everywhere → IAM (least privilege)

● And YOUR actual architecture ★



■ "Four clients talk to one Spring Boot API over REST with JWT. Persistence is MongoDB; S3 holds assets and serves them through presigned URLs; Cognito is the identity provider; Firebase handles push; Razorpay handles payments; Quartz

runs the scheduled jobs. Locally the AWS pieces are emulated with LocalStack, and CI/CD is Jenkins with Docker images."

★ Being able to draw *your own* architecture beats reciting a textbook one.

Q: How do you keep AWS costs low?

- Use the **right instance size** (don't over-provision); stop unused instances.
- Use **Auto Scaling** so capacity matches demand.
- Pick the **right S3 storage class** and lifecycle rules.
- Use **Reserved/Savings Plans** for steady workloads.
- Set **billing alarms** in CloudWatch.

Q: What are basic AWS security best practices?

- Enable **MFA** and never use the root account for daily work.
 - Apply **least privilege** IAM policies; use **roles** instead of hardcoded keys.
 - **Encrypt** data at rest (S3, RDS) and in transit (HTTPS/TLS).
 - Keep **security groups** tight (open only required ports).
 - Enable **CloudTrail** for auditing.
-

Quick Revision Sheet

THE MODEL "Rent, don't build." Managed ↔ control is the trade-off ★
EC2 compute | S3 storage | RDS database | Lambda ▲ per-invocation
IAM ◆ permissions | CloudWatch monitoring | Cognito auth ★

REGION vs AZ Region = a city | AZ = a separate data centre in it
Multi-AZ = survive one building failing ★

IAM user (a person) | role (assumed, TEMPORARY credentials ★) | policy (JSON)
Least privilege ★ | roles > hard-coded keys ★

EC2 vs LAMBDA EC2 = always on, you patch it | Lambda = per request, cold start ★
SG vs NACL SG = instance level, STATEFUL ★ | NACL = subnet, stateless

S3 buckets + objects | storage classes | versioning
PRESIGNED URL ★ = private bucket + a time-limited direct link

RDS Multi-AZ = HIGH AVAILABILITY (standby, no reads) ★
Read replica = SCALE READS ★ – different things!

CloudWatch = metrics/logs/alarms (what happened INSIDE) ★
CloudTrail = WHO did what in the account (audit) ★

- YOURS: S3 (PutObject + presigned URLs ★) · Cognito (3 pools + JWKS ★)
LocalStack for local dev ★ · Jenkins CI/CD + Docker ★
"I use S3 and Cognito in production", not "I know AWS basics" ★

Related files: [06 — Spring Boot](#) · [08 — Microservices](#) · [39 — RoboGebra Code Examples](#)

Microservices (Basics) — Interview Q&A (Easy Version)

▲ YOUR HONEST POSITION — read this first

I checked your code. **RoboGebra is a well-structured MODULAR MONOLITH, not microservices:**

robogebra-portal (Spring Boot 3.2, ONE deployable): Eureka / service discovery → 0 Feign clients → 0 Circuit breaker / Resilience4j → 0 Kafka / RabbitMQ → 0 WebClient (outbound HTTP) → 10 files ★
RestTemplate → 3 files Domain packages → exercise, quiz, payment, studygroup, learningtimeline, notification... ★

So DON'T say "we use microservices." One follow-up — "how do your services discover each other?" — and it falls apart.

✓ Say this instead ★

*"RoboGebra is a modular monolith — one Spring Boot deployment, but organised into clear domain packages that each own their repositories and services, so the seams are already there if we ever needed to split it. We do make outbound HTTP calls with **WebClient** — to the AI service and to Razorpay — and we use AWS Cognito as a separate identity provider and Firebase for push, so it's not a single closed box. I understand the microservice patterns — discovery, gateway, circuit breakers, database-per-service, saga — and the trade-offs, but on a team our size a monolith with clean module boundaries has been the right call. Martin Fowler's own advice is 'monolith first', and this is that."*

★ **That answer is STRONGER than claiming microservices.** It shows you can choose an architecture instead of following fashion — which is exactly what a lead-level interviewer is probing for.

Every question follows the same shape — **Easiest way to remember** → **simple explanation** → **real-world example** → **Easy memory box**.

◆ The easiest way to remember microservices

MONOLITH → one BIG restaurant kitchen
one menu, one team, one gas line.
The fryer breaks → the whole kitchen stops ▲

MICROSERVICES → a FOOD COURT ★
separate stalls, separate kitchens, separate owners.
The pizza stall closes → biryani keeps selling ✓
But now you need signage, a shared payment desk,
and someone to coordinate them

That ONE picture gives you every advantage AND every disadvantage:

✓ independent deployment · independent scaling · fault isolation · tech freedom
✗ network calls · no shared database · harder debugging · DevOps overhead ★

★ The sentence that wins the topic:

"Microservices trade code complexity for operational complexity. You don't remove the difficulty — you move it from inside one codebase to the network between many."

Easy memory

Monolith = ONE kitchen | Microservices = a FOOD COURT ★

✓ deploy + scale + fail INDEPENDENTLY
✗ the network is now part of your logic ▲ (latency, partial failure)

"You trade code complexity for OPERATIONAL complexity." ★
Start with a monolith. Split when a real boundary hurts. ★

Fundamentals

Q: What are microservices?

An architectural style where an application is built as a **collection of small, independent services**, each responsible for one business capability, communicating over the network (usually REST or messaging). Each service can be **developed, deployed, and scaled independently**.

Q: Monolith vs Microservices?

	Monolith	Microservices
Codebase	Single, unified	Many small services
Deployment	Deploy whole app	Deploy each service independently
Scaling	Scale entire app	Scale only the service that needs it
Tech stack	Usually one	Each service can use its own
Failure	One bug can crash all	Failure is isolated
Complexity	Simple to start	More operational complexity

Q: What are the advantages of microservices?

- **Independent deployment** — ship one service without redeploying everything.
- **Independent scaling** — scale only the hot service.
- **Technology freedom** — different languages/DBs per service.
- **Fault isolation** — one service failing doesn't bring down the whole system.
- **Team autonomy** — small teams own small services.

Q: What are the disadvantages / challenges?

- **Distributed system complexity** — network calls, latency, partial failures.
- **Data consistency** — no single database; eventual consistency.

- **Testing & debugging** harder across services.
- **Deployment & monitoring** overhead (many moving parts).
- **DevOps maturity** required (CI/CD, containers, orchestration).

Q: When should you NOT use microservices?

- Small/simple applications or early-stage products — a monolith is faster to build.
- Small teams without DevOps maturity.
- When the domain boundaries aren't clear yet. *Start with a monolith, split later when needed (see strangler pattern).*

Q: What are the key characteristics of a microservice?

Single responsibility, loosely coupled, independently deployable, owns its own data, communicates via well-defined APIs, and is **resilient** (handles failures of others gracefully).

Q: Microservices vs SOA?

Both break apps into services, but **SOA** typically uses a heavy central **ESB** (**Enterprise Service Bus**) and shared databases, while **microservices** are lighter, smaller, decentralized, each owning its own data, communicating via simple protocols (REST/messaging).

Communication Between Services

Q: How do microservices communicate?

- **Synchronous** — one service calls another and waits (usually **REST/HTTP**, sometimes gRPC).
- **Asynchronous** — services exchange **messages/events** via a broker (Kafka, RabbitMQ), without waiting.

Q: Synchronous vs Asynchronous — when to use which?

- **Synchronous (REST):** simple, immediate response needed. Risk: tight coupling, cascading failures.
- **Asynchronous (messaging):** decoupled, resilient, good for events ("OrderPlaced"), background work. Risk: eventual consistency, more complexity.

Q: What is a message broker?

Middleware that lets services communicate via messages without direct calls.

Kafka (high-throughput event streaming) and **RabbitMQ** (traditional message queue) are common. It decouples producers from consumers.

Q: What is an API contract?

The agreed-upon **interface** (endpoints, request/response formats) between services. Keeping contracts stable (and versioning them) lets services evolve independently. Tools like OpenAPI/Swagger document them.

Spring Cloud Building Blocks

Q: What is service discovery? (Eureka)

In a dynamic environment, service instances come and go with changing IPs. A **service registry** like **Netflix Eureka** lets services **register themselves** and **discover** others by name instead of hardcoding URLs.

Service A → asks Eureka "where is PAYMENT-SERVICE?" → gets instance address → calls it

Q: What is an API Gateway and why use it?

A **single entry point** for all client requests that routes them to the right backend service. **Spring Cloud Gateway** is common. Benefits: centralized **authentication**,

rate limiting, logging, routing, CORS, and it hides internal service structure from clients.

Q: What is a Config Server?

A **centralized configuration** service (Spring Cloud Config) that stores config for all microservices (often in Git). Services fetch their config at startup, so you manage settings in one place and across environments (dev/test/prod) without redeploying.

Q: What is client-side load balancing?

The calling service picks which instance of the target service to call (e.g., **Spring Cloud LoadBalancer**, formerly Ribbon), often using the registry's list of instances — distributing load without a central load balancer.

Q: What is a Feign client?

A **declarative REST client** in Spring Cloud. You define an interface with annotations, and Feign generates the HTTP call code — making inter-service REST calls clean:

```
@FeignClient(name = "payment-service")
public interface PaymentClient {
    @GetMapping("/payments/{id}")
    Payment getPayment(@PathVariable Long id);
}
```

Q: What is a Circuit Breaker and why is it needed?

If a downstream service is slow/failing, repeated calls pile up and can cascade. A **circuit breaker** (e.g., **Resilience4j**, formerly Hystrix) "trips" after repeated failures and **fails fast** (or returns a fallback) instead of waiting — protecting the whole system. States: **Closed** → **Open** → **Half-Open**.

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "fallback")
public Payment getPayment(Long id) { ... }
```

The circuit breaker, explained properly ★

```
CLOSED    → normal. Calls pass through. Failures are counted. ★
    ↓ too many failures
OPEN      → FAIL FAST. Don't even try – return a fallback immediately ★
    ↓ after a wait
HALF-OPEN → let ONE test call through
    ↓ it works → CLOSED ✓           ↓ it fails → OPEN again
```

Real-world idea: **the electrical trip switch in your house**. When something shorts, it cuts the circuit *instantly* rather than letting the whole house burn. Then you flip it back and see if the problem is gone.

★ **Why "fail fast" is the point** — this is the bit people miss:

```
WITHOUT a circuit breaker:
    the payment service is slow (30s timeouts)
    → every order request WAITS 30 seconds
    → all 200 Tomcat threads are stuck waiting ▲
    → your ENTIRE app is down, because ONE dependency is slow ★

WITH a circuit breaker:
    after N failures it OPENS → calls return instantly with a fallback
    → threads are freed → the rest of the app keeps working ✓
```

The failure CASCADES without one. That's the whole reason it exists ★

Easy memory

```
CLOSED → OPEN → HALF-OPEN → CLOSED ★ (a house trip switch )
OPEN = FAIL FAST with a fallback ★

Why: a SLOW dependency exhausts your thread pool and takes the
    WHOLE app down – the cascade is the danger, not the failure ★
Resilience4j (Hystrix is retired)
```

Common Patterns

Q: What is the API Gateway pattern?

Route all external traffic through one gateway that handles cross-cutting concerns (auth, routing, throttling) and aggregates responses — instead of clients calling many services directly.

Q: What is "database per service"?

Each microservice **owns its own database**; no other service accesses it directly. This keeps services loosely coupled and independently deployable — but means **no cross-service joins** and the need for eventual consistency.

Q: What is the Saga pattern?

A way to manage **distributed transactions** across services without a global lock. A business transaction is split into a sequence of **local transactions**; if one fails, **compensating transactions** undo the previous steps. Two styles: **choreography** (events) and **orchestration** (a coordinator).

The Saga, explained simply ★

One database → you have ROLLBACK. ✓
Five databases → there is NO global rollback. ★
So instead you do the UNDO YOURSELF, step by step.

Order Saga:

1. Payment taken ✓
2. Stock reserved ✓
3. Shipping booked ✗ FAILS
 ↓ now run the COMPENSATING transactions, in REVERSE ★
- 2'. Release the stock
- 1'. REFUND the payment ★

Real-world idea: **booking a holiday**. Flight ✓, hotel ✓, car hire ✗ sold out. Nobody can "roll back" your holiday — you have to **cancel the hotel and cancel the flight**, one at a time. Each cancellation is a compensating transaction.

CHOREOGRAPHY → each service listens for events and reacts
→ no coordinator, but the flow is hard to SEE ★
ORCHESTRATION → one coordinator tells each service what to do
→ easier to follow and debug, but it's a central point
★

Easy memory

Saga = many local transactions + COMPENSATING undo steps ★
(a holiday booking – you cancel, you don't "roll back")
CHOREOGRAPHY = events, no boss (hard to trace) ★
ORCHESTRATION = a coordinator (easier to debug) ★
Needed because there is NO rollback across separate databases ★

Q: What is CQRS? (brief)

Command Query Responsibility Segregation — separate the **write model** (commands) from the **read model** (queries), often with different data stores optimized for each. Useful for high-read systems; adds complexity.

Q: What is the Strangler pattern?

A migration strategy: incrementally replace a monolith by building new functionality as microservices and **routing traffic gradually** away from the monolith until it's "strangled" out. Lower risk than a big-bang rewrite. *(Relevant to your migration experience.)*

Q: What are retry and bulkhead patterns? (brief)

- **Retry:** automatically retry a failed call (with backoff) for transient errors.
- **Bulkhead:** isolate resources (e.g., separate thread pools) per dependency so one failing service can't exhaust all resources.

Data & Consistency

Q: Why does each service own its database?

To keep services **independent and loosely coupled** — a schema change in one service shouldn't break another. The trade-off is you lose ACID joins across services and must handle consistency at the application level.

Easy memory — database per service

Each service owns its OWN database. Nobody else touches it ★

- ✓ A schema change in one service can't break another
- ✓ Services deploy independently
- ✗ NO cross-service JOIN ▲ → you must call the other service, or duplicate data
- ✗ NO ACID transaction across services → you need a SAGA ★

Real-world idea: **separate bank accounts per family member**. Nobody can touch yours — but "how much does the family have?" now needs four phone calls instead of one query.

Q: What is eventual consistency?

In distributed systems you often can't have instant consistency everywhere.

Eventual consistency means data across services becomes consistent **after some delay** (e.g., via events). Acceptable for many business cases (e.g., an order summary updating moments later).

Q: Why are distributed transactions hard?

There's no single database to roll back. A transaction spanning services can partially fail, so you need patterns like **Saga** with compensating actions instead of a traditional commit/rollback.

Cross-Cutting Concerns

Q: How do you handle logging across microservices?

Centralized logging — each service ships logs to a central system (e.g., ELK stack: Elasticsearch + Logstash + Kibana, or CloudWatch). This lets you search logs across all services in one place.

Q: What is distributed tracing?

Tracking a single request as it flows through multiple services using a **correlation/trace ID**. Tools like **Spring Cloud Sleuth + Zipkin** add and propagate the ID so you can see the full path and find bottlenecks.

Q: How is security handled in microservices?

Typically **token-based (JWT)**. The API Gateway authenticates the user, then the **token is propagated** to downstream services, each validating it. OAuth2 is common for authorization.

Q: How does Docker fit microservices?

Docker packages each service with its dependencies into a **container** that runs identically anywhere. Each microservice → its own image → its own container, enabling consistent, isolated, independently deployable units. (*You used Docker on the Subsea project.*)

Q: What is Kubernetes (very brief)?

A **container orchestration** platform that automates deploying, scaling, and managing containers across machines. It handles load balancing, self-healing (restarting failed containers), rolling updates, and service discovery — the standard way to run microservices at scale.

Quick Honest Framing for the Interview

The 30-second answer ★

*"RoboGebra is a modular monolith — one Spring Boot 3.2 deployment organised into domain packages that each own their services and repositories, so the boundaries are already drawn. We do talk to external services: **WebClient** for the AI engine and Razorpay, AWS Cognito for identity, Firebase for push. I understand the microservice patterns and the trade-offs, but for our team size a monolith with clean modules has been the right call — you get most of the modularity without paying the operational tax."*

If they push: "So you haven't built microservices?" ★

"Not as separate deployables, no. What I've done is the part that decides whether microservices would even work — keeping domain boundaries clean, owning a feature from API to UI, and dealing with the failure modes you get once a call leaves your process. On RoboGebra that showed up as a bounded thread pool with a caller-runs rejection policy for our push-notification fan-out, so a slow external service can't exhaust the pool and take the app down. That's the same reasoning behind a bulkhead and a circuit breaker — I've just applied it inside one process."

★ **That is a genuinely strong answer.** It converts "I haven't done X" into "here is the underlying problem X solves, and here's how I've handled it." → [39](#) —

[RoboGebra Code Examples](#)

✗ What NOT to say

- ✗ "Yes, we use microservices."
 - "How do your services discover each other?" → over ▲
- ✗ "We use Eureka and a circuit breaker."
 - you have neither. Don't name a tool you can't discuss ★
- ✗ "Microservices are better."
 - they're a TRADE-OFF. Saying otherwise reads as inexperience ★

Quick Revision Sheet

Monolith = ONE kitchen | Microservices = a FOOD COURT ★

"You trade CODE complexity for OPERATIONAL complexity" ★

COMMUNICATION sync (REST/gRPC – simple, but coupling + cascades)
async (Kafka/RabbitMQ – decoupled, but eventual consistency)

SPRING CLOUD Eureka (discovery) | Gateway (one entry point ★)
Config Server | Feign (declarative REST) | Resilience4j

CIRCUIT BREAKER ★ CLOSED → OPEN (fail fast) → HALF-OPEN
Why: a SLOW dependency exhausts your threads and kills the WHOLE app ★

PATTERNS DB per service (no joins ★) | SAGA (compensating undo ★)
CQRS (split read/write) | STRANGLER (migrate gradually ★)
Retry + BULKHEAD (isolated pools ★)

CROSS-CUTTING centralised logs (ELK) | distributed tracing (correlation id ★)

JWT at the gateway | Docker + Kubernetes

★ YOUR ANSWER: "RoboGebra is a MODULAR MONOLITH – one deployment, clean domain boundaries. I know the patterns and the trade-offs; for our team size this was the right call. Monolith first."

Related files: [06 — Spring Boot](#) · [07 — AWS](#) · [32 — Multithreading \(bounded pools, bulkheads\)](#) · [39 — RoboGebra Code Examples](#)

Coding Problems — Solutions (JavaScript & Java) — Easy Version

Common coding-round problems, easy → medium, with the approach, the complexity, and clean code.

◆ The easiest way to solve ANY of these — spot the PATTERN

★ You are not memorising 60 solutions. You are learning **6 patterns**. Almost every problem below is one of them.

1. TWO POINTERS ★ "from both ends" or "slow + fast"
→ palindrome · reverse in place · pair with a sum (sorted) · move zeros
→ $O(n)$ time, $O(1)$ space ★
2. HASH MAP / SET ★ "have I seen this before?"
→ duplicates · two sum · anagrams · frequency count · first non-repeating
→ trades SPACE for TIME: $O(n^2) \rightarrow O(n)$ ★
3. SLIDING WINDOW "a contiguous chunk that grows and shrinks"
→ max subarray (Kadane) · longest substring without repeats
4. SORT FIRST "does sorting make it obvious?"
→ anagrams · second largest · merge · group anagrams
→ costs $O(n \log n)$, so only if a hash map can't do it ★
5. MATH / IN-PLACE "can I avoid extra memory?"
→ missing number (sum formula) · swap without temp · reverse in place
6. RECURSION "does the problem contain a smaller version of itself?"
→ factorial · fibonacci · power · sum of an array

The decision question ★

"Have I seen this value before?"	→ HASH SET ★
"Count how many of each?"	→ HASH MAP ★
"Is the array SORTED, or can I sort it?"	→ TWO POINTERS / BINARY SEARCH ★
"A contiguous run of elements?"	→ SLIDING WINDOW
"O(1) space required?"	→ TWO POINTERS or MATH ★

Real-world idea: these are **six tools in a toolbox**, not sixty separate skills. You don't memorise every repair — you learn to recognise which tool the job needs.

The complexity numbers to have ready ★

$O(1)$	a hash lookup, arithmetic
$O(\log n)$	binary search ★ (halving each step)
$O(n)$	one pass ★ ← the target for most of these
$O(n \log n)$	sorting ★
$O(n^2)$	nested loops ▲ ← usually the thing you're being asked to remove

★ **The single most common improvement in an interview:** a nested loop ($O(n^2)$) replaced by a **hash set** ($O(n)$). If you're stuck, ask yourself whether a set would help — it usually does.

Tips for the coding round

1. CLARIFY FIRST ★ – input type, size, edge cases (empty, null, negatives, duplicates)
"Can the array be empty?" "Are there duplicates?" "Is it sorted?"
→ this alone marks you as experienced ★
2. THINK ALOUD ★ – say the APPROACH before you write any code
"I'll use a hash set to track what I've seen – that makes it one pass, $O(n)$."
→ interviewers grade your REASONING, not your typing ★
3. BRUTE FORCE FIRST, then optimise ★
A working $O(n^2)$ beats an elegant solution that doesn't compile.
Say: "let me get it correct first, then improve it."
4. STATE THE COMPLEXITY ★ – time AND space, unprompted
5. DRY-RUN one example ★ – walk through it out loud and prove it works
6. HANDLE EDGE CASES – empty, one element, all duplicates, negatives

★ **The habit that scores highest:** narrate the trade-off. *"I could sort it for $O(n \log n)$ and no extra space, or use a set for $O(n)$ time and $O(n)$ space — I'll take the set unless memory is tight."*

Easy memory

6 PATTERNS ★ two pointers · HASH MAP ★ · sliding window · sort first · math · recursion

THE QUESTION "have I seen this before?" → a SET ★ (turns $O(n^2)$ into $O(n)$)

Clarify → say the approach → brute force → optimise → complexity → dry-run ★

Strings

Problem: Reverse a string

Approach: Two pointers, or use built-ins. **Complexity:** $O(n)$ time, $O(n)$ space.

```
function reverseString(str) {  
  return str.split("").reverse().join("");  
}  
// Without built-ins:  
function reverse(str) {  
  let res = "";  
  for (let i = str.length - 1; i >= 0; i--) res += str[i];  
  return res;  
}
```

```
public static String reverse(String str) {  
  return new StringBuilder(str).reverse().toString();  
}
```

Problem: Check if a string is a palindrome

Approach: Compare characters from both ends. **Complexity:** $O(n)$ time, $O(1)$ space.

```
function isPalindrome(str) {  
  let i = 0, j = str.length - 1;  
  while (i < j) {  
    if (str[i] !== str[j]) return false;  
    i++; j--;  
  }  
  return true;  
}
```

```

public static boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        if (s.charAt(i) != s.charAt(j)) return false;
        i++; j--;
    }
    return true;
}

```

Problem: Count vowels and consonants

Complexity: $O(n)$.

```

function countVowelsConsonants(str) {
    let vowels = 0, consonants = 0;
    for (const ch of str.toLowerCase()) {
        if ("aeiou".includes(ch)) vowels++;
        else if (ch >= "a" && ch <= "z") consonants++;
    }
    return { vowels, consonants };
}

```

Problem: Check if two strings are anagrams

Approach: Sort both, or compare character frequency. **Complexity:** $O(n)$ with frequency map.

```

function isAnagram(a, b) {
    if (a.length !== b.length) return false;
    const count = {};
    for (const ch of a) count[ch] = (count[ch] || 0) + 1;
    for (const ch of b) {
        if (!count[ch]) return false;
        count[ch]--;
    }
    return true;
}

```



```
public static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    char[] x = a.toCharArray(), y = b.toCharArray();
    Arrays.sort(x); Arrays.sort(y);
    return Arrays.equals(x, y);
}
```

Problem: First non-repeating character

Complexity: $O(n)$.

```
function firstNonRepeating(str) {
    const count = {};
    for (const ch of str) count[ch] = (count[ch] || 0) + 1;
    for (const ch of str) if (count[ch] === 1) return ch;
    return null;
}
```

Problem: Remove duplicate characters from a string

```
function removeDuplicates(str) {
    return [...new Set(str)].join("");
}
```

Problem: Reverse words in a sentence

```
function reverseWords(sentence) {
    return sentence.trim().split(/\s+/).reverse().join(" ");
}
```

Problem: Check if two strings are rotations of each other

Approach: `b` is a rotation of `a` if it's a substring of `a + a`. **Complexity:** $O(n)$.

```
function areRotations(a, b) {
    return a.length === b.length && (a + a).includes(b);
}
```

Problem: Count occurrences of each character (frequency map)

```
function charFrequency(str) {  
  const map = {};  
  for (const ch of str) map[ch] = (map[ch] || 0) + 1;  
  return map;  
}
```

Arrays

Problem: Find the maximum and minimum in an array

Complexity: $O(n)$.

```
function maxMin(arr) {  
  return { max: Math.max(...arr), min: Math.min(...arr) };  
}
```

Problem: Find the second largest element

Approach: Single pass, track largest & second largest. **Complexity:** $O(n)$.

```
function secondLargest(arr) {  
  let first = -Infinity, second = -Infinity;  
  for (const n of arr) {  
    if (n > first) { second = first; first = n; }  
    else if (n > second && n !== first) second = n;  
  }  
  return second;  
}
```

Problem: Reverse an array in place

```
function reverseArray(arr) {  
  let i = 0, j = arr.length - 1;  
  while (i < j) { [arr[i], arr[j]] = [arr[j], arr[i]]; i++; j--; }  
  return arr;  
}
```

Problem: Find the missing number (1..n)

Approach: Sum of 1..n minus actual sum. **Complexity:** $O(n)$ time, $O(1)$ space.

```
function missingNumber(arr, n) {  
  const expected = (n * (n + 1)) / 2;  
  const actual = arr.reduce((a, b) => a + b, 0);  
  return expected - actual;  
}
```

Problem: Two Sum (return indices of two numbers adding to target)

Approach: HashMap of value $\rightarrow$ index. **Complexity:** $O(n)$.

```
function twoSum(nums, target) {  
  const map = new Map();  
  for (let i = 0; i < nums.length; i++) {  
    const need = target - nums[i];  
    if (map.has(need)) return [map.get(need), i];  
    map.set(nums[i], i);  
  }  
  return [];  
}
```

```
public static int[] twoSum(int[] nums, int target) {  
  Map<Integer, Integer> map = new HashMap<>();  
  for (int i = 0; i < nums.length; i++) {  
    int need = target - nums[i];  
    if (map.containsKey(need)) return new int[]{map.get(need), i};  
    map.put(nums[i], i);  
  }  
  return new int[]{};  
}
```

Problem: Move all zeros to the end

Complexity: $O(n)$.

```
function moveZeros(arr) {
  let pos = 0;
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] !== 0) { [arr[pos], arr[i]] = [arr[i], arr[pos]]; pos++; }
  }
  return arr;
}
```

Problem: Find duplicates in an array

```
function findDuplicates(arr) {
  const seen = new Set(), dups = new Set();
  for (const n of arr) seen.has(n) ? dups.add(n) : seen.add(n);
  return [...dups];
}
```

Problem: Maximum subarray sum (Kadane's algorithm)

Approach: Track current sum; reset if it drops below 0. **Complexity:** $O(n)$.

```
function maxSubArray(nums) {
  let maxSoFar = nums[0], current = nums[0];
  for (let i = 1; i < nums.length; i++) {
    current = Math.max(nums[i], current + nums[i]);
    maxSoFar = Math.max(maxSoFar, current);
  }
  return maxSoFar;
}
```

Problem: Rotate an array by k positions

Complexity: $O(n)$.

```
function rotate(arr, k) {
  k = k % arr.length;
  return [...arr.slice(-k), ...arr.slice(0, -k)];
}
```

Problem: Merge two sorted arrays

Complexity: $O(n + m)$.

```
function mergeSorted(a, b) {
  const res = []; let i = 0, j = 0;
  while (i < a.length && j < b.length) {
    res.push(a[i] <= b[j] ? a[i++] : b[j++]);
  }
  return [...res, ...a.slice(i), ...b.slice(j)];
}
```

Problem: Find a pair with a given sum

```
function hasPairWithSum(arr, sum) {
  const set = new Set();
  for (const n of arr) {
    if (set.has(sum - n)) return true;
    set.add(n);
  }
  return false;
}
```

Problem: Majority element (appears > $n/2$ times) — Boyer-Moore

Complexity: $O(n)$ time, $O(1)$ space.

```
function majorityElement(nums) {
  let count = 0, candidate = null;
  for (const n of nums) {
    if (count === 0) candidate = n;
    count += (n === candidate) ? 1 : -1;
  }
  return candidate;
}
```

Numbers

Problem: Factorial (iterative + recursive)

```
function factorialIter(n) { let r = 1; for (let i = 2; i <= n; i++) r *= i; return r; }
function factorialRec(n) { return n <= 1 ? 1 : n * factorialRec(n - 1); }
```

```
public static long factorial(int n) {
    long r = 1;
    for (int i = 2; i <= n; i++) r *= i;
    return r;
}
```

Problem: Fibonacci (iterative + recursive + memoized)

```
function fibIter(n) {
    let a = 0, b = 1;
    for (let i = 0; i < n; i++) [a, b] = [b, a + b];
    return a;
}
function fibMemo(n, memo = {}) {
    if (n <= 1) return n;
    if (memo[n]) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}
```

Problem: Check if a number is prime

Complexity: $O(\sqrt{n})$.

```
function isPrime(n) {
    if (n < 2) return false;
    for (let i = 2; i * i <= n; i++) if (n % i === 0) return false;
    return true;
}
```

Problem: Print primes in a range

```
function primesInRange(start, end) {
    const res = [];
    for (let n = start; n <= end; n++) if (isPrime(n)) res.push(n);
    return res;
}
```

Problem: Armstrong number (e.g., $153 = 1^3 + 5^3 + 3^3$)

```
function isArmstrong(n) {  
  const digits = String(n).split("");  
  const p = digits.length;  
  const sum = digits.reduce((a, d) => a + Math.pow(+d, p), 0);  
  return sum === n;  
}
```

Problem: Check if a number is a palindrome

```
function isNumberPalindrome(n) {  
  const s = String(n);  
  return s === s.split("").reverse().join("");  
}
```

Problem: Swap two numbers without a temp variable

```
let a = 5, b = 10;  
a = a + b; b = a - b; a = a - b; // a=10, b=5  
// or with XOR: a ^= b; b ^= a; a ^= b;
```

Problem: GCD and LCM

```
function gcd(a, b) { return b === 0 ? a : gcd(b, a % b); }  
function lcm(a, b) { return (a * b) / gcd(a, b); }
```

Problem: Count digits & sum of digits

```
function sumOfDigits(n) {  
  let sum = 0;  
  n = Math.abs(n);  
  while (n > 0) { sum += n % 10; n = Math.floor(n / 10); }  
  return sum;  
}
```

Problem: FizzBuzz

```
function fizzBuzz(n) {
  for (let i = 1; i <= n; i++) {
    if (i % 15 === 0) console.log("FizzBuzz");
    else if (i % 3 === 0) console.log("Fizz");
    else if (i % 5 === 0) console.log("Buzz");
    else console.log(i);
  }
}
```

Sorting & Searching

Problem: Bubble sort

Complexity: $O(n^2)$.

```
function bubbleSort(arr) {
  for (let i = 0; i < arr.length - 1; i++)
    for (let j = 0; j < arr.length - 1 - i; j++)
      if (arr[j] > arr[j + 1]) [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
  return arr;
}
```

Problem: Binary search (sorted array)

Complexity: $O(\log n)$.

```
function binarySearch(arr, target) {
  let lo = 0, hi = arr.length - 1;
  while (lo <= hi) {
    const mid = Math.floor((lo + hi) / 2);
    if (arr[mid] === target) return mid;
    if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
  }
  return -1;
}
```



```

public static int binarySearch(int[] arr, int target) {
    int lo = 0, hi = arr.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

```

Problem: Quicksort (idea)

Pick a **pivot**, partition the array so smaller elements go left and larger go right, then recursively sort each partition. Average **$O(n \log n)$** , worst case $O(n^2)$.

HashMap-based

Problem: Group anagrams

Approach: Use sorted word as the map key. **Complexity:** $O(n \cdot k \log k)$.

```

function groupAnagrams(words) {
    const map = {};
    for (const w of words) {
        const key = w.split('').sort().join('');
        (map[key] || []).push(w);
    }
    return Object.values(map);
}

```

Problem: First repeating element

```
function firstRepeating(arr) {  
  const seen = new Set();  
  for (const n of arr) {  
    if (seen.has(n)) return n;  
    seen.add(n);  
  }  
  return null;  
}
```

Recursion

Problem: Sum of array elements (recursive)

```
function sumArray(arr, i = 0) {  
  return i === arr.length ? 0 : arr[i] + sumArray(arr, i + 1);  
}
```

Problem: Power(x, n)

Approach: Fast exponentiation. **Complexity:** $O(\log n)$.

```
function power(x, n) {  
  if (n === 0) return 1;  
  const half = power(x, Math.floor(n / 2));  
  return n % 2 === 0 ? half * half : x * half * half;  
}
```

Stack / Linked List

Problem: Check balanced parentheses

Approach: Push opening brackets, pop & match on closing. **Complexity:** $O(n)$.

```
function isBalanced(s) {
  const stack = [];
  const pairs = { "(": ")", "[": "]", "{": "}" };
  for (const ch of s) {
    if ("({[".includes(ch)) stack.push(ch);
    else if (pairs[ch]) {
      if (stack.pop() !== pairs[ch]) return false;
    }
  }
  return stack.length === 0;
}
```

Problem: Reverse a linked list (Java)

Complexity: $O(n)$ time, $O(1)$ space.

```
class Node { int val; Node next; Node(int v){ val = v; } }

public static Node reverse(Node head) {
  Node prev = null, curr = head;
  while (curr != null) {
    Node next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
  }
  return prev; // new head
}
```

Problem: Find the duplicate in an array (Floyd's cycle / set)

```
function findDuplicate(nums) {
  const seen = new Set();
  for (const n of nums) {
    if (seen.has(n)) return n;
    seen.add(n);
  }
  return -1;
}
```

Quick Revision Sheet — the patterns and the 12 problems that cover most rounds

THE 6 PATTERNS ★

1. TWO POINTERS both ends / slow+fast → $O(n)$ time, $O(1)$ space ★
2. HASH MAP/SET ★ "seen it before?" → turns $O(n^2)$ into $O(n)$ ★
3. SLIDING WINDOW contiguous chunk → Kadane, longest substring
4. SORT FIRST makes it obvious → costs $O(n \log n)$
5. MATH/IN-PLACE avoid extra memory → sum formula, XOR swap
6. RECURSION contains itself → factorial, power, fibonacci

THE 12 THAT COME UP MOST ★

- reverse a string two pointers / `StringBuilder.reverse()`
- palindrome two pointers ★ $O(1)$ space
- anagram sort both, or compare FREQUENCY MAPS ★
- first non-repeating `LinkedHashMap` (ORDER matters! ★)
- two sum hash map of value → index ★ (one pass)
- missing number $n(n+1)/2 - \text{sum}$ ★ ($O(1)$ space)
- second largest one pass, two variables ★ (don't sort!)
- max subarray KADANE ★ (keep or restart)
- move zeros two pointers, in place ★
- duplicates a SET ★
- fibonacci iterative, or memoised
- ★ (naive recursion is $O(2^n)$ ▲)
- balanced parentheses a STACK ★

THE TRAPS ★

- second largest → handle DUPLICATES ★ (sort+skip, or use `distinct`)
- binary search → $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$ ★ (avoids integer overflow)
- fibonacci → naive recursion is $O(2^n)$ – say that, then memoise ★
- strings in a loop → `StringBuilder` in Java, `array+join` in JS ★ ($O(n^2)$ otherwise)
- reverse in place → swap, don't build a new array, if $O(1)$ space is asked ★

IN THE ROOM ★

- Clarify → state the approach → brute force → optimise → complexity → dry-run
- "Have I seen this before?" → reach for a SET ★
- Narrate the TRADE-OFF: "sort for $O(n \log n)$ and no extra space, or a set for $O(n)$ time and $O(n)$ space – I'll take the set unless memory is tight." ★

Related files: [22 — Java Streams Coding Problems](#) · [01 — JavaScript](#) · [05 — Core Java](#) · [23 — Predict the Output](#)

Projects — Deep-Dive Interview Q&A (Easy Version)

Experience / "project difficulty" round. Interviewers cross-question on real decisions, trade-offs and concrete technical detail. Speak in the first person, give numbers where you can, and always close a "how did you solve it" answer with the **outcome**.

▲ **Corrected in this version:** RoboGebra's database is **MongoDB**, not MySQL (verified: 104 @Document classes, 27 @Aggregation pipelines). Two answers below still said MySQL — a follow-up like "*how did you model the joins?*" would have exposed it. ● blocks now cite the **real files**.

◆ The easiest way to answer ANY project question — CAPS

C – CONTEXT	what the product does, in ONE sentence
A – ACTION	what I BUILT (name the modules) ★
P – PROBLEM	one hard thing, and the trade-off I chose ★ ★
S – SCALE	numbers, versions, real details ★

★ **The P is what actually gets you hired.** Anyone can list features. Only someone who built it can explain what was *hard* and why they chose one option over another.

WEAK "I built the quiz module using Angular and Java."
↑ this is a resume line read aloud ▲

STRONG "I built the quiz module. The hard part was the live graph – recomputing on every slider tick janked the UI and hammered the AI endpoint, so I split it into two streams: local maths redraws instantly, and the AI insight is debounced 400ms with switchMap so a stale response can't overwrite a newer one." ★

The rule ★

Every project answer needs ONE hard problem you can go three questions deep on.

They will ask: "why?" → "what else did you consider?" → "what broke?"
If you only have the feature list, you run out at question one ▲

Real-world idea: **a cooking show**. Nobody wants the ingredients read out. They want the moment the sauce split and what you did about it.

Easy memory

CAPS ★ Context (1 line) → Action (modules) → PROBLEM + trade-off ★ ★ → Scale (numbers)

Have ONE hard problem per project, deep enough for THREE follow-ups ★
Always finish with the OUTCOME.

1) RoboGebra — AI-Driven Math Learning Platform

Project pitch: RoboGebra is an AI-driven math learning platform that helps students learn step-by-step, with personalized dashboards and bilingual (Tamil/English) explanations. I work as a frontend-focused full-stack engineer building the Angular + Ionic client and the supporting Java/**MongoDB** REST services. I own features like the personalized learning dashboard, the Study Reminder system, the Study List module, the AI math explanation engine integration, the interactive Quiz module, and real-time math computation with live graph visualization. My work directly improved learner retention through scheduled reminders and made complex problems easier to grasp via instant, interactive feedback.

Q: What was your exact role and the team size?

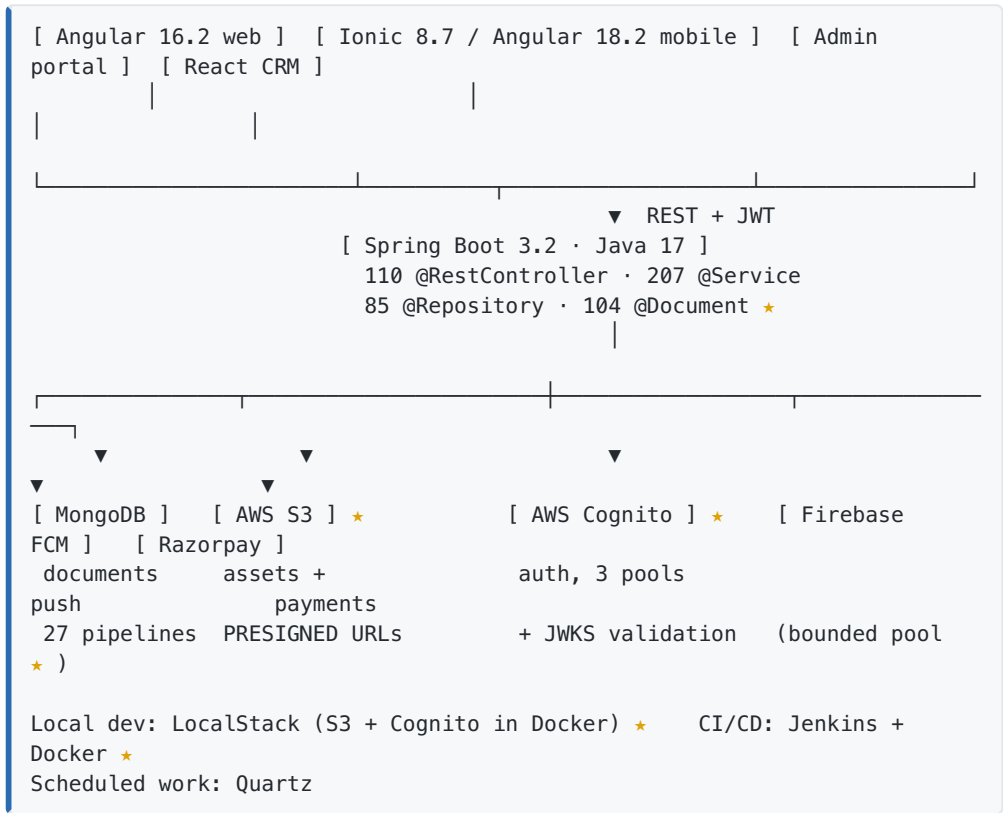
I was the primary frontend engineer (Angular/Ionic) and also wrote backend REST endpoints in **Java 17 / Spring Boot 3.2 with MongoDB** for my modules. The team was around 6–7: a couple of frontend devs, two backend devs, a designer, a

QA, and a lead/PM. I owned the dashboard, reminders, study list, quiz, and the real-time computation features end to end — UI, the API contract, and the persistence layer for those.

Q: Describe the overall architecture.

It's a hybrid mobile/web app: Angular as the framework with Ionic for the mobile/cross-platform UI layer, talking over REST to **Java 17 / Spring Boot 3.2 services backed by MongoDB** (Spring Data MongoDB, not JPA). The AI explanation/quiz generation lives behind a service that the Java layer proxies, so the client never calls the model directly — that keeps keys server-side and lets us cache/normalize responses. On the frontend I used a service-per-domain pattern (DashboardService, ReminderService, StudyListService, QuizService), RxJS for async streams, and a shared state layer for cross-component data like the current learner profile.

● The architecture, with the real numbers ★



■ **Say it like this:** "Four clients — the Angular web app, the Ionic mobile app, an admin portal and a React CRM — all talk to one Spring Boot API over REST with JWT. Persistence is MongoDB through Spring Data. The AI engine sits behind our own service, so the client never holds a key and we can cache and normalise the responses. S3 holds assets and serves them via presigned URLs; Cognito is the identity provider; Firebase does push; Razorpay does payments."

▲ And be precise if they probe the architecture style:

"It's a modular monolith, not microservices — one Spring Boot deployment organised into domain packages that each own their services and repositories. The boundaries are drawn, but we haven't paid the operational cost of splitting it."

Q: The real-time computation and graph visualization sounds expensive. How did you keep it performant?

The problem: when a student drags a slider or edits a value, we recompute the expression and redraw the graph plus fetch AI insights. Naively that fires a request and a re-render on every keystroke/drag tick, which janks the UI and hammers the AI endpoint. I split it into two streams — local math computation (instant, runs in the browser) and the AI insight call (debounced).

```
// Local recompute is instant; AI insight is debounced + cancellable
this.valueChange$.pipe(
  // redraw the graph immediately from the local math engine
  tap(v => this.redrawGraph(this.computeLocally(v))),
  debounceTime(400),
  distinctUntilChanged(),
  switchMap(v => this.quizApi.getAiInsight(v).pipe(
    catchError(() => of(null)) // never let one failed insight break the
    canvas
  ))
).subscribe(insight => this.insight = insight);
```

`switchMap` cancels the in-flight AI request when a newer value arrives, `debounceTime` collapses bursts, and `distinctUntilChanged` skips no-op changes. I also memoized the local compute and used `trackBy` / `OnPush` so the graph component only re-renders when its inputs actually change. Result: the graph feels instant and AI calls dropped by a large margin.

Q: How did you implement the bilingual (Tamil/English) rendering? Any gotchas?

Explanations come back as structured content (steps + formulas), so I keep the language as a key rather than baking translated strings into components. The AI engine returns both language variants (or we request a specific one), and the UI switches via a language signal/observable. The gotchas were font rendering and math: Tamil glyphs needed a font that supports the script, and mixing Tamil prose with LaTeX-style formulas meant I had to render the formula segments separately (MathJax/KaTeX) from the localized prose so the RTL/script handling didn't

mangle the math. I segmented each step into "text" and "formula" tokens and rendered them in their own spans.

Q: How does the Study Reminder system work, and what REST design did you use?

It's flexible scheduling with notifications. I exposed a standard REST resource for reminders with full CRUD:

```
@RestController
@RequestMapping("/api/reminders")
public class ReminderController {
    @GetMapping          // list a user's reminders
    @PostMapping         // create
    @PutMapping("/{id}") // update schedule/status
    @DeleteMapping("/{id}") // remove
}
```

A reminder stores a schedule (one-off or recurring), the target study item, and a status. A scheduler checks due reminders and pushes notifications. On the client I optimistically update the list on POST/PUT/DELETE and reconcile if the server rejects. Adding consistent reminders measurably improved retention because learners came back on cadence.

Q: How did you handle the Study List real-time sync?

The Study List lets learners create/organize/track goals and needs to reflect changes immediately across views (and devices). I kept a single source of truth in a shared service exposing a **BehaviorSubject** of the list; any mutation goes through that service so every subscribed component updates instantly. For cross-device sync, mutations hit the API and the service refreshes/merges so an edit on mobile shows on web. Reordering and completion toggles are optimistic with rollback on error.

Q: How do you manage state across these modules?

For most feature state I use RxJS services with **BehaviorSubject**s (a lightweight store pattern) rather than pulling in NgRx — the app's state is feature-scoped, so a store-per-domain keeps it simple and testable. Shared/global concerns (current

learner, language, auth) live in singleton root-provided services. Components subscribe via the `async` pipe so subscriptions are cleaned up automatically.

● The real state-management answer ★

File: `robogebra-mobile/src/app/core/services/user-summary.service.ts` —
122 `BehaviorSubject` usages, zero `NgRx`.

```
@Injectable({ providedIn: 'root' }) // ★ app-wide
singleton
export class UserSummaryService {
  private userSummarySubject$ = new
  BehaviorSubject<UserSummaryGroupDTO>(null);

  getUserSummarySubject(): Observable<UserSummaryGroupDTO> {
    return this.userSummarySubject$.asObservable(); // ★ READ-
ONLY outside
  }
  getUserSummary(): UserSummaryGroupDTO {
    return this.userSummarySubject$.getValue(); // ★
    SYNCHRONOUS read
  }
}
```

■ "State is `BehaviorSubject` -backed services rather than `NgRx` — that was a deliberate choice. `BehaviorSubject` gave us the two things we actually needed: a current value on subscribe, so a lazily-created page or modal still gets the logged-in user, and a synchronous `getValue()` for callers that can't subscribe — our institute-code HTTP interceptor, for instance. `NgRx` would have meant actions, reducers, effects and selectors for state that a handful of components read. I'd reach for it when state is shared across many unrelated features and I need time-travel debugging."

★ Note the subject is **private** and only `asObservable()` is exposed — so no component can call `.next()` and mutate global state from anywhere. Say that; it's an encapsulation point most people miss.

→ 21 — `NgRx`

Q: How do you handle API errors and loading states?

Every API call goes through typed service methods that return Observables; components render explicit loading/empty/error states rather than blank screens. I use a `catchError` to map server errors into a user-facing message and, where safe, fall back (e.g., the AI insight failing should not break the graph — see the snippet above). An HTTP interceptor centralizes auth headers, 401 handling, and a global error toast so I don't repeat that logic per call.

Q: How did you test this?

Unit tests for the services (Jasmine/Karma) with `HttpTestingController` to assert the right requests and to simulate error responses, and component tests for the dashboard/quiz logic. For the RxJS-heavy real-time code I tested the debounce/switchMap behavior with marble-style tests so I could prove old AI requests get cancelled. QA covered end-to-end flows on device.

Q: What would you do differently?

I'd introduce a proper state library (NgRx Component Store or Signals-based store) earlier — once several modules needed to share the learner/profile state, the hand-rolled subjects got repetitive. I'd also add response caching for AI explanations at the service layer from day one, since identical problems get explained repeatedly and that's wasted model cost and latency.

● Your three strongest "hard problem" stories ★

These are verified in your code. **Pick one and be ready to go three questions deep.**

1. The N+1 query — and the fix ★★

File: `domain/exerciseitem/repository/ExerciseItemRepository.java` (27 `@Aggregation` pipelines)

```
@Aggregation(pipeline = {
    "{ '$match': { 'exercise.$id': { '$in': ?0 } } }",    // ★ ?0 =
    the method parameter
    "{ '$sort': { 'displayOrder': 1 } }"
})
List<ExerciseItemEntity> findAllByExerciseIds(List<ObjectId> exerciseIds);
```

"Loading a chapter's exercises and then fetching each exercise's items one at a time is one query plus N — and @DBRef makes it very easy to fall into, because Spring Data resolves each reference with its own query. A chapter with fifty exercises meant fifty-one round trips. I replaced it with a single aggregation pipeline that matches on all the exercise ids at once with \$in and sorts by display order. It's the same fix as a JPA JOIN FETCH, expressed as a pipeline. What I took from it is that a document database doesn't remove the N+1 problem — it just changes what it looks like."

2. The push-notification thread pool ★★

File: `config/AsyncConfig.java`

```
@Bean(NOTIFICATION_TASK_EXECUTOR)
public ThreadPoolTaskExecutor notificationTaskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(4);
    executor.setMaxPoolSize(8);
    executor.setQueueCapacity(500);                // BOUNDED ★
    executor.setThreadNamePrefix("push-");         // readable thread
    dumps ★
    executor.setRejectedExecutionHandler(new
    ThreadPoolExecutor.CallerRunsPolicy()); // backpressure ★
    executor.setWaitForTasksToCompleteOnShutdown(true);
    executor.setAwaitTerminationSeconds(30);
    return executor;
}
```

"We had one shared @Async pool, and push fan-out was starving it — a single announcement touches every teacher, and each device is a blocking FCM round trip, so analytics events queued behind it. I gave push its own executor with a bounded 500-item queue and a caller-runs rejection policy, so a backlog slows the submitter down instead of growing an unbounded queue until the heap gives out."

A late push beats an `OutOfMemoryError`. The subtle part was that declaring any executor bean makes Spring Boot's auto-configured `applicationTaskExecutor` back off — it's `@ConditionalOnMissingBean(Executor.class)` — so every unqualified `@Async` silently moved onto the notification pool. I had to rebuild Boot's default from its own builder."

★ That last sentence is genuinely senior. It demonstrates the auto-configuration answer *and* a real production trap in one story.

3. The HTTP interceptor ★

File: `core/interceptors/header-authorization.interceptor.ts`

"One interceptor handles the token, the 401 refresh cycle and cancellation. Three details in it are worth calling out. `HttpRequest` is immutable, so you must `clone()` — mutating it silently does nothing. We skip the `/auth` endpoints, or login would need a token to get a token. And we deliberately don't set `ContentType` when the body is `FormData`, because the browser has to generate the multipart boundary itself — setting it manually breaks every file upload. There's also a `cancelRequests$` subject, so logout aborts every in-flight request at once; otherwise a slow response from the previous session can land after the user has signed out."

★ **The `FormData` detail is the one that makes interviewers sit up** — it's a real bug most people have shipped without ever diagnosing.

→ [39 — RoboGebra Code Examples](#) for all three in full.

Q: How did you collaborate?

Agile with two-week sprints, Jira for stories/bugs, Git with feature branches and PR reviews. I'd pick up a story, agree the API contract with the backend dev, build behind a feature branch, get a review, and demo in sprint review.

2) Subsea — Marine Equipment & Operations Management

Project pitch: Subsea is a marine equipment and operations management platform used to track and schedule subsea assets and operations. I worked on the Angular frontend with a .NET API backend, containerized with Docker and deployed on Azure. My main contributions were building the Schedule-Manager module with its .NET APIs, implementing complex grids with Kendo UI, adding dynamic category/subcategory management, and fixing production bugs. I also refactored parts of the architecture to be more modular and reusable.

Q: What was your role and team size?

Full-stack with an Angular emphasis on a team of roughly 5–6. I owned the Schedule-Manager module front to back — Angular UI plus the .NET endpoints — and I was one of the people who picked up production defects.

Q: Describe the architecture.

Angular SPA, .NET Web API services, containerized with Docker and running on Azure. The frontend talked to REST endpoints; Docker gave us consistent local/CI/prod environments, and Azure hosted the containers. The Schedule-Manager was a feature module with its own routing, services, and Kendo-based grid components.

Q: You used Kendo UI Grid with complex/large data. What was the challenge and how did you solve it?

The grids showed large schedule/equipment datasets with sorting, filtering, grouping, and inline editing, and the default client-side binding got sluggish as rows grew. I moved the grid to server-side data operations — paging, sorting, and filtering done in the .NET API rather than loading everything into the browser:

```
// Push paging/sort/filter to the server via Kendo DataSourceRequest
loadGrid(state: State) {
  const query = toDataSourceRequestString(state); // Kendo helper ->
  querystring
  return this.http.get<GridDataResult>(`/api/schedule?${query}`);
}
```



```
// .NET side: let Kendo's DataSourceRequest do paging/sort/filter in the
DB query
[HttpGet]
public DataSourceResult Get([DataSourceRequest] DataSourceRequest request)
    => _db.Schedules.ToDataSourceResult(request); // translates to SQL
paging
```

That meant only one page of rows crossed the wire. Combined with virtual scrolling on the grid and only requesting the columns we display, the grid stayed responsive even with large datasets.

Q: How did the dynamic category/subcategory management work?

Categories and their subcategories were data-driven, not hardcoded — admins could add/edit them and the UI had to reflect that hierarchy live. I modeled it as a parent/child relationship from the API and rendered cascading selects where choosing a category filters its subcategories. On change I reload dependent dropdowns reactively rather than re-fetching the whole tree.

Q: How did Docker fit into your workflow?

Docker gave us a reproducible environment — the same image ran on my machine, CI, and Azure, which killed the "works on my machine" class of bugs, especially for the .NET runtime and dependencies. I worked with the Dockerfile/compose setup to run the API and its dependencies locally so I could integrate the frontend against a realistic backend.

Q: Tell me about a production bug you fixed.

A common pattern was grid/state defects where filters or edits didn't persist correctly after a reload. My approach was reproduce-first: get exact repro steps from the report, reproduce locally against the same data shape, narrow it to frontend vs API by inspecting the network calls, fix at the right layer, and add a guard/test so it didn't regress. Doing it methodically rather than guessing is what kept fixes from bouncing back.

Q: How did you handle state and API calls here?

Feature-scoped Angular services wrapping the .NET endpoints, Observables consumed with the async pipe, and the grid driven by server-side state objects. Error handling went through a shared interceptor plus per-call `catchError` mapping to user-facing messages.

Q: What did you refactor for modularity/reusability?

I pulled repeated grid configuration and category/subcategory selectors into shared components and helpers so other modules could reuse them instead of copy-pasting Kendo setup. I also tightened the feature module boundaries so Schedule-Manager didn't leak dependencies into unrelated areas.

Q: How did you test?

Unit tests for services and component logic, and manual/QA verification of the grid operations and schedule flows. For bug fixes I added regression coverage for the specific failing case.

Q: What would you do differently?

I'd standardize the grid data-binding pattern across the app from the start — we had a mix of client- and server-bound grids, and unifying on server-side operations earlier would have prevented the performance issues. I'd also add more automated integration tests around the .NET endpoints.

Q: How did you collaborate?

Agile sprints, Jira, Git PRs, and close coordination with the backend on the .NET API contracts since I was building both sides of Schedule-Manager.

3) Backoffice UI — Angular Version Migration

Project pitch: This was a focused migration project to bring four Backoffice repositories up to Angular 16. I upgraded a Ticketing application from Angular 2 to 16 and three SSG web-plugin apps from Angular 9 and 4 up to 16. The work was

about navigating years of breaking changes safely and verifying nothing regressed through thorough testing. The payoff was modern, supported, faster-building apps on a single Angular version.

Q: What was your role and team size?

I led/owned the migrations across the four repos. It was a small effort — me driving the upgrades with the original feature teams and QA validating their apps. I was the person who understood the cross-version breaking changes and applied them consistently.

Q: What's your strategy for a multi-version jump like Angular 2/9 to 16?

I never jump straight across — Angular only officially supports one-major-at-a-time upgrades. I went incrementally (e.g., 9 → 10 → 11 ... → 16), using `ng update` per step which runs the migration schematics, building and running tests after each hop, and committing each version as its own checkpoint so I could bisect if something broke. The `update.angular.io` guide was my checklist for each step.

Q: What were the actual breaking changes you had to handle?

A lot. Key ones:

- **Ivy** became the default renderer (v9) — some libraries that relied on View Engine or non-Ivy-compatible metadata needed updates or replacement; I had to ensure all dependencies were Ivy-compatible.
- **RxJS 6 → 7** — typing changes and the move to fully pipeable operators; I had to remove any remaining deprecated operator imports and fix `toPromise()` usage (deprecated, later replaced with `firstValueFrom / lastValueFrom`).
- **Deprecated/removed APIs** — e.g. the `entryComponents` array became unnecessary with Ivy, `ViewChild`'s `static` flag semantics changed (v8/9), and various TypeScript strictness bumps surfaced type errors.
- **TypeScript and tooling version bumps** at each step, plus changes to the build (`angular.json`) schema and eventual move toward the newer build pipeline.
- **Strict mode and stricter templates** flagged previously-silent issues.

Q: How did RxJS 6→7 specifically bite you?

Mostly typing and deprecated patterns. `toPromise()` was deprecated, so I migrated to `firstValueFrom` / `lastValueFrom`. Some operator overloads got stricter so a few `map` / `reduce` chains that were loosely typed needed explicit types. The fix was mechanical but had to be done carefully across the codebases.

```
// before (RxJS 6, deprecated)
const data = await this.http.get('/api/tickets').toPromise();

// after (RxJS 7)
import { firstValueFrom } from 'rxjs';
const data = await firstValueFrom(this.http.get('/api/tickets'));
```

Q: How did you make the transition "seamless"? What was the testing strategy?

Build + test after every single version step, not just at the end — that localizes any breakage to one hop. I ran the existing unit tests, did smoke testing of the critical flows in each app, and leaned on QA for full regression on the Ticketing and plugin apps. Where tests were thin, I added coverage around the riskiest areas before upgrading so I'd notice regressions. Keeping each version in its own commit meant I could always roll back one step.

Q: The three plugin apps were SSG web plugins — anything special?

They were embedded plugins, so I had to make sure the upgraded Angular bundles still loaded correctly inside the host SSG environment and didn't clash with the host's runtime. Verifying the integration boundary (how the plugin is bootstrapped/mounted) was as important as the in-app behavior.

Q: How did you handle third-party libraries that weren't compatible?

I checked each dependency's Angular compatibility before each hop. If a library lagged, I either upgraded it to a compatible version, found a maintained alternative, or temporarily pinned and isolated it until a compatible release existed. Ivy compatibility was the big filter at v9.

Q: What would you do differently?

I'd add or strengthen automated tests before starting the migration, since strong test coverage is what makes a big upgrade safe and fast. I'd also script the per-step build/test loop in CI so each version bump is gated automatically rather than verified by hand.

Q: How did you manage this in Git?

A branch per repo, one commit per major version step with a clear message, and PRs reviewed by the owning team before merge. The granular commits doubled as documentation of exactly what changed at each version.

4) EasyVisa — Visa Management for Attorneys

Project pitch: EasyVisa is a visa case management platform for immigration attorneys — managing applications, documents, and the relationships between cases, applicants, and petitioners. I worked on the Angular frontend against a Grails/Groovy backend with PostgreSQL and Neo4j for graph relationships. I built the document portal module with accordion upload panels, created dynamic forms with Angular Formly including nested reactive forms, customized an image cropper, drove an Angular version upgrade, and built reusable components with unit tests and documentation. My work made complex, document-heavy visa workflows manageable for attorneys.

Q: What was your role and team size?

Frontend engineer on a team of around 6–8 (frontend, Grails/Groovy backend devs, QA, lead). I owned the document portal and the dynamic forms work, and I contributed shared/reusable components used across the app.

Q: Why Neo4j alongside Postgres — what's the architecture?

Postgres held the core relational/transactional data, while Neo4j modeled the relationships — applicants, petitioners, cases, family/employer links — because visa cases are fundamentally a graph of related entities. Querying "who is

connected to this case and how" is natural in a graph and painful with deep relational joins. The Grails/Groovy backend exposed REST endpoints to the Angular frontend; I consumed those, and where the data was relationship-shaped (e.g., showing related parties on a case) it was backed by Neo4j traversals on the server.

Q: The dynamic nested Formly forms sound hard. Walk me through a challenge.

Visa forms are huge, conditional, and repeatable — a section can have N applicants, each with nested address/employment histories, and fields appear/disappear based on earlier answers. Hardcoding that is unmaintainable, so I used Angular Formly to render forms from a JSON field config, with `fieldGroup` for nesting and `fieldArray` for repeatable sections:

```
const fields: FormlyFieldConfig[] = [{
  key: 'applicants',
  type: 'repeat', // custom repeating section
  fieldArray: {
    fieldGroup: [
      { key: 'firstName', type: 'input', props: { label: 'First name',
        required: true } },
      { key: 'hasEmployer', type: 'checkbox', props: { label: 'Currently
        employed?' } },
      {
        key: 'employer', type: 'input', props: { label: 'Employer' },
        expressions: { hide: 'model.hasEmployer === false' } //
        conditional visibility
      }
    ]
  }
}];
```

The hard parts were conditional logic (Formly `expressions` for show/hide and dynamic required), keeping nested reactive form state valid as sections were added/removed, and validation bubbling up from nested groups to the submit button. I solved validation by relying on Formly's nested `FormGroup` aggregation and writing custom validators for the visa-specific rules, then surfacing errors at the right nesting level.

Q: Tell me about the document portal and large uploads.

Attorneys upload many documents per case, organized in accordion panels by document type. Large files were the issue — uploading naively blocks the UI and risks timeouts. I used `HttpClient` with `reportProgress` to stream progress per file, allowed concurrent-but-throttled uploads, and gave each panel its own progress/error state so one failed upload didn't sink the others:

```
this.http.post('/api/documents', formData, {
  reportProgress: true, observe: 'events'
}).subscribe(event => {
  if (event.type === HttpEventType.UploadProgress) {
    this.progress = Math.round(100 * event.loaded / (event.total ?? 1));
  } else if (event.type === HttpEventType.Response) {
    this.markUploaded();
  }
});
```

For very large files the right move is chunked/resumable upload, which I flagged as the next improvement.

Q: You customized an image cropper — why and how?

Certain documents/photos (e.g., applicant photos) had to meet specific dimension/aspect requirements, so I wrapped and customized an Angular image-cropper component to enforce the aspect ratio, output size, and format we needed, and wired its output into the upload flow. The customization was mostly constraining the crop box and post-processing the output blob before upload.

Q: How did you handle state, APIs, and errors?

Feature services wrapping the Grails REST endpoints, Observables with the async pipe, and reactive forms as the source of truth for form state. Errors were handled per-call with `catchError` mapping to user messages plus a shared interceptor for auth and global failures. For the forms, I kept the Formly model as the single state object so save/restore (drafts) was straightforward.

Q: You built reusable components — give an example and the benefit.

I extracted things like the upload-panel, the Formly field types, and common form controls into shared, configurable components. The benefit was consistency and speed: a new form section or document type became a config change rather than new bespoke code, and bugs got fixed once in the shared component.

Q: How did you test, and what about the version upgrade?

I wrote unit tests (Jasmine/Karma) for the components and form logic and documented the reusable pieces so other devs could adopt them. For the Angular upgrade I followed the incremental **ng update** approach, building and testing at each step and fixing breaking changes/deprecations as they surfaced.

Q: What would you do differently?

I'd move large uploads to chunked/resumable from the start, and I'd consider a schema-driven approach where the form JSON configs come from the backend so non-developers could adjust forms without a release. I'd also add more integration tests around the conditional form logic, since that's where regressions hide.

Q: How did you collaborate?

Agile with Jira and Git PR reviews, and tight coordination with the Grails/Groovy team on REST contracts and on what data came from Postgres vs Neo4j.

General project & experience questions

Q: What was the most challenging project and why?

RoboGebra's real-time computation-and-visualization feature, because it combined frontend performance (instant graph updates), async orchestration (debounced, cancellable AI calls), and bilingual rendering of mixed prose and formulas. Getting it to feel instant while not flooding the AI backend forced careful

RxJS design — separating the instant local compute from the debounced/
`switchMap`-cancelled remote call.

Q: How do you approach building a new feature end to end?

Clarify the requirement and acceptance criteria, agree the API contract with the backend early, sketch the component/service breakdown, build behind a feature branch with the happy path plus loading/empty/error states, write unit tests as I go, self-review the diff, open a PR, address review comments, and demo in sprint review. I try to ship a thin vertical slice first, then iterate.

Q: How do you debug a production issue?

Reproduce first — get exact steps and the affected data shape, reproduce locally. Then isolate the layer using the network tab and logs (frontend vs API vs data), form a hypothesis, fix at the correct layer, and add a regression test so it can't silently come back. I avoid guess-and-deploy; methodical reproduction is what stops fixes from bouncing.

Q: What do you look for in code reviews?

Correctness first, then readability and maintainability: clear naming, proper unsubsubscription/async-pipe usage, error/loading handling, no obvious performance traps (unnecessary re-renders, client-side data loads that should be server-side), and adequate tests. I give specific, kind, actionable comments and distinguish blocking issues from nits.

Q: How do you ensure code quality?

Strong typing, small focused components/services, the async pipe and OnPush to avoid subscription leaks and extra renders, unit tests for logic, linting/formatting in CI, and reusing shared components instead of duplicating. Consistent patterns across the codebase matter as much as any single clever fix.

Q: Describe your Agile/Scrum process.

Two-week sprints with planning, daily standups, sprint review/demo, and retro. Work tracked in Jira as stories/bugs with estimates, code in Git with feature branches and PR reviews. Standup is where I flag blockers early.

Q: How do you handle changing requirements mid-sprint?

I clarify the change with the PM/stakeholder, assess impact on scope and the sprint commitment, and raise it transparently rather than silently absorbing it. Small changes I fold in; larger ones get re-estimated and either swapped for something or moved to the next sprint. Building things data/config-driven (like the Formly forms) also made many "changes" cheap.

Q: What was your biggest technical decision and the trade-off?

On EasyVisa, choosing Angular Formly with JSON-driven config for the visa forms instead of hand-coding each form. The trade-off was a steeper initial learning curve and some complexity in custom field types versus huge long-term savings — new forms and conditional logic became configuration, not new components. For the scale of forms there, it clearly paid off.

Q: How do you handle merge conflicts and what's your Git workflow?

Feature branches off main, small frequent commits, regular rebases/merges from main to keep the branch current so conflicts stay small. When conflicts happen I resolve them by understanding both sides' intent — not blindly picking one — re-run the build and tests, and if it's non-trivial I check with the other author. PRs are reviewed before merge.

Q: How do you estimate tasks?

I break a story into concrete sub-tasks, estimate each based on complexity and unknowns (relative sizing / story points), pad for testing and review, and flag risky areas as needing a spike. I'd rather surface uncertainty up front than commit to a number I can't hit. Past similar work calibrates the estimate.

Q: What's your experience with CI/CD and deployment?

On Subsea we used Docker for reproducible builds and deployed containers to Azure, so the same image ran locally, in CI, and in prod — which eliminated environment drift. Generally I work with pipelines that lint, build, and run tests on every PR so problems are caught before merge, and I treat a green pipeline as a gate to deploy.

Q: Tell me about a disagreement on a technical approach.

On grid performance in Subsea there was a question of client-side vs server-side data operations. I argued for server-side paging/sorting/filtering because client-side wouldn't scale with the dataset. Rather than just assert it, I demonstrated the lag with a realistic data volume, we agreed on the server-side approach, and the grid stayed responsive. I focus on evidence and the user impact, and I commit to the decision once it's made even if it isn't the one I proposed.

Quick Revision Sheet — the four projects, one page

CAPS ★ Context → Action (modules) → PROBLEM + trade-off ★ ★ → Scale (numbers)

Every project needs ONE hard problem, deep enough for THREE follow-ups ★

1. ROBOSGEBRA ★ FLAGSHIP

Oct 2024 – now

Angular 16 web · Ionic 8.7 / Angular 18.2 mobile · Java 17 / Spring Boot 3.2

MONGODB ★ (NOT MySQL!) · AWS S3 + Cognito · Firebase · Razorpay · Quartz

BUILT: dashboard · Study List · quiz · AI Tamil/English explanations + THE IONIC MOBILE APP (iOS/Android) ★ ← lead with this

HARD PROBLEM (pick one):

- N+1 → one \$match/\$in aggregation instead of 51 queries ★ ★
- push thread pool → bounded queue + CallerRunsPolicy ★ ★ (+ the @ConditionalOnMissingBean auto-config trap ★)
- interceptor → clone() · FormData Content-Type ★ · cancelRequests\$

STATE: BehaviorSubject services, NOT NgRx – a deliberate choice ★

STYLE: MODULAR MONOLITH, not microservices ★

2. SUBSEA

Apr 2024 – Sep 2024

Angular · .NET · Docker · Azure · Kendo UI Grid

BUILT: Schedule-Manager module + the .NET APIs · category/sub-category mgmt

HARD PROBLEM: large Kendo grid performance → virtualisation + server paging

STORY: learning .NET quickly, coming from Java ★

3. BACKOFFICE MIGRATION ★

Feb 2024 – Mar 2024

FOUR repos → Angular 16 (Ticketing v2→16, three SSG plugins v9/v4→16)

HARD PROBLEM: RxJS 6→7 breaking changes · incompatible third-party libs

STORY: this is your LEADERSHIP / OWNERSHIP answer ★

LINE: "the hard part is never Angular – it's the third-party libraries" ★

4. EASYVISA

Nov 2021 – Jan 2024

Angular · Grails/Groovy · Postgres · Neo4j · Angular FORMLY ★

BUILT: document portal (accordion uploads) · nested reactive forms · custom Formly field types · customised image cropper · unit

tests

HARD PROBLEM: dynamic nested Formly forms + large file uploads
STORY: "above and beyond" – reusable components the whole team adopted

★

LINE: "a new form field became a CONFIG change, not a release" ★

▲ NgRx and Formly are EASYVISA, not RoboGebra ★

—

THE THREE THINGS TO NEVER GET WRONG ★

1. RoboGebra = MONGODB (not MySQL) ★
2. RoboGebra = modular MONOLITH (no Eureka/Feign/circuit breaker) ★
3. NgRx + Formly = EasyVisa · signals + standalone = NOT adopted yet ★

Related files: [00 — Self-Introduction](#) · [11 — HR & Behavioural](#) · [39 — RoboGebra Code Examples](#) · [18 — RoboGebra Technical Versions](#)

HR & Behavioral Interview — Questions & Sample Answers (Easy Version)

▲ BEFORE YOU USE THIS FILE

Every "[COMPANY]" is a BLANK you fill in ★ Q2 ("why this company") and Q22 ("what do you know about them") MUST be rewritten for each interview — a generic answer there is worse than a short one ▲

Fixed in this version: "over four years" → **5 years** · RoboGebra's database → **MongoDB** · every hardcoded "Wipro" → **[COMPANY]** .

◆ The easiest way to answer ANY behavioural question — STAR

S – SITUATION	what was happening?	(1 sentence – set the scene)
T – TASK	what was MY job in it? responsibility)	(1 sentence – your
A – ACTION	what did I DO?	(2–3 sentences – the BIGGEST part ★)
R – RESULT	what happened because?	(1 sentence – the outcome)

S ____	T ____	A _____	R ____
10%	10%	60% ★	20%

★ Most people spend all their time on S and run out of time on A. The ACTION is the only part that tells them about YOU. ★

Real-world idea: **a news report**. *What happened* (situation), *whose job it was* (task), *what was actually done* (action — the real story), *what came of it* (result). A report that is all background and no events is useless.

The three rules

1. Say "I", not "we" ★
"We migrated four repos" tells them NOTHING about you.
"I planned it repo by repo and tested each one" tells them everything ★
2. ALWAYS finish with the RESULT ★
An answer with no result sounds like an unfinished story.
3. 30–90 SECONDS. Past 90 and you're narrating, not answering ★

Easy memory

S _ T _ A _ _ _ R _ ★ – the ACTION is 60% of it
Say "I", not "we" ★ | always land the RESULT ★ | 30–90 seconds ★

1. Tell me about yourself.

Sample answer:

"I'm a Full Stack Developer with **5 years of experience**, currently a Software Engineer at Provility Software Solutions in Chennai. I specialize in Angular on the front end and Java with Spring Boot on the back end. My current project is RoboGebra, an AI-driven math learning platform on Angular, Ionic, Java and **MongoDB**, where I built the learning dashboard, the Study List module and the quiz module — and built our Ionic mobile app for iOS and Android from the existing Angular codebase. I've also worked across .NET, microservices, and cloud on AWS and Azure. I'm now looking for a role with more scale and variety, which is what draws me to **[COMPANY]**."

(See the dedicated 00-self-introduction.md for the full 60-second and 2-minute scripts.)

2. Why do you want to join [COMPANY]? ▲ REWRITE THIS EVERY TIME

▲ This is the one question you cannot answer generically. A vague answer here undoes a good technical round.

The formula ★

1. ONE SPECIFIC thing about THEM (product, domain, scale, the stack in the JD)
2. What I bring that MATCHES it
3. What I'd LEARN there (shows a two-way fit, not desperation)

Fill-in-the-blank template:

"Two things. First, **[ONE SPECIFIC THING — their product, their domain, the scale, or something in the JD]** — that's the kind of work I want to be doing next. Second, the stack lines up closely with mine: Angular and Ionic on the front end, Java and Spring Boot on the back end, which is exactly what I've been building for five years, so I could contribute early rather than spending months ramping up. And what I'd get from it is **[scale / the domain / a bigger engineering team / more structured architecture practice]**, which is the part I don't get in a smaller product team."

Worked example — IQVIA (your next interview) ★

"Two things. First, the domain — healthcare and clinical data is a space where the software genuinely matters, and the constraints around data quality and compliance are the kind of engineering problem I find interesting. Second, the role is an Angular and Ionic technical lead position, and that's precisely what I've been doing — I built our Ionic mobile app for iOS and Android on top of an existing Angular codebase, with around nineteen native plugins. What I'd get from it is the scale and the lead-level responsibility: I've led an Angular 16 migration across four repositories, and I want to keep moving in that direction with a bigger team around me."

★ Notice it names a **real specific thing**, a **real match**, and is honest about what she wants out of it.

3. Why are you looking for a change?

Sample answer:

"I've had a great five years at Provility and I've grown a lot — from building reusable components on EasyVisa to leading an Angular migration and now working on AI-driven features in RoboGebra. At this stage I'm looking for a larger platform with bigger, more complex projects and clearer growth into senior roles. It's a move toward more scale and learning, not away from any problem. I want to keep building, just at a larger scale and with broader exposure."

4. What are your strengths?

Sample answer:

"My biggest strength is end-to-end ownership — I'm comfortable building a feature from the REST API in Spring Boot or .NET all the way up to the Angular UI, which means I can move fast without waiting on hand-offs. Second, I'm strong on reusable, maintainable front-end work — on EasyVisa I built custom Formly components and reusable form modules that the whole team used. And third, I'm adaptable: across my projects I've picked up Angular, React, .NET, Grails and Groovy as the work demanded, so learning a new stack quickly is something I'm confident about."

5. What are your weaknesses?

Sample answer:

"I tend to be a bit of a perfectionist with code — I used to spend extra time polishing things that were already good enough. I've learned to manage it by being more deliberate about priorities: I ask what's truly important for the release,

get it working and reviewed first, and refine later if there's time. I also lean on code reviews and feedback to know when something is genuinely 'done.' It's made me faster while still keeping quality high. The other thing I've been actively working on is public speaking and presenting in larger forums, which I'm improving by volunteering to demo features in our sprint reviews."

6. What is your biggest achievement?

Sample answer (STAR):

"Situation: On RoboGebra, one of the key differentiators was giving students math explanations in their own language. **Task:** I was responsible for building the front-end of the AI math explanation feature and the surrounding learning experience — dashboard, quiz, and reminders. **Action:** I built the personalized learning dashboard and quiz module in Angular and Ionic, integrated the AI explanation engine to deliver bilingual Tamil and English explanations, and wired up real-time math computation and visualization. I also built a study reminder system backed by REST APIs so students stayed on track. **Result:** The platform delivered a genuinely personalized, accessible learning experience, especially for Tamil-speaking students who could finally learn math in their native language. It's the project I'm most proud of because it combined clean engineering with real user impact."

7. Tell me about a time you had a conflict with a teammate.

Sample answer (STAR):

"Situation: On one project, a backend teammate and I disagreed on the API contract for a form-heavy module — they wanted one large endpoint, I wanted more granular ones for the nested reactive forms I was building. **Task:** We needed to agree quickly without blocking the sprint. **Action:** Instead of pushing my view, I set up a short call, walked through the actual UI flows and how the form state would be saved, and listened to their concern about round-trips and performance. We landed on a middle ground — a couple of focused endpoints plus one batch

save. **Result:** It unblocked both of us within a day, the integration was clean, and we actually used that same pattern on later modules. I learned that most 'conflicts' are just two valid priorities that haven't been talked through yet."

8. How do you handle tight deadlines?

Sample answer:

"I break the work down first and figure out what's truly must-have versus nice-to-have, then communicate early if something is at risk — surprises are worse than bad news. A real example: during the Subsea project I had bug fixes plus the Schedule-Manager module to deliver in a short window. I prioritized the blocking bugs, got the core Schedule-Manager flow with the Kendo UI grid working end to end first, and left polish for last. I also kept my lead updated daily so expectations were aligned. We shipped on time. Under pressure I stay calm, prioritize ruthlessly, and over-communicate."

9. Tell me about a time you failed, and what you learned.

Sample answer (STAR):

"**Situation:** Early in my career on EasyVisa, I built a reusable Formly component and assumed my local testing was enough, so I didn't write thorough unit tests for an edge case. **Task:** The component was meant to handle dynamic nested forms. **Action:** A configuration I hadn't tested broke in another part of the app during QA. I owned it immediately, fixed it, and then went back and added proper unit tests covering the edge cases. **Result:** It was caught before release, but it taught me to treat unit tests as part of 'done,' not an afterthought. Since then, writing tests for the components I own has been a standard part of my workflow, and it's saved me many times."

10. How do you handle feedback or criticism?

Sample answer:

"I genuinely value it — code reviews are one of the main ways I've grown. I try not to take feedback personally; I separate the code from my ego. When I get a review comment I don't agree with, I ask questions to understand the reasoning rather than just defending my approach, and often I end up learning something. When I do disagree after that, I explain my view with context and we decide together. Feedback is faster learning, so I'd rather hear it early than discover a problem in production."

11. Where do you see yourself in 5 years?

Sample answer:

"In five years I'd like to be a senior full-stack engineer or a technical lead — someone who not only delivers complex features but also makes architecture decisions and mentors junior developers. I've already had a taste of leadership through the Angular 16 migration I led, and I enjoyed owning that. At a company like **[COMPANY]**, with its scale and structured growth paths, I see a clear runway to deepen my technical expertise while taking on more ownership and eventually guiding a team."

12. What are your salary expectations?

Sample answer:

"I'm looking for a package that reflects my **5 years** of full-stack experience and is aligned with industry standards for this role and location. I'm open and flexible, and I trust **[COMPANY]** to make a fair offer based on my skills and the value I bring. If you can share the band for this position, I'm happy to discuss where I'd fit within it. Overall, the right role and growth opportunity matter as much to me as the number."

13. What is your notice period?

Sample answer:

"My current notice period is [your actual notice period, e.g. 60 days]. I'm committed to a smooth handover for my team, but I'd also do my best to wrap up responsibly and join as early as possible. If there's flexibility through a buyout or an early release, I'm open to exploring that."

Tip: state your real notice period and, if you can, signal willingness to expedite — companies value an earlier start.

14. Are you open to relocation?

Sample answer:

"Yes, I'm open to relocation for the right opportunity. I'm currently based in Chennai and comfortable there, but I'm flexible and would relocate to wherever the role is based. I'd just appreciate reasonable notice to plan the move. I'm also comfortable with hybrid or remote arrangements if that's how the team works."

Tip: adjust this honestly based on your actual willingness and any constraints.

15. Are you a team player or an individual contributor?

Sample answer:

"Both, depending on what the work needs. I can put my head down and deliver independently — I've owned full modules like the document portal on EasyVisa and the Schedule-Manager on Subsea end to end. But I also thrive in a team: I share reusable components, do code reviews, help unblock teammates, and I led the Angular 16 migration that touched four repositories and required coordinating across people. Good engineering is collaborative, so I'm comfortable switching between focused individual work and tight teamwork."

16. How do you stay updated with new technologies?

Sample answer:

"I follow the official docs and release notes for the frameworks I use — Angular especially, which is how I confidently led the migration to Angular 16. I read engineering blogs and follow the community, and I learn a lot hands-on by building small things to try new patterns, like RxJS operators or NgRx state patterns. I've also picked up new stacks on the job as projects demanded — React, .NET, Grails, Groovy — so I treat each project as a chance to learn. Staying current is just part of being a developer for me."

17. Why should we hire you?

Sample answer:

"Because I bring **5 years** of genuine full-stack delivery — Angular and Ionic on the front end, Java, Spring Boot and .NET on the back end — and a track record of owning features end to end across multiple domains: legal tech, marine operations, and AI-driven education. I ramp up quickly on new stacks, I write reusable and tested code, and I've already led work like an Angular migration. I'm low-maintenance, delivery-focused, and I communicate proactively. For a lateral role you need someone who can contribute from week one while still growing into more responsibility — that's exactly what I offer."

18. Describe a time you took ownership or showed leadership.

Sample answer (STAR):

"**Situation:** Four of our repositories were on an older Angular version and accumulating tech debt. **Task:** I was given the job of migrating them to Angular 16. **Action:** I planned the upgrade repo by repo, worked through breaking changes and deprecated APIs, updated dependencies carefully, and tested each migration before moving on so the team's work was never blocked for long. **Result:** All four repos were successfully migrated to Angular 16, the codebase

became easier to maintain, and we could use newer framework features. It was a project I largely drove myself, and it built my confidence in owning larger technical initiatives."

19. Tell me about a time you had to learn something new quickly.

Sample answer (STAR):

"Situation: When I moved to the Subsea project, the back end was .NET — and I'd been working primarily in Java and Grails. **Task:** I needed to build the Schedule-Manager module with .NET APIs and a Kendo UI grid. **Action:** I leaned on my general back-end fundamentals, went through the existing .NET codebase and docs, and built incrementally — starting with simple endpoints before the full category and subcategory management. I asked targeted questions when stuck rather than spinning. **Result:** I delivered the module on schedule, and adding ASP.NET Core to my toolkit made me a more versatile developer. It confirmed that I can pick up a new stack quickly when the work calls for it."

20. How do you prioritize when you have multiple tasks?

Sample answer:

"I look at impact and dependencies first — what's blocking others or the release goes to the top. On Subsea I was balancing bug fixes with building the Schedule-Manager module, so I cleared the blocking bugs first, then moved to feature work. I use Jira to keep everything visible and track status, and I align with my lead when priorities are unclear rather than guessing. I also time-box things so one task doesn't quietly eat the whole day. Clear priorities plus communication keeps me from getting overwhelmed."

21. How do you handle pressure or stress?

Sample answer:

"I deal with stress by getting organized — when things feel chaotic, breaking the work into a clear list immediately calms it down because I can see a path forward. I focus on what I can control, tackle the highest-priority item, and communicate early if a deadline is genuinely at risk. I've handled crunch periods, like delivering across bug fixes and a new module on Subsea in a short window, by staying methodical rather than panicking. And I make sure to step away briefly when I'm stuck — a short break often solves the bug faster than staring at it."

22. What do you know about [COMPANY]? ▲ RESEARCH THIS BEFOREHAND

▲ **Fifteen minutes of research before the call is the highest-value preparation you can do.** This question separates candidates who want *this* job from candidates who want a job.

What to look up — 15 minutes ★

1. What do they actually SELL? (the product, or the service line)
2. WHO are their customers / what industry? ★
3. Any recent NEWS — a launch, an acquisition, a funding round
4. The JD's stack — and which part matches you ★
5. Something from their engineering blog or careers page

The structure

"**[What they do, in one sentence.]** What stands out to me is **[one specific thing]**, and from the JD it looks like this role is about **[the actual work]**. That matches where I want to go — I've been building **[your matching experience]**, and I'd want to keep going deeper in that direction."

Worked example — IQVIA ★

"IQVIA works in healthcare data and clinical research technology — the products sit between clinical trials, real-world evidence, and the analytics built on top of that data. What stands out to me is that it's a domain where correctness genuinely matters; a bug in a marketing dashboard is one thing, a bug in clinical data is another. From the JD, this role is an Angular and Ionic technical lead position, which maps directly onto what I've been doing — I built our Ionic mobile app on top of an existing Angular codebase, and I've led an Angular 16 migration across four repositories. So the technology is familiar and the domain would be new, which is exactly the combination I want."

★ **"The technology is familiar and the domain would be new"** is a strong line — it says you can contribute *and* still be learning.

23. Tell me about a time you went above and beyond.

Sample answer (STAR):

"Situation: On EasyVisa, several teammates kept rebuilding similar form components from scratch, which was slow and inconsistent. **Task:** My assigned work was just my own module, but I saw the repeated effort. **Action:** I took the initiative to build a set of reusable Angular Formly custom components and reusable form modules — including things like an image cropper and nested reactive form handling — and documented how to use them. **Result:** The team adopted them, which sped up development across modules and kept the UI consistent. It wasn't strictly my task, but it had a bigger payoff for the whole team."

24. How do you ensure code quality?

Sample answer:

"A few habits. I write unit tests for the components I own — I learned that lesson early on EasyVisa. I participate actively in code reviews, both giving and receiving

feedback, and I build reusable components so logic isn't duplicated and bugs don't multiply. I follow the team's coding standards and keep things readable, because code is read far more than it's written. And I test edge cases, not just the happy path. Quality for me is about preventing problems early rather than firefighting later."

25. Do you have any questions for us?

Sample answer (always ask 2-3 — never say "no"):

"Yes, a few:

- What does the team I'd join look like, and what stack and projects would I be working on?
- How is success measured for this role in the first six months to a year?
- What do the growth and learning paths look like at **[COMPANY]** for someone at my level?
- How does the team handle code reviews, testing, and engineering best practices?
- What are the biggest technical challenges the team is currently working on?"

Tip: pick the 2-3 most relevant to the conversation. Asking thoughtful questions signals genuine interest and seniority.

★ The three questions that make you sound senior

1. "What does success look like in the first six months?" ★
→ you're thinking about DELIVERING, not about getting hired
2. "What's the biggest technical challenge the team has right now?" ★
→ you're interested in the WORK — and the answer is genuinely useful to you
3. "How does the team handle code review and testing?" ★
→ you care about HOW they build, not just what they build

- ▲ NEVER ask about salary, leave or working hours in a TECHNICAL round — those belong in the HR / offer conversation ★
- ▲ NEVER say "no, I don't have any questions" — it reads as no interest ▲

Quick Revision Sheet — read this before the HR round

STAR ★ S _ T _ A ____ R _ — the ACTION is 60%
Say "I", not "we" ★ | always land the RESULT ★ | 30–90 seconds

YOUR FACTS

5 years ★ · Provility Software Solutions, Chennai, Oct 2021 – now
RoboGebra (Angular · Ionic · Java · MONGODB ★) → dashboard, Study List,
quiz, AI Tamil/English explanations, and the IONIC MOBILE APP ★
Subsea (Angular/.NET, Schedule-Manager) · Angular 16 migration, 4 repos
★
EasyVisa (document portal, Angular Formly, nested reactive forms)
B.Tech IT – Anna University, IRTT Erode, 2020, 77.1%

THE STORIES – one each, ready to tell ★

LEADERSHIP / OWNERSHIP → the Angular 16 migration (4 repos, planned
repo by repo)
ACHIEVEMENT → RoboGebra bilingual AI explanations + the
mobile app
FAILURE → the untested Formly edge case → tests are part
of "done"
CONFLICT → the API-contract disagreement → talked through
the UI flow
LEARNING FAST → .NET on Subsea, coming from Java
ABOVE AND BEYOND → built reusable Formly components the whole
team adopted
PRESSURE → Subsea: blocking bugs first, then the module,
daily updates

▲ REWRITE PER COMPANY ★

Q2 "Why [COMPANY]?" → one SPECIFIC thing + the match + what
you'd learn
Q22 "What do you know?" → 15 minutes of research: product,
customers, news, stack
Closing line → "...which is why I'm interested in
[COMPANY]"

TONE

- ✓ Never criticise Provility – frame the move as GROWTH, not escape ★
- ✓ Real project NAMES make you credible
- ✓ Pause before hard questions – composure reads as seniority ★
- ✓ Be consistent on notice period, relocation and salary – consistency = trust

Quick Reminders for the HR Round

- **Stay positive about your current employer** — never badmouth Provility or teammates. Frame your move as growth, not escape.
- **Use real project names** — RoboGebra, Subsea, EasyVisa, the Angular 16 migration. Specifics make you credible.
- **Quantify or describe impact** where you can, even qualitatively ("reused across the team," "delivered on time," "all four repos migrated").
- **Keep answers 30-90 seconds.** Long enough to show substance, short enough to keep attention.
- **Be honest** about notice period, relocation, and salary — consistency builds trust.
- **Smile, make eye contact, and pause before answering** tough questions. Composure reads as seniority.

Java 17 — Version-Specific Features (Easy Version)

Why this file matters: almost every interview asks *"What's new since Java 8?"* or *"What Java 17 features have you used?"*

Every topic below follows the same shape — **Easiest way to remember** → **Old way vs New way** → **Real-world example** → **Diagram** → **Interview trap** → **Interview-ready answer** → **Easy memory box**.

Contents

1. [Java 17 Basics \(LTS, Java 8 vs 17\)](#)
 2. [Records](#)
 3. [Sealed Classes](#)
 4. [Switch Expressions](#)
 5. [Pattern Matching](#)
 6. [Text Blocks](#)
 7. [Other Features \(Java 9–17\)](#)
 8. [Interview Scenarios](#)
-

Java 17 Basics

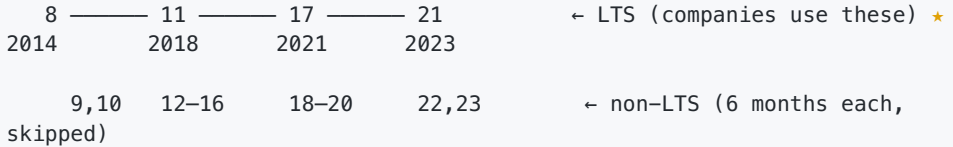
Q: Why is Java 17 important?

The easiest way to remember:

Java 17 = an LTS (Long-Term Support) release,
just like Java 8, 11 and 21.

Companies stay on LTS versions because they get security patches for
YEARS.

The LTS timeline



Real-world idea: **phone operating systems**.

LTS release → an Android version that gets security updates for 5
years
Non-LTS release → a beta build that is replaced in 6 months

A bank will never run its production system on something that stops receiving
patches in six months. That is the entire reason LTS matters.

What Java 17 actually bundles

Java 17 is not one big feature — it is **everything from Java 9 to 17 packaged
into one supported release**:

Java 9 → modules, `List.of()`, `takeWhile`
Java 10 → `var`
Java 11 → `HttpClient`, `String.strip()`, run a `.java` file directly
Java 14 → switch expressions, helpful `NullPointerExceptions`
Java 15 → text blocks
Java 16 → records, pattern matching for `instanceof`
Java 17 → sealed classes ★ LTS

Also important: **Spring Boot 3.x requires Java 17 as a minimum**. That is why so
many companies moved in 2023–24, and why interviewers now expect you to
know it.

Interview-ready answer

Java 17 is a Long-Term Support release from September 2021, in the same family as Java 8, 11 and 21. LTS versions receive security patches and updates for many years, so enterprises standardise on them, while the non-LTS releases in between are short-lived. Java 17 is significant because it bundles everything introduced from Java 9 onwards — records, sealed classes, pattern matching, switch expressions and text blocks — into one stable, supported release. It also became a practical requirement because Spring Boot 3 needs Java 17 as a minimum.

Easy memory

LTS = 8 → 11 → 17 → 21 (long support, companies use these) ★
Non-LTS = everything else (6 months, skipped)

Java 17 = Java 9...17 features + long support
Spring Boot 3 REQUIRES Java 17

Q: How is Java 17 different from Java 8?

The easiest way to remember — **Java 8 gave us functional programming, Java 17 removed boilerplate.**

Java 8 → HOW we process data → lambdas, streams, Optional
Java 17 → HOW we write classes → records, sealed, pattern matching, text blocks

The comparison table

Topic	Java 8	Java 17
DTO class	60 lines of getters>equals/hashCode	<code>record Employee(Long id, String name) {}</code> — 1 line
<code>switch</code>	statement, needs <code>break</code>	expression , returns a value, arrow syntax
Multi-line String	<code>"line1\n" + "line2\n"</code>	text block with <code>"""</code>

Topic	Java 8	Java 17
Type check	<code>instanceof</code> + explicit cast	pattern matching: <code>if (o instanceof String s)</code>
Restricting subclasses	not possible	sealed classes
Local variables	<code>ArrayList<String> list = new ArrayList<>();</code>	<code>var list = new ArrayList<String>();</code>
Immutable list	<code>Collections.unmodifiableList(...)</code>	<code>List.of("a","b")</code>
NullPointerException	"NullPointerException" (which one?!)	tells you the exact variable
HTTP calls	<code>URLConnection</code> (painful)	built-in <code>HttpClient</code> with HTTP/2

See the difference in one example

Java 8 DTO:

```
public class Employee {
    private final Long id;
    private final String name;

    public Employee(Long id, String name) {
        this.id = id; this.name = name;
    }
    public Long getId() { return id; }
    public String getName() { return name; }

    @Override public boolean equals(Object o) { /* 10 lines */ }
    @Override public int hashCode() { return Objects.hash(id, name); }
    @Override public String toString() { /* 3 lines */ }
}
```

Java 17:

```
public record Employee(Long id, String name) {}
```

Same functionality. 25 lines became 1.

Interview-ready answer

Java 8 was about functional programming — it introduced lambdas, the Stream API, `Optional` and functional interfaces, which changed how we process data. Java 17 is mainly about reducing boilerplate and improving language expressiveness. Records replace verbose DTO classes, sealed classes let us define a closed hierarchy, pattern matching removes the cast after `instanceof`, switch expressions return values with no fall-through, and text blocks make multi-line JSON or SQL readable. There are also practical improvements like `var`, immutable collection factories such as `List.of()`, helpful `NullPointerException` messages that name the exact variable, and a modern built-in `HttpClient`.

Easy memory

```
Java 8 = functional → lambda, stream, Optional      (HOW we process)
Java 17 = less code  → record, sealed, pattern, """" (HOW we write)

25-line DTO → record Employee(Long id, String name) {}
```

Records (Java 16)

Q: What is a record?

The easiest way to remember:

record = a class whose ONLY job is to carry data.

Java writes the constructor, getters, equals(), hashCode() and toString() for you.

The pain it removes

```
// BEFORE – the classic DTO
public class Employee {
    private final Long id;
    private final String name;
    private final String dept;

    public Employee(Long id, String name, String dept) { ... }

    public Long getId() { ... }
    public String getName() { ... }
    public String getDept() { ... }

    @Override public boolean equals(Object o) { ... }
    @Override public int hashCode() { ... }
    @Override public String toString() { ... }
}
```

```
// AFTER
public record Employee(Long id, String name, String dept) {}
```

What the compiler generates for you

```
record Employee(Long id, String name, String dept)
|
|— private final Long id;           ← fields (final!)
|— private final String name;
|— private final String dept;
|
|— Employee(Long, String, String)   ← canonical constructor
|
|— id()      name()    dept()      ← accessors (NO "get"
prefix ★ )
|
|— equals()      ← compares ALL fields
|— hashCode()    ← uses ALL fields
|— toString()    ← Employee[id=1, name=Priyanka,
dept=Engineering]
```

Usage:

```
Employee e = new Employee(1L, "Priyanka", "Engineering");

e.name();           // "Priyanka" ← note: name(), NOT getName() ★
e.toString();       // Employee[id=1, name=Priyanka, dept=Engineering]

Employee e2 = new Employee(1L, "Priyanka", "Engineering");
e.equals(e2);       // true – value-based equality, for free ✓
```

Real-world idea: an **ID card**.

An ID card only CARRIES information – name, number, photo.
It has no behaviour, and nobody edits it after printing.

That is exactly a record: data in, never changed.

Rules and limits ★

- ✓ Fields are implicitly private FINAL → immutable
- ✓ The record class itself is implicitly FINAL → cannot be extended
- ✓ CAN implement interfaces
- ✓ CAN have static fields and static methods
- ✓ CAN have extra instance methods
- ✓ CAN have a compact constructor for validation
- ✗ CANNOT extend another class (it already extends java.lang.Record)
- ✗ CANNOT add instance fields outside the header
- ✗ CANNOT have setters (it is immutable)

The compact constructor — for validation

```
public record Employee(Long id, String name) {

    public Employee {                               // no parameter
list! ★
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name is required");
        }
        name = name.trim();                          // you can
normalise too
    }
}
```

```
new Employee(1L, " Priya ")
    ↓
compact constructor runs → validates + trims
    ↓
fields are assigned automatically ✓
```

You can also add methods:

```
public record Employee(Long id, String firstName, String lastName) {

    public String fullName() {                // extra behaviour ✓
        return firstName + " " + lastName;
    }

    public static Employee of(String first, String last) {    // factory ✓
        return new Employee(null, first, last);
    }
}
```

Record vs Lombok `@Data` ★

	record	Lombok <code>@Data</code>
What it is	native Java language feature	third-party annotation processor
Extra dependency	✗ none	✓ needs the Lombok jar + IDE plugin
Mutability	immutable only	mutable (generates setters)
Accessors	<code>name()</code>	<code>getName()</code>
Inheritance	cannot extend a class	can
Best for	DTOs, API request/response, value objects	JPA entities, mutable beans

Where NOT to use a record — JPA entities ▲

```
@Entity
public record Employee(Long id, String name) {}    // ✗ does NOT work
```

Hibernate needs a **no-arg constructor**, **mutable fields** and proxy support — a record has none of those. So:

```
DTO / API response / value object → record ✓  
JPA @Entity                        → normal class (with Lombok) ✓
```

That distinction is exactly what interviewers probe.

Interview-ready answer

A record, finalised in Java 16, is a special class designed to carry immutable data. We declare the components in the header and the compiler generates the private final fields, the canonical constructor, accessor methods, and `equals()`, `hashCode()` and `toString()` based on all components. The accessors are named after the component, so it is `name()` rather than `getName()`. Records are implicitly final and cannot extend another class, though they can implement interfaces and can have static members, extra methods, and a compact constructor for validation. I use records for DTOs, API request and response objects and value objects. I don't use them for JPA entities, because Hibernate requires a no-argument constructor and mutable fields, so those remain normal classes.

Easy memory

record = data carrier → an ID card

```
record Employee(Long id, String name) {}  
    → final fields + constructor + id()/name() + equals + hashCode +  
    toString  FREE
```

Accessor is `name()`, NOT `getName()` ★
Implicitly FINAL, immutable, cannot extend a class
Compact constructor → validation: `public Employee { ... }`

✓ DTO / API response ✗ JPA @Entity (needs no-arg ctor + mutability)

Sealed Classes (Java 17)

Q: What are sealed classes?

The easiest way to remember:

```
sealed = "ONLY these specific classes are allowed to extend me."  
  
It's a guest list.
```

```
public sealed interface Shape permits Circle, Square, Rectangle {}  
  
public final class Circle implements Shape { }  
public final class Square implements Shape { }  
public non-sealed class Rectangle implements Shape { }    // reopened
```

Anyone else who tries:

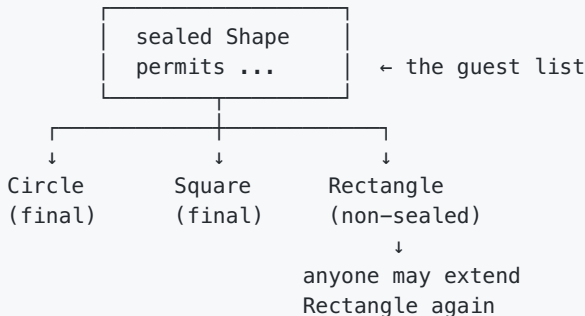
```
public class Triangle implements Shape { }    // ✗ compile error – not on  
the list
```

The problem it solves

Before sealed classes you had only two extremes:

```
public class Shape { }    → ANYONE can extend it → you lose control  
final class Shape { }    → NOBODY can extend it → too restrictive
```

sealed is the middle ground: **a controlled, known set of subclasses.**



Real-world idea: a **private wedding**.

```
public class      → open ground: anyone walks in
final class      → locked room: nobody enters
sealed ... permits → invitation only
```

Another real example: payment methods in your app are exactly UPI, Card and NetBanking. There is no fourth. Sealing that hierarchy documents the rule *and* enforces it.

Every permitted subclass must choose one of three words ★

```
final class Circle implements Shape { }           // 1. nobody extends me
sealed class Square implements Shape              // 2. only MY list may
extend me
    permits SmallSquare { }
non-sealed class Rectangle implements Shape { } // 3. I'm open again to
everyone
```

```
final      → the chain STOPS here
sealed     → the chain continues, but still controlled
non-sealed → the chain is reopened to everyone
```

If you write none of the three, it will not compile.

The real payoff — exhaustive switch ★★

This is the reason sealed classes exist:

```
String describe(Shape shape) {
    return switch (shape) {
        case Circle c   → "Circle radius " + c.radius();
        case Square s   → "Square side " + s.side();
        case Rectangle r → "Rectangle";
    };
    // ✓ NO default needed
}
```

The compiler knows the complete list of subtypes, so it can prove the switch covers everything.

And if someone later adds a fourth shape:


```
public sealed interface Shape permits Circle, Square, Rectangle, Triangle
{}
```

→ every switch that doesn't handle Triangle now FAILS TO COMPILE ★

You find the problem at BUILD time, not at 2 a.m. in production.

Without sealed classes you'd need a `default` branch, which silently swallows the new case.

Where the permitted classes must live

Same file	→ the <code>permits</code> clause can be omitted entirely
Same package	→ allowed
Same named module	→ allowed

Interview-ready answer

Sealed classes, finalised in Java 17, let a class or interface restrict which types are allowed to extend or implement it, using the `permits` clause. Before this we only had `public`, which lets anyone extend, or `final`, which lets nobody — sealed gives a controlled middle ground where the hierarchy is known and closed. Every permitted subclass must be declared `final`, `sealed` or `non-sealed`, so the author explicitly decides whether the hierarchy continues. The main benefit is exhaustive pattern matching: because the compiler knows all the subtypes, a switch over a sealed type does not need a `default` branch, and if a new subtype is added later, every switch that fails to handle it becomes a compile error rather than a runtime bug. I would use it to model a closed domain set, such as the payment types or event types supported by a service.

Easy memory

sealed = a GUEST LIST → "only these classes may extend me"

public → open ground | final → locked room | sealed → invitation only

sealed interface Shape permits Circle, Square, Rectangle {}

Each child MUST be: final | sealed | non-sealed

★ Payoff: exhaustive switch with NO default →
adding a new subtype breaks the BUILD, not production.

Switch Expressions (Java 14)

Q: What are switch expressions?

The easiest way to remember:

Old switch → a STATEMENT → does something → needs break → can fall through ▲

New switch → an EXPRESSION → RETURNS a value → arrow syntax → no fall-through ✓

The old pain

```
String size;
switch (day) {
    case MONDAY:
    case FRIDAY:
    case SUNDAY:
        size = "Busy";
        break;           // ← forget this and the bug is silent ▲
    case TUESDAY:
        size = "Light";
        break;
    default:
        size = "Normal";
}
```

Three problems: you must declare the variable first, you must repeat `break`, and forgetting one `break` causes fall-through that is very hard to spot.

The new way

```
String size = switch (day) {  
    case MONDAY, FRIDAY, SUNDAY -> "Busy";           // multiple labels ★  
    case TUESDAY                  -> "Light";  
    default                       -> "Normal";  
}; // ← note the semicolon
```

Old: switch DOES something → assigns a variable → needs break

New: switch RETURNS something → assign it directly → no break ✓

Real-world idea: a **vending machine**.

Old switch → you press a button, and hope the right item drops
(if the mechanism doesn't stop, more items keep falling = fall-through)

New switch → you press a button and it HANDS you exactly one item ✓

Multi-line branches — use **yield**

```
int result = switch (grade) {  
    case "A" -> 100;  
    case "B" -> 80;  
    default -> {  
        System.out.println("Unknown grade, defaulting");  
        yield 0; // ★ `yield` returns from a BLOCK  
    }  
};
```

Single expression → -> value (no yield needed)

Block { } → -> { ...; yield value; }

Note: **yield** returns from the *switch*; **return** would exit the whole method.

Exhaustiveness ★

When switching over an enum, the compiler now checks that every constant is handled:

```
enum Status { NEW, IN_PROGRESS, DONE }

String label = switch (status) {
    case NEW          -> "Just created";
    case IN_PROGRESS -> "Working";
    case DONE         -> "Finished";
};
// ✓ no default needed – all cases covered
```

Add a fourth enum constant later and this stops compiling — exactly what you want.

You can still use the old syntax with the new form

```
switch (day) {
    case MONDAY -> System.out.println("Start");    // arrow, no return
    value
    default     -> System.out.println("Other");
}
```

Arrow syntax alone already removes fall-through, even when you don't return a value.

Interview-ready answer

Switch expressions, finalised in Java 14, allow **switch** to return a value instead of only performing statements. They use arrow syntax, which removes fall-through entirely, so no **break** is needed, and multiple labels can be combined in a single case such as **case MONDAY, FRIDAY ->**. For multi-line branches we use **yield** to return the value from the block. Another benefit is exhaustiveness checking: when switching over an enum or a sealed type, the compiler verifies that every case is covered, so no **default** branch is needed and adding a new constant later produces a compile error instead of a silent runtime bug. Overall it removes the two classic switch problems — forgotten **break** statements and uninitialised result variables.

Easy memory

Old switch → statement → break → fall-through bugs ▲

New switch → EXPRESSION → returns a value → no break ✓

```
String s = switch (day) {  
    case MONDAY, FRIDAY -> "Busy";    ← multiple labels  
    default -> { yield "Normal"; }    ← yield for a BLOCK  
};
```

Enum/sealed → EXHAUSTIVE → no default needed → new constant breaks the build ★

Pattern Matching for instanceof (Java 16)

Q: What is pattern matching for instanceof ?

The easiest way to remember:

It merges the TYPE CHECK and the CAST into one step.

```
if (obj instanceof String s)    ← s is already a String, ready to use
```

Old vs new

```
// OLD – you say "String" three times  
if (obj instanceof String) {  
    String s = (String) obj;  
    System.out.println(s.length());  
}  
  
// NEW – Java 16 ✓  
if (obj instanceof String s) {  
    System.out.println(s.length());    // s is already cast  
}
```

```
instanceof String s  
    |  |  
    |  └─ the new variable, already cast ★  
    └── the type being tested
```

Real-world idea: a **security check at an airport**.

Old: guard verifies you're a passenger → then a SECOND person issues your pass

New: the guard verifies AND hands you the pass in one step ✓

Where it really shines — equals()

```
// OLD
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Employee)) return false;
    Employee other = (Employee) o;           // extra line
    return id == other.id;
}

// NEW
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    return o instanceof Employee other && id == other.id; // one line ✓
}
```

Scope rules — the clever part ★

The variable exists exactly where the check is guaranteed true:

```
if (obj instanceof String s) {
    s.length();           // ✓ available here
}
s.length();               // ✗ not available here

if (!(obj instanceof String s)) {
    return;               // s is NOT available here...
}
s.length();               // ✓ ...but IS available after, because we
                          // returned!
```

That last pattern is very useful for guard clauses.

You can also combine conditions:

```
if (obj instanceof String s && s.length() > 5) {    // ✓ s usable
    immediately
    System.out.println(s.toUpperCase());
}
```

Pattern matching for switch (preview in 17, final in 21)

```
String describe(Object obj) {
    return switch (obj) {
        case Integer i -> "Integer: " + i;
        case String s  -> "String of length " + s.length();
        case null      -> "It was null";           // null can be a case ★
        default        -> "Something else";
    };
}
```

With sealed types it becomes exhaustive:

```
String area(Shape shape) {
    return switch (shape) {
        case Circle c    -> "π r2 = " + (Math.PI * c.radius() *
c.radius());
        case Square s    -> "side2 = " + (s.side() * s.side());
        case Rectangle r -> "l × b";
    };
    // no default – the compiler knows the full
    list ✓
}
```

Java 21 adds **record patterns**, which destructure the object:

```
if (obj instanceof Employee(Long id, String name)) {    // Java 21
    System.out.println(name);                          // fields extracted directly
}
```

Interview-ready answer

Pattern matching for **instanceof**, finalised in Java 16, combines the type test and the cast into a single expression: **if (obj instanceof String s)** both checks the type and declares **s** already cast, which removes the redundant cast line. The binding variable's scope is limited to where the compiler can prove the check succeeded, so it also works after a negated check that returns early, and it can be combined with further conditions using **&&**. The most common practical

use is simplifying `equals()` implementations. This feature is the foundation for pattern matching in `switch`, which was a preview in Java 17 and finalised in Java 21, and which becomes exhaustive when used with sealed types.

Easy memory

`if (obj instanceof String s) → check + cast in ONE step ✓`

Old: `instanceof → then (String) obj → 3 mentions of String`

New: one line, ``s`` is ready to use

Scope: available only where the check is proven true
(also after ``if (!(o instanceof X x)) return;``)

`equals()`: `return o instanceof Employee e && id == e.id;` ★

Java 21: switch pattern matching + record patterns

Text Blocks (Java 15)

Q: What are text blocks?

The easiest way to remember:

Text block = a multi-line String written between triple quotes `"""`

No more `\n`, no more `+` concatenation, no more escaped quotes.

The old horror

```
String json = "{\n" +  
    "    \"name\": \"Priyanka\", \n" +  
    "    \"role\": \"Full Stack Developer\" \n" +  
    "}";
```

You cannot read that, and every `\"` is a chance to make a mistake.

The new way

```
String json = """"
{
    "name": "Priyanka",
    "role": "Full Stack Developer"
}
""";
```

It looks exactly like the JSON it produces. Quotes need no escaping at all.

SQL becomes readable too

```
String sql = """"
SELECT e.id, e.name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.id
WHERE e.salary > 50000
ORDER BY e.name
""";
```

Compare with the old version:

```
String sql = "SELECT e.id, e.name, d.dept_name " +
    "FROM employees e " +
    "JOIN departments d ON e.dept_id = d.id " +
    "WHERE e.salary > 50000 " +
    "ORDER BY e.name";
```

Notice the classic bug in the old style: forget one trailing space and you get

```
...e.nameFROM employees...
```

Real-world idea: **typing a letter in Notepad versus writing it as one long line with `\n` symbols.**

Indentation rules — the part people get wrong ★

Java removes the **common leading whitespace** automatically:

```
String s = """"
    Hello
    World
""";
```

The CLOSING `"""` decides the left margin.

```
Hello      ← 8 spaces
World      ← 8 spaces
"""        ← 8 spaces → strip 8 from every line
```

Result: `"Hello\nWorld\n"` (no leading spaces ✓)

Move the closing quotes left and you keep indentation:

```
String s = """
    Hello
    World
""";
// Result: "    Hello\n    World\n" ← 8 spaces kept
```

Two useful escapes

```
// \ → join lines (suppress the newline)
String s = """
    This is one \
    single line""";
// "This is one single line"

// \s → keep a trailing space (which would otherwise be stripped)
String t = """
    Name:\s
    """;
```

Where you'll use it in real work

- ✓ JSON test fixtures in unit tests
- ✓ Native SQL queries in `@Query` or `JdbcTemplate`
- ✓ HTML / XML templates
- ✓ Multi-line log or error messages

Interview-ready answer

Text blocks, finalised in Java 15, are multi-line string literals delimited by triple quotes. They remove the need for `\n` escapes and string concatenation, and double quotes inside them need no escaping, which makes embedded JSON, SQL, HTML and XML far more readable. Java automatically strips the common

leading indentation, and the position of the closing delimiter determines the left margin, so we can keep the code indented without that indentation ending up in the string. There are also two escapes specific to text blocks: a backslash at the end of a line joins it with the next, and `\s` preserves a trailing space. In practice I use them for native SQL queries and for JSON payloads in tests.

Easy memory

`"""` → multi-line String, no `\n`, no `+`, no escaped quotes ✓

```
String json = """
    { "name": "Priyanka" }
    """;
```

Indentation: the CLOSING `"""` sets the left margin ★

`\` at line end → join lines

`\s` → keep a trailing space

Use for: JSON in tests, native SQL, HTML

Other Notable Features (Java 9–17)

Q: What is `var` (local variable type inference, Java 10)?

The easiest way to remember:

```
var = "compiler, YOU work out the type."
```

Java is still statically typed – the type is fixed, just not typed out by hand.

```
var list = new ArrayList<String>();    // inferred: ArrayList<String>
var count = 10;                       // inferred: int
var name = "Priyanka";                // inferred: String
```

It shines when the type is long and repeated:

```
// Before
Map<String, List<Employee>> byDept = new HashMap<String,
List<Employee>>();

// After
var byDept = new HashMap<String, List<Employee>>();
```

The important trap ★ — `var` is NOT JavaScript's `var`

```
var x = 10;
x = "hello";           // ✗ compile error – x is permanently an int
```

JavaScript `var` → the TYPE can change at runtime (dynamic)
 Java `var` → the type is fixed at COMPILE time (static) ✓

Where it is NOT allowed

```
var x;                // ✗ no initialiser → nothing to infer
var y = null;         // ✗ null has no type
private var field = 10; // ✗ fields not allowed
void m(var param) { }  // ✗ parameters not allowed
var m() { return 10; } // ✗ return types not allowed
```

- ✓ Local variables WITH an initialiser
- ✓ for / for-each loop variables
- ✓ try-with-resources
- ✗ fields, parameters, return types, no-initialiser, null

When to use it (readability rule)

```
var employees = employeeService.findAll(); // ✓ clear from the name
var x = process();                         // ✗ what type is this?
```

Rule: use `var` when the right-hand side already makes the type obvious.

Interview-ready answer

`var`, introduced in Java 10, is local variable type inference — the compiler infers the type from the initialiser. Java remains statically and strongly typed; the type is fixed at compile time and simply not written out, so it is completely different from

JavaScript's `var`. It can only be used for local variables that have an initialiser, including loop variables and try-with-resources, and not for fields, method parameters, return types, or with `null`. I use it where the right-hand side already makes the type obvious, such as `var employees = service.findAll()`, and I avoid it where it hides useful information.

Easy memory

```
var = compiler infers the type → still STATIC, still fixed ✓  
var x = 10; then x = "hi"; → ✗ compile error
```

```
✓ local variables with an initialiser, loops, try-with-resources  
✗ fields, parameters, return types, var x; , var y = null;
```

Use it when the type is obvious from the right-hand side.

Q: What are the Java 9 collection factory methods?

The easiest way to remember:

```
List.of() / Set.of() / Map.of() → create an IMMUTABLE collection in one line.
```

```
List<String> list = List.of("a", "b", "c");  
Set<Integer> set = Set.of(1, 2, 3);  
Map<String, Integer> map = Map.of("x", 1, "y", 2);  
  
// more than 10 map entries:  
Map<String, Integer> big = Map.ofEntries(  
    Map.entry("a", 1),  
    Map.entry("b", 2));
```

The old way

```
List<String> list = new ArrayList<>();  
list.add("a"); list.add("b"); list.add("c");  
list = Collections.unmodifiableList(list);           // 4 lines
```

The traps ★

```
List<String> list = List.of("a", "b");

list.add("c");           // ▲ UnsupportedOperationException – truly
immutable               // ▲ NullPointerException – nulls are not
List.of("a", null);      // ▲
allowed
Set.of(1, 1);           // ▲ IllegalArgumentException – duplicates not
allowed
```

Also note:

```
Arrays.asList("a","b")   → fixed SIZE, but set() works, nulls allowed
List.of("a","b")         → fully IMMUTABLE, no nulls ★
```

That difference is a favourite follow-up question.

Easy memory

```
List.of() Set.of() Map.of() → one-line IMMUTABLE collections ✓
```

```
add()      → UnsupportedOperationException ▲
null       → NullPointerException ▲
duplicates in Set.of() → IllegalArgumentException ▲
```

```
Arrays.asList → fixed size but MUTABLE elements | List.of → fully
immutable
```

Q: What Stream and Optional improvements came after Java 8?

```
// STREAMS
Stream.takeWhile(p)           // Java 9 – take while true, then STOP
Stream.dropWhile(p)           // Java 9 – skip while true, then take the rest
Stream.ofNullable(x)          // Java 9 – empty stream if null
Stream.iterate(1, i -> i < 100, i -> i * 2) // Java 9 – with a
condition ★
stream.toList()               // Java 16 – shorter than
collect(Collectors.toList())

// OPTIONAL
optional.ifPresentOrElse(v -> use(v), () -> handleEmpty()); // Java 9 ★
optional.or(() -> Optional.of(fallback));                    // Java 9
optional.stream()                                                 // Java 9

// STRING
"hi ".strip()              // Java 11 – Unicode-aware trim()
"".isBlank()               // Java 11 – empty or only whitespace
"a\nb".lines()             // Java 11 – a Stream of lines
"ab".repeat(3)             // Java 11 – "ababab"

// COLLECTORS
Collectors.teeing(...)     // Java 12 – two collectors at once
```

takeWhile vs filter — the difference that gets asked ★

```
List.of(1, 2, 3, 10, 4, 5).stream()
    .filter(n -> n < 5)           // [1, 2, 3, 4] → checks EVERY element
    .toList();

List.of(1, 2, 3, 10, 4, 5).stream()
    .takeWhile(n -> n < 5)        // [1, 2, 3] → STOPS at 10 ★
    .toList();
```

filter → keeps every matching element, scans the whole stream
takeWhile → stops at the FIRST failure (needs sorted/ordered data)

Easy memory

```
Java 9 → takeWhile / dropWhile / ofNullable / ifPresentOrElse / or()  
Java 11 → strip() isBlank() lines() repeat()  
Java 16 → stream.toList() ★ (shortest form)
```

```
filter = check ALL | takeWhile = STOP at the first failure
```

Q: What are "helpful NullPointerExceptions" (Java 14)?

The easiest way to remember:

```
The JVM now names the EXACT variable that was null.
```

```
employee.getAddress().getCity().toUpperCase();
```

Before Java 14:

```
Exception in thread "main" java.lang.NullPointerException  
    at Main.main(Main.java:10)
```

Which one was null — the employee, the address, or the city? You had to guess or add print statements.

Java 14+:

```
java.lang.NullPointerException:  
  Cannot invoke "String.toUpperCase()" because the return value of  
  "Address.getCity()" is null ★
```

Now you know instantly: `getCity()` returned null.

Enabled by default since Java 15 (in 14 it needed `-XX:
+ShowCodeDetailsInExceptionMessages`).

Easy memory

Java 14+ NPE tells you WHICH variable was null ★

"Cannot invoke String.toUpperCase() because the return value of Address.getCity() is null"

Saves hours of debugging chained calls.

Q: What is the new HTTP Client (Java 11)?

The easiest way to remember:

A modern, built-in HTTP client – HTTP/2, async, WebSocket – replacing the painful old `URLConnection`.

```
HttpClient client = HttpClient.newHttpClient();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/employees"))
    .header("Content-Type", "application/json")
    .GET()
    .build();

// synchronous
HttpResponse<String> response =
    client.send(request, HttpResponse.BodyHandlers.ofString());
System.out.println(response.statusCode());
System.out.println(response.body());

// asynchronous ★
client.sendAsync(request, HttpResponse.BodyHandlers.ofString())
    .thenApply(HttpResponse::body)
    .thenAccept(System.out::println);
```

Old `URLConnection` → 30+ lines, manual streams, HTTP/1.1 only

New `HttpClient` → builder style, HTTP/2, async, WebSocket ✓

Note: in Spring Boot projects you'd normally still use `RestTemplate` or, preferably, `WebClient` — but knowing the built-in client exists is a good point.

Easy memory

```
Java 11 HttpClient → HTTP/2 + async (CompletableFuture) + WebSocket  
send()           → blocking  
sendAsync()      → non-blocking ★  
Replaces HttpURLConnection.
```

Q: What is the Java Module System / JPMS (Java 9)?

The easiest way to remember:

```
Modules = packages grouped together, declaring what they EXPORT  
          and what they REQUIRE.
```

```
// module-info.java  
module com.myapp.service {  
    requires com.myapp.data;           // what I need  
    exports com.myapp.service.api;    // what others may use  
}
```

```
Before: classpath → EVERY public class is visible to everyone  
After : module   → only EXPORTED packages are visible ✓
```

Real-world idea: an **office building**.

```
Classpath → all doors unlocked, anyone walks into any room  
Modules   → each department declares which rooms are open to visitors
```

Benefits: strong encapsulation, explicit dependencies, and **jlink** can build a small custom runtime containing only the modules you use (useful for containers).

Be honest in the interview: most business applications still run on the classpath. Knowing what it is and why it exists is enough.

Easy memory

```
module-info.java → requires (what I need) + exports (what I share)  
Benefit: real encapsulation + smaller runtimes via jlink  
Reality: most apps still use the classpath.
```

Q: Can you run a single-file Java program (Java 11)?

```
# Before
javac Hello.java
java Hello

# Java 11+
java Hello.java          # ✓ compiles in memory and runs
```

Great for scripts, quick tests and learning — no `.class` file is produced.

Likely Interview Scenarios

Q: What Java 17 features would you use in a Spring Boot microservice?

Answer it feature by feature, tied to real code:

```
Records          → DTOs, API request/response objects, value objects
Text blocks      → native SQL in @Query, JSON payloads in tests
Switch expressions → status → action mapping, cleanly and exhaustively
Sealed + pattern matching → a closed set of domain events or payment types
var              → concise local variables inside service methods
List.of()/Map.of() → immutable constants and test fixtures
Optional.ifPresentOrElse → cleaner service-layer null handling
```

Concrete example:

```
// DTO
public record EmployeeResponse(Long id, String name, String dept) {}

// Controller
@GetMapping("/{id}")
public EmployeeResponse get(@PathVariable Long id) {
    Employee e = service.findById(id);
    return new EmployeeResponse(e.getId(), e.getName(), e.getDept());
}

// Query with a text block
@Query(value = """
    SELECT * FROM employees
    WHERE dept = :dept AND salary > :min
    """, nativeQuery = true)
List<Employee> search(String dept, double min);

// Status mapping with a switch expression
String action = switch (order.status()) {
    case NEW      -> "Send confirmation email";
    case SHIPPED  -> "Send tracking link";
    case DELIVERED -> "Request a review";
    case CANCELLED -> "Process the refund";
};
```

● In your RoboGebra code

```
robogebra-portal → Java 17, Spring Boot 3.2.0
    26 files use `public record` (payment DTOs, exception payloads,
    service value objects)
    e.g. common/exception/QuizConflictException.java
        domain/payment/service/RazorPayService.java
    92 × @Valid with jakarta.validation-api 3.0.2 ← jakarta, NOT javax ★
```

■ *"We're on Java 17 and Spring Boot 3.2, so records are used for DTOs and small value objects — the payment service uses them for its request and response payloads. What we deliberately don't do is make a persistence model a record, because the Mongo mapper, like Hibernate, needs a no-arg constructor and mutable fields."*

That single sentence answers "**do you use records?**" and "**where would you not use one?**" at the same time. → [39 — RoboGebra Code Examples](#)

Q: How would you migrate a Java 8 project to Java 17?

Give it as a **sequence**, not a list of facts:

1. Update the JDK and set source/target (or `release`) to 17 in Maven/Gradle
2. Fix the REMOVED things ★
 - javax.* → jakarta.* (if moving to Spring Boot 3)
 - Nashorn JavaScript engine – removed
 - SecurityManager – deprecated
 - some internal sun.misc.* APIs are now blocked (strong encapsulation)
3. Upgrade dependencies
 - Spring Boot 3.x requires Java 17
 - Lombok, Hibernate, Mockito all need version bumps
4. Run the FULL test suite; fix warnings
5. THEN modernise gradually
 - records for DTOs, switch expressions, text blocks

★ The key sentence: "Migrate first, modernise second."
Get it compiling and green on 17 before rewriting any code style.

The most common real blocker is the `javax` → `jakarta` package rename, because it touches every entity, servlet and validation annotation.

Honesty tip for the interview

If you have mostly used Java 8 or 11, say this — it is far stronger than pretending:

"I've worked extensively with Java 8 features — streams, lambdas, `Optional` and functional interfaces — and I've been upskilling on Java 17. I'm comfortable with records, sealed classes, pattern matching for `instanceof`, switch expressions

and text blocks, and I understand where each one fits, for example records for DTOs but not for JPA entities."

That answer shows both real experience and current knowledge.

Quick Revision Sheet

LTS: 8 → 11 → 17 → 21 | Spring Boot 3 REQUIRES Java 17

RECORD (16) `record Employee(Long id, String name) {}`
→ final fields + ctor + name() + equals/hashCode/toString

FREE
→ accessor is name(), not getName()
→ ✓ DTO ✗ JPA entity

SEALED (17) `sealed interface Shape permits Circle, Square {}`
→ child must be final | sealed | non-sealed
→ payoff: EXHAUSTIVE switch with no default

SWITCH EXPR (14) `String s = switch (day) { case MON -> "Busy"; };`
→ returns a value, no break, no fall-through, `yield` in blocks

PATTERN MATCH (16) `if (obj instanceof String s)` → check + cast in one

TEXT BLOCK (15) `""" multi-line """` → closing `"""` sets the margin

var (10) local variables only, needs an initialiser, still STATIC typing

Java 9 `List.of() / Map.of()` (immutable, no nulls)
 `takeWhile / dropWhile / ifPresentOrElse`

Java 11 `HttpClient, strip(), isBlank(), lines(), repeat(), java`
`Hello.java`

Java 14 `helpful NullPointerExceptions` (names the null variable) ★

Java 16 `stream.toList()`

MIGRATION: migrate first, modernise second. Biggest blocker = `javax` → `jakarta`.

→ Next: Java 21

"And what's new in Java 21?" is now the standard follow-up to every Java 17 question. Java 21 is the **current LTS** most companies are migrating to.

Java 17 = less BOILERPLATE ★ (records, sealed, pattern matching)
Java 21 = CONCURRENCY ★★ (VIRTUAL THREADS) + pattern matching for switch and record patterns FINALISED + sequenced collections

→ [40 — Java 21 Features](#) ★

Related files

- [05-java.md](#) — Core Java (OOP, collections, streams, threads)
- [FLASHCARDS-12-java17.md](#) — quick recall cards for this file
- [06-spring-boot.md](#) — Spring Boot (uses records and text blocks heavily)
- [22-java-streams-coding-problems.md](#) — stream coding problems

MongoDB — Interview Q&A (Easy Version)

✓ POSITIONING FIX — read this first

An older version of this file said MongoDB was a *gap* on your resume. **That is wrong, and it was costing you marks.**

RoboGebra runs on MongoDB. Verified in your own workspace:

'''

robogebra-portal (Java 17 / Spring Boot 3.2)

104 × @Document entity classes

85 × @Repository MongoRepository interfaces

27 × @Aggregation pipelines

× @CompoundIndex compound indexes on hot query paths

× MongoTemplate in the persistence services

robogebra-crm (Node / Express)

Mongoose 8.8 — 10+ schemas with validators, plugins and hooks

'''

You have production MongoDB experience on your current project, in two languages. Say so with confidence. See [39 — RoboGebra Code Examples](#).

Every question follows the same shape — **Easiest way to remember** → **simple explanation** → **real-world example** → **diagram** → **interview trap** → **Easy memory box** — with ● blocks showing your real code.

Contents

1. [Fundamentals](#)
2. [CRUD Operations](#)
3. [Schema Design](#)
4. [Indexing & Performance](#)

- 5. [Aggregation](#)
 - 6. [Scaling & Reliability](#)
 - 7. [Spring Data MongoDB](#)
 - 8. [Mongoose \(Node\)](#)
 - 9. [Your Answer Scripts](#)
 - 10. [Quick Revision Sheet](#)
-

Fundamentals

Q: What is MongoDB?

The easiest way to remember:

MongoDB = a database that stores JSON-like DOCUMENTS instead of table rows. ★

SQL		MongoDB
database	→	database
table	→	COLLECTION
row	→	DOCUMENT ★
column	→	field
JOIN	→	embed, or \$lookup
primary key	→	_id

Real-world idea: a **filing cabinet vs a spreadsheet.**

SPREADSHEET (SQL) → every row must have the SAME columns.
Adding a column means changing every row.

FOLDER (MongoDB) → each folder holds whatever papers that case needs.
A new case can carry an extra document; the others don't care ★

Easy memory

Document = a row | Collection = a table | `_id` = the primary key
JSON-like (BSON), FLEXIBLE schema, scales HORIZONTALLY ★

Q: What is a document and a collection?

```
// one DOCUMENT in the "employees" collection
{
  "_id": ObjectId("64f..."),
  "name": "Priyanka",
  "role": "Full Stack Developer",
  "skills": ["Angular", "Java", "Spring Boot"],    // ★ an ARRAY inside
a row
  "address": { "city": "Chennai", "country": "India" } // ★ a NESTED
object
}
```

★ Look at what a single document does that a SQL row cannot:

"skills" is an ARRAY → in SQL that's a second table + a join
"address" is an OBJECT → in SQL that's a third table + another join
MongoDB reads all three in ONE trip ★

That is the whole value proposition: **model for how you READ.**

Easy memory

Document = a BSON object; can hold ARRAYS and NESTED objects ★
Collection = a group of documents, NO fixed schema
One document can replace three SQL tables + two joins ★

Q: What is BSON?

BSON = Binary JSON – MongoDB's storage format.
JSON + extra TYPES + optimised for speed and size. ★

JSON has: string, number, boolean, null, array, object
BSON adds: ObjectId ★, Date ★, Binary, int32/int64, Decimal128, Timestamp

★ Why the extra types matter:

JSON has ONE "number" type → you cannot tell an int from a double, and money in a float is a rounding bug waiting to happen ▲

BSON has Decimal128 for money, and a real Date type instead of a string ★

Easy memory

BSON = Binary JSON ★ → adds ObjectId, Date, Decimal128, Binary
Why: JSON has one number type; BSON can store money and dates properly ★

Q: What is `_id`?

`_id` = the automatic PRIMARY KEY on every document.
By default a 12-byte ObjectId. ★

```
ObjectId("64f3a1b2c9e77a0012ab34cd")
  _____|_____|_____|_____|
    4 bytes  5 bytes 3 bytes
  TIMESTAMP ★ random counter
```

★ Two useful consequences:

1. An ObjectId CONTAINS its creation time
→ sorting by `_id` ≈ sorting by creation date, for free ★
→ you can extract the date: `ObjectId.getTimestamp()`
2. It's generated CLIENT-side, so inserts don't need a round trip to ask the database for an id ★
(compare with SQL IDENTITY, which blocks batch inserts – [24](./24-orm-jpa-hibernate.md))

Easy memory

`_id` = the automatic primary key, a 12-byte ObjectId ★
It EMBEDS a timestamp → sort by `_id` ≈ sort by `createdAt` ★
Generated client-side → no round trip (unlike SQL auto-increment) ★
You can supply your own `_id`.

Q: SQL vs NoSQL (MySQL vs MongoDB)?

	MySQL (relational)	MongoDB (document)
Data model	tables, rows, columns	collections, documents
Schema	fixed, predefined	flexible, dynamic ★
Relationships	joins, foreign keys	embed or reference ★
Scaling	vertical (bigger server)	horizontal (sharding) ★
Transactions	strong ACID	ACID per document; multi-doc since v4.0 ★
Query language	SQL	JSON-shaped query language
Best for	structured, relational, reporting	flexible, nested, evolving, high write volume

The honest way to frame the choice:

Ask: "does the read pattern have a natural DOCUMENT boundary?"

An invoice with its line items → ONE document ✓ read in one trip
A social graph / heavy reporting → SQL joins or a graph DB ✓

▲ **The mistake to avoid in the answer:** saying "MongoDB is faster." It isn't inherently faster — it's faster *for reads that match your document shape*, and slower for anything needing joins across collections.

Easy memory

SQL → fixed schema, JOINS, vertical scaling, reporting
Mongo → flexible schema, EMBED, horizontal SHARDING ★

Choose by the READ PATTERN, not by fashion ★

▲ Never say "MongoDB is faster" – say "faster for reads shaped like the document"

Q: What is ACID vs BASE?

ACID (relational) → Atomicity, Consistency, Isolation, Durability → strong guarantees

BASE (many NoSQL) → Basically Available, Soft state, Eventual consistency
→ favours availability and scale ★

★ Where MongoDB actually sits — be precise, this gets probed:

SINGLE-document operations → ALWAYS atomic ✓ (always were)

MULTI-document transactions → supported since v4.0 ★
BUT they require a REPLICASET ▲

● In your RoboGebra code

robogebra-portal uses @Transactional in 130 places –
on MongoDB, which means the deployment MUST be a replica set ★

■ "*@Transactional on MongoDB isn't the free lunch it is on a relational database — multi-document transactions need a replica set, and they carry real overhead. Most of our write paths are single-document, which Mongo makes atomic by itself, so we only reach for a multi-document transaction where an operation genuinely spans collections.*"

★ That answer proves you've used it rather than read about it.

Easy memory

ACID = strong guarantees | BASE = availability + eventual consistency
MongoDB: single-document ALWAYS atomic ✓
multi-document transactions since v4.0 – need a REPLICASET ▲ ★
Good schema design usually REMOVES the need for a transaction ★

CRUD Operations

Q: Basic CRUD in the shell?

```
// CREATE
db.employees.insertOne({ name: "Priyanka", role: "Developer", exp: 4 });
db.employees.insertMany([ { name: "A" }, { name: "B" } ]);

// READ
db.employees.find({ role: "Developer" });
db.employees.findOne({ name: "Priyanka" });
db.employees.find({ exp: { $gte: 4 } }); // exp >= 4
db.employees.find({ role: "Developer" }, { name: 1, _id: 0 }); // ★
PROJECTION

// UPDATE
db.employees.updateOne({ name: "Priyanka" }, { $set: { exp: 5 } });
db.employees.updateMany({ role: "Developer" }, { $inc: { exp: 1 } });

// DELETE
db.employees.deleteOne({ name: "B" });
db.employees.deleteMany({ exp: { $lt: 2 } });
```

▲ The trap that destroys data — worth volunteering:

```
db.employees.updateOne({ name: "Priyanka" }, { exp: 5 }); // ✗
NO $set
```

Without \$set, MongoDB REPLACES the entire document.
→ name, role and every other field are GONE ▲
→ the document is now just { _id, exp: 5 }

Always use an update operator.

Easy memory

```
insertOne/Many | find/findOne | updateOne/Many | deleteOne/Many  
Projection: find(filter, { name: 1, _id: 0 }) ★
```

★ updateOne without \$set REPLACES the whole document ▲ ALWAYS use \$set
★

Q: Common query operators?

```
COMPARISON  $eq $ne $gt $gte $lt $lte $in $nin  
LOGICAL     $and $or $not $nor  
ELEMENT     $exists $type  
ARRAY       $all $elemMatch ★ $size  
UPDATE      $set ★ $unset $inc $push $pull $addToSet ★
```

★ The two array operators worth knowing properly:

```
// $elemMatch – ALL conditions must match the SAME array element ★  
db.orders.find({ items: { $elemMatch: { product: "Pen", qty: { $gt:  
5 } } } });  
// without $elemMatch, one element could match "Pen" and a DIFFERENT one  
match qty > 5 ▲  
  
// $addToSet vs $push  
{ $push:      { skills: "Java" } } // adds, even if it's already there  
{ $addToSet: { skills: "Java" } } // adds ONLY if absent ★ (a set)
```

Easy memory

```
$gte $in $exists | $and $or | $set ★ $inc $push $pull  
$elemMatch ★ = all conditions on the SAME array element  
$addToSet ★ = push only if it isn't already there
```

Schema Design

Q: Embedding vs referencing — how do you model relationships?

★ **This is the single most important MongoDB design question.**

The easiest way to remember:

EMBED → the data is read TOGETHER and is BOUNDED → "contains"
REFERENCE → the data is SHARED, LARGE, or grows forever → "relates to"

```
// EMBEDDING — one read, no join ★  
{ "name": "Order1",  
  "items": [ {"product":"Pen","qty":2}, {"product":"Book","qty":1} ] }  
  
// REFERENCING — a second lookup needed  
{ "name": "Order1", "customerId": ObjectId("...") }
```

EMBED

1 query ▲
data duplicated
update in many places ▲
▲ 16 MB document limit

REFERENCE

2 queries (or a \$lookup)
data stored once ★
update in ONE place ✓
no size limit

The decision rule to state

ONE-TO-FEW (an order's items, an address)	→ EMBED ★
ONE-TO-MANY (a user's 10,000 orders)	→ REFERENCE ★
MANY-TO-MANY (students ↔ courses)	→ REFERENCE
Changes independently / shared	→ REFERENCE
Read together, always	→ EMBED

Real-world idea:

EMBED → a passport: photo, name and visa pages are all IN the book
REFERENCE → a library card: it points to books it doesn't contain

▲ **The 16 MB limit is the killer constraint:**

Embedding an UNBOUNDED array (comments, logs, events)

- the document grows forever
- at 16 MB every write FAILS ▲
- and long before that, every read drags the whole array over the wire

● In your RoboGebra code — you use `@DBRef` (referencing) ★

File: `domain/exercisesolution/repository/model/`

`ExerciseSolutionEntity.java`

```
@Document(collection = "exercise_solution")
@CompoundIndexes({
    @CompoundIndex(name = "exercise_item_id_index", def =
    "{ 'exercise_item.$id': 1 }")
})
public class ExerciseSolutionEntity {
    @Id private String id;

    @Indexed
    @DBRef // ★ a REFERENCE, not embedded
    @Field("exercise_item")
    private ExerciseItemEntity exerciseItem;

    @Field("step_output_json") // ★ a large AI payload,
    embedded as-is
    private Object stepOutputJson;
    @Field("animated_step_output_json")
    private Object animatedStepOutputJson;
}
```

■ "Solutions reference their exercise item with `@DBRef`, because the same item is shared and changes independently — embedding it would duplicate it across every solution. But the AI-generated step output is embedded as a nested object, because it's meaningless outside its solution and is always read with it. That's the embed-versus-reference decision made per field rather than per entity."

★ **That is a genuinely senior answer** — most candidates treat it as an all-or-nothing choice.

▲ And the `@DBRef` caveat to volunteer:

@DBRef is convenient, but Spring Data resolves it with an EXTRA QUERY per reference – which is exactly the N+1 problem again.

↓

Which is why we also have 27 @Aggregation pipelines that fetch in one trip ★

Easy memory

EMBED ★ → read together + BOUNDED → 1 query (a passport)
REFERENCE ★ → shared / large / unbounded → 2 queries (a library card)

▲ 16 MB document limit → NEVER embed an unbounded array ▲

▲ @DBRef costs an extra query per reference → N+1 ★

Decide PER FIELD, not per entity ★ (your ExerciseSolutionEntity does both)

Q: Does schema-less mean no design?

NO. It means the DATABASE doesn't enforce the schema – YOU still design it. ★

And you design it around your QUERY PATTERNS, not around normalisation. ★

SQL design: "what is the correct normalised model?" → then write queries

Mongo design: "what queries do I need?" → then shape the document ★

▲ Flexible schema is a **migration** tool, not a licence:

Same collection, three different shapes because nobody agreed on one
→ every read needs defensive checks
→ "schema-less" becomes "schema in the application code, undocumented" ▲

★ In practice you enforce a schema *somewhere*: @Document classes in Spring, a Mongoose schema in Node, or MongoDB's own JSON-schema validation.

Easy memory

Schema-less ≠ design-less ★

Model for how you READ, not for normalisation ★

Enforce the schema in the app: @Document / Mongoose schema / \$jsonSchema validation

Indexing & Performance

Q: What are indexes in MongoDB?

An index = a sorted B-tree that lets Mongo JUMP to matching documents instead of scanning the whole collection. ★

```
db.employees.createIndex({ name: 1 });           // 1 = ascending
db.employees.createIndex({ role: 1, exp: -1 });  // COMPOUND ★
db.employees.createIndex({ email: 1 }, { unique: true });
```

WITHOUT an index → COLLSCAN → read all 1,000,000 documents

WITH an index → IXSCAN → jump straight to the matches ▲

Real-world idea: the **index at the back of a book**. Without it, you read every page.

▲ Indexes are not free:

- ✓ Reads get much faster
- ✗ Every WRITE must also update every index
- ✗ Indexes consume RAM – and Mongo wants them in memory ★

★ The compound-index rule that gets asked — the ESR / prefix rule:

An index on { role: 1, exp: -1 } can serve:

- find({ role }) ✓ (a PREFIX of the index)
- find({ role, exp }) ✓
- find({ exp }) ✗ NOT the prefix → no index used ▲

Order the fields: Equality → Sort → Range ("ESR") ★

● In your RoboGebra code — real compound indexes ★

```
@CompoundIndexes({
  @CompoundIndex(name = "exercise_item_id_index", def =
    "{ 'exercise_item.$id': 1 }")
})
// and in SelfQuizAttemptEntity:
@CompoundIndex(name = "user_exercise_index", def = "{ 'user.$id': 1,
  'exercise.$id': 1 }")
```

■ "The quiz-attempt collection has a compound index on **user** plus **exercise**, because the hot query is always 'this user's attempts at this exercise'. Field order matters — with user first, the same index also serves 'all attempts by this user', but it would not serve a query on exercise alone."

★ That last clause proves you understand the **prefix rule** rather than just having added an index.

Easy memory

Index = a B-tree → IXSCAN ▲ instead of COLLSCAN (a book's index)
createIndex({ a: 1, b: -1 }) – COMPOUND

- ▲ PREFIX RULE ★ {a,b} serves find(a) and find(a,b) – but NOT find(b) ▲
- ▲ Order: Equality → Sort → Range (ESR) ★
- ▲ Every index slows WRITES and consumes RAM

Q: Types of indexes?

SINGLE FIELD	{ name: 1 }	
COMPOUND ★	{ role: 1, exp: -1 }	→ the prefix rule applies
MULTIKEY	automatic on ARRAY fields ★	(one index entry per element)
TEXT	full-text search	
GEOSPATIAL	2dsphere – "near me"	
HASHED	for sharding	
TTL ★	auto-DELETE documents after n seconds	
PARTIAL	index only documents matching a filter	

★ **TTL is the one worth volunteering** — it does something SQL can't do natively:

```
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 });  
// → MongoDB DELETES the document itself, one hour later ★
```

Perfect for sessions, OTPs, password-reset tokens and cache entries — no cleanup job needed.

Easy memory

```
single | COMPOUND ★ | multikey (arrays, automatic) | text | geo | hashed  
TTL ★ = auto-delete after n seconds → sessions, OTPs, tokens (no cron  
job!)  
partial = index only some documents
```

Q: How do you analyse query performance?

```
db.employees.find({ name: "Priyanka" }).explain("executionStats");
```

What to look for:

COLLSCAN ✗ full collection scan → add an index
IXSCAN ✓ an index was used ★

totalDocsExamined vs nReturned

examined 1,000,000 → returned 10 = a terrible query ▲
examined 10 → returned 10 = perfect ★

★ **totalDocsExamined $\approx$ nReturned is the target** — that ratio is the single most useful number, and quoting it sounds like experience.

Also:

```
db.collection.getIndexes() // what indexes exist  
db.setProfilingLevel(1, 100) // log every query slower than 100 ms  
★
```

Easy memory

```
.explain("executionStats") ★  
COLLSCAN ✗ → IXSCAN ✓  
Target: totalDocsExamined ≈ nReturned ★  
Profiler: db.setProfilingLevel(1, 100) → log slow queries
```

Aggregation

Q: What is the aggregation pipeline?

A pipeline = documents flowing through STAGES, like a factory line. ★
Each stage transforms the output of the previous one.

```
db.orders.aggregate([  
  { $match: { status: "completed" } },           // WHERE  
  ★ put this FIRST  
  { $group: { _id: "$customerId", total: { $sum: "$amount" } } }, //  
  GROUP BY  
  { $sort: { total: -1 } },                       // ORDER BY  
  { $limit: 10 }  
]);
```

```
[all orders]  
  ↓ $match ← filter EARLY so later stages handle less ★  
[completed only]  
  ↓ $group  
[one row per customer, with a total]  
  ↓ $sort → $limit  
[the top 10] ✓
```

SQL

WHERE →
GROUP BY →
HAVING →
ORDER BY →
LIMIT →
SELECT →
JOIN →

Aggregation

\$match ★
\$group
\$match AFTER \$group ★
\$sort
\$limit
\$project
\$lookup

★ **The performance rule:** `$match` and `$limit` as early as possible, because `$match` can use an index — but only while it's still at the front of the pipeline.

The stages to know

```
$match ★ $group $project $sort $limit $skip  
$unwind ★ (one document per array element)  
$lookup ★ (a left outer join)  
$count $addFields $facet
```

★ `$unwind` is the one people can't explain:

```
{ name: "Priya", skills: ["Java", "Angular"] }  
  ↓ $unwind: "$skills"  
{ name: "Priya", skills: "Java" }  
{ name: "Priya", skills: "Angular" }      // ★ ONE document became TWO
```

Use it when you need to group or filter *by an array element*.

● In your RoboGebra code — 27 real pipelines ★

File: `domain/exerciseitem/repository/ExerciseItemRepository.java`

```
@Aggregation(pipeline = {  
    "{ '$match': { 'exercise.$id': { '$in': ?0 } } }",      // ★ ?0 =  
    the first parameter  
    "{ '$sort': { 'displayOrder': 1 } }"  
})  
List<ExerciseItemEntity> findAllByExerciseIds(List<ObjectId> exerciseIds);
```

■ **The N+1 story — your best MongoDB answer:** *"Loading a chapter's exercises and then fetching each exercise's items one at a time is one query plus N — the*

classic N+1, and `@DBRef` makes it easy to fall into. I replaced it with a single `@Aggregation` pipeline that `$match` es on `exercise.$id` with `$in` and sorts by `displayOrder`, so it's one round trip instead of N. It's the same fix as a JPA `JOIN FETCH`, just written as a pipeline."

★ Note `?0` — that's how Spring Data binds method parameters into the pipeline.

Easy memory

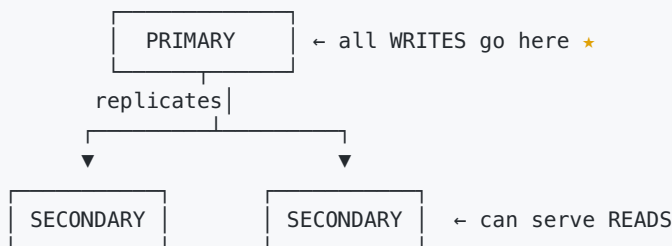
Pipeline = a factory line – each STAGE transforms the previous output
`$match` ★ (WHERE) → `$group` (GROUP BY) → `$sort` → `$limit`
`$unwind` ★ = one document PER array element | `$lookup` = a left join

▲ `$match` FIRST – only then can it use an index ★
Your N+1 fix: `$match` + `$in` in ONE pipeline instead of N queries ★

Scaling & Reliability

Q: What is replication / a replica set?

Replica set = several servers holding the SAME data.
ONE primary (writes) + secondaries (copies). ★



Primary dies ▲ → an ELECTION promotes a secondary → automatic failover
✓ ★

Real-world idea: a **manager and two deputies**. If the manager is unavailable, the team votes one deputy in and work continues.

★ The point that connects back to transactions:

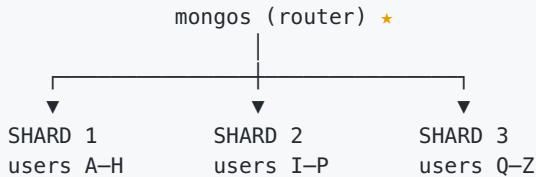
Multi-document transactions REQUIRE a replica set ★
→ which is why even a single-node dev setup is often run as a one-member replica set

Easy memory

Replica set = SAME data, many servers → PRIMARY (writes) + SECONDARIES ★
Primary fails → ELECTION → automatic failover ✓
Purpose: HIGH AVAILABILITY (and read scaling)
★ Required for multi-document transactions

Q: What is sharding?

Sharding = SPLITTING the data across servers by a SHARD KEY. ★
Horizontal scaling for size and write throughput.



★ **Everything depends on the shard key**, and picking it badly is unfixable-in-place:

A BAD shard key (e.g. a timestamp)
→ every new write goes to the SAME shard
→ a "hot shard" doing all the work while the others idle ▲

A GOOD shard key → high cardinality + even distribution + matches your queries ★

Replication vs sharding — the one-liner

REPLICATION → COPIES the same data → availability ★
SHARDING → SPLITS different data → capacity + write throughput ★
(production runs BOTH: each shard is itself a replica set)

Easy memory

Replication = COPIES (availability) ★ | Sharding = SPLITS (scale) ★
mongos routes the queries
▲ A bad shard key → a HOT SHARD doing all the work ▲
Production: each shard is itself a replica set

Spring Data MongoDB

Q: How do you use MongoDB with Spring Boot?

```
spring.data.mongodb.uri=mongodb://localhost:27017/robogebra
```

```
@Document(collection = "exercise_solution")    // ★ the collection name
@CompoundIndexes({
    @CompoundIndex(name = "exercise_item_id_index", def =
    "{ 'exercise_item.$id': 1 }")
})
public class ExerciseSolutionEntity {

    @Id
    private String id;                        // ★ String, not Long –
    it's an ObjectId

    @Indexed
    @DBRef                                    // ★ a reference to
    another document
    @Field("exercise_item")                  //
    ★ maps to a snake_case field name
    private ExerciseItemEntity exerciseItem;

    @Field("step_output_json")
    private Object stepOutputJson;
}
```

```

@Document    → class → COLLECTION
@Id          → the _id field (use String)
@Field       → rename the stored field ★ (Java camelCase ↔ Mongo snake_case)
@Indexed     → a single-field index
@CompoundIndex → a multi-field index ★
@DBRef       → a reference to another document ▲ (costs an extra query)

```

★ The Java-side difference from JPA worth naming:

```

JPA    → @Entity needs a no-arg constructor, and there IS a persistence context
Mongo → @Document maps a document, and there is NO persistence context ★
       → NO dirty checking → you MUST call save() explicitly ★

```

Easy memory

```

@Document(collection) | @Id (String) | @Field ★ | @Indexed |
@CompoundIndex | @DBRef ▲

```

★ NO persistence context, NO dirty checking → you MUST call save() ★
 (unlike JPA, where a managed entity saves itself – [24](./24-orm-jpa-hibernate.md))

Q: What is **MongoRepository** ?

```

@Repository
public interface ExerciseItemRepository extends
MongoRepository<ExerciseItemEntity, String> {

    // DERIVED queries – Spring writes them from the METHOD NAME ★
    Optional<ExerciseItemEntity> findByExercise_IdAndIndex(String
exerciseId, String index);
    List<ExerciseItemEntity> findAllByExerciseId(String exerciseId);
    Boolean existsByExercise_IdAndIndex(String exerciseId, String index);
    void deleteAllByExercise_Id(String exerciseId);
}

```

★ Note **findByExercise_Id** — the **underscore** traverses into a nested/referenced object.

Exactly the same programming model as JpaRepository ★
→ declare an interface, Spring generates the implementation

● In your RoboGebra code

85 repository interfaces extending MongoRepository ★

■ *"The repository model is identical to Spring Data JPA — that's the point of Spring Data. Which is also why I can move between them: the derived-query syntax, paging and sorting are the same, and what changes is the engine underneath — no persistence context, no lazy loading, no dirty checking."*

★ That sentence answers *both* "do you know Mongo?" and "could you pick up JPA?" at once.

Easy memory

MongoRepository<T, String> → CRUD + derived queries, like JpaRepository ★
findByExercise_Id → the UNDERSCORE traverses into a nested object ★
Same programming model as JPA; a completely different engine ★

Q: **MongoRepository** vs **MongoTemplate** ?

MongoRepository → HIGH level → derived queries → 90% of the work ★
MongoTemplate → LOW level → dynamic criteria, bulk ops, complex aggregation ★

```
// MongoTemplate – when the query is built at RUNTIME ★
Query query = new
Query(Criteria.where("role").is("Developer").and("exp").gte(4));
List<Employee> list = mongoTemplate.find(query, Employee.class);

// dynamic filters – impossible with a derived method name
if (city != null) query.addCriteria(Criteria.where("city").is(city));
```

★ The rule: **a method name can't be dynamic**. The moment filters depend on which parameters the user supplied, you need **MongoTemplate** (or Criteria/ **Query** by hand).

● In your RoboGebra code

`MongoTemplate` appears in the persistence services —

`ExerciseSolutionPersistenceService`, `LearningTimelineService`,
`SolutionPipelinePersistenceService`.

■ *"Repositories handle the standard access; the persistence services drop to `MongoTemplate` for the cases a derived method name can't express — dynamic filters, partial updates, and bulk operations."*

Easy memory

`MongoRepository` ★ → derived queries → the default
`MongoTemplate` ★ → dynamic Criteria, bulk ops, complex aggregation

Rule: a method NAME can't be dynamic → user-driven filters need
`MongoTemplate` ★

Mongoose (Node)

Q: What is Mongoose and how does it differ from Spring Data MongoDB?

Mongoose = the ODM for Node.
Same idea as Spring Data MongoDB, in JavaScript. ★

● In your RoboGebra CRM — real code ★

File: `robogebra-crm/src/models/user.model.js` (Mongoose 8.8, Express 4.21)

```
const userSchema = mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: {
    type: String, required: true, unique: true, trim: true, lowercase:
true,
    validate(value) {
      // ★ SCHEMA-
      LEVEL validation
      if (!validator.isEmail(value)) throw new Error('Invalid email');
    },
  },
  password: { type: String, required: true, minlength: 6 },
  role: { type: String, enum: roles, default: 'admin' }, // ★ enum +
  default
  isEmailVerified: { type: Boolean, default: false },
});
```

★ Three Mongoose features visible in that one file, and each is a talking point:

1. SCHEMA VALIDATION in the model – required, unique, minlength, a custom validator ★
→ MongoDB itself doesn't enforce this; Mongoose does, in the app layer
2. PLUGINS – `paginate` is imported from `./plugins` → reusable schema behaviour ★
3. bcryptjs – password hashing, almost certainly in a pre('save') HOOK ★

Mongoose HOOKS (middleware):

- pre('save') → hash the password before it's written ★
- post('save') → send a welcome email

Your other CRM models: `institute`, `transaction`, `coupon`, `affiliate`, `payout`, `webhook`, `token`, `feedback`, `counter`.

★ `counter.model.js` is worth calling out — it's the classic **auto-increment sequence** pattern, because MongoDB has no **AUTO_INCREMENT**:

Need a human-readable invoice number (INV-1001)?

- a `counters` collection + findOneAndUpdate with \$inc, atomically ★

Spring Data MongoDB vs Mongoose

	Spring Data MongoDB	Mongoose
Language	Java	JavaScript
Model	<code>@Document</code> class	<code>mongoose.Schema</code>
Validation	Bean Validation (<code>@Valid</code>)	in the schema ★
Repository	<code>MongoRepository</code> interface	<code>Model.find()</code>
Hooks	lifecycle events	<code>pre / post</code> middleware ★
Population	<code>@DBRef</code>	<code>.populate()</code> ★

Easy memory

Mongoose = the Node ODM ★ – schema, validators, plugins, pre/post HOOKS
Your CRM: 10+ schemas, bcrypt password hashing, a paginate plugin
`counter.model.js` ★ = the auto-increment pattern (`$inc`), since Mongo has none

`@DBRef` (Spring) ≈ `.populate()` (Mongoose) ★

Your Answer Scripts

■ "Have you worked with MongoDB?" (30 seconds) ★

"Yes — it's the database on my current project. RoboGebra's Spring Boot backend uses Spring Data MongoDB: around a hundred `@Document` classes, eighty-five repositories, and twenty-seven `@Aggregation` pipelines for the queries derived methods can't express. I've also worked with Mongoose on the Node CRM service, so I've used the document model from both Java and JavaScript."

★ Confident, specific, and every number is verifiable.

■ "What's the hardest MongoDB problem you've solved?" (45 seconds)



"An N+1 query. We load a chapter's exercises and then each exercise's items — and with `@DBRef`, Spring Data resolves each reference with its own query, so a chapter with fifty exercises meant fifty-one round trips. I replaced it with a single `@Aggregation` pipeline that `$match` es on `exercise.$id` with `$in` and sorts by `displayOrder` — one query instead of N. The lesson I took from it is that the document model doesn't remove the N+1 problem, it just changes what it looks like."

■ "How do you decide embed vs reference?" (30 seconds) ★

"By the read pattern and whether the data is bounded. Read together and bounded — embed; shared, large or unbounded — reference. In our solution entity we do both: the exercise item is a `@DBRef` because it's shared across solutions and changes independently, while the AI-generated step output is embedded, because it's meaningless outside its solution and always read with it. The hard constraint is the 16 MB document limit, so an unbounded array is never embedded."

■ If they ask about relational databases too

"I've worked relationally as well — MySQL and Postgres, and Neo4j on EasyVisa. Having used a document store, a relational store and a graph store makes the trade-offs concrete rather than theoretical: I pick by the access pattern, not by preference."

Quick Revision Sheet

BASICS

Document = row | Collection = table | `_id` = ObjectId (EMBEDS a timestamp ★)
BSON = JSON + ObjectId/Date/Decimal128 ★
Single-doc writes are ALWAYS atomic; multi-doc transactions need a REPLICASET ★

CRUD

`insertOne` | `find(filter, projection)` | `updateOne($set)` | `deleteOne`
▲ `updateOne` WITHOUT `$set` REPLACES the whole document ▲
`$elemMatch` ★ = same array element | `$addToSet` ★ = push if absent

SCHEMA DESIGN ★ (the most important question)

EMBED → read together + BOUNDED → 1 query
REFERENCE → shared / large / unbounded → 2 queries
▲ 16 MB limit → never embed an unbounded array ▲
▲ @DBRef = an extra query per reference → N+1 ★
Decide PER FIELD (your `ExerciseSolutionEntity` does both) ★
Model for how you READ ★

INDEXES

COLLSCAN ✗ → IXSCAN ✓ | `.explain("executionStats")` ★
Target: `totalDocsExamined` ≈ `nReturned` ★
▲ PREFIX RULE: {a,b} serves `find(a)` and `find(a,b)`, NOT `find(b)` ▲
ESR: Equality → Sort → Range ★
TTL index ★ = auto-delete (sessions, OTPs) – no cron job

AGGREGATION

`$match` ★ FIRST (so it can use an index) → `$group` → `$sort` → `$limit`
`$unwind` ★ = one doc per array element | `$lookup` = left join
Your 27 pipelines; ?0 binds a method parameter ★

SCALING

Replication = COPIES → availability ★ (primary + election)
Sharding = SPLITS → capacity ★ | ▲ a bad shard key = a HOT SHARD ▲

SPRING DATA

@Document | @Id (String) | @Field ★ | @Indexed | @CompoundIndex | @DBRef ▲
★ NO persistence context, NO dirty checking → you MUST call `save()`
MongoRepository (derived) ★ vs MongoTemplate (dynamic Criteria) ★

MONGOOSE

Schema validators + plugins + pre/post HOOKS ★ | `.populate()` ≈ @DBRef
`counter.model.js` = the auto-increment pattern (`$inc`) ★

YOUR POSITION ★

"MongoDB is the database on my current project – 104 @Document classes, 85 repositories, 27 aggregation pipelines, plus Mongoose on the Node CRM."

Related files: [24 — ORM/JPA/Hibernate \(ORM vs ODM\)](#) · [06 — Spring Boot](#) · [36 — SQL](#) · [39 — RoboGebra Code Examples](#) · [FLASHCARDS-13-mongodb.md](#)

Berribot Assessment — Master Prep Guide

Target role: Senior Full Stack Developer (5 years)
Stack: Java 17 · Spring Boot · Angular 16 · MongoDB · MySQL · Microservices · AWS S3

This guide maps the assessment's stack to your study files, flags gaps, and gives you a focused plan + key talking points.

▲ **Status note (3 Sept 2026).** This guide was written for the **Berribot** assessment in July. Two things have changed since, and they change the answers:
...
✗ "MongoDB is a gap" → ✓ It is your **CURRENT** production database ★
RoboGebra: 104 @Document · 85 repositories · 27 @Aggregation pipelines plus Mongoose on the Node CRM. Stop treating it as a weakness.
✗ "Java 17 is a gap" → ✓ The portal IS Java 17 / Spring Boot 3.2 ★
26 files use **record** . You ship on it daily.
...
The stack in this guide still maps well to most Java+Angular roles — keep using it for the full-stack design section. But read the gap table below with the correction above. See **13 MongoDB** and **39 RoboGebra Code Examples**.

1. Stack → Study-file map

Stack item	Status vs your resume	Study file(s)
Java 17	You list "Java" — brush up version features	12-java17-features.md + 05-java.md
Spring Boot	✓ On resume	06-spring-boot.md
Angular 16	✓ Strong — you migrated to Angular 16	04-angular.md + Angular 16 section below
MongoDB	▲ Gap — not on resume	13-mongodb.md

Stack item	Status vs your resume	Study file(s)
MySQL	✓ On resume	06-spring-boot.md (JPA)
Microservices	▲ Basic	08-microservices-basics.md
AWS S3	▲ Basic	07-aws-basics.md
Self-intro / HR	—	00, 11
Coding round	—	09-coding-problems.md
Projects	✓	10-projects-deep-dive.md

2. Honest gap analysis (prioritize these)

1. **MongoDB** — biggest gap. Learn the document model, indexing, aggregation pipeline, and Spring Data MongoDB (`@Document` , `MongoRepository` , `MongoTemplate`). Your **Neo4j** experience is your bridge — you already think NoSQL.
2. **Java 17 features** — you've used Java 8; learn records, sealed classes, pattern matching, switch expressions, text blocks.
3. **Microservices depth** — you have the basics; be ready for service discovery, API gateway, circuit breaker, inter-service communication.
4. **AWS S3 hands-on** — know buckets, the Spring Boot SDK upload/download flow, presigned URLs (example below).

Everything else (Angular, Spring Boot, MySQL/JPA, REST) is your strength — lead with it.

3. Likely assessment format (typical for this profile)

Berribot/product-company assessments for a senior full-stack role usually run in these stages — prepare for each:

1. **Online coding test** (HackerRank/Codility style) — DSA + maybe a small Java/JS task. → [09-coding-problems.md](#)
 2. **Technical Round 1 (Backend)** — Java 17, Spring Boot, REST, JPA/MongoDB, design of an API.
 3. **Technical Round 2 (Frontend)** — Angular 16, RxJS, state management, component design.
 4. **System/Full-stack design** — design a feature end-to-end (Angular → Spring Boot REST → MongoDB/MySQL → S3), microservices split, scaling.
 5. **Managerial / project deep-dive** — your real projects, decisions, trade-offs. → [10](#)
 6. **HR** — fit, expectations, notice period. → [11](#)
-

4. Angular 16 — version-specific talking points

Since they pin **Angular 16** (which you migrated to), be ready for:

- **Standalone components** — no NgModule needed; `standalone: true`, import dependencies directly. The future direction of Angular.
- **Angular Signals (developer preview in 16)** — a new reactive primitive for fine-grained change detection: `signal()`, `computed()`, `effect()`. Know it exists and the concept (reactivity without zone.js overhead).
- **Required inputs** — `@Input({ required: true })`.
- **Router data as input bindings** (`withComponentInputBinding`).
- `takeUntilDestroyed()` and `DestroyRef` — cleaner unsubscription.
- **Self-closing tags, esbuild-based dev server** (faster builds), Vite.
- **NgOptimizedImage** for image performance.

Talking point: "I led/contributed to migrating four repos to Angular 16. I dealt with breaking changes, RxJS upgrades, Ivy, and deprecated APIs, and I'm familiar with the new standalone-component direction and the signals preview."

5. Full-stack scenario you should be able to whiteboard

"**Design a file-upload + document feature**" (ties your EasyVisa document-portal experience to their stack):

```
[ Angular 16 client ]
- reactive form, file picker
- calls REST API; shows progress
  |
  v (HTTP, JWT auth)
[ Spring Boot service (Java 17) ]
- @RestController, @Valid DTOs (records)
- validates, stores metadata
- uploads file to S3 (presigned URL or SDK)
  |                               |
  v                               v
[ MongoDB ]                     [ AWS S3 ]
document metadata                actual file bytes
(filename, type, owner,          (key = userId/uuid)
s3Key, uploadedAt)
```

Key points to mention: store **metadata in MongoDB/MySQL**, **binary in S3** (never blobs in the DB); use **presigned URLs** so the client uploads/downloads directly from S3; secure with **JWT**; handle large files with multipart; add validation and a global exception handler.

S3 upload from Spring Boot (be ready to discuss)

```
@Service
public class S3Service {
    private final S3Client s3;    // AWS SDK v2

    public String upload(MultipartFile file, String key) throws
IOException {
        s3.putObject(
            PutObjectRequest.builder().bucket("berrirobot-
docs").key(key).build(),
            RequestBody.fromBytes(file.getBytes())
        );
        return key;
    }

    // Presigned URL so the client downloads directly from S3
    public String presignedDownloadUrl(String key) {
        S3Presigner presigner = S3Presigner.create();
        GetObjectPresignRequest req = GetObjectPresignRequest.builder()
            .signatureDuration(Duration.ofMinutes(15))
            .getObjectRequest(b -> b.bucket("berrirobot-docs").key(key))
            .build();
        return presigner.presignGetObject(req).url().toString();
    }
}
```

6. MongoDB + MySQL together (polyglot persistence)

They list **both** — expect "when do you use which?"

- **MySQL**: transactional, relational, strong consistency — users, payments, orders, structured relationships.
- **MongoDB**: flexible/nested data, high write volume, evolving schema — activity logs, content, catalogs, dashboards, embedded sub-documents.
- A single Spring Boot app can use **both** (Spring Data JPA + Spring Data MongoDB), each for the right data.

7. Focused 5-day plan for Berribot

Day	Focus
1	MongoDB (13) — model, queries, aggregation, Spring Data. This is your gap.
2	Java 17 (12) + revise Core Java (05)
3	Spring Boot + REST + JPA (06) + the S3/full-stack scenario above
4	Angular 16 (04) + the Angular 16 section above
5	Microservices (08) + coding practice (09) + self-intro & projects (00, 10)

8. Senior-level talking points (5-year framing)

- At "senior," they expect **ownership and judgment**, not just coding:
- Talk about **design decisions and trade-offs** (why MongoDB vs MySQL, why a service split).
 - Mention **mentoring, code reviews, and architecture** input (you "assisted in refining architecture" on Subsea).
 - Show **end-to-end ownership** — API → DB → UI → deployment.
 - Discuss **non-functionals**: performance, security (JWT), scalability, monitoring.
 - Quantify impact where you can (retention from Study Reminders, migration of 4 repos, etc.).

9. Pre-interview checklist (Berribot edition)

- [] Can explain **MongoDB vs MySQL** and when to use each
- [] Can write a basic **aggregation pipeline** and a **MongoRepository** interface
- [] Can name **5 Java 17 features** and where you'd use them

- [] Can describe **Angular 16 standalone components & signals** at a high level
- [] Can whiteboard the **upload-to-S3 + metadata-in-DB** flow with presigned URLs
- [] Can split a feature into **microservices** and justify the boundaries
- [] Self-intro polished; one strong **STAR story** per project ready
- [] Honest about MongoDB/Java 17 depth — but concepts solid

Good luck with the Berribot assessment, Priyanka! Lead with your Angular + Spring Boot strength, bridge MongoDB through your Neo4j NoSQL mindset, and show senior-level ownership.

Ionic — Level 1 Interview Q&A

◆ **Built for:** *Priyanka — Java with Angular Ionic — Level 1 — Virtual Interview — Codeboard Technology*

Level 1 = fundamentals round. They will NOT ask you to architect an offline-sync engine. They will ask: *what is Ionic, how is it different from Angular, which components have you used, how does it become a mobile app, what plugins did you use.* Answer crisply and always land it on **RoboGebra** (your Ionic project, Oct 2024 – Present).

For the harder follow-ups, see `../interview-preparation/Advanced_Ionic_Capacitor_Interview_10Q.md`.

◆ Your 30-second Ionic positioning statement

"I've used Ionic with Angular on **RoboGebra**, an AI-driven math learning platform. It's an Ionic + Angular front end talking to Java/Spring Boot APIs with MongoDB. I built the learning dashboard, the Study Reminder module with local notifications, the Study List, and the quiz module. Because Ionic sits on top of Angular, I reused everything I already know — components, services, RxJS, reactive forms, routing — and got a single codebase that runs as a web app and as a native Android/iOS build through Capacitor."

Memorize that. It answers "do you know Ionic?" in one breath and immediately gives them three follow-up hooks.

Section A — Ionic Fundamentals

Q1. What is Ionic?

Ionic is an **open-source UI toolkit for building cross-platform mobile and desktop apps using web technologies** — HTML, CSS and JavaScript/

TypeScript. You write one codebase and ship it to **Android, iOS, PWA and desktop**.

Key points to say:

- It gives you a large library of **pre-built, mobile-optimized UI components** (`ion-button` , `ion-list` , `ion-modal` , ...) that automatically adapt their look to the platform (Material Design on Android, iOS styling on iOS).
- It is **framework-agnostic today** — works with Angular, React, Vue, or plain JavaScript. Internally the components are **Web Components** built with Stencil.
- Ionic handles the **UI layer**. The **native layer** (camera, storage, notifications) is handled by **Capacitor** (or older Cordova).

Q2. Ionic vs Angular — what's the difference?

This is the #1 Level-1 question for your profile. Answer:

	Angular	Ionic
What it is	A full application framework — components, DI, routing, forms, HTTP, change detection	A UI component library + native runtime for mobile
Provides	App architecture and logic	Mobile-styled UI components + access to native device features
Runs on	Browser	Browser, Android, iOS, desktop
Relationship	Ionic uses Angular (or React/ Vue) underneath	Ionic is built <i>on top of</i> your framework

One-liner: *"Angular is the framework, Ionic is the mobile UI layer and native bridge on top of it. An Ionic-Angular app IS an Angular app — same modules, services, routing and RxJS — just with Ionic's components and Capacitor for native access."*

Q3. Native vs Hybrid vs Cross-platform apps?

- **Native** — written in the platform's own language (Kotlin/Java for Android, Swift/Objective-C for iOS). Best performance and deepest device access, but you maintain **two separate codebases**.
- **Hybrid** — web app (HTML/CSS/JS) running inside a **WebView** wrapped in a native shell. One codebase, faster and cheaper to build. **Ionic is here**.
- **Cross-platform (compiled)** — e.g. React Native / Flutter. One codebase that renders **actual native UI widgets** (React Native) or its own engine (Flutter), so performance is closer to native.

Q4. Advantages of Ionic?

1. **Single codebase** for Android, iOS and web → less cost and faster delivery.
2. **Web skills reused** — any Angular developer is productive immediately.
3. **Rich pre-built UI** that follows each platform's design language automatically.
4. **Live reload in the browser** — you develop 90% of the app with `ionic serve`, no emulator needed.
5. **Capacitor plugin ecosystem** for camera, geolocation, storage, notifications, etc.
6. **PWA out of the box** — the same build deploys to the web.

Q5. Limitations / disadvantages of Ionic?

Say these honestly — Level-1 interviewers respect it:

- Runs in a **WebView**, so heavy graphics/animation-intensive apps (3D games, video editing) perform worse than native.
- **Large lists / heavy DOM** can lag — you mitigate with `ion-virtual-scroll` / virtual scrolling and `trackBy`.
- Dependency on **third-party plugins** for native features; a badly maintained plugin can block you.
- App **bundle size** is larger than a lean native app.
- Debugging native-layer issues still needs Xcode/Android Studio knowledge.

Q6. What is Capacitor?

Capacitor is Ionic's native runtime. It takes your built web app, drops it into a native Android/iOS project, and gives you a **JavaScript API to call native device features**.

- Modern **Promise/async-await** based plugin API.
- The `android/` and `ios/` folders are **real native projects committed to git** — you can open them in Android Studio/Xcode and add native code.
- Backward compatible with most **Cordova** plugins.
- It has replaced Cordova as the default; **Cordova is in maintenance mode**.

Q7. Cordova vs Capacitor?

	Cordova	Capacitor
Native project	Auto-generated, gitignored	Committed to source control, editable
Plugin API	Callback-based	Promise / async-await
Config file	<code>config.xml</code>	<code>capacitor.config.ts</code>
Framework support	Ionic-oriented	Any web framework
PWA / web target	Not supported	First-class
Status	Maintenance mode	Actively developed

Q8. How does an Ionic app actually run on a phone?

1. You build the web app → static assets in `www/` (Angular) or `dist/`.
2. `npx cap sync` copies those assets into the native `android/` and `ios/` projects and installs plugins.
3. The native app launches a **WebView** that loads `index.html` from the local file system (no network needed).
4. When your JS calls something like `Camera.getPhoto()`, **Capacitor bridges** that call to the native Java/Kotlin or Swift code, which returns the result back to JavaScript as a Promise.

Section B — Setup, CLI & Project Structure

Q9. How do you create an Ionic project?

```
npm install -g @ionic/cli          # install the CLI
ionic start myApp blank --type=angular # create (blank | tabs | sidemenu
| list)
cd myApp
ionic serve                        # run in the browser with live reload
```

Q10. Common Ionic CLI commands you should be able to rattle off

```
ionic serve          # dev server in the browser, live reload
ionic serve --lab    # side-by-side iOS + Android preview
ionic generate page home # scaffold page / component / service / guard
ionic build          # production web build
ionic build --prod

npx cap add android  # add native platform
npx cap sync         # build assets + copy to native + update
plugins
npx cap copy         # copy web assets only (faster)
npx cap open android # open in Android Studio
ionic capacitor run android -l --external # run on device with live
reload
```

Q11. What's in an Ionic-Angular project?

```
src/
├── app/
│   ├── app.module.ts          # IonicModule.forRoot()
│   ├── app-routing.module.ts  # lazy-loaded page routes
│   └── pages/                 # each page = a folder (ts/html/scss/
module)
│   ├── services/             # API + business logic
│   └── shared/               # reusable components, pipes, guards
├── assets/                   # images, icons, i18n files
├── theme/variables.scss      # Ionic CSS variables / theming
├── global.scss
└── index.html

capacitor.config.ts          # app id, name, webDir, plugin config
android/ ios/               # native projects (Capacitor)
```

Q12. What is `IonicModule.forRoot()` ?

It's imported once in `AppModule` to **initialize Ionic** — registering the components, setting global config (mode, animations, back-button behaviour) and providing Ionic's singleton services. Feature/page modules import plain `IonicModule` (no `forRoot`). Same pattern as `RouterModule.forRoot()` vs `forChild()`.

With **standalone components** (Angular 15+/Ionic 7+), you instead use `provideIonicAngular()` in `bootstrapApplication` and import components individually.

Section C — Components & UI

Q13. Which Ionic components have you used? *(Be ready to name 10+)*

- **Layout:** `ion-app`, `ion-header`, `ion-toolbar`, `ion-title`, `ion-content`, `ion-footer`, `ion-grid` / `ion-row` / `ion-col`
- **Navigation:** `ion-tabs`, `ion-menu`, `ion-back-button`, `ion-segment`
- **Lists & data:** `ion-list`, `ion-item`, `ion-label`, `ion-avatar`, `ion-thumbnail`, `ion-item-sliding`, `ion-virtual-scroll`, `ion-infinite-scroll`, `ion-refresher`
- **Forms:** `ion-input`, `ion-textarea`, `ion-select`, `ion-checkbox`, `ion-radio`, `ion-toggle`, `ion-range`, `ion-datetime`, `ion-searchbar`
- **Overlays:** `ion-modal`, `ion-popover`, `ion-alert`, `ion-toast`, `ion-loading`, `ion-action-sheet`
- **Misc:** `ion-button`, `ion-icon`, `ion-card`, `ion-badge`, `ion-chip`, `ion-spinner`, `ion-progress-bar`, `ion-fab`, `ion-skeleton-text`, `ion-accordion`

◆ From RoboGebra you can honestly cite: `ion-card` + `ion-grid` for the dashboard, `ion-list` / `ion-item-sliding` for the Study List, `ion-datetime` for reminder scheduling, `ion-modal` + `ion-alert` + `ion-toast` for quiz feedback, `ion-segment` for switching between short and detailed answers.

Q14. **ion-content** — why is it required?

ion-content is the **scrollable area** of a page. It provides the scroll container, safe-area padding, and hosts **ion-refresher** , **ion-infinite-scroll** and the fullscreen behaviour. Without it your page won't scroll properly on device.

```
<ion-header>
  <ion-toolbar><ion-title>Study List</ion-title></ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <ion-refresher slot="fixed" (ionRefresh)="doRefresh($event)">
    <ion-refresher-content></ion-refresher-content>
  </ion-refresher>
  ...
</ion-content>
```

Q15. Grid system in Ionic?

A 12-column **flexbox** grid: **ion-grid** → **ion-row** → **ion-col** . Responsive breakpoints **size** , **size-sm** , **size-md** , **size-lg** , **size-xl** .

```
<ion-grid>
  <ion-row>
    <ion-col size="12" size-md="6">Left</ion-col>
    <ion-col size="12" size-md="6">Right</ion-col>
  </ion-row>
</ion-grid>
```

Q16. Alert vs Toast vs Loading vs ActionSheet vs Modal?

Component	Use it for
Alert	Ask a question / confirm — has buttons, blocks the UI
Toast	Short non-blocking feedback message ("Saved successfully")
Loading	Blocking spinner while an async call runs
ActionSheet	Menu of actions sliding up from the bottom

Component	Use it for
Modal	A full page shown over the current page (forms, details)
Popover	Small contextual overlay anchored to an element

```

constructor(private toastCtrl: ToastController,
             private alertCtrl: AlertController,
             private loadingCtrl: LoadingController,
             private modalCtrl: ModalController) {}

async showToast(msg: string) {
  const t = await this.toastCtrl.create({ message: msg, duration: 2000,
    position: 'bottom' });
  await t.present();
}

async confirmDelete() {
  const alert = await this.alertCtrl.create({
    header: 'Delete reminder?',
    message: 'This cannot be undone.',
    buttons: [
      { text: 'Cancel', role: 'cancel' },
      { text: 'Delete', role: 'destructive', handler: () =>
this.delete() }
    ]
  });
  await alert.present();
}

```

Q17. How do you open a modal and get data back?

```

async openQuiz() {
  const modal = await this.modalCtrl.create({
    component: QuizPage,
    componentProps: { topicId: 42 }           // pass data IN
  });
  await modal.present();

  const { data, role } = await modal.onWillDismiss(); // get data BACK
  if (role === 'confirm') { this.score = data.score; }
}

```

Inside `QuizPage`: receive with `@Input() topicId!: number;` and close with `this.modalCtrl.dismiss({ score: 8 }, 'confirm')`.

Q18. Infinite scroll and pull-to-refresh?

```
<ion-refresher slot="fixed" (ionRefresh)="refresh($event)">
  <ion-refresher-content pullingText="Pull to refresh"></ion-refresher-
content>
</ion-refresher>

<ion-list>
  <ion-item *ngFor="let q of questions; trackBy: trackById">{{ q.title }}
</ion-item>
</ion-list>

<ion-infinite-scroll (ionInfinite)="loadMore($event)">
  <ion-infinite-scroll-content loadingText="Loading..."></ion-infinite-
scroll-content>
</ion-infinite-scroll>
```

```
refresh(ev: any) {
  this.api.getQuestions().subscribe(d => { this.questions = d;
ev.target.complete(); });
}
loadMore(ev: any) {
  this.page++;
  this.api.getQuestions(this.page).subscribe(d => {
    this.questions.push(...d);
    ev.target.complete();
    if (!d.length) { ev.target.disabled = true; } // no more data
  });
}
```

Always call `event.target.complete()` — otherwise the spinner never stops. That's the follow-up they ask.

Section D — Navigation & Routing

Q19. How does navigation work in Ionic Angular?

It uses the **standard Angular Router** — no special Ionic navigation API needed (the old `NavController.push()` stack was Ionic 3).

```
// routing module — lazy loaded pages
const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: 'home', loadChildren: () => import('./pages/home/home.module').then(m => m.HomePageModule) },
  { path: 'study-list/:id', loadChildren: () => import('./pages/study/study.module').then(m => m.StudyPageModule) },
];
```

```
this.router.navigate(['/study-list', id]); // go
this.router.navigate(['/home'], { queryParams: { tab: 1 } });
// read it back
this.route.snapshot.paramMap.get('id');
this.route.paramMap.subscribe(p => this.id = p.get('id'));
```

Q20. What is `NavController` then?

Ionic's thin wrapper over the Angular Router that adds **native-feeling page transitions and stack direction**:

- `navCtrl.navigateForward('/details')` — slides in from the right (push)
- `navCtrl.navigateBack('/home')` — slides back (pop)
- `navCtrl.navigateRoot('/login')` — resets the stack (use after logout)

Q21. How do you pass data between pages?

1. **Route params** — `/study/:id` for IDs.
2. **Query params** — `{ queryParams: { mode: 'edit' } }`.
3. **A shared service** (best for objects) — a service with a `BehaviorSubject`.
4. **Router state** — `navigate(['/x'], { state: { item } })`, read via `router.getCurrentNavigation()?.extras.state`.
5. **Modal `componentProps`** — for modals/popovers.

▲ Never pass large objects through the URL. Interviewers like hearing "IDs in the URL, objects through a service."

Q22. Tabs vs Side menu?

- **ion-tabs** — 3–5 top-level destinations, always visible at the bottom, each tab keeps its own navigation stack. Configured with a parent route holding child routes.
- **ion-menu** — a drawer for many/secondary destinations; opened with **ion-menu-button** or **menuCtrl.open()**.

Q23. How do you handle the Android hardware back button?

```
this.platform.backButton.subscribeWithPriority(10, () => {  
  if (this.router.url === '/home') { App.exitApp(); } // @capacitor/app  
  else { this.navCtrl.back(); }  
});
```

Section E — Lifecycle Hooks ▲ *High-probability question*

Q24. Ionic page lifecycle hooks vs Angular lifecycle hooks?

Ionic **caches pages** in the navigation stack, so a page can be *left and re-entered without being destroyed*. That's why Ionic adds its own hooks.

Hook	When it fires
ionViewWillEnter	Just before the page becomes active — fires every time you enter
ionViewDidEnter	After the page is fully active and the transition finished
ionViewWillLeave	Just before leaving the page
ionViewDidLeave	After the page has been left

Hook	When it fires
<code>ngOnInit</code>	Once , when the component is created (may be cached afterwards)
<code>ngOnDestroy</code>	When the component is actually destroyed / removed from the stack

The key interview point:

"`ngOnInit` runs only once because Ionic caches the page. So one-time setup goes in `ngOnInit`, but **anything that must be fresh every time the user comes back to the page — like refreshing a list after an edit — goes in `ionViewWillEnter`.**"

```
export class StudyListPage {
  ngOnInit() { this.buildForm(); }           // once
  ionViewWillEnter() { this.loadStudyItems(); } // every entry -
  data stays fresh
  ionViewWillLeave() { this.sub?.unsubscribe(); }
}
```

The easiest way to remember it

Angular assumes a page is DESTROYED when you leave.
Ionic KEEPS IT ALIVE in the navigation stack. ★

- so `ngOnInit` fires ONCE, ever
- and `ionViewWillEnter` fires EVERY TIME you come back ★

```

Page A —push→ Page B
  |             |
  |             |
still ALIVE    active
in the stack   |
  |             |
←back          |
  |             |
ngOnInit does NOT run again ▲
ionViewWillEnter DOES ✓ ★
```

Real-world idea: **browser tabs**. Switching away from a tab doesn't close it — it's still there with all its state. Coming back doesn't reload the page.

▲ The bug this causes, and the one to describe:

User opens the Study List → ngOnInit loads 5 items
User edits an item on another page → saves
User presses back
 → ngOnInit does NOT run
 → the list still shows the OLD data ▲
Fix: load in ionViewWillEnter ★

Easy memory

Ionic CACHES pages in the navigation stack ★ (like browser tabs)

ngOnInit	→ ONCE ever	→ build the form, one-time setup
ionViewWillEnter	→ EVERY entry ★	→ refresh the data
ionViewWillLeave	→ every exit	→ pause timers, unsubscribe
ngOnDestroy	→ only when the page really leaves the stack	

THE question: "why doesn't my list refresh after an edit?"
→ because you loaded it in ngOnInit ★

Section F — Native Features / Capacitor Plugins

Q25. How do you access native device features?

Install a Capacitor plugin, import it, call it — it returns a Promise.

```
npm install @capacitor/camera
npx cap sync
```

```
import { Camera, CameraResultType, CameraSource } from '@capacitor/camera';

async takePhoto() {
  const photo = await Camera.getPhoto({
    quality: 80,
    resultType: CameraResultType.Uri,
    source: CameraSource.Camera
  });
  this.imgSrc = photo.webPath;
}
```

Q26. Which Capacitor plugins do you know?

▲ **Answer with the ones YOU actually ship, not a textbook list.** If you name `@capacitor/camera` and they ask what you used it for, you're stuck.

● **The 19 plugins actually in `robogebra-mobile/package.json` ★**

@capacitor/app	app state, deep links, hardware back button
@capacitor/browser	in-app browser
@capacitor/device	device info
@capacitor/haptics	tactile feedback
@capacitor/keyboard	keyboard events + resize behaviour
@capacitor/network	online/offline detection ★
@capacitor/preferences	native key-value storage ★
@capacitor/push-notifications	+ @capacitor-firebase/messaging (FCM)
★	
@capacitor/screen-orientation	
@capacitor/splash-screen	
@capacitor/status-bar	
@capacitor-community/firebase-analytics	
@capawesome/capacitor-app-update	in-app update prompts ★
capacitor-native-settings	deep-link into the OS settings screen
capacitor-razorpay	NATIVE payments ★

■ **The story to tell:** "About nineteen plugins in production. The two with real complexity were push notifications — Capacitor Firebase messaging on the client, Firebase Admin on the Spring Boot side, with a dedicated bounded thread pool for the fan-out — and native Razorpay, because a web checkout flow inside a WebView doesn't survive an app-store review. We also use `@capacitor/network`

for offline detection, and `@capacitor/preferences` rather than `localStorage`, since the OS can evict WebView storage under pressure."

★ Two details in there that score:

1. Preferences over `localStorage` – because the OS can EVICT WebView storage ★
2. `@capawesome/capacitor-app-update` – because users DON'T update apps, unlike a web deploy where everyone gets the new version instantly ★

That second one shows you've thought about the *operational* difference between shipping web and shipping mobile, which is exactly what a lead is asked about.

Easy memory

Name YOUR 19, not a textbook list ★

The headline three: PUSH (Firebase) | RAZORPAY (native checkout) | NETWORK (offline)

Preferences > `localStorage` (the OS can evict WebView storage) ★

`app-update` plugin → because users don't update apps ★

Q27. How do you check whether the app is running on a device or in the browser?

```
import { Platform } from '@ionic/angular';
import { Capacitor } from '@capacitor/core';

constructor(private platform: Platform) {}

ngOnInit() {
  this.platform.ready().then(() => {
    if (Capacitor.isNativePlatform()) { /* device-only code */ }
    if (this.platform.is('android')) { }
    if (this.platform.is('ios')) { }
    if (this.platform.is('mobileweb')) { }
  });
}
```

Also `Capacitor.isPluginAvailable('Camera')` before calling a plugin — this is the safe pattern, because plugins that need native code will fail in `ionic serve`.

Q28. How do you store data locally?

Option	Use for
@capacitor/preferences	Small key-value data — tokens, settings, flags
@ionic/storage-angular	Key-value with an IndexedDB/SQLite driver, larger data
SQLite plugin	Structured/relational offline data, large datasets
localStorage	Web only — not reliable on native , can be cleared by the OS

```
import { Preferences } from '@capacitor/preferences';
await Preferences.set({ key: 'token', value: jwt });
const { value } = await Preferences.get({ key: 'token' });
await Preferences.remove({ key: 'token' });
```

The easiest way to remember local storage

Preferences ★ → SMALL key-value, NATIVE storage → tokens, settings, flags
Ionic Storage → larger key-value, IndexedDB/SQLite driver
SQLite → structured, relational, big offline datasets
localStorage → WEB ONLY ▲ — the OS can wipe WebView storage ★

localStorage lives inside the WEBVIEW.
Android/iOS may CLEAR WebView data when the device is low on space. ▲
↓
Your user is silently logged out and loses their settings.
↓
Preferences writes to NATIVE storage (SharedPreferences / UserDefaults)
✓ ★

★ That is the single best Ionic storage answer — it explains why, not just what.

● In your RoboGebra code

@capacitor/preferences is in the dependency list — and the auth interceptor reads the token asynchronously (via `getAuthDetailsObservable()` + `switchMap()`) precisely because native storage is async, unlike `localStorage`.

■ "Native `Preferences` is promise-based, not synchronous like `localStorage`. That's why our HTTP interceptor `switchMap` s over an `auth-details` observable to attach the token — the token isn't available synchronously when the request is created."

★ That connects **storage** → **RxJS** → **the interceptor**, and it's true of your actual code.

Q29. How do you handle permissions?

Plugins expose `checkPermissions()` and `requestPermissions()`. You also declare them natively: `AndroidManifest.xml` for Android, `Info.plist` usage descriptions for iOS (iOS **rejects the build** without a usage description string).

Section G — Theming & Styling

Q30. How do you theme an Ionic app?

Ionic is styled with **CSS custom properties (variables)**, defined in `src/theme/variables.scss`.

```
:root {
  --ion-color-primary: #3880ff;
  --ion-color-primary-contrast: #ffffff;
  --ion-background-color: #ffffff;
}
```

Override per-component too:

```
ion-button { --background: #6c3; --border-radius: 12px; }
```

Because Ionic components use **Shadow DOM**, plain CSS selectors often don't reach inside them — **that's why you style through CSS variables** (or `::part()`). This is a classic follow-up.

Q31. Dark mode?

```
@media (prefers-color-scheme: dark) {  
  body { --ion-background-color: #121212; --ion-text-color: #fff; }  
}
```

Or toggle a class manually:

```
document.body.classList.toggle('dark', enabled) .
```

Q32. What is `mode` in Ionic?

`mode` decides the platform styling: `'md'` (Material Design) or `'ios'`. By default Ionic picks it from the device. You can force it globally:

```
IonicModule.forRoot({ mode: 'md' })
```

or per component: `<ion-button mode="ios">`.

Q33. Ionic utility CSS classes?

`ion-padding`, `ion-margin`, `ion-text-center`, `ion-float-right`, `ion-hide`,
`ion-hide-md-down`, `ion-justify-content-center`, `ion-align-items-center`.

Section H — Working with APIs

Q34. How do you call a REST API in Ionic?

Exactly like Angular — `HttpClient` in a service:

```

@Injectables({ providedIn: 'root' })
export class StudyService {
  private base = environment.apiUrl; // Spring Boot backend
  constructor(private http: HttpClient) {}

  getStudyList(): Observable<StudyItem[]> {
    return this.http.get<StudyItem[]>(`${this.base}/study-list`)
      .pipe(catchError(this.handleError));
  }
  create(item: StudyItem) { return this.http.post<StudyItem>(`${this.base}/study-list`, item); }
  update(id: string, item: StudyItem) { return this.http.put<StudyItem>(`${this.base}/study-list/${id}`, item); }
  delete(id: string) { return this.http.delete(`${this.base}/study-list/${id}`); }
}

```

Add a **JWT interceptor** for the auth header, and a **loading indicator** around slow calls.

Q35. CORS works in ionic serve but breaks on device — why?

On the browser, the origin is `http://localhost:8100`. On a device the WebView origin is different (`capacitor://localhost` on iOS, `http://localhost` on Android), so the server's CORS config must allow it — or you avoid CORS entirely because the native WebView isn't subject to the same rules once configured.

Fixes: proxy in `ionic.config.json` during development, correct **CORS configuration on the Spring Boot side** (`@CrossOrigin` / a `WebMvcConfigurer`), and `server.androidScheme` in `capacitor.config.ts`.

This is a great answer for a **Java + Ionic** interview because it shows you understand both ends.

Section I — Build & Deployment

Q36. Steps to build and run an Ionic app on Android?

```
ionic build --prod           # 1. build web assets
npx cap add android          # 2. once, adds the native project
npx cap sync android         # 3. copy assets + sync plugins
npx cap open android         # 4. opens Android Studio
# 5. Build > Generate Signed Bundle/APK → upload to Play Console
```

For iOS the same, but you need **macOS + Xcode**, a signing certificate and a provisioning profile, then upload via Xcode/Transporter to App Store Connect.

Q37. What's in `capacitor.config.ts` ?

```
const config: CapacitorConfig = {
  appId: 'com.robogebra.app',
  appName: 'RoboGebra',
  webDir: 'www',
  server: { androidScheme: 'https' },
  plugins: { SplashScreen: { launchShowDuration: 2000 } }
};
```

Q38. `cap copy` vs `cap sync` vs `cap update` ?

- `copy` — copies web assets + config into the native projects (fastest, use after a plain code change).
- `update` — updates the native dependencies/plugins.
- `sync` = `copy` + `update`. Use after installing a plugin.

Q39. Can an Ionic app be a PWA?

Yes — that's a headline Ionic feature. `ng add @angular/pwa` adds a service worker and manifest, then `ionic build --prod` gives you a deployable PWA from the same codebase.

Section J — Resume-linked questions (*expect these*)

Q40. "You mention Ionic on your resume — what exactly did you build with it?"

"On **RoboGebra**, an AI-driven math learning platform, the mobile app is **Ionic 8.7 + Capacitor 6.2 on Angular 18**, and the backend is **Java 17 / Spring Boot 3.2 with MongoDB**. My main modules were:

- the **personalized learning dashboard** — progress analytics and learning timelines, built with `ion-card` / `ion-grid` and Chart rendering;
- the **Study Reminder system** — flexible scheduling with `ion-datetime`, notification delivery, and full REST CRUD APIs;
- the **Study List module** — create/organize/track goals, with `ion-item-sliding` for swipe actions and real-time sync;
- the **Quiz module** — dynamic questions, instant evaluation and solution walkthroughs in modals;
- plus the **bilingual (Tamil/English) AI explanation** views with step-by-step reasoning."

● Numbers you can quote about the app ★

Angular 18.2 · Ionic 8.7.5 · Capacitor 6.2.1 · RxJS 7.8 · TypeScript 5.4
163 components · 89 services · 27 route resolvers · 4 guards · 3 HTTP interceptors
19 Capacitor plugins
takeUntil 426 · switchMap 246 · BehaviorSubject 122 · async pipe 311 · trackBy 55

★ Quoting real numbers is disproportionately convincing — nobody invents "27 resolvers".

Full inventory: [39 — RoboGebra Code Examples](#).

Q41. "Since Ionic is just Angular, what did you actually have to learn?"

"Three things: the **Ionic page lifecycle** — `ionViewWillEnter` versus `ngOnInit`, because pages are cached; **styling through CSS variables** instead of normal CSS, because the components use Shadow DOM; and the **Capacitor build cycle**

— build, `cap sync`, then run from Android Studio, plus handling code paths that only work on a real device."

Q42. "Any challenge you faced in the Ionic app?"

Pick one and tell it in STAR form. A safe, true-to-your-work example:

- S** — the Study List needed to feel instant, but every edit re-fetched from the server and the list flickered.
- T** — make updates feel real-time without hammering the API.
- A** — moved the list into a service-held `BehaviorSubject` so all screens shared one stream, refreshed on `ionViewWillEnter` instead of `ngOnInit`, added `trackBy` on the `*ngFor`, and used optimistic UI updates with rollback on API error.
- R** — no more flicker, fewer API calls, and the list stayed consistent across the dashboard and the detail page.

(Adjust the details to what you actually did — never invent numbers.)

Q43. "Did you publish to the Play Store / App Store?"

Answer honestly. If you didn't own the release, say:

"I worked on the app build and tested on device through Android Studio; the store release was handled by our lead — but I know the flow: signed bundle, versionCode bump, Play Console internal track, then production."

▲ Rapid-fire round (say the answer in one line)

Question	Answer
Latest Ionic version family?	Ionic 7/8 (Angular standalone support, <code>ion-input</code> label props)
Ionic components are built with?	Stencil — compiled to Web Components
Default native runtime?	Capacitor

Question	Answer
Which folder holds web assets after build?	<code>www/</code> (Angular)
How to preview both platforms at once?	<code>ionic serve --lab</code>
Component for a scrollable page body?	<code>ion-content</code>
Hook that fires on every page entry?	<code>ionViewWillEnter</code>
Storage plugin for key-value?	<code>@capacitor/preferences</code> — not <code>localStorage</code> , the OS can evict WebView storage ★
How to style inside a shadow-DOM component?	CSS variables / <code>::part()</code>
Force Material styling?	<code>mode: 'md'</code>
Navigate with a forward animation?	<code>navCtrl.navigateForward()</code>
Stop the infinite-scroll spinner?	<code>event.target.complete()</code>
Detect native vs web?	<code>Capacitor.isNativePlatform()</code> / <code>platform.is('android')</code>
Grid columns?	12
Is Ionic free?	Yes — MIT licensed open source (paid add-ons like Appflow exist)

✓ Ionic checklist before the call

- [] Can explain **Ionic vs Angular** in 20 seconds
- [] Can explain `ngOnInit` vs `ionViewWillEnter` and *why* it matters (page caching)
- [] Can name **10+ Ionic components** without pausing
- [] Can describe the **build** → `cap sync` → **Android Studio** flow
- [] Can name **5 Capacitor plugins** and one you used
- [] Know **theming = CSS variables** because of Shadow DOM
- [] Have the **RoboGebra Ionic story** ready with 4 concrete modules

- [] Have **one STAR challenge** from the Ionic app
-

Next: [16 — Codeboard Technology Level-1 Interview Guide](#)

● Codeboard Technology — "Java with Angular Ionic — Level 1 — Virtual Interview"

▲ **SINGLE ROUND → ONBOARDING.** This one call decides everything.

Candidate: Priyanka R · 5 years · Full Stack (Angular + Ionic + Java/Spring Boot)

Round: Level 1 — Virtual — **the only round.** HR confirmed: clear this → onboarding process.

Company: Codeboard Technology Pvt Ltd, Chennai (Alandur) — IT services / staffing, hires Java and Angular developers, often for client projects.

■ Status note (3 Sept 2026)

This round was attended — ✗ not selected (logged in [26 — Companies](#), 7 questions).

Keep this file. It is the best *template* in the pack for a **"Java + Angular + Ionic, single round"** interview, and that shape keeps recurring — it is close to the **IQVIA** brief. Reuse it as a rehearsal script; just swap the company facts.

▲ **Two answers in here need the current version of the facts:**

'''

MongoDB → NOT a gap. It is your production database ★
104 @Document · 85 repositories · 27 @Aggregation pipelines
+ Mongoose on the Node CRM

Ionic → name YOUR 19 Capacitor plugins, not a textbook list ★
push (Firebase) · native Razorpay · network · Preferences
(Preferences over localStorage — the OS can evict WebView storage)

'''

0. ● WHAT THEY ACTUALLY ASKED — the real question list (with answers)

This round has now happened. These are the questions Codeboard actually asked. Everything below is answered in full — **revise this section first**, because the same interviewer and the same question bank get reused, and these topics are the ones this company cares about.

The Java round was output-based, not theory — that's the lesson that cost the round. Full drill set in [23 — Java "Predict the Output"](#).

#	Asked	Type	Where it's covered in depth
1	Abstract class	Java theory	05 · 17
2	Data hiding	Java theory	05
3	<code>methodTest(null)</code> — overloading with <code>Object</code> vs <code>String</code>	Output	23 — Q1
4	Override that adds <code>throws IOException</code>	Output	23 — Q2
5	Ionic lifecycle hooks	Ionic	15 — Q24
6	Difference between Angular and Ionic lifecycle hooks	Ionic	15 — Q24
7	How do you handle offline support in Ionic	Ionic / design	below

Q1. What is an abstract class?

A class declared with the `abstract` keyword that **cannot be instantiated** and is meant to be extended. It can hold **both** abstract methods (no body, subclasses must implement them) and concrete methods with shared logic, plus constructors, instance fields and any access modifier.

```
abstract class Shape {
    private final String name;           // state – an interface can't
    have this
    Shape(String name) { this.name = name; } // constructors are allowed

    abstract double area();              // subclass MUST implement

    void describe() {                    // shared concrete behaviour
        System.out.println(name + " area = " + area());
    }
}

class Circle extends Shape {
    private final double r;
    Circle(double r) { super("Circle"); this.r = r; }
    @Override double area() { return Math.PI * r * r; }
}

// new Shape("x");    ✗ compile error – Shape is abstract
Shape s = new Circle(2); //
✓ reference of abstract type, object of the concrete subclass
```

Rules worth stating:

- An abstract class **need not** have any abstract method — `abstract` alone is enough to block instantiation.
- A class with **even one** abstract method **must** be declared abstract.
- `abstract` cannot combine with `final` (nothing could extend it), `static`, or `private` (nothing could override it).
- A subclass that doesn't implement all inherited abstract methods must itself be abstract.

The guaranteed follow-up: abstract class vs interface

	Abstract class	Interface
Inheritance	<code>extends</code> — only one	<code>implements</code> — many ✓
Fields	Any instance state, any modifier	<code>public static final</code> constants only
Constructors	✓ Yes	✗ No
Method access	public / protected / default / private	public (plus <code>default</code> , <code>static</code> , <code>private</code> since Java 8/9)
Purpose	" is-a " — shared identity + shared code	" can-do " — a capability contract
Since Java 8	—	<code>default</code> and <code>static</code> methods allow behaviour too

■ **Say this:** "I use an abstract class when subclasses genuinely share state and partial implementation — a base entity or a template-method flow. I use an interface to define a capability that unrelated classes can implement, and because a class can implement many interfaces but extend only one class. Since Java 8, interfaces can carry `default` methods, so the real distinguisher isn't 'code vs no code' any more — it's state and identity: only an abstract class can hold instance fields and a constructor."

Q2. What is data hiding?

Data hiding is restricting direct access to an object's internal state so it can only be changed through controlled methods. In Java you do it by declaring fields `private` and exposing `public` getters/setters that enforce the rules.

```

public class BankAccount {
    private double balance;           // hidden – nobody can touch
    it directly

    public double getBalance() { return balance; }

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Deposit must
be positive");
        balance += amount;           // the invariant is enforced
here
    }
}

// account.balance = -5000;  x  compile error – that's the whole point

```

Why it matters: the object protects its own invariants. Without it, any code anywhere can put the object into an invalid state, and you can never change the internal representation without breaking callers.

Follow-up: data hiding vs encapsulation vs abstraction

Term	What it means	How
Data hiding	Restricting access to internal state	<code>private</code> fields
Encapsulation	Bundling data + the methods that operate on it into one unit, and hiding the internals	<code>private</code> fields + <code>public</code> methods
Abstraction	Hiding complexity — showing <i>what</i> an object does, not <i>how</i>	<code>abstract</code> classes / interfaces

■ **The one-liner:** "Data hiding is the mechanism — `private` fields. Encapsulation is the design principle that bundles state with the behaviour that guards it. Abstraction hides complexity behind a contract. Data hiding is how you achieve encapsulation; abstraction is a different axis — it hides implementation, not data."

Follow-up: access modifiers (have this table cold)

Modifier	Same class	Same package	Subclass (other package)	Anywhere
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Q3. Output — `methodTest(null)` ●

```
public class MethodOverloadingExample {  
  
    public void methodTest(Object object) {  
        System.out.println("Calling object method");  
    }  
  
    public void methodTest(String object) {  
        System.out.println("Calling String method");  
    }  
  
    public static void main(String args[]) {  
        MethodOverloadingExample moe = new MethodOverloadingExample();  
        moe.methodTest(null);  
    }  
}
```

✓ Output

Calling String method

The rule

Overloading is resolved at compile time, and the compiler picks the *most specific applicable overload*. `null` is assignable to both `Object` and `String`, but `String` is-a `Object`, so `String` is more specific and wins.

Follow-ups they ask next

- "What if you add `methodTest(Integer)`?" → **Compile error: reference to `methodTest` is ambiguous.** `String` and `Integer` are siblings — neither is more specific.
- "How do you force the `Object` version?" → Cast it:
`moe.methodTest((Object) null);`
- "What if only `methodTest(Object)` existed?" → **Calling object method** — one applicable method, no ambiguity.
- "Overloading vs overriding?" → **"Overloading is decided by the compiler from the reference type; overriding is decided by the JVM from the object type."**

Q4. Output — override that adds `throws IOException` ●

```
class SuperClass {
    void method() {
        System.out.println("SuperClass");
    }
}

class SubClass extends SuperClass {
    void method() throws IOException {           // ▲ here
        System.out.println("SubClass");
    }

    public static void main(String args[]) {
        SuperClass s = new SubClass();
        s.method();
    }
}
```

✓ Output

Compile error — *"overridden method does not throw java.io.IOException".*

The rule

An overriding method cannot throw a broader or new *checked* exception than the method it overrides. It may throw fewer, narrower, or none — and it may throw **any unchecked** exception freely. The parent's `method()` declares no checked exceptions, so the child cannot introduce `IOException`.

Why the language does this: callers are checked against the **reference type**.

`SuperClass s` promises `method()` throws nothing; if the child could add a checked exception, that promise would be broken at runtime.

Follow-ups

- "Make it compile — three ways." → add `throws IOException` to the **parent**; or remove it from the child; or throw an **unchecked** exception (`RuntimeException` / `IOException`) instead.
- "What if the parent declared `throws Exception` and the child `throws IOException`?" → ✓ **Legal** — narrowing is allowed. Output: `SubClass`.
- "And if the parent declares `throws Exception` and the child declares nothing, then you call `s.method()`?" → **Compile error at the call site** — `unreported exception Exception` — because the reference type `SuperClass` declares it. Wrap in try/catch and it prints `SubClass`.
- "Which method actually runs?" → the child's, by dynamic dispatch — **if** it compiles.

▲ **Two more trip-wires in this exact program:** `IOException` needs `import java.io.IOException;`, and the version you were shown had `new SubClass()` written as `newSubClass()` — **always check whether it even compiles before you trace the output.** (That's the golden habit at the top of 23.)

Q5 & Q6. Ionic lifecycle hooks — and how they differ from Angular's ●

Both fire. An Ionic page *is* an Angular component, so all Angular hooks run; Ionic adds **four more** that are driven by the **navigation stack**.

Ionic hook	When it fires
<code>ionViewWillEnter</code>	Just before the page becomes active — every time you enter it
<code>ionViewDidEnter</code>	After the enter transition finishes — the page is visible and interactive
<code>ionViewWillLeave</code>	Just before you navigate away
<code>ionViewDidLeave</code>	After the leave transition finishes

● **The actual difference — and the reason the question is asked**

Ionic keeps pages alive in the DOM. When you push page B on top of page A, A is *not* destroyed — it stays in the nav stack. So when you come back to A:

- `ngOnInit` / `ngOnDestroy` — **do NOT run again.** They fire once, at creation and destruction.
- `ionViewWillEnter` / `ionViewDidEnter` — **run every single time** the page is entered.

	Angular hooks	Ionic hooks
Driven by	Component creation & destruction	Navigation stack entry & exit
<code>ngOnInit</code> on re-entry	✗ Doesn't re-fire	<code>ionViewWillEnter</code> ✓ fires every time
Available in	Any Angular app	Ionic pages only (<code>@ionic/angular</code>)
Use for	One-time setup: DI, form building, static config	Refreshing data, starting/ stopping timers, subscriptions tied to visibility

```
export class StudyListPage implements OnInit {
  ngOnInit() { this.buildForm(); } // once – one-time
  setup
  ionViewWillEnter() { this.loadItems(); } // every entry – data
  stays fresh ✓
  ionViewDidEnter() { this.chart.render(); } // after the animation
  – measure/animate safely
  ionViewWillLeave() { this.timer?.unsubscribe(); } // stop work the user
  can't see
}
```

■ **Say this:** "An Ionic page is an Angular component, so all the Angular hooks fire — but Ionic caches pages in the navigation stack instead of destroying them, so `ngOnInit` does not run again when the user navigates back. That's why anything that must be fresh on every entry — reloading a list after an edit — goes in `ionViewWillEnter`. One-time setup stays in `ngOnInit`. On RoboGebra that exact distinction caused a bug where an edited item still showed stale data on the list page until I moved the reload into `ionViewWillEnter`."

Two more marks if you add them:

- `ionViewCanEnter` / `ionViewCanLeave` were Ionic 3 only — removed in Ionic 4+. Use **Angular route guards** (`CanActivate` / `CanDeactivate`) instead.
- `@ionic/angular` also exposes `ionViewWillUnload`, and Angular's `ngOnDestroy` fires when the page is finally popped off the stack.

Q7. How do you handle offline support in Ionic? ●

▲ **Answer honestly about what RoboGebra actually does** — don't claim a full offline-sync engine if it isn't there. The safe, strong framing: "Here's what we do today, and here's how I'd extend it."

Answer it as **five layers** — this structure alone reads as senior:

1. Detect connectivity

```
import { Network } from '@capacitor/network';

const status = await Network.getStatus();           // { connected,
connectionType }
Network.addListener('networkStatusChange', s =>
this.online$.next(s.connected));
```

`@capacitor/network` works on device **and** web. (`navigator.onLine` alone is unreliable — it only says the interface is up, not that the internet is reachable.)

2. Store data locally — pick the right tool for the size

Need	Use
Small key-values (token, user prefs, last-sync timestamp)	<code>@capacitor/preferences</code>
Larger structured data, cached lists	<code>@ionic/storage-angular</code> (IndexedDB / SQLite driver)
Big relational datasets, querying offline	SQLite (<code>@capacitor-community/sqlite</code>)
Files, images, documents	<code>@capacitor/filesystem</code>
x Not <code>localStorage</code> on native	The OS can clear it, and it's synchronous + size-limited

3. Cache reads — serve from cache, refresh in the background

An **HTTP interceptor** that stores successful `GET` responses and replays them when offline (stale-while-revalidate):

```
intercept(req: HttpRequest<any>, next: HttpHandler) {
  if (req.method !== 'GET') return next.handle(req);
  return next.handle(req).pipe(
    tap(res => res instanceof HttpResponse &&
this.cache.set(req.urlWithParams, res.body)),
    catchError(err => this.cache.has(req.urlWithParams)
      ? of(new HttpResponse({ body:
this.cache.get(req.urlWithParams) })) // offline → cached
      : throwError(() => err))
  );
}
```

4. Queue writes — the outbox pattern

While offline, don't fail the user's action: **write to a local queue, update the UI optimistically**, and replay the queue when the connection returns.

```
async submit(answer: Answer) {
  await this.store.enqueue(answer);      // persisted outbox
  this.applyOptimistically(answer);      // UI updates immediately
}
// on reconnect
this.online$.pipe(filter(Boolean), switchMap(() =>
  this.flushQueue())).subscribe();
```

Each queued item carries a **client-generated id / idempotency key** so a replayed request can't create duplicates, plus a retry count with backoff.

5. Sync & conflict resolution

Track a `lastSyncedAt` timestamp and pull deltas. When the same record changed on both sides, decide the policy explicitly: **last-write-wins** by timestamp (simple), **server-wins** (safest), or **version numbers with a merge prompt** (best UX, most work). Say which one and why.

Plus the two UX points that make the answer complete:

- **Tell the user** — an offline banner, disabled submit buttons, and a "3 items pending sync" indicator. Silent failure is the worst outcome.
- **Web/PWA builds** get a **service worker** (`ng add @angular/pwa`) for precached app shell and assets; native builds already bundle assets locally.

■ **Full answer, out loud (~50 s):** "Five layers. First, detect connectivity with the Capacitor Network plugin and expose it as an observable. Second, persist locally — Capacitor Preferences for small key-values like the token and last-sync time, Ionic Storage or SQLite for larger cached datasets. Third, cache reads through an HTTP interceptor so `GET`s are served from the cache when we're offline and refreshed in the background. Fourth, queue writes in an outbox with idempotency keys and optimistic UI, then replay with retry and backoff on reconnect. Fifth, sync with a `lastSyncedAt` delta pull and an explicit conflict policy — I'd default to last-write-wins by server timestamp and escalate to versioning if the data justified it. And I'd always surface the state in the UI — an offline banner and a pending-sync count — because silent failures destroy trust."

When there's a single round, the interviewer is not screening you for the *next* round — they are making the **hire/no-hire decision live, on that call**. That changes three things:

Multi-round interview	▲ Single decisive round
Technical only; HR comes later	Everything gets compressed into one call — tech + project + behavioural + salary + notice period
A weak answer is recoverable later	A weak answer on your <i>headline</i> skill can end it — no second chance
Depth in one area	Breadth across the whole JD — Java, Angular, Ionic, DB, and how you work
30–45 min	Usually 60–90 min , sometimes with a decision-maker on the call
"Let's see how they do next round"	They need confidence to commit today — so your job is to remove doubt, not just answer correctly

The one mindset shift

Don't just *answer* questions. **Close**. Every answer should reduce their risk in saying yes: name the tech, name the project, name what you owned, and say what you'd do on day one.

Likely time split of a 60–75 minute single round

Minutes	What happens	Your goal
0–5	Intro, small talk, "tell me about yourself"	Land the 60-second script — this sets the frame for the whole call
5–20	Project deep-dive (RoboGebra)	Prove you actually built it — modules, decisions, a challenge

Minutes	What happens	Your goal
20–40	Java + Spring Boot questions	Your riskiest stretch — be crisp, no rambling
40–55	Angular + Ionic questions	Your strongest stretch — be specific, this is where you win
55–65	Maybe a small coding/logic problem or SQL	Think out loud before writing
65–75	Notice period, CTC, location, your questions	Be prepared with exact numbers — hesitation here delays the offer

◆ If they skip the coding problem, that's normal for a single-round services hire — they're weighting project experience and communication more heavily.

2. Your stack map — where you're strong, where to patch

Area	Your evidence	Confidence	Action before the call
Angular	5 yrs, 4 projects, v16 migration, Formly, NgRx, reactive forms	● Strong	Revise 04-angular.md — RxJS operators, lifecycle, change detection
Ionic	RoboGebra (Oct 2024 – present)	● Real but recent	Read 15-ionic-level1.md twice — it's in the job title
Java	Java + Spring Boot backend work	● Medium	05-java.md — collections, HashMap, streams, OOP. Highest-risk area
Spring Boot	REST APIs on RoboGebra; Grails/ Groovy on EasyVisa	● Medium	06-spring-boot.md — annotations, layers, JPA

Area	Your evidence	Confidence	Action before the call
TypeScript/JS	Daily driver	● Strong	01 / 03 — quick skim
DB	MySQL, Postgres, Neo4j, MongoDB	● Medium	Joins, indexes, basic queries + 13-mongodb.md
Coding	—	●	09-coding-problems.md — practise 5 out loud

▲ **The honest risk:** your resume is front-end-heavy, but the role is titled *Java with Angular Ionic*. In a single round they cannot "check Java later." Give Java the most revision time — it is the single biggest thing standing between you and the offer.

3. Your opening answer — memorize this

"Tell me about yourself" (*≈60 seconds — never exceed 90*)

"I'm Priyanka, a Full Stack Developer with **5 years of experience**, currently at Provility Software Solutions in Chennai. I work primarily with **Angular and Ionic on the front end and Java with Spring Boot on the back end**.

My current project is **RoboGebra**, an AI-driven math learning platform built with **Ionic + Angular, Java and MongoDB**. I own several modules end to end — a personalized learning dashboard with progress analytics, a Study Reminder system with scheduling and notification delivery backed by REST APIs, the Study List module with real-time sync, and an interactive quiz module integrated with an AI explanation engine that produces step-by-step solutions in both Tamil and English.

Before that I worked on **Subsea**, a marine operations platform where I built a Schedule-Manager module with Angular and .NET APIs; a **four-repository Angular version migration to v16**; and **EasyVisa**, a visa management system for attorneys where I built the document portal with Angular Formly and nested reactive forms.

So my strength is **end-to-end ownership** — API design through to responsive UI — and this role lines up almost exactly with what I do today, which is why I'm interested."

Why this works in a one-round setting: it names the exact JD stack in sentence one, leads with the Ionic+Java project, shows 4 projects of breadth, and closes on fit — so the interviewer starts the call already leaning yes.

4. The 25 questions most likely to come up (with short answers)

Java (expect 8–10 questions)

1. Four pillars of OOP with a real example?

Encapsulation (private fields + getters/setters), Inheritance (`extends` , code reuse), Polymorphism (overloading = compile-time, overriding = runtime), Abstraction (abstract class / interface hides implementation). *Example: a `NotificationService` interface with email/push implementations chosen at runtime.*

2. Abstract class vs Interface?

Abstract class: can have state, constructors, and both abstract and concrete methods; single inheritance; use for an "is-a" hierarchy with shared code.
Interface: a contract; multiple implementation allowed; since Java 8 can have `default` / `static` methods, since Java 9 `private` methods. Interfaces for capability, abstract classes for shared base behaviour.

3. `ArrayList` vs `LinkedList` ?

`ArrayList` = dynamic array → $O(1)$ random access, $O(n)$ insert/delete in the middle.
`LinkedList` = doubly linked nodes → $O(1)$ insert/delete given the node, $O(n)$ access. **`ArrayList` wins for most real cases; `LinkedList` only for heavy middle insertion.**

4. How does `HashMap` work internally? (near-guaranteed)

Array of buckets; `hashCode()` → hash → bucket index; collisions chained in a

linked list, converting to a **balanced tree above 8 entries in one bucket** (Java 8+); `equals()` distinguishes keys within a bucket. Default capacity 16, load factor 0.75, resizes by doubling. $O(1)$ average, $O(\log n)$ worst case.

5. `equals()` and `hashCode()` contract?

Equal objects **must** produce equal hash codes. Override one → override the other, or objects get lost in `HashMap/HashSet`.

6. `String` vs `StringBuilder` vs `StringBuffer` ?

`String` is immutable (string pool). `StringBuilder` is mutable, **not** thread-safe (faster). `StringBuffer` is mutable and synchronized. Use `StringBuilder` for concatenation in loops.

7. Checked vs unchecked exceptions?

Checked (`IOException` , `SQLException`) — compile-time enforced, catch or declare. Unchecked (`NullPointerException` , `IllegalArgumentException`) — extend `RuntimeException` , programming errors. `Error` (e.g. `OutOfMemoryError`) shouldn't be caught.

8. Java 8 features you've used?

Lambdas, Streams, functional interfaces, `Optional` , default methods, method references, new Date/Time API.

```
List<String> names = employees.stream()
    .filter(e -> e.getSalary() > 50000)
    .map(Employee::getName)
    .sorted()
    .collect(Collectors.toList());
```

9. `==` vs `.equals()` ? `==` compares references (or primitive values); `.equals()` compares content when overridden.

10. What is `Optional` and why? A container that may or may not hold a value — makes "no result" explicit and avoids NPEs. Use `orElse` , `orElseThrow` , `map` , `ifPresent` . Don't use it for fields or parameters.

Spring Boot (3–5 questions)

11. Spring Boot vs Spring?

Spring Boot = Spring + **auto-configuration** + **starter dependencies** + **embedded Tomcat** + **Actuator**. Removes XML/boilerplate so you get a runnable JAR with minimal setup.

12. `@SpringBootApplication` = `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`.

13. `@Controller` vs `@RestController`? `@RestController` = `@Controller` + `@ResponseBody` — methods return data (JSON), not view names.

14. Explain a REST API you built. Use your **Study Reminder APIs** — your resume literally says GET/POST/PUT/DELETE.

```
@RestController
@RequestMapping("/api/reminders")
public class ReminderController {
    private final ReminderService service;
    ReminderController(ReminderService service) { this.service =
service; }

    @GetMapping                                List<Reminder> all() { return
service.findAll(); }
    @GetMapping("/{id}")                      Reminder one(@PathVariable String id)
{ return service.findById(id); }
    @PostMapping                              @ResponseStatus(HttpStatus.CREATED)
Reminder create(@Valid @RequestBody
Reminder r) { return service.save(r); }
    @PutMapping("/{id}")                      Reminder update(@PathVariable String id,
@RequestBody Reminder r) { return service.update(id, r); }
    @DeleteMapping("/{id}")                  @ResponseStatus(HttpStatus.NO_CONTENT)
void delete(@PathVariable String id)
{ service.delete(id); }
}
```

15. **Dependency injection types?** Constructor (recommended — immutable, testable, catches circular deps), setter, field (`@Autowired` on a field — discouraged).

16. **Global exception handling?** `@RestControllerAdvice` + `@ExceptionHandler` returning a consistent error body with the correct HTTP status.

Angular (6–8 questions)

17. Lifecycle hooks in order?

`ngOnChanges` → `ngOnInit` → `ngDoCheck` → `ngAfterContentInit` → `ngAfterContentChecked` → `ngAfterViewInit` → `ngAfterViewChecked` → `ngOnDestroy` .

18. Template-driven vs Reactive forms?

Template-driven: `ngModel` , logic in the template, simple forms. Reactive: `FormGroup` / `FormControl` in the class — synchronous, testable, dynamic. *"I used reactive forms heavily on EasyVisa, including nested forms and custom Angular Formly components."*

19. Observable vs Promise?

Promise = single value, eager, not cancellable. Observable = stream of 0..n values, lazy, cancellable, with operators (`map` , `filter` , `switchMap` , `debounceTime`).

20. `switchMap` vs `mergeMap` vs `concatMap` vs `exhaustMap` ?

`switchMap` cancels the previous inner observable (**typeahead/search**), `mergeMap` runs all concurrently, `concatMap` queues in order, `exhaustMap` ignores new emissions while one is in flight (**login double-click**).

21. Preventing subscription memory leaks?

`async` pipe (best), `takeUntil(this.destroy$)` with a `Subject` completed in `ngOnDestroy` , or manual `unsubscribe()` .

22. What is a service, and why `providedIn: 'root'` ?

A singleton class for shared logic/state/HTTP, injected via DI. `providedIn: 'root'` registers it app-wide and keeps it **tree-shakable**.

23. Lazy loading — what and why?

Feature modules loaded on demand via `loadChildren` , shrinking the initial bundle. *"Every page in the Ionic app is lazy loaded by default."*

24. The Angular migration — how did you approach it? *(it's on your resume, they will ask)*

"We migrated four repositories to Angular 16 — the Ticketing app from v2 and three SSG plugin repos from v9/v4. Approach: upgrade **one major version at a time** using `ng update` and the official update guide, fixing breaking changes at each step — RxJS 6→7 imports, `HttpModule` → `HttpClientModule`, stricter TypeScript, deprecated APIs — then updating third-party libraries to compatible versions and regression-testing module by module. The v2→16 repo was hardest: we effectively rebuilt the build and config layer and migrated the module structure incrementally so features kept working throughout."

Ionic (4–6 questions — full set in 15-ionic-level1.md)

25. The four they almost always ask:

- *What is Ionic and how is it different from Angular?* → Ionic is the mobile UI toolkit + native bridge; Angular is the framework underneath. An Ionic-Angular app **is** an Angular app.
- `ngOnInit` vs `ionViewWillEnter`? → pages are cached in the nav stack, so `ionViewWillEnter` fires on **every** entry — use it to refresh data.
- *Cordova vs Capacitor?* → Capacitor is the modern runtime: Promise API, native projects committed to git, PWA support; Cordova is in maintenance mode.
- *How do you access camera/notifications?* → install a Capacitor plugin, `npx cap sync`, call the Promise API; guard with `Capacitor.isNativePlatform()`.

5. Coding round — the 5 to have cold

Practise **saying your approach out loud before writing** — reported interviews at this company focus on your *thought process*, not just the output.

1. **Reverse a string** (and reverse the word order in a sentence)
2. **Palindrome check**
3. **Find duplicates / first non-repeating character** — HashMap
4. **Second largest element** in an array — single pass, no sorting
5. **FizzBuzz** / count vowels / sum of digits

One Java 8 stream question — *"group employees by department"*:

```
Map<String, List<Employee>> byDept = employees.stream()  
    .collect(Collectors.groupingBy(Employee::getDepartment));
```

One SQL — *"second highest salary"*:

```
SELECT MAX(salary) FROM employee WHERE salary < (SELECT MAX(salary) FROM  
employee);
```

- ◆ They may share a CoderPad/HackerRank link or ask you to type in a shared doc **without autocomplete**. Practise writing a Java class with `public static void main` by hand.

6. Project walkthrough — the 3-minute RoboGebra script

They will say *"walk me through your current project."* In a single round this is the **highest-weight answer of the call**. Structure: **What** → **Stack** → **My modules** → **A challenge** → **Impact**.

What: RoboGebra is an AI-driven math learning platform for students — it explains problems step by step instead of just giving the answer.

Stack: Ionic + Angular front end, Java/Spring Boot REST APIs, MongoDB — running as both a web app and a mobile app through Capacitor.

My modules: the personalized learning dashboard (progress analytics, learning timelines, short and detailed answers); the Study Reminder system (flexible scheduling, notification delivery, full CRUD REST APIs); the Study List module (create/organize/track goals with real-time sync); the quiz module (dynamic questions, instant evaluation, solution walkthroughs); and the bilingual Tamil/English AI explanation views with formula breakdowns.

Challenge: (*pick one and tell it in STAR — e.g. keeping the Study List consistent across screens without flicker, or rendering step-by-step math reliably.*)

Impact: students got consistent reminders and a clear view of their progress, which was the retention goal for that release.

Be ready for the follow-ups: *"Why MongoDB and not MySQL?"* · *"How did you secure the APIs?"* (JWT + Spring Security) · *"How did the Angular app talk to the*

Java service?" (HttpClient service layer + interceptor, DTOs, error handling) · *"What was hard about Ionic specifically?"* (lifecycle caching, Shadow DOM styling, device-only plugin code) · *"What would you do differently?"* (have one honest answer — it signals seniority).

7. ★ Closing the single round — the last 10 minutes

This is what most candidates waste. With no second round, the ending *is* the decision.

When they ask "Do you have any questions?" — never say no

Pick 2–3:

1. "Is this role for an internal product or a client project — and which domain?"
2. "How is the team split between the Angular/Ionic front end and the Java services?"
3. "Is the mobile app on Capacitor or Cordova, and which Ionic/Angular versions is the project on?"
4. "What would success in the first three months look like?"
5. "Since this is the only technical round — is there anything about my background you'd like me to clarify before we finish?" ← **this one is gold in a single-round interview.** It surfaces doubts while you can still fix them.

Your closing statement (say it, don't wait to be asked)

"Thank you — this has been a good conversation. Just to close from my side: the stack here is exactly what I work on today — Angular and Ionic on the front end, Java and Spring Boot behind it — so there's no ramp-up on the technology. I'm genuinely interested, my notice period is [X], and I can start the onboarding formalities as soon as you need. Is there anything else you need from me?"

If you get stuck on a question

Never bluff — reported interviewers here probe *how you think*.

"I haven't worked on that directly. My understanding is *[whatever you do know]*, and if I hit it on the job I'd *[how you'd find out]*. Where I have done something close is *[nearest real experience]*."

That answer scores better than a confident wrong one, and it protects your credibility on everything else you said.

8. Onboarding readiness — have this ready BEFORE the call

Since clearing this round goes straight to onboarding, HR may ask for details **on the same call or within hours**. Fumbling here delays or weakens your offer.

Numbers to have on a sticky note in front of you

Item	Your answer
Current CTC (fixed + variable)	_____
Expected CTC	_____ (give a number, not "as per company standards")
Notice period	_____ (and whether buyout/early release is possible)
Earliest joining date	_____
Current location / work mode preference	Chennai / _____
Reason for change	Growth + full-stack ownership — never criticize your current employer
Any other offers in hand	Be honest but don't bluff a fake offer

On expected CTC: state a number with a small range and anchor it on scope, e.g. *"Based on my 5 years across Angular, Ionic and Java, and the end-to-end ownership in this role, I'm looking at ₹- LPA fixed. I'm open to discussing the overall structure."* If they push first with "what's your expectation?", answering with a number is fine — deflecting twice in a one-round process just stalls things.

Documents typically requested at onboarding (Indian IT)

Keep scanned PDFs in one folder so you can send within the hour:

- [] Updated resume (the one you applied with)
- [] Aadhaar + PAN
- [] Passport-size photo
- [] All education certificates + marksheets (B.Tech, 12th, 10th)
- [] Offer letter, appointment letter & increment/promotion letters from Provility (and any earlier employer)
- [] **Last 3 months' payslips**
- [] **Form 16** / last year's ITR
- [] Bank statement showing salary credits (last 3–6 months)
- [] Relieving & experience letters from previous employers
- [] UAN / PF number
- [] Two professional references (name, designation, official email, phone) — **ask them first**

After the call — same day

- [] Send a short thank-you email to HR/the interviewer (4–5 lines: thanks, one thing you enjoyed discussing, reconfirm interest and notice period)
- [] Ask HR for the **expected timeline for the offer** and what documents to prepare
- [] Do **not** resign until you have the written offer letter in hand

9. Virtual interview logistics ✓

- [] Test **camera, mic and the meeting link 15 minutes early**; keep a phone hotspot as backup
- [] Quiet room, plain background, front lighting, laptop at eye level
- [] Keep only these open: this file, [15-ionic-level1.md](#), an editor for the coding round
- [] Resume printed or on a second screen — **they read from it**, so know every line of it

- [] Pen and paper for the coding problem
 - [] Dress as for an office; join **2 minutes early**, not 10
 - [] Speak slightly slower than normal — audio lag eats clarity
 - [] Have your CTC/notice-period sticky note visible (Section 8)
 - [] Budget **90 minutes** free after the start time — don't schedule anything behind it
-

10. Two-day crash plan

Day 1 — Java first, because it's your gap

- Morning: [05-java.md](#) — OOP, collections, HashMap internals, exceptions, streams (2.5 h)
- Afternoon: [06-spring-boot.md](#) — annotations, REST layers, DI, exception handling (1.5 h)
- Evening: [09-coding-problems.md](#) — write the 5 problems in Java by hand (1 h)

Day 2 — Angular + Ionic + delivery

- Morning: [15-ionic-level1.md](#) — read fully, then answer Sections A, D, E from memory (1.5 h)
 - Midday: [04-angular.md](#) — lifecycle, RxJS operators, forms, lazy loading (2 h)
 - Afternoon: rehearse the **self-intro**, the **RoboGebra walkthrough** and the **closing statement** out loud, on camera, twice each (1 h)
 - Evening: Section 4 rapid answers + fill in the Section 8 numbers + logistics checklist (45 min)
-

11. Final reminders

1. **One round = no recovery.** Prepare breadth, not depth — they'll touch Java, Angular, Ionic, DB and behavioural in one sitting.
2. **Lead every answer with your project.** "Yes — on RoboGebra we..." beats a textbook definition every time.

3. **Java is the gap.** Front-end fluency won't carry a role titled *Java with Angular* Ionic.
4. **Ionic is your edge.** Very few Angular developers can discuss `ionViewWillEnter`, `cap sync` and Shadow DOM theming. You can.
5. **Close the call deliberately** — ask the "anything you'd like me to clarify?" question, state your notice period, and confirm interest.
6. **Have your CTC, notice period and documents ready** — onboarding starts immediately after a yes.
7. **Numbers you own:** 5 years · 4 major projects · 4 repos migrated to Angular 16 · 5 modules on RoboGebra.

You've built exactly what this JD is asking for. Go and close it.

Java — SOLID Principles & Design Patterns (Easy Version)

Interview Q&A for a **Full Stack Developer (Java + Spring Boot, 5 years)**. This is the round where interviewers separate "writes code" from "designs code". Examples are tied to your real projects — **RoboGebra** (AI math explanation engine, quiz module), **EasyVisa** (document portal), **Subsea** (Schedule-Manager).

How this gets asked: rarely as "define SRP". Usually as "*Which SOLID principle does this code violate?*", "*Which design pattern have you actually used?*", or "*How would you design X so adding a new type doesn't break existing code?*"
Prepare **one real story per principle/pattern** — that is what scores.

Every topic below follows the same shape:

Easiest way to remember → **bad code** → **good code** → **real-world example** → **diagram** → **Easy memory box**.

● About the "In your RoboGebra project" blocks

Throughout this file, after an interview answer you'll see a ● **In your RoboGebra project** block containing **real code from your own repositories** (`/Users/safi/workspace/robogebra-workspace`).

Use them like this ★ 1. Give the DEFINITION (the interview answer above) 2. Then "in our codebase..." ← the ● block ★ 3. Then ONE trade-off or trap

Three sentences. That is the difference between someone who read the docs and someone who has shipped it. Full index: **39 — RoboGebra Code Examples**.

Table of contents

- [Part 1 — SOLID Principles](#)
- [Part 2 — Design Patterns: fundamentals](#)
- [Part 3 — Creational patterns](#)
- [Part 4 — Structural patterns](#)
- [Part 5 — Behavioural patterns](#)
- [Part 6 — Patterns in the JDK, Spring & Angular](#)
- [Part 7 — Scenario & "spot the violation" questions](#)
- [Part 8 — Rapid-fire one-liners](#)
- [Quick Revision Sheet](#)

Part 1 — SOLID Principles

Q: What is SOLID and why does it matter?

The easiest way to remember — the **word SOLID itself**:

S	→ Single Responsibility	→ ONE job per class
O	→ Open/Closed	→ ADD new code, don't EDIT old code
L	→ Liskov Substitution	→ the child must not SURPRISE you
I	→ Interface Segregation	→ many SMALL interfaces, not one fat one
D	→ Dependency Inversion	→ depend on INTERFACES, not concrete classes

Letter	Principle	One-line meaning
S	Single Responsibility	a class should have only one reason to change
O	Open/Closed	open for extension, closed for modification
L	Liskov Substitution	a subtype must be usable anywhere its parent is, with no surprises

Letter	Principle	One-line meaning
I	Interface Segregation	many small, focused interfaces beat one fat interface
D	Dependency Inversion	depend on abstractions, not concrete implementations

Why it matters — the sentence to say

"Following SOLID means a new requirement usually means adding a class, not editing five existing ones — which keeps the regression surface small and unit tests easy, because dependencies can be mocked."

<u>WITHOUT SOLID</u>	<u>WITH SOLID</u>
New requirement	New requirement
↓	↓
edit 5 tested classes	ADD 1 new class ✓
↓	↓
re-test everything	old code untouched → nothing to re-test
↓	
something else breaks ▲	

Easy memory

S O L I D
One job | Add don't edit | No surprises | Small interfaces | Use interfaces

The payoff: a new feature = a NEW CLASS, not an edit to five old ones ★

Q: Explain the Single Responsibility Principle (SRP).

The easiest way to remember:

A class should have only ONE REASON TO CHANGE.

Test: "Can I name two different PEOPLE who would ask me to change this class?"

If yes → it's doing too much.

The violation

```
class InvoiceService {  
    double calculateTax(Invoice inv) { /* tax rules */ }  
    void saveToDatabase(Invoice inv) { /* JDBC */ }  
    void sendEmail(Invoice inv)      { /* SMTP */ }  
}
```

Three completely different people can force a change here:

The ACCOUNTANT	changes the tax rules	→ edit this class
The DBA	changes the schema	→ edit this class
MARKETING	changes the email template	→ edit this class

THREE reasons to change = SRP violated ▲

And they collide: a marketing tweak to the email risks breaking the tax calculation, because they live in the same file and the same tests.

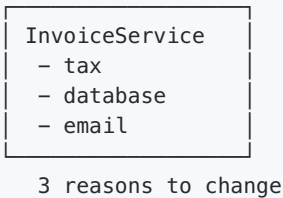
The fix — separate the axes of change

```
class TaxCalculator      { double calculate(Invoice inv) { ... } }
class InvoiceRepository  { void save(Invoice inv) { ... } }
class InvoiceNotifier    { void notifyCustomer(Invoice inv) { ... } }

@Service
class InvoiceService {                                     // now it only
ORCHESTRATES
    private final TaxCalculator tax;
    private final InvoiceRepository repo;
    private final InvoiceNotifier notifier;
    // constructor injection...

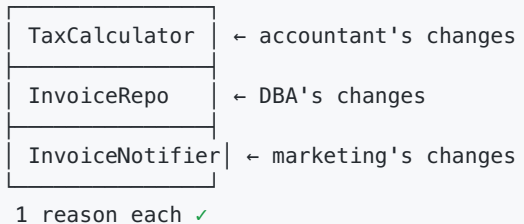
    void process(Invoice inv) {
        inv.setTax(tax.calculate(inv));
        repo.save(inv);
        notifier.notifyCustomer(inv);
    }
}
```

BEFORE



→

AFTER



Real-world idea: a **restaurant**.

Bad → one person cooks, serves, bills and cleans
Change the billing system → the cooking stops

Good → chef, waiter, cashier, cleaner ✓
Change the billing system → only the cashier is affected

Your project answer (RoboGebra)

"Our first cut of the explanation engine had one *ExplanationService* that built the AI prompt, called the model, parsed the step-by-step response, and did the Tamil/English translation. Any prompt tweak risked breaking translation. We split it

into *PromptBuilder*, *AIClient*, *ExplanationParser* and *TranslationService* — the service just orchestrates. After that, adding bilingual output only touched one class."

▲ **Common trap:** SRP does **not** mean "one method per class". Over-splitting creates anaemic classes and a maze of indirection. The real test is the "two different people" question above.

Easy memory

SRP = ONE reason to change ★

Test: can TWO different people ask me to change this class?
accountant (tax) + DBA (schema) + marketing (email) = 3 reasons ▲

Restaurant: chef | waiter | cashier — not one person doing all of it
▲ Not "one method per class" — don't over-split

Q: Explain the Open/Closed Principle (OCP).

The easiest way to remember:

OPEN for extension → you CAN add new behaviour
CLOSED for modification → by ADDING code, never by EDITING tested code

The smell that tells you it's violated: a growing if/else or switch. ★

The violation

```
class QuestionEvaluator {
    boolean evaluate(Question q, String answer) {
        if (q.getType().equals("MCQ")) { /* ... */ }
        else if (q.getType().equals("NUMERIC")) { /* ... */ }
        else if (q.getType().equals("MATCH")) { /* ... */ } // keeps
growing ▲
        return false;
    }
}
```

New question type



EDIT this method



re-test EVERY existing type (you touched their file)



risk of breaking working code ▲

The fix — polymorphism (Strategy + OCP together)

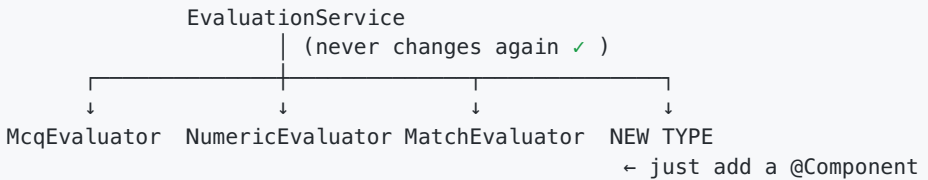
```
interface QuestionEvaluator {
    boolean supports(QuestionType type);
    boolean evaluate(Question q, String answer);
}

@Component
class McqEvaluator implements QuestionEvaluator {
    public boolean supports(QuestionType t) { return t ==
    QuestionType.MCQ; }
    public boolean evaluate(Question q, String answer) { /* ... */ }
}

@Service
class EvaluationService {
    private final List<QuestionEvaluator> evaluators;    // ★ Spring
    injects ALL implementations

    EvaluationService(List<QuestionEvaluator> evaluators)
    { this.evaluators = evaluators; }

    boolean evaluate(Question q, String answer) {
        return evaluators.stream()
            .filter(e -> e.supports(q.getType()))
            .findFirst()
            .orElseThrow(() -> new
    UnsupportedQuestionTypeException(q.getType()))
            .evaluate(q, answer);
    }
}
```



A new question type = **one new @Component**. `EvaluationService` is never touched again.

Real-world idea: a **power socket**. You don't rewire the house to plug in a new appliance — the socket is a fixed contract, and any device that fits can be added.

◆ **High-scoring detail:** "Spring injecting a `List<T>` or `Map<String, T>` of all beans implementing an interface is the practical way to do OCP in Spring Boot." Interviewers love this.

Easy memory

OCP = ADD new classes, never EDIT tested ones ★

Smell: a growing if/else or switch on a "type" field ▲

Fix: an interface + one @Component per type
+ Spring injects `List<Interface>` ★

Power socket: plug in a new device, don't rewire the house.

Q: Explain the Liskov Substitution Principle (LSP).

The easiest way to remember:

A CHILD must work ANYWHERE the PARENT works – with no surprises.

If you need an `instanceof` check to use a subclass safely, LSP is broken. ★

The classic violation — Square extends Rectangle

```
class Rectangle {
    protected int width, height;
    void setWidth(int w) { this.width = w; }
    void setHeight(int h) { this.height = h; }
    int area() { return width * height; }
}

class Square extends Rectangle {                // "a square IS-A rectangle"
    - mathematically true
    @Override void setWidth(int w) { this.width = w; this.height = w; }
    @Override void setHeight(int h) { this.width = h; this.height = h; }
}
```

```
void resize(Rectangle r) {
    r.setWidth(5);
    r.setHeight(4);
    assert r.area() == 20;    // ✓ Rectangle → 20
                             //
    ▲ Square    → 16 (setHeight also changed the width!)
}
```

Caller wrote code that works for Rectangle
↓ passes a Square instead
Silently WRONG answer – no exception, no warning ▲

★ The lesson: **"is-a" in mathematics is not the same as "is-a" in behaviour.**

A more realistic Java one

```
class ReadOnlyDocumentStore implements DocumentStore {
    public Document get(String id) { ... }

    public void save(Document d) {
        throw new UnsupportedOperationException();    // ✗ LSP violation
    }
}
```

The interface PROMISES save() works.
This implementation BREAKS that promise.
↓
Every caller must now know the CONCRETE type → the abstraction LIES ▲

This is exactly why `Arrays.asList()` — which returns a fixed-size list that throws on `add()` — is a famous LSP smell inside the JDK itself.

The rules a subclass must respect

1. PRECONDITIONS may not be STRENGTHENED
parent accepts any int → child must not reject negatives
2. POSTCONDITIONS may not be WEAKENED
parent guarantees a sorted list → child must still sort
3. INVARIANTS must be preserved
parent guarantees balance ≥ 0 → child must too
4. No NEW CHECKED EXCEPTIONS the parent didn't declare

Real-world idea: **a job replacement.**

Your manager hires a replacement for you.
If the replacement refuses half your tasks, or does them differently
and breaks things — the substitution FAILED. That is LSP. ★

Fix: prefer composition, or split the interface (`ReadableStore` / `WritableStore` — which is ISP).

Easy memory

LSP = the child must work anywhere the parent works, with NO SURPRISES ★

Violation smells:

```
throw new UnsupportedOperationException() ▲  
an instanceof check before using a subclass ▲
```

Square extends Rectangle → `setHeight` also changes the width → wrong area
`Arrays.asList().add()` → the JDK's own LSP smell

Fix: composition, or split the interface (→ ISP)

Q: Explain the Interface Segregation Principle (ISP).

The easiest way to remember:

No class should be FORCED to implement methods it doesn't need.

Many SMALL interfaces > one FAT interface. ★

The violation

```
interface DocumentOperations {
    void upload(MultipartFile f);
    void download(String id);
    void delete(String id);
    void digitallySign(String id);
    void ocrExtract(String id);
}

class SimpleAttachmentHandler implements DocumentOperations {
    public void digitallySign(String id) { throw new
UnsupportedOperationException(); } // ✗
    public void ocrExtract(String id)    { throw new
UnsupportedOperationException(); } // ✗
    // ...
}
```

A simple attachment doesn't need signing or OCR.
But the fat interface FORCES it to implement them.

↓

It throws UnsupportedOperationException

↓

which is ALSO an LSP violation ▲

The fix — split by capability

```
interface DocumentStorage { void upload(MultipartFile f); void
download(String id); }
interface Signable        { void digitallySign(String id); }
interface OcrCapable      { void ocrExtract(String id); }

class SimpleAttachmentHandler implements DocumentStorage
{ ... }                // ✓ 2 methods
class VisaFormHandler implements DocumentStorage, Signable, OcrCapable
{ ... }                // ✓ all of them
```

FAT interface

```
upload
download
delete
digitallySign ✗
ocrExtract ✗
```

→

SEGREGATED

```
DocumentStorage
Signable
OcrCapable
```

← everyone

← only those who sign

← only those who OCR

Real-world idea: a **restaurant menu**.

Bad → ONE menu forcing every customer to order a starter, main, dessert AND drink

Good → separate menus; you order only what you want ✓

Or: a printer/scanner/fax **all-in-one interface** forces a simple printer to implement `fax()` and throw.

Your project answer (EasyVisa)

"In the EasyVisa document portal, different panel types supported different capabilities — some documents needed attorney signature, some were plain uploads. Rather than one interface with `UnsupportedOperationException` holes, we kept a small `DocumentStorage` contract and mixed in capability interfaces. It also made the Angular side simpler, because the API contract per document type was honest."

◆ **ISP and LSP are cousins:** a fat interface forces implementations to throw `UnsupportedOperationException`, which is an LSP violation. Segregating the interface removes the cause.

Easy memory

ISP = don't force a class to implement methods it doesn't use ★

Smell: `UnsupportedOperationException` in an implementation ▲

Fix: split into small CAPABILITY interfaces
(`Signable`, `OcrCapable` — mix them in as needed)

ISP violated → causes an LSP violation. Fixing ISP fixes both ★

Q: Explain the Dependency Inversion Principle (DIP).

The easiest way to remember:

Depend on INTERFACES, not on concrete classes.

The smell: the word `new` inside a service. ★

Two formal parts:

1. High-level modules must not depend on low-level modules.
BOTH should depend on abstractions.
2. Abstractions must not depend on details.
Details must depend on abstractions.

The violation

```
@Service
class StudyListService {
    private final MongoStudyListRepository repo = new
MongoStudyListRepository(); // x concrete + new
    private final EmailNotifier notifier = new
EmailNotifier();           // x
}
```

Problems:

- you cannot unit-test it without a real MongoDB ▲
- switching to Postgres means EDITING this service
- switching to SMS means EDITING this service

The fix

```
public interface StudyListRepository {
    StudyList save(StudyList s);
    Optional<StudyList> findById(String id);
}

public interface Notifier { void notify(String userId, String message); }

@Service
public class StudyListService {
    private final StudyListRepository repo;
    private final Notifier notifier;

    public StudyListService(StudyListRepository repo, Notifier notifier)
    { // constructor injection ★
        this.repo = repo;
        this.notifier = notifier;
    }
}
```

Why it is called "INVERSION" ★ — the part people can't explain

BEFORE (normal dependency)

StudyListService (high level) —depends on→ MongoRepository (low level)

AFTER (inverted)

StudyListService —depends on→ StudyListRepository (interface)

 ▲
 | implements
 MongoRepository
 (low level now depends UPWARD)

The INTERFACE belongs to the DOMAIN (the high-level module).
The database adapter implements it.

So the arrow of dependency now points FROM the detail TOWARD the domain.
That reversal is the "inversion". ★

Real-world idea: a **wall socket**.

Your laptop doesn't depend on "the Chennai power plant".
Both depend on the SOCKET STANDARD.
Change the power source (solar, generator) — the laptop never changes ✓

Easy memory

DIP = depend on INTERFACES, inject them ★

Smell: `new ConcreteClass()` inside a service ▲

Fix: an interface + constructor injection

"Inversion" = the interface belongs to the DOMAIN;
the database adapter depends on the DOMAIN, not vice versa

★

Wall socket: laptop and power plant both depend on the STANDARD.

● In your RoboGebra project — DIP, for real ★

Files: `common/service/event/EventTrackingService.java` + two implementations

```
// The ABSTRACTION belongs to the DOMAIN ★
public interface EventTrackingService {
    void trackUserLogin(User user);
    void trackPaymentEvent(SubscriptionInstanceWithItems subscription);
}

// The DETAILS depend on the abstraction – the arrow points UPWARD ★
@Service @Profile({"local","dev"})
@ConditionalOnProperty(prefix="posthog", name="enabled",
    havingValue="false", matchIfMissing=true)
public class LoggingEventTrackingService implements EventTrackingService
{ ... }

@Service @Profile({"prod"})
@ConditionalOnProperty(prefix="posthog", name="enabled",
    havingValue="true")
public class PostHogEventTrackingService implements EventTrackingService
{ ... }
```

```
// Every caller depends on the INTERFACE – never on PostHog ★
@Service
public class UserService {
    private final EventTrackingService eventTracking;    // ★ the
    abstraction
    public UserService(EventTrackingService eventTracking) { ... }    //
    constructor injection
}
```

WITHOUT DIP
 UserService → PostHogClient
 (interface)

✗ can't test without a key
 ✗ dev machines need PostHog
 PostHogEventTrackingService

✗ swapping vendors = edit
 the DOMAIN ★)
 every call site

WITH DIP ★
 UserService → EventTrackingService

▲
 | implements

(the DETAIL depends on

■ "Our analytics is the cleanest DIP example in the codebase. Services depend on *EventTrackingService*, never on PostHog. In dev the implementation just logs, in production it posts to PostHog, and the switch is a Spring profile plus a property. The practical payoff is that nobody needs a PostHog key on their laptop, tests inject a stub with no mocking framework, and if we changed analytics vendor tomorrow it's one new class — no call site changes."

★ And note **where the interface lives**: in `common/service/event`, with the domain — **not** in a vendor package. That's the "inversion": the low-level detail depends on the high-level module's contract, not the other way round.

Q: What's the difference between Dependency Inversion, Dependency Injection, and IoC?

A very common follow-up — get this crisp:

```
DIP = the PRINCIPLE    → "depend on abstractions"
IoC = the PATTERN      → "the framework controls the flow, not your code"
DI  = the TECHNIQUE    → "dependencies are handed to you"
```

Term	What it is
Dependency Inversion (DIP)	a design principle — depend on abstractions
Inversion of Control (IoC)	a broader pattern — the framework controls flow and object creation ("don't call us, we'll call you")
Dependency Injection (DI)	a technique , one implementation of IoC — dependencies are handed to an object rather than created by it

DIP (principle: use interfaces)
 ↑ makes it work cleanly
 DI (technique: constructor injection)
 ↑ is one way of doing
 IoC (pattern: the container is in charge)

"Spring's `ApplicationContext` is an IoC container; `@Autowired` / constructor injection is DI; and the reason we code to interfaces so it works cleanly is DIP."

Easy memory

DIP = PRINCIPLE (use interfaces)
 IoC = PATTERN (container is in charge)
 DI = TECHNIQUE (constructor injection)

One sentence: "ApplicationContext = IoC, constructor injection = DI, coding to interfaces = DIP." ★

Q: Which injection type should you use and why?

CONSTRUCTOR injection. ★

1. Dependencies can be `final` → IMMUTABLE, thread-safe
2. FAILS FAST at startup if a dependency is missing
3. TESTABLE without Spring:

```
new StudyListService(mockRepo, mockNotifier);
```

 ✓
4. EXPOSES SRP violations — a 9-argument constructor screams "this class does too much" ★ (field injection HIDES that)

Since Spring 4.3, **@Autowired** is optional if the class has a single constructor.

Field injection

✗ can't be final | hides dependencies | needs reflection to test

Setter injection ● only for genuinely optional dependencies

Constructor ✓ everything above

Easy memory

CONSTRUCTOR injection ★

final ✓ | fail-fast ✓ | testable without Spring ✓ | exposes bloat ✓

@Autowired not needed for a single constructor (4.3+)

Q: Where does SOLID conflict with reality?

A genuinely senior answer:

"SOLID is a set of heuristics, not laws. Applying OCP everywhere leads to speculative abstraction — interfaces with exactly one implementation forever. My rule is the rule of three: hard-code the first case, note the duplication on the second, abstract on the third, when I actually know what varies. Premature abstraction is harder to remove than duplication."

1st case → just write it

2nd case → notice the duplication, don't abstract yet

3rd case → NOW abstract – you finally know what actually varies ★

★ Why this scores: it shows you can *apply judgement*, not just recite principles.

An interface with one implementation forever is ceremony, not design.

Easy memory

RULE OF THREE ★

1st → write it | 2nd → notice it | 3rd → abstract it

"Premature abstraction is harder to remove than duplication."

Part 2 — Design Patterns: fundamentals

Q: What is a design pattern? What are the categories?

The easiest way to remember:

A design pattern = a NAMED, REUSABLE SOLUTION to a recurring design problem.

A template – not copy-paste code.

The Gang of Four (GoF) catalogued 23, in three categories:

- CREATIONAL → how objects are CREATED
- STRUCTURAL → how objects are COMPOSED
- BEHAVIOURAL → how objects COMMUNICATE

Category	Concern	Key patterns
Creational	<i>how objects are created</i>	Singleton, Factory Method, Abstract Factory, Builder, Prototype
Structural	<i>how objects are composed</i>	Adapter, Decorator, Facade, Proxy, Composite, Bridge, Flyweight
Behavioural	<i>how objects communicate</i>	Strategy, Observer, Template Method, Chain of Responsibility, Command, Iterator, State, Mediator, Visitor

Real-world idea: patterns are like **recipes**. "Biryani" is a named approach with known steps — but every cook adapts it to their kitchen. You don't copy a photograph of biryani; you follow the method.

Easy memory

Creational = CREATION (Singleton, Factory, Builder, Prototype)
Structural = COMPOSITION (Adapter, Decorator, Facade, Proxy, Composite)
Behavioural = COMMUNICATION (Strategy, Observer, Template, Chain, Command, State)

C-S-B: Create → Compose → Communicate ★

Part 3 — Creational patterns

Q: Explain the Singleton pattern. How do you make it thread-safe?

The easiest way to remember:

Singleton = EXACTLY ONE instance for the whole application,
with a global access point.

Used for caches, configuration holders, connection pools and loggers.

Real-world idea: the **Prime Minister of a country**. There is exactly one, and everybody refers to the same one.

The broken version (not thread-safe)

```
class Config {  
    private static Config instance;  
    private Config() {}  
    public static Config getInstance() {  
        if (instance == null) instance = new Config();    // ✗ race  
        condition  
        return instance;  
    }  
}
```

Thread A: instance == null? YES
Thread B: instance == null? YES ← both passed the check!
Thread A: creates instance #1
Thread B: creates instance #2 → TWO singletons ▲

1. Eager initialisation — simplest, thread-safe by the classloader

```
class Config {  
    private static final Config INSTANCE = new Config();  
    private Config() {}  
    public static Config getInstance() { return INSTANCE; }  
}
```

Use when creation is cheap. Class loading is guaranteed thread-safe by the JVM.

2. Double-checked locking with **volatile** — lazy + thread-safe

```
class Config {  
    private static volatile Config instance;           // ★ volatile is  
MANDATORY  
    private Config() {}  
  
    public static Config getInstance() {  
        if (instance == null) {                        // 1st check – no lock,  
fast path ▲  
            synchronized (Config.class) {  
lock  
                if (instance == null) {                // 2nd check – under the  
lock  
                    instance = new Config();  
                }  
            }  
        }  
        return instance;  
    }  
}
```

Why **volatile?** ★ **new Config()** is not atomic — it is *allocate memory, run the constructor, assign the reference*. Without **volatile**, the JIT or CPU may reorder so the reference is assigned **before** the constructor finishes. Another thread then sees a non-null but **partially constructed** object and uses it.

Allowed reordering WITHOUT volatile:

1. allocate memory
3. assign the reference ← instance is now NON-NULL...
2. run the constructor ← ...but the object isn't ready ▲

Thread B: if (instance == null) → false → uses a half-built object ▲

3. Bill Pugh / holder idiom — lazy, thread-safe, no locking ★ best classic

```
class Config {  
    private Config() {}  
    private static class Holder { static final Config INSTANCE = new  
Config(); }  
    public static Config getInstance() { return Holder.INSTANCE; }  
}
```

The inner class isn't loaded until `getInstance()` is first called — so it is lazy — and class loading gives thread safety for free, with **no synchronization cost at all**.

4. Enum singleton — Joshua Bloch's recommendation ★ safest

```
public enum Config {  
    INSTANCE;  
    public void load() { ... }  
}
```

The only version immune to reflection **and** serialization attacks.

How to rank them out loud

```
Eager    → cheap object, always needed  
DCL      → the classic interview answer (volatile is the point ★ )  
Holder   → lazy + fast + no locks → the best classic approach ★  
Enum     → the SAFEST of all ★
```

Easy memory

Singleton = exactly ONE instance (the Prime Minister)

Eager | DCL (volatile MANDATORY ★) | Bill Pugh holder ★ | enum (safest) ★

volatile in DCL prevents seeing a HALF-CONSTRUCTED object
(allocate → assign → construct is a legal reordering ▲)

Q: How can a Singleton be broken, and how do you defend it?

1. REFLECTION → constructor.setAccessible(true) → a second instance ▲
2. SERIALIZATION → each deserialize creates a new object ▲
3. CLONING → clone() makes a copy ▲
4. MULTIPLE CLASSLOADERS → one instance per classloader

Attack	Defence
Reflection	throw from the constructor if the instance already exists; or use an enum
Serialization	implement readResolve() returning the existing instance; or use an enum
Cloning	override clone() to throw CloneNotSupportedException
Multiple classloaders	usually acceptable; otherwise load explicitly

★ Only ENUM is immune to all of them, for free.

Easy memory

Reflection | Serialization | Cloning | Classloaders

enum defeats ALL of them without any extra code ★

Otherwise: guard the constructor + readResolve() + throw from clone()

Q: Is a Spring @Service bean a Singleton pattern?

N0 – it is singleton SCOPE, which is different. ★

GoF Singleton	Spring singleton scope
one instance per JVM/classloader	one instance per ApplicationContext ★
enforced by a private constructor	managed by the container; the class stays a normal POJO

GoF Singleton	Spring singleton scope
hard to test or mock	trivially mockable — it's just a class
global static access	injected, so dependencies are explicit

Two ApplicationContexts in one JVM → TWO instances of the same @Service ★

And the point that matters in production:

▲ Singleton beans are NOT automatically thread-safe.

ONE instance serves ALL concurrent requests, so a mutable instance field on a @Service is a classic production bug.
→ Keep them STATELESS.

Easy memory

Spring singleton = per ApplicationContext (not per JVM) ★
Container-managed POJO → easily mockable (unlike GoF Singleton)

▲ NOT thread-safe by itself → keep @Service beans STATELESS

Q: Explain the Factory Method pattern.

The easiest way to remember:

Factory = "you tell me WHAT you want; I decide WHICH class to create."

It removes `new` from the caller.

```

public interface ExplanationGenerator { Explanation generate(MathProblem
p); }

class AlgebraExplanationGenerator    implements ExplanationGenerator
{ ... }
class GeometryExplanationGenerator  implements ExplanationGenerator
{ ... }
class CalculusExplanationGenerator   implements ExplanationGenerator
{ ... }

@Component
public class ExplanationGeneratorFactory {
    private final Map<Topic, ExplanationGenerator> registry;

    public ExplanationGeneratorFactory(List<ExplanationGenerator>
generators) {
        this.registry = generators.stream()
            .collect(Collectors.toMap(ExplanationGenerator::topic,
Function.identity()));
    }

    public ExplanationGenerator forTopic(Topic topic) {
        var gen = registry.get(topic);
        if (gen == null) throw new IllegalArgumentException("No generator
for " + topic);
        return gen;
    }
}

```

```

Caller:  factory.forTopic(ALGEBRA)
          ↓
        the factory looks up the registry
          ↓
        returns AlgebraExplanationGenerator ✓

```

The caller NEVER writes `new`, and never knows the concrete class.

Real-world idea: a **pizza counter**. You say "one margherita"; the kitchen decides which chef and which oven. You never walk in and start making it yourself.

Benefit: callers depend only on the interface, so adding a new topic doesn't touch the caller — that is **OCP**.

Simple Factory vs Factory Method: a "simple factory" is one class with a `switch` — not a GoF pattern, but common and perfectly fine. **Factory Method** puts creation behind an overridable method so subclasses decide the type.

Easy memory

Factory = the caller says WHAT, the factory decides WHICH class ★
Removes `new` from the caller → satisfies OCP

Pizza counter: you order, the kitchen decides who cooks it
Spring version: inject List<Interface> → build a Map<Type, Impl> registry ★

Q: Factory Method vs Abstract Factory?

The easiest way to remember:

Factory Method → creates ONE product
Abstract Factory → creates a FAMILY of related products ★

	Factory Method	Abstract Factory
Creates	one product	a family of related products
Mechanism	inheritance (override a method)	composition (a factory object with several create methods)
Example	<code>createExplanation()</code>	<code>UiFactory</code> → <code>createButton()</code> , <code>createCheckbox()</code>

```
interface ReportFactory {                                // Abstract Factory – a FAMILY
    Header createHeader();
    Body    createBody();
    Footer  createFooter();
}
class PdfReportFactory implements ReportFactory { ... }
class ExcelReportFactory implements ReportFactory { ... }
```

PdfReportFactory → PDF header + PDF body + PDF footer ✓ consistent
ExcelReportFactory → Excel header + Excel body + Excel footer ✓

You can NEVER accidentally mix a PDF header with an Excel footer ★

Real-world idea: a **furniture set**. An "Antique" factory gives you an antique chair, table *and* sofa — you can't end up with a modern chair beside an antique table.

Easy memory

Factory Method → ONE product → createButton()
Abstract Factory → a FAMILY → createButton() + createCheckbox() + createDialog()

Guarantee: you never mix a PDF header with an Excel footer ★

Furniture set: all pieces match.

Q: Explain the Builder pattern and when to use it.

The easiest way to remember:

Builder = construct a complex object STEP BY STEP,
instead of one constructor with 8 arguments.

The problem it solves — telescoping constructors

```
new QuizConfig("Algebra", 10);  
new QuizConfig("Algebra", 10, HARD);  
new QuizConfig("Algebra", 10, HARD, true);  
new QuizConfig("Algebra", 10, HARD, true, Duration.ofMinutes(20)); //  
▲ which is which?
```

```
new QuizConfig("Algebra", 10, HARD, true, false, true, 20, false)  
                        ↑      ↑      ↑      ↑  
                    what do these booleans even mean?
```



```

public final class QuizConfig {
    private final String topic;           // required
    private final int questionCount;     // required
    private final Difficulty difficulty;  // optional
    private final boolean bilingual;     // optional
    private final Duration timeLimit;    // optional

    private QuizConfig(Builder b) {
        this.topic = b.topic;
        this.questionCount = b.questionCount;
        this.difficulty = b.difficulty;
        this.bilingual = b.bilingual;
        this.timeLimit = b.timeLimit;
    }

    public static Builder builder(String topic, int questionCount) {
        return new Builder(topic, questionCount);    // required args
here ★
    }

    public static class Builder {
        private final String topic;
        private final int questionCount;
        private Difficulty difficulty = Difficulty.MEDIUM;    // sensible
defaults ★
        private boolean bilingual = false;
        private Duration timeLimit = Duration.ofMinutes(10);

        Builder(String topic, int questionCount) {
            this.topic = topic;
            this.questionCount = questionCount;
        }

        public Builder difficulty(Difficulty d) { this.difficulty = d;
return this; } // fluent ★
        public Builder bilingual(boolean b)      { this.bilingual = b;
return this; }
        public Builder timeLimit(Duration d)    { this.timeLimit = d;
return this; }

        public QuizConfig build() {
            if (questionCount <= 0) throw new
IllegalArgumentExcep­tion("questionCount must be > 0");
            return new QuizConfig(this);    // validate ONCE, then
create an immutable object ★
        }
    }
}

```

```
}  
}
```

```
QuizConfig cfg = QuizConfig.builder("Quadratic Equations", 10)  
    .difficulty(Difficulty.HARD)  
    .bilingual(true)  
    .build();
```

Every line NAMES what it sets ✓
Optional fields have defaults ✓
Validation happens ONCE, in build() ✓
The result is IMMUTABLE ✓

Real-world idea: **Subway**. You don't order "a sandwich(bread, cheese, veg, sauce, toasted)". You walk the counter saying *bread* → *cheese* → *veggies* → *sauce* → *done*, and you can skip anything.

In the JDK / ecosystem: `StringBuilder`, `Stream.Builder`, `Calendar.Builder`, `UriComponentsBuilder`, `SpringApplicationBuilder`, Lombok's `@Builder`.

Builder vs Factory: Factory answers "*which type do I create?*" in one call. Builder answers "*how do I assemble this one type?*" over several calls.

Easy memory

Builder = build STEP BY STEP → `.difficulty().bilingual().build()` ★

Use when: many parameters, several optional, and you want immutability
Validate inside build() | each setter returns ``this`` (fluent)

Subway sandwich – add what you want, skip what you don't
Factory = WHICH type (1 call) | Builder = HOW to assemble (many calls)

Q: Explain the Prototype pattern.

The easiest way to remember:

Prototype = create a new object by COPYING an existing one,
instead of building it from scratch.

Useful when construction is expensive — e.g. an object assembled from a database or a network call.

```
public class QuizTemplate implements Cloneable {
    private String name;
    private List<Question> questions;

    @Override
    public QuizTemplate clone() {
        QuizTemplate copy = new QuizTemplate();
        copy.name = this.name;
        copy.questions = new ArrayList<>(this.questions);    // copy the
list, not just the reference
        return copy;
    }
}
```

Real-world idea: **photocopying a filled form**. Rather than writing every field again, you copy the completed one and change the name.

▲ Shallow vs deep copy — the real question here

SHALLOW copy (Object.clone() by default)

copy.list ┌
 └─→ the SAME ArrayList object
original.list ┘

→ copy.list.add(x) also changes the ORIGINAL ▲

DEEP copy

copy.list → a NEW ArrayList with copied contents ✓

In real code, prefer a **copy constructor** or a **record** with **with...** methods over **Cloneable**, which is widely considered a broken API (it doesn't even declare **clone()**).

Note: Spring's **@Scope("prototype")** is a **scope**, not this pattern — it just means "a new bean per injection or lookup".

Easy memory

Prototype = COPY an existing object (photocopy a filled form)

▲ SHALLOW copy shares nested objects → mutating the copy corrupts the original ▲

Prefer a COPY CONSTRUCTOR over Cloneable (which is a broken API)

Spring @Scope("prototype") is a SCOPE, not this pattern.

Part 4 — Structural patterns

Q: Explain the Adapter pattern.

The easiest way to remember:

Adapter = a PLUG CONVERTER.

It makes an incompatible interface fit the one your code expects.

```
// A third-party math solver whose interface we DON'T control
class LegacySolver {
    String solveExpression(String latex) { ... }
}

// What our domain WANTS
public interface MathSolver { Solution solve(MathProblem problem); }

// The adapter
@Component
public class LegacySolverAdapter implements MathSolver {

    private final LegacySolver legacy = new LegacySolver();

    @Override
    public Solution solve(MathProblem problem) {
        String raw = legacy.solveExpression(problem.toLatex()); //
        translate the CALL
        return SolutionParser.parse(raw); //
        translate the RESULT
    }
}
```

<u>Our code</u>		<u>ADAPTER</u>		<u>Third-party</u>
MathSolver.solve()	→	translates in	→	legacy.solveExpression()
Solution	←	translates out	←	a raw String

Real-world idea: an **Indian-to-European plug adapter**. Neither the laptop nor the wall socket changes — the adapter sits between them.

Real use: wrapping a vendor SDK so your domain isn't polluted by their types, and so swapping vendors touches exactly one class.

In the JDK: `Arrays.asList()`, `InputStreamReader` (bytes → chars).

Easy memory

Adapter = a plug converter – makes an incompatible interface fit

Wrap the vendor SDK → your domain stays clean → swapping vendors touches ONE class ★

JDK: `Arrays.asList()`, `InputStreamReader`

Q: Explain the Decorator pattern. How is it different from inheritance?

The easiest way to remember:

Decorator = WRAP an object to add behaviour, at RUNTIME,
without changing its class.

```
public interface Notifier { void send(String userId, String message); }
```

```
@Component
```

```
class EmailNotifier implements Notifier {  
    public void send(String userId, String message) { /* SMTP */ }  
}
```

```
// Decorators WRAP and DELEGATE
```

```
class LoggingNotifier implements Notifier {  
    private final Notifier delegate;  
    LoggingNotifier(Notifier delegate) { this.delegate = delegate; }
```

```
    public void send(String userId, String message) {  
        log.info("Sending to {}", userId);  
        delegate.send(userId, message);           // pass it on ★  
        log.info("Sent");  
    }  
}
```

```
class RetryingNotifier implements Notifier {  
    private final Notifier delegate;  
    RetryingNotifier(Notifier delegate) { this.delegate = delegate; }
```

```
    public void send(String userId, String message) {  
        for (int attempt = 1; attempt <= 3; attempt++) {  
            try { delegate.send(userId, message); return; }  
            catch (TransientException e) { if (attempt == 3) throw e; }  
        }  
    }  
}
```

```
Notifier notifier = new LoggingNotifier(new RetryingNotifier(new  
EmailNotifier()));
```

```

LoggingNotifier
  wraps
RetryingNotifier
  wraps
  EmailNotifier      ← the real work

```

send() → log → retry loop → actual SMTP → back out through the wrappers ✓

Real-world idea: **dressing in layers**. Shirt → sweater → raincoat. Each layer adds a capability, and you choose the combination each morning. You don't buy a single "ShirtSweaterRaincoat" garment.

Decorator vs inheritance

Decorator	Inheritance
runtime composition ✓	compile-time , fixed
combine freely (log + retry + cache)	class explosion: <code>LoggingRetryingEmailNotifier</code> ▲
wraps an instance	extends a class

With inheritance, 3 features = $2^3 = 8$ classes to cover every combination

▲ With decorators, 3 wrappers = every combination for free ✓

In the JDK: `java.io` is the textbook case —

```

new BufferedReader(new InputStreamReader(new FileInputStream(f)))
//   ↑ adds buffering   ↑ adds char decoding   ↑ the real file

```

Also `Collections.unmodifiableList()`.

Easy memory

Decorator = WRAP and DELEGATE, at runtime ★

```

new LoggingNotifier(new RetryingNotifier(new EmailNotifier()))

```

Layers of clothing — mix and match, no class explosion
JDK: `java.io` streams, `Collections.unmodifiableList()`

Q: Explain the Facade pattern.

The easiest way to remember:

Facade = ONE simple front door to a complicated system behind it.

```
@Service
public class QuizFacade {                                // ONE call for the controller

    private final QuestionSelector selector;
    private final EvaluationService evaluator;
    private final ExplanationService explanations;
    private final ProgressTracker progress;

    public QuizResult submit(String studentId, QuizSubmission submission)
    {
        var scored      = evaluator.evaluate(submission);
        var walkthrough =
        explanations.forIncorrect(scored.getWrongAnswers());
        progress.record(studentId, scored);
        return new QuizResult(scored, walkthrough);
    }
}
```

BEFORE

Controller

- └─ EvaluationService
- └─ ExplanationService
- └─ ProgressTracker
- └─ QuestionSelector

(must know all 4, and the correct ORDER to call them)

AFTER

Controller

↓

QuizFacade.submit() ★

- └─ EvaluationService
- └─ ExplanationService
- └─ ProgressTracker

Real-world idea: a **hotel reception**. You ask reception for a taxi, laundry and a wake-up call. You never phone the taxi company, the laundry room and the operator yourself.

Facade vs Adapter:

Facade → SIMPLIFIES an interface you OWN

Adapter → CONVERTS an interface you DON'T own ★

Easy memory

Facade = one simple FRONT DOOR to a complex subsystem (hotel reception)

Controller makes ONE call; the facade orchestrates the four services
Facade SIMPLIFIES (yours) | Adapter CONVERTS (someone else's) ★

Q: Explain the Proxy pattern and its types.

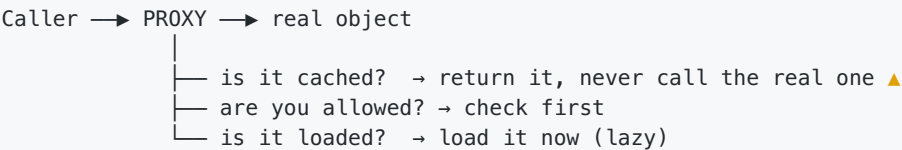
The easiest way to remember:

Proxy = a STAND-IN with the SAME interface, that CONTROLS ACCESS to the real object.

Type	Purpose
Virtual	lazy-load an expensive object (Hibernate lazy-loading proxies)
Protection	access control / authorization checks
Remote	a local stand-in for a remote object (RMI, Feign clients)
Caching	return a cached result instead of calling the real service

```
class CachingSolverProxy implements MathSolver {
    private final MathSolver real;
    private final Map<String, Solution> cache = new ConcurrentHashMap<>();

    public Solution solve(MathProblem p) {
        return cache.computeIfAbsent(p.getKey(), k -> real.solve(p));
    }
}
```



Real-world idea: a **personal secretary**. You call the CEO; the secretary answers, decides whether to put you through, and sometimes answers your question herself from her notes.

▲ This is the #1 Spring internals question

@Transactional, @Cacheable, @Async and Spring Security ALL work by wrapping your bean in a PROXY. ★

JDK dynamic proxy → if your class implements an interface
CGLIB subclass → otherwise

Spring injects the PROXY, not your class:

Caller → [PROXY: begin transaction] → YourService.method() → [PROXY: commit]

The consequence everyone gets wrong:

```
@Service
class OrderService {
    public void a() {
        this.b();           // x the call NEVER leaves the object
    }                       // → it never passes through the proxy
                           // → @Transactional is completely IGNORED ▲

    @Transactional
    public void b() { }
}
```

External call: caller → PROXY → a()	✓ proxy applies
Internal call: a() → this.b()	x proxy is BYPASSED ▲

Also, for the same reason, **@Transactional** on a **private** or **final** method does nothing.

Proxy vs Decorator: structurally identical — only the *intent* differs.

Decorator → ADDS behaviour
Proxy → CONTROLS ACCESS to the same behaviour ★

Easy memory

Proxy = a stand-in that CONTROLS ACCESS (a secretary)

Types: virtual (lazy) | protection | remote | caching

- ▲ `@Transactional` / `@Cacheable` / `@Async` / `Security = PROXIES` ★
JDK dynamic proxy (with an interface) | CGLIB (without)
- ▲ SELF-INVOCATION `this.method()` bypasses the proxy → the annotation is IGNORED
- ▲ `private/final` methods too

● In your RoboGebra code — proxies you actually rely on

```
robogebra-portal:
  16 × @Cacheable    → a PROXY intercepts and returns the cached value
  130 × @Transactional → a PROXY begins/commits the transaction
  12 × @Async        → a PROXY hands the call to a thread pool
```

File: `domain/exercisesolution/service/ExerciseSolutionService.java`

```
@Cacheable(value = CacheNames.EXERCISE_SOLUTION)
public ExerciseSolutionDTO getSolution(String exerciseItemId) { ... }

@Cacheable(value = CacheNames.EXERCISE_SOLUTION, key = "'admin-' +
#exerciseItemId")
public ExerciseSolutionDTO getAdminSolution(String exerciseItemId) { ... }
```

■ "Solution content is expensive to assemble and almost never changes, so it's cached. Note the custom key on the admin variant — two methods caching into the same region need distinct keys, or the admin view would serve the student's cached payload. And because `@Cacheable` is proxy-based, calling it from inside the same bean bypasses the cache entirely — the same self-invocation trap as `@Transactional`."

→ 39 — RoboGebra Code Examples

Q: Explain the Composite pattern.

The easiest way to remember:

Composite = treat ONE thing and a GROUP of things the SAME way.
(For TREE structures.)

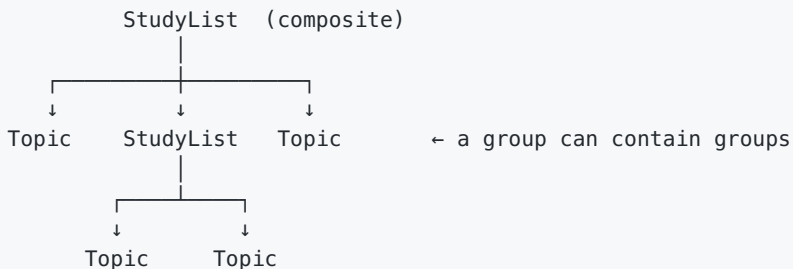
```
interface StudyNode {
    int totalTopics();
}

class Topic implements StudyNode { // LEAF
    public int totalTopics() { return 1; }
}

class StudyList implements StudyNode { // COMPOSITE
    private final List<StudyNode> children = new ArrayList<>();

    public void add(StudyNode node) { children.add(node); }

    public int totalTopics() {
        return
        children.stream().mapToInt(StudyNode::totalTopics).sum(); // recursive ★
    }
}
```



`totalTopics()` works IDENTICALLY on a leaf and on a branch ✓

Real-world idea: **folders on your computer**. A folder can contain files *and* other folders. "Calculate the size" works the same way on both — you never write different code for a file and a folder.

Real uses: the Subsea **category / sub-category** tree, file systems, Angular component trees, org charts, menu structures.

Easy memory

Composite = leaf and group share ONE interface → TREE structures ★

Folders – a folder holds files AND folders; "size" works on both
Recursive: the composite delegates to its children and sums the result

Part 5 — Behavioural patterns

Q: Explain the Strategy pattern.

The easiest way to remember:

Strategy = a family of INTERCHANGEABLE ALGORITHMS,
swappable at RUNTIME.

The single most-used pattern in Spring applications. ★

```

public interface DiscountStrategy { BigDecimal apply(BigDecimal amount); }

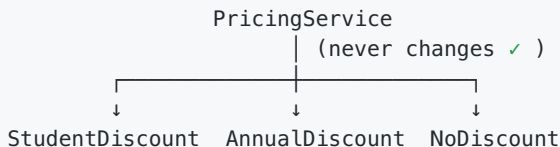
@Component("student") class StudentDiscount implements DiscountStrategy
{ ... }
@Component("annual") class AnnualDiscount implements DiscountStrategy
{ ... }
@Component("none") class NoDiscount implements DiscountStrategy {
    public BigDecimal apply(BigDecimal amount) { return amount; } //
★ Null Object pattern
}

@Service
public class PricingService {
    private final Map<String, DiscountStrategy> strategies; // Spring
    injects bean-name → bean ★

    public PricingService(Map<String, DiscountStrategy> strategies) {
        this.strategies = strategies;
    }

    public BigDecimal price(BigDecimal base, String planCode) {
        return strategies.getOrDefault(planCode,
        strategies.get("none")).apply(base);
    }
}

```



New discount type → add ONE @Component → nothing else changes ★

Real-world idea: **Google Maps travel modes.**

"Get me from A to B"

- by car
- by bike
- walking
- by train

Same GOAL, different ALGORITHM, chosen at the moment you tap it ★

★ Note the `NoDiscount` class — a strategy that does nothing. That is the **Null Object pattern**, and it removes every `if (discount != null)` check from the codebase. Worth mentioning.

Strategy vs Factory:

Factory → decides WHICH OBJECT to CREATE
Strategy → decides WHICH ALGORITHM to RUN

They are often used together – a factory returns the strategy.

Strategy vs State:

Structurally IDENTICAL.
Strategy → chosen by the CLIENT ("sort by price")
State → the OBJECT changes its own state ("draft → submitted → approved") ★

Easy memory

Strategy = interchangeable ALGORITHMS, swapped at runtime ★

Google Maps: car / bike / walk – same goal, different algorithm
Spring: inject `Map<String, Strategy>` → look it up by key ★
Null Object (`NoDiscount`) removes every null check

Factory = WHICH OBJECT | Strategy = WHICH ALGORITHM
Strategy = client chooses | State = the object drives itself

● In your RoboGebra project — Strategy, chosen by environment ★

```
public interface EventTrackingService { void trackUserLogin(User user); } // the strategy

@Service @Profile({"local", "dev"})
public class LoggingEventTrackingService implements EventTrackingService
{ ... } // strategy A

@Service @Profile({"prod"})
public class PostHogEventTrackingService implements EventTrackingService
{ ... } // strategy B
```

Classic Strategy → the CLIENT picks the algorithm at runtime
Your variant → SPRING picks it, from the active profile ★

Same pattern, and the selection logic lives in configuration
instead of in an if/else ★

■ *"It's Strategy with the selection delegated to the framework — the profile decides which implementation is in the context, so there's no factory and no conditional anywhere in the business code."*

★ Compare that with the classic runtime-selected form, which is also in your stack — the **Angular Formly** field registry on EasyVisa, where a JSON **type** picks the component. Same pattern, different selector.

Q: Explain the Observer pattern.

The easiest way to remember:

Observer = one-to-many.
When the SUBJECT changes, all registered LISTENERS are told automatically.

```
public interface QuizCompletedListener { void onQuizCompleted(QuizResult result); }

@Service
public class QuizService {
    private final List<QuizCompletedListener> listeners; // injected by Spring

    public void complete(QuizResult result) {
        save(result);
        listeners.forEach(l -> l.onQuizCompleted(result)); // notify ALL
    }
}
```

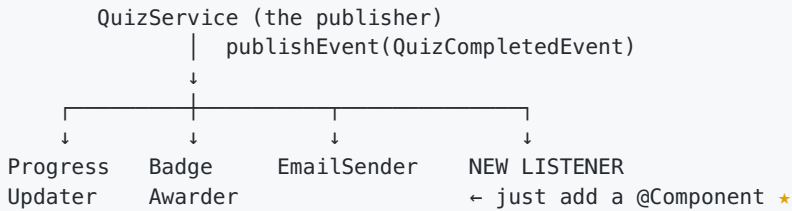

The idiomatic Spring version — application events ★

```
@Service
class QuizService {
    private final ApplicationEventPublisher publisher;

    void complete(QuizResult result) {
        save(result);
        publisher.publishEvent(new QuizCompletedEvent(result));    // ★
    }
    // knows NOTHING about listeners
}

@Component
class ProgressUpdater {
    @EventListener
    void on(QuizCompletedEvent event) { /* update the dashboard analytics */ }
}

@Component
class BadgeAwarder {
    @Async
    @EventListener
    void on(QuizCompletedEvent event) { /* award badges off the request thread */ }
}
```



The publisher never learns their names.

Real-world idea: a **YouTube channel**.

The creator uploads ONE video

↓

EVERY subscriber gets a notification automatically

The creator doesn't call each subscriber individually,
and doesn't even know who they are.

Adding a new reaction to quiz completion = **one new listener**, `QuizService` untouched. That is **OCP** again.

Elsewhere: RxJS `Observable` / `Subject` in Angular (your Study List real-time sync), DOM event listeners, `PropertyChangeListener`.

▲ Java's `java.util.Observer` was **deprecated in Java 9** — don't cite it as current practice.

Easy memory

Observer = one-to-many; the subject notifies all listeners ★

YouTube: upload once → every subscriber is notified

Spring: `ApplicationEventPublisher` + `@EventListener` (publisher knows nobody) ★

Angular: RxJS `Observable` / `BehaviorSubject`

▲ `java.util.Observer` is DEPRECATED (Java 9)

Q: Explain the Template Method pattern.

The easiest way to remember:

Template Method = the PARENT fixes the STEPS and their ORDER;
the CHILD fills in specific steps.

The skeleton method is `final` so nobody can reorder it. ★

```

public abstract class DocumentImportJob {

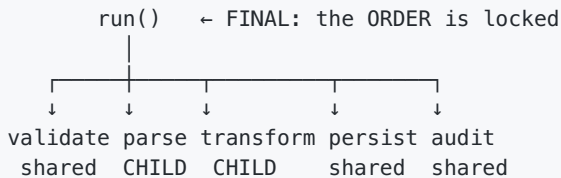
    public final ImportResult run(File file) {           // ★ final – the
        skeleton can't be altered
        validate(file);
        var parsed = parse(file);                       // varies by subclass
        var records = transform(parsed);                 // varies by subclass
        persist(records);
        return audit(records);
    }

    protected abstract ParsedData parse(File file);
    protected abstract List<Record> transform(ParsedData data);

    protected void validate(File file) { /* shared default */ } // a
HOOK – overridable
    private void persist(List<Record> records) { /* shared */ }
    private ImportResult audit(List<Record> r) { /* shared */ }
}

class CsvImportJob extends DocumentImportJob { ... }
class ExcelImportJob extends DocumentImportJob { ... }

```



Real-world idea: a **recipe**. The steps and their order are fixed — chop, cook, garnish. Each chef decides *what* to chop and *how* to garnish, but nobody is allowed to garnish before cooking.

Template Method vs Strategy:

Template Method → INHERITANCE → the structure is fixed at compile time
 Strategy → COMPOSITION → the whole algorithm is swapped at runtime

Prefer Strategy when you can (favour composition over inheritance);
 use Template Method when the steps genuinely share a FIXED ORDER. ★

In Spring: `JdbcTemplate`, `RestTemplate`, `TransactionTemplate` are literally named after it — they handle the boilerplate (open the connection, handle exceptions, close it) and call *your* callback for the varying part.

Easy memory

Template Method = the parent fixes the STEPS + ORDER (final method);
the child fills in the varying steps ★

Recipe – chop → cook → garnish, in that order, always
Spring: `JdbcTemplate` / `RestTemplate` / `TransactionTemplate` ★

vs Strategy: inheritance + fixed skeleton | composition + runtime swap

● In your RoboGebra project — Template Method, in the security layer ★★

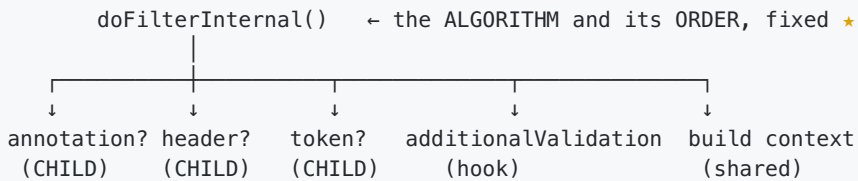
File: `security/AbstractServiceAuthenticationFilter.java`

```
public abstract class AbstractServiceAuthenticationFilter extends
OncePerRequestFilter {

    // ===== the SKELETON – written once, identical for every subclass ★
    =====
    // doFilterInternal(): is this endpoint annotated? → extract the
token
    //                               → compare it → additionalValidation() → build
the
    //                               SecurityContext → continue the chain

    // ===== the VARYING steps – each subclass supplies five ★ =====
    protected abstract Class<? extends Annotation> getAnnotationClass();
    protected abstract String getHeaderName();
    protected abstract String getExpectedToken();
    protected abstract String getFilterName();
    protected abstract UserRole getUserRole();

    // ===== HOOKS with sensible defaults – override only if you need to
★ =====
    protected String extractToken(HttpServletRequest request) { return
r.getHeader(getHeaderName()); }
    protected void additionalValidation(HttpServletRequest request) { /*
default: none */ }
}
```



Two subclasses, ~30 lines each: `CrmServiceAuthenticationFilter`, `WebhookAuthorizationFilter`.

■ "That's a template method. Both service-auth filters follow the same steps in the same order — check the annotation, read the header, compare the token, authenticate — so the algorithm is written once in the base class and each subclass supplies five small decisions. It also has two optional hooks with defaults, `extractToken` and `additionalValidation`, for the subclass that needs Bearer parsing. You couldn't express this with an interface: the base class needs three injected collaborators and a constructor."

★ And it sits inside another pattern: the whole thing is a **Chain of Responsibility** — Spring Security's filter chain, with `@Order(3)` deciding this filter's position.

Q: Explain the Chain of Responsibility pattern.

The easiest way to remember:

Chain of Responsibility = pass the request along a LINE of handlers until one of them deals with it.

The sender doesn't know WHO will handle it. ★

```

public interface ValidationHandler {
    void setNext(ValidationHandler next);
    void handle(VisaApplication app);
}

public abstract class AbstractValidator implements ValidationHandler {
    private ValidationHandler next;
    public void setNext(ValidationHandler next) { this.next = next; }
    protected void passOn(VisaApplication app) { if (next != null)
next.handle(app); }
}

class DocumentCompletenessValidator extends AbstractValidator {
    public void handle(VisaApplication app) {
        if (app.getDocuments().isEmpty()) throw new
ValidationException("Documents missing");
        passOn(app); // ★ hand it on
    }
}

class EligibilityValidator extends AbstractValidator { ... }
class AttorneySignatureValidator extends AbstractValidator { ... }

```

Request

↓

```

[ DocumentCompleteness ] → passes → [ Eligibility ] → passes →
[ Signature ] → ✓
                    |
                    or STOPS here if it fails ▲

```

Real-world idea: a **leave approval chain**.

Your request → Team Lead → Manager → HR → Director

Each one either approves and passes it up, or stops it.

You submit ONCE and don't need to know who finally signs it ★

Real-world in code: Servlet **Filter** chains, Spring Security's filter chain, Angular HTTP interceptors, logging levels, approval workflows.

Easy memory

Chain of Responsibility = pass it along until someone handles it ★

Leave approval: Lead → Manager → HR → Director

Spring: the SECURITY FILTER CHAIN, servlet filters, HandlerInterceptor

Angular: HTTP_INTERCEPTORS (auth → error → retry)

Q: Explain the Command pattern.

The easiest way to remember:

Command = wrap a REQUEST as an OBJECT,
so you can queue it, log it, pass it around – and UNDO it. ★

```
public interface Command {
    void execute();
    void undo();
}

class AddTopicCommand implements Command {
    private final StudyList list;
    private final Topic topic;

    public void execute() { list.add(topic); }
    public void undo()    { list.remove(topic); }      //
★ this is the payoff
}

class CommandHistory {
    private final Deque<Command> history = new ArrayDeque<>();

    void run(Command c) { c.execute(); history.push(c); }
    void undoLast()    { if (!history.isEmpty()) history.pop().undo(); }
}
```

```
run(AddTopic)    → history: [AddTopic]
run(RemoveTopic)→ history: [RemoveTopic, AddTopic]
undoLast()       → pops RemoveTopic → calls undo() ✓
```

Real-world idea: a **restaurant order slip**. The waiter writes the order on paper. That slip can be queued, passed to the kitchen, re-read, logged — and cancelled.

Or simply: **Ctrl+Z** in any editor. Every action is an object with an `undo()` .

In the JDK: `Runnable` is a Command — that is exactly why an `ExecutorService` can queue and schedule work. Also, Angular/NgRx **actions** are essentially commands.

Easy memory

Command = a request wrapped as an OBJECT → queue it, log it, UNDO it ★

Ctrl+Z ⇄ | a restaurant order slip

JDK: `Runnable` IS a Command → that's why executors can queue it ★

Angular: NgRx actions

Q: Explain the State pattern.

The easiest way to remember:

State = the object CHANGES ITS BEHAVIOUR when its internal state changes.
It appears to change class.

It replaces a sprawling if/switch on a `status` field. ★

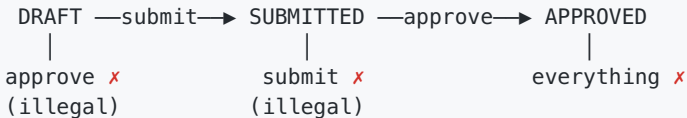

```

interface ApplicationState {
    ApplicationState submit(VisaApplication app);
    ApplicationState approve(VisaApplication app);
}

class DraftState implements ApplicationState {
    public ApplicationState submit(VisaApplication app) { return new
SubmittedState(); }
    public ApplicationState approve(VisaApplication app) {
        throw new IllegalStateException("Cannot approve a draft");    //
★ impossible, by design
    }
}

class SubmittedState implements ApplicationState {
    public ApplicationState submit(VisaApplication app) { throw new
IllegalStateException("Already submitted"); }
    public ApplicationState approve(VisaApplication app) { return new
ApprovedState(); }
}

```



Illegal transitions become IMPOSSIBLE, instead of being guarded by scattered `if` statements all over the codebase ★

Real-world idea: a **traffic light**. Red can only become green; green can only become amber. The light itself decides the next state — you don't hand it a strategy.

Easy memory

State = behaviour changes with internal state; illegal transitions become IMPOSSIBLE ★

Replaces: `if (status == "DRAFT") ... else if (status == "SUBMITTED") ...`
Traffic light – the object drives its OWN transitions

vs Strategy: identical structure | Strategy = the CLIENT chooses
| State = the OBJECT transitions
itself

Q: Explain the Iterator pattern.

The easiest way to remember:

Iterator = walk through a collection WITHOUT knowing how it stores things.

```
for (String s : list) { ... }           // you never see the array or the  
linked nodes ★
```

ArrayList → internally an array
LinkedList → internally nodes with pointers
TreeSet → internally a red-black tree

The for-each loop looks IDENTICAL for all three ✓

You use it every day: **Iterable** / **Iterator**, the enhanced for-loop, and **Stream**.

Custom implementations matter when you want to hide the backing structure — an array, a linked list, or a **paged API** — from the caller. A well-written iterator can even fetch the next page of a REST API transparently as you loop.

Real-world idea: a **TV remote's channel button**. You press "next" without knowing whether the channels are stored in an array, a list or a database.

Easy memory

Iterator = sequential access WITHOUT exposing the internal structure ★

for (String s : list) — identical for ArrayList, LinkedList and TreeSet
Custom use: hide paging — the iterator fetches the next API page transparently

Part 6 — Patterns in the JDK, Spring & Angular

Q: Name design patterns used in the JDK.

Pattern	JDK example
Singleton	<code>Runtime.getRuntime()</code> , <code>Desktop.getDesktop()</code>
Factory Method	<code>Calendar.getInstance()</code> , <code>NumberFormat.getInstance()</code> , <code>Optional.of()</code> , <code>List.of()</code>
Builder	<code>StringBuilder</code> , <code>Stream.Builder</code> , <code>Calendar.Builder</code>
Prototype	<code>Object.clone()</code>
Adapter	<code>Arrays.asList()</code> , <code>InputStreamReader</code>
Decorator	<code>java.io</code> streams, <code>Collections.unmodifiableList()</code>
Proxy	<code>java.lang.reflect.Proxy</code> , RMI
Observer	<code>PropertyChangeListener</code> , Swing listeners, the Flow API
Strategy	<code>Comparator</code> passed to <code>Collections.sort()</code> ★
Template Method	<code>AbstractList</code> , <code>AbstractMap</code> , <code>InputStream.read()</code>
Command	<code>Runnable</code> , <code>Callable</code>
Iterator	<code>Iterator</code> , <code>Iterable</code>
Flyweight	the <code>Integer.valueOf()</code> cache (-128..127), the String pool ★

The three easiest to say confidently:

Comparator	→ STRATEGY ★
new BufferedReader(new ...)	→ DECORATOR ★
Integer cache / String pool	→ FLYWEIGHT ★

Q: Which design patterns does Spring use?

This is asked in almost every Spring interview — memorise this table.

Pattern	Where in Spring
Singleton	the default bean scope
Prototype	@Scope("prototype")
Factory	BeanFactory , ApplicationContext , FactoryBean<T>
Proxy ★	@Transactional , @Cacheable , @Async , AOP, Spring Security, Hibernate lazy loading
Template Method ★	JdbcTemplate , RestTemplate , TransactionTemplate , MongoTemplate
Observer	ApplicationEvent / @EventListener / ApplicationEventPublisher
Strategy	injecting different implementations of an interface; Resource loading
Front Controller ★	DispatcherServlet
MVC	the whole Spring MVC layering
Chain of Responsibility ★	servlet filters, the Spring Security filter chain, HandlerInterceptor
Adapter	HandlerAdapter , MessageConverter
Dependency Injection / IoC	the core container

Pattern	Where in Spring
DAO / Repository	@Repository , Spring Data JPA / Mongo repositories
DTO	request/response objects at the API boundary

The five to say if you only get one sentence:

```

DispatcherServlet → Front Controller
JdbcTemplate      → Template Method
@Transactional     → PROXY ★
Security filters  → Chain of Responsibility
@EventListener     → Observer

```

Q: What patterns show up in your Angular/Ionic work?

Good to have ready, since your role is full stack:

```

Singleton → providedIn: 'root' services (one instance app-wide)
Observer  → RxJS Observable / Subject / BehaviorSubject
           → your real-time Study List sync ★
DI         → Angular's injector, @Injectable, injection tokens
Decorator  → literally @Component, @Injectable, @Input
           → also HTTP interceptors wrapping requests
Chain of Resp → the HTTP_INTERCEPTORS chain (auth token → error → retry) ★
Facade     → a facade service over an NgRx store, so components
           don't touch actions and selectors directly
Strategy   → Angular Formly: the field `type` selects which component
renders ★
Command    → NgRx actions dispatched to reducers and effects

```

◆ The Formly point is a strong, genuine answer: *"Formly is essentially a strategy/factory pattern — a JSON config picks the field type, and a registry maps that type to a component. In EasyVisa I registered custom field types for our nested reactive forms, which meant new form fields were config changes rather than template changes."*

Q: What are DTO, DAO/Repository, and why not expose entities directly?

DTO = a flat object for carrying data across a BOUNDARY (API request/response).

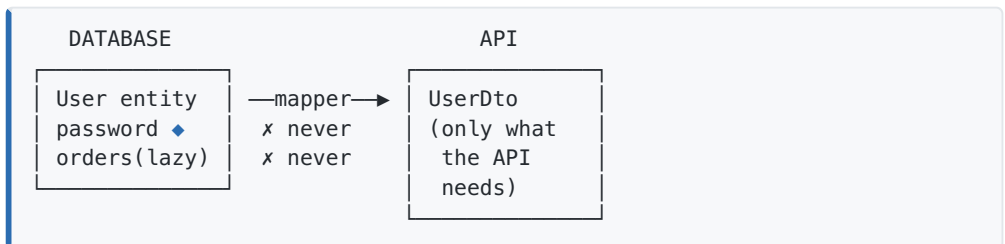
No business logic.

DAO / Repository = encapsulates DATA ACCESS, so the service layer never sees SQL or Mongo queries.

(Strictly, a Repository is a DDD concept working with aggregates, and a DAO is table-oriented — but interviewers usually use them interchangeably.)

Why not return JPA entities from controllers — five reasons

1. It LEAKS the DB schema into the API contract
→ a column rename becomes a BREAKING API change ▲
2. Risk of exposing SENSITIVE fields (password hash, internal flags)
3. LAZY-LOADING proxies blow up during JSON serialisation
→ LazyInitializationException
4. BIDIRECTIONAL relations cause INFINITE RECURSION in JSON
→ Employee → Department → employees → Employee → ... ▲
5. You cannot SHAPE the response per endpoint



Use a mapper (MapStruct, or plain code) and Java **records** for DTOs.

Q: What is an anti-pattern? Name a few.

An anti-pattern = a common "solution" that LOOKS reasonable but causes harm.

Anti-pattern	Problem
God Object / God Class	one class does everything — SRP violation, untestable
Singleton abuse	global mutable state, hidden dependencies, untestable
Anaemic Domain Model	entities are just getters/setters; all logic sits in services
Spaghetti / Big Ball of Mud	no layering; everything depends on everything
Magic numbers/strings	unnamed literals scattered through the code
Golden Hammer	forcing one favourite pattern onto every problem ★
Premature optimization	complexity for unmeasured gains
Circular dependencies	bean A needs B needs A — Spring now rejects this by default ★

★ The **Golden Hammer** is worth naming, because it shows judgement: *"knowing patterns is less valuable than knowing when NOT to use one."*

Part 7 — Scenario & "spot the violation" questions

Q: Which SOLID principle does this violate, and how would you fix it?

```
@Service
public class ReportService {
    public void generate(String type) {
        if (type.equals("PDF")) {
            new PdfWriter().write();
        } else if (type.equals("EXCEL")) {
            new ExcelWriter().write();
        }
        new SmtMailer().send();
        System.out.println("Report generated");
    }
}
```

Answer — three violations. Say all three:

1. OCP ▲ — a new format means EDITING this method
Fix: a ReportWriter interface + strategy/factory
2. DIP ▲ — it `new`s concrete PdfWriter, ExcelWriter, SmtMailer
Fix: inject ABSTRACTIONS through the constructor
3. SRP ▲ — it chooses a format, writes, emails AND logs
Fix: split the responsibilities; use a logger, not System.out

And it is UNTESTABLE:
you cannot generate a report without sending a REAL email ▲

The refactored shape:


```

@Service
public class ReportService {
    private final Map<ReportType, ReportWriter> writers;    // OCP + DIP
    private final Mailer mailer;                          // DIP

    public ReportService(List<ReportWriter> writers, Mailer mailer)
    { ... }

    public void generate(ReportType type) {
        writers.get(type).write();    // add a format = add a
    }
}
@Component ✓
    mailer.send();
}
}

```

Q: "Design a notification system that supports email and SMS today, and push/WhatsApp later, with retry and audit logging."

Walk through it **out loud, in this order**:

1. A `Notifier` interface with supports(Channel) + send(Notification)
→ STRATEGY, satisfies OCP
2. EmailNotifier, SmsNotifier as @Components
→ adding push = ONE new class ✓
3. A NotifierFactory injecting List<Notifier>, resolving by channel
→ FACTORY
4. Retry and audit as DECORATORS wrapping any notifier
→ DECORATOR, keeps SRP intact ★
5. A NotificationService FACADE so callers make one call
→ FACADE
6. Trigger it from a domain event via @EventListener
→ OBSERVER, decoupling the sender from the sending ★
7. Constructor injection everywhere
→ DIP + easily unit-tested with mocks

```
DomainEvent
  ↓ @EventListener (Observer)
NotificationService (Facade)
  ↓
NotifierFactory (Factory)
  ↓
AuditingNotifier → RetryingNotifier → EmailNotifier    (Decorators + Strategy)
```

Then — and this is the part that scores — **mention the trade-off**:

"I'd start with the interface and two implementations, and only add the decorators when retry and audit are actually required — otherwise it's speculative."

Q: "You have a class with an 8-argument constructor. What's wrong?"

Almost certainly an SRP violation —
8 collaborators means 8 REASONS TO CHANGE. ★

Options:

1. GROUP related dependencies into one cohesive collaborator
2. EXTRACT a facade for a subgroup
3. SPLIT the class along its axes of change
4. If they are VALUES rather than dependencies
→ use a BUILDER or a parameter object / record

★ The bonus point: *"This is actually an argument for constructor injection — field injection would have hidden the problem completely."*

Q: "Have you actually used a design pattern in your work?"

Never say "not really." Use one of these — all defensible from your resume.

Format: **Problem** → **Pattern** → **Result**. 30–45 seconds.

Strategy/Factory — RoboGebra: *"The quiz module supports several question types with different evaluation rules. Instead of a growing switch, I defined an*

evaluator interface, made each type a Spring component, and let Spring inject the full list. Adding a new question type became a single new class with no changes to the evaluation service."

Facade — RoboGebra: "Quiz submission touched scoring, AI explanation generation, and progress tracking. I put a facade service in front so the controller made one call — it kept the controller thin and let me change the internal ordering without touching the API layer."

Strategy — EasyVisa: "The document portal had multiple accordion panels with different upload rules. We drove them from configuration with a handler per document type rather than conditionals in the component."

Observer — RoboGebra (Angular side): "Real-time Study List sync used an RxJS *BehaviorSubject* in a shared service — the components subscribe and update reactively, which is the Observer pattern; I used *takeUntil* on a destroy subject to avoid leaks."

Adapter — Subsea: "We wrapped Kendo UI Grid interactions behind our own component API so the rest of the app didn't depend on the vendor's types."

Part 8 — Rapid-fire one-liners

Question	Answer
SRP in one line	one class, one reason to change
OCP in one line	add new classes, don't edit tested ones
LSP in one line	a subclass must not surprise code written for the parent
ISP in one line	no client forced to depend on methods it doesn't use
DIP in one line	depend on interfaces, inject them
DI vs IoC	DI is one way of achieving IoC
Best injection type	constructor — immutable, fail-fast, testable

Question	Answer
Why <code>volatile</code> in double-checked locking	prevents seeing a partially constructed object due to reordering ★
Safest singleton	<code>enum</code> (reflection- and serialization-proof)
Best lazy singleton without locks	the Bill Pugh static holder idiom
Spring singleton vs GoF singleton	per <code>ApplicationContext</code> vs per JVM; container-managed vs private constructor
Are singleton beans thread-safe?	No — keep them stateless ★
Factory vs Builder	Factory picks <i>which</i> object; Builder assembles <i>one</i> object step by step
Factory Method vs Abstract Factory	one product vs a family of related products
Strategy vs State	the client chooses the strategy; the object drives its own state transitions
Strategy vs Template Method	composition + runtime swap vs inheritance + fixed skeleton
Decorator vs Proxy	Decorator adds behaviour; Proxy controls access
Adapter vs Facade	Adapter converts an interface; Facade simplifies a subsystem
Composite is for	tree structures treated uniformly
Chain of Responsibility in Spring	the security filter chain, servlet filters, interceptors
Why <code>@Transactional</code> fails on self-invocation	the call never passes through the proxy ★
Pattern behind <code>JdbcTemplate</code>	Template Method
Pattern behind <code>DispatcherServlet</code>	Front Controller
Pattern behind <code>@Cacheable</code>	Proxy (AOP)
Pattern behind RxJS <code>Observable</code>	Observer
Pattern behind Angular <code>@Component</code>	Decorator

Question	Answer
Pattern behind <code>Comparator</code>	Strategy
Pattern behind the Integer cache	Flyweight
Favour composition over inheritance because	inheritance is compile-time and fragile; composition is runtime and flexible
God class	an anti-pattern violating SRP
Rule of three	hard-code, notice, then abstract — on the third case ★

Quick Revision Sheet

SOLID

- S one reason to change → accountant + DBA + marketing = 3 reasons
- ▲ 0 add, don't edit field → smell: a growing if/else on a "type" field
fix: interface + @Component + inject
- List<T> ★
 - L no surprises → smell: UnsupportedOperationException / instanceof
Square extends Rectangle breaks area()
 - I small interfaces → split by CAPABILITY (Signable, OcrCapable)
 - D depend on interfaces → smell: `new` inside a service
the interface belongs to the DOMAIN ★

DIP = principle | IoC = pattern | DI = technique

Rule of three: hard-code → notice → abstract ★

CREATIONAL

- Singleton eager | DCL (volatile MANDATORY ★) | Bill Pugh holder ★ | enum (safest)
Spring singleton = per ApplicationContext, NOT thread-safe ▲
- Factory caller says WHAT, factory decides WHICH class (pizza counter)
- Abstract Factory a FAMILY of products (never mix PDF header + Excel footer)
- Builder step by step, fluent, validate in build() (Subway)
- Prototype copy an object ▲ shallow vs deep

STRUCTURAL

- Adapter plug converter – convert someone else's interface
- Decorator WRAP and delegate at runtime (layers of clothing) – java.io
- Facade one front door (hotel reception)
- Proxy ▲ controls access – @Transactional/@Cacheable/@Async are

PROXIES ★

- self-invocation this.method() BYPASSES the proxy ▲
- Composite leaf + group share one interface → trees

BEHAVIOURAL

- Strategy ★ interchangeable algorithms (Google Maps: car/bike/walk)
Spring: inject Map<String, Strategy>
- Observer one-to-many (YouTube) – ApplicationEventPublisher + @EventListener
- Template parent fixes the ORDER (final method), child fills steps (recipe)
JdbcTemplate / RestTemplate
- Chain pass it along (leave approval) – Security filter chain ★

Command	request as an OBJECT → queue, log, UNDO (Ctrl+Z ↔) –
Runnable	
State	the object drives its own transitions (traffic light)
Iterator	walk without exposing the structure

SPRING PATTERNS (memorise)

DispatcherServlet → Front Controller | JdbcTemplate → Template Method
@Transactional → PROXY ★ | Security filters → Chain | @EventListener → Observer

THE #1 THING

@Transactional / @Cacheable / @Async are PROXIES,
and SELF-INVOCATION bypasses them. ★
Most 5-year candidates get this wrong.

Two-day drill

Day 1 — SOLID: write out each principle with a bad→good code pair from memory. Then do Part 7's "spot the violation". Be able to state DI vs IoC vs DIP without hesitating.

Day 2 — Patterns: memorise the Spring patterns table (Part 6) and be fluent in Singleton (all four variants **and** why `volatile`), Factory, Builder, Strategy, Observer, Decorator, Proxy and Template Method. Then rehearse your three real project stories from Part 7 out loud, 45 seconds each.

The single highest-yield thing here: knowing that `@Transactional` / `@Cacheable` / `@Async` are **proxies**, and that self-invocation bypasses them. It's asked constantly, and most 5-year candidates get it wrong.

Related files: [05 — Core Java](#) · [06 — Spring Boot](#) · [33 — Interface vs Abstract Class](#) · [32 — Multithreading \(singleton, proxies\)](#)

RoboGebra — Technical Stack & Versions (from the actual codebase)

Every number below was read from the real repos in `/Users/safi/workspace/robogebra-workspace/` — `package.json`, `pom.xml`, `angular.json`, `tsconfig.json`, `capacitor.config.ts`, `variables.gradle`, `application.yml`, `Dockerfile`. **Nothing here is guessed.** Audit date: **31 July 2026.**

Why this file matters: "Which version of Angular/Java/Spring Boot are you using?" is asked in almost every screening call. Vague answers ("latest", "Angular 17 or 18 I think") kill credibility instantly. Precise answers with a *reason* land you as someone who actually ships.

Table of contents

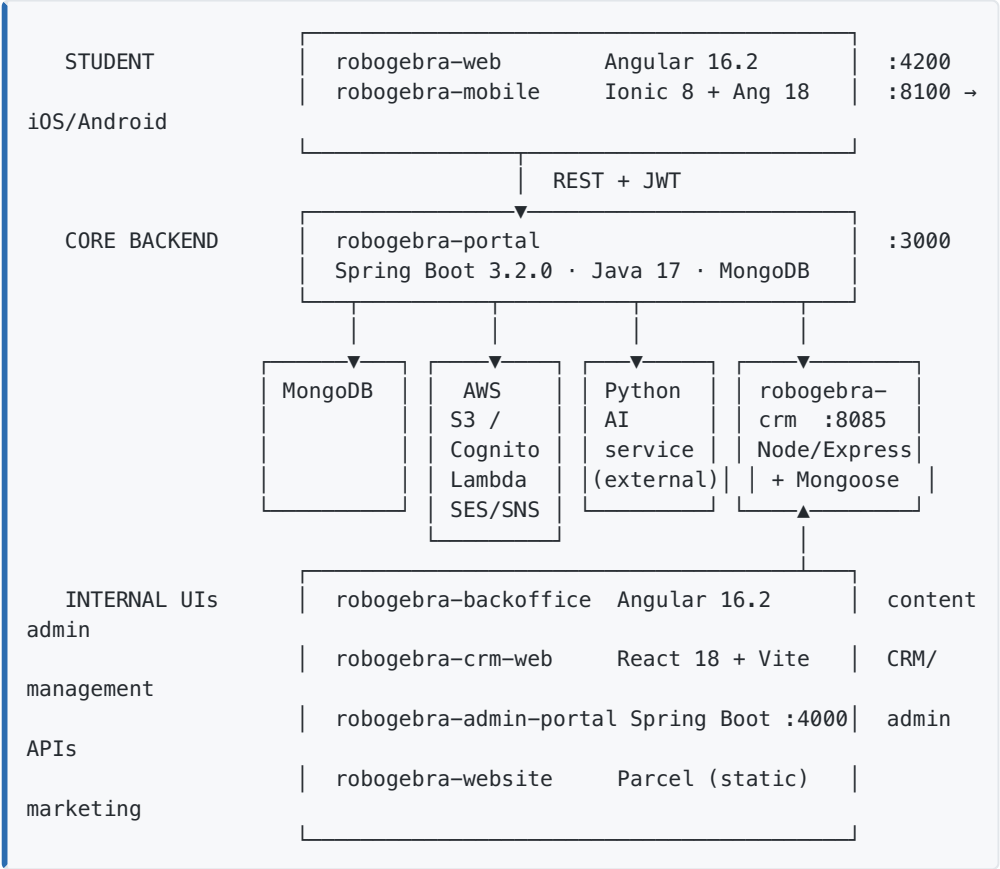
- ▲ [The 60-second cheat sheet](#)
- [System architecture — 8 repos](#)
 - 1. [robogebra-web](#) (frontend)
 - 2. [robogebra-mobile](#) (Ionic)
 - 3. [robogebra-backoffice](#) (admin)
 - 4. [robogebra-portal](#) (backend)
 - 5. [robogebra-admin-portal](#) (admin backend)
 - 6. [robogebra-crm](#) (management)
 - 7. [robogebra-crm-web](#) (management UI)
 - 8. [robogebra-website](#) (marketing)
- [Master version table](#)
- ▲ [Traps in your own stack](#)
- [Interview answer scripts](#)
- [Numbers worth quoting](#)

▲ The 60-second cheat sheet

Memorize this block. It answers 90% of stack questions.

Question	Your answer
Java version?	Java 17 (LTS) — <code><java.version>17</java.version></code> , runtime is Amazon Corretto 17.0.18
Spring Boot version?	3.2.0 — so Spring Framework 6.1 and Spring Security 6.2
Build tool?	Maven (wrapper <code>mvnw</code>), Maven 3.8.1
Database?	MongoDB — Spring Data MongoDB, <code>MongoRepository</code> , no SQL/Hibernate
Angular version (web + backoffice)?	Angular 16.2
Angular version (mobile)?	Angular 18.2 — the mobile app is on a newer line
Ionic version?	Ionic (Angular) 8.7.5 with Capacitor 6.2.1
TypeScript?	5.1.3 web · 4.9.5 backoffice · 5.4.5 mobile
RxJS?	7.8 everywhere
State management?	RxJS services with <code>BehaviorSubject</code> — no NgRx in RoboGebra
Standalone components?	No — NgModule-based across all three Angular apps
API docs?	springdoc-openapi 2.2.0 (Swagger UI at <code>/swagger-ui.html</code>)
Auth?	JWT + OAuth2 social login (Google, LinkedIn) + AWS Cognito
Deployment?	Docker (<code>eclipse-temurin:17-jre</code>) + Jenkins CI/CD pipelines
Cloud?	AWS — S3, Cognito, Lambda, SES, SNS

System architecture — 8 repos



Wiring you can quote (from the environment files):

- `robogeбра-web` / `robogeбра-mobile` → `API_URL` = portal on :3000
- `robogeбра-backoffice` → `API_URL` :3000 and `ADMIN_API_URL` :4000 (talks to both backends)
- `robogeбра-portal` → CRM on :8085 (coupons), Node exercise service :8082, Python AI assistant, voice-token server, LiveKit

1. robogeбра-web — frontend / student app

Path: robogeбра-workspace/robogeбра-web · 573 TS files, 182 templates, 8 NgModules

Core framework

Package	Version
@angular/core (and common/forms/router/animations/compiler)	^16.2.0
@angular/cli	~16.2.1
@angular-devkit/build-angular	^16.2.1
@angular/cdk , @angular/material	^16.2.0
TypeScript	~5.1.3
RxJS	~7.8.0
zone.js	~0.13.0

Build configuration (angular.json / tsconfig.json)

- Builder: @angular-devkit/build-angular:browser (the classic browser builder, not esbuild/ application)
- Compile target & module: ES2022
- strict: true , but strictNullChecks: false and strictPropertyInitialization: false
- Initial bundle budget: 8 MB warning/error; component style budget 100 KB
- Build configurations: production , dev , development

UI libraries (three design systems coexist)

Library	Version	Used for
VMware Clarity — @clr/angular , @clr/ui	^15.14.6	Core layout/components

Library	Version	Used for
Clarity Core — @cds/angular , @cds/core	^6.9.2	Web components
PrimeNG + primeicons	^16.1.0 / 7.0.0	Rich widgets
Angular Material	^16.2.0	Selected components
@angular-slider/ngx-slider , ng-circle-progress , ngx-spinner 16.0.2, angular-svg-icon 16.1.0		

Domain-specific libraries (the interesting ones)

Package	Version	Why it's there
mathlive	^0.101.0	Interactive math input/editing
katex	^0.16.11	LaTeX formula rendering
svg-pan-zoom	^3.6.1	Interactive graph pan/zoom
chart.js	^4.5.1	Progress analytics on the dashboard
livekit-client	^2.13.3	Real-time voice AI assistant (WebRTC)
wavesurfer.js	^7.9.5	Audio waveform playback
@gumlet/player.js	^3.0.3	Video player integration
firebase / @angular/fire	^10.1.0 / ^7.0.0	Auth + messaging
razorpay	^2.9.4	Payments
@ngx-translate/core / http-loader	^15.0.0 / ^8.0.0	Tamil/English i18n
ngx-quill	22.1.0	Rich text
posthog-js	1.268.8	Product analytics

Package	Version	Why it's there
<code>dompurify</code>	<code>^3.2.6</code>	XSS sanitization of AI-generated HTML
<code>intl-tel-input</code> 18.5.3, <code>ng-otp-input</code> 1.9.3		Phone/OTP login
<code>moment</code> 2.30.1, <code>jquery</code> 3.7.1, <code>shortid</code>		

Testing

Jasmine ~4.6 · Karma ~6.4 · karma-coverage 2.2

Architecture

`src/app/` → `core/` (services, guards, interceptors, resolvers, models, `AppEndpoints.ts`, `AppRoutes.ts`) · `features/` (pages, robo-chat) · `shared/` · `components/`

Interceptors: `header-authorization.interceptor`, `anonymous-lesson.interceptor`

Guards: `auth-guard`, `login-guard`, `public-preview-guard`

2. robogebra-mobile — Ionic hybrid app

Path: `robogebra-workspace/robogebra-mobile` · 502 TS files, 5 NgModules

▲ This app is on Angular 18, not 16. Know that and know why you'd say it.

Core framework

Package	Version
<code>@ionic/angular</code>	<code>8.7.5</code>
<code>@ionic/angular-toolkit</code>	<code>^12.1.1</code>
<code>@angular/core</code> and friends	<code>^18.2.0</code>

Package	Version
@angular/cli / build-angular	^18.2.0
@angular/material , @angular/cdk	^18.2.0
TypeScript	~5.4.5
RxJS	~7.8.0
zone.js	~0.14.10

Capacitor (native bridge)

Package	Version
@capacitor/core	6.2.1
@capacitor/android , @capacitor/ios	6.2.1
@capacitor/cli	6.2.0

Plugins in use: app 6.0.1 · browser 6.0.3 · device 6.0.2 · haptics 6.0.1 · keyboard 6.0.3 · network 6.0.3 · preferences 6.0.3 · push-notifications 6.0.4 · screen-orientation 6.0.3 · splash-screen 6.0.4 · status-bar 6.0.1 · @capacitor-firebase/messaging 6.3.1 · @capacitor-community/firebase-analytics 6.0.0 · @capawesome/capacitor-app-update 6.1.0 · capacitor-native-settings 6.0.1 · capacitor-razorpay

capacitor.config.ts

```
appId: 'io.robogeбра.one' // Android ('io.robogeбра.app' for iOS)
appName: 'Robogeбра'
webDir: 'www'
server: { androidScheme: 'https' }
plugins: PushNotifications (badge/sound/alert, ic_notification, #E83E6C),
SplashScreen
```

Android build

Setting	Value
minSdkVersion	22 (Android 5.1 Lollipop)
compileSdkVersion / targetSdkVersion	36
Gradle	8.2.1
versionCode / versionName	34 / "1.0"

Mobile-specific libraries

firebase **11.1.0** + @angular/fire 18.0 · mathlive **0.103.0** (newer than web) ·
katex 0.16.11 · quill 2.0.3 / ngx-quill 26.0.10 · swiper **11.1.15** · posthog-js 1.268.8
· @ngx-translate/core 15.0.0 · angular-svg-icon 18.0.1 · dompurify 3.2.6

Tooling (only app with linting configured)

ESLint **9.17** · @angular-eslint/* 18.4.2 · typescript-eslint 8.18.2 · eslint-
plugin-import / jsdoc / prefer-arrow · Jasmine 5.5 · Karma 6.4.4

Dev server

ng serve --port 8100 (Ionic convention)

3. robogebra-backoffice — admin UI

Path: robogebra-workspace/robogebra-backoffice · 497 TS files, 5 NgModules

Core framework

Package	Version
@angular/core	^16.2.12
@angular/cli	~16.2.16
TypeScript	~4.9.5 ← oldest TS in the workspace

Package	Version
RxJS	~7.8.2
zone.js	~0.13.3
Target / module	ES2022 / es2020, <code>strict: true</code>

UI stack (heaviest of the three Angular apps)

Library	Version
Clarity <code>@clr/angular</code> , <code>@clr/ui</code>	~15.14.8
<code>@cds/angular</code> , <code>@cds/core</code> , <code>@cds/city</code>	^6.4.4 / 1.1.0
<code>@porscheinformatik/clr-addons</code>	^15.13.0
PrimeNG	16.9.1
Angular Material / CDK	^16.2.0
<code>@swimlane/ngx-datatable</code>	^20.1.0
<code>ngx-bootstrap</code> / <code>@ng-bootstrap/ng-bootstrap</code>	^11.0.2 / ^15.1.2
<code>@ng-select/ng-select</code>	^11.2.0
FontAwesome (angular 0.10, free 5.15)	

Content-authoring libraries (this is a CMS)

Package	Version	Purpose
codemirror	^5.65.18	In-browser code editing
jsoneditor / <code>ang-jsoneditor</code>	^9.1.8 / ^3.1.1	JSON content editing
<code>@kolkov/angular-editor</code>	3.0.0-beta.2	WYSIWYG
<code>ngx-quill</code> / <code>quill</code>	23.0.3 / ^1.3.7	Rich text

Package	Version	Purpose
<code>ngx-highlightjs</code> / <code>prismjs</code> / <code>prism-themes</code>	<code>^4.1.3</code> / <code>^1.29.0</code>	Syntax highlighting
mathlive / katex	<code>^0.101.0</code> / <code>^0.16.11</code>	Math authoring
<code>@svgdotjs/svg.js</code>	<code>^3.1.2</code>	SVG manipulation
<code>fullcalendar</code>	<code>5.10.2</code>	Scheduling
<code>ngx-file-drop</code> 16.0.0, <code>ngx-filesize</code> , <code>file-saver</code> 2.0.5		Uploads
<code>ngx-color-picker</code> 13.0.0, <code>ngx-chips</code> 3.0.0, <code>ngx-toastr</code> 15.2.2		
<code>@ngx-translate/core</code>	<code>^14.0.0</code> ← older than web/mobile (<code>15.0.0</code>)	
<code>lodash</code> 4.17.21, <code>moment</code> 2.29.1, <code>short-uuid</code> , <code>http-status-codes</code>		

Guards worth mentioning

Six `CanDeactivate` guards protecting unsaved work in the generators (`solution-generator` , `solution-pipeline` , `solution-regenerator` , `solution-source` , `toc-generator` , `preview`) — a good concrete answer to "have you used route guards?"

Dev setup

`proxy.config.json` for local API proxying · Karma/Jasmine 4.x

4. robogebra-portal — main Java backend

Path: `robogebra-workspace/robogebra-portal` · 1,472 Java files across 70 domain packages

Platform

Item	Version
Spring Boot (<code>spring-boot-starter-parent</code>)	3.2.0
↳ implies Spring Framework	6.1.x
↳ implies Spring Security	6.2.x
Java	17 (<code><java.version>17</java.version></code>)
Local JDK	Amazon Corretto 17.0.18 LTS
Build	Maven (<code>mvnw</code> wrapper), Maven 3.8.1
GroupId / Artifact	<code>com.robogebra.cms</code> / <code>robogebra</code> 0.0.1-SNAPSHOT
Server port	3000

Spring starters in use

`spring-boot-starter-web` · `-security` · `-validation` · `-webflux` (for `WebClient`) · `-data-mongodb` · `-mail` · `-thymeleaf` · `-cache` · `-aop` · `-oauth2-client` · `-test`

Key dependencies

Dependency	Version	Purpose
springdoc-openapi-starter-webmvc-ui	2.2.0	Swagger UI <code>/swagger-ui.html</code> , docs <code>/api-docs</code>
Lombok	Boot-managed	Boilerplate reduction (<code>@AllArgsConstructor</code> etc.)
AWS SDK v1 — <code>aws-java-sdk-s3</code> , <code>aws-java-sdk-cognitoidp</code>	1.12.523	S3 storage, Cognito

Dependency	Version	Purpose
AWS SDK v2 — <code>cognitoidentityprovider</code> , <code>ses</code> , <code>sns</code>	2.20.109	Email, SMS, auth
AWS SDK v2 — <code>lambda</code>	2.20.0	Invokes <code>interactive-answer-dev</code> , <code>interactive-graph-dev</code> , <code>semantic_search</code> , <code>validate-coupon</code>
firebase-admin	9.3.0	Push notifications
Quartz Scheduler	2.3.2	Background jobs
Caffeine	3.2.2	In-memory cache behind <code>@Cacheable</code>
java-jwt (Auth0) / jwt-rsa	4.4.0 / 0.22.1	JWT verification
jjwt (api/impl/jackson)	0.11.5	JWT issuing
razorpay-java	1.4.3	Payments
iText kernel / <code>html2pdf</code>	7.2.1 / 4.0.1	PDF generation
Apache POI <code>poi-ooxml</code>	5.2.5	Excel export
opencsv	5.9 (and a stale 4.1 entry)	CSV import
Jakarta <code>servlet-api</code> / <code>annotation-api</code> / <code>validation-api</code>	5.0.0 / 2.1.0 / 3.0.2	
Testcontainers <code>junit-jupiter</code>	1.17.6	Integration tests
<code>spring-security-test</code>	Boot-managed	
<code>netty-resolver-dns-native-macos</code>	4.1.82.Final	Apple-Silicon dev workaround

Data layer

MongoDB — `spring-boot-starter-data-mongodb`, URI `mongodb://localhost:27017/robogebra cms_db`, `auto-index-creation: true`.

Repositories extend `MongoRepository<T, String>` with derived queries (`findByQuizDefinition_IdOrderByOrderAsc`) and `@Aggregation` pipelines for join-style reads that avoid N+1 DBRef loads.

Security architecture (strong talking point)

`SecurityConfig` is **Spring Security 6 style** — a `SecurityFilterChain` `@Bean` , no deprecated `WebSecurityConfigurerAdapter` . It composes **~15 custom filters** in ordered groups:

`LoggingFilter` → `FilterChainExceptionHandler` → `PreferredLanguageFilter`
→ `AnonymousContentAuthenticationFilter` →
`CrmServiceAuthenticationFilter` → `InternalServiceAuthenticationFilter`
→ `WebhookAuthorizationFilter` → `JwtAuthenticationFilter` →
`MultiLoginDetectionFilter` → `BookAccessRestrictionFilter` →
`FeatureUsageLimitFilter`

Plus `JwtAuthenticationEntryPoint` , `CustomUserDetailsService` , `PermissionInterceptor` , and OAuth2 handlers (`SocialAuthSuccessHandler` / `FailureHandler` , `HttpCookieOAuth2AuthorizationRequestRepository` , a custom `LinkedInAuthorizationRequestResolver`).

Auth methods: JWT (24 h expiry, `max-user-login: 3`) · **Google + LinkedIn OAuth2/OIDC** · AWS Cognito.

■ This is your best backend answer. "The filter chain is Chain of Responsibility — each filter has one concern and passes the request on. That's why adding LinkedIn SSO didn't touch JWT auth at all." Ties straight into [17 — SOLID & Design Patterns](#).

`@Configuration` classes (44 total)

`SecurityConfig` · `CacheConfig` · `CorsConfig` · `AwsConfig` · `CognitoConfig` ·
`FireBaseConfig` · `QuartzConfig` · `MailConfig` · `OpenApiConfig` ·
`WebClientConfig` · `WebMvcConfig` · `AuditingConfig` · `RazorPaymentConfig` ·
`EventTrackingConfig`

Package layout (domain-driven)

```
com.robogebra.cms
├─ config/           44 @Configuration classes
├─ security/         ~22 filters, handlers, UserDetailsService
├─ common/           util, model, service, exception, annotation,
properties, importer
├─ domain/           70 feature packages, each:
    │   └─ quiz/
    │       ├── controller/  @RestController
    │       ├── service/     @Service (business logic)
    │       ├── repository/  MongoRepository + @Aggregation
    │       └─ model/        entities + DTOs
```

Sample domains: `quiz`, `selfquiz`, `sectionquiz`, `robochat`, `voiceaiassistant`, `exercisesolution`, `exercisesolutionworkout`, `solutionpipeline`, `learningtimeline`, `parentanalytics`, `studymaterials`, `subscription`, `payment`, `institute`, `institutionlearning`, `usagequota`, `notification`, `webhook`, `refreshtoken`, `otp`, `favourites`, `learningasset`

Deployment

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/robogebra-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 3000
ENTRYPOINT ["java", "-jar", "app.jar"]
```

CI/CD: **Jenkins** — `jenkins/Jenkinsfile-CI` and `jenkins/Jenkinsfile-CD`.
Profiles: `application.yml`, `application-dev.yml`, `application-prod.yml`.
Logging via `logback-spring.xml`.

External services it calls

Python AI assistant (`/api/ai_assistant/generate/text|image|video`, `/api/semantic-search/execute`, `/api/robochat/chat`) · voice-token server (LiveKit) · CRM (coupon validation/redeem) · Node exercise service :8082 · Point2Space Studio

5. robogeбра-admin-portal — admin Java backend

Path: robogeбра-workspace/robogeбра-admin-portal · **218 Java files** · Port **4000**

Same platform as the portal, slimmer dependency set:

Item	Version
Spring Boot	3.2.0
Java	17
Artifact	robogeбра-admin 0.0.1-SNAPSHOT

Starters: web, security, validation, webflux, data-mongodb, mail, thymeleaf, test
Extras: springdoc-openapi 2.2.0 · Lombok · AWS SDK v1 (S3, Cognito) 1.12.523 · razorpay-java 1.4.3 · opencsv 4.1 · Testcontainers 1.17.6 · Docker

How to describe it: "A separate Spring Boot service for admin-only operations, so admin traffic and student traffic scale and deploy independently — the backoffice UI points at both :3000 and :4000 ."

6. robogeбра-crm — management backend (Node.js)

Path: robogeбра-workspace/robogeбра-crm · **166 JS files** · Port **8085**

Not Java. Be upfront: "The CRM is a Node/Express service — I've worked across it, but my primary backend work is the Java/Spring Boot portal."

Item	Version
Node engine	>=12.0.0 (local machine runs v22.12.0)
Express	^4.21.2
Mongoose (MongoDB ODM)	^8.8.0
passport / passport-jwt	^0.7.0 / ^4.0.1

Item	Version
jsonwebtoken / bcryptjs	^9.0.2 / ^2.4.3
Joi (validation)	^17.13.3
helmet / cors / compression	^8.1.0 / ^2.8.5 / ^1.8.1
express-rate-limit / express-mongo-sanitize / xss-clean	^5.5.1 / ^2.2.0 / ^0.1.4
winston / morgan	^3.17.0 / ^1.10.1
multer / nodemailer / axios	^2.0.2 / ^6.10.1 / ^1.11.0
swagger-jsdoc / swagger-ui-express	^6.2.8 / ^4.6.3
PM2 (process manager)	^5.4.3
Jest / supertest / mongodb-memory-server	^30.1.3 / ^7.1.4 / ^10.2.0
ESLint (airbnb-base) / Prettier / Husky	^7.32.0 / ^2.8.8 / ^9.1.7

Structure: `src/{routes, controllers, services, models, middlewares, validations, config, utils, seeders}` — a clean layered API, same controller→service→model separation as the Spring app.

Deployment: Docker + `docker-compose.{dev,test,prod}.yaml` , PM2 via `ecosystem.config.json` . Live at `testcrm.api.robogebra.ai` .

Domain: customers, affiliates, orders, coupons (the portal calls it for coupon validation/redemption).

7. robogebra-crm-web — management UI (React)

Path: `robogebra-workspace/robogebra-crm-web` · 129 TS/TSX files

Not Angular. Your resume already lists React/Formik, so this is consistent — but be clear it's a **React + Vite** app, not Angular.

Item	Version
React / react-dom	^18.3.1

Item	Version
Vite	^5.4.19
@vitejs/plugin-react-swc	^3.11.0
TypeScript	^5.8.3 ← newest TS in the workspace
Tailwind CSS	^3.4.17 (+ @tailwindcss/typography , tailwindcss-animate)
shadcn/ui on Radix UI	~28 @radix-ui/react-* primitives, v1.x–2.x
TanStack React Query	^5.83.0 (+ devtools 5.87.4)
react-router-dom	^6.30.1
react-hook-form + zod + @hookform/resolvers	^7.61.1 / ^3.25.76 / ^3.10.1
recharts	^2.15.4
lucide-react / sonner / cmdk / vaul / embla-carousel	icons, toasts, command palette, drawer, carousel
date-fns / axios / next-themes	^3.6.0 / ^1.11.0 / ^0.3.0
ESLint / typescript-eslint	^9.32.0 / ^8.38.0

Note: the package name is vite_react_shadcn_ts and lovable-tagger is a devDependency — the project was scaffolded with Lovable. Don't claim you hand-built the scaffold; talk about the features you built inside it.

8. robogebra-website — marketing site

Path: robogebra-workspace/robogebra-website

Item	Version
Parcel bundler	^2.13.3
svgo	^3.3.2

Static multi-page HTML: `index` , `about-us` , `leadership` , `team` , `contact-us` , `terms` , `policy` . No framework.

Master version table

Technology	Version	Where
Java	17 (Corretto 17.0.18)	portal, admin-portal
Spring Boot	3.2.0	portal, admin-portal
Spring Framework	6.1.x (via Boot)	portal, admin-portal
Spring Security	6.2.x (via Boot)	portal, admin-portal
Spring Data MongoDB	via Boot 3.2.0	portal, admin-portal
Maven	3.8.1 + <code>mvnw</code> wrapper	portal, admin-portal
springdoc-openapi	2.2.0	portal, admin-portal
MongoDB	server local 27017; Mongoose 8.8 on CRM	all backends
Angular	16.2	web, backoffice
Angular	18.2	mobile
Ionic Angular	8.7.5	mobile
Capacitor	6.2.1	mobile
TypeScript	5.1.3 / 4.9.5 / 5.4.5 / 5.8.3	web / backoffice / mobile / crm-web
RxJS	7.8	all Angular apps
zone.js	0.13 / 0.14	Ang 16 apps / mobile
Angular Material + CDK	16.2 / 18.2	web+backoffice / mobile
Clarity (<code>@clr</code>)	15.14	web, backoffice
PrimeNG	16.1 / 16.9.1	web / backoffice
ngx-translate	15.0 / 14.0	web+mobile / backoffice

Technology	Version	Where
KaTeX	0.16.11	web, mobile, backoffice
MathLive	0.101.0 / 0.103.0	web+backoffice / mobile
Firebase JS SDK	10.1 / 11.1	web / mobile
firebase-admin (Java)	9.3.0	portal
React	18.3.1	crm-web
Vite	5.4.19	crm-web
Tailwind CSS	3.4.17	crm-web
TanStack Query	5.83.0	crm-web
Node.js	engine >=12 , local v22.12.0, npm 11.6.0	crm
Express	4.21.2	crm
Jest	30.1.3	crm
Jasmine / Karma	4.6+5.5 / 6.4	Angular apps
Gradle	8.2.1	mobile/android
Android SDK	min 22, compile/target 36	mobile
Docker base image	eclipse-temurin:17-jre	portal
CI/CD	Jenkins (CI + CD pipelines)	portal, web
AWS SDK	v1 1.12.523 · v2 2.20.109	portal
Quartz	2.3.2	portal
Caffeine	3.2.2	portal
Testcontainers	1.17.6	portal, admin-portal
Parcel	2.13.3	website

▲ Traps in your own stack — read this twice

These are the follow-ups that catch people out. Each one has a *correct, honest* answer.

1. "You said Angular 16 — but your mobile app is 18?"

Yes, and say why: *"The web and backoffice are on 16.2; the Ionic mobile app was built later on Angular 18.2 with Ionic 8 and Capacitor 6. They're separate repos with separate release cycles, so we upgraded the newer app first rather than doing a big-bang upgrade of everything."* That's a strong answer — it shows you understand incremental migration.

2. "Angular 16+ — so you use standalone components?"

No, and you must not bluff this. All three Angular apps are **NgModule-based** — I counted **zero** components with `standalone: true`. Web has 8 NgModules, mobile 5, backoffice 5.

Say: *"Standalone components have been available since 14 and are the default from 17, but our apps are all NgModule-based — they predate the shift and there was no business case to migrate. I understand the model: standalone components declare their own `imports`, you bootstrap with `bootstrapApplication`, and routes can lazy-load with `loadComponent` instead of `loadChildren`."* Then read [04-angular.md](#) on standalone before the call.

3. "How do you manage state — NgRx?"

RoboGebra has no NgRx and no Angular Formly. Your resume lists both — they're from **EasyVisa**, and you must attribute them correctly.

Say: *"In RoboGebra we use RxJS-based service state — shared services with `BehaviorSubject`, `async` pipe in templates, and `takeUntil` for unsubscribe. I used NgRx and Angular Formly on EasyVisa, where the dynamic nested forms justified them."*

4. "Spring Boot 3 — what changed from Spring Boot 2?"

Have this ready, it's the #1 Boot 3 question:

- `javax.*` → `jakarta.*` (Jakarta EE 9+) — the single biggest migration cost
- **Java 17 minimum** (Boot 2 supported Java 8)
- **Spring Security 6** — `WebSecurityConfigurerAdapter` removed; you configure a `SecurityFilterChain` bean (which is exactly what your `SecurityConfig` does)
- Native image support via **GraalVM AOT**
- **Micrometer** observability/tracing built in
- springdoc **2.x** required (1.x doesn't work with Boot 3)
- Trailing-slash matching disabled by default

5. "You listed MySQL/Postgres — but what does RoboGebra use?"

MongoDB only. No SQL database, no Hibernate, no JPA entities in this codebase.

Say: "RoboGebra is MongoDB with Spring Data MongoDB — `MongoRepository`, derived query methods and `@Aggregation` pipelines. I used Postgres and Neo4j on EasyVisa."

▲ There is a `spring-data-jpa` dependency in the POM, but there's no JPA starter, no datasource, and no `@Entity` classes — it's a leftover. **Don't say "we use JPA."** If asked, "there's an unused JPA dependency on the classpath from an earlier direction" is a fine, honest answer.

6. "Java 17 — which Java 17 features do you actually use?"

Be precise; the codebase is conservative:

- `record` — 4 files use it
- `switch` expressions — ~20 files
- **Sealed classes** — none
- **Text blocks** — none
- Heavy use of **Streams** (198 files) and `Optional` (92 files) — Java 8 features

Say: "We're on Java 17 as the platform, but the codebase leans on Java 8 idioms — streams and Optional everywhere — with records and switch expressions used where they fit. I know records, sealed classes, pattern matching for `instanceof`

and text blocks; we just haven't retrofitted the older code." Then revise [12-java17-features.md](#).

7. "@Transactional with MongoDB?"

125 files use `@Transactional`. Know that **MongoDB multi-document transactions require a replica set** (they don't work on a standalone `mongod`). If pushed, say you know transactions are supported from MongoDB 4.0 on replica sets and 4.2 for sharded clusters, and that the document model means most operations are single-document and atomic by nature.

8. "Both AWS SDK v1 and v2?"

Yes — v1 `1.12.523` for S3 and Cognito, v2 `2.20.109` for Cognito/SES/SNS/Lambda. Honest framing: *"The service predates SDK v2, so newer integrations use v2 while the older S3 code is still on v1 — a migration we haven't finished."* That reads as awareness, not sloppiness.

9. "TypeScript 4.9 in backoffice?"

It's the oldest in the workspace. Fine answer: *"The backoffice lags the other apps — TS 4.9.5 and ngx-translate 14 — because it's an internal tool, so upgrades are prioritised behind the customer-facing apps."*

10. "Which build system does the Angular app use?"

`@angular-devkit/build-angular:**browser**` — the **webpack** builder, not the newer esbuild-based `application` builder (which became the default in Angular 17). Worth knowing the distinction if they probe build performance.

Interview answer scripts

■ "Walk me through your tech stack."

"RoboGebra is a microservice-style platform. The student-facing web app is Angular 16.2 with TypeScript 5.1, and the mobile app is Ionic 8 on Angular 18 with

Capacitor 6, shipping to iOS and Android from the same codebase. The main backend is Spring Boot 3.2 on Java 17 with MongoDB — about 70 domain modules, each with controller, service, repository and model layers. There's a separate admin Spring Boot service on port 4000, a Node/Express CRM for customers and coupons, and a React 18 + Vite UI on top of that CRM. Auth is JWT plus Google and LinkedIn OAuth2. It's deployed as Docker images on a temurin-17 base through Jenkins CI/CD, with S3, Cognito, Lambda, SES and SNS on AWS."

■ "Which version of Angular and why not the latest?"

"Web and backoffice are on 16.2, the mobile app on 18.2. We upgrade per-repo when there's a reason rather than chasing releases — the mobile app went to 18 because Ionic 8 and Capacitor 6 pair with it. Angular 16 gave us the things we actually wanted: required inputs, `takeUntilDestroyed`, and the `DestroyRef` API."

■ "Spring Boot version and what you get from it."

"3.2.0, which pulls in Spring Framework 6.1 and Spring Security 6.2. The big consequences are Java 17 as the floor, the `jakarta.*` namespace instead of `javax.*`, and Security 6's component-based config — our `SecurityConfig` is a `SecurityFilterChain` bean composing about fifteen custom filters rather than the old `WebSecurityConfigurerAdapter`."

■ "Tell me about the mobile app." (your strongest resume line)

"I built the Ionic app from the existing Angular web codebase. It's `@ionic/angular` 8.7.5 on Angular 18.2, with Capacitor 6.2.1 as the native bridge — push notifications through Firebase Messaging, plus preferences, network, keyboard, screen orientation, haptics and in-app update plugins. Android targets SDK 36 with a minimum of 22, built with Gradle 8.2.1; we're at version code 34. Payments go through the Razorpay Capacitor plugin. The shared pieces — KaTeX and MathLive for math rendering, ngx-translate for Tamil/English — carried over from the web app."

■ "How big is the codebase you work on?"

"The Java backend alone is about 1,500 files across 70 domain modules — roughly 100 REST controllers and 200 services. The web app is ~570 TypeScript files, mobile ~500, backoffice ~500."

Numbers worth quoting

Metric	Value
Java files (portal)	1,472
Domain packages (portal)	70
@RestController	101
@Service	199
@Repository	84
@Configuration	44
@Transactional	125
@Cacheable	16
@Async	8
@ControllerAdvice	1 (global exception handling)
Java files (admin-portal)	218
TS files — web / mobile / backoffice	573 / 502 / 497
HTML templates (web)	182
JS files (CRM)	166
TS/TSX files (crm-web)	129
Custom security filters	~15
Android version code	34

✓ Re-audit — 3 September 2026

This file was first audited on **31 July 2026**. I re-read the repos on **3 Sept 2026**. **Every version number below was confirmed unchanged.** The counts have grown slightly (the codebase is active) and there are new figures worth quoting.

Confirmed unchanged ✓

Java 17 · Spring Boot 3.2.0 · MongoDB (no JPA) ✓
Angular 16.2 (web + backoffice) · Angular 18.2 (mobile) ✓
Ionic 8.7.5 · Capacitor 6.2.1 · RxJS 7.8 ✓
TypeScript 5.4 (mobile) / 5.1 (web) / 4.9 (backoffice) ✓

Updated counts (3 Sept 2026) ★

Metric	31 Jul	3 Sept
@RestController	101	110
@Service	199	207
@Repository	84	85
@Transactional	125	130
@Async	8	12
@Document (MongoDB)	—	104 ★
@Aggregation pipelines	—	27 ★
@Valid	—	92
Java record	—	26

New figures worth quoting — the frontend ★

```
robogebra-mobile (Angular 18.2 / Ionic 8.7.5):  
  163 components · 89 services · 27 route resolvers · 4 guards · 3  
  interceptors  
  19 Capacitor plugins ★  
  
  RxJS usage:  map 496 · filter 485 · takeUntil 426 ★ · catchError 338  
               finalize 254 · switchMap 246 · BehaviorSubject 122 ★  
               forkJoin 22 · combineLatest 21 · debounceTime 13  
  
  Angular:     async pipe 311 · @Input 252 · ngOnDestroy 100  
               trackBy 55 ★ · inject() 30 · OnPush 6  
               takeUntilDestroyed 5 · standalone 2 ▲ · signals 0 ▲
```

★ **Quote a count, not an adjective.** "Over four hundred `takeUntil` calls" is unforgeable; "we handle subscriptions carefully" is what everyone says.

The seven "do NOT claim" items — confirmed by count ★

```
signals                → 0 ▲ "studied, not shipped"  
standalone components→ 2 ▲ "new code is standalone-first"  
NgRx                   → 0 ▲ that's EasyVisa  
Angular Formly         → 0 ▲ that's EasyVisa  
JPA / Hibernate        → 0 ▲ it's Spring Data MongoDB (an ODM)  
MySQL / Postgres       → 0 ▲ MongoDB only  
Eureka/Feign/circuit breaker/Kafka → 0 ▲ modular MONOLITH
```

▲ The `spring-data-jpa` line in the portal's `pom.xml` is unused — no JPA starter, no `DataSource`, no `@Entity`. It is the single most dangerous trap in your own stack. Confirmed still present on 3 Sept.

New findings not in the original audit ★

AWS	S3 (PutObjectRequest + GeneratePresignedUrlRequest ★) Cognito with THREE user pools ★ + JWKS validation (jwks-rsa)
DEV	LocalStack in Docker – S3 + Cognito emulated locally ★
CI/CD	Jenkinsfile-CI + Jenkinsfile-CD ★
CONFIG	AsyncConfig.java – a dedicated bounded push pool with CallerRunsPolicy, and Boot's applicationTaskExecutor redeclared by hand (@ConditionalOnMissingBean back-off) ★ ★
SECURITY	FilterChainExceptionHandler – because @ControllerAdvice does NOT catch exceptions thrown in a security filter ★
INDEXES	@CompoundIndex on the hot query paths, e.g. {'user.\$id': 1, 'exercise.\$id': 1} on quiz attempts ★

→ All of these are written up as answers in [39 – RoboGebra Code Examples](#).

✓ Pre-interview checklist for this file

- [] Can say **Angular 16.2 / Ionic 8.7.5 / Capacitor 6.2.1 / Angular 18.2 / Spring Boot 3.2.0 / Java 17 / MongoDB** without pausing
- [] Have the **"why two Angular versions"** answer ready
- [] Will **not** claim standalone components, NgRx, Formly, JPA, MySQL or Postgres on RoboGebra
- [] Can name **five Spring Boot 3 changes** from Boot 2
- [] Can describe the **security filter chain** as Chain of Responsibility (17)
- [] Can explain **MongoRepository derived queries + @Aggregation** (13-mongodb.md)
- [] Know the honest limits of your **Java 17 feature usage** (12)
- [] Can draw the **8-repo architecture diagram** from memory

Golden rule: quote the version, then give the *reason*. "Angular 16.2, because..." beats "Angular 16" every time — and it beats "the latest one" by a mile.

JavaScript Variables — `var`, `let`, `const` Made Easy

A slow, from-zero explanation with **real problems you can run**. If [01-javascript.md](#) felt too compressed, start here — then go back to it.

Why this topic is worth a whole file: `var` vs `let` vs `const` is asked in every *single* JavaScript screening round, from fresher to 5 years. And the follow-up — "what will this loop print?" — is the single most common trick question in the language. Get this airtight and you start every interview on the front foot.

◆ The easiest way to remember this whole file — three pictures

1 Scope = ROOMS in a house

```
var   → ignores the walls. Declare it in a room, it LEAKS into the corridor ▲  
let   → stays in its room ✓  
const → stays in its room, and the nameplate is screwed on ✓
```

2 Hoisting = a GUEST LIST read out before the party

Every name is REGISTERED before anyone arrives (the declaration moves up). But the guests arrive at their own time (the assignment stays put). ★

```
var guests   → registered as "unknown person" → you get `undefined` ▲  
let/const    → registered but BARRED at the door until their moment  
              → ReferenceError, which is BETTER (it fails loudly) ★
```

3 `const` = a fixed ADDRESS, movable furniture

```
const user = { name: 'Priya' };  
user.name = 'Ravi';           ✓ rearranged the furniture  
user = { name: 'Ravi' };      ✗ you cannot move the house ★
```

★ **Every question in this file is one of those three pictures.** The loop problem (§9) is picture 1. The TDZ (§8) is picture 2. The "but I changed the array!" confusion (§6) is picture 3.

Easy memory

- | | |
|-------------|--|
| 1. SCOPE | var leaks out of blocks let/const stay in ★ |
| 2. HOISTING | declarations move up, assignments stay
var → undefined ▲ let/const → TDZ ReferenceError ★ |
| 3. const | locks the ADDRESS, not the CONTENTS ★ |

Table of contents

- 1. What a variable actually is
- 2. The 4 questions that define every declaration
- 3. Scope — where can I see this variable?
- 4. `var` — the old one
- 5. `let` — the fixed one
- 6. `const` — the one everyone misunderstands
- 7. Hoisting — what really happens before your code runs
- 8. The Temporal Dead Zone (TDZ)
- 9. ▲ THE loop problem
- 10. Real bugs from real code
- 11. Practice problems (with answers)
- 12. Output-based drills
- 13. Interview Q&A
- 14. One-page cheat sheet

1. What a variable actually is

A variable is a **name that points to a value**.

```
let price = 100;
```

Three separate things are happening:

Step	Name	What happens
1	Declaration	JavaScript creates a name <code>price</code> in memory
2	Initialization	That name is given a starting value
3	Assignment	<code>100</code> is stored under that name

You can split them:

```
let price;           // declaration only → price is `undefined`
price = 100;         // assignment
price = 200;         // re-assignment
```

Keep these three words separate in your head. **Every confusing thing about `var` / `let` / `const` comes down to *when* each step happens.**

2. The 4 questions that define every declaration

Whenever you see `var`, `let` or `const`, ask these four questions. That's the whole topic.

#	Question	var	let	const
1	What scope?	Function	Block	Block
2	Can I re-assign it?	✓ Yes	✓ Yes	✗ No

#	Question	var	let	const
3	Can I re-declare it in the same scope?	✓ Yes	✗ No	✗ No
4	What if I use it before declaring?	undefined	✗ ReferenceError (TDZ)	✗ ReferenceError (TDZ)

Plus one bonus:

| 5 | Does it attach to **window**? | ✓ Yes (at top level) | ✗ No | ✗ No |

If you memorize only one table in this file, memorize this one. Every question in this document is just one of these five rows in disguise.

3. Scope — where can I see this variable?

Scope = the region of code where a variable is visible.

There are three kinds:

```

let planet = "Earth";           // 1 GLOBAL scope – visible everywhere

function greet() {
  let city = "Chennai";        // 2 FUNCTION scope – only inside
  greet()

  if (true) {
    let street = "Anna Salai";  // 3 BLOCK scope – only inside these
  }
  console.log(planet, city, street); // ✓ all three visible
}

console.log(street);           // ✗ ReferenceError – street died at
the }
}

```

What is a "block"?

Anything between `{` and `}` that isn't a function body:

```
if (x) { /* block */ }
for (...) { /* block */ }
while (x) { /* block */ }
{ /* even a bare block is a block */ }
```

The one-line rule that explains everything:

◆ **var** ignores blocks. **let** and **const** respect them.

Watch it happen:

```
function test() {
  if (true) {
    var a = 1;      // var: escapes the if-block, belongs to test()
    let b = 2;      // let: trapped inside the if-block
  }

  console.log(a);   // ✓ 1
  console.log(b);   // ✗ ReferenceError: b is not defined
}
```

Say this in an interview: "**var** is function-scoped, so it leaks out of **if** and **for** blocks. **let** and **const** are block-scoped, so they're confined to the nearest pair of braces."

4. **var** — the old one

var is the original JavaScript declaration (pre-2015). It has **three behaviours that cause bugs**.

Problem 1 — it leaks out of blocks

```
if (true) {  
  var role = "admin";  
}  
console.log(role);           // "admin" ← still alive outside the if!
```

You expected `role` to be temporary. It isn't.

Problem 2 — it can be silently re-declared

```
var user = "Priyanka";  
// ... 200 lines later, a teammate writes:  
var user = "Admin";           // ✓ No error. No warning. Your value is gone.  
  
console.log(user);           // "Admin"
```

This is a real production bug pattern in large files. `let` makes it a hard error instead.

Problem 3 — it pollutes `window`

```
var appName = "RoboGebra";  
console.log(window.appName);  // "RoboGebra" ← now a global property  
  
let appVersion = "1.0";  
console.log(window.appVersion); // undefined ← let stays clean
```

Two libraries both doing `var $ = ...` will overwrite each other.

So when do you use `var`?

You don't. In modern code (ES6+, TypeScript, Angular) `var` has **no remaining use case**.

■ **Interview answer:** "I don't use `var` in new code. It's function-scoped, silently re-declarable, and attaches to `window`, which causes bugs that `let` and `const` prevent at compile time. I only need to understand it for reading legacy code."

5. `let` — the fixed one

`let` (ES6, 2015) fixes all three `var` problems.

```
let count = 0;
count = 1;           // ✓ re-assign – fine
count = count + 1;   // ✓ fine

let count = 5;       //
✗ SyntaxError: Identifier 'count' has already been declared
```

Block-scoped, as promised:

```
let x = "outer";
{
  let x = "inner";           // ✓ different variable – this is "shadowing",
  not re-declaring           // "inner"
  console.log(x);
}
console.log(x);              // "outer"
```

▲ **Shadowing vs re-declaring.** Same name in a **different** scope = shadowing (allowed). Same name in the **same** scope = re-declaring (error with `let`).

Use `let` when the value genuinely changes

```
let attempts = 0;
let currentUser = null;
let total = 0;

for (let i = 0; i < items.length; i++) { // i changes each iteration
  total += items[i].price;               // total changes
}
```

6. `const` — the one everyone misunderstands

`const` means the **binding** can't be reassigned. It does **not** mean the value is frozen.

Rule 1 — must be initialized immediately

```
const rate = 0.18;    // ✓  
const tax;            // ✗ SyntaxError: Missing initializer in const  
declaration
```

Rule 2 — cannot be re-assigned

```
const rate = 0.18;  
rate = 0.20;          // ✗ TypeError: Assignment to constant variable.
```

Rule 3 — ▲ objects and arrays CAN still be changed

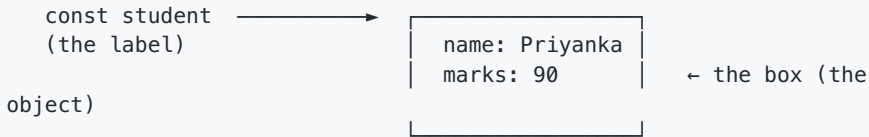
This is the #1 misunderstanding in JavaScript interviews.

```
const student = { name: "Priyanka", marks: 90 };  
  
student.marks = 95;          // ✓ ALLOWED — changing what's *inside*  
the object  
student.grade = "A";        // ✓ ALLOWED — adding a property  
delete student.grade;       // ✓ ALLOWED  
  
student = { name: "Someone" }; // ✗ TypeError — pointing the name at a  
NEW object
```

```
const marks = [90, 85, 78];  
  
marks.push(100);            // ✓ ALLOWED → [90, 85, 78, 100]  
marks[0] = 95;              // ✓ ALLOWED  
marks.length = 0;          // ✓ ALLOWED (empties it!)  
  
marks = [1, 2, 3];         // ✗ TypeError
```

◆ The mental model that makes this click

Think of a **variable** as a **label stuck on a box**:



const stops you MOVING THE LABEL to a different box.
It does NOT stop you REARRANGING WHAT'S INSIDE the box.

- `student.marks = 95` → rearranging inside the box → ✓ fine
- `student = {...}` → moving the label to a new box → ✗ blocked by `const`

Why does it work this way? Because objects and arrays are stored **by reference**. The variable holds a *pointer* to the object, not the object itself. `const` protects the pointer, not the thing it points to.

■ **Interview answer (say it exactly like this):** "`const` prevents re-assignment of the binding, not mutation of the value. For an object or array, the reference is constant but the contents are still mutable. If I need real immutability I'd use `Object.freeze()`, or `readonly` / `ReadonlyArray<T>` in TypeScript."

If you actually want immutability

```
const config = Object.freeze({ apiUrl: "https://api.robogebra.ai",
  retries: 3 });

config.retries = 5; // silently ignored (throws in strict mode)
console.log(config.retries); // 3
```

▲ `Object.freeze()` is **shallow** — nested objects are still mutable:
`js const settings = Object.freeze({ user: { name: "Priyanka" } });`
`settings.user.name = "Changed";` // ✓ still works! freeze didn't go deep

For deep freezing you recurse over the properties yourself.

In TypeScript (your daily language)

```
const rate = 0.18; // value can't be
reassigned

interface Config { readonly apiUrl: string; } // property can't be
reassigned
const c: Config = { apiUrl: "..." };
c.apiUrl = "x"; // x compile error

const ids: ReadonlyArray<number> = [1, 2, 3];
ids.push(4); // x compile error
```

- ◆ TypeScript's **readonly** is **compile-time only** — it disappears at runtime. **Object.freeze()** is the runtime guarantee. Good detail to mention.

7. Hoisting — what really happens before your code runs

Hoisting is the #2 most-asked question here. Here's the honest explanation.

The two-phase model

JavaScript does **not** run your code top-to-bottom in one pass. It runs each scope in **two phases**:

```
PHASE 1 – CREATION (before any line executes)
  → Scan the scope
  → Register every declared name in memory
  → var names → set to `undefined`
  → let/const names → registered but marked "not initialized" (TDZ)
  → function declarations → fully stored, body and all

PHASE 2 – EXECUTION (now run line by line)
  → Assignments actually happen
```

Why `var` prints `undefined` instead of crashing

```
console.log(name);    // undefined ← not an error!  
var name = "Priyanka";  
console.log(name);    // "Priyanka"
```

What JavaScript effectively does:

```
var name = undefined;    // ← Phase 1 put this here  
console.log(name);       // undefined  
name = "Priyanka";       // ← Phase 2: only the assignment stayed put  
console.log(name);       // "Priyanka"
```

◆ Only the *declaration* is hoisted, never the *assignment*.

Function declarations are hoisted completely

```
greet();                // ✓ "Hello!" – works before it's written  
  
function greet() {  
  console.log("Hello!");  
}
```

Function expressions are NOT

```
greet();                // ✗ TypeError: greet is not a function  
  
var greet = function () {  
  console.log("Hello!");  
};
```

```
greet();  
before initialization    // ✗ ReferenceError: Cannot access 'greet'  
  
const greet = () => {  
  console.log("Hello!");  
};
```

■ **Notice the two different errors.** `TypeError` vs `ReferenceError` is a classic interview trap — see [drill 4](#).

8. The Temporal Dead Zone (TDZ)

Scary name, simple idea.

TDZ = the gap between when a `let` / `const` variable is created and when it's initialized. Touching it in that gap throws a `ReferenceError`.

```
{
  // ↓ TDZ for `x` starts here — x exists, but is unusable
  console.log(x);    // ✗ ReferenceError: Cannot access 'x' before
  initialization
  let x = 10;        // ↑ TDZ ends here
  console.log(x);    // ✓ 10
}
```

Proof `let` really is hoisted

People often say "`let` isn't hoisted." That's wrong — and interviewers love catching it. Compare:

```
console.log(a);    // ✗ ReferenceError: a is not defined    ← never
declared anywhere
```

```
console.log(b);    // ✗ ReferenceError: Cannot access 'b' before
initialization
let b = 1;          // ↑ different message!
```

Two **different** error messages. The second one proves JavaScript already *knows* `b` exists — it's hoisted, just parked in the TDZ.

■ **Interview answer:** "`let` and `const` are hoisted, but unlike `var` they aren't initialized to `undefined`. They sit in the Temporal Dead Zone until the declaration

line executes, and accessing them there throws a **ReferenceError**. That's deliberate — it turns a silent **undefined** bug into a loud error."

Why the TDZ is a good thing

```
// With var – silent wrong answer:
console.log(discount);    // undefined
var discount = 0.10;
const price = 100 * (1 - discount);    // NaN – and no one notices until
production

// With let – immediate, obvious crash:
console.log(discount);    // ✗ ReferenceError – you fix it in 5 seconds
let discount = 0.10;
```

9. ▲ THE loop problem — the most asked question

If you learn one thing from this file, learn this. It comes up constantly.

The problem

```
for (var i = 1; i <= 3; i++) {
  setTimeout(function () {
    console.log(i);
  }, 1000);
}
```

What most people say: 1, 2, 3

What actually prints: 4, 4, 4

Why — step by step

Step 1. **var i** is **function-scoped**, not block-scoped. So there is exactly **ONE** **i** shared by all three iterations.

```
Iteration 1 ┌──┐
Iteration 2 ┌──┐ → the SAME single `i` in memory
Iteration 3 ┌──┐
```


Step 2. `setTimeout` doesn't run the callback now. It schedules it for **later** and moves on.

Step 3. The loop finishes first. Trace it:

Moment	i	What happens
Start	1	schedule callback #1, <code>i++</code>
	2	schedule callback #2, <code>i++</code>
	3	schedule callback #3, <code>i++</code>
	4	<code>4 <= 3</code> is false → loop exits
1 second later	4	callback #1 runs → reads <code>i</code> → 4
	4	callback #2 runs → 4
	4	callback #3 runs → 4

By the time the callbacks run, the loop is long finished and the one shared `i` is stuck at `4`.

✓ **Fix 1 — just use `let` (the modern answer)**

```
for (let i = 1; i <= 3; i++) {
  setTimeout(function () {
    console.log(i);
  }, 1000);
}
// 1, 2, 3 ✓
```

Why this works: `let` in a `for` loop creates a **brand-new binding for every iteration**. This is a special rule in the spec — the loop head is treated as its own block, and the value is copied forward each pass.

```
Iteration 1 → i = 1    (its own copy)
Iteration 2 → i = 2    (its own copy)
Iteration 3 → i = 3    (its own copy)
```

Each callback closes over its **own** `i`. Done.

✓ Fix 2 — an IIFE (the pre-ES6 answer)

```
for (var i = 1; i <= 3; i++) {  
  (function (copy) {  
    setTimeout(function () {  
      console.log(copy);  
    }, 1000);  
  })(i);           // pass the CURRENT value in as an argument  
}  
// 1, 2, 3 ✓
```

Each call to the immediately-invoked function creates a new function scope with its own `copy` parameter.

✓ Fix 3 — bind the value as an argument

```
for (var i = 1; i <= 3; i++) {  
  setTimeout(console.log, 1000, i); // extra args to setTimeout are  
  passed to the callback  
}  
// 1, 2, 3 ✓
```

■ **Full interview answer:** "It prints 4, 4, 4. `var` is function-scoped, so all three callbacks close over the same `i`, and by the time they run — asynchronously, after the loop — `i` is 4. Changing `var` to `let` fixes it, because `let` creates a fresh binding per iteration. Before ES6 you'd wrap it in an IIFE to capture the value."

▲ The `const` variant

```
for (const i = 0; i < 3; i++) { }  
// ✗ TypeError: Assignment to constant variable. (i++ can't reassign a const)  
  
for (const item of items) { }  
// ✓ Perfectly fine! for...of creates a NEW binding each iteration —  
nothing is reassigned
```

for...of with **const** is the idiomatic modern loop. Know the difference — it's a good follow-up question.

Easy memory — THE loop problem ★★

```
for (var i ...)  setTimeout(() => log(i))  →  4, 4, 4  ▲  
for (let i ...)  setTimeout(() => log(i))  →  1, 2, 3  ✓ ★
```

WHY: ``var`` = ONE shared ``i`` for the whole loop.

By the time the callbacks run, the loop has FINISHED and `i` is 4 ★

``let`` = a NEW binding EVERY iteration, so each callback captured its own copy ★

Real-world idea: **one shared whiteboard vs three separate notepads**. With `var`, all three callbacks read the same whiteboard — and by the time they look, it says 4. With `let`, each one wrote the number on its own notepad at the time.

The three fixes, in order of what an interviewer wants to hear:

1. use ``let`` ★ the modern answer
2. an IIFE per iteration (the pre-ES6 answer — shows you know why)
3. `setTimeout`'s 3rd argument (pass `i` in as an argument)

★ **Say why, not just what:** "`var` is function-scoped, so all three closures captured the same variable. `let` is block-scoped and the spec creates a fresh binding per iteration, so each closure captured its own."

10. Real bugs from real code

These are the shapes this topic takes in an Angular/TypeScript job — much more convincing than textbook examples.

Bug 1 — event listeners in a loop

```
const buttons = document.querySelectorAll(".chapter-btn");

// ✗ Every button alerts the LAST index
for (var i = 0; i < buttons.length; i++) {
  buttons[i].addEventListener("click", () => alert("Chapter " + i));
}

// ✓ Fix
for (let i = 0; i < buttons.length; i++) {
  buttons[i].addEventListener("click", () => alert("Chapter " + i));
}
```

Same root cause as the `setTimeout` problem: the callback runs *later*, and `var` gave everyone the same `i`.

Bug 2 — mutating a `const` array shared across components

```
// shared.constants.ts
export const DEFAULT_FILTERS = ["algebra", "geometry"];

// some.component.ts
ngOnInit() {
  this.filters = DEFAULT_FILTERS;
  this.filters.push("calculus"); //
  ▲ mutates the SHARED array for the whole app!
}
```

`const` did **not** protect you — `push` mutates, it doesn't reassign. Every other component now sees `"calculus"`.

Fixes:

```
this.filters = [...DEFAULT_FILTERS]; // copy before mutating
// or make the source genuinely safe:
export const DEFAULT_FILTERS: ReadonlyArray<string> =
Object.freeze(["algebra", "geometry"]);
```

◆ This is an excellent thing to mention in an interview: *"The `const` + shared-array mutation bug is one I actively watch for in Angular — a constant exported from a*

shared module gets mutated by one component and silently changes behaviour everywhere."

Bug 3 — `var` inside a `switch`

```
switch (type) {
  case "quiz":
    var handler = quizHandler;    // leaks to the whole function
    break;
  case "lesson":
    var handler = lessonHandler; // silent re-declaration – no error
    break;
}
```

With `let` you'd get `SyntaxError: Identifier 'handler' has already been declared` — because a `switch` body is **one single block**. The correct fix is braces per case:

```
switch (type) {
  case "quiz": {
    let handler = quizHandler;    // ✓ own block
    break;
  }
  case "lesson": {
    let handler = lessonHandler; // ✓ own block
    break;
  }
}
```

Bug 4 — a `let` you meant to declare outside

```
let subscription;                                // declared outside ✓

ngOnInit() {
  this.data$.subscribe(d => {
    let subscription = somethingElse;           // ✗ shadows the outer one
  });
}

ngOnDestroy() {
  subscription.unsubscribe();                   // ✗ undefined — memory
  leak
}
```

Shadowing is legal, which makes this bug quiet. Always check whether you meant to declare a *new* variable or assign to an existing one.

Bug 5 — accidental global (no keyword at all)

```
function calculateTotal() {
  total = 0;                                // ✗ no let/const/var → creates a GLOBAL
  variable
  // ...
}
```

In non-strict mode this silently creates `window.total`. In `"use strict"` (and every ES module, and all TypeScript) it throws `ReferenceError: total is not defined` — one more reason strict mode is on by default in modern code.

11. Practice problems

Try each one **before** reading the answer. Write down your prediction first.

Problem 1

```
function test() {  
  console.log(a);  
  console.log(b);  
  var a = 1;  
  let b = 2;  
}  
test();
```

→ Answer

```
undefined  
✗ ReferenceError: Cannot access 'b' before initialization
```

`var a` is hoisted and initialized to `undefined`, so line 1 prints `undefined`. `let b` is hoisted but sits in the TDZ, so line 2 throws — and execution stops there.

Problem 2

```
const arr = [1, 2, 3];  
arr.push(4);  
console.log(arr);  
  
arr = [5, 6];  
console.log(arr);
```

→ Answer

```
[1, 2, 3, 4]  
✗ TypeError: Assignment to constant variable.
```

`push` mutates the existing array — allowed. `arr = [5, 6]` re-assigns the binding — blocked.

Problem 3

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log("var:", i), 0);  
}  
for (let j = 0; j < 3; j++) {  
  setTimeout(() => console.log("let:", j), 0);  
}
```

→ Answer

```
var: 3  
var: 3  
var: 3  
let: 0  
let: 1  
let: 2
```

One shared `i`` (ends at 3) vs a fresh `j`` per iteration. Note all six run *after* both loops finish, because `setTimeout`` is async even with a `0`` delay.

Problem 4

```
var x = 10;  
  
function show() {  
  console.log(x);  
  var x = 20;  
}  
show();
```

→ Answer

```
undefined
```

The **inner** `var x`` is hoisted to the top of `show()`` and initialized to `undefined``. It shadows the global `x`` for the entire function body — so `console.log(x)`` reads the local one, not the global `10``. This is the classic hoisting-plus-shadowing trap.

Problem 5

```
let count = 0;
{
  let count = 10;
  count++;
}
console.log(count);
```

→ Answer

```
0
```

The inner `count` is a completely separate variable that lives and dies inside the block. The outer one was never touched.

Problem 6

```
const student = { name: "Priyanka", skills: ["Angular"] };

student.skills.push("Java");
student.name = "Priyanka R";
Object.freeze(student);
student.name = "Changed";
student.skills.push("Spring");

console.log(student);
```

→ Answer

```
{ name: "Priyanka R", skills: ["Angular", "Java", "Spring"] }
```

Line by line: `push` and the `name` change happen before the freeze, so both apply. After `Object.freeze`, `student.name = "Changed"` is silently ignored. But **freeze is shallow** — the nested `skills` array is *not* frozen, so `push("Spring")` still works.

Problem 7

```
function counter() {  
  var count = 0;  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const c1 = counter();  
const c2 = counter();  
  
console.log(c1(), c1(), c2());
```

→ Answer

```
1 2 1
```

Each call to `counter()` creates a **new** function scope with its own `count`. `c1` and `c2` are independent closures. (`var` is fine here — it's function-scoped and there's one function scope per call.)

Problem 8 — write the code

Write a loop that prints **1** after 1 second, **2** after 2 seconds, and **3** after 3 seconds. Then write the same thing **without** using `let`.

→ Answer

```
// With let  
for (let i = 1; i <= 3; i++) {  
  setTimeout(() => console.log(i), i * 1000);  
}  
  
// Without let - IIFE  
for (var i = 1; i <= 3; i++) {  
  (function (n) {  
    setTimeout(function () { console.log(n); }, n * 1000);  
  })(i);  
}
```

12. Output-based drills

Rapid-fire. Cover the right column and answer out loud.

Code	Output
<code>console.log(a); var a = 1;</code>	<code>undefined</code>
<code>console.log(b); let b = 1;</code>	<code>✗ ReferenceError (TDZ)</code>
<code>console.log(c);</code> (never declared)	<code>✗ ReferenceError: c is not defined</code>
<code>let x = 1; let x = 2;</code>	<code>✗ SyntaxError: already declared</code>
<code>var y = 1; var y = 2;</code> <code>console.log(y);</code>	<code>2</code>
<code>const z = {a:1}; z.a = 2;</code> <code>console.log(z.a);</code>	<code>2</code>
<code>const z = {a:1}; z = {a:2};</code>	<code>✗ TypeError</code>
<code>if(true){var m=1} console.log(m);</code>	<code>1</code>
<code>if(true){let n=1} console.log(n);</code>	<code>✗ ReferenceError</code>
<code>for(var i=0;i<3;i++){}</code> <code>console.log(i);</code>	<code>3</code>
<code>for(let j=0;j<3;j++){}</code> <code>console.log(j);</code>	<code>✗ ReferenceError</code>
<code>const arr=[1]; arr.length=0;</code> <code>console.log(arr);</code>	<code>[]</code>
<code>var v=1; console.log(window.v);</code>	<code>1</code>
<code>let l=1; console.log(window.l);</code>	<code>undefined</code>
<code>f(); function f()</code> <code>{console.log("hi")}</code>	<code>hi</code>
<code>g(); var g=function(){};</code>	<code>✗ TypeError: g is not a function</code>
<code>h(); const h={()=>{};}</code>	<code>✗ ReferenceError (TDZ)</code>
<code>const o=Object.freeze({a:1});</code> <code>o.a=9; console.log(o.a);</code>	<code>1</code>

13. Interview Q&A

Q: Difference between `var`, `let` and `const`?

"Three differences that matter. Scope — `var` is function-scoped, so it leaks out of `if` and `for` blocks; `let` and `const` are block-scoped. Re-assignment — `var` and `let` can be reassigned, `const` can't. Hoisting — all three are hoisted, but `var` is initialized to `undefined` while `let` and `const` sit in the Temporal Dead Zone and throw a `ReferenceError` if you touch them early. `var` also attaches to `window` at the top level; `let` and `const` don't."

Q: Which one do you use by default?

"`const` by default, `let` when the value genuinely needs to change, and `var` never in new code. Defaulting to `const` documents intent — a reader can see at a glance which variables are stable — and it prevents accidental reassignment."

Q: Does `const` make a value immutable?

"No. `const` prevents reassigning the binding, not mutating the value. `const arr = [1,2]; arr.push(3)` is legal because the reference hasn't changed. For actual immutability you'd use `Object.freeze()` at runtime — though that's shallow — or `readonly` and `ReadonlyArray<T>` in TypeScript at compile time."

Q: Is `let` hoisted?

"Yes — this is a common misconception. `let` and `const` are hoisted, but not initialized. They're in the TDZ until their declaration executes. You can prove it: accessing an undeclared variable gives 'x is not defined', while accessing a `let` before its declaration gives 'Cannot access x before initialization' — a different error, which means the engine already knows the variable exists."

Q: Why does `for (var i...)` with `setTimeout` print the wrong values?

(See the full answer in [section 9](#).)

Q: What's the TDZ for, conceptually?

"It converts a silent bug into a loud one. With `var`, using a variable too early gives you `undefined`, which then quietly propagates as `NaN` or a wrong branch. With `let`, you get an immediate `ReferenceError` at the exact line."

Q: Can you re-declare a variable?

"`var` yes, silently. `let` and `const` no — it's a `SyntaxError` in the same scope. But shadowing in a nested scope is allowed for all three, and that's a different thing: a new variable with the same name in an inner block."

14. One-page cheat sheet

	var	let	const
Scope	function	block	block
Re-assign	✓ yes	✓ yes	✗ no
Re-declare	✓ yes	✗ no	✗ no
Hoisted	✓ yes	✓ yes	✓ yes
Initialized	undefined	✗ TDZ	✗ TDZ
Use before decl.	undefined	ReferenceErr	ReferenceErr
Must initialize	✗ no	✗ no	✓ YES
On window	✓ yes	✗ no	✗ no

THE FOUR SENTENCES THAT COVER 90% OF QUESTIONS

1. var ignores blocks. let and const respect them.
2. All three are hoisted – var to `undefined`, let/const into the TDZ.
3. const locks the binding, not the contents.
4. `let` in a for-loop creates a fresh binding every iteration; `var` doesn't.

DAILY RULE

- const → default. Use it until the code forces you not to.
- let → only when the value is genuinely reassigned.
- var → never.

Where to go next

- [01-javascript.md](#) — closures, `this`, the event loop, promises (the loop problem sits right next to closures — study them together)
- [03-typescript.md](#) — `readonly`, `as const`, and how TypeScript layers compile-time safety on top of all of this
- [09-coding-problems.md](#) — practice problems where scope bugs actually bite

RxJS — Interview Questions & Operator Guide

The deep-dive companion to the short RxJS section in [04-angular.md](#). Written for an **Angular developer with 5 years' experience** — every example is a real Angular scenario, not a toy.

Your stack context: RoboGebra runs **RxJS 7.8** on Angular 16.2 (web/backoffice) and Angular 18.2 (mobile), with **service + BehaviorSubject** state — no NgRx. That's exactly the setup this file targets. See [18 — Technical Versions](#).

Why RxJS decides Angular interviews: anyone can list lifecycle hooks. What separates a 2-year candidate from a 5-year one is **switchMap** vs **mergeMap**, why a **Subject** leaked memory, and what **shareReplay** actually does. Expect **20–30% of an Angular technical round** to be RxJS.

Table of contents

- [▲ The 60-second cheat sheet](#)
- [Part 1 — Fundamentals](#)
- [Part 2 — Subjects](#)
- [Part 3 — Creation operators](#)
- [Part 4 — Transformation & the flattening four](#)
- [Part 5 — Filtering operators](#)
- [Part 6 — Combination operators](#)
- [Part 7 — Error handling](#)
- [Part 8 — Utility & multicasting](#)
- [Part 9 — Unsubscribing & memory leaks](#)
- [Part 10 — Real Angular scenarios](#)
- [Part 11 — Common mistakes](#)
- [Part 12 — RxJS 7 changes](#)
- [Part 13 — Output-based drills](#)

▲ The 60-second cheat sheet

Question	Answer
Observable vs Promise?	Observable = multiple values, lazy, cancellable, operators. Promise = one value, eager, not cancellable
Cold vs hot?	Cold = new producer per subscriber (HTTP). Hot = shared producer (Subject, <code>fromEvent</code>)
<code>Subject</code> vs <code>BehaviorSubject</code> ?	<code>BehaviorSubject</code> needs an initial value and replays the latest to new subscribers
Search box operator?	<code>debounceTime</code> → <code>distinctUntilChanged</code> → <code>switchMap</code>
Save button operator?	<code>exhaustMap</code> (ignore double-clicks)
Ordered writes?	<code>concatMap</code>
Independent parallel calls?	<code>mergeMap</code>
Parallel HTTP, wait for all?	<code>forkJoin</code>
React whenever any input changes?	<code>combineLatest</code>
How do you unsubscribe?	<code>async</code> pipe first; else <code>takeUntilDestroyed()</code> (Angular 16+) or <code>takeUntil(destroy\$)</code>
Cache an HTTP result?	<code>shareReplay({ bufferSize: 1, refCount: true })</code>
Retry a failed call?	<code>retry({ count: 3, delay: 1000 })</code> (RxJS 7)
Observable → Promise?	<code>firstValueFrom()</code> / <code>lastValueFrom()</code> — <code>toPromise()</code> is deprecated

◆ The easiest way to remember ALL of RxJS

Everything in this file hangs off **three ideas**. If you can say these, you can reason your way to any answer.

1. A stream is a CONVEYOR BELT.
Values arrive over time. Operators are MACHINES sitting on the belt.
Nothing moves until someone switches it on (subscribe). ★

2. An ERROR is a BROKEN BELT. It stops forever.
→ which is why WHERE you put catchError decides whether the belt survives ★

3. A subscription is a RUNNING MOTOR.
If you never switch it off, it keeps running after the component is gone.
→ that's a memory leak ★

The real-world analogy for every operator group

Group	Analogy	Operators
Creation	starting the belt	of from timer interval fromEvent
Transformation	a machine that reshapes each item	map scan
Flattening	what to do when a <i>new order</i> arrives mid-job	switchMap mergeMap concatMap exhaustMap ★
Filtering	a quality-control gate	filter debounceTime distinctUntilChanged take
Combination	merging two belts into one	forkJoin combineLatest zip merge
Error handling	a repair station	catchError retry finalize

Group	Analogy	Operators
Multicasting	one belt feeding several packers	<code>share</code> <code>shareReplay</code>

Easy memory

Stream = a CONVEYOR BELT – lazy until subscribed ★

Error = the belt BREAKS permanently ★

Subscription = a MOTOR you must switch off ★

The 3 answers that decide the round:

1. the four flattening operators, and when each is WRONG
2. WHERE catchError goes, and why
3. how you prevent subscription leaks

Part 1 — Fundamentals

Q: What is RxJS and why does Angular use it?

RxJS is a library for **reactive programming with observable streams**. Angular uses it because most of what an app does is asynchronous *and repeated over time*: HTTP responses, router events, form value changes, DOM events, WebSocket messages.

Angular ships RxJS into `HttpClient`, `Router` (`events`, `params`, `queryParams`), reactive forms (`valueChanges`, `statusChanges`), and `EventEmitter`.

■ "A Promise models one future value. RxJS models a stream of values over time, with operators to transform, filter and combine those streams declaratively — and, crucially, to cancel them."

Q: Observable vs Promise?

	Observable	Promise
Values	Many over time	Exactly one
Execution	Lazy — nothing runs until <code>.subscribe()</code>	Eager — runs the moment it's created
Cancellable	✓ <code>.unsubscribe()</code>	✗ No
Operators	✓ 100+ (<code>map</code> , <code>filter</code> , <code>retry</code> ...)	✗ Only <code>.then</code> / <code>.catch</code> / <code>.finally</code>
Retry	✓ Built in	✗ Manual
Sync or async	Either	Always async

```
// Promise – fires immediately, even if nobody uses it
const p = fetch('/api/quiz');
```

```
// Observable – nothing happens until you subscribe
const o$ = this.http.get('/api/quiz');
o$.subscribe(); // ← only NOW does the request go out
```

◆ **The laziness point wins interviews.** *"Because observables are lazy, `this.http.get(...)` on its own sends no request. That's also why calling `subscribe()` twice fires two HTTP calls — a bug people hit constantly."*

Q: What are the three Observer callbacks?

```
source$.subscribe({
  next:    value => console.log('value:', value),    // 0..n times
  error:   err  => console.error(err),               // 0 or 1 time – then
  complete: () => console.log('done'),               // 0 or 1 time – then
  stream is DEAD
});
```

Contract: `next*` then **either** `error` **or** `complete`, never both, never anything after.

▲ Once a stream errors, it's finished. New values will not arrive. That's why an error inside a `valueChanges` pipe silently kills your form subscription forever — see [Part 7](#).

Q: Cold vs hot observables?

Cold — the producer is created *per subscriber*. Everyone gets their own independent execution.

```
const cold$ = this.http.get('/api/chapters');
cold$.subscribe(); // HTTP request #1
cold$.subscribe(); // HTTP request #2 ← two separate calls!
```

Hot — one producer, shared by all subscribers. Late subscribers miss earlier values.

```
const hot$ = fromEvent(document, 'click'); // the clicks happen whether
you subscribe or not
const subject$ = new Subject();           // Subjects are hot
```

Turn cold into hot: `share()`, `shareReplay()`.

■ *"HTTP observables are cold and unicast — each subscribe re-runs the request. If two components need the same data I add `shareReplay({bufferSize:1, refCount:true})` so one request is shared and replayed."*

Q: Unicast vs multicast?

- **Unicast** — one producer per subscriber (a plain `Observable`)
- **Multicast** — one producer, many subscribers (a `Subject`, or any observable through `share()`)

Part 2 — Subjects

A **Subject** is both an **Observable** and an **Observer** — you can `subscribe()` to it *and* push values into it with `next()`.

The four Subject types

Type	Initial value	New subscriber gets	Typical use
Subject	✗ None	Only values emitted after subscribing	Events, notifications, <code>destroy\$</code>
BehaviorSubject	✓ Required	The latest value immediately	★ Shared state in services
ReplaySubject	✗ None	The last N values (buffer)	History, late-joining components
AsyncSubject	✗ None	Only the final value, on complete	Rare — one-shot results

```
// Subject – late subscriber misses everything
const s = new Subject<number>();
s.next(1);
s.subscribe(v => console.log('A:', v)); // hears nothing yet
s.next(2);                               // A: 2

// BehaviorSubject – late subscriber gets the current value
const b = new BehaviorSubject<number>(0);
b.next(1);
b.subscribe(v => console.log('B:', v)); // B: 1 ← immediately
b.next(2);                               // B: 2

// ReplaySubject – late subscriber gets the buffer
const r = new ReplaySubject<number>(2);
r.next(1); r.next(2); r.next(3);
r.subscribe(v => console.log('R:', v)); // R: 2, R: 3
```

● In your RoboGebra code — the real state layer ★

File: `robogeбра-mobile/src/app/core/services/user-summary.service.ts`

122 BehaviorSubject usages across the app — this is your state management.

```
@Injectable({ providedIn: 'root' }) // ★ app-wide
singleton
export class UserSummaryService {

    private userSummarySubject$ = new
BehaviorSubject<UserSummaryGroupDTO>(null);
    private bannerVisibilitySubject$ = new
BehaviorSubject<boolean>(false);
    private expiredModalVisibilitySubject$ = new
BehaviorSubject<boolean>(false);

    getUserSummarySubject(): Observable<UserSummaryGroupDTO> {
        return this.userSummarySubject$.asObservable(); //
★ READ-ONLY to the outside
    }

    getUserSummary(): UserSummaryGroupDTO {
        return this.userSummarySubject$.getValue(); // ★
    }
    SYNCHRONOUS read

    // The signed-in user's institute code – read synchronously so callers
that
    // CAN'T subscribe (the X-Institute-Code interceptor) still get the
value.
    getInstituteCode(): string | null {
        return this.getUserSummary()?.getInstituteCode() || null;
    }
}
```

■ **Say this:** "We manage shared state with services holding a *BehaviorSubject*, exposed as a read-only observable via *asObservable()* so only the service can push. *BehaviorSubject* specifically, for two reasons: a component created later — a lazy page or a modal — still needs the current user, not just the next change; and *getValue()* gives a synchronous read for callers that can't subscribe, like our institute-code HTTP interceptor. Components consume it with the *async* pipe, which handles unsubscription for free."

★ Two details that score: **the private subject + `asObservable()` encapsulation**, and **why a synchronous accessor exists at all**.

The generic pattern, for when they ask you to write one

```
@Injectable({ providedIn: 'root' })
export class StudyListService {
  private studyListSubject = new BehaviorSubject<StudyItem[]>([]);
  readonly studyList$ = this.studyListSubject.asObservable(); // ★
  read-only

  get current(): StudyItem[] { return this.studyListSubject.value; }

  addItem(item: StudyItem): void {
    this.studyListSubject.next([...this.current, item]); // ★
    IMMUTABLE update
  }
}
```

```
<!-- component template – no manual subscription at all -->
<div *ngFor="let item of studyListService.studyList$ |
  async">{{ item.title }}</div>
```

■ **Say this:** *"In RoboGebra we manage shared state with services holding a `BehaviorSubject`, exposed as a read-only observable via `asObservable()` so only the service can push. Components consume it with the `async` pipe, which also handles unsubscription. It gave us real-time sync across the Study List without the ceremony of `NgRx`."*

Why `asObservable()` ? Without it, any component could call `service.studyListSubject.next(...)` and bypass your service logic — an encapsulation leak. Same reasoning as private fields in Java ([17 – SOLID](#)).

Part 3 — Creation operators

Operator	What it does	Example
<code>of(...)</code>	Emits the given values, then completes	<code>of(1, 2, 3)</code>
<code>from(...)</code>	Converts array / Promise / iterable to an Observable	<code>from([1,2,3])</code> , <code>from(promise)</code>
<code>fromEvent</code>	DOM events as a stream	<code>fromEvent(input, 'keyup')</code>
<code>interval(n)</code>	<code>0,1,2,...</code> every <code>n</code> ms, forever	<code>interval(1000)</code>
<code>timer(d, p?)</code>	Emits after delay <code>d</code> , then every <code>p</code> ms	<code>timer(0, 5000)</code> — poll immediately then every 5s
<code>EMPTY</code>	Completes immediately, emits nothing	Error-recovery fallback
<code>NEVER</code>	Never emits, never completes	Testing
<code>throwError(() => e)</code>	Errors immediately	Error simulation
<code>defer(fn)</code>	Creates the observable per subscriber , at subscribe time	Lazy/fresh values
<code>range(s, c)</code>	A range of numbers	<code>range(1, 5)</code> → 1,2,3,4,5

▲ `of([1,2,3])` vs `from([1,2,3])` — a favourite trick question.
`of([1,2,3])` emits **one value: the array**. `from([1,2,3])` emits **three values: 1, 2, 3**.

```
// defer – why it exists
const now$ = of(Date.now()); // timestamp fixed at creation, same
for every subscriber
const fresh$ = defer(() => of(Date.now())); // evaluated at each
subscribe
```


Part 4 — Transformation & the flattening four

Simple transformations

Operator	Does
<code>map</code>	Transform each value — <code>map(u => u.name)</code>
<code>scan</code>	Like <code>reduce</code> , but emits the running accumulator every time
<code>reduce</code>	Emits only the final accumulated value on complete
<code>toArray</code>	Collects all values into one array on complete
<code>bufferTime(n)</code>	Groups values emitted within n ms into arrays
<code>pairwise</code>	Emits <code>[previous, current]</code>

```
// scan — a running total, emits every step
of(1, 2, 3).pipe(scan((acc, v) => acc + v, 0)).subscribe(console.log);
// 1, 3, 6

// reduce — only the final value
of(1, 2, 3).pipe(reduce((acc, v) => acc + v, 0)).subscribe(console.log);
// 6
```

▲ switchMap vs mergeMap vs concatMap vs exhaustMap

This is the single most-asked RxJS question. All four "flatten" an inner observable — they differ only in *what happens when a new outer value arrives while an inner one is still running*.

The one-line rule for each

Operator	When a new value arrives while inner is active	Memory hook
<code>switchMap</code>	Cancels the previous inner, switches to the new one	"switch to the latest"
<code>mergeMap</code>	Runs both concurrently, order not guaranteed	"merge everything, all at once"
<code>concatMap</code>	Queues the new one until the current finishes	"concat = one after another"
<code>exhaustMap</code>	Ignores the new one until the current finishes	"exhaust the current first"

Marble diagrams

Outer emits `a` then `b`; each inner takes time to produce `1` then `2`.

```
outer:      --a-----b-----

switchMap:  ----a1--X   ← a's inner CANCELLED when b arrives
              --b1--b2|
result:     ----a1-----b1--b2|

mergeMap:   ----a1--a2|
              --b1--b2|           ← both run at once
result:     ----a1--a2b1--b2|      (interleaved)

concatMap:  ----a1--a2|
              --b1--b2|           ← b WAITS for a to finish
result:     ----a1--a2----b1--b2|  (strict order)

exhaustMap: ----a1--a2|
              x                   ← b DISCARDED, a was still running
result:     ----a1--a2|
```

When to use each — with the real Angular case

`switchMap` — **typeahead search** ★ most common

```
this.searchControl.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(term => this.api.search(term)), // cancels the in-flight
  request
  takeUntilDestroyed(this.destroyRef)
).subscribe(results => this.results = results);
```

Why: if the user types "alge" then "algebra", the response for "alge" is worthless. `switchMap` cancels it — which for `HttpClient` **actually aborts the HTTP request**. It also prevents out-of-order responses overwriting newer results.

`concatMap` — ordered writes

```
this.saveQueue$.pipe(
  concatMap(change => this.api.saveProgress(change))
).subscribe();
```

Why: progress updates must hit the server in the order they happened. Never `switchMap` a save — you'd cancel real user data.

`mergeMap` — independent parallel work

```
from(fileList).pipe(
  mergeMap(file => this.api.upload(file), 3) // 3 concurrent uploads max
).subscribe();
```

Why: uploads don't depend on each other and order doesn't matter. The second argument caps concurrency.

`exhaustMap` — submit button / login

```
fromEvent(this.submitBtn.nativeElement, 'click').pipe(
  exhaustMap(() => this.api.submitQuiz(this.answers))
).subscribe();
```

Why: an impatient user clicks Submit four times. `exhaustMap` ignores clicks 2–4 while the first request is in flight. No duplicate submissions, no disabled-button plumbing.

■ **The answer that lands:** *"They all flatten inner observables; the difference is the cancellation strategy. `switchMap` cancels the previous — right for search, wrong for saves because you'd cancel a real write. `concatMap` queues and preserves order — right for saves. `mergeMap` runs everything in parallel — right for independent uploads. `exhaustMap` ignores new emissions while one is in flight — right for a submit button, since it kills double-submits for free."*

▲ **Danger:** using `switchMap` for a POST/PUT can cancel a request the server has already started processing, leaving you inconsistent. **Default to `concatMap` for writes and `switchMap` for reads.**

◆ Real-world analogies — the fastest way to keep these four straight

`switchMap` → changing the TV CHANNEL
the old programme stops INSTANTLY. Only the latest matters.

`mergeMap` → opening SEVERAL TAPS
all fill at once, in whatever order they finish.

`concatMap` → a QUEUE at a ticket counter
one at a time, strictly in the order people arrived.

`exhaustMap` → a LIFT
pressing the button again while the doors close does NOTHING.

● In your RoboGebra code — be precise and honest

`switchMap` → 246 usages ★ (the auth interceptor, every search, dependent loads)
`mergeMap` → 0
`concatMap` → 0
`exhaustMap` → 0

■ *"We use `switchMap` heavily — in the HTTP auth interceptor, where the token itself arrives asynchronously, and in every search box, because latest-wins is what you want for reads. We haven't needed `concatMap` or `exhaustMap` yet, but I know exactly where they belong: `concatMap` for ordered writes so a save is never cancelled, and `exhaustMap` on a submit button to kill double-submits without any disabled-button plumbing."*

★ Saying "we haven't needed X, but here's where it belongs" is far stronger than pretending. It proves the knowledge is real rather than recited.

Easy memory

switchMap CANCEL previous ★ TV channel → SEARCH / reads (latest wins)

mergeMap PARALLEL many taps → independent work (uploads)

concatMap QUEUE in order ★ ticket queue → WRITES / saves (never cancel a write!)

exhaustMap IGNORE new ★ a lift → submit button, login (no double-submit)

RULE: switchMap for READS, concatMap for WRITES ★

Why switchMap for search: it prevents an OUT-OF-ORDER response – a slow "alge" landing after a fast "algebra" and overwriting it ★

Part 5 — Filtering operators

Operator	What it does
<code>filter(fn)</code>	Only pass values matching the predicate
<code>take(n)</code>	Take the first n, then complete (auto-unsubscribes)
<code>first()</code> / <code>last()</code>	First/last value then complete (errors if empty — <code>first(pred, default)</code> avoids that)
<code>takeUntil(notifier\$)</code>	Emit until <code>notifier\$</code> emits — ★ the unsubscribe pattern
<code>takeWhile(fn, inclusive?)</code>	Emit while the predicate holds, then complete
<code>skip(n)</code> / <code>skipWhile</code> / <code>skipUntil</code>	Ignore values
<code>debounceTime(ms)</code>	Wait for a pause of <code>ms</code> — emits the last value ★ search
<code>throttleTime(ms)</code>	Emit the first value, then ignore for <code>ms</code> ★ scroll/resize

Operator	What it does
<code>auditTime(ms)</code>	Ignore for <code>ms</code> , then emit the latest value
<code>distinctUntilChanged()</code>	Skip if identical to the previous value
<code>distinctUntilKeyChanged('id')</code>	Same, comparing one key
<code>sample(notifier\$)</code>	Emit the latest value whenever the notifier fires

debounceTime vs throttleTime — know the difference

```

input:           --a-b-c-----d-e-----

debounceTime:    -----c-----e-----   ← waits for silence, emits
the LAST

throttleTime:    --a-----d-----       ← emits the FIRST, then
blocks

```

- **debounceTime** — search box. You want the final query after they stop typing.
- **throttleTime** — scroll/resize/mousemove. You want regular sampling, not the end.

◆ A crisp way to say it: *"Debounce waits for the user to stop; throttle limits the rate while they continue."*

Why **distinctUntilChanged** matters

```

searchControl.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),      // ← without this, typing "a", backspace,
  "a" refires the same search
  switchMap(t => this.api.search(t))
)

```

For objects it compares by **reference**, so pass a comparator:

```
distinctUntilChanged((a, b) => a.id === b.id).
```

Part 6 — Combination operators

Very commonly asked as "difference between forkJoin, combineLatest and zip". Here's the table that separates them:

Operator	Emits when	How many times	Needs completion?
<code>forkJoin</code>	All sources complete	Once — array/object of last values	✓ Yes
<code>combineLatest</code>	Any source emits (after all have emitted ≥ 1)	Many	✗ No
<code>zip</code>	All have emitted at the same index	Many — paired by index	✗ No
<code>merge</code>	Any source emits — pass-through	Many	✗ No
<code>concat</code>	Sequentially, after the previous completes	Many	✓ Yes
<code>withLatestFrom</code>	Source emits (samples the others)	Many	✗ No
<code>race</code>	Only the first source to emit; others unsubscribed	Many	✗ No
<code>startWith(v)</code>	Prepends a value immediately	—	—

Marble comparison

A: --1-----2-----3|
B: ----a-----b-----|

forkJoin:	-----[3,b]	← ONE emission, on completion
combineLatest:	----[1,a]-[2,a]-[2,b]-[3,b]	← every change
zip:	----[1,a]--[2,b]	← paired by index
merge:	--1-a---2--b-----3	← flat pass-through
withLatestFrom (A source)	-----[2,a]----[3,b]	← only when A emits

forkJoin — parallel HTTP, wait for all ★

```
// Loading a RoboGebra dashboard: three independent calls in parallel
forkJoin({
  profile: this.api.getProfile(),
  progress: this.api.getProgress(),
  timeline: this.api.getTimeline()
}).subscribe(({ profile, progress, timeline }) => {
  this.profile = profile;
  this.progress = progress;
  this.timeline = timeline;
});
```

▲ Three **forkJoin** gotchas interviewers love:

1. **If any source errors, the whole thing errors** and you lose the successful results. Guard each: `this.api.getProgress().pipe(catchError(() => of(null)))`.
2. **If a source completes without emitting, **forkJoin** emits nothing at all.**
3. **It never fires with a **Subject** or **valueChanges**** — those don't complete. Use `combineLatest` there.

combineLatest — react to any change ★

```
// Re-filter whenever the search term, chapter or difficulty changes
combineLatest([
  this.searchTerm$,
  this.selectedChapter$,
  this.difficulty$
]).pipe(
  debounceTime(200),
  switchMap(([term, chapter, difficulty]) =>
    this.api.getExercises({ term, chapter, difficulty }))
).subscribe(list => this.exercises = list);
```

▲ **combineLatest** emits nothing until every source has emitted at least once.

The usual fix is `startWith(initialValue)` on each — or use

`BehaviorSubject`s, which always have a value. This is the #1 "why isn't my combineLatest firing?" bug.

withLatestFrom — one stream drives

```
// Only when Save is clicked, grab whatever the form currently holds
fromEvent(saveBtn, 'click').pipe(
  withLatestFrom(this.form.valueChanges),
  exhaustMap([_, formValue]) => this.api.save(formValue))
).subscribe();
```

Form changes alone don't trigger anything — only the click does.

merge VS concat

```
merge(a$, b$);    // both start now, interleaved
concat(a$, b$);   // b$ doesn't start until a$ completes
```

Part 7 — Error handling

`catchError` — recover or rethrow

```
this.api.getQuiz(id).pipe(
  catchError(err => {
    console.error(err);
    return of(null);           // ✓ recover with a fallback value
    // return EMPTY;          // ✓ or emit nothing and complete
  })
  // return throwError(() => err); // ✓ or rethrow for an outer handler
).subscribe(quiz => this.quiz = quiz);
```

of(fallback) vs EMPTY: `of(null)` emits one value then completes (your `next` handler runs). `EMPTY` completes with no value at all (your `next` handler never runs).

▲ The most important error-handling question: WHERE do you put

`catchError`?

```
// ✗ WRONG — one failed search kills the stream forever.
// The user types again and nothing happens.
this.search$.pipe(
  switchMap(term => this.api.search(term)),
  catchError(() => of([]))           // outer pipe — stream is dead after
  this fires
).subscribe();

// ✓ RIGHT — catch INSIDE the inner observable.
// The outer stream survives; the next keystroke still works.
this.search$.pipe(
  switchMap(term =>
    this.api.search(term).pipe(catchError(() => of([]))) // ← inside
  )
).subscribe();
```

■ **This answer marks you as senior:** "`catchError` placed in the outer pipe kills the source stream — an error is terminal, so no further values arrive. For a long-

lived stream like `valueChanges`, I put `catchError` inside the `switchMap`'s inner observable so a failed request doesn't take the whole stream down."

`retry` — RxJS 7 syntax

```
this.api.getData().pipe(
  retry({ count: 3, delay: 1000 }), // 3 retries, 1s
  apart
  catchError(err => { this.toast.error('Failed'); return EMPTY; })
).subscribe();

// exponential backoff
retry({ count: 3, delay: (err, retryCount) => timer(retryCount * 1000) })
```

▲ `retryWhen` is **deprecated in RxJS 7** — use `retry({ delay })`.

`finalize` — always runs

```
this.loading = true;
this.api.getData().pipe(
  finalize(() => this.loading = false) // runs on complete, error, AND
  unsubscribe
).subscribe();
```

The RxJS equivalent of `finally`. Perfect for spinners.

Part 8 — Utility & multicasting

Operator	Purpose
<code>tap</code>	Side effects (logging, setting a flag) without changing the stream
<code>delay(ms)</code>	Time-shift emissions
<code>finalize</code>	Cleanup on complete/error/unsubscribe
<code>share()</code>	Multicast — one execution shared across subscribers

Operator	Purpose
<code>shareReplay({bufferSize, refCount})</code>	Multicast + replay to late subscribers ★ caching
<code>toArray()</code>	Collect everything into one array

tap — debugging tool

```
this.api.getQuiz(id).pipe(
  tap(() => this.loading = true),
  tap(quiz => console.log('received', quiz)),
  map(quiz => quiz.questions),
  finalize(() => this.loading = false)
).subscribe();
```

`tap` never modifies values — if you're changing something, use `map`.

shareReplay — caching an HTTP call ★

```
@Injectable({ providedIn: 'root' })
export class ConfigService {
  readonly config$ = this.http.get<AppConfig>('/api/config').pipe(
    shareReplay({ bufferSize: 1, refCount: true })
  );
}
```

Ten components subscribing → **one** HTTP request, and every late subscriber immediately gets the cached response.

▲ The `refCount` trap — a genuinely senior detail.

`shareReplay(1)` (without `refCount`) keeps the source subscription alive **forever**, even after every subscriber has left. With an `interval` or a WebSocket, that's a permanent memory leak.

`shareReplay({ bufferSize: 1, refCount: true })` unsubscribes from the source when the last subscriber leaves.

Rule: `refCount: true` for long-lived/infinite sources; plain `shareReplay(1)` is acceptable for a one-shot HTTP call you genuinely want cached for the app's lifetime.

Part 9 — Unsubscribing & memory leaks

Q: What happens if you don't unsubscribe?

The subscription stays alive after the component is destroyed. The callback keeps running, keeps a reference to the dead component (so it's never garbage-collected), and can throw errors on destroyed views. With a route the user visits repeatedly, subscriptions **stack up** — 10 visits, 10 live subscriptions all firing.

What you DON'T need to unsubscribe from

- `HttpClient` calls — they complete after one emission
- Anything ending in `take(1)`, `first()`, `takeUntil(...)`
- Anything consumed via the `async` pipe
- `ActivatedRoute` params **in a routed component** (Angular cleans these up) — but do unsubscribe in a non-routed child

What you MUST unsubscribe from

- `Subject` / `BehaviorSubject` in services
- `valueChanges` / `statusChanges`
- `interval` / `timer` / `fromEvent`
- `Router.events`
- Anything infinite

✓ Pattern 1 — `async` pipe (best; use it whenever you can)

```
quiz$ = this.api.getQuiz(this.id);
```

```
<div *ngIf="quiz$ | async as quiz">{{ quiz.title }}</div>
```

Angular subscribes and unsubscribes for you. No `ngOnDestroy` needed.

▲ Two `async` pipes on the same cold observable = two HTTP calls. Fix with `shareReplay(1)` or `*ngIf="x$ | async as x"` and reuse `x`.

✓ Pattern 2 — `takeUntilDestroyed()` (Angular 16+, the modern way)

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

export class QuizComponent {
  private destroyRef = inject(DestroyRef);

  ngOnInit() {
    this.service.data$.pipe(
      takeUntilDestroyed(this.destroyRef)    // destroyRef needed outside
the injection context
    ).subscribe(d => this.data = d);
  }
}
```

In a **field initializer or constructor** you can omit the argument — it picks up the injection context automatically:

```
readonly data$ = this.service.data$.pipe(takeUntilDestroyed());
```

◆ **Both your apps can use this** — Angular 16.2 (web/backoffice) and 18.2 (mobile). Mentioning it shows you're current.

✓ Pattern 3 — `takeUntil(destroy$)` (the classic, works everywhere)

```
export class QuizComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();

  ngOnInit() {
    this.service.data$.pipe(takeUntil(this.destroy$)).subscribe(d =>
this.data = d);
    this.form.valueChanges.pipe(takeUntil(this.destroy$)).subscribe(v =>
this.onChange(v));
  }

  ngOnDestroy() {
    this.destroy$.next();
    this.destroy$.complete();    // don't forget – completes the Subject
itself
  }
}
```

▲ **`takeUntil` must be LAST in the pipe.** Any operator after it (especially `switchMap` or `shareReplay`) can create a new subscription that `takeUntil` no longer guards.

Pattern 4 — collecting Subscriptions

```
private subs = new Subscription();

ngOnInit() {
  this.subs.add(this.a$.subscribe());
  this.subs.add(this.b$.subscribe());
}

ngOnDestroy() { this.subs.unsubscribe(); }    // unsubscribes all children
```

Fine, but more manual bookkeeping than `takeUntil`.

■ **Interview answer:** "My order of preference is: `async` pipe first, because Angular handles the lifecycle; `takeUntilDestroyed` when I need an imperative subscription — we're on Angular 16 and 18 so it's available; and `takeUntil` with a destroy Subject on anything older. I avoid manual `unsubscribe()` calls because they're easy to forget when a component grows."

● In your RoboGebra code — leak prevention at scale ★

takeUntil	→ 426 usages ★
takeUntilDestroyed	→ 5 usages (the newer code)
async pipe	→ 311 usages ★
ngOnDestroy	→ 100 components
finalize	→ 254 usages ★ (hiding the busy spinner)
catchError	→ 338 usages

Even the **HTTP interceptor** uses `takeUntil` — for something clever:

File: `core/interceptors/header-authorization.interceptor.ts`

```
private readonly cancelRequests$: Subject<void> = new Subject<void>();

private invokeHttpRequest(req, next): Observable<HttpEvent<any>> {
  return next.handle(req).pipe(
    takeUntil(this.cancelRequests$), //
    ★ ONE emit cancels EVERY in-flight request
    catchError(error => this.handleUnAuthResponse(error, req, next))
  );
}

ngOnDestroy(): void {
  this.cancelRequests$.next();
  this.cancelRequests$.complete();
}
```

■ "`takeUntil` isn't only for component cleanup. In our auth interceptor there's a `cancelRequests$` subject, so on logout a single `next()` aborts every in-flight HTTP request at once — otherwise a slow response from the previous session can land after the user has already signed out."

★ That is a genuinely good, non-textbook use of `takeUntil`.

And the 254 `finalize` usages: "`finalize` guarantees the busy spinner is hidden whether the call succeeded, failed, or was cancelled — `catchError` alone wouldn't cover the unsubscribe path."

Part 10 — Real Angular scenarios

Rehearse these — they're what "have you used RxJS in production?" is really asking.

1. Typeahead search (the classic — memorize it)

```
results$ = this.searchControl.valueChanges.pipe(
  debounceTime(300), // wait for a typing pause
  map(term => term?.trim() ?? ''),
  distinctUntilChanged(), // ignore no-op changes
  filter(term => term.length >= 2), // don't search 1 character
  tap(() => this.loading = true),
  switchMap(term => this.api.search(term).pipe(
    catchError(() => of([])), // ← INSIDE, so the stream
    survives
    finalize(() => this.loading = false)
  )),
  takeUntilDestroyed()
);
```

Be ready to justify **every operator** — that's the real question.

● Your real one — [features/pages/study-materials/study-materials.component.ts:288](#)

```
this.loadVideosSubject$.pipe(
  takeUntil(this.destroy$),
  debounceTime(500), //
  ★ wait for the typing to stop
  tap(() => (this.isLoading = true)),
  filter(() => { // ★ don't search on 1-2
    characters
    const query = this.searchTerm?.trim() || '';
    return query.length === 0 || query.length >= 3;
  }),
  switchMap(() => this.studyMaterialService.search(query,
    tags, ...)) // ★ cancels the previous
  )
```

The same pipeline appears in `help-page.component.ts` and `parent-info.component.ts` (there with `distinctUntilChanged` for an async availability check). **13 `debounceTime` usages** in total.

2. Dependent dropdowns (chapter → exercise)

```
this.chapterControl.valueChanges.pipe(
  filter(Boolean),
  tap(() => this.exerciseControl.reset()),
  switchMap(chapterId => this.api.getExercises(chapterId)),
  takeUntilDestroyed()
).subscribe(list => this.exercises = list);
```

3. Polling with a stop condition

```
timer(0, 5000).pipe(                                // fire immediately, then
  every 5s
  switchMap(() => this.api.getSolutionStatus(jobId)),
  takeWhile(res => res.status === 'PROCESSING', true), // `true` = emit
  the final one too
  takeUntilDestroyed()
).subscribe(res => this.status = res.status);
```

Real use: waiting on the AI solution pipeline to finish.

4. Prevent double-submit

```
this.submitClicks$.pipe(
  exhaustMap(() => this.api.submitQuiz(this.answers).pipe(
    catchError(err => { this.toast.error('Submit failed'); return
    EMPTY; })
  )),
  takeUntilDestroyed()
).subscribe(result => this.router.navigate(['/result', result.id]));
```

● Your real `forkJoin` — `app.component.ts:166`

```
this.isLoggedIn$.pipe(
  filter(Boolean),
  filter(() => !this.authService.isLoggedInAsParent()),
  switchMap(() => forkJoin({                                // ★ three calls IN
    PARALLEL
      userSummary:    this.userSummaryService.fetchUserSummary(),
      userProfile:    this.userService.fetchUserProfile(false),
      userPreference:
this.userService.fetchUserPreference(false)
    })),
  takeUntil(this.destroy$)
).subscribe({
  next: async ({ userSummary, userProfile, userPreference }) => { ... }
});
```

■ "Three independent calls fire after login. Sequentially that's the sum of the latencies; `forkJoin` makes it the slowest of the three. Note it's the object form, so the result destructures by name rather than by array index — much harder to get wrong when someone later adds a fourth call."

▲ And the trap to volunteer: " `forkJoin` emits nothing at all if any source errors, so for a dashboard where partial data is better than none, each call needs its own `catchError` returning a fallback."

22 `forkJoin` usages in the app.

5. Parallel dashboard load with per-call fallback

```
forkJoin({
  profile: this.api.getProfile(),
  progress: this.api.getProgress().pipe(catchError(() => of(null))),
  timeline: this.api.getTimeline().pipe(catchError(() => of([])))
}).pipe(takeUntilDestroyed())
  .subscribe(data => this.dashboard = data);
```

The `catchError` per source means one failing widget doesn't blank the whole dashboard.

6. Route param → data

```
quiz$ = this.route.paramMap.pipe(  
  map(params => params.get('id')!),  
  distinctUntilChanged(),  
  switchMap(id => this.api.getQuiz(id)),  
  shareReplay({ bufferSize: 1, refCount: true })  
);
```

`switchMap` cancels the previous fetch when the user navigates quickly between quizzes.

7. Auth-token refresh in an HTTP interceptor

```
intercept(req: HttpRequest<any>, next: HttpHandler) {  
  return next.handle(this.addToken(req)).pipe(  
    catchError(err => {  
      if (err.status === 401) {  
        return this.auth.refreshToken().pipe(  
          switchMap(newToken => next.handle(this.addToken(req,  
newToken))),  
          catchError(() => { this.auth.logout(); return EMPTY; })  
        );  
      }  
      return throwError(() => err);  
    })  
  );  
}
```

Your `header-authorization.interceptor.ts` does this shape of work — good to reference.

Part 11 — Common mistakes interviewers probe for

✗ 1. Nested subscribes

```
// WRONG – callback hell, no cancellation, hard to handle errors
this.route.params.subscribe(params => {
  this.api.getUser(params['id']).subscribe(user => {
    this.api.getOrders(user.id).subscribe(orders => { ... });
  });
});

// RIGHT
this.route.params.pipe(
  switchMap(p => this.api.getUser(p['id'])),
  switchMap(user => this.api.getOrders(user.id)),
  takeUntilDestroyed()
).subscribe(orders => ...);
```

◆ If you spot and name this, it reads as real experience. "A subscribe inside a subscribe is the RxJS equivalent of callback hell — you lose cancellation and centralised error handling. Flatten it with switchMap."

✗ 2. Subscribing just to reassign a field

```
// Meh
ngOnInit() { this.api.getQuiz(id).subscribe(q => this.quiz = q); }

// Better – let the async pipe manage it
quiz$ = this.api.getQuiz(id);
```

✗ 3. Multiple subscriptions to a cold observable

```
<div>{{ (user$ | async)?.name }}</div>
<div>{{ (user$ | async)?.email }}</div>    <!-- ✗ two HTTP calls -->

<ng-container *ngIf="user$ | async as user">    <!-- ✓ one -->
  <div>{{ user.name }}</div><div>{{ user.email }}</div>
</ng-container>
```

X 4. `catchError` in the outer pipe (see Part 7)

X 5. `forkJoin` with a non-completing source

`forkJoin([this.mySubject$, ...])` never emits, because a Subject never completes. Use `combineLatest`, or `take(1)` each source.

X 6. `combineLatest` that never fires

Because one source hasn't emitted yet. Add `startWith(...)` or use `BehaviorSubject`.

X 7. Operators after `takeUntil`

X 8. `shareReplay(1)` on an infinite source without `refCount`

X 9. Mutating state inside a BehaviorSubject

```
this.subject.value.push(item);           // x no emission, subscribers
never update
this.subject.next([...this.subject.value, item]); // ✓
```

Part 12 — RxJS 7 changes you must know

You're on **RxJS 7.8** — know what changed from 6.

Change	Old (RxJS 6)	New (RxJS 7)
Observable → Promise	<code>toPromise()</code> deprecated	<code>firstValueFrom(obs\$)</code> / <code>lastValueFrom(obs\$)</code>
Retry with delay	<code>retryWhen(...)</code> deprecated	<code>retry({ count, delay })</code>
Property extraction	<code>pluck('a', 'b')</code> deprecated	<code>map(x => x.a.b)</code>
<code>combineLatest(a, b)</code>	multiple args deprecated	<code>combineLatest([a, b])</code>

Change	Old (RxJS 6)	New (RxJS 7)
Bundle size	—	~40% smaller, better tree-shaking
Types	—	Much stronger TypeScript inference

```
// Why toPromise() was replaced: it resolved to `undefined` if the
// observable
// completed without emitting — silently. The new APIs are explicit:
const value = await firstValueFrom(this.api.getConfig()); // rejects if
empty
const value = await lastValueFrom(this.api.getConfig());
const value = await firstValueFrom(src$, { defaultValue: null }); // no
throw
```

■ Dropping this in unprompted is a strong signal: "We're on RxJS 7.8, so *toPromise()* is deprecated — we use *firstValueFrom*, which fails loudly on an empty stream instead of silently resolving to *undefined*."

Part 13 — Output-based drills

Drill 1

```
of(1, 2, 3).pipe(map(x => x * 2), filter(x => x >
2)).subscribe(console.log);
```

Answer

`4`, `6` — 1→2 is filtered out (not `> 2`).

Drill 2

```
const s = new Subject<number>();
s.subscribe(v => console.log('A', v));
s.next(1);
s.subscribe(v => console.log('B', v));
s.next(2);
```

Answer

A 1
A 2
B 2

B subscribed after `1` and a plain `Subject` doesn't replay.

Drill 3

```
const b = new BehaviorSubject<number>(0);  
b.next(1);  
b.subscribe(v => console.log('X', v));  
b.next(2);
```

Answer

X 1 ← the current value, immediately on subscribe
X 2

Drill 4

```
of([1, 2, 3]).subscribe(console.log);  
from([1, 2, 3]).subscribe(console.log);
```

Answer

[1, 2, 3] ← of emits the array as ONE value
1
2
3 ← from emits each element

Drill 5

```
const a$ = interval(1000).pipe(take(3));    // 0,1,2 at 1s,2s,3s  
const b$ = interval(1500).pipe(take(2));   // 0,1 at 1.5s,3s  
forkJoin([a$, b$]).subscribe(console.log);
```

Answer

[2, 1] at 3s — one emission with the **last** value of each, only once both complete.

Drill 6

```
concat(of(1, 2), of(3, 4)).subscribe(console.log);  
merge(of(1, 2), of(3, 4)).subscribe(console.log);
```

Answer

Both print `1 2 3 4` here, because `of` is synchronous. With async sources `merge` would interleave and `concat` would not — that's the real distinction, and a good thing to point out.

Drill 7

```
of(1, 2, 3).pipe(  
  tap(x => console.log('tap', x)),  
  map(x => x * 10)  
)  
.subscribe(x => console.log('sub', x));
```

Answer

```
tap 1 / sub 10  
tap 2 / sub 20  
tap 3 / sub 30
```

Interleaved, not grouped — each value flows through the whole pipe before the next one starts.

Drill 8

```
const src$ = of(1, 2, 3).pipe(tap(() => console.log('side effect')));  
src$.subscribe();  
src$.subscribe();
```

Answer

`side effect` prints **6 times** — cold observable, one full execution per subscriber. Add `share()` to make it 3.

Part 14 — Rapid-fire

Question	Answer
Observable vs Promise	Many values, lazy, cancellable vs one value, eager, not cancellable
Cold vs hot	Producer per subscriber vs shared producer
Subject vs BehaviorSubject	BehaviorSubject has an initial value and replays the latest
BehaviorSubject vs ReplaySubject	Latest one vs last N
<code>.value</code> on a Subject?	Only <code>BehaviorSubject</code> has it
<code>asObservable()</code> — why?	Stops consumers calling <code>.next()</code> — encapsulation
<code>switchMap</code>	Cancels previous — search/reads
<code>mergeMap</code>	All in parallel — independent uploads
<code>concatMap</code>	Queued, order preserved — writes
<code>exhaustMap</code>	Ignores new while busy — submit buttons
Never <code>switchMap</code> on a...	POST/PUT — you'd cancel a real write
<code>forkJoin</code>	Waits for all to complete , emits once
<code>combineLatest</code>	Emits on any change, after all emitted once
<code>zip</code>	Pairs by index
<code>withLatestFrom</code>	Source drives, samples the rest
<code>debounceTime</code>	Waits for a pause, emits the last — search
<code>throttleTime</code>	Emits the first, then blocks — scroll
<code>distinctUntilChanged</code>	Skips consecutive duplicates
<code>take(1)</code>	One value then completes — auto-unsubscribes
<code>takeUntil</code>	Emit until the notifier fires — must be last

Question	Answer
catchError placement	Inside the inner observable for long-lived streams
retry (RxJS 7)	<code>retry({ count: 3, delay: 1000 })</code>
finalize	Runs on complete, error and unsubscribe
tap	Side effects only, never transforms
share vs shareReplay	Multicast vs multicast + replay to late subscribers
shareReplay trap	Needs <code>refCount: true</code> on infinite sources or it leaks
Best unsubscribe	<code>async</code> pipe → <code>takeUntilDestroyed()</code> → <code>takeUntil(destroy\$)</code>
Nested subscribe	Anti-pattern — flatten with <code>switchMap</code>
toPromise()	Deprecated — <code>firstValueFrom</code> / <code>lastValueFrom</code>
retryWhen	Deprecated in RxJS 7
Angular 16 unsubscribe helper	<code>takeUntilDestroyed()</code> from <code>@angular/core/rxjs-interop</code>

🕒 Two-day drill

Day 1 — Parts 1, 2, 4, 9. Be able to draw the four marble diagrams for `switchMap` / `mergeMap` / `concatMap` / `exhaustMap` from memory and give the Angular use case for each. Write the typeahead pipeline from scratch, no notes.

Day 2 — Parts 5–8 and 10. Memorize the `forkJoin` vs `combineLatest` vs `zip` table. Rehearse three real RoboGebra scenarios out loud (Study List `BehaviorSubject`, search with `switchMap`, dashboard `forkJoin`), 45 seconds each.

The three answers that most often decide the round: the four flattening operators and when each is wrong; where `catchError` goes and why; and how you prevent subscription leaks. Everything else is detail.

NgRx — Interview Questions & Complete Guide

State management for Angular, the Redux way. Companion to [20 — RxJS & Operators](#) — NgRx is built entirely on RxJS, so read that first if the operators aren't solid.

▲ Read this before anything else — your honesty position

NgRx is on your resume. RoboGebra does not use it.

I checked every `package.json` in `robogebra-workspace/` — there is **no** `@ngrx/*` **package** in the web, mobile or backoffice apps. RoboGebra state is services holding `BehaviorSubject`s ([18 — Technical Versions](#)).

So when NgRx comes up, **attribute it correctly**:

✓ *"I used NgRx on EasyVisa, where the visa application had deeply nested forms and shared state across many modules. On RoboGebra we deliberately went simpler — services with `BehaviorSubject` and the `async` pipe — because the state is mostly per-feature and NgRx would have been ceremony without payoff."*

That answer is **stronger** than claiming NgRx everywhere. It shows you can choose a tool rather than apply one reflexively — and interviewers actively probe for that. What you must not do is describe RoboGebra as an NgRx app; a follow-up question about your store structure will expose it.

Table of contents

- ▲ [60-second cheat sheet](#)
- [Part 1 — Why NgRx exists](#)
- [Part 2 — The data flow](#)
- [Part 3 — Actions](#)

- [Part 4 — Reducers](#)
- [Part 5 — Selectors](#)
- [Part 6 — Effects](#)
- [Part 7 — Wiring the store up](#)
- [Part 8 — @ngrx/entity](#)
- [Part 9 — The Facade pattern](#)
- [Part 10 — Full worked example](#)
- [Part 11 — ComponentStore & SignalStore](#)
- [Part 12 — Router Store & DevTools](#)
- [Part 13 — When NOT to use NgRx](#)
- [Part 14 — Common mistakes](#)
- [Part 15 — Interview Q&A](#)
- [Part 16 — Rapid-fire](#)

▲ 60-second cheat sheet

Question	Answer
What is NgRx?	Redux-inspired reactive state management for Angular, built on RxJS
The core building blocks?	Store, Actions, Reducers, Selectors, Effects
The three principles?	Single source of truth · state is read-only · changes via pure reducer functions
What is an Action?	A plain object <code>{ type, ...payload }</code> describing that something happened
What is a Reducer?	A pure function <code>(state, action) => newState</code>
What is a Selector?	A memoized query function for reading a slice of state

Question	Answer
What is an Effect?	A side-effect handler — listens for actions, does async work, dispatches new actions
Where does HTTP go?	Effects. Never in a reducer
Which RxJS operator in effects?	<code>concatMap</code> for writes, <code>switchMap</code> for reads/search, <code>exhaustMap</code> for submits, <code>mergeMap</code> for independent work
Why do selectors matter?	Memoization — they don't recompute unless their inputs change
@ngrx/entity ?	Normalized CRUD collections — <code>{ ids: [], entities: {} }</code> + a generated adapter
When NOT to use it?	Small apps, mostly-local state, small teams — the boilerplate isn't free
NgRx version?	Tracks Angular majors — Angular 16 → NgRx 16

◆ The easiest way to remember ALL of NgRx

One analogy carries the entire library — **a bank.**

You NEVER edit the balance directly.
 You submit a SLIP saying what happened.
 The bank applies a RULE and writes a NEW ledger entry.
 The balance is DERIVED from the ledger.

Component	→ the customer	"I want to withdraw ₹500"
ACTION	→ the deposit/withdrawal command	SLIP ★ (what HAPPENED – never a command)
REDUCER	→ the bank's RULE	(pure: old balance + slip → NEW balance) ★
STORE	→ the LEDGER	(one source of truth, never overwritten)
SELECTOR	→ the BALANCE ENQUIRY	(derived, and CACHED until something changes) ★
EFFECT	→ the BACK OFFICE	(calls the other bank, then files another slip) ★

★ Three consequences fall straight out of the analogy, and they're the three most-asked questions:

1. Why must a reducer be PURE?
A bank rule that phoned someone would be unauditable ★
2. Why NEVER mutate state?
You don't erase a ledger line – you add a new one ★
3. Why do effects exist?
Anything that talks to the outside world happens in the BACK OFFICE, never inside the rule ★

Easy memory

NgRx = a BANK LEDGER
 Action = the SLIP (what happened) ★ | Reducer = the RULE (pure) ★
 Store = the LEDGER | Selector = the BALANCE ENQUIRY (memoised) ★
 Effect = the BACK OFFICE (async, then files another slip) ★

Never edit the ledger. Add a new entry. ★

Part 1 — Why NgRx exists

Q: What problem does NgRx solve?

In a mid-size Angular app, shared state gets messy:

- The same data is fetched by several components, each with its own copy
- Component A changes something, component B doesn't know

- `@Input` / `@Output` chains get passed through five layers ("prop drilling")
- Services with `BehaviorSubject`s multiply until nobody knows who mutates what
- Reproducing a bug is guesswork because state changes are untraceable

NgRx centralizes application state in one immutable store and makes every change go through an explicit, logged, replayable action.

Q: What are the three Redux principles?

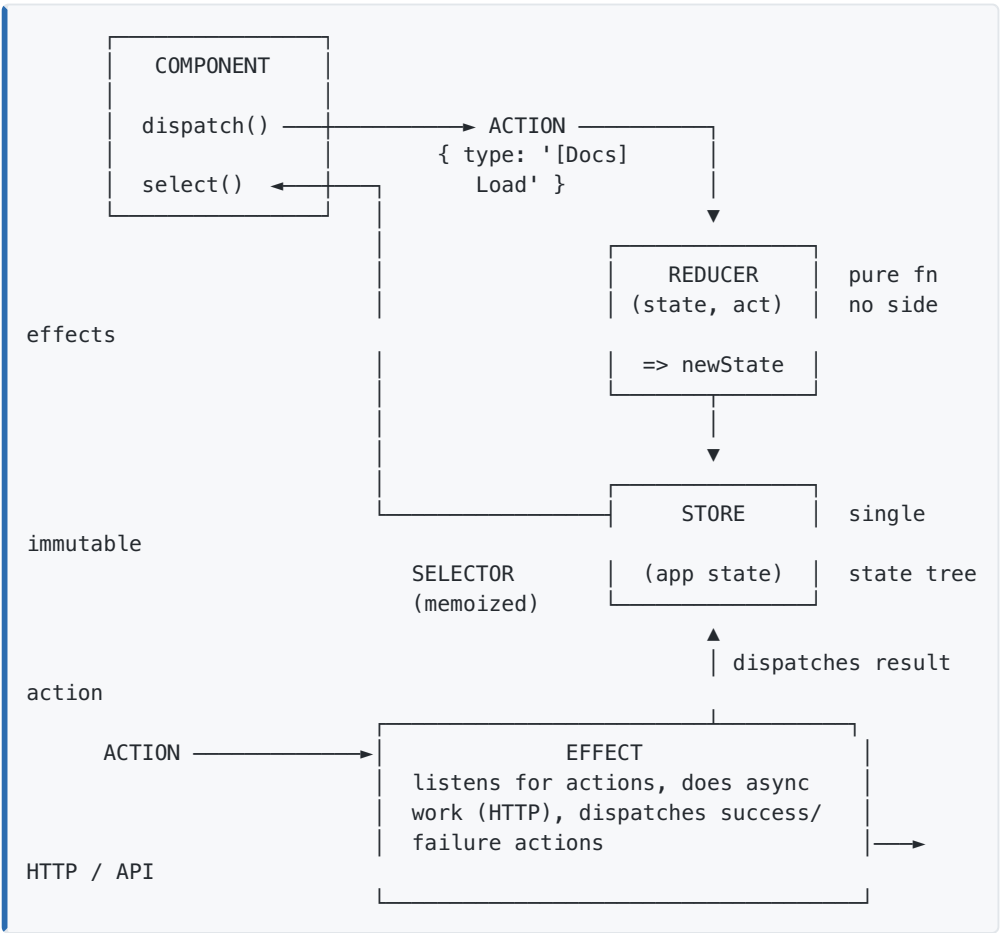
1. **Single source of truth** — the whole app's state lives in one object tree.
2. **State is read-only** — you never mutate it; you dispatch an action describing what happened.
3. **Changes are made by pure functions** — reducers take the previous state and an action and return **new** state.

Q: What do you actually gain?

Benefit	Why
Predictability	Same state + same action → same result, always
Traceability	Every change is a named action visible in DevTools
Time-travel debugging	Replay actions, jump to any previous state
Testability	Reducers and selectors are pure functions — trivial unit tests, no mocks
Performance	Immutable state + memoized selectors → <code>OnPush</code> change detection works properly
Decoupling	Components dispatch and select; they don't know where data comes from

Part 2 — The data flow

Strictly unidirectional. Be able to draw this on a whiteboard.



Say it out loud like this:

"A component dispatches an action. The reducer handles it synchronously and produces new state. If the action needs async work, an effect picks it up, calls the service, and dispatches a success or failure action — which the reducer then handles. Components read state through memoized selectors. Data flows one way, so any state change is traceable to a single named action."

Why "unidirectional" actually matters ★

TWO-WAY / ad-hoc state:

Component A sets user.name

Component B sets user.name

A service sets user.name

↓

The name is wrong. WHO changed it? → you add console.logs everywhere



UNIDIRECTIONAL:

EVERY change is a NAMED ACTION in DevTools, in order, with the state before and after ★

↓

"The name changed on [Profile] Update Success at 14:03." ✓

★ That traceability — not the code structure — is the actual reason NgRx exists. Say *that* when asked "why NgRx?".

Easy memory

dispatch(ACTION) → REDUCER (pure, sync) → STORE → SELECTOR (memoised) → component

↑
└── EFFECT dispatches success/failure ★ async work

ONE direction ★ → every change is a NAMED, replayable event → real traceability

Part 3 — Actions

An action is a **plain object with a type** describing that something happened.

```
import { createAction, props } from '@ngrx/store';

export const loadDocuments = createAction(
  '[Document Portal] Load Documents'
);

export const loadDocumentsSuccess = createAction(
  '[Document API] Load Documents Success',
  props<{ documents: Document[] }>()
);

export const loadDocumentsFailure = createAction(
  '[Document API] Load Documents Failure',
  props<{ error: string }>()
);

export const uploadDocument = createAction(
  '[Document Portal] Upload Document',
  props<{ file: File; panelId: string }>()
);
```

The naming convention — [Source] Event

Part	Meaning
[Source]	Where it came from — a page, a component, an API
Event	What happened , in past tense

- ✓ '[Login Page] Login Submitted'
- ✓ '[Auth API] Login Success'
- ✓ '[Document Portal] Document Deleted'
- ✗ 'LOGIN' — no source, no event
- ✗ '[Auth] Set User' — a command, not an event
- ✗ '[Docs] Load' (reused everywhere) — you lose traceability

◆ The senior insight: actions are events, not commands

"Actions should describe what happened, not what to do. '[Login Page] Login Submitted' rather than 'Set Loading True'. Unique, source-prefixed actions

mean the DevTools log reads like a story of what the user did, and multiple reducers or effects can react to the same event independently."

This is the **Observer pattern** — one event, many independent listeners ([17 — SOLID & Design Patterns](#)).

`createActionGroup` (NgRx 15+) — less boilerplate

```
export const DocumentActions = createActionGroup({
  source: 'Document Portal',
  events: {
    'Load Documents': emptyProps(),
    'Load Documents Success': props<{ documents: Document[] }>(),
    'Load Documents Failure': props<{ error: string }>(),
  },
});

// Usage: DocumentActions.loadDocuments()
```

Part 4 — Reducers

A reducer is a **pure function** that takes the current state and an action and returns **new** state.

```

import { createReducer, on } from '@ngrx/store';

export interface DocumentState {
  documents: Document[];
  loading: boolean;
  error: string | null;
  selectedId: string | null;
}

export const initialState: DocumentState = {
  documents: [],
  loading: false,
  error: null,
  selectedId: null,
};

export const documentReducer = createReducer(
  initialState,

  on(loadDocuments, state => ({
    ...state,
    loading: true,
    error: null,
  })),

  on(loadDocumentsSuccess, (state, { documents }) => ({
    ...state,
    documents,
    loading: false,
  })),

  on(loadDocumentsFailure, (state, { error }) => ({
    ...state,
    loading: false,
    error,
  })),

  on(documentDeleted, (state, { id }) => ({
    ...state,
    documents: state.documents.filter(d => d.id !== id), // new array,
    not splice
  })),
);

```

The rules a reducer must obey

Rule	Why
Pure — same inputs, same output	Predictability, time-travel
No mutation — always return a new object	Change detection + DevTools rely on reference changes
No side effects — no HTTP, no <code>console.log</code> , no <code>Date.now()</code> , no <code>Math.random()</code>	Reproducibility
Handle unknown actions by returning state unchanged	<code>createReducer</code> does this for you

✗ The mutation bug (asked constantly)

```
// WRONG – mutates existing state
on(addDocument, (state, { doc }) => {
  state.documents.push(doc);           // ✗ same array reference
  return state;                       // ✗ same state reference
});
// Result: OnPush components never update, DevTools shows nothing, tests
pass misleadingly

// RIGHT
on(addDocument, (state, { doc }) => ({
  ...state,
  documents: [...state.documents, doc],
}));
```

■ "Reducers must return a new reference. Angular's **OnPush** change detection and NgRx's memoized selectors both compare by reference — mutate in place and the UI simply doesn't update, which is a maddening bug to trace."

Immutable update cheat sheet:

```
[...arr, item]                // add
arr.filter(x => x.id !== id)   // remove
arr.map(x => x.id === id ? { ...x, ...changes } : x) // update
{ ...obj, key: value }        // object property
{ ...state, nested: { ...state.nested, k: v } } // nested
```

createFeature (NgRx 13+) — reducer + selectors together

```
export const documentFeature = createFeature({
  name: 'documents',
  reducer: createReducer(initialState, /* ...on() handlers */),
});

// Auto-generates: selectDocumentsState, selectDocuments, selectLoading,
selectError...
export const { selectDocuments, selectLoading, selectError } =
documentFeature;
```

Part 5 — Selectors

A selector is a **memoized** function for reading a slice of state.


```

import { createFeatureSelector, createSelector } from '@ngrx/store';

// 1. Grab the feature slice
export const selectDocumentState =
  createFeatureSelector<DocumentState>('documents');

// 2. Simple derived selectors
export const selectAllDocuments = createSelector(
  selectDocumentState,
  state => state.documents
);

export const selectLoading = createSelector(
  selectDocumentState,
  state => state.loading
);

// 3. Compose selectors
export const selectApprovedDocuments = createSelector(
  selectAllDocuments,
  docs => docs.filter(d => d.status === 'APPROVED')
);

// 4. Combine multiple selectors
export const selectDocumentSummary = createSelector(
  selectAllDocuments,
  selectApprovedDocuments,
  (all, approved) => ({
    total: all.length,
    approved: approved.length,
    pending: all.length - approved.length,
  })
);

// 5. Selector with a runtime argument (props)
export const selectDocumentsByPanel = (panelId: string) => createSelector(
  selectAllDocuments,
  docs => docs.filter(d => d.panelId === panelId)
);

```

◆ Memoization — the whole point of selectors

`createSelector` caches its last inputs and last result. If the inputs are **reference-equal** to last time, it returns the cached result **without re-running the projector**.

selectAllDocuments returns the SAME array reference

↓

selectApprovedDocuments: inputs unchanged → skips the filter, returns cached array

↓

component gets the SAME reference → OnPush skips re-render

That's why an unrelated state change (say **loading** flipping) doesn't cause an expensive **filter** or **sort** to re-run across your whole component tree.

■ **The answer:** "Selectors are memoized — **createSelector** caches on input reference equality, so an expensive derivation only recomputes when its actual inputs change. Combined with **OnPush**, that's what keeps an NgRx app fast. It also means reducers must never mutate — mutation defeats the reference check and memoization silently returns stale data."

Real-world idea

A memoised selector = a CACHED bill total

Nothing in the basket changed → don't add it all up again; hand back the same total, with the SAME reference.

↓

Same reference → an OnPush component doesn't re-render ★

▲ And the trap that follows immediately:

MUTATE the array in a reducer → the reference is UNCHANGED

↓

memoisation thinks nothing changed → returns the STALE cached result

↓

the UI silently doesn't update ▲

★ That is the deep reason "never mutate" is a rule, and it's a much better answer than "because Redux says so."

Easy memory

createSelector CACHES on INPUT REFERENCE equality ★ (a cached bill total)

Unchanged inputs → the projector is SKIPPED → the same reference back
→ OnPush skips the re-render ★

▲ Mutation keeps the same reference → memoisation returns STALE data ▲
→ THAT is why reducers must be immutable ★

Using selectors in a component

```
export class DocumentListComponent {
  documents$ = this.store.select(selectAllDocuments);
  loading$   = this.store.select(selectLoading);
  summary$   = this.store.select(selectDocumentSummary);

  constructor(private store: Store) {}

  ngOnInit() {
    this.store.dispatch(loadDocuments());
  }

  onDelete(id: string) {
    this.store.dispatch(documentDeleted({ id }));
  }
}
```

```
<app-spinner *ngIf="loading$ | async"></app-spinner>

<div *ngFor="let doc of documents$ | async">
  {{ doc.name }}
  <button (click)="onDelete(doc.id)">Delete</button>
</div>
```

▲ Use `store.select(...)` + the `async` pipe. **Never**
`store.select(x).subscribe(v => this.x = v)` without unsubscribing — the
store is an infinite stream.

Part 6 — Effects

Effects handle everything impure: HTTP, routing, localStorage, timers, toasts.

The easiest way to remember

An EFFECT listens for an action, does the messy real-world work, and then dispatches ANOTHER action with the result. ★

Action IN → side effect → Action OUT

```
[Docs] Load (dispatched by the component)
  ↓ the effect hears it
  HTTP GET /documents ← the impure part lives HERE, never in a
  reducer ★
  ↓
[Docs] Load Success or [Docs] Load Failure
  ↓
the reducer handles it → the store updates → the UI reacts ✓
```

Real-world idea: the **back office of the bank**. The counter rule (reducer) is instant and never phones anyone. Anything involving another institution goes to the back office, which comes back with its own slip.

★ The two operator choices that get asked here:

```
LOAD (a read) → switchMap ★ cancel the stale request
SAVE (a write) → concatMap ★ NEVER cancel a write
```

→ the same rule as [20 — RxJS](#).

```

import { createEffect, Actions, ofType } from '@ngrx/effects';

@Injectable()
export class DocumentEffects {

  loadDocuments$ = createEffect(() =>
    this.actions$.pipe(
      ofType(loadDocuments),                                // 1. listen for
this action                                              // 2. do the
      switchMap(() =>                                     // 2. do the
async work
        this.documentService.getAll().pipe(
          map(documents => loadDocumentsSuccess({ documents })), // 3.
success action
          catchError(error => of(loadDocumentsFailure({          // 4.
failure action
            error: error.message
          })))
        )
      )
    )
  );

  constructor(
    private actions$: Actions,
    private documentService: DocumentService
  ) {}
}

```

▲ The two rules that get asked

Rule 1 — **catchError** goes **INSIDE** the inner observable.

```
// ✗ WRONG – the effect DIES on the first error and never runs again
this.actions$.pipe(
  ofType(loadDocuments),
  switchMap(() => this.service.getAll()),
  map(docs => loadDocumentsSuccess({ documents: docs })),
  catchError(err => of(loadDocumentsFailure({ error: err }))) // outer =
terminal
);

// ✓ RIGHT – the inner stream absorbs the error, the effect keeps
listening
this.actions$.pipe(
  ofType(loadDocuments),
  switchMap(() => this.service.getAll().pipe(
    map(docs => loadDocumentsSuccess({ documents: docs })),
    catchError(err => of(loadDocumentsFailure({ error: err })))
  ))
);
```

Same principle as 20 — RxJS, Part 7: an error terminates a stream. `actions$` is infinite — kill it and that feature stops responding for the rest of the session, with no error in the console.

Rule 2 — pick the right flattening operator.

Operator	Use for	Example
<code>switchMap</code>	Reads/search — cancel the previous	Load list, typeahead
<code>concatMap</code>	Writes — preserve order, never cancel	Save, update, delete
<code>exhaustMap</code>	Submissions — ignore duplicates	Login, submit form
<code>mergeMap</code>	Independent parallel work	Bulk file uploads

▲ **Never `switchMap` a save effect.** If two saves fire quickly, the first request gets cancelled — potentially after the server already started processing it. `concatMap` is the safe default for anything that writes.

Non-dispatching effects

Some effects only cause a side effect and dispatch nothing:

```
navigateAfterSave$ = createEffect(() =>
  this.actions$.pipe(
    ofType(saveDocumentSuccess),
    tap(() => this.router.navigate(['/documents']))
  ),
  { dispatch: false } // ← required, or NgRx tries to
dispatch `undefined`
);

showErrorToast$ = createEffect(() =>
  this.actions$.pipe(
    ofType(loadDocumentsFailure, saveDocumentFailure), // ofType takes
multiple actions
    tap(({ error }) => this.toastr.error(error))
  ),
  { dispatch: false }
);
```

Reading state inside an effect — `concatLatestFrom`

```
import { concatLatestFrom } from '@ngrx/operators';

saveDocument$ = createEffect(() =>
  this.actions$.pipe(
    ofType(saveDocument),
    concatLatestFrom(() => this.store.select(selectCurrentUser)), //
lazily reads state
    concatMap(([action, user]) =>
      this.service.save(action.document, user.id).pipe(
        map(saved => saveDocumentSuccess({ document: saved })),
        catchError(e => of(saveDocumentFailure({ error: e.message })))
      )
    )
  )
);
```

◆ `concatLatestFrom` is preferred over `withLatestFrom` because it evaluates the selector **lazily** — only when the action actually arrives, not on every subscription.

Effect for initial load

```
init$ = createEffect(() =>
  this.actions$.pipe(
    ofType(ROOT_EFFECTS_INIT),           // fires once when effects are
    initialized
    map(() => loadAppConfig())
  )
);
```

Part 7 — Wiring the store up

NgModule style (what your Angular 16 apps would use)

```
// app.module.ts - root
@NgModule({
  imports: [
    StoreModule.forRoot({}, {
      runtimeChecks: {
        strictStateImmutability: true,           // throws if you mutate state
        strictActionImmutability: true,
        strictStateSerializability: true,
        strictActionSerializability: true,
      }
    }),
    EffectsModule.forRoot([]),
    StoreDevtoolsModule.instrument({
      maxAge: 25,
      logOnly: environment.production,
    }),
  ],
})
export class AppModule {}
```



```
// documents.module.ts – lazy-loaded feature
@NgModule({
  imports: [
    StoreModule.forFeature('documents', documentReducer),
    EffectsModule.forFeature([DocumentEffects]),
  ],
})
export class DocumentsModule {}
```

- ◆ **forRoot vs forFeature**: **forRoot** sets up the store once in the root injector. **forFeature** registers a lazily-loaded slice under a key — the reducer is added to the store dynamically when the feature module loads. Same distinction as **RouterModule.forRoot / forFeature**.
- ◆ **runtimeChecks** are a great detail to mention — they catch accidental mutation at development time instead of letting it become a silent UI bug.

Standalone style (NgRx 15+, Angular 15+)

```
bootstrapApplication(AppComponent, {
  providers: [
    provideStore(),
    provideEffects([]),
    provideStoreDevtools({ maxAge: 25 }),
  ],
});

// route-level feature state
export const routes: Routes = [{
  path: 'documents',
  providers: [
    provideState(documentFeature),
    provideEffects(DocumentEffects),
  ],
  loadComponent: () => import('./documents.component'),
}];
```

Part 8 — @ngrx/entity

For collections, `@ngrx/entity` gives you **normalized state plus generated CRUD helpers**.

The shape it produces

```
{
  ids: ['d1', 'd2', 'd3'],           // ordered
  entities: {                       // 0(1) lookup by id
    d1: { id: 'd1', name: 'Passport.pdf' },
    d2: { id: 'd2', name: 'I-140.pdf' },
    d3: { id: 'd3', name: 'Photo.jpg' },
  }
}
```

Why normalize? A plain array means `0(n)` lookup and awkward updates (`map` with a ternary on every change). A keyed dictionary gives `0(1)` lookup, no duplication, and much simpler updates.

Setup

```
export interface DocumentState extends EntityState<Document> {
  loading: boolean;
  selectedId: string | null;
}

export const adapter: EntityAdapter<Document> =
createEntityAdapter<Document>({
  selectId: doc => doc.id,
  sortComparer: (a, b) => a.name.localeCompare(b.name),
});

export const initialState: DocumentState = adapter.getInitialState({
  loading: false,
  selectedId: null,
});

export const documentReducer = createReducer(
  initialState,
  on(loadDocumentsSuccess, (state, { documents }) =>
    adapter.setAll(documents, { ...state, loading: false })),
  on(documentAdded, (state, { document }) =>
    adapter.addOne(document, state)),
  on(documentUpdated, (state, { update }) =>
    adapter.updateOne(update, state)),          // { id, changes: {...} }
  on(documentDeleted, (state, { id }) =>
    adapter.removeOne(id, state)),
);

// Adapter-generated selectors
const { selectAll, selectEntities, selectIds, selectTotal } =
adapter.getSelectors();

export const selectDocumentState =
createFeatureSelector<DocumentState>('documents');
export const selectAllDocuments = createSelector(selectDocumentState,
selectAll);
export const selectDocumentTotal = createSelector(selectDocumentState,
selectTotal);
```

Adapter methods: `addOne` · `addMany` · `setAll` · `setOne` · `updateOne` ·
`updateMany` · `upsertOne` · `upsertMany` · `removeOne` · `removeMany` · `removeAll` ·
`map`

Part 9 — The Facade pattern

A facade service wraps the store so components never touch actions or selectors directly.

```
@Injectable({ providedIn: 'root' })
export class DocumentFacade {
  // Reads
  readonly documents$ = this.store.select(selectAllDocuments);
  readonly loading$   = this.store.select(selectLoading);
  readonly summary$   = this.store.select(selectDocumentSummary);

  constructor(private store: Store) {}

  // Writes
  load(): void { this.store.dispatch(loadDocuments()); }
  upload(file: File, panelId: string): void {
    this.store.dispatch(uploadDocument({ file, panelId }));
  }
  delete(id: string): void
  { this.store.dispatch(documentDeleted({ id })); }
}
```

```
// Component – clean, and doesn't know NgRx exists
export class DocumentListComponent {
  documents$ = this.facade.documents$;
  loading$   = this.facade.loading$;

  constructor(private facade: DocumentFacade) {}

  ngOnInit() { this.facade.load(); }
  onDelete(id: string) { this.facade.delete(id); }
}
```

✓ Pros	✗ Cons
Components decoupled from NgRx	One more layer
Easy to swap state library later	Can hide the action flow, hurting traceability
Much simpler component tests (mock the facade)	Teams sometimes turn it into a dumping ground

■ "This is the Facade pattern from GoF — one simplified interface over a subsystem (17). It's also what I'd use in a plain Angular app: a service exposing *BehaviorSubject* -backed observables is the same idea with less machinery — which is essentially what we do on RoboGebra."

Part 10 — Full worked example: EasyVisa document portal

The complete slice, end to end. This is your reference implementation.

document.actions.ts

```
export const DocumentActions = createActionGroup({
  source: 'Document Portal',
  events: {
    'Enter': emptyProps(),
    'Load Documents': emptyProps(),
    'Load Documents Success': props<{ documents: Document[] }>(),
    'Load Documents Failure': props<{ error: string }>(),
    'Upload Document': props<{ file: File; panelId: string }>(),
    'Upload Document Success': props<{ document: Document }>(),
    'Upload Document Failure': props<{ error: string }>(),
    'Delete Document': props<{ id: string }>(),
    'Delete Document Success': props<{ id: string }>(),
    'Panel Selected': props<{ panelId: string }>(),
  },
});
```

document.reducer.ts

```
export interface DocumentState extends EntityState<Document> {
  loading: boolean;
  uploading: boolean;
  error: string | null;
  selectedPanelId: string | null;
}

export const adapter = createEntityAdapter<Document>();

export const initialState: DocumentState = adapter.getInitialState({
  loading: false, uploading: false, error: null, selectedPanelId: null,
});

export const documentFeature = createFeature({
  name: 'documents',
  reducer: createReducer(
    initialState,
    on(DocumentActions.loadDocuments, s => ({ ...s, loading: true, error:
null })),
    on(DocumentActions.loadDocumentsSuccess, (s, { documents }) =>
      adapter.setAll(documents, { ...s, loading: false })),
    on(DocumentActions.loadDocumentsFailure, (s, { error }) =>
      ({ ...s, loading: false, error })),

    on(DocumentActions.uploadDocument, s => ({ ...s, uploading: true })),
    on(DocumentActions.uploadDocumentSuccess, (s, { document }) =>
      adapter.addOne(document, { ...s, uploading: false })),
    on(DocumentActions.uploadDocumentFailure, (s, { error }) =>
      ({ ...s, uploading: false, error })),

    on(DocumentActions.deleteDocumentSuccess, (s, { id }) =>
      adapter.removeOne(id, s)),
    on(DocumentActions.panelSelected, (s, { panelId }) =>
      ({ ...s, selectedPanelId: panelId })),
  ),
});
```

document.selectors.ts

```
const { selectAll } = adapter.getSelectors();

export const selectDocumentState = documentFeature.selectDocumentsState;
export const selectAllDocuments = createSelector(selectDocumentState,
  selectAll);
export const selectLoading = documentFeature.selectLoading;
export const selectSelectedPanel = documentFeature.selectSelectedPanelId;

export const selectDocumentsForSelectedPanel = createSelector(
  selectAllDocuments,
  selectSelectedPanel,
  (docs, panelId) => panelId ? docs.filter(d => d.panelId === panelId) :
  docs
);

export const selectPanelCompletion = createSelector(
  selectDocumentsForSelectedPanel,
  docs => ({
    total: docs.length,
    approved: docs.filter(d => d.status === 'APPROVED').length,
    isComplete: docs.length > 0 && docs.every(d => d.status ===
    'APPROVED'),
  })
);
```



```

@Injectables()
export class DocumentEffects {

  load$ = createEffect(() => this.actions$.pipe(
    ofType(DocumentActions.enter, DocumentActions.loadDocuments),
    switchMap(() => this.api.getDocuments().pipe(          // read →
switchMap
    map(documents =>
DocumentActions.loadDocumentsSuccess({ documents })),
    catchError(e => of(DocumentActions.loadDocumentsFailure({ error:
e.message }))))
    ))
  ));

  upload$ = createEffect(() => this.actions$.pipe(
    ofType(DocumentActions.uploadDocument),
    concatMap(({ file, panelId }) =>                      // write →
concatMap
    this.api.upload(file, panelId).pipe(
    map(document =>
DocumentActions.uploadDocumentSuccess({ document })),
    catchError(e => of(DocumentActions.uploadDocumentFailure({ error:
e.message }))))
    ))
  ));

  delete$ = createEffect(() => this.actions$.pipe(
    ofType(DocumentActions.deleteDocument),
    concatMap(({ id }) => this.api.delete(id).pipe(
    map(() => DocumentActions.deleteDocumentSuccess({ id })),
    catchError(e => of(DocumentActions.loadDocumentsFailure({ error:
e.message }))))
    ))
  ));

  notifyError$ = createEffect(() => this.actions$.pipe(
    ofType(DocumentActions.loadDocumentsFailure,
DocumentActions.uploadDocumentFailure),
    tap(({ error }) => this.toastr.error(error))
  ), { dispatch: false });

  constructor(
    private actions$: Actions,
    private api: DocumentApiService,
    private toastr: ToastrService
  ) {}
}

```

Unit tests — the payoff for all this structure

```
// Reducer test – pure function, zero mocks
it('sets loading on loadDocuments', () => {
  const state = documentReducer(initialState,
  DocumentActions.loadDocuments());
  expect(state.loading).toBe(true);
});

// Selector test – call the projector directly
it('counts approved documents', () => {
  const result = selectPanelCompletion.projector([
    { status: 'APPROVED' }, { status: 'PENDING' }
  ] as Document[]);
  expect(result.approved).toBe(1);
  expect(result.isComplete).toBe(false);
});

// Effect test – marble testing
it('dispatches success on load', () => {
  actions$ = hot('-a', { a: DocumentActions.loadDocuments() });
  api.getDocuments.and.returnValue(cold('-b|', { b: mockDocs }));
  expect(effects.load$.toBeObservable(
    hot('--c', { c: DocumentActions.loadDocumentsSuccess({ documents:
  mockDocs }) })
  ));
});
```

■ **Strong closing line:** *"**.projector()** is how you test a selector without building a whole state object — you call the projection function with its inputs directly. That testability is the main thing you buy with NgRx's boilerplate."*

Part 11 — ComponentStore & SignalStore

@ngrx/component-store — local state, no global store

For state that belongs to **one component or feature**, not the whole app.

```

interface QuizState { currentIndex: number; answers: Record<string,
string>; }

@Injectable()
export class QuizStore extends ComponentStore<QuizState> {
  constructor() { super({ currentIndex: 0, answers: {} }); }

  // selectors
  readonly currentIndex$ = this.select(s => s.currentIndex);
  readonly answers$      = this.select(s => s.answers);

  // updaters (synchronous, like reducers)
  readonly next = this.updater(state => ({ ...state, currentIndex:
state.currentIndex + 1 }));
  readonly setAnswer = this.updater((state, a: { q: string; v: string })
=> ({
    ...state, answers: { ...state.answers, [a.q]: a.v }
}));

  // effects (async)
  readonly submit = this.effect<void>(trigger$ => trigger$.pipe(
    withLatestFrom(this.answers$),
    exhaustMap([, answers]) => this.api.submit(answers).pipe(
      tapResponse(r => this.router.navigate(['/result', r.id]),
        e => this.toastr.error('Submit failed'))
    ))
  );
}

```

Provided at the **component** level, so it's created and destroyed with the component. No actions, no global store, far less boilerplate.

Global Store	ComponentStore
App-wide, shared state	Local/feature state
Actions + reducers + effects + selectors	Updaters + effects + selectors
DevTools time-travel	No actions to time-travel
Survives navigation	Dies with the component

@ngrx/signals SignalStore (NgRx 17+) — the modern direction

```
export const QuizStore = signalStore(  
  { providedIn: 'root' },  
  withState({ currentIndex: 0, answers: {} as Record<string, string> } ),  
  withComputed(({ answers }) => ({  
    answeredCount: computed(() => Object.keys(answers()).length),  
  })),  
  withMethods(store => ({  
    next: () => patchState(store, s => ({ currentIndex: s.currentIndex +  
1 })),  
  })),  
);
```

■ **Worth one sentence, honestly framed:** "NgRx is moving toward the Signal Store — signals-based, far less boilerplate, no actions or reducers. I've read about it but haven't shipped it; our apps are Angular 16 and 18 with NgModules, so signals-first architecture wasn't on the table." Don't overclaim here.

Part 12 — Router Store & DevTools

@ngrx/router-store

Puts router state into the store so route params/query params become selectable:

```
StoreModule.forRoot({ router: routerReducer } ),  
StoreRouterConnectingModule.forRoot(),  
  
export const selectRouteParams = createSelector(selectRouterState, r =>  
  r.state.params);  
export const selectApplicationId = createSelector(selectRouteParams, p =>  
  p['applicationId']);
```

@ngrx/store-devtools

The Redux DevTools browser extension gives you:

- Every dispatched action in order, with its payload
- The full state tree before and after each action

- A diff view of exactly what changed
- **Time travel** — click any past action to restore that state
- Export/import state, so a tester can send you the exact state that broke

◆ Great practical answer: *"DevTools is genuinely the biggest day-to-day benefit. A QA can export the store state at the moment of a bug and I can import it and reproduce it exactly — that turned 'cannot reproduce' tickets into fixable ones."*

Part 13 — When NOT to use NgRx

Being able to argue against NgRx is a senior signal. Interviewers ask this deliberately.

✗ Don't use NgRx when

- The app is small or mostly CRUD screens
- State is **local** to a component — a form, a modal, a toggle
- Only one or two components share the data
- The team is small and unfamiliar with Redux — the learning curve is real
- You'd be writing 4 files (action, reducer, selector, effect) to store one boolean

✓ Do use NgRx when

- Many unrelated components read and write the same state
- State must survive navigation between routes
- You need an audit trail / undo-redo / time-travel debugging
- The team is large enough that convention beats individual judgement
- There's genuinely complex state interaction — optimistic updates, offline sync, websockets

Alternatives on the spectrum

Approach	Fits
Plain service + <code>BehaviorSubject</code>	★ Small-medium shared state — what RoboGebra uses
<code>@ngrx/component-store</code>	Local/feature state with async work
Angular Signals (16+)	Synchronous local state, fine-grained reactivity
Full NgRx Store	Large app, many teams, complex shared state

■ **Your best answer to "would you use NgRx?":** *"It depends on how much state is genuinely shared. On EasyVisa the visa application state was read and written across many modules, so NgRx paid for itself in traceability. On RoboGebra most state is per-feature, so we used services with `BehaviorSubject` and the `async` pipe — the same unidirectional discipline without four files per feature. I'd rather start simple and introduce NgRx when the pain is real than pay the boilerplate tax up front."*

Part 14 — Common mistakes

✗ 1. Mutating state in a reducer

Covered in [Part 4](#). Enable `runtimeChecks` to catch it.

✗ 2. HTTP calls in reducers

Reducers must be pure. All async work belongs in effects.

✗ 3. `catchError` in the outer effect pipe

Kills the effect permanently. See [Part 6](#).

X 4. `switchMap` on a save effect

Cancels in-flight writes. Use `concatMap`.

X 5. Putting everything in the store

Form values mid-edit, modal open/closed, hover state — these don't belong in global state. Store what's **shared** and **needs to survive**.

X 6. Reusing one action across sources

```
// X Now you can't tell which screen triggered it
export const loadData = createAction('[App] Load Data');

// ✓ Distinct actions, both handled by the same effect via ofType(a, b)
export const dashboardOpened = createAction('[Dashboard Page] Opened');
export const refreshClicked = createAction('[Dashboard Page] Refresh
Clicked');
```

X 7. Selecting state inside a subscribe instead of using `async`

```
// X leaks unless you unsubscribe
this.store.select(selectDocuments).subscribe(d => this.docs = d);

// ✓
docs$ = this.store.select(selectDocuments); // + async pipe
```

X 8. Deriving data in the component instead of a selector

Put derivation in a selector so it's memoized and testable — not in a getter that re-runs on every change detection cycle.

X 9. Dispatching from inside a reducer

Impossible by design, and the impulse means the action modelling is wrong. Use an effect.

Part 15 — Interview Q&A

Q: Explain the NgRx data flow.

"Unidirectional. A component dispatches an action describing what happened. Reducers handle it synchronously and return new immutable state. Effects listen to the same action stream for anything async — HTTP, routing, notifications — and dispatch follow-up success or failure actions that reducers then handle. Components read state through memoized selectors, usually with the async pipe. Nothing writes to the store directly; every change traces back to one named action."

Q: Reducer vs Effect?

"A reducer is a pure synchronous function — state in, new state out, no side effects. An effect is where impurity lives: HTTP, router navigation, localStorage, toasts. Effects don't touch state directly; they dispatch actions that reducers handle."

Q: Why must reducers be pure?

*"Predictability and tooling. Same state plus same action must always give the same result, which is what makes time-travel debugging and replay work. And purity plus immutability is what lets memoized selectors and **OnPush** change detection use reference equality — mutate state and both silently break."*

Q: What is selector memoization and why does it matter?

(See [Part 5](#).)

Q: How do you handle errors in effects?

*"**catchError** inside the inner observable, returning a failure action via **of(...)**. If you put it in the outer pipe the error terminates the **actions\$** stream and the effect stops working for the rest of the session — silently, with no console error. I*

also usually have one non-dispatching effect that listens to all the failure actions and shows a toast, so error presentation is in one place."

Q: `forRoot` vs `forFeature` ?

"`forRoot` initializes the store and effects once in the root injector. `forFeature` registers a state slice and its effects for a lazily-loaded module — the reducer is added to the store dynamically when that module loads, so feature state doesn't cost anything until the user goes there."

Q: What is `@ngrx/entity` for?

"Normalized collections. Instead of an array you keep `{ ids, entities }`, giving $O(1)$ lookup by id and no duplication. `createEntityAdapter` generates `addOne`, `updateOne`, `removeOne`, `setAll` and matching selectors, which removes most of the repetitive immutable-update code from reducers."

Q: How do you test NgRx code?

"Reducers are pure functions — call them with a state and an action and assert on the result, no mocks. Selectors are tested with `.projector()`, passing inputs directly rather than constructing a full state tree. Effects are tested with `jasmine-marbles`, mocking the service and asserting the emitted action stream. That testability is really the return on NgRx's boilerplate."

Q: NgRx vs a service with a BehaviorSubject?

(See [Part 13](#) — this is the question where you cite EasyVisa vs RoboGebra.)

Q: How does NgRx improve performance?

"Two ways. Immutable state means `OnPush` components can skip change detection by reference comparison. And memoized selectors mean expensive derivations — filtering, sorting, aggregating — only recompute when their actual inputs change, not on every tick."

Part 16 — Rapid-fire

Question	Answer
NgRx in one line	Redux for Angular, built on RxJS
The 5 building blocks	Store, Actions, Reducers, Selectors, Effects
The 3 principles	Single source of truth · read-only state · pure reducers
Action shape	<code>{ type: '[Source] Event', ...props }</code>
Action naming	<code>[Source] Event</code> , past tense, unique per source
Actions are...	Events, not commands
Reducer signature	<code>(state, action) => newState</code>
Reducer must be	Pure, immutable, side-effect free
Where does HTTP go?	Effects
Effect operator for reads	<code>switchMap</code>
Effect operator for writes	<code>concatMap</code>
Effect operator for submits	<code>exhaustMap</code>
<code>catchError</code> placement	Inside the inner observable
Effect with no dispatch	<code>{ dispatch: false }</code>
Read state in an effect	<code>concatLatestFrom</code>
Selector superpower	Memoization on input reference equality
Test a selector	<code>.projector(inputs)</code>
Test an effect	<code>jasmine-marbles</code> (<code>hot</code> / <code>cold</code>)
<code>createFeatureSelector</code>	Grabs a named feature slice

Question	Answer
<code>createFeature</code>	Reducer + auto-generated selectors (NgRx 13+)
<code>createActionGroup</code>	Grouped actions, less boilerplate (NgRx 15+)
<code>@ngrx/entity</code> state shape	<code>{ ids: [], entities: {} }</code>
<code>forRoot</code> vs <code>forFeature</code>	Root store vs lazy-loaded slice
<code>runtimeChecks</code>	Dev-time guards against mutation/non-serializable state
ComponentStore	Local state, no global actions
SignalStore	NgRx 17+, signals-based, minimal boilerplate
DevTools killer feature	Time travel + exportable state
Biggest downside	Boilerplate — 4 files to change one value
NgRx version	Tracks Angular majors (Angular 16 → NgRx 16)
Facade pattern	Service wrapping dispatch + select

One-day drill

Morning — Parts 2–6. Draw the data-flow diagram from memory. Write an action, reducer, selector and effect for one feature without looking.

Afternoon — Parts 10 and 13. Rehearse the full EasyVisa slice, then rehearse the "why NgRx on EasyVisa but not RoboGebra" answer until it's natural. That comparison is the single most valuable thing in this file — it turns a potential gap into evidence of engineering judgement.

Final reminder: never describe RoboGebra as an NgRx app. NgRx = **EasyVisa**. RoboGebra = **services with BehaviorSubject**. Being precise about which project used what is exactly the credibility that gets offers.

Java 8 Streams — 20 Coding Problems (Easy Version)

The 20 questions that come up again and again in Java interviews, each with a **simple, readable solution** you can write on a whiteboard.

Every problem follows the same shape:

Easiest way to remember → **the code** → **how it works step by step** → **the trap** → **Easy memory box**.

Your context: the RoboGebra backend is **Java 17**, and 198 files already use `.stream()` (18 — [Technical Versions](#)). So `.toList()` (Java 16+) is available to you — but I've shown `Collectors.toList()` too, since some interviewers still expect it.

Before you start — 4 things to know

1. A stream pipeline has three parts

```
list.stream()                // SOURCE
    .filter(n -> n > 10)      // INTERMEDIATE (lazy, returns a
Stream)                      // INTERMEDIATE
    .map(n -> n * 2)          // INTERMEDIATE
    .collect(Collectors.toList()); // TERMINAL (triggers execution,
returns a result)
```

SOURCE	INTERMEDIATE	TERMINAL
list.stream()	→ filter → map → sorted → collect	
	└─ lazy, nothing runs ─┘	└─ everything runs HERE ▲

Real-world idea: a **washing machine**. Loading clothes, adding detergent and choosing the cycle are all just settings (intermediate). Pressing **START** is the terminal operation — only then does anything happen.

2. Nothing runs until the terminal operation

Intermediate operations are **lazy** — they only build the pipeline.

3. A stream can only be consumed ONCE

Reuse it and you get `IllegalStateException: stream has already been operated upon or closed`.

4. Streams don't modify the source

They produce a **new** result. The original list is untouched.

Static imports that make the code readable (assume these in every answer):

```
import java.util.*;
import java.util.stream.*;
import static java.util.stream.Collectors.*;
import static java.util.function.Function.identity;
```

The sample class used throughout:

```
record Employee(int id, String name, String department, double salary, int
age) {}
// (Java 17 record – or a normal class with getters if the interviewer
prefers)
```

Table of contents

#	Problem	Key API
1	Count character occurrences	<code>groupingBy</code> + <code>counting</code>
2	First non-repeated character	<code>LinkedHashMap</code> + <code>findFirst</code>
3	Find duplicates	<code>Set.add()</code> trick

#	Problem	Key API
4	Max and min	<code>max</code> / <code>min</code> / <code>summaryStatistics</code>
5	Sort objects by field	<code>Comparator.comparing</code>
6	List → Map	<code>Collectors.toMap</code>
7	Sum and average	<code>mapToInt</code>
8	Element frequency	<code>groupingBy</code> + <code>counting</code>
9	Remove duplicates	<code>distinct</code>
10	Reverse a string	<code>reduce</code>
11	Second highest	<code>distinct</code> + <code>skip</code>
12	Anagrams	<code>chars().sorted()</code>
13	Group by field	<code>groupingBy</code>
14	Partition even/odd	<code>partitioningBy</code>
15	Longest string	<code>max(comparingInt)</code>
16	Match conditions	<code>allMatch</code> / <code>anyMatch</code> / <code>noneMatch</code>
17	Find any / first	<code>findFirst</code> / <code>findAny</code>
18	Stream → List/Set	<code>toList</code> / <code>toSet</code>
19	Join strings	<code>Collectors.joining</code>
20	Flatten nested lists	<code>flatMap</code>

1. Count character occurrences in a string

The easiest way to remember:

"Group by the character, then count each group."

```
groupingBy(identity(), counting())
```

```
String str = "programming";

Map<Character, Long> counts = str.chars()           // IntStream of
char codes
    .mapToObj(c -> (char) c)                       // IntStream ->
Stream<Character>
    .collect(groupingBy(identity(), counting()));

System.out.println(counts);
// {p=1, a=1, r=2, g=2, i=1, m=2, n=1, o=1}
```

How it works

```
"programming"
  ↓ .chars()
IntStream: 112, 114, 111, 103, ...    ← char CODES, not characters
  ↓ .mapToObj(c -> (char) c)
Stream<Character>: p, r, o, g, r, a, m, m, i, n, g
  ↓ groupingBy(identity(), counting())
{p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
```

`identity()` just means "group by the element itself".

Real-world idea: **sorting a bag of mixed coins into piles**, then counting each pile.

Keep insertion order — use a `LinkedHashMap`

```
Map<Character, Long> ordered = str.chars()
    .mapToObj(c -> (char) c)
    .collect(groupingBy(identity(), LinkedHashMap::new, counting()));
// {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
```

Count one specific character

```
long count = str.chars().filter(c -> c == 'g').count();    // 2
```

Follow-up: "without streams?"

```
java Map<Character, Integer> map = new HashMap<>(); for (char c :
str.toCharArray()) { map.put(c, map.getOrDefault(c, 0) + 1); }
```

`map.merge(c, 1, Integer::sum)` is the shorter modern version. **Know both** — some interviewers specifically ask for the loop.

Easy memory

```
str.chars() → mapToObj(c -> (char) c) → groupingBy(identity(), counting())
```

▲ `chars()` gives an `IntStream` (codes), so you **MUST** `mapToObj`

★ Need order? → `groupingBy(identity(), LinkedHashMap::new, counting())`

Loop version: `map.merge(c, 1, Integer::sum)`

2. Find the first non-repeated character

The easiest way to remember:

Count every character (KEEPING ORDER), then take the first one whose count is 1.

```
String str = "swiss";
```

```
Character first = str.chars()
    .mapToObj(c -> (char) c)
    .collect(groupingBy(identity(), LinkedHashMap::new, counting())) //
keep order! ★
    .entrySet().stream()
    .filter(e -> e.getValue() == 1L)
    .map(Map.Entry::getKey)
    .findFirst()
    .orElse(null);
```

```
System.out.println(first); // w
```


How it works

```
"swiss"
  ↓ count, keeping insertion order
{s=3, w=1, i=1}
  ↓ filter value == 1
{w=1, i=1}
  ↓ findFirst
w ✓
```

The trick: `LinkedHashMap::new` is essential. A plain `HashMap` has **no ordering**, so "first" would be meaningless — you might get `i` instead of `w`.

Real-world idea: a **queue of people**. You need the first person wearing a unique badge colour — so the queue order must be preserved.

One-pass alternative (better for very long strings... or is it?)

```
Character first = str.chars()
    .mapToObj(c -> (char) c)
    .filter(c -> str.indexOf(c) == str.lastIndexOf(c))    // appears
    exactly once
    .findFirst()
    .orElse(null);
```

▲ Mention the trade-off out loud: this version is **$O(n^2)$** because of the repeated `indexOf` scans. The map version is **$O(n)$** . Interviewers like candidates who notice.

Easy memory

```
LinkedHashMap is MANDATORY here (HashMap has no order → wrong answer) ★

count → filter(count == 1) → findFirst

Alternative: indexOf(c) == lastIndexOf(c) → simpler but  $O(n^2)$ 
```

3. Identify duplicate elements in a list

The easiest way to remember:

`Set.add()` returns `FALSE` when the element is already there.
So `!seen.add(n)` is `TRUE` exactly for duplicates.

```
List<Integer> nums = List.of(1, 2, 3, 2, 4, 5, 1, 6, 3);

Set<Integer> seen = new HashSet<>();
List<Integer> duplicates = nums.stream()
    .filter(n -> !seen.add(n))    // add() returns false if already
    present
    .distinct()                  // in case something appears 3+ times
    .collect(toList());

System.out.println(duplicates);  // [2, 1, 3]
```

How it works

```
1 → seen.add(1) = true → !true = false → dropped
2 → seen.add(2) = true → dropped
3 → seen.add(3) = true → dropped
2 → seen.add(2) = FALSE → !false = TRUE → KEPT ✓ duplicate
```

Real-world idea: a **guest list at a party**. You tick each name as it arrives; if a name is already ticked, that person is a repeat.

Frequency-based version (cleaner, no shared mutable state)

```
List<Integer> duplicates = nums.stream()
    .collect(groupingBy(identity(), counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(toList());
```

Follow-up: "which is better?" The **second**.

The `Set.add()` trick **mutates an external variable**, which silently breaks with parallel streams. Say that — it's exactly the kind of detail that separates candidates.

Easy memory

Trick: `.filter(n -> !seen.add(n))` ← `add()` returns false for a duplicate

Better: `groupingBy(identity(), counting()) → filter(count > 1) → map(getKey)`
(no external mutation → parallel-safe ★)

4. Find max and min using Streams

The easiest way to remember:

Need ONE thing? → `max()` / `min()`

Need EVERYTHING? → `summaryStatistics()` ★ one pass, all five values

```
List<Integer> nums = List.of(45, 12, 89, 7, 63);

// With Optional
Optional<Integer> max = nums.stream().max(Comparator.naturalOrder());
Optional<Integer> min = nums.stream().min(Comparator.naturalOrder());
System.out.println(max.get() + " " + min.get());    // 89 7

// With IntStream – no Optional boxing
int maxVal =
nums.stream().mapToInt(Integer::intValue).max().getAsInt();    // 89
int minVal =
nums.stream().mapToInt(Integer::intValue).min().getAsInt();    // 7

// ★ Best: ONE pass, everything at once
IntSummaryStatistics stats =
nums.stream().mapToInt(Integer::intValue).summaryStatistics();
System.out.println(stats.getMax());    // 89
System.out.println(stats.getMin());    // 7
System.out.println(stats.getSum());    // 216
System.out.println(stats.getAverage());    // 43.2
System.out.println(stats.getCount());    // 5
```

FIVE separate streams max → pass 1 ✓ min → pass 2 sum → pass 3 avg → pass 4 count → pass 5	ONE <code>summaryStatistics()</code> → pass 1 → max, min, sum, average, count
--	---

Real-world idea: instead of walking through the whole class list five times to find the topper, the failer, the total, the average and the headcount — you walk **once** and note all five as you go.

On objects

```
Employee highestPaid = employees.stream()
    .max(Comparator.comparingDouble(Employee::salary))
    .orElseThrow();
```

◆ **`summaryStatistics()` is the answer that impresses.** It computes max, min, sum, average and count in **one pass** instead of five separate streams.

Easy memory

```
max(Comparator.naturalOrder()) → Optional
mapToInt(...).max().getAsInt() → primitive, no Optional boxing
summaryStatistics()           → max + min + sum + average + count in ONE
pass ★

Objects: max(Comparator.comparingDouble(Employee::salary))
```

5. Sort a list of objects by a field

The easiest way to remember:

```
Comparator.comparing(Employee::field)
    .reversed()           ← descending
    .thenComparing(...)   ← tie-breaker
```

```

List<Employee> employees = List.of(
    new Employee(1, "Priya", "IT", 75000, 28),
    new Employee(2, "Arun", "HR", 55000, 35),
    new Employee(3, "Divya", "IT", 85000, 30),
    new Employee(4, "Karthik", "Sales", 65000, 26)
);

// Ascending
List<Employee> bySalary = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::salary))
    .collect(toList());

// Descending
List<Employee> desc = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::salary).reversed())
    .collect(toList());

// Multiple fields: department ascending, then salary descending
List<Employee> multi = employees.stream()
    .sorted(Comparator.comparing(Employee::department)
        .thenComparing(Comparator.comparingDouble(Employee::
salary).reversed()))
    .collect(toList());

// Null-safe on a nullable field
Comparator.comparing(Employee::name,
    Comparator.nullsLast(Comparator.naturalOrder()));

```

thenComparing = "if the first field is EQUAL, use this next"

```

HR    → Arun
IT    → Divya (85000) ← salary descending WITHIN the department
IT    → Priya (75000)
Sales → Karthik

```

Real-world idea: a **class ranking**. Sort by marks; if two students have the same marks, sort by name.

▲ **sorted()** does NOT modify the original list — it returns a new stream.
list.sort(...) mutates in place. Interviewers ask this.

Easy memory

```
sorted(Comparator.comparing(Employee::getName))
sorted(Comparator.comparingDouble(Employee::salary).reversed()) ←
descending
    .thenComparing(...) ← tie-
breaker
Comparator.nullsLast(...) ←
null-safe

sorted() → NEW stream, original untouched ✓
list.sort() → mutates in place ▲
```

6. Convert a List into a Map

The easiest way to remember:

```
toMap(keyFunction, valueFunction)
```

▲ AND ALWAYS think about the third argument – the merge function.

```
Map<Integer, String> idToName = employees.stream()
    .collect(toMap(Employee::id, Employee::name));
// {1=Priya, 2=Arun, 3=Divya, 4=Karthik}

// Whole object as the value
Map<Integer, Employee> byId = employees.stream()
    .collect(toMap(Employee::id, identity()));
```

▲ The duplicate-key trap — the #1 follow-up

```
// ✗ Throws IllegalStateException: Duplicate key IT
Map<String, String> byDept = employees.stream()
    .collect(toMap(Employee::department, Employee::name));
```

```
Priya → key "IT" → stored
Divya → key "IT" → ▲ IllegalStateException: Duplicate key IT
```

```
// ✓ Supply a merge function to say what to do on a collision
Map<String, String> byDept = employees.stream()
    .collect(toMap(Employee::department, Employee::name,
        (existing, replacement) -> existing + ", " +
        replacement));
// {IT=Priya, Divya, HR=Arun, Sales=Karthik}
```

```
(a, b) -> a      // keep the FIRST
(a, b) -> b      // keep the LAST
(a, b) -> a + ", " + b // combine them
```

Real-world idea: two people arrive with the **same house number**. The merge function is your rule: *"keep the first", "keep the newest", or "put both on the nameplate"*.

Choose the map type (4-argument form)

```
Map<Integer, String> sorted = employees.stream()
    .collect(toMap(Employee::id, Employee::name, (a, b) -> a,
        TreeMap::new));
```

▲ **toMap** also throws a **NullPointerException** if a **VALUE** is null — a real gotcha that surprises people. **groupingBy** doesn't have that problem.

Easy memory

toMap(key, value)	→ ▲ on duplicate keys
toMap(key, value, (a,b) -> a)	→ ✓ first wins
toMap(key, value, (a,b) -> b)	→ last wins
toMap(key, value, merge, TreeMap::new)	→ choose the map type

▲ Duplicate key → **IllegalStateException**
 ▲ Null VALUE → **NullPointerException**

7. Sum and average of a list of numbers

The easiest way to remember:

```
mapToInt / mapToDouble first → then .sum() or .average()
(this drops the Integer boxing and gives you the primitive methods)
```

```
List<Integer> nums = List.of(10, 20, 30, 40, 50);

int sum =
nums.stream().mapToInt(Integer::intValue).sum();           //
150
double avg =
nums.stream().mapToInt(Integer::intValue).average().orElse(0.0); // 30.0

// With reduce
int sum2 = nums.stream().reduce(0,
Integer::sum);           // 150

// With a collector
double avg2 =
nums.stream().collect(averagingInt(Integer::intValue));    // 30.0
int sum3 =
nums.stream().collect(summingInt(Integer::intValue));       // 150
```

How **reduce** works, step by step

```
[10, 20, 30, 40, 50]    identity = 0

0 + 10 = 10
10 + 20 = 30
30 + 30 = 60
60 + 40 = 100
100 + 50 = 150 ← final result
```

On an object field

```
double totalSalary =
employees.stream().mapToDouble(Employee::salary).sum();
double avgSalary =
employees.stream().mapToDouble(Employee::salary).average().orElse(0);
```

Why does `average()` return `OptionalDouble` ? An **empty list has no average** — dividing by zero is undefined. `.orElse(0.0)` handles it. Saying this shows you think about edge cases.

Easy memory

```
sum      → mapToInt(...).sum()
average → mapToInt(...).average().orElse(0.0)  ← OptionalDouble! empty
list has no average ★
reduce   → reduce(0, Integer::sum)
```

```
Objects: mapToDouble(Employee::salary).sum()
```

8. Find the frequency of each element in a list

The easiest way to remember:

```
Same recipe as problem 1: groupingBy(identity(), counting())
```

```
List<String> items = List.of("apple", "banana", "apple", "orange",
    "banana", "apple");
```

```
Map<String, Long> frequency = items.stream()
    .collect(groupingBy(identity(), counting()));
```

```
System.out.println(frequency);
// {banana=2, orange=1, apple=3}
```

Sorted by count, highest first

```
LinkedHashMap<String, Long> sorted = frequency.entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
    .collect(toMap(Map.Entry::getKey, Map.Entry::getValue,
        (a, b) -> a, LinkedHashMap::new));
// {apple=3, banana=2, orange=1}
```

▲ `LinkedHashMap::new` again — collecting into a plain `HashMap` would **throw away** the sorting you just did.

The most frequent element

```
String mostCommon = frequency.entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey)
    .orElse(null);    // apple
```

Real-world idea: counting **votes**. Group the ballots by candidate, count each pile, then take the biggest pile.

Easy memory

```
frequency → groupingBy(identity(), counting())
sort by count → .sorted(Map.Entry.comparingByValue().reversed())
                + collect into LinkedHashMap ★ (else the order is lost)
most common → .max(Map.Entry.comparingByValue()).map(Map.Entry::getKey)
```

9. Remove duplicates from a list

The easiest way to remember:

```
distinct() → keeps ORDER, uses equals() + hashCode()
new HashSet<>(list) → loses order
new LinkedHashSet<>(list) → keeps order
```

```
List<Integer> nums = List.of(1, 2, 3, 2, 4, 1, 5);

List<Integer> unique = nums.stream().distinct().collect(toList());
// [1, 2, 3, 4, 5] - order preserved ✓

// Via a Set (order NOT preserved)
Set<Integer> set = new HashSet<>(nums);

// Order preserved with a Set
Set<Integer> ordered = new LinkedHashSet<>(nums);
```

▲ Duplicates of objects — the important follow-up

`distinct()` uses `equals()` and `hashCode()`.

Without them, two objects with IDENTICAL fields are treated as different.



```
// Records generate equals/hashCode automatically ✓  
// A normal class MUST override both, or distinct() does nothing useful
```

Distinct by ONE field (no `equals` / `hashCode` needed):

```
List<Employee> uniqueByDept = employees.stream()  
    .collect(toMap(Employee::department, identity(), (a, b) -> a))  
    .values().stream()  
    .collect(toList());
```

`toMap` with `(a, b) -> a` → "if this department already has an employee, keep the first"



`.values()` → one employee per department ✓

Or with a `TreeSet` comparator:

```
List<Employee> byName = employees.stream()  
    .collect(collectingAndThen(  
        toCollection(() -> new  
        TreeSet<>(Comparator.comparing(Employee::name))),  
        ArrayList::new));
```

Easy memory

```
distinct()                → order kept, needs equals()+hashCode() ★  
new HashSet<>(list)        → order lost  
new LinkedHashSet<>(list) → order kept
```

Distinct by ONE field:

```
toMap(Employee::department, identity(), (a,b) -> a).values()
```

10. Reverse a string using Stream operations

The easiest way to remember:

```
reduce normally APPENDS (a + b).  
Swap it to (b + a) and each new character goes IN FRONT → reversed.
```

```
String str = "interview";  
  
String reversed = str.chars()  
    .mapToObj(c -> String.valueOf((char) c))  
    .reduce("", (a, b) -> b + a); // prepend each character  
System.out.println(reversed); // weivretni
```

How it works

```
"" + i → "i"  
i → "n" + "i" → "ni"  
n → "t" + "ni" → "tni"  
... each new letter jumps to the FRONT
```

The honest answer — say this too ★

```
String reversed = new StringBuilder(str).reverse().toString(); // ✓  
simplest and fastest
```

◆ **How to handle this in an interview:** "Streams can do it with *reduce*, but string concatenation inside a reduce is $O(n^2)$ because each step creates a new String. In real code I'd use *StringBuilder.reverse()* — $O(n)$."

Showing you know when **not** to use a stream is a plus, not a minus.

Reverse the WORDS instead of the characters

```
String s = "Java is fun";  
String result = Arrays.stream(s.split(" "))  
    .reduce((a, b) -> b + " " + a)  
    .orElse("");  
// fun is Java
```

Easy memory

Characters: `.reduce("", (a, b) -> b + a)` $\leftarrow b + a = \text{prepend} = \text{reverse}$

Words: `.reduce((a, b) -> b + " " + a)`

▲ $O(n^2)$ – in REAL code use: `new StringBuilder(str).reverse().toString()`

★
Saying this out loud SCORES points.

11. Find the second highest number

The easiest way to remember:

`distinct() → sorted(reverseOrder()) → skip(1) → findFirst()`

▲ `distinct()` is the whole question. Forget it and you get the duplicate.

```
List<Integer> nums = List.of(45, 12, 89, 7, 89, 63);
```

```
Optional<Integer> secondHighest = nums.stream()
    .distinct()                      // ★ essential – 89 appears
    twice
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();
```

```
System.out.println(secondHighest.orElse(-1)); // 63
```

How it works

```
[45, 12, 89, 7, 89, 63]
  ↓ distinct()
[45, 12, 89, 7, 63]
  ↓ sorted(reverseOrder())
[89, 63, 45, 12, 7]
  ↓ skip(1)
[63, 45, 12, 7]
  ↓ findFirst()
63 ✓
```

The trap: without `distinct()`, the sorted list is `[89, 89, 63, ...]`, so `skip(1).findFirst()` gives **89** — the duplicate. Interviewers plant a duplicate specifically to test this.

Real-world idea: the **silver medallist**. If two athletes tie for gold, the second person on the list is still gold — you must remove the tie first.

Second highest salary (a classic SQL-style question in Java form)

```
Optional<Employee> second = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::salary).reversed())
    .skip(1)
    .findFirst();
```

Nth highest, generalised

```
static Optional<Integer> nthHighest(List<Integer> list, int n) {
    return list.stream().distinct()
        .sorted(Comparator.reverseOrder())
        .skip(n - 1)
        .findFirst();
}
```

Easy memory

`distinct()` → `sorted(reverseOrder())` → `skip(1)` → `findFirst()`

▲ FORGET `distinct()` AND YOU FAIL THE QUESTION ★

Nth highest → `skip(n - 1)`

12. Check if two strings are anagrams

The easiest way to remember:

Anagram = same letters, different order.

So: SORT both strings and compare. (Or compare their letter counts.)

```
String a = "listen", b = "silent";

boolean isAnagram = sortChars(a).equals(sortChars(b));

static String sortChars(String s) {
    return s.replaceAll("\\s", "").toLowerCase()
        .chars()
        .sorted()
        .mapToObj(c -> String.valueOf((char) c))
        .collect(joining());
}

System.out.println(isAnagram);    // true
```

```
"listen" → sorted → "eilnst"
"silent" → sorted → "eilnst"    ← identical ✓
```

Real-world idea: two bags of **Scrabble tiles**. Line the tiles up alphabetically; if both rows look identical, they're anagrams.

Frequency-map version ($O(n)$ instead of $O(n \log n)$)

```
static Map<Character, Long> freq(String s) {
    return s.toLowerCase().chars()
        .mapToObj(c -> (char) c)
        .collect(groupingBy(identity(), counting()));
}

boolean isAnagram = freq(a).equals(freq(b));
```

One-liner without streams — mention it as the practical answer

```
char[] x = a.toCharArray(), y = b.toCharArray();
Arrays.sort(x); Arrays.sort(y);
boolean isAnagram = Arrays.equals(x, y);
```

▲ Always **check the lengths first** as a fast exit, and decide out loud whether case and spaces matter. **Stating your assumptions is half the marks.**

Easy memory

Sort both → compare → $O(n \log n)$

Count both → compare maps → $O(n)$ ★

Fast exit: if (a.length() != b.length()) return false;

Ask out loud: case-sensitive? spaces? ★

13. Group objects by a specific field

The easiest way to remember:

`groupingBy(classifier)`

→ `Map<Key, List<T>>`

`groupingBy(classifier, downstream)`

→ `Map<Key, whatever-you-want>` ★

That **second argument** is the most valuable thing in the entire Streams API.


```

// Simple grouping
Map<String, List<Employee>> byDept = employees.stream()
    .collect(groupingBy(Employee::department));
// {IT=[Priya, Divya], HR=[Arun], Sales=[Karthik]}

// Count per group
Map<String, Long> countByDept = employees.stream()
    .collect(groupingBy(Employee::department, counting()));
// {IT=2, HR=1, Sales=1}

// Only the NAMES per group
Map<String, List<String>> namesByDept = employees.stream()
    .collect(groupingBy(Employee::department, mapping(Employee::name,
toList())));
// {IT=[Priya, Divya], HR=[Arun], Sales=[Karthik]}

// Average salary per group
Map<String, Double> avgByDept = employees.stream()
    .collect(groupingBy(Employee::department,
averagingDouble(Employee::salary)));
// {IT=80000.0, HR=55000.0, Sales=65000.0}

// Total salary per group
Map<String, Double> totalByDept = employees.stream()
    .collect(groupingBy(Employee::department,
summingDouble(Employee::salary)));

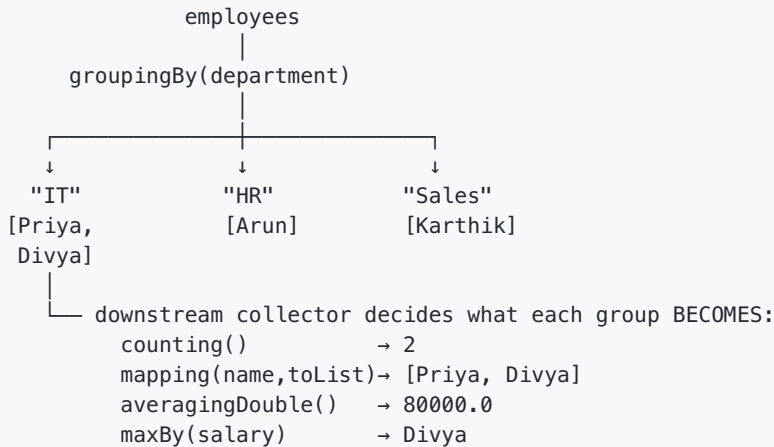
// Highest earner per group
Map<String, Optional<Employee>> topByDept = employees.stream()
    .collect(groupingBy(Employee::department,
maxBy(Comparator.comparingDouble(Employee::salary))));

// Sorted output (TreeMap)
Map<String, List<Employee>> sortedGroups = employees.stream()
    .collect(groupingBy(Employee::department, TreeMap::new, toList()));

// TWO-LEVEL grouping – department, then age band
Map<String, Map<String, List<Employee>>> nested = employees.stream()
    .collect(groupingBy(Employee::department,
groupingBy(e -> e.age() > 30 ? "Senior" : "Junior")));

```

The picture



Real-world idea: sorting a **pile of exam papers into subject stacks**, and then — for each stack — either counting them, averaging the marks, or pulling out just the topper.

◆ **groupBy** with a downstream collector is the single most valuable Streams API to know. `counting()`, `mapping()`, `averagingX()`, `summingX()`, `maxBy()`, `toSet()` and `joining()` all work as the second argument.

Easy memory

<code>groupBy(Employee::department)</code>	→ <code>Map<String, List<Employee>></code>
<code>groupBy(dept, counting())</code>	→ <code>Map<String, Long></code>
<code>groupBy(dept, mapping(Employee::name, toList()))</code>	→ <code>Map<String, List<String>></code>
<code>groupBy(dept, averagingDouble(Employee::salary))</code>	→ <code>Map<String, Double></code>
<code>groupBy(dept, maxBy(comparing(salary)))</code>	→ <code>Map<String, Optional<Employee>></code>
<code>groupBy(dept, TreeMap::new, toList())</code>	→ sorted keys
<code>groupBy(dept, groupBy(ageBand))</code>	→ two levels ★

14. Partition a list into even and odd numbers

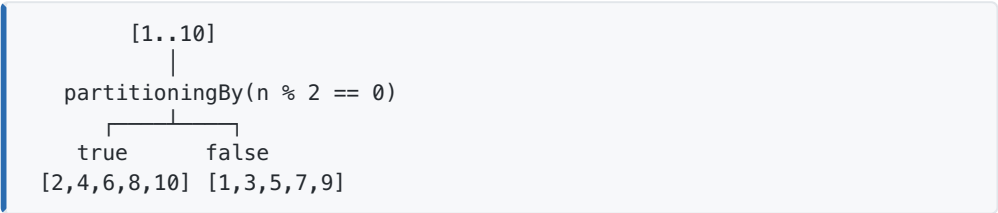
The easiest way to remember:

`partitioningBy` = `groupingBy` with EXACTLY two keys: `true` and `false`.

```
List<Integer> nums = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

Map<Boolean, List<Integer>> partitioned = nums.stream()
    .collect(partitioningBy(n -> n % 2 == 0));

System.out.println(partitioned.get(true));    // [2, 4, 6, 8, 10] - even
System.out.println(partitioned.get(false));   // [1, 3, 5, 7, 9]  - odd
```



Real-world idea: a **pass/fail split** of one exam. There are always exactly two buckets, even if one is empty.

With a downstream collector

```
Map<Boolean, Long> counts = nums.stream()
    .collect(partitioningBy(n -> n % 2 == 0, counting()));
// {false=5, true=5}
```

`partitioningBy` vs `groupingBy` — expect this question

	<code>partitioningBy</code>	<code>groupingBy</code>
Keys	always exactly two : <code>true</code> and <code>false</code>	any number
Key type	<code>Boolean</code>	anything
Empty groups	both keys always present , even if empty	missing keys are simply absent

	partitioningBy	groupingBy
Argument	a <code>Predicate</code>	a classifier <code>Function</code>
Performance	slightly faster for a boolean split	general purpose

- ◆ The **"both keys always present"** point is the detail most candidates miss.

```
java List.<Integer>of().stream().collect(partitioningBy(n -> n > 0)); // {false=[], true=[]} ← both keys, even on an empty list ★
```

Easy memory

```
partitioningBy(predicate) → Map<Boolean, List<T>>
    .get(true) → matched
    .get(false) → didn't match
```

★ BOTH keys ALWAYS exist, even when empty (groupingBy would omit them)
Two buckets only → use groupingBy if you need more

15. Find the longest string in a list

The easiest way to remember:

```
max(Comparator.comparingInt(String::length))
```

```
List<String> words = List.of("java", "streams", "interview", "code");

Optional<String> longest = words.stream()
    .max(Comparator.comparingInt(String::length));
System.out.println(longest.get());    // interview

// Shortest
Optional<String> shortest =
words.stream().min(Comparator.comparingInt(String::length));
// code
```

ALL strings of maximum length (handles ties) ★

```
int maxLen = words.stream().mapToInt(String::length).max().orElse(0);

List<String> allLongest = words.stream()
    .filter(w -> w.length() == maxLen)
    .collect(toList());
```

`max()` returns ONE element.

Real data has TIES. → two passes: find the max length, then filter by it.

Longest, with alphabetical order breaking ties

```
Optional<String> longestThenAlpha = words.stream()
    .max(Comparator.comparingInt(String::length)
        .thenComparing(Comparator.reverseOrder()));
```

◆ The **ties** version is worth volunteering unprompted — `max()` returns only one element, and real data has ties.

Easy memory

```
longest → max(comparingInt(String::length))
shortest → min(comparingInt(String::length))
```

TIES ★ → find `maxLen` first, then `filter(w -> w.length() == maxLen)`
(volunteer this — it shows you think about real data)

16. Check if all elements match a condition

The easiest way to remember:

```
allMatch → EVERY element passes?
anyMatch → at least ONE passes?
noneMatch → ZERO pass?
```

All three SHORT-CIRCUIT (they stop as soon as the answer is known). ▲

```
List<Integer> nums = List.of(2, 4, 6, 8, 10);

boolean allEven    = nums.stream().allMatch(n -> n % 2 == 0);    // true
boolean anyOver5   = nums.stream().anyMatch(n -> n > 5);         // true
boolean noneNeg    = nums.stream().noneMatch(n -> n < 0);        // true

// On objects
boolean allInIT     = employees.stream().allMatch(e ->
e.department().equals("IT"));
boolean anyHighEarner = employees.stream().anyMatch(e -> e.salary() >
80000);
```

▲ The empty-stream trap — a favourite trick question ★

```
List<Integer> empty = List.of();

empty.stream().allMatch(n -> n > 100);    // true  ▲  !!
empty.stream().anyMatch(n -> n > 100);    // false
empty.stream().noneMatch(n -> n > 100);   // true
```

Why is **allMatch** true on an empty stream?

It's VACUOUS TRUTH.

"All elements are > 100" means "there is NO element that is ≤ 100."
 An empty list has no elements at all → nothing violates it → TRUE ✓

Real-world idea: *"Every student in this empty classroom passed."* There's nobody who failed, so technically the statement holds. Mathematically correct, and it surprises almost everyone.

Easy memory

```
allMatch  → all pass    → EMPTY stream = TRUE  ▲ ▲  (vacuous truth)
anyMatch  → one passes  → EMPTY stream = false
noneMatch → none pass   → EMPTY stream = true
```

All three SHORT-CIRCUIT ▲ (stop as soon as the answer is decided)

17. Find any / first element

The easiest way to remember:

`findFirst` → the FIRST in order (costs coordination on a parallel stream)
`findAny` → WHICHEVER is quickest (faster in parallel)

On a SEQUENTIAL stream they behave the same.

```
List<String> names = List.of("Priya", "Arun", "Divya", "Karthik");

Optional<String> first = names.stream().findFirst();           // Priya
Optional<String> any  = names.stream().findAny();             // usually
Priya (sequential)

Optional<String> firstD = names.stream()
    .filter(n -> n.startsWith("D"))
    .findFirst();                                             // Divya

String result = names.stream()
    .filter(n -> n.startsWith("Z"))
    .findFirst()
    .orElse("Not found");                                     // Not
found
```

findFirst VS findAny

	findFirst	findAny
Sequential stream	first element	usually the first
Parallel stream	first in encounter order (costs coordination)	whatever finishes first — faster
Use when	order matters	order doesn't matter

■ "On a sequential stream they behave the same. On a parallel stream `findFirst` has to respect encounter order, which costs synchronisation, whereas `findAny` returns whichever thread finds a match first — so `findAny` is the faster choice when you genuinely don't care which element you get."

Real-world idea: **four people searching a library for any copy of a book.**

`findAny` = "shout as soon as you find one". `findFirst` = "we must have the copy from the lowest shelf number", so everyone must wait and compare.

Handling the `Optional` — know all four

```
opt.get()                // ✗ throws if empty – avoid
opt.orElse("default")    // default value
opt.orElseGet(() -> compute()) // LAZY default – only computed if
                             empty ★
opt.orElseThrow(() -> new NotFoundException("no match"))
opt.ifPresent(v -> System.out.println(v));
```

Easy memory

`findFirst` → order guaranteed | `findAny` → fastest, parallel-friendly
Same behaviour on a sequential stream ★

`Optional`: `orElse` (eager) | `orElseGet` (lazy) ★ | `orElseThrow` | `ifPresent`
NEVER just call `.get()`

18. Convert a Stream to a List or Set

The easiest way to remember:

```
.toList()           → Java 16+, shortest, but IMMUTABLE ▲
collect(toList())   → MUTABLE ArrayList
```



```

List<Integer> nums = List.of(5, 3, 8, 1, 9, 3);

// Java 16+ – shortest (▲ returns an UNMODIFIABLE list)
List<Integer> list1 = nums.stream().filter(n -> n > 3).toList();

// Classic – returns a mutable ArrayList
List<Integer> list2 = nums.stream().filter(n -> n > 3).collect(toList());

// Set (duplicates removed, unordered)
Set<Integer> set = nums.stream().collect(toSet());

// Specific implementations
List<Integer> arrayList =
    nums.stream().collect(toCollection(ArrayList::new));
Set<Integer> linkedSet =
    nums.stream().collect(toCollection(LinkedHashSet::new)); // keeps order
TreeSet<Integer> treeSet =
    nums.stream().collect(toCollection(TreeSet::new)); // sorted

// Array
Integer[] arr = nums.stream().toArray(Integer[]::new);
int[] ints = nums.stream().mapToInt(Integer::intValue).toArray();

```

▲ The `.toList()` gotcha

```

java List<Integer> l = nums.stream().toList(); l.add(10); // ▲
UnsupportedOperationException

```

`stream().toList()` (Java 16+) returns an **immutable** list. Use `collect(toList())` when you need to modify the result. **Your codebase is Java 17, so this is a live issue.**

Easy memory

<code>.toList()</code>	→ Java 16+, IMMUTABLE ▲ (add() throws)
<code>collect(toList())</code>	→ mutable ArrayList ✓
<code>collect(toSet())</code>	→ duplicates removed, no order
<code>toCollection(LinkedHashSet::new)</code>	→ unique + order kept ★
<code>toCollection(TreeSet::new)</code>	→ unique + sorted
<code>toArray(Integer[]::new)</code>	→ array

19. Join a list of strings with a delimiter

The easiest way to remember:

```
Collectors.joining(", ")
```

No transformation needed? → just use `String.join(", ", list)`

```
List<String> names = List.of("Priya", "Arun", "Divya");

String joined = names.stream().collect(joining(", "));
// Priya, Arun, Divya

// With prefix and suffix
String bracketed = names.stream().collect(joining(", ", "[", "]"));
// [Priya, Arun, Divya]

// Simplest of all – no stream needed
String simple = String.join(", ", names);
// Priya, Arun, Divya

// Joining a FIELD from objects
String allNames = employees.stream()
    .map(Employee::name)
    .collect(joining(" | "));
// Priya | Arun | Divya | Karthik

// Transform, then join
String upper = names.stream()
    .map(String::toUpperCase)
    .sorted()
    .collect(joining(", "));
// ARUN, DIVYA, PRIYA
```

◆ Mention `String.join()` for a plain list — it's clearer than a stream when there's no transformation. **Reaching for a stream where a one-liner exists is a small red flag.**

Easy memory

```
joining(", ")           → "a, b, c"
joining(" ", "[", "]")  → "[a, b, c]"
String.join(" ", list)  → same thing, NO stream needed ★
```

```
Objects: .map(Employee::name).collect(joining(" | "))
```

20. Flatten a list of lists

The easiest way to remember:

```
map      → 1 element → 1 element
flatMap → 1 element → MANY elements, all merged into one stream ★
```

```
List<List<Integer>> nested = List.of(
    List.of(1, 2, 3),
    List.of(4, 5),
    List.of(6, 7, 8, 9)
);

List<Integer> flat = nested.stream()
    .flatMap(List::stream) // each inner List becomes a Stream,
all merged
    .collect(toList());

System.out.println(flat); // [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

The picture

```
[[1,2,3], [4,5], [6,7,8,9]]
  |
  .map(List::stream)      → Stream<Stream<Integer>>  ✗ still nested
  .flatMap(List::stream) → Stream<Integer>           ✓ flattened

[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Real-world idea: you have **three boxes of chocolates**. **map** gives you three boxes; **flatMap** **tips them all onto one table** so you can see every chocolate at once.

map vs flatMap — the question behind the question

	map	flatMap
Transforms	one value → one value	one value → a stream of values
On <code>List<List<Integer>></code>	<code>Stream<Stream<Integer></code> > (still nested)	<code>Stream<Integer></code> (flattened)
Think of it as	transform	transform + flatten

Real-world uses

```
// All words from a list of sentences
List<String> sentences = List.of("Java is fun", "Streams are powerful");
List<String> words = sentences.stream()
    .flatMap(s -> Arrays.stream(s.split(" ")))
    .collect(toList());
// [Java, is, fun, Streams, are, powerful]

// All skills across all employees, deduplicated and sorted
List<String> allSkills = employees.stream()
    .flatMap(e -> e.skills().stream())
    .distinct()
    .sorted()
    .collect(toList());

// Flatten a Map's values
Map<String, List<Employee>> byDept = ...;
List<Employee> everyone = byDept.values().stream()
    .flatMap(List::stream)
    .collect(toList());

// Unique characters across a list of words
List<Character> chars = words.stream()
    .flatMap(w -> w.chars().mapToObj(c -> (char) c))
    .distinct()
    .collect(toList());
```

Easy memory

map → 1 → 1 → Stream<Stream<T>> if the element is a list ✗
flatMap → 1 → MANY → Stream<T> ✓

flatMap(List::stream) ← flatten lists
flatMap(s → Arrays.stream(s.split(" "))) ← sentences → words

Three boxes of chocolates → tip them all onto ONE table

Bonus problems that often follow

Sum of even numbers:

```
int sum = nums.stream().filter(n -> n % 2 == 0).mapToInt(Integer::intValue).sum();
```

First 5 elements / skip the first 5:

```
nums.stream().limit(5).toList();  
nums.stream().skip(5).toList();
```

Check if a string is a palindrome:

```
String s = "madam";  
boolean isPalindrome = IntStream.range(0, s.length() / 2)  
    .allMatch(i -> s.charAt(i) == s.charAt(s.length() - 1 - i));
```

```
m a d a m  
↑       ↑   compare 0 and 4  
  ↑     ↑   compare 1 and 3  
    ↑     middle – no need to check ✓
```

Count words in a sentence:

```
long words = Arrays.stream(sentence.split("\\s+")).filter(w -> !  
w.isBlank()).count();
```

Squares of the first 10 numbers:

```
List<Integer> squares = IntStream.rangeClosed(1, 10).map(i -> i * i).boxed().toList();
```

```
range(1, 10)      → 1..9    (end EXCLUSIVE)
rangeClosed(1, 10) → 1..10  (end INCLUSIVE) ★
boxed()           → IntStream → Stream<Integer>
```

Employees with salary > 60000, names sorted:

```
List<String> names = employees.stream()
    .filter(e -> e.salary() > 60000)
    .map(Employee::name)
    .sorted()
    .toList();
```

Merge two lists and remove duplicates:

```
List<Integer> merged = Stream.concat(a.stream(),
    b.stream()).distinct().toList();
```

Find common elements (intersection):

```
List<Integer> common = a.stream().filter(b::contains).distinct().toList();
```

API quick reference

Need	Use
Transform each element	<code>map</code>
Transform + flatten	<code>flatMap</code>
Keep matching elements	<code>filter</code>
Remove duplicates	<code>distinct</code>
Sort	<code>sorted(Comparator...)</code>
Take / skip N	<code>limit(n)</code> / <code>skip(n)</code>

Need	Use
Count	<code>count()</code>
Sum / average	<code>mapToInt(...).sum()</code> / <code>.average()</code>
Max / min	<code>max(cmp)</code> / <code>min(cmp)</code>
All stats at once	<code>summaryStatistics()</code> ★
Group	<code>groupingBy(fn)</code>
Group + aggregate	<code>groupingBy(fn, counting())</code> ★
Split in two by predicate	<code>partitioningBy(pred)</code>
List → Map	<code>toMap(key, value, merge)</code>
Join strings	<code>joining(", ")</code>
Any / all / none match	<code>anyMatch</code> / <code>allMatch</code> / <code>noneMatch</code>
Get one element	<code>findFirst()</code> / <code>findAny()</code>
To List (immutable)	<code>.toList()</code> — Java 16+
To List (mutable)	<code>collect(toList())</code>
To Set	<code>collect(toSet())</code>
To a specific collection	<code>collect(toCollection(TreeSet::new))</code>
Combine into one value	<code>reduce(identity, accumulator)</code>
Peek at values (debug only)	<code>peek</code>

▲ The 12 traps to remember

1. A stream is SINGLE-USE → reusing it throws `IllegalStateException`
2. `toMap` throws on DUPLICATE KEYS → always consider a merge function
3. `toMap` throws NPE on a null VALUE → `groupingBy` doesn't
4. `stream().toList()` is IMMUTABLE → `collect(toList())` is mutable
5. `allMatch` on an EMPTY stream = TRUE → vacuous truth ★
6. `distinct()` on objects needs `equals()` + `hashCode()`
7. `sorted()` returns a NEW stream → `list.sort()` mutates in place
8. `findFirst` vs `findAny` only differ on PARALLEL streams
9. Don't mutate external state in a lambda → breaks with parallel streams
10. `average()` returns `OptionalDouble` → handle the empty case
11. Don't force a stream where a loop is clearer (`StringBuilder.reverse` beats `reduce`)
12. `parallelStream()` is NOT automatically faster → only for large + CPU-heavy work

■ How to answer these in the room

1. RESTATE the problem and confirm edge cases out loud
"Should this be case-sensitive? What if the list is empty?"
2. SAY YOUR APPROACH before you write
"I'll group by the character and count each group."
3. WRITE it, then TRACE one example through the pipeline
4. VOLUNTEER the complexity
"This is $O(n)$, one pass."
5. OFFER the alternative
"There's a non-stream version with a for loop and `map.merge`, if you'd prefer."

The most valuable habit: name a trade-off unprompted.

"I'd use *StringBuilder* here rather than *reduce*, because string concatenation in a reduce is $O(n^2)$."

That one sentence marks you as someone who has written production Java, not just practised puzzles.

Related files

- [05-java.md](#) — OOP, collections, HashMap internals, Java 8 basics
- [12-java17-features.md](#) — records, sealed classes, pattern matching
- [09-coding-problems.md](#) — 35+ general DSA problems in JS and Java

★ **Before the round:** the one-page recall version of this file is [42 — Java + Streams Final Memory Sheet](#) — all 32 problems, one line each, then the code for every one.

Java "Predict the Output" — 60 Tricky Programs with Answers

● **Why this file exists:** in your last interview you were asked **output-based Java programs**, not theory. Two of them are broken down in full at the top of this file (Q1 and Q2). Everything after that is the same *style* of question, grouped by the trick being tested.

How interviewers use these: they don't want the answer only — they want "why". For every program below, the format is:

1. **The code**
2. **The output** (or *compile error*)
3. **The rule** — the one sentence that makes it obvious
4. **The follow-up** they ask next

◆ **Golden habit:** before you answer any output question, ask yourself in this order —

(a) does it even compile? → (b) is anything resolved at *compile* time instead of runtime? → (c) then trace the runtime.

Most "tricky" Java questions are won at step (a) or (b). Candidates lose because they jump straight to (c).

Easy memory — the 6 rules that decide 90% of these programs

Before you trace a single line, run through these. Most "tricky" programs are decided by one of them.

1. OVERLOADING is decided at COMPILE time by the REFERENCE type
 OVERRIDING is decided at RUNTIME by the OBJECT type ★
 → methodTest(null) picks the MOST SPECIFIC type (String beats Object)
2. FIELDS and STATIC methods are NOT polymorphic – they use the REFERENCE type ★
 Only INSTANCE methods are polymorphic
 → Parent p = new Child(); p.field → PARENT's | p.method() → CHILD's
3. An OVERRIDE may not: widen `throws`, reduce visibility, or change the signature
4. String LITERALS are POOLED; `new String()` never is
 Compile-time constant folding ("hel" + "lo") is pooled;
 runtime concatenation is not ★
5. Integer CACHE is -128..127. Mixing a wrapper with a primitive UNBOXES.
 Unboxing a null → NullPointerException ★
6. `finally` ALWAYS runs, and a `return` inside it WINS
 (and silently swallows the exception) ★

The order to think in:

- (a) Does it even COMPILE? ← most questions are won here
 ↓ yes
- (b) Is anything resolved at COMPILE time rather than runtime?
 ↓ no
- (c) NOW trace the runtime.

Candidates lose marks by jumping straight to (c).

Table of contents

#	Section	What it tests
A	The 2 questions you were asked	Overloading with <code>null</code> , overriding + checked exceptions

#	Section	What it tests
B	Overloading traps (Q3–Q10)	Compile-time resolution, most-specific rule, widening vs boxing vs varargs
C	Overriding & polymorphism (Q11–Q19)	Static binding vs dynamic binding, field hiding, covariant returns
D	Initialization order (Q20–Q23)	static block → instance block → constructor
E	String traps (Q24–Q30)	String pool, <code>==</code> vs <code>equals</code> , immutability, <code>intern()</code>
F	Autoboxing & Integer cache (Q31–Q36)	<code>-128..127</code> cache, unboxing NPE, ternary promotion
G	Operators & numbers (Q37–Q44)	<code>i = i++</code> , integer division, float precision, compound assignment
H	try/catch/finally (Q45–Q50)	<code>finally</code> beating <code>return</code> , try-with-resources order
I	Collections (Q51–Q57)	<code>remove(int)</code> vs <code>remove(Object)</code> , CME, <code>equals</code> / <code>hashCode</code>
J	Static & threads (Q58–Q60)	Static on <code>null</code> , <code>run()</code> vs <code>start()</code>
K	Rapid-fire table	1-line answers for last-minute revision
L	20 self-test drills	Answers hidden — test yourself

A — The two questions you were actually asked

Q1. Method overloading with `null`

```
public class MethodOverloadingExample {  
  
    public void methodTest(Object object) {  
        System.out.println("Calling object method");  
    }  
  
    public void methodTest(String object) {  
        System.out.println("Calling String method");  
    }  
  
    public static void main(String args[]) {  
        MethodOverloadingExample moe = new MethodOverloadingExample();  
        moe.methodTest(null);  
    }  
}
```

✓ Output

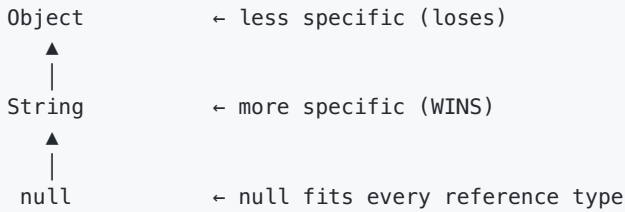
```
Calling String method
```

The rule (say this exact sentence)

"Method overloading is resolved at *compile* time, and the compiler picks the **most specific** applicable method. `null` is assignable to both `Object` and `String`, but `String` is a subtype of `Object`, so `String` is more specific — so `methodTest(String)` wins."

Why "most specific" means `String`

`String` **IS-A** `Object`. So anything you can pass to `methodTest(String)` you could also pass to `methodTest(Object)` — but not the other way round. When one candidate can be forwarded to the other, the compiler picks the one lower in the hierarchy.



▲ The 4 follow-ups they ask next

Follow-up 1 — "How do you force the **Object** version?"

```
moe.methodTest((Object) null);    // Calling object method
Object o = null;
moe.methodTest(o);                // Calling object method
```

The cast changes the **compile-time type** of the argument, which is the only thing overload resolution looks at.

Follow-up 2 — "What if the two overloads were **String** and **Integer** instead?"

```
public void methodTest(Integer i) { }
public void methodTest(String s) { }

moe.methodTest(null);    // ✗ COMPILE ERROR
// error: reference to methodTest is ambiguous
//      both method methodTest(Integer) and method methodTest(String)
//      match
```

String and **Integer** are siblings — neither is a subtype of the other, so there is no "most specific" one. Ambiguous → **compile error**, not a runtime error. Fix it with a cast: `moe.methodTest((String) null);`

Follow-up 3 — "And with three overloads: **Object**, **String**, **StringBuilder**?"

```
public void methodTest(Object o)      { }  
public void methodTest(String s)      { }  
public void methodTest(StringBuilder s) { }
```

```
moe.methodTest(null);    // ✗  COMPILE ERROR – ambiguous
```

`Object` is eliminated (both others beat it), but `String` vs `StringBuilder` are siblings → ambiguous.

Follow-up 4 — "Is `String.valueOf(null)` the same trap?"

```
System.out.println(String.valueOf(null));    // ✗  throws  
NullPointerException at runtime
```

Yes — the *same* rule bites you in the JDK. `String.valueOf` is overloaded for `Object`, `char[]`, `int`, `char[]` is more specific than `Object`, so the compiler picks `valueOf(char[])`, which does `new String(null)` → **NPE**.
`java System.out.println(String.valueOf((Object) null));` // prints "null" – safe

Dropping this line in the interview makes you look like you've actually hit the bug in real code.

Q2. Overriding + checked exception

```
class SuperClass {
    void method() {
        System.out.println("SuperClass");
    }
}

class SubClass extends SuperClass {
    void method() throws IOException {        // ▲ broader checked
exception
        System.out.println("SubClass");
    }

    public static void main(String args[]) {
        SuperClass s = new SubClass();
        s.method();
    }
}
```

✓ Output

✗ COMPILE ERROR

error: method() in SubClass cannot override method() in SuperClass
overridden method does not throw java.io.IOException

▲ Note there are also **two typos** planted in the original snippet — `newSubClass()` should be `new SubClass()`, and `IOException` needs `import java.io.IOException;`. But even after fixing both, **it still does not compile**, and that is the point of the question.

The rule (say this exact sentence)

"An overriding method may throw fewer or narrower checked exceptions than the method it overrides — never new or broader ones. It may throw any unchecked exception freely."

Why the language does this

Polymorphism must stay safe. The caller only sees the **compile-time type**:

```
SuperClass s = new SubClass();
s.method();           // compiler checks SuperClass.method() → declares NO
exceptions
                        // → caller writes no try/catch
                        // → but at runtime SubClass.method() could throw
IOException
                        // → a checked exception would escape completely
unhandled ▲
```

The compiler blocks this at the source: **you cannot widen the `throws` clause.**

The complete legal / illegal table — memorise this

Super declares: `void method() throws IOException`

Sub declares	Legal?	Why
<code>throws IOException</code>	✓	Same exception
<code>throws FileNotFoundException</code>	✓	Narrower (subclass of <code>IOException</code>)
<i>(no <code>throws</code> at all)</i>	✓	Fewer — always allowed
<code>throws IOException, SQLException</code>	✗	<code>SQLException</code> is new and checked
<code>throws Exception</code>	✗	Broader than <code>IOException</code>
<code>throws RuntimeException</code>	✓	Unchecked — never restricted
<code>throws Error / throws Throwable</code>	✓ / ✗	<code>Error</code> unchecked ✓ ; <code>Throwable</code> broader-checked ✗

Super declares: `void method()` *(nothing)*

Sub declares	Legal?
<i>(nothing)</i>	✓
throws IOException	✗ ← your question
throws NullPointerException	✓ (unchecked)
throws IllegalArgumentException	✓ (unchecked)

▲ The 3 follow-ups they ask next

Follow-up 1 — "Make it compile."

Three valid fixes; say the first one, then mention the other two:

```
// Fix A – remove the throws from the subclass (best: super's contract is
honoured)
class SubClass extends SuperClass {
    void method() { System.out.println("SubClass"); }
}

// Fix B – declare it in the SUPERCLASS too (widen the contract at the
top)
class SuperClass {
    void method() throws IOException { System.out.println("SuperClass"); }
}
// ...then main must handle it:
public static void main(String[] args) throws IOException {
    SuperClass s = new SubClass();
    s.method();                      // prints "SubClass"
}

// Fix C – throw an UNCHECKED exception instead
class SubClass extends SuperClass {
    void method() throws RuntimeException
{ System.out.println("SubClass"); }
}
```

Follow-up 2 — "If it did compile, what would print?"

```
SuperClass s = new SubClass();
s.method();      // "SubClass"
```

Overriding is resolved at runtime (dynamic dispatch) by the *object's actual type*, not the reference type. This is the exact opposite of Q1 — say that contrast out loud, it's the connection between the two questions.

Follow-up 3 — "Reverse it: super throws `IOException` , sub throws nothing. What does the caller do?"

```
class SuperClass { void method() throws IOException { } }
class SubClass extends SuperClass { void method() { } }

SuperClass s = new SubClass();
s.method();           // x compile error at the CALL SITE: unhandled
IOException
```

The caller is checked against the **reference type** `SuperClass` , which declares `throws IOException` — so `try/catch` (or `throws`) is still required even though the real object never throws.

◆ **The one slide that answers both questions**

	Q1 — Overloading	Q2 — Overriding
Also called	Compile-time / static polymorphism	Runtime / dynamic polymorphism
Decided by	The reference/declared type of the argument	The actual object on the heap
Decided when	Compile time	Run time
Signature rule	Parameters must differ	Signature must be identical (or covariant return)
<code>throws</code> rule	No restriction	Cannot widen checked exceptions
Access modifier	No restriction	Cannot reduce visibility
<code>static</code> / <code>private</code> / <code>final</code>	Can be overloaded	Cannot be overridden

Memorise this line: **"Overload = compiler decides by the reference type. Override = JVM decides by the object type."** It answers roughly half of all Java output questions.

B — Method overloading traps

Q3. Widening vs autoboxing vs varargs

```
public class Test {
    static void show(int x)      { System.out.println("int"); }
    static void show(long x)    { System.out.println("long"); }
    static void show(Integer x) { System.out.println("Integer"); }
    static void show(int... x)  { System.out.println("varargs"); }

    public static void main(String[] a) {
        show(5);
    }
}
```

✓ **Output:** `int`

The rule — the compiler tries three phases, in this order:

Phase 1: exact match / WIDENING	(int → long → float → double)
Phase 2: AUTOBOXING	(int → Integer → Object)
Phase 3: VARARGS	(int → int...)

"Widening beats boxing beats varargs." It stops at the first phase that finds a match.

Trace it: delete the methods one at a time and the answer walks down the list:

Methods present	Output
all four	<code>int</code> (exact)
remove <code>int</code>	<code>long</code> (widening)

Methods present	Output
remove <code>int</code> , <code>long</code>	<code>Integer</code> (boxing)
remove <code>int</code> , <code>long</code> , <code>Integer</code>	<code>varargs</code> (last resort)

Follow-up: "why is widening preferred over boxing?" — Backward compatibility. Varargs and autoboxing arrived in Java 5; code written before that had to keep behaving identically, so the old resolution rules run first.

Q4. `Integer` vs `long` — the trap in the trap

```
static void show(long x)    { System.out.println("long"); }
static void show(Integer x) { System.out.println("Integer"); }

show(5);
```

✓ **Output:** `long`

Widening (`int` → `long`) is **phase 1**; boxing (`int` → `Integer`) is **phase 2**. Phase 1 wins. Candidates almost always guess `Integer` because it "looks closer" — it isn't.

But reverse it:

```
static void show(long x)    { System.out.println("long"); }
static void show(Integer x) { System.out.println("Integer"); }

Integer i = 5;
show(i);           // "Integer" — an Integer cannot widen to long, only unbox
                    // then widen (2 steps, not allowed)
```

Q5. `char` in overloading

```
static void show(int x)    { System.out.println("int"); }
static void show(double x) { System.out.println("double"); }

show('a');
```

✓ **Output:** `int`

`char` widens along `char → int → long → float → double`. `int` is the **nearest** widening target, so it wins.

If you delete `show(int)`: output becomes `double` (still legal widening).

If only `show(Character)` and `show(int)` exist: `int` wins (widening beats boxing again).

Q6. Overloading is NOT decided by the object's real type

```
class Parent { }
class Child extends Parent { }

public class Test {
    static void show(Parent p) { System.out.println("Parent version"); }
    static void show(Child c) { System.out.println("Child version"); }

    public static void main(String[] a) {
        Parent p = new Child();    // reference type Parent, object type
        Child
        show(p);
    }
}
```

✓ **Output:** `Parent version`

The rule: overload resolution uses the **declared (compile-time) type** of the argument — `Parent`. The fact that the object is really a `Child` is irrelevant. *This is the single most important overloading fact, and it's the mirror image of Q1.*

```
show((Child) p);      // "Child version" – cast changes the compile-time
type
show(new Child());    // "Child version"
```

Q7. Ambiguous `null` — sibling types

```
static void show(String s)      { System.out.println("String"); }
static void show(StringBuilder b) { System.out.println("StringBuilder"); }

show(null);
```

✓ **Output:** ✗ **Compile error** — `reference to show is ambiguous`

Neither type is a subtype of the other → no "most specific" candidate. **Fix:**
`show((String) null);`

Q8. Two varargs methods

```
static void show(int... x)      { System.out.println("int varargs"); }
static void show(Object... x) { System.out.println("Object varargs"); }

show(1, 2);
```

✓ **Output:** `int varargs`

Within the varargs phase, the most specific applicable one still wins. `show()` with **no arguments** would be ambiguous, and `show(1, "a")` picks `Object...`.

Q9. Overloading on return type only

```
static int  show() { return 1; }
static long show() { return 1L; }
```

✓ **Output:** ✗ **Compile error** — `method show() is already defined`

The rule: return type is not part of the method signature. Overloading requires a different *parameter list* (number, types, or order). This also means you can't overload by changing only `throws` or only the access modifier.

Q10. Overloaded `println` and `char` arithmetic

```
char a = 'A';  
System.out.println(a);           // ?  
System.out.println(a + 1);       // ?  
System.out.println((char)(a + 1)); // ?  
System.out.println("" + a + 1);  // ?
```

✓ **Output:**

```
A  
66  
B  
A1
```

Line 1 picks `println(char)`. Line 2: `a + 1` promotes `char` to `int` → `66` → picks `println(int)`. Line 3: the cast forces `println(char)`. Line 4: string concatenation, no promotion.

C — Overriding & polymorphism traps

Q11. Static methods are hidden, not overridden

```
class Parent {
    static void show() { System.out.println("Parent static"); }
}
class Child extends Parent {
    static void show() { System.out.println("Child static"); }
}

public class Test {
    public static void main(String[] a) {
        Parent p = new Child();
        p.show();
    }
}
```

✓ **Output:** `Parent static`

The rule: static methods are bound at compile time by the reference type.

This is *method hiding*, not overriding — there is no dynamic dispatch for statics.

Adding `@Override` to `Child.show()` is a **compile error**.

Follow-up: "how do you prove it's not overriding?" — `p.show()` compiles down to a call on `Parent`. Also, the IDE warns "*static method should be accessed in a static way*" — you should really write `Parent.show()`.

Q12. Fields are hidden, never polymorphic

```
class Parent {
    String name = "Parent";
    String getName() { return name; }
}
class Child extends Parent {
    String name = "Child";
    String getName() { return name; }
}

public class Test {
    public static void main(String[] a) {
        Parent p = new Child();
        System.out.println(p.name);           // ?
        System.out.println(p.getName());      // ?
    }
}
```

✓ Output:

```
Parent
Child
```

The rule: Variables are resolved at compile time by the reference type; methods are resolved at runtime by the object type. A `Child` object physically contains *both* `name` fields; `p.name` reads the `Parent` one because `p` is declared as `Parent`.

This is a favourite because it puts both rules in one 6-line program.

Q13. Constructor calls an overridden method

```
class Parent {
    Parent() {
        System.out.println("Parent constructor");
        print();
    }
    void print() { System.out.println("Parent print"); }
}

class Child extends Parent {
    int x = 5;

    Child() {
        System.out.println("Child constructor");
    }

    @Override
    void print() { System.out.println("Child print, x = " + x); }
}

new Child();
```

✓ Output:

```
Parent constructor
Child print, x = 0      ← ▲ x is 0, not 5!
Child constructor
```

The rule: the `Parent` constructor runs **before** `Child`'s field initialisers. `print()` dispatches dynamically to `Child.print()`, but `x` still holds its **default value 0** because `int x = 5;` hasn't executed yet.

This is a real bug class — the reason the rule "*never call an overridable method from a constructor*" exists (Effective Java, Item 19). Mentioning that book by name lands well.

Fix: make `print()` `private`, `static`, or `final`, or move the call into a separate `init()` method the caller invokes after construction.

Q14. Covariant return types

```
class Parent {
    Object get() { return "parent"; }
}
class Child extends Parent {
    @Override
    String get() { return "child"; }           // narrower return type
}

Parent p = new Child();
System.out.println(p.get());
```

✓ **Output:** `child` — and it compiles.

The rule: since Java 5, an overriding method may return a **subtype** of the original return type (*covariant return*). `String` IS-A `Object`, so it's fine.

But the reverse fails:

```
class Child extends Parent {
    Number get() { return 1; }           // ✗ compile error – Number is not a
    subtype of String
}
```

And **primitives have no covariance:** `int` overriding `long` is a compile error.

Q15. Overriding cannot reduce visibility

```
class Parent {
    public void show() { System.out.println("Parent"); }
}
class Child extends Parent {
    protected void show() { System.out.println("Child"); }
}
```

✓ **Output:** ✗ **Compile error** — `attempting to assign weaker access privileges; was public`

The rule: an override may **widen** access (`protected` → `public`) but never **narrow** it. Otherwise `((Parent) child).show()` would call a method you're not allowed to call.

Legal direction:

```
class Parent { protected void show() { } }
class Child { public void show() { } } // ✓ widening is fine
```

Q16. `private` methods are not overridden

```
class Parent {
    private void show() { System.out.println("Parent"); }
    void call() { show(); }
}
class Child extends Parent {
    void show() { System.out.println("Child"); } // NOT an override –
    a brand new method
}

Parent p = new Child();
p.call();
```

✓ **Output:** `Parent`

The rule: `private` methods are **not inherited and not virtual** — `Parent.call()` is bound to `Parent.show()` at compile time. Putting `@Override` on `Child.show()` is a compile error.

Same for `final` (cannot be overridden at all) and `static` (hidden — see Q11).

Q17. `final` method

```
class Parent { final void show() { } }
class Child extends Parent { void show() { } }
```

✓ **Output:** ✗ **Compile error** — `show() in Child cannot override show() in Parent; overridden method is final`

Q18. Upcast / downcast and `ClassCastException`

```
class Parent { }  
class Child extends Parent { void childOnly() { } }  
  
Parent p = new Parent();  
Child c = (Child) p;           // compiles fine!
```

✓ **Output:** `Exception in thread "main" java.lang.ClassCastException: class Parent cannot be cast to class Child`

The rule: a **downcast compiles** (the compiler allows it because it *might* be valid) but is checked at runtime. Guard with `instanceof`:

```
if (p instanceof Child c) {           // Java 16+ pattern matching – you're on  
    Java 17 ✓  
    c.childOnly();  
}
```

Q19. Interface default methods — the diamond

```
interface A { default void show() { System.out.println("A"); } }  
interface B { default void show() { System.out.println("B"); } }  
  
class C implements A, B { }
```

✓ **Output:** ✗ **Compile error** — `class C inherits unrelated defaults for show() from types A and B`

Fix — you must override and disambiguate:

```

class C implements A, B {
    @Override
    public void show() {
        A.super.show();    // explicitly choose A → prints "A"
    }
}

```

`A.super.show()` is the special syntax for "call interface A's default method". Knowing that one token is a strong signal.

D — Initialization order traps

Q20. The full initialization order

```

class Parent {
    static { System.out.println("1. Parent static block"); }
    { System.out.println("3. Parent instance block"); }
    Parent() { System.out.println("4. Parent constructor"); }
}

class Child extends Parent {
    static { System.out.println("2. Child static block"); }
    { System.out.println("5. Child instance block"); }
    Child() { System.out.println("6. Child constructor"); }
}

public class Test {
    public static void main(String[] a) {
        new Child();
        System.out.println("----");
        new Child();
    }
}

```

✓ **Output:**

1. Parent static block
2. Child static block
3. Parent instance block
4. Parent constructor
5. Child instance block
6. Child constructor

-
3. Parent instance block
 4. Parent constructor
 5. Child instance block
 6. Child constructor

The rule — memorise this order:

ONCE per class, at class-load time:

1. Parent static blocks + static fields (top to bottom)
2. Child static blocks + static fields

EVERY time you call new:

3. Parent instance blocks + instance fields
4. Parent constructor body
5. Child instance blocks + instance fields
6. Child constructor body

Static blocks run only once — note they don't reappear for the second `new Child()`. **Instance blocks run before the constructor body**, and are copied into every constructor.

Q21. Instance block vs field initializer

```
class Test {  
    int x = printAndReturn("field initializer", 1);  
    { printAndReturn("instance block", 2); }  
    int y = printAndReturn("field after block", 3);  
  
    Test() { System.out.println("constructor"); }  
  
    static int printAndReturn(String s, int v) { System.out.println(s);  
    return v; }  
  
    public static void main(String[] a) { new Test(); }  
}
```

✓ Output:

```
field initializer  
instance block  
field after block  
constructor
```

The rule: field initialisers and instance blocks run **in the order they appear in the source**, all before the constructor **body**.

Q22. `this()` and `super()`

```
class Test {  
    Test()      { this(10); System.out.println("no-arg"); }  
    Test(int x) { System.out.println("int-arg " + x); }  
  
    public static void main(String[] a) { new Test(); }  
}
```

✓ Output:

```
int-arg 10  
no-arg
```

this() / **super()** must be the **FIRST** statement in a constructor, and you can't have both. If you write neither, the compiler inserts an implicit **super();** .

The classic compile error:

```
class Parent { Parent(int x) { } }           // no no-arg constructor!
class Child extends Parent { Child() { } } //
✗ error: constructor Parent in class Parent // cannot be applied to
given types
```

Q23. Static initializer and **final** constants

```
class Test {
    static final int A = 10;           // compile-time constant
    static int B = init();
    static { System.out.println("static block, B = " + B); }

    static int init() { System.out.println("init() called"); return 20; }

    public static void main(String[] a) { System.out.println("A = " + A +
        ", B = " + B); }
}
```

✓ Output:

```
init() called
static block, B = 20
A = 10, B = 20
```

static final int A = 10 is a **compile-time constant** — it's inlined at every use site and doesn't even trigger class loading. **B** needs a method call, so it initialises in source order, before the static block.

E — String traps

Q24. == vs equals and the string pool

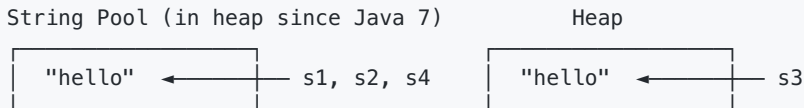
```
String s1 = "hello";
String s2 = "hello";
String s3 = new String("hello");
String s4 = s3.intern();

System.out.println(s1 == s2);           // ?
System.out.println(s1 == s3);           // ?
System.out.println(s1.equals(s3));       // ?
System.out.println(s1 == s4);           // ?
```

✓ Output:

```
true
false
true
true
```

The rule: string **literals** are interned in the **String Constant Pool** — identical literals share one object. `new String(...)` **always** allocates a fresh heap object. `intern()` returns the pooled instance.



Q25. Compile-time constant concatenation — the `final` twist

```
String s1 = "hello";
String s2 = "hel" + "lo";           // ?

String part = "hel";
String s3 = part + "lo";           // ?

final String fpart = "hel";
String s4 = fpart + "lo";          // ?

System.out.println(s1 == s2);
System.out.println(s1 == s3);
System.out.println(s1 == s4);
```

✓ Output:

```
true
false
true
```

The rule:

- `"hel" + "lo"` — both operands are literals → the **compiler folds it into `"hello"`** → pooled → `true`.
- `part + "lo"` — `part` is a variable, so concatenation happens **at runtime** (via `StringBuilder` / `invokedynamic`) → new heap object → `false`.
- `final String fpart = "hel"` — a **compile-time constant**, so the compiler folds it just like a literal → `true`.

That third case is the one that separates people. The magic word is **"compile-time constant expression"**.

Q26. Strings are immutable

```
String s = "hello";
s.toUpperCase();
s.concat(" world");
s.replace('h', 'H');
System.out.println(s);
```

✓ **Output:** `hello`

The rule: every `String` method **returns a new String** — it never mutates the receiver. You must reassign:

```
s = s.toUpperCase();    // HELLO
```

Q27. `StringBuilder` has no `equals`

```
StringBuilder a = new StringBuilder("abc");
StringBuilder b = new StringBuilder("abc");

System.out.println(a == b);                // ?
System.out.println(a.equals(b));            // ?
System.out.println(a.toString().equals(b.toString())); // ?
```

✓ **Output:**

```
false
false
true
```

`StringBuilder` does NOT override `equals()` — it inherits `Object.equals`, which is reference equality. Always compare via `.toString()` (or `compareTo`, added in Java 11).

Q28. `switch` on `String` and `null`

```
String s = null;
switch (s) {
    case "a": System.out.println("A"); break;
    default:  System.out.println("default");
}
```

✓ **Output:** `NullPointerException`

`switch` on a `String` internally calls `s.hashCode()`. **default** does not catch `null` — null-check before the switch. (Java 21's pattern-matching `switch` allows an explicit `case null` — but on Java 17 you must guard.)

Q29. String concatenation in `println`

```
System.out.println(1 + 2 + "3");      // ?
System.out.println("1" + 2 + 3);     // ?
System.out.println('a' + 'b');       // ?
System.out.println("" + 'a' + 'b');  // ?
System.out.println(1 + 2 + "" + 3 + 4); // ?
```

✓ **Output:**

```
33
123
195
ab
334
```

The rule: `+` is **left-associative**. Once one operand is a `String`, everything to the right becomes concatenation. Before that, it's arithmetic — and `char + char` promotes to `int` (`97 + 98 = 195`).

Q30. `substring` and `equals` / `hashCode` contract

```
String s = "hello world";  
System.out.println(s.substring(6));           // ?  
System.out.println(s.substring(0, 5));        // ?  
System.out.println(s.substring(5, 5));        // ?  
System.out.println(s.substring(12));          // ?
```

✓ **Output:**

```
world  
hello  
      ← empty string  
StringIndexOutOfBoundsException
```

`substring(begin, end)` is **begin-inclusive, end-exclusive**. `substring(5,5)` is legal and returns `""`. `substring(s.length())` is legal (returns `""`); anything beyond throws.

F — Autoboxing & Integer cache traps

Q31. The Integer cache — the #1 asked output question

```
Integer a = 127, b = 127;  
Integer c = 128, d = 128;  
int     e = 128;  
  
System.out.println(a == b);    // ?  
System.out.println(c == d);    // ?  
System.out.println(c == e);    // ?  
System.out.println(c.equals(d)); // ?
```

✓ **Output:**

```
true
false
true
true
```

The rule: `Integer.valueOf()` — which autoboxing calls — **caches objects for -128 to +127**. Within that range, boxing the same value returns the *same object*, so `==` is true. Outside it, each boxing creates a new object.

Line 3 is the twist: `c == e` compares `Integer` with `int`, so `c` is **UNBOXED** and it becomes a numeric comparison → `true`.

Cached types: `Boolean` (both), `Byte` (all), `Character` (0–127), `Short` & `Integer` & `Long` (-128..127). `Float` and `Double` are **never** cached.

Rule for real code: never use `==` on wrapper types. Always `.equals()` or unbox explicitly.

Q32. Unboxing `NullPointerException`

```
Map<String, Boolean> map = new HashMap<>();
boolean flag = map.get("missing");
```

✓ **Output:** `NullPointerException`

`map.get()` returns `null` (a `Boolean`), and assigning to `boolean` triggers **unboxing of null** → NPE. The stack trace points at a line with no visible method call, which is why it's confusing in production.

Same trap, harder:

```
Integer count = null;
if (count == 0) { }           // NPE – count is unboxed to compare with int 0
if (count == null) { }       // ✓ fine – comparing two references, no
                              unboxing
```


Q33. Ternary operator numeric promotion — the sneaky one

```
Map<String, Boolean> map = new HashMap<>();
Boolean b = (map != null) ? map.get("missing") : false;
System.out.println(b);
```

✓ **Output:** `NullPointerException` — even though the target is a `Boolean` !

The rule: the two branches of the ternary are `Boolean` and `boolean`. The compiler applies **binary numeric promotion**, making the *whole expression's type* `boolean` — so `map.get("missing")` gets **unboxed** → NPE, before it can ever be re-boxed into `b`.

Fix: make both branches the same reference type: `: Boolean.FALSE`.

Q34. Ternary with `Integer` and `Double`

```
Object o = true ? new Integer(1) : new Double(2.0);
System.out.println(o);
```

✓ **Output:** `1.0` ← not `1` !

Same rule as Q33. `Integer` and `Double` both unbox, binary numeric promotion makes the expression type `double`, so `1` becomes `1.0` and is re-boxed as a `Double`. The `false` branch never even runs, yet it changed the result's type.

Q35. `equals` across wrapper types

```
Integer i = 1;
Long    l = 1L;
Short   s = 1;

System.out.println(i.equals(l));    // ?
System.out.println(i.equals(s));    // ?
System.out.println(i.equals(1));    // ?
System.out.println(l.equals(1));    // ?
```

✓ **Output:**

```
false
false
true
false
```

The rule: `Integer.equals` starts with `if (obj instanceof Integer)` — a `Long` or `Short` fails the type check immediately, **regardless of numeric value**. Line 4 is the classic bug: the literal `1` boxes to `Integer`, so `Long(1).equals(Integer(1))` is `false`. Write `l.equals(1L)`.

Q36. The `Set<Short>` remove trap

```
Set<Short> set = new HashSet<>();
for (short i = 0; i < 100; i++) {
    set.add(i);
    set.remove(i - 1);
}
System.out.println(set.size());
```

✓ **Output:** `100` ← not `1`!

The rule: `i - 1` promotes `short` to `int`, so it boxes to an `Integer`, and `remove(Object)` looks for an `Integer` in a set full of `Short`s — never matching. Nothing is ever removed.

Fix: `set.remove((short)(i - 1));`

G — Operator & numeric traps

Q37. `i = i++`

```
int i = 0;
i = i++;
System.out.println(i);    // ?

int j = 0;
j = ++j;
System.out.println(j);    // ?

int k = 0;
k = k++ + ++k;
System.out.println(k);    // ?
```

✓ Output:

```
0
1
2
```

The rule for `i = i++`: Java evaluates in this order —

1. Read `i`'s current value (0) → save it as the result of `i++`
2. Increment `i` → `i` is now 1
3. ASSIGN the saved value (0) back to `i` → `i` is 0 again ✗

The assignment **overwrites** the increment. `++i` (pre-increment) returns the *new* value, so `j = ++j` gives 1.

For `k = k++ + ++k`: `k++` yields 0 ($k \rightarrow 1$), `++k` yields 2 ($k \rightarrow 2$), so `0 + 2 = 2` is assigned.

Q38. Integer division and modulo with negatives

```
System.out.println(10 / 3);      // ?
System.out.println(10 % 3);      // ?
System.out.println(-10 / 3);     // ?
System.out.println(-10 % 3);     // ?
System.out.println(10 / 3.0);    // ?
```

✓ Output:

```
3
1
-3
-1
3.3333333333333335
```

Integer division **truncates toward zero** (not floor). The sign of `%` follows the **dividend**, so `-10 % 3` is `-1`, not `2`. (Use `Math.floorMod(-10, 3)` → `2` if you want the mathematical modulo.)

Q39. Floating-point precision

```
System.out.println(0.1 + 0.2);      // ?
System.out.println(0.1 + 0.2 == 0.3); // ?
System.out.println(1.03 - 0.42);    // ?
```

✓ Output:

```
0.30000000000000004
false
0.6100000000000001
```

The rule: `double` / `float` are IEEE-754 **binary** fractions — `0.1` has no exact binary representation. **Never use `double` for money.** Use `BigDecimal` (with the `String` constructor):

```
new BigDecimal("0.1").add(new BigDecimal("0.2")); // exactly 0.3
new BigDecimal(0.1)                               //
▲ WRONG – captures the binary error
```

For comparisons, use an epsilon: `Math.abs(a - b) < 1e-9`.

Q40. Division by zero — int vs double

```
System.out.println(1 / 0);           // ?
System.out.println(1.0 / 0);        // ?
System.out.println(-1.0 / 0);       // ?
System.out.println(0.0 / 0.0);      // ?
System.out.println(1 % 0);          // ?
```

✓ Output:

```
ArithmeticException: / by zero
Infinity
-Infinity
NaN
ArithmeticException: / by zero
```

Integer division by zero throws; **floating-point** division by zero returns `Infinity` / `NaN` and never throws.

Q41. NaN comparisons

```
double nan = 0.0 / 0.0;
System.out.println(nan == nan);           // ?
System.out.println(Double.valueOf(nan).equals(nan)); // ?
System.out.println(Double.isNaN(nan));     // ?
```

✓ Output:

```
false
true
true
```

NaN != NaN by IEEE-754 rule — but `Double.equals` compares the raw bits, so it says `true`. This inconsistency is why `Double` behaves oddly as a `HashMap` key. Always test with `Double.isNaN()`.

Q42. Compound assignment hides a cast

```
byte b = 10;
b = b + 5;      // ?
b += 5;         // ?
```

✓ Output:

- `b = b + 5;` → ✗ **compile error:** `incompatible types: possible lossy conversion from int to byte`
- `b += 5;` → ✓ compiles, `b` becomes `15`

The rule: `b + 5` promotes to `int`, and `int` won't fit into `byte` without a cast. But **compound assignment operators** (`+=`, `-=`, `*=`, ...) **contain an implicit cast** — `b += 5` is really `b = (byte)(b + 5)`.

The dangerous consequence:

```
byte b = 127;
b += 1;
System.out.println(b);      // -128 ← silent overflow, no warning!
```

Q43. Integer overflow

```
System.out.println(Integer.MAX_VALUE + 1);           // ?
System.out.println(Math.abs(Integer.MIN_VALUE));    // ?
int seconds = 24 * 60 * 60 * 1000 * 1000;
System.out.println(seconds);                         // ?
```

✓ Output:

```
-2147483648
-2147483648
500654080
```

Java integers **wrap around silently**. `Math.abs(Integer.MIN_VALUE)` is negative because `+2147483648` doesn't exist in `int`. The third line is the famous microseconds-per-day bug — fix with `24L * 60 * 60 * 1000 * 1000`.

Java 8+ gives you `Math.addExact()` / `multiplyExact()`, which throw `ArithmeticException` on overflow instead of wrapping.

Q44. Short-circuit vs non-short-circuit

```
int i = 0;
if (false && (i++ > 0)) { }
System.out.println(i);           // ?

int j = 0;
if (false & (j++ > 0)) { }
System.out.println(j);           // ?

String s = null;
if (s != null && s.length() > 0) { } // ?
if (s != null & s.length() > 0) { }  // ?
```

✓ Output:

```
0
1
(no exception)
NullPointerException
```

`&&` and `||` **short-circuit** — the right side isn't evaluated if the result is already known. `&` and `|` **always evaluate both sides**. That's why `s != null && s.length() > 0` is safe and `&` is not.

H — try/catch/finally traps

Q45. `finally` overrides `return`

```
static int test() {
    try {
        return 1;
    } finally {
        return 2;
    }
}
System.out.println(test());
```

✓ **Output:** 2

The rule: a `return` inside `finally` **discards** the `try` block's return value — and **swallows any pending exception too**:

```
static int test2() {
    try {
        throw new RuntimeException("boom");
    } finally {
        return 2;           // exception silently disappears!
    }
}
test2();           // returns 2, no exception
```


Say: "Never **return** or **throw** from a **finally** block — most static analysers (SonarQube, SpotBugs) flag it as a bug."

Q46. **finally** mutating the return value

```
static int test() {  
    int x = 1;  
    try {  
        return x;  
    } finally {  
        x = 2;  
        System.out.println("finally ran, x = " + x);  
    }  
}  
System.out.println(test());
```

✓ Output:

```
finally ran, x = 2  
1
```

The rule: **return x** evaluates and saves **x** (value **1**) onto the stack *before* **finally** runs. Reassigning **x** afterwards changes the variable, not the already-captured return value.

▲ But with a mutable object it's different:

```
java static List<String> test() { List<String> list = new  
ArrayList<>(); try { return list; } finally  
{ list.add("added"); } // mutates the same object the caller gets }  
test(); // ["added"] ← the mutation IS visible
```

The *reference* is captured, but the object it points to is still mutable.

Q47. Execution order of try / catch / finally

```
try {
    System.out.println("try");
    throw new RuntimeException("x");
} catch (RuntimeException e) {
    System.out.println("catch");
} finally {
    System.out.println("finally");
}
System.out.println("after");
```

✓ Output:

```
try
catch
finally
after
```

When does `finally` NOT run? Only three cases — worth naming all three:

1. `System.exit(0)` inside the `try`
2. The JVM crashes / is killed
3. The thread is killed or enters an infinite loop / deadlock

```
try { System.exit(0); } finally { System.out.println("never printed"); }
```

Q48. Catch order — subclass must come first

```
try {
    throw new FileNotFoundException();
} catch (IOException e) {
    System.out.println("IOException");
} catch (FileNotFoundException e) {
    System.out.println("FileNotFoundException");
}
```

✓ **Output:** ✗ **Compile error** — `exception FileNotFoundException has already been caught`

The rule: catch blocks are tested **top to bottom**, so a **subclass must be caught before its superclass** — otherwise the later block is unreachable, and Java makes unreachable catch a compile error.

Correct order → output **FileNotFoundException** :

```
catch (FileNotFoundException e) { ... } // most specific FIRST
catch (IOException e)           { ... }
catch (Exception e)             { ... } // most general LAST
```

Q49. try-with-resources — closing order

```
class Resource implements AutoCloseable {
    private final String name;
    Resource(String name) { this.name = name; System.out.println("open " +
name); }
    public void close() { System.out.println("close " + name); }
}

try (Resource a = new Resource("A"); Resource b = new Resource("B")) {
    System.out.println("body");
} finally {
    System.out.println("finally");
}
```

✓ **Output:**

```
open A
open B
body
close B          ← REVERSE order of declaration
close A
finally          ← resources close BEFORE finally
```

The rule: resources are closed in the **reverse order** they were declared (like a stack), and **before** any **catch** or **finally** block runs.

Q50. `throw` in both `try` and `finally` — suppressed exceptions

```
try (Resource r = new ResourceThatThrowsOnClose()) {
    throw new RuntimeException("from body");
} catch (Exception e) {
    System.out.println(e.getMessage());
    System.out.println(Arrays.toString(e.getSuppressed()));
}
```

✓ Output:

```
from body
[java.lang.RuntimeException: from close]
```

The rule: with try-with-resources, if both the body and `close()` throw, the **body's exception wins** and the close exception is attached as a **suppressed exception** (`getSuppressed()`). With an old-style `finally`, the second exception would simply **destroy** the first — which is exactly the bug try-with-resources was introduced to fix.

I — Collections traps

Q51. `remove(int)` vs `remove(Object)` — the classic

```
List<Integer> list = new ArrayList<>(Arrays.asList(10, 20, 30));
list.remove(1);
System.out.println(list);                                // ?

List<Integer> list2 = new ArrayList<>(Arrays.asList(10, 20, 30));
list2.remove(Integer.valueOf(20));
System.out.println(list2);                                // ?
```

✓ Output:

```
[10, 30]
[10, 30]
```

...but for completely different reasons.

The rule: `List` has two `remove` methods — `remove(int index)` and `remove(Object o)`. With a `List<Integer>`, the literal `1` is an `int`, so `remove(int)` wins (no boxing needed — remember Q3: exact match beats boxing) and it removes index `1` (the value `20`). To remove *by value*, box explicitly: `remove(Integer.valueOf(20))` or `remove((Integer) 20)`.

This is Q1's rule showing up in the JDK's own API. Connect the two out loud — it shows you understand the principle rather than memorising trivia.

Q52. `Arrays.asList` is fixed-size

```
List<String> list = Arrays.asList("a", "b", "c");
list.set(0, "z");           // ?
list.add("d");              // ?
list.remove("a");           // ?
```

✓ **Output:**

- `set(0, "z")` → ✓ works, list becomes `[z, b, c]`
- `add("d")` → `UnsupportedOperationException`
- `remove("a")` → `UnsupportedOperationException`

The rule: `Arrays.asList` returns a **fixed-size view backed by the array** — you can *replace* elements but not *resize*. `List.of(...)` (Java 9+) is **fully immutable** — even `set` throws.

	set	add / remove	null allowed
<code>Arrays.asList(..)</code>	✓	✗ UOE	✓
<code>List.of(...)</code>	✗ UOE	✗ UOE	✗ NPE
<code>new ArrayList<>(Arrays.asList(...))</code>	✓	✓	✓

Q53. `ConcurrentModificationException`

```
List<String> list = new ArrayList<>(List.of("a", "b", "c"));
for (String s : list) {
    if (s.equals("b")) list.remove(s);
}
System.out.println(list);
```

✓ **Output:** `ConcurrentModificationException`

The rule: the for-each loop uses an `Iterator` that tracks a `modCount`. Structurally modifying the list behind the iterator's back makes `next()` throw — **fail-fast** behaviour.

Three correct fixes:

```
list.removeIf(s -> s.equals("b")); // ✓ best, Java 8+

Iterator<String> it = list.iterator(); // ✓ classic
while (it.hasNext()) { if (it.next().equals("b")) it.remove(); }

for (int i = list.size() - 1; i >= 0; i--) { ... } // ✓ backwards index
loop
```

The nastiest variant: removing the **second-to-last** element does *not* throw — the `hasNext()` check (`cursor != size`) happens to become true and the loop exits early with no exception. Undefined-ish behaviour, which is why CME is documented as *best-effort*.

```
java List<String> l = new ArrayList<>(List.of("a","b","c")); for
(String s : l) { if (s.equals("b")) l.remove(s); } // NO exception,
prints [a, c]
```

Q54. `HashSet` without `equals` / `hashCode`

```
class Person {
    String name;
    Person(String name) { this.name = name; }
}

Set<Person> set = new HashSet<>();
set.add(new Person("Priya"));
set.add(new Person("Priya"));
System.out.println(set.size());
```

✓ **Output:** 2

The rule: `HashSet` deduplicates using `hashCode()` and `equals()`. Without overriding them, `Object`'s identity-based versions are used, so two distinct objects are always "different".

Fix (Java 17 — a `record` gives you both for free):

```
record Person(String name) { }    // auto-generates equals, hashCode,
toString → size() = 1
```

The contract: *equal objects must have equal hash codes.* Overriding only `equals` and not `hashCode` is the classic bug — the set puts them in different buckets and `equals` is never even called.

Q55. Mutating a key after inserting it

```
Map<List<String>, String> map = new HashMap<>();
List<String> key = new ArrayList<>(List.of("a"));
map.put(key, "value");

key.add("b");                                // mutate the key AFTER
insertion

System.out.println(map.get(key));             // ?
System.out.println(map.size());              // ?
System.out.println(map.containsKey(key));    // ?
```

✓ Output:

```
null
1
false
```

The rule: the entry was stored in the bucket for the **old** hash. Changing the key changes its `hashCode()`, so lookups now search the wrong bucket — the entry is **stranded**: still counted in `size()`, but unreachable.

This is the argument for **immutable keys**. `String` is the ideal `HashMap` key precisely because it's immutable and caches its hash.

Q56. `HashMap` vs `TreeMap` vs `LinkedHashMap` — null and ordering

```
Map<String, Integer> hm = new HashMap<>();
hm.put("b", 2); hm.put("a", 1); hm.put(null, 0);
System.out.println(hm);                                // ?

Map<String, Integer> tm = new TreeMap<>();
tm.put("b", 2); tm.put("a", 1);
System.out.println(tm);                                // ?
tm.put(null, 0);                                        // ?

Map<String, Integer> lhm = new LinkedHashMap<>();
lhm.put("b", 2); lhm.put("a", 1);
System.out.println(lhm);                                // ?
```

✓ Output:

<code>{null=0, a=1, b=2}</code>	← <code>HashMap</code> order is NOT guaranteed (this is just how these hash)
<code>{a=1, b=2}</code>	← <code>TreeMap</code> : sorted by key
<code>NullPointerException</code> <code>compare null</code>	← <code>TreeMap</code> must compare the key, and you can't compare null
<code>{b=2, a=1}</code>	← <code>LinkedHashMap</code> : insertion order

	Order	Null key	Null values
HashMap	none (don't rely on it)	✓ one	✓
LinkedHashMap	insertion (or access)	✓ one	✓
TreeMap	sorted	✗ NPE	✓
Hashtable	none	✗ NPE	✗ NPE

▲ **Never claim "HashMap prints in random order"** — it's *unspecified*, and small String keys often look sorted by accident. Say **"no ordering guarantee"**.

Q57. Array covariance — `ArrayStoreException`

```
Object[] objects = new String[3];
objects[0] = "hello";           // fine
objects[1] = 42;                // ?
```

✓ **Output:** `ArrayStoreException: java.lang.Integer`

The rule: arrays are covariant (`String[]` IS-A `Object[]`) but **generics are not** (`List<String>` is NOT a `List<Object>`). Array covariance is unsound, so the JVM has to check every store at runtime.

```
List<Object> list = new ArrayList<String>(); // ✗ compile error –
generics caught it at COMPILE time
```

Generics chose compile-time safety; arrays chose runtime checks. That contrast is a strong closing line.

J — Static & thread traps

Q58. Static method on a `null` reference

```
class Test {
    static void hello() { System.out.println("hello"); }

    public static void main(String[] a) {
        Test t = null;
        t.hello();
    }
}
```

✓ **Output:** `hello` — no `NullPointerException`

The rule: a static method is resolved from the **compile-time type of the reference**, not the object. The compiler rewrites `t.hello()` into `Test.hello()` and the null value is never dereferenced.

Same story for a static field: `t.staticField` works on a null reference. (An *instance* field or method would throw NPE.)

Q59. `run()` vs `start()`

```
public class Test {
    public static void main(String[] a) {
        Thread t = new Thread() ->
            System.out.println("running in " +
                Thread.currentThread().getName());

        t.run();    // ?
        t.start();  // ?
    }
}
```

✓ **Output:**

running in main	← run() is just an ordinary method call!
running in Thread-0	← start() creates a new OS thread

The rule: `start()` creates a new thread and the JVM calls `run()` on it.

Calling `run()` yourself executes it on the current thread — no concurrency at all.

Follow-up: "what if you call `start()` twice?"

```
t.start();
t.start();    // IllegalThreadStateException – a Thread can never be
restarted
```

Q60. `static` counter shared across instances

```
class Counter {
    static int staticCount = 0;
    int instanceCount = 0;

    Counter() { staticCount++; instanceCount++; }
}

new Counter(); new Counter(); Counter c = new Counter();
System.out.println(c.staticCount + " " + c.instanceCount);
System.out.println(Counter.staticCount);
```

✓ **Output:**

```
3 1
3
```

One `staticCount` exists **per class**; one `instanceCount` exists **per object**. This is the simplest way to demonstrate the difference, and it's a common warm-up question.

K — Rapid-fire answer table

Cover the right column and go down the list. Target: **under 4 minutes**, no hesitation.

Program	Output	The one-line reason
<code>methodTest(null)</code> with <code>Object / String</code>	<code>String</code> version	Most specific type wins
<code>methodTest(null)</code> with <code>String / Integer</code>	Compile error	Ambiguous — siblings
Sub overrides with broader checked exception	Compile error	Can't widen <code>throws</code>
Sub overrides with <code>RuntimeException</code>	Compiles	Unchecked is unrestricted
Sub overrides with <code>protected</code> (super <code>public</code>)	Compile error	Can't reduce visibility
<code>Parent p = new Child();</code> <code>p.staticMethod()</code>	Parent's	Static = hidden , compile-time bound
<code>Parent p = new Child();</code> <code>p.field</code>	Parent's	Fields are not polymorphic
<code>Parent p = new Child();</code> <code>p.method()</code>	Child's	Methods are polymorphic
Constructor calls overridden method	Child's method, fields = 0	Super ctor runs before child fields
<code>show(5)</code> with <code>int / long / Integer / int...</code>	<code>int</code>	Widening > boxing > varargs
<code>s1 == s2</code> for two <code>"hello"</code> literals	<code>true</code>	String pool
<code>"hello" == new String("hello")</code>	<code>false</code>	<code>new</code> always allocates

Program	Output	The one-line reason
<code>"hel" + "lo" == "hello"</code>	<code>true</code>	Compile-time constant folding
<code>part + "lo" == "hello"</code> (non-final var)	<code>false</code>	Runtime concatenation
<code>Integer 127 == 127 / 128 == 128</code>	<code>true</code> / <code>false</code>	Cache is -128..127
<code>Integer c = 128; int e = 128; c == e</code>	<code>true</code>	Mixing wrapper+primitive → unboxes
<code>boolean b = map.get(missing)</code>	NPE	Unboxing <code>null</code>
<code>x ? map.get(k) : false</code> (Boolean/boolean)	NPE	Ternary promotion unboxes
<code>true ? Integer : Double</code>	<code>1.0</code>	Ternary numeric promotion
<code>i = i++</code>	<code>0</code>	Assignment overwrites the increment
<code>0.1 + 0.2</code>	<code>0.30000000000000004</code>	IEEE-754 binary fractions
<code>1/0</code> vs <code>1.0/0</code>	Exception vs <code>Infinity</code>	int throws, double doesn't
<code>NaN == NaN</code>	<code>false</code>	IEEE-754 rule
<code>byte b=10; b = b+5;</code>	Compile error	Promotes to <code>int</code>
<code>byte b=10; b += 5;</code>	<code>15</code>	<code>+=</code> has an implicit cast
<code>try{return 1;} finally{return 2;}</code>	<code>2</code>	<code>finally</code> 's return wins
<code>try{return x;} finally{x=2;}</code>	<code>1</code>	Return value captured first
<code>catch(IOException)</code> before <code>catch(FNFE)</code>	Compile error	Subclass must come first
try-with-resources close order	Reverse of declaration	Stack (LIFO), before <code>finally</code>

Program	Output	The one-line reason
<code>list.remove(1)</code> on <code>List<Integer></code>	removes index 1	<code>remove(int)</code> beats <code>remove(Object)</code>
<code>Arrays.asList(...).add(x)</code>	UOE	Fixed-size array view
Remove during for-each	CME	Fail-fast iterator (<code>modCount</code>)
<code>HashSet</code> of 2 equal objects, no <code>equals</code>	size 2	Identity <code>equals</code> / <code>hashCode</code>
<code>TreeMap.put(null, v)</code>	NPE	Must compare the key
<code>Object[] o = new String[3]; o[1] = 42;</code>	ArrayStoreException	Arrays are covariant
<code>nullRef.staticMethod()</code>	Runs fine	Statics don't dereference
<code>t.run()</code> vs <code>t.start()</code>	<code>main</code> vs <code>Thread-0</code>	<code>run()</code> is a plain method call
<code>t.start()</code> twice	IllegalThreadStateException	Threads can't restart
<code>String.valueOf(null)</code>	NPE	Picks <code>valueOf(char[])</code>
<code>System.exit(0)</code> inside <code>try</code>	<code>finally</code> skipped	JVM dies

L — 20 self-test drills (answers hidden)

Write your answer down **before** expanding. Aim for 17/20.

Drill 1

```
public class T {  
    void f(Object o) { System.out.println("Object"); }  
    void f(String s) { System.out.println("String"); }  
    void f(Integer i){ System.out.println("Integer"); }  
  
    public static void main(String[] a) {  
        new T().f(null);  
    }  
}
```

→ Answer

****Compile error — ambiguous.**** `Object` is eliminated, but `String` and `Integer` are siblings with no most-specific candidate. *(If `Integer` were removed, the answer would be `String`.)*

Drill 2

```
class A { void f() throws Exception { System.out.println("A"); } }  
class B extends A { void f() { System.out.println("B"); } }  
  
public static void main(String[] args) {  
    A a = new B();  
    a.f();  
}
```

→ Answer

****Compile error at the call site**** — `unreported exception Exception; must be caught or declared to be thrown`. The override (removing `throws`) is perfectly legal, but the **caller** is checked against the ****reference type**** `A`, which declares `throws Exception`. Fix: wrap in `try/catch`, or add `throws Exception` to `main`. Then it prints ****`B`****.

Drill 3

```
class A { A() { print(); } void print() { System.out.println("A"); } }  
class B extends A {  
    String s = "hello";  
    void print() { System.out.println("B: " + s.length()); }  
}  
new B();
```

→ Answer

NullPointerException. **`A`'s constructor calls the overridden `B.print()` before `s = "hello"` runs, so `s` is still `null` and `s.length()` throws. *(Compare Q13 where the field was an `int` and quietly gave `0` — with an object it blows up.)*

Drill 4

```
Integer a = 1000, b = 1000;  
System.out.println(a == b);  
System.out.println(a.equals(b));  
System.out.println(a.intValue() == b.intValue());
```

→ Answer

```
false  
true  
true
```

1000 is outside the `-128..127` cache → two distinct objects.

Drill 5

```
String a = "java";  
String b = "ja";  
String c = b + "va";  
final String d = "ja";  
String e = d + "va";  
System.out.println((a == c) + " " + (a == e));
```

→ Answer

``false true`` — ``b`` is a normal variable (runtime concatenation), ``d`` is ``final`` and initialised with a literal, making it a **``compile-time constant``** the compiler folds into ``"java"``.

Drill 6

```
static int f() {  
    int x = 0;  
    try { x = 1; return x; }  
    catch (Exception e) { x = 2; return x; }  
    finally { x = 3; System.out.println("finally x=" + x); }  
}  
System.out.println(f());
```

→ Answer

```
finally x=3  
1
```

``return x`` captured the value ``1`` before ``finally`` ran.

Drill 7

```
List<Integer> l = new ArrayList<>(List.of(1, 2, 3));  
l.remove(2);  
System.out.println(l);
```

→ Answer

``[1, 2]`` — ``remove(int index)`` wins over ``remove(Object)``, so index **``2``** (value ``3``) is removed. ``l.remove(Integer.valueOf(2))`` would give ``[1, 3]``.

Drill 8

```
System.out.println(Math.min(Double.MIN_VALUE, 0.0d));
```

→ Answer

``0.0`` — ``Double.MIN_VALUE`` is the smallest **``positive``** value (`4.9E-324``), **``not``** the most negative one. For that you'd want ``-Double.MAX_VALUE``. **``(By contrast**

`Integer.MIN\_VALUE` really is the most negative int — the inconsistency is the trap.)*

Drill 9

```
class A { static void f() { System.out.println("A.f"); } }  
class B extends A { static void f() { System.out.println("B.f"); } }  
  
A a = new B();  
a.f();  
((B) a).f();  
B.f();
```

→ Answer

```
A.f  
B.f  
B.f
```

Static binding follows the **reference type** every time.

Drill 10

```
System.out.println('a' + 'b' + "c");  
System.out.println("c" + 'a' + 'b');
```

→ Answer

```
195c  
cab
```

Left-associative: ``a'+b`` is int arithmetic (195) before the String appears; in the second line the String comes first, so everything is concatenation.

Drill 11

```
Set<String> set = new HashSet<>();
set.add("a"); set.add("b"); set.add("a");
System.out.println(set.size());

List<String> list = new ArrayList<>();
list.add("a"); list.add("b"); list.add("a");
System.out.println(list.size());
System.out.println(list.indexOf("a") + " " + list.lastIndexOf("a"));
```

→ Answer

```
2
3
0 2
```

`Set` deduplicates (String has proper `equals`/`hashCode`); `List` allows duplicates.

Drill 12

```
public class T {
    static int x = getX();
    static { System.out.println("static block"); }
    static int getX() { System.out.println("getX"); return 5; }
    public static void main(String[] a) { System.out.println("main, x=" +
x); }
}
```

→ Answer

```
getX
static block
main, x=5
```

Static fields and static blocks run ****in source order**** at class load, before `main`.

Drill 13

```
int[] arr = new int[3];
String[] strs = new String[3];
boolean[] flags = new boolean[3];
System.out.println(arr[0] + " " + strs[0] + " " + flags[0]);
System.out.println(arr[3]);
```

→ Answer

```
0 null false
ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
```

Array elements get **default values** (0, `null`, `false`) — unlike local variables, which must be explicitly initialised or the code won't compile.

Drill 14

```
Object o = "hello";
if (o instanceof Integer) System.out.println("Integer");
else if (o instanceof String s) System.out.println("String of length " +
s.length());
```

→ Answer

String of length 5 — Java 16+ **pattern matching for `instanceof`** binds `s` automatically, no cast needed. You're on **Java 17**, so use this in interviews ([18 — Technical Versions](./18-robogebra-technical-versions.md)).

Drill 15

```
StringBuilder sb = new StringBuilder("abc");
modify(sb);
System.out.println(sb);

static void modify(StringBuilder s) {
    s.append("def");
    s = new StringBuilder("xyz");
    s.append("!");
}
```

→ Answer

``abcdef`` — Java is **pass-by-value of the reference**. `append` mutates the shared object (visible to the caller), but reassigning `s` only changes the local copy of the reference (invisible to the caller). *This is the answer to "is Java pass-by-value or pass-by-reference?" — always **pass-by-value**.*

Drill 16

```
public class T {
    public static void main(String[] args) {
        System.out.println(args.length);
        System.out.println(args[0]);
    }
}
// run as: java T
```

→ Answer

```
0
ArrayIndexOutOfBoundsException
```

With no command-line arguments, `args` is an **empty array**, not `null`. (In C it would include the program name; in Java it does not.)

Drill 17

```
interface Shape { default String name() { return "shape"; } }
interface Square extends Shape { default String name() { return
"square"; } }
class MySquare implements Shape, Square { }

System.out.println(new MySquare().name());
```

→ Answer

``square`` — no ambiguity error here because `Square` **extends** `Shape`, so it's the more specific interface and wins. *(Compare Q19, where `A` and `B` were unrelated → compile error.)*

Drill 18

```
try {
    int[] arr = new int[2];
    arr[5] = 10;
    System.out.println("A");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("B");
} catch (Exception e) {
    System.out.println("C");
} finally {
    System.out.println("D");
}
System.out.println("E");
```

→ Answer

B
D
E

Only the **first matching** catch block runs — `catch` is not a fall-through switch.

Drill 19

```
class A {
    void f(int x) { System.out.println("A.int"); }
}
class B extends A {
    void f(long x) { System.out.println("B.long"); }
}
new B().f(5);
```

→ Answer

`A.int` — `B.f(long)` **overloads** rather than overrides, and `B` inherits `f(int)` from `A`. Both are candidates; **widening comes second to an exact match**, so `f(int)` wins. *Overload resolution searches the whole inherited hierarchy, not just the declaring class.*

Drill 20

```
class Parent {
    Parent() { System.out.println("Parent()"); }
    Parent(int x) { System.out.println("Parent(int)"); }
}
class Child extends Parent {
    Child() { System.out.println("Child()"); }
    Child(int x) { this(); System.out.println("Child(int)"); }
}
new Child(5);
```

→ Answer

```
Parent()
Child()
Child(int)
```

`Child(int)` calls `this()` first → `Child()` runs, which has an implicit `super()` → `Parent()` runs first of all. Note `Parent(int)` is **never** called.

How to answer these in the actual interview

1. Never blurt the answer. Say the *rule* first, then the answer:

*"This is overload resolution, which happens at compile time — the compiler picks the most specific applicable method. **String** is more specific than **Object**, so it prints **Calling String method**."*

2. Always check "does it compile?" first. Roughly a third of these questions are compile errors, and candidates who trace the runtime immediately walk straight past them.

3. Name the error precisely. *"Compile error"* is worth one point. *"Compile error — overridden method does not throw IOException, because you can't widen the throws clause of an override"* is worth five.

4. Volunteer the fix. After every wrong-looking program, say how you'd make it correct. That's what turns a quiz into a conversation.

- 5. Connect the questions.** Both of the programs you were asked test the same axis — **compile-time vs runtime binding**. Pointing that out ("these are really the same question from two directions") is a senior-level move.
- 6. If you genuinely don't know:** *"I'd guess X because of [rule], but I'd want to run it to be sure — this is exactly the kind of thing I'd write a quick unit test for."*
Honest reasoning beats a confident wrong answer.
-

Quick Revision Sheet — the traps grouped by memory hook

OVERLOADING (compile time) ★

- null → the MOST SPECIFIC type wins (String beats Object)
- two SIBLING types (String/Integer) → AMBIGUOUS → compile error
- resolution order: WIDENING > BOXING > VARARGS ★
- return type ALONE can never overload

OVERRIDING (runtime) ★

- cannot WIDEN throws | cannot REDUCE visibility | return must be covariant
- static → HIDDEN, not overridden → the REFERENCE type wins
- fields → NOT polymorphic → the REFERENCE type wins ★
- private/final/static → not overridable
- calling an overridable method from a CONSTRUCTOR → child fields are still 0/null ▲

INITIALISATION ORDER

- static block → instance block → constructor
- parent FIRST, always (super() runs before the child's field initialisers) ★

STRING

- literals are POOLED → "a" == "a" is true
- new String() → always a new object → == is false
- "hel" + "lo" → folded at COMPILE time → pooled → true ★
- part + "lo" (non-final var) → RUNTIME concat → not pooled → false
- String is IMMUTABLE → s.concat("x") does NOTHING unless you assign it

AUTOBOXING

- Integer cache = -128..127 → == true inside, false outside ★
- wrapper == primitive → the wrapper is UNBOXED → compares VALUES → true
- unboxing NULL → NullPointerException ★
- ternary numeric promotion: true ? Integer : Double → 1.0

OPERATORS / NUMBERS

- i = i++ → 0 (the assignment overwrites the increment) ★
- 0.1 + 0.2 → 0.30000000000000004 (IEEE-754)
- 1/0 throws | 1.0/0 → Infinity | NaN == NaN → false
- byte b = b + 5; → compile error (promoted to int)
- byte b += 5; → fine (+= has an IMPLICIT cast) ★

try / catch / finally

- finally ALWAYS runs; a return in finally WINS and SWALLOWS the exception ▲
- the return VALUE is captured BEFORE finally runs (primitives)
- catch order must be SPECIFIC → GENERAL, else compile error
- try-with-resources closes in REVERSE order, BEFORE catch/finally ★
- System.exit(0) → finally is SKIPPED

COLLECTIONS

`list.remove(1)` on `List<Integer>` → removes INDEX 1 (`remove(int)` wins) ★
`Arrays.asList(...).add()` → `UnsupportedOperationException`
removing during a for-each → `ConcurrentModificationException` → use `removeIf` ★
`HashSet` of "equal" objects without equals/hashCode → size 2
`TreeMap.put(null, v)` → `NPE` (it must compare the key)
`Object[] o = new String[3]; o[1] = 42;` → `ArrayStoreException`

STATIC / THREADS

`nullRef.staticMethod()` → RUNS FINE (statics don't dereference) ★
`t.run()` → "main" | `t.start()` → "Thread-0" ★
`t.start()` twice → `IllegalThreadStateException`
`String.valueOf(null)` → `NPE` (it picks `valueOf(char[])`)

Related files in this pack

- **05 — Core Java** — the theory behind all of this: OOP, collections internals, `HashMap`, multithreading
- **12 — Java 17 Features** — records, sealed classes, pattern matching (use `instanceof` patterns in your answers)
- **17 — SOLID & Design Patterns** — Liskov Substitution is the *principle* behind the Q2 `throws` rule
- **22 — Java Streams: 20 Coding Problems** — the coding half of the Java round
- **18 — RoboGebra Technical Versions** — your real stack: Java 17, Spring Boot 3.2.0

Drill routine: cover the answers, work through Section K's rapid-fire table daily (under 4 min), and do 5 of the Section L drills on paper each morning. These questions repeat across companies almost verbatim — the same 40 programs cover the vast majority of Java output rounds.

ORM — JPA, Hibernate, Spring Data JPA (Easy Version) (+ ODM: MongoDB & Mongoose)

◆ **Why this file exists:** "Have you worked with ORM?" is asked in almost every Java/Spring round, and the follow-ups (`persist` vs `merge` , N+1, `LazyInitializationException` , `@Transactional`) are where candidates get exposed.

Your situation is specific: your current project **RoboGebra uses MongoDB — that is an ODM, not an ORM**. This file gives you (a) the honest one-liner so you never get caught claiming JPA on RoboGebra, and (b) full ORM depth so you can still answer every JPA/Hibernate question confidently.

Sections **B–K** follow the easy format — **Easiest way to remember** → **explanation** → **real-world example** → **diagram** → **trap** → **Easy memory box**. Section **L** is already in quiz format (code → answer → rule → follow-up) and is kept that way, because that is exactly how it gets asked.

Table of contents

#	Section	What it covers
A	Your honest position	what you have really used — RoboGebra = ODM. Exact wording.
B	ORM fundamentals	what/why, impedance mismatch, ORM vs JDBC vs ODM
C	JPA vs Hibernate vs Spring Data JPA	spec vs implementation vs abstraction (top-3 asked)
D	Entity mapping	<code>@Entity</code> , ids, generators, <code>@Column</code> , <code>@Embeddable</code> , inheritance, enums

#	Section	What it covers
E	Lifecycle & persistence context	4 states, dirty checking, flush, <code>persist</code> vs <code>merge</code> vs <code>save</code> — the heart of ORM
F	Relationships	all 4, owning side, <code>mappedBy</code> , cascade, <code>orphanRemoval</code> , helpers
G	Fetching & N+1	LAZY/EAGER defaults, N+1 with 4 fixes, <code>LazyInitializationException</code>
H	Transactions	<code>@Transactional</code> , propagation, isolation, rollback rules, self-invocation
I	Caching	L1 vs L2 vs query cache
J	Locking	optimistic <code>@Version</code> , pessimistic locks
K	equals/hashCode & Lombok	the <code>@Data</code> -on-entity disaster, DTO vs entity
L	● Predict the output (20)	same style as file 23, but ORM
M	ODM — what you actually use	Spring Data MongoDB (RoboGebra), Mongoose (CRM)
N	Schema & migrations	<code>ddl-auto</code> values, Flyway/Liquibase
O	Performance checklist	the 12 things that make ORM slow
P	Rapid-fire table	1-line answers for last-minute revision
Q	12 drills	answers hidden — test yourself

#	Section	What it covers
R	Answer scripts	word-for-word answers for the 6 likely questions

A — Your honest position (read this first)

The truth about your stack

Project	Persistence	Is it ORM?
RoboGebra (current)	MongoDB + Spring Data MongoDB (<code>MongoRepository</code> , <code>@Aggregation</code>)	✗ ODM (Object–Document Mapper)
RoboGebra CRM (Node)	Mongoose on MongoDB	✗ ODM
EasyVisa	Postgres + Neo4j behind a Grails/Groovy backend (GORM). You worked on the Angular side.	▲ ORM existed, but not your code
Subsea / Backoffice	—	—

▲ There is an unused `spring-data-jpa` dependency in the RoboGebra POM — but no JPA starter, no `DataSource` , and **no** `@Entity` classes. Do not say "we use JPA on RoboGebra." (Same warning as in 18 — RoboGebra Technical Stack, trap #5.)

✓ Say this when asked "Have you worked with ORM / Hibernate?"

"My hands-on object-mapping work is with Spring Data MongoDB on RoboGebra — that's an ODM rather than an ORM: `@Document` models, `MongoRepository` derived queries and `@Aggregation` pipelines. I'm solid on JPA/Hibernate concepts — entity lifecycle, the persistence context, relationship mapping, lazy loading and the N+1 problem, `@Transactional` — and I've worked against a Grails/GORM + Postgres backend on EasyVisa from the frontend side. I haven't

shipped a large Hibernate codebase myself, but I can pick it up quickly because Spring Data's repository model is identical across both."

Why this works: it's honest, it names the *concepts* (which is what they're actually testing), and it immediately gives you credibility on the part you *do* own.

✗ Never say

- ✗ "Yes, we use Hibernate on RoboGebra."
→ one follow-up ("show me an entity") and you're finished ▲
- ✗ "MongoDB is an ORM."
→ it's an ODM. This exact slip is a common junior tell.
- ✗ "Spring Data JPA and Spring Data MongoDB are the same thing."
→ same programming model, completely different engine –
no persistence context, no lazy loading, no dirty checking in
Mongo.

If they push: "So you've never written an @Entity?"

"Not in production, no — my production persistence is document-based. I've written JPA entities while learning and I understand the mapping model well. What I'd want to be careful about on day one is the things that bite you at scale: lazy loading boundaries, N+1 queries, and transaction demarcation."

★ Naming the *risks* is what a 4-year developer sounds like. Naming nothing is what a fresher sounds like.

B — ORM fundamentals

Q: What is ORM?

The easiest way to remember:

ORM = a TRANSLATOR between Java objects and database tables.

class → table
field → column
object reference → foreign key

class User { Long id; String name; }	ORM	users
	≈	id name
new User(1, "Priya")		1 Priya

Without ORM you write SQL, then manually convert every `ResultSet` row into an object. With ORM you work with objects and the framework writes the SQL.

Real-world idea: an **interpreter in a meeting**. You speak Java; the database speaks SQL. The ORM sits in the middle and translates both ways.

Q: What problem does ORM solve? (the "impedance mismatch")

The easiest way to remember:

Objects and tables think DIFFERENTLY about the same data.
That gap is the "object-relational impedance mismatch".

The five disagreements:

Mismatch	Objects	Relational tables
Granularity	many small classes (<code>Address</code> inside <code>User</code>)	fewer, wider tables
Inheritance	<code>Employee extends Person</code>	✗ no inheritance at all
Identity	<code>==</code> (reference) and <code>equals()</code>	primary key only
Associations	directional references (<code>user.getOrders()</code>)	foreign keys — inherently bidirectional
Navigation	walk the graph: <code>a.getB().getC()</code>	joins; walking = N queries ▲

JAVA WORLD

inheritance ✓
object references
a.getB().getC() (free)

DATABASE WORLD

inheritance ✗
foreign keys
each hop = another QUERY ▲

★ That last row is the seed of the **N+1 problem** — in Java, walking a graph is free; in SQL, every hop is a round trip.

Real-world idea: translating **Tamil to English**. Both languages express the same ideas, but the grammar doesn't line up — some things need restructuring, and a literal word-for-word translation produces nonsense.

Easy memory

Impedance mismatch = objects and tables DISAGREE in 5 ways ★
Granularity | Inheritance | Identity | Associations | Navigation

Navigation is the dangerous one:

a.getB().getC() is free in Java, but 2 QUERIES in SQL → N+1 ▲

Q: Advantages and disadvantages of ORM?

Advantages

- ✓ Removes JDBC boilerplate (Connection, PreparedStatement, ResultSet, mapping)
- ✓ DATABASE PORTABILITY — the dialect handles vendor SQL differences
- ✓ Built-in caching, dirty checking, lazy loading, transaction management
- ✓ Type safety + refactorability — rename a field, the compiler finds every usage
- ✓ Automatic SQL-injection protection (bind parameters)

Disadvantages

- ✗ HIDES THE SQL – it's easy to generate awful queries (N+1) without noticing ★
- ✗ Real learning curve; the "magic" bites when you don't know the persistence context
- ✗ Poor fit for BULK operations and heavy reporting → use native SQL or JDBC there
- ✗ Extra memory and CPU overhead vs plain JDBC

50 lines of JDBC per table interface ✓ → ORM: 1 entity + 1 repository
but the SQL is now INVISIBLE ▲

■ **Great line to say:** "ORM removes boilerplate, not the need to understand SQL. The two things I always check are the generated SQL in the logs and whether any collection access is causing N+1."

Easy memory

- ✓ no boilerplate | portable | caching + dirty checking | type safe | no SQL injection
 - ✗ HIDES the SQL (N+1) ★ | learning curve | bad for bulk/reporting | overhead
- Line to say: "ORM removes boilerplate, not the need to understand SQL." ★

Q: ORM vs JDBC vs ODM?

The easiest way to remember:

JDBC → you write SQL yourself	→ rows and columns
ORM → objects ↔ TABLES	→ JPA/Hibernate (relational)
ODM → objects ↔ DOCUMENTS	→ MongoDB/Mongoose ★ (what you use)

	JDBC	ORM (JPA/ Hibernate)	ODM (Spring Data MongoDB / Mongoose)
Data model	rows/columns	objects ↔ tables	objects ↔ documents (BSON/ JSON)
You write	SQL + mapping code	entities + JPQL/ derived queries	documents + derived queries/aggregations
Schema	you manage it	generated/validated from entities	schema-flexible (Mongoose can enforce it)
Joins	SQL joins	relationship mapping + fetch strategies	embed, or <code>\$lookup</code> / <code>populate()</code>
Persistence context	none	Yes — L1 cache + dirty checking ★	No — you must call <code>save()</code> ★
Lazy loading	none	yes (proxies)	only <code>@DBRef(lazy = true)</code> ; usually you embed instead
Transactions	manual	declarative <code>@Transactional</code>	needs a replica set (Mongo 4.0+)

★ The two rows that matter most for you: Mongo has **no persistence context** and **no dirty checking** — so in Mongo you must always call `save()` explicitly, whereas in JPA an entity loaded inside a transaction updates itself.

Easy memory

JDBC → raw SQL | ORM → objects ↔ TABLES | ODM → objects ↔ DOCUMENTS ★

The two ODM differences to name:
NO persistence context → you MUST call `save()` ★
NO dirty checking / lazy loading

C — JPA vs Hibernate vs Spring Data JPA

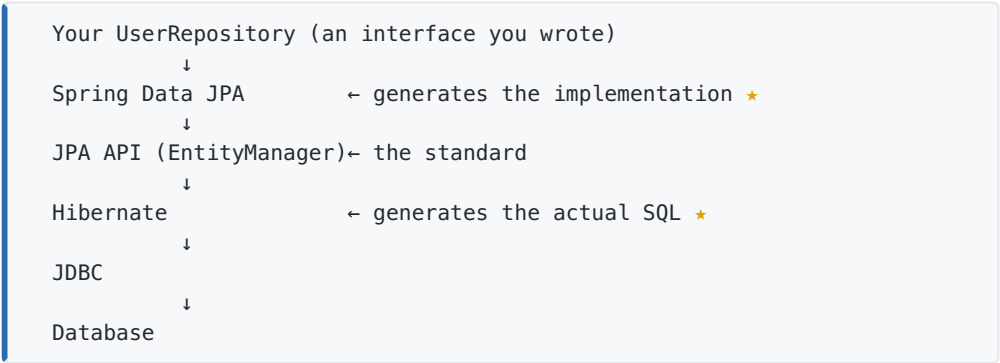
● This is the single most-asked ORM question. Have the answer memorised.

The easiest way to remember:

JPA	= the RULE BOOK	(specification – interfaces + annotations)
Hibernate	= the PLAYER	(implementation – actually writes the SQL)
Spring Data JPA	= the COACH	(removes the boilerplate on top)

	JPA	Hibernate	Spring Data JPA
What is it	a specification	an implementation of JPA	a Spring abstraction on top of JPA
Ships as	<code>jakarta.persistence-api</code>	<code>hibernate-core</code>	<code>spring-boot-starter-data-jpa</code>
Gives you	<code>EntityManager</code> , <code>@Entity</code> , JPQL	the engine + extras (<code>Session</code> , HQL, Criteria, L2 cache, <code>@BatchSize</code>)	repository interfaces , derived queries, pagination, auditing
Can run alone?	✗ it's just an API	✓	✗ needs a JPA provider

The layering — draw this



Real-world idea:

JPA → the DRIVING RULES (the law says: drive on the left)
Hibernate → the CAR that actually drives
Spring Data JPA → the CHAUFFEUR who drives it for you ★

✓ The memorised answer

"JPA is the specification — the standard API and annotations. Hibernate is the implementation that actually generates the SQL; it's the default provider in Spring Boot. Spring Data JPA sits on top and removes the DAO boilerplate — I declare a repository interface and Spring generates the implementation at runtime. So the layering is: my repository → Spring Data JPA → JPA API → Hibernate → JDBC → database."

Q: Other JPA implementations?

EclipseLink (the reference implementation) and **OpenJPA**. Hibernate dominates in Spring.

Q: **EntityManager** vs **Hibernate Session** ?

Session = Hibernate's NATIVE API
EntityManager = the JPA STANDARD API

In modern Hibernate, Session EXTENDS EntityManager. ★

Prefer **EntityManager** for portability; unwrap when you need a Hibernate-only feature:

```
Session session = entityManager.unwrap(Session.class);
```

Q: EntityManagerFactory vs EntityManager ?

EntityManagerFactory → HEAVY, THREAD-SAFE, ONE per application, made at startup
EntityManager → LIGHT, NOT thread-safe, ONE per transaction/request ★

EntityManagerFactory (created once at startup – expensive)
| creates
├─ EntityManager (request 1)
├─ EntityManager (request 2)
└─ EntityManager (request 3) ← cheap, short-lived, one per transaction

Real-world idea: the **factory** builds cars; each **car** is driven by one person at a time.

★ Spring injects a **thread-bound proxy** of `EntityManager`, which is exactly why sharing one field in a singleton `@Service` is safe even though `EntityManager` itself is not thread-safe.

Easy memory

JPA = spec | Hibernate = implementation | Spring Data JPA = chauffeur

Layering: repository → Spring Data → JPA API → Hibernate → JDBC → DB ★

EntityManagerFactory = heavy, thread-safe, ONE

EntityManager = light, NOT thread-safe, one per transaction ★

Session extends EntityManager

▲ `javax` vs `jakarta` (the Boot 3 trap — RoboGebra is Boot 3.2)

Spring Boot 2.x → `import javax.persistence.Entity;`
Spring Boot 3.x → `import jakarta.persistence.Entity;` ✓

Oracle donated Java EE to the Eclipse Foundation,
but kept the "javax" TRADEMARK.

↓

Every package had to be renamed javax.* → jakarta.*

↓

Boot 3 upgrade = thousands of import changes ▲

■ "On Boot 3.2 all the persistence and validation imports are *jakarta.**, not *javax.** — that's the single biggest migration cost when upgrading from Boot 2."

D — Entity mapping reference

Q: What makes a class a valid JPA entity?

The easiest way to remember — **five rules**:

1. @Entity annotation
2. an @Id
3. a public/protected NO-ARG CONSTRUCTOR ★ (Hibernate uses reflection + proxies)
4. NOT final, and no final methods/fields ★ (proxies SUBCLASS it)
5. a top-level class (not an interface or enum)

★ Rules 3 and 4 together explain something people ask about constantly:

Why can't a `record` be a JPA entity?

records are FINAL, have no no-arg constructor, and are immutable
→ all three rules broken ▲

The core annotations

```
@Entity
@Table(name = "app_user",
      uniqueConstraints = @UniqueConstraint(columnNames = "email"),
      indexes = @Index(name = "idx_user_status", columnList = "status"))
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "full_name", nullable = false, length = 100)
    private String name;

    @Column(unique = true)
    private String email;

    @Enumerated(EnumType.STRING)           // ▲ NEVER ORDINAL – see below
    private Status status;

    private LocalDateTime createdAt;       // java.time needs no
    annotation ✓

    @Transient                             // NOT persisted
    private String fullDisplayName;

    @Lob
    private String description;             // CLOB / BLOB for byte[]

    protected User() { }                   // ★ the required no-arg
    constructor
}
```

Annotation	Purpose
@Entity	marks the class as persistent
@Table	table name, schema, unique constraints, indexes
@Id	primary key
@GeneratedValue	how the PK is generated

Annotation	Purpose
<code>@Column</code>	column name, nullable, length, unique, precision, <code>insertable</code> / <code>updatable</code>
<code>@Transient</code>	the field is not persisted
<code>@Enumerated(EnumType.STRING)</code>	store the enum by name ★
<code>@Lob</code>	large object (CLOB/BLOB)
<code>@Embedded</code> / <code>@Embeddable</code>	a value object flattened into the same table
<code>@ElementCollection</code>	a collection of basic/embeddable types in a side table
<code>@Version</code>	optimistic locking
<code>@CreatedDate</code> / <code>@LastModifiedDate</code>	Spring Data auditing

▲ `@Enumerated(EnumType.ORDINAL)` — the classic bug

ORDINAL is the DEFAULT, and it stores the enum's POSITION, not its name.

```
enum Status { ACTIVE, INACTIVE, DELETED }
//           0         1         2

DB now holds: 0, 1, 2
```

Six months later someone adds a value in the middle:

```
enum Status { ACTIVE, PENDING, INACTIVE, DELETED }
//           0         1         2         3
```

Row with 1 used to mean INACTIVE
Row with 1 now means PENDING ▲

EVERY existing row silently means something else.
No exception. No error. Just wrong data. ▲

```
@Enumerated(EnumType.STRING)    // ✓ ALWAYS do this
private Status status;
```

Now the database stores "ACTIVE", "INACTIVE" — reordering the enum can never corrupt anything.

Easy memory

Entity rules: @Entity + @Id + NO-ARG CONSTRUCTOR + not final + top-level

★

→ which is exactly why a `record` can NEVER be an @Entity

▲

@Enumerated: the DEFAULT is ORDINAL and it stores the POSITION.
Insert one enum value in the middle → every existing row means something different ▲ → ALWAYS use EnumType.STRING ★

Q: @GeneratedValue strategies?

Strategy	How it works	Notes
IDENTITY	DB auto-increment column	simple; disables JDBC batch inserts ▲
SEQUENCE	a database sequence	✓ preferred on Postgres/Oracle; supports batching
TABLE	a separate table simulates a sequence	portable but slow (row locking) — avoid
AUTO	the provider picks	on Hibernate 6 + Boot 3 this usually means a sequence
UUID	Hibernate 6 generates a UUID	good for distributed ids; worse index locality

Why does IDENTITY block batch inserts? ★

IDENTITY:

Hibernate MUST know the id immediately after each insert,
and only the DB can tell it.

↓

INSERT → ask the DB for the id → INSERT → ask again → ...

↓

1,000 inserts = 1,000 round trips

SEQUENCE:

Hibernate asks for a BLOCK of 50 ids in ONE call,
then batches 50 inserts into one round trip ▲

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"user_seq")
@SequenceGenerator(name = "user_seq", sequenceName = "user_seq",
allocationSize = 50)
private Long id;
```

■ "*IDENTITY* is the easy default but it kills batch inserts — Hibernate needs the generated id immediately, so it can't queue the statements. On Postgres I'd use *SEQUENCE* with an *allocationSize* so it fetches a block of ids at once."

Easy memory

IDENTITY → MySQL auto-increment → simple → ▲ NO batch inserts (1 round trip each)

SEQUENCE → Postgres/Oracle → ★ batch-friendly (allocationSize = 50)

TABLE → slow, avoid | AUTO → provider decides | UUID → distributed ids

Q: @Embeddable — when do you use it?

The easiest way to remember:

@Embeddable = a value object with NO IDENTITY of its own.
It lives INSIDE the parent's table. ★

```

@Embeddable
public class Address {
    private String street;
    private String city;
    private String zip;
}

@Entity
public class User {
    @Id @GeneratedValue private Long id;

    @Embedded private Address address;          // → street, city, zip columns
    on `user` ★

    @Embedded
    @AttributeOverride(name = "city", column = @Column(name =
"billing_city"))
    private Address billingAddress;             // reuse the same class,
    different columns
}

```

ONE table `user`

id	name	street	city	zip	billing_city
----	------	--------	------	-----	--------------

└─ from the embedded Address ─┘

No separate `address` table. No join. ✓

Real-world idea: an address written **on** an envelope, not on a separate card clipped to it. It has no life of its own — the envelope is thrown away, the address goes with it.

Q: @Embeddable vs @Entity?

```

@Entity      → has its OWN identity, its OWN table, its OWN lifecycle
@Embeddable → has NONE of those; it is part of the owner and DIES with it
★

```

Is it meaningful on its own, and would you ever query it alone?

YES → @Entity

NO → @Embeddable

Q: Composite primary keys?

Two options — @EmbeddedId (preferred, cleaner) or @IdClass .

```
@Embeddable
public class OrderItemId implements Serializable {    // must be
Serializable + equals/hashCode ★
    private Long orderId;
    private Long productId;
}

@Entity
public class OrderItem {
    @EmbeddedId private OrderItemId id;
}
```

- ▲ The composite-key class MUST:
 - implement Serializable
 - override equals() and hashCode()
 - have a no-arg constructor

Q: Inheritance mapping strategies?

The easiest way to remember — **how many tables?**

SINGLE_TABLE → ONE table for the whole hierarchy + a discriminator column (default)
JOINED → parent table + one table PER CHILD, linked by joins
TABLE_PER_CLASS → one table per CONCRETE class, no sharing
@MappedSuperclass → NO table for the parent; it just shares fields ★

Strategy	Tables	Pros	Cons
SINGLE_TABLE (default)	one + discriminator	fastest — no joins	subclass columns must be nullable ▲

Strategy	Tables	Pros	Cons
JOINED	parent + child tables	normalised, no nulls	every read is a join
TABLE_PER_CLASS	one per concrete class	no joins for single-type reads	polymorphic queries need UNION
@MappedSuperclass	none for the parent	shares fields (id , createdAt)	the parent is not an entity — can't query it polymorphically

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "payment_type")
public abstract class Payment { ... }

@Entity @DiscriminatorValue("CARD")
public class CardPayment extends Payment { ... }
```

SINGLE_TABLE `payment`

id	payment_type	amount	card_no	upi_id
1	CARD	500	1234...	NULL ▲
2	UPI	300	NULL ▲	a@ybl

↑ the discriminator ↑ nullable columns are the cost

★ **@MappedSuperclass** is the one you'll actually use most — a **BaseEntity** holding **id**, **createdAt** and **updatedAt**, shared by every entity without creating a table of its own.

Easy memory

```
SINGLE_TABLE ★ default → 1 table + discriminator → fast, but NULLABLE columns
JOINED → normalised → but every read is a JOIN
TABLE_PER_CLASS → one per concrete class → polymorphic queries need UNION
@MappedSuperclass ★ → NO table — just shares id/createdAt (what you'll really use)
```

E — Entity lifecycle & the persistence context

● **This section is the heart of ORM.** If you understand the persistence context, most "tricky" JPA questions answer themselves.

Q: What is the persistence context?

The easiest way to remember:

The persistence context = Hibernate's MEMORY of the entities it is currently managing, for the duration of ONE transaction.

It is three things at once:

1. the FIRST-LEVEL CACHE → always on, cannot be disabled ★
2. where DIRTY CHECKING happens → it auto-generates UPDATES
3. an IDENTITY MAP → one DB row = exactly ONE Java object, inside that transaction

```
@Transactional
public void m() {
    User a = repo.findById(1L).get();    // SQL runs → object cached
    User b = repo.findById(1L).get();    // NO SQL – served from the L1
    cache ★

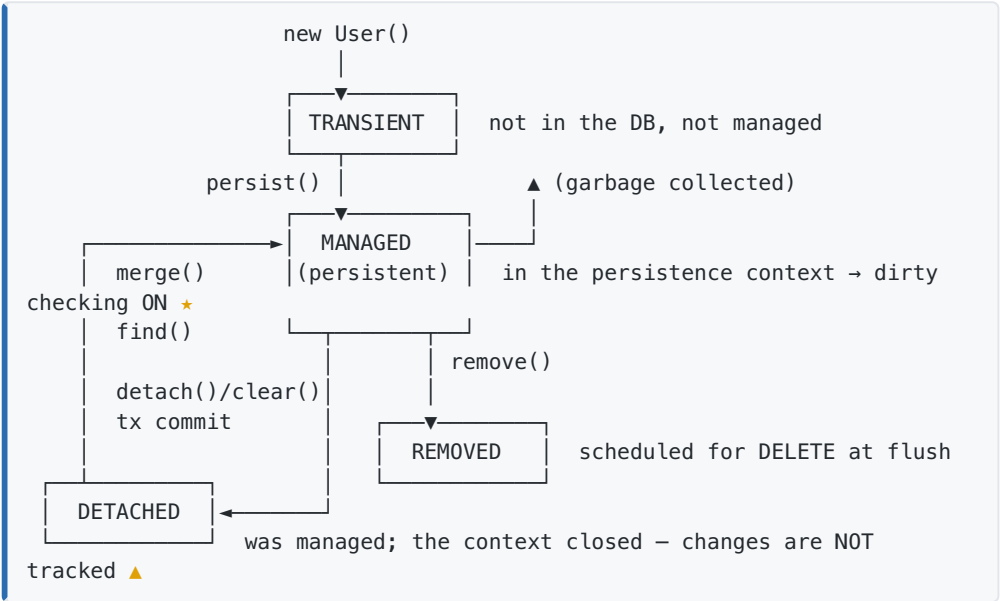
    System.out.println(a == b);          // TRUE – the identity map ★
}
```

Real-world idea: a **shopping trolley**.

While you're in the shop (the transaction), the trolley holds everything you've picked up.

You can put things in, take things out, change your mind – nothing reaches the BILL until you go to the checkout (commit). ★

Q: The four entity states — draw this



State	In the DB?	Tracked?	How you get there
Transient (new)	✗	✗	<code>new User()</code>
Managed (persistent)	✓ (or will be at flush)	✓	<code>persist()</code> , <code>find()</code> , <code>merge()</code> , a JPQL result
Detached	✓	✗ ▲	the transaction ended, <code>detach()</code> , <code>clear()</code> , <code>close()</code>
Removed	✓ until flush	✓	<code>remove()</code>

Real-world idea: a **hospital patient**.

TRANSIENT → a person at home; the hospital has never seen them
MANAGED → ADMITTED – every change is monitored and recorded ★
DETACHED → DISCHARGED – still alive, but nobody is monitoring them ▲
REMOVED → marked for discharge-and-delete at the end of the shift

★ The whole of "tricky JPA" comes down to one question: **is this entity**

MANAGED right now?

MANAGED → changes are saved automatically, no `save()` needed ✓
DETACHED → changes go NOWHERE ▲

Q: What is dirty checking?

The easiest way to remember:

Dirty checking = Hibernate remembers what the entity looked like when it was loaded, and at commit it compares. Anything different → UPDATE.

You NEVER call `update()`. ★

```
@Transactional
public void renameUser(Long id, String newName) {
    User user = userRepository.findById(id).orElseThrow();
    user.setName(newName);           // managed → dirty checked
    // no save() needed – the UPDATE fires automatically at commit ✓
}
```

`findById()` → Hibernate stores a SNAPSHOT: `{name: "Priya"}`
`setName()` → current state is now: `{name: "Ravi"}`
`commit` → compare snapshot vs current → they differ
→ UPDATE users SET name='Ravi' WHERE id=1 ✓

Real-world idea: a **hotel minibar**. Nobody watches you drink. At checkout they compare the fridge against the original list and bill you for the difference.

■ **Interview gold:** "Inside a transaction I don't need to call `save()` on an entity I loaded — it's managed, so dirty checking generates the UPDATE at flush. People add a redundant `save()` out of habit; it's harmless but shows they don't know the persistence context."

Easy memory

Persistence context = L1 cache + dirty checking + identity map ★
(a shopping trolley until checkout)

4 states: TRANSIENT → MANAGED → DETACHED / REMOVED
(at home → admitted → discharged)

MANAGED → changes saved automatically (dirty checking, hotel minibar)
DETACHED → changes go NOWHERE ▲

The one question behind every tricky JPA question:
"Is this entity MANAGED right now?" ★

Q: What is flush? When does it happen?

The easiest way to remember:

FLUSH = send the pending SQL to the database (but NOT commit) ★
COMMIT = make it permanent

flush → the SQL runs, still inside the transaction → can still be rolled back
commit → flush + make it permanent ✓

Flush happens in three situations:

1. On transaction COMMIT
2. BEFORE a query whose result could be affected by pending changes
(FlushModeType.AUTO – the default) ★
3. When you call flush() explicitly

★ Point 2 is clever and worth explaining: if you've changed a `User`'s status in memory and then run a query for "all active users", Hibernate flushes first so the query sees your change.

```
@Transactional(readOnly = true)           // → FlushMode.MANUAL → NOTHING is  
ever flushed
```

That is exactly why **writes silently disappear inside a read-only transaction** — no exception, no error, just no UPDATE.

Real-world idea: **saving a document vs printing it**. Flush = writing to disk; commit = handing over the signed final copy.

Q: `persist()` vs `merge()` vs `save()` — the most-asked lifecycle question

The easiest way to remember:

`persist()` → for a NEW object → the SAME object becomes managed
`merge()` → for a DETACHED object → returns a DIFFERENT, managed COPY ★ ★
`save()` → Spring Data → decides between the two for you

	<code>persist(e)</code>	<code>merge(e)</code>	Spring Data <code>save(e)</code>
Argument state	must be transient (or already managed)	transient or detached	anything
Returns	<code>void</code> — the argument becomes managed	a different, managed copy ; the argument stays detached ▲	the managed instance
On a detached entity	throws <code>EntityExistsException</code>	copies the state onto a managed instance (SELECT then UPDATE)	delegates to <code>merge</code>
SQL	INSERT	SELECT + INSERT/UPDATE	<code>isNew()</code> ? <code>persist</code> : <code>merge</code>

```
// Spring Data's SimpleJpaRepository, essentially:
public <S extends T> S save(S entity) {
    if (entityInformation.isNew(entity)) { em.persist(entity); return
entity; }
    else                                { return em.merge(entity); }
}
```

▲ The trap — merge returns a DIFFERENT object

```
detachedUser  —merge()→  managedCopy  ← a DIFFERENT object!
    |                      |
    still DETACHED        MANAGED ✓
    changes here go       changes here are saved
    NOWHERE ▲
```

```
User saved = repo.save(detachedUser);

detachedUser.setName("X"); // ✗ goes nowhere
saved.setName("X");        // ✓ this one is managed
```

Rule: always keep working with the object `save()` RETURNS.

Real-world idea: `merge` is like **photocopying a form onto the official letterhead**. The office keeps and processes the official copy; your original piece of paper is now irrelevant, no matter what you scribble on it.

Q: How does Spring Data decide an entity is "new"?

```
By default: the @Id field is NULL (or 0 for a primitive) → it's new →
persist()
Otherwise → merge()
```

▲ The consequence:

```
If YOU assign ids yourself (UUIDs, natural keys)
    → the id is never null
    → Spring thinks it already exists
    → it issues a pointless SELECT before EVERY insert
```

Fix: implement `Persistable<ID>` and override `isNew()`.

Q: `find()` vs `getReference()` ?

```
find()          → hits the DB IMMEDIATELY → returns the entity (or null)
getReference() → returns a LAZY PROXY → no SQL until you touch a field ★
```

	<code>find(User.class, 1L)</code>	<code>getReference(User.class, 1L)</code>
Hits the DB	immediately	not until you touch a non-id field
Returns	the entity (or <code>null</code>)	a lazy proxy
If the row is missing	<code>null</code>	throws <code>EntityNotFoundException</code> later ▲

The real use — setting a foreign key without loading the row:

```
order.setUser(em.getReference(User.class, userId)); // ✓ no SELECT on user at all
```

```
find()           → SELECT * FROM users WHERE id=5   ← wasted, you only needed the FK
getReference() → no query at all; the INSERT just writes user_id = 5 ▲
```

Spring Data equivalents: `findById()` → `find`, `getReferenceById()` → `getReference`.

Q: `remove()` vs `deleteById()` vs bulk delete?

```
remove(entity)      → the entity must be MANAGED → DELETE at flush →
cascades apply
deleteById(id)      → Spring does a findById FIRST, then remove → TWO
statements ▲
deleteAllInBatch() → ONE bulk DELETE ▲ but BYPASSES the persistence
context
/ @Modifying        and all cascades → loaded entities go STALE ▲
```

Deleting 1,000 rows:

```
deleteAll()          → 1,000 SELECTs + 1,000 DELETEs
deleteAllInBatch() → 1 DELETE ▲ (but no cascade, no callbacks)
```

Easy memory

```
persist() → NEW → the SAME object becomes managed
merge()   → DETACHED → returns a DIFFERENT managed copy ★ ★ (use the
RETURN value!)
save()    → id is null ? persist : merge

flush = send the SQL | commit = make it permanent
readOnly = true → FlushMode.MANUAL → writes SILENTLY vanish ▲

find() → SQL now | getReference() → lazy proxy, no SQL ★ (great for
setting a FK)
deleteById → 2 statements | deleteAllInBatch → 1, but skips cascades ▲
```

F — Relationships

The four types

The easiest way to remember:

```
@OneToOne → 1 ↔ 1      User ↔ Profile
@ManyToOne → many → 1   Order → User      ★ ALWAYS the owning side
(holds the FK)
@OneToMany → 1 → many   User → Orders    ← the mirror (mappedBy)
@ManyToMany → many ↔ many Student ↔ Course (needs a join table)
```

```

// — @OneToOne —————
@Entity
public class User {
    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY,
    orphanRemoval = true)
    @JoinColumn(name = "profile_id")           // owning side – holds the
    FK ★
    private Profile profile;
}
@Entity
public class Profile {
    @OneToOne(mappedBy = "profile")           // inverse side
    private User user;
}

// — @ManyToOne / @OneToMany (the most common pair) ———
@Entity
public class Order {
    @ManyToOne(fetch = FetchType.LAZY)        //
    ▲ ALWAYS override – the default is EAGER
    @JoinColumn(name = "user_id")             // owning side ★
    private User user;
}
@Entity
public class User {
    @OneToMany(mappedBy = "user",              // "user" = the FIELD NAME
    in Order ★
    cascade = CascadeType.ALL,
    orphanRemoval = true)
    private List<Order> orders = new ArrayList<>();
}

// — @ManyToMany —————
@Entity
public class Student {
    @ManyToMany
    @JoinTable(name = "student_course",
    joinColumns = @JoinColumn(name = "student_id"),
    inverseJoinColumns = @JoinColumn(name = "course_id"))
    private Set<Course> courses = new HashSet<>();
}
@Entity
public class Course {
    @ManyToMany(mappedBy = "courses")
    private Set<Student> students = new HashSet<>();
}

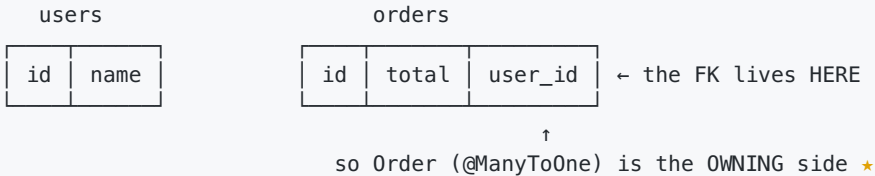
```

Q: What is the "owning side" and why does it matter?

★ **This single concept causes more JPA bugs than anything else.**

The OWNING side is the one that HOLDS THE FOREIGN KEY COLUMN.

It is the ONLY side Hibernate looks at when WRITING to the database.
The inverse side (mappedBy) is READ-ONLY for persistence. ★



@ManyToOne → ALWAYS the owning side
mappedBy → goes on the INVERSE side, and names the FIELD on the owning side

▲ The #1 relationship bug

```
user.getOrders().add(order);    // ✗ only touched the INVERSE side
userRepository.save(user);
// → nothing is saved. order.user_id is still NULL ▲
```

```
order.setUser(user);            // ✓ set the OWNING side
orderRepository.save(order);
```

Real-world idea: **only the person holding the pen can sign the register.** You can shout your name from the queue (the inverse side) all day; unless the pen-holder writes it, nothing is recorded.

✓ **Always write bidirectional helper methods**

```
public void addOrder(Order order) {
    orders.add(order);
    order.setUser(this);           // ★ keeps BOTH sides consistent
}
public void removeOrder(Order order) {
    orders.remove(order);
    order.setUser(null);
}
```

This one habit removes an entire class of bug permanently.

Q: What happens if you use @OneToMany without mappedBy ?

Hibernate assumes UNIDIRECTIONAL and silently creates an EXTRA JOIN TABLE (user_orders) instead of using the FK on `order`. ▲

What you expected: orders.user_id
What you got: a third table `user\_orders(user\_id, order\_id)`
▲

Almost always a mistake — either add mappedBy , or put @JoinColumn on the @OneToMany .

Q: Cascade types?

Cascade = "when this operation happens to the PARENT, do it to the CHILDREN too."

Cascade	Propagates
PERSIST	persist() → children inserted
MERGE	merge()
REMOVE	remove() → children deleted
REFRESH	refresh()

Cascade	Propagates
DETACH	detach()
ALL	all of the above

▲ There is NO cascade by default.

CascadeType.ALL is FINE on a true parent-child composition (Order → OrderItem)

CascadeType.ALL is DANGEROUS on @ManyToOne / @ManyToMany ▲

```
@ManyToOne(cascade = CascadeType.ALL)    //  
▲ deleting ONE ORDER deletes the USER  
private User user;  
  
@ManyToMany(cascade = CascadeType.ALL)    // ▲ deleting ONE STUDENT  
deletes shared COURSES  
private Set<Course> courses;
```

★ The rule: cascade **downwards** from a parent that truly owns its children — never upwards or sideways.

Q: CascadeType.REMOVE vs orphanRemoval = true ?

The easiest way to remember:

CascadeType.REMOVE → deletes children when the PARENT is deleted
orphanRemoval → ALSO deletes a child REMOVED FROM THE COLLECTION ★

	CascadeType.REMOVE	orphanRemoval = true
Parent deleted → children deleted	✓	✓
Child removed from the collection	✗ (the FK is just orphaned)	✓

```
user.getOrders().remove(order);
```

```
// orphanRemoval = true      → DELETE FROM orders WHERE id = ? ✓  
// CascadeType.REMOVE only  → nothing happens (or an FK constraint  
violation) ▲
```

Real-world idea:

CascadeType.REMOVE → the company shuts down, so all employees leave
orphanRemoval → you FIRE one employee, and they cease to exist ★

Q: List vs Set for collections?

Set → no duplicates, and avoids MultipleBagFetchException ★

List → a "bag"; preserves DB order only if you add @OrderBy

RULE: use Set for @ManyToMany and anywhere you might fetch-join two collections ★

Easy memory

@ManyToOne = the FK side = the OWNING side ★ (@JoinColumn)

@OneToMany = mappedBy = the MIRROR (read-only for persistence)

▲ #1 BUG: adding to the inverse collection saves NOTHING

→ always write addOrder() helpers that set BOTH sides ★

▲ @OneToMany without mappedBy → a surprise extra JOIN TABLE ▲

Cascade: NONE by default | ALL is fine for Order→OrderItem,

DANGEROUS on @ManyToOne / @ManyToMany ▲

orphanRemoval → also deletes a child removed from the collection ★

Use Set for @ManyToMany.

G — Fetching, N+1, and LazyInitializationException

Q: Lazy vs eager — and what are the defaults?

★ Memorise this with one phrase: "To-One is EAGER, To-Many is LAZY."

Association	Default fetch	What you should use
@ManyToOne	EAGER ▲	LAZY
@OneToOne	EAGER ▲	LAZY
@OneToMany	LAZY ✓	LAZY
@ManyToMany	LAZY ✓	LAZY

■ **Say this:** "The `-ToOne` associations default to EAGER, which is the wrong default — every `findById` silently drags in a join or an extra query. I set `fetch = FetchType.LAZY` on all associations and fetch what I need explicitly with a join fetch or an entity graph."

How lazy actually works — proxies

LAZY field
↓
Hibernate returns a generated SUBCLASS (a proxy) instead of the real object
↓
the first time you touch a NON-ID property → the real SELECT fires ★

```
order.getUser()           → returns a proxy, NO query
order.getUser().getId()    → still NO query (the id is already known!)
★
order.getUser().getName() → NOW the SELECT runs
```

★ That is also exactly why **entities can't be final** — Hibernate must be able to subclass them.

Q: What is the N+1 problem?

```
1 query to load N parents
+ N queries, one per parent, for its association
= N + 1 queries ▲
```

```
List<User> users = userRepository.findAll();           // 1 query
for (User u : users) {
    System.out.println(u.getOrders().size());         // N queries – one per
user ✗
}
// 100 users → 101 queries
```

```
SELECT * FROM users                ← 1
SELECT * FROM orders WHERE user_id = 1 ← 2
SELECT * FROM orders WHERE user_id = 2 ← 3
SELECT * FROM orders WHERE user_id = 3 ← 4
...
                                  ← 101 ▲
```

Real-world idea: going to the market **101 times** — once for the list, then once per item — instead of buying everything in one trip.

The 4 fixes

1. **JOIN FETCH (JPQL)** — one query, explicit

```
@Query("select distinct u from User u left join fetch u.orders")
List<User> findAllWithOrders();
```

2. **@EntityGraph** — declarative; works with derived queries and pagination ★

```
@EntityGraph(attributePaths = {"orders", "profile"})
List<User> findByStatus(String status);
```

3. **@BatchSize** — turns N queries into N/size queries using **IN (...)**

```
@OneToMany(mappedBy = "user")
@BatchSize(size = 25)
private List<Order> orders;
// or globally:
spring.jpa.properties.hibernate.default_batch_fetch_size=25
```

Without: 100 queries

With size 25: 4 queries → WHERE user_id IN (1,2,...,25) ▲

4. DTO projection — fetch only the columns you need, no entities at all ★

```
@Query("select new com.app.dto.UserSummary(u.id, u.name, count(o)) " +
        "from User u left join u.orders o group by u.id, u.name")
List<UserSummary> findSummaries();
```

■ "My default fix is an *@EntityGraph* because it composes with derived queries and pagination. For read-only screens I prefer a DTO projection — it avoids loading entities into the persistence context at all."

● In your RoboGebra code — the N+1 fix, in MongoDB ★

File: `robogebra-portal/.../domain/exerciseitem/repository/ExerciseItemRepository.java`

(27 *@Aggregation* pipelines across the codebase)

```

@Repository
public interface ExerciseItemRepository extends
MongoRepository<ExerciseItemEntity, String> {

    // derived query – Spring writes it from the METHOD NAME
    Optional<ExerciseItemEntity> findByExercise_IdAndIndex(String
exerciseId, String index);

    // the N+1 fix: ONE pipeline instead of one query per exercise ★
    @Aggregation(pipeline = {
        "{ '$match': { 'exercise.$id': { '$in': ?0 } } }",
        "{ '$sort': { 'displayOrder': 1 } }"
    })
    List<ExerciseItemEntity> findAllByExerciseIds(List<ObjectId>
exerciseIds);
}

```

■ "N+1 shows up in Mongo too. Loading a chapter's exercises and then fetching each exercise's items one at a time is one query plus N. I replaced it with a single **@Aggregation** pipeline that **\$match** es on **exercise.\$id: { \$in: [...] }** and sorts by **displayOrder** — one round trip instead of N. It's the same fix as a JPA **JOIN FETCH**, just expressed as a pipeline."

★ This is your **best** N+1 answer because it is real, it is yours, and it shows the concept transfers across databases.

▲ Three N+1 gotchas worth naming

JOIN FETCH + pagination

```

join fetch on a COLLECTION + Pageable
    ↓
HHH000104: firstResult/maxResults specified with collection fetch;
           applying in memory
    ↓
Hibernate loads EVERY row and paginates in Java ▲

```

Fix: use **@EntityGraph** with **@BatchSize**, or do it in two queries (page the ids first, then fetch by those ids).

MultipleBagFetchException

Fetch-joining TWO `List` collections at once
→ MultipleBagFetchException: cannot simultaneously fetch multiple bags ▲

Fix: make them `Set`, or fetch one per query.

Why **distinct** in a **JOIN FETCH** ?

A join to a collection MULTIPLIES the parent rows:

user 1 × 3 orders → 3 rows → 3 duplicate User objects in the result ▲

select DISTINCT u → de-duplicates the OBJECT references ✓

(Hibernate 6 de-duplicates entity results automatically, so **distinct** is no longer required there — but say you know both.)

Q: What is **LazyInitializationException** ?

Accessing a LAZY association AFTER the persistence context has CLOSED. ▲

```
@Transactional
public User getUser(Long id) { return
    userRepository.findById(id).orElseThrow(); }

// in the controller – the transaction has already committed
user.getOrders().size(); // x could not initialize proxy – no Session
```

INSIDE @Transactional → the session is OPEN → the proxy can load ✓
after the method ends → the session is CLOSED → the proxy is helpless ▲

Real-world idea: **the kitchen closes at 10 pm**. Order at 9:59 and you're fine.
Order at 10:01 and the proxy has nobody to ask.

Fixes, best first:

1. RETURN A DTO from the service – the mapping happens INSIDE the transaction ✓ best
2. Fetch what you need with JOIN FETCH / @EntityGraph
3. Touch it inside the transactional method:
`Hibernate.initialize(user.getOrders())`
4. `spring.jpa.open-in-view=true` – ON BY DEFAULT in Spring Boot ▲

★ About that fourth one:

`open-in-view` keeps the session open for the WHOLE HTTP request.

↓

It HIDES the problem – and pushes N+1 queries into the VIEW layer, while holding a database connection for the entire request ▲

Best practice: `spring.jpa.open-in-view=false`, and fix the fetching properly.

■ "Boot logs a warning about *open-in-view* at startup for a reason — it hides lazy loading problems until production. I'd turn it off and return DTOs from the service layer."

Easy memory

"To-One is EAGER, To-Many is LAZY" ★ → always override @ManyToOne to LAZY
Lazy works via PROXIES → which is why entities can't be final ★

N+1 = 1 + N queries (going to the market 101 times) ▲

FIX: JOIN FETCH (use distinct) | @EntityGraph ★ | @BatchSize | DTO projection

▲ JOIN FETCH + Pageable → paginates IN MEMORY (HHH000104)

▲ Two List fetch joins → MultipleBagFetchException → use Set

LazyInitializationException = touching a lazy field after the session closed

(the kitchen closed at 10pm)

Fix: return a DTO from the service ★ | `open-in-view=false`

H — Transactions

Q: What does `@Transactional` do?

Spring wraps the bean in a PROXY that:
begins a transaction before the method
COMMITs on a normal return
ROLLS BACK on a runtime exception

```
@Transactional
public void transfer(Long from, Long to, BigDecimal amount) {
    Account a = repo.findById(from).orElseThrow();
    Account b = repo.findById(to).orElseThrow();
    a.debit(amount);
    b.credit(amount);           // both UPDATES flush at commit – all or
    nothing ✓
}
```

Without it: debit succeeds -1000 → credit CRASHES ▲ → the money VANISHES
With it: debit succeeds → credit crashes → EVERYTHING rolls back ✓

Real-world idea: **a bank transfer**. It cannot be half done.

Q: Propagation levels?

Propagation	Behaviour
REQUIRED (default) ★	join the existing transaction, or start one
REQUIRES_NEW ★	suspend the current one; always start a new independent transaction
SUPPORTS	join if one exists, otherwise run non-transactionally
NOT_SUPPORTED	suspend any transaction and run without one
MANDATORY	must already be in a transaction, else IllegalTransactionStateException

Propagation	Behaviour
NEVER	must not be in a transaction, else an exception
NESTED	a savepoint inside the current transaction — an inner rollback doesn't kill the outer

A (REQUIRED) calls B (REQUIRED)
rolls back too

→ SAME transaction → B fails → A

A (REQUIRED) calls B (REQUIRES_NEW)
rolls back ★

→ SEPARATE → B commits even if A

★ **The classic use of REQUIRES_NEW** : audit or log writes that must survive even when the business transaction rolls back.

Q: Isolation levels?

The easiest way to remember by **what each one prevents**:

Isolation	Prevents
READ_UNCOMMITTED	nothing (dirty reads possible)
READ_COMMITTED	dirty reads (<i>Postgres/Oracle default</i>)
REPEATABLE_READ	dirty + non-repeatable reads (<i>MySQL default</i>)
SERIALIZABLE	everything, including phantom reads — slowest

The three read phenomena — explain them with one example each:

DIRTY READ

You read a value another transaction wrote but hasn't COMMITTED.
They roll back → you acted on data that never existed ▲

NON-REPEATABLE READ

You read row 5 → someone UPDATES it → you read row 5 again
→ a DIFFERENT value. The same query, two answers.

PHANTOM READ

You run "SELECT * WHERE age > 30" → 10 rows
Someone INSERTS a new matching row
You run it again → 11 rows. A new row APPEARED.

non-repeatable = an EXISTING row CHANGED

phantom = a NEW row APPEARED ★

▲ The rollback rule (asked constantly)

Spring rolls back on RuntimeException and Error ONLY.
NOT on checked exceptions. ★

```
@Transactional
public void save() throws IOException {
    repo.save(x);
    throw new IOException();           // ✗ CHECKED → NO ROLLBACK → it
COMMITTS ▲
}
```

```
@Transactional(rollbackFor = Exception.class) // ✓ the fix
public void save() throws IOException { ... }
```

★ And the second half of the trap:

```

@Transactional
public void m() {
    try {
        somethingThatThrows();
    } catch (Exception e) {
        log.error("failed", e);    // ✗ you SWALLOWED it → Spring never
    }                             //      → the transaction COMMITS ▲
}

```

▲ The self-invocation trap (the #1 `@Transactional` bug)

```

@Service
public class OrderService {

    public void placeOrder() {
        saveOrder();    //
    }                  ✗ an internal call – bypasses the proxy, NO transaction ▲

    @Transactional
    public void saveOrder() { ... }
}

```

```

External call: caller → PROXY → placeOrder()    ✓ the proxy is
involved
Internal call: placeOrder() → this.saveOrder()  ✗ never leaves the
object
                                                    → the proxy never
sees it ▲

```

Same reason `@Transactional` on a `private`, `final` or `static` method does nothing at all.

Fixes: move the method to another bean, self-inject, or use `AopContext.currentProxy()`.

Q: @Transactional(readOnly = true) — what does it actually do?

1. Sets FlushMode.MANUAL → Hibernate SKIPS dirty checking
→ less memory, faster (no snapshots kept) ▲
2. Hints the driver/DB that the transaction is read-only

Use it on every read-only service method.

- ▲ And it means ANY modification silently does NOT persist.
No exception. No warning. The UPDATE simply never happens ▲

Easy memory

@Transactional = a PROXY: begin → commit on return → rollback on RuntimeException

REQUIRED ★ default (join) | REQUIRES_NEW ★ (independent – audit logs survive)

Isolation: READ_COMMITTED (Postgres) | REPEATABLE_READ (MySQL)
non-repeatable = a row CHANGED | phantom = a NEW row APPEARED

- ▲ CHECKED exceptions do NOT roll back → rollbackFor = Exception.class
- ▲ CATCHING the exception also prevents rollback ▲
- ▲ SELF-INVOCATION (this.method()) → no transaction at all ▲
- ▲ readOnly = true → writes SILENTLY vanish ▲

I — Caching

The easiest way to remember:

- L1 (first level) → per TRANSACTION → ALWAYS ON, can't be disabled ★
- L2 (second level) → per APPLICATION → OFF by default, opt-in
- Query cache → caches query → IDS → needs L2

	First-level (L1)	Second-level (L2)	Query cache
Scope	EntityManager / transaction	app-wide, shared across sessions	query + parameters → ids

	First-level (L1)	Second-level (L2)	Query cache
On by default	✓ always on, cannot be disabled ★	✗ off	✗ off
Provider	built in	EHCache, Caffeine, Infinispan, Hazelcast	needs L2 enabled

```
@Entity
@Cacheable
@org.hibernate.annotations.Cache(usage =
CacheConcurrencyStrategy.READ_WRITE)
public class Country { ... }
```

L2 is for READ-MOSTLY REFERENCE DATA: countries, currencies, config ✓
 NEVER cache hot, frequently-written tables ✗ (invalidation costs more than it saves)

Real-world idea:

L1 → your own notepad during ONE meeting (thrown away afterwards)
 L2 → the office noticeboard, shared by everyone, all day ★

Q: How do you clear the L1 cache and why would you?

```
em.clear()           → detach EVERYTHING
em.detach(entity) → detach one
```

★ The real reason: **batch jobs**.

```
for (int i = 0; i < items.size(); i++) {
    em.persist(items.get(i));
    if (i % 50 == 0) { em.flush(); em.clear(); }    // ★ or you will OOM
}
```

Without flush + clear:
 the persistence context holds ALL 100,000 entities in memory
 + a SNAPSHOT of each for dirty checking
 → OutOfMemoryError ▲

Q: Hibernate cache vs Spring's @Cacheable ?

Spring @Cacheable → caches METHOD RETURN VALUES (any bean, any type)
Hibernate L2 → caches ENTITIES and COLLECTIONS by id, inside the ORM

They are different layers and can both be on at once. RoboGebra uses `spring-boot-starter-cache` — that is the **Spring** one.

Easy memory

L1 = per transaction, ALWAYS ON ★ (your notepad)
L2 = app-wide, OPT-IN, for read-mostly reference data (the noticeboard)
Query cache = query → ids (needs L2)

BATCH JOBS: flush() + clear() every 50 rows, or you OOM ★
Spring @Cacheable = method return values ≠ Hibernate L2 = entities by id

J — Concurrency & locking

Q: Optimistic vs pessimistic locking?

The easiest way to remember:

OPTIMISTIC → "conflicts are RARE" → no lock; DETECT the clash at commit
★
PESSIMISTIC → "conflicts are LIKELY" → LOCK the row up front

	Optimistic	Pessimistic
Assumes	conflicts are rare	conflicts are likely
Mechanism	a <code>@Version</code> column checked on UPDATE	a DB row lock (<code>SELECT ... FOR UPDATE</code>)
Cost	no locks held — scales well ▲	holds locks; risk of deadlock and contention
On conflict	<code>OptimisticLockException</code> at commit → retry	the second transaction waits

	Optimistic	Pessimistic
Use for	web apps, long "think time" between read and write ★	short critical sections: inventory, seat booking, balances

How optimistic locking actually works

```
@Entity
public class Product {
    @Id private Long id;
    @Version private Integer version;    // ★ Hibernate increments it on
    every UPDATE
    private int stock;
}
```

```
UPDATE product SET stock = ?, version = 2 WHERE id = ? AND version = 1
                                     ↑ the guard ★
```

```
Both users load version 1
User A saves → version becomes 2 ✓
User B saves → WHERE version = 1 matches 0 ROWS
               → Hibernate sees 0 rows updated
               → OptimisticLockException ▲ → retry
```

★ Nothing was locked. The clash is simply *detected*, not prevented.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select p from Product p where p.id = :id")
Product findByIdForUpdate(@Param("id") Long id);
```

Real-world idea:

```
OPTIMISTIC → editing a shared Google Doc: everyone edits freely,
              and a conflict is flagged if two people change the same
              line ★
PESSIMISTIC → checking a library book out: nobody else can touch it
              until you return it
```

■ "For a checkout flow I'd use *@Version* optimistic locking with a retry, because the user has think time and holding a DB lock across that is unacceptable. For

decrementing stock inside a short server-side transaction, pessimistic write is simpler and correct."

Easy memory

OPTIMISTIC → @Version column → UPDATE ... WHERE version = 1
→ 0 rows updated → OptimisticLockException → RETRY ★
(Google Docs: edit freely, flag the conflict)

PESSIMISTIC → SELECT ... FOR UPDATE → others WAIT
(library book: checked out, nobody else can touch it)

Long user think time → OPTIMISTIC ★ | short server-side critical section
→ PESSIMISTIC

K — equals, hashCode, Lombok & DTO traps

Q: How should you implement `equals` / `hashCode` on an entity?

★ This is genuinely hard in JPA, and the reason is worth stating clearly:

An entity's id is NULL before it's saved, and NOT NULL afterwards.
So its hashCode CHANGES mid-life – which breaks every HashSet it's in. ▲

```
new User() → id = null → hashCode = 31 → put in a HashSet → bucket 31
save()      → id = 42   → hashCode = 42
set.contains(user) → looks in bucket 42 → NOT FOUND ▲
                                   (the object is sitting in bucket 31, invisible
forever)
```

The rules:

1. NEVER use a generated @Id in hashCode ★
2. Prefer a BUSINESS/NATURAL key (email, order number, a UUID you assign yourself)
3. No natural key? → a CONSTANT hashCode + an id-based equals
4. Use ``instanceof``, not `getClass()` – Hibernate PROXIES are subclasses, so `getClass()` comparisons fail against a proxy ★

```

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof User other)) return false;           // handles
proxies ★
    return id != null && id.equals(other.getId());
}

@Override
public int hashCode() { return getClass().hashCode(); }      // stable
across the id transition ★

```

A constant `hashCode` looks wrong, but it is correct here: it means every entity of that class lands in one bucket, and `equals` does the real work. Entity sets are almost always tiny, so the performance cost is nil — and correctness beats distribution.

▲ Lombok `@Data` on an entity — three bugs at once

```

@Data                      // ✗ generates equals/hashCode/toString over ALL
fields
@Entity
public class User {
    @OneToMany(mappedBy = "user") private List<Order> orders;
}

```

BUG 1 – equals/hashCode over ALL fields
including the id (see above) and the lazy collections ▲

BUG 2 – toString() touches EVERY field
→ triggers lazy loading (a surprise query, or
LazyInitializationException)
→ or INFINITE RECURSION on a bidirectional relationship:
User.toString() → Order.toString() → User.toString() →
StackOverflowError ▲

BUG 3 – @Setter on everything
breaks the encapsulation of your invariants

✓ Use `@Getter @Setter @NoArgsConstructor` and write `equals / hashCode / toString` yourself — or at minimum `@ToString(exclude = "orders")`.

Q: DTO vs Entity — why never return an entity from a controller?

1. LEAKS INTERNALS – password hashes, audit columns, the whole object graph
2. Serialisation TRIGGERS LAZY LOADING
 - LazyInitializationException, or an accidental N+1 during JSON writing ▲
3. COUPLES the API contract to the schema
 - a column rename becomes a BREAKING API change
4. Bidirectional relations cause INFINITE JSON RECURSION
 - the @JsonIgnore / @JsonManagedReference band-aid

✓ Map entity → DTO **inside the transactional service** (MapStruct, or by hand).

Easy memory

Entity equals/hashCode is HARD because the id is null → then not null ★
→ NEVER put a generated id in hashCode
→ constant hashCode + id-based equals
→ use `instanceof`, NOT getClass() (proxies are SUBCLASSES) ★

▲ Lombok @Data on an @Entity = 3 bugs:
equals/hashCode over all fields | toString → lazy load / StackOverflow
| setters
→ use @Getter @Setter @NoArgsConstructor instead ✓

Never return an entity from a controller → map to a DTO INSIDE the transaction ★

L — Predict the output — ORM edition (20 programs)

● **Your last interview was output-based.** These are the same *style* of question for ORM. For each: **the code** → **what happens** → **the rule** → **the follow-up**. Assume Spring Boot 3.2, Hibernate 6, `spring.jpa.show-sql=true`.

Q1. Does this UPDATE the database?

```
@Transactional
public void rename(Long id) {
    User user = userRepository.findById(id).orElseThrow();
    user.setName("Priyanka");
    // no save() call
}
```

✓ Answer

Yes — an **UPDATE** is issued at commit.

The rule

An entity loaded inside a transaction is **managed**. Hibernate snapshots it and **dirty checks** at flush — `save()` is unnecessary.

Follow-up: "And if you remove `@Transactional`?"

`findById` opens and closes its own transaction (Spring Data methods are transactional), so `user` comes back **detached** — the `setName` is lost. **No UPDATE.**

Q2. Same query twice

```
@Transactional
public void test(Long id) {
    User a = userRepository.findById(id).orElseThrow();
    User b = userRepository.findById(id).orElseThrow();
    System.out.println(a == b);
    System.out.println("queries: check the log");
}
```

✓ Answer

```
true
```

and **only ONE SELECT** appears in the log.

The rule

The persistence context is an **identity map** and the **first-level cache**: within one transaction, one row = one object instance. The second lookup is served from L1.

Follow-up: "What if the two calls are in different transactions?"

Two SELECTs, and `a == b` is **false** — different persistence contexts.

Q3. `merge` returns a different object

```
@Transactional
public void update(User detachedUser) {           // came from the request
    body, has an id
        detachedUser.setName("New Name");
        userRepository.save(detachedUser);
        System.out.println(detachedUser.getName());
        // ...later code uses detachedUser
}
```

✓ Answer

Prints **New Name**, and the DB **is** updated — but `detachedUser` itself is **still detached**. Any change you make to it *after* the `save()` line is **not** persisted.

The rule

`save()` on an entity with an id delegates to `merge()`, which copies the state onto a **different, managed instance** and returns it. **Always use the returned value:**

```
User managed = userRepository.save(detachedUser); // ✓
managed.setStatus(ACTIVE);                       // this WILL persist
```

Follow-up: "How many SQL statements does that `save()` produce?"

Two — a `SELECT` (to load the current row into the context) then an `UPDATE` .
`persist()` would be a single `INSERT` .

Q4. Lazy access outside the transaction

```
@Transactional
public User getUser(Long id) {
    return userRepository.findById(id).orElseThrow();
}

// controller
User user = service.getUser(1L);
System.out.println(user.getOrders().size());
```

✓ Answer

LazyInitializationException: could not initialize proxy – no Session
...unless `spring.jpa.open-in-view=true` (Boot's default), in which case it
silently works — and fires an extra query in the view layer.

The rule

A lazy proxy needs an **open** persistence context. The transaction ended when the
service method returned.

Follow-up: "Three ways to fix it?"

Return a DTO / `JOIN FETCH` or `@EntityGraph` / initialize inside the transaction.
(Not "make it EAGER" — that's the answer they're hoping you don't give.)

Q5. Adding to the inverse side only

```
@Transactional
public void addOrder(Long userId) {
    User user = userRepository.findById(userId).orElseThrow();
    Order order = new Order("ORD-1");
    user.getOrders().add(order);        // orders is mappedBy = "user"
    // order.setUser(user) ← missing
}
```

✓ Answer

With `cascade = ALL`: the order **is inserted**, but with `user_id = NULL`.

Without cascade: **nothing happens at all** (or a `TransientObjectException` on flush).

The rule

Only the **owning side** (`@ManyToOne`, the FK holder) is used to write the relationship. `mappedBy` marks the inverse side, which Hibernate ignores when generating SQL.

Follow-up: "How do you prevent this class of bug?"

Bidirectional helper methods (`addOrder` / `removeOrder`) that always set both sides.

Q6. Self-invocation

```
@Service
public class UserService {
    public void register(User u) {
        save(u);                // internal call
    }
    @Transactional
    public void save(User u) { repo.save(u); throw new
RuntimeException("boom"); }
}
```


✓ Answer

The exception propagates — **but there is no rollback of `save`**, because `save()` ran **without a transaction** at all.

The rule

`@Transactional` is proxy-based AOP. Calls from **inside** the same object never pass through the proxy. Same for `private`, `final` and `static` methods.

Follow-up: "How do you fix it?"

Move `save` into a separate bean, or self-inject the proxy, or `AopContext.currentProxy()`.

Q7. Write inside a read-only transaction

```
@Transactional(readOnly = true)
public void deactivate(Long id) {
    User user = userRepository.findById(id).orElseThrow();
    user.setActive(false);
}
```

✓ Answer

Nothing is written. No exception, no UPDATE — the change is silently lost.

The rule

`readOnly = true` sets Hibernate's flush mode to **MANUAL**, so dirty checking never flushes. (*On some databases/drivers you'd instead get a "cannot execute UPDATE in a read-only transaction" error — but the classic Hibernate answer is silent no-op.*)

Follow-up: "Then why use `readOnly` at all?"

Skipping dirty checking saves memory and CPU on read paths, and it lets the driver route to a read replica.

Q8. Checked exception and rollback

```
@Transactional
public void process() throws IOException {
    repo.save(new Order("ORD-1"));
    throw new IOException("file missing");
}
```

✓ Answer

The order **is committed**. Spring rolls back on `RuntimeException` / `Error` only.

The rule

Default rollback rule = unchecked exceptions. Add `@Transactional(rollbackFor = Exception.class)`.

Follow-up: "And if you catch the exception inside the method?"

No rollback either — Spring never sees it. (And if the transaction was already marked rollback-only by an inner call, the commit throws

`UnexpectedRollbackException`.)

Q9. Entity in a `HashSet` before it's saved

```
@Entity
class User {
    @Id @GeneratedValue Long id;
    @Override public int hashCode() { return Objects.hash(id); }
    @Override public boolean equals(Object o) { ... id-based ... }
}

Set<User> set = new HashSet<>();
User u = new User();
set.add(u); // id == null → hashCode based on null
userRepository.save(u); // id becomes 1
System.out.println(set.contains(u));
```

✓ Answer

```
false
```

The rule

`HashSet` buckets by `hashCode` **at insertion time**. Persisting changes the id, so the hash changes and the object lands in a different bucket — it's lost. **Never use a generated id in `hashCode`**.

Follow-up: "What do you use instead?"

A business key, or a **constant** `hashCode` (`getClass().hashCode()`) with id-based `equals`.

Q10. Bulk update and the stale entity

```
@Transactional
public void run(Long id) {
    User user = userRepository.findById(id).orElseThrow(); // status =
    ACTIVE
    userRepository.deactivateAll(); // @Modifying
    UPDATE user SET status='OFF'
    System.out.println(user.getStatus());
}
```

✓ Answer

```
ACTIVE
```

— the **stale** value, even though the DB now says `OFF`.

The rule

`@Modifying` bulk queries go **straight to the database** and bypass the persistence context. The already-loaded entity is not refreshed.

Follow-up: "Fix?"

```
@Modifying(clearAutomatically = true, flushAutomatically = true), or  
em.refresh(user) .
```

Q11. @Modifying missing

```
@Query("update User u set u.status = 'OFF' where u.id = :id")  
void deactivate(@Param("id") Long id);
```

✓ Answer

Runtime exception: **Not supported for DML operations** /
InvalidDataAccessApiUsageException .

The rule

Any JPQL **update** / **delete** in a Spring Data repository needs **@Modifying** (and a transaction).

Q12. getReference on a missing row

```
@Transactional  
public void test() {  
    User u = em.getReference(User.class, 999L);    // no such row  
    System.out.println("got proxy");  
    System.out.println(u.getName());  
}
```

✓ Answer

```
got proxy
```

then **EntityNotFoundException** .

The rule

`getReference` returns an **uninitialized proxy** — no SQL yet. The database is hit on the first non-id property access, and that's where it fails. `find()` would have returned `null` immediately.

Q13. `JOIN FETCH` duplicates

```
@Query("select u from User u join fetch u.orders")
List<User> findAllWithOrders();
// 1 user with 3 orders
System.out.println(findAllWithOrders().size());
```

✓ Answer

Hibernate 5: `3` (the SQL join produces 3 rows → 3 references to the same `User`).

Hibernate 6 / Boot 3: `1` — entity results are de-duplicated automatically.

The rule

A collection fetch join multiplies parent rows. Pre-Hibernate-6 you needed `select distinct u`. **Know both** — the interviewer's mental model is probably Hibernate 5.

Q14. Two `List` fetch joins

```
@Query("select u from User u join fetch u.orders join fetch u.addresses")
List<User> findAll();           // both are List
```

✓ Answer

MultipleBagFetchException: cannot simultaneously fetch multiple bags at startup.

The rule

Two unordered `List`s ("bags") can't be fetch-joined in one query — the Cartesian product is ambiguous. **Fix:** make them `Set`, or fetch one and use `@BatchSize` for the other.

Q15. `remove(int)` vs the collection you thought you had

```
@Transactional
public void deleteOrder(Long userId, Order order) {
    User user = userRepository.findById(userId).orElseThrow();
    user.getOrders().remove(order);    // @OneToMany(mappedBy="user",
    cascade=ALL) - no orphanRemoval
}
```

✓ Answer

No **DELETE**. The row stays with its `user_id` intact (or, if the FK is non-null and Hibernate nulls it, you get a constraint violation).

The rule

`CascadeType.REMOVE` only cascades when the **parent** is removed. Removing a child *from the collection* needs `orphanRemoval = true`.

Q16. `equals` against a proxy

```
User a = em.find(User.class, 1L);
User b = em.getReference(User.class, 1L);
System.out.println(a.getClass() == b.getClass());
System.out.println(a.equals(b));
```

✓ Answer

```
false
```

then — with a `getClass()` -based `equals` — `false` (wrong!). With an `instanceof` -based `equals` — `true` ✓.

The rule

A lazy proxy is a **generated subclass** (`User$HibernateProxy$xyz`), so `getClass()` never matches. Write `equals` with `instanceof`, or use `Hibernate.getClass(o)`.

Q17. Enum ordinal drift

```
public enum Status { ACTIVE, INACTIVE }           // stored with default
@Enumerated → ORDINAL
// later someone changes it to:
public enum Status { PENDING, ACTIVE, INACTIVE }
```

✓ Answer

Every existing row silently shifts meaning: old `0` (`ACTIVE`) now reads as `PENDING`. No error, corrupted data.

The rule

`@Enumerated` 's default is `ORDINAL`. Always write `@Enumerated(EnumType.STRING)`.

Q18. Deleting a parent with 1000 children

```
@Transactional
public void deleteUser(Long id) {
    userRepository.deleteById(id);    // @OneToMany(cascade = ALL) with
    1000 orders
}
```

✓ Answer

1002 statements — 1 SELECT for the user, 1 SELECT for the orders, then **1000 individual DELETES**.

The rule

JPA cascade removal is **entity-by-entity** (it must run lifecycle callbacks). For bulk deletes use `@Modifying @Query("delete from Order o where o.user.id = :id")` or an `ON DELETE CASCADE FK` — and remember those bypass the persistence context.

Q19. `@Transactional` on a `private` method

```
@Service
public class ReportService {
    @Transactional
    private void generate() { ... }
}
```

✓ Answer

No transaction. It compiles, it runs, the annotation is silently ignored. (*Spring cannot proxy a private method; with class-based CGLIB proxies the same is true for `final` and `static`.*)

The rule

Proxy-based AOP only intercepts **public, non-final, externally called** methods.

Q20. Which entity state?

```
@Transactional
public void states() {
    User u = new User("a@b.com");           // (1)
    em.persist(u);                           // (2)
    em.flush();
    em.detach(u);                             // (3)
    u.setName("changed");
    em.merge(u);                              // (4)
    em.remove(em.find(User.class, u.getId())); // (5)
}
```

✓ **Answer**

Point	State of u
(1)	Transient
(2)	Managed (INSERT queued)
(3)	Detached — setName is not tracked
(4)	u is still detached ; the <i>returned</i> copy is managed and carries "changed"
(5)	The managed instance is Removed — DELETE at flush

The rule

merge never makes the argument managed. Memorize the state diagram in [section E](#).

M — ODM side — what you actually use (MongoDB & Mongoose)

This is **your** ground. Be fluent here — it's the honest counterweight to the JPA section.

Spring Data MongoDB — mapping (RoboGebra)

```
@Document(collection = "quiz_definition")
public class QuizDefinition {

    @Id
    private String id;                // maps to _id (ObjectId as
String)

    @Field("quiz_title")
    private String title;

    @Indexed(unique = true)
    private String slug;

    private List<Question> questions;    // ✓ EMBEDDED — no join
needed

    @DBRef(lazy = true)                //
    ▲ reference — extra query per document
    private Course course;

    @CreatedDate private Instant createdAt;
    @LastModifiedDate private Instant updatedAt;

    @Version private Long version;      // optimistic locking works
in Mongo too
}
```

```

public interface QuizRepository extends MongoRepository<QuizDefinition,
String> {

    List<QuizDefinition> findByQuizDefinition_IdOrderByOrderAsc(String
id);    // derived query

    @Query("{ 'status': ?0, 'score': { $gte: ?1 } }")
    List<QuizDefinition> findByStatusAndMinScore(String status, int
score);

    @Aggregation(pipeline = {
        "{ $match: { courseId: ?0 } }",
        "{ $lookup: { from: 'question', localField: '_id', foreignField:
'quizId', as: 'questions' } }",
        "{ $sort: { order: 1 } }"
    })
    List<QuizWithQuestions> findQuizzesWithQuestions(String courseId);
}

```

▲ The three differences that catch people out

	JPA / Hibernate	Spring Data MongoDB
Dirty checking	Automatic — change a managed entity, it updates	✗ None. You must call <code>save()</code> explicitly
Persistence context / L1 cache	Yes	✗ None. Two <code>findById</code> calls = two queries, two objects
Lazy loading	Proxies on every association	Only <code>@DBRef(lazy = true)</code> ; the model answer is to embed instead

■ **Killer line:** *"The repository programming model is identical, but the engine isn't. In Mongo there's no persistence context — no dirty checking, no L1 cache — so I always call `save()` explicitly. And instead of solving N+1 with fetch joins, I solve it at the schema level by embedding, or with a `$lookup` in an `@Aggregation`."*

Q: **MongoRepository** vs **MongoTemplate** ?

MongoRepository = declarative, derived queries, great for CRUD. **MongoTemplate** = imperative, full control — dynamic criteria, partial updates (**\$set** on one field instead of rewriting the whole document), bulk operations.

Q: How do you avoid the N+1 equivalent in MongoDB?

1. **Embed** the child documents (the default answer — one read, one document).
2. **\$lookup** in an aggregation pipeline (a server-side join).
3. Fetch the ids and do **one** **findAllById** batch query — never a loop of **findById**.

▲ **@DBRef** is the N+1 machine of MongoDB — each reference is a separate round trip. RoboGebra's **@Aggregation** pipelines exist precisely to avoid that.

Mongoose (RoboGebra CRM — Node)

```
const couponSchema = new mongoose.Schema({
  code:      { type: String, required: true, unique: true, uppercase:
true },
  discount:  { type: Number, min: 0, max: 100 },
  customer:  { type: mongoose.Schema.Types.ObjectId, ref:
'Customer' }, // reference
  expiresAt: Date,
}, { timestamps: true }); // createdAt / updatedAt

couponSchema.pre('save', function (next) { // middleware / hook
  this.code = this.code.trim();
  next();
});

couponSchema.index({ code: 1, expiresAt: -1 });

const Coupon = mongoose.model('Coupon', couponSchema);

// "join" — populate resolves the reference (an extra query, like @DBRef)
const coupons = await Coupon.find({ active: true })
  .populate('customer', 'name email')
  .lean(); // lean() = plain JS objects,
no Mongoose docs → much faster reads
```

Mongoose concept	JPA equivalent
Schema	@Entity mapping metadata
Model	Repository/ EntityManager
populate()	JOIN FETCH (but as an extra query)
pre / post hooks	@PrePersist / @PostLoad entity listeners
lean()	DTO projection (skip the managed-object overhead)
timestamps: true	@CreatedDate / @LastModifiedDate
Schema validators	Bean Validation (@NotNull , @Size)

■ "So across the stack I've used two ODMs — Spring Data MongoDB on the Java side and Mongoose on the Node CRM — and the concepts map cleanly onto JPA: schema ↔ entity, model ↔ repository, populate ↔ fetch join, hooks ↔ entity listeners."

Q: ORM vs ODM — the summary answer

"An ORM maps objects to rows across normalized tables, so its hard problems are joins, lazy loading and N+1. An ODM maps objects to documents, so related data is usually embedded and read in a single round trip — the hard problems move to schema design, duplication and keeping denormalized copies consistent. JPA gives you a persistence context with dirty checking; Spring Data MongoDB doesn't, so writes are always explicit."

N — Schema management & migrations

Q: `spring.jpa.hibernate.ddl-auto` values?

Value	What it does	Use in
<code>none</code>	Nothing	✓ Production
<code>validate</code>	Verifies the schema matches the entities, fails fast otherwise	✓ Production (with Flyway)
<code>update</code>	Adds missing tables/ columns; never drops or alters	Local only ▲
<code>create</code>	Drops and recreates at startup	Local/tests
<code>create-drop</code>	<code>create</code> , plus drop on shutdown	Tests

▲ **Never update in production** — it can't rename, drop, change a type, or apply a data migration, and it silently drifts from what you think the schema is.

Q: Flyway vs Liquibase?

Both are **versioned migration** tools that keep an applied-migrations table and run pending scripts at startup.

- **Flyway** — plain SQL files (`V1_create_user.sql`), simple, DB-specific. Most common.

- **Liquibase** — changelogs in XML/YAML/JSON/SQL, database-agnostic, supports rollback.

✓ **Production setup:** `ddl-auto: validate` + Flyway. Hibernate proves the entities match; Flyway owns the schema.

O — Performance checklist

The 12 things that actually make an ORM app slow:

1. **N+1 queries** — the single biggest cause. Fix with `@EntityGraph` / `JOIN FETCH` / `@BatchSize`.
2. **EAGER -ToOne associations** — set everything to `LAZY`.
3. **`open-in-view=true`** — turn it off; it hides N+1 in the view layer and holds connections.
4. **Loading entities for read-only screens** — use DTO projections instead.
5. **Missing `@Transactional(readOnly = true)`** on reads — you're paying for dirty checking you don't need.
6. **`IDENTITY` ids** — disables JDBC batching; use `SEQUENCE` with `allocationSize`.
7. **No batching configured** — `hibernate.jdbc.batch_size=50`, `order_inserts=true`, `order_updates=true`.
8. **Cascade deletes over large collections** — 1000 DELETES; use a bulk query.
9. **Never clearing the persistence context in batch jobs** — `flush()` + `clear()` every N rows.
10. **Missing DB indexes** on FK and filter columns — ORM won't save you here.
11. **`select *` on wide tables with `@Lob` columns** — project only what you need.
12. **Not looking at the SQL** — turn on `show-sql` / `p6spy` / Hibernate statistics in dev.

```
spring.jpa.open-in-view=false
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.jdbc.batch_size=50
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
spring.jpa.properties.hibernate.default_batch_fetch_size=25
logging.level.org.hibernate.SQL=DEBUG
```

P — Rapid-fire answer table

Question	One-line answer
ORM?	Maps Java objects to relational tables so you work with objects, not SQL
JPA vs Hibernate?	Spec vs implementation. Spring Data JPA is an abstraction on top of both
Persistence context?	The set of managed entities for a transaction — L1 cache + dirty checking + identity map
Entity states?	Transient → Managed → Detached / Removed
Dirty checking?	Hibernate compares an entity to its load-time snapshot at flush and auto-generates the UPDATE
<code>persist</code> vs <code>merge</code> ?	<code>persist</code> = insert a transient entity, makes the argument managed. <code>merge</code> = copy a detached entity's state onto a different managed instance and return it
What does <code>save()</code> do?	<code>isNew()</code> ? <code>persist</code> : <code>merge</code>
Flush vs commit?	Flush writes SQL; commit ends the transaction. Flush ≠ commit
Default fetch types?	<code>-ToOne</code> = EAGER ▲, <code>-ToMany</code> = LAZY. Set everything to LAZY
N+1 problem?	1 query for parents + N for children. Fix: <code>JOIN FETCH</code> , <code>@EntityGraph</code> , <code>@BatchSize</code> , DTO projection
<code>LazyInitializationException</code> ?	Touching a lazy proxy after the session closed. Fix: DTO from the service, or fetch it eagerly for that query
Owning side?	The side holding the FK (<code>@ManyToOne</code>). <code>mappedBy</code> marks the inverse side, which is ignored on write

Question	One-line answer
Cascade default?	None
<code>REMOVE</code> vs <code>orphanRemoval</code> ?	<code>REMOVE</code> = when the parent is deleted. <code>orphanRemoval</code> = also when the child leaves the collection
L1 vs L2 cache?	L1 = per-EntityManager, always on. L2 = app-wide, off by default, needs a provider
<code>@Transactional</code> default rollback?	<code>RuntimeException</code> + <code>Error</code> only — not checked exceptions
Default propagation?	REQUIRED
Why doesn't <code>@Transactional</code> work?	Self-invocation, or a <code>private</code> / <code>final</code> / <code>static</code> method — the proxy is bypassed
<code>readOnly = true</code> ?	Flush mode MANUAL → no dirty checking → writes silently don't persist
Optimistic vs pessimistic?	<code>@Version</code> check at UPDATE vs a DB row lock
<code>@Enumerated</code> default?	ORDINAL — always override with STRING
<code>ddl-auto</code> in production?	<code>validate</code> (or <code>none</code>) + Flyway. Never <code>update</code>
Lombok <code>@Data</code> on an entity?	Don't — broken <code>equals</code> / <code>hashCode</code> , and <code>toString</code> triggers lazy loads / infinite recursion
ORM vs ODM?	Objects↔ tables (joins, lazy loading) vs objects↔ documents (embedding, aggregation). No persistence context in Mongo
Do you use ORM at work?	"ODM — <i>Spring Data MongoDB on RoboGebra. I know JPA concepts; my production mapping work is document-based.</i> "

Q — 12 self-test drills (answers hidden)

1. You load a `User` in a `@Transactional` method, change a field, and never call `save()`. Is the DB updated?

→ Answer

****Yes**** — dirty checking issues the UPDATE at commit. Without `@Transactional` on your method, the entity is detached and ****nothing**** happens.

2. `em.persist(user)` where `user` already has an id set and the row exists. What happens?

→ Answer

`EntityExistsException` (or a `PersistenceException` at flush). Use `merge()` for detached entities.

3. How many SELECTs does `userRepository.findById(1L)` twice in one transaction produce?

→ Answer

****One.**** The second is served by the first-level cache, and the two references are `==` equal.

4. What's wrong with `@OneToMany(fetch = EAGER) private List<Order> orders;` on a user with 50 000 orders?

→ Answer

Every single `findById(user)` loads 50 000 rows into memory. And eager collections can't be paginated. Use LAZY + an explicit fetch when needed.

5. `@Query("select u from User u join fetch u.orders")` with `Pageable`. What does Hibernate log?

→ Answer

`HHH000104: firstResult/maxResults specified with collection fetch; applying in memory` — it loads ****every**** row and paginates in Java. Use `@EntityGraph` + `@BatchSize`, or page ids first.

6. Name the four entity states and one way into each.

→ Answer

Transient (new), **Managed** (persist/find/merge), **Detached** (transaction ends / detach / clear), **Removed** (remove).

7. `@Transactional` method throws `IOException`. Rollback?

→ Answer

No. Only `RuntimeException/Error` roll back by default. Add `rollbackFor = Exception.class`.

8. Why must a JPA entity not be `final`?

→ Answer

Hibernate creates **proxy subclasses** for lazy loading. A final class (or final getter) can't be subclassed/overridden, so lazy loading silently degrades or fails.

9. You remove a child from `parent.getChildren()` and nothing is deleted. Why?

→ Answer

`orphanRemoval = true` is missing. `CascadeType.REMOVE` only fires when the **parent** is deleted.

10. Does Spring Data MongoDB do dirty checking?

→ Answer

No. There is no persistence context. You must call `save()` explicitly — this is the biggest behavioural difference from JPA and a great thing to volunteer in an interview.

11. What's the MongoDB equivalent of the N+1 problem and how do you avoid it?

→ Answer

A loop of `findById`, or `@DBRef` resolution — one round trip per reference. Avoid it by **embedding**, by `$lookup` in an `@Aggregation`, or by one batched `findAllById`.

12. Interviewer: *"So you've used Hibernate in production?"* — your answer?

→ Answer

"Not Hibernate — my production persistence is **Spring Data MongoDB**, which is an ODM. I'm strong on the JPA concepts: entity lifecycle, persistence context, relationship mapping, lazy loading and N+1, `@Transactional`. The repository model is the same, so the transition is short."**Honest + specific + confident.**
Never bluff this one — the follow-up is always "show me an entity."

R — Answer scripts (say these out loud)

1. "Explain ORM in your own words." (45 s)

"ORM maps Java classes to database tables — class to table, field to column, object reference to foreign key — so instead of writing JDBC and converting `ResultSet` rows by hand, I work with objects and the framework generates the SQL. It solves the object-relational impedance mismatch: objects have inheritance, references and identity; tables have rows, foreign keys and primary keys. The trade-off is that it hides the SQL, so you have to know what it's generating — most ORM performance problems are N+1 queries that nobody looked at."

2. "JPA vs Hibernate vs Spring Data JPA." (30 s)

"JPA is the specification — the API and annotations. Hibernate is the implementation that generates the SQL, and it's Spring Boot's default provider. Spring Data JPA sits on top and removes the DAO boilerplate: I declare a repository interface and Spring generates the implementation. So it's my repository → Spring Data JPA → JPA → Hibernate → JDBC → the database."

3. "What's the persistence context?" (40 s)

"It's the set of entities the `EntityManager` is managing for the current transaction. It acts as a first-level cache, so within one transaction the same row always gives me the same object instance. It's also where dirty checking happens — Hibernate keeps a snapshot at load time and, at flush, compares it and generates the UPDATE. That's why I don't need to call `save()` on an entity I loaded inside a transaction."

4. "Tell me about the N+1 problem." (45 s)

"It's when one query loads N parents and then each parent's lazy association triggers its own query — 100 users becomes 101 queries. I usually catch it by turning on SQL logging and watching the query count on a list endpoint. The fixes, in the order I'd reach for them: an `@EntityGraph` on the repository method

because it composes with derived queries and pagination; a `JOIN FETCH` for a one-off query; `@BatchSize` to turn N queries into N -over-25 using `IN` clauses; or a DTO projection when the screen is read-only and doesn't need entities at all."

5. "Have you used ORM on your current project?" (35 s — the honest one)

"RoboGebra is MongoDB, so it's an ODM rather than an ORM — Spring Data MongoDB with `@Document` models, `MongoRepository` derived queries, and `@Aggregation` pipelines where I need join-style reads without the round trips that `@DBRef` would cost. The repository programming model is identical to Spring Data JPA; what's different underneath is that there's no persistence context, so no dirty checking and no lazy loading — every write is an explicit `save()`. On the Node CRM side it's Mongoose, which is the same idea: schemas, models, `populate` for references, and pre-save hooks."

6. "How would you design the persistence layer for a new module?" (60 s)

"I'd start from the read patterns rather than the tables. Entities with all associations LAZY, `open-in-view` off, and DTOs returned from the service layer so nothing lazy escapes the transaction. `@Transactional(readOnly = true)` on reads and a normal `@Transactional` on writes, with the transaction boundary at the service, never the controller. Sequence-based ids so JDBC batching works, `ddl-auto=validate` with Flyway owning the schema, and indexes on every foreign key and filter column. Then I'd watch the SQL log on the main list endpoints — the first thing that goes wrong in any ORM codebase is $N+1$."

✓ Before the interview, make sure you can

- [] Say the **JPA vs Hibernate vs Spring Data JPA** answer in 30 seconds without hesitating
- [] Draw the **four entity states** and name a transition into each

- [] Explain **dirty checking** and why `save()` is unnecessary inside a transaction
 - [] Explain **persist vs merge vs save** — including that `merge` returns a *different* object
 - [] Explain the **N+1 problem** and give **four** fixes in preference order
 - [] Explain **LazyInitializationException** and why `open-in-view=false` is the right default
 - [] Name the **owning side** rule and why `mappedBy` collections don't write to the DB
 - [] State the **default fetch types** (`-ToOne` = EAGER ▲) and the **default rollback rule** (unchecked only)
 - [] Explain the **@Transactional self-invocation** trap
 - [] Say the **honest RoboGebra = ODM** answer without sounding apologetic
 - [] Explain **ORM vs ODM** in two sentences
 - [] Answer at least 15 of the 20 output questions in [section L](#) cold
-
-

Quick Revision Sheet — read this the night before

YOUR HONEST LINE

RoboGebra = MongoDB = ODM (not ORM). Say it first, then pivot to the CONCEPTS ★

"Spring Data MongoDB is an ODM; I'm solid on JPA/Hibernate concepts."

LAYERING

repository → Spring Data JPA → JPA API → Hibernate → JDBC → DB ★

JPA = spec | Hibernate = implementation | Spring Data = chauffeur

Boot 3 → jakarta.*, not javax.* ▲

ENTITY

@Entity + @Id + NO-ARG CONSTRUCTOR + not final

★ (so a record can never be one)

@Enumerated: the DEFAULT is ORDINAL → stores the POSITION → ALWAYS use

STRING ★

IDENTITY (MySQL, no batching ▲) | SEQUENCE (Postgres, batch-friendly

★)

@MappedSuperclass = a BaseEntity with id/createdAt, no table of its own

PERSISTENCE CONTEXT ★ (the heart of it)

= L1 cache + dirty checking + identity map (a shopping trolley)

4 states: TRANSIENT → MANAGED → DETACHED / REMOVED

MANAGED → changes auto-saved (dirty checking – the hotel minibar), no save() needed

DETACHED → changes go NOWHERE ▲

The question behind every tricky JPA question: "is it MANAGED right now?" ★

persist() → NEW → the same object becomes managed

merge() → DETACHED → returns a DIFFERENT copy → USE THE RETURN VALUE

★ ★

save() → id null ? persist : merge

flush = send the SQL | commit = make it permanent

find() → SQL now | getReference() → lazy proxy, no SQL ★

RELATIONSHIPS

@ManyToOne = the FK side = the OWNING side ★ | @OneToMany = mappedBy = the mirror

▲ #1 BUG: adding to the inverse collection saves NOTHING

→ helper methods that set BOTH sides ★

▲ @OneToMany without mappedBy → a surprise extra JOIN TABLE

No cascade by default. ALL is fine for Order-OrderItem, DANGEROUS on

@ManyToOne ▲

orphanRemoval → also deletes a child removed from the collection

FETCHING

"To-One is EAGER ▲ , To-Many is LAZY" → override @ManyToOne to LAZY ★

Lazy = proxies → which is why entities can't be final
N+1 = 1 + N queries (the market 101 times) ▲
FIX: JOIN FETCH (distinct) | @EntityGraph ★ | @BatchSize | DTO
projection
▲ JOIN FETCH + Pageable → paginates IN MEMORY | two List fetches →
MultipleBagFetch
LazyInitializationException = touching lazy data after the session
closed
→ return a DTO from the service ★ | open-in-view=false

TRANSACTIONS

A proxy: begin → commit on return → rollback on RuntimeException
REQUIRED ★ (join) | REQUIRES_NEW ★ (independent – audit logs survive)
▲ CHECKED exceptions do NOT roll back → rollbackFor = Exception.class
▲ CATCHING the exception also prevents rollback
▲ SELF-INVOCATION this.method() → NO transaction at all ▲
▲ readOnly = true → writes SILENTLY vanish ▲
Isolation: non-repeatable = a row CHANGED | phantom = a NEW row
APPEARED

CACHING

L1 = per transaction, ALWAYS ON ★ | L2 = app-wide, opt-in, reference
data only
Batch jobs: flush() + clear() every 50 rows, or you OOM ★

LOCKING

Optimistic @Version → UPDATE ... WHERE version = 1 → 0 rows → exception
→ retry ★
(Google Docs – edit freely, flag the conflict)
Pessimistic → SELECT ... FOR UPDATE → others wait (a library book)

equals/hashCode

NEVER put a generated id in hashCode (null → not null breaks HashSet) ★
constant hashCode + id-based equals | use instanceof, NOT getClass()
(proxies!)
▲ Lombok @Data on an entity = 3 bugs → use @Getter @Setter
@NoArgsConstructor
NEVER return an entity from a controller → map to a DTO inside the
transaction ★

Related files: 06 — Spring & Spring Boot · 13 — MongoDB · 18 — RoboGebra
Technical Stack · 23 — Java Predict the Output

Angular — Data Binding & Forms (Deep Dive)

◆ **Why this file exists:** binding and forms are the two things an Angular interviewer *always* drills, because they separate people who wrote real screens from people who read a tutorial. [04 — Angular](#) covers them in about 10 questions; this file goes all the way down — including `setValue` vs `patchValue`, `value` vs `getRawValue()`, `ControlValueAccessor`, and a "what happens?" section in the same style as [23](#), because your rounds are output-based.

▲ **Version note — attribute your work correctly.** RoboGebra web is **Angular 16.2**; the Ionic app is **Angular 18.2** (Ionic 8.7.5). **Angular Formly is EasyVisa, not RoboGebra.** Features below are tagged with the version that introduced them — don't claim `model()` signals on a 16.2 codebase. (See [18 — RoboGebra Technical Stack](#).)

Table of contents

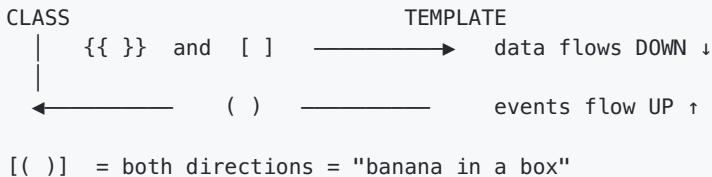
#	Section	What it covers
A	The 4 binding types	Syntax table, direction, when each is used
B	Property vs attribute	The <code>[attr.colspan]</code> question, <code>[disabled]</code> , class & style binding
C	Two-way binding	Desugaring <code>[(x)]</code> , custom two-way <code>@Input</code> / <code>@Output</code> , <code>model()</code>
D	<code>@Input</code> / <code>@Output</code>	Setters vs <code>ngOnChanges</code> , <code>required</code> , <code>transform</code> , signal inputs
E	Binding gotchas	<code>ExpressionChangedAfterItHasBeenChecked</code> , function calls in templates, <code>OnPush</code>

#	Section	What it covers
F	TD vs Reactive	The full comparison table + when you'd pick each
G	Reactive building blocks	<code>FormControl</code> , <code>FormGroup</code> , <code>FormArray</code> , <code>FormBuilder</code> , typed forms
H	Reading & writing state	<code>setValue</code> vs <code>patchValue</code> , <code>value</code> vs <code>getRawValue()</code> , <code>reset</code> , <code>updateOn</code>
I	Control state	pristine/dirty/touched/valid/pending/disabled + the ng-classes
J	Validation	Built-in, custom, cross-field , async , dynamic validators, error display
K	<code>FormArray</code> & dynamic forms	Add/remove rows, nested groups, Formly (EasyVisa)
L	<code>ControlValueAccessor</code>	The senior question — full working implementation
M	● What happens? (18 traps)	Output-style questions on binding & forms
N	Testing forms	<code>TestBed</code> , setting values, async validators
O	Rapid-fire table	1-line answers
P	12 drills	Answers hidden
Q	Answer scripts	Word-for-word answers for 6 likely questions

◆ The easiest way to remember ALL of this file

Two pictures cover binding and forms completely.

Binding — follow the ARROWS



Forms — WHERE does the truth live?

TEMPLATE-DRIVEN → the form's truth lives in the HTML → ngModel
REACTIVE ★ → the form's truth lives in the CLASS → FormGroup

The deciding line: "reactive forms are testable WITHOUT rendering the template." ★

The five traps this whole file exists to teach ★

1. [disabled]="x" on a REACTIVE control → Angular warns and it doesn't work ★
2. setValue = STRICT (throws) | patchValue = SILENT (a typo does nothing) ★
3. form.value EXCLUDES disabled controls → getRawValue() includes them ★
4. valueChanges + setValue inside it = an INFINITE LOOP ★
5. Changing a bound value too late → ExpressionChangedAfterItHasBeenChecked ★

★ If you remember nothing else from this file, remember **#3** — it silently wipes a database column.

Easy memory

Binding: {{ }} [] down ↓ | () up ↑ | [()] both

Forms: truth in the HTML (template-driven) vs truth in the CLASS (reactive ★)

THE five traps: [disabled] | setValue/patchValue | getRawValue ★ ★ valueChanges loop | ExpressionChanged...

A — Data binding: the four types

Q: What are the types of data binding in Angular?

Type	Syntax	Direction	Example
Interpolation	{{ }}	Component → View	{{ title }}
Property binding	[prop]	Component → View	
Event binding	(event)	View → Component	<button (click)="save()">
Two-way binding	[(ngModel)]	Both	<input [(ngModel)]="name">

■ **The line they want:** "Interpolation and property binding are one-way from component to view, event binding is one-way from view to component, and two-way binding is just those two combined — [(x)] desugars to [x] plus (xChange) ."

Q: Interpolation vs property binding — is there a difference?

For strings they're equivalent: ≡ . But:

- Interpolation **always stringifies** — binding an object gives [object Object] .

- Property binding can pass **non-string values** — objects, arrays, booleans, numbers. `[disabled]="false"` works; `disabled="{{false}}"` sets the attribute to the string `"false"`, which is truthy in the DOM.

✓ **Rule: use property binding for anything that isn't text.**

Q: What is `$event` ?

The payload of the event. For DOM events it's the native event object; for `@Output()` it's whatever the child emitted.

```
<input (input)="onInput($event)">                                <!-- native InputEvent
-->
<app-child (saved)="onSaved($event)"></app-child> <!-- the emitted value
-->
```

Q: What's the safe navigation operator?

`?.` — guards against `null` / `undefined` in a template so it renders nothing instead of throwing.

```
{{ user?.address?.city }}
{{ items?.length ?? 0 }}      <!-- ?? nullish coalescing, Angular 12+ -->
<div *ngIf="user as u">{{ u.name }}</div>
```

`$any(expr)` is the escape hatch that turns off template type-checking for one expression — use it rarely.

B — Property vs attribute binding

Q: What's the difference between an HTML attribute and a DOM property?

Attributes are the initial value in the HTML; properties are the current state of the DOM object. Angular binding works on **properties**, not attributes.

```
<input value="hello">
<!-- attribute stays "hello" forever; the .value PROPERTY changes as the
user types -->
```

Q: So when do you need `[attr.x]` ?

When the element has **no matching DOM property** — Angular would throw *"Can't bind to 'x' since it isn't a known property"*.

```
<td [attr.colspan]="span"></td>           <!-- colSpan property exists but
attr form is clearer -->
<button [attr.aria-label]="label">         <!-- ARIA has NO DOM property →
attr required -->
<svg><rect [attr.width]="w"></rect></svg><!-- SVG has no width property --
>
<div [attr.data-id]="id">                   <!-- data-* attributes -->
```

- "`[attr.]` is for attributes with no DOM property — ARIA, `data-*`, SVG, and `colspan`. Everything else binds to the property directly."

Setting `[attr.x]="null"` **removes** the attribute — useful for conditional ARIA.

Q: Class and style binding — all the forms

```
<!-- single class, toggled by a boolean -->
<div [class.active]="isActive">

<!-- multiple classes from an object -->
<div [ngClass]="{ active: isActive, disabled: isDisabled }">
<div [ngClass]="isActive ? 'active' : 'inactive'">
<div [ngClass]="['a', 'b']">

<!-- single style, with a unit -->
<div [style.width.px]="width">
<div [style.backgroundColor]="color">

<!-- multiple styles -->
<div [ngStyle]="{ 'font-size.px': size, color: color }">
```

	<code>[class.x]</code> / <code>[style.x]</code>	<code>ngClass</code> / <code>ngStyle</code>
What	Built-in binding syntax	Directives
Best for	One known class/style	A dynamic set
Cost	Cheaper	Object is diffed each CD cycle

✓ **Prefer `[class.x]` when you know the class name** — it's faster and clearer.
 Reach for `ngClass` when the set is dynamic.

▲ The `[disabled]` trap

```
<button disabled="{{ isDisabled }}">      <!--
✗ always disabled - "false" is a non-empty string -->
<button [disabled]="isDisabled">          <!-- ✓ real boolean -->
```

C — Two-way binding — how it actually works

Q: How does `[(ngModel)]` work under the hood?

The "banana in a box" `[(x)]` is **pure syntactic sugar**. Angular desugars it into a property binding plus an event binding whose name is the property + **Change**:

```
<input [(ngModel)]="name">
<!-- desugars to -->
<input [ngModel]="name" (ngModelChange)="name = $event">
```

Q: How do you create your own two-way bindable property?

The convention: `@Input() x` + `@Output() xChange`. The **Change** suffix is not optional — that's what the desugaring looks for.


```

@Component({
  selector: 'app-counter',
  template: `
    <button (click)="decrement()">--</button>
    {{ count }}
    <button (click)="increment()">+</button>
  `,
})
export class CounterComponent {
  @Input() count = 0;
  @Output() countChange = new EventEmitter<number>();

  increment() { this.count++; this.countChange.emit(this.count); }
  decrement() { this.count--; this.countChange.emit(this.count); }
}

```

```

<app-counter [(count)]="total"></app-counter>    <!-- ✓ works -->

```

★ `model()` — signal-based two-way binding (Angular 17.2+ — the Ionic app on 18.2, not the 16.2 web app)

```

export class CounterComponent {
  count = model(0); // input + output in one
  increment() { this.count.update(c => c + 1); } // auto-emits
}

```

■ **Say this:** "`[(x)]` is sugar for `[x]` plus `(xChange)` — so any component becomes two-way bindable just by naming the output `<inputName>Change`. From Angular 17.2 there's `model()`, which is a writable signal that is an input and an output at once. RoboGebra web is on 16.2, so there I'd use the `@Input` / `@Output` pair; the Ionic app is on 18.2 where `model()` is available."

▲ Two-way binding vs the store

Two-way binding is fine for local UI state. For shared/app state it hides *who* changed *what* — that's why NgRx (EasyVisa) pushes you to one-way data flow with explicit actions.

D — @Input / @Output deep dive

Q: How do you react to an @Input change?

Two ways — know both and when to pick each.

```
// 1. Setter — for ONE input, runs immediately on set
@Input()
set userId(value: string) {
  this._userId = value;
  this.loadUser(value);
}
private _userId!: string;

// 2. ngOnChanges — for MULTIPLE inputs, or when you need the previous
// value / first-change flag
ngOnChanges(changes: SimpleChanges) {
  if (changes['userId'] && !changes['userId'].firstChange) {
    this.loadUser(changes['userId'].currentValue);
    console.log(changes['userId'].previousValue);
  }
}
```

	Setter	ngOnChanges
Runs	Immediately on assignment	Once per CD cycle, before ngOnInit
Previous value	You track it yourself	SimpleChange.previousValue ✓
Multiple inputs	One setter each	All in one object ✓
First change?	Track manually	.firstChange ✓

▲ ngOnChanges only fires for template-bound inputs and only when the reference changes

```
this.items.push(newItem);           // ✗ same array reference — ngOnChanges
does NOT fire
this.items = [...this.items, newItem]; // ✓ new reference — fires
```

This is exactly why **OnPush** components need immutable updates.

★ Input options (Angular 16+ — available on RoboGebra web 16.2)

```
@Input({ required: true }) userId!: string;           // compile
error if not provided
@Input({ transform: booleanAttribute }) disabled = false; // <app-x
disabled> works like HTML
@Input({ transform: numberAttribute }) count = 0;
@Input('aliasName') internalName!: string;           // alias –
avoid, hurts readability
```

★ Signal inputs (Angular 17.1+ — the Ionic app only)

```
userId = input.required<string>();
count   = input(0, { transform: numberAttribute });
total   = computed(() => this.count() * 2);           // reacts automatically,
no ngOnChanges
```

Q: @Output best practices

```
@Output() saved = new EventEmitter<User>();           // name it as a past-
tense event, not "onSave"
this.saved.emit(user);
```

- Don't prefix with **on** — the template already reads `(saved)="onSaved()"`.
- Emit **data**, not DOM events.
- **EventEmitter** extends **Subject**, but **only use it for @Output** — for anything else use a **Subject** directly.

E — Binding gotchas & change detection

● Q: What is ExpressionChangedAfterItHasBeenCheckedError?

The most-asked Angular binding error. In **dev mode only**, Angular runs change detection a second time and verifies nothing changed. If a bound value changed *during* or *after* the first pass, it throws.

```
@Component({ template: `{{ value }}` })
export class BadComponent implements AfterViewInit {
  value = 'initial';
  ngAfterViewInit() {
    this.value = 'changed';    // ✗ view was already checked → throws
  }
}
```

Common causes:

1. Changing a bound value in `ngAfterViewInit` / `ngAfterViewChecked`.
2. A **child** changing a **parent's** state during the parent's check (data flows top-down; Angular calls this unidirectional data flow).
3. A getter or method in the template returning a **new object/array each call**:
`html {{ getItems() }} <!-- returns a fresh array every call → never "equal" → throws -->`

Fixes:

- Move the change to `ngOnInit`.
- `cdr.detectChanges()` to run CD again immediately, or `cdr.markForCheck()`.
- Defer a tick: `Promise.resolve().then(() => this.value = 'changed')` or `setTimeout`.
- **Best:** restructure so the value is computed before the view is checked.

■ "It only appears in dev mode, and it's Angular telling me I've broken unidirectional data flow — something changed after the view was already checked. I fix the data flow rather than papering over it with `setTimeout`."

▲ Never call functions in templates (for anything expensive)

```
{{ calculateTotal() }}    <!--
✗ runs on EVERY change detection cycle – dozens of times -->
```

Fixes: compute in `ngOnInit` / a setter, use a **pure pipe**, use a `computed()` signal (16+), or `OnPush`.

Q: How does binding interact with `OnPush` ?

An `OnPush` component is re-checked only when:

- 1. an `@Input` reference changes,
- 2. an event fires **from its own template**,
- 3. an `async` pipe in its template emits,
- 4. you call `markForCheck()` explicitly.

■ "With `OnPush` I have to treat inputs as immutable — mutating an object in place won't trigger a re-render because the reference didn't change."

Q: What's the `async` pipe and why is it the right way to bind an Observable?

```
<div *ngIf="user$ | async as user">{{ user.name }}</div>
```

It subscribes, renders each emission, **unsubscribes automatically on destroy**, and calls `markForCheck()` so it works with `OnPush`. No manual `takeUntil` needed. (Full detail in [20 — RxJS](#).)

F — Template-driven vs Reactive forms

● The comparison table (memorize this)

	Template-driven	Reactive (model-driven)
Module	<code>FormsModule</code>	<code>ReactiveFormsModule</code>
Form model	Created implicitly by directives in the template	Created explicitly in the class
Source of truth	The template	The component class
Data flow	Asynchronous	Synchronous ✓

	Template-driven	Reactive (model-driven)
Directives	<code>ngModel</code> , <code>ngForm</code> , <code>ngModelGroup</code>	<code>formControl</code> , <code>formGroup</code> , <code>formControlName</code> , <code>formArrayName</code>
Validation	Directives in the template (<code>required</code> , <code>minlength</code>)	Functions in the class (<code>Validators.required</code>)
Dynamic fields	Hard	Easy (<code>FormArray</code>) ✓
Type safety	None	Typed forms (Angular 14+) ✓
Testability	Needs a fixture + <code>whenStable()</code>	Unit-testable without the DOM ✓
Scales to	Small, simple forms (login, a search box)	Complex, dynamic, conditional forms ✓
Mutability	Mutable — the directive mutates your model	Immutable — <code>valueChanges</code> emits a new value

✓ **The answer they want**

"Template-driven forms build the model implicitly from directives in the template, are asynchronous, and suit simple forms. Reactive forms create the model explicitly in the class, are synchronous and immutable, and give you typed forms, `FormArray` for dynamic fields, and easy unit testing. I use reactive forms for anything non-trivial — on EasyVisa the visa forms were reactive and rendered from JSON config through Angular Formly, with `fieldGroup` for nesting and `fieldArray` for repeatable sections."

Q: Can you mix them?

Technically in the same app, yes — but **never on the same control**.
`[(ngModel)]` together with `formControlName` throws an error (it was deprecated in Angular 6 and removed in v11+).


```

<form [formGroup]="form" (ngSubmit)="onSubmit()">
  <input formControlName="name">

  <div formGroupName="address">
    <input formControlName="city">
  </div>

  <div formArrayName="phones">
    <div *ngFor="let ctrl of phones.controls; let i = index">
      <input [formControlName]="i">
    </div>
  </div>

  <button type="submit" [disabled]="form.invalid">Save</button>
</form>

```

● Q: What are typed forms? (Angular 14+ — available on both your codebases)

Before v14 every form value was `any`. Now `FormGroup` infers its value type:

```

form = this.fb.group({
  name: ['', Validators.required],
  age:  [0],
});

this.form.value; // { name?: string | null; age?: number | null }
this.form.controls.name; // FormControl<string | null>
this.form.get('name'); // ✗ compile error – typo caught

```

Two things to know:

1. Values are `| null` because `reset()` sets controls to `null` — unless you use **`nonNullable`**:

```

ts name: this.fb.nonNullable.control('', Validators.required); //
FormControl<string> // reset() now restores '' instead of null // or:
new FormControl('', { nonNullable: true, validators:
[Validators.required] })

```

`this.fb.nonNullable.group({...})` makes the whole group non-nullable.

2. `value` is **`Partial`** because disabled controls are excluded — use `getRawValue()` for the complete typed value.

Opting out during migration: `UntypedFormGroup` / `UntypedFormControl` — that's what `ng update` generates so nothing breaks.

■ "Typed forms landed in Angular 14 and the migration just renames everything to `Untyped*` so nothing breaks; you then opt in file by file. The two things that surprise people are that values are nullable because `reset()` sets `null` — which `nonNullable` fixes — and that `.value` is a `Partial` because disabled controls are omitted."

H — Reading & writing form state

● **Q: `setValue()` vs `patchValue()` — the most-asked forms question**

	<code>setValue()</code>	<code>patchValue()</code>
Requires	Every control, exactly matching the structure	Any subset
Extra/missing key	Throws an error ✓ (strict)	Silently ignored ▲
Use for	Full replacement — safer, catches structure drift	Partial updates (patching a server response)

```
form = this.fb.group({ name: [''], email: [''] });

this.form.setValue({ name: 'Priyanka' });
// ✗ Error: Must supply a value for form control with name: 'email'

this.form.patchValue({ name: 'Priyanka' });           // ✓ email untouched
this.form.patchValue({ nmae: 'typo' });               // ▲ silently does
nothing — no error!
```

■ "`setValue` is strict — it throws if the shape doesn't match, which is exactly what I want when I'm loading a record into a form, because a renamed field fails loudly. `patchValue` ignores unknown keys silently, so I use it only for genuine partial updates."

Real-world idea

`setValue` → filling in an OFFICIAL FORM: every box must be completed, and an unknown box is rejected at the counter ✓ ★

`patchValue` → editing a few lines of a document: everything else is left alone, and a misspelt heading is simply... ignored ▲

★ The trap is `patchValue`'s **silence**:

```
this.form.patchValue({ nmae: 'typo' });    //  
▲ no error, no warning, no change
```

The field just never updates.
No console message. You debug the API, the service, the template...
and the bug is a typo in a key. ▲

That is why `setValue` is safer for loading a record — a renamed backend field fails loudly instead of silently blanking a screen.

Easy memory

`setValue` ★ → STRICT → every control required → a typo THROWS ✓ (an official form)

`patchValue` → PARTIAL → a subset is fine → a typo is SILENT ▲ ▲

Loading a record? → `setValue` (fail loudly)

Genuine partial update? → `patchValue`

● Q: `value` vs `getRawValue()`

`value` excludes disabled controls. `getRawValue()` includes them.

```
form = this.fb.group({  
  name: ['Priyanka'],  
  email: [{ value: 'p@x.com', disabled: true }],  
});  
  
this.form.value;           // { name: 'Priyanka' }           ▲ email  
missing!  
this.form.getRawValue(); // { name: 'Priyanka', email: 'p@x.com' } ✓
```

▲ **The bug this causes:** you disable a field, submit `form.value`, and the API wipes that column because the key wasn't sent. **If you disable fields, submit `getRawValue()`.**

● Trace the bug — this is worth memorising ★

1. You disable the `email` field because the user can't edit it
2. The user saves
3. `form.value` → `{ name: 'Priyanka' }` ← `email` is NOT in the object ★
4. PUT `/users/1` with that body
5. The backend treats a MISSING key as "set it to null"
6. The user's email is now EMPTY in the database ▲

No exception. No red text. No failing test.
Just a column quietly blanked in production. ★

Real-world idea: a form where the greyed-out boxes are **cut off the page** before it's posted. The clerk assumes those fields were meant to be cleared.

The three fixes

- | | |
|---|---|
| 1. <code>form.getRawValue()</code> | ★ include disabled controls |
| 2. Use <code>readonly</code> instead of <code>disabled</code> | (<code>readonly</code> stays in <code>.value</code> ★) |
| 3. Send a PATCH, not a PUT | (a missing key means "don't change it") |

★ Option 2 is a genuinely good point to raise: **readonly is a display concern, disabled is a form-state concern** — and people reach for `disabled` when they only wanted the field to look uneditable.

Easy memory

`form.value` → EXCLUDES disabled controls ▲
`form.getRawValue()` → INCLUDES them ★

THE BUG: disable a field → submit `.value` → the API blanks that column ▲
FIX: `getRawValue()`

★ | or `readonly` instead of `disabled` | or PATCH not PUT

Q: `valueChanges` and `statusChanges`

```
this.form.valueChanges // Observable<T> – every value
change
  .pipe(debounceTime(300), distinctUntilChanged(),
takeUntil(this.destroy$))
  .subscribe(v => this.autoSave(v));

this.form.get('country')!.valueChanges // per-control
  .subscribe(c => this.loadStates(c));

this.form.statusChanges // 'VALID' | 'INVALID' |
'PENDING' | 'DISABLED'
  .subscribe(s => console.log(s));
```

▲ **Always unsubscribe** (`takeUntil` or `takeUntilDestroyed()` in v16+) — a form subscription is a classic memory leak.

▲ The infinite loop

```
this.form.valueChanges.subscribe(() => {
  this.form.patchValue({ total: this.calc() }); //
  x fires valueChanges again → infinite loop
});
```

Fix: `patchValue(..., { emitEvent: false })`.

Q: `updateOn` — controlling *when* the value updates

```
name = new FormControl('', { updateOn: 'blur' }); // 'change'
(default) | 'blur' | 'submit'
form = this.fb.group({...}, { updateOn: 'submit' }); // for the
whole group
```

Great for expensive async validators — validate on blur instead of every keystroke.

Q: `reset()`

```
this.form.reset(); // all controls → null, pristine + untouched
this.form.reset({ name: 'Priyanka', email: '' }); // reset to specific values
```

▲ `reset()` sets `null`, not `''` — which is why bound `<input>`s go empty but your typed value becomes `null`. `nonNullable` controls reset to their **initial** value instead.

Q: Enable/disable programmatically

```
this.form.get('email')!.disable(); // emits valueChanges
this.form.get('email')!.disable({ emitEvent: false });
this.form.enable();
```

▲ **Never** use `[disabled]="cond"` in the template with reactive forms — Angular logs a warning and the state can desync. Do it in the class.

I — Control state & the CSS classes

Property	Meaning	Opposite
<code>valid</code>	Passes all validators	<code>invalid</code>
<code>pending</code>	An async validator is running	—
<code>disabled</code>	Excluded from value and from validation	<code>enabled</code>
<code>pristine</code>	Value never changed by the user	<code>dirty</code>
<code>touched</code>	The control has been blurred	<code>untouched</code>

Angular mirrors these onto the element as CSS classes:

`ng-valid` / `ng-invalid` / `ng-pending` / `ng-pristine` / `ng-dirty` / `ng-touched` / `ng-untouched`

```
input.ng-invalid.ng-touched { border-color: red; } /* only show red
AFTER they leave the field */
```

● The disabled-control trap

A disabled control is excluded from validation, so an invalid-but-disabled control makes the form `VALID`.

```
form.get('email')!.setValidators(Validators.required);
form.get('email')!.disable();
form.valid; // true ✓ - even though email is empty and required
```

Q: How do you show errors only when appropriate?

```
<input formControlName="email" [class.is-invalid]="isInvalid('email')">
<div class="error" *ngIf="isInvalid('email')">
  <span *ngIf="form.get('email')?.errors?.['required']">Email is
required</span>
  <span *ngIf="form.get('email')?.errors?.['email']">Enter a valid email</
span>
</div>
```

```
isInvalid(name: string): boolean {
  const c = this.form.get(name);
  return !!c && c.invalid && (c.dirty || c.touched);
}
```

Q: How do you show all errors when the user hits Submit without touching anything?

```
onSubmit() {  
  if (this.form.invalid) {  
    this.form.markAllAsTouched();    // ✓ turns on every error message  
    at once  
    return;  
  }  
  this.api.save(this.form.getRawValue()).subscribe();  
}
```

Related: `markAsTouched()`, `markAsDirty()`, `markAsPristine()`,
`updateValueAndValidity()`.

J — Validation

Built-in validators

`required`, `requiredTrue` (checkboxes — terms & conditions), `min`, `max`,
`minLength`, `maxLength`, `pattern`, `email`, `nullValidator`, `compose`,
`composeAsync`.

```
age: [null, [Validators.required, Validators.min(18),  
Validators.max(120)]],  
terms: [false, Validators.requiredTrue],    // ▲ NOT  
Validators.required — false is "empty"
```

▲ `Validators.pattern` is auto-anchored

`Validators.pattern('\\d+')` becomes `^\\d+$`. If you pass a **RegExp** object it is **not** anchored — a common source of "why does my partial match pass?"

Custom sync validator

```
export function noWhitespace(): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    const isBlank = (control.value || '').trim().length === 0;
    return isBlank ? { whitespace: true } : null; // null = VALID
  };
}
// usage
name: ['', [Validators.required, noWhitespace()]]
```

The rule to say out loud: "A validator returns *null* when the control is valid, and an error object when it isn't. It's counter-intuitive the first time — null means success."

● Cross-field validator (goes on the FormGroup)

```
export const passwordsMatch: ValidatorFn = (group: AbstractControl):
ValidationErrors | null => {
  const pwd = group.get('password')?.value;
  const confirm = group.get('confirmPassword')?.value;
  return pwd === confirm ? null : { passwordMismatch: true };
};

form = this.fb.group({
  password: ['', Validators.required],
  confirmPassword: ['', Validators.required],
}, { validators: passwordsMatch }); // ← second argument
```

```
<div *ngIf="form.errors?.['passwordMismatch'] &&
form.get('confirmPassword')?.touched">
  Passwords do not match
</div>
```

▲ The error lands on the **group**, not the control — `form.errors`, not `form.get('confirmPassword').errors`.

● Async validator (server-side uniqueness)

```
emailTaken(): AsyncValidatorFn {
  return (control: AbstractControl): Observable<ValidationErrors | null>
  => {
    if (!control.value) return of(null);
    return timer(400).pipe(                                // debounce
      switchMap(() => this.api.checkEmail(control.value)),
      map(taken => (taken ? { emailTaken: true } : null)),
      catchError(() => of(null)),                          // never block
      the user on an API failure
    );
    first()                                                // ✓ MUST
  );
  complete
};
}
// third argument position:
email: ['', [Validators.required, Validators.email], [this.emailTaken()]]
```

Four things interviewers check here:

1. Async validators go in the **third** argument (or `{ asyncValidators: [...] }`).
2. They run **only after the sync validators pass** — no wasted API calls on an invalid email.
3. The control's status is **PENDING** while in flight — disable submit on `form.invalid || form.pending`.
4. The Observable **must complete** (`first()` / `take(1)`) or the status stays **PENDING** forever. ▲ This is the classic bug.

Dynamic validators

```
const phone = this.form.get('phone')!;
if (contactByPhone) {
  phone.setValidators([Validators.required, Validators.pattern(/
^d{10}$/)]);
} else {
  phone.clearValidators();
}
phone.updateValueAndValidity(); //
▲ REQUIRED — without this nothing re-evaluates
```

▲ **setValidators** alone does nothing until you call `updateValueAndValidity()` . Guaranteed follow-up question.

setErrors — manual server-side errors

```
this.api.save(v).subscribe({
  error: err => {
    if (err.status === 409) {
      this.form.get('email')!.setErrors({ serverTaken: true });
    }
  }
});
```

K — **FormArray** & dynamic forms

Q: When do you use **FormArray** ?

When the **number of controls isn't known at compile time** — repeatable rows: phone numbers, line items, applicants on a visa case.

```

form = this.fb.group({
  name: ['', Validators.required],
  phones: this.fb.array([ this.createPhone() ]),
});

get phones(): FormArray {                                // ✓ getter
  for the template
    return this.form.get('phones') as FormArray;
}

createPhone(): FormGroup {
  return this.fb.group({
    type: ['mobile'],
    number: ['', [Validators.required, Validators.pattern(/^\d{10}$/)]],
  });
}

addPhone()          { this.phones.push(this.createPhone()); }
removePhone(i: number){ this.phones.removeAt(i); }
clearPhones()       { this.phones.clear(); }

```

```

<div formArrayName="phones">
  <div *ngFor="let group of phones.controls; let i = index"
    [formGroupName]="i">
    <select formControlName="type">
      <option value="mobile">Mobile</option>
      <option value="home">Home</option>
    </select>
    <input formControlName="number">
    <button type="button" (click)="removePhone(i)">Remove</button>
    <div *ngIf="group.get('number')?.touched &&
group.get('number')?.invalid">
      10-digit number required
    </div>
  </div>
</div>
<button type="button" (click)="addPhone()">Add phone</button>

```

Three **FormArray** gotchas:

1. `form.get('phones')` returns **AbstractControl** — **cast to FormArray** (that's what the getter is for).
2. Inside `*ngFor` use `[formGroupName]="i"` for groups, `[formControlName]="i"` for plain controls — **the index, in brackets**.
3. Iterate `phones.controls`, not `phones.value`.

Q: How do you build fully dynamic forms from config? (Your EasyVisa story)

Angular Formly — render a form from a JSON field config instead of hand-writing templates.

```

fields: FormlyFieldConfig[] = [{
  key: 'applicants',
  type: 'repeat', // fieldArray →
  repeatable: true,
  fieldArray: {
    fieldGroup: [ // nesting
      { key: 'firstName', type: 'input', props: { label: 'First name',
        required: true } },
      { key: 'hasEmployer', type: 'checkbox', props: { label:
        'Employed?' } },
      {
        key: 'employer', type: 'input',
        props: { label: 'Employer' },
        expressions: { // conditional show/
          hide: '!model.hasEmployer',
          'props.required': 'model.hasEmployer',
        },
      },
    ],
  },
},
],
},
}],
}];

```

```
<form [formGroup]="form" (ngSubmit)="submit()">
  <formly-form [form]="form" [fields]="fields" [model]="model"></formly-
form>
</form>
```

■ **Your answer:** "On EasyVisa the visa forms were huge, conditional and repeatable — a section could have N applicants each with nested address and employment history, and fields appeared based on earlier answers. Hardcoding that was unmaintainable, so I drove it from a JSON field config with Formly: `fieldGroup` for nesting, `fieldArray` for repeatable sections, and `expressions` for conditional show/hide and dynamic required. Underneath it's still reactive forms — Formly builds the same `FormGroup` tree — so validity aggregates up from the

nested groups to the submit button automatically. A new form section became a config change instead of a new component."

▲ **Attribution:** Formly = **EasyVisa**. RoboGebra has **no Formly and no NgRx**.

L — `ControlValueAccessor` — custom form controls

● **This is the senior-level forms question.** If you can implement it on a whiteboard, you're clearly above the "I used `formControlName`" level.

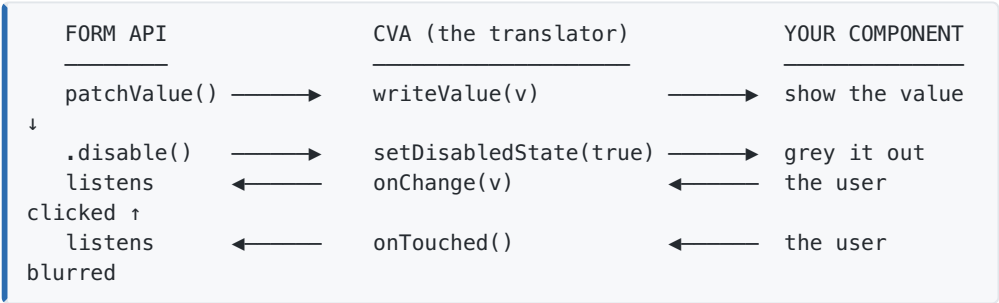
Q: What is `ControlValueAccessor` ?

The **bridge between Angular's form API and a custom component**. It's how Angular knows how to write a value into your component and how to hear about changes coming out of it. Implement it and your component works with `formControlName` , `ngModel` , validators, `touched / dirty` — everything.

The easiest way to remember it

Angular already knows how to talk to `<input>`.
It has NO IDEA how to talk to YOUR component.

`ControlValueAccessor` is the TRANSLATOR between them. ★



Real-world idea: an **interpreter at a meeting**. Angular speaks "form API"; your star-rating component speaks "clicks and stars". The CVA translates in both

directions — and once it's in place, the two sides behave as if they always understood each other.

★ The memory hook for the four methods — **two in, two out**:

```
IN  (forms → you):  writeValue ↓ | setDisabledState ↓
OUT (you → forms):  registerOnChange ↑ | registerOnTouched ↑
```

▲ And the two provider details that are always forgotten:

```
forwardRef → the class isn't defined yet when the decorator is evaluated
★
multi: true → NG_VALUE_ACCESSOR is a MULTI provider; without it you
REPLACE
                every other accessor in the app ▲
```

The four methods

Method	Direction	Purpose
<code>writeValue(v)</code>	Forms → your component	Angular pushes a value in (<code>patchValue</code> , <code>reset</code> , initial value)
<code>registerOnChange(fn)</code>	Your component → forms	Save <code>fn</code> ; call it whenever the value changes
<code>registerOnTouched(fn)</code>	Your component → forms	Save <code>fn</code> ; call it on blur to mark <code>touched</code>
<code>setDisabledState(isDisabled)</code>	Forms → your component	Called by <code>.disable()</code> / <code>.enable()</code>

Full working implementation — a star-rating control

```
import { Component, forwardRef, Input } from '@angular/core';
import { ControlValueAccessor, NG_VALUE_ACCESSOR } from '@angular/forms';

@Component({
  selector: 'app-rating',
  template: `
    <span *ngFor="let star of stars; let i = index"
      (click)="select(i + 1)"
      (blur)="onTouched()"
      [class.filled]="i < value"
      [class.disabled]="disabled">★</span>
  `,
  providers: [{
    provide: NG_VALUE_ACCESSOR,
    useExisting: forwardRef(() => RatingComponent), // forwardRef: the
class isn't defined yet
    multi: true, // ▲ multi:true is
mandatory
  }],
})
export class RatingComponent implements ControlValueAccessor {
  @Input() max = 5;
  get stars() { return Array(this.max); }

  value = 0;
  disabled = false;

  private onChange: (v: number) => void = () => {};
  private onTouch: () => void = () => {};

  writeValue(value: number): void { this.value = value ?? 0; }
  registerOnChange(fn: (v: number) => void): void { this.onChange = fn; }
  registerOnTouched(fn: () => void): void { this.onTouched = fn; }
  setDisabledState(isDisabled: boolean): void { this.disabled =
isDisabled; }

  select(v: number): void {
    if (this.disabled) return;
    this.value = v;
    this.onChange(v); // ✓ tell the form
    this.onTouched();
  }
}
```

```
<!-- now it's a first-class form control -->
<app-rating formControlName="rating" [max]="5"></app-rating>
```

Three things that trip people up:

1. **multi: true** — **NG_VALUE_ACCESSOR** is a multi-provider; without it you break every other accessor.
2. **forwardRef** — the class is referenced inside its own decorator, before it exists.
3. **writeValue** must not call **onChange** — that's an infinite loop.

Bonus: make the component validate itself

```
providers: [
  { provide: NG_VALUE_ACCESSOR, useExisting: forwardRef(() =>
    RatingComponent), multi: true },
  { provide: NG_VALIDATORS, useExisting: forwardRef(() =>
    RatingComponent), multi: true },
]
// ...
validate(control: AbstractControl): ValidationErrors | null {
  return this.value > 0 ? null : { ratingRequired: true };
}
```

M — What happens? (18 binding & forms traps)

- Same format as 23: the code → what happens → the rule → the follow-up.

Q1. **setValue** with a missing control

```
form = this.fb.group({ name: [''], email: [''] });
this.form.setValue({ name: 'Priyanka' });
```

✓ What happens

Runtime error: Must supply a value for form control with name: 'email'.

The rule

`setValue` requires the **exact** structure. `patchValue` would have worked.

Follow-up: "And `patchValue({ nmae: 'typo' })` ?"

No error, no effect — unknown keys are silently ignored. That's the real danger of `patchValue`.

Q2. Submitting a form with a disabled field

```
form = this.fb.group({
  name: ['Priyanka'],
  email: [{ value: 'p@x.com', disabled: true }],
});
console.log(this.form.value);
```

✓ What happens

```
{ name: 'Priyanka' }
```

email is missing.

The rule

`value` **excludes disabled controls**; `getRawValue()` includes them.

Follow-up: "What bug does this cause in production?"

You PATCH the API with `form.value` and the disabled field's column gets wiped, because the key wasn't sent.

Q3. Is this form valid?

```
form = this.fb.group({ email: ['', Validators.required] });
this.form.get('email')!.disable();
console.log(this.form.valid);
```

✓ What happens

```
true
```

The rule

Disabled controls are excluded from validation entirely. The form's status ignores them.

Follow-up: *"How do you make it read-only but still validated?"*

Use the `readonly` HTML attribute (or CSS `pointer-events`) instead of `disable()` — the control stays enabled in the form model.

Q4. `[disabled]` with reactive forms

```
<input formControlName="email" [disabled]="isReadOnly">
```

✓ What happens

Angular logs: *"It looks like you're using the disabled attribute with a reactive form directive"*, and the state can **desync** from the form model.

The rule

With reactive forms the **model owns the disabled state** — call `control.disable()` / `.enable()` in the class.

Q5. The infinite loop

```
this.form.valueChanges.subscribe(v => {  
  this.form.patchValue({ total: v.price * v.qty });  
});
```

✓ What happens

Infinite loop — the patch fires `valueChanges`, which patches again. The tab freezes / stack overflow.

The rule

Pass `{ emitEvent: false }` to any programmatic update inside a `valueChanges` handler.

Follow-up: "A cleaner way?"

Derive `total` as a getter or a `computed()` signal instead of storing it in the form at all.

Q6. Async validator that never completes

```
emailTaken(): AsyncValidatorFn {  
  return c => this.api.checkEmail(c.value).pipe(map(t => t ? { taken:  
    true } : null));  
}
```

✓ What happens

If `checkEmail` returns a **Subject/long-lived Observable**, the control stays **PENDING forever** and the submit button never enables.

The rule

An async validator's Observable **must complete**. Add `first()` or `take(1)`.

Follow-up: "How do you disable submit correctly with async validators?"

```
[disabled]="form.invalid || form.pending".
```

Q7. `setValidators` with no follow-up call

```
this.form.get('phone')!.setValidators(Validators.required);  
console.log(this.form.get('phone')!.valid); // phone is empty
```

✓ What happens

```
true
```

— the validator hasn't run yet.

The rule

`setValidators` / `clearValidators` / `addValidators` only **register**. You must call `updateValueAndValidity()` to re-evaluate.

Q8. `ExpressionChangedAfterItHasBeenCheckedError`

```
@Component({ template: `{{ label }}` })  
export class C implements AfterViewInit {  
  label = 'a';  
  ngAfterViewInit() { this.label = 'b'; }  
}
```

✓ What happens

`ExpressionChangedAfterItHasBeenCheckedError` — in **dev mode only**. In production the value just updates.

The rule

Dev mode runs a second verification CD pass; changing a bound value after the view was checked breaks unidirectional data flow.

Follow-up: "Fixes?"

Move it to `ngOnInit` ; `cdr.detectChanges()` ; defer with `Promise.resolve().then(...)` . Best: fix the data flow.

Q9. Mutating an array bound to an `OnPush` child

```
// parent
this.items.push(newItem);
```

```
<app-list [items]="items"></app-list>    <!-- child is
ChangeDetectionStrategy.OnPush -->
```

✓ What happens

The child does not re-render, and `ngOnChanges` does **not** fire.

The rule

`OnPush` and `ngOnChanges` both compare by **reference**. Mutation keeps the same reference.

Follow-up: "Fix?"

`this.items = [...this.items, newItem];` — immutable update.

Q10. `disabled="{{{ flag }}}"` where `flag === false`

```
<button disabled="{{{ isDisabled }}}">Save</button>
```

✓ What happens

The button is **always disabled**, even when `isDisabled` is `false`.

The rule

Interpolation produces the **string** `"false"`, and the presence of the `disabled` attribute is what disables a button. Use `[disabled]="isDisabled"`.

Q11. `ngModel` + `formControlName` together

```
<input formControlName="name" [(ngModel)]="name">
```

✓ What happens

Runtime error — `ngModel` cannot be used to register form controls with a parent `formGroup` directive. (*Deprecated in v6, removed in v11+.*)

The rule

One control, one directive. Reactive **or** template-driven, never both.

Q12. `{{ }}` on an object

```
<p>{{ user }}</p>          <!-- user = { name: 'Priyanka' } -->
```

✓ What happens

```
[object Object]
```

The rule

Interpolation stringifies. Use `{{ user | json }}` for debugging, or bind the property you actually want.

Q13. `reset()` on a typed form

```
form = this.fb.group({ name: [''] });
this.form.reset();
console.log(this.form.value.name);
```

✓ What happens

```
null
```

— not `''`.

The rule

`reset()` sets controls to `null`. Use `this.fb.nonNullable.control('')` to reset back to the **initial value** instead, which also removes `| null` from the type.

Q14. Required checkbox

```
terms: [false, Validators.required]
```

✓ What happens

The form is invalid when unchecked — **but also stays invalid logic-wise for the wrong reason**, and any *falsy-but-valid* value (like `0` on a number field) is rejected too.

The rule

For "must be ticked" use `Validators.requiredTrue`. `Validators.required` treats `false`, `''`, `null` and `[]` as empty.

Q15. Removing a `FormArray` row inside a loop

```
for (let i = 0; i < this.phones.length; i++) {  
  if (!this.phones.at(i).value.number) this.phones.removeAt(i);  
}
```

✓ What happens

Rows are skipped — removing at `i` shifts everything left while `i` still increments.

The rule

Iterate **backwards** (`for (let i = len - 1; i >= 0; i--)`) or collect indices first. (Same class of bug as `ConcurrentModificationException` in Java — see [23.](#))

Q16. Cross-field error lookup

```
form = this.fb.group({ pwd: [''], confirm: [''] }, { validators:  
  passwordsMatch });
```

```
<div *ngIf="form.get('confirm')?.errors?.['passwordMismatch']">Mismatch</div>
```

✓ What happens

The message never shows, even when the passwords differ.

The rule

A validator on the **group** puts the error on the **group**: `form.errors?.['passwordMismatch']`. To attach it to the control you'd have to call `setErrors` on the control yourself.

Q17. Calling a method in a template

```
<div *ngFor="let item of getFilteredItems()">{{ item.name }}</div>
```

✓ What happens

`getFilteredItems()` runs on **every change detection cycle** — often dozens of times per second on mouse move. And if it returns a new array each call inside a dev-mode-checked binding, you can also get

`ExpressionChangedAfterItHasBeenCheckedError`.

The rule

Templates re-evaluate every CD cycle. Precompute in the class, use a **pure pipe**, or a `computed()` signal.

Q18. Two-way binding with the wrong output name

```
@Input() count = 0;  
@Output() countChanged = new EventEmitter<number>(); // ← "Changed", not  
"Change"
```

```
<app-counter [(count)]="total"></app-counter>
```

✓ What happens

Compile error: *"Can't bind to 'countChange' since it isn't a known property"* — or the value simply never flows back up.

The rule

`[(x)]` desugars to `[x] + (xChange)`. The output must be named **exactly** `<input>Change`.

N — Testing forms

Reactive forms are testable **without touching the DOM** — that's one of their selling points.

```

describe('UserFormComponent', () => {
  let component: UserFormComponent;
  let fixture: ComponentFixture<UserFormComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      declarations: [UserFormComponent],
      imports: [ReactiveFormsModule],
    }).compileComponents();

    fixture = TestBed.createComponent(UserFormComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('is invalid when empty', () => {
    expect(component.form.valid).toBeFalse();
  });

  it('requires a valid email', () => {
    const email = component.form.get('email')!;
    email.setValue('not-an-email');
    expect(email.hasError('email')).toBeTrue();
    email.setValue('p@x.com');
    expect(email.valid).toBeTrue();
  });

  it('flags mismatched passwords on the group', () => {
    component.form.patchValue({ pwd: 'abc123', confirm: 'xyz789' });
    expect(component.form.errors?.['passwordMismatch']).toBeTrue();
  });

  it('submits the raw value including disabled controls', () => {
    const spy = spyOn(component['api'], 'save').and.returnValue(of({}));
    component.form.patchValue({ name: 'Priyanka', email: 'p@x.com' });
    component.form.get('email')!.disable();
    component.onSubmit();
    expect(spy).toHaveBeenCalledWith(jasmine.objectContaining({ email:
'p@x.com' }));
  });
});

```

Async validators: use `fakeAsync` + `tick()`, and assert the `PENDING` → `VALID` transition.

Template-driven forms need `fixture.whenStable()` because `ngModel` updates asynchronously — another reason reactive forms are easier to test.

O — Rapid-fire answer table

Question	One-line answer
Types of binding?	Interpolation, property, event, two-way
Interpolation vs property binding?	Interpolation always stringifies; property binding passes real objects/booleans
<code>[attr.x]</code> vs <code>[x]</code> ?	<code>[attr.]</code> for attributes with no DOM property — ARIA, <code>data-*</code> , SVG, <code>colspan</code>
How does <code>[(x)]</code> work?	Sugar for <code>[x] + (xChange)</code>
Custom two-way binding?	<code>@Input() x + @Output() xChange</code> ; or <code>model()</code> in v17.2+
<code>[class.x]</code> vs <code>ngClass</code> ?	Binding syntax for one known class vs a directive for a dynamic set
React to an <code>@Input</code> change?	An input setter , or <code>ngOnChanges</code> when you need the previous value / multiple inputs
Why doesn't <code>ngOnChanges</code> fire?	The reference didn't change — you mutated instead of replacing
<code>ExpressionChangedAfterItHasBeenChecked</code> ?	Dev-mode-only check; a bound value changed after the view was checked. Fix the data flow
TD vs Reactive forms?	Model in the template, async, simple forms vs model in the class, sync, typed, dynamic, testable
<code>FormControl</code> / <code>FormGroup</code> / <code>FormArray</code> ?	One field / an object of controls / a list of controls — all extend <code>AbstractControl</code>
What is <code>FormBuilder</code> ?	A helper that builds the same tree with less boilerplate: <code>fb.group({...})</code>

Question	One-line answer
Typed forms?	Angular 14+; the value type is inferred. <code>Untyped*</code> is the migration escape hatch
Why is the value nullable?	<code>reset()</code> sets <code>null</code> — use <code>nonNullable</code> to reset to the initial value instead
<code>setValue</code> vs <code>patchValue</code> ?	Strict full structure (throws) vs partial (unknown keys silently ignored)
<code>value</code> vs <code>getRawValue()</code> ?	<code>value</code> omits disabled controls; <code>getRawValue()</code> includes them
Is a form with an invalid disabled control valid?	Yes — disabled controls are excluded from validation
What does a validator return?	<code>null</code> if valid, an error object if invalid
Cross-field validation?	A <code>ValidatorFn</code> on the FormGroup ; the error lands on <code>form.errors</code>
Async validator rules?	Third argument, runs after sync validators pass, status is PENDING , must complete
Changed validators don't apply?	You forgot <code>updateValueAndValidity()</code>
Show all errors on submit?	<code>form.markAllAsTouched()</code>
Dynamic number of fields?	<code>FormArray</code> + <code>formArrayName</code> + <code>[formGroupName]="i"</code>
<code>ControlValueAccessor</code> ?	The bridge that makes a custom component a form control — <code>writeValue</code> , <code>registerOnChange</code> , <code>registerOnTouched</code> , <code>setDisabledState</code> , provided via <code>NG_VALUE_ACCESSOR</code> with <code>multi: true</code>
<code>updateOn</code> ?	<code>'change'</code> (default), <code>'blur'</code> , <code>'submit'</code> — controls when value/validation update
Form memory leak?	Unsubscribed <code>valueChanges</code> — use <code>takeUntil</code> / <code>takeUntilDestroyed()</code> / the <code>async</code> pipe

P — 12 self-test drills (answers hidden)

1. `form.setValue({ name: 'a' })` on a group with `name` and `email`. What happens?

→ Answer

Throws: `*Must supply a value for form control with name: 'email'*. `patchValue` would silently succeed.`

2. You disabled `email` and submit `form.value`. What's in the payload?

→ Answer

`**No `email` key.** Disabled controls are excluded from `value` — use getRawValue().`

3. `[(size)]` on your component doesn't work. What did you name the output?

→ Answer

Anything other than `**`sizeChange`**. [(x)] desugars to [x] + `(xChange)`.`

4. A validator returns `{ required: true }`. Is the control valid?

→ Answer

`**Invalid.** Validators return `null` for valid and an error object for invalid — the reverse of what most people guess.`

5. Why can't you bind `[aria-label]`?

→ Answer

There's no DOM `**property**` called ``aria-label``. Use `[attr.aria-label]`.

6. Your submit button is permanently disabled with `form.invalid || form.pending`. What's the likely bug?

→ Answer

An `**async validator whose Observable never completes**` — status stays ``PENDING``. Add ``first()` / `take(1)``.

7. You call `setValidators` on a control and nothing changes. Why?

→ Answer

You didn't call `**`updateValueAndValidity()`**`.

8. Where does a `passwordsMatch` group validator put its error?

→ Answer

On the `**group**` — ``form.errors?.['passwordMismatch']`, not on ``confirmPassword.errors``.

9. Name the four `ControlValueAccessor` methods.

→ Answer

``writeValue``, ``registerOnChange``, ``registerOnTouched``, ``setDisabledState``.

Registered with ``NG_VALUE_ACCESSOR``, ``useExisting: forwardRef(...)``, `**multi: true**`.

10. `this.items.push(x)` and the `OnPush` child doesn't update. Fix?

→ Answer

Replace the reference: ``this.items = [...this.items, x]``. ``OnPush`` and ``ngOnChanges`` compare by reference.

11. How do you show validation errors for a form the user submitted without touching?

→ Answer

``this.form.markAllAsTouched()`` before returning from ``onSubmit()``.

12. `Validators.required` on a terms-and-conditions checkbox — what's wrong?

→ Answer

``false`` counts as empty in a confusing way and the intent is unclear — use `**`Validators.requiredTrue`**`, which specifically requires ``true``.

Q — Answer scripts (say these out loud)

1. "Explain data binding in Angular." (40 s)

"There are four types. Interpolation and property binding go component to view — one-way. Event binding goes view to component. Two-way binding with the banana-in-a-box syntax is just those two combined: `[(ngModel)]` desugars to `[ngModel]` plus `(ngModelChange)`. That's also how I make my own components two-way bindable — an `@Input` called `x` and an `@Output` called `xChange`. The

one distinction I'd add is property versus attribute binding: Angular binds to DOM properties, so for things with no property — ARIA, `data-*`, SVG, `colspan` — I use `[attr.]`."

2. "Template-driven vs reactive forms — which do you use?" (45 s)

"Reactive, for anything beyond a login box. Template-driven builds the form model implicitly from directives in the template and updates asynchronously, which is fine for something tiny. Reactive forms create the model explicitly in the class, so the data flow is synchronous and immutable, I get typed forms from Angular 14, `FormArray` for dynamic fields, custom and cross-field validators as plain functions, and I can unit test the whole form without rendering the DOM. On EasyVisa the visa forms were reactive and rendered from a JSON config through Angular Formly — nested `fieldGroup`s and repeatable `fieldArray`s with conditional expressions."

3. "setValue vs patchValue?" (25 s)

"`setValue` needs the exact structure — every control, no extras — and throws if it doesn't match, which is what I want when loading a record, because a renamed field fails loudly instead of silently. `patchValue` takes a subset and ignores unknown keys without any error, so I use it only for genuine partial updates. And when I submit, I use `getRawValue()` rather than `value`, because `value` drops disabled controls and that quietly wipes columns on a PATCH."

4. "How do you do async validation?" (40 s)

"An `AsyncValidatorFn` in the third argument position, returning an Observable of `ValidationErrors | null`. I debounce it with a `timer` and `switchMap` so I'm not hitting the API on every keystroke, `catchError` to `of(null)` so an API outage doesn't block the user, and `first()` so the Observable completes — if it doesn't, the control stays `PENDING` forever and the submit button never enables. While it's in flight the status is `PENDING`, so the button binds to `form.invalid || form.pending`. Async validators only run after the sync ones pass, so I don't waste calls on a malformed email."

5. "How would you build a form with a repeatable section?" (40 s)

"`FormArray`. I expose a typed getter that casts `form.get('phones')` to `FormArray`, a factory method that returns a `FormGroup` for one row, and `push / removeAt` for add and delete. In the template it's `formArrayName` on the wrapper, `*ngFor` over `array.controls`, and `[formGroupName]="i"` on each row — the index in brackets. Validity aggregates automatically, so the submit button just binds to `form.invalid`. The one bug I watch for is removing rows in a forward loop, which skips entries — I iterate backwards."

6. "How do you make a custom component work with `formControlName`?" (45 s)

"Implement `ControlValueAccessor` and register it with the `NG_VALUE_ACCESSOR` token using `useExisting`, a `forwardRef` because the class isn't defined yet inside its own decorator, and `multi: true` because it's a multi-provider. Then four methods: `writeValue` for values Angular pushes in, `registerOnChange` where I store the callback and call it whenever my value changes, `registerOnTouched` which I call on blur so `touched` works, and `setDisabledState` so `.disable()` reaches my component. The trap is calling the `onChange` callback from inside `writeValue` — that's an infinite loop. If the component should validate itself, I also provide `NG_VALIDATORS` and implement `validate`."

✓ Before the interview, make sure you can

- [] Name the **4 binding types** and say which direction each goes
- [] Explain `[attr.x]` vs `[x]` with a concrete example (ARIA, `colspan`, SVG)
- [] Desugar `[(ngModel)]` on the spot and write a custom two-way binding
- [] Explain `ExpressionChangedAfterItHasBeenCheckedError` and give 2 fixes
- [] Give the **template-driven vs reactive** table from memory
- [] Explain `setValue` vs `patchValue` and `value` vs `getRawValue()`
- [] Say why a form with an **invalid disabled control** is **VALID**

- [] Write a **custom validator**, a **cross-field validator**, and an **async validator** on paper
 - [] Remember `updateValueAndValidity()` after changing validators
 - [] Build a `FormArray` with add/remove in the class *and* the template
 - [] Implement `ControlValueAccessor` — all 4 methods + `multi: true` + `forwardRef`
 - [] Answer 14 of the 18 traps in [section M](#) cold
 - [] Attribute correctly: **Formly & NgRx = EasyVisa. RoboGebra = Angular 16.2 web + Angular 18.2 / Ionic 8 mobile**
-

Related files: [04 — Angular](#) · [20 — RxJS & Operators](#) · [21 — NgRx](#) · [15 — Ionic Level 1](#) · [18 — RoboGebra Technical Stack](#)

Companies — Questions Actually Asked (with answers)

◆ **This is your recall log.** Every question below was **actually asked in a real round** — not a prediction. Source: your [altemertic,infosys,Interview_Questions.pdf](#) (Infosys · Altimetrik · Photon Interactive) plus the Codeboard Technology round.

How to use it: read the question, cover the answer, say it out loud, then check. Before any interview, re-read the company's section — and read **§ Most-repeated** every single time, because those questions came up at *four different companies*.

Keep appending. After every round, add the company, the date, and the questions **the same day** while you still remember them. That's what makes this file compound.

Track record — every company attended so far

#	Company	Round(s)	Outcome	Questions logged
1	Virtusa	Online assessment	✗ Not selected	▲ <i>not logged</i> — <i>add them</i>
2	Infosys	Technical	✗ Not selected	✓ 6 → §
3	Altimetrik	Level 1 (Java) + Angular round	✗ Not selected	✓ 24 → §
4	Photon Interactive	2 rounds (Frontend + Backend)	✗ Not selected	✓ 46 → §
5	Codeboard Technology	Level 1 (single round → onboarding)	✗ Not selected	✓ 7 → §

#	Company	Round(s)	Outcome	Questions logged
6	Mphasis	L1 Technical (Java/Spring/ microservices) — 6 Aug 2026	● CLEARED → L2 + client round scheduled	✓ 10 → §
7	Virtusa	Round 2 (Java core) — ~18 Aug 2026	✗ Not cleared	✓ 4 → §
8	Tech Mahindra	Angular Developer — ~22 Aug 2026	● RESULT PENDING	✓ 15 → §
9	Deloitte (USI)	Fullstack Java + Angular · AI&E- EaaS · SWE II — 25 Aug 2026	● RESULT PENDING	→ prep: 35
10	Altimetrik	Fullstack Java + Angular (2nd attempt) — 31 Aug 2026	✗ Not cleared	✓ 22 → §
11	IQVIA	★ Angular/Ionic Technical Lead — coming days	● SCHEDULED	→ prep: 38
12	Capgemini	★ Java + Spring + Angular — Sept 2026	● RESULT PENDING	✓ 10 → § · full answers: 41
				144 logged

Read this as data, not as a verdict. Nine companies, **144 questions**, and the **same topics keep repeating**. That's genuinely good news: the question bank is small and knowable. See § **What the pattern says** below.

Companies logged

Company	Round	Qs	Section
Virtusa	Online assessment	—	→
Infosys	Technical	6	→
Altimetrik	Level 1 (Java)	13	→
Altimetrik	Angular round	11	→
Photon Interactive	Round 1 — Frontend	35	→
Photon Interactive	Round 2 — Backend	11	→
Codeboard Technology	Level 1 (single round)	7	→
Mphasis	Technical — Java/ Spring/microservices	10	→
Virtusa	★ Round 2 — Java core (HashMap, threads, interface vs abstract)	4	→
Tech Mahindra	Angular Developer — full breadth sweep	15	→
Altimetrik	★ Fullstack Java + Angular (31 Aug 2026) — Spring depth + Angular + Streams	22	→
Capgemini	★ Java + Spring + Angular (Sept 2026) — breadth sweep + 1 Streams program	10	→
		144	

What the pattern says (across 6 rejections)

1. The question bank is small and it repeats. Lifecycle hooks came up in **4** rounds. Component communication in **4**. **Subject** vs **BehaviorSubject** , unsubscribing, and reactive-vs-template forms in **2–3** each. You are not being asked 93 different things — you're being asked roughly **25 things, six times**. Master those 25 and most of a round is already won.

2. The failures cluster in three places, not everywhere:

Gap	Evidence	Fix
● Output-based Java — "what does this print?" not "what is X?"	Codeboard asked 2 output programs and that round ended there	23 — Predict the Output — 60 programs. Do these before every Java round
● Live coding under pressure	Altimetrik: Stream API salary problem, first non-repeating char. Photon: build a standalone login form live	22 — Streams + 09 — Coding Problems — <i>type</i> them, don't read them
● The "why / what breaks" follow-up	Photon asked "what if forkJoin fails", "disadvantages of lazy loading", "root + component provider — error?"	Every answer needs a second sentence : the trade-off, the failure mode, or what you'd do differently

2b. ★ A round can be 100% backend. Mphasis (6 Aug 2026) asked **zero Angular questions** — ten questions of Java, Spring Boot, microservices, multithreading and SQL. Your resume reads full-stack, so you cannot treat the Java side as the part you revise last. And **SQL appeared for the first time in six rounds** — joins, **GROUP BY / HAVING** and window functions are now a live gap.

3. Breadth is not your problem — depth on the follow-up is. You clearly answered enough to reach 2 rounds at Photon and a final round at Codeboard. The rounds are being lost on the *second* question, not the first.

◆ **The habit that changes outcomes:** for every answer, say the definition, then **one concrete thing from your own project**, then **one trade-off or failure mode**. Three sentences. That's what "4+ years" sounds like versus "read the docs".

4. **Log every round the same day.** This file only exists because you kept the Infosys/Altimetrik/Photon list. Five companies of question data is a real asset — most candidates walk out and forget by evening.

◆ Virtusa — Online Assessment

▲ **Not logged yet.** You attended a Virtusa **online assessment** and weren't selected, but the questions weren't recorded.

→ The **later Virtusa Round 2 (Java core) IS logged** — see § [Virtusa — Round 2](#).

Add whatever you can still recall, even partially — section counts, topics, difficulty, time limits. Even *"3 coding questions, 45 minutes, one on arrays"* is worth writing down, because Virtusa reuses assessment platforms across drives.

Things worth capturing for any online assessment:

Capture	Why it matters
Sections & timing (aptitude / logical / verbal / MCQ / coding)	Tells you how to pace next time — most OA failures are time, not knowledge
Number of coding questions + difficulty	Easy/medium DSA vs framework-specific
Topics of the coding questions	Arrays, strings, HashMap, streams — that's the drill list
MCQ topics (Java? Angular? SQL? OOP?)	Sets what to revise
Was it proctored / camera on / tab-switch locked?	Practical prep for the next one
Platform (HackerRank, HackerEarth, Mettl, Codility...)	Practise on that exact platform's editor

■ **For online assessments specifically:** the usual killer is **time management, not difficulty** — attempt every question you can solve fast first, and never sink 20 minutes into one hard problem while three easy ones go unattempted. Drill on [09 — Coding Problems](#) with a timer running.

● **Answer these with YOUR CODE, not with definitions** ★

The questions in this file repeat across companies. What changes the outcome is **finishing each answer with one concrete thing from your own codebase**:

Lifecycle hooks (5 rounds ★) → "ngOnDestroy in 100 components, 426 takeUntil" Interceptors (4 rounds ★) → the FormData Content-Type trap + cancelRequests\$ Observable vs Promise (4 ★) → "switchMap 246 times – the auth interceptor and every search" Data binding (4 rounds) → 252 @Input, and OnPush on the 6 heavy components HashMap internals (3 rounds) → the N+1 aggregation fix (same reasoning, in Mongo) Lazy loading (3 rounds) → 27 route resolvers, access-ruler resolvers

Definition → **one thing from my code** → **one trade-off**. Three sentences. That is what five years sounds like.

Full inventory: [39 – RoboGebra Code Examples](#).

Most-repeated questions across ALL companies

If you prepare nothing else, prepare these. The count is how many separate rounds asked it.

Asked in	Question	Answer lives in
5 rounds ★	Angular lifecycle hooks / ngOnInit vs constructor	04 · 37 Q15 · Ionic variant in 15
4 rounds	HTTP interceptors ★	34 Q4 · 37 Q17
4 rounds	Component communication	04 · below
4 rounds	Observables vs Promises	20 — RxJS · 34 Q6
4 rounds	Data binding / forms ★	25 — Binding & Forms · 37 Q20

Asked in	Question	Answer lives in
3 rounds	Observable / Subject / BehaviorSubject	20 — RxJS
3 rounds ★	Highest salary per department (Streams and SQL)	22 · 36 · 37 Q22
3 rounds	HashMap internals	31
3 rounds ★	Lazy loading (+ its disadvantages)	04 · 37 Q21
3 rounds ★	Microservices — communication & failure handling	08 · 37 Q11–13
2 rounds	Reactive vs template-driven forms	25 — Binding & Forms
2 rounds	Interface vs abstract class	05 · 33
2 rounds	Unsubscribing / memory leaks	20
2 rounds	Global exception handling	06 · below
2 rounds	Preventing duplicate API calls on rapid clicks	below (Altimetrik Q9) · 37 Q18 (refresh stampede — same technique)
3 rounds ★	Spring stereotypes / DI wiring (@Component vs @Bean , constructor vs setter)	06 · 37 Q3–7 · 41 Q3–4
2 rounds ★	ngOnChanges vs ngOnChecked	04 · Photon Q6 below · 41 Q9
2 rounds ★	Non-repeating character (first / second-from-the-end)	22 · Altimetrik Q3 below · 41 Q10
2 rounds ★	Externalised config — dev/ QA/prod, how values are loaded	06 · 37 Q14 · 41 Q5

● Infosys — Technical Round

Frontend-weighted round: JS basics, Angular, CSS.

1. How do you declare a variable?

JavaScript/TypeScript: `var` (function-scoped, hoisted as `undefined`), `let` (block-scoped, TDZ), `const` (block-scoped, must be initialized, binding can't be reassigned — but object contents can still change).

```
let count: number = 0;
const API = 'https://...';
```

Java: `type name = value;` — `int count = 0;` (Java 10+ also allows `var count = 0;` for local variables only, inferred at compile time).

✓ **Default to `const`, use `let` when you must reassign, never `var`.** Full depth: [19 — JS Variables Made Easy.](#)

2. Angular lifecycle hooks

In firing order:

`ngOnChanges` → `ngOnInit` → `ngDoCheck` → `ngAfterContentInit` → `ngAfterContentChecked` → `ngAfterViewInit` → `ngAfterViewChecked` → `ngOnDestroy`

Hook	Use it for
<code>ngOnChanges</code>	React to <code>@Input</code> changes (gives previous + current value)
<code>ngOnInit</code>	One-time init — API calls, building forms
<code>ngDoCheck</code>	Custom/deep change detection — expensive, use rarely
<code>ngAfterViewInit</code>	Access <code>@ViewChild</code> — the view exists now

Hook	Use it for
<code>ngOnDestroy</code>	Unsubscribe , clear timers, remove listeners

▲ `ngOnChanges` runs **before** `ngOnInit`, and only for **template-bound** inputs whose **reference** changed.

3. How do components communicate?

Relationship	Mechanism
Parent → Child	<code>@Input()</code>
Child → Parent	<code>@Output()</code> + <code>EventEmitter</code>
Parent → Child (direct call)	<code>@ViewChild()</code>
Siblings / unrelated	A shared service with a <code>BehaviorSubject</code>
Across routes	Route params / query params / router state
App-wide	NgRx store (EasyVisa)
Content projection	<code>@ContentChild</code> + <code><ng-content></code>

4. How do you change text colour on click without using classes?

Style binding — bind the style property directly:

```
<p [style.color]="textColor" (click)="textColor = 'red'">Click me</p>
```

Also acceptable: `[ngStyle]="{ color: textColor }"`, or `Renderer2` when you need to touch the DOM safely:

```
constructor(private el: ElementRef, private renderer: Renderer2) {}
onClick() { this.renderer.setStyle(this.el.nativeElement, 'color', 'red'); }
```

■ "I'd use `[style.color]` — it's declarative and stays inside Angular's rendering. `Renderer2` if I needed it to work with server-side rendering or a non-DOM renderer." Full detail: [25 §B](#).

5. Box model

Every element is a box of four layers, inside out: **content** → **padding** → **border** → **margin**.

```
* { box-sizing: border-box; } /* width now INCLUDES padding + border */
```

- **content-box** (default): `width` = content only; padding and border are **added** on top.
- **border-box**: `width` = content + padding + border — far easier to reason about, which is why resets set it globally.
- **Margins collapse** vertically between siblings; padding never collapses.

6. Positioning

position	Behaviour
static	Default. <code>top/left/...</code> do nothing
relative	Offset from its normal spot; still occupies its original space
absolute	Removed from flow; positioned against the nearest positioned ancestor
fixed	Removed from flow; positioned against the viewport — doesn't scroll
sticky	<code>relative</code> until it hits a threshold, then behaves like <code>fixed</code>

▲ The classic follow-up: "`absolute` relative to what?" → the nearest ancestor with a `position` other than `static`; otherwise the initial containing block. Full CSS depth: [02 — HTML & CSS](#).

● Altimetrik — Level 1 Round (Java)

1. Stream API — highest salary per department ●

```
Map<String, Optional<Employee>> topPerDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,

Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))));

// cleaner — just the salary value, no Optional:
Map<String, Double> maxSalary = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.collectingAndThen(

Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary)),
        e -> e.map(Employee::getSalary).orElse(0.0))));

// or with reducing:
Map<String, Optional<Employee>> r = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment,
        Collectors.reducing(BinaryOperator.maxBy(
            Comparator.comparingDouble(Employee::getSalary))));
```

▲ **maxBy** returns an **Optional** — the follow-up is always "how do you get rid of the Optional?" → **collectingAndThen**. More: [22 — Java Streams \(20 problems\)](#).

2. When was **forEach** introduced?

Java 8 (2014) — **Iterable.forEach(Consumer)** and **Stream.forEach(Consumer)**, both enabled by lambdas and **default** methods on interfaces.

▲ If the interviewer meant JavaScript: **Array.prototype.forEach** came in **ES5 (2009)**. Ask which one they mean — it shows precision.

Bonus: **forEach** vs **forEachOrdered** — on a parallel stream **forEach** gives no order guarantee; **forEachOrdered** preserves encounter order.

3. First non-repeating character

```
public static Character firstNonRepeating(String s) {
    Map<Character, Integer> counts = new LinkedHashMap<>();    //
    LinkedHashMap preserves order
    for (char c : s.toCharArray()) counts.merge(c, 1, Integer::sum);
    return counts.entrySet().stream()
        .filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey)
        .findFirst().orElse(null);
}
// "swiss" -> 'w'
```

The whole trick is **LinkedHashMap** — a plain **HashMap** loses insertion order, so "first" becomes meaningless. Say that out loud; it's what they're testing. **O(n)** time, **O(k)** space.

4. ArrayList vs LinkedList

	ArrayList	LinkedList
Backing	Dynamic array	Doubly-linked nodes
<code>get(i)</code>	O(1) ✓	O(n)
<code>add / remove</code> at end	Amortized O(1)	O(1)
<code>add / remove</code> in middle	O(n) — shifting	O(1) <i>once you're at the node</i> — but O(n) to find it
Memory	Compact, cache-friendly	Extra prev/next pointers per node
Implements	List , RandomAccess	List + Deque

5. For searching, which would you use and why? ●

ArrayList. Two reasons:

1. **O(1) random access** — and if the list is sorted, `Collections.binarySearch` gives **O(log n)**, which needs random access to work at all. On a **LinkedList** binary search degrades to O(n) per probe.
2. **Cache locality** — the array is one contiguous block, so scanning it is

dramatically faster in practice than chasing pointers, even for the same $O(n)$ linear search.

■ **Best answer:** "*ArrayList* for searching. If it's sorted I get $O(\log n)$ with binary search, which needs random access; and even for a linear scan the contiguous memory is far more cache-friendly than following node pointers. And honestly, if searching is the main operation I'd question the *List* entirely — a *HashMap* / *HashSet* gives $O(1)$ lookup." ← **that last line is what gets you the mark.**

6. Converting to a text block

Text blocks — Java 15 (preview in 13/14). Triple quotes, no escaping, natural formatting:

```
// before
String json = "{\n  \"name\": \"Priyanka\",\n  \"role\": \"Developer\"\n}";

// after – text block
String json = """
    {
        "name": "Priyanka",
        "role": "Developer"
    }
    """;
```

Rules: the opening `"""` must be followed by a newline; **incidental leading whitespace is stripped** based on the least-indented line (the closing delimiter's position matters); `\` at end of line joins lines; `\s` keeps a trailing space. More: [12 — Java 17 Features](#).

7. HashMap vs ConcurrentHashMap

	HashMap	ConcurrentHashMap
Thread-safe	✗	✓
Null key/values	1 null key, many null values	✗ Neither (ambiguous with <code>get</code> returning null)

	HashMap	ConcurrentHashMap
Locking	—	Bucket/node-level CAS + synchronized (Java 8+); <i>segments</i> pre-Java-8
Concurrent modification	ConcurrentModificationException	Fail-safe iterator — weakly consistent, no exception
Performance	Fastest single-threaded	Scales across threads
vs Hashtable	—	Hashtable locks the whole map — legacy, don't use

8. Internal working of **HashMap** ●

1. **hash(key)** — **key.hashCode()** **XOR (h >>> 16)** to spread high bits into the low bits.
2. Bucket index = **(n - 1) & hash** — works because capacity is always a **power of 2**.
3. Collision → the entry is appended to that bucket's **linked list**.
4. **Treeify**: when a bucket exceeds **8** nodes *and* capacity ≥ **64**, the list becomes a **red-black tree** → $O(\log n)$ instead of $O(n)$. It untreeifies below **6**.
5. **Resize**: when **size > capacity × loadFactor** (**0.75**), capacity **doubles** and entries are rehashed.
6. Default capacity **16**; lookup is **$O(1)$** average, **$O(\log n)$** worst case since Java 8.

▲ **The follow-up**: "What if you use a mutable object as a key?" → mutate it after insertion and its hash changes, so the entry is stranded in the wrong bucket and becomes unreachable. **Keys must be immutable** with consistent **equals** / **hashCode**. (Same trap as JPA entities in 24 §L Q9.)

9. Immutable class in Java

A class whose state cannot change after construction. Built-in examples: **String**, all wrapper types (**Integer**, **Long**, **Double** ...), **BigDecimal**, **BigInteger**, **LocalDate** / **LocalDateTime**, **UUID**, records (shallowly).

Benefits: inherently **thread-safe** (no synchronization needed), safe as `HashMap` keys and in `Set` s, freely cacheable and shareable, no defensive copying by callers.

10. Mutable class

State can change after construction: `StringBuilder` , `StringBuffer` , `ArrayList` , `HashMap` , `Date` , and any normal POJO with setters.

- *"`String` is immutable, which is why every 'modification' creates a new object — that's exactly why you use `StringBuilder` inside a loop instead of `+=`."*

11. Criteria for an immutable class ● (they want the checklist)

1. Declare the **class** `final` — or make all constructors private with a static factory — so nobody can subclass and add mutability.
2. Make all fields `private final` .
3. **No setters** — and no method that changes state.
4. **Initialize everything in the constructor.**
5. **Defensive copy on the way in** — copy any mutable argument before storing it.
6. **Defensive copy on the way out** — never return a reference to a mutable field.
7. Don't let `this` escape during construction.

```

public final class Employee {                                // 1
    private final String name;                                // 2
    private final Date joinDate;                              // mutable type!
    private final List<String> skills;

    public Employee(String name, Date joinDate, List<String> skills) {
        this.name = name;
        this.joinDate = new Date(joinDate.getTime());        // 5
defensive copy IN
        this.skills = List.copyOf(skills);                    // 5
unmodifiable copy
    }

    public String getName() { return name; }
    public Date getJoinDate() { return new Date(joinDate.getTime()); } //
6 copy OUT
    public List<String> getSkills() { return skills; }         // already
unmodifiable
    }

```

▲ Steps 5 and 6 are what separate a real answer from a memorized one.

Without them the class *looks* immutable but the caller still holds a live reference to the `Date`.

12. `@Controller` vs `@RestController`

- `@Controller` — returns a **view name** resolved by a `ViewResolver` (JSP/Thymeleaf). To return data you must add `@ResponseBody` per method.
- `@RestController` = `@Controller` + `@ResponseBody` **on every method** — the return value is serialized straight to JSON. This is what you use for REST APIs.

```

@RestController @RequestMapping("/api/users")
public class UserController {
    @GetMapping("/{id}")
    public ResponseEntity<UserDto> get(@PathVariable Long id) { ... }
}

```

13. PUT vs PATCH

	PUT	PATCH
Semantics	Full replacement of the resource	Partial update — only the sent fields
Payload	The complete representation	Just the changed fields
Idempotent	✓ Yes	▲ Not guaranteed (e.g. <code>{"op": "increment"}</code>)
Missing fields	Get overwritten/nulled	Left untouched

■ "PUT replaces the whole resource and is idempotent — sending it ten times leaves the same state. PATCH sends only the delta and isn't necessarily idempotent. The practical trap is using PUT with a partial body and silently wiping the fields you didn't send."

▲ Exactly the same bug as `form.value` dropping disabled controls — see 25 §H.

● Altimetrik — Angular Round

1. Angular component communication

→ Same as Infosys Q3 above. Lead with `@Input` / `@Output` , then shared service with `BehaviorSubject` for unrelated components.

2. Angular lifecycle hooks

→ Same as Infosys Q2 above.

3. How do you initialize an Observable?

```
// 1. from scratch
const obs$ = new Observable<number>(subscriber => {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.complete();
  return () => console.log('teardown on unsubscribe');
});

// 2. creation operators
of(1, 2, 3); // emits values synchronously, then completes
from([1, 2, 3]); // from array / Promise / iterable
fromEvent(button, 'click'); // from a DOM event
interval(1000); timer(500, 1000);
EMPTY; NEVER; throwError(() => new Error('x'));

// 3. from a Subject (expose it read-only)
private state$ = new BehaviorSubject<User | null>(null);
readonly user$ = this.state$.asObservable();

// 4. what you use 99% of the time
this.http.get<User[]>('/api/users'); // HttpClient returns a cold Observable
```

▲ **Observables are lazy** — nothing runs until `subscribe()` (or the `async` pipe).
Full guide: [20 — RxJS](#).

4. Reactive forms vs template-driven forms

Model in the class, synchronous, immutable, typed, `FormArray` for dynamic fields, unit-testable without the DOM — **vs** model built implicitly by directives in the template, asynchronous, fine for simple forms.

→ **Full table + everything they can follow up with: [25 §F](#).**

5. Formly

Angular Formly renders a form from a **JSON field config** instead of hand-written templates — you describe fields as data and it builds the reactive `FormGroup` for you.

```
fields: FormlyFieldConfig[] = [
  { key: 'firstName', type: 'input', props: { label: 'First name',
    required: true } },
  { key: 'employer', type: 'input',
    expressions: { hide: '!model.employed', 'props.required':
      'model.employed' } },
];
```

`fieldGroup` = nesting, `fieldArray` = repeatable sections, `expressions` = conditional show/hide and dynamic required.

▲ **Attribution: Formly is EasyVisa, NOT RoboGebra.** Your story: "Visa forms were huge, conditional and repeatable — N applicants each with nested address and employment history. Hardcoding was unmaintainable, so a new section became a config change instead of a new component." (25 \$K)

6. What is an Observable?

A **stream of values over time** that you subscribe to. Key properties vs a Promise:

	Observable	Promise
Values	Many over time	Exactly one
Execution	Lazy — starts on subscribe	Eager — starts immediately
Cancellable	✓ <code>unsubscribe()</code>	✗
Operators	100+ (<code>map</code> , <code>switchMap</code> , <code>retry</code> ...)	<code>.then</code> chaining only

7. Subject vs BehaviorSubject ●

	Subject	BehaviorSubject
Initial value	✗ None	✓ Required
Late subscriber gets	Only values emitted after subscribing	The current value immediately , then subsequent ones
Read current value	✗	✓ <code>.getValue()</code>

	Subject	BehaviorSubject
Use for	Events — "a button was clicked", notifications	State — current user, cart, filters

Also: **ReplaySubject(n)** replays the last *n* values to every new subscriber;
AsyncSubject emits only the final value, and only on complete.

■ "For shared state I always use **BehaviorSubject** — a component that subscribes late still gets the current value instead of sitting empty until the next change."

8. In the Angular migration you mentioned, what challenges did you face?

Your real story (**Angular 16 migration**):

"I went version by version with **ng update** rather than jumping, because Angular only supports one-major-at-a-time. The main challenges were: third-party libraries that had no compatible release yet, so I had to check every peer dependency and in a couple of cases replace or wait; RxJS 6 → 7 deprecations — **toPromise()** removed in favour of **firstValueFrom**, and the safe-subscribe overloads; stricter TypeScript and template type-checking surfacing real bugs that had been hidden; and Ivy/ **ngcc** build changes. I did it on a branch with the full test suite as the safety net, upgraded one major at a time, and ran the app after each step so I always knew which version broke what."

✓ Adjust to your real details — but **keep the structure**: one major at a time → dependency compatibility → RxJS/TS breaking changes → tests as the net.

9. If a button is clicked multiple times triggering multiple API calls, how do you restrict them? ●

The best RxJS answer is **exhaustMap** — it ignores new clicks while a request is in flight:

```
fromEvent(this.btn.nativeElement, 'click').pipe(
  exhaustMap(() => this.api.save(this.form.value)), //
  ✓ ignores clicks until this completes
  takeUntilDestroyed(this.destroyRef)
).subscribe();
```

Know all four and when each is right:

Operator	Behaviour	Right for
<code>exhaustMap</code>	Ignore new while one is running	✓ Submit / save buttons — no duplicate writes
<code>switchMap</code>	Cancel the previous , keep the latest	Type-ahead search, filters
<code>concatMap</code>	Queue them, run in order	Ordered writes that all must happen
<code>mergeMap</code>	Run all in parallel	Independent parallel work

Also mention the non-RxJS layers — a complete answer covers all three:

1. **Disable the button** while in flight: `[disabled]="form.invalid || (loading$ | async)"`.
2. `throttleTime(1000)` / `debounceTime(300)` to rate-limit at the source.
3. **Server-side idempotency key** — because a determined user (or a flaky network retry) can still double-submit. *That last point is the senior answer.*

10. In the frontend, if the request body is missing, how do you handle the error?

Three layers:

1. **Prevent it** — don't let an invalid form submit: `if (this.form.invalid) { this.form.markAllAsTouched(); return; }`, and send `getRawValue()` so disabled controls aren't dropped (25 §H).
2. **Handle the server's 400** — the backend replies with field errors; map them onto the form:

```

this.api.save(dto).pipe(
  catchError((err: HttpErrorResponse) => {
    if (err.status === 400) {
      Object.entries(err.error.errors ?? {}).forEach(([field, msg]) =>
        this.form.get(field)?.setErrors({ server: msg }));
    }
    this.toast.error('Please correct the highlighted fields');
    return EMPTY; // swallow – the UI already told
the user
  })
).subscribe();

```

1. **Global fallback** — an HTTP interceptor catches anything unhandled so the user never sees a silent failure (next question).

11. Global exception handling

Angular — two mechanisms, mention both:

```

// 1. HTTP errors → interceptor
@Injectable()
export class ErrorInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    return next.handle(req).pipe(
      catchError((err: HttpErrorResponse) => {
        if (err.status === 401) this.auth.logout();
        if (err.status >= 500) this.toast.error('Something went wrong,
please try again');
        return throwError(() => err);
      })
    );
  }
}

// 2. Everything else (runtime JS errors) → a global ErrorHandler
@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  handleError(error: unknown) { this.logger.send(error); }
}
// providers: [{ provide: ErrorHandler, useClass: GlobalErrorHandler }]

```

Spring Boot side (they often ask both):


```

@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> notFound(ResourceNotFoundException ex)
    {
        return ResponseEntity.status(404).body(new ApiError(404,
ex.getMessage()));
    }
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError>
invalid(MethodArgumentNotValidException ex) { ... }
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> generic(Exception ex) { ... }    //
catch-all last
}

```

→ **06 — Spring Boot**

● Photon Interactive — Round 1 (Frontend)

35 questions — the deepest Angular round you've had. Routing, RxJS and change detection dominate.

1. Angular 16 features

Signals (developer preview) · `computed()` / `effect()` · `takeUntilDestroyed()` and `DestroyRef` · **required inputs** `@Input({ required: true })` · **input transforms** · standalone APIs matured (`ng new --standalone`) · **non-destructive full-app hydration** for SSR · esbuild-based dev server (preview) · self-closing tags in templates · `bootstrapApplication` improvements.

▲ **RoboGebra web is on 16.2** — so these are the features you can legitimately claim.

2. Latest changes in Angular

- **v17** — new **built-in control flow** `@if` / `@for` / `@switch`; **deferrable views** `@defer`; esbuild/Vite builder becomes default; SSR made a first-class `ng new` option; new docs site (angular.dev).
- **v18** — **zoneless change detection** (experimental), Material 3, **event replay** for SSR, signal APIs stabilizing, `@angular/build`.
- **v17.1/17.2** — **signal inputs** `input()`, `output()`, and `model()` for signal-based two-way binding.

✓ Your Ionic app is **Angular 18.2**, so you can speak to v17/v18 features from real code — say which app you mean.

3. Zone and zoneless ●

Zone.js monkey-patches every async browser API — `setTimeout`, `addEventListener`, XHR/fetch, promises — so that when any of them completes, Angular knows *something might have changed* and runs **change detection across the whole component tree**.

Zoneless (v18, experimental) removes zone.js entirely: change detection is triggered **explicitly** by signals, the `async` pipe, `markForCheck()`, and event bindings.

```
bootstrapApplication(App, { providers:
  [provideExperimentalZonelessChangeDetection()] });
```

Why it matters: smaller bundle (no zone.js ~13 kB), no more "CD ran 40 times because a `setInterval` ticked", better interop with third-party async libraries, and faster, more predictable rendering.

■ "Zone.js is how Angular currently knows when to check for changes — it patches async APIs. Zoneless flips it to explicit notification via signals, which is more precise and lets Angular skip the whole tree."

4. What is a component and how do components communicate?

A **component** = a TypeScript class with `@Component` (template + styles + selector) that controls a piece of the screen. Communication → **Infosys Q3**.

5. Lifecycle hooks

→ **Infosys Q2**.

6. `ngOnChanges` VS `ngDoCheck` ●

	<code>ngOnChanges</code>	<code>ngDoCheck</code>
Fires when	A template-bound <code>@Input</code> changes by reference	Every change detection cycle
Gives you	<code>SimpleChanges</code> — previous + current + <code>firstChange</code>	Nothing — you check yourself
Cost	Cheap	▲ Expensive — runs constantly
Use for	Reacting to input changes	Detecting changes Angular can't see — a mutated array, a deep object property

```
ngDoCheck() {
  if (this.items.length !== this.prevLength) {    // Angular missed the
    push()
    this.prevLength = this.items.length;
    this.recalculate();
  }
}
```

■ "`ngOnChanges` only fires when the input reference changes, so mutating an array in place is invisible to it. `ngDoCheck` catches that — but it runs on every cycle, so I'd rather fix the root cause with immutable updates than pay that cost."

7. Memory leak issue

The main causes in an Angular app:

1. **Unclosed subscriptions** (the big one) — `valueChanges` , `route.params` , `interval` , WebSockets, a service `Subject` .
2. **DOM event listeners** added manually and never removed.
3. `setInterval` / `setTimeout` not cleared in `ngOnDestroy` .
4. **Detached DOM nodes** still referenced by a closure or a service.
5. **Long-lived services holding component references** (e.g. pushing `this` into a service array).

8, 9 & 10. What happens if you don't unsubscribe — and how does it behave in the browser? ●

They asked this three ways, so give a layered answer:

What happens technically: the subscription holds a reference to the component's callback, so **the component instance can never be garbage collected** even after Angular destroys it. The stream keeps emitting into a dead component.

How it behaves in the app:

- The callback keeps running against a **destroyed component** — updating properties nobody renders, and often throwing `Cannot read property of undefined` when it touches something already torn down.
- **Duplicates multiply:** navigate to a page and back five times and you now have five live subscriptions — so one button click fires **five** HTTP requests, or a toast appears five times.
- Timer/WebSocket streams keep **network and CPU work running** for screens the user left.

How it behaves in the browser:

- **Heap grows monotonically** — visible in DevTools → Memory as a rising sawtooth that never returns to baseline; the detached component shows up in a heap snapshot as a **detached DOM tree**.
- Increasing **CPU usage and jank** as more zombie subscribers process every emission.

- Eventually the tab becomes unresponsive or the browser kills it — **worst on mobile / Ionic**, where memory limits are much tighter.

The fixes (say all four):

```
// 1. async pipe – best, unsubscribes automatically
users$ = this.service.getUsers();           // <div *ngIf="users$ | async
as users">

// 2. takeUntilDestroyed (Angular 16+) – cleanest imperative option
this.service.data$.pipe(takeUntilDestroyed(this.destroyRef)).subscribe();

// 3. takeUntil with a destroy Subject – the classic pattern
private destroy$ = new Subject<void>();
ngOnDestroy() { this.destroy$.next(); this.destroy$.complete(); }
this.x$.pipe(takeUntil(this.destroy$)).subscribe();

// 4. manual – Subscription.add() then unsubscribe() in ngOnDestroy
```

▲ **HttpClient** calls **complete** after one emission, so they self-unsubscribe — the leaks come from **infinite** streams: **valueChanges**, **route.params**, **interval**, **fromEvent**, **Subject** s. Say this and you sound like you've actually debugged one. Full detail: [20 – RxJS](#).

11. Component communication

→ asked twice in the same round. [Infosys Q3](#).

12. Service provided in root and also privately in a component — error or not? ●

No error. It works — and you get TWO separate instances.

Angular's injector is **hierarchical**. A component-level **providers: [MyService]** creates a **new instance in that component's own injector**, which **shadows** the root one for that component and all of its children. The rest of the app keeps using the root singleton.

```

@Injectable({ providedIn: 'root' }) // instance A – app-wide singleton
export class CartService {}

@Component({
  selector: 'app-widget',
  providers: [CartService], // instance B – this component + its
  children only
})
export class WidgetComponent {
  constructor(private cart: CartService) {} // gets B, NOT A
}

```

▲ **The bug this causes:** shared state silently stops being shared — the widget updates its own cart and the header never sees it. When it's deliberate it's useful (per-component state isolation, like `NgRx ComponentStore`); when it's accidental it's a nasty debugging session.

■ *"It won't throw. Angular resolves from the nearest injector upward, so the component gets its own instance and everything else gets the root one — two instances, not one. If I actually wanted a singleton I'd remove the component-level provider."*

13. `HttpClient` — why does Angular use it and what are its advantages?

1. Returns **Observables** — cancellable, retryable, composable with RxJS operators.
2. **Typed responses** — `http.get<User[]>()`.
3. **Interceptors** — one place for auth headers, error handling, logging, caching, loaders.
4. **Automatic JSON** parsing (and `HttpErrorResponse` on failure).
5. **Testability** — `HttpClientTestingModule` + `HttpTestingController` to assert requests without a server.
6. **Progress events** for uploads/downloads, plus typed `HttpParams` / `HttpHeaders` (immutable).
7. Built-in **XSRF** protection support.

14. Routing design for home, admin, login, user, product, productList, productDetail ●

```
export const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: 'login', component: LoginComponent },
  { path: 'home', component: HomeComponent },

  { path: 'products',
    children: [
      { path: '', component: ProductListComponent },      // /products
      { path: ':id', component: ProductDetailComponent,    // /products/
        resolve: { product: productResolver } },
    ]},

  { path: 'user',
    canActivate: [authGuard],
    loadChildren: () => import('./user/user.routes').then(m =>
m.USER_ROUTES) },

  { path: 'admin',
    canActivate: [authGuard, roleGuard('ADMIN')],          // ▲ role
    // check here
    loadChildren: () => import('./admin/admin.routes').then(m =>
m.ADMIN_ROUTES) },

  { path: '**', component: NotFoundComponent },            // wildcard
  LAST
];
```

The four points they're grading:

- **productList** and **productDetail** are **parent/child** with **:id** — not two flat routes.
- **pathMatch: 'full'** on the empty redirect, and the **wildcard route last** (routes match top-down, first match wins).
- **Lazy-load** the **admin** and **user** feature areas — they aren't needed on first paint.
- Guards on the route, not sprinkled inside components.

15, 16, 17 & 18. Security after login — where do you configure it, and how do you protect a URL?

You configure it on the route, not in the component. Attach guards in the route config so the component is never even constructed for an unauthorized user.

```
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService), router = inject(Router);
  return auth.isLoggedIn()
    ? true
    : router.createUrlTree(['/login'], { queryParams: { returnUrl:
state.url } });
};

export const roleGuard = (role: string): CanActivateFn => () =>
  inject(AuthService).hasRole(role) || inject(Router).createUrlTree(['/
forbidden']);
```

Then: **canActivate** on the parent (`/admin`), **canActivateChild** for every child, **canMatch** to stop the lazy chunk from even downloading, and an **HTTP interceptor** attaching the JWT to every request.

● **The point that wins this question:** *"Guards are UX, not security — anyone can edit the JS bundle and reach the route. The API must enforce authorization server-side on every endpoint; the guard just stops the user seeing a screen that would fail anyway. On the Spring side that's Spring Security with role checks or `@PreAuthorize`."* **Say this.** Most candidates don't.

19. Which component can activate? (Route Guards)

The guard types: **CanActivate** (enter a route), **CanActivateChild** (enter any child), **CanDeactivate** (leave — "you have unsaved changes"), **CanMatch** (whether a route config matches at all — replaces the old **CanLoad**), **Resolve** (prefetch data).

▲ **Angular 15+ prefers functional guards** (**CanActivateFn** with **inject()**) over class-based ones — class guards are deprecated. Mentioning that shows currency.

20. When would you use a Resolver?

When the route **must not activate until the data is there** — so the component renders once, fully populated, with no empty flash and no `*ngIf="data"` scaffolding. Typical: a detail page where a missing record should redirect to 404 *before* navigating.

```
export const productResolver: ResolveFn<Product> = route =>
  inject(ProductService).getById(route.paramMap.get('id')!);
// component: this.product = this.route.snapshot.data['product'];
```

21. Does every component need a Resolver?

No — and usually you shouldn't. A resolver **blocks navigation** until the request finishes, so the app feels frozen on a slow network with no feedback. Prefer navigating immediately and showing a skeleton/spinner (`data$ | async`), and reserve resolvers for cases where partial rendering is genuinely wrong.

■ *"I use a resolver when the page is meaningless without the data or when a 404 should stop the navigation. Otherwise I navigate straight away and render a loading state — it feels faster even though the total time is the same."*

22. Lazy loading

Loading a feature's JavaScript **only when its route is visited**, instead of shipping everything in the initial bundle.

```
{ path: 'admin', loadChildren: () => import('./admin/admin.routes').then(m
=> m.ADMIN_ROUTES) }
{ path: 'settings', loadChildren: () => import('./
settings.component').then(m => m.SettingsComponent) }
```

Benefits: smaller initial bundle → faster first paint/TTI; users never download features they don't use.

23. Disadvantages of lazy loading ● (the one most candidates fumble)

1. **Delay on first navigation** to that route — the chunk downloads then; on a slow connection the user sees a blank pause. (Mitigate with `PreloadAllModules` or a custom preloading strategy.)
2. **More HTTP requests / chunk waterfalls** — many small chunks can be slower than one, especially pre-HTTP/2.
3. **Duplicated code across chunks** if shared dependencies aren't hoisted properly, which can *increase* total download size.
4. **Accidental multiple service instances** — a service provided in a lazy module gets its own injector instance instead of the root singleton. ▲ (Same hierarchy issue as Q12 — use `providedIn: 'root'`.)
5. **More complexity** — routing config, shared modules, guards (`canMatch`) all get harder to reason about.
6. **Errors surface late** — a broken lazy chunk only fails when someone navigates there, so it can escape testing.

24. RxJS

A library for **reactive programming with Observables** — treating events, HTTP responses and state as **streams** you compose with operators (`map` , `filter` , `switchMap` , `combineLatest` , `catchError` , `retry`). Angular uses it for `HttpClient` , router events, `valueChanges` , and `EventEmitter` . → 20

25. Reactive pattern

Programming with **asynchronous data streams and declarative composition**: instead of imperatively calling and storing, you describe how data flows and transforms, and the framework pushes values through it. It's **push-based** (the producer tells you) rather than pull-based (you ask). Benefits: less state to track, cancellation for free, and complex async coordination becomes a readable pipeline.

26. Observable vs Observer

- **Observable** = the **producer** — the stream that emits values.

- **Observer** = the **consumer** — the object with `next`, `error`, `complete` callbacks that you pass to `subscribe()`.

```
const observer = { next: v => console.log(v), error: e =>
console.error(e), complete: () => console.log('done') };
observable$.subscribe(observer);
```

Subject is unusual in being **both** — it's an Observable *and* an Observer, which is why you can `.next()` into it.

27. Hot vs Cold Observable ●

	Cold	Hot
Producer	Created per subscriber	Shared by all subscribers
Late subscriber	Gets the whole sequence from the start	Gets only what's emitted from now on
Analogy	A movie on demand — everyone starts at 0:00	A live broadcast — you join mid-stream
Examples	<code>HttpClient.get()</code> , <code>of</code> , <code>from</code> , <code>interval</code>	<code>Subject</code> , <code>BehaviorSubject</code> , <code>fromEvent(document, 'click')</code>

▲ **The practical consequence:** `http.get()` is **cold**, so **each subscription fires its own HTTP request** — subscribe three times and you make three calls. `shareReplay(1)` (or `share()`) makes it hot and shares one response.

```
readonly config$ = this.http.get<Config>('/api/
config').pipe(shareReplay(1)); // one request, many subscribers
```

28. Two independent APIs needed in a single subscription — which operator?

forkJoin — waits for **all** to complete, then emits one combined array/object. It's the RxJS `Promise.all`.

```
forkJoin({
  user: this.http.get<User>('/api/user'),
  orders: this.http.get<Order[]>('/api/orders'),
}).subscribe(({ user, orders }) => { ... }); // both requests fire in
parallel
```

Alternatives and when they're right: `combineLatest` when the sources keep emitting and you want the latest of each (it emits only after **every** source has emitted at least once); `zip` to pair emissions by index. For **one-shot HTTP calls**, `forkJoin` is the answer.

29. If you use `forkJoin`, what happens if one API call fails? ●

The entire `forkJoin` errors immediately and you lose *all* results — including the responses that already succeeded. Only the `error` callback runs; `next` never fires.

Fix — catch inside each inner observable so the failure becomes a value:

```
forkJoin({
  user: this.http.get<User>('/api/user').pipe(catchError(() =>
of(null))),
  orders: this.http.get<Order[]>('/api/orders').pipe(catchError(() =>
of([]))),
}).subscribe(({ user, orders }) => { /* runs even if one failed */ });
```

The rule: `catchError` inside the inner pipe = partial success. `catchError` outside, on the `forkJoin` itself = you still lose everything, you just handle the error gracefully. **Where you put it is the whole question.**

30. State management libraries

NgRx (Redux pattern — store/actions/reducers/selectors/effects), **NgRx ComponentStore** (local, per-component), **NgRx SignalStore**, **NGXS** (less boilerplate, decorator-based), **Akita/Elf**, or simply a **service with a BehaviorSubject** for smaller apps.

▲ Attribution: NgRx = EasyVisa, not RoboGebra. (21 — NgRx)

31. localStorage / sessionStorage — why use a state management library instead? ●

They solve **different problems**: storage is **persistence**, a store is **runtime state**.

	localStorage / sessionStorage	State library (NgRx)
Reactive	✗ Not reactive — nothing re-renders when it changes	✓ Selectors push updates to every subscriber
Data types	Strings only — manual <code>JSON.stringify/parse</code>	Real typed objects
API	Synchronous — blocks the main thread	Async-friendly, effects for side effects
Scope	Per origin, survives reload (<code>session</code> = per tab)	In-memory, lost on reload
Tooling	None	Redux DevTools — time-travel, action log
Structure	Ad-hoc keys, no single source of truth	Immutable single source of truth
Security	▲ Readable by any script — XSS-exposed	In memory only

■ "They're complementary, not alternatives. The store is the single reactive source of truth the UI renders from; storage is where I persist a slice of it — a token or user preferences — so it survives a reload. I'd hydrate the store from storage on bootstrap rather than reading storage from components. And I wouldn't put anything sensitive in localStorage because any XSS can read it."

32. Side effects

Anything outside pure state updates — HTTP calls, routing, localStorage, toasts, logging. In NgRx they live in **Effects**: listen for an action, do the async work, dispatch a success/failure action. Reducers stay **pure**.

```
loadUsers$ = createEffect(() => this.actions$.pipe(
  ofType(UserActions.load),
  switchMap(() => this.api.getUsers().pipe(
    map(users => UserActions.loadSuccess({ users })),
    catchError(error => of(UserActions.loadFailure({ error }))) // ▲
  INNER catchError
  ))
));
```

▲ If **catchError** is on the outer pipe, the effect dies after the first error and never runs again. Guaranteed follow-up. (21)

33. Angular 17 features

Built-in control flow `@if` / `@else` / `@for` (with mandatory `track`) / `@switch` — no `CommonModule` import, better type narrowing, faster; **deferrable views** `@defer (on viewport)` for lazy-loading a *component* declaratively; **esbuild/Vite** application builder by default (much faster builds); **SSR + hydration** as a first-class `ng new` option; standalone by default; angular.dev docs.

```
@if (user(); as u) { <p>{{ u.name }}</p> } @else { <app-login /> }
@for (item of items(); track item.id) { <li>{{ item.name }}</li> } @empty
{ <li>None</li> }
@defer (on viewport) { <app-heavy-chart /> } @placeholder { <div>Loading...
</div> }
```

34. Standalone component: username + password form, submit disabled until typed ●

```

import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/
forms';

@Component({
  selector: 'app-login',
  standalone: true,                                // ← standalone: no
  NgModule
  imports: [CommonModule, ReactiveFormsModule],      // ← imports live on
the component
  template: `
    <form [formGroup]="form" (ngSubmit)="onSubmit()">
      <input formControlName="username" placeholder="Username">
      <div class="error" *ngIf="isInvalid('username')">Username is
required</div>

      <input type="password" formControlName="password"
placeholder="Password">
      <div class="error" *ngIf="isInvalid('password')">Minimum 6
characters</div>

      <button type="submit" [disabled]="form.invalid">Login</button>
    </form>
  `,
})
export class LoginComponent {
  form = this.fb.nonNullable.group({
    username: ['', Validators.required],
    password: ['', [Validators.required, Validators.minLength(6)]],
  });

  constructor(private fb: FormBuilder) {}

  isInvalid(name: string): boolean {
    const c = this.form.get(name);
    return !!c && c.invalid && (c.dirty || c.touched);
  }

  onSubmit(): void {
    if (this.form.invalid) { this.form.markAllAsTouched(); return; }
    console.log(this.form.getRawValue());
  }
}

```


What they're checking: `standalone: true` + `imports` on the component (no `NgModule`), `ReactiveFormsModule` imported, `[disabled]="form.invalid"` for the button, and validators declared in the class. → 25

35. During sign-up, how do you check if the user already exists? ●

An **async validator** on the email/username control:

```
userExists(): AsyncValidatorFn {
  return (control: AbstractControl): Observable<ValidationErrors | null>
=> {
  if (!control.value) return of(null);
  return timer(400).pipe(                                // debounce
    keystrokes
      switchMap(() => this.api.checkUsername(control.value)), // cancels
the previous check
      map(exists => (exists ? { userExists: true } : null)),
      catchError(() => of(null)),                          // don't
block on API failure
      first()                                              // ▲ MUST
complete
  );
};
}
// username: ['', [Validators.required], [this.userExists()]] ← 3rd
argument
```

```
<div *ngIf="form.get('username')?.pending">Checking...</div>
<div *ngIf="form.get('username')?.hasError('userExists')">That username is
taken</div>
<button [disabled]="form.invalid || form.pending">Sign up</button>
```

The four things they're grading: async validators go in the **third** argument; the observable **must complete** (`first()`) or the control is stuck **PENDING** forever; debounce so you don't hit the API per keystroke; and — **the senior point** — *"the client check is a UX convenience; the real uniqueness constraint is a unique index in the database and a **409 Conflict** from the API, because two users can sign up in the same second."*

● Photon Interactive — Round 2 (Backend)

1 & 2. Is your application built on microservices? How many services?

✓ **Honest RoboGebra answer:** *"It's a microservice-style platform rather than a textbook microservices architecture. There are separate deployable services — the main Spring Boot backend, a separate admin Spring Boot service on port 4000, and a Node/Express CRM — plus the Angular web app, the Ionic mobile app and a React CRM UI, across about eight repos. Inside the main backend it's modular — around 70 domain modules each with controller, service, repository and model layers — but they deploy together, so I'd call that a modular monolith with a couple of satellite services rather than fine-grained microservices."*

▲ **Don't oversell.** Saying "modular monolith + satellite services" and being right beats claiming 20 microservices and collapsing on the follow-up ("how do you handle distributed transactions?"). → [18 · 08](#)

3. Interface and abstract class — real-time examples ●

Abstract class — a shared base with state and partial implementation:

```
abstract class PaymentProcessor {                                // shared state +
    template method
        protected final AuditLog log;
        PaymentProcessor(AuditLog log) { this.log = log; }

        public final void process(Payment p) {                    // template method —
fixed flow
            validate(p);
            doCharge(p);                                          // varies per subclass
            log.record(p);
        }
        protected void validate(Payment p) { /* shared */ }
        protected abstract void doCharge(Payment p);
    }
    class CardProcessor extends PaymentProcessor { protected void
doCharge(Payment p) { ... } }
    class UpiProcessor extends PaymentProcessor { protected void
doCharge(Payment p) { ... } }
```

Interface — a capability that unrelated classes can implement:

```
interface Exportable { byte[] toCsv(); }           // Invoice, Report and
User can all be Exportable
// Real-world: List, Comparable, Runnable, JpaRepository, Angular's OnInit
```

■ "Abstract class when subclasses genuinely share state and a flow — my payment processors share the audit log and the template method, and differ only in how they charge. Interface when the classes are unrelated but share a capability, or when I need multiple inheritance of type. Since Java 8 interfaces can have **default** methods, so the real line is state and constructors — only an abstract class has those." → **16 §0 Q1**

4. Create a Java string array with strings and numbers

```
String[] mixed = { "Priyanka", "42", "Angular", "2026" }; // numbers as
text
System.out.println(mixed[1]);                             // "42" — a
String
int n = Integer.parseInt(mixed[1]);                         // convert
when needed
```

If they mean genuinely mixed types (the likely trap):

```
Object[] mixed = { "Priyanka", 42, 3.14, true };           // Object[]
holds anything
for (Object o : mixed) System.out.println(o + " : " +
o.getClass().getSimpleName());
```

The point: a **String[]** can only ever hold **String**s — Java arrays are typed and **covariant**, so **Object[] a = new String[2]; a[0] = 42;** compiles but throws **ArrayStoreException** at runtime. Mention that and you've answered the question behind the question.

5, 6 & 7. Create an interface, create an object of it, try/catch

```
interface Greeting {
    String greet(String name);           // implicitly public abstract
    default String hello() { return greet("World"); } // Java 8 default
    method
}
```

"Create an object of that interface" — the trap. You **cannot** instantiate an **interface** (`new Greeting()` **X**). Three legal ways:

```
// 1. A concrete implementing class
class English implements Greeting { public String greet(String n) { return
"Hello " + n; } }
Greeting g1 = new English();

// 2. An anonymous inner class — this is what "new Greeting() { }" really
means
Greeting g2 = new Greeting() {
    @Override public String greet(String n) { return "Hi " + n; }
};

// 3. A lambda — only for a FUNCTIONAL interface (exactly one abstract
method)
Greeting g3 = n -> "Hey " + n;
```

■ *"You can't instantiate an interface directly — you instantiate something that implements it. `new Greeting() { ... }` looks like instantiating the interface but it's actually an anonymous class. And because `Greeting` has a single abstract method, it's a functional interface, so a lambda works too."*

try/catch:

```
try {
    int result = 10 / divisor;
} catch (ArithmeticException e) {           // most specific FIRST
    log.error("Divide by zero", e);
} catch (Exception e) {                   // broadest LAST
    log.error("Unexpected", e);
} finally {
    // always runs
}
```

▲ Catching a **broader** exception before a narrower one is a **compile error** (unreachable catch block).

8. **finally** block

Runs **whether or not** an exception is thrown, and **even if the try or catch block returns** — it's for cleanup: closing connections, files, streams, releasing locks. **The only cases it doesn't run:** `System.exit()`, a JVM crash, or the thread being killed.

9. What happens if you remove the **catch** block? ●

try – finally without a **catch** is perfectly legal and compiles. What changes:

- The **finally** block still runs (cleanup is guaranteed).
- Then the exception **propagates up** to the caller — this method does **not** handle it.
- If nothing up the stack handles it, the thread terminates and the stack trace is printed.

```
try {  
    riskyOperation();  
} finally {  
    connection.close();           // ✓ still runs, then the exception  
    keeps propagating  
}
```

▲ **try** completely alone — no **catch** and no **finally** — IS a compile error. You need at least one of them.

▲ **The killer follow-up:** "What if **finally** has a **return**?" → it **swallows the exception entirely** and the method returns normally. Never put a **return** in **finally**. (See 23 §H.)

10. If the try block succeeds, what is the purpose of **finally**?

Its purpose isn't the failure path — it's the guarantee. **finally** runs on **every** exit path: success, exception, and even an early **return** from inside **try**. That's

what makes it the right place for cleanup, because you write the release **once** instead of duplicating it in the happy path and every catch block.

```
Connection conn = null;
try {
    conn = ds.getConnection();
    return query(conn);          // ▲ even this return doesn't skip
finally
} finally {
    if (conn != null) conn.close(); // runs after the return value is
    computed
}
```

✓ **Land it here:** *"In modern Java I'd use try-with-resources instead — anything implementing **AutoCloseable** gets closed automatically, in reverse order, and it correctly handles an exception thrown by **close()** itself by adding it as a suppressed exception. That's cleaner than a manual **finally**."*

```
try (Connection conn = ds.getConnection();
    PreparedStatement ps = conn.prepareStatement(SQL)) {
    return query(ps);
} // both closed automatically, ps first
```

11. Spring Boot validation

```
// 1. Annotate the DTO (jakarta.validation – Boot 3 uses jakarta, not
javax)
public record UserDto(
    @NotBlank(message = "Name is required")           String name,
    @Email @NotBlank                                   String email,
    @Min(18) @Max(120)                                 int age,
    @Size(min = 8, message = "Min 8 characters")       String password,
    @Pattern(regexp = "\\d{10}$")                      String phone,
    @NotNull @Valid                                     Address
    address // @Valid = cascade to nested
) {}

// 2. Trigger it with @Valid on the controller parameter
@PostMapping("/users")
public ResponseEntity<UserDto> create(@Valid @RequestBody UserDto dto)
{ ... }

// 3. Turn the failure into a clean response
@RestControllerAdvice
public class ValidationHandler {
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
    handle(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
            .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
        return ResponseEntity.badRequest().body(errors); // 400
    }
    with field-level messages
}
```

Points that earn marks:

- Needs **spring-boot-starter-validation** on the classpath — it's not in **starter-web** since Boot 2.3.
- **@Valid** vs **@Validated** — **@Valid** is the JSR-380 standard (method params, nested objects); **@Validated** is Spring's, and it's what enables **validation groups** and class-level validation on **@Service** beans.
- **@NotNull** vs **@NotEmpty** vs **@NotBlank** — not null / not null and size>0 / not null and contains non-whitespace. ▲ A guaranteed follow-up.
- The failure is **MethodArgumentNotValidException** for **@RequestBody**, but **ConstraintViolationException** for **@RequestParam** / **@PathVariable** on a

`@Validated` class — handle both.

- **Custom validator:** a `@Constraint` annotation + a `ConstraintValidator<A, T>` implementation.

● Codeboard Technology — Level 1 (single round)

Single round → onboarding. The Java portion was output-based, not theory — that's the lesson that cost it.

#	Question	Answer
1	Abstract class	16 \$0 Q1 — + abstract vs interface table
2	Data hiding	16 \$0 Q2 — + encapsulation vs abstraction + access modifiers
3	Output: <code>methodTest(null)</code>	Calling String method — most-specific overload wins at compile time
4	Output: override adding <code>throws IOException</code>	Compile error — can't widen the checked-exception clause
5	Ionic lifecycle hooks	<code>ionViewWillEnter</code> / <code>DidEnter</code> / <code>WillLeave</code> / <code>DidLeave</code>
6	Angular vs Ionic lifecycle hooks	Ionic caches pages in the nav stack → <code>ngOnInit</code> doesn't re-fire; <code>ionViewWillEnter</code> does
7	Offline support in Ionic	5 layers: detect → store → cache reads → queue writes → sync

- **Full answers with follow-ups: 16 §0 — What they actually asked.**
- **The real lesson:** they asked "what does this print?", not "what is polymorphism?". **Do 23 — Java Predict the Output before every Java round from now on.**

● Mphasis — Technical Round (6 Aug 2026)

- **RESULT: CLEARED.** Mphasis is moving her forward to **Level 2** plus a **client technical round** — the first time in six companies a next round has been offered.
- ◆ **Prep for those two rounds: 30 — Mphasis Level 2 + Client Round.** HR's brief: "Java only mostly. With Angular, what you have used in recent projects they'll ask. Basics of Angular not mandatory."
- **The 10 questions below are the L2 syllabus, not just history — L2** interviewers read the L1 feedback form and re-probe the same threads one level deeper. File 30 Part A lists the expected follow-up to each one.

Backend-weighted Java + Spring Boot + microservices round — 10 questions. Mostly theory, plus one coding problem (thread-safe Singleton) and one SQL query. Notably: no Angular at all, and no predict-the-output — a different shape from Codeboard.

#	Question	Type
1	What is Microservices?	Theory
2	Design patterns in microservices	Theory ●
3	CircuitBreaker annotation	Theory
4	Which dependency is added for circuit breaker?	Theory ▲
5	Write a program for a thread-safe Singleton class	Coding ●

#	Question	Type
6	Spring Boot Actuator	Theory
7	Difference between Association and Aggregation in OOP	Theory
8	Total employees + average salary per department	SQL ●
9	What is deadlock in multithreading?	Theory
10	How do you improve API performance in backend and frontend?	Theory / experience

1. What is Microservices?

"Microservices is an architectural style where an application is built as a set of small, independently deployable services, each owning one business capability and its own database, communicating over lightweight protocols — usually REST or messaging. The opposite is a monolith, where everything ships as one unit."

The second sentence — from your own project:

"On RoboGebra we had 8 separate repositories with independent Spring Boot services, so a change to one service could be deployed without redeploying the whole platform."

The third sentence — the trade-off. Say this unprompted:

*"The cost is real though: distributed transactions instead of a single **@Transactional**, network calls that can fail, eventual consistency, and much harder debugging and monitoring. For a small team or an unclear domain a well-structured monolith is usually the better first move — you split it once you know where the seams actually are."*

Monolith	Microservices
One deployable unit	Many, deployed independently
One shared database	Database per service
In-process method calls	Network calls (REST / gRPC / message queue)
Scale the whole app	Scale only the hot service
Simple to debug & test	Needs distributed tracing, log aggregation
One tech stack	Polyglot possible
ACID transactions	Saga / eventual consistency

■ Full depth: **08 — Microservices Basics**. Photon Round 2 asked the same thing ("is your application built on microservices, how many services?") — this is now a **2-round repeat**.

2. Design patterns in microservices ●

The ones to name — group them, don't list randomly. Grouping is what sounds senior.

Group	Pattern	What it solves
Decomposition	Decompose by business capability / subdomain	How to split the monolith
	Strangler Fig	Migrating a monolith incrementally instead of a big-bang rewrite
Data	Database per Service	Each service owns its schema — no shared tables
	Saga (choreography or orchestration)	Distributed transactions without 2PC; compensating actions on failure

Group	Pattern	What it solves
	CQRS	Separate read and write models when their needs diverge
	Event Sourcing	Store the sequence of events, not just current state
	API Composition / Aggregator	Join data that lives in several services
Communication	API Gateway	One entry point: routing, auth, rate limiting, SSL termination
	BFF (Backend for Frontend)	A tailored gateway per client — web vs mobile
	Service Discovery (Eureka, Consul)	Services find each other without hard-coded hosts
Reliability	Circuit Breaker	Stop calling a failing service; fail fast (→ Q3)
	Retry + Timeout + Backoff	Handle transient failures without hammering
	Bulkhead	Isolate thread pools so one slow service can't exhaust all threads
	Sidecar / Ambassador	Cross-cutting concerns as a separate process (service mesh)
Observability	Log Aggregation (ELK)	One place to search logs across services
	Distributed Tracing (Sleuth/ Micrometer + Zipkin)	Follow one request across services via a correlation id
	Health Check API	Load balancers know who's alive (→ Q6, <i>Actuator</i>)
Config	Externalized Configuration (Spring Cloud Config)	Config outside the artefact, per environment

If they want depth, pick Saga — it's the one they're usually fishing for:

"Saga handles a transaction spanning services, because you can't hold a database transaction across a network. It's a sequence of local transactions where each one publishes an event triggering the next, and if a step fails you run compensating transactions to undo the earlier ones. Two flavours: choreography, where services react to each other's events — simple but the flow is implicit and hard to trace; and orchestration, where a central coordinator drives the steps — easier to reason about but it's another component to run. For an order flow across Order, Payment and Inventory I'd choose orchestration, because 'what state is this order in?' has to be answerable."

3. CircuitBreaker annotation

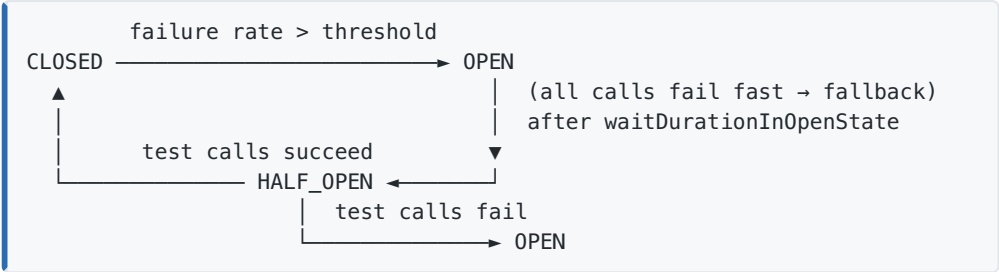
```
@Service
public class InventoryClient {

    @CircuitBreaker(name = "inventoryService", fallbackMethod =
"fallbackStock")
    @Retry(name = "inventoryService")
    @TimeLimiter(name = "inventoryService")
    public String getStock(String sku) {
        return restTemplate.getForObject("http://inventory/stock/" + sku,
String.class);
    }

    // ▲ Must have the SAME return type + the SAME parameters + a
    Throwable as the last param
    private String fallbackStock(String sku, Throwable t) {
        return "UNAVAILABLE";
    }
}
```

*"**@CircuitBreaker** is from Resilience4j — [io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker](https://github.com/resilience4j/circuitbreaker-annotation). It wraps the method so that when the failure rate crosses a threshold, the breaker opens and subsequent calls fail immediately into the fallback method instead of waiting on a dead service. That protects the caller's threads — without it, one slow downstream service exhausts your thread pool and takes your service down too."*

The three states — draw them if there's a whiteboard:



State	Behaviour
CLOSED	Normal — calls pass through, failures are counted
OPEN	Calls fail immediately into the fallback; the downstream service is left alone to recover
HALF_OPEN	After a wait, a few trial calls are permitted — succeed → CLOSED, fail → OPEN again

Configuration lives in `application.yml` , not the annotation:

```
resilience4j:
  circuitbreaker:
    instances:
      inventoryService:
        slidingWindowSize: 10
        failureRateThreshold: 50          # % of failures that opens the
breaker
        waitDurationInOpenState: 10s
        permittedNumberOfCallsInHalfOpenState: 3
        registerHealthIndicator: true    # surfaces in /actuator/health
→ Q6
```

**** The legacy answer they may be testing for: *"The older Netflix option was Hystrix — @HystrixCommand(fallbackMethod = "...") with @EnableCircuitBreaker on the main class.*** Hystrix has been in maintenance mode since 2018* and Spring Cloud removed it, so Resilience4j is the current answer. If a codebase still has Hystrix, that's a migration item."

4. Which dependency is added for circuit breaker? ▲

Spring Boot 3 + Resilience4j (the current answer):

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>    <!-- ...-spring-
boot2 for Boot 2.x -->
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>      <!-- ● REQUIRED
-->
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId> <!-- to see
breaker state -->
</dependency>
```

Or the Spring Cloud starter, which pulls in the above:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</
artifactId>
</dependency>
```

● **The detail that earns the mark:** " *spring-boot-starter-aop* is mandatory and it's the thing people miss. The annotations are implemented with AOP proxies — without it the app starts fine, the annotation is silently ignored, and there's no circuit breaker at all. Same class of failure as a *@Transactional* that does nothing."

And the second AOP trap, which is a classic follow-up:

"Because it's proxy-based, calling an annotated method from within the same class bypasses the proxy and the breaker never fires. *this.getStock()* is a plain Java call. The fix is to move the annotated method into a separate bean and inject it — exactly the same self-invocation problem as *@Transactional*."

- That self-invocation trap is covered in [24 — ORM/JPA §@Transactional](#).

5. Write a program for a thread-safe Singleton class ● (coding)

Lead with this one — it's the best answer, and short:

```
// ✓ BILL PUGH / static holder – lazy, thread-safe, no synchronization cost
public class Singleton {

    private Singleton() { }

    private static class Holder {
        private static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}
```

"The inner class isn't loaded until `getInstance()` is called for the first time, so it's lazy. And the JVM guarantees class initialization is thread-safe — the classloader holds a lock — so there's no `synchronized` in my code and no cost on every subsequent call. This is my default."

If they specifically ask for double-checked locking:


```

public class Singleton {

    private static volatile Singleton instance;    // ● volatile is
    ESSENTIAL

    private Singleton() { }

    public static Singleton getInstance() {
        if (instance == null) {                    // 1st check – no
lock, fast path
            synchronized (Singleton.class) {
                if (instance == null) {            // 2nd check – under
the lock
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}

```

● **Why `volatile` ? This is the whole question.** "*`new Singleton()` is three steps — allocate memory, run the constructor, assign the reference. Without `volatile` the JIT is allowed to reorder steps 2 and 3, so another thread can see a non-null reference pointing at a partially constructed object and use it. `volatile` forbids that reordering and guarantees visibility across threads. Double-checked locking was famously broken in Java before the Java 5 memory model fixed `volatile`.*"

The other approaches — know the trade-offs:

```
// Eager – thread-safe (class init), but built even if never used
public class Singleton {
    private static final Singleton INSTANCE = new Singleton();
    private Singleton() { }
    public static Singleton getInstance() { return INSTANCE; }
}

// Synchronized method – correct but slow: EVERY call takes the lock,
// forever
public static synchronized Singleton getInstance() {
    if (instance == null) instance = new Singleton();
    return instance;
}

// ✓ ENUM – Joshua Bloch's recommendation in Effective Java
public enum Singleton {
    INSTANCE;
    public void doWork() { }
}
```

▲ The follow-up they ask next: "Can your singleton be broken?"

Attack	Breaks Bill Pugh / DCL?	Breaks enum?
Reflection — <code>setAccessible(true)</code> on the private constructor	✓ yes	✗ no — the JVM forbids reflective enum instantiation
Serialization — deserializing creates a new instance	✓ yes	✗ no
Cloning	✓ if it implements <code>Cloneable</code>	✗ no
Multiple classloaders	✓ yes	✓ yes

```
// Reflection breaking it:
Constructor<Singleton> c = Singleton.class.getDeclaredConstructor();
c.setAccessible(true);
Singleton second = c.newInstance();           // ▲ a second instance exists

// Defences:
private Singleton() {
    if (Holder.INSTANCE != null) throw new IllegalStateException("Use
getInstance()");
}
protected Object readResolve() { return getInstance(); } //
serialization
```

The closing line: *"Enum handles reflection, serialization and cloning for free, which is why Effective Java recommends it. The reason people still use the holder pattern is that an enum can't extend a class and can't take constructor arguments — so for a singleton that needs injected dependencies I'd let Spring manage it instead, since a Spring bean is singleton-scoped by default and this whole problem disappears."*

That last sentence is the senior move: it connects the puzzle back to how you'd actually build it.

6. Spring Boot Actuator

"Actuator adds production-ready endpoints for monitoring and managing a running app — health, metrics, environment, beans, mappings, log levels — without writing any of it yourself. In a microservices setup it's what a load balancer or Kubernetes probes to decide whether an instance is alive."

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Endpoint	Shows
<code>/actuator/health</code>	UP/DOWN — plus DB, disk, Redis, circuit breaker sub-checks
<code>/actuator/info</code>	Build version, git commit
<code>/actuator/metrics</code>	JVM memory, threads, HTTP request timings, connection pool
<code>/actuator/env</code>	▲ All config properties — leaks secrets
<code>/actuator/beans</code>	Every bean in the context
<code>/actuator/mappings</code>	All request mappings — useful for "why is my endpoint 404?"
<code>/actuator/loggers</code>	Read and change log levels at runtime , no restart
<code>/actuator/heapdump</code>	● Thread dump — this is how you diagnose a deadlock (→ Q9)
<code>/actuator/heapdump</code>	▲ Downloads the heap — leaks everything in memory
<code>/actuator/prometheus</code>	Metrics in Prometheus format (via Micrometer)

```
management:
  endpoints.web.exposure.include: health,info,metrics,prometheus
  endpoint.health.show-details: when-authorized
```

Two points that earn marks:

- "By default only `/health` and `/info` are exposed over HTTP — the rest are opt-in. That's a deliberate secure default."
- "`/env` and `/heapdump` must never be publicly reachable — they expose connection strings, passwords and everything in memory. I'd put actuator on a separate management port and behind auth. It's a real breach vector, not a theoretical one."

Custom health check — a likely follow-up:

```

@Component
public class PaymentGatewayHealth implements HealthIndicator {
    @Override
    public Health health() {
        return gateway.ping()
            ? Health.up().withDetail("latencyMs", 12).build()
            : Health.down().withDetail("reason", "timeout").build();
    }
}

```

■ 06 — Spring Boot

7. Difference between Association and Aggregation in OOP

The key sentence: *"Association is the general relationship; aggregation and composition are two specialised kinds of association, and they differ by who owns the lifecycle."*

	Association	Aggregation	Composition
Relationship	"uses-a" / general link	"has-a", weak ownership	" part-of ", strong ownership
Lifecycle	Fully independent	Part survives the whole	Part dies with the whole
UML	plain line	hollow ◇ diamond	filled ◆ diamond
Example	Teacher ↔ Student	Department has Employees	House has Rooms / Car has Engine
Who creates the part?	Neither owns it	Passed in from outside	Created inside the owner

```
// — ASSOCIATION — two independent classes that just interact
class Teacher {
    void teach(Student s) { }          // no ownership either way
}

// — AGGREGATION — Department HAS employees, but they outlive it
class Department {
    private List<Employee> employees;

    Department(List<Employee> employees) {    // ♦ passed IN — created elsewhere
        this.employees = employees;
    }
}

// Delete the Department → the Employee objects still exist and can join another one.

// — COMPOSITION — the Engine cannot exist without the Car
class Car {
    private final Engine engine = new Engine();    // ♦ created INSIDE, owned

    class Engine { }
}

// Destroy the Car → the Engine goes with it. Nothing else holds a reference.
```

● **The discriminator to say out loud:** *"The test I use is — can the part exist on its own after the whole is destroyed? If yes it's aggregation, if no it's composition. And practically, look at who calls **new**: if the object is passed into the constructor it's aggregation; if the owner creates it internally, it's composition."*

The likely follow-up — "which do you prefer?"

"Composition, generally — 'favour composition over inheritance'. Inheritance is a compile-time IS-A that you can't change; composition is a runtime HAS-A that you can swap, which makes it far more testable. That's the Strategy pattern, and it's the reason Spring's constructor injection is composition rather than a class hierarchy."

■ 17 — SOLID & Design Patterns

8. Total employees and average salary per department ● (SQL)

■ Full depth on this and every other SQL question → [36 — SQL: The Complete Interview File](#).

Given:

```
dept      ( dept_id, dept_name )
employee  ( emp_id, emp_name, salary, dept_id )
```

The answer:

```
SELECT  d.dept_id,
        d.dept_name,
        COUNT(e.emp_id)                AS total_employees,
        ROUND(AVG(e.salary), 2)        AS average_salary
FROM    dept d
LEFT JOIN employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY d.dept_name;
```

● Three things here separate a junior answer from a mid-level one. Say them:

① **LEFT JOIN**, not **INNER JOIN**.

*"With an inner join, a department with zero employees disappears from the result entirely. The question says 'total employees per department', and zero is a valid answer — so I use a left join from **dept**."*

② **COUNT(e.emp_id)**, not **COUNT(*)**.

*"This is the subtle one. With a **LEFT JOIN**, an empty department still produces one row with all-NULL employee columns. **COUNT(*)** counts that row and returns 1, which is wrong. **COUNT(column)** skips NULLs and correctly returns 0."*

③ Every non-aggregated column must be in **GROUP BY**.

"*dept_name* is selected but not aggregated, so it has to be in the **GROUP BY**. MySQL is lenient about this by default; Postgres, SQL Server and MySQL in **ONLY_FULL_GROUP_BY** mode reject it."

Bonus: **AVG** ignores NULLs, so an empty department returns **NULL** for average salary. If they want **0**:

```
COALESCE(ROUND(AVG(e.salary), 2), 0) AS average_salary
```

▲ The follow-ups they ask after this query — have these ready

"Only departments with more than 5 employees?" → **HAVING**, not **WHERE**:

```
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(e.emp_id) > 5;
```

"**WHERE** filters rows before grouping; **HAVING** filters groups after aggregation. You can't put an aggregate in **WHERE** because it doesn't exist yet at that point."

"Employees earning more than their department's average" → correlated subquery:

```
SELECT e.emp_name, e.salary, d.dept_name
FROM   employee e
JOIN   dept d ON d.dept_id = e.dept_id
WHERE  e.salary > (SELECT AVG(e2.salary)
                  FROM   employee e2
                  WHERE  e2.dept_id = e.dept_id);
```

...or the window-function version, which is faster and scores better:

```
SELECT emp_name, salary, dept_id
FROM ( SELECT e.*, AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
      FROM   employee e ) t
WHERE salary > dept_avg;
```

"Second-highest salary" — asked in a huge share of rounds:


```

SELECT MAX(salary) FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);           -- classic

SELECT DISTINCT salary FROM employee ORDER BY salary DESC LIMIT 1 OFFSET
1;  -- MySQL/Postgres

SELECT salary FROM (                                         -- handles
ties correctly
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rnk FROM
employee
) t WHERE rnk = 2;

```

"Highest-paid employee per department":

```

SELECT dept_id, emp_name, salary FROM (
    SELECT e.*, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC)
    rnk
    FROM    employee e
) t WHERE rnk = 1;

```

- The Java Streams equivalent of that last one — *highest salary per department* — was **Altimetrik's Q1** and is in [22 — Streams](#). Worth noticing that **the same problem gets asked in both SQL and Streams form**.

9. What is deadlock in multithreading?

"A deadlock is when two or more threads are each holding a lock the other one needs, so none of them can ever proceed. The application doesn't crash — it just hangs, which is what makes it nasty to diagnose."

```

public class DeadlockDemo {

    private static final Object LOCK_A = new Object();
    private static final Object LOCK_B = new Object();

    public static void main(String[] args) {

        new Thread(() -> {
            synchronized (LOCK_A) {                // T1 takes A
                sleep(100);
                synchronized (LOCK_B) {            // ...then wants B
                    System.out.println("T1 done");
                }
            }
        }, "T1").start();

        new Thread(() -> {
            synchronized (LOCK_B) {                // T2 takes B ←
                sleep(100);
                synchronized (LOCK_A) {            // ...then wants A
                    System.out.println("T2 done");
                }
            }
        }, "T2").start();

        OPPOSITE ORDER

        static void sleep(long ms) {
            try { Thread.sleep(ms); } catch (InterruptedException e)
        { Thread.currentThread().interrupt(); }
        }
    }

    // Neither line ever prints. The JVM hangs.
}

```

The four Coffman conditions — all four must hold, so breaking any one prevents deadlock:

#	Condition	How you break it
1	Mutual exclusion — a resource is held exclusively	Use immutable data / lock-free structures
2	Hold and wait — hold one lock while requesting another	Acquire everything at once, or nothing

#	Condition	How you break it
3	No preemption — a lock can't be forcibly taken	tryLock with a timeout
4	Circular wait — A waits for B, B waits for A	● Global lock ordering — the standard fix

The two fixes, in order of preference:

```
// ✓ FIX 1 – LOCK ORDERING. Every thread takes locks in the same global
order.
synchronized (LOCK_A) { synchronized (LOCK_B) { } } // both threads,
same order
```

"Lock ordering is the fix I'd reach for first — if every thread acquires in the same order, a cycle is impossible. For dynamic objects you can order by something stable like `System.identityHashCode`."

```
// ✓ FIX 2 – tryLock with a timeout, so a thread gives up instead of
waiting forever
if (lockA.tryLock(1, TimeUnit.SECONDS)) {
    try {
        if (lockB.tryLock(1, TimeUnit.SECONDS)) {
            try { /* work */ } finally { lockB.unlock(); }
        } else { /* back off and retry */ }
    } finally { lockA.unlock(); }
}
```

▲ "How would you detect one in production?" — this is the follow-up, and it's where Q6 pays off:

"Take a thread dump — `jstack <pid>`, or `/actuator/threads` if Actuator is on. The JVM literally prints `Found one Java-level deadlock:` and names the two threads and the locks. JConsole and VisualVM show it visually, and programmatically `ThreadMXBean.findDeadlockedThreads()` returns the thread ids — you can wire that into a health check."

"Deadlock vs livelock vs starvation?" — the standard trio follow-up:

	What happens
Deadlock	Threads blocked forever, waiting on each other. CPU idle.
Livelock	Threads keep responding to each other and never progress — like two people stepping aside in a corridor. CPU busy , no work done.
Starvation	A thread never gets scheduled or never wins the lock, because higher-priority/greedier threads keep taking it.

The closing line that scores: *"Honestly, the best prevention is not writing the locks at all — use `java.util.concurrent : ExecutorService`, `ConcurrentHashMap`, `BlockingQueue`, atomic classes. Most deadlocks come from hand-rolled `synchronized` blocks that a higher-level utility would have handled."*

■ 05 — Core Java §Multithreading

10. How do you improve API performance — backend and frontend?

★ **Answer this in layers and lead with "measure first."** That framing alone separates you from a candidate reciting a list.

Open with:

"First I'd measure rather than guess — where is the time actually going? Server processing, database, network, or rendering? I'd use Actuator metrics or APM on the backend and the browser Network/Performance tab on the frontend. Then I'd fix in this order, because that's roughly the order of impact."

Backend

Layer	What I'd do
● Database	Fix N+1 with <code>JOIN FETCH</code> / <code>@EntityGraph</code> ; add indexes on filtered and joined columns; paginate instead of returning everything; check the execution plan
Payload	Return DTOs / projections , not full entities — stop over-fetching columns and lazy relations
Caching	<code>@Cacheable</code> + Redis for hot reads; HTTP caching with ETag / Cache-Control so the client skips the call entirely
Connections	Tune the HikariCP pool — an undersized pool looks exactly like a slow database
Concurrency	<code>@Async</code> / non-blocking for independent work; move long jobs to a queue and return 202 Accepted
Chattiness	Batch endpoints so the client makes 1 call instead of 20
Transport	Gzip/Brotli compression; HTTP/2
Resilience	Circuit breaker + timeouts (→ Q3) — a hung downstream call is a performance problem, not just a reliability one

Frontend *(your strength — spend time here)*

Technique	Why
● <code>debounceTime</code> + <code>distinctUntilChanged</code>	Stop firing a request per keystroke on search
● <code>switchMap</code>	Cancels the in-flight request when a new one starts — no wasted work, no out-of-order responses
<code>shareReplay(1)</code>	Multiple subscribers share one HTTP call instead of triggering several

Technique	Why
<code>forkJoin</code>	Fire independent calls in parallel instead of sequentially
HTTP interceptor cache	Serve repeat GETs from memory
Pagination / virtual scrolling	Never render 10,000 rows
<code>trackBy</code> in <code>*ngFor</code>	Angular reuses DOM nodes instead of rebuilding the list
<code>OnPush</code> change detection	Cuts change-detection work dramatically on large trees
Lazy loading + code splitting	Smaller initial bundle → faster first paint
Optimistic UI / skeletons	<i>Perceived</i> performance — often the one users actually notice

◆ **Close with something concrete from your own work:** *"On the Angular side I've used `debounceTime` with `switchMap` to stop a search box firing a request per keystroke — that's the same fix as the duplicate-API-call-on-rapid-clicks problem. And the biggest backend win I've seen is almost always the data layer: one N+1 or one missing index costs more than every micro-optimisation put together."*

■ **20 — RxJS · 24 — ORM/JPA §N+1** · Altimetrik Q9 above (duplicate calls on rapid clicks) is the **same technique** — that's now a **2-round repeat**.

■ What this round tells you

1. **It was a backend round with no Angular at all.** Five of ten questions were microservices/Spring (Q1–Q4, Q6). Your resume reads full-stack, but some companies will drill **only** the Java side — so Java/Spring depth can't be the part you revise last.
2. **Two of ten were hands-on** — a coding problem (Singleton) and a SQL query. Both are *writing*, not talking. **Type them, don't read them.**

3. **SQL appeared for the first time in six rounds.** Nothing in this pack covered it before today. If more rounds ask, that's a gap worth its own file — the joins/`GROUP BY` / `HAVING` / window-function set above is the core of it.
4. **Q3 and Q4 are the same topic split in two** — concept, then "*which dependency?*". Expect that pattern: they check whether you've actually wired it up or only read about it. The `spring-boot-starter-aop` answer is the tell.
5. **Q10 was your best opportunity in the round** — an open question where you can lead with your real Angular experience. Prepare the layered answer; open-ended questions are where you can out-answer a backend-only candidate.

◆ Virtusa — Round 2 (Java core), Aug 2026

✗ **Not cleared.** A **Java-fundamentals** round — no Angular, no framework. It went deep on **one topic** rather than broad, which is the opposite of the Tech Mahindra round below. Full answers: **31 — HashMap Internals**, **32 — Multithreading**, **33 — Interface vs Abstract Class**.

1. How does HashMap internally work?

The 5 beats they wanted: `Node<K,V>[] table` (array of buckets) → `hash()` spreads with `h ^ (h >>> 16)` → index via `(n - 1) & hash` (power-of-two capacity makes `&` a fast modulo) → each bucket is a **linked list of Node s** chained by `next` → **treeified at 8 nodes** if the table is ≥ 64 → **resize doubles at load factor 0.75**. Average $O(1)$, worst $O(\log n)$. → **31 §Part 0**

2. What is a bucket/bin in a HashMap?

One slot of the internal array — `table[i]`. Not a class. It holds either `null`, the head `Node` of a linked list, or the root `TreeNode` of a red-black tree. → **31 §Part 2**

3. What is the internal structure of a bucket?

A chain of `Node<K,V>` objects: `final int hash` (cached — computed once), `final K key`, `V value`, `Node<K,V> next`. The `next` pointers are the linked list; there is no separate list object. After treeification the nodes become `TreeNode`s, which keep the `next / prev` links **as well as** the tree pointers — which is how a bin can be untreeified cheaply on resize. → [31 §Part 1](#)

4. Does a bucket contain a linked list?

Yes by default — but not always. A singly linked list of `HashMap.Node` objects (▲ **not** a `java.util.LinkedList` — that answer sounds right and is wrong). Since Java 8, a bucket holding **8** nodes in a table of length ≥ 64 converts to a **red-black tree** — $O(n) \rightarrow O(\log n)$, which was added both for performance and as a defence against hash-collision DoS. It reverts to a list at **6** during a resize; the 8/6 gap prevents convert-revert thrashing. → [31 §Part 5](#)

Also probed in this round: multithreading and **interface vs abstract class** — hence files [32](#) and [33](#).

■ **What this round says:** Virtusa drilled **one** topic to the floor. That is the "second question" pattern from §What the pattern says, in its purest form — the opening question is easy and the round is decided by the **third** follow-up. `HashMap` internals has now been asked at **Altimetrik**, **Mphasis L1** and **Virtusa R2** — it is the single most-repeated Java question in this log.

● Tech Mahindra — Angular Developer round, Aug 2026

● **RESULT PENDING.** A **breadth sweep** — 15 short questions across Angular, TypeScript, the HTTP layer, core JavaScript and HTML/CSS. Full answers to every one: [34 — Tech Mahindra Angular Round](#).

#	Question	Notes
1	State management	Answer as a ladder : component → <code>@Input</code> / <code>@Output</code> → service + <code>BehaviorSubject</code> → NgRx
2	NgRx and its features	Store, actions, reducers, selectors, effects + <code>@ngrx/</code> <code>entity</code> , DevTools, ComponentStore
3	"Any idea of TypeScript?"	Don't answer "yes" — 45 seconds of range: interfaces, generics, unions, utility types, strict mode
4	HTTP interceptor and its benefits	Auth, errors, loading, logging, retry, caching + clone the request , order, <code>multi:</code> <code>true</code>
5	String interpolation	<code>{{ }}</code> , one-way, sanitized, no side-effects; don't call methods in templates
6	Observables vs Promises	★ 4th round in a row — stream/lazy/cancellable/ operators vs single/eager/ uncancellable
7	HttpClient — features & benefits	Observables, typed JSON, interceptors, <code>HttpErrorResponse</code> , progress events, testing, XSRF
8	Callback, callback hell	Pyramid of doom → Promises → async/await → RxJS; add <code>Promise.all</code> for independent calls

#	Question	Notes
9	Event capturing and bubbling	3 phases, <code>addEventListener(..., true)</code> , <code>stopPropagation</code> vs <code>preventDefault</code> , delegation
10	ES5 vs ES6	<code>let / const</code> , arrows, classes, template literals, destructuring, spread, Promises, modules
11	Arrow functions	The point is lexical this — plus no <code>arguments</code> , not a constructor, wrong for object methods
12	OOP in TypeScript	4 pillars + <code>private</code> vs <code>#private</code> , structural typing , one class / many interfaces
13	Modules — how, and types	Both meanings: ESM/ CommonJS/AMD/UMD and Angular root/feature/shared/ core/routing/lazy + standalone
14	<code>unknown</code> type and <code>never</code> type	<code>unknown</code> = type-safe <code>any</code> (narrow before use); <code>never</code> = no value → exhaustiveness check
15	HTML and CSS	Semantics, box model, positioning, Flexbox vs Grid, specificity, responsive

■ **What this round says:** a **breadth** round is won with crisp 3-sentence answers, not long ones — and it confirms the repeat list. **Observables vs Promises has now been asked in 4 rounds; interceptors in 3; state management/NgRx in 3.** Those aren't "likely" questions any more, they're certainties. If Tech Mahindra calls

back, expect the inverse: 3 of these drilled deep, most likely NgRx effects, `switchMap` vs `mergeMap`, and a live component.

● Altimetrik — Fullstack Java + Angular round (31 Aug 2026)

★ Attended Sun 31 Aug 2026 · result pending. This is your **SECOND Altimetrik round** — the first (Level 1 Java + Angular, 24 questions) is [§ above](#).

Every one of these 22 questions is answered in full in → [37 — Altimetrik Fullstack Java + Angular \(31 Aug\)](#).

Shape: 2 Java (records + pattern matching) → 10 Spring/Spring Boot → 2 microservices → 1 config/environments → 6 Angular → 1 Java Streams live-coding. Not a breadth sweep — a *depth* round on Spring wiring.

#	Question	Notes
1	● What is a record ? Why are we using it?	The opening question. Transparent immutable data carrier (Java 16) — compiler generates ctor, <code>name()</code> accessors, <code>equals</code> / <code>hashCode</code> / <code>toString</code>; implicitly <code>final</code>; compact constructor for validation. Why: kills DTO boilerplate, immutable/thread-safe/safe map key, correct <code>equals</code> for free, signals intent. Traps: shallow immutability, <code>name()</code> not <code>getName()</code>, can't be a JPA <code>@Entity</code>; pairs with sealed types + pattern matching

#	Question	Notes
2	Pattern matching	<code>instanceof</code> pattern (16), <code>switch</code> patterns + <code>when</code> guards + record patterns (21); killer combo = <code>sealed</code> + <code>switch</code> → compiler-checked exhaustiveness
3	Spring Boot annotations	Answer in groups (bootstrap / stereotype / DI / web / validation / JPA / config / test), close with "they're metadata — proxies do the work, which is why self- invocation breaks <code>@Transactional</code> "
4	<code>@Component</code> and <code>@Service</code>	All stereotypes are <code>@Component</code> ; <code>@Repository</code> is the only one with real behaviour (exception translation); <code>@Service</code> = intent + AOP pointcut
5	Multiple packages — how does Spring read all the annotations?	Default scan = main class's package + sub-packages → <code>scanBasePackages</code> , <code>scanBasePackageClasses</code> s (refactor-safe), filters; plus <code>@EntityScan</code> + <code>@EnableJpaRepositories</code> s , and auto-configuration for a shared jar
6	Reading the bean annotations (<code>@Bean</code>)	Stereotype = Spring builds it (your classes) · <code>@Bean</code> = you build it (third-party); bean name = method name; the <code>proxyBeanMethods</code> trap; bean lifecycle order

#	Question	Notes
7	<code>@Autowired</code> , <code>@Qualifier</code> , <code>@Primary</code>	Resolution is by type first → NoUniqueBeanDefinitionException . Precedence: @Qualifier > @Primary > bean name . Constructor injection; inject <code>Map / List</code> of all impls
8	<code>CrudRepository</code> and <code>JpaRepository</code>	<code>JpaRepository</code> → <code>PagingAndSortingRepository</code> → <code>CrudRepository</code> ; <code>List</code> vs <code>Iterable</code> , paging, <code>deleteAllInBatch</code> , <code>saveAndFlush</code> ; <code>SimpleJpaRepository</code> behind a proxy
9	Pagination methods	<code>Page<T> findByX(..., Pageable)</code> ; <code>PageRequest.of(page, size, Sort)</code> ; Boot binds <code>Pageable</code> from <code>? page&size&sort</code> ; OFFSET degrades → keyset paging ; pagination + <code>JOIN FETCH</code> = in-memory paging
10	What classes are created in pagination	<code>Pageable</code> · <code>PageRequest</code> · <code>Sort / Sort.Order / Direction</code> · <code>Page / PageImpl</code> · <code>Slice / SliceImpl</code> · <code>Window / ScrollPosition</code> . ★ Page runs 2 queries (incl. COUNT), Slice runs 1

#	Question	Notes
11	Communication in microservices	Sync (REST, <code>WebClient</code> / <code>RestClient</code> , <code>Feign</code> , gRPC) vs async (<code>Kafka</code> /Rabbit/SQS) + discovery, gateway, config server, Resilience4j, tracing, Saga. Rule: <i>sync when I need the answer now, async when B just needs to eventually know</i>
12	How do you handle error cascading?	Latency problem before an error problem: timeouts → circuit breaker → retry w/ backoff → bulkhead → fallback , <code>@RestControllerAdvice</code> + <code>ProblemDetail</code> + trace ID
13	★ Two microservices, network failure — A errors, how do you inform B?	The sharpest question. Caller can't tell "never arrived" from "response lost" → idempotency key. To <i>reliably</i> inform B: transactional outbox + Kafka + idempotent consumer + DLQ , compensating transaction if it can't complete
14	Dev/QA/Prod properties, DB changed per environment	<code>application-{profile}.yaml</code> + <code>SPRING_PROFILES_ACTIVE</code> ; one jar, many environments — env vars/ CLI override; <code>@Profile</code> beans; Config Server / ConfigMaps; secrets in Vault; Flyway + <code>ddl-auto: validate</code> in prod

#	Question	Notes
15	★ <code>ngOnInit</code> and constructor	Constructor = DI only , @Input s are undefined ; <code>ngOnInit</code> = after first <code>ngOnChanges</code> , inputs available → API calls. Full hook order. 5th round running
16	Decorator	Metadata function: class / property / method / parameter. <code>@Injectable({providedIn: 'root'})</code> = tree-shakable; Angular 16+ moving to <code>input()</code> / <code>output()</code> signal functions
17	★ Interceptor	Functional <code>HttpInterceptorFn</code> + <code>withInterceptors</code> ; <code>req.clone()</code> — requests are immutable, <code>multi: true</code> , order reverses on the response. 4th round running
18	Token expired — how do you notify the user?	401 in the interceptor → silent refresh → toast + redirect to <code>/login</code> with <code>returnUrl</code> ; proactive <code>exp</code> decode → "session expiring" dialog; auth guard; gate the refresh stampede with a <code>BehaviorSubject</code>

#	Question	Notes
19	How do you redirect to a page?	<code>routerLink</code> · <code>router.navigate /</code> <code>navigateByUrl</code> (+ <code>replaceUrl</code>) · <code>redirectTo</code> with <code>pathMatch: 'full'</code> (else infinite loop) · guard returning a <code>UrlTree</code> · <code>window.location</code> for external
20	★ Angular core — data binding	4 types: interpolation · property · event · two-way; <code>[(x)] = [x] +</code> <code>(xChange)</code> ; don't call methods in templates; signals version
21	★ Lazy loading	<code>loadChildren /</code> <code>loadComponent</code> ; never also import the lazy module eagerly; <code>PreloadAllModules</code> / custom strategy; disadvantages: first-nav delay, duplicated deps, per- module service instance, <code>ChunkLoadError</code> ; <code>@defer</code>
22	● Highest-paid employee per department → department + name	<code>groupBy</code> + <code>collectingAndThen(maxBy</code> <code>y(...), e -></code> <code>e.map(Employee::getNam</code> <code>e))</code> , or the cleaner 3-arg <code>toMap</code> + <code>BinaryOperator.maxBy</code> . Ties → <code>maxBy</code> keeps one. Output: <code>IT -> Peter</code> , <code>Sales -> Liza</code>

■ **What this round says:** Altimetrik tested **Spring wiring you can only know from building** — scanning across packages, `@Qualifier` vs `@Primary`, the repository hierarchy, the *classes* inside pagination. And **Q22 is the third time you've been asked highest-salary-per-department** (Altimetrik R1 in Streams, Mphasis in SQL, here again). Together with lifecycle hooks (**5 rounds**), interceptors (**4**), data binding (**4**) and lazy loading (**3**), the repeat list is now undeniable. The new muscle to build is **microservice failure vocabulary**: timeout · circuit breaker · retry+backoff · idempotency key · **transactional outbox** · DLQ · compensating transaction.

● Capgemini — Java + Spring + Angular round (Sept 2026)

★ **Attended early Sept 2026 · result pending.** A **breadth sweep**, not a depth round: 2 Java core → 4 Spring/architecture → 3 Angular → 1 Streams live-coding. **All 10 questions are answered in full in → 41 — Capgemini Java + Spring + Angular.**

▲ **Q10 was not written in the round** — the program is now solved four ways in 41 §Q10. Type it.

7 of the 10 were repeats of questions already in this file, and **two of them** (`@Component` vs `@Bean`, and dev/QA/prod config) had been asked at Altimetrik four days earlier.

#	Question	Notes
1	Java 8 new features	Answer in 5 groups: lambdas + functional interfaces · Stream API · interface default / static methods · Optional · java.time . Traps: streams are lazy, a stream is single-use, parallelStream() uses the shared common ForkJoinPool. Memory hook: L.S.I.O.D.
2	HashMap and LinkedHashMap	Same family, one difference — order. HashMap none · LinkedHashMap insertion order (HashMap + a doubly-linked list; accessOrder=true + removeEldestEntry = LRU cache) · TreeMap sorted, $O(\log n)$. Internals: $\text{hash} \rightarrow \& (n-1) \rightarrow \text{bucket} \rightarrow \text{list} \rightarrow \text{treeify at 8 with table} \geq 64$, load factor 0.75. ▲ mutable keys, equals / hashCode as a pair
3	Setter injection and constructor injection	Constructor = mandatory (fields final , fail-fast at startup, testable with plain new , @Autowired optional since 4.3); setter = optional with a default. Field injection X. ★ Circular deps: constructor → BeanCurrentlyInCreationException at startup (good); setter hides it — and Boot 2.6+ disallows circular refs by default

#	Question	Notes
4	@Component and @Bean	@Component = class, Spring builds it, found by scanning, name = decapitalised class · @Bean = method in @Configuration , you build it, name = method name — the only way to register a third-party class, and the way to do conditional/multiple instances. ★ The proxyBeanMethods trap: calling one @Bean method from another returns the <i>same</i> singleton
5	How do microservices load values?	Externalised config — one artefact, many environments. @Value · @ConfigurationProperties (type-safe, validated) ✓ · Environment · @Profile. ★ Precedence: CLI args > env vars > external application-{profile}.yml > external application.yml > the ones in the jar > defaults, with relaxed binding (payment.timeout-ms == PAYMENT_TIMEOUT_MS). At scale: Spring Cloud Config Server on git + @RefreshScope + Cloud Bus; K8s ConfigMap/Secret; secrets in Vault / AWS Secrets Manager, never in git

#	Question	Notes
6	Monolithic and microservices	One deployable + one DB vs many deployables + database per service. Microservices are an organisational answer first; the cost is that a method call becomes a network call (slow · fails · arrives twice) and @Transactional becomes a Saga. Name the distributed monolith anti-pattern. ● RoboGebra is a MODULAR MONOLITH — no Eureka, Feign, circuit breaker or Kafka. Say so
7	Components in Angular	@Component decorator + class = one piece of screen + its logic. The 4 files, the metadata (selector , providers = new instance per component, changeDetection: OnPush , encapsulation), the component tree, and communication: @Input ↓ · @Output / EventEmitter ↑ · shared service + BehaviorSubject · @ViewChild . ▲ Honesty guard: your codebase is NgModule-based, no signals

#	Question	Notes
8	How to create a new component	<code>ng g c student-card</code> → 4 files + it registers the class in the nearest NgModule's declaration s (say that fourth step — it's the actual answer). Flags: <code>--dry-run</code>, <code>--skip-tests</code>, <code>--module</code>, <code>--standalone</code>, <code>--change-detection=OnPush</code>, <code>--flat</code>. ▲ <code>'app-x'</code> is not a known element = not declared · declared but not exported · typo
9	★ <code>ngOnChanges</code> and <code>ngDoCheck</code>	<code>ngOnChanges</code> = only when an @Input reference changes, gets <code>SimpleChanges</code> (<code>previousValue</code> / <code>currentValue</code> / <code>firstChange()</code>); <code>ngDoCheck</code> = every change-detection cycle, no arguments, you do the comparing. ★ Angular compares inputs with === — so a <i>mutated</i> object never fires <code>ngOnChanges</code>, and that's why <code>ngDoCheck</code> exists. Keep it cheap; better still, make inputs immutable and use <code>OnPush</code>. 2nd round running (Photon asked it too)

#	Question	Notes
10	● String name = "Priyanka" — 2nd non-repeating character from the END, using Stream API	Answer: <code>n</code>. Counts: <code>a=2</code>, everything else 1 → non-repeating in order <code>P r i y n k</code> → from the end: <code>k</code>, then <code>n</code>. Stream API in 3 simple steps: ① count each character — <code>groupingBy(c -> c, LinkedHashMap::new, counting())</code> → <code>{P=1, r=1, i=1, y=1, a=2, n=1, k=1}</code> · ② keep count == 1 → <code>[P, r, i, y, n, k]</code> · ③ take <code>single.get(single.size() - 2)</code> → <code>n</code>. ★ LinkedHashMap is the whole trick — a plain HashMap loses the order and "2nd from the end" stops meaning anything. O(n). ▲ Not written in the round — simple solution, explained line by line

■ **What this round says:** nine questions you already had answers for, then **one program that decides the outcome**. That is now the pattern at Altimetrik, Photon, Codeboard **and** Capgemini. Theory gets you to the coding question; the coding question gets you the offer. **Type one problem a day from 22 — Java Streams** — reading them is a different skill from writing them under a shared screen.

● Next up — IQVIA (Angular / Ionic Technical Lead)

They gave you the syllabus. Ten areas, from Angular/Signals to Technical-Lead behaviour to AI-assisted development — all prepared in [38 — IQVIA Full Prep Pack](#).

~75% of that checklist is frontend, which is your strongest ground, plus a lead layer (code review, mentoring, incidents, tech debt, stakeholders) that is about judgement rather than syntax. **Log every question here the same day.**

Template — add your next round here

```
# ● <Company> — <Round name> (<date>)
```

```
### 1. <Question exactly as asked>  
<answer>
```

```
### 2. ...
```

Log it the same day. Note not just the question but:

- **which ones you fumbled** (mark them ▲ — those are your next study list),
- **the follow-up** they asked after your answer (that's where the real signal is),
- whether it was **theory or output/coding**.

✓ Pre-interview 20-minute revision path

1. [§ Most-repeated](#) — always, every time.
2. The section for **this company** if you've interviewed there before.
3. If it's a **Java** round → [23 — Predict the Output](#).
4. If it's an **Angular** round → lifecycle hooks, component communication, [Subject](#) vs [BehaviorSubject](#), unsubscribing, forms.
5. If it's an **Ionic** round → [ionViewDidEnter](#) vs [ngOnInit](#), Capacitor, offline support.

6. If it's a **Spring** round → `@Controller` vs `@RestController`, validation, global exception handling, `PUT` vs `PATCH`.
 - 6b. If it's a **Spring depth** round (Altimetrik-style wiring questions) → **37 Parts 3** — component scanning across packages, `@Qualifier` vs `@Primary`, `CrudRepository` vs `JpaRepository`, the pagination classes.
 7. If it's a **microservices/backend** round → Mphasis §Q1–Q4 + Q6 (microservices, patterns, circuit breaker + its dependency, Actuator), thread-safe Singleton, deadlock.
 8. If **SQL** is likely → **36 — SQL: The Complete Interview File** — read **Part 0** (the two questions you were actually asked) and **Part 24** (the cheat sheet).
Locally: Mphasis §Q8 below — `LEFT JOIN` + `COUNT(column)`, `WHERE` vs `HAVING`, second-highest salary, `RANK()` / `PARTITION BY`.
-

Related files: [16 — Codeboard §0](#) · [23 — Java Output](#) · [25 — Binding & Forms](#) · [20 — RxJS](#) · [22 — Streams](#) · [15 — Ionic](#)

C# / .NET "Predict the Output" — 75 Tricky Programs with Answers

● **Why this file exists:** your Java round was **output-based** ("*what does this print?*"), and .NET rounds are exactly the same — often worse, because C# has **more** places where the compiler decides something you assumed was decided at runtime. This is the .NET twin of [23 — Java Predict the Output](#).

The format for every program below:

1. **The code**
2. **The output** (or *compile error*)
3. **The rule** — the one sentence that makes it obvious
4. **The follow-up** they ask next

◆ **Golden habit** — ask these four in order, every time:

(a) **Does it compile?** → (b) **Is it a `struct` or a `class`?** (value vs reference decides half of all C# traps) → (c) **Was anything bound at *compile* time — overloads, `new` hiding, optional parameters?** → (d) **Only then trace the runtime.**

Candidates lose these questions at (a), (b) and (c). They almost never lose them at (d).

▲ **You are coming from Java. That is an advantage and a trap.** ~40% of C# output questions are designed to punish a Java reflex. **Section A is the eight places where the same code prints something different in C# than in Java** — do that section first, twice.

Table of contents

#	Section	What it tests
A	Java → C# flips (Q1–Q8)	The 8 programs where your Java instinct gives the wrong answer
B	Value types, structs & boxing (Q9–Q16)	Copy semantics, defensive copies, unboxing rules
C	<code>virtual</code> / <code>override</code> / <code>new</code> (Q17–Q24)	Static vs dynamic binding, hiding, interface re-implementation
D	Initialization order (Q25–Q29)	Field initializers, base ctors, static ctors, <code>beforefieldinit</code>
E	Strings (Q30–Q36)	Interning, <code>==</code> overload, immutability, <code>null</code> in concat
F	Numbers & operators (Q37–Q45)	<code>checked</code> / <code>unchecked</code> , <code>i</code> <code>= i++</code> , integer division, float precision
G	Nullable & null operators (Q46–Q50)	Lifted operators, <code>??</code> , <code>?.</code> , <code>null</code> comparisons that are <i>both</i> false
H	<code>try</code> / <code>catch</code> / <code>finally</code> / <code>using</code> (Q51–Q56)	Why C# <code>finally</code> can't swallow a return, disposal order
I	Closures, LINQ & iterators (Q57–Q64)	Loop capture, deferred execution, double enumeration, <code>yield</code>
J	<code>async</code> / <code>await</code> (Q65–Q70)	Execution order, <code>.Result</code> deadlock, <code>async void</code> , <code>WhenAll</code> exceptions
K	Collections & equality (Q71–Q75)	Modify-during- <code>foreach</code> , mutable keys, <code>record</code> vs <code>class</code> equality

#	Section	What it tests
L	Rapid-fire table	1-line answers for last-minute revision
M	20 self-test drills	Answers hidden — test yourself
N	How to answer in the room	The script that turns a quiz into a conversation

A — The 8 places C# prints something different from Java

These are the highest-value questions in the whole file. Every one of them is a program that a good Java developer gets **wrong**, which is exactly why interviewers who know you came from Java reach for them.

Q1. Method overloading with `null` — the one you were already asked, in C

```
using System;

class OverloadDemo
{
    public void Test(object o) => Console.WriteLine("object method");
    public void Test(string s) => Console.WriteLine("string method");

    static void Main()
    {
        new OverloadDemo().Test(null);
    }
}
```

✓ Output

```
string method
```

The rule (say this exact sentence)

"Overload resolution happens at *compile* time and the compiler picks the *most specific* applicable overload. `null` converts to both `object` and `string`, but `string` derives from `object`, so `string` is more specific and wins."

Same answer as Java, same reason. Lead with that — "*this is identical to Java's rule*" — it shows you understand the principle, not a memorized fact.

▲ The follow-ups

Follow-up 1 — "Force the `object` version."

```
new OverloadDemo().Test((object)null);    // object method
object o = null;
new OverloadDemo().Test(o);                // object method
```

The cast changes the argument's **compile-time type**, which is the only thing overload resolution looks at.

Follow-up 2 — "What if the overloads were `string` and `int[]`?"

```
public void Test(string s) { }
public void Test(int[] a) { }

Test(null);    // ✗ COMPILE ERROR CS0121: the call is ambiguous
```

`string` and `int[]` are **siblings** — neither converts to the other, so there is no "most specific" one. Fix with a cast.

Follow-up 3 — "C# has something Java doesn't here. What?"

```
public void Test(object o)    => Console.WriteLine("object");
public void Test(dynamic d) => Console.WriteLine("dynamic");
```

dynamic moves overload resolution to **runtime**. If any argument is **dynamic**, the whole call is resolved at runtime against the actual types — the single biggest difference from Java's model. Mentioning **dynamic** unprompted is a strong signal.

Q2. Field initializer vs base constructor — C# and Java print opposite things

```
using System;

class Base
{
    public Base()
    {
        Console.WriteLine("Base ctor");
        Show(); // virtual call from a constructor
    }
    public virtual void Show() { }
}

class Derived : Base
{
    private string name = "initialized";

    public Derived() => Console.WriteLine("Derived ctor");

    public override void Show() => Console.WriteLine($"name = {name ??
    "null"}");
}

class Program
{
    static void Main() => new Derived();
}
```

✓ Output

```
Base ctor
name = initialized
Derived ctor
```

The surprising part is not the order of the three lines — it's line 2. `Show()` is called from inside the base constructor, before `Derived`'s constructor has run at all... and yet `name` is **already assigned**.

The rule

"In C#, the derived class's *instance field initializers* run BEFORE the base constructor. In Java they run AFTER it."

Full C# construction order:

1. Derived field initializers ← C# does this FIRST
2. Base field initializers
3. Base constructor body
4. Derived constructor body ← last

Java's order is `super()` first, *then* the subclass's field initializers. So **the identical program in Java prints `name = null`** — the classic "virtual call from constructor" bug. In C# the field is already assigned, so it prints `initialized`.

▲ The follow-up

"So calling a virtual method from a constructor is safe in C#?"

No. Field *initializers* have run, but the **derived constructor body has not**.

Anything assigned in the ctor body is still `null / 0`:

```
csharp class Derived : Base { private string name; // no initializer
public Derived() { name = "set in ctor"; } public override void
Show() => Console.WriteLine(name ?? "null"); } // prints: null
```

The professional answer: *"Never call a virtual method from a constructor in either language. C# just changes which half of the state is missing."*

Q3. Methods are not virtual by default

```
using System;

class Animal
{
    public void Speak() => Console.WriteLine("Animal speaks");
}

class Dog : Animal
{
    public void Speak() => Console.WriteLine("Dog barks");
}

class Program
{
    static void Main()
    {
        Animal a = new Dog();
        a.Speak();
    }
}
```

✓ Output

```
Animal speaks
```

(plus compiler warning CS0108: "'Dog.Speak()'" hides inherited member 'Animal.Speak()'. Use the new keyword if hiding was intended.)

The rule

"In C#, methods are non-virtual by default. Without **virtual** + **override**, **Dog.Speak** *hides* rather than overrides, so the call is bound to the compile-time type **Animal**."

In Java this prints **Dog barks** — Java methods are virtual by default. This single difference is the most-asked C#-vs-Java question in existence.

▲ The follow-ups

Follow-up 1 — "Fix it."

```
class Animal { public virtual void Speak() => Console.WriteLine("Animal speaks"); }  
class Dog : Animal { public override void Speak() =>  
    Console.WriteLine("Dog barks"); }  
// now prints: Dog barks
```

Follow-up 2 — "Does it compile without `new`?"

Yes — it's a *warning*, not an error. `new` only silences the warning; it changes nothing at runtime.

Follow-up 3 — "Why did C# make this choice?"

"Explicit opt-in to polymorphism. You can't accidentally override a base method that a later library version adds — the 'brittle base class' / fragile-inheritance problem. Java chose convenience, C# chose versioning safety." That's a senior-level answer.

Q4. `finally` cannot change the return value in C

```
using System;  
  
class Program  
{  
    static int Test()  
    {  
        try { return 1; }  
        finally { return 2; }      // ← what happens?  
    }  
    static void Main() => Console.WriteLine(Test());  
}
```


✓ Output

✗ COMPILE ERROR CS0157: Control cannot leave the body of a finally clause

The rule

"C# forbids **return**, **break**, **continue** or **goto** inside a **finally** block. Java allows it — and in Java it silently swallows the original return value and any in-flight exception."

The same program in Java compiles and prints **2**. C# closed the hole at the language level.

▲ The follow-up

"Then can **finally** affect the returned value at all?"

```
static int Test()
{
    int x = 1;
    try { return x; }
    finally { x = 2; Console.WriteLine("finally runs"); }
}
// Output:
// finally runs
// 1
```

No. The return value is **evaluated and copied** before **finally** runs. **x** becomes 2, but the value **1** was already captured. Same in Java for primitives.

▲ **But with a reference type, mutation IS visible:**

```
csharp static List<int> Test() { var list = new List<int> { 1 }; try
{ return list; } finally { list.Add(2); } } // caller sees: [1, 2] ←
the reference was copied, the object was mutated
```

Q5. **struct** — assignment copies

```
using System;

struct PointS { public int X; }
class PointC { public int X; }

class Program
{
    static void Main()
    {
        PointS s1 = new PointS { X = 1 };
        PointS s2 = s1;
        s2.X = 99;

        PointC c1 = new PointC { X = 1 };
        PointC c2 = c1;
        c2.X = 99;

        Console.WriteLine($"{s1.X}, {c1.X}");
    }
}
```

✓ Output

```
1, 99
```

The rule

"A **struct is a value type — assignment copies the whole value. A **class** is a reference type — assignment copies only the reference, so both variables see the same object."**

Java has **no** user-defined value types (until Valhalla ships), so there is no Java equivalent to get wrong — which is exactly why interviewers use it on Java converts.

▲ The follow-up

"Where else does that copy happen?"

Everywhere a value moves: passing to a method, returning from a method, storing in a field, boxing, and — the killer — **reading from a property or a `List<T>` indexer** (see Q11).

Q6. `int.MaxValue + 1` — constants overflow at compile time

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine(int.MaxValue + 1);
    }
}
```

✓ Output

```
x  COMPILE ERROR CS0220: The operation overflows at compile time in
checked mode
```

The rule

"Constant expressions are always evaluated in a *checked* context at compile time. Runtime arithmetic is *unchecked* by default and wraps silently."

```

int max = int.MaxValue;
Console.WriteLine(max + 1);           // -2147483648 ← wraps silently,
no error

Console.WriteLine(unchecked(int.MaxValue + 1)); // -2147483648 ←
explicitly allowed

checked
{
    int m = int.MaxValue;
    Console.WriteLine(m + 1);         // ▲ OverflowException at
runtime
}

```

Java prints `-2147483648` for all of these — Java has no `checked` / `unchecked` and never errors on overflow.

▲ The follow-up

"How would you make overflow throw across a whole project?"

Set `<CheckForOverflowUnderflow>true</CheckForOverflowUnderflow>` in the `.csproj`. That flips the default for the entire assembly. Financial code should almost always turn it on.

Q7. `foreach` closure capture — C# 5 changed the answer

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        var actions = new List<Action>();

        for (int i = 0; i < 3; i++)
            actions.Add(() => Console.Write(i));

        Console.WriteLine();

        foreach (var x in new[] { 0, 1, 2 })
            actions.Add(() => Console.Write(x));

        foreach (var a in actions) a();
    }
}
```

✓ Output

```
333012
```

The rule

"A `for` loop has **ONE** loop variable for the whole loop, so all three lambdas capture the same `i` — which is 3 when they finally run. A `foreach` loop (C# 5 and later) creates a **FRESH** variable each iteration, so each lambda captures its own copy."

- **Before C# 5**, `foreach` also printed `222` — this was one of the most complained-about behaviours in the language, and Microsoft made a **breaking change** to fix it.
- **Java** has no equivalent problem because it forces captured variables to be *effectively final* — the `for` version simply **doesn't compile** in Java.

▲ The follow-ups

Follow-up 1 — "Fix the `for` version."

```
for (int i = 0; i < 3; i++)
{
    int copy = i;                // fresh variable per iteration
    actions.Add(() => Console.Write(copy));
}
// now prints 012
```

Follow-up 2 — "Where does this bite you in real code?"

Registering event handlers or `Task`s inside a `for` loop:

```
csharp for (int i = 0; i < 3; i++) tasks.Add(Task.Run(() =>
Process(i))); // all three may process 3
```

Q8. Optional parameter defaults come from the compile-time type

```
using System;

class Base    { public virtual void Show(int x = 1) =>
Console.WriteLine($"Base {x}"); }
class Derived : Base { public override void Show(int x = 2) =>
Console.WriteLine($"Derived {x}"); }

class Program
{
    static void Main()
    {
        Base b = new Derived();
        b.Show();
    }
}
```

✓ Output

```
Derived 1
```

The rule

"The *body* is chosen at runtime by the actual type (**Derived**). The *default value* is baked in at compile time from the static type (**Base**). You get **Derived** 's code with **Base** 's default."

This is the single most surprising line in C#. Java has no optional parameters at all, so there's nothing to transfer.

▲ The follow-ups

Follow-up 1 — "What's the versioning danger?"

Default values are **compiled into the caller's assembly**. If a library changes `void Log(int level = 1)` to `= 2`, every caller keeps passing `1` until they are **recompiled**. Optional parameters are a **binary-compatibility hazard** in public APIs.

Follow-up 2 — "What would you do instead in a public API?"

Use **overloads** — they're resolved against the callee's real signature:

```
csharp public void Log() => Log(1); public void Log(int level) { }
```

✓ **Section A checkpoint.** If you can state the eight rules above cold — non-virtual by default, field-initializers-before-base-ctor, no `return` in `finally`, structs copy, constants are checked, `for` vs `foreach` capture, optional-parameter defaults, and `dynamic` — you already handle most of a .NET output round.

B — Value types, reference types & boxing

Q9. Boxing takes a snapshot

```
int i = 5;
object o = i;      // boxing – copies the value onto the heap
i = 10;
Console.WriteLine(o);
```

✓ Output

```
5
```

The rule

"Boxing copies the value into a new heap object. The box has no link back to the variable."

▲ Follow-up — "And unboxing?"

```
object o = 5;
int a = (int)o;      // ✓ fine
long b = (long)o;    // ▲ InvalidCastException at runtime
```

Unboxing must name the exact original type. You cannot unbox-and-widen in one step. Correct form: `long b = (int)o;` or `Convert.ToInt64(o)`.

Q10. Two boxes are never ==

```
object a = 5;
object b = 5;
Console.WriteLine(a == b);
Console.WriteLine(a.Equals(b));
```


✓ Output

```
False
True
```

The rule

"**==** on two **object** variables is *reference* comparison — boxing created two different objects. **Equals** is virtual and dispatches to **Int32.Equals**, which compares values."

▲ Follow-up — "And if they were **int** variables?"

```
int a = 5, b = 5;
Console.WriteLine(a == b);    // True - no boxing, numeric == is a value
                                comparison
```

The **compile-time type** decides which **==** is used. This is the C# analogue of Java's **Integer** cache trap — except C# has no cache, so the answer is **False** for every value, not just outside **-128..127**.

Q11. Mutating a struct inside a **List<T>** vs an array

```
struct Point { public int X; }

var list  = new List<Point> { new Point { X = 1 } };
var array = new Point[]    { new Point { X = 1 } };

array[0].X = 99;           // line A
list[0].X  = 99;           // line B
```

✓ Output

```
✗ COMPILE ERROR on line B - CS1612:
  Cannot modify the return value of 'List<Point>.this[int]' because it is
  not a variable
```

Line A compiles and works fine.

The rule

"An array indexer returns a *reference* to the slot, so you can assign into it. A `List<T>` indexer is a *property* — it returns a *copy* — and assigning to a copy is meaningless, so the compiler blocks it."

▲ Follow-up — "How do you actually update it?"

```
var p = list[0];
p.X = 99;
list[0] = p;           // read-modify-write

// ...or just don't do this. Make the struct immutable:
readonly struct Point { public int X { get; init; } }
```

The real answer interviewers want: *"Mutable structs are a design smell."*

Microsoft's own guidance is to make structs immutable — this compile error is the language pushing back."

Q12. A `readonly` struct field silently mutates a copy

```
using System;

struct Counter
{
    public int Count;
    public void Increment() => Count++;
}

class Program
{
    private readonly Counter _readonlyField = new Counter();
    private          Counter _normalField  = new Counter();

    static void Main()
    {
        var p = new Program();
        p._readonlyField.Increment();
        p._normalField.Increment();
        Console.WriteLine($"{p._readonlyField.Count},
{p._normalField.Count}");
    }
}
```

✓ Output

```
0, 1
```

The rule

"Calling a method on a `readonly` struct field makes the compiler create a *defensive copy* first — the method mutates the copy and the copy is thrown away. The field never changes."

▲ Follow-up — "How would you catch this in review?"

Two ways: mark the struct `readonly struct` (then a mutating method won't compile at all), or enable the analyzer for **CA1815 / defensive-copy** warnings.

Mention that `readonly struct` also *removes* the hidden copies, so it's a performance win too.

Q13. Passing a reference type "by reference"?

```
using System;

class Program
{
    static void Modify(int[] arr)
    {
        arr[0] = 99;           // mutation – visible to caller
        arr = new int[] { 7, 7, 7 }; // reassignment – NOT visible
        arr[1] = 42;
    }

    static void Main()
    {
        int[] a = { 1, 2, 3 };
        Modify(a);
        Console.WriteLine(string.Join(",", a));
    }
}
```

✓ Output

```
99,2,3
```

The rule

"C# is *always* pass-by-value. For a reference type, the *reference* is passed by value — so mutating the object is visible, but reassigning the parameter is not."

Identical to Java. Say "same as Java" — but be ready for the next question:

▲ Follow-up — "Can C# do what Java can't here?"

```
static void Modify(ref int[] arr) => arr = new int[] { 7, 7, 7 };

int[] a = { 1, 2, 3 };
Modify(ref a);
Console.WriteLine(string.Join(",", a));    // 7,7,7
```

Yes — `ref` . It passes the *variable itself*, so reassignment is visible. Java has no equivalent. Also mention **`out`** (must be assigned before return, doesn't need initialising by the caller) and **`in`** (pass-by-reference, read-only, avoids copying large structs).

Q14. `default` for struct vs class

```
struct S { public int X; public string Name; }
class C { public int X; public string Name; }

S s = default;
C c = default;

Console.WriteLine(s.X);
Console.WriteLine(s.Name ?? "null");
Console.WriteLine(c == null);
```

✓ Output

```
0
null
True
```

The rule

"**`default(struct)`** is a real instance with every field zeroed — you can use it.
`default(class)` is **`null`** — touching it throws."

▲ Follow-up — "So `new S()` and `default(S)` are the same?"

They were identical before C# 10. Since **C# 10** a struct may declare an explicit parameterless constructor, and then `new S()` runs it while `default(S)` still just zeroes memory — they can differ. Also note **`default(S)` bypasses field initializers**, which is why struct field initializers were illegal for so long.

Q15. Struct equality without an override

```
struct P { public int X, Y; }

var a = new P { X = 1, Y = 2 };
var b = new P { X = 1, Y = 2 };
Console.WriteLine(a.Equals(b));
Console.WriteLine(a == b);
```

✓ Output

```
True
✗ COMPILE ERROR CS0019 on the second line:
  Operator '==' cannot be applied to operands of type 'P' and 'P'
```

The rule

"**`ValueType.Equals`** gives structs *value* equality for free. But **`==`** is NOT generated — you must overload it yourself."

▲ Follow-up — "Any problem with the free `Equals`?"

Yes — for a struct containing any reference-type field, the default `Equals` falls back to **reflection**, which is dramatically slow. Always override `Equals`, `GetHashCode` and `==` / `!=` on a struct you use in hot paths or as a dictionary key. (Or use a **`record struct`**, which generates all of it.)

Q16. Interfaces box structs

```
using System;

interface ICounter { void Increment(); int Value { get; } }

struct Counter : ICounter
{
    public int Value { get; private set; }
    public void Increment() => Value++;
}

class Program
{
    static void Main()
    {
        Counter c = new Counter();
        c.Increment();
        c.Increment();

        ICounter i = c;           // boxing!
        i.Increment();

        Console.WriteLine($"{c.Value}, {i.Value}");
    }
}
```

✓ Output

2, 3

The rule

"Assigning a struct to an interface variable **BOXES** it. **i** is a separate heap copy — from that point the two evolve independently."

▲ Follow-up — "Where does this cost you in production?"

Any `List<IShape>` of structs, any `foreach` over `IEnumerable` of a struct enumerator, any LINQ over a struct collection — each boxes and allocates. It's the standard answer to *"why did adding an interface slow down my hot loop?"*

C — `virtual`, `override`, `new` & interfaces

Q17. `override` vs `new` side by side

```
using System;

class A { public virtual void Show() => Console.WriteLine("A"); }
class B : A { public override void Show() => Console.WriteLine("B"); }
class C : A { public new      void Show() => Console.WriteLine("C"); }

class Program
{
    static void Main()
    {
        A ab = new B();  ab.Show();
        A ac = new C();  ac.Show();
        C cc = new C();  cc.Show();
    }
}
```

✓ Output

```
B
A
C
```


The rule

"**override** replaces the slot in the vtable — dynamic binding. **new** creates a brand-new, unrelated method — binding follows the *compile-time* type of the variable."

▲ Follow-up — "One sentence on when **new** is legitimate?"

Almost never in your own code. Its real purpose is **versioning**: a base class you don't own adds a method with your method's name, and **new** lets you keep compiling without accidentally overriding it.

Q18. **new** cannot be un-hidden by a further override

```
using System;

class A { public virtual void M() => Console.WriteLine("A.M"); }
class B : A { public new virtual void M() => Console.WriteLine("B.M"); }
class C : B { public override void M() => Console.WriteLine("C.M"); }

A a = new C();
a.M();
```

✓ Output

```
A.M
```

The rule

"**B.M** started a *new* virtual chain. **C.M** overrides **B** 's chain, not **A** 's. Through an **A** reference you still reach the original **A.M**."

▲ Follow-up — "And through a B reference?"

```
B b = new C();  
b.M();      // C.M
```

Q19. sealed override

```
class A { public virtual void M() { } }  
class B : A { public sealed override void M() { } }  
class C : B { public override void M() { } }
```

✓ Output

```
x  COMPILER ERROR CS0239: 'C.M()' cannot override inherited member 'B.M()' because it is sealed
```

The rule

"sealed override stops the virtual chain at that class. It's C#'s final on a method."

▲ Follow-up — "Can C still declare an M?"

Yes, with `public new void M()` — hiding is always allowed, it's *overriding* that's sealed off.

Q20. Interface re-implementation — the nastiest one in this section

```
using System;

interface IShape { void Draw(); }

class Square : IShape
{
    public void Draw() => Console.WriteLine("Square.Draw");
}

class ColoredSquare : Square, IShape          // ← note: IShape is listed
AGAIN
{
    public new void Draw() => Console.WriteLine("ColoredSquare.Draw");
}

class Program
{
    static void Main()
    {
        IShape i = new ColoredSquare(); i.Draw();
        Square s = new ColoredSquare(); s.Draw();
    }
}
```

✓ Output

```
ColoredSquare.Draw
Square.Draw
```

The rule

"Re-listing the interface makes the derived class *re-implement* it — the interface map is rebuilt and now points at **ColoredSquare.Draw**. The class-level call still uses hiding rules and reaches **Square.Draw**."

▲ Follow-up — "And if `ColoredSquare` did NOT list `IShape` again?"

```
class ColoredSquare : Square           // IShape not re-listed
{
    public new void Draw() => Console.WriteLine("ColoredSquare.Draw");
}
IShape i = new ColoredSquare(); i.Draw(); // Square.Draw ← both calls
now print Square.Draw
```

That's the whole trap: the same class body behaves differently depending on one word in the base list. The correct fix is always `virtual/override`, never `new`.

Q21. Explicit interface implementation is invisible on the class

```
using System;

interface ILogger { void Log(); }

class FileLogger : ILogger
{
    void ILogger.Log() => Console.WriteLine("ILogger.Log");
}

class Program
{
    static void Main()
    {
        var f = new FileLogger();
        f.Log();
    }
}
```

✓ Output

```
✗ COMPILE ERROR CS1061: 'FileLogger' does not contain a definition for
'Log'
```

The rule

"An explicit interface implementation has no access modifier and is NOT a member of the class — it's only reachable through the interface."

```
((ILogger)f).Log();           // ✓  ILogger.Log
ILogger l = new FileLogger();
l.Log();                      // ✓  ILogger.Log
```

▲ Follow-up — "When would you use it?"

Two real cases: (1) **name collisions** — a class implementing `IEnumerable<T>` and `IEnumerable` needs two `GetEnumerator` s; (2) **hiding infrastructure** from the public API, e.g. implementing `IDisposable` explicitly so `Dispose` doesn't appear on IntelliSense for a type you want people to use with `using` only.

Q22. Abstract class calling an abstract method

```
using System;

abstract class Shape
{
    protected Shape() { Console.WriteLine($"Area = {GetArea()}"); }
    public abstract double GetArea();
}

class Circle : Shape
{
    private readonly double _r;
    public Circle(double r) => _r = r;
    public override double GetArea() => 3.14 * _r * _r;
}

new Circle(2);
```

✓ Output

```
Area = 0
```

The rule

"**_r** is assigned in the *derived constructor body*, which runs **AFTER** the base constructor. At the time the base ctor calls **GetArea()**, **_r** is still **0**."

▲ Contrast with **Q2**: field *initializers* (**double _r = 2;**) would already have run. Constructor *bodies* have not. Know which half you're in.

▲ Follow-up — "Give a design fix."

Don't compute in the constructor — make it a lazy property or a **Create** factory that constructs then computes. Also: analyzer **CA2214** ("Do not call overridable methods in constructors") flags exactly this.

Q23. Static methods are never polymorphic

```
using System;

class A { public static void Who() => Console.WriteLine("A"); }
class B : A { public static new void Who() => Console.WriteLine("B"); }

A.Who();
B.Who();
// A a = new B(); a.Who(); ← what about this?
```

✓ Output

```
A
B
```

And the commented line is a **compile error CS0176**: "*Member 'A.Who()' cannot be accessed with an instance reference; qualify it with a type name instead.*"

The rule

"Static members are bound to the *type*, never to an instance. C# goes further than Java and forbids calling a static member through an instance reference at all."

In Java, `a.Who()` **compiles** (with a warning) and prints `A`. C# rejects it outright — a small but well-loved gotcha for Java converts.

Q24. Overload resolution beats inheritance

```
using System;

class A { public void F(int x) => Console.WriteLine("A.F(int)"); }
class B : A { public void F(long x) => Console.WriteLine("B.F(long)"); }

new B().F(5);
```

✓ Output

```
B.F(long)
```

The rule

"C# searches for applicable methods starting at the **MOST DERIVED** type and stops at the first class that has *any* applicable candidate. **B** has one (`F(long)` via implicit widening), so `A.F(int)` is never even considered."

▲ **This is the opposite of Java**, which merges the whole hierarchy into one candidate set and would pick the exact match `A.F(int)`. Same code, different language, different answer — a favourite of interviewers who know both.

▲ Follow-up — "So how do I reach `A.F(int)`?"

```
((A)new B()).F(5);    // A.F(int)
```

D — Initialization order

Q25. The full order, one program

```
using System;

class Base
{
    static Base() => Console.WriteLine("1. Base static ctor");
    public static int BaseStaticField = Print("0. Base static field");
    public int BaseInstanceField      = Print("3. Base instance field");
    public Base() => Console.WriteLine("4. Base ctor body");
    static int Print(string s) { Console.WriteLine(s); return 0; }
}

class Derived : Base
{
    public int DerivedInstanceField = Print2("2. Derived instance field");
    public Derived() => Console.WriteLine("5. Derived ctor body");
    static int Print2(string s) { Console.WriteLine(s); return 0; }
}

class Program { static void Main() => new Derived(); }
```

✓ Output

```
2. Derived instance field
0. Base static field
1. Base static ctor
3. Base instance field
4. Base ctor body
5. Derived ctor body
```


The rule — memorize this ladder

- ① Derived instance field initializers ← FIRST (C#-specific!)
- ② Base static field initializers } triggered on first use of Base
- ③ Base static constructor }
- ④ Base instance field initializers
- ⑤ Base constructor body
- ⑥ Derived constructor body ← LAST

▲ Follow-up — "Why do the statics appear in the middle?"

A static constructor runs **lazily, exactly once, on first use of the type**. Here the first use of **Base** is the implicit **base()** call — which happens *after* the derived field initializers. Move the trigger and the statics move with it.

Q26. Static constructor runs exactly once

```
using System;

class Config
{
    public static int Count;
    static Config() { Count++; Console.WriteLine("static ctor"); }
    public Config() { }
}

new Config(); new Config(); new Config();
Console.WriteLine(Config.Count);
```

✓ Output

```
static ctor
1
```

The rule

"A static constructor runs at most once per AppDomain, on first instantiation or first static-member access — whichever comes first — and the CLR guarantees it is thread-safe."

▲ Follow-up — "So a static ctor is a valid singleton?"

Yes — the classic **type-initializer singleton** is thread-safe with no locking:

```
csharp public sealed class Singleton { private static readonly
Singleton _instance = new Singleton(); static Singleton() { } // ←
disables 'beforefieldinit', forcing lazy init private Singleton()
{ } public static Singleton Instance => _instance; }
```

Mentioning **beforefieldinit** — that without an explicit static ctor the CLR may initialize the type *earlier* than you expect — is a strong senior signal.

Q27. **const** vs **static readonly**

```
// Assembly Lib.dll
public class Config
{
    public const          int MaxUsers = 100;
    public static readonly int MaxItems = 100;
}

// Assembly App.exe — compiled against Lib v1, Lib later rebuilt with 200
Console.WriteLine(Config.MaxUsers);
Console.WriteLine(Config.MaxItems);
```

✓ Output (after Lib is changed to 200 and only Lib is recompiled)

```
100      ← stale!
200
```

The rule

"**const** is inlined into the *calling* assembly at compile time.
static readonly is a real field read at runtime. Changing a public **const** in a library is a binary-breaking change."

Same trap as optional parameters (Q8) — **compile-time baking**.

▲ Follow-up — "When is **const** still right?"

Only for values that are true forever and are part of the type's identity — **Math.PI**, **int.MaxValue**, **""**. Anything configurable should be **static readonly**.

Q28. Field initializers can't reference **this**

```
class Program
{
    private int a = 5;
    private int b = a + 1;
}
```

✓ Output

✗ COMPILE ERROR CS0236: A field initializer cannot reference the non-static field, method, or property 'Program.a'

The rule

"Instance field initializers run before the object is fully formed, so they cannot reference *any* instance member — including each other. Order is not guaranteed to be usable."

Java allows this (**int b = a + 1;** compiles and works if **a** is declared first).
Another Java reflex to unlearn.

▲ Follow-up — "Fix?"

Move it into the constructor: `public Program() { b = a + 1; }.`

Q29. Static field initializer order

```
using System;

class Program
{
    static int A = B + 1;
    static int B = 10;
    static void Main() => Console.WriteLine($"A={A}, B={B}");
}
```

✓ Output

A=1, B=10

The rule

"Static field initializers run in *textual order*. When **A is initialized, **B** hasn't run yet, so **B** is still its default **0** — **A** becomes 1."**

Identical behaviour in Java. Swap the two lines and you get `A=11, B=10`.

E — String traps

Q30. `==` on strings is a value comparison

```
string a = "hello";
string b = "hel" + "lo";
string c = new string("hello".ToCharArray());

Console.WriteLine(a == b);
Console.WriteLine(a == c);
Console.WriteLine((object)a == (object)b);
Console.WriteLine((object)a == (object)c);
```

✓ Output

```
True
True
True
False
```

The rule

"**string** overloads `==` to compare *contents* — unlike Java, where `==` on strings is always reference comparison. Casting to **object** removes the overload and gives you back reference comparison."

Line 3 is **True** because `"hel" + "lo"` is folded by the compiler into the literal `"hello"` and **interned** — same object. Line 4 is **False** because `new string(...)` allocates at runtime and is not interned.

▲ The follow-ups

Follow-up 1 — "Make line 4 print **True**."

```
Console.WriteLine((object)a == (object)string.Intern(c)); // True
```

Follow-up 2 — "What about runtime concatenation?"

```
string x = "hel";
string y = x + "lo"; // built at runtime, not
interned
Console.WriteLine((object)a == (object)y); // False
Console.WriteLine(a == y); // True
```

Follow-up 3 — "So should I ever use `==` on strings?"

Yes, for ordinal equality it's fine and fast. But for anything **culture-sensitive or case-insensitive**, use `string.Equals(a, b, StringComparison.OrdinalIgnoreCase)` — the explicit comparison type is what reviewers look for.

Q31. Strings are immutable

```
string s = "hello";
s.ToUpper();
s.Replace("h", "H");
Console.WriteLine(s);
```

✓ Output

```
hello
```

The rule

"Every **string** method RETURNS a new string; none mutate. Ignoring the return value does nothing."

▲ Follow-up — "Fix it, then tell me about the loop version."

```
s = s.ToUpper(); // HELLO
```

And in a loop, `s += x` allocates a **new string every iteration** — $O(n^2)$ copying. Use `StringBuilder`. Interviewers love the follow-up *"at roughly what size does*

StringBuilder win?" — the honest answer is "a few concatenations in a loop; for 3–4 fixed concatenations the compiler turns + into a single `string.Concat` call and `StringBuilder` is actually slower."

Q32. `null` inside string concatenation

```
string s = null;
Console.WriteLine("Value: " + s + "!");
Console.WriteLine($"Value: {s}!");
Console.WriteLine(s.Length);
```

✓ Output

```
Value: !
Value: !
▲ NullReferenceException
```

The rule

"Concatenation and interpolation treat `null` as `string.Empty` — they never throw. Calling a *member* on a `null` string does throw."

▲ Follow-up — "And `Console.WriteLine(s)` on its own?"

Prints an **empty line**, no exception — `Console.WriteLine(string)` handles `null`.

Q33. `1 + 2 + "3" + 4 + 5`

```
Console.WriteLine(1 + 2 + "3" + 4 + 5);
Console.WriteLine("1" + 2 + 3);
```

✓ Output

```
3345
123
```

The rule

" + is left-associative. Before the first string appears you're doing arithmetic; from the first string onward everything becomes concatenation."

```
1 + 2      → 3      (arithmetic)
3 + "3"    → "33"   (concat)
"33" + 4   → "334"
"334" + 5  → "3345"
```

Identical to Java — say so, it's a free point.

Q34. `string.Empty`, `""` and `null`

```
string a = "";
string b = string.Empty;
string c = null;

Console.WriteLine(a == b);
Console.WriteLine(a == c);
Console.WriteLine(string.IsNullOrEmpty(c));
Console.WriteLine(string.IsNullOrEmpty("  "));
Console.WriteLine(c?.Length ?? -1);
```

✓ Output

```
True
False
True
True
-1
```


The rule

"`""` and `string.Empty` are the same interned instance. `null` is not a string at all. `?.` short-circuits to `null`, and `??` supplies the fallback."

Q35. Comparing with `Equals` vs `CompareTo`

```
string a = "STRASSE";
string b = "STRASSE";

Console.WriteLine(a.Equals(b));
Console.WriteLine(string.Equals("i", "I",
StringComparison.OrdinalIgnoreCase));
Console.WriteLine("apple".CompareTo("Apple"));
```

✓ Output

```
True
True
-1      (on .NET Core / .NET 5+ with the ICU collation; culture-
dependent)
```

The rule

"`Equals` without a `StringComparison` is ordinal. `CompareTo` is *culture-sensitive*, so its result can differ between machines and .NET versions."

▲ Follow-up — "What's the safe habit?"

Always pass an explicit `StringComparison`. Use `Ordinal` for identifiers, keys and paths; `CurrentCulture` only for text shown to a human. This is a real production-bug class (the famous "Turkish I" problem: in `tr-TR`, `"i".ToUpper()` is `"İ"`, not `"I"`).

Q36. **switch** on a string, and no fall-through

```
string s = "a";
switch (s)
{
    case "a":
        Console.WriteLine("A");
    case "b":
        Console.WriteLine("B");
        break;
}
```

✓ Output

```
x COMPILE ERROR CS0163: Control cannot fall through from one case label
to another
```

The rule

"C# forbids implicit fall-through — every non-empty **case must end with **break**, **return**, **throw** or **goto case**. Java allows fall-through and it's a classic bug source."**

▲ Follow-up — "How do you share a body between cases?"

```
case "a":
case "b": // ✓ empty case labels CAN stack
    Console.WriteLine("A or B");
    break;

// or explicitly:
case "a": Console.WriteLine("A"); goto case "b";
case "b": Console.WriteLine("B"); break;
```

F — Numbers & operators

Q37. `i = i++`

```
int i = 0;
i = i++;
Console.WriteLine(i);

int j = 0;
j = ++j;
Console.WriteLine(j);
```

✓ Output

```
0
1
```

The rule

"`i++` returns the OLD value, then increments. The assignment then writes that old value back over the increment. `++j` increments first and returns the new value."

Identical to Java. The trace:

```
i = i++      →   temp = i (0);  i = i + 1 (i is 1);  i = temp (i is 0)
```

▲ Follow-up — "And `i = i++ + ++i`; starting from 1?"

```
int i = 1;
i = i++ + ++i;      // (1) + (3) = 4
Console.WriteLine(i);  // 4
```

Left to right: `i++` yields 1 (i becomes 2), `++i` makes i 3 and yields 3, sum 4 is assigned. **Say you'd never write this in real code** — that comment scores points.

Q38. Integer division and modulo with negatives

```
Console.WriteLine(5 / 2);  
Console.WriteLine(-5 / 2);  
Console.WriteLine(-5 % 2);  
Console.WriteLine(5.0 / 2);
```

✓ Output

```
2  
-2  
-1  
2.5
```

The rule

"Integer division truncates *toward zero* (not toward negative infinity), and `%` takes the sign of the *dividend*."

Same as Java. Contrast with Python, where `-5 // 2` is `-3`.

Q39. Division by zero — int vs double

```
int a = 0;  
double b = 0;  
  
Console.WriteLine(10.0 / 0);  
Console.WriteLine(-10.0 / 0);  
Console.WriteLine(0.0 / 0);  
Console.WriteLine(10 / a);  
Console.WriteLine(10 / 0);
```

✓ Output

```
∞           (prints "Infinity" on .NET Framework, "∞" on .NET Core 3.0+)
-∞
NaN
▲ DivideByZeroException
✗ COMPILE ERROR CS0020: Division by constant zero
```

The rule

"Floating-point division by zero follows IEEE 754 and produces Infinity/NaN. Integer division by zero throws. Integer division by a *literal* zero doesn't even compile."

▲ Follow-up — "What is `double.NaN == double.NaN`?"

```
Console.WriteLine(double.NaN == double.NaN);           // False
Console.WriteLine(double.NaN.Equals(double.NaN));       // True  ← !
Console.WriteLine(double.IsNaN(double.NaN));            // True
```

`==` follows IEEE 754 (NaN is unequal to everything, including itself). `Equals` deliberately breaks that so that `NaN` works as a dictionary key and sorts consistently. This inconsistency is a favourite closing question.

Q40. Floating-point precision

```
Console.WriteLine(0.1 + 0.2);
Console.WriteLine(0.1 + 0.2 == 0.3);
Console.WriteLine(0.1m + 0.2m == 0.3m);
Console.WriteLine((0.1f + 0.2f) == 0.3f);
```

✓ Output

```
0.30000000000000004    (.NET Core 3.0+; older .NET Framework printed 0.3)
False
True
True
```

The rule

" **double / float** are binary floating point — 0.1 and 0.2 have no exact binary representation. **decimal** is base-10 with 28–29 significant digits, so it represents them exactly."

The **float** line is **True** only because **float**'s lower precision happens to round both sides to the same value — *"accidentally right, still wrong to rely on."*

▲ Follow-up — "Rule for choosing?"

decimal for money and anything a human will audit (it's also ~10× slower and can't do transcendental math). **double** for science, graphics and measurement. Never compare floats with **==** — use **Math.Abs(a - b) < epsilon**.

Q41. Compound assignment hides a cast

```
byte b = 10;
b = b + 5;
b += 5;
```

✓ Output

```
✗ COMPILE ERROR CS0266 on line 2: Cannot implicitly convert type 'int' to
'byte'
  (line 3 compiles fine)
```

The rule

" **byte** + **int** promotes to **int** , and **int** won't implicitly narrow back to **byte** . But **b += 5** contains an *implicit cast* — the spec defines **b += x** as **b = (byte)(b + x)** ."

Identical to Java. And the sting:

```
byte b = 250;
b += 10;
Console.WriteLine(b);    // 4 ← silently wraps, no warning
```

Q42. **char** arithmetic

```
Console.WriteLine('a' + 1);
Console.WriteLine((char)('a' + 1));
Console.WriteLine('a' + 'b');
Console.WriteLine("a" + 'b');
```

✓ Output

```
98
b
195
ab
```

The rule

" **char** implicitly widens to **int** in arithmetic. Only an explicit cast back to **char** gives you a character. But **string** + **char** is concatenation, because **+** with a string operand always concatenates."

Same as Java.

Q43. Ternary operator type unification

```
Console.WriteLine(true ? 1 : 2.0);
Console.WriteLine(true ? 1 : "two");
object o = true ? (object)1 : "two";
Console.WriteLine(o);
```

✓ Output

```
1
✗ COMPILE ERROR CS0173: Type of conditional expression cannot be
determined
  because there is no implicit conversion between 'int' and 'string'
1
```

The rule

"A ternary has **ONE** static type, computed from both branches even though only one runs. **int** widens to **double**, so the whole expression is **double** — and **1.0.ToString()** is **"1"**. **int** and **string** have no common type, so it doesn't compile."

▲ Follow-up — "Then why does **1** print, not **1.0**?"

Because **double.ToString()** uses the shortest round-trippable form — **1.0** renders as **"1"**. The value **is** a **double**; only its rendering hides it. Prove it with **(true ? 1 : 2.0).GetType() → System.Double**.

Q44. ++ on a property vs a field

```
using System;

class Counter
{
    private int _c;
    public int Value { get => _c; set => _c = value; }
}

var c = new Counter();
c.Value++;
Console.WriteLine(c.Value);
```

✓ Output

1

The rule

"**c.Value++** on a property compiles to **set_Value(get_Value() + 1)** — a get, an add, and a set. It works, but it is NOT atomic and it hides two method calls."

▲ Follow-up — "Why does that matter?"

Two threads doing **c.Value++** can lose an update — the read-modify-write is not atomic. The fix is **Interlocked.Increment(ref _c)**, which needs a **field**, not a property. This is the classic "*why is my counter wrong under load?*" question.

Q45. `as` vs `cast` vs `is`

```
object o = "hello";

string s1 = o as string;
int? n1 = o as int?;
Console.WriteLine(s1 ?? "null");
Console.WriteLine(n1?.ToString() ?? "null");

Console.WriteLine(o is string);
Console.WriteLine(null is object);

int i = (int)o;
```

✓ Output

```
hello
null
True
False
▲ InvalidCastException on the last line
```

The rule

"`as` returns `null` on failure (reference and nullable types only) — a cast throws. `is` is a type test that is ALWAYS false for `null`, because `null` has no type."

▲ Follow-up — "Which do you use and why?"

The modern form is pattern matching, one test instead of two:

```
csharp if (o is string str) Console.WriteLine(str.Length);
```

Using `is` followed by a cast does the type check **twice** — the pattern form does it once.

G — Nullable & the null operators

Q46. Both comparisons are false

```
int? a = null;
int? b = 5;

Console.WriteLine(a > b);
Console.WriteLine(a < b);
Console.WriteLine(a >= b);
Console.WriteLine(a == b);
Console.WriteLine(a != b);
```

✓ Output

```
False
False
False
False
True
```

The rule

"Lifted RELATIONAL operators (`<` `>` `<=` `>=`) return **false** if either operand is null — so `a > b` and `a <= b` can BOTH be false, breaking the usual logic. Lifted EQUALITY operators do work normally: null equals null, null differs from anything else."

▲ Follow-up — "Where does that cause a real bug?"

`if (!(a > b))` is **not** the same as `if (a <= b)` when nulls are involved. Any validation written as a negated comparison silently changes meaning. Always null-check first:

```
csharp if (a.HasValue && b.HasValue && a > b) { }
```

Q47. Nullable arithmetic swallows everything

```
int? a = null;
int? b = 5;

Console.WriteLine(a + b);
Console.WriteLine((a + b) ?? -1);
Console.WriteLine(a.GetValueOrDefault());
Console.WriteLine(a.Value);
```

✓ Output

```
(blank line – the result is null)
-1
0
▲ InvalidOperationException: Nullable object must have a value
```

The rule

"Any lifted arithmetic with a null operand yields null — nulls propagate through the whole expression. `.Value` on a null throws `InvalidOperationException`, NOT `NullReferenceException`."

Naming the *right* exception is worth extra credit.

Q48. `??` and `?.` short-circuit further than you think

```
string s = null;

Console.WriteLine(s?.Length ?? -1);
Console.WriteLine(s?.ToUpper().Trim().Length ?? -1);

int[] arr = null;
Console.WriteLine(arr?[0] ?? -1);
```

✓ Output

```
-1  
-1  
-1
```

The rule

"**?.** short-circuits the **ENTIRE** remaining chain, not just the next member. Once it hits null, **.ToUpper().Trim().Length** is never evaluated and the whole expression is null."

▲ Follow-up — "What's the type of **s?.Length**?"

int?, not **int** — **?.** on a value-typed member always lifts the result to nullable. That's why **?? -1** compiles at all.

Q49. **??=** and evaluation order

```
using System;  
  
class Program  
{  
    static int _calls;  
    static string Get() { _calls++; return "value"; }  
  
    static void Main()  
    {  
        string a = null;  
        a ??= Get();  
        a ??= Get();  
        Console.WriteLine($"{a}, calls={_calls}");  
    }  
}
```

✓ Output

```
value, calls=1
```

The rule

"**??=** assigns only if the left side is null, and the right side is NOT evaluated when the left side is non-null — it short-circuits like **&&.**"

▲ Follow-up — "Compare with `a = a ?? Get();`"

Behaviourally identical, but **??=** avoids re-assigning `a` to itself — which matters for properties with side-effecting setters and for thread safety.

Q50. Nullable reference types are compile-time only

```
#nullable enable

string NotNull(string? input) => input!;    // != null-forgiving

string result = NotNull(null);
Console.WriteLine(result.Length);
```

✓ Output

```
▲ NullReferenceException
```

(plus a compile-time warning CS8625 on `NotNull(null)`)

The rule

"Nullable reference types are a *compile-time analysis* with warnings only. There is no runtime enforcement, and **!** just silences the analyser — it does not check anything."

▲ Follow-up — "How do you make the warnings bite?"

`<WarningsAsErrors>nullable</WarningsAsErrors>` (or `<TreatWarningsAsErrors>`) in the `.csproj`. And note the boundary problem: **data from JSON, EF Core, or a non-annotated library can be null regardless of what the type says** — always validate at the edge.

H — `try / catch / finally` & `using`

Q51. `finally` always runs

```
using System;

class Program
{
    static void Main()
    {
        try
        {
            Console.WriteLine("try");
            throw new Exception("boom");
        }
        catch (Exception e)
        {
            Console.WriteLine("catch: " + e.Message);
            return;
        }
        finally
        {
            Console.WriteLine("finally");
        }
    }
}
```

✓ Output

```
try
catch: boom
finally
```

The rule

" **finally** runs before the method actually returns — even when **catch** contains **return** ."

▲ Follow-up — "Is there anything that skips **finally** ?"

Yes, three things: `Environment.FailFast()` , a `StackOverflowException` (uncatchable since .NET 2.0 — the process dies), and killing the process. Also, an unhandled exception on a **background thread** tears down the process without unwinding other threads' **finally** blocks.

Q52. Exception filters run BEFORE the stack unwinds

```
using System;

class Program
{
    static bool Log(Exception e) { Console.WriteLine("filter"); return false; }

    static void Main()
    {
        try
        {
            try { throw new InvalidOperationException("x"); }
            catch (Exception e) when (Log(e)) { Console.WriteLine("inner
catch"); }
            finally { Console.WriteLine("inner finally"); }
        }
        catch (Exception) { Console.WriteLine("outer catch"); }
    }
}
```

✓ Output

```
filter
inner finally
outer catch
```

The rule

"An exception filter (**when**) is evaluated in the **FIRST** pass, before any **finally** block runs. Only if it returns true does the second pass unwind the stack into that **catch** ."

Note "filter" prints before "inner finally" — proof of the two-pass model. Java has no exception filters at all.

▲ Follow-up — "What's a filter good for?"

Two real uses: (1) logging without catching — `catch (Exception e) when (Log(e))` where `Log` returns `false`, so the exception continues with its stack trace **intact**; (2) conditional handling — `catch (SqlException e) when (e.Number == 1205)` for deadlock retries.

Q53. `throw` vs `throw ex`

```
try
{
    try { throw new Exception("original"); }
    catch (Exception ex) { throw ex; }           // ← A
    // catch (Exception) { throw; }             // ← B
}
catch (Exception e) { Console.WriteLine(e.StackTrace); }
```

✓ Output

A prints a stack trace starting at the `throw ex;` line — **everything below it is lost**.

B preserves the original trace all the way to where the exception was created.

The rule

"`throw ex;` **RESETS** the stack trace. `throw;` rethrows the original exception with its trace intact. Always use bare `throw;`."

▲ Follow-up — "And if you must wrap it?"

```
catch (Exception ex) { throw new DataException("Failed to load user", ex); }
```

Pass the original as the **inner exception** — never discard it. Also mention `ExceptionDispatchInfo.Capture(ex).Throw()` for rethrowing a captured exception from a different context (how `await` preserves traces).

Q54. `using` disposal order

```
using System;

class Res : IDisposable
{
    private readonly string _n;
    public Res(string n) { _n = n; Console.WriteLine($"open {_n}"); }
    public void Dispose() => Console.WriteLine($"dispose {_n}");
}

class Program
{
    static void Main()
    {
        using (var a = new Res("A"))
        using (var b = new Res("B"))
        {
            Console.WriteLine("body");
            throw new Exception("boom");
        }
    }
}
```

✓ Output

```
open A
open B
body
dispose B
dispose A
Unhandled exception: boom
```

The rule

"**using** compiles to **try/finally**, so disposal happens even on exception — and nested **using** s dispose in **REVERSE** order, innermost first."

▲ Follow-up — "What about the **using declaration** form?"

```
using var a = new Res("A");  
using var b = new Res("B");  
// disposed at the end of the enclosing scope, still B then A
```

Same semantics, less nesting. C# 8+. And the trap: **using** does nothing for **async disposal** — you need **await using** with **IAsyncDisposable**, otherwise you can block a thread inside **Dispose**.

Q55. Catch order

```
try { throw new ArgumentNullException("x"); }  
catch (Exception e)           { Console.WriteLine("Exception"); }  
catch (ArgumentException e)   { Console.WriteLine("ArgumentException"); }
```

✓ Output

```
✗ COMPILE ERROR CS0160: A previous catch clause already catches all  
  exceptions  
    of this or of a super type ('Exception')
```

The rule

"Catch blocks are tested top-down and must go **MOST** specific → **LEAST** specific. The compiler rejects an unreachable catch."

Same as Java. Correct order:

```
catch (ArgumentNullException) { } // most specific
catch (ArgumentException)     { }
catch (Exception)             { } // least specific
```

Q56. Exception from a **finally** block

```
try
{
    try { throw new Exception("original"); }
    finally { throw new Exception("from finally"); }
}
catch (Exception e) { Console.WriteLine(e.Message); }
```

✓ Output

```
from finally
```

The rule

"An exception thrown in **finally** REPLACES the in-flight exception, and the original is lost with no inner-exception link."

▲ Follow-up — "So what's the guidance?"

Never let **finally throw.** In cleanup code, wrap risky calls in their own **try/catch** and log. This is also why **Dispose()** implementations are expected never to throw.

I — Closures, LINQ & iterators

Q57. LINQ is deferred

```
using System;
using System.Collections.Generic;
using System.Linq;

var numbers = new List<int> { 1, 2, 3 };
var query = numbers.Where(n => n > 1);

numbers.Add(4);

Console.WriteLine(string.Join(",", query));
```

✓ Output

```
2,3,4
```

The rule

"LINQ query operators are **LAZY** — **Where** just builds a description. Nothing executes until you enumerate, and enumeration reads the collection *as it is at that moment*."

▲ Follow-up — "How do you force it?"

```
var query = numbers.Where(n => n > 1).ToList();    // executes NOW
numbers.Add(4);
Console.WriteLine(string.Join(",", query));        // 2,3
```

ToList, **ToArray**, **Count**, **First**, **Sum**, **Any** — the **terminal** operators execute immediately.

Q58. Double enumeration runs the query twice

```
using System;
using System.Linq;

int calls = 0;
var query = Enumerable.Range(1, 3).Select(x => { calls++; return x * 2; });

Console.WriteLine(query.Count());
Console.WriteLine(query.Sum());
Console.WriteLine($"lambda calls = {calls}");
```

✓ Output

```
3
12
lambda calls = 6
```

The rule

"Every enumeration of a deferred query re-executes the whole pipeline. Two terminal operators = two full runs."

▲ Follow-up — "When is that a production incident?"

When the source is a **database or an HTTP call** — you fire the query twice. Or when the projection has side effects. Or with **IEnumerable** over a network stream, where the second enumeration throws because the stream is consumed.

Rule: materialize with `ToList()` the moment you need the results more than once.

Q59. Closure capture in LINQ

```
using System;
using System.Linq;
using System.Collections.Generic;

var funcs = new List<Func<int>>();
for (int i = 0; i < 3; i++) funcs.Add(() => i);

Console.WriteLine(string.Join(",", funcs.Select(f => f())));
```

✓ Output

```
3,3,3
```

The rule

Same as **Q7** — the `for` loop has one `i`, hoisted into a single closure object, and it's `3` by the time the lambdas run.

▲ Follow-up — "How does the compiler actually do this?"

It generates a **display class** — a hidden compiler-generated class holding the captured variable — and rewrites `i` to be a field on one shared instance. Being able to say "display class" out loud is a strong signal you understand closures rather than memorizing the symptom.

Q60. First vs FirstOrDefault vs Single

```
using System;
using System.Linq;
using System.Collections.Generic;

var empty = new List<int>();
var two   = new List<int> { 1, 1 };

Console.WriteLine(empty.FirstOrDefault());
Console.WriteLine(two.First());
Console.WriteLine(empty.First());
Console.WriteLine(two.Single());
```

✓ Output

```
0
1
▲ InvalidOperationException: Sequence contains no elements
▲ InvalidOperationException: Sequence contains more than one element
```

The rule

"**First** throws on empty. **FirstOrDefault** returns **default(T)** — which is **0** for **int**, not **null**. **Single** throws on empty AND on more than one."

▲ Follow-up — "What's the trap with **FirstOrDefault** on a value type?"

You **cannot distinguish "not found" from "found the value 0"**. Use **FirstOrDefault()** on a nullable (**Cast<int?>().FirstOrDefault()**) or check **Any()** first. Same trap with **default(DateTime)** giving **01/01/0001** instead of **null**.

Q61. `yield return` doesn't run until you enumerate

```
using System;
using System.Collections.Generic;

class Program
{
    static IEnumerable<int> Gen()
    {
        Console.WriteLine("start");
        yield return 1;
        Console.WriteLine("middle");
        yield return 2;
        Console.WriteLine("end");
    }

    static void Main()
    {
        Console.WriteLine("before");
        var g = Gen();
        Console.WriteLine("after call");
        foreach (var x in g) Console.WriteLine(x);
    }
}
```

✓ Output

```
before
after call
start
1
middle
2
end
```

The rule

"Calling an iterator method executes NOTHING — it just returns a state machine. The body runs one `yield` at a time, driven by `MoveNext()` ."

Notice `"start"` prints **after** `"after call"`.

▲ Follow-up — "So where do you validate arguments in an iterator?"

```
public static IEnumerable<int> Take(IEnumerable<int> src, int n)
{
    if (src == null) throw new ArgumentNullException(nameof(src));    //
    thrown LAZILY
    return Impl(src, n);

    static IEnumerable<int> Impl(IEnumerable<int> s, int c) { foreach (var
x in s) yield return x; }
}
```

Split it into a non-iterator wrapper + a private iterator, exactly as shown.

Otherwise the `ArgumentNullException` doesn't surface until the caller enumerates — possibly in a completely different stack frame. This is how the BCL implements every LINQ operator.

Q62. Modifying the source while enumerating a LINQ query

```
using System;
using System.Linq;
using System.Collections.Generic;

var list = new List<int> { 1, 2, 3 };
var query = list.Where(x => x > 1);

foreach (var x in query)
{
    Console.WriteLine(x);
    list.Add(99);
}
```

✓ Output

```
2
▲ InvalidOperationException: Collection was modified; enumeration
operation may not execute
```

The rule

"`List<T>`'s enumerator holds a version stamp. Any structural change bumps it and the next `MoveNext()` throws — the deferred LINQ query is enumerating that same live list."

The C# analogue of Java's `ConcurrentModificationException`.

▲ Follow-up — "Fixes?"

Iterate a snapshot (`foreach (var x in query.ToList())`), collect-then-apply, loop backwards with a `for`, or use `ConcurrentBag` / `ConcurrentDictionary` for genuinely concurrent access.

Q63. `OrderBy` is stable, `List.Sort` is not

```
using System;
using System.Linq;
using System.Collections.Generic;

var items = new List<(string Name, int Group)>
{
    ("a", 1), ("b", 1), ("c", 1), ("d", 1), ("e", 1)
};

var linq = items.OrderBy(x => x.Group).Select(x => x.Name);
items.Sort((x, y) => x.Group.CompareTo(y.Group));

Console.WriteLine(string.Join("", linq));
Console.WriteLine(string.Join("", items.Select(x => x.Name)));
```

✓ Output

```
abcde      ← LINQ OrderBy is guaranteed stable
```

The `List.Sort` line prints **some permutation** — with all keys equal, introsort may reorder them arbitrarily. It is documented as **unstable**.

The rule

"**Enumerable.OrderBy** is documented as a **STABLE** sort. **List<T>.Sort** and **Array.Sort** use introsort and are **UNSTABLE** — equal elements may be reordered."

▲ Follow-up — "Why care?"

Anywhere you sort by one key and expect a previous sort to survive — e.g. "sort by date, then by priority" done in two passes. With an unstable sort the first pass is destroyed. Use **.OrderBy(...).ThenBy(...)** instead of two sorts.

Q64. **GroupBy** and **ToDictionary** on duplicate keys

```
using System;
using System.Linq;
using System.Collections.Generic;

var words = new[] { "apple", "avocado", "banana" };

var grouped = words.GroupBy(w => w[0]);
Console.WriteLine(grouped.Count());

var dict = words.ToDictionary(w => w[0]);
```

✓ Output

```
2
▲ ArgumentException: An item with the same key has already been added.
Key: a
```

The rule

"**GroupBy** collects duplicates into groups. **ToDictionary** requires keys to be **UNIQUE** and throws on the first duplicate."

▲ Follow-up — "How do you make `ToDictionary` safe?"

```
var dict = words.GroupBy(w => w[0]).ToDictionary(g => g.Key, g =>
g.ToList());
// or, keeping only the first:
var dict2 = words.GroupBy(w => w[0]).ToDictionary(g => g.Key, g =>
g.First());
```

J — `async / await`

Q65. Execution order

```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        Console.WriteLine("1");
        var t = DoWork();
        Console.WriteLine("3");
        await t;
        Console.WriteLine("5");
    }

    static async Task DoWork()
    {
        Console.WriteLine("2");
        await Task.Delay(100);
        Console.WriteLine("4");
    }
}
```

✓ Output

```
1  
2  
3  
4  
5
```

The rule

"An **async** method runs **SYNCHRONOUSLY** until the first **await** that actually needs to wait. So **2** prints before **3** — calling an async method is not the same as starting a thread."

This is the single most-asked async output question.

▲ Follow-up — "Where does the code after **await** run?"

On a captured **SynchronizationContext** (UI thread in WPF/WinForms, request context in classic ASP.NET) or on the thread pool if there is none — which is the case in Console apps and ASP.NET Core. **ConfigureAwait(false)** opts out of capturing it.

Q66. **.Result** deadlock

```
// In a WinForms/WPF/classic-ASP.NET app:  
public async Task<string> GetAsync()  
{  
    await Task.Delay(100);  
    return "done";  
}  
  
public void Button_Click(object sender, EventArgs e)  
{  
    string s = GetAsync().Result;    // ← what happens?  
    MessageBox.Show(s);  
}
```

✓ Output

DEADLOCK – the UI freezes forever

(the exact same code in a Console app or ASP.NET Core completes fine)

The rule

"**.Result** blocks the UI thread. When **Task.Delay** completes, the continuation needs the captured **SynchronizationContext** – the UI thread – which is blocked waiting on **.Result**. Each waits for the other."

▲ The follow-ups

Follow-up 1 – "Fix it."

```
public async void Button_Click(object sender, EventArgs e)    // async void
is OK for event handlers
{
    string s = await GetAsync();
    MessageBox.Show(s);
}
```

"**Async all the way down.**" Never mix blocking and async.

Follow-up 2 – "Why doesn't it deadlock in ASP.NET Core?"

ASP.NET Core removed the **SynchronizationContext**, so continuations go to the thread pool. It still **wastes a thread** and can cause pool starvation under load – it's a scalability bug rather than a deadlock.

Follow-up 3 – "And **ConfigureAwait(false)**?"

```
await Task.Delay(100).ConfigureAwait(false);    // library code – don't
capture the context
```

It fixes this specific deadlock, but the real fix is not blocking. Rule: **library code uses **ConfigureAwait(false)** ; application code doesn't need to.**

Q67. `async void` exceptions can't be caught

```
using System;
using System.Threading.Tasks;

class Program
{
    static async void Fire() { await Task.Delay(10); throw new
Exception("boom"); }
    static async Task FireT() { await Task.Delay(10); throw new
Exception("boom"); }

    static async Task Main()
    {
        try { Fire(); }
        catch (Exception e) { Console.WriteLine("caught void: " +
e.Message); }

        try { await FireT(); }
        catch (Exception e) { Console.WriteLine("caught task: " +
e.Message); }

        await Task.Delay(200);
    }
}
```

✓ Output

```
caught task: boom
▲ ...then the process CRASHES with the unhandled "boom" from Fire()
```

The rule

"An `async void` method has no `Task` to carry the exception, so it is re-thrown on the `SynchronizationContext` and becomes an **UNHANDLED** exception that kills the process. `async Task` stores the exception in the task, where `await` rethrows it into your `catch`."

Note the `try/catch` around `Fire()` catches **nothing** — the method returned at its first `await`, long before it threw.

▲ Follow-up — "When is `async void` acceptable?"

Only for event handlers, where the signature is fixed by the framework — and then the body should be wrapped in its own `try/catch`. Everything else returns `Task`.

Q68. Sequential `await` vs `WhenAll`

```
using System;
using System.Diagnostics;
using System.Threading.Tasks;

class Program
{
    static Task<int> Work(int ms) => Task.Delay(ms).ContinueWith(_ => ms);

    static async Task Main()
    {
        var sw = Stopwatch.StartNew();
        var a = await Work(1000);
        var b = await Work(1000);
        Console.WriteLine($"sequential: ~{sw.ElapsedMilliseconds}ms");

        sw.Restart();
        var t1 = Work(1000);
        var t2 = Work(1000);
        await Task.WhenAll(t1, t2);
        Console.WriteLine($"parallel: ~{sw.ElapsedMilliseconds}ms");
    }
}
```

✓ Output

```
sequential: ~2000ms
parallel: ~1000ms
```

The rule

"**await** in sequence starts the second task only after the first completes. Starting both tasks first and awaiting **WhenAll** overlaps them."

▲ Follow-up — "What's the trap in a **foreach**?"

```
foreach (var id in ids) await FetchAsync(id);           // N × latency
var results = await Task.WhenAll(ids.Select(FetchAsync)); // ✓
overlapped
```

But add the caveat interviewers wait for: *"...as long as the downstream can take the concurrency — otherwise throttle with **SemaphoreSlim** or **Parallel.ForEachAsync** with **MaxDegreeOfParallelism**."*

Q69. **Task.WhenAll** and multiple exceptions

```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task Fail(string m) { await Task.Delay(10); throw new
Exception(m); }

    static async Task Main()
    {
        var t = Task.WhenAll(Fail("A"), Fail("B"));
        try { await t; }
        catch (Exception e) { Console.WriteLine("caught: " + e.Message); }

        Console.WriteLine("inner count: " +
            (t.Exception?.InnerExceptions.Count ?? 0));
    }
}
```

✓ Output

```
caught: A  
inner count: 2
```

The rule

"**await** on a faulted task rethrows only the **FIRST** exception, to keep stack traces meaningful. All of them are still on **task.Exception**, which is an **AggregateException**."

▲ Follow-up — "How do you get all of them?"

```
try { await t; }  
catch { foreach (var e in t.Exception.InnerExceptions)  
    Console.WriteLine(e.Message); }
```

Compare with **.Wait()** / **.Result**, which throw the **AggregateException** **directly** — another visible difference between blocking and awaiting.

Q70. **async** without **await**

```
using System;  
using System.Threading.Tasks;  
  
class Program  
{  
    static async Task<int> NoAwait()  
    {  
        Console.WriteLine("running");  
        return 42;  
    }  
  
    static async Task Main() => Console.WriteLine(await NoAwait());  
}
```

✓ Output

```
running
42
```

(plus compiler warning CS1998: "This async method lacks 'await' operators and will run synchronously")

The rule

"**async** without **await** compiles and works, but it runs completely synchronously while still paying for the state machine. The compiler warns you."

▲ Follow-up — "How do you write a synchronous implementation of an async interface method?"

```
public Task<int> GetAsync() => Task.FromResult(42);           //
✓ no async, no state machine
public Task DoAsync()      => Task.CompletedTask;
```

Also mention **ValueTask<T>** for hot paths that usually complete synchronously — it avoids the **Task** allocation entirely.

K — Collections & equality

Q71. Modify during **foreach**

```
var list = new List<int> { 1, 2, 3 };
foreach (var x in list)
{
    if (x == 2) list.Remove(x);
}
```

✓ Output

▲ `InvalidOperationException: Collection was modified; enumeration operation may not execute`

The rule

"`List<T>` keeps a `_version` counter; the enumerator snapshots it and throws on the next `MoveNext()` if it changed."

▲ Follow-up — "Which is the sneakiest way this passes silently?"

Removing the **second-to-last** element: `MoveNext()` sees `index >= _size` and returns `false` *before* checking the version, so the loop just ends early with **no exception**. That's a silent data bug — worse than the crash.

Safe forms:

```
list.RemoveAll(x => x == 2);           // best
for (int i = list.Count - 1; i >= 0; i--) { } // backwards
foreach (var x in list.ToList()) { }    // snapshot
```

Q72. Mutating a dictionary key

```
using System;
using System.Collections.Generic;

class Key
{
    public int Id;
    public override int GetHashCode() => Id.GetHashCode();
    public override bool Equals(object o) => o is Key k && k.Id == Id;
}

var k = new Key { Id = 1 };
var d = new Dictionary<Key, string> { [k] = "value" };

Console.WriteLine(d.ContainsKey(k));
k.Id = 2;
Console.WriteLine(d.ContainsKey(k));
Console.WriteLine(d.Count);
```

✓ Output

```
True
False
1
```

The rule

"The dictionary placed the entry in the bucket for hash(1). Changing **Id** changes the hash, so lookup goes to the wrong bucket. The entry is still there — it's now **UNREACHABLE**."

Same as Java's `HashMap`.

▲ Follow-up — "The rule for keys?"

Dictionary keys must be immutable — use `string`, `int`, a `readonly record struct`, or a class whose hash-relevant fields are `readonly`. And: "if you override `Equals` you must override `GetHashCode`" — equal objects must have equal hashes, or a `Dictionary` / `HashSet` will lose them.

Q73. **record** vs **class** equality

```
using System;

record PersonR(string Name, int Age);
class PersonC { public string Name; public int Age; }
readonly record struct PointR(int X, int Y);

var r1 = new PersonR("Ann", 30);
var r2 = new PersonR("Ann", 30);
var c1 = new PersonC { Name = "Ann", Age = 30 };
var c2 = new PersonC { Name = "Ann", Age = 30 };

Console.WriteLine(r1 == r2);
Console.WriteLine(r1.Equals(r2));
Console.WriteLine(ReferenceEquals(r1, r2));
Console.WriteLine(c1 == c2);
Console.WriteLine(r1);
```

✓ Output

```
True
True
False
False
PersonR { Name = Ann, Age = 30 }
```

The rule

"A **record** gets value-based **Equals**, **GetHashCode**, **==** / **!=** and a readable **ToString** generated by the compiler. A plain **class** gets reference equality and a type-name **ToString**."

ReferenceEquals is still **False** — a record is a **class** with value semantics, not a value type.

▲ Follow-up — "What else does a record give you?"

with expressions for non-destructive mutation, **Deconstruct**, and **init**-only setters:

```
csharp var older = r1 with { Age = 31 }; // new instance, Name copied
```

▲ **Trap:** **with** is a **shallow** copy — a **List<T>** property is shared between the original and the copy.

Q74. Array covariance

```
object[] arr = new string[2];  
arr[0] = "ok";  
arr[1] = 42;
```

✓ Output

▲ **ArrayTypeMismatchException** at runtime on the last line

(it compiles without warning)

The rule

"Arrays are covariant — **string[]** is assignable to **object[]** — but the runtime enforces the real element type on every write. It's a known design mistake in both C# and Java."

▲ Follow-up — "How did generics fix it?"

List<string> is **not** assignable to **List<object>** — generic type parameters are invariant unless declared **out** (covariant, output-only, e.g. **IEnumerable<out T>**) or **in** (contravariant, input-only, e.g. **IComparer<in T>**). That's why **IEnumerable<string>** → **IEnumerable<object>** works but **IList<string>** → **IList<object>** doesn't.

Q75. Dictionary ordering is not guaranteed

```
using System;
using System.Collections.Generic;

var d = new Dictionary<string, int> { ["z"] = 1, ["a"] = 2, ["m"] = 3 };
foreach (var kv in d) Console.Write(kv.Key + " ");
```

✓ Output

z a m ← usually insertion order... but this is NOT guaranteed

The rule

"**Dictionary<K,V>** makes NO ordering guarantee. It happens to enumerate in insertion order when nothing has been removed, but a single **Remove** followed by an **Add** reuses the freed slot and scrambles it. Never rely on it."

▲ Follow-up — "What if I need order?"

SortedDictionary<K,V> (key order, tree, $O(\log n)$), **SortedList<K,V>** (key order, array, faster lookup / slower insert), or a **List<KeyValuePair<K,V>>** for true insertion order. Java's answer — **LinkedHashMap** — has no direct BCL equivalent, which is a good thing to know out loud.

L — Rapid-fire answer table

Cover the right column. Target: **under 5 minutes**, daily.

#	Program	Output	Why (one line)
1	<code>Test(object)</code> / <code>Test(string)</code> with <code>Test(null)</code>	<code>string method</code>	Most-specific overload, compile-time
2	Virtual call from base ctor, derived field initialized	field is set	C# runs derived field initializers <i>before</i> base ctor
3	<code>Speak()</code> hidden without <code>virtual</code>	base version	Non-virtual by default → compile-time binding
4	<code>return</code> inside <code>finally</code>	✗ CS0157	C# forbids leaving a <code>finally</code>
5	<code>struct</code> assigned then mutated	original unchanged	Value types copy on assignment
6	<code>int.MaxValue + 1</code> (literal)	✗ CS0220	Constants are evaluated <code>checked</code>
7	Lambdas in <code>for</code> vs <code>foreach</code>	<code>333</code> then <code>012</code>	<code>for</code> has one variable; <code>foreach</code> a fresh one (C# 5+)
8	Optional param on <code>override</code>	derived body, base default	Defaults baked at compile time
9	Box, then change the original	old value	Boxing snapshots
10	<code>object a=5, b=5; a==b</code>	<code>False</code>	Two boxes, reference comparison
11	<code>list[0].X = 5</code> on <code>List<struct></code>	✗ CS1612	Indexer is a property → returns a copy
12	Method call on a <code>readonly</code> struct field	no change	Defensive copy
13	Reassign an array parameter	caller unaffected	Reference passed <i>by value</i>

#	Program	Output	Why (one line)
14	<code>default(struct)</code> vs <code>default(class)</code>	zeroed / <code>null</code>	Value vs reference
15	<code>structA == structB</code>	✗ CS0019	<code>==</code> isn't auto-generated for structs
16	struct → interface variable	boxes, diverges	Interface assignment boxes
17	<code>override</code> vs <code>new</code> via base ref	<code>B</code> / <code>A</code>	vtable slot vs new method
18	<code>new virtual</code> then <code>override</code>	<code>A.M</code>	A second, separate virtual chain
19	<code>override</code> a <code>sealed override</code>	✗ CS0239	Chain terminated
20	Interface re-listed + <code>new</code>	<code>ColoredSquare.Dr</code> <code>aw</code>	Interface re-implementation
21	Explicit interface impl on the class	✗ CS1061	Not a class member
22	Abstract method called from base ctor	<code>0</code>	Derived ctor body hasn't run
23	<code>instance.StaticMethod()</code>	✗ CS0176	C# forbids it (Java allows)
24	<code>B.F(long)</code> vs inherited <code>A.F(int)</code>	<code>B.F(long)</code>	Search stops at the most-derived hit
25	Full init order	derived fields → base statics → base fields → base ctor → derived ctor	Memorize the ladder
26	Three <code>new Config()</code>	static ctor once	Type initializer runs once

#	Program	Output	Why (one line)
27	<code>const</code> vs <code>static</code> <code>readonly</code> across assemblies	stale / current	<code>const</code> is inlined into callers
28	<code>int b = a + 1;</code> as a field initializer	✗ CS0236	Can't touch instance members
29	<code>static int A = B</code> <code>+ 1; static int</code> <code>B = 10;</code>	A=1	Textual order
30	<code>(object)a ==</code> <code>(object)new</code> <code>string(...)</code>	False	Not interned
31	<code>s.ToUpper();</code> ignoring the result	unchanged	Strings are immutable
32	<code>"x" + null</code>	"x"	Concat treats null as empty
33	<code>1 + 2 + "3" + 4</code> <code>+ 5</code>	3345	Left-associative
34	<code>"" ==</code> <code>string.Empty</code>	True	Same interned instance
35	<code>"apple".CompareTo</code> <code>("Apple")</code>	culture-dependent	Always pass a <code>StringComparison</code>
36	<code>switch</code> case without <code>break</code>	✗ CS0163	No implicit fall- through
37	<code>i = i++</code>	0	Post-increment returns the old value
38	<code>-5 / 2, -5 % 2</code>	-2, -1	Truncates toward zero; <code>%</code> takes the dividend's sign
39	<code>10.0/0</code> vs <code>10/a</code>	∞ / ▲	IEEE 754 vs <code>DivideByZeroExcep</code> <code>tion</code>

#	Program	Output	Why (one line)
40	<code>0.1 + 0.2 == 0.3</code>	<code>False</code>	Binary floating point
41	<code>b = b + 5</code> on a <code>byte</code>	<code>×</code> CS0266 (but <code>b += 5</code> is fine)	Compound assignment has a hidden cast
42	<code>'a' + 1</code>	<code>98</code>	<code>char</code> widens to <code>int</code>
43	<code>true ? 1 : "two"</code>	<code>×</code> CS0173	No common type
44	<code>property++</code>	works, not atomic	get + add + set
45	<code>null is object</code>	<code>False</code>	<code>null</code> has no type
46	<code>a > b</code> and <code>a < b</code> with <code>a == null</code>	both <code>False</code>	Lifted relational operators
47	<code>null + 5 (int?)</code>	<code>null</code>	Nulls propagate
48	<code>s?.A().B().C</code> on <code>null</code>	<code>null</code> , whole chain	<code>?.</code> short-circuits everything after it
49	<code>a ??= Get()</code> twice	one call	Short-circuits
50	<code>input!</code> returning <code>null</code>	<code>▲</code> NRE	NRTs are compile-time only
51	<code>return</code> in <code>catch</code> + <code>finally</code>	<code>finally</code> still runs	Runs before the actual return
52	Exception filter + inner <code>finally</code>	filter first	Two-pass exception model
53	<code>throw ex;</code>	trace reset	Use bare <code>throw;</code>
54	Nested <code>using</code>	disposes B then A	Reverse order
55	<code>catch(Exception)</code> before <code>catch(ArgumentException)</code>	<code>×</code> CS0160	Unreachable catch

#	Program	Output	Why (one line)
56	<code>throw</code> inside <code>finally</code>	replaces the original	Original exception is lost
57	LINQ then <code>Add</code> to source	new item included	Deferred execution
58	<code>Count()</code> then <code>Sum()</code> on a query	pipeline runs twice	Re-enumeration
59	Lambdas capturing <code>for</code> 's <code>i</code>	<code>3,3,3</code>	Shared display-class field
60	<code>empty.First()</code>	▲ <code>InvalidOperationException</code>	<code>FirstOrDefault</code> returns <code>default(T)</code>
61	Calling an iterator method	nothing runs	State machine built lazily
62	<code>list.Add</code> inside a LINQ <code>foreach</code>	▲ <code>InvalidOperationException</code>	Version stamp
63	<code>OrderBy</code> vs <code>List.Sort</code>	stable vs unstable	Introsort reorders equals
64	<code>ToDictionary</code> with duplicate keys	▲ <code>ArgumentException</code>	Keys must be unique
65	Async method call before first real <code>await</code>	runs synchronously	<code>2</code> prints before <code>3</code>
66	<code>.Result</code> on a UI thread	deadlock	Blocked context can't run the continuation
67	<code>async void</code> throwing	process crash	No Task to carry the exception
68	<code>await</code> in a loop vs <code>WhenAll</code>	2000ms vs 1000ms	Sequential vs overlapped

#	Program	Output	Why (one line)
69	<code>WhenAll</code> with 2 failures	first message only	<code>await</code> rethrows the first; all are in <code>.Exception</code>
70	<code>async</code> with no <code>await</code>	runs sync + CS1998	Pointless state machine
71	<code>Remove</code> inside <code>foreach</code>	▲ (or silently short!)	Version check happens after the bounds check
72	Mutate a dictionary key	entry unreachable	Hash bucket no longer matches
73	<code>record ==</code>	<code>True</code>	Compiler-generated value equality
74	<code>object[] a = new string[2]; a[1] = 42;</code>	▲ ArrayTypeMismatchException	Array covariance
75	<code>Dictionary</code> enumeration order	unspecified	Never rely on it

M — 20 self-test drills (answers hidden)

Cover the answer, commit out loud, then check. Do **five a day**.

Drill 1

```
struct S { public int X; }
var arr = new S[1];
arr[0].X = 5;
var copy = arr[0];
copy.X = 10;
Console.WriteLine(arr[0].X);
```

→ Answer

5 — `arr[0].X = 5` writes into the array slot directly, but `var copy = arr[0]` takes a **copy**, so mutating `copy` doesn't touch the array.

Drill 2

```
class A { public A() { Console.WriteLine("A"); } }
class B : A { public B() : this(1) { Console.WriteLine("B"); } public B(int x)
{ Console.WriteLine("C"); } }
new B();
```

→ Answer

ACB — `B()` chains to `B(int)` via `this(1)`. `B(int)` has an implicit `: base()`, so `A()` runs first (A), then `B(int)`'s body (C), then `B()`'s body (B).

Drill 3

```
Console.WriteLine("5" + 3 + 2);
Console.WriteLine(5 + 3 + "2");
```

→ Answer

532 and **82** — left-associative. First line: string from the start. Second line: `5+3=8` arithmetic, then concat.

Drill 4

```
int? x = null;
Console.WriteLine(x == null);
Console.WriteLine(x.HasValue);
Console.WriteLine(x.GetType());
```

→ Answer

`True`, `False`, then  **NullReferenceException** — calling `GetType()` on a null `Nullable` boxes it, and boxing a null nullable produces a **real null** reference. Great trap: `int?` isn't a reference type, but boxing it can still give you `null`.

Drill 5

```
var list = new List<int> { 1, 2, 3, 4 };
list.Remove(3);
list.RemoveAt(0);
Console.WriteLine(string.Join(",", list));
```

→ Answer

2,4 — `Remove(3)` removes the **value** 3 → `[1,2,4]`; `RemoveAt(0)` removes the **index** 0 → `[2,4]`. (The `remove(int)` vs `remove(Object)` trap from Java is **explicit** in C#, which is why C# names them differently.)

Drill 6

```
static void Swap(int a, int b) { var t = a; a = b; b = t; }
static void SwapRef(ref int a, ref int b) { var t = a; a = b; b = t; }
int x = 1, y = 2;
Swap(x, y);    Console.Write($"{x}{y} ");
SwapRef(ref x, ref y); Console.Write($"{x}{y}");
```

→ Answer

12 21 — plain parameters are copies; `ref` passes the variable itself.

Drill 7

```
Console.WriteLine(typeof(int) == typeof(Int32));
Console.WriteLine(typeof(int?) == typeof(Nullable<int>));
Console.WriteLine(typeof(string) == typeof(String));
```

→ Answer

True True True — `int`, `int?` and `string` are C# **aliases** for `System.Int32`, `System.Nullable` and `System.String`. They're the same types, so `string.Empty` and `String.Empty` are also identical.

Drill 8

```
static class Ext { public static bool IsEmpty(this string s) => s == null
|| s.Length == 0; }
string s = null;
Console.WriteLine(s.IsEmpty());
```

→ Answer

****True**** — no `NullReferenceException`! An extension method is a ****static call**** (`Ext.IsEmpty(s)`), so `this` may legally be null. Only *instance* method calls null-check the receiver.

Drill 9

```
enum Color { Red = 1, Green, Blue }
Color c = (Color)10;
Console.WriteLine(c);
Console.WriteLine(Enum.IsDefined(typeof(Color), c));
Console.WriteLine((Color)0);
```

→ Answer

****10****, ****False****, ****0**** — casting an out-of-range value to an enum ****never throws****. `ToString()` falls back to the number. And since no member has value 0, `(Color)0` and `default(Color)` also print `0` — which is why you should always define a `None = 0` member.

Drill 10

```
var a = new { X = 1, Y = 2 };
var b = new { X = 1, Y = 2 };
Console.WriteLine(a.Equals(b));
Console.WriteLine(ReferenceEquals(a, b));
Console.WriteLine(a.GetType() == b.GetType());
```

→ Answer

****True False True**** — anonymous types get compiler-generated ****value equality**** (like records), and two anonymous types with the same property names, types and order in the same assembly are the ****same generated type****.

Drill 11

```
static int F(int x) { Console.Write("F"); return x; }  
if (false && F(1) > 0) { }  
if (false & F(2) > 0) { }
```

→ Answer

****F**** (printed once) — `&&` short-circuits, `&` does not. The non-short-circuiting `&/|` still evaluate both operands.

Drill 12

```
var sb = new StringBuilder("a");  
void Mutate(StringBuilder b) { b.Append("b"); b = new StringBuilder("z");  
b.Append("!"); }  
Mutate(sb);  
Console.WriteLine(sb);
```

→ Answer

****ab**** — `Append` mutates the shared object; reassigning the parameter is invisible to the caller. Same as Q13.

Drill 13

```
Console.WriteLine(Math.Round(2.5));  
Console.WriteLine(Math.Round(3.5));  
Console.WriteLine(Math.Round(2.5, MidpointRounding.AwayFromZero));
```

→ Answer

****2**, **4**, **3**** — .NET's default is **banker's rounding** (round-half-to-even), which surprises almost everyone. It exists to avoid statistical bias when summing many rounded values. For money, pass `MidpointRounding.AwayFromZero` explicitly.

Drill 14

```
IEnumerable<int> Numbers() { for (int i = 0; i < 3; i++) yield return i; }  
var n = Numbers();  
Console.WriteLine(n.Count());  
Console.WriteLine(n.Count());
```

→ Answer

3 and **3** — each `Count()` starts a **fresh** enumeration; the iterator restarts from the top. (Contrast with an `IEnumerator`, which is one-shot.)

Drill 15

```
class Node { public Node Next; }  
var a = new Node();  
a.Next = a;  
Console.WriteLine(a.Next.Next.Next == a);
```

→ Answer

True — self-reference; every hop lands back on `a`. Follow-up they'll ask: "will the GC collect this?" → **Yes.** .NET uses a **tracing** collector (mark-and-sweep from GC roots), not reference counting, so cycles unreachable from a root are collected.

Drill 16

```
try  
{  
    using var fs = new MemoryStream();  
    throw new Exception("x");  
}  
catch (Exception e) { Console.WriteLine("caught " + e.Message); }  
Console.WriteLine("done");
```

→ Answer

caught x then **done** — the `using` declaration disposes the stream as the scope unwinds (before the `catch` body runs), and the exception is caught normally.

Drill 17

```
static T Max<T>(T a, T b) where T : IComparable<T> => a.CompareTo(b) > 0 ?  
a : b;  
Console.WriteLine(Max(3, 5));  
Console.WriteLine(Max("apple", "banana"));  
Console.WriteLine(Max(3, 5.0));
```

→ Answer

`**5**`, `**banana**`, then ✗ `**compile error CS0411**` on the third — type inference can't unify ``int`` and ``double`` into a single ``T``. Fix: ``Max(3, 5.0)``.

Drill 18

```
object o = null;  
Console.WriteLine(o?.ToString() ?? "null");  
Console.WriteLine($"{o}");  
Console.WriteLine(o + "x");
```

→ Answer

`**null**`, `**""` (blank line), `**x**` — `?.`` short-circuits; interpolation and ``,`` both convert null to empty. None of them throw.

Drill 19

```
var d = new Dictionary<string, int>();  
d["a"] = 1;  
d["a"] = 2;  
d.Add("b", 1);  
d.Add("b", 2);
```

→ Answer

▲ `**ArgumentException: An item with the same key has already been added**` on the last line. `**The indexer `d["a"] = 2` overwrites silently; `Add` throws on a duplicate key.**` Knowing which one throws is the whole question. (`TryAdd`` returns ``false`` instead of throwing.)

Drill 20

```
class A { public virtual string Name => "A"; }
class B : A { public override string Name => "B"; }
class C : B { public new string Name => "C"; }

A x = new C();
B y = new C();
C z = new C();
Console.WriteLine($"{x.Name} {y.Name} {z.Name}");
```

→ Answer

****`B B C`**** — `C.Name`` uses `new``, so it only exists for compile-time type `C``. Through `A`` or `B`` you reach the virtual chain, whose most-derived override is `B.Name``. (Properties follow the exact same virtual/hiding rules as methods.)

N — How to answer these in the actual interview

1. Never blurt the answer. State the rule, then the answer.

"This is overload resolution, which is a compile-time decision — the compiler picks the most specific applicable overload. `string` is more specific than `object`, so it prints `string method`."

2. Ask "does it compile?" first. Roughly a quarter of these are compile errors, and candidates who start tracing the runtime walk straight past them.

3. Ask "struct or class?" second. In C# this single question decides Q5, Q9–Q16, Q71 and half the drills. Say it out loud — *"first thing I'm checking is whether `Point` is a struct"* — because that's the reasoning they're grading.

4. Name the error precisely. *"Compile error"* is worth one point. *"CS0157 — control cannot leave the body of a finally clause, because C# forbids `return` inside `finally`, unlike Java"* is worth five.

5. Use your Java background deliberately. When the answer differs, say so:

"In Java this would print *Dog barks* because methods are virtual by default. In C# they're non-virtual by default, so without *virtual / override* this binds to the compile-time type and prints *Animal speaks*."

That one sentence proves you understand the *mechanism*, not just this language. It is the strongest thing you can say in a .NET interview as a Java developer, and it turns your background from a liability into an asset.

6. Volunteer the fix. After every broken program, say how you'd correct it. That's what turns a quiz into a conversation.

7. If you genuinely don't know:

"My reasoning says *X* because of [rule], but this is exactly the kind of thing I'd verify with a quick unit test or on sharplab.io before relying on it."

Mentioning sharplab.io — which shows the lowered C# and the IL — is a real signal. It's how you'd *actually* answer "does the compiler add a cast here?" in a code review.

Related files in this pack

- **23 — Java "Predict the Output"** — the Java twin of this file; Section A above maps directly onto it
- **17 — SOLID & Design Patterns** — language-agnostic; the Liskov principle behind Q3 and Q17
- **22 — Java Streams** — every Stream problem there has a LINQ equivalent; `stream().filter().map().collect()` → `.Where().Select().ToList()`
- **24 — ORM, JPA & Hibernate** — the .NET equivalent is **Entity Framework Core**; lazy loading, change tracking and N+1 are the same problems with different names
- **26 — Companies: Questions Actually Asked** — log every .NET round here the same day

Drill routine: Section A twice (it's where the marks are), then the Section L rapid-fire table daily under 5 minutes, then 5 of the Section M drills on paper each morning. **Type the programs — don't just read them.** Reading these gives you recognition; typing them gives you the answer under pressure.

ASP.NET MVC — 62 Tricky "What Happens?" Programs with Answers

● **Why this file exists:** your interviews ask **output-based programs**, not theory ("what does this print?"). MVC rounds do the same thing in a different shape: *"this action runs — what does the browser get?", "this filter and that filter — which order?", "the model says X but the textbox shows Y — why?"*

This is the MVC twin of [23 — Java Predict the Output](#).

Every question below uses the same format:

1. **The code**
2. **What actually happens** (or *exception / 404 / 400*)
3. **The rule** — the one sentence that makes it obvious
4. **The follow-up** they ask next

■ **Framework coverage.** Answers are for **ASP.NET Core MVC** (the default in any new .NET job). Wherever classic **ASP.NET MVC 5** behaves differently there's a *** In MVC 5 box*. *Knowing both* is the point — *"what changed in Core?"* is itself one of the most-asked MVC questions, and answering it proves you understand the mechanism rather than the syntax.

◆ **Golden habit — ask these four in order, every time:**

(a) Did routing even match this action? → (b) Did model binding succeed, and where did the value come from? → (c) Which filters ran, in what order? → (d) Only then look at the action body.

Candidates lose MVC questions at (a), (b) and (c). Almost nobody loses them at (d) — but almost everybody *starts* at (d).

Table of contents

#	Section	What it tests
A	Routing & action selection (Q1–Q8)	Route order, attribute vs conventional, ambiguity, optional params
B	Model binding (Q9–Q17)	Source priority, <code>[FromBody]</code> , prefixes, collections, silent failures
C	Validation & ModelState (Q18–Q22)	<code>IsValid</code> timing, <code>[Required]</code> on value types, auto-400
D	ViewBag / ViewData / TempData (Q23–Q29)	<code>dynamic</code> typos, the shared dictionary, one-request lifetime
E	Action results & redirects (Q30–Q35)	<code>View</code> vs <code>Redirect</code> , PRG, code after a redirect, <code>JsonRequestBehavior</code>
F	Filters & order (Q36–Q42)	Global/controller/action order, short-circuiting, what exception filters miss
G	Razor traps (Q43–Q51)	▲ The <code>TextBoxFor</code> / <code>ModelState</code> trap , <code>@</code> parsing, encoding, layout order
H	Controller lifecycle & state (Q52–Q55)	Per-request instances, static fields, <code>async void</code> , <code>HttpContext.Current</code>
I	DI & middleware (Q56–Q62)	Lifetimes, captive dependencies, pipeline order, <code>UseX</code> placement
J	Rapid-fire table	1-line answers for last-minute revision
K	18 self-test drills	Answers hidden — test yourself

#	Section	What it tests
L	How to answer in the room	The script that turns a quiz into a conversation

A — Routing & action selection

Q1. Which route wins?

```
// Program.cs
app.MapControllerRoute(
    name: "blog",
    pattern: "{controller}/{action}/{slug}");

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

GET /Products/Details/5

```
public class ProductsController : Controller
{
    public IActionResult Details(int id) => Content($"id={id}");
    public IActionResult Details(string slug) => Content($"slug={slug}");
}
```

✓ What happens

▲ `AmbiguousMatchException`:
The request matched multiple endpoints.

The rule

"Conventional routes are matched in the order they're registered — the *first* match wins and routing stops. But two actions with the same name and the

same HTTP verb are ambiguous at *action selection* time, which happens after routing."

Two separate mechanisms, two separate failures. Route order picks the **route**; action selection picks the **method** — and here it can't.

▲ The follow-ups

Follow-up 1 — "Fix it without renaming."

```
[HttpGet("Products/Details/{id:int}")]
public IActionResult Details(int id) => Content($"id={id}");

[HttpGet("Products/Details/{slug}")]
public IActionResult Details(string slug) => Content($"slug={slug}");
```

The `:int` **route constraint** makes the two mutually exclusive, so only one endpoint can ever match.

Follow-up 2 — "If I remove the second action, which route matches?"

The **blog** route — it was registered first and `{controller}/{action}/{slug}` matches `/Products/Details/5` perfectly. `slug` would bind the string `"5"`. **Registration order is the whole answer.** Put specific routes before general ones.

Q2. Attribute routing silently disables conventional routing

```
app.MapControllerRoute("default", "{controller=Home}/{action=Index}/{id?}");

public class ProductsController : Controller
{
    [Route("catalog/items")]
    public IActionResult Index() => Content("Index");

    public IActionResult List() => Content("List");
}
```

```
GET /catalog/items → ?  
GET /Products/Index → ?  
GET /Products/List → ?
```

✓ What happens

```
GET /catalog/items → 200 "Index"  
GET /Products/Index → 404  
GET /Products/List → 200 "List"
```

The rule

"Once an action has a **[Route]** attribute, it is reachable **ONLY** through that attribute — conventional routes can no longer reach it. Actions *without* an attribute on the same controller still use conventional routing."

That mixed state (**List** works conventionally, **Index** doesn't) is exactly what makes this a good interview question. It looks like the whole controller switched over. It didn't.

▲ Follow-up — "And if I put **[Route]** on the controller?"

```
[Route("catalog/{controller}/{action}")]  
public class ProductsController : Controller { }
```

Then **every** action is attribute-routed and the conventional route reaches none of them. **[controller]** and **[action]** are **token replacements** filled in at startup — using them keeps the URL in sync when you rename the class.

Q3. Two actions, one name, different verbs

```
[HttpGet]
public IActionResult Edit(int id) => View();

[HttpPost]
public IActionResult Edit(Product p) => RedirectToAction("Index");
```

✓ What happens

✓ Works perfectly – no ambiguity

The rule

"Action selection filters candidates by HTTP verb FIRST. Two same-named actions are fine as long as no single request could match both."

▲ But this is **not** overloading in the C# sense — the CLR allows it only because the *signatures* differ. Two GET actions with the **same signature** don't compile at all, and two GET actions with **different** signatures give you Q1's `AmbiguousMatchException` at runtime.

▲ Follow-up — "How do you have two GETs with the same name?"

```
[HttpGet]
public IActionResult Edit(int id) => View();

[HttpGet]
[ActionName("EditByName")]           // ← different *action* name, same C#
method name
public IActionResult Edit(string name) => View();
```

`[ActionName]` decouples the **routing name** from the **method name**. Same trick is used to expose `Delete` as a POST-only action named `Delete` while the C# method is `DeleteConfirmed`.

Q4. Missing `id` on a non-nullable parameter

```
// route: {controller=Home}/{action=Index}/{id?}

public IActionResult Details(int id) => Content($"id = {id}");
```

```
GET /Home/Details
```

✓ **What happens — and this is where Core and MVC 5 differ completely**

ASP.NET Core:

```
200 OK
id = 0
```

The action **runs**, `id` is `0`, and `ModelState` contains an error — which nobody checks, so the bug ships silently.

In MVC 5

▲ `ArgumentException`:
The parameters dictionary contains a null entry for parameter 'id' of non-nullable type 'System.Int32' for method 'Details(Int32)'.
An optional parameter must be a reference type, a nullable type, or be declared as an optional parameter.

Memorize that error message — it's one of the most-quoted exceptions in ASP.NET MVC, and being able to recite what it *means* ("the route made `id` optional but the method signature didn't") is an instant credibility win.

The rule

"A route can make a segment optional; a non-nullable parameter cannot be optional. MVC 5 throws. Core binds the default value and records a `ModelState` error instead — so it fails silently unless you check `ModelState.IsValid`."

▲ Follow-up — "Three ways to fix it."

```
public IActionResult Details(int? id)    { if (id == null) return
NotFound(); } // nullable
public IActionResult Details(int id = 0)
{ }                                     // default value
[HttpGet("Home/Details/
{id:int}")]                           // constraint →
404 instead
public IActionResult Details(int id)    { }
```

The **third** is usually the right one: a request with no id isn't a valid request, so it should 404 at routing, not reach your code at all.

Q5. Route constraint mismatch

```
app.MapControllerRoute("default", "{controller=Home}/{action=Index}/
{id:int}");
```

```
public IActionResult Details(int id) => Content($"id = {id}");
```

```
GET /Home/Details/abc
GET /Home/Details
```

✓ What happens

```
GET /Home/Details/abc    → 404
GET /Home/Details        → 404
```

The rule

"A route constraint is part of MATCHING, not validation. If the constraint fails, the route simply doesn't match — you get a 404, never a 400 and never an exception."

The second 404 catches people out: `{id:int}` without `?` makes the segment **required**.

▲ Follow-up — "Constraints you'd actually use?"

`{id:int}`, `{id:guid}`, `{id:min(1)}`, `{slug:alpha}`, `{code:length(6)}`, `{date:datetime}`, `{id:regex(^\d{4}$)}`. And the point worth making: **`:int` is a routing decision, `[Range]` is a validation decision.** Use routing to reject requests that aren't for you; use validation to reject requests that are for you but wrong.

Q6. Route defaults vs the action's default

```
app.MapControllerRoute("default", "{controller=Home}/{action=Index}/{id=99}");
```

```
public IActionResult Index(int id = 5) => Content($"id = {id}");
```

GET /

✓ What happens

`id = 99`

The rule

"The route default supplies a *value* into route data, and model binding finds it there. The C# default only applies when binding produces nothing at all — and here it produced 99."

▲ Follow-up — "Remove `=99` from the route. Now?"

`id = 5` — nothing binds, so the C# default kicks in. **Route defaults win over method defaults**, because they run first.

Q7. Catch-all and the order of `MapControllerRoute`

```
app.MapControllerRoute("catchall", "{*url}",
    defaults: new { controller = "Error", action = "NotFound" });

app.MapControllerRoute("default", "{controller=Home}/{action=Index}/{id?}");
```

```
GET /Products/Index
```

✓ What happens

Every request goes to Error/NotFound – the whole site is broken

The rule

"`{*url}`" is a catch-all: it matches everything, including the empty string. Registered first, it shadows every route below it. Conventional routes are evaluated top-down."

▲ Follow-up — "So where does a catch-all belong?"

Last, always. And in Core the better tool is

`app.UseStatusCodePagesWithReExecute("/Error/{0}")` or exception-handling middleware — a catch-all *route* also swallows requests you'd rather see fail honestly.

In **MVC 5** the same rule applies to `RouteConfig.RegisterRoutes` — and it's why the generated template always ends with the `Default` route and puts `routes.IgnoreRoute("{resource}.axd/{*pathInfo}")` *first*.

Q8. Every public method is an action

```
public class AccountController : Controller
{
    public IActionResult Index() => View();

    public string BuildConnectionString(string server, string user, string
pwd)
        => $"Server={server};User={user};Password={pwd}";
}
```

GET /Account/BuildConnectionString?server=db1&user=sa&pwd=secret

✓ What happens

200 OK
Server=db1;User=sa;Password=secret

The rule

"Every PUBLIC method on a controller is an action by default and is routable. A helper method you forgot to make private is a public endpoint."

▲ The follow-ups

Follow-up 1 — "How do you stop it?"

Make it **private** / **protected** (best), or mark it **[NonAction]** if it genuinely must stay public.

Follow-up 2 — "Why does a **string** return type still work?"

Core wraps a non- **IActionResult** return value automatically — a **string** becomes a **ContentResult** with **text/plain**. A **void** action becomes an **EmptyResult** (200, empty body).

Follow-up 3 — "Is this a real vulnerability?"

Yes, and it's a standard finding in .NET security reviews. Combine it with the fact that `[Authorize]` on the *controller* would still protect it, and you've given the complete answer: **defence in depth — make it private AND authorize the controller.**

B — Model binding

Q9. Same key in the form, the route and the query string

```
// route: {controller}/{action}/{id?}
public IActionResult Save(int id) => Content($"id = {id}");
```

```
POST /Home/Save/1?id=2
Content-Type: application/x-www-form-urlencoded
```

```
id=3
```

✓ What happens

```
id = 3
```

The rule

"The default value providers are tried in a fixed order: FORM → ROUTE → QUERY STRING. The first one holding the key wins."

Form data	← highest priority (3)
Route values	← (1)
Query string	← lowest priority (2)

▲ Follow-up — "How do you force a specific source?"

```
public IActionResult Save([FromQuery] int id) => Content($"id = {id}"); // → 2
public IActionResult Save([FromRoute] int id) => Content($"id = {id}"); // → 1
public IActionResult Save([FromForm] int id) => Content($"id = {id}"); // → 3
```

`[FromQuery]` , `[FromRoute]` , `[FromForm]` , `[FromHeader]` , `[FromBody]` , `[FromServices]` . **Being explicit is the right habit** — implicit source precedence is exactly the kind of thing that changes behaviour when someone adds a query parameter months later.

Q10. `[FromBody]` on two parameters

```
[HttpPost]
public IActionResult Save([FromBody] Product p, [FromBody] int quantity)
=> Ok();
```

✓ What happens

- ▲ `InvalidOperationException` at startup / first request:
Action has more than one parameter bound from the request body.

The rule

"The request body is a forward-only stream that can be read **ONCE**. At most one parameter may be `[FromBody]` ."

▲ Follow-up — "So how do you send two things?"

Wrap them in one DTO — which is the design the framework is pushing you toward:

```
public record SaveRequest(Product Product, int Quantity);

[HttpPost]
public IActionResult Save([FromBody] SaveRequest req) => Ok();
```

Q11. Complex type from a form vs from JSON

```
public class Product { public int Id { get; set; } public string Name { get; set; } }

public class ProductsController : Controller // ← MVC controller, no [ApiController]
{
    [HttpPost]
    public IActionResult Save(Product p) => Content($"{p.Id}/{p.Name}");
}
```

Request A: POST with form fields Id=1&Name=Pen
Request B: POST with JSON body {"Id":1,"Name":"Pen"}

✓ What happens

A → 1/Pen
B → 0/ ← everything is default!

The rule

"In an MVC controller, a complex type binds from FORM VALUES by default, never from the JSON body. You must add `[FromBody]` to read JSON."

▲ Follow-up — "And in an API controller?"

```
[ApiController]                                // ← this changes the default
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    [HttpPost]
    public IActionResult Save(Product p) => Ok();    // now binds from the
JSON body
}
```

[ApiController] flips the default inference: complex types come from the body, simple types from the route/query. That one attribute is behind a large share of "why is my model empty?" questions.

In MVC 5 there are two entirely separate stacks — `System.Web.Mvc` (form-first) and `System.Web.Http` / Web API (`[FromBody]` , body-first), with **different Controller base classes and different filter types**. Core unified them, and that unification is a great thing to mention.

Q12. The name has to match — including the prefix

```
public class Order { public Customer Customer { get; set; } }
public class Customer { public string Name { get; set; } }

[HttpPost]
public IActionResult Save(Order order) => Content(order.Customer?.Name ??
"null");
```

Form A: Name=Ann
Form B: Customer.Name=Ann
Form C: order.Customer.Name=Ann

✓ What happens

```
A → null
B → Ann
C → Ann
```

The rule

"Binding is by NAME PATH, not by shape. Nested properties need the dotted path. The top-level parameter name is an OPTIONAL prefix — the binder tries `order.Customer.Name` first, then falls back to `Customer.Name`."

▲ Follow-up — "How do I force the prefix?"

```
public IActionResult Save([Bind(Prefix = "o")] Order order) { } // now
expects o.Customer.Name
```

Needed when one form posts two objects of the same type (`billing.` and `shipping.`). And in Razor, `@Html.EditorFor(m => m.Customer.Name)` generates `name="Customer.Name"` for free — which is the real reason the helpers exist.

Q13. Binding a collection

```
[HttpPost]
public IActionResult Save(List<Product> items) => Content($"count =
{items?.Count}");
```

```
Form A: items[0].Name=Pen&items[1].Name=Ink
Form B: items[0].Name=Pen&items[2].Name=Ink
Form C: items=Pen&items=Ink
```

✓ What happens

```
A → count = 2
B → count = 1    ← ▲ index 2 is silently dropped
C → count = 0
```

The rule

"Collection indices must be **CONSECUTIVE** from zero. The binder stops at the first gap. And a repeated simple key only binds to a collection of *simple* types, not complex ones."

▲ Follow-up — "How do you handle deletes in a dynamic form then?"

```
<input type="hidden" name="items.Index" value="7" />
<input type="text"   name="items[7].Name" />
```

The **.Index hidden field** lets you use non-consecutive keys — the standard trick for "add/remove row" forms where the user deleted row 1. Knowing this is a strong sign of real MVC form experience rather than tutorial experience.

Q14. Binding failure does not stop the action

```
[HttpPost]
public IActionResult Save(int quantity)
{
    return Content($"quantity = {quantity}, valid = {ModelState.IsValid}");
}
```

```
POST  quantity=abc
```

✓ What happens

```
quantity = 0, valid = False
```

The rule

"A binding failure sets the parameter to its default and records a ModelState error — it does NOT prevent the action from running. If you don't check `ModelState.IsValid`, you process garbage."

This is the single most consequential MVC trap on this page, because nothing crashes and nothing logs.

▲ Follow-up — "Make it impossible to forget."

```
// Option 1 – [ApiController] auto-returns 400 before the action runs
[ApiController] public class OrdersController : ControllerBase { }

// Option 2 – a global filter for MVC controllers
public class ValidateModelAttribute : ActionFilterAttribute
{
    public override void OnActionExecuting(ActionExecutingContext c)
    {
        if (!c.ModelState.IsValid) c.Result = new
        BadRequestObjectResult(c.ModelState);
    }
}

builder.Services.AddControllers(o =>
o.Filters.Add<ValidateModelAttribute>());
```

"I don't rely on every developer remembering `if (!ModelState.IsValid)` — I make it a filter." That's the answer that sounds like 4 years of experience.

Q15. Over-posting / mass assignment

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    public bool IsAdmin { get; set; }           // ← not on the form
}

[HttpPost]
public IActionResult Register(User user) { db.Add(user); db.SaveChanges();
return Ok(); }
```

POST Name=Ann&IsAdmin=true

✓ What happens

A user is created with IsAdmin = true. ♦

The rule

"The binder fills EVERY public settable property it finds a matching key for — including ones your form never rendered. Attackers add the extra field by hand."

▲ Follow-up — "Fixes, best first."

```
// 1. BEST – bind to a DTO that physically cannot carry the field
public record RegisterDto(string Name);

// 2. Whitelist
public IActionResult Register([Bind("Name")] User user) { }

// 3. Blacklist – weakest; a new property added later is unprotected by default
public IActionResult Register([Bind(Exclude = "IsAdmin")] User user) { }
```

Say **"never bind directly to your EF entity"** out loud. It answers the security question and the architecture question in one sentence.

Q16. Unchecked checkbox

```
public class Settings { public bool SendEmail { get; set; } }
```

```
<input type="checkbox" name="SendEmail" value="true" />
```

✓ What happens

```
Checked    → SendEmail = true
Unchecked → the browser posts NOTHING for that field → SendEmail =
false
```

...which *looks* right, until you realise you can never tell "unchecked" from "field absent" — so a partial update wipes the setting.

The rule

"An unchecked checkbox is not submitted at all by the browser. MVC handles it by rendering a HIDDEN companion input with `value=\"false\"`."

```
<!-- what @Html.CheckBoxFor(m => m.SendEmail) actually renders -->
<input type="checkbox" name="SendEmail" value="true" />
<input type="hidden" name="SendEmail" value="false" />
```

With both present, the binder takes the **first** value for `true` and falls back to the hidden `false`. **This is why you use `CheckBoxFor` instead of hand-writing the input** — a genuinely good "why do the helpers exist?" answer.

Q17. [FromServices] vs constructor injection

```
public IActionResult Report([FromServices] IReportService svc) =>
    Content(svc.Run());
```

✓ What happens

- ✓ Works – the service is resolved from DI per request, not bound from the request

The rule

" **[FromServices]** resolves the parameter from the DI container instead of the request. Use it when only **ONE** action needs a heavy dependency, so the controller's constructor doesn't resolve it on every request."

▲ Follow-up – "Downside?"

It hides the dependency from the constructor, which makes unit testing and dependency review harder. Default to constructor injection; reach for **[FromServices]** only for the expensive-and-rarely-used case.

C — Validation & ModelState

Q18. **[Required]** on a non-nullable **int**

```
public class Order
{
    [Required] public int Quantity { get; set; }
}
```

POST Quantity=0

✓ What happens

ModelState.IsValid == true ← 0 passes [Required]

The rule

" **[Required]** only checks for NULL. A non-nullable **int** can never be null — **0** is a perfectly good value — so **[Required]** on it can only ever catch a *missing field*, never a zero."

▲ Follow-up — "So how do you require a real quantity?"

```
[Range(1, int.MaxValue, ErrorMessage = "Quantity must be at least 1")]
public int Quantity { get; set; }

// or, to distinguish 'not supplied' from 'zero':
[Required] public int? Quantity { get; set; }
```

Note the second form: **making it nullable is what gives [Required] something to do**. Same trap shows up with **DateTime** (**01/01/0001** passes) and **bool** (**false** passes).

Q19. Adding an error after checking **IsValid**

```
[HttpPost]
public IActionResult Save(Product p)
{
    if (ModelState.IsValid)
    {
        if (db.Exists(p.Name))
            ModelState.AddModelError("Name", "Already exists");

        db.Add(p); // ← runs anyway
        return RedirectToAction("Index");
    }
    return View(p);
}
```

✓ What happens

The duplicate is saved. The error message is added and then thrown away by the redirect.

The rule

"**ModelState.IsValid** is evaluated at the moment you read it — it is not a latch. Adding an error afterwards changes **IsValid** to **false**, but your code has already committed to the success path."

▲ Follow-up — "Correct shape?"

```
if (!ModelState.IsValid) return View(p);

if (db.Exists(p.Name))
{
    ModelState.AddModelError("Name", "Already exists");
    return View(p);           // ← re-check by returning, not
    by re-reading IsValid
}

db.Add(p);
return RedirectToAction("Index");
```

Guard clauses, one return per failure. Also mention the `""` key:

`AddModelError("", "...")` produces a *model-level* error shown by `@Html.ValidationSummary()` rather than next to a field.

Q20. `[ApiController]` returns 400 before your action

```
[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    [HttpPost]
    public IActionResult Save(Order o)
    {
        Console.WriteLine("action ran");
        if (!ModelState.IsValid) return BadRequest();
        return Ok();
    }
}
```

with `Order.Name` marked `[Required]`, posting `{}`.

✓ What happens

400 Bad Request with an RFC 7807 ProblemDetails body
"action ran" is NEVER printed

The rule

"`[ApiController]` installs an automatic model-validation filter that short-circuits with a 400 *before* the action executes. Your `ModelState.IsValid` check inside the action is dead code."

▲ Follow-up — "How would you customise or disable that?"

```
builder.Services.Configure<ApiBehaviorOptions>(o =>
{
    o.SuppressModelStateInvalidFilter = true;           // turn it
    off
    // or shape the response:
    o.InvalidModelStateResponseFactory = ctx => new
    BadRequestObjectResult(
        new { errors = ctx.ModelState.Where(e => e.Value.Errors.Any()) });
});
```

Q21. Client-side validation without the script

```
public class Product { [Required] public string Name { get; set; } }
```

```
@Html.TextBoxFor(m => m.Name)
@Html.ValidationMessageFor(m => m.Name)
<!-- jquery.validate.unobtrusive.js NOT referenced -->
```

✓ What happens

The form submits with an empty Name.
The SERVER still rejects it – ModelState.IsValid is false.

The rule

" **[Required]** produces **data-val-*** attributes; the *jQuery unobtrusive validation* script turns those into browser-side blocking. Server-side validation runs regardless — client validation is a UX feature, never a security control."

▲ Follow-up — "Name a validation that only exists on the server."

[Remote] needs a round trip by definition, and any custom **IDataableObject** / **ValidationAttribute** without a matching client adapter runs **server-only**. Also: *"I never trust client validation, because anyone can POST with curl."*

Q22. Validation doesn't recurse into an unbound child

```
public class Order
{
    [Required] public string Ref { get; set; }
    public Customer Customer { get; set; }      // null when nothing
    posted for it
}
public class Customer { [Required] public string Name { get; set; } }
```

```
POST  Ref=A1
```

✓ What happens

```
ModelState.IsValid == true
```

The rule

"Validation walks the object graph that binding actually produced. **Customer** is null, so there is nothing to validate on it — a **[Required]** inside a null child never fires."

▲ Follow-up — "Make it fail."

```
[Required] public Customer Customer { get; set; }    // require the child
itself
```

And mention **MvcOptions.MaxValidationDepth** (default 32) — deeply nested or cyclic graphs are cut off, which is another way validation quietly stops.

D — ViewBag, ViewData & TempData

Q23. ViewBag is **dynamic** — typos are silent

```
public IActionResult Index()
{
    ViewBag.UserName = "Ann";
    return View();
}
```

```
<p>Hello @ViewBag.UserName</p>           @* typo *@
```

✓ What happens

```
<p>Hello </p>
```

No exception. No warning. No compile error.

The rule

"**ViewBag** is **dynamic** over a dictionary. A missing key returns **null**, and Razor renders **null** as an empty string. Every typo is a silent blank."

▲ The follow-ups

Follow-up 1 — "When DOES it throw?"

```
@ViewBag.UserName.ToUpper()    @* ▲ RuntimeBinderException: 'object'
does not contain a definition for 'ToUpper' *@
@ViewBag.Items.Count           @* ▲ same, if Items was never set *@
```

Reading is safe; calling a **member on the null** is not — and it's a *runtime binder* exception, which is worth naming precisely.

Follow-up 2 — "What do you use instead?"

A strongly typed view model. `@Model.UserName` is compile-checked, refactor-safe, IntelliSense-friendly and testable. *"ViewBag is fine for a page title; anything the view depends on belongs on the model."*

Q24. ViewBag and ViewData are the same storage

```
ViewData["Message"] = "from ViewData";
Console.WriteLine(ViewBag.Message);

ViewBag.Title = "from ViewBag";
Console.WriteLine(ViewData["Title"]);
```

✓ What happens

```
from ViewData
from ViewBag
```

The rule

"**ViewBag** is nothing but a **dynamic** wrapper over **ViewData**. One dictionary, two syntaxes — they see each other's values."

▲ Follow-up — "Then why do both exist, and where does the difference bite?"

<code>@ViewData["Count"] + 1</code>	<code>@*</code> renders "5 + 1" – Razor ends the expression at <code>]</code> <code>*@</code>
<code>@(ViewData["Count"] + 1)</code>	<code>@*</code> ✗ COMPILE ERROR: operator '+' cannot be applied to 'object' and 'int' <code>*@</code>
<code>@((int)ViewData["Count"] + 1)</code>	<code>@*</code> ✓ 6 <code>*@</code>
<code>@(ViewBag.Count + 1)</code>	<code>@*</code> ✓ 6 – dynamic dispatch resolves it at runtime <code>*@</code>

ViewData is **object** and needs a cast; **ViewBag** is **dynamic** and doesn't.

The trade is compile-time safety for convenience — and **ViewBag** moves the failure from compile time to a customer's browser.

Q25. TempData survives exactly one redirect

```
public IActionResult Save()
{
    TempData["Msg"] = "Saved!";
    return RedirectToAction("Index");
}

public IActionResult Index()
{
    var a = TempData["Msg"];
    var b = TempData["Msg"];
    return Content($"a={a}, b={b}");
}
```

✓ What happens

```
a=Saved!, b=Saved!
```

...but on the **next** request, `TempData["Msg"]` is `null`.

The rule

"Reading **TempData** MARKS the entry for deletion at the end of the current request — it doesn't remove it immediately. So repeated reads in the same request all succeed; the value is gone from the request after."

▲ The follow-ups

Follow-up 1 — "**Peek** and **Keep**?"

```
TempData.Peek("Msg");    // read WITHOUT marking for deletion → survives
                           another request
TempData["Msg"];          // read AND mark
TempData.Keep("Msg");     // un-mark after reading → survives another
                           request
TempData.Keep();          // un-mark everything
```

Follow-up 2 — "Where is it stored?"

Core: `CookieTempDataProvider` by default (encrypted cookie) — or `SessionStateTempDataProvider` if you register it.

** MVC 5: **always** Session**, so TempData silently breaks if session state is disabled.

Follow-up 3 — "Store a `List<Product>` in TempData. What happens in Core?"

▲ `InvalidOperationException`: The '`CookieTempDataProvider`' cannot serialize an object of type '`Product`'

The cookie provider only serializes simple types (`string` , `int` , `bool` , `DateTime` , `Guid` , and collections of those). Store an **id**, not an object graph — a cookie is also size-limited to ~4KB.

Q26. ViewBag does not survive a redirect

```
public IActionResult Save()
{
    ViewBag.Msg = "Saved!";
    return RedirectToAction("Index");
}

public IActionResult Index() => Content($"Msg = {ViewBag.Msg ?? "null"}");
```

✓ What happens

```
Msg = null
```

The rule

"A redirect is a 302 — the browser makes a brand-new HTTP request, which gets a brand-new controller instance and a brand-new ViewData . ViewBag lives for ONE request; TempData is the one designed to cross a redirect."

▲ Follow-up — "One line comparing all four."

	Lifetime	Typed?	Survives redirect
ViewBag	current request	x dynamic	x
ViewData	current request	x object	x
TempData	until read (≈ next request)	x object	✓
Session	until it expires	x object	✓
View model	current request	✓ compile-time	x

Q27. Setting TempData without ever reading it

```
public IActionResult A() { TempData["X"] = 1; return  
RedirectToAction("B"); }  
public IActionResult B() { return RedirectToAction("C"); }           //  
doesn't read X  
public IActionResult C() => Content($"X = {TempData["X"]} ?? "null"");
```

✓ What happens

```
X = 1
```


The rule

"TempData is deleted on READ, not on age. An unread value is automatically carried forward to the next request, and the next, until something reads it."

Most people answer "null" here because they've memorised "TempData lasts one request". The precise rule is **"until read"**.

Q28. TempData in an AJAX call

```
public IActionResult Save()
{
    TempData["Msg"] = "Saved!";
    return Json(new { ok = true });
}
```

The page then does a client-side `location.reload()`, and the reloaded page shows the message.

✓ What happens

The message appears – but only because the reload was a real HTTP request. If the page had updated via JS without reloading, TempData would still be sitting there, and would surface on some LATER, unrelated navigation.

The rule

"TempData is keyed to the browser session, not to a page. Mixing it with AJAX leaks messages onto unrelated pages later."

▲ Follow-up – "So what do you use for an AJAX flash message?"

Return the message **in the JSON response** and let the client render it. TempData exists for the **POST-Redirect-GET** pattern specifically — outside PRG it's the wrong tool.

Q29. Session in Core without `UseSession`

```
builder.Services.AddSession();
var app = builder.Build();
app.UseRouting();
app.MapControllers();           // ← UseSession() is missing

// in a controller:
HttpContext.Session.SetString("k", "v");
```

✓ What happens

▲ `InvalidOperationException: Session has not been configured for this application or request.`

The rule

"In Core, `AddSession()` registers the services and `UseSession()` inserts the middleware. Both are required, and `UseSession()` must come BEFORE the endpoint that uses it."

▲ Follow-up — "General shape of that rule?"

It's the whole Core mental model: **`AddX()` in `Services` = registration; `UseX()` in the pipeline = execution, in order.** The same pair applies to authentication, authorization, CORS, response caching and session. Nearly every "why is this null in Core?" question is a missing `UseX()` or one in the wrong place.

E — Action results & redirects

Q30. Code after `return RedirectToAction`

```
public IActionResult Save()
{
    db.Save();
    return RedirectToAction("Index");
    Log("saved");                // ← ?
}
```

✓ What happens

It compiles, with warning CS0162: "Unreachable code detected" `Log()` never runs.

The rule

"`RedirectToAction` doesn't redirect — it RETURNS an `IActionResult` describing a redirect. The framework executes it after the action returns. So `return` is a normal return, and anything after it is unreachable."

In MVC 5 — the trap that actually bites

```
public ActionResult Save()
{
    Response.Redirect("/Home/Index");    // writes the 302 header
    immediately
    Log("saved");                        // ▲ THIS RUNS
    db.DeleteEverything();                // ▲ SO DOES THIS
    return View();
}
```

`Response.Redirect` does NOT stop execution. It sets the header and carries on — the classic "I redirected but the code kept running" bug.

`Response.Redirect(url, endResponse: true)` throws a `ThreadAbortException` to stop it, which is itself a well-known anti-pattern.

The MVC answer: always `return RedirectToAction(...)` , never `Response.Redirect` .

Q31. Returning `View()` from a POST

```
[HttpPost]
public IActionResult Save(Product p)
{
    if (!ModelState.IsValid) return View(p);
    db.Add(p);
    return View("Index");           // ← instead of
    RedirectToAction
}
```

✓ What happens

The Index view renders, but the URL bar still says `POST /Products/Save`
→ pressing F5 re-submits the form and creates a DUPLICATE record
→ the browser shows a "Confirm Form Resubmission" dialog

The rule

" `return View()` renders in the SAME request, so the URL doesn't change and the POST stays in browser history. `RedirectToAction` issues a 302 so the browser does a fresh GET."

▲ Follow-up — "Name the pattern."

POST-Redirect-GET (PRG). POST → validate → save → **redirect** → GET renders. Returning `View()` on the *failure* path is correct (you want the posted values back on screen); returning `View()` on the *success* path is the bug.

Q32. `return Json(data)` on a GET

This one is a pure MVC 5 question

```
public ActionResult GetUsers()
{
    return Json(db.Users.ToList()); // MVC 5
}
```

✓ What happens (MVC 5)

▲ `InvalidOperationException`:
This request has been blocked because sensitive information could be disclosed to third party web sites when this is used in a GET request.
To allow GET requests, set `JsonRequestBehavior` to `AllowGet`.

The fix and the reason

```
return Json(db.Users.ToList(), JsonRequestBehavior.AllowGet);
```

The rule

"MVC 5 blocks JSON on GET by default to mitigate JSON hijacking — a cross-site `<script src>` could read a JSON array returned to a GET while the user was authenticated."

▲ Follow-up — "And in Core?"

`JsonRequestBehavior` doesn't exist — `return Json(data)` just works. The underlying attack was closed by browsers (Array constructor poisoning was fixed) and by SameSite cookies. Knowing *why it was removed* rather than just *that it was removed* is the answer they want.

Q33. `View()` can't find the view

```
public class ProductsController : Controller
{
    public IActionResult Summary() => View();    // Views/Products/
    Summary.cshtml doesn't exist
}
```

✓ What happens

▲ `InvalidOperationException`: The view 'Summary' was not found. The following locations were searched:
/Views/Products/Summary.cshtml
/Views/Shared/Summary.cshtml

The rule

"`View()` with no name uses the ACTION name, and searches `/Views/{Controller}/` then `/Views/Shared/`. The exception helpfully lists every path it tried."

▲ Follow-up — "What if the action name was changed with `[ActionName]`?"

```
[ActionName("Report")]
public IActionResult Summary() => View();    // looks for Report.cshtml,
not Summary.cshtml
```

It searches for the *action* name, not the method name. A genuinely nasty five-minute debugging session the first time it happens.

Q34. `View()` vs `PartialView()`

```
public IActionResult Row() => View();
public IActionResult RowPartial() => PartialView("Row");
```

✓ What happens

`View()` → renders `Row.cshtml` INSIDE `_Layout` (full `<html>` page)
`PartialView()` → renders `Row.cshtml` ALONE (just the fragment)

The rule

"**`ViewResult`** applies **`_ViewStart.cshtml`**, which is what sets **`Layout`**.
`PartialViewResult` skips **`_ViewStart`** entirely, so no layout."

▲ Follow-up — "So an AJAX call returning `View()` gives you...?"

A full HTML page — `<html>`, `<head>`, navbar and all — injected into your `<div>`. It's the standard "why does my modal contain the whole site?" bug. Use `PartialView()`, or set `Layout = null;` in the view.

Q35. Returning a `Task` from an action without `await`

```
public IActionResult Save()
{
    db.SaveChangesAsync();           // ← not awaited
    return RedirectToAction("Index");
}
```

✓ What happens

The redirect is issued immediately.
The save MAY complete... or may be aborted when the request ends and the `DbContext` is disposed.
Often: ▲ `ObjectDisposedException` on a background thread, and the record is silently missing.

The rule

"An un-awaited **Task** is fire-and-forget. The request completes, the DI scope is disposed, and the scoped **DbContext** disappears out from under the running task."

▲ Follow-up — "Fix, and the compiler hint you ignored."

```
public async Task<IActionResult> Save()
{
    await db.SaveChangesAsync();
    return RedirectToAction("Index");
}
```

The compiler emits **CS4014** — *"Because this call is not awaited, execution of the current method continues before the call is completed"*. Treat warnings as errors and this class of bug disappears. For genuine background work, use **IHostedService** / a queue, **not** a fire-and-forget task in a controller.

F — Filters & execution order

Q36. Global, controller and action filters together

```
builder.Services.AddControllers(o => o.Filters.Add(new
LogFilter("GLOBAL")));

[LogFilter("CONTROLLER")]
public class HomeController : Controller
{
    [LogFilter("ACTION")]
    public IActionResult Index() { Console.WriteLine("ACTION METHOD");
return View(); }
}

public class LogFilter : Attribute, IActionFilter
{
    private readonly string _n;
    public LogFilter(string n) => _n = n;
    public void OnActionExecuting(ActionExecutingContext c) =>
Console.WriteLine($"{_n} executing");
    public void OnActionExecuted(ActionExecutedContext c) =>
Console.WriteLine($"{_n} executed");
}
```

✓ Output

```
GLOBAL executing
CONTROLLER executing
ACTION executing
ACTION METHOD
ACTION executed
CONTROLLER executed
GLOBAL executed
```

The rule

"Filters nest like Russian dolls. On the way IN it's Global → Controller → Action. On the way OUT it's the exact reverse: Action → Controller → Global."

▲ Follow-up — "How do you override that order?"

```
[LogFilter("ACTION", Order = -1)]    // lower Order runs FIRST, before  
Global
```

Order beats scope. Filters are sorted by **Order** first (default 0), and scope only breaks ties. Mentioning that the default is 0 for all three scopes — which is *why* scope decides normally — is the complete answer.

Q37. The full Core filter pipeline

Which of these runs FIRST — and where does model binding fit?
Authorization filter · Resource filter · Action filter · Exception filter
· Result filter

✓ The order

1. Authorization filters ← reject unauthenticated requests as early as possible
2. Resource filters (before) ← caching lives here; runs BEFORE model binding
3. — MODEL BINDING —
4. Action filters (before)
5. == THE ACTION METHOD ==
6. Action filters (after)
7. Result filters (before)
8. — RESULT EXECUTION (the view renders here) —
9. Result filters (after)
10. Resource filters (after)

Exception filters wrap steps 3–6.

The rule

"Authorization is first because there's no point binding a model for a request you're going to reject. Resource filters bracket model binding, which is why output caching is a resource filter."

▲ Follow-up — "Where would you put each of these?"

Need	Filter type
Check a permission	Authorization
Short-circuit with a cached response	Resource
Log/audit arguments, start a stopwatch	Action
Turn an exception into a friendly view	Exception
Add a header to every response	Result
Anything that must run for non-MVC requests too (static files)	Middleware, not a filter

That last row is the one that separates people who've read the docs from people who've shipped.

Q38. Short-circuiting a filter

```
public class CacheFilter : Attribute, IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext c)
    {
        c.Result = new ContentResult { Content = "from cache" };    // ←
        set Result
        Console.WriteLine("executing");
    }
    public void OnActionExecuted(ActionExecutedContext c) =>
    Console.WriteLine("executed");
}

[CacheFilter]
public IActionResult Index() { Console.WriteLine("ACTION"); return
View(); }
```

✓ Output

executing

Response body: `from cache`. **ACTION** never prints — and neither does `executed`.

The rule

"Setting `context.Result` in `OnActionExecuting` short-circuits the pipeline: the action is skipped, and so is the `OnActionExecuted` half of THIS filter. Result filters and the result itself still run."

The missing `executed` is the part people get wrong — they assume the "after" half always pairs with the "before" half. It doesn't.

▲ Follow-up — "And in an async filter?"

```
public async Task OnActionExecutionAsync(ActionExecutingContext c,
    ActionExecutionDelegate next)
{
    if (cached) { c.Result = new ContentResult { Content = "hit" };
    return; } // don't call next()
    var executed = await
    next();                                     // action runs here
}
```

Not calling `next()` is how you short-circuit an async filter — same idea as middleware. And you must never call `next()` twice.

Q39. What exception filters do not catch

```
public class MyExceptionHandler : IExceptionHandler
{
    public void OnException(ExceptionContext c)
    {
        Console.WriteLine("caught: " + c.Exception.Message);
        c.Result = new ContentResult { Content = "handled" };
        c.ExceptionHandled = true;
    }
}
```

The exception is thrown **inside the Razor view** (`@Model.Customer.Name` where `Customer` is null).

✓ What happens

The filter does NOT run. The user gets a 500 / the developer exception page.

The rule

"Exception filters cover controller creation, model binding, action filters and the action method. They do NOT cover result execution — so an exception while the VIEW renders escapes them entirely."

▲ Follow-up — "So what does catch it?"

Exception-handling middleware — `app.UseExceptionHandler("/Error")` — because middleware wraps the whole pipeline including result execution. The rule of thumb: *"filters for MVC-shaped errors, middleware for everything."*

Q40. `[Authorize]` on the controller, `[AllowAnonymous]` on the action

```
[Authorize]
public class AccountController : Controller
{
    [AllowAnonymous] public IActionResult Login() => View();
                    public IActionResult Profile() => View();
}
```

✓ What happens

/Account/Login → 200 (anonymous allowed)
/Account/Profile → 302 to the login page (or 401/403 for an API)

The rule

" `[AllowAnonymous]` always wins — it short-circuits authorization regardless of scope, including a globally registered `AuthorizeFilter` ."

▲ Follow-up — "Why does that make `[AllowAnonymous]` dangerous?"

Because it overrides even a global authorization policy. One `[AllowAnonymous]` left on a controller during debugging opens the whole controller, and no global setting will save you. It's a standard code-review grep.

Q41. `[ValidateAntiForgeryToken]` without the token

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Save(Product p) => Ok();
```

The form is hand-written HTML with no `@Html.AntiForgeryToken()` .

✓ What happens

```
400 Bad Request
(AntiforgeryValidationException: The required antiforgery cookie "... " is
not present)
```

The rule

"The token is two halves — a cookie AND a form field — that must match. It stops CSRF: an attacker's site can make your browser send the cookie, but can't read or forge the form field."

▲ Follow-up — "How do you send it from AJAX?"

```
services.AddAntiforgery(o => o.HeaderName = "X-CSRF-TOKEN");
```

```
fetch(url, { method: 'POST', headers: { 'X-CSRF-TOKEN': token } })
```

And in Core, `<form asp-action="Save">` injects the hidden token **automatically** — which is why hand-written `<form>` tags are the usual cause of this 400.

Q42. `OnActionExecuted` when the action throws

```
public void OnActionExecuted(ActionExecutedContext c)
    => Console.WriteLine($"executed, exception = {c.Exception?.Message ??
"none"}");

public IActionResult Index() => throw new Exception("boom");
```

✓ Output

```
executed, exception = boom
```

...and then the exception continues to propagate.

The rule

"**`OnActionExecuted`** still runs when the action throws — the exception is handed to you on `context.Exception` rather than being thrown past you."

▲ Follow-up — "How do you swallow it there?"

```
c.ExceptionHandled = true;
c.Result = new ContentResult { Content = "recovered" };
```

Setting `ExceptionHandled = true` **without** setting `Result` gives the client an empty 200 — a silent failure that's arguably worse than the 500. Always set both.

G — Razor view traps

Q43. ▲ The model changed, but the textbox didn't

```
[HttpPost]
public IActionResult Edit(Product p)
{
    p.Name = "CHANGED";
    return View(p);
}
```

```
@model Product
@Html.TextBoxFor(m => m.Name)
<p>Model value: @Model.Name</p>
```

The user posted `Name=original`.

✓ What renders

```
<input type="text" name="Name" value="original" />    ← ▲ NOT "CHANGED"
<p>Model value: CHANGED</p>
```

The rule (this is the most-asked MVC view question there is)

"HTML helpers read their value from ModelState FIRST, then ViewData, and only then from the model. The posted value is sitting in ModelState, so `TextBoxFor` shows the POSTED value and ignores your change."

Lookup order:

1. ModelState ← the posted value lives here
2. ViewData / ViewBag
3. The model ← only reached if the first two have nothing

Why the framework does this: when validation fails you *want* the user's original input redisplayed next to the error message — not the sanitised version your code produced. It's correct behaviour that looks like a bug.

▲ The follow-ups

Follow-up 1 — "Force the new value to show."

```
ModelState.Remove(nameof(Product.Name)); // ✓ surgical – clears just
that key
// or
ModelState.Clear(); //
▲ blunt – also destroys all validation errors
```

Follow-up 2 — "Why does `@Model.Name` show the new value?"

Because raw Razor output reads the model **directly** — it doesn't go through the helper's ModelState lookup at all. The two lines disagreeing on the same page is the tell.

Follow-up 3 — "When have you hit this?"

Say it honestly: server-side normalisation (trimming, uppercasing a code, formatting a phone number) that "didn't take" on redisplay. It's the classic real-world encounter, and naming it makes the answer sound lived rather than memorised.

Q44. Razor's `@` parsing

```
@{ int count = 5; }

<p>@count + 1</p>
<p>@(count + 1)</p>
<p>Email: user@domain.com</p>
<p>Cost: @@50</p>
```

✓ What renders

```
<p>5 + 1</p>
<p>6</p>
<p>Email: user@domain.com</p>
<p>Cost: @50</p>
```

The rule

"Razor ends an implicit expression at the first character that can't continue it — so `@count` stops before the space. Parentheses make it explicit. `@@` escapes a literal `@`, and Razor is smart enough to leave email addresses alone."

▲ Follow-up — "What about a method call with arguments?"

<code>@Model.Items.Count</code>	<code>@*</code> ✓ works — dots and property access
<code>continue the expression *@</code>	
<code>@Model.GetName("a", "b")</code>	<code>@*</code> ✓ works — parens are part of an implicit expression *
<code>@Model.Price.ToString("C")</code>	<code>@*</code> ✓ works *
<code>@if (x) { <p>@x</p> }</code>	<code>@*</code> ✓ code block *
<code>@:Plain text with @count</code>	<code>@*</code> ✓ <code>@:</code> forces the rest of the line to be literal output *

Q45. HTML encoding and `Html.Raw`

```
var model = new Product { Name = "<script>alert('xss')</script>" };
```

```
<p>@Model.Name</p>
<p>@Html.Raw(Model.Name)</p>
<p>@Html.DisplayFor(m => m.Name)</p>
```

✓ What renders

<code><p>&lt;script&gt;alert(&#x27;xss&#x27;)&lt;/script&gt;</p></code>	← safe, visible as text
<code><p><script>alert('xss')</script></p></code>	← ◆ EXECUTES
<code><p>&lt;script&gt;...&lt;/script&gt;</p></code>	← safe

The rule

"Razor HTML-encodes every `@expression` by default. `Html.Raw` is the **ONLY** way to opt out — and it is the standard XSS vector in .NET applications."

▲ Follow-up — "When is `Html.Raw` legitimate?"

Only for HTML **you generated**, or user HTML **after sanitising it** with a library like `HtmlSanitizer`. Never for raw user input. Add: "*`Html.Raw` in a pull request is an automatic review comment for me.*"

Q46. `Html.Partial` vs `Html.RenderPartial`

```
@Html.Partial("_Row")
@Html.RenderPartial("_Row")
@{ Html.RenderPartial("_Row"); }
```

✓ What happens

```
Line 1 → ✓ renders
Line 2 → ✗ COMPILE ERROR – cannot implicitly convert 'void' to 'object'
Line 3 → ✓ renders (and is faster)
```

The rule

"**Partial** RETURNS an `IHtmlString`, so it works in an `@` expression.
RenderPartial WRITES straight to the output stream and returns `void`, so it must be called from a code block."

`RenderPartial` avoids building an intermediate string — marginally faster, meaningfully so inside a big loop.

▲ Follow-up — "And in modern Core?"

<code><partial name="_Row" model="item" /></code>	@* tag helper – preferred
<code>*@</code>	
<code>@await Html.PartialAsync("_Row")</code>	@* async, avoids sync-over-
<code>async *@</code>	
<code>@await Component.InvokeAsync("Cart")</code>	@* view component – has its
<code>own logic + DI *@</code>	

`Html.Partial` is **flagged by an analyzer in Core** because it blocks on async rendering. The modern answer is the `<partial>` tag helper, and a **view component** when the fragment needs its own data-fetching logic.

Q47. `@section` inside a partial view

```
@* _Row.cshtml (a partial) *@
@section Scripts { <script>console.log('row');</script> }
```

✓ What happens

▲ `InvalidOperationException`: The layout page cannot be rendered after 'RenderBody' has been called. / the section is simply never rendered.

The rule

"Sections belong to the view/layout relationship. A partial has no layout, so `@section` inside one has nothing to render into — it's ignored or throws."

▲ Follow-up — "So how does a partial add a script?"

Three real options: put the script in the **parent view's `@section Scripts`**; use a **view component** with its own script include; or write to a shared collection (`ViewData` / a tag helper like `<script asp-append-version>`) that the layout renders. Say *"I'd avoid inline scripts in partials"* — it's the answer that shows you've maintained a codebase.

Q48. Layout and view execution order

```
@* _Layout.cshtml *@  
<title>@ViewBag.Title</title>  
{ ViewBag.FromLayout = "layout"; }  
@RenderBody()  
  
@* Index.cshtml *@  
{ ViewBag.Title = "Home"; }  
<p>@ViewBag.FromLayout</p>
```

✓ What renders

<title>Home</title>	← the view's value IS visible in the layout
	← the layout's value is NOT visible in the view

The rule

"The VIEW executes first, completely. Its output is then handed to the layout, which executes and calls `RenderBody()` to splice it in. So the view can set values for the layout, but never the other way round."

This surprises everyone, because the layout *looks* like the outer thing.

▲ Follow-up — "Where does `_ViewStart.cshtml` fit?"

`_ViewStart.cshtml` → the View → `_Layout.cshtml`

`_ViewStart` runs **before** the view (that's where `Layout = "_Layout";` is set), and it cascades down folders — a `_ViewStart` in `/Views/Admin/` overrides the one in `/Views/`. `_ViewImports.cshtml` is the sibling that handles `@using`, `@inject` and `@addTagHelper`.

Q49. `RenderSection` required by default

```
@* _Layout.cshtml *@  
@RenderSection("Scripts")  
  
@* Index.cshtml – has no @section Scripts *@
```

✓ What happens

▲ `InvalidOperationException`: The layout page '`_Layout.cshtml`' cannot find the section '`Scripts`' in the content page.

The rule

" `RenderSection(name)` is **REQUIRED** by default. Every view using this layout must define the section — one that doesn't, breaks."

```
@RenderSection("Scripts", required: false)           @* ✓ optional *@  
@await RenderSectionAsync("Scripts", required: false) @* ✓ Core, async  
*@
```

Q50. `DisplayNameFor` on an empty collection

```
@model IEnumerable<Product>  
  
<th>@Html.DisplayNameFor(model => model.First().Name)</th>  
  
@foreach (var p in Model) { <tr><td>@p.Name</td></tr> }
```

`Model` is an **empty** list.

✓ What happens

```
<th>Name</th>           ← ✓ renders fine, no exception
```

...even though `Model.First()` on an empty sequence would normally throw `InvalidOperationException`.

The rule

"**DisplayNameFor** never **EXECUTES** the lambda — it only parses the expression **TREE** to find the property and read its metadata. So **First()** is never called."

This is the line the MVC scaffolder generates for every list view, and almost nobody can explain why it doesn't crash.

▲ The follow-ups

Follow-up 1 — "Where does the name come from?"

```
[Display(Name = "Product name")] public string Name { get; set; }
```

From `[Display]`, falling back to the property name. The whole point of the metadata system.

Follow-up 2 — "So can I use `First()` in `DisplayFor` too?"

No. `DisplayFor` / `TextBoxFor` **compile and invoke** the expression to get the *value* — so `Model.First()` on an empty list throws there. `DisplayNameFor` reads metadata; everything else reads data. That distinction is the whole question.

Q51. `@Html.ActionLink` with a null route value

```
@Html.ActionLink("Edit", "Edit", new { id = Model.Id })
@Html.ActionLink("Edit", "Edit", new { id = (int?)null })
@Html.ActionLink("Edit", "Edit", "Products", new { id = 5 }, null)
```

✓ What renders

```
<a href="/Products/Edit/5">Edit</a>
<a href="/Products/Edit">Edit</a>      ← null values are simply
dropped
<a href="/Products/Edit/5">Edit</a>
```

The rule

"Null route values are omitted from the generated URL. And watch the overloads — the 4-argument form is **(text, action, routeValues, htmlAttributes)**, so passing a controller name there makes it a query string parameter instead."

```
@Html.ActionLink("Edit", "Edit", "Products")
@* ▲ 3-arg overload is (text, action, routeValues) → /Products/Edit?
Length=8 *@
```

That **?Length=8** is legendary: **"Products"** was treated as an **object of route values**, and **string** has a **Length** property. If you ever see **?Length=** in a URL, this is why.

▲ Follow-up — "What do you use in Core?"

```
<a asp-controller="Products" asp-action="Edit" asp-route-id="5">Edit</a>
```

Tag helpers — no overload ambiguity, real IntelliSense, and the markup stays HTML.

H — Controller lifecycle & state

Q52. Instance fields don't survive

```
public class CounterController : Controller
{
    private int _count = 0;

    public IActionResult Index()
    {
        _count++;
        return Content($"count = {_count}");
    }
}
```

Called three times.

✓ What happens

```
count = 1
count = 1
count = 1
```

The rule

"A controller is instantiated PER REQUEST and disposed at the end of it. Instance state never survives a request."

▲ Follow-up — "Make it count. Then tell me what breaks."

```
private static int _count = 0;    // now: 1, 2, 3
```

...and it's **shared across every user and every thread**, so `_count++` is a data race (see Q53). The correct answers are Session (per user), a distributed cache (per app), or the database (durable).

Q53. Static state is shared across all requests

```
public class CartController : Controller
{
    private static List<string> _items = new();

    public IActionResult Add(string item)
    {
        _items.Add(item);
        return Content(string.Join(",", _items));
    }
}
```

✓ What happens

Every user sees every other user's cart.
Under concurrent load: ▲ `IndexOutOfRangeException` / corrupted state / lost items

The rule

"Static fields live for the lifetime of the process and are shared by all concurrent requests. `List<T>` is not thread-safe, so concurrent `Add` calls can corrupt its internal array."

▲ Follow-up — "If you genuinely need process-wide shared state?"

`IMemoryCache` (built for it, with expiry and size limits), `ConcurrentDictionary` (thread-safe), or a **singleton service with proper locking**. And the scaling caveat interviewers wait for: *"none of that survives a restart or works across multiple instances behind a load balancer — that's what a distributed cache like Redis is for."*

Q54. `async void` action

```
public async void Save() // ← void, not Task
{
    await db.SaveChangesAsync();
}
```

✓ What happens

The response completes immediately with 200 and an empty body, before the save has finished. Any exception CRASHES THE PROCESS – it cannot be caught by exception middleware or a filter.

In MVC 5:

▲ `InvalidOperationException`: An asynchronous module or handler completed while an asynchronous operation was still pending.

The rule

"An `async void` method returns nothing the framework can await, so MVC treats the action as finished the moment it hits the first `await`. Exceptions have no `Task` to land in and are raised on the thread pool – which terminates the process."

Exactly the same rule as Q67 in [file 27], now with an HTTP-shaped consequence.

▲ Follow-up – "The one place `async void` is acceptable?"

Event handlers only, where the framework fixes the signature. An MVC action is never one. Always `public async Task<IActionResult>`.

Q55. `HttpContext` after `ConfigureAwait(false)`

Classic MVC 5

```
public async Task<ActionResult> Index()
{
    await SomeApiCallAsync().ConfigureAwait(false);
    var user = HttpContext.Current.User.Identity.Name;    // ← ?
    return View();
}
```

✓ What happens

▲ `NullReferenceException` — `HttpContext.Current` is null

The rule

"`HttpContext.Current` is stored on the ASP.NET `SynchronizationContext`. `ConfigureAwait(false)` tells the continuation not to resume on that context — so it resumes on a bare thread-pool thread where `HttpContext.Current` is null."

▲ Follow-up — "And in Core?"

`HttpContext.Current` doesn't exist. Core removed the ambient static entirely. You inject `IHttpContextAccessor` where you need it, and inside a controller you just use the `HttpContext` property — which is a field on the controller, not ambient state, so `ConfigureAwait(false)` can't break it.

The deeper point worth saying: "Core removed the `SynchronizationContext`, which is also why the classic `.Result` deadlock doesn't happen there." That ties this question to Q66 in file 27 and shows you understand one mechanism, not two facts.

I — DI & the middleware pipeline (Core)

Q56. Scoped inside Singleton — the captive dependency

```
builder.Services.AddScoped<IRepository, Repository>();
builder.Services.AddSingleton<ICacheService, CacheService>();

public class CacheService : ICacheService
{
    public CacheService(IRepository repo) { }    // singleton depending
    on scoped
}
```

✓ What happens

▲ `InvalidOperationException` at startup:
Cannot consume scoped service 'IRepository' from singleton 'ICacheService'.

The rule

"A singleton outlives every scope, so capturing a scoped service would keep the **FIRST** request's instance alive forever — a *captive dependency*. Core's scope validation catches it at startup in Development."

▲ The follow-ups

Follow-up 1 — "Why only in Development?"

`ValidateScopes` and `ValidateOnBuild` default to **true only in the Development environment**. In Production the app starts happily and you get a stale `DbContext` shared across all users — a genuinely dangerous silent failure. Turn it on explicitly:

```
csharp builder.Host.UseDefaultServiceProvider(o =>
{ o.ValidateScopes = true; o.ValidateOnBuild = true; });
```

Follow-up 2 — "Fix it."

```

public class CacheService : ICacheService
{
    private readonly IServiceScopeFactory _factory;
    public CacheService(IServiceScopeFactory factory) => _factory =
factory;

    public void Refresh()
    {
        using var scope = _factory.CreateScope();
        var repo =
scope.ServiceProvider.GetRequiredService<IRepository>();
    }
}

```

Inject **IServiceScopeFactory** and create a scope per unit of work. This is also exactly how you use a **DbContext** inside a background **IHostedService**.

Q57. Transient vs Scoped vs Singleton in one request

```

builder.Services.AddTransient<ITransient, Svc>();
builder.Services.AddScoped<IScoped, Svc>();
builder.Services.AddSingleton<ISingleton, Svc>();

public HomeController(ITransient t1, ITransient t2,
                    IScoped s1,    IScoped s2,
                    ISingleton g1, ISingleton g2) { }

```

✓ What happens

```

t1 != t2    ← two different instances, in the SAME request
s1 == s2    ← one instance per request
g1 == g2    ← one instance for the whole application

```

And across two requests: transient differs, **scoped differs**, singleton is the same.

The rule

"Transient = a new instance every time it's INJECTED. Scoped = one per HTTP request. Singleton = one per application lifetime."

▲ Follow-up — "Which one for `DbContext`, and why?"

Scoped — and `AddDbContext` registers it as scoped by default. It gives you one unit of work and one change-tracker per request, which is exactly the EF Core model. **Singleton would be a disaster:** `DbContext` isn't thread-safe and its change tracker would grow forever. **Transient** would break transactions across repositories, because each repository would get its own context.

Q58. Last registration wins

```
builder.Services.AddScoped<IGreeter, EnglishGreeter>();
builder.Services.AddScoped<IGreeter, FrenchGreeter>();

public HomeController(IGreeter g, IEnumerable<IGreeter> all)
    => Console.WriteLine($"{g.GetType().Name}, count = {all.Count()}");
```

✓ Output

```
FrenchGreeter, count = 2
```

The rule

"Registering the same interface twice does NOT replace — both stay in the container. Resolving a single instance gives you the LAST one registered; resolving `IEnumerable<T>` gives you ALL of them, in registration order."

▲ Follow-up — "How do you actually replace one?"

```
builder.Services.Replace(ServiceDescriptor.Scoped<IGreeter,  
FrenchGreeter>());  
builder.Services.TryAddScoped<IGreeter, EnglishGreeter>();    // only if  
none registered
```

TryAdd* is what **libraries** use, so your app's registration wins over the library's default. Knowing **TryAdd** exists and why is a strong signal.

Q59. Middleware order

```
app.Use(async (ctx, next) => { Console.WriteLine("A in"); await next();  
Console.WriteLine("A out"); });  
app.Use(async (ctx, next) => { Console.WriteLine("B in"); await next();  
Console.WriteLine("B out"); });  
app.Run(async ctx => { Console.WriteLine("terminal"); await  
ctx.Response.WriteAsync("done"); });
```

✓ Output

```
A in  
B in  
terminal  
B out  
A out
```

The rule

"Middleware is a nested pipeline, not a list. `await next()` is the seam — everything before it runs on the way in, everything after it runs on the way OUT, in reverse order."

Same doll-nesting shape as filters (Q36) — say that out loud, it's the same idea at a different layer.

▲ Follow-up — "What if B doesn't call `next()` ?"

```
A in
B in
A out
```

B short-circuits. `terminal` never runs. That's exactly how `UseStaticFiles` works: it serves the file and doesn't call `next()`, so MVC never sees the request — which is *why* it belongs early in the pipeline.

Q60. Writing to the response after `next()`

```
app.Use(async (ctx, next) =>
{
    await next();
    ctx.Response.Headers["X-Custom"] = "value";    // ← ?
});
```

✓ What happens

▲ `InvalidOperationException: Headers are read-only, response has already started.`

The rule

"Once the response body has started being written, headers are already on the wire and cannot be changed. Anything downstream that wrote a body has closed that window."

▲ Follow-up — "So how do you add a header from middleware?"

```
app.Use(async (ctx, next) =>
{
    ctx.Response.OnStarting(() =>
    {
        ctx.Response.Headers["X-Custom"] = "value";
        return Task.CompletedTask;
    });
    await next();
});
```

Register a callback with `OnStarting`, which fires just before the first byte goes out — or set the header *before* calling `next()`. Also check `ctx.Response.HasStarted` before touching the response in any error handler.

Q61. `UseRouting` / `UseAuthorization` in the wrong order

```
app.UseAuthorization();
app.UseRouting();
app.MapControllers();
```

✓ What happens

▲ `InvalidOperationException:`
Endpoint routing does not support
'IApplicationBuilder.UseAuthorization'
being called before 'UseRouting'.
...or [Authorize] is silently NOT enforced.

The rule

" **`UseRouting`** is what **SELECTS** the endpoint. **`UseAuthorization`** needs to know which endpoint was selected in order to read its **`[Authorize]`** metadata — so it must come after **`UseRouting`** and before **`UseEndpoints`** / **`MapControllers`**."

The order to memorize

```
app.UseExceptionHandler("/Error"); // outermost – must wrap everything
app.UseHsts();
app.UseHttpsRedirection();
app.UseStaticFiles(); // before routing – short-circuits
                        // for files
app.UseRouting(); // ← selects the endpoint
app.UseCors();
app.UseAuthentication(); // who are you?
app.UseAuthorization(); // are you allowed? ← must follow
UseRouting
app.MapControllers(); // ← executes the endpoint
```

▲ Follow-up — "Why is `UseStaticFiles` before `UseRouting`?"

Performance — a static file short-circuits before any routing work happens. The trade-off worth naming: **static files served before `UseAuthorization` are not protected by `[Authorize]`**. Anything in `wwwroot` is public. If files need protection, serve them through a controller action.

Q62. `IOptions` vs `IOptionsSnapshot`

```
builder.Services.Configure<MySettings>(builder.Configuration.GetSection("MySettings"));

public HomeController(IOptions<MySettings> a, IOptionsSnapshot<MySettings> b) { }
```

`appsettings.json` is edited while the app is running.

✓ What happens

a.Value → the OLD value (read once at startup, cached forever)
b.Value → the NEW value (re-read per request)

The rule

" `IOptions<T>` is a **SINGLETON** computed once. `IOptionsSnapshot<T>` is **SCOPED** and recomputed per request, so it picks up `reloadOnChange` config edits. `IOptionsMonitor<T>` is a singleton that also gets change notifications — the only one usable inside another singleton."

▲ Follow-up — "Which do you inject into a singleton service?"

`IOptionsMonitor<T>` — injecting `IOptionsSnapshot<T>` (scoped) into a singleton is exactly the captive-dependency error from Q56.

J — Rapid-fire answer table

Cover the right column. Target: **under 5 minutes**, daily.

#	Scenario	Answer	Why (one line)
1	Two same-name, same-verb actions	<code>AmbiguousMatchException</code>	Routing matched; action selection couldn't
2	<code>[Route]</code> on one action	conventional routes stop reaching it	Attribute routing opts that action out
3	<code>[HttpGet]</code> + <code>[HttpPost]</code> same name	✓ fine	Verb disambiguates
4	<code>{id?}</code> route + <code>int id</code> param	Core: <code>0</code> · MVC 5: exception	Non-nullable can't be optional
5	<code>{id:int}</code> with <code>/abc</code>	404	Constraints are matching, not validation
6	Route default vs C# default	route default wins	Binding runs before the C# default applies

#	Scenario	Answer	Why (one line)
7	<code>{*url}</code> registered first	breaks the whole site	Catch-all shadows everything below
8	Public helper on a controller	it's a routable endpoint	All public methods are actions
9	Same key in form + route + query	form wins	Form → Route → Query
10	Two <code>[FromBody]</code> params	exception	Body is read once
11	JSON posted to an MVC controller	model is empty	Complex types bind from form unless <code>[FromBody]</code> / <code>[ApiController]</code>
12	<code>Name=Ann</code> for a nested property	<code>null</code>	Needs <code>Customer.Name</code>
13	<code>items[0]</code> and <code>items[2]</code>	count = 1	Indices must be consecutive
14	<code>quantity=abc</code> for <code>int quantity</code>	<code>0</code> , action still runs	Binding failure ≠ short-circuit
15	Extra <code>isAdmin=true</code> posted	it binds ♦	Over-posting — bind to a DTO
16	Unchecked checkbox	field absent → <code>false</code>	Hidden companion input supplies false
17	<code>[FromServices]</code>	resolved from DI	Not from the request
18	<code>[Required] int Quantity = 0</code>	valid	Required only checks null
19	<code>AddModelError</code> after <code>IsValid</code>	too late, code continues	<code>IsValid</code> isn't a latch
20	<code>[ApiController]</code> + invalid model	400 before the action	Auto model-validation filter

#	Scenario	Answer	Why (one line)
21	No unobtrusive JS	server still rejects	Client validation is UX only
22	<code>[Required]</code> inside a null child	valid	Nothing to walk
23	<code>@ViewBag.Typo</code>	blank, no error	<code>dynamic</code> returns null
24	<code>ViewData["X"]</code> then <code>ViewBag.X</code>	same value	One dictionary
25	Read <code>TempData</code> twice in one request	works both times	Deleted at end of request, not on read
26	<code>ViewBag</code> across a redirect	<code>null</code>	New request, new ViewData
27	<code>TempData</code> never read	survives to the next request	Deleted on read, not by age
28	<code>TempData</code> + AJAX	leaks onto a later page	TempData is per-session, not per-page
29	<code>AddSession</code> without <code>UseSession</code>	exception	<code>AddX</code> registers, <code>UseX</code> executes
30	Code after <code>return RedirectToAction</code>	unreachable	It just returns a result object
31	<code>Response.Redirect</code> (MVC 5)	code keeps running	Doesn't stop execution
32	<code>return View()</code> from POST	F5 re-submits	Use POST-Redirect-GET
33	<code>return Json(x)</code> on GET (MVC 5)	exception	JSON hijacking guard; removed in Core

#	Scenario	Answer	Why (one line)
34	<code>View()</code> with a missing file	exception listing search paths	<code>/Views/{Ctrl}/</code> then <code>/Views/Shared/</code>
35	<code>PartialView()</code>	no layout	Skips <code>_ViewStart</code>
36	Un-awaited <code>SaveChangesAsync</code>	may never complete	Scope disposed under it
37	Global + controller + action filters	in: $G \rightarrow C \rightarrow A$ · out: $A \rightarrow C \rightarrow G$	Nested, reversed on the way out
38	Filter sets <code>context.Result</code>	action skipped, <code>OnActionExecuted</code> skipped too	Short-circuit
39	Exception thrown in the view	exception filter misses it	Filters don't cover result execution
40	<code>[Authorize]</code> + <code>[AllowAnonymous]</code>	anonymous wins	Overrides every scope
41	<code>[ValidateAntiForgeryToken]</code> no token	400	Cookie + form field must match
42	Action throws	<code>OnActionExecuted</code> still runs	Exception arrives on <code>context.Exception</code>
43	▲ <code>p.Name = "X"; return View(p);</code>	textbox shows the posted value	Helpers read ModelState first
44	<code>@count + 1</code>	<code>5 + 1</code>	Implicit expression ends at the space
45	<code>@Html.Raw(userInput)</code>	◆ XSS	Only way to bypass encoding
46	<code>@Html.RenderPartial("x")</code>	compile error	Returns <code>void</code> ; needs a code block

#	Scenario	Answer	Why (one line)
47	@section in a partial	ignored / throws	Sections belong to view↔layout
48	ViewBag set in the view	visible in the layout	View executes before the layout
49	@RenderSection("Scripts")	required by default	Pass required: false
50	Lambda capturing for 's i	all get the last value	One shared variable
51	ActionLink("t", "a", "Products")	?Length=8	3-arg overload treats it as route values
52	Instance field on a controller	resets every request	New controller per request
53	static List<T> on a controller	shared + not thread-safe	Process-wide state
54	async void action	empty 200, crash on throw	Nothing to await
55	ConfigureAwait(false) + HttpContext.Current	null (MVC 5)	Left the SynchronizationContext
56	Scoped injected into Singleton	startup exception (Dev)	Captive dependency
57	Two ITransient in one ctor	different instances	Transient = per injection
58	Same interface registered twice	last wins; IEnumerable gets both	Registrations append
59	Two middlewares + terminal	A in, B in, term, B out, A out	Nested pipeline
60	Set a header after next()	exception	Response already started

#	Scenario	Answer	Why (one line)
61	<code>UseAuthorization</code> before <code>UseRouting</code>	exception / not enforced	Needs the selected endpoint
62	<code>IOptions</code> vs <code>IOptionsSnapshot</code>	stale vs fresh	Singleton vs scoped

K — 18 self-test drills (answers hidden)

Cover the answer, commit out loud, then check. **Five a day.**

Drill 1

```
public IActionResult Index(string name = "guest") => Content(name ?? "null");
```

`GET /Home/Index?name=`

→ Answer

`**empty string**`, not ``"guest"``. The key `*is*` present with an empty value, so binding succeeds with ```` and the C# default never applies. To treat empty as missing you need ``string.IsNullOrEmpty(name)`` explicitly.

Drill 2

```
[HttpPost] public IActionResult Save(Product p) => View(p);
```

Posted: `Name=Ann`. The view has `@Html.HiddenFor(m => m.Id)` and `Id` was never posted.

→ Answer

Renders ``value="0"`` — ``Id`` is a non-nullable ``int``, binding produced nothing, so it's ``0``. On the next POST the record updates `**entity 0**`, i.e. creates a new one. This

is why edit forms must round-trip the id in a hidden field *and* validate it server-side.

Drill 3

```
public IActionResult A() { TempData["m"] = "x"; return View(); }           //  
NOT a redirect  
public IActionResult B() => Content(TempData["m"]?.ToString() ?? "null");
```

Call **A** then **B**.

→ Answer

****x**** — TempData works across *any* two requests, not just redirects. The redirect is just the usual reason you need it.

Drill 4

```
@{ var items = new List<string>(); }  
<p>@items.FirstOrDefault().Length</p>
```

→ Answer

▲ ****NullReferenceException**** at render time. `FirstOrDefault()` on an empty `List` returns `null`. And because it's thrown during ****result execution****, an exception **filter** won't catch it (Q39) — only exception middleware will.

Drill 5

```
[HttpGet] public IActionResult Delete(int id) { db.Delete(id); return  
RedirectToAction("Index"); }
```

→ Answer

It works — and it's a ****serious bug****. A GET must be safe and idempotent. A search-engine crawler, a browser prefetch, or an `` on any other website will delete your data. Destructive actions must be `[HttpPost]` + `[ValidateAntiForgeryToken]`.

Drill 6

```
[HttpPost]
public IActionResult Save(Product p)
{
    if (!ModelState.IsValid) return View(p);
    db.Add(p);
    return RedirectToAction(nameof(Index));
}
```

The `Product` has `[Required] public string Name` and the user submits an empty name.

→ Answer

Returns the `**Edit view with the validation message**`, and the textbox redisplay the user's empty input — correct behaviour on all three counts. The trap is the `*next* question: **`ViewBag`/`ViewData` you set in the GET action are gone**`, so any dropdown populated from ``ViewBag.Categories`` will now throw ``NullReferenceException`` on redisplay. You must repopulate them before every ``return View(p)``. This is the #1 real-world MVC form bug.

Drill 7

```
@model IEnumerable<Product>
@foreach (var p in Model) { @Html.TextBoxFor(m => p.Name) }
```

→ Answer

Every input renders with `**`name="p.Name"`**` — the same name for every row, and a name that doesn't match anything on POST. Lambda-based helpers read the `*expression text*`, and here the expression is the loop variable. Fix with an indexed ``for`` loop (`m => m[i].Name``) or an ``EditorTemplate``, which generates correct indexed names automatically.

Drill 8

```
public class HomeController : Controller
{
    public HomeController(ILogger<HomeController> l) { }
    public HomeController() { }           // second constructor
}
```

→ Answer

▲ **InvalidOperationException** — the DI container requires an **unambiguous** constructor. Multiple public constructors where more than one is satisfiable is an error. Controllers get exactly one constructor.

Drill 9

```
[HttpPost] public IActionResult Save([FromBody] string name) =>
    Content(name ?? "null");
```

Body: **name=Ann** with **Content-Type: application/x-www-form-urlencoded**.

→ Answer

415 Unsupported Media Type (with `[ApiController]`) or `null`. `[FromBody]` uses the configured **input formatters**, which by default handle JSON only. To send a bare string as JSON the body must be `"Ann"` — with the quotes — and `Content-Type: application/json`.

Drill 10

```
@* _Layout.cshtml *@
@RenderBody()
@RenderBody()
```

→ Answer

▲ **InvalidOperationException** — `RenderBody` may be called **once**. The body is a forward-only buffer, not a re-readable string. (Sections, by contrast, can only be rendered once each too.)

Drill 11

```
public IActionResult Index()
{
    ViewBag.Items = new List<string> { "a" };
    return View();
}
```

```
@foreach (var i in ViewBag.Items) { <p>@i</p> }
```

→ Answer

It **works** — `foreach` over a `dynamic` resolves at runtime. But `@ViewBag.Items.Where(x => x.Length > 0)` would throw `RuntimeBinderException`, because **extension methods can't be called on `dynamic`**. That one catches almost everybody. Fix: `((List)ViewBag.Items).Where(...)`.

Drill 12

```
app.UseEndpoints(e => e.MapControllers());
app.UseAuthentication();
```

→ Answer

`UseAuthentication` **never runs for MVC requests** — `MapControllers` is terminal, so the pipeline ends there. `User` is anonymous everywhere and `[Authorize]` rejects every request. No exception, just a totally broken auth system: the worst kind of ordering bug.

Drill 13

```
public class Product { public string Name { get; } } // no setter
```

Posted: **Name=Ann**

→ Answer

`Name` stays `null` — **model binding requires a public setter**. Same for `init`-only properties bound from a **form** (they work via the constructor for JSON body

binding, but not for form binding). This is why binding to `record` types trips people up.

Drill 14

```
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    [HttpGet] public IActionResult All() => Ok();
    [HttpGet] public IActionResult Active() => Ok();
}
```

→ Answer

▲ **AmbiguousMatchException** — both actions map to `GET /api/Products`. `[HttpGet]` with no template doesn't append the action name. Fix: `[HttpGet("all")]` and `[HttpGet("active")]`, or add `[action]` to the controller route.

Drill 15

```
public IActionResult Index()
{
    Response.StatusCode = 404;
    return View();
}
```

→ Answer

The **Index** view renders with a 404 status code. Setting `StatusCode` doesn't change what's returned — the two are independent. This is actually the standard way to serve a friendly "not found" page that still tells crawlers the truth.

Drill 16

```
public class LogFilter : IActionFilter
{
    private int _count;
    public void OnActionExecuting(ActionExecutingContext c) =>
        Console.WriteLine(++_count);
}
builder.Services.AddControllers(o => o.Filters.Add(new LogFilter())); //
← instance
```

→ Answer

Prints `1, 2, 3, ...` — adding an **instance** makes it a de-facto singleton shared across all requests, so `\_count` accumulates and `++\_count` is a **data race**.
`o.Filters.Add()` (the generic form) creates one per request instead. Filter **attributes** are also cached and shared — never put mutable state in a filter.

Drill 17

```
[HttpPost]
public async Task<IActionResult> Upload(IFormFile file)
{
    var path = Path.Combine("uploads", file.FileName);
    using var s = System.IO.File.Create(path);
    await file.CopyToAsync(s);
    return Ok();
}
```

→ Answer

Works — and is a **path-traversal vulnerability**. `file.FileName` comes from the client and can be `..\..\Windows\System32\x.dll`. Always `Path.GetFileName(file.FileName)`, or better, generate your own name (Guid.NewGuid()) and store the original only as metadata. Also validate content type and size.

Drill 18

```
public IActionResult Index()
{
    var users = db.Users.Where(u => u.Active); // IQueryable, not
    executed
    return View(users);
}
```

```
@foreach (var u in Model) { <p>@u.Name</p> }
```

→ Answer

Either it works (the query executes during rendering, **after** the action returned) or you get ▲ **ObjectDisposedException** if the `DbContext` scope has closed. Either way the DB query runs inside the view, so a slow query looks like a slow

render and exceptions dodge your exception filter. **Always materialize before returning: ``await db.Users.Where(...).ToListAsync()``.** Same deferred-execution rule as LINQ in [file 27, Q57].

L — How to answer these in the actual interview

1. Never blurt the answer. Name the stage first.

"This is a model-binding question. The binder tries form, then route, then query string — so the form value 3 wins."

Saying **which stage of the pipeline** you're in (routing / binding / validation / filters / action / result) is the whole skill. It's also the reason MVC questions feel hard: candidates try to answer them as C# questions.

2. Trace the request out loud. For anything you're unsure of:

"Request comes in → middleware → routing selects the endpoint → authorization filter → model binding → action filters → action → result filters → the view renders. The exception is in the view, so we're past the action — which means an exception filter won't see it."

That sentence answers half the questions in section F on its own.

3. Say "Core or MVC 5?" when it matters. Q4, Q11, Q30, Q32, Q54 and Q55 have genuinely different answers. Asking *"is this Core or Framework?"* is not dodging — it's the correct question, and interviewers read it as experience.

4. Name the exception precisely. *"It errors"* is worth one point.

"`InvalidOperationException` — a singleton can't consume a scoped service, that's a captive dependency, and scope validation only runs in Development by default" is worth five.

5. Always give the fix, then the trade-off. Every answer above has a follow-up for a reason — the rounds you've lost were lost on the *second* question, not the first. Definition → what you'd do → what it costs.

6. Use your Java/Spring background deliberately. The concepts map almost one-to-one, and saying so proves you understand mechanisms rather than syntax:

Spring / Java	ASP.NET Core MVC
@Controller / @RestController	Controller / [ApiController]
@RequestMapping / @GetMapping	[Route] / [HttpGet]
@RequestBody / @RequestParam / @PathVariable	[FromBody] / [FromQuery] / [FromRoute]
@Valid + BindingResult	[Required] + ModelState
HandlerInterceptor	Action filter
@ControllerAdvice / @ExceptionHandler	Exception filter / UseExceptionHandler
Servlet Filter	Middleware
@Autowired , @Service , @Scope	Constructor injection, AddScoped / AddSingleton
@Transactional	DbContext scope / explicit transaction
Thymeleaf / JSP	Razor
Spring Data JPA	Entity Framework Core

*"In Spring this would be a **HandlerInterceptor** ; here it's an action filter — same position in the pipeline, and the same **preHandle** / **postHandle** shape." **That's the sentence that makes 4 years of Java count as 4 years of relevant experience** instead of starting from zero.*

7. If you genuinely don't know:

*"My reasoning says X because the binder runs before the action, but that's exactly the kind of thing I'd confirm with a quick integration test using **WebApplicationFactory**."*

Related files in this pack

- **27 — C# / .NET "Predict the Output"** — the language half; several traps here (closures, deferred LINQ, `async void`, DI lifetimes) are that file's rules wearing an HTTP costume
- **23 — Java "Predict the Output"** — the same *style* of question in the language you already know
- **06 — Spring & Spring Boot** — read alongside section L's mapping table; every Spring concept you know has a named equivalent here
- **24 — ORM, JPA & Hibernate** — the EF Core equivalents: lazy loading, change tracking, N+1, and `DbContext` scope (Q57 and Drill 18)
- **17 — SOLID & Design Patterns** — DI, the filter pipeline and middleware are Chain of Responsibility + Dependency Inversion in production
- **26 — Companies: Questions Actually Asked** — **log every .NET round here the same day**

Drill routine: section G first (the `TextBoxFor` /ModelState trap in **Q43** is the single most-asked MVC view question), then sections A/B (routing + binding are where the round starts), then the section J rapid-fire table daily under 5 minutes, then five section K drills each morning.

Type the code — don't just read it. Spin up one `dotnet new mvc` project and actually run the ten you find least obvious. Reading gives you recognition; running gives you the answer under pressure.

ASP.NET MVC – 72 Interview Questions with Answers

◆ **What this file is:** the **spoken Q&A round** — "*what is X?*", "*explain Y*", "*how would you do Z?*" — for ASP.NET MVC.

Its sibling **28 – MVC "What Happens?"** is the *code round* ("*this action runs — what does the browser get?*"). Most .NET rounds use **both**: theory for the first 20 minutes, then a program on screen. Do 29 first, 28 second.

■ **Framework coverage.** Answers are **ASP.NET Core MVC** first, with ****** In MVC 5 boxes wherever the classic framework differs. "*What changed in Core?*" is itself one of the most-asked questions in this list — you'll find it at **Q7****.

▲ **Read § 0 – The honesty position before anything else.** You have Angular + Java/Spring Boot production experience, not .NET production experience. How you frame that in the first two minutes decides how the rest of the round goes.

Table of contents

#	Section	Questions
0	● The honesty position	How to answer " <i>have you worked on .NET?</i> "
★	★ The 15 most-likely questions	If you only have one hour
A	MVC fundamentals	Q1–Q9
B	The request lifecycle	Q10–Q13
C	Routing	Q14–Q21
D	Controllers & action results	Q22–Q29
E	Views & Razor	Q30–Q39
F	Passing data to the view	Q40–Q44

#	Section	Questions
G	Model binding & validation	Q45–Q51
H	Filters	Q52–Q56
I	DI & middleware	Q57–Q61
J	Security	Q62–Q66
K	EF Core, performance & testing	Q67–Q72
L	Answer scripts	Self-intro, project walkthrough, closing
M	Rapid-fire table	1-line answers

● 0 — The honesty position (read this first)

They will ask, in the first five minutes: *"How much .NET have you worked on?"*

- You have a real answer to this — lead with it. Do not undersell.

Subsea was an Angular front-end on a **.NET Web API** backend, containerized with Docker and deployed on **Azure**. You **owned the Schedule-Manager module front to back — the Angular UI and the .NET endpoints** — including pushing grid paging/sorting/filtering down into the C# API with Kendo's **DataSourceRequest**, and you coordinated API contracts with the backend team. That is production .NET, shipped.

- Full detail: **10 — Projects Deep-Dive § 2 Subsea** — re-read that section before any .NET interview.

✓ The script (memorize this shape, use your own words)

"I've worked on both sides of a .NET stack. On Subsea — a marine equipment and operations platform — the backend was a .NET Web API with an Angular SPA in

front, running in Docker on Azure. I owned the Schedule-Manager module end to end: the Angular feature module and the .NET endpoints behind it. The piece I'd point to is a performance fix — the Kendo grids were client-bound and got sluggish as the datasets grew, so I moved paging, sorting and filtering into the API using `DataSourceRequest` so it translated to SQL paging and only one page crossed the wire.

Alongside that, my other backend experience is Java with Spring Boot, which maps very closely — filters are Spring's interceptors, middleware is the servlet filter chain, the built-in DI container plays the same role, EF Core sits where JPA does. So I'm comfortable in the architecture from both directions.

To be straight about the boundary: my .NET work has been Web API — controllers, endpoints, DTOs — rather than MVC with Razor views, since the UI was Angular. The routing, model binding, filters and DI are the same stack, and I've been closing the Razor and view-layer gap deliberately."

Why this works: it opens with a real shipped project and a specific technical decision (the one that proves you were actually in the code), backs it with the Spring mapping, and draws the boundary *precisely* — Web API rather than Razor MVC — instead of vaguely conceding "I haven't really done .NET". Precision reads as confidence; vagueness reads as bluffing.

▲ Two things to nail down before the interview

1. **Which .NET version was Subsea on?** — .NET Framework 4.x with classic Web API, or .NET Core/5+? It changes half the answers in this file, and "I don't remember which .NET we were on" is the one answer that damages you.

Check the repo or your notes and know it.

2. **Was there any Razor, or was it API-only?** If the project had any `.cshtml`, say so. If it was purely JSON endpoints, say that — it's a completely normal shape and nobody penalises it.

● **Never bluff a version number or an API signature.** Everything else here is true, so there is nothing you need to invent.

The mapping table — know it cold

Spring / Java	ASP.NET Core MVC
@Controller / @RestController	Controller / [ApiController]
@RequestMapping , @GetMapping	[Route] , [HttpGet]
@RequestBody / @RequestParam / @PathVariable	[FromBody] / [FromQuery] / [FromRoute]
@Valid + BindingResult	Data annotations + ModelState
HandlerInterceptor (preHandle / postHandle)	Action filter (OnActionExecuting / OnActionExecuted)
@ControllerAdvice + @ExceptionHandler	Exception filter / UseExceptionHandler
Servlet Filter	Middleware
@Autowired , @Service , @Component	Constructor injection, AddScoped , AddSingleton
@Scope("singleton"/"request")	AddSingleton / AddScoped
application.properties / @Value	appsettings.json / IOptions<T>
Spring Data JPA, EntityManager	Entity Framework Core, DbContext
@Transactional	DbContext scope + SaveChanges / explicit transaction
Thymeleaf / JSP	Razor
Maven / Gradle	NuGet / .csproj
Tomcat	Kestrel (behind IIS/Nginx)
JUnit + Mockito	xUnit + Moq

◆ **Use it live.** Whenever you're asked about a .NET concept you're shaky on, say *"in Spring this is X — is it the same idea here?"* You'll usually be right, and it converts a gap into a conversation between two engineers.

★ The 15 questions most likely to be asked

If you have one hour before the interview, these are it. Full answers are linked.

#	Question	The one-line answer
1	What is MVC?	A separation-of-concerns pattern: Model = data + rules, View = UI, Controller = handles the request and picks the response
2	Explain the request lifecycle	Middleware → routing → filters → model binding → action → result → response
3	ViewBag vs ViewData vs TempData	Same request / same request / survives a redirect
4	What are filters and their order?	Authorization → Resource → Action → Exception → Result; Global → Controller → Action, reversed on the way out
5	Routing: conventional vs attribute	Pattern registered centrally vs <code>[Route]</code> on the action
6	What is model binding?	Maps HTTP request values onto action parameters: Form → Route → Query
7	How does validation work?	Data annotations → <code>ModelState.IsValid</code> → redisplay or save; client validation is UX only
8	Partial view vs View Component	Partial = markup only; View Component = markup + its own logic and DI

#	Question	The one-line answer
9	What are action results?	<code>ViewResult</code> , <code>JsonResult</code> , <code>RedirectToActionResult</code> , <code>FileResult</code> , <code>StatusCodeResult</code> ...
10	DI lifetimes	Transient = per injection, Scoped = per request, Singleton = per app
11	What is middleware?	A nested request pipeline; runs for every request, not just MVC
12	MVC 5 vs Core	Cross-platform, built-in DI, unified MVC+API, middleware instead of HTTP modules
13	How do you prevent CSRF?	<code>[ValidateAntiForgeryToken]</code> + the hidden token the tag helper emits
14	Layout, <code>_ViewStart</code> , sections	<code>_ViewStart</code> → view → layout; <code>RenderBody</code> once, sections optional
15	How would you improve performance?	Async all the way, <code>AsNoTracking</code> , fix N+1, cache, project to DTOs, bundle assets

A — MVC fundamentals

Q1. What is MVC and why use it?

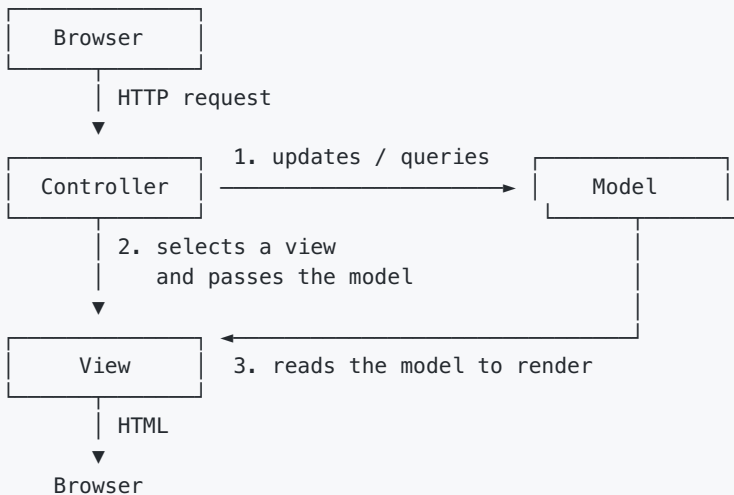
Answer (say this):

"MVC is a presentation-layer pattern that splits an application into three responsibilities: the Model holds the data and business rules, the View renders the UI, and the Controller receives the request, coordinates the model, and chooses which result to return. The benefit is separation of concerns — the view has no business logic and the controller has no HTML, so each part is testable and changeable on its own."

The third sentence (the trade-off) — this is what scores:

"The cost is more files and more ceremony than putting logic in the page, so for a truly simple CRUD page Razor Pages is often the better fit. MVC pays for itself once there's real logic to keep out of the view."

If they push: "who talks to whom?"



The key rule: the **View never talks to the Model directly for data-fetching** — the controller hands it what it needs. In web MVC the view is passive.

Q2. What are the advantages of MVC over Web Forms?

	Web Forms	MVC
Markup control	ViewState, generated IDs	Full control over the HTML
Testability	Code-behind is tied to the page lifecycle	Controllers are plain classes — unit testable
URLs	.aspx file paths	Clean, routed, SEO-friendly
Statefulness	Stateful (ViewState, postbacks)	Stateless — matches how HTTP actually works
Separation	Code-behind mixes concerns	Enforced by the pattern
Front-end fit	Fights modern JS frameworks	Natural fit for Angular/React

◆ **Tie it to your experience:** *"The last point matters to me — I build Angular front-ends. MVC returning JSON from clean routed URLs is exactly what an Angular app wants; Web Forms' postback model fights it."*

Q3. Is MVC a design pattern or an architectural pattern?

"It's an architectural pattern for the presentation layer — it structures the whole layer, not one class interaction. People often call it a design pattern loosely, but it's built out of design patterns: the filter pipeline is Chain of Responsibility, DI is Dependency Inversion, and the view engine selection is a Strategy."

That last sentence connects it to **17 — SOLID & Design Patterns** and reliably scores.

Q4. What is the "Model" exactly? Domain model vs view model vs DTO

Type	Purpose	Lives where
Domain / entity model	Business data + rules; maps to DB tables via EF	Domain / Data layer
View model	Exactly what ONE view needs — may combine several entities, adds display-only fields	Web layer
DTO	Data crossing a boundary (API request/response)	Contracts / API layer

Answer:

"The 'M' in MVC is usually a view model, not the entity. I'd never bind a Razor form directly to an EF entity — that's how you get over-posting, where an attacker adds `IsAdmin=true` to the form and the binder happily sets it. A view model only carries the fields the view actually uses, so the extra field has nowhere to land."

That answers the architecture question *and* the security question at once. (Live demo of the attack: [28](#) — [Q15](#).)

Q5. MVC vs Razor Pages vs Minimal APIs — when do you use each?

	Best for	Shape
MVC	Apps with real logic, multiple views per controller, or a mix of pages + APIs	<code>Controller</code> + actions + views
Razor Pages	Page-focused CRUD — one URL, one file pair	<code>Page.cshtml</code> + <code>PageModel</code> with <code>OnGet</code> / <code>OnPost</code>

	Best for	Shape
Minimal APIs	Small JSON APIs, microservices	<code>app.MapGet("/x", () => ...)</code> in <code>Program.cs</code>
Web API (controllers)	Larger JSON APIs needing filters, versioning, conventions	<code>[ApiController] : ControllerBase</code>

"Microsoft's own guidance since .NET 6 is Razor Pages for page-based scenarios and MVC when you need the controller abstraction. They share the same underlying stack — routing, binding, filters, DI are identical — so it's an organisational choice, not a technical one."

Q6. What's the difference between .NET Framework, .NET Core and .NET 5+?

".NET Framework (up to 4.8) is Windows-only, IIS-hosted, and in maintenance mode — no new features. .NET Core (1.0–3.1) was the cross-platform rewrite. From .NET 5 they dropped 'Core' from the name and unified everything into one platform — .NET 6, 8, 9 are all the same lineage. .NET Standard was a compatibility spec that let libraries target both; it's effectively legacy now that everything is on .NET 5+."

The version fact worth knowing: even-numbered releases are LTS (.NET 6, 8, 10 — three years of support); odd-numbered are STS (18 months). If they ask what you'd start a new project on: *"the current LTS."*

Q7. What's the difference between ASP.NET MVC 5 and ASP.NET Core MVC?

This is the single most-asked comparison question. Know at least six rows.

	ASP.NET MVC 5	ASP.NET Core MVC
Platform	Windows only, <code>System.Web</code>	Cross-platform (Windows/Linux/macOS)
Hosting	IIS only	Kestrel; IIS/Nginx/Apache as reverse proxy; Docker
Startup	<code>Global.asax</code> + <code>Web.config</code> + <code>RouteConfig</code>	<code>Program.cs</code> (minimal hosting) + <code>appsettings.json</code>
DI	None built in — Unity, Ninject, Autofac	Built in — <code>IServiceCollection</code>
MVC + Web API	Two separate stacks, two <code>Controller</code> base classes, two filter systems	Unified — one <code>Controller</code> / <code>ControllerBase</code>
Pipeline	HTTP modules & handlers	Middleware
Config	<code>Web.config</code> (XML)	<code>appsettings.json</code> + env vars + user secrets, layered
Views	Razor + ASPX engines; HTML helpers	Razor only; tag helpers + view components
Cross-cutting	<code>HttpContext.Current</code> (ambient static)	Injected <code>IHttpContextAccessor</code> — no ambient state
Performance	Good	Substantially faster (Kestrel, less allocation)
Open source	Partially	Fully, on GitHub

Answer out loud (pick four):

"The big ones are: Core is cross-platform and self-hosted on Kestrel instead of IIS-only; dependency injection is built in instead of bolted on; MVC and Web API were merged into one stack instead of two parallel ones; and HTTP modules were replaced by middleware, which is a much simpler nested pipeline. Configuration also moved from `Web.config` XML to layered `appsettings.json` plus environment variables."

Q8. What is Kestrel, and why is there still IIS in front of it?

"Kestrel is the cross-platform web server built into ASP.NET Core — it's what actually runs your app. It's fast but deliberately minimal, so in production you usually put a reverse proxy in front of it — IIS on Windows, Nginx or Apache on Linux — which handles TLS termination, request filtering, port sharing across sites, and process management/restarts."

If they push — "can Kestrel be edge-facing?" Yes, it's supported and hardened enough for it now, but the reverse proxy still buys you host-level concerns you'd otherwise implement yourself.

Q9. What is `wwwroot`, and how do static files work?

"`wwwroot` is the web root — the only folder served directly to browsers. Everything else in the project is invisible over HTTP, which is a safe default: your `.cs` files, `appsettings.json` and `Views` folder can't be downloaded."

```
app.UseStaticFiles();    // serves wwwroot, short-circuits — must come
BEFORE UseRouting
```

The trade-off to volunteer: "Because `UseStaticFiles` runs before `UseAuthorization`, anything in `wwwroot` is public — `[Authorize]` doesn't protect it. Files that need protection have to be served through a controller action that checks permissions and returns a `FileResult`."

B — The request lifecycle

Q10. Walk me through the ASP.NET Core MVC request lifecycle

This is the highest-value answer in the entire file. Almost every other question is a zoom-in on one of these steps, so knowing the sequence lets you locate any question.

1. Browser sends an HTTP request
2. Kestrel receives it, builds an HttpContext
3. — MIDDLEWARE PIPELINE (in order) —
 - UseExceptionHandler → UseHttpsRedirection → UseStaticFiles
 - UseRouting ← selects the endpoint from the route table
 - UseAuthentication (who are you?) → UseAuthorization (allowed?)
 - UseEndpoints / MapControllers ← executes the endpoint
4. — FILTER PIPELINE —
 - Authorization filters
 - Resource filters (before)
5. Controller instantiated (dependencies injected)
6. MODEL BINDING – request values → action parameters
7. VALIDATION – data annotations populate ModelState
 - Action filters (before)
8. == THE ACTION METHOD RUNS ==
 - Action filters (after)
 - Result filters (before)
9. RESULT EXECUTION – the view engine renders Razor to HTML
 - Result filters (after)
 - Resource filters (after)
10. Response travels back OUT through the middleware, in reverse
11. Kestrel writes it to the wire

The 30-second spoken version:

*"The request hits Kestrel, goes through the middleware pipeline where routing selects an endpoint and authentication and authorization run, then into the MVC filter pipeline. MVC creates the controller with its dependencies injected, binds the request values onto the action parameters, validates them into ModelState, runs the action filters and then the action, and the action returns an **IActionResult** which is executed afterwards — for a **ViewResult** that's when Razor actually*

renders. Then the response unwinds back out through the middleware in reverse order."

▲ The follow-ups they ask next

"Where exactly does model binding happen?" — Between resource filters and action filters. That's *why* an authorization filter can reject a request before you pay the cost of binding it, and why resource filters are where output caching lives.

"Where does the view render?" — In step 9, **after the action has already returned**. This is why an exception in a view isn't caught by an exception filter (28 — Q39) and why a lazy `IQueryable` returned from an action executes inside the view.

Q11. And the MVC 5 lifecycle?

```
Request → IIS → UrlRoutingModule (HTTP module)
        → RouteTable match → MvcRouteHandler → MvcHandler
        → ControllerFactory creates the controller
        → ActionInvoker
            → Authentication filters
            → Authorization filters
            → Model binding
            → Action filters → ACTION → Action filters
            → Result filters → ActionResult executes
                → ViewEngine (Razor) locates & renders the view
        → Response
```

The differences worth naming: MVC 5 uses **HTTP modules and handlers** instead of middleware, has a separate **Authentication filter** type that Core removed (Core does authentication in middleware instead), and everything hangs off `System.Web.HttpContext.Current` rather than an injected context.

Q12. What is `Program.cs` doing in a Core app?

```
var builder = WebApplication.CreateBuilder(args);

// — 1. REGISTER services into the DI container —
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("Default")));
builder.Services.AddScoped<IProductService, ProductService>();

var app = builder.Build();

// — 2. BUILD the middleware pipeline (order matters!) —
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute("default", "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

The one-sentence mental model:

"`builder.Services.AddX()` is registration — what's available. `app.UseX()` is execution — what runs, in order. Nearly every 'why is this null in Core?' bug is a missing `UseX()` or one in the wrong place."

In **MVC 5** this was split across `Global.asax` (`Application_Start`), `RouteConfig.cs`, `FilterConfig.cs`, `BundleConfig.cs` and `Web.config`. Core collapsed all of it into `Program.cs`. (Versions 3.1–5.0 used a `Startup.cs` with `ConfigureServices` and `Configure` — same two halves, different file.)

Q13. How does configuration work in Core?

"*IConfiguration* layers multiple sources, and later sources override earlier ones: `appsettings.json`, then `appsettings.{Environment}.json`, then user secrets in Development, then environment variables, then command-line args. So the same build runs in every environment and only the config differs — which is what you want for containers."

```
// Bind a section to a strongly typed class – preferred over magic strings
builder.Services.Configure<SmtpSettings>(builder.Configuration.GetSection(
    "Smtp"));

public class EmailService
{
    public EmailService(IOptions<SmtpSettings> options) { _settings =
options.Value; }
}
```

The trade-off to volunteer: "Secrets never go in `appsettings.json` — that's committed to git. In development I'd use user secrets (`dotnet user-secrets`), and in production environment variables or Azure Key Vault."

`IOptions` vs `IOptionsSnapshot` vs `IOptionsMonitor` is a common follow-up — see [28](#) — [Q62](#).

C — Routing

Q14. What is routing, and what are the two kinds?

"Routing maps an incoming URL to a controller action, and also generates URLs in the other direction so your links don't hard-code paths. There are two styles: conventional routing, where you register URL patterns centrally, and attribute routing, where you put `[Route]` directly on the controller or action."

```
// CONVENTIONAL – one pattern serves the whole app
app.MapControllerRoute("default", "{controller=Home}/{action=Index}/{id?}");

// ATTRIBUTE – the URL lives next to the code
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    [HttpGet("{id:int}")] // GET /api/products/5
    public IActionResult Get(int id) => Ok();
}
```

When to use which:

"Conventional for a traditional MVC site where URLs follow a predictable / *controller/action* shape. Attribute routing for APIs, where the URLs are resource-shaped and don't match the controller/action naming — and it's the only sensible option for REST versioning."

Q15. Explain {controller=Home}/{action=Index}/{id?}

Segment	Meaning
{controller=Home}	Route parameter with the default value Home
{action=Index}	Defaults to Index
{id?}	Optional — the URL matches with or without it

So / → HomeController.Index(), /Products → ProductsController.Index(), /Products/Details/5 → Details(5).

▲ **The trap they follow up with:** an optional route segment does **not** make the C# parameter optional. Details(int id) with no id throws in MVC 5 and silently binds 0 in Core — 28 — Q4. The parameter must be int? or have a default.

Q16. What are route constraints?

```
[HttpGet("products/{id:int:min(1)}")] // must be an int ≥ 1
[HttpGet("posts/{slug:alpha:length(3,50)}")] // letters only, 3–50 chars
[HttpGet("archive/{date:datetime}")]
[HttpGet("items/{code:regex(^[A-Z]{3}$)}")]
```

"A constraint restricts what a segment can match. The important thing to be clear about is that a constraint is a matching decision, not a validation decision — if it fails you get a 404, not a 400 and not a ModelState error. So I use constraints to reject requests that were never for me, and data annotations to reject requests that are for me but wrong."

Built-ins worth naming: `int`, `bool`, `datetime`, `decimal`, `guid`, `alpha`, `length`, `min/max`, `minlength/maxlength`, `range`, `regex`, `required`.

Q17. How do you write a custom route constraint?

```
public class EvenConstraint : IRouteConstraint
{
    public bool Match(HttpContext ctx, IRouter route, string key,
                      RouteValueDictionary values, RouteDirection
direction)
    => values.TryGetValue(key, out var v)
        && int.TryParse(v?.ToString(), out var n)
        && n % 2 == 0;
}

builder.Services.Configure<RouteOptions>(o => o.ConstraintMap.Add("even",
typeof(EvenConstraint)));
// usage: [HttpGet("items/{id:even}")]
```

Q18. What is attribute route token replacement?

```
[Route("api/[controller]/[action]")]
public class ProductsController : Controller
{
    public IActionResult Search() => Ok();    // → /api/Products/Search
}
```

*"**[controller]**, **[action]** and **[area]** are replaced at startup with the actual names. The benefit is that renaming the class renames the URL automatically — no stale strings. The trade-off is that renaming a class silently breaks every existing link and bookmark, so on a public API I'd write the literal string instead and treat the URL as a contract."*

That trade-off sentence is exactly the "second question" depth your rounds have been lost on.

Q19. What are Areas and when would you use them?

"An Area is a sub-application inside the project with its own Controllers, Views and Models folders — the standard use is separating an Admin section from the public site in a large app."

```
[Area("Admin")]
public class DashboardController : Controller { }

app.MapControllerRoute("areas", "{area:exists}/{controller=Home}/{action=Index}/{id?}");
```

```
/Areas/Admin/Controllers/DashboardController.cs
/Areas/Admin/Views/Dashboard/Index.cshtml
```

Trade-off: *"Areas are folder-level separation only — they don't give you independent deployment or independent teams. Past a certain size I'd reach for separate projects or services rather than more areas."*

Q20. How do you generate URLs instead of hard-coding them?

```
Url.Action("Details", "Products", new { id = 5 })           // in a controller
```

```
<a asp-controller="Products" asp-action="Details" asp-route-id="5">View</a>
@* tag helper *@
@Html.ActionLink("View", "Details", "Products", new { id = 5 },
null)           @* HTML helper *@
```

"Always generate, never hard-code. If the route pattern changes, generated URLs follow automatically and hard-coded ones become 404s. The tag helper form is preferred in Core — the HTML helper has an overload trap where a controller name passed in the wrong position becomes a query-string parameter." (That trap: [28](#) — [Q51](#).)

Q21. What is a named route, and why name one?

```
[HttpGet("products/{id}", Name = "ProductDetails")]
public IActionResult Details(int id) => View();

// then:
Url.Link("ProductDetails", new { id = 5 });
return CreatedAtRoute("ProductDetails", new { id = p.Id }, p);    // 201 +
Location header
```

"A name gives you a stable handle for URL generation that survives route changes. The main real-world use is REST — `CreatedAtRoute` returns a 201 with a `Location` header pointing at the new resource."

D — Controllers & action results

Q22. What is a controller, and what makes a method an action?

"A controller is a class that groups related actions. In Core it inherits from `Controller` (for views) or `ControllerBase` (for APIs, no view support). Every public method is an action by default and is routable — which means a public helper method you forgot to make private is a live endpoint."

Suppress with `[NonAction]` , or just make it private. (The security angle: [28 — Q8](#) — worth telling as a story.)

Conventions: the class name must end in `Controller` (or carry `[Controller]`), and it must be public, non-abstract, non-generic.

Q23. `Controller` vs `ControllerBase` — which do you inherit?

	<code>ControllerBase</code>	<code>Controller</code>
Model binding, validation, <code>HttpContext</code> , <code>User</code>	✓	✓
<code>Ok()</code> , <code>NotFound()</code> , <code>BadRequest()</code> , <code>CreatedAtAction()</code>	✓	✓
<code>View()</code> , <code>PartialView()</code> , <code>ViewBag</code> , <code>ViewData</code> , <code>TempData</code>	✗	✓

" `ControllerBase` for APIs — it keeps view-related members off the class you'll never use. `Controller` inherits from it and adds the Razor bits."

Q24. What are the main action result types?

Return	Produces	Typical use
<code>View()</code> / <code>View(model)</code>	<code>ViewResult</code>	Render Razor with the layout
<code>PartialView()</code>	<code>PartialViewResult</code>	Fragment, no layout — AJAX
<code>Json(obj)</code>	<code>JsonResult</code>	JSON response
<code>Content("text")</code>	<code>ContentResult</code>	Raw text
<code>RedirectToAction("Index")</code>	<code>RedirectToActionResult</code>	302 to another action
<code>Redirect(url)</code> / <code>LocalRedirect(url)</code>	<code>RedirectResult</code>	302 to a URL (<code>Local</code> blocks open-redirect)
<code>File(bytes, type, name)</code>	<code>FileResult</code>	Downloads
<code>Ok()</code> / <code>Ok(obj)</code>	<code>OkObjectResult</code>	200
<code>NotFound()</code>	404	
<code>BadRequest(ModelState)</code>	400	
<code>Unauthorized()</code> / <code>Forbid()</code>	401 / 403	
<code>CreatedAtAction(...)</code>	201 + <code>Location</code>	REST create
<code>NoContent()</code>	204	REST update/delete
<code>StatusCode(418)</code>	any status	
<i>(nothing</i> / <code>void</code>)	<code>EmptyResult</code>	200, empty body

Q25. IActionResult vs ActionResult<T> vs a concrete type?

```
public IActionResult Get(int id)           // ← flexible: can return Ok()
or NotFound()
    => id > 0 ? Ok(new Product()) : NotFound();

public ActionResult<Product> Get2(int id) // ← flexible AND documents the
success type
    => id > 0 ? new Product() : NotFound();

public Product Get3(int id) => new Product(); // ← always 200; can't
return NotFound()
```

"*IActionResult* when the action can return different status codes.

ActionResult<T> is the best of both — you keep the flexibility but the return type documents the success shape, which is what Swagger and API clients read. A concrete return type is fine only when there's genuinely one outcome."

Q26. Why does the action return a result object instead of writing the response?

"Because it makes the action testable and composable. *RedirectToAction* doesn't redirect — it returns an object describing a redirect, which the framework executes afterwards. So a unit test can assert

Assert.IsType<RedirectToActionResult>(result) without any HTTP at all. It also means result filters get a chance to run between the action returning and the response being written."

The practical consequence: code after `return RedirectToAction(...)` is unreachable, whereas MVC 5's `Response.Redirect` **does not stop execution** —
28 — Q30.

Q27. What are action selectors / action method selectors?

Attribute	Effect
<code>[HttpGet] , [HttpPost] , [HttpPut] , [HttpDelete] , [HttpPatch]</code>	Restrict by verb
<code>[AcceptVerbs("GET","POST")]</code>	Multiple verbs
<code>[ActionName("Report")]</code>	Decouple the routing name from the method name
<code>[NonAction]</code>	Public method that is <i>not</i> an endpoint

The classic use of `[ActionName]` :

```
[HttpGet] public IActionResult Delete(int id) => View();
[HttpPost, ActionName("Delete")]
public IActionResult DeleteConfirmed(int id) { db.Delete(id); return
RedirectToAction("Index"); }
```

"Two methods can't have the same name and signature, but both need to be reachable at `/Products/Delete`. `[ActionName]` solves it — this exact pattern is what MVC scaffolding generates."

Q28. Why must Delete be a POST?

"Because GET must be safe and idempotent. If deleting is a GET, a search-engine crawler, a browser prefetch, or an `` on any other site will silently delete your data — no user interaction needed. Destructive actions are `[HttpPost]` plus `[ValidateAntiForgeryToken]`."

A genuinely strong answer because it's a real vulnerability, not a convention.

Q29. How do you handle multiple submit buttons on one form?

```
<button type="submit" name="action" value="save">Save</button>
<button type="submit" name="action" value="delete">Delete</button>
```

```
[HttpPost]
public IActionResult Handle(Product p, string action)
    => action == "delete" ? Delete(p.Id) : Save(p);
```

"Simplest approach: give the buttons the same *name* and different *value*, and bind that name as a parameter. The alternative is *asp-page-handler* in Razor Pages, or a custom *ActionNameSelectorAttribute* if you want each button to route to its own action."

E — Views & Razor

Q30. What is Razor?

"Razor is the view engine — a templating syntax that mixes C# into HTML using *@*. The parser is smart enough to work out where code ends and markup begins, so you rarely need explicit delimiters. It compiles to a C# class, so views are strongly typed and compile-checked when you enable Razor compilation."

@model Product	@* the view's model type *@
@{ var count = Model.Items.Count; }	@* code block *@
@Model.Name	@* implicit expression *@
@(count + 1)	@* explicit expression *@
@if (count > 0) { In stock }	@* control flow *@
@foreach (var i in Model.Items) { @i }	
@@	@* literal @ *@
@* this is a Razor comment *@	

The two traps to mention unprompted: *@count + 1* renders *"5 + 1"* because the implicit expression ends at the space; and **Razor HTML-encodes everything**

by default, so `@Model.Name` is XSS-safe and `@Html.Raw` is the only way to opt out. Both are in [28 – Q44/Q45](#).

Q31. Strongly typed views vs `dynamic` – why does it matter?

```
@model ProductViewModel
<p>@Model.Name</p>           @* ✓ compile-checked, IntelliSense,
refactor-safe *@
<p>@ViewBag.Name</p>        @* ✗ a typo renders blank, silently *@
```

"A strongly typed view declares `@model`, so property access is compile-checked and refactoring renames it everywhere. `ViewBag` is `dynamic`, so a typo just renders an empty string with no error at all – the failure reaches the customer. I use `ViewBag` for something trivial like a page title and a view model for anything the view actually depends on."

Q32. HTML helpers vs tag helpers

```
@Html.TextBoxFor(m => m.Name, new { @class = "form-control" })
@Html.ValidationMessageFor(m => m.Name)

<input asp-for="Name" class="form-control" />
<span asp-validation-for="Name"></span>
```

	HTML helpers	Tag helpers
Looks like	C# method calls	HTML attributes
Designer/HTML-editor friendly	✗	✓
IntelliSense on attributes	Limited	✓
Available in	MVC 5 + Core	Core only
<code>class</code> attribute	<code>new { @class = ... }</code> (needs <code>@</code> escape)	just <code>class="..."</code>

"Tag helpers are preferred in Core — the markup stays valid HTML, so a front-end developer can work in it without knowing Razor. Both generate the same output and both read their value from ModelState first."

That last clause is the thing to say, because it sets up the biggest MVC view trap — 28 — Q43: after a POST, `asp-for / TextBoxFor` shows the **posted** value even if your code changed the model.

Q33. Explain `_Layout`, `_ViewStart`, `_ViewImports` and sections

File	Job
<code>_Layout.cshtml</code>	The site template — <code><html></code> , nav, footer; calls <code>@RenderBody()</code>
<code>_ViewStart.cshtml</code>	Runs before every view ; this is where <code>Layout = "_Layout";</code> is set. Cascades down folders
<code>_ViewImports.cshtml</code>	Shared <code>@using</code> , <code>@inject</code> , <code>@addTagHelper</code> — no output
<code>@RenderBody()</code>	Where the view's content is spliced in. Exactly once per layout
<code>@RenderSection("Scripts", required: false)</code>	Optional named slot the view can fill

Execution order — the counter-intuitive bit:

`_ViewStart.cshtml` → the VIEW (fully) → `_Layout.cshtml`

"The view executes first, completely, and its output is handed to the layout. So a view can set `ViewBag.Title` and the layout will see it — but not the other way round. That surprises people because the layout looks like the outer thing." (28 — Q48)

▲ **RenderSection** is required by default — a view that doesn't define the section throws. Pass `required: false`.

Q34. How does the view engine find a view?

```
View() in ProductsController.Details()
    → /Views/Products/Details.cshtml
    → /Views/Shared/Details.cshtml
    → (Razor Pages / area paths if configured)
```

"*View()* with no name uses the action name — not the method name, so `[ActionName]` changes what it looks for. If it can't find it, the exception helpfully lists every path it searched."

Q35. Partial view vs View Component vs child action

	Partial view	View Component
Has its own logic / DI	✗ — gets data from the parent	✓ — its own class with constructor injection
Can query a database	✗ (shouldn't)	✓
Invoked with	<code><partial name="_Row" model="x" /></code>	<code>@await Component.InvokeAsync("Cart")</code>
Async	✓ (<code>PartialAsync</code>)	✓ natively
Use for	Reusable markup	A self-contained widget

"A partial view is markup reuse — a row template, a form fragment. A view component is for something that needs its own data, like a shopping-cart summary or a nav menu driven by permissions: it has a class with DI, an `InvokeAsync` method, and its own view. In MVC 5 you'd have used `Html.Action` to call a child action for this; Core removed that and view components are the replacement."

That last sentence is a strong "what changed in Core?" data point.

Partial rendering options in Core:

```
<partial name="_Row" model="item" />           @* preferred *@
@await Html.PartialAsync("_Row", item)         @* async *@
@Html.Partial("_Row")                          @*
▲ analyzer-flagged: blocks on async *@
@{ Html.RenderPartial("_Row"); }               @* writes direct to output;
returns void *@
```

Q36. Why can't a partial view define a `@section`?

"Sections belong to the view↔layout relationship. A partial isn't rendered by a layout — it's rendered inline into another view — so there's no section slot for it to fill. It's either ignored or throws. If a partial needs a script, the parent view's `@section Scripts` has to carry it, or you use a view component."

Q37. What does `@inject` do?

```
@inject IProductService ProductService
@inject IStringLocalizer<Home> Localizer

<p>@ProductService.GetFeaturedCount()</p>
```

"It resolves a service from DI directly into the view. It's handy for genuinely view-level concerns like localization or feature flags — but I'd be careful with it, because pulling data in the view means queries execute during rendering, after the action has returned, where exception filters can't see them."

Q38. What is `IHtmlHelper.Raw`, and when is it acceptable?

"Razor encodes every `@expression`, so `<script>` in user data renders as visible text rather than executing. `Html.Raw` is the only way to opt out — which makes it the standard XSS vector in .NET apps. It's acceptable only for HTML you generated, or user HTML after sanitising it with something like `HtmlSanitizer`. `Html.Raw` in a pull request is an automatic review comment for me."

Q39. How do you build a reusable editor for a type?

```
/Views/Shared/EditorTemplates/DateTime.cshtml
/Views/Shared/DisplayTemplates/Money.cshtml
```

<code>@Html.EditorFor(m => m.StartDate)</code>	<code>@*</code> picks up
<code>DateTime.cshtml</code> automatically <code>*@</code>	
<code>@Html.EditorFor(m => m.Items)</code>	<code>@*</code> renders the template
once per item,	
	with CORRECT indexed
names <code>*@</code>	

"Editor and display templates are convention-based — a file named after the type or specified with `[UIHint]`. The underrated benefit is collections: `EditorFor` on a list generates properly indexed input names like `Items[0].Name`, which is exactly what model binding needs. Hand-rolling that inside a `foreach` is the usual cause of forms that post nothing back."

(The hand-rolled failure: [28 — Drill 7.](#))

F — Passing data to the view

Q40. ViewBag vs ViewData vs TempData vs Session

The table to memorize:

	Type	Lifetime	Survives redirect	Cast needed
ViewBag	dynamic	Current request	✗	No
ViewData	ViewDataDictionary (object)	Current request	✗	Yes
TempData	ITempDataDictionary (object)	Until read (≈ next request)	✓	Yes
Session	object / byte[]	Until it expires	✓	Yes
View model	Strongly typed	Current request	✗	No — compile-checked

Answer:

" *ViewBag* and *ViewData* are literally the same dictionary — *ViewBag* is just a *dynamic* wrapper over *ViewData* , so a value set through one is visible through the other. Both last one request. *TempData* is the one designed to cross a redirect, which is what makes the POST-Redirect-GET pattern work. *Session* is per-user and lasts until it expires."

The trade-off: "All four are untyped, so I use a view model for anything the view genuinely depends on and keep *ViewBag* for incidentals like a page title."

Q41. How does TempData actually work internally?

"It's backed by a provider — in Core the default is `CookieTempDataProvider`, an encrypted cookie; MVC 5 always used Session. The mechanic people get wrong is the deletion rule: reading marks an entry for deletion at the end of the current request — it isn't removed the instant you read it, and it isn't deleted by age. So you can read the same key twice in one request, and an entry nobody reads carries forward indefinitely."

```
TempData["Msg"];           // read AND mark for deletion
TempData.Peek("Msg");      // read WITHOUT marking → survives another
                             request
TempData.Keep("Msg");      // un-mark after reading → survives another
                             request
```

The Core-specific gotcha worth volunteering: *"With the cookie provider, values must be serializable to simple types — storing a `List<Product>` throws. And a cookie is capped around 4KB. So I store an id, not an object graph."*

Q42. Why doesn't ViewBag survive a redirect?

"Because a redirect is a 302 — the browser makes a completely new HTTP request. New request means a new controller instance and a new `ViewData` dictionary, so anything in `ViewBag` is gone. That's precisely the gap `TempData` exists to fill."

Q43. What's the single most common ViewBag bug you'd expect to see?

"A dropdown that works on GET and crashes on POST. The GET action populates `ViewBag.Categories` for the `<select>`; validation fails on POST, the action does `return View(model)` — and `ViewBag.Categories` was never repopulated, so

the view throws a `NullReferenceException`. Every `return View(model)` on a failure path has to repopulate the same data the GET populated. I'd factor that into a private method so it can't be forgotten."

This is the highest-signal answer in section F — it's a bug you only know about from building forms.

Q44. When would you use Session, and what's the cost?

"Session for genuinely per-user state that has to survive several requests — a multi-step wizard, a cart for an anonymous user. The costs are real: it needs `AddSession()` and `UseSession()` in the right order; it's untyped; it consumes server memory; and by default it's in-process, so it breaks the moment you scale to more than one instance behind a load balancer or the app pool recycles."

The fix to name: a distributed cache — `AddStackExchangeRedisCache` — so session survives restarts and works across instances. "Better still, avoid server session for anything that could live in a signed cookie or the database."

G — Model binding & validation

Q45. What is model binding, and where do values come from?

"Model binding maps values from the HTTP request onto your action parameters — it's why you can write `Save(Product p)` instead of reading `Request.Form` by hand. The default value providers are tried in a fixed order: Form → Route → Query string, first one holding the key wins."

Force a source explicitly:

`[FromForm]`, `[FromRoute]`, `[FromQuery]`, `[FromHeader]`, `[FromBody]`,
`[FromServices]`

"I prefer to be explicit. Implicit precedence is exactly the kind of thing that changes behaviour six months later when someone adds a query parameter with the same name."

Q46. Why is my model empty when I POST JSON?

"Because in a plain MVC controller a complex type binds from form values, not the JSON body. You need `[FromBody]` — or the `[ApiController]` attribute, which flips the default so complex types come from the body and simple types from the route/query. That one attribute is behind most 'why is my model null?' questions."

In **MVC 5** this confusion was structural: `System.Web.Mvc` (form-first) and Web API (body-first) were two separate stacks with different `Controller` base classes and different filter types. Core unified them.

Q47. What is over-posting, and how do you prevent it?

```
public class User { public string Name { get; set; } public bool IsAdmin { get; set; } }  
// attacker posts: Name=Ann&IsAdmin=true → IsAdmin binds. ♦
```

"The binder fills every public settable property it finds a matching key for — including ones your form never rendered. An attacker just adds the field by hand. The fix, best first: bind to a DTO or view model that physically doesn't have the field; or whitelist with `[Bind("Name")]`. I'd never bind a form directly to an EF entity."

Blacklisting (`[Bind(Exclude=...)]`) is weakest — a property added later is unprotected by default.

Q48. How does validation work end to end?

```
public class Product
{
    [Required(ErrorMessage = "Name is required")]
    [StringLength(100, MinimumLength = 3)]
    public string Name { get; set; }

    [Range(0.01, 10000)]
    public decimal Price { get; set; }

    [EmailAddress] public string ContactEmail { get; set; }
    [Compare("Password")] public string ConfirmPassword { get; set; }
    [RegularExpression(@"^\d{6}$")] public string Pincode { get; set; }
}
```

```
[HttpPost]
public IActionResult Save(Product p)
{
    if (!ModelState.IsValid) return View(p);    // redisplay with
messages
    db.Add(p); db.SaveChanges();
    return RedirectToAction(nameof(Index));    // POST-Redirect-GET
}
```

```
<span asp-validation-for="Name" class="text-danger"></span>
<div asp-validation-summary="ModelOnly"></div>
```

Answer:

"Data annotations on the model are read during model binding and populate `ModelState`. The action checks `ModelState.IsValid`, and on failure returns the same view so the user sees their input plus the messages. On success it redirects, so refresh doesn't re-post."

Three things to volunteer that most candidates miss:

1. **A binding failure does not stop the action.** Posting `abc` to an `int` gives you `0` and an invalid `ModelState` — but the action still runs. If you don't check, you process garbage. (28 — Q14)

2. **Client validation is UX only.** It's `data-val-*` attributes plus jQuery unobtrusive validation. Anyone can POST with curl, so the server check is the real one.
 3. **[Required] on a non-nullable int can't catch 0** — `Required` only checks null. Use `[Range(1, ...)]`, or make it `int?`.
-

Q49. How do you write a custom validation attribute?

```
public class FutureDateAttribute : ValidationAttribute
{
    protected override ValidationResult IsValid(object value,
        ValidationContext ctx)
    => value is DateTime d && d > DateTime.UtcNow
        ? ValidationResult.Success
        : new ValidationResult($"{ctx.DisplayName} must be in the
future");
}
```

And for a rule that spans two properties, `IValidatableObject`:

```
public class Booking : IValidatableObject
{
    public DateTime Start { get; set; }
    public DateTime End { get; set; }

    public IEnumerable<ValidationResult> Validate(ValidationContext ctx)
    {
        if (End <= Start)
            yield return new ValidationResult("End must be after start",
new[] { nameof(End) });
    }
}
```

"A *ValidationAttribute* for a rule about one property, *IValidatableObject* for a rule across properties. *IValidatableObject* only runs after all the attribute-level validations pass, which is usually what you want."

Q50. What is **[Remote]** validation?

```
[Remote(action: "IsEmailAvailable", controller: "Account")]
public string Email { get; set; }

public IActionResult IsEmailAvailable(string email)
    => Json(!db.Users.Any(u => u.Email == email));
```

"It validates against the server without a full postback — the classic 'is this username taken?' check. Two caveats: it's an extra round trip on every field change, so debounce it; and it's not a substitute for the server-side check on submit, because of the race between checking and saving. I'd still have a unique constraint in the database."

Q51. How do you return validation errors from an API?

```
[ApiController]                                // ← automatic 400 with
ProblemDetails
[Route("api/[controller]")]
public class ProductsController : ControllerBase { }
```

*"**[ApiController]** installs a filter that returns a 400 with an RFC 7807 **ProblemDetails** body automatically, before the action runs — so an **if (!ModelState.IsValid)** inside the action is dead code. You can customise the shape with **ApiBehaviorOptions.InvalidModelStateResponseFactory**, or suppress it if you want full control."*

H — Filters

Q52. What are filters, and what types are there?

"Filters let you run code at specific stages of the MVC pipeline — they're the cross-cutting-concern mechanism, the equivalent of a Spring *HandlerInterceptor*. There are five types, and they run in a fixed order."

Type	Runs	Use for
Authorization	First, before everything	Permission checks
Resource	Around model binding	Caching, short-circuiting
Action	Around the action method	Logging, auditing, timing
Exception	On unhandled exceptions	Turning errors into friendly results
Result	Around result execution	Adding response headers

Authorization → Resource(before) → [MODEL BINDING] → Action(before) → ACTION → Action(after) → Result(before) → [VIEW RENDERS] → Result(after) → Resource(after)

Two things to volunteer:

- "Authorization is first because there's no point binding a model for a request you're about to reject."
- "Exception filters cover controller creation, binding, action filters and the action — but not result execution, so an exception while the view renders escapes them. That needs *UseExceptionHandler* middleware."

MVC 5 also had an **Authentication** filter type; Core moved authentication into middleware.

Q53. In what order do global, controller and action filters run?

IN: Global → Controller → Action
ACTION METHOD
OUT: Action → Controller → Global

*"They nest like Russian dolls — reverse order on the way out. And **Order** beats scope: filters are sorted by the **Order** property first, which defaults to 0 for all three scopes, and scope only breaks the tie. So setting **Order = -1** on an action filter makes it run before a global one."*

Q54. Write a custom action filter

```
public class LogActionFilter : IActionFilter
{
    private readonly ILogger<LogActionFilter> _logger;
    public LogActionFilter(ILogger<LogActionFilter> logger) => _logger =
        logger;

    public void OnActionExecuting(ActionExecutingContext ctx)
        => _logger.LogInformation("Executing {Action}",
            ctx.ActionDescriptor.DisplayName);

    public void OnActionExecuted(ActionExecutedContext ctx)
        => _logger.LogInformation("Executed, exception: {Ex}",
            ctx.Exception?.Message ?? "none");
}
```

Registering it — three scopes:

```
builder.Services.AddControllersWithViews(o =>
o.Filters.Add<LogActionFilter>()); // global
[ServiceFilter(typeof(LogActionFilter))]
// controller/action, with DI
public class HomeController : Controller { }
```

"If the filter needs dependencies injected, register it in DI and apply it with `[ServiceFilter]` or `[TypeFilter]` — a plain attribute can't take constructor injection, because attribute arguments must be compile-time constants."

Two traps worth naming:

- "Adding a filter instance (`o.Filters.Add(new LogFilter())`) makes it a singleton shared across all requests — any mutable field in it is a data race. Use the generic form."
- "Setting `context.Result` in `OnActionExecuting` short-circuits: the action is skipped, and so is that filter's own `OnActionExecuted`."

Q55. How do you handle exceptions globally?

Three layers, and knowing which to use is the question:

```
// 1. MIDDLEWARE – catches everything, including view rendering. The
outermost net.
app.UseExceptionHandler("/Home/Error");

// 2. EXCEPTION FILTER – MVC-aware; has ActionContext. Doesn't cover
result execution.
public class ApiExceptionHandler : IExceptionHandler
{
    public void OnException(ExceptionContext ctx)
    {
        ctx.Result = new ObjectResult(new { error = "Something went
wrong" }) { StatusCode = 500 };
        ctx.ExceptionHandled = true;
    }
}

// 3. try/catch – only where you can actually recover or add context
```

"Middleware for the safety net, because it wraps the whole pipeline. A filter when I need MVC context — which action, which model — to shape the response, typically to return JSON for an API instead of an HTML error page. And local `try/catch` only where I can genuinely do something about it. What I don't do is catch, log and rethrow at every layer — that's just noise in the log."

▲ `ExceptionHandler = true` without setting `Result` gives the client an empty 200 — a silent failure worse than the 500. Always set both.

Q56. Give a real example of when you'd write each filter type

Need	Filter
Only tenant admins may reach this controller	Authorization
Return a cached response and skip the action entirely	Resource
Audit-log every action's arguments and duration	Action
Convert a <code>DomainException</code> into a 422 with a message	Exception
Add <code>X-Correlation-Id</code> to every response	Result
Rate-limit <i>all</i> requests, including static files	Middleware, not a filter

That last row is the one that separates people who've read the docs from people who've shipped.

I — Dependency injection & middleware

Q57. Explain the three DI lifetimes

	Instance per	Use for
Transient	Every injection	Cheap, stateless services
Scoped	HTTP request	<code>DbContext</code> , repositories, per-request context
Singleton	Application lifetime	Caches, config, expensive stateless clients

```
builder.Services.AddTransient<IEmailFormatter, EmailFormatter>();  
builder.Services.AddScoped<IProductRepository, ProductRepository>();  
builder.Services.AddSingleton<ICacheService, CacheService>();
```

The follow-up that always comes: "Which for `DbContext` ?"

"Scoped — and `AddDbContext` registers it as scoped by default. You get one unit of work and one change tracker per request, which is exactly the EF model. Singleton would be a disaster: `DbContext` isn't thread-safe and the change tracker would grow forever. Transient would break transactions spanning two repositories, because each would get its own context."

Q58. What is a captive dependency?

```
builder.Services.AddScoped<IRepository, Repository>();  
builder.Services.AddSingleton<ICacheService, CacheService>(); // ←  
injects IRepository
```

▲ `InvalidOperationException: Cannot consume scoped service 'IRepository' from singleton 'ICacheService'.`

"A singleton outlives every request, so if it captures a scoped service it keeps the first request's instance alive forever — a stale `DbContext` shared by every user. Core's scope validation catches it at startup, but only in the Development environment by default, so in production the app would start happily and misbehave. I'd turn `ValidateScopes` and `ValidateOnBuild` on explicitly."

The fix: inject `IServiceScopeFactory` and create a scope per unit of work — the same pattern you need for a `DbContext` inside a background `IHostedService`.

Q59. What is middleware, and how is it different from a filter?

```
app.Use(async (ctx, next) =>
{
    // runs on the way IN
    await next();
    // runs on the way OUT
});
```

A in → B in → endpoint → B out → A out

	Middleware	Filter
Scope	Every request, including static files	MVC actions only
Knows about	<code>HttpContext</code>	Action, arguments, model, <code>ModelState</code>
Registered	<code>app.UseX()</code> in <code>Program.cs</code>	Globally / on a controller / on an action
Short-circuit	Don't call <code>next()</code>	Set <code>context.Result</code>

"Middleware is the outer, framework-agnostic pipeline — logging, HTTPS redirect, static files, auth, CORS. Filters are MVC-aware and can see the action and the model. Rule of thumb: if it must run for non-MVC requests too, it's middleware; if it needs to know which action is running, it's a filter. In Spring terms, middleware is a servlet `Filter` and a filter is a `HandlerInterceptor`."

Q60. Why does middleware order matter? Give the standard order.

```
app.UseExceptionHandler("/Error"); // outermost – must wrap everything
below
app.UseHsts();
app.UseHttpsRedirection();
app.UseStaticFiles(); // before routing – short-circuits
for files
app.UseRouting(); // ← SELECTS the endpoint
app.UseCors();
app.UseAuthentication(); // who are you?
app.UseAuthorization(); // are you allowed? ← must be AFTER
UseRouting
app.MapControllers(); // ← EXECUTES the endpoint
```

The two rules to state:

*" **UseAuthorization** must come after **UseRouting**, because it needs to know which endpoint was selected in order to read its **[Authorize]** metadata — put it before and you either get an exception or, worse, authorization silently doesn't run. And **UseExceptionHandler** goes first so it wraps everything downstream."*

The trade-off worth volunteering: " **UseStaticFiles** before **UseAuthorization** means everything in **wwwroot** is public. That's the right default for CSS and images, but files that need protecting have to go through a controller."

Q61. How do you write custom middleware?

```
public class RequestTimingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<RequestTimingMiddleware> _logger;

    public RequestTimingMiddleware(RequestDelegate next,
        ILogger<RequestTimingMiddleware> logger)
        => (_next, _logger) = (next, logger);

    public async Task InvokeAsync(HttpContext ctx)
    {
        var sw = Stopwatch.StartNew();
        await _next(ctx);
        _logger.LogInformation("{Path} took {Ms}ms", ctx.Request.Path,
            sw.ElapsedMilliseconds);
    }
}

app.UseMiddleware<RequestTimingMiddleware>();
```

The gotcha to mention: *"The middleware class itself is effectively a singleton — constructor dependencies must be singletons. For a scoped service, take it as a parameter on `InvokeAsync` instead, which is resolved per request."*

```
public async Task InvokeAsync(HttpContext ctx, IMyScopedService svc)
{ }    // ✓
```

J — Security

Q62. How do you prevent CSRF?

<code><form asp-action="Save" method="post"></code>	@* tag helper injects the token
<code>automatically *@</code>	
<code>@Html.AntiForgeryToken()</code>	@* explicit form, MVC 5 style
<code>*@</code>	
<code></form></code>	

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Save(Product p) { }

[AutoValidateAntiforgeryToken] // ← better: applies to ALL
unsafe verbs
public class ProductsController : Controller { }
```

"CSRF is when another site makes the user's browser send an authenticated request to yours — the cookie goes along automatically. The defence is a token in two places: a cookie and a hidden form field, which must match. The attacker's site can cause the cookie to be sent but can't read or forge the form field, because the same-origin policy stops it."

Two things to add:

- "In Core, `<form asp-action>` injects the token automatically — hand-written `<form>` tags are the usual cause of the resulting 400."
- "For AJAX, configure a header name (`o.HeaderName = "X-CSRF-TOKEN"`) and send it from JavaScript. And `SameSite=Lax` cookies are a good defence in depth, not a replacement."

Q63. How do you prevent XSS and SQL injection?

XSS:

"Razor HTML-encodes every `@expression` by default, so the common case is safe for free. The risk is `Html.Raw`, and `@Html.Raw(userInput)` is a live vulnerability — I'd only use it on HTML I generated or user HTML that's been through a sanitiser. A Content-Security-Policy header is the second layer."

SQL injection:

"EF Core parameterises everything by default, including interpolated `FromSqlInterpolated`. The way people still get hit is string-concatenating into `FromSqlRaw` or `ExecuteSqlRaw`."


```
db.Users.FromSqlRaw($"SELECT * FROM Users WHERE Name = '{name}'"); //
◆ injectable
db.Users.FromSqlInterpolated($"SELECT * FROM Users WHERE Name =
{name}"); // ✓ parameterised
db.Users.Where(u => u.Name ==
name); // ✓ best
```

Q64. Authentication vs authorization — and how are they done in Core?

"Authentication establishes who you are; authorization decides what you're allowed to do. In Core they're separate middleware in that order, and `[Authorize]` is what enforces the second."

Scheme	Use for
Cookie authentication	Traditional server-rendered MVC apps
ASP.NET Core Identity	Full user store — registration, password hashing, 2FA, lockout
JWT bearer	APIs and SPAs (your Angular front-end)
OpenID Connect / OAuth 2	External providers — Azure AD, Google, Auth0

```
[Authorize] // any authenticated user
[Authorize(Roles = "Admin,Manager")] // role-based
[Authorize(Policy = "MinimumAge18")] // policy-based (preferred)
[AllowAnonymous] // overrides everything, at
any scope
```

Three authorization models:

- **Role-based** — simple, but roles multiply badly
- **Claims-based** — checks a claim's value
- **Policy-based** — *"the one I'd default to; it's a named rule you can change centrally instead of hunting `Roles = "..."` strings through the codebase"*

```
builder.Services.AddAuthorization(o =>
    o.AddPolicy("MinimumAge18", p => p.RequireClaim("Age", "18")));
```

▲ **The trap:** " `[AllowAnonymous]` always wins — it overrides even a global authorization policy. One left on a controller after debugging opens the whole thing, and no global setting will save you. It's a standard code-review grep."

Q65. What is CORS and when do you need it?

"CORS is a browser rule: JavaScript on `site-a.com` can't read a response from `site-b.com` unless the server explicitly allows it. It matters to me directly — an Angular app on `localhost:4200` calling an API on `localhost:5000` is cross-origin, so without CORS the browser blocks it."

```
builder.Services.AddCors(o => o.AddPolicy("Spa", p => p
    .WithOrigins("https://myapp.com")           // ▲ not AllowAnyOrigin in
production
    .AllowAnyHeader()
    .AllowAnyMethod()
    .AllowCredentials()));

app.UseCors("Spa");                               // after UseRouting, before
UseAuthorization
```

Two things to volunteer: "CORS is enforced by the browser, not the server — it's not a security boundary against curl or Postman. And `AllowAnyOrigin` combined with `AllowCredentials` is rejected outright, because it would let any site read authenticated responses."

Q66. How would you secure a file upload?

```
[HttpPost]
[RequestSizeLimit(5_000_000)]
public async Task<IActionResult> Upload(IFormFile file)
{
    var safeName = $"{Guid.NewGuid()}_{Path.GetExtension(file.FileName)}";
    var path = Path.Combine(_uploadRoot, safeName);
    using var stream = System.IO.File.Create(path);
    await file.CopyToAsync(stream);
    return Ok();
}
```

"Four things. Never trust `file.FileName` — it can contain `..\..\` and traverse out of your folder, so I generate my own name and keep the original only as metadata. Limit the size, or you have a denial-of-service. Validate the content type and the actual file signature, not just the extension. And store uploads outside `wwwroot`, because anything in `wwwroot` is publicly downloadable and an uploaded `.html` or `.js` becomes stored XSS."

K — EF Core, performance & testing

Q67. How does Entity Framework Core fit into MVC?

```
builder.Services.AddDbContext<AppDbContext>(o =>
o.UseSqlServer(builder.Configuration.GetConnectionString("Default")));    /
/ scoped

public class ProductsController : Controller
{
    private readonly AppDbContext _db;
    public ProductsController(AppDbContext db) => _db = db;

    public async Task<IActionResult> Index()
        => View(await _db.Products.AsNoTracking().ToListAsync());
}
```

"EF Core is the ORM — it's to .NET what Hibernate/JPA is to Spring. *DbContext* is registered scoped, so it's one unit of work per request, and it's injected into the controller. Code-first with migrations is the usual workflow: change the model, *dotnet ef migrations add*, *dotnet ef database update*."

Cross-reference [24 — ORM, JPA & Hibernate](#) — the concepts map directly: persistence context $\approx$ change tracker, *LazyInitializationException* $\approx$ *ObjectDisposedException* after the scope closes, and N+1 is N+1 in both.

Q68. Do you need a repository pattern on top of EF Core?

A deliberately opinionated question. Have a position.

"Often not. *DbSet<T>* is already a repository and *DbContext* is already a unit of work, so a generic *IRepository<T>* wrapping them usually just adds a layer that leaks *IQueryable* anyway. I'd add a repository when I want to hide EF from the domain layer, or to make a specific set of queries mockable in tests. What I'd avoid is a generic repository per entity added reflexively because a tutorial said so."

Having a **position with a caveat** is what a senior answer sounds like. Agreeing with whatever the interviewer implies is not.

Q69. What is the N+1 problem and how do you fix it?

```
var orders = db.Orders.ToList();                // 1 query
foreach (var o in orders)
    Console.WriteLine(o.Customer.Name);         // + N queries

var orders = db.Orders.Include(o => o.Customer).ToList(); // ✓ 1 query
// with a JOIN
```

"Fetching a list and then touching a navigation property per row fires one query per row. The fixes are eager loading with *Include*, projection to a DTO with *Select* so you only fetch the columns you need, or split queries when a big *Include*

produces a cartesian explosion. Projection is usually the best of the three, because it also stops you over-fetching."

```
var dto = await db.Orders
    .Select(o => new OrderDto { Id = o.Id, CustomerName =
o.Customer.Name })
    .AsNoTracking()
    .ToListAsync();
```

Q70. How would you improve the performance of an MVC application?

Answer in layers — this structure is what makes it sound experienced:

Layer	What I'd do
Measure first	Profile before guessing — MiniProfiler, Application Insights, EF logging. <i>"I wouldn't optimise anything I hadn't measured."</i>
Database	Fix N+1 with <code>Include</code> /projection, <code>AsNoTracking()</code> for read-only queries, index the columns you filter on, paginate instead of <code>ToList()</code> on everything
Async	<code>async</code> / <code>await</code> all the way down — it doesn't make one request faster, it frees the thread so the server handles more concurrent requests
Caching	<code>IMemoryCache</code> for per-instance, <code>IDistributedCache</code> /Redis across instances, <code>[ResponseCache]</code> / output caching for whole responses
Payload	Return DTOs not entities, enable response compression, paginate APIs
Front-end	Bundle and minify, <code>asp-append-version</code> for cache-busting, CDN for static assets

Layer	What I'd do
Views	Avoid queries in views; prefer view components with their own cached data

The sentence that lands: *"The single biggest win in almost every app I've seen is the data layer — an N+1 or a missing index costs more than every micro-optimisation combined."*

Q71. How do you cache in ASP.NET Core?

Type	API	Scope
In-memory	<code>IMemoryCache</code>	One server instance
Distributed	<code>IDistributedCache</code> (Redis, SQL)	All instances
Response caching	<code>[ResponseCache]</code> + middleware	Client/proxy via HTTP headers
Output caching (.NET 7+)	<code>app.UseOutputCache()</code>	Server-side full responses

```
public async Task<List<Category>> GetCategoriesAsync()
    => await _cache.GetOrCreateAsync("categories", async entry =>
    {
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10);
        return await _db.Categories.AsNoTracking().ToListAsync();
    });
```

The trade-off to volunteer: *"`IMemoryCache` is per-instance, so behind a load balancer users get inconsistent data and a cache invalidation only clears one server. That's when you move to Redis. And every cache needs an expiry — an unbounded cache is a memory leak."*

MVC 5 used `[OutputCache]`, which Core replaced with `[ResponseCache]` (header-driven) and later true server-side output caching.

Q72. How do you test an MVC application?

```
// UNIT TEST – the controller is just a class
[Fact]
public async Task Index_ReturnsViewWithProducts()
{
    var mockService = new Mock<IProductService>();
    mockService.Setup(s => s.GetAllAsync())
        .ReturnsAsync(new List<Product> { new() { Name =
"Pen" } });

    var controller = new ProductsController(mockService.Object);
    var result = await controller.Index();

    var view = Assert.IsType<ViewResult>(result);
    var model = Assert.IsAssignableFrom<List<Product>>(view.Model);
    Assert.Single(model);
}
```

```
// INTEGRATION TEST – the real pipeline, in memory
public class ProductsApiTests :
    IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;
    public ProductsApiTests(WebApplicationFactory<Program> f) => _client =
        f.CreateClient();

    [Fact]
    public async Task Get_ReturnsSuccess()
    {
        var response = await _client.GetAsync("/api/products");
        response.EnsureSuccessStatusCode();
    }
}
```

*"Unit tests for controllers and services — this is the payoff for returning result objects instead of writing to the response: I can assert on a **ViewResult** with no HTTP involved. Integration tests with **WebApplicationFactory**, which spins up the real app in memory so routing, filters, binding and DI all actually run — that's where you catch the middleware-order and binding bugs a unit test can't see."*

Tools to name: xUnit (or NUnit/MSTest), **Moq** for mocking, **FluentAssertions** for readability, **WebApplicationFactory** for integration, EF Core InMemory or Testcontainers for the database.

◆ **Bridge to your experience:** "Same split as JUnit plus Mockito for units and **@SpringBootTest** with MockMvc for integration — **WebApplicationFactory** is the direct equivalent of **@SpringBootTest**."

L — Answer scripts

Script 1 — "Tell me about yourself" (for a .NET role, 60 seconds)

"I'm Priyanka, a full-stack developer with 5 years' experience. My front-end is Angular — Angular 16 and 18 with RxJS, NgRx and Ionic, including a hybrid mobile app.

On the backend I've worked across two stacks. On Subsea, a marine equipment and operations platform, the backend was a .NET Web API with Angular in front, in Docker on Azure — I owned the Schedule-Manager module end to end, both the Angular feature module and the .NET endpoints behind it. My other backend experience is Java with Spring Boot — REST APIs, dependency injection, layered architecture and JPA.

Having done both, what strikes me is how closely they map — filters are Spring's interceptors, middleware is the servlet filter chain, EF Core sits where JPA does. My .NET work has been API-side rather than Razor MVC, since the UI was always Angular; and that's the part I've been deliberately closing.

I'm looking for a role where I can keep doing full-stack work — Angular on the front, .NET on the back — which is exactly the shape Subsea had."

Why this works: real shipped .NET first, the second stack second, the honest boundary (API not Razor) third and briefly, and it closes by naming the role shape you've already succeeded in. It never apologises and never overclaims.

Script 2 — Project walkthrough, translated

For a .NET role, lead with Subsea — it's the .NET one. Don't default to RoboGebra out of habit.

"Subsea is a marine equipment and operations management platform — Angular SPA, .NET Web API services, Docker, Azure. I owned the Schedule-Manager module front to back. The part I'd highlight is a performance problem: the Kendo grids were client-bound, so as the schedule datasets grew the browser was loading everything and the grid got sluggish. I moved paging, sorting and filtering server-side using Kendo's `DataSourceRequest`, which translates straight into SQL paging, so only one page crossed the wire. Combined with virtual scrolling and trimming the columns to what we actually display, the grid stayed responsive at scale. I also built the dynamic category/subcategory hierarchy as cascading selects driven from the API rather than hardcoded, and refactored the repeated grid configuration into shared components so other modules weren't copy-pasting Kendo setup."

Then, if the second project comes up, translate as you go:

"RoboGebra is an education platform — Angular and Ionic on the front, Spring Boot services on the back, MongoDB for storage. In .NET terms that architecture would be an ASP.NET Core Web API with controllers and DTOs, EF Core where we had the ODM layer, JWT bearer authentication where we used tokens, and the cross-cutting logging we did with interceptors would be an action filter or middleware."

◆ **Say the translation out loud** for the Java project — it demonstrates the mapping instead of asserting it. But lead with Subsea: for a .NET interviewer, a shipped .NET project beats any amount of mapping.

Script 3 — When you genuinely don't know

"I haven't used that specific feature. My understanding is it does X — in Spring the equivalent is Y — but I'd want to confirm that rather than guess. How is it used in your codebase?"

Three things at once: honesty, demonstrated reasoning, and turning it into a conversation. **Never bluff a .NET version number or an API signature** — it's checkable in seconds and it costs you the round.

Script 4 — Questions to ask them

- *"Is the codebase on .NET Framework or .NET 6/8? Is there a migration in progress?"* — tells you which half of this file matters, and shows you know they're different
- *"MVC with Razor views, or an API with a separate SPA front-end?"* — lets you steer toward your Angular strength
- *"How is the team structured — is the front-end and back-end the same people?"*
- *"What does the testing and code-review process look like?"*
- *"What would the first three months look like for someone joining?"*

M — Rapid-fire table

Cover the right column. Target: **under 6 minutes**, daily.

#	Question	Answer
1	What is MVC?	Model = data + rules, View = UI, Controller = handles request, picks result

#	Question	Answer
2	Why MVC over Web Forms?	Testable, clean URLs, full HTML control, stateless, fits SPAs
3	Model vs view model vs DTO	Entity / what one view needs / what crosses a boundary
4	MVC vs Razor Pages	Controller-centric vs page-centric; same underlying stack
5	.NET Framework vs .NET 5+	Windows-only legacy vs cross-platform unified; even versions are LTS
6	Biggest MVC 5 → Core changes	Cross-platform, built-in DI, MVC+API unified, middleware, <code>appsettings.json</code>
7	What is Kestrel?	Core's built-in cross-platform web server; reverse proxy in front
8	What is <code>wwwroot</code> ?	The only publicly served folder — and it's not covered by <code>[Authorize]</code>
9	Request lifecycle	Middleware → routing → filters → binding → action → result → response
10	Where does the view render?	Result execution — after the action returns
11	<code>AddX</code> vs <code>UseX</code>	Register a service vs run it in the pipeline
12	Config sources	<code>appsettings</code> → <code>appsettings.{Env}</code> → user secrets → env vars → CLI

#	Question	Answer
13	Conventional vs attribute routing	Central pattern vs <code>[Route]</code> on the action
14	<code>{id?}</code> + <code>int id</code>	Route optional ≠ parameter optional → MVC5 throws, Core binds 0
15	Route constraint fails	404 — matching, not validation
16	What are Areas?	Sub-app folders (Admin) with their own controllers/views
17	Generate URLs how?	<code>Url.Action</code> / <code>asp-action</code> tag helper — never hard-code
18	What makes a method an action?	It's public on a controller — use <code>[NonAction]</code> or private
19	<code>Controller</code> vs <code>ControllerBase</code>	Views vs API-only
20	<code>IActionResult</code> vs <code>ActionResult<T></code>	Flexible vs flexible and self-documenting
21	Why return a result object?	Testable, composable; result filters run in between
22	<code>[ActionName]</code> use	Two reachable actions at the same URL (Delete/ DeleteConfirmed)
23	Why must Delete be POST?	GET must be safe — crawlers and <code></code> would delete data
24	What is Razor?	View engine mixing C# into HTML with <code>@</code> ; compiles to a class
25	Strongly typed vs ViewBag	Compile-checked vs a typo that renders blank

#	Question	Answer
26	HTML vs tag helpers	Method calls vs HTML-looking attributes; tag helpers are Core-only
27	<code>_ViewStart</code> / <code>_Layout</code> / <code>_ViewImports</code>	Pre-view setup / template / shared usings
28	Layout execution order	View first , then layout — view can set ViewBag for the layout
29	<code>RenderBody</code> vs <code>RenderSection</code>	Once, required / named slot, optional with <code>required:false</code>
30	Partial vs view component	Markup reuse vs a widget with its own logic and DI
31	<code>@section</code> in a partial	Doesn't work — sections are view↔layout
32	Editor templates	Convention-based; give collections correct indexed names
33	ViewBag vs ViewData	The same dictionary — dynamic wrapper vs <code>object</code>
34	TempData lifetime	Deleted on read , not by age; <code>Peek</code> / <code>Keep</code> to preserve
35	TempData storage in Core	Encrypted cookie by default — simple types only, ~4KB
36	ViewBag across a redirect	Gone — 302 means a new request
37	#1 real ViewBag bug	Dropdown data not repopulated before <code>return View(model)</code>

#	Question	Answer
38	Session cost	Needs <code>AddSession</code> + <code>UseSession</code> ; in-process by default → breaks when scaled
39	Model binding order	Form → Route → Query
40	JSON posts as empty model	Complex types bind from form — add <code>[FromBody]</code> or <code>[ApiController]</code>
41	Over-posting	Binder sets fields your form never rendered → bind to a DTO
42	Binding failure	Sets default + ModelState error — the action still runs
43	<code>[Required]</code> on <code>int</code>	Can't catch 0 — use <code>[Range]</code> or <code>int?</code>
44	Client validation	UX only — <code>data-val-*</code> + jQuery unobtrusive; server is the real check
45	Cross-property validation	<code>InvalidatableObject</code>
46	<code>[ApiController]</code> + invalid model	Automatic 400 before the action runs
47	Five filter types	Authorization, Resource, Action, Exception, Result
48	Filter scope order	Global → Controller → Action in; reversed out; <code>Order</code> beats scope
49	What exception filters miss	Result execution — a view exception needs middleware
50	Filter with dependencies	<code>[ServiceFilter]</code> / <code>[TypeFilter]</code> — attributes can't take DI

#	Question	Answer
51	Short-circuit a filter	Set <code>context.Result</code> — also skips its own <code>OnActionExecuted</code>
52	DI lifetimes	Transient = per injection, Scoped = per request, Singleton = per app
53	Lifetime for <code>DbContext</code>	Scoped — not thread-safe, tracker would grow forever
54	Captive dependency	Singleton holding a scoped service; validated in Dev only
55	Middleware vs filter	Every request vs MVC-aware; servlet Filter vs HandlerInterceptor
56	<code>UseAuthorization</code> position	After <code>UseRouting</code> , before <code>MapControllers</code>
57	Custom middleware DI	Class is a singleton — take scoped services on <code>InvokeAsync</code>
58	CSRF defence	Token in a cookie and a form field; <code>[ValidateAntiForgeryTokenTo ken]</code>
59	XSS defence	Razor auto-encodes; <code>Html.Raw</code> is the vector; add CSP
60	SQL injection in EF	Only via string-concatenated <code>FromSqlRaw</code>
61	AuthN vs AuthZ	Who you are vs what you may do; separate middleware, in that order
62	Authorization models	Role / claims / policy (preferred)

#	Question	Answer
63	[AllowAnonymous]	Always wins , overrides global policies
64	What is CORS?	Browser rule for cross-origin JS; not a defence against curl
65	Secure file upload	Don't trust FileName , limit size, check signature, store outside wwwroot
66	EF Core in MVC	ORM; DbContext scoped, injected; code-first + migrations
67	Repository over EF?	Often unnecessary — DbSet is a repo, DbContext is a UoW
68	N+1 fix	Include , projection with Select , or split queries
69	Performance priorities	Measure → data layer → async → cache → payload → front-end
70	Caching options	IMemoryCache / IDistributedCache / [ResponseCache] / output caching
71	IMemoryCache limitation	Per-instance — inconsistent behind a load balancer
72	Testing an MVC app	xUnit + Moq for units; WebApplicationFactory for integration

How to answer in the room

- 1. Three sentences, always.** Definition → one concrete thing (from your project or a named example) → one trade-off or failure mode. Your rounds have been lost on the **second** question, not the first — the trade-off sentence *is* the second question, answered before it's asked.
- 2. Locate the question in the lifecycle.** *"That's a model-binding question, so we're after routing and before the action."* Naming the stage is the skill; the fact is secondary.
- 3. Say "Core or Framework?" when it genuinely matters.** Not dodging — it's the correct question, and interviewers read it as experience.
- 4. Use the Spring mapping out loud.** *"In Spring this is a `HandlerInterceptor`; here it's an action filter — same position in the pipeline."* That single sentence converts four years of Java into four years of relevant experience.
- 5. Never bluff a version number or an API signature.** Checkable in seconds. *"I'd confirm that rather than guess"* costs you nothing; a confident wrong answer costs you the round.

Related files in this pack

- **28 — MVC "What Happens?" (62 programs)** — the code round. **Do this file first, then 28.**
- **27 — C# / .NET "Predict the Output" (75 programs)** — the language round
- **06 — Spring & Spring Boot** — read alongside § 0's mapping table
- **24 — ORM, JPA & Hibernate** — EF Core is the same concepts: change tracker, N+1, lazy loading
- **17 — SOLID & Design Patterns** — filters are Chain of Responsibility; DI is Dependency Inversion
- **00 — Self Introduction** — adapt with Script 1 above for .NET roles
- **26 — Companies: Questions Actually Asked** — **log every .NET round here the same day**

Study order: § 0 (the honesty position) → the ★ 15 most-likely → sections B (lifecycle) and F (ViewBag/TempData), which are the two most-asked areas → then G, H, I. Run the rapid-fire table daily under 6 minutes. **Then move to file 28** for the code half — theory gets you through the first twenty minutes, the programs decide the round.

● Mphasis — Level 2 + Client Technical Round

✓ **You cleared Level 1.** The 6 Aug 2026 round (10 backend questions) went well enough that they're scheduling you forward. That is the first time in six companies you've been moved to a *next* round by a tier-1 service company. Read that as evidence, not luck.

◆ **What HR told you:**

"Java only mostly. With Angular, what you have used in recent projects they'll ask. Basics of Angular not mandatory."

This file decodes that sentence, then prepares both remaining rounds.

Read alongside: [26 — Companies: Questions Actually Asked § Mphasis](#) — the 10 questions they already asked you. **L2 interviewers read the L1 feedback form.** They will re-probe those exact threads, one level deeper. That section is not revision — it's the syllabus.

Table of contents

Part	What it covers
0 — Decoding the HR message	What "Java only mostly" and "basics not mandatory" actually mean
A — Level 2: the re-probes	The follow-up to each of the 10 L1 questions
B — Level 2: Java core depth	Collections, HashMap internals, equals/hashCode, immutability, Streams

Part	What it covers
C — Level 2: multithreading & concurrency	● The biggest section — they asked 2 of 10 L1 questions here. The two already asked, answered deeper; locks, <code>volatile</code> / atomics, thread pools, <code>CompletableFuture</code>, concurrent collections, <code>ThreadLocal</code>, threading inside Spring Boot, virtual threads, output programs
D — Level 2: Spring & Spring Boot depth	Bean lifecycle, <code>@Transactional</code> , AOP, auto-configuration, REST design
E — Level 2: JPA / Hibernate	N+1, lazy loading, caching, locking
F — Level 2: microservices, one level down	Feign vs WebClient, gateway, config server, tracing, JWT
G — SQL: your live gap	The queries that get asked, plus indexes and ACID
H — Live coding	What they'll make you type
I — System design, lite	The 5-minute answer structure
J — Angular, the way THEY will ask it	Project-anchored, not theory
K — The client technical round	● Different round, different game. 17 questions in 3 buckets — project deep dive · scenario/troubleshooting · client interaction & delivery (the bucket candidates skip)
L — Logistics & fit questions	Notice period, location, why leaving
M — Your 48-hour plan	What to do with the time you have
N — One-page cheat sheet	Print this

0 — Decoding the HR message

"Java only mostly"

Level 2 is a **backend depth round**. L1 was breadth — ten topics, one question each, mostly definitions. L2 at Mphasis is where they pick **three or four** of those and push until you stop knowing. Expect fewer questions, longer each, with the interviewer following your answer wherever it goes.

The single biggest change from L1 to L2: L1 rewards *knowing*. L2 rewards *having done*. Every answer needs a project sentence in it.

"With Angular, what you have used in recent projects they'll ask"

● **This is the most useful thing HR said.** It tells you the *form* of the Angular questions:

✗ What they will NOT ask	✓ What they WILL ask
"What are Angular lifecycle hooks?"	"Walk me through a component you built recently."
"What is BehaviorSubject?"	"How did the dashboard get its data?"
"Explain lazy loading."	"How did you structure the RoboGebra app — modules? routing?"
"What is a reactive form?"	"You mention Formly on EasyVisa — what was that for?"

So the preparation is **not** flashcards. It's being able to narrate three features you built, with the Angular decisions inside the narration. → [Part J](#)

"Basics of Angular not mandatory"

Two readings, and you must prepare for both:

- **The generous reading (likely correct):** they won't fail you on Angular theory. The role is Java-weighted.

• **The trap reading:** "not mandatory" ≠ "not asked." If you answer "*I built the dashboard in Angular*" and they say "*how did you handle the subscription cleanup?*" and you blank, that's a bad moment — because **you claimed it on your resume**. Anything you say you built, you own the details of.

◆ **The rule:** don't study Angular broadly. Study **your own Angular** narrowly and completely.

A — Level 2: the re-probes of your L1 answers

They have your L1 sheet. For each of the 10 questions you were asked, here is the **next** question. This is the highest-yield section in the file.

L1 question (already asked ✓)	● The L2 re-probe
1. What is microservices?	"How would you split RoboGebra into services? Where do you draw the boundary?"
2. Design patterns in microservices	"Explain Saga with a concrete failure. What compensates what?"
3. @CircuitBreaker	"What values did you set for the threshold and why? What happens to in-flight requests when it opens?"
4. Circuit-breaker dependency	"Why does AOP matter here? What else in Spring breaks the same way?"
5. Thread-safe Singleton	"Now make it a Spring bean. Is a Spring singleton the same as a Java singleton?"
6. Actuator	"How do you secure it in production? What do you actually monitor?"
7. Association vs aggregation	"Show me composition in your own codebase. Why not inheritance?"
8. SQL group-by query	"Add an index. How would you know the query is slow?"

L1 question (already asked ✓)	● The L2 re-probe
9. Deadlock	"You have a hung production app. Walk me through diagnosing it."
10. API performance	"Give me a number. What was slow, what did you change, what did it become?"

Now the answers to the five that are hardest.

A1. "How would you split RoboGebra into services?"

Don't invent an architecture. Describe yours, then reason about it.

"RoboGebra runs as separate Spring Boot services — the split is by business capability. User/Auth owns accounts and JWT issuing. Content owns the study lists and question bank. Quiz owns attempts and scoring. Reminders owns the scheduling and notification dispatch. AI Explanation wraps the math explanation engine, because it's the only service with a completely different scaling profile — it's slow, it's expensive per call, and it fails differently from everything else."

"The boundary test I'd use is: does this thing change for its own reasons, and does it own data nobody else writes? Reminders qualifies — its schema is scheduling data, and a change to reminder logic never touches quiz scoring. What I'd resist splitting is Quiz and Content, because a quiz attempt constantly needs question data; that split would turn one query into a chatty network call for every question."

● The senior close:

"And I'd be honest that the AI service is the one that justified the split on its own. If I were starting over with a small team I'd build a modular monolith with those boundaries as packages, and extract the AI service first — because you can always split a well-structured monolith, but you can't easily un-split a distributed system."

A2. "Explain Saga with a concrete failure"

Use an order flow — it's the canonical example and it's clean.

Orchestrated saga – Order across 3 services

1. Order Service

→ CREATE order (status = PENDING)
2. Payment Service

→ charge card

✓
3. Inventory Service

→ reserve stock

✗ OUT OF STOCK

Compensate, in reverse:

4. Payment Service

→ REFUND the charge

← compensating txn
5. Order Service

→ status = CANCELLED

"The key point is that a compensating transaction is not a rollback — the payment genuinely happened and is visible in the ledger. You're issuing a new business transaction that semantically undoes it. That's why compensations have to be designed per step, and why some steps are hard to compensate — you can't un-send an email, so you send a correction."

The three details that separate you from a memorised answer:

Detail	Say this
Idempotency	"Every step must be idempotent, because the orchestrator will retry. I'd key each step by the saga id so a repeated 'reserve stock' doesn't reserve twice."
Isolation	"Sagas have no isolation — mid-saga, another transaction can read the half-done state. You handle it with a status field like PENDING that the rest of the system knows not to trust."
Choreography vs orchestration	"Choreography = services react to each other's events; less coupling, but nobody can answer 'where is this order?'. Orchestration = a coordinator drives it; one more component, but the flow is explicit and traceable. For anything with money in it I choose orchestration."

A5. ● "Is a Spring singleton the same as a Java singleton?"

No — and this is a favourite L2 question because most people say yes.

	Java singleton (GoF)	Spring singleton scope
Scope of uniqueness	One per classloader / JVM	One per Spring container
Enforced by	Private constructor	The container's bean registry
Can you make another?	No (that's the point)	✓ Yes — <code>new MyBean()</code> works fine
Two contexts in one JVM	Still one instance	Two instances , one per context
Lifecycle	Yours	Container's — <code>@PostConstruct</code> / <code>@PreDestroy</code>
Testability	Poor — global state, hard to mock	Good — inject a mock

"A Spring singleton is a container-scoped singleton, not a JVM-wide one. The class has a public constructor; nothing stops you calling `new`. If an app has two `ApplicationContext`s — which happens in tests, or with a parent/child web context — you get two instances of the 'singleton'."

▲ The follow-up that always comes next: "What's the danger of singleton beans?"

"Singleton beans are shared across all request threads, so any mutable instance field is a race condition. A `private List<String> cache` on a `@Service` is a bug in production and works perfectly on your laptop. Keep beans stateless — state lives in method parameters or local variables."

▲ And the one after that: "How do you inject a prototype bean into a singleton?"

"Naively it doesn't work — the prototype is injected once, at singleton creation, so you get the same instance forever. The fixes are `@Lookup` method injection,

injecting an `ObjectProvider<T>` and calling `getObject()` per use, or a scoped proxy with `@Scope(value = "prototype", proxyMode = TARGET_CLASS)` ."

```
@Service
public class ReportService {

    @Autowired
    private ObjectProvider<ReportBuilder> builderProvider;    // ✓ fresh
    one per call

    public Report build() {
        return builderProvider.getObject().build();
    }
}
```

A9. ● "Production app is hung. Walk me through diagnosing it."

This is a **scenario question**, and scenario questions are the whole game in L2 and the client round. Answer as a procedure, not a fact.

"First I'd confirm what 'hung' means — is it all requests, or one endpoint? Is CPU at 0% or 100%? 0% CPU with requests piling up says lock contention or an exhausted thread pool. 100% CPU says an infinite loop or GC thrashing. That one observation splits the problem in half."

"Then I take a thread dump — `jstack <pid>` , or `/actuator/heapdump` if it's exposed. Take three, ten seconds apart: if the same threads are in the same stack frames across all three, they're genuinely stuck, not just busy. The JVM detects Java-monitor deadlocks itself, so `jstack` prints a `Found one Java-level deadlock` section naming the two threads and the locks."

"What I usually find in a Spring Boot app isn't a classic deadlock — it's the connection pool. All HikariCP connections are held by threads waiting on a slow downstream call, so every new request blocks on `getConnection()` . The stack trace shows threads parked in `HikariPool.getConnection` . The real fix isn't a bigger pool — it's a timeout on the downstream call, which is exactly what the circuit breaker from L1 Q3 is for."

Symptom	Likely cause	Tool
0% CPU, requests queue	Deadlock / pool exhaustion	<code>jstack</code> , <code>/actuator/</code> <code>threaddump</code>
100% CPU, no progress	GC thrashing or infinite loop	<code>jstat -gc</code> , GC logs, heap dump
Memory climbing, then OOM	Leak — unbounded cache/ collection	heap dump + Eclipse MAT
Slow but working	N+1 queries, missing index	slow query log, <code>EXPLAIN</code> , tracing

● **Close by connecting it back:** *"And the reason I'd want Actuator and distributed tracing already in place is precisely this moment — diagnosing it after the fact with no instrumentation is guesswork."*

A10. ● "Give me a number" — API performance

You answered this at L1 in general terms. At L2 they want **one concrete before/after**. Pick one and rehearse it with numbers.

The RoboGebra dashboard story:

"The learning dashboard was making one call per widget — progress, streak, recommended list, recent quizzes — four sequential round trips from a mobile client, and it felt slow on first load. Two changes: on the backend I added an aggregate endpoint so the dashboard is one call, and on the frontend I moved the remaining parallel calls into `forkJoin` instead of nested subscribes. The backend query itself had an N+1 — the study-list fetch was lazily loading each item — so I added a `JOIN FETCH`. First paint went from roughly two seconds to under half a second on the same device."

The structure to reuse for any perf answer:

- | | |
|---|----------------------------|
| ① What was slow, and how did you KNOW | → measurement, not vibes |
| ② What was the actual cause
index / no cache | → N+1 / round trips / no |
| ③ What you changed | → specific |
| ④ The number | → before → after |
| ⑤ What you'd do next if it grew | → caching, pagination, CDN |

The backend/frontend split they asked at L1 — have both halves ready:

Backend	Frontend
Fix N+1 (<code>JOIN FETCH</code> , <code>@EntityGraph</code>)	Lazy-load routes; smaller initial bundle
Add the right index; check <code>EXPLAIN</code>	<code>OnPush</code> change detection
Pagination instead of returning everything	<code>trackBy</code> on <code>*ngFor</code>
Cache (<code>@Cacheable</code> / Redis) for hot reads	<code>debounceTime</code> + <code>switchMap</code> on search
Return a DTO, not the whole entity graph	Cache HTTP responses; <code>shareReplay</code>
Async the non-critical work (<code>@Async</code> , queue)	Image sizing, compression, CDN
Connection-pool sizing, gzip, HTTP keep-alive	<code>forkJoin</code> instead of sequential calls

B — Level 2: Java core depth

L1 didn't test core Java at all — no collections, no OOP internals, no streams.
That is exactly why L2 will.

B1. ● HashMap internals — the most-asked L2 Java question anywhere

"A `HashMap` is an array of buckets. On `put` , it takes the key's `hashCode()` , applies a spreading function — $h \wedge (h \ggg 16)$ — to mix the high bits down, then indexes with `hash & (n-1)` , which works because the capacity is always a power of two. If the bucket is empty it stores the node; if not, it walks the chain comparing `hash` first and then `equals` . Since Java 8, once a bucket has 8 entries and the table is at least 64 slots, that chain converts to a red-black tree, so worst-

case lookup goes from $O(n)$ to $O(\log n)$ — that was the fix for hash-collision DoS attacks."

Property	Value
Default capacity	16
Load factor	0.75 → resizes at 12 entries
Resize	Doubles capacity, rehashes
Treeify threshold	8 (and table ≥ 64)
Untreeify	6
Null keys	✓ one, always in bucket 0
Thread-safe?	✗ — use <code>ConcurrentHashMap</code>

▲ "What if you use a mutable object as a key?"

"You lose the entry. The hash is computed at insert time and stored in the node. Mutate a field that `hashCode()` depends on and the key now hashes to a different bucket — `get()` looks in the new bucket and finds nothing, but `size()` still counts it. It's a silent memory leak. Keys should be immutable; that's why `String` and the wrappers are the usual choice."

▲ "What breaks if you override `equals` but not `hashCode`?"

"Two objects that are `equals` can land in different buckets, so a `HashMap` treats them as different keys and a `HashSet` accepts both as distinct. The contract is: equal objects must have equal hash codes. The reverse isn't required — unequal objects may collide, that's fine."

```
// The contract, in one block
a.equals(b)  ⇒ a.hashCode() == b.hashCode()    // REQUIRED
a.hashCode() == b.hashCode() ⇒/ a.equals(b)    // NOT required
(collision)
```

B2. Collections — the comparison table they ask from

	ArrayLi st	LinkedL ist	HashMa p	LinkedH ashMap	TreeMa p	HashSe t	Concurr entHash Map
Backing	Array	Doubly- linked list	Bucket array	Buckets + linked list	Red- black tree	HashMap	Bucket array + CAS
Get by index / key	O(1)	O(n)	O(1) avg	O(1)	O(log n)	O(1)	O(1)
Insert at end	O(1) amortise d	O(1)	—	—	—	—	—
Insert in middle	O(n) shift	O(1) <i>if you have the node</i>	—	—	—	—	—
Order	Insertion	Insertion	✗ none	✓ insertion or access	✓ sorted	✗ none	✗ none
Nulls	✓	✓	1 key, many values	✓	✗ null key	✓ one	✗ neither
Thread- safe	✗	✗	✗	✗	✗	✗	✓

▲ **"ArrayList vs LinkedList — which do you use?"** → "**ArrayList** , almost always. **LinkedList** 's O(1) insert assumes you already hold the node — via an iterator. If you're calling **add(index, e)** , it still walks there in O(n), and it loses on cache locality and memory (an object header plus two pointers per element). The honest answer is that **LinkedList** is rarely the right choice outside of a **Deque** ."

▲ **Hashtable vs ConcurrentHashMap** → "**Hashtable** synchronizes every method on the whole map, so it's one thread at a time. **ConcurrentHashMap** locks per bucket (CAS plus a **synchronized** on the bin head since Java 8) so reads are

lock-free and writes only contend on the same bucket. `Hashtable` is legacy — never use it."

▲ **fail-fast vs fail-safe** → "`ArrayList`'s iterator is fail-fast: it holds a `modCount` and throws `ConcurrentModificationException` if the list is structurally modified during iteration — including by your own code inside the loop. `ConcurrentHashMap`'s iterator is fail-safe: it iterates a snapshot view and never throws, but may not reflect the newest writes. To remove during iteration, use `iterator.remove()` or `removeIf`."

B3. `String` — pool, immutability, and the output trap

```
String a = "java";
String b = "java";
String c = new String("java");
String d = c.intern();

a == b           // true   - both point at the SAME pooled literal
a == c           // false  - new String() forces a fresh heap object
a.equals(c)      // true   - same characters
a == d           // true   - intern() returns the pooled reference
```

"String literals live in the string pool, which since Java 7 sits in the heap, not PermGen. Identical literals are the same object. `new String("java")` deliberately creates a second object — which is why `==` on strings is a bug and `.equals` is the answer."

▲ **"Why is `String` immutable?"** → "Four reasons: the pool only works if nobody can mutate a shared literal; the hash is cached, which makes `String` a fast `HashMap` key; it's inherently thread-safe; and security — a filename or connection URL validated and then mutated by another thread would be a real vulnerability."

▲ **`StringBuilder` vs `StringBuffer`** → "Same API. `StringBuffer` is synchronized, `StringBuilder` isn't. Use `StringBuilder` — the synchronization is almost never useful, since a builder is nearly always local to one method. Concatenating in a loop with `+` creates a new `String` each iteration — $O(n^2)$; that's the classic answer."

B4. Immutability — how to write an immutable class

```
public final class Money {                                // ① final – can't be
    subclassed

    private final String currency;                        // ② all fields final +
private
    private final List<String> tags;

    public Money(String currency, List<String> tags) {
        this.currency = currency;
        this.tags = new ArrayList<>(tags);                // ③ DEFENSIVE COPY in
    }

    public List<String> getTags() {
        return List.copyOf(tags);                          // ④ DEFENSIVE COPY out
    }
    // ⑤ no setters
}
```

● "Points ③ and ④ are the ones people miss — without them the caller still holds a reference to the internal list and can mutate it. And in Java 17, a *record* gives you final fields, a constructor, *equals* / *hashCode* / *toString* for free — but records are only shallowly immutable, so a record holding a *List* still needs a defensive copy in a compact constructor."

■ More Java 17: [12 — Java 17 Features](#)

B5. Streams — what they'll ask beyond "what is a stream"

Question	The answer in one line
Intermediate vs terminal	Intermediate returns a stream and is lazy ; terminal triggers execution. No terminal op = nothing runs.
map vs flatMap	map is 1→1; flatMap is 1→many, flattening nested streams (List<List<T>> → List<T>).
findFirst vs findAny	Same in sequential; in parallel findAny returns whichever finishes first — faster, non-deterministic.

Question	The answer in one line
Can you reuse a stream?	<code>x IllegalStateException: stream has already been operated upon or closed</code> .
<code>Collectors.toMap</code> danger	Throws on duplicate keys unless you pass a merge function; also NPEs on a null value, unlike <code>HashMap</code> .
Parallel streams — when?	Large data, CPU-bound, no shared mutable state. They use the common <code>ForkJoinPool</code> , so one slow parallel stream can starve every other one in the JVM . Rarely worth it in a web app where the container already gives you thread-per-request.
<code>reduce</code> vs <code>collect</code>	<code>reduce</code> for immutable accumulation (sum); <code>collect</code> for mutable containers (list, map) — far more efficient for building collections.

```
// The three they ask you to write live
Map<String, List<Employee>> byDept =
    emps.stream().collect(groupingBy(Employee::getDept));

Map<String, Double> avgByDept =
    emps.stream().collect(groupingBy(Employee::getDept,
    averagingDouble(Employee::getSalary)));

Optional<Employee> topPaid =
    emps.stream().max(Comparator.comparingDouble(Employee::getSalary));

// Highest-paid per department — the Altimetrik question, and the SQL
RANK() question in Java form
Map<String, Optional<Employee>> topPerDept =
    emps.stream().collect(groupingBy(Employee::getDept,
    maxBy(Comparator.comparingDouble(Employee::getSalary))));
```

■ 22 — Java Streams Coding Problems — type these, don't read them.

B6. Exceptions

	Checked	Unchecked
Extends	Exception	RuntimeException
Compiler forces handling	✓	✗
Examples	IOException , SQLException	NullPointerException , IllegalArgumentException on
Use for	Recoverable, caller can act	Programming errors

- ▲ " throw vs throws , final / finally / finalize " — quick-fire, know them cold.
- ▲ "Can finally be skipped?" → "Yes — System.exit() , JVM crash, or the thread being killed. Also, a return inside finally swallows an exception thrown in try , which is a real bug. Prefer try-with-resources."
- ▲ "Custom exception in Spring Boot?" → @ControllerAdvice + @ExceptionHandler returning a ProblemDetail /error DTO with the right status.
- Say this — it connects core Java to the framework, which is what L2 rewards.

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse>
    handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse("NOT_FOUND", ex.getMessage(),
Instant.now()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class) // @Valid
failures
    public ResponseEntity<ErrorResponse>
    handleValidation(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldErrors().stream()
            .map(f -> f.getField() + ": " + f.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return ResponseEntity.badRequest().body(new
ErrorResponse("VALIDATION_ERROR", msg, Instant.now()));
    }
}

```

■ Predict-the-output practice: **23 — Java Output Tricky Questions**. L1 didn't ask any — but Codeboard did, and a second interviewer may. Do 15 of them the night before.

C — Level 2: multithreading & concurrency

● **Treat this as the most important technical section in the file.** Mphasis asked **two** multithreading questions at L1 out of ten — "*write a thread-safe Singleton*" and "*what is deadlock*". That's 20% of the round on concurrency. **An interviewer who asks two multithreading questions at L1 is a concurrency interviewer,** and L2 will go further, not elsewhere.

Sub-section	What
C0	● The two they already asked — deeper, with every follow-up
C1	Creating threads · the lifecycle
C2	<code>synchronized</code> and the monitor
C3	<code>volatile</code> vs <code>synchronized</code> vs Atomic
C4	<code>synchronized</code> vs <code>ReentrantLock</code>
C5	<code>ExecutorService</code> and thread pools
C6	<code>CompletableFuture</code>
C7	Concurrent collections
C8	<code>CountDownLatch</code> , <code>Semaphore</code> , producer-consumer
C9	<code>ThreadLocal</code> — and the leak
C10	● Multithreading inside Spring Boot
C11	Virtual threads (Java 21)
C12	Predict-the-output programs
C13	Rapid-fire table

C0 — the two questions they already asked, answered deeper

● Asked at L1: *"Write a program for a thread-safe Singleton class"*

You answered this. **L2 will not ask it again — it will ask what's underneath it.**
 There are five follow-ups, and they get progressively harder.

The answer itself, one more time, in its best form:

```
public class ConfigManager {

    private ConfigManager() { }

    private static class Holder {                                // not
loaded until first use
        private static final ConfigManager INSTANCE = new ConfigManager();
    }

    public static ConfigManager getInstance() {
        return Holder.INSTANCE;
    }
}
```

"Bill Pugh / static holder. The inner class isn't initialised until `getInstance()` first touches it, so it's lazy; and the JVM guarantees class initialisation is thread-safe because the classloader holds a lock during `<clinit>`, so it's thread-safe with zero synchronization on every subsequent call."

▲ Follow-up 1 — "Why exactly is double-checked locking broken without `volatile`?"

This is the single most likely L2 concurrency question, because it's the one that separates people who memorised the pattern from people who understand the memory model.

```
instance = new Singleton();
```

"That one line is three bytecode-level steps:

- 1. allocate memory for the object,*
- 2. run the constructor to initialise the fields,*
- 3. assign the reference to `instance`.*

The JVM and the CPU are both allowed to reorder 2 and 3, because within a single thread the result is indistinguishable — that's the 'as-if-serial' rule. But another thread doing the first, unsynchronized `if (instance == null)` check can now see a non-null reference to a half-constructed object — fields still at their defaults.

It uses it, and you get a null field or a zero where there shouldn't be one, intermittently, usually in production and never on your laptop."

*"**volatile** fixes it two ways: it forbids that reordering (it inserts memory barriers around the write), and it guarantees the write is visible to other threads immediately rather than sitting in a CPU cache. This is why DCL was genuinely, famously broken in Java 1.4 — the pre-Java-5 memory model didn't give **volatile** those guarantees. Java 5's JSR-133 fixed the model, not the pattern."*

```
public class Singleton {
    private static volatile Singleton instance;    //
    • remove `volatile` and this is subtly broken

    public static Singleton getInstance() {
        if (instance == null) {                    // ① cheap check, no
lock – the fast path
            synchronized (Singleton.class) {
                if (instance == null) {            // ② the real check,
under the lock
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

▲ **"Why two checks?"** → *"The outer one is a performance optimisation — once initialised, 99.99% of calls skip the lock entirely. The inner one is the correctness check — two threads can both pass the outer check simultaneously, and only one may create the instance."*

▲ Follow-up 2 — "What is the happens-before relationship?"

The concept underneath all of it. Be able to say this:

"Happens-before is the JVM's ordering guarantee: if action A happens-before action B, then everything A did is visible to B. Without such a relationship, there is

no guarantee at all that one thread ever sees another thread's write — the JIT and the CPU cache are both free to hide it indefinitely."

The relationships worth naming:

Rule	Guarantee
Program order	Within one thread, statements happen-before later ones
Monitor lock	Unlocking a monitor happens-before any later lock of the <i>same</i> monitor
<code>volatile</code>	A write to a volatile happens-before every later read of it
<code>Thread.start()</code>	Everything before <code>start()</code> happens-before the new thread's first action
<code>Thread.join()</code>	Everything in the thread happens-before <code>join()</code> returns
<code>final</code> fields	Correctly-constructed final fields are visible without synchronization

● "The practical consequence: `synchronized` isn't only about mutual exclusion — it's also about visibility. Two threads that never actually run at the same time can still see stale data if neither synchronizes."

▲ Follow-up 3 — "Can your singleton be broken?" (you have this from L1 — keep it)

Attack	Bill Pugh / DCL	enum
Reflection (<code>setAccessible(true)</code>)	✓ broken	✗ JVM forbids reflective enum construction
Serialization	✓ broken (new instance on deserialize)	✗ safe
Cloning	✓ if <code>Cloneable</code>	✗ safe

Attack	Bill Pugh / DCL	enum
Two classloaders	✓ broken	✓ broken

```
public enum ConfigManager {           // ✓ safe against all three, for free
    INSTANCE;
    public String get(String key) { return props.get(key); }
}
```

▲ Follow-up 4 — ● "Now make it a Spring bean. Is that the same thing?"

No. → the full answer is in **A5**. Container-scoped, not JVM-scoped; public constructor; two contexts give two instances.

▲ Follow-up 5 — "Is your singleton thread-safe once it's created?"

● **This one catches almost everyone.** They're testing whether you understand that *creating* the instance safely says nothing about *using* it safely.

```
public class CounterSingleton {
    private static class Holder { static final CounterSingleton I = new
CounterSingleton(); }
    public static CounterSingleton getInstance() { return Holder.I; }

    private int count = 0;                               // ● shared mutable state

    public void increment() { count++; }                  // ✗ NOT thread-safe –
read-modify-write
}
```

"The construction is thread-safe. The object isn't. Every thread in the application shares this one instance, so any mutable field on it is contended. `count++` is three operations — read, add, write — and two threads interleaving lose an increment. The fixes are `AtomicInteger`, a `synchronized` method, or best of all making the singleton stateless so the question doesn't arise. That's exactly the rule for Spring `@Service` beans too."

● Asked at L1: "What is deadlock in multithreading?"

You gave the definition and the code. **Here is everything they can ask next.**

▲ **Follow-up 1 — "How would you detect it in a running system?"** → the full procedure is in **A9**. Short version: three thread dumps ten seconds apart; `jstack` prints a `Found one Java-level deadlock:` section naming both threads and both locks.

```
Found one Java-level deadlock:
=====
"T1":  waiting to lock monitor 0x00007f9 (object 0x000000076ab, a
      java.lang.Object),
      which is held by "T2"
"T2":  waiting to lock monitor 0x00007f8 (object 0x000000076aa, a
      java.lang.Object),
      which is held by "T1"
```

▲ **The honest caveat that earns credit:** "`jstack` only detects deadlocks on intrinsic monitors and `ReentrantLock`. A deadlock built out of `Semaphore`s, or a logical one where two threads are waiting on each other's `Future`, won't be reported — you have to read the stacks yourself."

▲ **Follow-up 2 — "Can you get a deadlock with a database?"**

● **Say yes — this is the version that actually happens in the projects you've worked on.**

```
-- Transaction A          -- Transaction B
UPDATE account SET ... WHERE id=1;  UPDATE account SET ... WHERE id=2;  --
each holds a row lock
UPDATE account SET ... WHERE id=2;  UPDATE account SET ... WHERE id=1;  --
▲  each wants the other's
```

"Exactly the same circular wait, but on database row locks instead of Java monitors. The difference is that the database detects it and picks a victim — MySQL/InnoDB rolls one transaction back with error 1213, `Deadlock found when trying to get lock`. So it doesn't hang forever, it fails one transaction. The fix is the same principle: touch rows in a consistent order — for a transfer, always lock

the lower account id first — plus a retry on the deadlock error, since it's transient by nature."

▲ **Follow-up 3 — "Show me the fix"** — both, in order of preference:

```
// ✓ FIX 1 – GLOBAL LOCK ORDERING. Break the circular-wait condition.
public void transfer(Account from, Account to, BigDecimal amount) {
    Account first = from.getId() < to.getId() ? from : to;    //
    ◆ always the same order
    Account second = from.getId() < to.getId() ? to : from;

    synchronized (first) {
        synchronized (second) {
            from.debit(amount);
            to.credit(amount);
        }
    }
}
```

▲ *"And if the two ids can be equal — a self-transfer — that's a re-entrant lock on the same object, which is fine in Java because **synchronized** is reentrant. But it's worth guarding explicitly anyway."*

```
// ✓ FIX 2 – tryLock with timeout. Break the no-preemption condition.
if (lockA.tryLock(1, TimeUnit.SECONDS)) {
    try {
        if (lockB.tryLock(1, TimeUnit.SECONDS)) {
            try { /* work */ }
            finally { lockB.unlock(); }
        } else {
            // back off, release everything, retry with a randomised delay
        }
    } finally { lockA.unlock(); }           // ● unlock in finally, ALWAYS
}
```

● **The random back-off matters:** *"If two threads both fail, both release and both retry immediately at the same interval, you get a livelock — they keep colliding politely forever. Randomised back-off breaks the symmetry."*

▲ **Follow-up 4 — "What are the other concurrency hazards?"** — know all four by name, they're a common quick-fire:

Hazard	What it is	Symptom
Deadlock	Circular wait on locks	Hangs forever, 0% CPU
Livelock	Threads keep reacting to each other, no progress	100% CPU, nothing completes
Starvation	A thread never gets the resource — low priority, or unfair locks	One thread hangs, others fine
Race condition	Result depends on interleaving	Wrong data, intermittent, unreproducible

"The distinction I'd draw is that a deadlock gives you no result, and a race condition gives you a wrong result — which is worse, because nothing looks broken."

▲ **Follow-up 5 — "Where have you actually hit a concurrency issue?"**

Have an honest answer ready. If you haven't hit a classic deadlock, say what you have hit:

"I haven't debugged a textbook two-lock deadlock in production. What I have dealt with is the shape that actually shows up in a Spring Boot app — the reminder scheduler running on more than one instance, where a `@Scheduled` job on two replicas picked up the same due reminders and sent duplicate notifications. That's a race on shared rows rather than a lock cycle, and the fix was making the claim atomic — a conditional update so only one instance can transition a row from PENDING to SENDING, `SELECT ... FOR UPDATE SKIP LOCKED` being the other option — plus making the send idempotent so a retry can't double-deliver."

● **That answer is much stronger than inventing a deadlock story**, because it's specific, it's yours, and it shows you understand that in real systems the concurrency is *distributed*, not just in-JVM.

C1. Creating threads & the lifecycle

Four ways, and which one to say you use:

```
// ① extend Thread – rarely, because it burns your one inheritance slot
class Worker extends Thread { public void run() { } }
new Worker().start();

// ② implement Runnable – the classic answer
Runnable r = () -> System.out.println("work");
new Thread(r).start();

// ③ Callable + ExecutorService – returns a value AND can throw a checked
exception
Future<Integer> f = pool.submit(() -> 42);

// ④ CompletableFuture – composable, non-blocking → C6
```

"*Runnable* over extending *Thread*, because Java has single inheritance and a task isn't a kind of thread — it's a job that a thread runs. But in real code I don't create threads directly at all; I submit tasks to an *ExecutorService*."

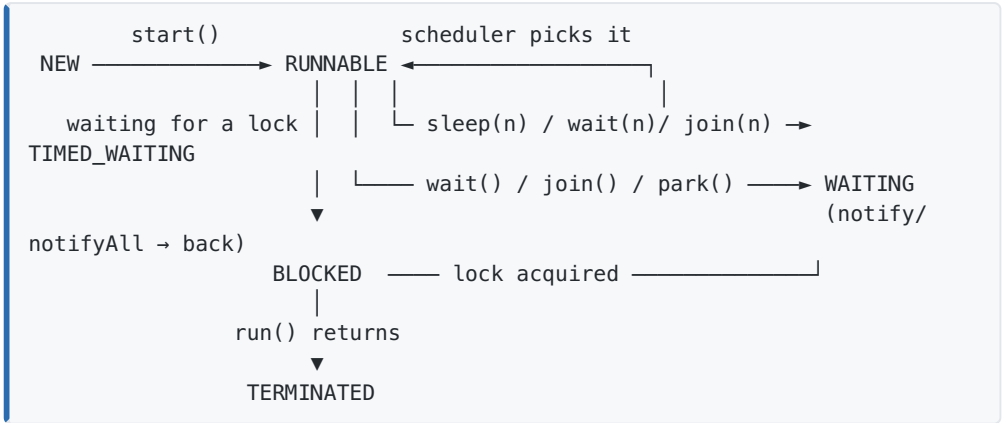
▲ **Runnable** vs **Callable** — asked constantly:

	Runnable	Callable<T>
Method	<code>void run()</code>	<code>T call()</code>
Returns a value	✗	✓
Can throw a checked exception	✗	✓
Since	1.0	1.5

▲ **start()** vs **run()** — ● the classic trick question:

"*start()* asks the JVM to create a new OS thread which then calls *run()*. Calling *run()* directly is just an ordinary method call on the current thread — the code executes, everything looks fine, and there is no concurrency at all. And calling *start()* twice throws *IllegalThreadStateException*: a thread is not reusable, which is precisely the reason thread pools exist."

The lifecycle — **Thread.State**:



State	Meaning	Where you see it in a thread dump
NEW	Created, <code>start()</code> not called	—
RUNNABLE	Running or ready to run	Normal work
BLOCKED	● Waiting to enter a <code>synchronized</code> block	Lock contention / deadlock
WAITING	<code>wait()</code> , <code>join()</code> , <code>LockSupport.park()</code> — no timeout	Waiting on a condition
TIMED_WAITING	Same, with a timeout	<code>sleep</code> , <code>poll(timeout)</code> , <code>Future.get(timeout)</code>
TERMINATED	Finished	—

● " **BLOCKED** versus **WAITING** is the distinction that matters when you read a thread dump. **BLOCKED** means it wants a monitor someone else holds — many **BLOCKED** threads on the same lock is contention, and a cycle of them is a deadlock. **WAITING** usually means the thread is idle by design, like a pool thread waiting on its queue."

Daemon threads: " `setDaemon(true)` before `start()` . The JVM exits when only daemon threads remain — so background workers are daemons, but anything that must finish its work must not be, or it gets killed mid-flight at shutdown."

Interruption — the right way to stop a thread:

```
// ✗ Thread.stop() is deprecated and unsafe – it can leave objects half-
// modified.
// ✓ Cooperative interruption:
while (!Thread.currentThread().isInterrupted()) {
    try {
        queue.take(); // blocking calls throw
        InterruptedException
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); // ● RESTORE the flag –
        catching it clears it
        break;
    }
}
```

● "Catching *InterruptedException* clears the interrupt flag. If you swallow it without restoring, code further up the stack never learns the thread was asked to stop — the classic 'why won't this app shut down' bug. Either rethrow it or restore the flag."

C2. *synchronized* — the monitor

"Every Java object has an intrinsic lock, a monitor. *synchronized* acquires it on entry and releases it on exit — including when an exception is thrown, which is one of its advantages over an explicit lock."

```
public synchronized void m() { } // locks on `this`
public static synchronized void s() { } //
● locks on MyClass.class – a DIFFERENT lock
public void block() {
    synchronized (lockObject) { } // locks on a specific object –
    preferred
}
```

● **The trap they ask:** "An instance method and a static method that are both *synchronized* on the same class do not exclude each other — one holds the instance monitor, the other holds the class monitor. Two different locks. People assume *synchronized* is one global thing and it isn't."

Why a private final lock object beats `synchronized(this)` :

"`synchronized` on `this` means the lock is public — any outside code holding a reference to your object can lock on it and block your methods. A `private final Object lock = new Object()` is encapsulated. Same reason you don't synchronize on a `String` literal: it's pooled, so unrelated classes may share the same monitor."

Reentrancy: "Intrinsic locks are reentrant — a thread that holds a lock can acquire it again, which is what makes a synchronized method calling another synchronized method on the same object safe rather than an instant self-deadlock."

`wait()` / `notify()` — and the two rules:

```
synchronized (lock) {
    while (!condition) {           // ● while, NOT if
        lock.wait();              // releases the monitor and parks
    }
    // condition is true and we hold the lock
}

synchronized (lock) {
    condition = true;
    lock.notifyAll();             // ● notifyAll, not notify
}
```

Rule	Why
<code>while</code> , not <code>if</code>	Spurious wakeups are permitted by the spec, and another thread may consume the condition between the notify and your re-acquiring the lock
<code>notifyAll</code> , not <code>notify</code>	<code>notify</code> wakes one arbitrary waiter — if it's waiting on a different condition, the right thread is never woken and you get a lost-wakeup hang
Must hold the monitor	Otherwise <code>IllegalMonitorStateException</code>

▲ `wait()` vs `sleep()` — one of the most-asked:

	<code>wait()</code>	<code>sleep()</code>
Declared on	<code>Object</code>	<code>Thread</code> (static)
Releases the lock	✓ yes	✗ no — keeps holding it
Needs <code>synchronized</code>	✓	✗
Woken by	<code>notify</code> / <code>notifyAll</code> / timeout	Time elapsing / interrupt

● "The one-sentence version: `sleep` holds the lock, `wait` gives it up. A `sleep` inside a `synchronized` block blocks every other thread for the whole duration — which is exactly how you turn a slow call into an outage."

C3. `volatile` vs `synchronized` vs `Atomic`

	Visibility	Atomicity	Blocks	Cost
<code>volatile</code>	✓	✗	✗	Cheapest
<code>AtomicInteger</code> etc.	✓	✓ (CAS)	✗ (lock-free spin)	Cheap, contention- sensitive
<code>synchronized</code> / <code>Lock</code>	✓	✓	✓	Most expensive

"`volatile count++` is still broken, because `++` is read-modify-write — three operations, and two threads interleave and lose an update. `volatile` fixes visibility, not atomicity. For a counter you want `AtomicInteger.incrementAndGet()`."

Where `volatile` alone is exactly right — the stop flag:

```
private volatile boolean running = true;    // ● remove volatile →
possible infinite loop

public void run() { while (running) { work(); } }
public void stop() { running = false; }
```


"Without `volatile`, the JIT is allowed to hoist the read out of the loop — it sees nothing in the loop modifying `running` — and the thread spins forever even after another thread sets it false. It's not a theoretical optimisation; it happens once the loop is hot enough to be compiled."

How CAS works — say this, it's the mechanism behind the whole `java.util.concurrent` package:

"Compare-and-swap is a single CPU instruction: 'if this memory location still holds the value I read, replace it — otherwise tell me you failed.'
`AtomicInteger.incrementAndGet` is a loop around it: read, compute, CAS, retry if someone beat you. It's optimistic — no lock, no thread parking, no context switch. The trade-off is that under heavy contention the retry loop burns CPU, so at very high contention a lock can actually win. `LongAdder` exists for exactly that case: it spreads the count over multiple cells and sums on read."

Atomic class	Use
<code>AtomicInteger</code> / <code>AtomicLong</code>	Counters, sequence numbers
<code>AtomicBoolean</code>	One-shot flags — <code>compareAndSet(false, true)</code> to elect a single winner
<code>AtomicReference<T></code>	Lock-free swap of an immutable object
<code>LongAdder</code>	High-contention counters — faster than <code>AtomicLong</code>

C4. `synchronized` vs `ReentrantLock`

<code>synchronized</code>	<code>ReentrantLock</code>
Language keyword	Class in <code>java.util.concurrent.locks</code>
Auto-released on exit/exception	● You must <code>unlock()</code> in a <code>finally</code>
No timeout	✓ <code>tryLock(1, SECONDS)</code> — the deadlock escape hatch
Not interruptible	✓ <code>lockInterruptibly()</code>

synchronized	ReentrantLock
No fairness option	✓ optional FIFO fairness (<code>new ReentrantLock(true)</code>)
One wait-set	✓ multiple <code>Condition</code> s
Can't query it	✓ <code>isLocked()</code> , <code>getHoldCount()</code>

```
private final ReentrantLock lock = new ReentrantLock();

lock.lock();
try { /* work */ }
finally { lock.unlock(); }           // ● if you forget this, the lock is
held forever
```

"I'd default to `synchronized` — it's simpler and the JVM optimises it well (biased/ thin locks). I reach for `ReentrantLock` when I specifically need a timeout, an interruptible acquire, fairness, or multiple conditions. The cost is that you own the unlock, and forgetting it in a `finally` is a permanent hang."

▲ `ReadWriteLock` → "`ReentrantReadWriteLock` allows many concurrent readers but an exclusive writer. Right for a read-heavy cache — but if writes are frequent, readers starve or the overhead outweighs it, and `StampedLock` with optimistic reads is the faster modern option."

C5. `ExecutorService` & thread pools

"Creating threads directly doesn't scale — each one costs about a megabyte of stack and an OS context switch, and unbounded threads will OOM the JVM. An `ExecutorService` gives you a fixed pool that pulls tasks off a queue, so threads are reused."

```

ExecutorService pool = Executors.newFixedThreadPool(10);

Future<Integer> f = pool.submit(() -> compute());    // Callable → value
+ checked exceptions
Integer result = f.get();                            // ▲ BLOCKS until
done

pool.shutdown();                                    // no new tasks;
finish what's queued
if (!pool.awaitTermination(30, TimeUnit.SECONDS)) {
    pool.shutdownNow();                            // interrupt the
stragglers
}

```

	<code>execute(Runnable)</code>	<code>submit(Callable/ Runnable)</code>
Returns	void	<code>Future<T></code>
Return value	✗	✓
Exception	Goes to the uncaught- exception handler	● Swallowed until you call <code>get()</code>

● **A genuine production trap worth saying out loud:** "An exception inside a task submitted with `submit()` is captured in the `Future` and disappears silently if nobody calls `get()`. The task looks like it succeeded; there's nothing in the logs. Either call `get()`, or wrap the task body in a try/catch that logs."

The factory methods, and why you shouldn't use them:

Factory	What it really is	Risk
<code>newFixedThreadPool(n)</code>	n threads, unbounded <code>LinkedBlockingQueue</code>	● Queue grows until OOM — no back-pressure
<code>newCachedThreadPool()</code>	0.. <code>Integer.MAX_VALUE</code> threads, <code>SynchronousQueue</code>	● Unbounded threads under load
<code>newSingleThreadExecutor()</code>	1 thread, unbounded queue	Serialises work; same queue risk

Factory	What it really is	Risk
<code>newScheduledThreadPool(n)</code>	Delayed / periodic tasks	The <code>@Scheduled</code> engine

● "In production I'd construct a `ThreadPoolExecutor` explicitly rather than use the factories, because both common ones are unbounded in one direction or the other. A bounded queue plus an explicit rejection policy is what gives you back-pressure instead of a slow-motion OOM."

```
new ThreadPoolExecutor(  
    10, 20,                                // core, max  
    60L, TimeUnit.SECONDS,                 // keep-alive for threads  
    above core  
    new ArrayBlockingQueue<>(500),         // ● BOUNDED  
    new ThreadFactoryBuilder().setNameFormat("reminder-%d").build(), //  
    named → readable dumps  
    new ThreadPoolExecutor.CallerRunsPolicy() // back-pressure: the  
    submitter runs it  
);
```

▲ "How does it decide to create a thread?" — ● the order surprises people:

"Below core size → create a new thread. At core size → queue the task. Only when the queue is full does it create threads up to max. So with an unbounded queue, `maximumPoolSize` is dead code — the pool never grows past core, because the queue never fills."

Rejection policy	Behaviour
<code>AbortPolicy</code> (default)	Throws <code>RejectedExecutionException</code>
<code>CallerRunsPolicy</code>	✓ The submitting thread runs it — natural back-pressure
<code>DiscardPolicy</code> / <code>DiscardOldestPolicy</code>	Silently drops — dangerous

Pool sizing: "CPU-bound → roughly `Runtime.getRuntime().availableProcessors()`; more threads than cores just adds context switching. IO-bound → higher, because threads are mostly parked

waiting; the rough formula is `cores × (1 + wait/compute)` . And I'd measure rather than trust the formula."

C6. `CompletableFuture` — parallel calls, the microservices link

```
CompletableFuture<Profile>    p = CompletableFuture.supplyAsync(() ->
userClient.get(id), pool);
CompletableFuture<List<Quiz>> q = CompletableFuture.supplyAsync(() ->
quizClient.get(id), pool);

Dashboard dash = p.thenCombine(q, Dashboard::new)
                    .orTimeout(2, TimeUnit.SECONDS)
                    .exceptionally(ex -> Dashboard.partial())    // graceful
degradation
                        .join();
```

"This is the backend equivalent of `forkJoin` in RxJS — two independent downstream calls in parallel instead of sequentially, so the latency is the slower of the two rather than the sum. That's exactly the dashboard-aggregation problem from [A10](#), solved on the server side."

Method	Meaning
<code>supplyAsync</code> / <code>runAsync</code>	Start async — with, or without, a result
<code>thenApply</code>	<code>map</code> — transform the result
<code>thenCompose</code>	<code>flatMap</code> — when the function itself returns a <code>CompletableFuture</code>
<code>thenCombine</code>	Merge two independent futures
<code>allOf</code> / <code>anyOf</code>	Wait for all / the first
<code>exceptionally</code> / <code>handle</code>	Fallback on failure / handle both outcomes
<code>orTimeout</code> (Java 9+)	Fail after a duration

● **Two things to say:** "First — the `Async` variants without an explicit executor use the common `ForkJoinPool`, which is shared JVM-wide and sized to `cores minus one`. Putting blocking IO on it starves parallel streams and every other user of it, so

I always pass my own pool. Second — `join()` and `get()` are blocking; if you block on every call you've written synchronous code with extra ceremony. The value is in composing and blocking once, at the edge."

▲ `thenApply` vs `thenCompose` → "`thenApply` is $1 \rightarrow 1$. `thenCompose` flattens — use it when the mapper returns a `CompletableFuture`, otherwise you end up with `CompletableFuture<CompletableFuture<T>>`. Identical to `map` vs `flatMap`, and to `map` vs `switchMap` in RxJS."

C7. Concurrent collections

Instead of	Use	Why
<code>HashMap</code>	<code>ConcurrentHashMap</code>	Per-bucket locking; lock-free reads
<code>ArrayList</code>	<code>CopyOnWriteArrayList</code>	Read-heavy, write-rare (listener lists)
<code>Collections.synchronizedList</code>	Usually the above	One global lock; and iteration still needs manual synchronization
<code>Hashtable</code> / <code>Vector</code>	Anything else	Legacy, whole-object lock
A hand-rolled queue	<code>BlockingQueue</code>	Blocking put/take — producer/consumer for free

"`ConcurrentHashMap` since Java 8 uses CAS to insert into an empty bin and `synchronized` on the bin head otherwise, so writes only contend when two threads hit the same bucket, and reads are lock-free because the nodes' value and next fields are `volatile`. `Hashtable` synchronizes every method on the whole map — one thread at a time, always."

● The trap: individually atomic ≠ atomic together.

```
// ✗ NOT atomic – two separate operations; another thread can slip
between them
if (!map.containsKey(k)) map.put(k, v);

// ✓ Atomic
map.putIfAbsent(k, v);
map.computeIfAbsent(k, key -> expensiveLoad(key));
map.merge(k, 1, Integer::sum); // a thread-safe
counter
```

"Every method on `ConcurrentHashMap` is atomic on its own, but a check-then-act sequence isn't. That's the most common misuse — people assume the collection being 'thread-safe' makes their compound operation safe, and it doesn't. Same reason a `synchronizedList` still needs an explicit `synchronized` block around iteration."

CopyOnWriteArrayList: "Every write copies the whole backing array, so writes are $O(n)$ — but reads need no lock at all and its iterator never throws `ConcurrentModificationException` because it iterates the snapshot. Right for a listener list read constantly and written at startup; wrong for anything write-heavy."

C8. Coordination utilities & producer–consumer

Utility	One-line purpose
<code>CountDownLatch</code>	Wait for N things to finish. One-shot — can't be reset
<code>CyclicBarrier</code>	N threads wait for each other, then all proceed. Reusable
<code>Semaphore</code>	Limit concurrent access to N permits — a rate limiter / connection pool
<code>BlockingQueue</code>	Hand work between threads with blocking put/take
<code>Exchanger</code>	Two threads swap objects
<code>Phaser</code>	A flexible, multi-phase barrier

```
// CountdownLatch – the standard "wait for all the parallel calls" pattern
CountDownLatch latch = new CountDownLatch(3);
for (Task t : tasks) pool.submit(() -> { try { t.run(); } finally
{ latch.countDown(); } });
latch.await(5, TimeUnit.SECONDS);      // ● always the timed version in
production
```

● " `countDown()` goes in a `finally`. If a task throws and skips it, the latch never reaches zero and `await()` blocks forever — which is why I use the timed `await` as a second line of defence."

Producer–consumer with `BlockingQueue` — a very common "write this" request:

```
BlockingQueue<Order> queue = new ArrayBlockingQueue<>(100);    // bounded →
back-pressure

// Producer
new Thread(() -> {
    while (running) queue.put(nextOrder());                  // BLOCKS when full –
this IS the back-pressure
}).start();

// Consumer
new Thread(() -> {
    while (running) process(queue.take());                    // BLOCKS when empty
}).start();
```

"The whole point is that `put` and `take` handle the waiting — no `wait` / `notify`, no lost wakeups, no spurious-wakeup loop. And using a bounded queue is deliberate: an unbounded one lets a fast producer outrun the consumer until it exhausts memory. Blocking the producer is the feature."

Graceful shutdown — the poison-pill pattern:

```
queue.put(POISON_PILL);      // consumer sees it, stops, and re-inserts it
for its siblings
```

Semaphore — limiting concurrency against a fragile downstream:


```
private final Semaphore permits = new Semaphore(5); // at most 5
concurrent AI calls

permits.acquire();
try { return aiClient.explain(problem); }
finally { permits.release(); } // ● finally,
always
```

"This is the bulkhead pattern from the microservices question at L1 — capping how many threads can be tied up by one slow dependency so it can't exhaust the whole pool."

C9. ThreadLocal

*"A **ThreadLocal** gives each thread its own copy of a variable. Spring uses it heavily — the security context, the transaction's **EntityManager**, and the request context are all thread-locals, which is how **@Transactional** knows which transaction you're in without passing it through every method signature."*

```
private static final ThreadLocal<SimpleDateFormat> FORMAT =
    ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));
```

● The two things they're testing:

1. **SimpleDateFormat is not thread-safe** — a shared static instance silently produces wrong dates under load. *"Which is why in modern code I'd use **DateTimeFormatter** from **java.time**, which is immutable and thread-safe, and skip the **ThreadLocal** entirely."*
2. **ThreadLocal leaks in a thread pool.** *"Pool threads live forever, so a value set during one request is still there for the next request on that same thread — a stale-data bug and a memory leak. You must **remove()** in a **finally**. This is also why **ThreadLocal** state doesn't survive an **@Async** hop unless you propagate it explicitly."*

```
try { CONTEXT.set(user); doWork(); }
finally { CONTEXT.remove(); } // ● mandatory on pooled threads
```

C10. Multithreading inside Spring Boot — ● the question they will actually ask

Pure-Java concurrency is the textbook half. **The half that shows experience is knowing where threads live in a Spring app** — and this is where an L2 interviewer for a Java/Spring role will take it.

① "Is a Spring @Service thread-safe?"

"The bean is a singleton shared by every request thread, and Tomcat handles each request on its own thread — so any mutable instance field on a @Service is a race condition. Keep beans stateless: state belongs in method parameters and local variables, which are per-thread on the stack. A private List<String> items field on a service is the single most common concurrency bug in Spring codebases, and it works perfectly with one user testing on a laptop."

② @Async

```
@Configuration
@EnableAsync
public class AsyncConfig {
    @Bean("notificationExecutor")
    public Executor executor() {
        ThreadPoolTaskExecutor e = new ThreadPoolTaskExecutor();
        e.setCorePoolSize(5);
        e.setMaxPoolSize(10);
        e.setQueueCapacity(100);           // bounded
        e.setThreadNamePrefix("notify-");  // readable thread dumps
        e.initialize();
        return e;
    }
}

@Async("notificationExecutor")
public CompletableFuture<Void> sendReminder(Long userId) { ... }
```

● @Async trap	Why
Self-invocation	Proxy-based — calling it from the same class runs it synchronously, silently

● @Async trap	Why
Must be public	Private/final methods can't be proxied
Return type	void or CompletableFuture<T> . A plain String return is always null
Exceptions in void methods	Vanish — supply an AsyncUncaughtExceptionHandler
No default executor	Without a configured one, older Boot used SimpleAsyncTaskExecutor , which creates a new thread per call and doesn't pool at all
Nothing propagates	Security context, transaction and MDC do not cross the thread boundary automatically

*"It's the same AOP-proxy family of traps as **@Transactional** and **@CircuitBreaker** — which is the point I made at L1 about **spring-boot-starter-aop** . One mechanism, three annotations, the same silent failure."*

③ ● **"Does **@Transactional** work inside **@Async** ?"** — an excellent discriminator question.

*"Not the caller's transaction. The transaction is bound to the calling thread via a **ThreadLocal**, and the async method runs on a different thread — so it starts its own transaction, or none. The practical consequence is a real bug: you commit, fire an async job, and the async job can't see the data if it started before the commit landed. The fix is **@TransactionalEventListener(phase = AFTER_COMMIT)** so the async work is only triggered once the transaction has actually committed."*

④ **@Scheduled** — and the multi-instance problem

*"**@Scheduled** runs on a single-threaded scheduler by default, so one long task delays every other scheduled job in the app — you configure a **ThreadPoolTaskScheduler** to fix that. But the bigger issue is that in production you run more than one instance, and every instance fires the same job. That's*

exactly the duplicate-reminder problem I described — you need ShedLock, a database-level claim with `FOR UPDATE SKIP LOCKED`, or an external scheduler."

⑤ The database is where your real concurrency lives

"Honestly, in a web application most of the concurrency isn't threads I create — it's many requests hitting the same rows. Two users updating the same record is handled with optimistic locking, a `@Version` column that fails the second commit with `OptimisticLockException` and lets me retry. And the thread pool that matters most in practice is the HikariCP connection pool — more Tomcat threads than DB connections means threads queue on `getConnection()`, which is what a hung Spring Boot app usually turns out to be."

C11. Virtual threads (Java 21)

Worth one paragraph — it signals you're current, and Mphasis works on modernisation programmes.

"Java 21 made virtual threads final. They're lightweight threads scheduled by the JVM rather than the OS, so you can have millions of them; when one blocks on IO it unmounts from its carrier thread instead of holding an OS thread hostage. The point is that the thread-per-request model becomes cheap again — you get the scalability of reactive code while writing plain blocking code that's readable and debuggable. In Spring Boot 3.2 it's one property:

```
spring.threads.virtual.enabled=true."
```

"Two caveats: don't pool virtual threads — they're cheap to create, pooling defeats the purpose; and `synchronized` blocks used to pin a virtual thread to its carrier, so `ReentrantLock` was preferred in hot paths."

C12. Predict-the-output — multithreading

L1 didn't ask output questions, but Codeboard did, and these are the concurrency ones that show up.

```
// ① What prints?
Thread t = new Thread(() -> System.out.println("A"));
t.run(); // ← run(), not start()
System.out.println("B");
```

A then B, always, on the main thread. `run()` is an ordinary method call — no new thread at all.

```
// ② What prints?
Thread t = new Thread(() -> System.out.println("A"));
t.start();
t.start();
```

A, then `IllegalThreadStateException`. A thread can't be restarted.

```
// ③ What's the final count?
class Counter { int count = 0; void inc() { count++; } }
// 2 threads × 10,000 increments each
```

Something ≤ 20000 , non-deterministic. `count++` is read-modify-write. Marking it `volatile` does **not** fix it — you need `AtomicInteger` or `synchronized`.

```
// ④ Does this terminate?
static boolean running = true; // no volatile
new Thread(() -> { while (running) { } }).start();
Thread.sleep(1000);
running = false;
```

Possibly never. With no `volatile` and an empty loop body, the JIT can hoist the read out of the loop. It often terminates in `-Xint` and hangs once compiled — which is exactly why it's a good interview question.

```
// ⑤ What prints?
synchronized (this) { Thread.sleep(1000); }
```

Nothing about output — the point is: **sleep does not release the lock**. Every other thread waiting on that monitor is blocked for the full second.

// ⑥ HashMap under concurrent writes?

"Lost updates, and on Java 7 an infinite loop in `get()` because the resize could create a circular linked list — a genuine production hang, and one of the reasons the Java 8 rewrite happened. Java 8 removed the cycle, but it's still unsafe: entries are still lost."

■ More: [23 — Java Predict the Output](#)

C13. Rapid-fire

Question	Answer
Process vs thread	Process = own memory space; threads share the heap within one process (and have their own stack)
What's per-thread vs shared?	● Stack, program counter and locals are per-thread; heap and static fields are shared. That's why only object state races
<code>start()</code> vs <code>run()</code>	New thread vs plain method call on the current thread
<code>Runnable</code> vs <code>Callable</code>	No return / no checked exceptions vs both
<code>wait()</code> vs <code>sleep()</code>	<code>wait</code> releases the lock (and needs <code>synchronized</code>); <code>sleep</code> holds it
<code>wait</code> in <code>while</code> , not <code>if</code> ?	Spurious wakeups + another thread may consume the condition first
<code>notify</code> vs <code>notifyAll</code>	<code>notify</code> wakes one arbitrary waiter → lost-wakeup risk. Prefer <code>notifyAll</code>
Thread states	NEW · RUNNABLE · BLOCKED · WAITING · TIMED_WAITING · TERMINATED

Question	Answer
<code>BLOCKED</code> vs <code>WAITING</code>	Waiting for a monitor vs waiting for a condition
Daemon thread	JVM exits when only daemons remain; <code>setDaemon</code> before <code>start()</code>
Stop a thread	Cooperative interruption — <code>Thread.stop()</code> is deprecated and unsafe
Caught <code>InterruptedException</code> ?	● Restore the flag — catching it clears it
<code>volatile</code> guarantees	Visibility + no reordering. Not atomicity
Atomic classes	CAS — lock-free, optimistic retry loop
<code>synchronized</code> static vs instance	● Different locks — class monitor vs instance monitor. They don't exclude each other
Reentrant?	Yes — <code>synchronized</code> and <code>ReentrantLock</code> both
<code>ConcurrentHashMap</code> vs <code>Hashtable</code>	Per-bucket CAS/sync vs one lock for the whole map
Check-then-act on a concurrent map	● Not atomic — use <code>putIfAbsent</code> / <code>computeIfAbsent</code> / <code>merge</code>
Executor <code>submit</code> exception	Swallowed into the <code>Future</code> until <code>get()</code> is called
Thread pool growth order	core → queue → max. Unbounded queue = max never used
<code>CountDownLatch</code> vs <code>CyclicBarrier</code>	One-shot, one waiter set vs reusable, threads wait for each other
Deadlock's four conditions	Mutual exclusion · hold-and-wait · no preemption · circular wait
Deadlock fix	● Global lock ordering; or <code>tryLock</code> with timeout + randomised back-off
Detect a deadlock	3 thread dumps 10s apart; <code>jstack</code> prints Found one Java-level deadlock

Question	Answer
Deadlock vs livelock vs starvation	No progress + no CPU · no progress + full CPU · one thread never scheduled
<code>ThreadLocal</code> in a pool	● Must <code>remove()</code> in a <code>finally</code> — pool threads outlive the request
Is a Spring <code>@Service</code> thread-safe?	Only if it's stateless. Singleton bean + mutable field = race
<code>@Async</code> + <code>@Transactional</code>	Different thread → not the caller's transaction
Virtual threads	Java 21, JVM-scheduled, unmount on blocking IO. Don't pool them

D — Level 2: Spring & Spring Boot depth

D1. IoC and DI

"Inversion of Control means the framework creates and wires objects instead of my code calling `new`. Dependency Injection is how it does it. The practical benefit is testability — my service takes a `PaymentGateway` interface in its constructor, so in a unit test I pass a mock and never touch the network."

▲ **"Constructor vs field vs setter injection — which and why?"** — ● **an extremely common L2 question.**

"Constructor injection, always. Three reasons: the dependencies can be `final`, so the object is immutable and fully valid the moment it exists; a missing dependency fails at startup rather than with an NPE at 2 a.m.; and I can construct it in a test with plain `new`, no Spring at all. Field injection with `@Autowired` hides dependencies, can't be `final`, and needs reflection to test. Since Spring 4.3 a single constructor doesn't even need `@Autowired`."

"One honest caveat: constructor injection makes a circular dependency fail at startup, where field injection would have quietly allowed it. People treat that as a downside — it's the feature. A cycle means the design is wrong."

D2. Bean scopes and lifecycle

Scope	One bean per...
<code>singleton</code> (default)	container
<code>prototype</code>	injection / <code>getBean()</code> call
<code>request</code>	HTTP request
<code>session</code>	HTTP session
<code>application</code>	ServletContext

Instantiate → populate properties → *Aware callbacks → BeanPostProcessor.before
→ @PostConstruct → afterPropertiesSet() → BeanPostProcessor.after → ●
READY
→ @PreDestroy → destroy()

▲ "Spring does not call `@PreDestroy` on prototype beans — the container hands the instance over and forgets it. If a prototype holds a resource, you close it yourself."

D3. @Component vs @Bean vs @Configuration

	@Component (+ @Service / @Repository)	@Bean
Where	On your own class	On a method in a <code>@Configuration</code> class
Detected by	Component scanning	Explicit method call
Use for	Classes you own	● Third-party classes you can't annotate

"`@Repository` additionally translates vendor-specific persistence exceptions into Spring's `DataAccessException` hierarchy — it's not just a documentation marker."

▲ "Why is `@Configuration` different from `@Component` for `@Bean` methods?"

"`@Configuration` classes are CGLIB-proxied, so calling one `@Bean` method from another returns the existing singleton rather than creating a second instance. With plain `@Component` (`proxyBeanMethods = false` semantics) that same call is an ordinary Java call and you get a duplicate object. It's the kind of thing that produces two connection pools and a very confusing bug."

D4. ● `@Transactional` — the most re-probed Spring topic

Propagation:

Value	Behaviour
<code>REQUIRED</code> (default)	Join the existing transaction, or start one
<code>REQUIRES_NEW</code>	● Suspend the current one, always start a new one — commits independently
<code>SUPPORTS</code>	Join if one exists, else run non-transactionally
<code>MANDATORY</code>	Must already be in one, else throw
<code>NEVER</code> / <code>NOT_SUPPORTED</code>	Throw / suspend
<code>NESTED</code>	Savepoint inside the current transaction

"`REQUIRES_NEW` is what you use for audit logging — you want the audit row to survive even if the business transaction rolls back."

Isolation levels — and what each prevents:

Level	Dirty read	Non-repeatable read	Phantom read
<code>READ_UNCOMMITTED</code>	✗ possible	✗	✗
<code>READ_COMMITTED</code> (Postgres/Oracle default)	✓ prevented	✗	✗

Level	Dirty read	Non-repeatable read	Phantom read
REPEATABLE_READ (MySQL default)	✓	✓	✗
SERIALIZABLE	✓	✓	✓

● The three traps — know all three, they're the question:

```
// TRAP 1 – SELF-INVOCATION. The proxy is bypassed; there is NO
transaction.
@Service
public class OrderService {
    public void placeOrder() {
        this.save();                // ✗ plain Java call – @Transactional
    }
    @Transactional
    public void save() { }
}

// TRAP 2 – @Transactional only rolls back on RuntimeException / Error by
default.
@Transactional                    // ✗ IOException will
NOT roll back
public void process() throws IOException { }

@Transactional(rollbackFor = Exception.class)    // ✓
public void process2() throws IOException { }

// TRAP 3 – a private / final method can't be proxied. Silently no
transaction.
@Transactional
private void save() { }           // ✗ never
transactional
```

"All three are the same root cause: `@Transactional` is implemented with AOP proxies. Anything that bypasses the proxy — self-invocation, a private or final method, a missing `spring-boot-starter-aop` — silently disables it. And it's silent: the code runs, the data isn't protected. Exactly the same failure mode as `@CircuitBreaker`, which is why I mentioned AOP in the L1 round."

■ Deep dive: **24 — ORM / JPA / Hibernate**

D5. Spring Boot auto-configuration

"`@SpringBootApplication` is three annotations: `@SpringBootConfiguration`, `@ComponentScan`, and `@EnableAutoConfiguration`. Auto-configuration reads `META-INF/spring/...AutoConfiguration.imports` from every jar on the classpath and applies each configuration conditionally — `@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`. So if `spring-boot-starter-data-jpa` is present and a `DataSource` isn't already defined, Boot configures one from your properties."

"`@ConditionalOnMissingBean` is the important one — it means defining your own bean silently wins. That's how Boot stays out of your way. And `/actuator/conditions` shows exactly which auto-configurations matched and which were skipped and why — that's the first place I look when a bean I expected isn't there."

Excluding one: `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)`

D6. REST API design — likely in both rounds

Topic	The answer
Status codes	200 OK · 201 Created (+ <code>Location</code>) · 204 No Content · 400 Bad Request · 401 unauthenticated · 403 authenticated but not allowed · 404 · 409 Conflict · 422 · 500
<code>PUT</code> vs <code>PATCH</code>	PUT replaces the whole resource and is idempotent; PATCH is partial.
Idempotency	GET/PUT/DELETE idempotent; POST is not. For payments, use an <code>Idempotency-Key</code> header.
Versioning	URI (<code>/api/v1/...</code>) — simplest and most common. Also header or content negotiation.
Validation	<code>@Valid</code> on the <code>@RequestBody</code> + JSR-380 (<code>@NotNull</code> , <code>@Size</code> , <code>@Email</code>), handled in <code>@RestControllerAdvice</code> .

Topic	The answer
● DTO vs entity	Never return the entity. It leaks the schema, drags lazy proxies into serialisation (<code>LazyInitializationException</code> / infinite recursion on bidirectional relations), and couples your API to your DB. Map to a DTO or a record.
Pagination	<code>Pageable</code> → <code>Page<T></code> with <code>?page=0&size=20&sort=name,asc</code> .
<code>@RequestParam</code> vs <code>@PathVariable</code>	Path variable identifies the resource; request param filters/sorts/pages.
Filter vs Interceptor	Filter is Servlet-level, sees every request incl. static, no Spring context. Interceptor is Spring MVC, knows the handler — right place for auth checks and logging with controller context.

E — Level 2: JPA / Hibernate

They asked association vs aggregation at L1 — an OOP question that sits right next to entity mapping. Expect JPA at L2.

E1. ● The N+1 problem — the single most asked ORM question

```
List<Department> depts = repo.findAll();           // 1 query
for (Department d : depts) {
    d.getEmployees().size();                       // N more queries — one
    per department
}
```

Three fixes:

```
// ① JOIN FETCH – one query, best for a targeted case
@Query("SELECT DISTINCT d FROM Department d LEFT JOIN FETCH d.employees")
List<Department> findAllWithEmployees();

// ② @EntityGraph – declarative, works with derived queries + Pageable
@EntityGraph(attributePaths = "employees")
List<Department> findAll();

// ③ Batch fetching – N+1 becomes N/batch_size + 1
@BatchSize(size = 25)
private List<Employee> employees;
```

▲ " *JOIN FETCH* plus *Pageable* is a trap — Hibernate can't paginate a joined result set in SQL, so it pulls everything into memory and pages there, logging *HHH000104: firstResult/maxResults specified with collection fetch; applying in memory*. On a large table that's an OOM. For paginated fetches use *@EntityGraph* or a two-query approach: page the IDs, then fetch the collection for those IDs. "

How you'd notice it: `spring.jpa.show-sql=true`, or better, `p6spy` / Hibernate statistics — "I'd count the queries, not eyeball the logs."

E2. FetchType and LazyInitializationException

	LAZY	EAGER
Default for	@OneToMany , @ManyToMany	@ManyToOne , @OneToOne
Loads	On first access, via proxy	With the parent

" *LazyInitializationException* means you touched a lazy association after the persistence context closed — typically in the controller or during JSON serialisation. The wrong fix is `spring.jpa.open-in-view=true` (it's on by default and I'd turn it off — it holds the DB connection for the whole request) or switching to EAGER (which reintroduces N+1 everywhere). The right fix is to fetch what you need inside the transaction with *JOIN FETCH*, and return a DTO. "

E3. Quick-fire JPA

Question	Answer
Entity states	Transient → Managed → Detached → Removed
<code>save</code> vs <code>saveAndFlush</code>	<code>save</code> may defer the SQL to commit; <code>saveAndFlush</code> pushes it now (so you can read it back in the same transaction)
<code>getReferenceById</code> vs <code>findById</code>	Returns a lazy proxy without hitting the DB (throws on access if missing) vs an actual <code>SELECT</code>
First-level cache	The persistence context / <code>EntityManager</code> — always on, per transaction. Same entity twice in one transaction = one query.
Second-level cache	Across sessions, opt-in — EhCache/Hazelcast/Redis
<code>CascadeType</code>	ALL, PERSIST, MERGE, REMOVE, REFRESH, DETACH. ▲ <code>REMOVE</code> on <code>@ManyToMany</code> deletes the other side's rows — almost always a bug
<code>orphanRemoval</code> vs <code>CascadeType.REMOVE</code>	Orphan removal deletes a child <i>removed from the collection</i> ; cascade remove only fires when the parent is deleted
Optimistic vs pessimistic locking	<code>@Version</code> column, fails at commit with <code>OptimisticLockException</code> — right for low contention. <code>SELECT ... FOR UPDATE</code> blocks others — right for high contention/money
Bidirectional relation + JSON	Infinite recursion → <code>@JsonIgnore</code> / <code>@JsonManagedReference</code> — or just return a DTO

F — Level 2: microservices, one level down

F1. How do services talk to each other?

	RestTemplate	WebClient	FeignClient
Style	Blocking	Non-blocking, reactive	Declarative, blocking
Status	● In maintenance since Spring 5	Recommended	Spring Cloud OpenFeign
Code	Verbose	Fluent	An interface + annotations

```
@FeignClient(name = "inventory-service", fallback =
InventoryFallback.class)
public interface InventoryClient {
    @GetMapping("/stock/{sku}")
    StockResponse getStock(@PathVariable String sku);
}
```

"Feign gives you the cleanest call site — it's just an interface, and it integrates with service discovery and Resilience4j. **WebClient** is what I'd use where I need real concurrency or streaming. **RestTemplate** still works but new code shouldn't use it."

▲ **"Sync or async between services?"** → "Synchronous REST is fine when the caller genuinely needs the answer to continue. But every sync hop couples availability — if A calls B calls C, A is only as available as the weakest link. Anything that doesn't need an immediate answer should be an event on a queue — Kafka or RabbitMQ. On RoboGebra the reminder dispatch is exactly that shape: the quiz service doesn't need to wait for a notification to be sent."

F2. The supporting cast

Component	What it does	Typical tech
API Gateway	Single entry: routing, auth, rate limiting, CORS, SSL termination	Spring Cloud Gateway
Service Discovery	Services register and find each other by name, not host	Eureka, Consul
Config Server	Externalised, per-environment config, refreshable	Spring Cloud Config
Distributed tracing	One trace id across all hops	Micrometer Tracing + Zipkin
Log aggregation	Search all services' logs in one place	ELK / Loki
Message broker	Async, decoupled communication	Kafka, RabbitMQ

F3. Security — likely if the client is BFSI/insurance (Mphasis's core domains)

"Auth at the gateway, stateless downstream. The user logs in and gets a JWT; the gateway validates the signature and forwards the token. Each service can then authorise from the claims without a call back to the auth service, which is what makes it stateless and horizontally scalable."

Question	Answer
JWT parts	<code>header.payload.signature</code> , base64url-encoded, dot-separated
Is it encrypted?	● No — it's signed, not encrypted. Anyone can decode the payload. Never put secrets in it.
Why a short expiry + refresh token?	You can't revoke a JWT once issued, so you keep the access token short-lived (~15 min) and revoke at the refresh step

Question	Answer
Where to store it in Angular?	<code>httpOnly</code> cookie is safest (XSS can't read it, but needs CSRF protection); <code>localStorage</code> is convenient and XSS-readable
Attaching it in Angular	An <code>HttpInterceptor</code> that adds the <code>Authorization: Bearer</code> header — ● you've built this, say so
Spring Security basics	<code>SecurityFilterChain</code> bean, <code>authorizeHttpRequests</code> , a JWT filter before <code>UsernamePasswordAuthenticationFilter</code> , <code>@PreAuthorize("hasRole('ADMIN')")</code> for method-level

G — SQL: your live gap

● **SQL appeared for the first time in six rounds — at Mphasis, at L1.** They asked a `GROUP BY` with a join. That thread is now open and L2 will pull it. This is the section to over-prepare.

■ **This section is the compact version. The full treatment — every join, the whole salary-question family, window functions, NULL traps, indexing, 18 predict-the-output programs and 15 drills — is now in 36 — SQL: The Complete Interview File.**

G1. Joins — the diagram to have in your head

Join	Returns
<code>INNER</code>	Only matching rows on both sides
<code>LEFT</code>	All of the left + matches (NULLs where none)

Join	Returns
RIGHT	All of the right + matches
FULL OUTER	Everything from both (not in MySQL — emulate with UNION)
CROSS	Cartesian product
SELF	A table joined to itself — employee → manager

```
-- Self join: every employee with their manager's name
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM   employee e
LEFT JOIN employee m ON m.emp_id = e.manager_id;  -- LEFT, so the CEO
still appears
```

G2. The queries that actually get asked

```
-- ① Nth-highest salary (handles ties)
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rnk FROM
employee
) t WHERE rnk = 2;

-- ② Highest-paid per department
SELECT * FROM (
  SELECT e.*, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) rnk
FROM employee e
) t WHERE rnk = 1;

-- ③ Find duplicates
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

-- ④ Delete duplicates, keep the lowest id
DELETE u1 FROM users u1 JOIN users u2 ON u1.email = u2.email AND u1.id >
u2.id;

-- ⑤ Departments with NO employees
SELECT d.* FROM dept d LEFT JOIN employee e ON e.dept_id = d.dept_id WHERE
e.emp_id IS NULL;

-- ⑥ Employees earning above their department average
SELECT emp_name, salary FROM (
  SELECT e.*, AVG(salary) OVER (PARTITION BY dept_id) avg_sal FROM
employee e
) t WHERE salary > avg_sal;

-- ⑦ Running total
SELECT order_date, amount,
      SUM(amount) OVER (ORDER BY order_date ROWS UNBOUNDED PRECEDING) AS
running_total
FROM orders;
```

G3. RANK vs DENSE_RANK vs ROW_NUMBER — asked constantly

salary	ROW_NUMBER	RANK	DENSE_RANK
900	1	1	1
800	2	2	2
800	3	2	2

salary	ROW_NUMBER	RANK	DENSE_RANK
700	4	4 ← gap	3 ← no gap

G4. Indexes — the natural follow-up to "make it faster"

"An index is usually a B-tree that lets the engine seek instead of scanning. The cost is slower writes and disk space, so you index what you filter, join and sort on — not everything."

Concept	Say this
Clustered vs non-clustered	Clustered <i>is</i> the table's physical order — one per table (the PK by default). Non-clustered is a separate structure pointing back.
Composite index order	● Leftmost prefix rule — an index on (a, b) helps WHERE a and WHERE a AND b , but not WHERE b alone.
Covering index	Contains every column the query needs, so it never touches the table — "index-only scan".
● When an index is ignored	A function on the column (WHERE YEAR(created) = 2026), a leading wildcard (LIKE '%x'), an implicit type cast, or low selectivity (a gender column). Rewrite as a range: WHERE created >= '2026-01-01' AND created < '2027-01-01' .
How do you know?	EXPLAIN / EXPLAIN ANALYZE — look for type: ALL (full scan) vs ref / range , and the rows estimate.

G5. ACID, and DELETE vs TRUNCATE vs DROP

Atomicity · Consistency · Isolation · Durability — be able to give a one-line example of each.

	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
WHERE	✓	✗	✗
Rollback	✓	✗ (auto-commits)	✗
Resets AUTO_INCREMENT	✗	✓	—
Table survives	✓	✓	✗

Normalization: 1NF atomic columns → 2NF no partial dependency on part of a composite key → 3NF no transitive dependency. *"And denormalise deliberately for read performance — a reporting table that duplicates a department name is a legitimate trade, not a mistake."*

H — Live coding: what to expect

L1 made you write a thread-safe Singleton. **L2 will make you write something.** Type these, out loud, before the round.

Problem	Where
Second-highest salary — SQL and Streams	G2 · 22
Group employees by department / average salary	B5
First non-repeating character in a string	09
Reverse a string / check palindrome without library methods	09
Count word/char frequency with a Map	09
Remove duplicates from a list — with and without Streams	22
Find a missing number in 1..n	09

Problem	Where
Fibonacci / factorial, iterative + recursive	09
Producer-consumer with <code>BlockingQueue</code>	C8
Thread-safe counter / two threads incrementing	C3 · C12
Print odd/even numbers alternately with 2 threads	<code>wait</code> / <code>notifyAll</code> — C2
Thread-safe Singleton (<i>already asked — they may ask you to extend it</i>)	26 § Mphasis Q5

● How to behave during live coding — this matters as much as the code:

1. **Restate the problem** and confirm one edge case before typing. ("Empty list — return empty or throw?")
2. **Narrate as you type.** Silence reads as being stuck.
3. **Brute force first, then improve.** A working $O(n^2)$ you then optimise beats a broken clever answer.
4. **State the complexity** unprompted at the end.
5. **Name one edge case you'd test:** null, empty, duplicates, single element.

I — System design, lite

If L2 includes design, it will be a 10-minute conversation, not a whiteboard marathon. Use this skeleton and you'll never be lost:

- ① CLARIFY — who uses it, how many, read-heavy or write-heavy? (60 seconds, always)
- ② API — 3 or 4 endpoints. Makes it concrete immediately.
- ③ DATA MODEL — the 3 main tables/entities and the key relationships.
- ④ HIGH LEVEL — client → gateway → services → DB. Draw the boxes.
- ⑤ SCALE — where's the bottleneck? cache / index / queue / replicas.
- ⑥ TRADE-OFFS — what you chose NOT to do, and why.

The one to rehearse: "design the reminder/notification system" — because it's *your* feature, so you can't be caught out.

"Endpoints: create a reminder, list mine, cancel one. The table is `reminder(id, user_id, scheduled_at, type, status, payload)` with an index on `(status, scheduled_at)` since that's the only query the dispatcher makes. The naive version is a `@Scheduled` job polling every minute for `status = PENDING AND scheduled_at <= now()`."

"That's fine at small scale, and it breaks in two predictable ways. First, multiple instances — every replica picks the same rows and the user gets three notifications. You fix it with `SELECT ... FOR UPDATE SKIP LOCKED`, or `ShedLock`, or by moving to a queue. Second, the polling itself becomes a hot query as the table grows, which is why you'd move due reminders onto a delay queue and have the DB hold only the source of truth."

"And delivery has to be idempotent — dedupe on `(reminder_id, channel)` — because at-least-once delivery is the only thing a queue actually promises."

That answer demonstrates scale thinking, concurrency, indexing and idempotency in ninety seconds, all anchored in something you genuinely built.

J — Angular, the way THEY will ask it

◆ **HR's exact framing: "what you have used in recent projects."** So prepare **four narrations**, not a syllabus. Each one is: *what the feature was* → *what I used* → *why* → *what went wrong*.

J1. RoboGebra — the personalized learning dashboard

"The dashboard shows a student's progress, streak, recommended study list and recent quiz results. It's an Angular + Ionic app. The page is a smart container component that calls a facade service; the individual cards are dumb presentational components taking `@Input` s, which made them easy to test and reuse. The four calls originally ran sequentially — I moved them into a `forkJoin` so they go in parallel, and later collapsed them into one aggregate endpoint on the Java side. State that several components needed — the current student profile —

sits in a **BehaviorSubject** in a service, so any component subscribing gets the current value immediately rather than waiting for the next emission."

Follow-ups they'll ask, and your answers:

Follow-up	Answer
"How do you unsubscribe?"	" async pipe in the template wherever possible — Angular handles it. Where I subscribe in TypeScript, takeUntil(this.destroy\$) with a Subject completed in ngOnDestroy . In newer code, takeUntilDestroyed() . HTTP calls complete on their own, but a BehaviorSubject never does — that's the actual leak."
"What if one of the forkJoin calls fails?"	● " forkJoin fails the whole thing — you get nothing, not three out of four. So I add catchError on the individual streams to return a fallback, and the dashboard degrades gracefully instead of going blank."
" Subject vs BehaviorSubject ?"	" BehaviorSubject holds a current value and replays it to new subscribers, and needs an initial value. Subject only emits to whoever is already listening. For shared state you want BehaviorSubject , because a component created later still needs the current user."
"Why OnPush ?"	"Default change detection re-checks every binding on every event in the tree. OnPush only re-checks when an @Input reference changes, an event fires in the component, or an async pipe emits — which is why immutable updates matter. On the dashboard's list of cards that was a visible improvement."

J2. The Angular 16 migration — four repositories

● **This is your strongest Angular story, because almost nobody has done it.** It shows ownership, risk management and framework depth all at once.

*"I led an upgrade of four repositories to Angular 16. The approach was version-by-version with **ng update**, never skipping — Angular only supports one major at a time and jumping introduces failures you can't attribute. For each repo: upgrade, run the build, run the tests, fix what broke, commit, then the next version. The hard part wasn't Angular itself — it was third-party libraries that hadn't released a compatible version, where I had to either wait, find a replacement, or pin. And RxJS 6→7 was its own migration, with **toPromise()** deprecated in favour of **firstValueFrom** and the older operator import paths gone."*

Have ready: what actually broke, and how you verified nothing regressed ("the test suite plus a manual pass on the critical flows, and we shipped it repo by repo rather than all four at once so a rollback was small").

■ Full detail: [angular-migration-project-interview-pack](#)

J3. EasyVisa — Formly and nested reactive forms

*"The visa application forms were long, deeply nested and varied by visa type, so hard-coding each one wasn't viable. I used Formly, which builds the form from a JSON config, and wrote custom field types for the ones it didn't cover — an image cropper for document uploads, and a couple of domain-specific inputs. Underneath it's a reactive form: **FormGroup** containing nested **FormGroup**s and **FormArray**s for the repeated sections, so I could add and remove address blocks dynamically."*

Follow-up	Answer
"Reactive vs template-driven?"	"Reactive — the model is defined in TypeScript, so it's synchronous, unit-testable without the DOM, and supports dynamic controls and custom validators. Template-driven is fine for a login form; anything with conditional fields should be reactive."

Follow-up	Answer
"Custom validator?"	"A function returning <code>ValidationErrors null</code> . For async — checking an ID against the server — an <code>AsyncValidatorFn</code> returning an Observable, with <code>debounceTime</code> so it doesn't fire on every keystroke."
" <code>FormArray</code> ?"	"For a repeated group — a variable number of dependants or addresses. You push and remove <code>FormGroup</code> s at runtime and the form value stays a clean array."

J4. Subsea — .NET + Kendo grid

"Schedule Manager, with .NET Core APIs and a Kendo UI grid on the Angular side — category and subcategory management with server-side paging and filtering on the grid."

Keep this one short unless they follow up. **But mention .NET exists in your background** — Mphasis staffs both, and it can only help.

J5. The small handful of Angular theory to still know

Even in a "basics not mandatory" round, these come up because they're one step from a project answer. **They were each asked in 2–4 previous rounds** — see § [most-repeated](#).

Topic	Have one sentence
Lifecycle hooks	asked in 4 rounds — <code>ngOnInit</code> , <code>ngOnChanges</code> , <code>ngOnDestroy</code> , <code>ngAfterViewInit</code> and when each fires
Component communication	asked in 4 rounds — <code>@Input</code> / <code>@Output</code> , service + <code>BehaviorSubject</code> , <code>@ViewChild</code> , route params
Observable vs Promise	lazy + multi-value + cancellable vs eager + single-value

Topic	Have one sentence
<code>switchMap</code> vs <code>mergeMap</code> vs <code>concatMap</code> vs <code>exhaustMap</code>	cancel previous / all in parallel / queue in order / ignore while busy — <code>switchMap</code> for search-as-you-type, <code>exhaustMap</code> for a submit button
Lazy loading	<code>loadChildren</code> / <code>loadComponent</code> — smaller initial bundle; the cost is a delay on first navigation
Standalone components	Angular 14+, no NgModule — the default in 17+. Relevant because you migrated to 16
Signals	Angular 16 introduced them — know what they are and that they're an alternative to <code>Zone.js</code> -based change detection
Interceptors	JWT attachment, error handling, loaders — you've built this

■ 04 — Angular · 20 — RxJS Operators · 25 — Angular Binding & Forms

K — The client technical round

● This is a different round with a different objective. Most candidates prepare for it as "another tech round" and that's the mistake.

K1. Who's in the room and what they're deciding

	Mphasis L2	Client round
Interviewer	Mphasis senior engineer / architect	The end customer's tech lead or manager
Deciding	"Is she technically good enough to hire?"	"Do I want this person on my project on Monday?"
Style	Structured Q&A, depth probes	Conversational, scenario-led, project-focused

	Mphasis L2	Client round
Weight on theory	High	Low
Weight on "have you done this"	Medium	● Very high
What sinks you	Not knowing	Vagueness about your own work

◆ **The mental switch:** L2 asks "do you know Spring?". The client asks "if I give you a ticket on Monday, will it come back done?" Every answer should demonstrate **ownership, communication and independence** — not just knowledge.

K2. The questions they will actually ask

■ **Corroborated against reported Mphasis client-round questions** — the round splits into three buckets, and candidates report the same shapes repeatedly:

Bucket	What it's testing
A — Project & resume deep dive	Is the resume real? Architecture end to end, your contribution vs the team's, bottlenecks you solved, what you'd redesign
B — Technical & scenario	Structured troubleshooting of a live production issue, advanced SQL, framework internals for your stack, handling tight deadlines and changing requirements
C — Client interaction & delivery	Communicating blockers to non-technical stakeholders, conflicting priorities between the vendor's process and the client's asks

● **Bucket C is the one candidates don't prepare, and it's the one that's specific to a client round.** Bucket A and B they'd get in any interview; C is why this round exists.

Bucket A — project & resume deep dive

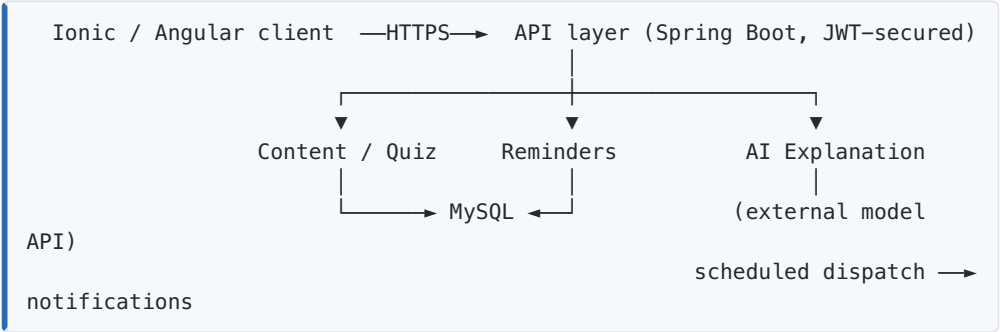
① "Walk me through your current project / your project architecture end to end." — ● the most important two minutes of the round.

Structure it as: **what the product does** → **the architecture** → **what you own** → **one hard thing you solved**.

"RoboGebra is an AI-driven math learning platform — students work through problems and get step-by-step explanations, in Tamil and English. The client is Angular and Ionic so it runs as a web app and on mobile; the backend is Java 17 with Spring Boot 3 and MongoDB, split into services, with an AI explanation engine behind its own service. I own the learner-facing side end to end — the personalized dashboard, the study list and quiz modules, and the study reminder system, which means both the Angular components and the REST APIs behind them."

"The hardest piece was the real-time computation and visualization — as the student edits an expression, the graph updates instantly while the AI explanation is fetched remotely. I split it: local computation renders immediately, and the remote call is debounced and `switchMap` ped so an in-flight request is cancelled when the input changes again. Without that we were flooding the AI backend with a request per keystroke."

● Have the architecture as a drawable picture, because "end to end" means they want the boxes:



Then say **where you sit on that diagram**: *"I own the learner-facing path — the Angular components and the APIs behind the dashboard, study list, quiz and reminders."*

② **"What was your specific contribution vs the team's?"** — ● **they are explicitly checking for resume inflation.** Be precise and volunteer the boundary. *"I owned the dashboard, study list, quiz and reminder modules end to end — Angular and the Spring Boot APIs. The AI explanation engine itself was another developer's; I integrated against it and built the bilingual rendering on the client."*

Honesty here buys more credibility than a bigger claim. The interviewer has seen candidates claim whole architectures; a clean, confident boundary is rare and it reads as senior.

③ **"What challenges or bottlenecks did you face, and how did you solve them?"** — have **two** ready, one performance and one behavioural/logic.

	Story
Performance	The dashboard's four sequential calls + an N+1 on the study list → aggregate endpoint, <code>forkJoin</code> , <code>JOIN FETCH</code> . ~2s → under 0.5s. → A10
Design under a constraint	Real-time computation + AI explanation: instant local compute, then a <code>debounceTime</code> + <code>switchMap</code> remote call so an in-flight request is cancelled on the next keystroke — otherwise we flooded the AI backend with one request per character

Structure each one: *what was wrong → how I knew → what I tried → what actually fixed it → the measurable result.*

④ ● "If you could redesign that module differently, what approach would you take?"

This question is a **maturity test**, and it's the one people fumble — they either defend the original design (reads as inflexible) or trash it (reads as if they didn't think it through the first time). **The right answer does three things: name a real limitation, propose a concrete alternative, and say what the original constraint was.**

"Two things. The reminder dispatch is a @Scheduled job polling the table, which was the right call for the scale and timeline we had — but it doesn't survive running on multiple instances cleanly, and the polling query gets hot as the table grows. I'd move due reminders onto a delay queue and keep the database purely as the source of truth, so the dispatcher isn't polling at all and horizontal scaling is free rather than something I have to defend with a lock."

"The second is that the learner APIs grew feature by feature, so the dashboard needed four calls before I aggregated them. If I started again I'd design the client's screen contract first and shape the endpoints to it — effectively a BFF — instead of exposing entity-shaped endpoints and stitching them together on the client."

"What I wouldn't change is keeping the AI explanation behind its own service. That boundary earned itself — it fails differently and scales differently from everything else."

● **That last sentence is the one that lands.** Naming something you'd *keep*, with a reason, proves you're evaluating rather than just apologising.

Bucket B — technical & scenario

⑤ ● "Walk me through your structured approach to troubleshooting a live production issue."

They want a **process**, not an anecdote. Answer in named steps — it's the single clearest signal of someone who has actually been on a production call.

Step	What you say
1. Assess impact	"How many users, which flow, is it total or degraded? That decides urgency and who needs to know."
2. Communicate immediately	"A short factual update to the client/lead before I start digging — 'checkout is failing for ~10% of users, investigating, next update in 30 minutes.' Silence is what damages trust, not the bug."
3. Mitigate before diagnosing	● "Restore service first — roll back the release, disable the feature flag, scale out. Root cause can wait; the outage can't."
4. Narrow it down	"What changed? Last deploy, config change, data volume, an upstream dependency. Then logs, the trace id across services, / actuator/health , DB slow query log, thread dump if it's hung."
5. Fix and verify	"Fix, verify in a lower environment where possible, deploy, and confirm with the same metric that showed the problem."
6. Prevent recurrence	"A regression test and, usually more importantly, the alert that would have caught it before the client did. Then a short RCA write-up."

◆ **The line to close on:** "The order matters more than the tooling — communicate, mitigate, then diagnose. The instinct is to jump straight to the interesting technical problem, and that's the mistake."

Pair it with a real story — C0's duplicate-reminder incident works: symptom (users got the same reminder three times), first hypothesis, actual cause (the scheduler firing on every replica), the fix (atomic claim + idempotent send), and the prevention (an alert on duplicate sends per user per day).

⑥ **"Deep questions on your core stack."** Expect the client's own priorities here — **advanced SQL** (Part G), **framework internals** (Parts D, E), CI/CD and cloud if the JD mentions them. Know your AWS/Azure honestly: EC2, S3, Lambda, RDS, CloudWatch — and say where the boundary of your experience is rather than being caught at it.

⑦ ● **"You're on a tight deadline / the client changes requirements mid-sprint. What do you do?"**

The wrong answers: *"I work extra hours"* (unsustainable, and it tells them you'll hide problems) and *"I refuse, it's out of scope"* (unworkable for a services engagement).

"First I'd understand what's actually driving the change and how urgent it really is — sometimes there's a demo date behind it and only part of it is needed by then. Then I'd make the trade-off explicit rather than silently absorbing it: 'this is roughly three days; if it comes into this sprint, either X moves out or we go a few days beyond — which do you prefer?' That's a decision for the lead and the client to make, not for me to make quietly by working weekends."

"If it genuinely can't move, I'd look for a smaller version that delivers the value now — the core path working, the edge cases in the next sprint — and be explicit that it's a first cut, not the finished thing. What I won't do is silently cut testing and let it surface as a defect later, because that costs more than the delay would have."

● **The principle to name:** *"Scope, time, quality — you can't hold all three. My job is to make sure the person who owns the priority is the one choosing which one gives."*

Bucket C — client interaction & delivery (the one candidates skip)

⑧ ● **"How do you communicate a technical blocker or delay to a non-technical stakeholder?"**

"Three things, in this order: impact, then cause in plain language, then options with a recommendation. I lead with what it means for them, not the technical detail. So — 'the reports feature won't be ready for Thursday's demo' first, then 'the third-party data feed is returning a different format than documented', then 'we can either demo with sample data on Thursday and go live Monday, or push the demo — I'd suggest the first'."

"And I raise it as soon as I know it's a risk, not when it becomes a fact. A stakeholder can plan around a delay they hear about a week early; they can't do anything with one they hear about the morning it's due. No jargon, no blame, and always at least one option — arriving with only a problem puts the whole thing on them."

	
<i>"The API is throwing a 500 because of a serialization issue in the DTO mapper."</i>	<i>"The reports page won't load — an integration with their data feed is failing. Fix is about a day."</i>
Reporting it on the due date	Flagging it as a risk the moment you see it
Problem with no options	Two options and a recommendation
<i>"The other team broke it"</i>	<i>"Here's where it's blocked and what I need to unblock it"</i>

⑨ ● **"How do you handle conflicting priorities between internal Mphasis guidelines/process and a direct client request?"**

This is a **loyalty and judgment test**, and it's specific to working at a services company. They want to know you won't go rogue for the client, and won't hide behind process either.

"My default is that the client's priority is the priority — that's the point of the engagement. But I wouldn't just quietly bypass our process, because the process usually exists for a reason someone else owns — code review, security review, the release calendar."

"So practically: I'd take the request seriously and immediately, and if honouring it conflicts with something internal, I raise it with my Mphasis lead the same day rather than deciding on my own. Most of the time it's not a real conflict once someone with the full picture looks at it — an approval gets expedited, or the client's need is met a slightly different way."

"The one place I'd hold firm is anything touching security, data handling or a compliance control, where 'the client asked for it directly' isn't enough on its own — that has to go through the right approval. Everything else I'd treat as a scheduling conversation, not a standoff."

● **The sentence that answers what they're really asking:** *"What I wouldn't do is either of the two failure modes — go around my own organisation to please the client, or hide behind process and tell the client no. Both of those end up on my lead's desk anyway, just later and worse."*

⑩ **"Have you worked directly with clients or onshore teams?"** → say yes if you have, with a specific example — a demo you ran, a requirement you clarified directly, a standup with an onshore team. **Mphasis is a services company: the client is buying communication as much as code**, and this round exists because someone got burned by an engineer who couldn't do that part.

If your client exposure has been limited, say so honestly and pivot to evidence of the underlying skill: *"Most client contact went through our lead, but I've presented my modules in sprint demos and worked directly with the BA on requirements, so I'm comfortable being the person explaining the work."*

⑪ **"What do you do when you disagree with a technical decision?"** → *"Make the case once, with reasoning and preferably a small proof — then commit to the decision either way. What I won't do is quietly build it my way, or agree in the meeting and complain afterwards."*

⑫ **"How do you approach a ticket you don't understand?"** → *"Read the ticket and the surrounding code first and try to reproduce it, so that when I do ask, I'm*

asking something specific rather than 'I don't understand this'. I'd rather spend twenty minutes clarifying than two days building the wrong thing."

⑬ **"How do you estimate?"** → *"Break it into the pieces I actually know — API, component, tests, integration — estimate each, and add buffer for the unknowns and for review and QA feedback. And I re-check against the estimate mid-way; if it's slipping I raise it then. Nobody minds an estimate being wrong. They mind finding out on the due date."*

⑭ **"Tell me about your agile process."** → sprint length, ceremonies (standup, planning, grooming, retro), Jira, story points, definition of done, PR review on Bitbucket/GitHub. ● **Have your real numbers:** team size, sprint length, release cadence. Vagueness here reads as not having been close to the process.

⑮ **"Are you comfortable with our stack / picking something new up?"** → yes, with **evidence rather than enthusiasm:** *"I've moved between Angular+Grails, Angular+.NET and Angular+Java across three projects, and I led an Angular version migration across four repositories. Picking up a new stack is something I've actually done, not just something I'm willing to do."*

⑯ **"How do you write tests?"** → be honest and specific. Jasmine/Karma for Angular components (you wrote these on EasyVisa), JUnit + Mockito for services. *"I mock the repository and assert behaviour, and I make a point of testing the edge cases — nulls, empty collections, the error path — because that's where the bugs actually are."*

⑰ **"Where do you see yourself / why this role?"** → growth, scale, enterprise exposure. **Name the client's domain if you know it** — *"and this is a much larger platform than I've worked on, which is exactly the exposure I'm looking for."*

K3. Red flags to avoid — each one has sunk a client round

✗ Don't say	✓ Say instead
"I only did the frontend part."	"I owned the frontend and the APIs behind it; the AI engine was another team's."
"My manager assigned it, I just did it."	"I picked it up in planning and owned it through to release."
"We used microservices" (with no detail)	Name the services and why they're split that way.
Badmouthing your current company	"I've learnt a lot; I'm looking for larger scale."
"I don't know" and stopping	"I haven't used that directly. What I have done that's closest is... and I'd approach it by..."
Vague numbers	"Team of six, two-week sprints, released every sprint."
"I'd just put in extra hours to hit the date."	"I'd make the trade-off explicit and let the lead choose what moves."
"The client asked, so I'd just do it."	"I'd take it seriously and raise the process conflict with my lead the same day."
Explaining a blocker in jargon	Impact first, in their language, with two options
Defending your design as flawless	Name one real limitation — and one thing you'd deliberately keep

K4. ● Questions YOU ask — never skip this

Not asking questions reads as not being interested. Have four ready; ask two or three.

1. "What does the team look like, and where would I fit in it?"
2. "What's the current state of the codebase — greenfield, or maintaining and extending something existing?"
3. "What would you want me to have accomplished in the first three months?"
4. "What's the biggest technical challenge the team is facing right now?"

5. "How do the onshore and offshore teams work together day to day?"

◆ Question 4 is the best one — the answer tells you exactly what they need, and asking it makes you sound like someone thinking about the *work*, not the *job*.

L — Logistics & fit questions

Have these answers ready and consistent — HR will cross-check them against what you told them earlier.

Question	The approach
Notice period	State it exactly. If negotiable, say by how much. Don't guess.
Why are you leaving?	Growth and scale. Never money first, never negative about Provility.
Current & expected CTC	Know your number before the call. Give a range with a reason ("based on my experience and the market for full-stack Angular/Java").
Location / onsite days	Know the client's location and be clear about what you can do.
Any other offers in progress?	Honest but brief. It's leverage, not a threat.
Total experience	5 years , since Oct 2021 at Provility. Consistent everywhere.
Are you okay with a support/maintenance project?	Say yes if you are — many client projects are. <i>"I'd want some feature work too, but I'm comfortable with production support."</i>

M — Your 48-hour plan

Adjust to the time you actually have. **Do it in this order — it's sorted by return on effort.**

Day 1 — the re-probes and the gaps

Time	What
90 min	● Part A — the 10 re-probes. Say every answer out loud . This is the highest-yield block in the file.
60 min	● Part G — SQL . Type the 7 queries. It's your newest gap and they've already asked once.
60 min	Part D4 — @Transactional + Part E1 — N+1 . The two most-asked Spring/JPA topics anywhere.
45 min	Part B1–B2 — HashMap internals, collections table.
30 min	Rehearse K2 ① — "walk me through your project" until it's smooth. Time it. Aim for 2 minutes.

Day 2 — coding, Angular, and the client round

Time	What
60 min	Type 5 problems from Part H . In an IDE, no autocomplete help, out loud.
60 min	● Part C — multithreading. Start with C0 (the two they already asked) and C10 (threading in Spring), then the C13 rapid-fire .
45 min	Part J — narrate all four project stories out loud. Not read — <i>narrate</i> .

Time	What
60 min	● Part K — the client round. Rehearse ① the architecture walkthrough, ④ what you'd redesign, ⑤ the production-troubleshooting process, and the whole of Bucket C — the delivery/communication questions are the ones candidates don't prepare. Pick your 3 questions to ask.
30 min	15 problems from 23 — Predict the Output .
20 min	Part L — get your numbers straight.

The morning of

Read only **Part N** and **Part A's table**. Nothing new. New material on the day creates doubt, not knowledge.

N — One-page cheat sheet

◆ The three rules

- 1. **Three sentences, always.** Definition → *something from my project* → the trade-off or failure mode. This is the exact gap the **pattern analysis** says has been losing you rounds — the answers were right, the second sentence was missing.
- 2. **Everything routes back to RoboGebra.** They can't check your theory against reality, but they can hear whether you've done it.
- 3. **"I haven't used that" is a complete, safe answer** — as long as it's followed by *"the closest I've done is..."*. Bluffing is what actually fails a round.

▲ The facts to have instant

Circuit breaker	Resilience4j + ● spring-boot-starter-aop . CLOSED → OPEN → HALF_OPEN
-----------------	---

Thread-safe singleton	Bill Pugh static holder. DCL needs <code>volatile</code> (reordering → partially constructed object)
Spring singleton ≠ Java singleton	Per- container , not per-JVM. Mutable fields on a bean = race condition
Deadlock	4 Coffman conditions; fix = global lock ordering or <code>tryLock</code> with timeout
Diagnose a hang	3 thread dumps 10s apart; 0% CPU = locks/pool, 100% = GC or loop
HashMap	16 / 0.75 / treeify at 8 with table ≥ 64. Mutable key = lost entry
<code>equals</code> / <code>hashCode</code>	Equal objects must have equal hashes; the converse isn't required
<code>@Transactional</code> traps	Self-invocation · rolls back only on <code>RuntimeException</code> · private/final methods
N+1	<code>JOIN FETCH</code> / <code>@EntityGraph</code> / <code>@BatchSize</code> · ▲ <code>JOIN FETCH</code> + <code>Pageable</code> pages in memory
Constructor injection	<code>final</code> fields · fails fast at startup · testable with plain <code>new</code>
SQL <code>LEFT JOIN</code> + <code>COUNT</code>	● <code>COUNT(column)</code> not <code>COUNT(*)</code> — otherwise an empty department returns 1
<code>WHERE</code> vs <code>HAVING</code>	Rows before grouping vs groups after aggregation
<code>RANK</code> vs <code>DENSE_RANK</code>	Gaps after a tie vs no gaps
Index ignored when	Function on the column · leading <code>%</code> · implicit cast · low selectivity
<code>volatile</code>	Visibility only — <code>count++</code> is still broken. Use <code>AtomicInteger</code>
<code>forkJoin</code> fails	● One error kills all of them → <code>catchError</code> per stream

Unsubscribe	<code>async pipe · takeUntil(destroy\$) · takeUntilDestroyed()</code>
JWT	Signed, not encrypted — never put secrets in the payload

● Client round — the five answers to have ready

Question	The shape of the answer
Architecture end to end	Draw the boxes → then say where you sit on the diagram
Your contribution vs the team's	Volunteer the boundary . "I owned X and Y; Z was another developer's"
What would you redesign?	One real limitation + a concrete alternative + one thing you'd keep, and why
Production issue — your process	● Assess → communicate → mitigate → diagnose → fix → prevent . In that order
Blocker to a non-technical stakeholder	Impact → plain-language cause → two options + a recommendation . Raise it as a <i>risk</i> , early
Client request vs internal process	Client's priority is the priority, but escalate to your lead the same day rather than going around either side. Hold firm only on security/compliance

■ Lines worth having word-perfect

- "You can always split a well-structured monolith; you can't easily un-split a distributed system."
- "It's proxy-based, so anything that bypasses the proxy silently disables it — and silent is what makes it dangerous."
- " `volatile` fixes visibility, not atomicity."
- "A compensating transaction isn't a rollback — it's a new business transaction that undoes the previous one."

- "I owned that feature end to end — the Angular components and the APIs behind them."
- "Communicate, mitigate, then diagnose. The instinct is to jump straight to the interesting technical problem, and that's the mistake."
- "Scope, time, quality — you can't hold all three. My job is to make sure the person who owns the priority is the one choosing which one gives."
- "What I wouldn't do is go around my own organisation to please the client, or hide behind process and tell the client no."
- "I haven't used that directly. The closest I've done is , **and I'd approach it by** ."

✗ The five things not to do

1. Don't answer in one sentence and stop.
2. Don't claim anything on your resume you can't take three follow-ups on.
3. Don't go silent while coding — narrate.
4. Don't say "we" when you mean "I", or "I" when you mean "we".
5. Don't skip asking them questions at the end.

✓ **You reached round two.** In six companies that hasn't happened before — the L1 answers were good enough. The work now is depth on the follow-up, which is the one thing the [pattern analysis](#) has been pointing at all along.

After each round, log every question in 26 the same day — append to the Mphasis section, don't start a new file.

● HashMap Internals — Easy Version

This is the file for the exact questions Virtusa Round 2 asked (Aug 2026):

"How does HashMap internally work?" · "What is a bucket / bin?" · "What is the internal structure of a bucket?" · "Does a bucket contain a linked list?"

You have hit HashMap internals in **three** rounds now (Altimetrik, Mphasis L1, Virtusa R2). It is the single most-asked Java question in your log.

The trap in this question: most candidates say *"it uses hashing and stores in buckets"* and stop. The interviewer is waiting for **five** specific things:

1. the **Node[] table** array
2. **hash()** spreading + **(n-1) & hash** indexing
3. a bucket is a **linked list of Node objects**
4. it becomes a **red-black tree at 8 nodes** if the table is ≥ 64
5. **resize doubles capacity at load factor 0.75**

Say all five and the question is over.

Table of contents

- [Part 0 — The 90-second answer \(memorise this\)](#)
- [Part 1 — The picture that explains everything](#)
- [Part 2 — The internal data structure](#)
- [Part 3 — What exactly is a bucket?](#)
- [Part 4 — put\(\) step by step](#)
- [Part 5 — get\(\) step by step](#)
- [Part 6 — Collisions: linked list → red-black tree](#)
- [Part 7 — Load factor, threshold and resize](#)
- [Part 8 — hashCode\(\) and equals\(\) contract](#)
- [Part 9 — Java 7 vs Java 8 \(the infinite-loop story\)](#)

- Part 10 — HashMap vs the other Maps
- Part 11 — Thread safety & ConcurrentHashMap
- Part 12 — Follow-up questions they ask next
- Part 13 — Predict the output
- Part 14 — Rapid-fire table
- Part 15 — Drills

Part 0 — The 90-second answer (memorise this)

"A `HashMap` is an array of buckets — internally `Node<K,V>[]` table. On `put(key, value)` it calls `key.hashCode()`, then applies a spreading function `h ^ (h >>> 16)` so the high bits also influence the result, and finds the bucket index with `(n - 1) & hash`, where `n` is the table length — that works as a fast modulo because the capacity is always a power of two.

Each bucket holds a `Node<K,V>` object — hash, key, value and a `next` pointer — so a bucket is really a singly linked list of Nodes. When two different keys land in the same bucket (a collision), the new node is appended to the end of that list. On lookup, Java walks the list comparing `hash` first and then `equals()`.

Since Java 8, if one bucket grows to 8 nodes and the table is at least 64 long, that bucket is converted from a linked list into a red-black tree, which drops worst-case lookup from $O(n)$ to $O(\log n)$. If it shrinks back to 6 during a resize it turns back into a list.

The map also grows: default capacity 16, load factor 0.75, so at 12 entries it doubles the table to 32 and redistributes the nodes. Average lookup is $O(1)$."

If they only let you say one sentence:

"Array of buckets; each bucket is a linked list of Nodes that becomes a red-black tree after 8 collisions; it doubles in size at 75% full."

Part 1 — The picture that explains everything

The easiest way to remember the whole topic:

```
hashCode() → WHERE to look    (which bucket)
equals()   → WHICH one it is (which entry in that bucket)
```

Real-world idea: a **postal address**.

```
PIN code    → tells the postman which AREA      → hashCode()
House name  → identifies the exact HOUSE there → equals()
```

Many houses share a PIN code — that is a **collision**, and it is completely normal. The postman goes to the right area, then reads the name boards one by one.

Second real-world idea, for the structure itself: a **library**.

```
Subject shelf → hashCode() picks the shelf
Book title    → equals() finds the exact book on that shelf

One shelf with 3 books → a small linked list, walk it
One shelf with 500 books → sort them into a tree so you can binary-search
```

★

And the picture of the actual memory:

```
table (Node<K,V>[16])

index 0 | null
index 1 | → Node{hash, "Ram", 25, next=null}
index 2 | null
index 3 | → Node{"Anu",30} → Node{"Kiran",41} → Node{"Zoe",19} →
null
index 4 | null                                ↑ collision chain – SAME bucket,
different keys
...
index 9 | → TreeNode root (red-black tree of 9+ entries)
          /      \
        TreeNode  TreeNode
index 15 | null
```

Keep this diagram in your head. Every question below is just a detail of this picture.

Part 2 — The internal data structure

Q: What are the actual fields inside `java.util.HashMap`?

```
public class HashMap<K,V> extends AbstractMap<K,V> implements Map<K,V>,
Cloneable, Serializable {

    transient Node<K,V>[] table;    // THE BUCKET ARRAY – created lazily on
the first put()
    transient Set<Map.Entry<K,V>> entrySet;
    transient int size;              // number of key-value mappings (NOT
table.length)
    transient int modCount;          // structural modification count →
fail-fast iterators
    int threshold;                  // capacity × loadFactor → resize when
size exceeds it
    final float loadFactor;          // default 0.75f
}
```

★ Notice `size` and `table.length` are **different things**. `size` is how many entries you stored; `table.length` is how many buckets exist. Interviewers ask this to check you're not guessing.

The constants they ask for by number

Constant	Value	Meaning
<code>DEFAULT_INITIAL_CAPACITY</code>	<code>1 << 4 = 16</code>	table length created on the first <code>put</code>
<code>MAXIMUM_CAPACITY</code>	<code>1 << 30</code>	upper bound on table length
<code>DEFAULT_LOAD_FACTOR</code>	<code>0.75f</code>	resize when <code>size > capacity × 0.75</code>
<code>TREEIFY_THRESHOLD</code>	<code>8</code>	list → red-black tree at this bin size

Constant	Value	Meaning
<code>UNTREEIFY_THRESHOLD</code>	6	tree → list when it shrinks to this during a resize
<code>MIN_TREEIFY_CAPACITY</code>	64	the table must be this long before treeifying; otherwise resize instead

▲ **The most-missed detail in the whole topic:** 8 alone does **not** treeify.

bucket reaches 8 nodes | `table.length < 64` ? → `resize()` instead ★
`table.length ≥ 64` ? → `treeify`

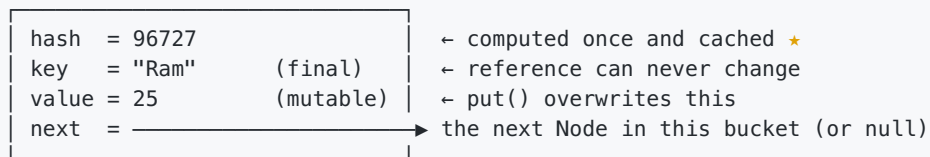
Why? In a small table, a long chain usually means **too few buckets**, not a bad `hashCode`. Growing the table is the cheaper cure.

Q: What is a `Node<K,V>` ?

The unit of storage. **One Node = one key-value mapping.**

```
static class Node<K,V> implements Map.Entry<K,V> {
    final int hash;    // cached spread hash – computed ONCE, never
    recomputed
    final K key;       // final: the key reference can never change
    V value;           // mutable: put() on an existing key overwrites this
    Node<K,V> next;    // the linked-list pointer → THIS is what makes a
    bucket a list
}
```

Picture one node:



Three things to say out loud in the interview:

1. ``hash`` is CACHED
 - `hashCode()` is called once per key, on insert
 - resizing and lookups reuse the stored value
 - that's why a resize is cheap-ish, and why an expensive `hashCode()` hurts less than people assume
2. ``key`` is FINAL
 - you can never change WHICH object a node points to as its key
 - which is exactly why MUTATING a key after insertion is a bug (Part 8)
3. ``next`` exists on EVERY node, even when the bucket has one entry (it's just null)
 - the linked list is not a separate object – it IS the chain of ``next`` pointers

Q: What is a `TreeNode`?

The tree version of a node, used after treeification:

```
static final class TreeNode<K,V> extends LinkedHashMap.Entry<K,V> { //
    which extends HashMap.Node
        TreeNode<K,V> parent, left, right, prev; // prev keeps the doubly-
        linked order
        boolean red; // the red-black colour bit
    }
```

◆ **Nice detail to drop:** a `TreeNode` still inherits `next` from `Node`, so a treeified bin is **simultaneously a red-black tree AND a doubly-linked list**. That is how it can be untreeified back into a plain list cheaply during a resize — the list is still right there.

Part 3 — What exactly is a bucket?

Q: What is a bucket / bin in a `HashMap`?

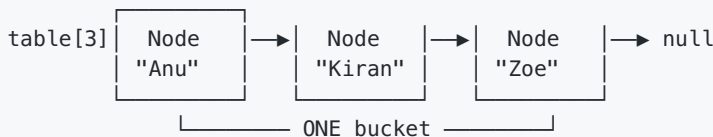
The easiest way to remember:

A bucket = ONE SLOT of the internal array → `table[i]`

It is NOT a class. It is just an array position.

That slot holds a reference to the **first node** of whatever lives there:

```
table[i] = null      → empty bucket
table[i] = Node      → a SINGLY LINKED LIST (1 or more entries chained by
`next`)
table[i] = TreeNode  → the ROOT of a red-black tree (after 8+ collisions,
table ≥ 64)
```



Real-world idea: a **shelf in a shoe rack**. The shelf itself is just a space. What's on it might be nothing, a row of boxes, or a neatly sorted arrangement.

Q: So does a bucket contain a linked list? (their exact words)

"Yes — by default a bucket is a singly linked list of `Node` objects, chained through the `next` field. It's not a `java.util.LinkedList`; it's a hand-rolled chain inside `HashMap`, which avoids an extra object per bucket.

But it is not always a list — since Java 8, once a single bucket holds 8 nodes and the table is at least 64 long, that bucket is converted into a red-black tree, so a pathological bucket degrades to $O(\log n)$ instead of $O(n)$. So the precise answer is: a bucket is either null, a linked list, or a red-black tree."

▲ **Trap they follow with:** "Is it a `LinkedList`?"

No. Saying "it stores a `java.util.LinkedList` in each bucket" is a wrong answer that *sounds* right. It is a chain of `HashMap.Node` objects, with the `next` pointer built into the node itself.

```
java.util.LinkedList → a separate object, with its own header and Node
class ✗
HashMap bucket      → the entries chain THEMSELVES via
`next`              ✓
```

Q: Why chaining and not open addressing / linear probing?

Java chose **separate chaining**: collisions go into a structure hanging off the bucket. The alternative, open addressing, would put the colliding entry in the *next free slot* of the array.

	Chaining (HashMap)	Open addressing (probe the next slot)
Deletion	simple — unlink the node	needs "tombstone" markers
Load factor > 1	still works, just slower	breaks down badly near 1.0
Memory	one Node object per entry	no per-entry object
Cache locality	worse (pointer chasing)	better

Real-world idea:

```
Chaining      → two families share a PIN code; both keep their own house
Open addressing → the second family is told "go live in the next empty
house"
               → now deleting the first family confuses every future
search
```

◆ **ThreadLocalMap** inside the JDK *does* use linear probing — a good "yes, I've read the JDK" remark if the conversation goes there.

Part 4 — put() step by step

Q: Walk me through `map.put("Ram", 25)`.

Four steps. Learn them in order.

Step 1 — hash the key (spread it)

```
static final int hash(Object key) {  
    int h;  
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);  
}
```

null key → hash 0 → ALWAYS bucket 0 (HashMap allows exactly ONE null key)
otherwise → hashCode() XOR its own upper 16 bits

Why the XOR? Because the index is computed with `(n-1) & hash`, and for a small table **only the low bits survive the mask**. Two keys differing only in their high bits would collide constantly. XOR-ing the high half down mixes those bits in, very cheaply. This is called **spreading** (or perturbation).

```
hashCode : 1111 1111 1111 1111 0000 0000 0000 0000  
h >>> 16 : 0000 0000 0000 0000 1111 1111 1111 1111  
XOR      : 1111 1111 1111 1111 1111 1111 1111 1111  
                                     ↑ high bits now influence the index ★
```

Real-world idea: shuffling a deck before dealing. Without the shuffle, cards that were together stay together.

Step 2 — find the bucket index

```
i = (n - 1) & hash;    // n = table.length, ALWAYS a power of two
```

`n = 16` → `n-1 = 15 = 11112` ← a clean 4-bit mask
`hash & 1111` keeps only the last 4 bits → a value 0..15 ✓

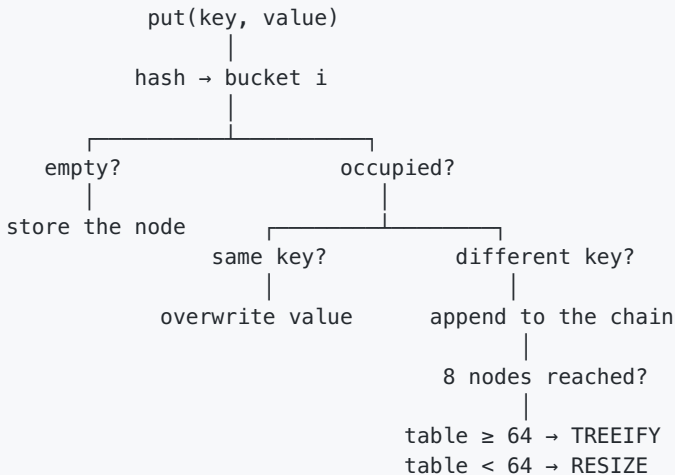
`&` with `n-1` is exactly the same as `hash % n` when `n` is a power of two, but a bitwise AND is far faster than a division. **That is the entire reason capacity is always a power of two.**

If you pass `new HashMap<>(20)`, Java does **not** use 20 — `tableSizeFor(20)` rounds it **up to the next power of two = 32**.

Step 3 — place the node

```
Is table null or empty?      → resize() creates the 16-slot array (LAZY INIT)
Is table[i] null?           → table[i] = new Node(hash, key, value, null) ✓ done
Else there's a COLLISION:
  └─ does the first node have the same hash AND (key == k || key.equals(k))?
    └─ → overwrite its value ✓ done
    └─ is the first node a TreeNode?
      └─ → putTreeVal() into the red-black tree
    └─ else walk the linked list:
      └─ matching key found → overwrite that node's value
      └─ reached the tail → append a new Node (TAIL insertion since Java 8)
        └─ if the bin now has ≥ 8 nodes → treeifyBin()
          └─ but if table.length < 64 → resize()
instead ★
Finally: if (++size > threshold) resize();
```

Draw it as the decision it is:



Step 4 — the return value

```
V old = map.put("Ram", 25);
```

Key already existed → returns the OLD value
Key was new → returns null

That is why `map.put(k, v) == null` is the common "was it new?" idiom — and also why it is **ambiguous** when the map legitimately stores `null` values. Use `containsKey()` when that matters.

Part 5 — get() step by step

Q: How does `get()` find the value?

```
final Node<K,V> getNode(int hash, Object key) {  
    // 1. compute the SAME spread hash  
    // 2. index = (n - 1) & hash  
    // 3. always check the FIRST node in the bucket (the common case – no  
    loop needed)  
    // 4. if that isn't it, and next != null:  
    //     TreeNode? → getNode(hash, key)    O(log n)  
    //     else      → walk the linked list   O(k)  
}
```

```
"Ram"  
↓  
hashCode() → spread → (n-1) & hash → bucket 7    ← STEP 1: WHERE  
↓  
walk bucket 7  
↓  
hash == hash? → == ? → equals()                  ← STEP 2: WHICH  
↓  
return the value
```

The comparison inside the loop — the part interviewers probe

```
if (e.hash == hash && ((k = e.key) == key || (key != null &&  
key.equals(k))))
```

Three checks, in a **deliberate order**, cheapest first:

1. `e.hash == hash` → an int compare, extremely cheap
→ rejects almost every non-match instantly ▲
2. `k == key` → reference equality
→ a free hit for interned Strings and cached Integers
3. `key.equals(k)` → only NOW does the potentially expensive user code run

Real-world idea: airport security. They check your **boarding pass** (instant), then your **face against the photo** (quick), and only then do a full **bag search** (slow).

◆ **Say this:** "*hashCode narrows it down to a bucket, equals decides the actual match — hashCode alone can never confirm equality, because two unequal objects are allowed to share a hash.*"

Complexity

Case	Structure	Lookup
Good hash distribution	bucket has 0–1 nodes	O(1) ▲
Heavy collisions, pre-Java 8 or small table	linked list	O(n)
Heavy collisions, Java 8+, table ≥ 64	red-black tree	O(log n)

Part 6 — Collisions: linked list → red-black tree

Q: What is a hash collision and how does HashMap handle it?

A collision = two DIFFERENT keys landing in the SAME bucket index.

It happens for two reasons:

1. The keys genuinely have equal hashCodes
2. Different hash codes collapse to the same index after $(n-1)$ & hash
→ e.g. hash 5 and hash 21 both give index 5 in a 16-slot table

Handling: the second key is **appended to the bucket's linked list**. Both entries live in the same bucket, and `equals()` tells them apart on lookup.

You can force a collision to see it:

```
class BadKey {
    private final String name;
    BadKey(String name) { this.name = name; }

    @Override public int hashCode() { return 1; } // ▲
    everything → bucket 1
    @Override public boolean equals(Object o) {
        return o instanceof BadKey b && b.name.equals(name);
    }
}
// 10 BadKeys → ALL land in ONE bucket → a 10-node chain → treeified once
// table ≥ 64
```

★ Important: a collision is **not a bug and not data loss**. Nothing is overwritten — the entries simply share a bucket. Only an *equal key* overwrites a value.

Q: Why was treeification added in Java 8?

Two reasons — give both.

1. PERFORMANCE under bad hash functions
A degenerate bucket turned `get()` into a linear scan.
1,000 colliding keys → 1,000 comparisons per lookup
2. SECURITY — hash-collision DoS ★ (this one impresses)
Attackers could craft thousands of colliding String keys and POST them as form or JSON parameters. The server's map operations became quadratic
and the CPU pinned at 100%.
Treeification bounds the damage to $O(\log n)$.

That second point is the one most candidates don't know. It is worth saying.

Q: Exactly when does a bin become a tree, and when does it revert?

An insert makes a bin reach 8 nodes

```
graph TD
    A[An insert makes a bin reach 8 nodes] --> B[table.length < 64 ? → resize() - double the table and re-split the bin]
    B --> C[chain up naturally]
    C --> D[table.length ≥ 64 ? → treeifyBin(): convert the chain to a red-black tree]
```

(a bigger table usually breaks the chain up naturally)

Later, during a resize, if a tree bin splits down to ≤ 6 nodes
→ untreeify back to a plain linked list

Why 8 and 6, and not 8 and 7? ★

Two separate reasons, and **both** score:

Why 8?

The JDK source cites a Poisson distribution.

With a good hashCode and load factor 0.75, the chance of any bucket reaching 8 entries is about 0.00000006.

- Trees are meant to NEVER be used in a healthy map.
- They are an insurance policy, not the normal path.
- And a `TreeNode` is roughly 2× the size of a `Node`, so you don't want them casually.

Why 6, not 7?

The GAP prevents thrashing.

If both thresholds were 8, a bin sitting exactly at the boundary would convert list → tree → list → tree on every single add and remove. ▲

The gap (8 up, 6 down) is hysteresis – flapping becomes impossible. ✓

Real-world idea: a **thermostat**. It switches the AC on at 26°C and off at 24°C, not both at 25°C — otherwise it would click on and off every few seconds.

Q: How does a red-black tree order keys that aren't Comparable ?

A good question to be ready for, because it shows real reading:

- 1. Order primarily by the HASH VALUE
- 2. If hashes tie AND the key implements Comparable AND both keys are the same class → use compareTo()
- 3. Otherwise → tieBreakOrder(): compare class names, then System.identityHashCode() – an arbitrary but CONSISTENT total order, purely to keep the tree balanced

So: making your key Comparable makes treeified bins behave better, but it is **not required**.

Part 7 — Load factor, threshold and resize

Q: What is the load factor and why 0.75?

The easiest way to remember:

Load factor = how FULL the table may get before it grows.
threshold = capacity × loadFactor

Default: $16 \times 0.75 = 12$
→ the 13th entry triggers a resize to capacity 32 (threshold 24)

It is a **space/time trade-off** — say it as one:

Load factor	Effect
Low (e.g. 0.5)	fewer collisions, faster lookups, more wasted memory , resizes more often
0.75	empirically the sweet spot — good collision behaviour, acceptable memory
High (e.g. 1.0)	dense table, less memory, longer chains → slower gets

Real-world idea: a **car park**.

25% full → find a space instantly, but you built a huge car park
75% full → a little searching, sensible size ✓
100% full → you circle for ten minutes

Q: What happens during a resize?

1. A new table of DOUBLE the capacity is allocated; the threshold doubles too
2. Every existing node is redistributed into the new table
3. ★ The clever Java 8 part:

Because the capacity doubles, a node's new index is either **the same index** or **index + oldCapacity** — decided by a single bit test:

```
if ((e.hash & oldCap) == 0) → stays at index j           (the "lo" list)
else                        → moves to index j + oldCap  (the "hi" list)
```

```
oldCap = 16, bucket 3 holds A, B, C, D
      |
      split by (hash & 16)
      └─ lo list → new table[3]
      └─ hi list → new table[19]      (3 + 16)
```

So Java 8 **never recomputes the hash or the modulo** during a resize. It splits each bucket into two lists and attaches them — and it **preserves relative order**, which matters a lot (see Part 9).

Q: Why is resizing expensive, and how do you avoid it?

A resize is $O(n)$, and it happens REPEATEDLY while a map grows:
16 → 32 → 64 → 128 → 256 → ...

Inserting 1M entries into a default map means about 17 rehash passes.

The fix — size it up front:

```
// You know you'll hold ~1000 entries:
Map<String,User> m = new HashMap<>(1365);    // 1000 / 0.75 ≈ 1334 →
rounded to 2048

// Java 19+ has the ergonomic version – no arithmetic, no mistakes:
Map<String,User> m2 = HashMap.newHashMap(1000);
```

▲ **Trap:** `new HashMap<>(1000)` does **not** mean "holds 1000 without resizing".
capacity 1000 → rounded up to 1024 threshold = $1024 \times 0.75 = 768$ →
it resizes at the 769th entry ▲

Always **divide by the load factor first**.

◆ **Real-project line:** *"In the Subsea schedule loader we were building a map of ~40k parts inside a loop and it was resizing all the way up from 16. Pre-sizing the map removed a visible chunk of the load time — it's a one-line fix that people never make."*

Part 8 — hashCode() and equals() contract

Q: What is the contract?

1. `a.equals(b)` is TRUE → `a.hashCode() == b.hashCode()` MUST be true ✓
2. hashCodes equal → objects MAY OR MAY NOT be equal (collisions are legal)
3. `hashCode()` must be CONSISTENT — the same object returns the same value, as long as the fields used in `equals()` don't change

The one-line version:

<code>equals()</code> TRUE → hashCode SAME	✓	guaranteed
hashCode SAME → equals() TRUE?	✗	not guaranteed

Q: What breaks if I override `equals()` but not `hashCode()` ?

The classic. Two "equal" objects get different (identity) hash codes → they land in **different buckets** → the map never finds the entry.

```
class Employee {
    int id;
    Employee(int id) { this.id = id; }

    @Override public boolean equals(Object o) {
        return o instanceof Employee e && e.id == id;
    }
    // ✗ hashCode() NOT overridden
}
```

```
Map<Employee,String> map = new HashMap<>();
map.put(new Employee(1), "Priyanka");

System.out.println(map.get(new Employee(1))); // null ← equal object,
WRONG bucket
System.out.println(map.size());                // 1

map.put(new Employee(1), "Again");
System.out.println(map.size());                // 2 ← DUPLICATE "equal"
keys in one map ▲
```

Bucket 100 | Employee(1) → "Priyanka" ← stored here

Bucket 500 | (searched with the second Employee(1)) → empty → null ▲

The postman went to the wrong PIN code, so he never even read the name boards.

The reverse mistake (overriding `hashCode` only) does not lose data — both objects land in the same bucket, but `equals` says they're different, so you get two entries. Slower, and still wrong.

Q: What happens if a key is mutated after being put in the map?

You orphan the entry.

The node keeps the **old cached hash**, so the map still looks in the old bucket, but `equals()` now fails there — and the object's new hash points somewhere else entirely.

```
List<String> key = new ArrayList<>(List.of("a"));
Map<List<String>,String> map = new HashMap<>();

map.put(key, "v");
key.add("b");                                // ▲ the hashCode just changed

System.out.println(map.get(key));             // null – unreachable, but still
occupying memory
System.out.println(map.size());               // 1    (a slow memory leak in long-
lived maps)
```

```
put:  hash was 97 → stored in bucket 1
key mutated → its hash is now 3105
get:  looks in bucket 1 (the CACHED hash) → equals() fails → null
      the entry is invisible forever, but never garbage collected ▲
```

Rule to state:

"Map keys must be immutable — or at least, the fields used in `equals` / `hashCode` must never change. That's why `String`, `Integer` and records make ideal keys."

Q: How should you write `hashCode()` in real code?

```
public record EmployeeKey(int id, String dept) { }    // record → equals +
hashCode generated ✓
```

Or the classic form:

```

@Override public int hashCode() {
    return Objects.hash(id, dept);
}

@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Employee e)) return false;
    return id == e.id && Objects.equals(dept, e.dept);
}

```

★ The golden rule: **both methods must use exactly the same fields.**

▲ **Lombok trap they love:** `@Data` on a JPA entity generates `equals` / `hashCode` over **all** fields, including the DB-generated `id` and lazy collections.

new entity → id is null → hash X → put into a HashSet
 save() flushes → id becomes 42 → hash Y ▲ the entity is now unreachable in the set
 touching a lazy collection → triggers a query, or `LazyInitializationException`

See [24 — ORM/JPA](#).

Part 9 — Java 7 vs Java 8 (the infinite-loop story)

Q: What changed in HashMap in Java 8?

	Java 7	Java 8+
Bucket structure	linked list only	linked list → red-black tree at 8
Insert position	head insertion	tail insertion
Resize transfer	recomputes the index, reverses list order	splits into lo/hi lists, preserves order
Hash function	4 shifts + 4 XORs	a single <code>h ^ (h >>> 16)</code>
Entry class	<code>Entry<K,V></code>	<code>Node<K,V></code> (+ <code>TreeNode</code>)
Worst-case get	O(n)	O(log n)

Q: What was the famous Java 7 HashMap infinite loop?

Tell this as a story — it lands very well.

Java 7 resize used HEAD insertion, which REVERSED each chain.

Thread A starts transferring bucket 3: $A \rightarrow B \rightarrow C$

Thread B does the same at the same time and completes the reversal

Thread A wakes up mid-transfer, holding stale pointers

↓

Result: $A.next = B$ and $B.next = A$ ← a CYCLE

↓

A later `get()` walks that cycle FOREVER

The symptom was brutal:

100% CPU on one thread

No exception. No error. Nothing in the logs.

Teams only found it by taking a thread dump and seeing a thread stuck inside `HashMap.get`.

Java 8's lo/hi split **preserves order** and never reverses, so this specific infinite loop is gone.

▲ Do NOT conclude "so Java 8 HashMap is thread-safe."

It is still **not** thread-safe. You can still lose updates, see a stale `size`, or get a `ConcurrentModificationException`. Java 8 removed one specific catastrophic failure, not the need for synchronization.

Interviewers *actively fish* for the wrong conclusion here.

Part 10 — HashMap vs the other Maps

	HashMap	Hashtable	LinkedHashMap	TreeMap	ConcurrentHashMap
Ordering	none (bucket order)	none	insertion (or access) order	sorted by key	none
Structure	Node array + list/tree	Node array + list	HashMap + doubly-linked list	red-black tree	Node array + list/tree
Thread-safe	✗	✓ (method-level synchronized)	✗	✗	✓ (CAS + per-bin lock)
null key	1 allowed	✗ NPE	1 allowed	✗ NPE (needs compare)	✗ NPE
null values	✓	✗	✓	✓	✗
get/put	O(1)	O(1)	O(1)	O(log n)	O(1)
Since	1.2	1.0 (legacy)	1.4	1.2	1.5

Say this about Hashtable:

"Legacy — it locks the whole object on every method, so it doesn't scale. Nobody should choose it in new code; **ConcurrentHashMap** replaced it."

LinkedHashMap bonus — an LRU cache in 5 lines ★

A classic follow-up, and a genuinely useful thing to know:

```

class LruCache<K,V> extends LinkedHashMap<K,V> {

    private final int cap;

    LruCache(int cap) {
        super(cap, 0.75f, true);    // true = ACCESS order, not
insertion order ★
        this.cap = cap;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K,V> eldest) {
        return size() > cap;        // evict the least recently used
    }
}

```

access order = true
 ↓
 every get() moves that entry to the END of the internal linked list
 ↓
 the entry at the FRONT is the least recently used
 ↓
 removeEldestEntry() drops it when the cache is full ✓

HashSet — the one-liner that impresses

*"**HashSet** is literally a **HashMap**. It holds one internally and stores every element as a key, with a shared dummy **PRESENT** object as the value. That's why **HashSet.add** returns a boolean — it's just **map.put(e, PRESENT) == null**."*

```

private static final Object PRESENT = new Object();

public boolean add(E e) {
    return map.put(e, PRESENT) == null;
}

```

Part 11 — Thread safety & ConcurrentHashMap

Q: Is HashMap thread-safe? What are the options?

No. Concurrent `put` s can lose entries, corrupt a chain, or leave `size` wrong.

Option	How it locks	Verdict
<code>Hashtable</code>	<code>synchronized</code> on every method	legacy, one global lock ✗
<code>Collections.synchronizedMap(map)</code>	wrapper, one lock for the whole map	works, but iteration must be manually synchronized
<code>ConcurrentHashMap</code>	CAS for empty bins + <code>synchronized</code> on the bin head	✓ the right answer

Hashtable / synchronizedMap

◆ ONE lock for the WHOLE map

| → all threads queue

ConcurrentHashMap

- bucket 2 ◆ Thread A works ✓

bucket 9 ◆ Thread B works ✓

bucket 14 ◆ Thread C works ✓
- ← at the SAME time

Real-world idea: a bank with one key for the whole building versus a separate lock on each counter.

Q: How does ConcurrentHashMap work internally (Java 8)?

- SAME layout as HashMap – Node[] table, lists that treeify at 8
- NO more segments ★
 - Java 7 used ~16 Segment locks (ReentrantLock)
 - Java 8 REMOVED them entirely
- Empty bucket → insert with a CAS (compare-and-swap), NO lock at all ▲
- Non-empty bucket → synchronized (firstNode)
 - lock granularity is ONE BUCKET
 - two threads writing to different buckets never contend
- Resize is COOPERATIVE
 - threads that find a resize in progress HELP transfer bins (a ForwardingNode marks a bin that has already moved)
- size() uses a striped LongAdder-style counter (baseCount + CounterCell[]),
 - not a single contended field
- Iterators are WEAKLY CONSISTENT, not fail-fast
 - they never throw ConcurrentModificationException
 - they may or may not reflect concurrent updates
- NO nulls for keys or values
 - because map.get(k) == null would be ambiguous between "absent" and "mapped to null", and containsKey() can't be atomic with get() while other threads are mutating

Q: Is ConcurrentHashMap fully atomic?

▲ **Trap.** Individual operations are atomic, but **compound** ones are not.

<code>// ✗ RACE</code>	<code>// ✓ ATOMIC</code>
<code>if (!map.containsKey(k)) map.put(k, v);</code>	<code>map.putIfAbsent(k, v);</code>
<code>map.put(k, map.get(k) + 1);</code>	<code>map.merge(k, 1,</code>
<code>Integer::sum);</code>	
<code>if (map.get(k) == null)</code>	<code>map.computeIfAbsent(k, key -></code>
<code>load(key));</code>	
<code>map.put(k, load(k));</code>	

Thread A: containsKey → false	
Thread B: containsKey → false	← both saw "absent"
Thread A: put(v1)	
Thread B: put(v2)	← one update is silently lost ▲

The atomic methods to name: `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`.

Q: What is a fail-fast iterator?

HashMap iterators record modCount when created, and compare it on EVERY next().

Mismatch → ConcurrentModificationException ▲

```
for (String k : map.keySet()) {
    if (k.startsWith("tmp")) map.remove(k);    // ▲ CME
}
```

```
// ✓ iterator.remove()
Iterator<String> it = map.keySet().iterator();
while (it.hasNext()) {
    if (it.next().startsWith("tmp")) it.remove();
}

// ✓ or, simplest
map.keySet().removeIf(k -> k.startsWith("tmp"));
```

Say the caveat: *"Fail-fast is best-effort — it's a bug-detection aid, not a guarantee. You cannot rely on the exception being thrown."*

Part 12 — Follow-up questions they ask next

Q: Why is HashMap capacity always a power of two?

So that $(n - 1) \& \text{hash}$ is a valid, FAST substitute for $\text{hash} \% n$.

$n = 16 \rightarrow n-1 = 15 = 1111_2$ ← a clean low-bit mask, every index reachable ✓

$n = 15 \rightarrow n-1 = 14 = 1110_2$ ← the last bit is always 0
→ NO ODD INDEX can ever be produced
→ half the table is unreachable ▲

That is the killer detail: with a non-power-of-two capacity, **half your buckets would never be used.**

Q: Can HashMap have a null key? How, if hashing null would NPE?

```
static final int hash(Object key) {  
    int h;  
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);  
    //    ↑ SPECIAL-CASED — it never calls hashCode() on null ★  
}
```

Exactly ONE null key allowed → always bucket 0
Null VALUES → unrestricted, as many as you like

TreeMap → ✗ (it must compare keys)
ConcurrentHashMap → ✗ (ambiguous get())

Q: What is the time complexity of HashMap operations?

Operation	Average	Worst (Java 8+)
get / put / remove / containsKey	O(1)	O(log n) (treeified) — O(n) if the table is < 64
containsValue	O(n) — scans every bucket	O(n)
Iteration	O(n + capacity) — it walks empty buckets too	same

◆ **Nice depth:** "Iteration cost depends on capacity, not just size. A map that once held a million entries and was then cleared still has a huge table — `clear()` doesn't shrink it. `HashMap` never shrinks; only creating a fresh map does."

Q: Why is `String` a good `HashMap` key?

- 1. IMMUTABLE → the hash can never go stale ★
- 2. hashCode is CACHED → computed once, stored inside the `String` object
- 3. equals is cheap → length check first, then characters
- 4. Good distribution → the 31-based polynomial hash spreads well

The same reasoning makes **records** and **enums** good keys. For enums, `EnumMap` is even better — it is a plain array indexed by `ordinal()`, with no hashing at all.

Q: Is `HashMap` ordering guaranteed to be stable?

No. Iteration order depends on hash values **and the current capacity**, so it can change on a resize.

- Need insertion order? → `LinkedHashMap`
- Need sorted order? → `TreeMap`
- Never rely on `HashMap`'s order.

Part 13 — Predict the output

Q1

```
Map<String,Integer> m = new HashMap<>();
m.put("a", 1); m.put("a", 2);
System.out.println(m.size() + " " + m.get("a"));
```

Answer

`1 2` Same key (equal hash + equals) → the existing node's **value** is overwritten; the size is unchanged.

Q2

```
Map<Integer,String> m = new HashMap<>();
m.put(null, "x");
m.put(null, "y");
System.out.println(m.size() + " " + m.get(null));
```

Answer

`1 y` One null key is allowed, always in bucket 0; the second `put` overwrites it.

Q3 ★ (the mutable-key classic)

```
class K {
    int id;
    K(int id) { this.id = id; }
    public boolean equals(Object o) { return o instanceof K k && k.id ==
id; }
    public int hashCode() { return id; }
}
Map<K,String> m = new HashMap<>();
K k = new K(1);
m.put(k, "one");
k.id = 99;
System.out.println(m.get(k) + " " + m.get(new K(1)) + " " + m.size());
```

Answer

`null null 1`

```

put    → cached hash 1 → entry stored in bucket 1
k.id = 99

m.get(k)        → hashes to bucket 99 → EMPTY → null
m.get(new K(1)) → reaches bucket 1, but equals() compares against the
                  MUTATED object whose id is now 99 → false → null
m.size()        → still 1

```

The entry is **unreachable but still counted** — the mutable-key leak.

Q4

```

Map<String,String> m = new HashMap<>();
m.put("k", null);
System.out.println(m.get("k"));
System.out.println(m.get("missing"));
System.out.println(m.containsKey("k"));

```

Answer

``null`, `null`, `true`` ``get()`` returning null is **ambiguous** — only ``containsKey()`` distinguishes "present with a null value" from "absent". This is exactly why ``ConcurrentHashMap`` bans null values.

Q5

```

Map<String,Integer> m = new HashMap<>(3);
System.out.println(m.size());
m.put("a",1);

```

Answer

``0`` Two points worth making: 1. The table is **not allocated in the constructor** — it is lazily created on the first ``put``. 2. Capacity 3 is rounded up by ``tableSizeFor`` to **4**, so the threshold becomes $4 \times 0.75 = 3$.

Q6

```

Set<Employee> s = new HashSet<>(); // Employee overrides equals() ONLY
s.add(new Employee(1));
s.add(new Employee(1));
System.out.println(s.size());

```

Answer

`2` Different identity hash codes → different buckets → `equals` is never even consulted. The broken-contract classic.

Q7

```
Map<String,Integer> m = new HashMap<>();
m.put("A", 1); m.put("B", 2); m.put("C", 3);
for (String k : m.keySet()) if (k.equals("B")) m.remove(k);
System.out.println(m);
```

Answer

Throws `ConcurrentModificationException` ... *usually*. With exactly one removal it can sometimes finish silently, if the removal happens on the second-to-last element — `hasNext()` returns false before the `modCount` check runs. That unpredictability *is* the point: fail-fast is best-effort. Use `removeIf`.

Q8

```
Map<Integer,String> m = new HashMap<>();
for (int i = 1; i <= 5; i++) m.put(i, "v" + i);
System.out.println(m.keySet());
```

Answer

`[1, 2, 3, 4, 5]` — and this misleads people. Small `Integer` keys hash to themselves, so `(16-1) & i == i`: they land in buckets 1..5, in order. It is *not* an ordering guarantee. Add key 20 and it appears at bucket 4, out of order.

Part 14 — Rapid-fire table

Question	One-line answer
Internal structure?	<code>Node<K,V>[] table</code> — an array of buckets
What is a bucket?	one array slot: null, a linked list of Nodes, or a red-black tree
Fields of a Node?	<code>final int hash</code> , <code>final K key</code> , <code>V value</code> , <code>Node next</code>

Question	One-line answer
Index formula?	$(n - 1) \& \text{hash}$
Hash function?	$h \wedge (h \ggg 16)$ — spreads high bits into the low ones
Why power-of-two capacity?	makes $\&$ a valid, fast modulo (and keeps every index reachable)
Default capacity / load factor?	$16 / 0.75 \rightarrow$ resizes on the 13th entry
Treeify threshold?	8 nodes and table length ≥ 64
Untreeify threshold?	6
Why 8 up, 6 down (not 7)?	the gap prevents convert/revert thrashing
Why tree at all?	$O(\log n)$ worst case + hash-collision DoS defence
Resize behaviour?	doubles; splits each bin by $(\text{hash} \& \text{oldCap})$ into lo/hi lists
Null key?	one, hash forced to 0, bucket 0
Thread-safe?	x — use <code>ConcurrentHashMap</code>
CHM lock granularity?	CAS on an empty bin, <code>synchronized</code> on the bin's head node
Java 7 vs 8 insertion?	head $\rightarrow$ tail insertion
Java 7 infinite loop?	head-insert resize reversed chains $\rightarrow$ cyclic list under concurrency
Iterator type?	fail-fast (<code>modCount</code>); CHM is weakly consistent
get/put complexity?	$O(1)$ average, $O(\log n)$ worst
HashSet internally?	a HashMap with a dummy PRESENT value

Part 15 — Drills

Cover the answers and say them out loud.

1. Draw a 16-slot table and place keys with hashes 5, 21, 37. Which bucket, and why?

Answer

All three → **bucket 5**:

```
5 & 15 = 5
21 & 15 = 5
37 & 15 = 5
```

A 3-node chain. After a resize to 32:

```
5 & 31 = 5
21 & 31 = 21
37 & 31 = 5
```

→ the chain splits into bucket 5 (two nodes) and bucket 21.

2. A map has 100,000 keys all returning `hashCode() == 7`. What is `get()` complexity in Java 7 and in Java 8?

Answer

Java 7: **$O(n)$** — a 100,000-node linked list. Java 8: **$O(\log n)$** — the bin treeifies (the table is well over 64).

3. You need a map for ~500 entries. What capacity do you pass?

Answer

$500 / 0.75 = 667$ → the next power of two is 1024, so `new HashMap<>(667)` (or simply `HashMap.newHashMap(500)` on Java 19+). Passing 500 gives capacity 512 and threshold 384 → it resizes before you finish.

4. Why can't `ConcurrentHashMap` store null values?

Answer

`get()` returning null would be ambiguous between "absent" and "mapped to null", and you cannot disambiguate with `containsKey` atomically while other threads

mutate the map. Doug Lea calls it a design error in `HashMap` that he refused to repeat.

5. Two threads call `put` on the same `HashMap` at the same time. Name three things that can go wrong.

Answer

1. **Lost update** — both read `table[i]` as null and one overwrites the other. 2. **`size` becomes wrong** — `++size` is not atomic. 3. **Concurrent resize** can drop entries or, in Java 7, create a cyclic chain that pins a CPU forever.

6. Is it safe to use a mutable object as a key if you never mutate it? Is it *good practice*?

Answer

Safe in that instant, but poor practice — nothing enforces the discipline, and the failure is **silent**: the entry becomes unreachable with no exception at all. Prefer immutable keys: `String`, `Integer`, enums, records.

7. Explain to a junior why `hashCode` alone can never decide equality.

Answer

A hash maps a huge key space onto 32 bits, so collisions are mathematically unavoidable — the pigeonhole principle. The hash narrows the search to a bucket; `equals` decides the match. That is exactly the two-step lookup inside `getNode`.

Quick Revision Sheet

STRUCTURE

Node<K,V>[] table → bucket = ONE array slot
bucket = null | linked list of Nodes | red-black tree
Node = {final hash, final key, value, next}

put(k,v)

hash = h ^ (h >>> 16) ← spread the high bits
index = (n-1) & hash ← fast modulo (capacity is a power of 2)
empty → store | same key → overwrite | else → append to the tail

get(k)

hash → bucket → compare: hash == , then == , then equals()
hashCode = WHERE ★ | equals = WHICH ★

NUMBERS

capacity 16 | load factor 0.75 | resize at 12 (the 13th entry)
treeify at 8 nodes AND table ≥ 64 | untreeify at 6
resize: doubles, splits by (hash & oldCap) into lo / hi, preserves order

CONTRACT

equals TRUE → hashCode SAME ✓ | hashCode SAME → equals? ✗
override equals → ALWAYS override hashCode, using the SAME fields
MUTABLE KEY = orphaned entry = silent leak ▲

JAVA 7 → 8

head → TAIL insertion; resize no longer reverses chains
→ the Java 7 concurrent infinite loop is gone
→ but HashMap is STILL not thread-safe ▲

THREADS

ConcurrentHashMap: CAS on empty bins + synchronized on the bin head
no segments (Java 8), no nulls, weakly consistent iterators
compound ops need putIfAbsent / computeIfAbsent / merge

HashSet = a HashMap with a dummy PRESENT value

LinkedHashMap(cap, 0.75f, true) + removeEldestEntry = LRU cache

What to say when you don't know a detail

"I know the shape of it — array of buckets, chained Nodes, treeified after 8 collisions, resize at 0.75. I haven't memorised that specific constant, but I'd check `HashMap`'s source; the constants are all at the top of the class."

That is a **strong** answer. Bluffing a wrong number is much worse than that sentence.

Related files: [05 — Core Java](#) · [23 — Predict the Output](#) · [32 — Multithreading](#) · [26 — Companies: Questions Actually Asked](#)

● Java Multithreading & Concurrency — Easy Version

The topic that has now appeared in **Mphasis L1** ("*write a thread-safe singleton*", "*what is deadlock?*") and **Virtusa Round 2**. It is asked at every experienced-level Java round, and it's where 4-year candidates get separated from 1-year candidates — because the follow-up is never "what is a thread", it's "**why does that break?**"

How this round actually goes: one definition question to warm up (**Thread** vs **Runnable**), then straight into "*how do you make this thread-safe?*", "*synchronized vs volatile*", "*what's a deadlock and how do you avoid it*", and usually **one live program** — thread-safe singleton, producer–consumer, or print odd/even alternately with two threads. Have those three typed from memory.

Table of contents

- [Part 0 — The vocabulary they expect](#)
- [Part 1 — Creating threads](#)
- [Part 2 — Thread lifecycle](#)
- [Part 3 — The core problem: race conditions](#)
- [Part 4 — synchronized](#)
- [Part 5 — volatile and the Java Memory Model](#)
- [Part 6 — Atomic classes & CAS](#)
- [Part 7 — Locks: ReentrantLock, ReadWriteLock](#)
- [Part 8 — wait / notify and inter-thread communication](#)
- [Part 9 — Deadlock, livelock, starvation](#)
- [Part 10 — ExecutorService & thread pools](#)
- [Part 11 — Callable, Future, CompletableFuture](#)
- [Part 12 — Concurrent collections](#)

- Part 13 — Thread-safe singleton (the live-coding one)
- Part 14 — Classic live-coding programs
- Part 15 — Multithreading in Spring Boot (your real answer)
- Part 16 — Virtual threads (Java 21)
- Part 17 — Predict the output
- Part 18 — Rapid-fire table
- Part 19 — Drills

Part 0 — The vocabulary they expect

The one idea that organises the whole topic ★

Thread safety is THREE separate problems:

1. ATOMICITY → "did my whole operation finish without interruption?"
2. VISIBILITY → "can the other thread SEE my change?"
3. ORDERING → "did the CPU run my lines in the order I wrote them?"

And each tool solves a different subset:

<code>synchronized</code>	→ ✓	<code>atomicity</code>	✓	<code>visibility</code>	✓	<code>ordering</code>	(all three)
<code>volatile</code>	→ ✗	<code>atomicity</code>	✓	<code>visibility</code>	✓	<code>ordering</code>	(NOT
<code>atomicity!)</code>							
<code>AtomicInteger</code>	→ ✓	<code>atomicity on ONE variable, lock-free</code>					

◆ **The one-liner that impresses:** *"Thread safety is three separate problems — atomicity, visibility and ordering. **synchronized** gives you all three; **volatile** gives you visibility and ordering but not atomicity; atomics give you atomicity on a single variable. Choosing the wrong one is where concurrency bugs come from."*

The terms

Term	One-line definition
Process	an independent program with its own memory space
Thread	a unit of execution inside a process, sharing its heap; has its own stack and program counter
Concurrency	multiple tasks making progress in overlapping time (may be interleaved on 1 core)
Parallelism	multiple tasks executing at the same instant on multiple cores
Race condition	the result depends on unpredictable thread timing
Critical section	code that must be executed by only one thread at a time
Mutual exclusion	the guarantee that only one thread is in the critical section
Atomicity	an operation completes fully or not at all — no visible half-done state
Visibility	one thread's write becomes observable to another thread
Ordering	whether instructions may be reordered by the compiler or CPU
Context switch	the OS saving one thread's state and restoring another's — not free

Concurrency vs parallelism — the easy picture:

CONCURRENCY (1 cook, 2 dishes)

chop → stir → chop → stir → chop

← switching between tasks

PARALLELISM (2 cooks, 2 dishes)

cook A: chop chop chop

cook B: stir stir stir

← genuinely at the same time

What each thread owns vs shares

PROCESS (the JVM)

HEAP (objects, static fields)

← SHARED → needs synchronization

Metaspace (class data)

← shared

Thread-1: stack, PC, locals

← PRIVATE ✓

Thread-2: stack, PC, locals

← PRIVATE ✓

Real-world idea: a **shared office**.

Your own desk drawer (stack) → nobody else can touch it → always safe ✓

The shared printer (heap) → everyone uses it → you need a queue/lock

Say it: "Local primitives are always thread-safe because they live on the thread's own stack. The moment state is on the heap and shared — an instance field, a static field, a collection — you need synchronization."

Part 1 — Creating threads

Q: What are the ways to create a thread?

```
// 1. extend Thread – uses up your ONE inheritance slot ✗
class MyThread extends Thread {
    public void run() { System.out.println(getName()); }
}
new MyThread().start();

// 2. implement Runnable – preferred: separates the TASK from the WORKER ✓
class MyTask implements Runnable {
    public void run() { System.out.println("running"); }
}
new Thread(new MyTask()).start();

// 3. lambda (Runnable is a functional interface)
new Thread(() -> System.out.println("running")).start();

// 4. Callable + ExecutorService – returns a value, can throw ✓ ✓ (real
production code)
ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> f = pool.submit(() -> 42);
System.out.println(f.get());
pool.shutdown();

// 5. Java 21 virtual threads
Thread.startVirtualThread(() -> System.out.println("virtual"));
```

Why **Runnable** over **Thread**?

1. Java allows only ONE parent class – don't waste it on Thread
2. Separates WHAT to do (Runnable) from WHO runs it (Thread)
3. The task can be REUSED across pools and executors ★

Real-world idea: a **delivery company**.

```
extends Thread      → every parcel becomes its own delivery boy
implements Runnable → parcels (tasks) go to a pool of delivery boys ✓
```

In production you rarely create a `Thread` at all — you submit tasks to an `ExecutorService`.

Q: `start()` vs `run()` — the most-asked trap ★

```
Thread t = new Thread(() ->
    System.out.println(Thread.currentThread().getName()));

t.start();    // prints "Thread-0" → a NEW thread; the JVM calls run() on
it ✓
t.run();      // prints "main" → a plain method call, NO new thread at
all ✗
```

```
start()
  ↓
JVM asks the OS for a new thread + a new stack
  ↓
that new thread calls run()           ← real concurrency ✓

run()
  ↓
just an ordinary method call on the CURRENT thread ← nothing concurrent
✗
```

Interviewers love this because the wrong version **compiles and appears to work**.

Q: What happens if you call `start()` twice?

`IllegalThreadStateException`.

```
t.start();
t.start();    // ▲
```

A `Thread` object is **single-use**. Once it has terminated it can never be restarted — you must create a new one. (Another reason thread pools exist: they reuse the *thread*, not the *Thread object's task*.)

Real-world idea: a **used train ticket**. It got you there once; you cannot travel on it again.

Q: Daemon vs user thread?

USER thread → keeps the JVM ALIVE until it finishes ✓
DAEMON thread → does NOT keep the JVM alive; killed abruptly when the last user thread ends (no `finally`, no cleanup) ▲

```
Thread t = new Thread(() -> cleanupLoop());  
t.setDaemon(true);      // ▲ must be called BEFORE start()  
t.start();
```

The GC and JIT compiler threads are daemons.

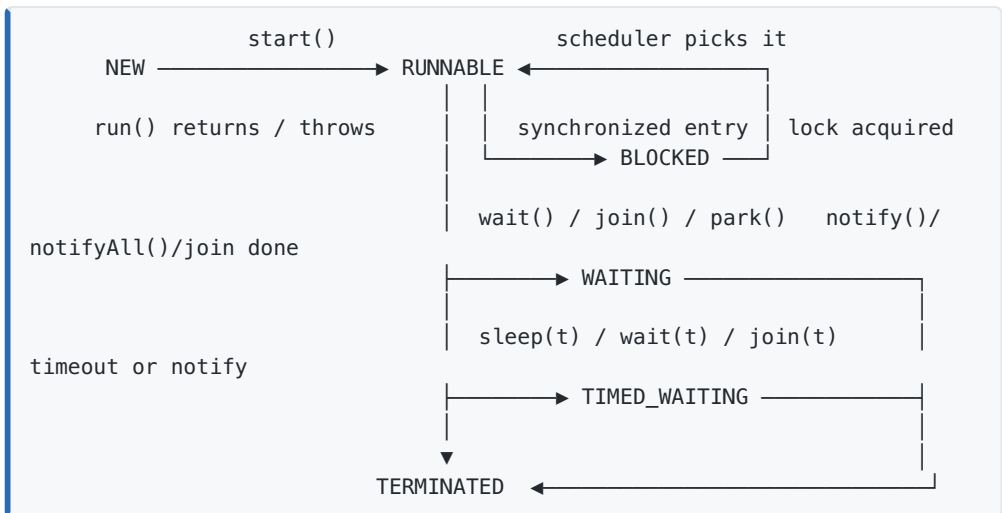
✓ Use daemons for: background housekeeping, monitoring, cache refresh
✗ NEVER for: writing a file, flushing a queue, anything that must finish

Real-world idea: the **office cleaner**. When the last employee leaves, the building closes and the cleaner goes home — even mid-mop.

Part 2 — Thread lifecycle

The easiest way to remember:

NEW → RUNNABLE → (BLOCKED / WAITING / TIMED_WAITING) → TERMINATED



The six states are exactly `Thread.State` :

NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED

Real-world idea — a **doctor's clinic**:

- NEW → you decide to visit the doctor (haven't left home)
- RUNNABLE → you're in the waiting room with a token
- BLOCKED → the consulting room is occupied by another patient (forced wait)
- TIMED_WAITING → the nurse says "come back in 10 minutes"
- WAITING → "wait until we call your name" (no fixed time)
- TERMINATED → consultation over; the token cannot be reused

▲ **Trap:** "Is there a *RUNNING* state?" — **No.** Java collapses "ready to run" and "actually on a CPU" into **RUNNABLE**; which one it is right now is the OS scheduler's business, not the JVM's.

BLOCKED vs WAITING — the follow-up

BLOCKED → wants to ENTER a synchronized block, but someone holds the lock
→ wakes automatically when the lock is free
→ the thread did NOT choose this

WAITING → called wait() / join() and gave up voluntarily
→ needs notify() / notifyAll() to wake up
→ the thread CHOSE to wait

Q: sleep() vs wait() — asked in nearly every round ★

The one line that answers it:

sleep() KEEPS the lock ♦

wait() RELEASES the lock ♦

	Thread.sleep(ms)	obj.wait()
Defined in	Thread (static)	Object
Releases the lock?	✗ keeps every lock it holds	✓ releases that object's monitor
Needs a synchronized block?	no	yes — else IllegalMonitorStateException
Woken by	timeout / interrupt	notify() , notifyAll() , timeout, interrupt
State	TIMED_WAITING	WAITING (or TIMED_WAITING with a timeout)
Purpose	pause	coordinate between threads

Real-world idea:

`sleep()` → you fall asleep **INSIDE** the bathroom with the door locked ♦
nobody else can get in for the whole duration

`wait()` → you step **OUT**, unlock the door, and wait to be called ♦
someone else can go in and change the situation

The sentence: "`sleep` is a pause that keeps the lock — using it inside a synchronized block blocks everyone else for the whole duration. `wait` releases the lock so another thread can make the condition true and notify you."

Q: `join()`, `yield()`, `interrupt()` ?

```
t.join();           // the CALLING thread waits until t finishes
Thread.yield();     // a HINT: "I'm willing to give up the CPU" – no
guarantee
t.interrupt();      // COOPERATIVE cancellation – it does NOT stop the
thread
```

`join()` — waiting for results before aggregating:

```
main → starts t1, t2
main → t1.join() → waits here until t1 finishes
main → t2.join() → waits here until t2 finishes
main → now safely reads both results ✓
```

`interrupt()` is the one people misunderstand:

`interrupt()` does NOT kill the thread. ★

It sets a FLAG. If the thread is in sleep/wait/join it throws `InterruptedException` – AND CLEARS the flag.

```
// ✓ correct handling – never swallow it
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // restore the flag for callers
    above you
    return;                             // and actually STOP doing work
}
```

```
// x the classic bug – swallowing it
catch (InterruptedException e) { }           // the cancellation signal is
lost forever
```

Real-world idea: `interrupt()` is a **polite knock on the door** saying "please wrap up". It is not someone dragging you out.

▲ `stop()`, `suspend()` and `resume()` are **deprecated and dangerous**. `stop()` releases all locks instantly, leaving shared objects half-modified. The only sanctioned way to stop a thread is to ask it to finish — a `volatile boolean` flag or interruption.

Part 3 — The core problem: race conditions

Q: Show me a race condition.

```
class Counter {
    private int count = 0;
    public void increment() { count++; }           // ▲ NOT atomic
    public int get() { return count; }
}
```

```
Counter c = new Counter();
ExecutorService pool = Executors.newFixedThreadPool(10);
for (int i = 0; i < 1000; i++) pool.submit(c::increment);
pool.shutdown();
pool.awaitTermination(1, TimeUnit.MINUTES);

System.out.println(c.get());           // expected 1000 – prints 973, 991, 1000,
987...
```

Why? `count++` is **three** bytecode operations:

1. READ count
2. ADD 1
3. WRITE count

And two threads can slot into each other's gaps:

```
Thread A reads  count = 5
Thread B reads  count = 5      ← before A wrote back
Thread A writes count = 6
Thread B writes count = 6      ← A's increment VANISHED ("lost update") ▲
```

Real-world idea: **two people editing the same Excel file on a shared drive.** Both open the version with 5 rows, both add a row, both save. The file ends up with 6 rows, not 7. One person's work is silently gone.

★ Note it prints 1000 *sometimes*. That non-determinism is exactly what makes these bugs escape testing and appear in production.

Three correct fixes — know all three and when to use each

```
// 1. LOCK — works, coarsest, simplest
public synchronized void increment() { count++; }

// 2. CAS — best for a single counter ★
private final AtomicInteger count = new AtomicInteger();
public void increment() { count.incrementAndGet(); }

// 3. LongAdder — best under HIGH contention
private final LongAdder count = new LongAdder();
public void increment() { count.increment(); }      // read with
count.sum()
```

```
Few threads / simple          → synchronized
One counter, moderate traffic → AtomicInteger ★
Hot metrics counter, many threads → LongAdder (striped cells)
```

▲ **volatile** does NOT fix this.

```
java volatile int count; count++; // STILL BROKEN ▲
```

volatile guarantees you read a *fresh* value — not that read-modify-write is atomic. Stating this correctly is a strong signal.

Part 4 — synchronized

Q: What does `synchronized` actually do?

The easiest way to remember:

Every Java object has a MONITOR (an intrinsic lock – one key ♦).

`synchronized` = take the key on entry, give it back on exit
(including when an exception is thrown ✓)

It provides **all three** guarantees:

1. MUTUAL EXCLUSION → only one thread inside at a time
2. VISIBILITY → releasing the lock flushes your changes to main memory,
acquiring it refreshes your view ★
3. ORDERING → no reordering across the lock boundary

Point 2 is the one candidates forget, and it is half the value of `synchronized` .

Real-world idea: a **single toilet with one key**. One person inside at a time; and whatever they did in there is visible to the next person.

The three forms

```
// 1. synchronized instance method → locks THIS
public synchronized void a() { }

// 2. synchronized static method → locks the CLASS object (MyClass.class)
public static synchronized void b() { }

// 3. synchronized block → locks whatever object you name – the PREFERRED
form ★
private final Object lock = new Object();           // dedicated, private lock
object
public void c() {
    // unsynchronized preamble – stays parallel
    synchronized (lock) {
        // ONLY the critical section
    }
}
```

Method-level → locks the ENTIRE method → slower
Block-level → locks only the risky lines → faster ▲

◆ **Say this trade-off:** *"I prefer a synchronized block on a private final lock object over a synchronized method. A synchronized method locks **this**, which any outside code can also lock — so a caller can accidentally block your class. And a block lets me keep the critical section as small as possible, which matters for throughput."*

Common mistakes:

```
synchronized ("lock") { }           // ✗ String literals are POOLED and
shared globally
private Integer lock = 0;
synchronized (lock) { lock++; }     // ✗ the lock object itself changes →
broken

private final Object lock = new Object();
synchronized (lock) { }             // ✓ correct
```

Q: Do an instance method and a static method block each other?

No. Different locks.

```
public synchronized void a()      { count++; }    // locks THIS
public static synchronized void b() { count++; }    // locks MyClass.class
```

Thread 1 inside a() ♦ this
Thread 2 inside b() ♦ MyClass.class

Two DIFFERENT keys → both run at the same time → the shared field is corrupted ▲

A classic subtle bug when a class mixes instance and static synchronized state. Both must lock the **same** object.

Q: Is **synchronized** **reentrant**?

Yes. A thread already holding a lock can acquire it again — the JVM keeps a hold count.

```
synchronized void a() {
    b();                // ✓ no self-deadlock
}
synchronized void b() { }
```

Thread enters a() → hold count 1
Thread enters b() → hold count 2 (same thread, same lock – allowed)
b() returns → hold count 1
a() returns → hold count 0 → lock released

Without reentrancy, a synchronized method could never call another synchronized method on the same object.

Q: What is the cost?

Uncontended lock → cheap on modern JVMs (fast paths, historically biased locking)
CONTENDED lock → expensive: threads park, the OS context-switches, CPU caches invalidate

Rules:

- ✓ Hold locks for as SHORT a time as possible
- ✗ Never do I/O while holding a lock
- ✗ Never call unknown / external code while holding a lock
(that's how you get lock-order inversions and deadlocks)

```
// ✗ locks for 5 seconds to protect a 1-microsecond operation
synchronized void process() {
    callSlowApi();      // 5 seconds
    count++;
}

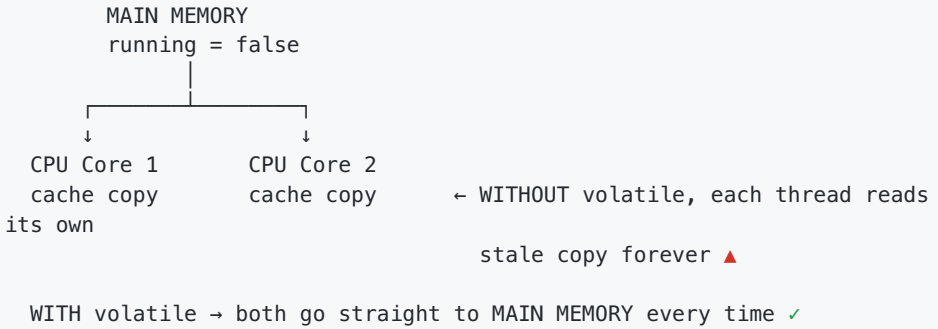
// ✓
void process() {
    callSlowApi();
    synchronized (lock) { count++; }
}
```

Part 5 — volatile and the Java Memory Model

Q: What does **volatile** guarantee?

- ✓ VISIBILITY – every read goes to main memory, every write is published immediately; no thread can see a stale cached copy
- ✓ ORDERING – the compiler/CPU may not reorder around a volatile access (it inserts a memory barrier)
- ✗ ATOMICITY – compound operations like `i++` or `check-then-act` are STILL broken

The picture:



Real-world idea: a **shared office whiteboard**.

Without volatile → everyone keeps a personal notepad copy → they go stale
With volatile → everyone must read the whiteboard every time ✓

Q: Give the canonical example.

```
class Worker implements Runnable {
    private volatile boolean running = true; //
    x without volatile this loops FOREVER

    public void run() {
        while (running) { /* work */ }
        System.out.println("stopped");
    }

    public void stop() { running = false; }
}
```

Why does it hang without **volatile** ? Because the JIT compiler is **entitled** to hoist the read out of the loop — nothing *inside* the loop modifies **running** , so it effectively compiles to:

```
if (running) { while (true) { /* work */ } } // ▲ never re-reads the flag
```

The thread never sees the write and never stops.

This is a REAL production hang. The fix is one keyword.

Q: volatile vs synchronized vs atomic — the comparison table

	volatile	synchronized	AtomicInteger
Visibility	✓	✓	✓
Ordering	✓	✓	✓
Atomicity of x++	✗	✓	✓
Blocks threads?	✗ never	✓	✗ (CAS spin)
Scope	one variable	a block/method	one variable
Cost	cheapest	highest	in between
Use for	flags, and the DCL singleton reference	compound state, invariants over several fields	counters, sequences, single-reference swaps

The decision line:

A simple FLAG (one writes, others read)

A COUNTER (x++)

SEVERAL fields that must change together

→ volatile ★

→ AtomicInteger ★

→ synchronized ★

Q: What is "happens-before"?

The easiest way to remember:

"A happens-before B" = whatever A did is GUARANTEED VISIBLE to B.

Every synchronization tool exists only to create one of these edges.

The rules worth naming:

- Program order within a single thread
- Monitor UNLOCK happens-before a later LOCK of the same monitor
- A volatile WRITE happens-before a later volatile READ of the same field
- Thread.start() happens-before everything in that thread
- Everything in a thread happens-before another thread's successful join() on it
- Constructor completion happens-before a `final` field is read
→ which is why immutable objects are safely publishable ★

◆ **Say:** "Concurrency bugs aren't really about 'two threads at once' — they're about the absence of a happens-before edge. Every synchronization tool is just a way of creating one."

Part 6 — Atomic classes & CAS

Q: How do the **Atomic*** classes work without locks?

Via **CAS — compare-and-swap**, a single CPU instruction (`lock cmpxchg` on x86):

"If this memory location STILL holds the value I expected, replace it with the new value. Otherwise, tell me I failed."

```
// conceptually, what incrementAndGet does:
int prev, next;
do {
    prev = get();
    next = prev + 1;
} while (!compareAndSet(prev, next)); // retry until nobody beat us to
it
```

Thread A: reads 5 → wants to write 6 → is it still 5? YES → write ✓
 Thread B: reads 5 → wants to write 6 → is it still 5? NO (it's 6) → RETRY
 reads 6 → wants to write 7 → is it still 6? YES → write ✓

Nothing was lost, and nobody was ever BLOCKED ▲

This is **optimistic, non-blocking, lock-free**: no thread is ever suspended; a loser simply retries.

Real-world idea: booking the **last cinema seat online**. You click, the system checks "is seat 7 still free?" — if someone beat you, it says "try again" rather than making you stand in a queue.

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();
AtomicLong id = new AtomicLong(); id.getAndIncrement();
AtomicBoolean flag = new AtomicBoolean(); flag.compareAndSet(false,
true); // run-once guard
AtomicReference<Config> cfg = new AtomicReference<>(initial);
cfg.updateAndGet(c -> c.withTimeout(30)); // lambda-based atomic
update
```

Trade-off to state:

```
LOW contention → CAS wins (no blocking) ▲
HIGH contention → CAS retries burn CPU; a lock can actually win
                  → that's what LongAdder fixes: it spreads updates across
                  per-thread cells and sums them on read
```

LongAdder is ideal for hot metrics counters — but not for a value you must read exactly on every update.

◆ **The ABA problem** (if they go deep): a value changes $A \rightarrow B \rightarrow A$ between your read and your CAS, so the CAS succeeds even though the world changed underneath. **AtomicStampedReference** adds a version stamp to detect it.

Part 7 — Locks: ReentrantLock, ReadWriteLock

Q: Why would you use **ReentrantLock** over **synchronized** ?

```
private final ReentrantLock lock = new ReentrantLock();

lock.lock();
try {
    // critical section
} finally {
    lock.unlock();    //
    ▲ MUST be in finally – synchronized does this for you
}
```

▲ That **finally** is the whole risk. Forget it and the lock is **never released** — the application freezes with no exception.

Capability	synchronized	ReentrantLock
Try without blocking	✗	✓ <code>tryLock()</code>
Timeout	✗	✓ <code>tryLock(5, SECONDS)</code> — deadlock escape hatch ★
Interruptible while waiting	✗	✓ <code>lockInterruptibly()</code>
Fairness (longest waiter first)	✗	✓ <code>new ReentrantLock(true)</code>
Multiple condition queues	✗ (one wait-set)	✓ <code>newCondition()</code>
Lock across methods	✗ (block-scoped)	✓
Auto-release	✓	✗ you must use <code>finally</code>

Real-world idea — a **restaurant table**:

```
synchronized → you stand at the door waiting for a table, forever
tryLock()      → "is a table free?" → no → you go somewhere else ✓
```

Say: " *synchronized* first — it's simpler and impossible to leak. I reach for *ReentrantLock* when I need *tryLock* with a timeout, interruptible acquisition, or separate condition queues."

Q: ReadWriteLock ?

```
ReadWriteLock rw = new ReentrantReadWriteLock();

rw.readLock().lock();    // MANY readers concurrently ✓
rw.writeLock().lock();   // exclusive – blocks all readers and writers
```

Readers: R R R R R ← all at once, they don't disturb each other
Writer: W ← alone; everyone else waits

Right for **read-heavy, write-rare** shared state — a cached config, a lookup table.

Not free, though: there is bookkeeping overhead, and readers can **starve** writers under a constant read load. Use the fair constructor, or prefer *StampedLock* 's optimistic read (Java 8+).

Real-world idea: a **notice board**. Any number of people can read it at once, but the person updating it needs everyone to step back.

Q: What are the coordination utilities?

Class	What it does	Reusable?
<i>CountDownLatch</i>	threads wait until a count reaches 0 (<i>await</i> / <i>countDown</i>)	✗ one-shot
<i>CyclicBarrier</i>	N threads wait for each other at a barrier, then all proceed	✓ resets
<i>Semaphore</i>	permits — caps concurrent access to a resource	✓
<i>Phaser</i>	flexible multi-phase barrier with dynamic parties	✓

Class	What it does	Reusable?
Exchanger	two threads swap objects at a rendezvous	✓

```

CountDownLatch latch = new CountDownLatch(3);           // wait for 3 services
to warm up
// each worker: ... latch.countDown();
latch.await();                                         // main proceeds after
all 3

Semaphore sem = new Semaphore(5);                      // max 5 concurrent
calls to a fragile API
sem.acquire();
try { callLegacyApi(); } finally { sem.release(); }

```

Real-world ideas:

```

CountDownLatch → a rocket launch countdown: 3... 2... 1... GO (one-shot)
CyclicBarrier  → a tour group: the bus leaves only when all 10 people are
back
                (and it does this at every stop – reusable)
Semaphore      → a parking lot with 5 spaces: 6th car waits for someone to
leave

```

Part 8 — wait / notify and inter-thread communication

Q: Rules for wait / notify?

Three rules. State all three.

1. You must HOLD the object's monitor
 - call inside synchronized on that same object
 - otherwise: `IllegalMonitorStateException` ▲
2. ALWAYS wait in a WHILE loop, never an IF ★ ★
 - spurious wakeups are legal
 - another thread may consume the condition between the notify and your reacquiring the lock
3. Prefer `notifyAll()` over `notify()`
 - `notify()` wakes ONE arbitrary waiter, which may be waiting on a different condition → the right thread sleeps forever (a hang)

```
class BoundedBuffer<T> {

    private final Queue<T> q = new LinkedList<>();
    private final int cap;
    BoundedBuffer(int cap) { this.cap = cap; }

    public synchronized void put(T item) throws InterruptedException {
        while (q.size() == cap) wait();           // while, NOT if ★
        q.add(item);
        notifyAll();
    }

    public synchronized T take() throws InterruptedException {
        while (q.isEmpty()) wait();
        T item = q.poll();
        notifyAll();
        return item;
    }
}
```

The flow:

PRODUCER	CONSUMER
buffer FULL wait() ♦ releases the lock ← wakes up RE-CHECKS the while condition produces ✓	takes the lock ♦ consumes an item notifyAll() releases the lock ♦

Real-world idea: a **restaurant kitchen counter**. The chef puts a plate down and rings the bell ; the waiter, who was waiting, hears it and collects. If the counter is full the chef waits; if it's empty the waiter waits.

◆ **The senior answer:** *"I know how to write producer–consumer with wait/notify, but in real code I'd use a `BlockingQueue` — `ArrayBlockingQueue` or `LinkedBlockingQueue` — because `put` / `take` already block correctly and there's no way to get the loop or the notify wrong."*

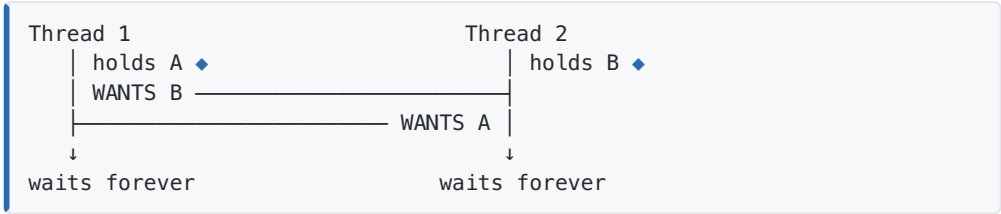
```
BlockingQueue<Order> queue = new LinkedBlockingQueue<>(100);  
// producer: queue.put(order);           blocks when full  
// consumer: Order o = queue.take();      blocks when empty
```

Part 9 — Deadlock, livelock, starvation

Q: What is a deadlock? (asked verbatim at Mphasis L1)

Two or more threads, each holding a lock the other needs, so none can ever proceed. ◆ ↔ ◆

```
// ▲ classic lock-ordering deadlock  
void transfer(Account from, Account to, BigDecimal amt) {  
    synchronized (from) {  
        synchronized (to) {  
            from.debit(amt);  
            to.credit(amt);  
        }  
    }  
}  
  
// Thread 1: transfer(A, B) → holds A, wants B  
// Thread 2: transfer(B, A) → holds B, wants A → both stuck forever
```



Real-world idea: **two people sharing one plate of food**. A grabs the spoon and waits for the fork; B grabs the fork and waits for the spoon. Neither will let go. Nobody eats.

The four Coffman conditions

All four must hold — break **any one** and deadlock becomes impossible:

- 1. MUTUAL EXCLUSION → resources aren't shareable
- 2. HOLD AND WAIT → a thread holds one lock while requesting another
- 3. NO PREEMPTION → locks can't be forcibly taken away
- 4. CIRCULAR WAIT → a cycle in the "waiting for" graph ← easiest to break ★

Q: How do you prevent it?

Strategy	How
Global lock ordering ★	always acquire locks in the same order — e.g. by account id. Breaks <i>circular wait</i> . Give this answer first.
Timeout	<code>tryLock(2, SECONDS)</code> — back off, release everything, retry. Breaks <i>hold and wait</i> .
Single coarse lock	fewer locks, no cycles. Costs concurrency.
Avoid nested locks	never call unknown / foreign code while holding a lock.
Immutability / no shared state	no lock, no deadlock.

```
// ✓ ordered acquisition
void transfer(Account a, Account b, BigDecimal amt) {

    Account first = a.getId() < b.getId() ? a : b;
    Account second = a.getId() < b.getId() ? b : a;

    synchronized (first) {
        synchronized (second) {
            a.debit(amt);
            b.credit(amt);
        }
    }
}
```

Both threads now take the LOWER id first, always.
A cycle can never form ✓

Back to the dinner table: agree that **everyone picks up the spoon before the fork**. Now the second person just waits for the spoon, and nobody is stuck holding half a set.

Q: How would you detect one in production?

*"Take a thread dump — `jstack <pid>`, or `jcmd / VisualVM / JMC`. The JVM prints a **Found one Java-level deadlock:** section naming both threads and the monitors. That's also how I'd diagnose a hung app generally: dump twice, a minute apart, and compare — threads stuck on the same stack are the problem.*

`ThreadMXBean.findDeadlockedThreads()` can do it programmatically for a health check."

```
jps                                # find the process id
jstack <pid>                       # → "Found one Java-level deadlock:"
```

Q: Deadlock vs livelock vs starvation?

DEADLOCK → everyone blocked, nothing moves, CPU IDLE
LIVELock → threads keep reacting to each other and changing state, but make no progress. CPU BUSY, work ZERO
STARVATION → a thread is runnable but never scheduled

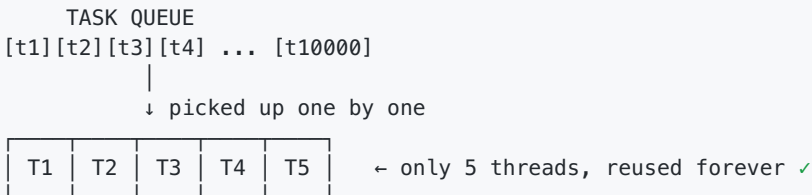
Real-world ideas:

Deadlock → two cars nose-to-nose in a one-lane road, both refusing to reverse
Livelock → two people in a corridor stepping aside at the same time, again and again, never passing
Starvation → a low-priority thread against a greedy one, or a writer stuck behind an endless stream of readers → use a FAIR lock

Part 10 — ExecutorService & thread pools

Q: Why use a pool instead of `new Thread()` ?

1. Thread creation is EXPENSIVE – ~1MB stack each, plus an OS call
2. BOUNDED concurrency – `new Thread()` per request means a traffic spike creates 10,000 threads and the JVM dies with "OutOfMemoryError: unable to create new native thread" ▲
3. A queue + rejection policy gives BACKPRESSURE instead of collapse
4. Lifecycle, naming, monitoring and Future results – all built in



Real-world idea: a **restaurant**.

Without a pool → hire a NEW waiter for every customer, then fire him ✗
With a pool → keep 5 waiters; each serves customer after customer ✓

```

ExecutorService pool = Executors.newFixedThreadPool(10);
pool.submit(() -> doWork());

pool.shutdown(); // no new tasks;
finish queued work
if (!pool.awaitTermination(30, TimeUnit.SECONDS))
    pool.shutdownNow(); // interrupt running
tasks

```

▲ If you never call `shutdown()`, the JVM **never exits** — the pool's non-daemon threads keep it alive.

The factory methods, and what each one hides

Factory	Behaviour	Watch out
<code>newFixedThreadPool(n)</code>	n threads, unbounded queue	the queue grows without limit → OOM
<code>newCachedThreadPool()</code>	unbounded threads, 60s idle reap	a spike creates thousands of threads
<code>newSingleThreadExecutor()</code>	one thread, sequential	fine for ordered work
<code>newScheduledThreadPool(n)</code>	delayed / periodic	replaces <code>Timer</code> (which dies on an uncaught exception)
<code>newWorkStealingPool()</code>	ForkJoinPool, per-thread dequeues	good for many small CPU tasks
<code>newVirtualThreadPerTaskExecutor()</code>	Java 21, a virtual thread per task	ideal for blocking I/O

▲ **The senior answer:** "In production I configure a `ThreadPoolExecutor` directly rather than using `Executors`, because the factory methods hide either an unbounded queue or unbounded thread creation — both of which fail as OOM under load."

```

new ThreadPoolExecutor(
    10, 20, // core, max
    60L, TimeUnit.SECONDS, // idle keep-alive for non-
core threads
    new ArrayBlockingQueue<>(500), // BOUNDED queue =
backpressure ★
    new ThreadFactoryBuilder().setNameFormat("order-%d").build(), //
named → readable dumps
    new ThreadPoolExecutor.CallerRunsPolicy() // rejection: the submitter
runs it → throttles input
);

```

How the pool actually decides — the order surprises people ★

```

Task arrives
├── fewer than CORE threads? → create a new thread ✓
├── else, is the QUEUE not full? → QUEUE it ★ (note: queue BEFORE
max threads)
├── else, fewer than MAX threads? → create a new thread
└── else → REJECTION POLICY

```

That middle step catches people out: the pool prefers **queuing** over growing to **max**. So with an unbounded queue, **maxPoolSize** is **never reached** — which is exactly why **newFixedThreadPool** can OOM on the queue rather than on threads.

The rejection policies:

```

AbortPolicy (default) → throws RejectedExecutionException
CallerRunsPolicy → the submitting thread runs it → natural
throttling ★
DiscardPolicy → silently drops the task
DiscardOldestPolicy → drops the oldest queued task

```

Sizing:

CPU-bound $\approx \text{cores} + 1$
I/O-bound $\approx \text{cores} \times (1 + \text{waitTime} / \text{computeTime}) \rightarrow \text{much larger,}$
because the threads are mostly blocked waiting

Part 11 — Callable, Future, CompletableFuture

Q: Runnable vs Callable?

Runnable $\rightarrow$ returns NOTHING, cannot throw a checked exception
Callable $\rightarrow$ RETURNS a value, CAN throw a checked exception ★

	Runnable	Callable<V>
Method	<code>void run()</code>	<code>V call()</code> throws Exception
Returns a value	✗	✓
Can throw checked exceptions	✗	✓
Since	1.0	1.5

Real-world idea: a **dry cleaner**.

Runnable $\rightarrow$ you drop off clothes and walk away, expecting nothing back
Callable $\rightarrow$ you drop off clothes and get a TOKEN (Future)
 $\rightarrow$ later you show the token and collect (`future.get()`)

```
Future<Integer> f = pool.submit(() -> 42); // returns IMMEDIATELY (the token)
// ... do other work ...
Integer result = f.get();                  // NOW it blocks until ready
```

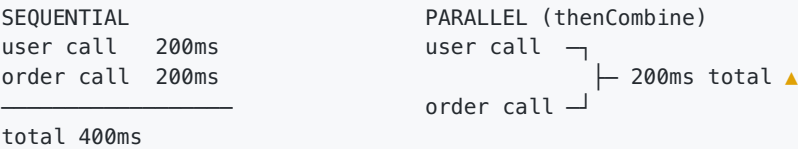
Q: What's wrong with **Future** ?

future.get() BLOCKS.
There's no callback.
You can't chain or combine futures without blocking.

CompletableFuture (Java 8) fixes all of that.

```
CompletableFuture<User> u = CompletableFuture.supplyAsync(() ->
userClient.get(id), pool);
CompletableFuture<Order> o = CompletableFuture.supplyAsync(() ->
orderClient.get(id), pool);

u.thenCombine(o, Dashboard::new) // two PARALLEL
calls, combined ★
  .thenApply(Dashboard::summarize) // transform
  .exceptionally(ex -> Dashboard.empty()) // recover
  .thenAccept(this::render); // terminal side
effect
```



Method	Meaning
supplyAsync / runAsync	start async (with a value / without)
thenApply vs thenCompose	map vs flatMap — use thenCompose when the function itself returns a CompletableFuture, or you get CF<CF<T>>
thenCombine	wait for two and merge
allOf / anyOf	wait for all / for the first
exceptionally / handle / whenComplete	recover / handle both outcomes / peek
orTimeout(...) (Java 9+)	fail after a deadline

▲ Two traps:

1. **Always pass your own executor.** The default `ForkJoinPool.commonPool()` is shared JVM-wide and sized to `cores - 1`, so one blocking task can starve everything else — including every parallel stream in the application.
2. `join()` throws unchecked `CompletionException`; `get()` throws checked `ExecutionException`.

◆ **Your project line:** "On the RoboGebra explanation flow, fetching the student profile and the question metadata were two independent service calls done sequentially. Moving them to `supplyAsync` + `thenCombine` on a dedicated pool cut the endpoint latency to roughly the slower of the two calls instead of their sum."

Part 12 — Concurrent collections

Need	Use	Not
Map	<code>ConcurrentHashMap</code>	<code>Hashtable</code> , <code>synchronizedMap</code>
List, read-heavy	<code>CopyOnWriteArrayList</code>	<code>Vector</code>
Queue (producer–consumer)	<code>LinkedBlockingQueue</code> , <code>ArrayBlockingQueue</code>	hand-rolled wait/notify
Direct hand-off, no capacity	<code>SynchronousQueue</code>	
Priority + blocking	<code>PriorityBlockingQueue</code>	
Delayed tasks	<code>DelayQueue</code>	
Non-blocking queue/deque	<code>ConcurrentLinkedQueue</code> / <code>Deque</code>	
Sorted concurrent map	<code>ConcurrentSkipListMap</code>	<code>TreeMap</code>

`ConcurrentHashMap` in one breath:

"Same structure as `HashMap` — bucket array, chains that treeify at 8 — but it CASes into empty bins and `synchronized` es on the bin's head node for the rest, so lock granularity is one bucket. No nulls. Iterators are weakly consistent, so no `ConcurrentModificationException`. Compound operations still need `putIfAbsent` / `computeIfAbsent` / `merge` to be atomic."

(Full detail in [31 — HashMap internals](#).)

<code>Hashtable</code>	◆ ONE lock for the whole map	→ everyone queues
<code>ConcurrentHashMap</code>	◆ one lock per BUCKET	→ parallel writes ▲

`CopyOnWriteArrayList` :

Every WRITE copies the entire backing array. $O(n)$ per add
Every READ is lock-free and never throws CME ▲

- ✓ Perfect for: listener lists, config caches — many reads, near-zero writes
- ✗ Terrible for: anything write-heavy or large

Q: `Collections.synchronizedList` — what's the catch?

Each individual method is synchronized, but **iteration is not atomic**:

```
List<String> list = Collections.synchronizedList(new ArrayList<>());

synchronized (list) {
    for (String s : list) { ... }
}
```

// required manually, or CME

And even then you hold **one global lock** for the entire loop — no better than `Hashtable`. Prefer a real concurrent collection.

Q: ThreadLocal ?

ThreadLocal = a PRIVATE COPY of a variable for each thread.
No sharing → no synchronization needed.

```
private static final ThreadLocal<SimpleDateFormat> FMT =  
    ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));
```

Thread 1 → its own SimpleDateFormat
Thread 2 → its own SimpleDateFormat ← never shared, never corrupted
✓

Real-world idea: instead of everyone sharing one office pen (and fighting over it),
each person gets their own pen.

Used by Spring for the transaction / `EntityManager` binding, by SLF4J's MDC for correlation ids, and historically for `SimpleDateFormat` (which is not thread-safe).

▲ Memory leak warning — say this unprompted:

"In a thread pool, threads are reused forever, so a `ThreadLocal` value is never garbage collected unless you `remove()` it in a `finally`. In a web container that leaks per-request objects and can eventually OOM — and it also leaks data across requests, which is a security issue."

```
try {  
    CONTEXT.set(userContext);  
    doWork();  
} finally {  
    CONTEXT.remove();           // ★ mandatory in a pooled environment  
}
```

Modern code prefers `java.time` (`DateTimeFormatter` — immutable and thread-safe) over `SimpleDateFormat` entirely.

Part 13 — Thread-safe singleton (the live-coding one)

Asked at Mphasis L1 — be able to type all five and rank them.

```

// 1 EAGER – simple and thread-safe (class loading is guaranteed thread-
safe by the JVM)
public class Eager {
    private static final Eager INSTANCE = new Eager();
    private Eager() {}
    public static Eager getInstance() { return INSTANCE; }
}

// 2 SYNCHRONIZED accessor – correct, but EVERY call pays for a lock
public class Sync {
    private static Sync instance;
    private Sync() {}
    public static synchronized Sync getInstance() {
        if (instance == null) instance = new Sync();
        return instance;
    }
}

// 3 DOUBLE-CHECKED LOCKING – the one they want, and volatile is the
point ★
public class Dcl {
    private static volatile Dcl instance;           // ▲ volatile is NOT
optional
    private Dcl() {}
    public static Dcl getInstance() {
        if (instance == null) {                     // 1st check – no lock,
fast path ▲
            synchronized (Dcl.class) {
                if (instance == null) {             // 2nd check – someone may
have won the race
                    instance = new Dcl();
                }
            }
        }
        return instance;
    }
}

// 4 BILL PUGH HOLDER idiom – lazy, no locking, no volatile. Cleanest. ★
public class Holder {
    private Holder() {}
    private static class H { static final Holder INSTANCE = new
Holder(); }
    public static Holder getInstance() { return H.INSTANCE; }    // inner
class loaded on first use
}

```

```
// 5 ENUM – the Effective Java answer; serialization- and reflection-
proof ★
public enum EnumSingleton {
    INSTANCE;
    public void doWork() { }
}
```

How to rank them out loud:

```
Eager    → fine if the object is cheap and always needed
Sync     → correct but slow (locks on EVERY call, forever)
DCL      → the classic answer; volatile is mandatory
Holder   → lazy + fast + no volatile → the cleanest lazy version ★
Enum     → the safest of all (immune to reflection and serialization)
```

Q: Why is **volatile** mandatory in double-checked locking?

The killer follow-up.

```
instance = new Dcl();
```

That single line is actually **three** steps:

1. allocate memory
2. run the constructor
3. assign the reference to `instance`

The JVM is **allowed to reorder steps 2 and 3**:

- | | |
|-------------------------|--|
| 1. allocate memory | |
| 3. assign the reference | ← `instance` is now NON-NULL... |
| 2. run the constructor | ← ...but the object isn't built yet! ▲ |

Now another thread doing the **first, unlocked** null check sees a non-null reference to a **half-constructed object** and uses it — reading default-valued fields, or crashing.

```
Thread B: if (instance == null) → false (it was just assigned)
Thread B: instance.getConfig() → returns null ▲ the constructor hadn't
run
```

`volatile` inserts the memory barrier that **forbids that reordering** (and guarantees visibility).

Before Java 5 the memory model was too weak for DCL to work at all — which is exactly why the Bill Pugh holder idiom became popular.

What breaks a singleton anyway?

1. REFLECTION → `setAccessible(true)` on the private constructor → a second instance
2. SERIALIZATION → each `deserialize` creates a new one, unless you add `readResolve()`
3. CLONING → override `clone()` to throw

Only `ENUM` is immune to all three ★

◆ **Bonus that always lands:** "A Spring `@Service` is a singleton too, but per `ApplicationContext`, not per JVM — it's a container guarantee, not the GoF pattern. And it's only thread-safe if it's stateless: mutable instance fields on a singleton bean are shared across every concurrent request. That's the single most common thread-safety bug I look for in Spring code."

Part 14 — Classic live-coding programs

Type these from memory before the interview. They are the three that actually get asked.

1. Print odd and even alternately with two threads

```
class OddEven {  
  
    private int n = 1;  
    private final int max = 10;  
  
    public synchronized void printOdd() throws InterruptedException {  
        while (n <= max) {  
            while (n % 2 == 0) wait();           // not my turn → release  
the lock  
            System.out.println("Odd : " + n++);  
            notifyAll();                         // wake the other thread  
        }  
    }  
  
    public synchronized void printEven() throws InterruptedException {  
        while (n <= max) {  
            while (n % 2 == 1) wait();  
            System.out.println("Even: " + n++);  
            notifyAll();  
        }  
    }  
}
```

n=1 → odd thread prints, n becomes 2, notifyAll
n=2 → odd thread sees $n\%2==0$ → wait ♦
even thread wakes, prints, n becomes 3, notifyAll
n=3 → even waits, odd prints... ← perfect alternation ✓

The key insight to explain: **the while is what makes it alternate**. Each thread parks itself when it isn't its turn.

2. Producer-consumer with **BlockingQueue** (the version to write FIRST)

```
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(5);

Thread producer = new Thread(() -> {
    try {
        for (int i = 1; i <= 20; i++) {
            q.put(i);                                // BLOCKS when
full ✓                                             full
            System.out.println("produced " + i);
        }
    } catch (InterruptedException e)
    { Thread.currentThread().interrupt(); }
});

Thread consumer = new Thread(() -> {
    try {
        while (true) System.out.println("consumed " + q.take()); //
BLOCKS when empty ✓
    } catch (InterruptedException e)
    { Thread.currentThread().interrupt(); }
});

producer.start();
consumer.start();
```

Write this first, then say: *"and if you'd like the manual version, here it is with wait/notify"* — showing both is the strongest possible answer.

3. Run N tasks in parallel and collect results

```
ExecutorService pool = Executors.newFixedThreadPool(4);

List<Callable<Integer>> tasks = List.of(() -> 1, () -> 2, () -> 3);
List<Future<Integer>> futures = pool.invokeAll(tasks);    // blocks
until ALL are done

int sum = 0;
for (Future<Integer> f : futures) sum += f.get();

pool.shutdown();
```

4. Thread-safe counter — three ways

See Part 3: `synchronized`, `AtomicInteger`, `LongAdder`.

5. Wait for N services to start

```
CountDownLatch latch = new CountDownLatch(3);

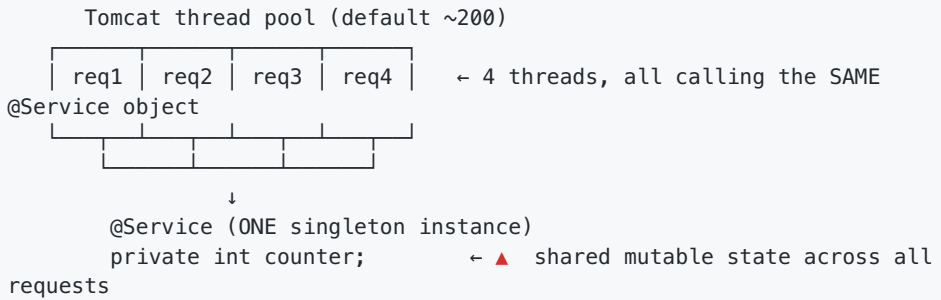
for (Service s : services)
    pool.submit(() -> { s.start(); latch.countDown(); });

latch.await(30, TimeUnit.SECONDS);
System.out.println("all services up");
```

Part 15 — Multithreading in Spring Boot (your real answer)

They will ask "where have you used threads?" — do not say "I haven't". You have, through the framework.

"Most of my concurrency is framework-level rather than raw threads. Every HTTP request in Spring Boot runs on its own Tomcat worker thread, so any singleton bean with mutable state is a shared-mutable-state bug — I keep `@Service` beans stateless and pass state as method parameters. Where I did explicit async work: `@Async` with a configured `ThreadPoolTaskExecutor` for fire-and-forget notification sending, and `CompletableFuture` to parallelise two independent downstream calls that had been sequential. And on the data side, optimistic locking with `@Version` for concurrent edits of the same record."



```

@Configuration
@EnableAsync
class AsyncConfig {

    @Bean("notificationExecutor")
    public Executor executor() {
        ThreadPoolTaskExecutor e = new ThreadPoolTaskExecutor();
        e.setCorePoolSize(5);
        e.setMaxPoolSize(10);
        e.setQueueCapacity(100); // bounded ★
        e.setThreadNamePrefix("notify-");
        e.setRejectedExecutionHandler(new
ThreadPoolExecutor.CallerRunsPolicy());
        return e;
    }
}

@Service
class NotificationService {
    @Async("notificationExecutor")
    public void send(Notification n) { /* runs OFF the request thread */ }
}
  
```

● In your RoboGebra code — this is the answer, verbatim

File: `robogeбра-portal/src/main/java/com/robogeбра/cms/config/AsyncConfig.java`

Your own codebase already contains a textbook thread-pool configuration. Use it instead of a generic answer:

```

@Bean(NOTIFICATION_TASK_EXECUTOR)
public ThreadPoolTaskExecutor notificationTaskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(4);
    executor.setMaxPoolSize(8);
    executor.setQueueCapacity(500); // BOUNDED ★
    executor.setThreadNamePrefix("push-"); // readable thread
    dumps ★
    executor.setRejectedExecutionHandler(new
    ThreadPoolExecutor.CallerRunsPolicy()); // backpressure ★
    executor.setWaitForTasksToCompleteOnShutdown(true); // graceful
    shutdown ★
    executor.setAwaitTerminationSeconds(30);
    return executor;
}

```

■ "We had one shared `@Async` pool and push-notification fan-out was starving it — one announcement touches every teacher, and each device is a blocking FCM round trip, so analytics events queued behind it. I gave push its own executor with a bounded 500-item queue and `CallerRunsPolicy`, so a backlog slows the submitter instead of growing an unbounded queue until the heap gives out — a late push beats an OOM. I also had to declare Boot's own `applicationTaskExecutor` by hand, because `TaskExecutorConfiguration` is `@ConditionalOnMissingBean(Executor.class)` — declaring any executor backs the auto-configured one out, and then every unqualified `@Async` silently resolves to the notification pool."

★ That last sentence is a genuinely senior observation and doubles as your Spring auto-configuration answer.

Full detail: [39 — RoboGebra Code Examples](#).

The four `@Async` traps ★ (excellent follow-up material)

1. SELF-INVOCATION doesn't work
 `@Async` is proxy-based, so calling `this.send(...)` from inside the same bean runs SYNCHRONOUSLY. Same rule as `@Transactional`.
2. Must be PUBLIC, and the return type must be void, Future or `CompletableFuture`
3. EXCEPTIONS VANISH on void methods
 unless you register an `AsyncUncaughtExceptionHandler`
4. CONTEXT DOESN'T PROPAGATE
 `SecurityContextHolder`, the transaction, and MDC are ThreadLocal-based,
 so the async thread sees NONE of it unless you configure propagation.
 `@Transactional` on an `@Async` method starts a DIFFERENT transaction.

That last one is the source of many real production bugs — worth saying unprompted.

Full Java 21 file: virtual threads in the context of everything else 21 added (pattern matching for `switch`, record patterns, sequenced collections, structured concurrency) → [40 — Java 21 Features](#) ★

Part 16 — Virtual threads (Java 21)

Worth two sentences — it signals you keep current. Don't oversell it if the project is on Java 17.

PLATFORM thread → maps 1:1 to an OS thread → ~1MB stack → thousands max
VIRTUAL thread → JVM-scheduled, a few hundred bytes → MILLIONS possible
and when it blocks on I/O it UNMOUNTS from its carrier thread instead of holding it ★

Platform threads, 10,000 requests
→ 10,000 OS threads → ~10 GB → dead ▲

Virtual threads, 10,000 requests
→ 10,000 virtual threads on ~8 carrier threads → fine ✓

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {  
    for (var task : tasks) executor.submit(task);  
} // close() waits for all – try-with-resources
```

Say: "They make the simple blocking style scale like reactive code for I/O-bound work — a thread per request becomes viable again. They do not speed up CPU-bound work, and **synchronized** blocks used to pin a virtual thread to its carrier, so **ReentrantLock** was preferred there. RoboGebra is on Java 17, so this is reading rather than production experience for me."

That last sentence is the honest framing that scores better than pretending.

Part 17 — Predict the output

Q1

```
Thread t = new Thread(() -> System.out.println("A"));  
t.run();  
System.out.println("B");
```

Answer

`A` then `B`, ****both on the main thread****. `run()` is a plain method call — no thread is started at all.

Q2

```
Thread t = new Thread(() -> System.out.println("A"));  
t.start();  
t.start();
```

Answer

Prints `A`, then throws `IllegalThreadStateException` on the second `start()`. A `Thread` object is single-use.

Q3

```
class C { int n = 0; void inc() { n++; } }  
// 1000 threads each call inc() once
```

Answer

Anything ≤ 1000 , typically slightly under. Lost updates, because `n++` is read-modify-write. It is non-deterministic — it may print exactly 1000 on some runs, which is precisely what makes these bugs so hard to catch in testing.

Q4

```
synchronized (this) {  
    Thread.sleep(5000);  
}
```

Answer

It compiles (once `InterruptedException` is handled) and holds the lock for the full 5 seconds — every other thread needing that monitor is blocked. `sleep` does not release locks. This is a design smell, not an error.

Q5

```
public class T {  
    public static void main(String[] a) {  
        Thread t = new Thread(() -> {  
            try { Thread.sleep(2000); } catch (InterruptedException e) {}  
            System.out.println("worker done");  
        });  
        t.setDaemon(true);  
        t.start();  
        System.out.println("main done");  
    }  
}
```

Answer

Prints only `main done`. The daemon thread is killed when main — the last user thread — exits, so `worker done` never prints. Remove `setDaemon(true)` and both print.

Q6

```
private static int count = 0;
public static synchronized void a() { count++; }
public void b() { synchronized (this) { count++; } }
```

Answer

****Not thread-safe together.**** `a()` locks `T.class`; `b()` locks `this`. Two different monitors guarding the ****same static field**** → they don't exclude each other. Both must lock the same object.

Q7

```
List<Integer> list = new ArrayList<>();
// 10 threads each add 100 elements
System.out.println(list.size());
```

Answer

Usually ****< 1000****, and it can also throw `ArrayIndexOutOfBoundsException` or `NullPointerException` — a concurrent `add` can interleave with the internal array grow and corrupt it. Use `Collections.synchronizedList`, `CopyOnWriteArrayList`, or per-thread lists merged at the end.

Q8

```
CompletableFuture.supplyAsync(() -> { throw new
RuntimeException("boom"); })
    .thenApply(x -> "never")
    .exceptionally(ex -> "recovered: " + ex.getMessage());
```

Answer

Result is `recovered: java.util.concurrent.CompletionException: java.lang.RuntimeException: boom`. Two details people get wrong: 1. `thenApply` is ****skipped**** entirely on the exceptional path. 2. The original exception arrives ****wrapped**** in a `CompletionException`. Also: nothing prints at all unless you `join()` or add a terminal stage.

Part 18 — Rapid-fire table

Question	Answer
<code>start()</code> vs <code>run()</code>	<code>start()</code> creates a thread; <code>run()</code> is a normal call on the current thread
Thread states	NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED
<code>sleep</code> vs <code>wait</code>	<code>sleep</code> keeps locks (Thread, static); <code>wait</code> releases the monitor (Object, needs synchronized)
<code>wait</code> in <code>if</code> or <code>while</code> ?	<code>while</code> — spurious wakeups and stolen conditions
<code>notify</code> vs <code>notifyAll</code>	<code>notifyAll</code> is safe; <code>notify</code> can wake the wrong waiter and hang
The three thread-safety problems	atomicity, visibility, ordering
What <code>volatile</code> gives	visibility + ordering. Not atomicity
Does <code>volatile</code> fix <code>i++</code> ?	x — use <code>AtomicInteger</code> or a lock
What <code>synchronized</code> gives	mutual exclusion + visibility + ordering; reentrant; releases on exception
Instance vs static synchronized	different locks (<code>this</code> vs <code>Class</code>) — they don't exclude each other
CAS	compare-and-swap; a lock-free optimistic retry loop behind <code>Atomic*</code>
<code>ReentrantLock</code> over <code>synchronized</code>	<code>tryLock</code> , timeout, interruptible, fairness, multiple conditions
Deadlock's 4 conditions	mutual exclusion, hold-and-wait, no preemption, circular wait
Best deadlock prevention	global lock ordering ★
Detect a deadlock	<code>jstack</code> thread dump → "Found one Java-level deadlock"

Question	Answer
Livelock	threads keep changing state but make no progress
Why pools	thread creation cost + bounded concurrency + backpressure
<code>Executors</code> factory risk	unbounded queue (fixed) or unbounded threads (cached) → OOM
Pool sizing	CPU-bound $\approx$ cores+1; I/O-bound much larger
Pool decision order	core threads → queue → max threads → rejection policy
<code>Runnable</code> vs <code>Callable</code>	<code>Callable</code> returns a value and can throw checked exceptions
<code>Future</code> limitation	<code>get()</code> blocks, no chaining → use <code>CompletableFuture</code>
<code>thenApply</code> vs <code>thenCompose</code>	map vs flatMap
CHM lock granularity	CAS on an empty bin; <code>synchronized</code> on the bin's head node
<code>ThreadLocal</code> risk	leaks in pooled threads unless <code>remove()</code> in a <code>finally</code>
<code>@Async</code> traps	self-invocation, must be public, swallowed exceptions, no context propagation
Virtual threads	cheap JVM-scheduled threads; unmount on blocking I/O; Java 21
Stop a thread properly	interrupt, or a <code>volatile</code> flag — never <code>stop()</code>
Safest singleton	<code>enum</code> (immune to reflection + serialization)
Why <code>volatile</code> in DCL	forbids reordering → no half-constructed object leaks

Part 19 — Drills

Cover the answers and say them out loud.

1. Why is `count++` unsafe but `count = 5` (an int assignment) atomic?

Answer

`count++` is read-modify-write — three steps that can interleave. An `int` assignment is a single atomic store per the JMM. Note the exception: `long` and `double` writes are **not** guaranteed atomic on 32-bit JVMs unless declared `volatile`.

2. Someone writes `volatile int count; count++;` and calls it thread-safe. What do you say?

Answer

`volatile` fixes **visibility**, not **atomicity**. Two threads can still both read 5 and both write 6. Use `AtomicInteger.incrementAndGet()` or `synchronize`.

3. Your service singleton has `private SimpleDateFormat fmt = new SimpleDateFormat(...)`. What's the bug?

Answer

`SimpleDateFormat` is mutable and not thread-safe, and a Spring singleton is shared across all request threads → garbled or wrong dates, sometimes exceptions. Fix: use `DateTimeFormatter` (immutable, thread-safe), or a local instance, or a `ThreadLocal`.

4. Two threads deadlock in production. Walk me through your first five minutes.

Answer

1. Confirm the symptoms — requests hanging, CPU low. 2. `jstack` twice, a minute apart. 3. Look for the "Found one Java-level deadlock" block, and for threads stuck on the same monitor across **both** dumps. 4. Identify the two lock-acquisition paths. 5. Fix by enforcing a consistent lock order, or introduce `tryLock` with a timeout.

5. When is `CopyOnWriteArrayList` the right choice, and when is it a disaster?

Answer

****Right**** for read-mostly, small, rarely-written lists — listener and observer registries. ****A disaster**** for write-heavy or large lists: every single ``add`` copies the whole array ($O(n)$), so a loop of 10,000 adds means 10,000 array copies.

6. You have 100 independent REST calls to make, each ~200ms. Sequential = 20s. How do you fix it, and what's the risk?

Answer

Fan out with ``CompletableFuture.supplyAsync`` on a ****dedicated bounded pool**** (or a virtual-thread executor on Java 21), and combine with ``allOf``. Risks: overwhelming the downstream service (rate-limit, or gate with a ``Semaphore``), exhausting the connection pool, and having no per-call timeout — add ``orTimeout`` and a fallback. Never use the default common pool for blocking I/O.

7. Why prefer `BlockingQueue` over `wait / notify` for producer-consumer?

Answer

It is already correct: blocking ``put`/`take``, a bounded capacity giving backpressure, and no chance of a missed signal or an ``if`` where a ``while`` was needed. Less code, and no subtle bug surface.

Quick Revision Sheet

THE FRAME

Thread safety = ATOMICITY + VISIBILITY + ORDERING

synchronized → all 3 | volatile → visibility+ordering only | Atomic → atomicity on 1 var

CREATING

implements Runnable > extends Thread (inheritance slot + pools + lambda)

start() = NEW thread ✓ | run() = same thread ✗

start() twice → IllegalThreadStateException

daemon → killed when the last user thread ends (no cleanup)

LIFECYCLE

NEW → RUNNABLE → (BLOCKED / WAITING / TIMED_WAITING) → TERMINATED

BLOCKED = waiting for a LOCK (forced) | WAITING = called wait()/join() (chosen)

sleep() KEEPS the lock ♦ | wait() RELEASES it ♦

interrupt() = a polite request; ALWAYS restore the flag in the catch

RACE CONDITION

count++ = read + add + write → lost updates

Fix: synchronized | AtomicInteger ★ | LongAdder (high contention)

volatile does NOT fix it ▲

LOCKS

synchronized: instance→this, static→Class, block→your own private final object ★

reentrant ✓ | releases on exception ✓

ReentrantLock adds: tryLock(timeout) ★, interruptible, fairness, conditions

ReadWriteLock: many readers, one writer

wait/notify – 3 RULES

1. inside synchronized 2. use WHILE not IF 3. prefer notifyAll()

Real code: use BlockingQueue instead ★

DEADLOCK

4 conditions; break CIRCULAR WAIT with consistent LOCK ORDERING ★ or tryLock(timeout). Detect with jstack.

livelock = busy but no progress | starvation = never scheduled

POOLS

new Thread() per task → OOM. Pool = reuse + bounded + backpressure

Decision order: core → QUEUE → max → rejection policy ★

Production: ThreadPoolExecutor + BOUNDED queue + CallerRunsPolicy

ALWAYS shutdown()

ASYNC

Runnable (void) vs Callable (value + checked exception → Future)

CompletableFuture: supplyAsync / thenApply / thenCompose / thenCombine /
allOf / exceptionally – ALWAYS pass your own

executor ▲

SINGLETON

Eager | synchronized | DCL (volatile mandatory!) | Holder ★ | Enum
(safest) ★

volatile in DCL prevents seeing a HALF-CONSTRUCTED object

SPRING

Every request = its own thread → @Service must be STATELESS ★

@Async traps: self-invocation, must be public, swallowed exceptions,
no SecurityContext/transaction/MDC propagation

Related files: [05 — Core Java](#) · [30 — Mphasis L2 §C concurrency](#) · [31 — HashMap internals](#) · [23 — Predict the Output](#)

● Interface vs Abstract Class — Easy Version

Asked at **Virtusa Round 2 (Aug 2026)** and in almost every Java OOP round.

It looks like a definition question, but it isn't. The interviewer wants to know two things:

1. Can you **choose** between them for a real design?
2. Do you know what **Java 8 and 9 changed**?

The mistake that costs the question: answering with the old textbook line *"interfaces can't have method bodies."* That has been wrong since 2014. Interfaces have had **default** and **static** methods since Java 8 and **private** methods since Java 9. Say the old line and the interviewer quietly downgrades everything you say afterwards.

● About the "In your RoboGebra project" blocks

Throughout this file, after an interview answer you'll see a ● **In your RoboGebra project** block containing **real code from your own repositories** (`/Users/safi/workspace/robogebra-workspace`).

Use them like this ★ 1. Give the **DEFINITION** (the interview answer above) 2. Then **"in our codebase..."** ← the ● block ★ 3. Then **ONE trade-off or trap**

Three sentences. That is the difference between someone who read the docs and someone who has shipped it. Full index: **39 — RoboGebra Code Examples**.

Table of contents

- [Part 0 — The 60-second answer](#)
- [Part 1 — The one-picture difference](#)

- Part 2 — The comparison table
- Part 3 — Interfaces after Java 8 and 9
- Part 4 — Abstract classes in depth
- Part 5 — When to choose which (the real question)
- Part 6 — Diamond problem & multiple inheritance
- Part 7 — Functional interfaces
- Part 8 — Real examples from the JDK, Spring and your projects
- Part 9 — Related concepts they drift into
- Part 10 — Predict the output
- Part 11 — Rapid-fire table
- Part 12 — Drills
- Part 13 — The C#/.NET version

Part 0 — The 60-second answer

The easiest way to remember the whole topic:

Abstract class	→ "IS-A"	→ share CODE and STATE	→ extends	(only ONE)
Interface	→ "CAN-DO"	→ share a CONTRACT	→ implements	(MANY)

What to actually say

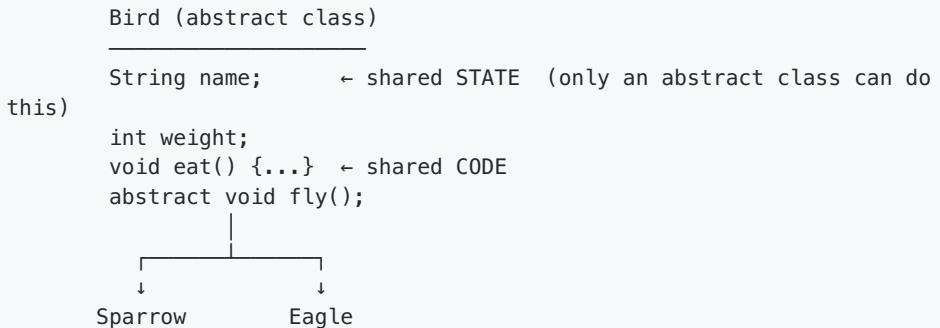
"Both are abstraction tools, but they answer different questions. An abstract class models an 'is-a' relationship and exists to share state and common implementation across closely related subclasses — it can have constructors, instance fields and any access modifier, and a class can extend only one.

*An interface models a capability — 'can-do' — and a class can implement many. Since Java 8 an interface can carry **default** and **static** method bodies, and since Java 9 **private** helper methods, so 'no implementation' is no longer the difference. What an interface still cannot have is instance state — its fields are implicitly **public static final** constants — and it has no constructor.*

The practical rule I use: interface for the contract, abstract class for shared code. If I catch myself wanting a field or a constructor, that's an abstract class. If I need a type that several unrelated classes can adopt, that's an interface. I default to interfaces, because Java gives you only one superclass and spending it is a big commitment."

Part 1 — The one-picture difference

Use **birds** for the whole topic. It makes both answers obvious.



Every bird **IS-A** bird, they all eat the same way, and they all have a name and a weight.

Shared state + shared code + a real family = **abstract class**.

Now, swimming:



A fish is not a bird, and a human is not a fish. Swimming is an **ability**, not a family. A capability adopted by unrelated types = **interface**.

And **Duck** needs both:

```
class Duck extends Bird implements Swimmable, Quackable {
//      └ ONE parent      └ MANY capabilities
}
```

extends → only 1 allowed (your one and only inheritance slot)
 implements → as many as you like

Real-world idea:

Abstract class → your FAMILY: you get the surname and the family property.
 You can only be born into one family.

Interface → a CERTIFICATE: swimming, driving, first aid.
 You can hold as many as you want, and so can strangers.

Part 2 — The comparison table

	Interface	Abstract class
Keyword	<code>interface</code> / <code>implements</code>	<code>abstract class</code> / <code>extends</code>
Multiple inheritance	✓ implement many	✗ extend exactly one
Instance fields (state)	✗ — fields are implicitly <code>public static final</code>	✓ any fields, any modifier
Constructor	✗ never	✓ (runs via <code>super()</code> from the subclass)
Method bodies	✓ <code>default</code> , <code>static</code> (8), <code>private</code> (9)	✓ concrete + abstract methods
Access modifiers	methods implicitly <code>public</code>	<code>public</code> , <code>protected</code> , <code>private</code> , default
<code>final</code> methods	✗	✓ (this is how a template method locks the algorithm)
Can be a lambda target	✓ if it has exactly 1 abstract method	✗ never

	Interface	Abstract class
Relationship it models	CAN-DO (capability)	IS-A (family)
Typical use	contract, plugin point, capability	shared code + shared state

★ The two rows that decide almost every real design question: **instance fields** and **multiple inheritance**.

Do I need a FIELD or a CONSTRUCTOR? → abstract class
 Do unrelated classes need this ability? → interface
 Might the implementer already extend something? → interface

Part 3 — Interfaces after Java 8 and 9

Q: What can an interface contain today?

The easiest way to remember — **six things**:

1. constants (implicitly public static final)
2. abstract methods (implicitly public abstract)
3. default methods (Java 8) → inherited body, overridable
4. static methods (Java 8) → utility, called on the interface
5. private methods (Java 9) → shared helper for default methods
6. private static (Java 9) → shared helper for static methods

All six in one example:

```

public interface PaymentProcessor {

    // 1. constant – implicitly public static final (no other option)
    int MAX_RETRIES = 3;

    // 2. abstract method – implicitly public abstract
    PaymentResult process(Payment p);

    // 3. DEFAULT method (Java 8) – an inheritable implementation
    default PaymentResult processWithRetry(Payment p) {
        for (int i = 0; i < MAX_RETRIES; i++) {
            PaymentResult r = process(p);
            if (r.isSuccess()) return r;
            log("retry " + i);                // calls the private
helper
        }
        return PaymentResult.failed();
    }

    // 4. STATIC method (Java 8) – belongs to the type, NOT inherited
    static PaymentProcessor noop() {
        return p -> PaymentResult.skipped();
    }

    // 5. PRIVATE method (Java 9) – shared logic, hidden from implementers
    private void log(String msg) {
        System.out.println("[pay] " + msg);
    }

    // 6. private static (Java 9) – helper for static methods
    private static boolean valid(Payment p) {
        return p != null && p.amount() > 0;
    }
}

```

What it still **cannot** have:

- ✗ instance fields (state) → int retryCount; ← not allowed
- ✗ constructors
- ✗ instance initialiser blocks
- ✗ final or synchronized default methods

Q: Why were default methods added?

The easiest way to remember:

To add NEW methods to an old interface WITHOUT breaking every class that already implements it.

Java 8 needed to add `stream()`, `forEach()` and `removeIf()` to `Collection` and `Iterable`.

Adding an ABSTRACT method to `Collection`

↓

`ArrayList`, `LinkedList`, `HashSet`, `TreeSet`, `Vector`,
+ every custom `Collection` ever written, worldwide

↓

ALL of them stop compiling ▲

```
default Stream<E> stream() {                                // ✓ has a body
    return StreamSupport.stream(spliterator(), false);
}
```

Old classes → inherit the body → still compile ✓

New classes → may override it ✓

Real-world idea: your **mobile operator** adds a new free service. Existing customers get it automatically without changing their SIM. Anyone who wants a different version can configure their own.

▲ **The follow-up:** "So are default methods just multiple inheritance?"

"Multiple inheritance of behaviour, yes. Not of state — interfaces still cannot hold instance fields, and duplicated state is what made C++-style multiple inheritance genuinely hard. That is the deliberate line Java drew."

Q: Can a default method be `final` or `synchronized`?

No — neither.

```
interface I {
    final default void m() { }           // ✗ compile error
    synchronized default void n() { }    // ✗ compile error
}
```

final → forbids overriding, but the whole point of a default method is that implementers **CAN** override it
 synchronized → locking belongs to the object, and the interface doesn't own one

Also: a default method **cannot override** `equals()`, `hashCode()` or `toString()`. Those come from `Object`, and the class-wins rule means the interface version could never be used anyway.

Q: What is a private interface method for?

To share code BETWEEN two default methods without exposing it publicly.

```
interface Report {

    default String daily() { return build("DAILY"); }
    default String weekly() { return build("WEEKLY"); }

    private String build(String type) {           // Java 9 ★ hidden
helper
        return "Report[" + type + "]";
    }
}
```

Before Java 9 that helper had to be `public default build(...)`, which then appeared on **every implementing class's public API** — leaking an internal detail to the whole world.

Java 8: helper must be public default → pollutes every implementer
 Java 9: helper can be private → clean public API ✓

Part 4 — Abstract classes in depth

Q: Can an abstract class have a constructor if it can't be instantiated?

Yes — and this is a favourite trick question.

The constructor is not there to create an abstract object. It runs as part of the **subclass's** construction chain, to initialise the shared fields.

```
abstract class Vehicle {  
    protected final String vin;           // shared state – impossible in  
    an interface  
  
    protected Vehicle(String vin) {       // ✓ constructor allowed  
        this.vin = vin;  
    }  
  
    abstract void start();                 // each subclass decides  
  
    void honk() {                          // shared behaviour  
        System.out.println("beep");  
    }  
}  
  
class Car extends Vehicle {  
    Car(String vin) { super(vin); }        // must call it  
    void start() { System.out.println("ignition"); }  
}
```

```
new Vehicle("x");    // ✗ compile error: Vehicle is abstract; cannot be  
instantiated  
new Car("x");        // ✓
```

What actually happens:


```

new Car("VIN123")
  ↓
Car() → super("VIN123")
  ↓
Vehicle() ← the abstract class's constructor RUNS here ★
  ↓ sets vin
back to Car()

```

Real-world idea: a **house foundation**. You cannot live in a foundation on its own, but every house built on it uses it — and it must be laid first.

Q: Can an abstract class have zero abstract methods?

Yes.

```

abstract class Base {
    void m() { System.out.println("works"); }    // no abstract methods
at all
}

```

abstract on a class means only one thing: **"you cannot instantiate me directly."** It says nothing about whether the methods have bodies.

This is useful when the type only makes sense as a base — Spring's `AbstractApplicationContext` is a real example.

Q: Can an abstract method be **private**, **static**, or **final**?

None of the three — each is a self-contradiction:

Combination	Why it fails
<code>private abstract</code>	<code>private</code> is not inherited → nothing could ever implement it
<code>static abstract</code>	static methods are hidden, not overridden — polymorphism doesn't apply
<code>final abstract</code>	<code>final</code> forbids overriding; <code>abstract</code> demands it

The easy way to remember:

```
abstract means "SOMEONE ELSE must write this."  
private / static / final all mean "nobody else can." → contradiction ▲
```

Q: What happens if a subclass doesn't implement all abstract methods?

The subclass must itself be declared **abstract**, passing the obligation further down.

```
abstract class A { abstract void m(); abstract void n(); }  
  
class B extends A {                // ✗ compile error  
    void m() { }                  // n() is missing  
}  
  
abstract class B extends A { // ✓ B stays abstract, C must finish the job  
    void m() { }  
}  
  
class C extends B {  
    void n() { }                  // ✓ now everything is implemented  
}
```

```
Implement ALL abstract methods → concrete class ✓  
Implement SOME                 → the subclass must be abstract too
```

Q: Can an abstract class implement an interface without implementing its methods?

Yes — it can leave them abstract for its own subclasses. This is the **skeletal implementation** (or adapter) pattern, and the JDK uses it everywhere.

```

interface List<E> { /* ~25 methods */ }

abstract class AbstractList<E> implements List<E> {
    // implements ~23 of them using get() and size()
    public abstract E get(int index);
    public abstract int size();
}

class MyList<E> extends AbstractList<E> {
    public E get(int i) { ... }      // only TWO methods to write ✓
    public int size() { ... }
}

```

Interface → the full contract (25 methods)
 Abstract class → does the boring 23 for you
 Concrete class → writes only the 2 that actually differ ★

Part 5 — When to choose which (the real question)

Q: When would you use an abstract class over an interface?

Choose an abstract class when...	Choose an interface when...
Subclasses share state (fields)	Unrelated classes need the same capability
You want to share constructor logic	You need a class to have multiple types
You need protected members or a final method	You want loose coupling and easy testing
The relationship is genuinely is-a	The relationship is can-do (Serializable , Comparable , Runnable)
You're writing a template method with fixed steps	You're defining a plugin point with several implementations
The hierarchy is yours and closely related	Implementers may already extend something else

The quick decision tree:

Do I need an instance FIELD or a CONSTRUCTOR?

YES → abstract class

NO → Might the implementer already extend something?

YES → interface

NO → interface anyway (it's cheaper to change later)

The Template Method — where an abstract class truly earns its place ★

This is the single strongest example to give.

```
abstract class ReportGenerator {  
  
    // final: the ALGORITHM is fixed; only the steps vary  
    public final Report generate(Query q) {  
        var data = fetch(q);           // subclass decides  
        var body = format(data);       // subclass decides  
        audit(q);                     // shared, identical for  
everyone  
        return new Report(body);  
    }  
  
    protected abstract List<Row> fetch(Query q);  
    protected abstract String format(List<Row> rows);  
  
    private void audit(Query q) { /* shared implementation + shared state  
*/ }  
}
```

generate() ← FINAL: nobody can change the order of the steps



```
graph TD  
    generate[generate()] --> fetch[fetch  
(child)]  
    generate --> format[format  
(child)]  
    generate --> audit[audit  
(shared)]  
    generate --> return[return]
```

You **cannot** express this cleanly with an interface:

`generate()` must be `final` → interfaces can't have final methods ✗
`audit()` uses shared private state → interfaces can't have state ✗

Real-world idea: a **recipe**. The steps and their order are fixed — chop, cook, garnish. Each chef decides *what* to chop and *how* to garnish, but nobody is allowed to garnish before cooking.

● In your RoboGebra project — you made BOTH choices, correctly ★★

This is the strongest possible answer to "*when would you use which?*" — because you have one of each, and the reason is textbook.

You chose an ABSTRACT CLASS — security/

AbstractServiceAuthenticationFilter.java

```
public abstract class AbstractServiceAuthenticationFilter extends
OncePerRequestFilter {

    protected final RequestMappingHandlerMapping handlerMapping;    // ★
    SHARED STATE
    protected final ExceptionResponseWriter responseWriter;
    protected final UserDetailsService userDetailsService;

    protected AbstractServiceAuthenticationFilter(...) { ... }    // ★
    CONSTRUCTOR

    // the ALGORITHM is written ONCE here (doFilterInternal) — a TEMPLATE
    METHOD ★

    protected abstract Class<? extends Annotation>
getAnnotationClass(); // subclass decides
    protected abstract String getHeaderName();
    protected abstract String getExpectedToken();
    protected abstract String getFilterName();
    protected abstract UserRole getUserRole();

    protected String extractToken(HttpServletRequest r) {            // ★
    overridable HOOK
        return r.getHeader(getHeaderName());
    }
    protected void additionalValidation(HttpServletRequest r) { }    // ★
    default: nothing
}
```

Subclasses: `CrmServiceAuthenticationFilter`, `WebhookAuthorizationFilter`

— about thirty lines each.

WHY an abstract class here:

- ✓ shared STATE (three collaborators) → an interface can't hold these ★
- ✓ a CONSTRUCTOR to initialise them → an interface has none ★
- ✓ one shared ALGORITHM, five varying steps → template method ★
- ✓ `protected` members → interfaces are all public

★

You chose an INTERFACE — `common/service/event/`

`EventTrackingService.java`

```
public interface EventTrackingService {           // ★ pure contract, NO
state
    void trackUserLogin(User user);
    void trackPaymentEvent(SubscriptionInstanceWithItems subscription);
    void trackSolutionEvent(String userId, ExerciseSolutionDTO dto);
}

@Service @Profile({"local","dev"})
public class LoggingEventTrackingService implements EventTrackingService
{ ... }

@Service @Profile({"prod"})
public class PostHogEventTrackingService implements EventTrackingService
{ ... }
```

WHY an interface here:

- ✓ the two implementations share NO code at all ★
(one writes a log line, the other posts JSON to PostHog)
- ✓ nothing to inherit — only a contract to honour
- ✓ swappable by environment, and trivial to stub in a test ★

■ The answer to give — 45 seconds ★

"I've used both on the same codebase and the deciding factor was exactly the textbook one — shared state and shared code.

The service-authentication filters are an abstract class. CRM auth and webhook auth follow an identical algorithm: check whether the handler method carries a particular annotation, read a token from a header, compare it against configuration, and build the security context. Only five things differ — which annotation, which header, which expected token, the filter name for logging, and which role to authenticate as. So the algorithm is written once in `doFilterInternal` as a template method, the base class holds the three collaborators all subclasses need, and each subclass is about thirty lines of five overrides. There are also two optional hooks with defaults — token extraction and extra validation — for the subclasses that need them.

The analytics tracker is an interface, because the logging implementation and the PostHog implementation share nothing at all. There's no code to inherit — just a contract, with the implementation chosen by Spring profile.

So the rule I actually apply: if I catch myself wanting a field or a constructor, it's an abstract class. If it's only a contract, it's an interface."

★ That answer works because it names **the concrete reason for each**, not a preference.

Q: Why does modern design favour interfaces?

Four reasons, in order of how convincing they sound in an interview:

1. Single inheritance is SCARCE
→ spend it on an abstract class and the class can never extend anything else
2. TESTABILITY
→ mocking an interface is trivial; a base class drags its constructor and its state into every test
3. Dependency Inversion (the D in SOLID)
→ depend on abstractions. Spring wires interfaces; @Transactional and AOP originally used JDK dynamic proxies, which require one
4. Composition over inheritance
→ deep class hierarchies become rigid; interfaces + delegation don't

▲ **Don't over-swing.** "Always use interfaces" is also a wrong answer.

An interface with exactly one implementation, created purely "for the pattern" (`UserService` + `UserServiceImpl`), is ceremony, not design.

Say this instead: "I add the interface when there's a real second implementation, a real test seam, or a module boundary." That nuance reads as experience.

Q: Can you use both together?

Yes — and this is the best answer to "which one?", because the JDK itself does it:

```
interface List<E> { ... } // the
contract
abstract class AbstractList<E> implements List<E> { ... } // skeletal
shared code
class ArrayList<E> extends AbstractList<E> { ... } // concrete
```

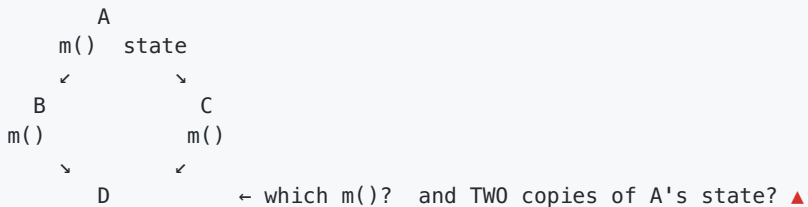
```
List (interface)      ← callers depend on THIS
  ↑
AbstractList (abstract) ← optional convenience base
  ↑
ArrayList (concrete)
```

◆ **Say this:** "Interface for the contract, abstract class for the convenience base. Callers depend on the interface; implementers can extend the base if it helps them — or ignore it entirely."

Part 6 — Diamond problem & multiple inheritance

Q: Why doesn't Java allow multiple class inheritance?

The **diamond problem**:



Ambiguity → which m() does D inherit?
Worse → D would hold TWO copies of A's fields

C++ solves it with virtual inheritance and a lot of complexity. Java removed the ambiguity by allowing only **one superclass**, then restored the useful half — multiple **types** — through interfaces, which have no state to duplicate.

Real-world idea: you can have only **one** biological family (state you inherit), but any number of **certificates** (capabilities).

Q: Java 8 default methods brought the diamond back. How is it resolved?

Compile error — you must disambiguate explicitly.

```
interface A { default String hello() { return "A"; } }
interface B { default String hello() { return "B"; } }

class C implements A, B {
    // ✗ without this: "class C inherits unrelated defaults for hello()
    from types A and B"
    @Override
    public String hello() {
        return A.super.hello();    //
    }
    ✓ the special InterfaceName.super syntax
}
```

```
A.hello()      B.hello()
   ↘           ↙
      C  ← the compiler REFUSES to guess; you must choose
```

Note the syntax carefully — it is `A.super.hello()`, not `super.hello()`.

Q: What if a class inherits the same method from a superclass and an interface?

"Class wins." Always.

```
class Base { public String hi() { return "class"; } }
interface I { default String hi() { return "interface"; } }

class C extends Base implements I { }

new C().hi();    // "class" ★
```

The rule is deliberate: adding a **default** method to an interface must never silently change the behaviour of an existing class.

The three resolution rules, in order:

1. The most specific CLASS wins over any interface ★
2. The most SPECIFIC INTERFACE wins (a sub-interface beats its parent)
3. Otherwise → compile error → you must override and choose

Part 7 — Functional interfaces

Q: What is a functional interface?

The easiest way to remember:

```
Functional interface = EXACTLY ONE abstract method (SAM = Single Abstract Method)
→ so it can be the target of a lambda
```

default, **static** and **private** methods do **not** count, and neither do public **Object** methods like **equals**.

```
@FunctionalInterface // optional, but the compiler then
ENFORCES SAM
interface Validator<T> {

    boolean validate(T t); // the ONE
    abstract method

    default Validator<T> and(Validator<T> other) { // doesn't
break SAM ✓
        return t -> this.validate(t) && other.validate(t);
    }
}
```

```
Validator<String> notBlank = s -> s != null && !s.isBlank();
Validator<String> maxLen = s -> s.length() <= 50;

boolean ok = notBlank.and(maxLen).validate(input);
```

The built-ins to be able to name:

Function<T,R>	Supplier<T>	Consumer<T>	Predicate<T>
BiFunction	UnaryOperator	BinaryOperator	
Runnable	Callable	Comparator	

▲ **Trap:** *"Can an abstract class be a lambda target?"*

No. Lambdas only target functional **interfaces**, even if the abstract class happens to have exactly one abstract method. That is a concrete, technical reason to prefer an interface for a single-method contract.

Q: What is a marker interface?

Marker interface = an interface with NO members at all,
used purely to TAG a type.

```
public interface Serializable { } // completely empty
```

```
class Employee implements Serializable { } // "I am safe to serialise"
```

The JVM and libraries then check `instanceof Serializable` at runtime.

Serializable

→ the JVM checks it during object serialisation

Cloneable

→ `Object.clone()` checks it, else `CloneNotSupportedException`

RandomAccess

→ `Collections.binarySearch()` picks a faster algorithm for lists

Real-world idea: a **"Fragile" sticker** on a parcel. It carries no instructions — it just marks the box so the handler behaves differently.

Modern code usually prefers **annotations** for tagging, but `Serializable` remains an interface because the JVM itself checks it.

Part 8 — Real examples from the JDK, Spring and your projects

Example	Which	Why
<code>Comparable</code> , <code>Runnable</code> , <code>Serializable</code>	interface	a capability adopted by unrelated types
<code>List</code> / <code>AbstractList</code> / <code>ArrayList</code>	both	contract + skeletal base + concrete
<code>HttpServlet</code>	abstract class	shares request-dispatch logic; you override <code>doGet</code> / <code>doPost</code>
<code>Thread</code>	class	...which is exactly why <code>Runnable</code> is preferred — extending <code>Thread</code> burns your inheritance slot
Spring <code>JpaRepository</code>	interface	Spring Data generates the implementation at runtime
Spring <code>OncePerRequestFilter</code>	abstract class	shared "run once per request" plumbing around your hook

Example	Which	Why
<code>WebMvcConfigurer</code>	interface (all defaults)	implement only the callbacks you care about — this replaced the old adapter class

That last row is a great one to mention: Spring **deleted**

`WebMvcConfigurerAdapter` once Java 8 default methods existed, because the adapter class was only ever a workaround for interfaces having no bodies.

Your project answers — have one of each ready

RoboGebra (interface): *"The explanation engine had to support more than one AI provider. I defined an `ExplanationProvider` interface with a single `explain(Problem)` method and had two implementations behind it. The service depends on the interface, Spring injects the configured one, and tests inject a stub — no mocking framework needed. Adding a provider is a new class, not an edit to existing code."*

EasyVisa (abstract class): *"Document validators shared real state and real steps — file-size limits, the audit trail, the error-collection list. I made an `AbstractDocumentValidator` with a `final validate()` template method that ran the shared checks and then called an abstract `validateSpecific()`. The passport, photo and bank-statement validators each implemented only that one method. Template method — the shared part genuinely needed fields, so an interface would have meant copy-pasting it into every validator."*

Notice the shape of both answers: **what the problem was** → **which one I chose** → **why the other one wouldn't work**. That last clause is what scores.

Part 9 — Related concepts they drift into

Q: Abstraction vs encapsulation?

Abstraction → hides COMPLEXITY → design level → interface / abstract class → WHAT
Encapsulation → hides DATA → code level → private fields + getters → HOW

"An interface abstracts; *private* encapsulates. *List* is abstraction — I don't know or care that it's an array underneath. *ArrayList* 's private *elementData* field is encapsulation."

That single sentence answers the question completely.

Q: Overloading vs overriding?

Overloading = same NAME, different PARAMETERS → same class → COMPILE time
Overriding = same SIGNATURE, child rewrites → parent/child → RUNTIME

	Overloading	Overriding
Same class?	Yes (or inherited)	Subclass
Signature	different parameters	identical signature
Return type	can differ freely	same or covariant
Binding	compile-time (static)	runtime (dynamic)
Access modifier	anything	cannot be more restrictive
Exceptions	anything	no new or broader checked exceptions
<code>static / private / final</code>	overloadable	not overridable (static = hiding)

★ Trap: you cannot overload by changing **only** the return type — Java picks the method from the arguments you pass.

Q: What are sealed classes/interfaces (Java 17)?

sealed = an abstraction that controls EXACTLY who may implement it.

```
public sealed interface Shape permits Circle, Square, Triangle { }  
public record Circle(double r) implements Shape { }
```

Two wins:

1. Nobody outside the `permits` list can implement it
2. A switch over Shape is EXHAUSTIVE – the compiler proves every case is covered, so no `default` is needed ★

Ideal for closed domain models — a result type, a state machine, a fixed set of payment methods. See [12 — Java 17](#).

public → open ground | final → locked room | sealed → invitation only

Part 10 — Predict the output

Q1

```
interface A { default void hi() { System.out.println("A"); } }  
interface B { default void hi() { System.out.println("B"); } }  
class C implements A, B { }
```

Answer

****Compile error**** — `class C inherits unrelated defaults for hi() from types A and B`. The compiler will not guess. Fix it by overriding `hi()` in C, optionally delegating with `A.super.hi()`.

Q2


```
class P { public String who() { return "class"; } }
interface I { default String who() { return "interface"; } }
class C extends P implements I { }
System.out.println(new C().who());
```

Answer

`class` ****Class wins over interface — always.**** The rule exists so that adding a `default` method to an interface can never silently change an existing class's behaviour.

Q3

```
interface Config { int TIMEOUT = 30; }
class Impl implements Config {
    void change() { TIMEOUT = 60; }
}
```

Answer

****Compile error**** — interface fields are implicitly `public static final`, so `TIMEOUT` cannot be reassigned. This is the "interfaces can't hold state" rule showing up in practice.

Q4 ★ (the nastiest one, and a great one to raise proactively)

```
abstract class Animal {
    Animal() { System.out.println("Animal ctor"); speak(); }
    abstract void speak();
}
class Dog extends Animal {
    private String sound = "woof";
    void speak() { System.out.println(sound); }
}
new Dog();
```

Answer

```
Animal ctor
null
```

Why? The construction order is:

```

new Dog()
  ↓
Dog() calls super() implicitly
  ↓
Animal() runs → prints "Animal ctor" → calls speak()
  ↓
speak() is OVERRIDDEN, so Dog.speak() runs
  ↓
but Dog's field initialisers have NOT run yet → sound is still null
  ↓
prints null
  ↓
NOW `sound = "woof"` executes

```

****Never call an overridable method from a constructor.**** This is a genuinely nasty real-world bug.

Q5

```

abstract class A { abstract void m(); }
class B extends A { }

```

Answer

****Compile error**** — `B` is not abstract and does not override abstract method `m()`. Either implement `m()` in `B`, or declare `B` abstract as well.

Q6

```

interface I {
    static void s() { System.out.println("static"); }
    default void d() { System.out.println("default"); }
}
class C implements I { }
C c = new C();
c.d();
c.s();

```

Answer

`c.d()` prints `default`. `c.s()` is a ****compile error**** — interface `static` methods are ****not inherited**** by implementing classes. You must call `I.s()`. ★ This differs from class static methods, which *are* inherited by subclasses.

Q7

```
@FunctionalInterface
interface F {
    void a();
    void b();
}
```

Answer

****Compile error**** — `@FunctionalInterface` requires exactly one abstract method, and F has two. Without the annotation it would compile fine, but it still could not be used as a lambda target.

Part 11 — Rapid-fire table

Question	Answer
Multiple inheritance?	Interface ✓ many · abstract class ✗ one
Instance fields?	Interface ✗ (constants only) · abstract class ✓
Constructor?	Interface ✗ · abstract class ✓ (runs via <code>super()</code>)
Method bodies in an interface?	✓ <code>default</code> / <code>static</code> (8), <code>private</code> (9)
Can an abstract class have no abstract methods?	✓ — <code>abstract</code> only means "not instantiable"
Can an abstract method be private/static/final?	✗ all three — contradictions
Interface field modifiers?	implicitly <code>public static final</code>
Interface method modifiers?	implicitly <code>public abstract</code> (unless default/static/private)
Can an interface extend another?	✓ and multiple — <code>interface A extends B, C</code>
Can an abstract class implement an interface?	✓ without implementing its methods

Question	Answer
Diamond with defaults?	compile error → override, use <code>A.super.m()</code>
Class vs interface method conflict?	class wins
Are interface static methods inherited?	x — call <code>Interface.method()</code>
Lambda target?	functional interface only, never an abstract class
Functional interface?	exactly one abstract method (SAM)
Marker interface?	empty interface used as a tag — <code>Serializable</code>
Can a default method be final/synchronized?	x neither
Why prefer interfaces?	one-superclass scarcity, testability, DIP, loose coupling
Why still use abstract classes?	shared state + constructor + template method + <code>final</code> methods
Rule of thumb	Interface = contract / can-do · Abstract class = shared code / is-a

Part 12 — Drills

Cover the answers and say them out loud.

1. You need a `Bird` and a `Plane` to both be flyable. Interface or abstract class?

Answer

`**Interface `Flyable`.` The classes are completely unrelated and ``Bird`` almost certainly already extends ``Animal``. A "can-do" capability across unrelated hierarchies is the textbook interface case.

2. Three payment gateways share retry logic, a timeout field and an audit log, but each calls a different API. Design it.

Answer

Both.

```
interface PaymentGateway {                                // the contract callers
    depend on
    PaymentResult pay(Payment p);
}

abstract class AbstractPaymentGateway implements PaymentGateway {
    private final int timeout;                            // shared STATE
    public final PaymentResult pay(Payment p) {           // shared retry + audit
        ...
        return callProvider(p);                          // the only varying step
    }
    protected abstract PaymentResult callProvider(Payment p);
}
```

Template method + Dependency Inversion. The shared part needs a field, so it has to be an abstract class; the contract stays an interface so callers and tests stay decoupled.

3. Can you add a method to a published interface without breaking implementers? How?

Answer

Yes — add it as a `default` method with a sensible body. That is exactly why Java 8 added them, so `Collection.stream()` could ship. Adding an **abstract** method breaks every implementer.

4. Why can't interfaces have constructors?

Answer

Constructors exist to initialise **instance state**, and interfaces have none. There is also no single super-interface chain to order construction — a class can implement many interfaces, so which constructor would run, and in what order?

5. Your teammate creates an interface for every service class, each with exactly one ***Impl**. Push back or accept?

Answer

Push back, politely and with nuance. Ceremony interfaces double the file count for no seam. Modern Spring proxies concrete classes with CGLIB, and

Mockito mocks concrete classes fine. Add the interface when there is a second implementation, a module or API boundary, or a genuine test seam.

6. Give one thing an abstract class can do that an interface never can, and vice versa.

Answer

****Abstract class only:**** hold mutable instance state (and run a constructor to initialise it); also `final` methods and `protected` members. ****Interface only:**** be implemented by a class that already extends something else — participating in multiple inheritance of type. Also: be a lambda target.

7. An interface has a `default` method and the implementing class extends a parent that has the same method. Which runs?

Answer

****The parent class's method.**** Class always wins over interface.

Part 13 — The C#/.NET version

Your log shows .NET rounds too ([29 — ASP.NET MVC](#)). The same question in C# has different details, and mixing them up is a visible error:

	Java	C#
Default methods in interfaces	Java 8 (<code>default</code>)	C# 8 (default interface members)
Virtual by default?	methods are virtual unless <code>final</code>	methods are non-virtual unless <code>virtual</code> ; overriding needs <code>override</code>
Explicit implementation	<code>x</code>	<code>✓ void IFoo.Bar() { }</code> — resolves name clashes cleanly
Diamond disambiguation	<code>A.super.m()</code>	explicit interface implementation

	Java	C#
Constants in interfaces	implicit <code>public static final</code>	not allowed pre-C# 8 (<code>static readonly</code> on a class instead)
Abstract keyword	<code>abstract</code>	<code>abstract</code> , and the subclass must use <code>override</code>

The biggest one to keep straight: **in Java every method is virtual by default; in C# you must opt in with `virtual`.**

Quick Revision Sheet

Abstract class → IS-A → shared STATE + CODE → extends (ONE) → Bird
Interface → CAN-DO → CONTRACT → implements (MANY) → Swimmable

Interface today: constants, abstract, default (8), static (8), private (9)
Interface never: instance fields, constructors, final/synchronized defaults

Abstract class CAN have: constructor ✓, zero abstract methods ✓, final methods ✓
abstract + private/static/final = contradiction ✗

Diamond → compile error → override → A.super.hello()
Class vs interface → CLASS WINS ★
Interface static methods are NOT inherited → call I.s()

Functional interface = 1 abstract method (SAM) → lambda target
(an abstract class can NEVER be a lambda target)
Marker interface = empty tag → Serializable

Template method → the reason abstract classes still exist:
`public final generate() { fetch(); format(); audit(); }`

Choose: need a FIELD or CONSTRUCTOR? → abstract class. Otherwise → interface.

Best answer: "Interface for the contract, abstract class for the shared base."

Related files: [05 — Core Java](#) · [17 — SOLID & Design Patterns](#) · [12 — Java 17 \(sealed\)](#) · [23 — Predict the Output](#)

● Tech Mahindra — Angular Developer Round (Aug 2026) · RESULT PENDING

Every question below was actually asked in this round, in roughly this order. Result is pending — so this file has two jobs: **(a)** be the answer key for the follow-up/next round if they call you back, and **(b)** feed the recall log, because these same questions recur everywhere (Observables vs Promises has now been asked in **4** rounds; interceptors in **3**).

Read the shape of this round: it was a **breadth sweep** — Angular state → NgRx → TypeScript → HTTP layer → core JavaScript → HTML/CSS. Short questions, many topics, no deep coding. That kind of round is won by **crisp 3-sentence answers with one concrete example each**, not by long explanations. If you get a second round with them, expect the opposite: they'll pick 3 of these and drill.

● Answer these with YOUR CODE, not with definitions ★

The questions in this file repeat across companies. What changes the outcome is **finishing each answer with one concrete thing from your own codebase**:

Lifecycle hooks (5 rounds ★) → "ngOnDestroy in 100 components, 426 takeUntil" Interceptors (4 rounds ★) → the FormData Content-Type trap + cancelRequests\$ Observable vs Promise (4 ★) → "switchMap 246 times — the auth interceptor and every search" Data binding (4 rounds) → 252 @Input, and OnPush on the 6 heavy components HashMap internals (3 rounds) → the N+1 aggregation fix (same reasoning, in Mongo) Lazy loading (3 rounds) → 27 route resolvers, access-ruler resolvers

Definition → **one thing from my code** → **one trade-off**. Three sentences. That is what five years sounds like.

Full inventory: **39 — RoboGebra Code Examples**.

The questions they asked

#	Question	Section
1	State management	→
2	NgRx and NgRx features	→
3	Any idea of TypeScript?	→
4	HTTP interceptor and its benefits	→
5	String interpolation	→
6	Observables and Promises	→
7	HttpClient — features & benefits	→
8	Callback, callback hell	→
9	Event capturing and event bubbling	→
10	ES5 vs ES6	→
11	Arrow functions	→
12	OOP in TypeScript	→
13	Modules — how, and types	→
14	<code>unknown</code> type and <code>never</code> type	→
15	HTML and CSS	→

1. State management in Angular

The answer they want is a ladder, not one tool. Start by naming the levels, then say where you land.

"State management is deciding who owns data and how changes propagate. In Angular I think of it as four levels:

1. Component state — a plain field in the component. Fine for anything not shared: a toggle, a form's local flag.
2. Parent-child — `@Input()` down, `@Output()` up. Fine for one or two levels.
3. A shared service with a `BehaviorSubject` (or a `signal` in newer Angular) — this is the workhorse. The service owns the state, exposes it as a read-only `Observable` via `asObservable()`, and any component subscribes. It covers the large majority of apps.
4. `NgRx` — a single immutable store with actions/reducers/effects, for genuinely app-wide state with complex flows and a need for traceability.

My default is level 3, and I move to `NgRx` when several unrelated features read and write the same state, or when I need time-travel debugging and a strict audit of what changed."

The service-with-subject pattern — be ready to write it:

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private readonly items$ = new BehaviorSubject<CartItem[]>([]);
  readonly cart$ = this.items$.asObservable(); // read-only
  outward
  readonly total$ = this.cart$.pipe(
    map(items => items.reduce((s, i) => s + i.price * i.qty, 0))
  );

  add(item: CartItem): void {
    this.items$.next([...this.items$.value, item]); // new array —
    immutability
  }
}
```

Why `BehaviorSubject` specifically: it holds a current value, so a component subscribing *later* (e.g. a lazily-loaded route) immediately receives the latest state instead of waiting for the next emission. `Subject` would leave it blank.

◆ **Modern add-on that scores:** "On Angular 16+ I'd consider signals for this — `signal()` for the state, `computed()` for derived values. It's synchronous, needs

no subscription or `async` pipe, and works with zoneless change detection. RxJS still wins for anything `async` or stream-shaped, which is why the two are designed to interoperate via `toSignal` / `toObservable`."

▲ **Honesty position:** your NgRx is from **EasyVisa**, not RoboGebra. Say it that way — see [21](#) — NgRx.

2. NgRx and its features

"NgRx is Redux for Angular — a single immutable store as the source of truth, changed only by dispatching actions, applied by pure reducers, read through memoized selectors, with side effects isolated in effects. The whole point is that state changes are traceable: every change has a named action, so the DevTools show you exactly what happened, in order, and you can replay it."

The building blocks — name all five plus the extras:

Feature	What it is
Store	One immutable state tree for the whole app; an <code>Observable</code> of state
Actions	Plain objects <code>{ type, payload }</code> describing <i>what happened</i> (not what to do)
Reducers	Pure functions <code>(state, action) => newState</code> . No mutation, no HTTP, no <code>Date.now()</code>
Selectors	Query the store; memoized — recompute only when their inputs change
Effects	Where the impure work lives — HTTP, routing, storage. Listen for an action, do the work, dispatch a result action
<code>@ngrx/entity</code>	Normalised collections — <code>EntityAdapter</code> gives <code>addMany</code> / <code>upsertOne</code> / <code>selectAll</code> for free

Feature	What it is
@ngrx/store-devtools	Time-travel debugging in the browser
@ngrx/component-store	Local, component-scoped store — NgRx patterns without the global store
@ngrx/signals (SignalStore)	The newer signal-based store
@ngrx/router-store	Puts router state in the store

The data flow — draw this if there's a whiteboard:

```

Component —dispatch(action)—> Reducer —> new State —> Selector —>
Component (async pipe)

|
|
|_____> Effect —> HTTP/API —> ^
dispatch(successAction)——|

```

Naming convention worth quoting: `'[Source] Event'` — e.g. `'[Visa Page] Load Documents'`, `'[Documents API] Load Documents Success'`. Actions describe **events that happened**, not commands, which is why one event can be handled by several reducers.

```

export const loadDocs = createAction('[Visa Page] Load Documents',
  props<{ id: string }>());
export const loadDocsSuccess = createAction('[Docs API] Load Success',
  props<{ docs: Doc[] }>());

export const reducer = createReducer(initialState,
  on(loadDocs, s => ({ ...s, loading: true })), // new
  object, never mutate
  on(loadDocsSuccess, (s, { docs }) => ({ ...s, docs, loading: false })),
);

loadDocs$ = createEffect(() => this.actions$.pipe(
  ofType(loadDocs),
  switchMap(({ id }) => this.api.getDocs(id).pipe(
    map(docs => loadDocsSuccess({ docs })),
    catchError(err => of(loadDocsFailure({ error: err.message }))) //
  ))
  inner catch — keeps the effect alive
));

```

▲ The two traps they follow up with:

1. **catchError must be on the inner observable.** Put it on the outer pipe and the error kills the effect stream permanently — the effect stops listening and that feature silently dies.
2. **Reducers must be pure.** No `Math.random()`, no `new Date()`, no service calls, no mutation — otherwise time-travel and memoization both break.

When NOT to use NgRx (say this unprompted — it reads as judgement):

"For a small or medium app it's a lot of boilerplate for state a service with a `BehaviorSubject` would hold fine. I'd use it when several features share state, when the async flows are complex enough that traceability matters, or on a big team that needs one enforced pattern."

3. "Any idea of TypeScript?"

An invitation to demonstrate range in 45 seconds. Don't answer "yes".

"Yes — all my Angular work is TypeScript. It's a typed superset of JavaScript that compiles down to JS; the types exist only at compile time, so there's no runtime cost and no runtime checking. What it buys is catching errors at build time and, honestly just as much, the editor experience — autocomplete, safe rename, find-usages across a large codebase.

Day to day I use: interfaces and type aliases for API models, union and literal types for finite states, generics for reusable services, enums, the utility types — `Partial`, `Pick`, `Omit`, `Record`, `Readonly` — optional chaining and nullish coalescing, and strict mode with `strictNullChecks` on, which is what actually removes the whole class of undefined bugs."

Have these ready if they probe:

```
// interface vs type
interface User { id: number; name: string; }           // extendable,
declaration merging, best for objects
type Status = 'draft' | 'submitted' | 'approved';      // unions,
primitives, tuples – types only

// generics
function first<T>(arr: T[]): T | undefined { return arr[0]; }

// utility types
type UserPatch = Partial<User>;                       // all optional – perfect for
a PATCH payload
type UserCard  = Pick<User, 'id' | 'name'>;
type NoId      = Omit<User, 'id'>;
type ById      = Record<number, User>;

// type guard – narrowing
function isUser(x: unknown): x is User {
  return typeof x === 'object' && x !== null && 'id' in x;
}
```

▲ **The follow-up:** *"Does TypeScript validate an HTTP response at runtime?"* — **No.**
`http.get<User[]>()` is a compile-time assertion only; if the API returns something else, TypeScript never notices. For real validation you need runtime checks (a guard, or a library like zod). That answer separates people who understand types from people who trust them.

4. HTTP interceptor and its benefits

"An interceptor sits in the middle of Angular's HTTP pipeline. Every request from `HttpClient` passes through the chain of interceptors on the way out and every response on the way back, so it's the one place to put cross-cutting concerns instead of repeating them in every service."

Functional interceptor (Angular 15+, the current style):

```

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).token;
  // requests are IMMUTABLE – you must clone to modify
  const authReq = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(authReq).pipe(
    catchError((err: HttpResponseError) => {
      if (err.status === 401) inject(Router).navigate(['/login']);
      return throwError(() => err);
    })
  );
};

// registration
bootstrapApplication(App, {
  providers: [provideHttpClient(withInterceptors([authInterceptor,
    loadingInterceptor]))]
});

```

Class-based (older / NgModule apps) — know both:

```

@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler):
    Observable<HttpEvent<any>> {
    return next.handle(req.clone({ setHeaders: { Authorization: `Bearer $
    {token}` } })));
  }
}

// providers: [{ provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor,
multi: true }] ← multi:true is essential

```

The benefits — list them, that's literally the question:

Benefit	Example
Auth	Attach the JWT/bearer token to every outgoing request
Centralised error handling	Map 401 → logout, 403 → forbidden page, 5xx → toast

Benefit	Example
Loading indicator	Increment a counter on request, decrement on response
Logging / correlation ids	Add a request id header, log timings
Base URL / headers	Prefix the API URL, set <code>Content-Type</code> , add locale
Retry	<code>retry({ count: 2, delay: 1000 })</code> on idempotent calls
Caching	Return a cached response without hitting the network
Mocking	Short-circuit to fake data during development
DRY	None of the above is repeated in 30 services

▲ Three follow-ups they ask:

1. **Order matters** — interceptors run in registration order on the request and in **reverse** order on the response.
2. **req is immutable** — you must `req.clone()`; mutating it directly does nothing.
3. **multi: true** — omit it in the class-based form and your interceptor **replaces** the whole array instead of joining it.
4. Interceptors only see traffic from **HttpClient** — not `fetch`, not `XMLHttpRequest`, not a third-party SDK.

5. String interpolation

"Interpolation is Angular's one-way binding from the component class to the template, using double curly braces: `{{ expression }}`. Angular evaluates the expression in the component's context, converts it with `toString()`, and inserts it into the DOM — and it re-evaluates on every change-detection cycle."

```

<h1>Hello {{ user.name }}</h1>
<p>Total: {{ price * qty | currency:'INR' }}</p>
<p>{{ user?.address?.city ?? 'Not provided' }}</p>
<img [src]="avatarUrl">                                <!-- property binding, not
interpolation -->
                                <!-- works for strings, but
[src] is preferred -->

```

Points that turn a 10-second answer into a good one:

- It is **one-way**, component → template. It's shorthand for a property binding to `textContent`.
- Angular **sanitizes** the output, so interpolated values are escaped — `{{ '<b>hi</b>' }}` renders as literal text, not bold. That's built-in XSS protection.
- **Template expressions are restricted:** no assignments, no `new`, no `++` / `--`, no bitwise ops, no `;`. Deliberately — they must be side-effect free.
- **Keep them cheap.** `{{ getTotal() }}` calls the method on **every** change-detection run, many times a second. Use a property, a pure pipe, or a memoized value instead — a real performance answer they like hearing.
- The delimiters are configurable via `interpolation: ['[', '']` in `@Component` — rarely needed, but a nice detail.

The four binding types, since interpolation is one of them:

Syntax	Direction	Name
<code>{{ value }}</code>	Component → View	Interpolation
<code>[prop]="value"</code>	Component → View	Property binding
<code>(event)="handler()"</code>	View → Component	Event binding
<code>[(ngModel)]="value"</code>	Both	Two-way (banana-in-a-box)

6. Observables vs Promises

Asked in 4 separate rounds now. This must be automatic.

	Promise	Observable
Values	One value (or one error)	Zero to many , over time
Execution	Eager — starts the moment it's created	Lazy — nothing runs until <code>subscribe()</code>
Cancellable	✗	✓ <code>unsubscribe()</code> — cancels the in-flight XHR
Operators	<code>.then</code> / <code>.catch</code> chaining only	100+ RxJS operators: <code>map</code> , <code>filter</code> , <code>switchMap</code> , <code>debounceTime</code> , <code>retry</code> ...
Retry	Manual — you must re-call the function	<code>retry()</code> / <code>retryWhen()</code> built in
Native?	✓ JavaScript	✗ RxJS library (Angular ships it)
Multicast	Always single-cast to its <code>.then</code> s	Unicast by default; multicast via <code>Subject</code> / <code>share()</code>
Angular uses it for	—	<code>HttpClient</code> , router events, forms <code>valueChanges</code> , <code>@Output EventEmitter</code>

The 20-second answer: "A *Promise* is a single future value, eager and not cancellable. An *Observable* is a stream of zero-to-many values, lazy — nothing happens until you subscribe — cancellable via `unsubscribe`, and composable with RxJS operators. That's why Angular's `HttpClient` returns an *Observable* even though an HTTP call yields one response: you get cancellation and operators like `retry`, `debounceTime` and `switchMap` for free."

The killer demonstration — typeahead search:

```
this.searchControl.valueChanges.pipe(
  debounceTime(300),           // wait for a pause in typing
  distinctUntilChanged(),      // ignore "same text again"
  switchMap(term => this.api.search(term)), // CANCELS the previous in-
    flight request
  takeUntilDestroyed()         // auto-unsubscribe on destroy
).subscribe(results => this.results = results);
```

"With Promises this needs manual timers and flags to discard stale responses, and you still can't cancel the request — so a slow earlier response can overwrite a newer one. `switchMap` solves that in one word."

Converting between them: `firstValueFrom(obs$)` / `lastValueFrom(obs$)` (RxJS 7+, replacing the deprecated `toPromise()`), and `from(promise)` the other way.

▲ **Follow-up: "what if you forget to unsubscribe?"** — memory leak: the subscription keeps the component alive and the callback keeps firing after navigation. Fixes: the **async pipe** (unsubscribes automatically — the best answer), `takeUntilDestroyed()`, `takeUntil(this.destroy$)`, or a manual `unsubscribe()` in `ngOnDestroy`. **HttpClient observables complete after one emission**, so those self-unsubscribe — but `valueChanges`, router events and `interval` never complete.

7. HttpClient — features & benefits

"`HttpClient` is Angular's HTTP API from `@angular/common/http` — a wrapper over `XMLHttpRequest` that returns Observables. It replaced the old `Http` service in Angular 4.3."

The features — this is the answer they're scoring:

Feature	Why it matters
Returns Observables	Cancellable, retryable, composable with RxJS
Automatic JSON parsing	No <code>res.json()</code> step — and the body is typed
Typed responses	<code>http.get<User[]>(url)</code> gives you <code>Observable<User[]></code>
Interceptors	Auth, logging, errors, caching — centrally (see §4)

Feature	Why it matters
<code>HttpErrorResponse</code>	Structured errors with <code>status</code> , <code>statusText</code> , <code>error</code> , plus client-vs-server distinction
Request/response headers & params	Immutable <code>HttpHeaders</code> / <code>HttpParams</code> builders
Progress events	<code>reportProgress: true</code> + <code>observe: 'events'</code> → upload/download progress bars
<code>observe: 'response'</code>	Access the full response — status code, headers — not just the body
Testing	<code>HttpClientTestingModule</code> + <code>HttpTestingController</code> — flush fake responses, assert requests
XSRF/CSRF protection	Built in — reads the <code>XSRF-TOKEN</code> cookie, sends <code>X-XSRF-TOKEN</code>
<code>provideHttpClient(withFetch())</code>	Fetch backend option, needed for SSR

```

@Injectables({ providedIn: 'root' })
export class UserService {
  private http = inject(HttpClient);

  getUsers(page: number): Observable<User[]> {
    const params = new HttpParams().set('page', page).set('size', 20);
    return this.http.get<User[]>('/api/users', { params }).pipe(
      retry({ count: 2, delay: 500 }),
      catchError(this.handle)
    );
  }

  upload(file: File): Observable<HttpEvent<unknown>> {
    const form = new FormData();
    form.append('file', file);
    return this.http.post('/api/files', form, { reportProgress: true,
observe: 'events' });
  }

  private handle(err: HttpResponse) {
    const msg = err.error instanceof ProgressEvent
      ? 'Network error' // client-side / connection
failure
      : `Server ${err.status}: ${err.error?.message}`;
    return throwError(() => new Error(msg));
  }
}

```

▲ **The trap:** "I called `http.get()` but nothing happened." — Observables are **lazy**; without a `subscribe()` (or an `async` pipe) the request is **never sent**. Guaranteed follow-up, and it connects straight back to §6.

8. Callback and callback hell

"A callback is just a function passed to another function to be run later — the original way of handling async work in JavaScript. Callback hell is what happens when async steps depend on each other: each one nests inside the previous one, and you get a pyramid that's hard to read, hard to error-handle, and hard to change."

```
// ▲ callback hell – the "pyramid of doom"
getUser(id, (err, user) => {
  if (err) return handle(err);
  getOrders(user.id, (err, orders) => {
    if (err) return handle(err); // error handling
    repeated at EVERY level
    getInvoice(orders[0].id, (err, invoice) => {
      if (err) return handle(err);
      sendEmail(invoice, (err) => {
        if (err) return handle(err);
        console.log('done');
      });
    });
  });
});
```

Three real problems, not just "ugly": error handling is duplicated at every level; you cannot **return** a value out of the pyramid; and *inversion of control* — you hand your continuation to someone else's library and trust it to call you exactly once.

The three fixes, in historical order:

```
// 1. Promises – flat chain, ONE catch
getUser(id)
  .then(user => getOrders(user.id))
  .then(orders => getInvoice(orders[0].id))
  .then(invoice => sendEmail(invoice))
  .then(() => console.log('done'))
  .catch(handle); // one handler for the whole chain

// 2. async/await – reads like synchronous code (ES2017)
async function run(id) {
  try {
    const user    = await getUser(id);
    const orders  = await getOrders(user.id);
    const invoice = await getInvoice(orders[0].id);
    await sendEmail(invoice);
    console.log('done');
  } catch (e) { handle(e); }
}

// 3. RxJS – in Angular, the same thing with cancellation
this.getUser(id).pipe(
  switchMap(user => this.getOrders(user.id)),
  switchMap(orders => this.getInvoice(orders[0].id)),
  catchError(handle)
).subscribe();
```

◆ **Add the parallelism point** — it's what senior answers include: *"Sequential `await` s are only right when each step needs the previous one. For independent calls, `await Promise.all([a(), b()])` — or `forkJoin` in RxJS — so they run concurrently instead of adding up their latencies."*

9. Event bubbling vs event capturing

"DOM events travel in three phases: capturing from `window` down to the target, then the target phase, then bubbling back up from the target to `window`. By default `addEventListener` listens in the bubbling phase; pass `true` (or `{ capture: true }`) as the third argument to listen while capturing."



```
<div id="outer">
  <button id="inner">Click</button>
</div>
```

```
outer.addEventListener('click', () => console.log('outer bubble'));
outer.addEventListener('click', () => console.log('outer capture'), true);
inner.addEventListener('click', () => console.log('inner'));

// Clicking the button logs: outer capture → inner → outer bubble
```

The controls:

Method	Effect
<code>e.stopPropagation()</code>	Stops travelling further up (or down) the tree
<code>e.stopImmediatePropagation()</code>	Also stops other listeners on the same element
<code>e.preventDefault()</code>	Cancels the default browser action (form submit, link navigation) — does not stop propagation
<code>e.target</code>	The element that was actually clicked
<code>e.currentTarget</code>	The element whose listener is running (<code>this</code>)

Why it matters practically — event delegation:

```
// ✓ ONE listener for a 1000-row table, and it works for rows added later
document.getElementById('table').addEventListener('click', e => {
  const row = e.target.closest('tr');
  if (row) select(row.dataset.id);
});
```

▲ **The Angular tie-in they may ask for:** " (*click*) in an Angular template listens in the bubbling phase. For a click-outside directive I use

`@HostListener('document:click', ['$event'])` and check `contains(e.target)` . And in a nested clickable card I call `$event.stopPropagation()` on the inner button so the card's own handler doesn't also fire."

Note: **focus** and **blur** do not bubble (use `focusin` / `focusout`), and neither do `mouseenter` / `mouseleave` (use `mouseover` / `mouseout`).

10. ES5 vs ES6

"ES6 — ES2015 — was the biggest release in JavaScript's history, and everything since has been yearly and incremental. The headline changes are `let` / `const` , arrow functions, classes, template literals, destructuring, spread/rest, default parameters, Promises, modules and `Map` / `Set` ."

Area	ES5	ES6 (2015)
Variables	<code>var</code> — function-scoped, hoisted as <code>undefined</code>	<code>let</code> / <code>const</code> — block-scoped , TDZ
Functions	<code>function</code> expressions	Arrow functions , lexical <code>this</code>
Classes	Constructor functions + prototypes	<code>class</code> , <code>extends</code> , <code>super</code> , <code>static</code>
Strings	<code>'a' + b + 'c'</code>	Template literals <code>`a\${b}c`</code> + multiline
Objects/arrays	Index access	Destructuring <code>const { a, b } = obj</code>
Args	<code>arguments</code> , <code>apply</code>	Spread/rest <code>...args</code> , default params
Async	Callbacks	Promises
Modules	IIFE / CommonJS / globals	<code>import</code> / <code>export</code> (native ES modules)

Area	ES5	ES6 (2015)
Collections	Objects & arrays only	<code>Map</code> , <code>Set</code> , <code>WeakMap</code> , <code>WeakSet</code>
Iteration	<code>for</code> , <code>forEach</code>	<code>for...of</code> , iterators, generators
Object literals	verbose	Shorthand <code>{ x, y }</code> , computed keys <code>{ [k]: v }</code>

```
// the three var/let/const behaviours they test
console.log(a); var a = 1;      // undefined (hoisted, initialised to undefined)
console.log(b); let b = 1;     // ✗ ReferenceError – Temporal Dead Zone

for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 0 1 2
← per-iteration binding
```

▲ **const trap:** `const` prevents **reassignment**, not mutation. `const a = [1]; a.push(2);` is legal; `a = []` is not. Use `Object.freeze()` for shallow immutability.

Post-ES6 worth naming (shows you didn't stop in 2015): `async/await` (ES2017), optional chaining `?.` and nullish coalescing `??` (ES2020), `Object.entries` / `Array.flat` (2017/2019), `Array.at()` (2022). Full drills in [19 – JS Variables Made Easy](#).

11. Arrow functions

"Arrow functions are a shorter function syntax, but the real difference isn't the syntax — it's that they don't have their own `this`. They inherit `this` lexically from the enclosing scope at definition time, which is why they solved the entire `var self = this` / `.bind(this)` problem."

```
const add = (a, b) => a + b;           // implicit return
const square = x => x * x;             // single param, no parens
needed
const make = () => ({ id: 1 });        // returning an object literal
needs parentheses
const noop = () => {};
```

Differences from a regular function — the checklist:

	Regular function	Arrow function
<code>this</code>	Dynamic — depends on how it's called	Lexical — from the enclosing scope, fixed
<code>arguments</code> object	✓	✗ — use rest (<code>...args</code>)
Can be a constructor (<code>new</code>)	✓	✗ <code>TypeError</code>
<code>prototype</code> property	✓	✗
Hoisted (declarations)	✓	✗ (it's an expression)
Usable as an object method	✓	▲ <code>this</code> won't be the object
Usable as a generator	✓	✗ no <code>yield</code>

The `this` demonstration — the actual interview question:

```
const timer = {
  seconds: 0,
  startBad() {
    setInterval(function () { this.seconds++; }, 1000); // ✗ this =
    window/undefined
  },
  startGood() {
    setInterval(() => { this.seconds++; }, 1000); // ✓ this =
    timer
  }
};

// ▲ but the reverse trap — arrows are WRONG for object methods:
const counter = {
  count: 0,
  inc: () => { this.count++; } // ✗ `this` is the enclosing (module/
  window) scope, not counter
};
```

The Angular angle: "Arrow functions are why callbacks in Angular components work without binding — inside `subscribe(res => this.data = res)`, `this` is still the component. Using `function(res) { this.data = res }` there is a classic bug."

12. OOP in TypeScript

"TypeScript adds real OO structure on top of JavaScript's prototypal model — classes with typed members, access modifiers, interfaces, abstract classes, generics. All four pillars are expressible."

```
// — ENCAPSULATION —————
class Account {
  private balance = 0; // compile-time private
  #pin = '0000'; // JS-native private field –
  enforced at RUNTIME
  protected readonly id: string; // visible to subclasses,
  never reassignable
  public owner: string;

  // parameter properties – declares AND assigns in one line
  constructor(id: string, owner: string) { this.id = id; this.owner =
owner; }

  get available(): number { return this.balance; } // getter
  deposit(amount: number): void {
    if (amount <= 0) throw new Error('invalid'); // the
invariant lives with the data
    this.balance += amount;
  }
}

// — ABSTRACTION —————
abstract class Shape {
  abstract area(): number; // must be implemented
  describe(): string { return `area = ${this.area()}`; } // shared
}

// — INHERITANCE —————
class Circle extends Shape {
  constructor(private r: number) { super(); }
  area(): number { return Math.PI * this.r ** 2; }
}

// — POLYMORPHISM —————
const shapes: Shape[] = [new Circle(2), new Square(3)];
shapes.forEach(s => console.log(s.area())); // each runs its own
implementation

// — INTERFACES: contracts, and structural typing —————
interface Persistable { save(): Promise<void>; }
class Doc implements Persistable { async save() {} }
```

Points that lift this answer:

- **private vs #private**: `private` is erased at compile time — the field is still reachable from plain JS. `#field` is genuinely private at runtime. Say this and you've clearly used both.
- **Structural typing ("duck typing")**: TypeScript matches on **shape**, not name. An object literal with the right members satisfies an interface without `implements`. This is the biggest difference from Java/C#.
- **A class can implement many interfaces but extend one class** — same rule as Java (see 33).
- **No method overloading with real bodies** — you declare overload signatures and write **one** implementation that handles all of them.
- **Angular ties in**: components and services *are* classes; DI is constructor injection; `implements OnInit` is an interface contract; `@Input` / `@Output` are decorators — which is metaprogramming, another OO tool.

13. Modules — how, and what types

Answer **both** meanings, because "modules" is ambiguous between JS and Angular — and asking which they mean is itself a good move.

(a) JavaScript / TypeScript modules

"Any file with a top-level `import` or `export` is a module — it gets its own scope instead of leaking into the global namespace."

```
// named exports – many per file
export interface User { id: number; }
export const MAX = 10;
export function load() {}

// default export – one per file
export default class ApiClient {}

// importing
import ApiClient, { User, MAX as LIMIT } from './api';
import * as Api from './api'; // namespace import
const mod = await import('./heavy'); // dynamic import → code
splitting
```

Module system	Syntax	Where
ES Modules (ESM)	<code>import</code> / <code>export</code>	The standard — browsers and modern Node
CommonJS	<code>require</code> / <code>module.exports</code>	Classic Node.js
AMD	<code>define()</code>	Legacy browser (RequireJS)
UMD	Wrapper for all of them	Legacy libraries

ESM vs CommonJS — the difference to state: ESM is **static** (imports resolved at parse time), which enables **tree-shaking** — the bundler drops unused exports. CommonJS is dynamic (`require` can be conditional), so it can't be tree-shaken reliably.

Named vs default: prefer **named** exports — better autocomplete, safe renaming, and no arbitrary renaming at each import site.

(b) Angular modules

" *NgModule* was Angular's compilation and DI container: *declarations* for components/directives/pipes, *imports* for other modules, *providers* for services, *exports* for what you make public, and *bootstrap* in the root module."

Type	Purpose
Root (<code>AppModule</code>)	Bootstraps the app
Feature	Groups one domain area (<code>OrdersModule</code>)
Shared	Common components/pipes/directives re-exported to features
Core	Singleton services, imported once by the root
Routing	<code>RouterModule.forRoot()</code> / <code>.forChild()</code> per feature
Lazy-loaded	Loaded on navigation → smaller initial bundle

```
// lazy loading – the practical benefit
{ path: 'orders', loadChildren: () => import('./orders/
orders.module').then(m => m.OrdersModule) }
```

◆ **Say the modern part — it shows you're current:** "From Angular 14/15 onwards, standalone components removed the need for `NgModules`. A component declares its own `imports` array, routes use `loadComponent`, and providers are configured functionally with `provideHttpClient` / `provideRouter` in `bootstrapApplication`. New apps default to standalone; `NgModules` are still supported for existing codebases, and my earlier projects use them."

```
@Component({
  standalone: true,
  imports: [CommonModule, RouterLink, MatButtonModule], // per-
  component, explicit
  template: `...`
})
export class OrderListComponent {}
```

▲ **`forRoot()` vs `forChild()`:** `forRoot()` registers the singleton providers and must be called **once** in the root; a lazy module calling `forRoot()` creates a **second instance** of those services — the classic "why do I have two copies of my service?" bug.

14. `unknown` vs `never` (vs `any`, `void`)

They asked about `unknown` and `never` specifically — a deliberate depth check. Answer with the whole set; it's a short, high-scoring answer.

`unknown` — the type-safe `any`

"`unknown` means 'a value whose type I don't know yet'. Anything is assignable to `unknown`, but `unknown` is assignable to nothing without first narrowing it — so the compiler forces you to check before you use it. It's the safe counterpart of `any`, which switches type checking off entirely."

```
let a: any = 'hello';
a.toFixed(); // ✓ compiles –
▲ crashes at runtime. any = no checking

let u: unknown = 'hello';
u.toFixed(); // ✗ compile error: 'u' is of type
'unknown'
if (typeof u === 'number') u.toFixed(); // ✓ narrowed → allowed

// where it belongs: values crossing a trust boundary
function parse(json: string): unknown { return JSON.parse(json); } //
JSON.parse returns `any` – bad
try { /* ... */ } catch (e: unknown) { //
  catch is unknown under strict
    if (e instanceof HttpResponse) console.log(e.status);
}
```

`never` — the type with no values

"`never` is the type of something that never produces a value — a function that always throws or never returns, or a branch the compiler has proved is unreachable. It's the empty set, so it's assignable to every type and nothing is assignable to it."

```

function fail(msg: string): never { throw new Error(msg); } //
never returns
function loop(): never { while (true) {} }

type A = string & number; // never – impossible intersection
type B = Exclude<'a' | 'b', 'a' | 'b'>; // never

// ★ the practical use – EXHAUSTIVENESS CHECKING
type Status = 'draft' | 'sent' | 'paid';

function label(s: Status): string {
  switch (s) {
    case 'draft': return 'Draft';
    case 'sent': return 'Sent';
    case 'paid': return 'Paid';
    default:
      const exhaustive: never = s; // ✓ compiles today
      return exhaustive; // add 'void' to Status → this
line FAILS TO COMPILE
  }
}

```

◆ **This example is the answer.** "Adding a new value to the union breaks the build at exactly the places that need updating, instead of silently falling through at runtime. That's *never* earning its keep."

The full comparison

Type	Meaning	Assign anything to it?	Assign it to anything?	Use it for
any	Turn off checking	✓	✓	Escape hatch — avoid; <code>noImplicitAny</code> flags it
unknown	Unknown-but-typed	✓	✗ (narrow first)	JSON, <code>catch</code> , third-party boundaries

Type	Meaning	Assign anything to it?	Assign it to anything?	Use it for
<code>never</code>	No value possible	<code>×</code>	<code>✓</code> (to everything)	Throwing functions, exhaustiveness checks
<code>void</code>	Returns nothing useful	only <code>undefined</code> / <code>null</code>	<code>×</code>	Function return type
<code>null</code> / <code>undefined</code>	Absence	<code>—</code>	<code>—</code>	With <code>strictNullChecks</code> , must be in the union

▲ **void vs never**: a `void` function **returns** — it just gives you nothing. A `never` function **never returns at all** (throws or loops forever). `() => void` completes; `() => never` doesn't.

15. HTML & CSS

The round closed here — general, and probably a quick check rather than depth. Have these five ready (02 — HTML & CSS has the full set):

Semantic HTML: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>` — they describe *meaning*, not appearance. Benefits: accessibility (screen readers navigate by landmark), SEO, and readable markup. `<div>` is what you use when nothing semantic fits.

The box model: content → padding → border → margin. `box-sizing: border-box` makes `width` include padding and border, which is why almost every project sets `*, *::before, *::after { box-sizing: border-box; }`.

Positioning: `static` (default) · `relative` (offset from its normal spot, still occupies space) · `absolute` (removed from flow, positioned against the nearest

positioned ancestor) · `fixed` (against the viewport) · `sticky` (relative until it hits a threshold, then fixed).

Flexbox vs Grid: Flexbox is **one-dimensional** — a row or a column (navbars, toolbars, centring). Grid is **two-dimensional** — rows and columns together (page layouts, card grids). They compose: a Grid page with Flexbox inside each cell.

Specificity: inline (1000) > id (100) > class/attribute/pseudo-class (10) > element/pseudo-element (1). `!important` overrides everything and is a smell. Equal specificity → the later rule wins.

Responsive: `@media (max-width: 768px)`, relative units (`rem`, `%`, `vw`, `clamp()`), `<meta name="viewport" content="width=device-width, initial-scale=1">`, mobile-first (write the small layout, then add `min-width` queries).

■ After this round — what to do

1. **Add these 15 to the recall log** ✓ (done — see [26 — Companies](#)).
 2. **The repeats are now undeniable:** Observables vs Promises (4 rounds), interceptors (3), state management/NgRx (3), lifecycle hooks (4). These are not "possible" questions — they are **guaranteed**. They should be word-perfect.
 3. **If Tech Mahindra calls back**, expect depth on 3 of these, not breadth. Most likely drilled: **NgRx effects with a real flow, switchMap vs mergeMap**, and **a live component**. Prepare [21 — NgRx](#) and [20 — RxJS](#).
 4. **The pattern from your log holds:** you clear the breadth sweep and lose the second question. For each of the 15 above, rehearse the answer **plus one trade-off or failure mode** — that second sentence is the whole difference.
-

Related files: [04 — Angular](#) · [20 — RxJS](#) · [21 — NgRx](#) · [03 — TypeScript](#) · [01 — JavaScript](#) · [19 — JS Variables](#) · [02 — HTML & CSS](#) · [26 — Companies log](#)

● TOMORROW — DELOITTE · Fullstack Java with Angular · Tue 25 Aug 2026, 3:00–4:00 PM IST

Company: Deloitte USI

Role: USI-EH27 – Consulting Services – AI&E – EaaS – SWE – Software Engineer II – Fullstack Java with Angular

Format: Virtual · 60 minutes · Software Engineer II (mid-level: ~3–6 yrs)

Decoding the requisition — don't say any of this out loud, just use it:

- **USI = Deloitte U.S.–India**, the India delivery arm serving Deloitte US engagements. Expect a **structured, rubric-scored** round — the interviewer works down a checklist across Java, Spring, Angular, SQL and communication, and scores each. **Breadth beats one deep dive**: a decent answer on ten topics scores higher than a brilliant answer on three and blanks elsewhere. Never go silent — always give the part you do know.

- **Consulting Services / EaaS (Engineering-as-a-Service)** = you'd be **staffed onto client projects**, sometimes client-facing. So they screen hard for **client-readiness**: clear structured communication, explaining technical work to someone outside your team, and flexibility across the stack. **Say once, explicitly: "I'm comfortable moving between frontend and backend depending on what the project needs."** That sentence is written for this role.

- **AI&E = Deloitte's "AI & Engineering" service line**. Your **RoboGebra AI explanation engine is directly on-theme** — a Spring Boot service that builds prompts, calls an AI model and parses structured output. **Lead your intro with it**. Most candidates for a Java+Angular role have nothing AI-adjacent; you do. This is your single biggest differentiator tomorrow.

- **Software Engineer II** = they expect you to code well and hold opinions, **not** to architect systems. Fundamentals + one solid project story wins it. Don't over-reach into system design you haven't done.

What Deloitte's process usually looks like

Stage	What it is
Round 1 — Technical (this one)	60 min. Java + Spring + Angular fundamentals, some SQL, a project walkthrough. Sometimes light live coding.
Round 2 — Technical / Project deep-dive	Deeper on your projects, design choices, "why did you do it that way", occasionally a hands-on task.
Round 3 — Managerial / Behavioural	STAR questions: conflict, deadline pressure, learning something new, working with a client. Deloitte weights this genuinely — it isn't a formality.
HR	Notice period, compensation, location/hybrid, documents.

◆ **So tomorrow's job is to survive the checklist and be easy to talk to.** Deloitte rejects far more people for vague communication than for a missing framework detail. Answer in **structure** — *"There are three ways to do this... I'd use the second one, because..."* — and you'll sound exactly like the person they're hiring.

Your plan — tonight and tomorrow morning

Do not open new topics tomorrow. Everything below is revision of what's already in this pack. New material the morning of an interview only creates doubt.

TONIGHT (24 Aug) — 90 minutes max, then sleep

Time	Do	File
30 min	Say your self-intro out loud 5 times. Not read — <i>said</i> , timed to 60–75 seconds. Record it once on your phone and listen back.	00
30 min	RoboGebra walkthrough out loud — problem → your role → stack → one hard thing you solved → the outcome. 2 minutes flat.	10, 18
20 min	Skim the Most-repeated questions table. Just read it.	26
10 min	Set up the room: test the meeting link, camera, mic, lighting, charger, backup mobile hotspot. Keep water and a notepad on the desk.	—
	Sleep by 11. A tired brain fails exactly where you've been failing — the second follow-up question.	

TOMORROW MORNING (25 Aug) — the concentrated block

Slot	Topic	What exactly	File
07:00–08:00	● Java core — the guaranteed block	HashMap internals (say the 90-second answer aloud 3×), <code>equals</code> / <code>hashCode</code> contract, ArrayList vs LinkedList, <code>String</code> immutability & the pool, <code>==</code> vs <code>equals</code>	31 Part 0 + Parts 1–2 · 05
08:00–08:30	Break — eat properly	Not at your desk.	
08:30–09:15	● OOP + interface vs abstract	The 60-second answer, the table, "when would you use which", the diamond rule	33 Parts 0, 1, 4
09:15–10:00	● Streams + Java 8	<code>map</code> / <code>filter</code> / <code>collect</code> , <code>groupingBy</code> , <code>Optional</code> , functional interfaces. Type 3 problems, don't read them.	22

Slot	Topic	What exactly	File
10:00–10:45	● Spring Boot	IoC/DI, @Component vs @Service vs @Repository , annotation-to-REST mapping, @Transactional , exception handling with @ControllerAdvice , auto-configuration in one sentence	06
10:45–11:00	Break		
11:00–11:45	● Angular — the four that always come	Lifecycle hooks (asked in 4 rounds), component communication (4), Observables vs Promises (4), services & DI	04 · 34 §6
11:45–12:30	● RxJS + interceptors + forms	switchMap vs mergeMap vs concatMap , async pipe & unsubscribe, HTTP interceptor + benefits, reactive vs template-driven	20 · 25 · 34 §4
12:30–13:15	Lunch — properly, away from the screen		

Slot	Topic	What exactly	File
13:15–13:45	● SQL + microservices	Joins, GROUP BY vs HAVING , indexing, one window function; then REST vs microservices, service discovery, one resilience pattern	36 Parts 0, 4, 6, 24 · 08
13:45–14:15	● Multithreading — 30 min, high yield	Thread vs Runnable, start() vs run() , synchronized vs volatile , deadlock + how you'd prevent and detect it	32 Parts 0, 4, 5, 9
14:15–14:40	◆ Dry run	Say your intro, your project story, and 5 rapid-fire answers out loud. Then stop revising .	
14:40–14:55	Log in early	Join 5 minutes before. Camera on. Notepad, water, resume printout, JD open in a tab.	
15:00			

The rule for the last 20 minutes: no new information. Confidence beats one extra fact. Every round you've lost was lost on delivery of the *second* answer, not on a missing fact.

◆ The 25 questions most likely to come (from YOUR own log)

These are ranked by how often they've actually been asked across your six logged rounds. **If you can answer these 25 fluently, you have covered most of a 60-minute round.**

Java (expect 6–8)

1. **How does HashMap work internally?** → the 5 beats: `Node[] table` · `hash()` spread + `(n-1) & hash` · bucket = **linked list of Nodes** · tree at 8 (`table ≥ 64`) · **resize at 0.75**. 31
2. **`equals()` and `hashCode()` contract** — **what breaks if you override only one?** → equal objects land in different buckets → `get` returns null, duplicates in a `Set`.
3. **`ArrayList` vs `LinkedList`** → array-backed, $O(1)$ random access, $O(n)$ middle insert vs node-based, $O(1)$ insert *if you already hold the node*, $O(n)$ traversal to reach it. In practice `ArrayList` wins almost always (cache locality).
4. **`String` immutability / `String` pool / `==` vs `equals`** → `new String("a") != "a"`; `.intern()`; why immutability makes `String` a safe map key and thread-safe.
5. **`Interface` vs `abstract class`** → contract vs shared code; and Java 8 default methods. 33
6. **`Java 8` features / `Streams`** → lambdas, functional interfaces, `Stream` API, `Optional`, default methods, `java.time`. Then a live one: group employees by department, find the second-highest salary. 22
7. **`final` vs `finally` vs `finalize` · **checked vs unchecked exceptions** · **overloading vs overriding**.**
8. **Multithreading:** `synchronized` vs `volatile`; what is a deadlock and how do you prevent it (**lock ordering**). 32

Spring Boot (expect 4–6)

1. **What is Spring Boot / how is it different from Spring?** → auto-configuration, starters, embedded server, no XML, production-ready actuator.
2. **IoC and Dependency Injection** → the container owns object creation; **constructor injection** is preferred (immutable, testable, catches circular deps at startup).
3. **@Component** vs **@Service** vs **@Repository** vs **@Controller** → all stereotypes; **@Repository** adds exception translation; the others are semantic.
4. **How do you build a REST API?** → **@RestController**, **@GetMapping** / **@PostMapping**, **@PathVariable** vs **@RequestParam** vs **@RequestBody**, **ResponseEntity** with proper status codes.
5. **Exception handling** → **@ControllerAdvice** + **@ExceptionHandler** returning a consistent error DTO. Say *"one place, consistent shape, correct status codes."*
6. **@Transactional** → the three traps: **self-invocation** doesn't work (proxy), only **unchecked** exceptions roll back by default (**rollbackFor** for checked), and it needs a public method.
7. **Is a Spring singleton the same as a Java singleton?** → No: one per **ApplicationContext**, not per JVM. And it's only thread-safe if it's **stateless**.

Angular (expect 6–8)

1. **Lifecycle hooks** (your #1 repeat — asked in 4 rounds) → order: **ngOnChanges** → **ngOnInit** → **ngDoCheck** → **ngAfterContentInit/Checked** → **ngAfterViewInit/Checked** → **ngOnDestroy**. Say *what you actually use each for*.
2. **Component communication** (4 rounds) → **@Input** / **@Output**, a shared service with **BehaviorSubject**, **@ViewChild**, and the store for app-wide.
3. **Observables vs Promises** (4 rounds) → single/eager/uncancellable vs stream/lazy/cancellable/operators. Close with the typeahead **switchMap** example. 34 §6

4. **Services & DI** / `providedIn: 'root'` → tree-shakable singleton; provider scope; the lazy-module `forRoot()` double-instance trap.
5. **Reactive vs template-driven forms** → reactive: defined in the class, testable, dynamic, scales; TD: simple, `ngModel`, small forms. Plus `setValue` vs `patchValue`.
6. **RxJS operators** → `map`, `filter`, `switchMap` (cancel previous — search), `mergeMap` (parallel), `concatMap` (ordered), `debounceTime`, `catchError`, `forkJoin`.
7. **Change detection** → default checks the whole tree; `OnPush` only on input reference change / event / async pipe. Mention immutability as the enabler.
8. **Unsubscribing / memory leaks** → `async` pipe first, then `takeUntilDestroyed()` / `takeUntil`.
9. **State management / NgRx** → the four-level ladder; NgRx = store/actions/reducers/selectors/effects. Your honesty position: **NgRx = EasyVisa**. 21

Cross-cutting (expect 2–4)

1. **SQL**: a join + `GROUP BY ... HAVING` question, and "how would you speed up a slow query?" 36 → index the filtered/joined columns, check the execution plan, avoid `SELECT *`, watch the N+1.

Microservices: why split, how services talk (REST/Feign/messaging), service discovery, one resilience pattern (circuit breaker), and **one honest sentence about the scale you actually worked at**.

■ Your opening — the first 3 minutes decide the tone

"Tell me about yourself" — 60–75 seconds, in this order:

1. **Now**: "I'm a full-stack developer with 5 years, working primarily with Angular on the frontend and Java with Spring Boot on the backend."

2. **The AI&E hook (say this — it's the role's own theme):** *"My most recent work is RoboGebra, an AI-powered math explanation engine — a Spring Boot service that builds prompts, calls an AI model, parses the step-by-step explanation and serves it to an Angular and Ionic frontend, with bilingual output."*
3. **Range:** *"Before that I worked on EasyVisa, a document portal with reactive forms and NgRx, and Subsea's schedule manager. So I've moved between frontend-heavy and backend-heavy work depending on what the project needed."*
4. **Close on fit:** *"This role is full-stack Java with Angular on client engagements under Deloitte's AI&E group, which is exactly the mix I've been doing — and the AI side is where I want to keep growing. That's why I'm interested."*

▲ **Do not** start with your college, your hometown, or a chronological life story. Start with what you do now.

When they ask about a project, use this shape every time — four beats, two minutes:

Context (what the product does, who uses it) → **Your role** (what *you* built, first person singular) → **One hard problem** and how you solved it → **Outcome** (a number if you have one; an honest qualitative result if you don't).

The three habits that decide this round

Your own log says you clear the first question and lose the follow-up. Fix that with three mechanical habits:

1. Every answer gets a second sentence

Definition → one concrete thing from your project → one trade-off or failure mode.

"An interceptor sits in the HTTP pipeline and lets you handle cross-cutting concerns centrally. In EasyVisa I used one to attach the JWT and redirect to login on a 401. The catch is that ordering matters — request order is registration order, response order is reversed — and it only sees traffic that goes through `HttpClient`."

That is what **"4+ years"** sounds like. One sentence sounds like documentation.

2. Say "I don't know" cleanly, then bridge

"I haven't used Kafka in production. I know it's a distributed log with topics and partitions and that consumers track offsets, and the closest thing I've built is an async notification flow with `@Async` and a bounded thread pool. I'd pick it up quickly, but I'd rather be accurate than guess."

That answer **scores**. Bluffing loses the whole round because the interviewer stops trusting everything else you said.

3. Own the scale honestly

Don't inflate numbers. *"Our traffic was modest — a few hundred concurrent users — so I haven't tuned for very high load, but here's how I'd approach it..."* is a **strong** answer for Software Engineer II. Consulting firms screen hard for candidates who over-claim, because they'll be in front of a client.

The Deloitte behavioural layer — 10 minutes of this round is not technical

Even in a technical round, a Deloitte interviewer scores **communication and fit**. Have short **STAR** answers ready (Situation → Task → Action → Result), 60–90 seconds each. Full versions in **11 — HR & Behavioural**.

Likely question	Your material
"Tell me about a challenging problem you solved."	The hardest technical thing in RoboGebra — parsing inconsistent AI output into reliable step-by-step explanations, and how you made it deterministic
"A time you had a disagreement with a teammate."	Pick a technical disagreement resolved with data, not a personality clash. End with what you learned
"A tight deadline — how did you handle it?"	Scope negotiation: what you cut, what you protected, how you communicated it early
"Something new you had to learn fast."	Ionic/Capacitor, or MongoDB — say how you learned it, not just that you did
"Why Deloitte?"	Consulting exposure across clients and domains, the AI&E service line matching where you want to grow (your last project was an AI engine), and the scale/structure to learn from senior engineers
"Where do you see yourself in 3 years?"	Deeper full-stack ownership, moving toward leading a module and mentoring — with AI-integrated engineering as the direction
"Why are you leaving your current role?"	Forward-looking only: scope, scale, technology. Never criticise your current employer

▲ **The client-facing tell they listen for:** when you explain RoboGebra, can you describe it **without jargon first** and then add the technical detail? Practise the one-sentence version: *"It's a learning app that explains maths problems step by step in the student's own language, using an AI model behind a Spring Boot service."* Say that, then go technical. That's the consulting communication pattern.

■ Questions to ask them (have 3 ready — never say "no questions")

- 1. "What does the delivery model look like — would I be staffed on one client project, or moving across engagements?"
 - 2. "How is the work split between frontend and backend on the current team — is there a preference for where I'd start?"
 - 3. "Given this is under AI&E, is there AI/ML-adjacent work on the roadmap for this team? My last project was an AI explanation engine, so that's an area I'd like to grow in."
 - 4. "What does the first 90 days look like for a Software Engineer II here?"
 - 5. "What are the next steps and the expected timeline?" ← **always close with this one.**
-

One-page cheat sheet — read at 14:45, nothing else

Topic	The line
HashMap	<code>Node[] table · h ^ (h>>>16) · (n-1) & hash</code> · bucket = linked list of Nodes · tree at 8 if table ≥ 64 · resize at 0.75 (16→12 entries)
equals/hashCode	Equal ⇒ same hash. Override one only → entry unreachable, duplicates in a Set
ArrayList vs LinkedList	Array + O(1) index vs nodes + O(1) insert-at-node; ArrayList wins in practice
String	Immutable, pooled, <code>==</code> compares references, safe as a map key

Topic	The line
Interface vs abstract	Contract/can-do/many vs shared state+code/ is-a/one. Java 8: <code>default + static</code> ; Java 9: <code>private</code>
<code>synchronized</code> vs <code>volatile</code>	Lock = atomicity+visibility+ordering; volatile = visibility+ordering only, doesn't fix <code>i++</code>
Deadlock	4 conditions; prevent with consistent lock ordering or <code>tryLock</code> timeout; detect with <code>jstack</code>
DI	Constructor injection — immutable, testable, circular deps caught at startup
<code>@Transactional</code>	Self-invocation fails · only unchecked rolls back by default · must be public
Spring singleton	Per ApplicationContext, not per JVM; safe only if stateless
REST codes	200 · 201 created · 204 no content · 400 bad request · 401 unauth · 403 forbidden · 404 · 409 conflict · 500
Lifecycle hooks	<code>ngOnChanges</code> → <code>ngOnInit</code> → <code>ngDoCheck</code> → AfterContent → AfterView → <code>ngOnDestroy</code>
Observable vs Promise	Stream/lazy/cancellable/operators vs single/eager/not cancellable
<code>switchMap</code> vs <code>mergeMap</code>	<code>switchMap</code> cancels the previous (search); <code>mergeMap</code> runs all in parallel (independent writes)
Unsubscribe	<code>async</code> pipe > <code>takeUntilDestroyed()</code> > <code>takeUntil(destroy\$)</code>
OnPush	Checks only on input reference change, events, <code>async</code> pipe → needs immutable updates

Topic	The line
Interceptor	Auth · errors · loading · logging · retry. Clone the request (immutable), order matters, <code>multi: true</code>
N+1	Detect in the SQL log → fix with <code>JOIN FETCH</code> / <code>@EntityGraph</code> / batch size
Slow query	Execution plan → index the filter/join columns → avoid <code>SELECT *</code> → check for N+1
Your honesty lines	NgRx = EasyVisa · MongoDB = ODM not ORM · .NET = Subsea Web API · scale = modest, and that's fine

✓ Final checklist — tonight

- [] Meeting link tested, camera + mic checked **in that app**
- [] Laptop charger plugged in · **mobile hotspot ready as backup**
- [] Resume printed or open in a tab — they *will* ask about something on it
- [] The JD open in a tab (the requisition text above)
- [] Quiet room booked, door closed, phone on silent
- [] Water, notepad, pen
- [] Intro rehearsed **out loud** 5 times
- [] RoboGebra story rehearsed out loud, 2 minutes
- [] 3 questions to ask them, written on the notepad
- [] Asleep by 11 PM

After the round tomorrow — while it's still fresh, the same evening: write every question they asked into **26 — Companies log**. That file is the reason you now know Observables vs Promises has come up four times. Six rounds of data is already an asset; seven is better.

**You've got this. Breadth is not your problem — you reach second rounds.
Tomorrow, just add the second sentence to every answer.**

Related: [00 Intro](#) · [11 HR & Behavioural](#) · [05 Java](#) · [06 Spring Boot](#) · [04 Angular](#) · [31 HashMap](#) · [32 Multithreading](#) · [33 Interface vs Abstract](#) · [26 Companies log](#)

SQL – The Complete Interview File

Why this file exists. SQL appeared for the first time at **Mphasis L1** (a **GROUP BY** with a join) and it is now on the checklist for **every** round — Deloitte's rubric has a SQL row, and every Java/Spring JD you've applied to lists MySQL or Postgres. This file takes the two questions you were actually asked and builds outward until nothing in a normal SQL round is new.

■ Companion sections you already have: **30 – Mphasis L2 §G** (the compact version) · **26 – Companies §8** (the exact question Mphasis asked) · **24 – ORM/JPA** (the N+1 and the SQL Hibernate generates).

Table of contents

Part	What's in it
0	● The two questions you were actually asked — answered to mid-level depth
1	The schema + sample data used everywhere below (memorise these 3 tables)
2	● Logical execution order — the single idea that explains WHERE vs HAVING , aliases, and half the traps
3	Joins in full — with row-by-row worked output, plus §3.6 join cardinality: the 1,1,1,1 vs 1,1,1,1 row-count puzzle
4	COUNT(*) vs COUNT(col) vs COUNT(DISTINCT) — the join+count trap
5	GROUP BY , HAVING , aggregates and their NULL behaviour
6	● The salary questions — Nth highest, 5 ways; per-department; above-average; top-3 per dept

Part	What's in it
7	● Window functions — RANK / DENSE_RANK / ROW_NUMBER , LAG / LEAD , running totals, frames
8	Subqueries — scalar, correlated, IN vs EXISTS vs JOIN , and the NOT IN NULL bomb
9	Duplicates — find them, delete them, prevent them
10	● NULLs and three-valued logic — where most candidates get caught
11	UNION vs UNION ALL vs INTERSECT vs EXCEPT
12	CTEs and recursive CTEs — the employee hierarchy question
13	Self joins — employee/manager, consecutive rows
14	Dates, strings, CASE , and pivoting without PIVOT
15	● Indexes and "make this query faster" — the follow-up that always comes
16	Transactions, ACID, isolation levels, locking, deadlock
17	Normalization, keys, constraints
18	DDL vs DML, DELETE vs TRUNCATE vs DROP
19	Views, stored procedures, functions, triggers
20	● SQL from the Java/Spring side — JPQL, N+1, pagination, SQL injection
21	● Predict the output — 23 SQL programs with real data

Real-world idea: an **unmarked exam paper**. You cannot say it scored more than 50, and you cannot say it scored 50 or less. Both answers are *unknown*, so it's excluded from either list.

That ONE idea explains:

NULL = NULL is NOT true → use IS NULL ★

NOT IN with a NULL returns ZERO rows ▲ ★

"salary > 50000 OR salary <= 50000" MISSES the NULL row ★

COUNT(col) skips NULLs but COUNT(*) doesn't ★

3 A JOIN MULTIPLIES rows ★

A join doesn't "add columns" – it produces every MATCHING PAIR. ★

4 matching rows × 4 matching rows = 16 rows ★

Real-world idea: **a wedding seating plan**. Four groomsmen and four bridesmaids don't make eight — they make **sixteen possible pairs**.

That ONE idea explains:

the $1,1,1,1 \times 1,1,1,1 = 16$ puzzle (§3.6) ★

why SUM() double-counts after a join ▲ ★

why DISTINCT after a join is a SMELL, not a fix ★

why COUNT(*) ≠ COUNT(b.id) on a LEFT JOIN ★

Easy memory — the three master ideas

1. EXECUTION ORDER ★ FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

(the kitchen, not the menu)

2. NULL = UNKNOWN ★ (an unmarked exam paper) → NOT IN ▲ , IS NULL, COUNT

3. JOINS MULTIPLY ★ (a wedding seating plan) → $4 \times 4 = 16$, SUM double-counts

Part 0 — The two questions you were actually asked

● Q0.1 — Mphasis L1: "Total employees and average salary per department"

```
SELECT  d.dept_id,
        d.dept_name,
        COUNT(e.emp_id) AS total_employees,
        COALESCE(ROUND(AVG(e.salary), 2), 0) AS average_salary
FROM    dept d
LEFT JOIN employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY d.dept_name;
```

The three sentences that make this a mid-level answer. Do not just write the query — narrate these while you type:

① **LEFT JOIN**, not **INNER JOIN**.

"With an inner join a department with zero employees disappears from the result entirely. The question says 'per department', and zero is a valid answer — so I drive from **dept** with a left join."

② **COUNT(e.emp_id)**, not **COUNT(*)**.

"This is the subtle one. With a **LEFT JOIN** an empty department still produces one row, with all the employee columns NULL. **COUNT(*)** counts that row and returns 1, which is wrong. **COUNT(column)** skips NULLs and correctly returns 0."

③ **Every non-aggregated column must be in the GROUP BY**.

"**dept_name** is selected but not aggregated, so it has to be in **GROUP BY**. MySQL 5.6 was lenient; Postgres, SQL Server and MySQL 5.7+ under **ONLY_FULL_GROUP_BY** reject it."

And the NULL average. `AVG` also ignores NULLs, so an empty department returns `NULL` , not `0` . `COALESCE(..., 0)` is the fix — mention it even if they don't ask.

The four follow-ups they ask next — have all four ready

They ask	You answer
"Only departments with more than 5 employees"	<code>HAVING COUNT(e.emp_id) > 5</code> — not <code>WHERE</code> . " <code>WHERE</code> filters rows before grouping; <code>HAVING</code> filters groups after aggregation. An aggregate doesn't exist yet at <code>WHERE</code> time."
"Only permanent employees, but still show every department"	The condition goes in the <code>ON</code> , not the <code>WHERE</code> : <code>LEFT JOIN employee e ON e.dept_id = d.dept_id AND e.emp_type = 'PERM'</code> . "Putting it in <code>WHERE</code> turns the left join back into an inner join, because a <code>NULL</code> row fails the predicate." ● This is the single most-asked LEFT JOIN follow-up in the industry.
"Departments with no employees at all"	<code>... LEFT JOIN ... WHERE e.emp_id IS NULL</code> — the anti-join.
"Sort by headcount descending"	<code>ORDER BY total_employees DESC</code> — " <code>ORDER BY</code> runs after <code>SELECT</code> , so it can use the alias. <code>WHERE</code> and <code>GROUP BY</code> can't."

● Q0.2 — The classic: "Second-highest salary"

They will ask this in some form. **The answer they want is `DENSE_RANK` , and the sentence they're listening for is "what about ties".**

```
-- ✓ The answer to give first
SELECT DISTINCT salary
FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employee
) t
WHERE rnk = 2;
```

"I'd use **DENSE_RANK** rather than **ROW_NUMBER** or **RANK**, because if two people share the top salary, **ROW_NUMBER** would call the second of them the 'second highest' — which is wrong, it's the same salary. **DENSE_RANK** gives both of them rank 1 and the genuinely-second salary rank 2."

Full five-way comparison, including the versions for a MySQL 5.7 box with no window functions, is in **Part 6**. Learn at least two: the **DENSE_RANK** one and the **LIMIT 1 OFFSET 1** one.

◆ How to answer *any* SQL question — the 4-step script

Interviewers score **method**, not just the final query. Do these four out loud, in order:

1. **Restate the schema and confirm.** "So **employee** has **emp_id**, **salary**, **dept_id**, and **dept** has **dept_id**, **dept_name** — and **dept_id** can be NULL for an unassigned employee?" **Asking about NULLs and duplicates before you write scores immediately.**
2. **Say the shape before the syntax.** "This is an aggregate per group, so it's a join, a **GROUP BY** on department, and a **HAVING** if we filter on the count."
3. **Write it, narrating the non-obvious choices** — the left join, the **COUNT(column)**, the alias.
4. **Volunteer the edge case.** "One thing to watch — if a department has no employees, the average comes back NULL rather than 0." This one sentence is what separates 3/5 from 5/5 on a rubric.

✗ **Never go silent and never say "I'd have to look it up".** Say the shape even if you're unsure of a function name: *"I'd use a window function here — **RANK** or **DENSE_RANK** partitioned by department — let me write it and we can check the exact syntax."*

Part 1 — The schema used in this whole file

Every query below runs against these three tables. **Memorise them** — if the interviewer doesn't give you a schema, invent this one out loud and they will accept it.

```
CREATE TABLE dept (  
    dept_id    INT PRIMARY KEY,  
    dept_name  VARCHAR(50) NOT NULL,  
    location   VARCHAR(50)  
);  
  
CREATE TABLE employee (  
    emp_id     INT PRIMARY KEY,  
    emp_name   VARCHAR(50) NOT NULL,  
    salary     DECIMAL(10,2),  
    hire_date  DATE,  
    dept_id    INT,                -- nullable: an unassigned employee  
    manager_id INT,                -- nullable: the CEO has no manager  
    CONSTRAINT fk_dept    FOREIGN KEY (dept_id)    REFERENCES dept(dept_id),  
    CONSTRAINT fk_manager FOREIGN KEY (manager_id) REFERENCES  
employee(emp_id)  
);  
  
CREATE TABLE orders (  
    order_id    INT PRIMARY KEY,  
    emp_id      INT,                -- the salesperson  
    order_date  DATE,  
    amount      DECIMAL(10,2),  
    status      VARCHAR(20)  
);
```

The sample data (used by every "predict the output" question in Part 21)

dept

dept_id	dept_name	location
10	Engineering	Chennai
20	Sales	Bangalore
30	HR	Chennai
40	Legal	Pune

employee

emp_id	emp_name	salary	hire_date	dept_id	manager_id
1	Asha	90000	2019-01-15	10	NULL
2	Bala	75000	2020-03-01	10	1
3	Chitra	75000	2020-07-20	10	1
4	Dev	60000	2021-02-10	20	1
5	Esha	50000	2022-05-05	20	4
6	Farid	NULL	2023-01-09	30	1
7	Gita	45000	2023-06-30	NULL	1

- **Note the three deliberate landmines**, because interview data always has them:
 - **Dept 40 (Legal) has zero employees** → shows up only with a **LEFT JOIN** .
 - **Farid's salary is NULL** → breaks **COUNT** , **AVG** and **NOT IN** .
 - **Gita's dept_id is NULL** → she disappears from any inner join to **dept** .
 - Plus **Bala and Chitra tie at 75,000** → this is what separates **RANK** from **DENSE_RANK** from **ROW_NUMBER** .

Part 2 — Logical query execution order — the idea that explains half the answers

● If you learn one thing from this file, learn this. SQL is *written* in one order and *executed* in another. Most SQL interview traps are just this table.

WRITTEN ORDER		LOGICAL EXECUTION ORDER
SELECT	(1st)	1. FROM + JOIN ... ON
FROM	(2nd)	2. WHERE
JOIN	(3rd)	3. GROUP BY
WHERE	(4th)	4. HAVING
GROUP BY	(5th)	5. SELECT (aliases born ↓)
HAVING	(6th)	6. DISTINCT
ORDER BY	(7th)	7. ORDER BY
LIMIT	(8th)	8. LIMIT / OFFSET

The five consequences — every one of them is an interview question

Consequence	Why	Example
WHERE cannot use an aggregate	WHERE (2) runs before GROUP BY (3), so COUNT(*) doesn't exist yet	WHERE COUNT(*) > 5 ✗ → HAVING COUNT(*) > 5 ✓
WHERE cannot use a SELECT alias	Aliases are created at step 5	WHERE total > 5 where total is an alias ✗ (MySQL bends this in HAVING / ORDER BY , Postgres doesn't in WHERE either)
ORDER BY can use a SELECT alias	ORDER BY (7) runs after SELECT (5)	ORDER BY total_employees DESC ✓
A WHERE on the right table kills a LEFT JOIN	The join (1) produces NULL rows, then WHERE (2) deletes them	Move the condition into ON

Join	Returns	Mental picture
<code>FULL [OUTER] JOIN</code>	Everything from both sides	A U B — not in MySQL , emulate with <code>LEFT UNION RIGHT</code>
<code>CROSS JOIN</code>	Cartesian product — every row x every row	7 x 4 = 28 rows
<code>SELF JOIN</code>	A table joined to itself (not a keyword — just an alias)	employee → manager

```
-- FULL OUTER JOIN emulated in MySQL
SELECT d.dept_name, e.emp_name FROM dept d LEFT JOIN employee e ON
e.dept_id = d.dept_id
UNION
SELECT d.dept_name, e.emp_name FROM dept d RIGHT JOIN employee e ON
e.dept_id = d.dept_id;
```

Easy memory — the six joins

INNER → only MATCHING rows in both ★ the overlap

LEFT → ALL of the left + matches ★ (the most-used)

RIGHT → ALL of the right + matches

FULL OUTER → everything from both sides

CROSS → every combination (n x m) ▲

SELF → a table joined to itself (employee → manager) ★

LEFT

●●●○○○

keep all L + matches

INNER

○○●●○○

keep only the OVERLAP ★

RIGHT

○○●●●●

keep all R + matches

▲ A JOIN MULTIPLIES rows — 4 matches x 4 matches = 16 ★ (§3.6)

3.2 Worked output on the real data — this is what they want you to be able to predict

```
SELECT d.dept_name, e.emp_name FROM dept d INNER JOIN employee e ON
e.dept_id = d.dept_id;
```

→ **6 rows.** Engineering×3, Sales×2, HR×1. **Legal is gone** (no employees) and **Gita is gone** (NULL dept_id).

```
SELECT d.dept_name, e.emp_name FROM dept d LEFT JOIN employee e ON
e.dept_id = d.dept_id;
```

→ **7 rows.** The 6 above **plus** ('Legal', NULL). Gita is *still* gone — she has no dept, and we drove from **dept**.

```
SELECT d.dept_name, e.emp_name FROM dept d RIGHT JOIN employee e ON
e.dept_id = d.dept_id;
```

→ **7 rows.** The 6 above **plus** (NULL, 'Gita'). Legal is gone.

```
SELECT COUNT(*) FROM dept CROSS JOIN employee;
```

→ **28.** 4×7 .

● **The trap question: "Does a LEFT JOIN always return at least as many rows as the left table?"**

Yes — at least. It can return *more*: if one left row matches three right rows, you get three output rows. "A left join guarantees every left row appears at least once, not exactly once." That row-multiplication is exactly why **SUM** after a join can double-count.

3.3 ● The **ON** vs **WHERE** question — asked constantly

```
-- ✗ Silently becomes an INNER JOIN
SELECT d.dept_name, e.emp_name
FROM dept d LEFT JOIN employee e ON e.dept_id = d.dept_id
WHERE e.salary > 50000;

-- ✓ Keeps every department
SELECT d.dept_name, e.emp_name
FROM dept d LEFT JOIN employee e ON e.dept_id = d.dept_id AND e.salary >
50000;
```

"In an outer join, a predicate in **ON** decides which rows match; a predicate in **WHERE** filters the result after the join. Because unmatched left rows carry NULLs, and **NULL > 50000** is unknown, a **WHERE** on the right-hand table deletes exactly the rows the left join was there to preserve. For an inner join it makes no difference — **ON** and **WHERE** are equivalent."

3.4 The anti-join — "find rows in A with no match in B"

Three ways, and you should know which is fastest:

```
-- ① LEFT JOIN ... IS NULL      ✓ portable, usually fast
SELECT d.* FROM dept d
LEFT JOIN employee e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;

-- ② NOT EXISTS                  ✓ safest – NULL-proof, usually the
optimiser's favourite
SELECT d.* FROM dept d
WHERE NOT EXISTS (SELECT 1 FROM employee e WHERE e.dept_id = d.dept_id);

-- ③ NOT IN                      ▲ BREAKS if the subquery can return NULL –
see Part 10
SELECT d.* FROM dept d
WHERE d.dept_id NOT IN (SELECT dept_id FROM employee);  -- returns ZERO
rows here!
```

● Query ③ returns **nothing at all** on our data, because Gita's **dept_id** is NULL. **This is the most popular SQL gotcha in interviews** — full explanation in [Part 10](#).

3.5 Join algorithms — the senior-sounding follow-up

If they ask "how does the database actually do a join?":

Algorithm	When the optimiser picks it
Nested loop	One side is small, or there's an index on the join column of the inner table. $O(n \times m)$ without an index.

Algorithm	When the optimiser picks it
Hash join	Big unsorted sets, equality join. Builds a hash table on the smaller side. (MySQL 8.0.18+, Postgres always.)
Merge join	Both inputs already sorted on the join key — e.g. both sides have indexes.

*"You don't choose it, the optimiser does — but you influence it, mainly by indexing the join columns and keeping statistics fresh. **EXPLAIN** tells you which one it picked."*

3.6 ● Join cardinality — "Table A has 1,1,1,1 and Table B has 1,1,1,1 — how many rows?"

This is the single most common SQL puzzle in Indian service-company interviews. They give you two tiny tables with duplicate values and ask for the row count of each join. It looks like a trick; it's actually just multiplication — but you have to know the rule cold, because they'll fire five counts at you in ten seconds.

● The headline question

Table A	Table B
-----	-----
id	id
--	--
1	1
1	1
1	1
1	1

```
SELECT COUNT(*) FROM A INNER JOIN B ON A.id = B.id;    -- ?
SELECT COUNT(*) FROM A LEFT  JOIN B ON A.id = B.id;    -- ?
SELECT COUNT(*) FROM A RIGHT JOIN B ON A.id = B.id;    -- ?
SELECT COUNT(*) FROM A FULL  JOIN B ON A.id = B.id;    -- ?
SELECT COUNT(*) FROM A CROSS JOIN B;                  -- ?
```

✓ **The answer: 16 — every single one of them.**

Join	Rows	Why
INNER	16	Every one of A's 4 rows matches every one of B's 4 rows → 4 × 4
LEFT	16	A left join adds rows only for unmatched left rows. There are none — so it's identical to the inner join
RIGHT	16	Same reasoning from B's side
FULL OUTER	16	Nothing is unmatched on either side
CROSS	16	4 × 4 — and here it <i>coincidentally</i> equals the others because every row matches

■ **Say this, don't just say "16":**

"Sixteen for all of them. A join on equality pairs every matching left row with every matching right row, so with four 1s on each side that's four times four. And the outer joins only add rows for values that have no partner on the other side — here nothing is unmatched, so left, right and full all collapse to the same sixteen rows as the inner join."

● **The follow-up is guaranteed: "So when would LEFT and INNER differ?" →**

"The moment either side has a value the other doesn't. The left join preserves those rows with NULLs; the inner join drops them." Have Case ③ below ready as your example.

The formula — compute any of these in 10 seconds

Group both sides by the join key, then:

INNER = $\sum (\text{countA}(v) \times \text{countB}(v))$ for every value v present in BOTH

LEFT = INNER + (number of A rows whose key is not in B)

RIGHT = INNER + (number of B rows whose key is not in A)

FULL = INNER + unmatched-A rows + unmatched-B rows

CROSS = $|A| \times |B|$ (no ON clause, so nothing to match)

▲ **NULL keys always count as "unmatched"** — **NULL = NULL** is UNKNOWN, so a NULL never joins to anything, **not even to another NULL**. It contributes 0 to the inner join and 1 to its own side's outer join.

Six worked cases — cover the answers and do them

Case ① — the headline: **A = 1,1,1,1 · B = 1,1,1,1**

	INNER	LEFT	RIGHT	FULL	CROSS
	16	16	16	16	16

4×4 = 16 ; nothing unmatched anywhere.

Case ② — unequal duplicates: **A = 1,1,1,1 · B = 1,1,1**

	INNER	LEFT	RIGHT	FULL	CROSS
	12	12	12	12	12

4×3 = 12 . Still all identical — every row on both sides has a partner. **Note: LEFT returns 12, not 4.** "A left join guarantees each left row appears at least once, not exactly once."

Case ③ — ● the one that actually separates the joins: A = 1,1,2,2 · B = 1,1,3

Group first: A has 1×2, 2×2 . B has 1×2, 3×1 .

	Count	Working
INNER	4	only value 1 is in both: 2 × 2 = 4
LEFT	6	4 + the two unmatched A rows (2,2) → each gets one row with NULL B columns
RIGHT	5	4 + the one unmatched B row (3)
FULL	7	4 + 2 + 1
CROSS	12	4 × 3 — ignores the values entirely

This is the case to reach for when they ask you to explain the difference.

Case ④ — ● with NULLs: A = 1, 1, NULL · B = 1, NULL

	Count	Working
INNER	2	only value 1 matches: 2 × 1 = 2 . The two NULLs do not match each other
LEFT	3	2 + A's NULL row (unmatched)
RIGHT	3	2 + B's NULL row (unmatched)
FULL	4	2 + 1 + 1
CROSS	6	3 × 2 — a cross join has no ON , so NULLs are irrelevant

"The NULL on each side joins to nothing — not even to the NULL on the other side — because `NULL = NULL` is UNKNOWN, not true. So each NULL row survives only in the outer join on its own side."

◆ In MySQL you can force NULLs to match with the null-safe operator: `ON A.id <=> B.id` → then the inner join returns 3. In Postgres: `ON A.id IS NOT DISTINCT FROM B.id`. Mention it only if they push.

Case ⑤ — ● the empty-table trap: `A = 1, 2, 3` · `B = (no rows)`

	Count
INNER	0
LEFT	3 — every A row, with all B columns NULL
RIGHT	0
FULL	3
CROSS	0 ▲ — <code>3 × 0 = 0</code> , not 3

The cross-join-with-an-empty-table answer catches a lot of people. Anything times zero is zero.

Case ⑥ — duplicates on both sides, mixed: `A = 1,1,2` · `B = 1,1,2,2`

`INNER = (2×2) + (1×2) = 4 + 2 = 6`. Everything matches on both sides, so `LEFT = RIGHT = FULL = 6`, and `CROSS = 3 × 4 = 12`.

● 3.6.1 The killer follow-up: `COUNT(*)` vs `COUNT(b.id)` on these joins

Go back to **Case ③** (`A = 1,1,2,2` · `B = 1,1,3`) with a `LEFT JOIN`:


```

SELECT COUNT(*)      FROM A LEFT JOIN B ON A.id = B.id;    -- 6
SELECT COUNT(b.id)   FROM A LEFT JOIN B ON A.id = B.id;    -- 4 ← the NULL
placeholder rows aren't counted
SELECT COUNT(a.id)   FROM A LEFT JOIN B ON A.id = B.id;    -- 6
SELECT COUNT(DISTINCT a.id) FROM A LEFT JOIN B ON A.id = B.id; -- 2
(values 1 and 2)

```

This is the same trap as the department-headcount question in [Part 0](#) —

`COUNT(*)` counts the placeholder row that a left join manufactures for an unmatched left row; `COUNT(right_column)` doesn't. If you can connect these two questions out loud, you've shown you understand the concept rather than memorised a number.

● 3.6.2 Why this matters in real code — the `SUM` double-count

This puzzle is not academic. It is the number-one cause of wrong numbers in reports:

```

-- ✗ Asha's salary is counted once PER ORDER
SELECT e.emp_name, SUM(e.salary) AS total_salary, COUNT(o.order_id) AS
orders
FROM   employee e
JOIN   orders o ON o.emp_id = e.emp_id
GROUP BY e.emp_id, e.emp_name;

```

If Asha has 3 orders, the join produces 3 rows for her, so `SUM(e.salary)` returns **3 × 90000 = 270000**. Her salary didn't triple — the join multiplied her row.

The fixes:

```
-- ① Aggregate the many-side FIRST, then join to the one-side
SELECT e.emp_name, e.salary, COALESCE(o.order_count, 0) AS orders
FROM   employee e
LEFT JOIN (SELECT emp_id, COUNT(*) AS order_count FROM orders GROUP BY
emp_id) o
        ON o.emp_id = e.emp_id;

-- ② Or use a correlated scalar subquery per row
SELECT e.emp_name, e.salary,
       (SELECT COUNT(*) FROM orders o WHERE o.emp_id = e.emp_id) AS orders
FROM   employee e;

-- ③ If you must join first, de-duplicate the one-side aggregate
SELECT e.emp_name, MAX(e.salary) AS salary, COUNT(o.order_id) AS orders
FROM   employee e LEFT JOIN orders o ON o.emp_id = e.emp_id
GROUP BY e.emp_id, e.emp_name;
```

■ *"The moment I aggregate across two joins to different child tables — say orders and payments — the counts multiply against each other. The safe pattern is to aggregate each child separately in a subquery and then join those single-row-per-parent results together."* **Saying this unprompted reads as production experience, because that's exactly where the bug shows up.**

3.6.3 The related counting questions they pair with it

"Three tables — A has 2 rows of 1, B has 3 rows of 1, C has 2 rows of 1. A JOIN B JOIN C on the id?"

→ **12.** $2 \times 3 \times 2$. "Joins compose — the multiplication just keeps going, which is why an unfiltered three-way join on a low-cardinality column explodes."

"A table has 4 rows all in the same department. SELECT COUNT(*) FROM employee e1 JOIN employee e2 ON e1.dept_id = e2.dept_id?"

→ **16.** Same 4×4 — a self join is not special.

→ **With AND e1.emp_id < e2.emp_id : 6** — that's $C(4,2)$, each unordered pair once, no self-pairs. (This is the "pairs of employees in the same department" question from [Part 13](#).)

→ **With AND e1.emp_id <> e2.emp_id : 12** — each pair twice, in both orders.

"Does **A LEFT JOIN B** ever return fewer rows than A?"
→ **Never.** At least **|A|** , possibly many more. *"That's the guarantee — every left row appears at least once."*

"Is **A LEFT JOIN B** the same as **B RIGHT JOIN A** ?"
→ **Yes, identical results** (column order aside). *"I write left joins exclusively — reading a query is easier when the driving table is always the first one. Right joins mostly exist for symmetry."*

"**SELECT DISTINCT** on Case ① — how many?"
→ **1 row.** All 16 output rows are **(1, 1)** . *"Which is exactly why **DISTINCT** after a join is a smell — it hides the fan-out instead of preventing it."*

▲ 3.6.4 The 20-second recipe to say out loud

"Count how many of each value are on each side. For the values in both, multiply and add up — that's the inner join. Then a left join adds one row for each left row whose value isn't on the right, a right join does the mirror image, and a full outer adds both. A cross join ignores the values completely and is just left count times right count. And NULL never matches anything, including another NULL, so a NULL row is always 'unmatched'."

Part 4 — **COUNT** — the question behind the question

● **This is the highest-yield 60 seconds in the whole file**, because "join + count" is the shape of the question you were actually asked.

Expression	Counts	On our data
COUNT (*)	Rows, NULLs included	SELECT COUNT(*) FROM employee → 7

Expression	Counts	On our data
<code>COUNT(col)</code>	Rows where <code>col</code> IS NOT NULL	<code>COUNT(salary)</code> → 6 (Farid's is NULL)
<code>COUNT(DISTINCT col)</code>	Distinct non-NULL values	<code>COUNT(DISTINCT salary)</code> → 5 (75000 counted once, NULL ignored)
<code>COUNT(1)</code>	Identical to <code>COUNT(*)</code>	7 — " <code>COUNT(1)</code> vs <code>COUNT(*)</code> is a myth; every modern optimiser treats them the same. I write <code>COUNT(*)</code> ."

```
-- The complete demonstration – run this mentally, it's a classic whiteboard question
SELECT COUNT(*)           AS rows_total,      -- 7
       COUNT(salary)      AS with_salary,     -- 6
       COUNT(DISTINCT salary) AS distinct_salary, -- 5
       COUNT(dept_id)     AS with_dept,       -- 6
       SUM(salary)        AS total_salary,    -- 395000
       AVG(salary)        AS avg_salary       -- 65833.33 ←
395000/6, NOT /7
FROM employee;
```

● The killer follow-up: "Why is `AVG(salary)` not `SUM(salary)/COUNT(*)` ?"

"Because `AVG` ignores NULLs — it divides by `COUNT(salary)` , which is 6, not by `COUNT(*)` which is 7. If you want NULL treated as zero you have to say so: `SUM(salary)/COUNT(*)` or `AVG(COALESCE(salary,0))` ."

The `LEFT JOIN + COUNT(*)` trap, drawn out

```
SELECT d.dept_name, COUNT(*) AS wrong, COUNT(e.emp_id) AS correct
FROM dept d LEFT JOIN employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

dept_name	wrong (<code>COUNT(*)</code>)	correct (<code>COUNT(e.emp_id)</code>)
Engineering	3	3

dept_name	wrong (COUNT(*))	correct (COUNT(e.emp_id))
Sales	2	2
HR	1	1
Legal	1 ✗	0 ✓

"Legal has no employees, but the left join still emits one row for it with every employee column NULL. `COUNT(*)` counts rows, so it says 1. `COUNT(e.emp_id)` counts non-NULL values, so it correctly says 0."

Part 5 — GROUP BY, HAVING and aggregates

5.1 The rule

Every column in the **SELECT** list must either be inside an aggregate function, or listed in the **GROUP BY**.

```
-- ✗ Rejected by Postgres / SQL Server / MySQL with ONLY_FULL_GROUP_BY
-- (the default since 5.7)
SELECT dept_id, emp_name, COUNT(*) FROM employee GROUP BY dept_id;
--          ^^^^^^^ which employee's name? The engine can't know.
```

"Old MySQL would silently pick an arbitrary row's `emp_name`, which is a genuine source of production bugs. Since 5.7 `ONLY_FULL_GROUP_BY` is on by default and it errors, matching the standard."

The exception worth knowing: Postgres allows selecting columns that are **functionally dependent on a grouped primary key** — `GROUP BY d.dept_id` lets you select `d.dept_name` because `dept_id` is the PK. Grouping by both is still the safe habit.

5.2 WHERE vs HAVING — the table to recite

	WHERE	HAVING
Runs	Before grouping	After grouping
Operates on	Individual rows	Groups
Aggregates allowed	✗ No	✓ Yes
Can use SELECT alias	✗ No	▲ MySQL yes, standard/ Postgres no
Uses indexes	✓ Yes	✗ No (data is already aggregated)
Performance	Filter here when you can	Only for aggregate conditions

```
-- Both, correctly used: only 2020+ hires, only departments where they
average > 60k
SELECT d.dept_name, AVG(e.salary) AS avg_sal
FROM   dept d JOIN employee e ON e.dept_id = d.dept_id
WHERE  e.hire_date >= '2020-01-01'      -- row filter, before grouping,
index-friendly
GROUP BY d.dept_id, d.dept_name
HAVING AVG(e.salary) > 60000           -- group filter, after aggregation
ORDER BY avg_sal DESC;
```

● **Performance point worth making unprompted:** "I push every filter I can into *WHERE* rather than *HAVING*, because *WHERE* reduces the rows before they're aggregated and can use an index. *HAVING* only makes sense for conditions on the aggregate itself."

Easy memory — WHERE vs HAVING

WHERE → filters ROWS → BEFORE grouping → cannot see COUNT() ★

HAVING → filters GROUPS → AFTER grouping → CAN see COUNT() ★

WHERE salary > 50000 ✓ (a row-level fact)

WHERE COUNT(*) > 5 ✗ COUNT doesn't exist yet ★

HAVING COUNT(*) > 5 ✓ ★

You can use BOTH:

WHERE (drop rows) → GROUP BY → HAVING (drop groups) ★

Real-world idea: **sorting exam papers**. **WHERE** throws out individual papers *before* you stack them by class; **HAVING** throws out whole classes *after* you've counted each stack.

5.3 Aggregate functions and their NULL behaviour

Function	NULLs	Empty input returns	Note
COUNT(*)	counted	0	The only aggregate that never returns NULL
COUNT(col)	skipped	0	
SUM	skipped	NULL ▲	Not 0! Wrap in COALESCE(SUM(x), 0)
AVG	skipped	NULL	Divides by the non-NULL count
MIN / MAX	skipped	NULL	Work on strings and dates too
GROUP_CONCAT (MySQL) / STRING_AGG (Postgres)	skipped	NULL	GROUP_CONCAT(emp_name ORDER BY salary DESC SEPARATOR ', ')

5.4 Bonus: **GROUP BY** with **ROLLUP** — subtotals in one query

```
SELECT d.dept_name, COUNT(*) AS headcount
FROM dept d JOIN employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_name WITH ROLLUP;    -- MySQL; Postgres: GROUP BY
ROLLUP(d.dept_name)
```

Adds a final row with `dept_name = NULL` holding the **grand total**. Mention it if they ask "how would you add a total row" — most candidates reach for a **UNION**.

Part 6 — The salary questions — every variant

● **The single most-asked family of SQL interview questions.** Learn the shapes, not the memorised text.

6.1 Highest salary — the warm-up

```
SELECT MAX(salary) FROM employee;    --
90000

-- "...and who earns it?" MAX alone can't tell you – you need one of these:
SELECT * FROM employee WHERE salary = (SELECT MAX(salary) FROM employee);
-- handles ties ✓
SELECT * FROM employee ORDER BY salary DESC LIMIT 1;
-- drops ties ▲
```

"***ORDER BY ... LIMIT 1** is fine unless two people tie for the top — then it arbitrarily returns one of them. The subquery version returns both, which is usually what's meant.*"

6.2 ● Nth-highest salary — five ways, with the trade-offs

Take **N = 2**. On our data the distinct salaries descending are `90000, 75000, 60000, 50000, 45000` → **the answer is 75000**.


```

-- ① DENSE_RANK ✓ the answer to lead with (MySQL 8+, Postgres, SQL
Server)
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM
employee
) t WHERE rnk = 2;

-- ② LIMIT/OFFSET on distinct salaries ✓ works on MySQL 5.7, no window
functions needed
SELECT DISTINCT salary FROM employee ORDER BY salary DESC LIMIT 1 OFFSET
1;
-- Postgres/standard: ... OFFSET 1 FETCH NEXT 1 ROWS ONLY;

-- ③ Correlated subquery – "count how many distinct salaries are above
me" ✓ portable, slow
SELECT DISTINCT e.salary FROM employee e
WHERE 1 = (SELECT COUNT(DISTINCT e2.salary) FROM employee e2 WHERE
e2.salary > e.salary);
-- generalise: N-1 = (...)

-- ④ MAX of the ones below the MAX ✓ neat for N=2 only
SELECT MAX(salary) FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);

-- ⑤ NOT IN / correlated NOT EXISTS – mention but don't lead with it
SELECT MAX(salary) FROM employee
WHERE salary NOT IN (SELECT MAX(salary) FROM employee);

```

● Which do you say when?

Situation	Use
Modern DB, they want the <i>best</i> answer	① DENSE_RANK
They say " <i>without window functions</i> " or "MySQL 5.7"	② LIMIT 1 OFFSET 1 on DISTINCT salary
They say " <i>without LIMIT</i> " (SQL Server / Oracle flavour)	③ correlated COUNT(DISTINCT)
They want it parameterised for any N	① with WHERE rnk = :n

▲ **The trap in ②:** **SELECT salary FROM employee ORDER BY salary DESC LIMIT 1 OFFSET 1 without DISTINCT** returns 75000 here by luck — but if Asha

and a second person both earned 90000, it would return **90000** and be wrong.

Always say **DISTINCT in the **LIMIT/OFFSET** version and explain why.**

▲ **The other trap:** if there is no Nth salary (N=99), ①②③ return **no rows**, while ④⑤ return a single row containing **NULL**. If they ask "what if there is no second-highest?", that's the answer — and **LeetCode 176** specifically wants **NULL**, which is why the wrapped-subquery form **SELECT (SELECT DISTINCT salary ... LIMIT 1 OFFSET 1) AS second** is used there: a scalar subquery with no rows evaluates to **NULL**.

6.3 Highest-paid employee per department — the natural escalation

```
-- ✓ Window function – handles ties, one pass
SELECT dept_name, emp_name, salary FROM (
    SELECT d.dept_name, e.emp_name, e.salary,
           RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS
rnk
    FROM employee e JOIN dept d ON d.dept_id = e.dept_id
) t
WHERE rnk = 1;
```

→ Engineering **Asha 90000**, Sales **Dev 60000**, HR **Farid** (*the only row in its partition, so it ranks 1 even with a NULL salary*). **Gita is absent** — her **dept_id** is **NULL**, so the inner join drops her.

▲ **Where do NULLs sort?** It differs by engine and it's a legitimate follow-up: **MySQL** treats **NULL** as lowest (first in **ASC**, last in **DESC**); **Postgres/Oracle** treat it as highest (last in **ASC**, first in **DESC**) and let you say **ORDER BY salary DESC NULLS LAST**. "If **NULLs** matter I make it explicit rather than relying on the default."

```
-- ✓ The pre-window-function version – correlated subquery
SELECT e.emp_name, e.salary, e.dept_id
FROM   employee e
WHERE  e.salary = (SELECT MAX(e2.salary) FROM employee e2 WHERE e2.dept_id
= e.dept_id);

-- ✓ Or the join-to-derived-table version (usually the fastest of the
three)
SELECT e.emp_name, e.salary, e.dept_id
FROM   employee e
JOIN   (SELECT dept_id, MAX(salary) AS max_sal FROM employee GROUP BY
dept_id) m
      ON m.dept_id = e.dept_id AND m.max_sal = e.salary;
```

● **RANK vs ROW_NUMBER here matters.** **ROW_NUMBER() = 1** returns **exactly one** employee per department even when two tie for the top. **RANK() = 1** returns **both**. Say which you chose and why: *"I used **RANK** so tied top earners both appear; if the requirement is exactly one row per department I'd switch to **ROW_NUMBER** with a deterministic tiebreak like **ORDER BY salary DESC, emp_id**."*

6.4 Top 3 per department

```
SELECT dept_name, emp_name, salary FROM (
  SELECT d.dept_name, e.emp_name, e.salary,
         DENSE_RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC)
  AS rnk
  FROM employee e JOIN dept d ON d.dept_id = e.dept_id
) t
WHERE rnk <= 3
ORDER BY dept_name, salary DESC;
```

*"**DENSE_RANK** <= 3 means 'the top three salary levels', which can be more than three people if there are ties. **ROW_NUMBER** <= 3 means 'exactly three rows'. I'd ask which they want."* **Asking that question is the answer they're scoring.**

6.5 Employees earning more than their department's average

```
-- ✓ Window function – single scan
SELECT emp_name, salary, dept_id FROM (
  SELECT e.*, AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg FROM
employee e
) t
WHERE salary > dept_avg;

-- ✓ Correlated subquery – the classic textbook answer
SELECT e.emp_name, e.salary
FROM   employee e
WHERE  e.salary > (SELECT AVG(e2.salary) FROM employee e2 WHERE e2.dept_id
= e.dept_id);
```

→ Engineering avg = 80000 → **Asha**. Sales avg = 55000 → **Dev**.

"The correlated version re-runs the inner query once per row — fine for thousands of rows, poor for millions. The window-function version computes each department's average once and is what I'd ship."

6.6 Department-level salary questions

```
-- Department with the highest total salary bill
SELECT d.dept_name, SUM(e.salary) AS total
FROM dept d JOIN employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY total DESC
LIMIT 1;

-- Departments where the average beats the company average
SELECT dept_id, AVG(salary) AS dept_avg
FROM employee
GROUP BY dept_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM employee);

-- Salary bands – a CASE + GROUP BY question they like
SELECT CASE WHEN salary >= 80000 THEN 'A: 80k+'
           WHEN salary >= 60000 THEN 'B: 60-80k'
           WHEN salary IS NULL THEN 'D: unknown'
           ELSE 'C: under 60k' END AS band,
       COUNT(*) AS headcount
FROM employee
GROUP BY band
CASE or use GROUP BY 1
ORDER BY band;
```

-- MySQL allows the alias; Postgres: repeat the

6.7 The "median salary" question — the hard one they sometimes end on

```
-- Postgres / Oracle: built in
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) FROM employee;

-- MySQL 8 – the portable window-function trick (works for even and odd counts)
SELECT AVG(salary) AS median FROM (
    SELECT salary,
           ROW_NUMBER() OVER (ORDER BY salary) AS rn,
           COUNT(*) OVER () AS cnt
    FROM employee WHERE salary IS NOT NULL
) t
WHERE rn IN (FLOOR((cnt+1)/2), CEIL((cnt+1)/2));
```

"There's no `MEDIAN()` in MySQL. The trick is to number the rows in order, then average the middle one — or the middle two when the count is even, which `FLOOR / CEIL` of $(n+1)/2$ gives you for free."

Part 7 — Window functions

The one-sentence definition to memorise: "A window function computes a value across a set of rows related to the current row, but unlike `GROUP BY` it doesn't collapse the rows — every input row still comes out, with the aggregate attached."

That contrast — *aggregate collapses, window preserves* — is the answer to "what's the difference between `GROUP BY` and a window function?"

7.1 Anatomy

```
FUNCTION(...) OVER (  
    PARTITION BY dept_id          -- optional: restart the calculation per  
group                             group  
    ORDER BY      salary DESC     -- optional: order within the partition  
    ROWS BETWEEN ... AND ...     -- optional: the frame (which rows are  
in the window)  
)
```

- **No `PARTITION BY`** → the whole result set is one window.
- **No `ORDER BY`** → the window is the entire partition (used for `AVG() OVER (PARTITION BY ...)`).
- **`ORDER BY` with no frame** → the default frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` — which is why `SUM(...) OVER (ORDER BY x)` gives a **running total**, not the grand total. ● *That default is itself an interview question.*

7.2 ● ROW_NUMBER vs RANK vs DENSE_RANK — the table to have burned in

```
SELECT emp_name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,
       RANK()      OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense,
       NTILE(2)    OVER (ORDER BY salary DESC) AS half
FROM employee WHERE salary IS NOT NULL;
```

emp_name	salary	ROW_NUMBE R	RANK	DENSE_RAN K	NTILE (2)
Asha	90000	1	1	1	1
Bala	75000	2	2	2	1
Chitra	75000	3	2	2	1
Dev	60000	4	4 ← gap	3 ← no gap	2
Esha	50000	5	5	4	2
Gita	45000	6	6	5	2

The one-liner:

" **ROW_NUMBER** is always unique and arbitrary between ties. **RANK** gives ties the same number and then skips — 1,2,2,4. **DENSE_RANK** gives ties the same number and doesn't skip — 1,2,2,3. For 'Nth highest salary' you want **DENSE_RANK**, because you're ranking salary levels, not people. "

Easy memory — the three ranking functions ★

```
Salaries: 100, 90, 90, 80

ROW_NUMBER() → 1, 2, 3, 4  ★ ALWAYS unique, ties broken arbitrarily
RANK()       → 1, 2, 2, 4  ★ ties share, then it SKIPS (like the Olympics )
DENSE_RANK() → 1, 2, 2, 3  ★ ties share, NO gap
```

Real-world idea: **an Olympic podium.**

Two athletes tie for silver

RANK → nobody gets 3rd; the next is 4th ★ (a real medal table)

DENSE_RANK → the next athlete IS 3rd ★

ROW_NUMBER → the tie is broken arbitrarily – someone is just "2nd" ★

"Nth HIGHEST SALARY, counting ties as one" → DENSE_RANK ★

"Top 3 rows per department, no duplicates" → ROW_NUMBER ★

★ That distinction is exactly what the "second-highest salary" question is testing.

7.3 The functions worth knowing, with a use for each

Function	Does	Real use
ROW_NUMBER()	1,2,3...	Deduplication (keep <code>rn = 1</code>), stable pagination
RANK() / DENSE_RANK()	ranking with ties	Nth highest, top-N per group
NTILE(n)	split into n buckets	Salary quartiles, percentile bands
LAG(col, 1) / LEAD(col, 1)	previous / next row's value	Month-over-month change, gaps between dates
SUM/AVG/COUNT/MIN/MAX ... OVER	aggregate without collapsing	Running total, % of department total
FIRST_VALUE / LAST_VALUE	first/last in the window	"compare each employee to the top earner"
NTH_VALUE(col, 2)	the nth row of the window	second-highest, inline

7.4 The four window queries they actually ask

```
-- ① Running total of orders by date
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
AS running_total
FROM orders;

-- ② Each employee's salary as a % of their department's total
SELECT emp_name, salary, dept_id,
       ROUND(100.0 * salary / SUM(salary) OVER (PARTITION BY dept_id), 1)
AS pct_of_dept
FROM employee;

-- ③ Month-over-month change (LAG)
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS
prev_month,
       revenue - LAG(revenue) OVER (ORDER BY month) AS
change,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
            / LAG(revenue) OVER (ORDER BY month), 2) AS
pct_change
FROM monthly_revenue;

-- ④ Deduplicate: keep the newest row per email
DELETE FROM users WHERE id IN (
    SELECT id FROM (
        SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY created_at
        DESC) AS rn
        FROM users
    ) t WHERE rn > 1
);
```

▲ MySQL's "you can't specify target table for update in FROM clause" error bites on ④ — the extra `SELECT ... FROM (...) t` wrapper is exactly the workaround. Mention it: *"MySQL won't let you read the same table you're deleting from in a subquery, so you wrap it in a derived table, which forces materialisation."*

7.5 ● The `LAST_VALUE` trap

```
-- ✗ Returns the CURRENT row, not the last one
SELECT emp_name, LAST_VALUE(salary) OVER (PARTITION BY dept_id ORDER BY
salary) FROM employee;

-- ✓ Give it the full frame
SELECT emp_name, LAST_VALUE(salary) OVER (
    PARTITION BY dept_id ORDER BY salary
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) FROM
employee;
```

"Because with an `ORDER BY` and no explicit frame the window ends at the current row, so 'last value' is the current value. `FIRST_VALUE` looks fine by accident. The fix is to state the frame."

7.6 Where window functions may not appear

● You cannot use a window function in `WHERE`, `GROUP BY` or `HAVING` — they're evaluated at the `SELECT` step, after all of those.

```
SELECT * FROM employee WHERE ROW_NUMBER() OVER (ORDER BY salary) = 2;
-- ✗ error
```

The fix is always the same: wrap it in a subquery or CTE and filter on the outside. *"That's exactly why every top-N query you see has a derived table around it."*

Part 8 — Subqueries: IN / EXISTS / JOIN

8.1 The three kinds

Kind	Returns	Example
Scalar	one value	WHERE salary > (SELECT AVG(salary) FROM employee)
Row / column (multi-row)	a list	WHERE dept_id IN (SELECT dept_id FROM dept WHERE location='Chennai')
Correlated	re-evaluated per outer row — references the outer query	WHERE EXISTS (SELECT 1 FROM orders o WHERE o.emp_id = e.emp_id)
Derived table / inline view	a table in FROM	FROM (SELECT ...) t

"What's a correlated subquery?" → "One that references a column from the outer query, so it can't be run once up-front — logically it re-executes for each outer row. That makes it powerful but potentially O(n) subquery executions; good optimisers rewrite many of them into joins."

8.2 IN vs EXISTS vs JOIN — the comparison they want

```
-- Employees who have placed at least one order
SELECT * FROM employee e WHERE e.emp_id IN      (SELECT o.emp_id FROM
orders o);                                     -- IN
SELECT * FROM employee e WHERE EXISTS (SELECT 1 FROM orders o WHERE
o.emp_id = e.emp_id);    -- EXISTS
SELECT DISTINCT e.* FROM employee e JOIN orders o ON o.emp_id =
e.emp_id;                -- JOIN
```

	IN	EXISTS	JOIN
Semantics	value ∈ list	at least one matching row	combine rows
Stops early	no	yes — short-circuits on the first match	n/a
Best when	the subquery result is small	the subquery is large / correlated	you need columns from both tables
NULL-safe	▲ NOT IN breaks on NULL	✓ NOT EXISTS is safe	✓
Duplicates	none introduced	none introduced	▲ multiplies rows — needs DISTINCT

"In modern MySQL 8 and Postgres the optimiser usually rewrites all three into the same plan, so I choose on readability and NULL-safety: **EXISTS** when I only need to test existence, a join when I need the other table's columns. **NOT IN** I avoid on any nullable column."

8.3 The **DISTINCT** -after-join smell

● If your answer needs **SELECT DISTINCT** after a join, **say why**: "The join multiplies rows when an employee has several orders, so I either **DISTINCT** it or switch to **EXISTS**, which doesn't multiply in the first place — **EXISTS** is the cleaner intent here." Interviewers notice this.

Part 9 — Duplicates: find them, delete them, prevent them

```
-- ① Find duplicate emails
SELECT email, COUNT(*) AS cnt
FROM   users
GROUP BY email
HAVING COUNT(*) > 1;

-- ② Show the full duplicate rows, not just the key
SELECT * FROM users
WHERE email IN (SELECT email FROM users GROUP BY email HAVING COUNT(*) > 1)
ORDER BY email;

-- ③ Delete duplicates, keeping the LOWEST id – the self-join classic
DELETE u1 FROM users u1
JOIN users u2 ON u1.email = u2.email AND u1.id > u2.id;

-- ④ Same thing with a window function – clearer, and lets you choose
"keep the newest"
DELETE FROM users WHERE id IN (
    SELECT id FROM (
        SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY created_at
        DESC) rn FROM users
    ) t WHERE rn > 1
);

-- ⑤ Prevent it happening again
ALTER TABLE users ADD CONSTRAINT uq_users_email UNIQUE (email);
```

● The follow-up: "how would you do it if there's no unique id?"

"Postgres gives you the hidden `ctid`; Oracle gives you `ROWID`. In MySQL with no PK I'd create a copy of the table with `SELECT DISTINCT`, or add a surrogate key first. And then I'd add the unique constraint so the question can't come back."

● "Duplicate on a combination of columns?" → `PARTITION BY first_name, last_name, dob`, or `GROUP BY` all three. Same shape.

Part 10 — NULLs and three-valued logic

● This is where most candidates get caught. SQL logic is **three-valued**: **TRUE**, **FALSE**, and **UNKNOWN**. NULL means "unknown", not "empty" and not "zero".

10.1 The rules

Expression	Result	Why
<code>NULL = NULL</code>	UNKNOWN (not TRUE)	Two unknowns aren't known to be equal
<code>NULL <> 5</code>	UNKNOWN	
<code>NULL + 10</code>	NULL	Any arithmetic with NULL is NULL
<code>'abc' NULL</code> (Postgres)	NULL	Concatenation too. MySQL's CONCAT also returns NULL; CONCAT_WS skips them
<code>WHERE x = NULL</code>	matches nothing	must use <code>IS NULL</code>
<code>NULL AND FALSE</code>	FALSE	← the one that surprises people
<code>NULL OR TRUE</code>	TRUE	
<code>NULL AND TRUE</code>	UNKNOWN	
<code>COUNT(*)</code>	counts NULL rows	the only aggregate that does
<code>GROUP BY</code>	all NULLs form one group	even though <code>NULL = NULL</code> is unknown
<code>ORDER BY</code>	NULLs cluster at one end	MySQL: first in <code>ASC</code> . Postgres: last in <code>ASC</code>
<code>UNIQUE</code> constraint	usually allows many NULLs	because they're not "equal" to each other
<code>DISTINCT</code>	treats all NULLs as one value	

Only rows where the **WHERE** clause evaluates to **TRUE** are returned. **UNKNOWN** is discarded exactly like **FALSE** — that one sentence explains most NULL surprises.

10.2 ● The **NOT IN** bomb — the most popular SQL gotcha in interviews

```
SELECT * FROM dept WHERE dept_id NOT IN (SELECT dept_id FROM employee);
```

On our data this returns **ZERO** rows — even though Legal (40) genuinely has no employees.

Why: the subquery returns **{10,10,10,20,20,30,NULL}** (Gita). **NOT IN** expands to

dept_id <> 10 AND dept_id <> 20 AND dept_id <> 30 AND dept_id <> NULL .

That last term is **UNKNOWN**, and **TRUE AND UNKNOWN = UNKNOWN** — so no row is ever TRUE.

Three fixes:

```
WHERE dept_id NOT IN (SELECT dept_id FROM employee WHERE dept_id IS NOT
NULL)    -- ① filter
WHERE NOT EXISTS (SELECT 1 FROM employee e WHERE e.dept_id =
dept.dept_id)    -- ② ✓ preferred
LEFT JOIN employee e ON e.dept_id = d.dept_id WHERE e.emp_id IS
NULL            -- ③ anti-join
```

*"**NOT EXISTS** is my default because it's NULL-safe by construction — it asks 'does a matching row exist', which is a two-valued question."*

Easy memory — the NULL bomb

NULL = UNKNOWN (an unmarked exam paper), not empty and not zero ★

NOT IN + a NULL in the subquery → ZERO rows, always ▲ ★
because x <> NULL is UNKNOWN, and TRUE AND UNKNOWN = UNKNOWN

FIX ★ NOT EXISTS — it asks "does a row exist?", a TWO-valued question
or filter: WHERE dept_id IS NOT NULL
or an anti-join: LEFT JOIN ... WHERE b.id IS NULL

NULL = NULL → NOT true ★ → use IS NULL
COUNT(*) counts rows | COUNT(col) SKIPS NULLs ★

10.3 The NULL-handling functions

Function	Engine	Does
COALESCE(a, b, c)	standard, all engines	first non-NULL argument. Use this one.
IFNULL(a, b)	MySQL	2-arg COALESCE
ISNULL(a, b)	SQL Server	2-arg COALESCE
NVL(a, b)	Oracle	2-arg COALESCE
NULLIF(a, b)	all	returns NULL if a = b — the divide-by-zero guard: x / NULLIF(y, 0)
IS DISTINCT FROM	Postgres	NULL-safe <>
<=>	MySQL	NULL-safe = — NULL <=> NULL is TRUE

10.4 The NULL question that catches everyone

"SELECT COUNT(*) FROM employee WHERE salary > 50000 OR salary <= 50000 — does that return 7?"

No — 6. The condition looks exhaustive, but **Farid's salary is NULL**, so both `NULL > 50000` and `NULL <= 50000` evaluate to UNKNOWN, `UNKNOWN OR UNKNOWN` is UNKNOWN, and the row is discarded. Everyone else passes one side or the other.

"A NULL row satisfies neither a condition nor its negation. If I want it included I have to say so explicitly — `OR salary IS NULL`. This is the reason a `WHERE status <> 'CLOSED'` filter silently drops every row where `status` is NULL, which is a real production bug I'd watch for."

Part 11 — Set operations

Operator	Does	Duplicates	Sorting cost
UNION	combine + remove duplicates	removed	▲ implicit sort/hash — expensive
UNION ALL	combine, keep everything	kept	✓ cheap — use this unless you need dedup
INTERSECT	rows in both	removed	(MySQL 8.0.31+, Postgres always)
EXCEPT / MINUS	in the first, not the second	removed	EXCEPT = standard/ Postgres, MINUS = Oracle

Rules: same number of columns, compatible types, column names come from the **first** query, and a single `ORDER BY` goes at the very **end** and applies to the whole result.

```
SELECT emp_name, 'current' AS src FROM employee
UNION ALL
SELECT emp_name, 'archived'          FROM employee_archive
ORDER BY emp_name;                  -- applies to the combined result
```

● **The interview line:** " *UNION* has to deduplicate, which means an implicit sort or hash over the whole result. If I know the sets are disjoint — or duplicates are fine — I use *UNION ALL* , and on large result sets that difference is significant."

UNION vs JOIN : "A join combines columns side by side; a union stacks rows on top of each other."

Part 12 — CTEs and recursive CTEs

12.1 Plain CTE — **WITH**

```
WITH dept_stats AS (  
    SELECT dept_id, COUNT(*) AS headcount, AVG(salary) AS avg_sal  
    FROM    employee  
    GROUP BY dept_id  
)  
SELECT d.dept_name, s.headcount, ROUND(s.avg_sal, 2) AS avg_sal  
FROM    dept_stats s  
JOIN     dept d ON d.dept_id = s.dept_id  
WHERE   s.headcount > 1;
```

Why use one? Readability, referencing the same intermediate result twice, and replacing deeply nested subqueries. **Multiple CTEs** chain with commas, and a later one can reference an earlier one.

"CTE vs subquery vs temp table vs view?"

- **Subquery** — inline, single use, can get unreadable when nested.
- **CTE** — named, reusable within the one statement, self-documenting. In

Postgres ≤ 11 a CTE was an **optimisation fence** (always materialised); from **12** it inlines unless you say **MATERIALIZED** . MySQL 8 may materialise or merge.

- **Temp table** — persists for the session, can be **indexed**, worth it when you reuse a big intermediate result across several statements.

- **View** — a stored query definition, permanent, no data of its own. A **materialized view** does store data and needs refreshing.

12.2 ● Recursive CTE — the employee hierarchy question

```
WITH RECURSIVE org_chart AS (  
  -- anchor: the top of the tree  
  SELECT emp_id, emp_name, manager_id, 1 AS level,  
         CAST(emp_name AS CHAR(500)) AS path  
  FROM   employee  
  WHERE  manager_id IS NULL  
  
  UNION ALL  
  
  -- recursive member: joins the CTE back to the base table  
  SELECT e.emp_id, e.emp_name, e.manager_id, oc.level + 1,  
         CONCAT(oc.path, ' > ', e.emp_name)  
  FROM   employee e  
  JOIN   org_chart oc ON e.manager_id = oc.emp_id  
)  
SELECT level, emp_name, path FROM org_chart ORDER BY path;
```

Output on our data:

level	emp_name	path
1	Asha	Asha
2	Bala	Asha > Bala
2	Chitra	Asha > Chitra
2	Dev	Asha > Dev
3	Esha	Asha > Dev > Esha
2	Farid	Asha > Farid
2	Gita	Asha > Gita

The four things to say:

1. **Anchor member** — the non-recursive seed (`manager_id IS NULL`).
2. **UNION ALL** — joins the recursive member; **UNION** would dedupe and cost a sort.
3. **Termination** — it stops when the recursive member returns no rows. **A cycle in the data means infinite recursion**, so guard it: MySQL's `cte_max_recursion_depth` (default 1000), or carry a `level` and add

`WHERE level < 10`, or track the path and exclude nodes already in it.

4. **RECURSIVE** keyword — required in MySQL and Postgres, **not used** in SQL Server (just `WITH`).

Other uses to mention: generating a date series to fill gaps in a report, exploding a bill-of-materials, walking a category tree.

Part 13 — Self joins and hierarchies

```
-- ① Every employee with their manager's name – LEFT so the CEO still
appears
SELECT e.emp_name AS employee, COALESCE(m.emp_name, '- none -') AS manager
FROM   employee e
LEFT JOIN employee m ON m.emp_id = e.manager_id;

-- ② Employees who earn MORE than their manager
SELECT e.emp_name, e.salary, m.emp_name AS manager, m.salary AS mgr_salary
FROM   employee e
JOIN   employee m ON m.emp_id = e.manager_id
WHERE  e.salary > m.salary;

-- ③ Managers with more than 2 direct reports
SELECT m.emp_name, COUNT(*) AS reports
FROM   employee e
JOIN   employee m ON m.emp_id = e.manager_id
GROUP BY m.emp_id, m.emp_name
HAVING COUNT(*) > 2;

-- ④ Employees who are NOT managers (nobody reports to them)
SELECT e.emp_name FROM employee e
WHERE NOT EXISTS (SELECT 1 FROM employee r WHERE r.manager_id = e.emp_id);

-- ⑤ Pairs of employees in the same department (each pair once, no self-
pairs)
SELECT a.emp_name, b.emp_name, a.dept_id
FROM   employee a
JOIN   employee b ON a.dept_id = b.dept_id AND a.emp_id < b.emp_id;
--                                     ~~~~~~ the
classic trick
```

● **a.emp_id < b.emp_id** is the detail they're checking for in ⑤. With <> you'd get each pair twice in both orders; with nothing you'd also pair every employee with themselves. Say it out loud.

Consecutive rows — the "three consecutive days" / "logged in 3 days running" family:

```
-- Consecutive rows using LAG
SELECT * FROM (
  SELECT id, num,
         LAG(num, 1) OVER (ORDER BY id) AS prev1,
         LAG(num, 2) OVER (ORDER BY id) AS prev2
  FROM logs
) t
WHERE num = prev1 AND num = prev2;
```

Part 14 — Dates, strings and CASE

14.1 Dates

```
-- Employees hired in 2023
WHERE hire_date >= '2023-01-01' AND hire_date < '2024-01-01'    -- ✓
sargable, uses the index
WHERE YEAR(hire_date) = 2023                                    -- ✗
function on the column kills the index

-- Tenure in years
SELECT emp_name, TIMESTAMPDIFF(YEAR, hire_date, CURDATE()) AS years    --
MySQL
SELECT emp_name, DATE_PART('year', AGE(CURRENT_DATE, hire_date))      --
Postgres

-- Last 30 days
WHERE order_date >= CURDATE() - INTERVAL 30 DAY                  -- MySQL
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days'            -- Postgres

-- Group by month
SELECT DATE_FORMAT(order_date, '%Y-%m') AS ym, SUM(amount)        -- MySQL
FROM orders GROUP BY ym ORDER BY ym;
SELECT DATE_TRUNC('month', order_date) AS ym, SUM(amount)        -- Postgres
FROM orders GROUP BY 1 ORDER BY 1;
```

● **The rule to say:** "Never wrap the indexed column in a function — rewrite it as a range. `YEAR(hire_date)=2023` forces a full scan; `hire_date >= '2023-01-01' AND hire_date < '2024-01-01'` is sargable and seeks the index." The word *sargable* ("Search-ARGument-able") lands well.

▲ **BETWEEN with datetimes:** `BETWEEN '2023-01-01' AND '2023-12-31'` misses everything on 31 Dec after midnight, because the end is `00:00:00`. Use the half-open range `>= start AND < next_start`.

14.2 Strings

Task	MySQL	Postgres
Concatenate	<code>CONCAT(a,b) /</code> <code>CONCAT_WS('-',a,b)</code>	<code>a b</code>
Length	<code>LENGTH</code> (bytes) / <code>CHAR_LENGTH</code>	<code>LENGTH</code>
Substring	<code>SUBSTRING(s, 1, 3)</code>	same
Position	<code>LOCATE('x', s)</code>	<code>POSITION('x' IN s)</code>
Upper/lower/trim	<code>UPPER</code> <code>LOWER</code> <code>TRIM</code>	same
Replace	<code>REPLACE(s, 'a', 'b')</code>	same
Split	<code>SUBSTRING_INDEX(email, '@', -1)</code>	<code>SPLIT_PART(email, '@', 2)</code>
Case sensitivity	▲ default collation is case-insensitive	case-sensitive ; use <code>ILIKE</code>

```
-- Domain from an email – a small live question they like
SELECT SUBSTRING_INDEX(email, '@', -1) AS domain, COUNT(*)
FROM users GROUP BY domain ORDER BY 2 DESC;
```

14.3 CASE — and pivoting without PIVOT

```
-- ① Conditional aggregation — headcount per band in one row
SELECT COUNT(CASE WHEN salary >= 70000 THEN 1 END) AS high,
       COUNT(CASE WHEN salary < 70000 THEN 1 END) AS low,
       SUM(CASE WHEN salary IS NULL THEN 1 ELSE 0 END) AS unknown
FROM employee;

-- ② Pivot: departments as columns
SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS ym,
       SUM(CASE WHEN o.status='PAID'      THEN o.amount ELSE 0 END) AS
paid,
       SUM(CASE WHEN o.status='CANCELLED' THEN o.amount ELSE 0 END) AS
cancelled
FROM orders o
GROUP BY ym ORDER BY ym;

-- ③ Custom sort order
ORDER BY CASE status WHEN 'URGENT' THEN 1 WHEN 'HIGH' THEN 2 ELSE 3 END,
order_date;
```

● **"Conditional aggregation" is the phrase to use** — `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` is how you pivot in any database without a `PIVOT` clause, and it's a genuinely common real-world pattern. Note `COUNT(CASE WHEN ... THEN 1 END)` works too because the implicit `ELSE NULL` isn't counted.

Part 15 — Indexes and query tuning

● **"How would you speed up a slow query?" is asked in almost every backend round.** Have a *method*, not a list of tricks.

15.1 The answer script — say it in this order

"First I'd measure, not guess — run `EXPLAIN ANALYZE` and look at what the plan actually does: is it a full table scan, how many rows is it estimating versus returning, and where is the time going. Then, in order: ① is there an index on what I filter, join and sort by — and is it usable, or am I wrapping the column in a

function; ② am I selecting more than I need — `SELECT *`, or rows I then throw away; ③ is this actually an N+1 coming from the ORM rather than one slow query; ④ can I reduce the work — filter earlier, avoid `SELECT DISTINCT` after a fan-out join, replace a correlated subquery with a join or window function. Only after that would I look at caching, denormalisation or pagination changes."

15.2 What an index is

"Usually a B+ tree: sorted, balanced, and it lets the engine seek to a range instead of scanning the table. The cost is slower writes — every `INSERT / UPDATE / DELETE` maintains every index — plus disk and memory. So I index what I filter, join and sort on, not everything."

Type	What it is
Clustered	Is the table — rows are physically stored in the index's order. One per table (the PK in InnoDB).
Non-clustered / secondary	A separate structure holding the key + a pointer back (in InnoDB, the PK value , so a secondary-index lookup costs a second seek — "bookmark lookup").
Composite	Multiple columns, order matters — see the leftmost rule.
Covering	Contains every column the query needs → index-only scan , never touches the table. <code>EXPLAIN</code> shows "Using index".
Unique	Index + constraint.
Partial / filtered	<code>WHERE deleted = false</code> — Postgres/SQL Server only.
Full-text / hash / GIN	For text search and specialised lookups.

15.3 ● The leftmost-prefix rule — the composite-index question

Index on (dept_id, salary, hire_date):

Query	Uses the index?
WHERE dept_id = 10	✓
WHERE dept_id = 10 AND salary > 50000	✓ both columns
WHERE dept_id = 10 ORDER BY salary	✓ — and the sort is free
WHERE salary > 50000	✗ no — skips the leading column
WHERE salary > 50000 AND hire_date > '2020-01-01'	✗
WHERE dept_id = 10 AND hire_date > '2020-01-01'	▲ partial — seeks on dept_id , then filters

"A composite index is sorted by the first column, then the second within that — like a phone book sorted by surname then first name. You can look up 'everyone called Kumar', and 'Kumar, Priya', but not 'everyone called Priya'." **That phone-book line is worth memorising.**

15.4 When an index is silently ignored

Cause	Fix
Function on the column: WHERE YEAR(d) = 2023 , WHERE UPPER(name) = 'X'	Rewrite as a range, or add a functional/ expression index
Leading wildcard: LIKE '%abc'	Full-text index, or store a reversed column. LIKE 'abc%' is indexable
Implicit type cast: WHERE varchar_col = 123	Match the types

Cause	Fix
<code>OR</code> across different columns	Rewrite as <code>UNION</code> , or index both and hope for an index merge
Low selectivity — a <code>gender</code> or <code>is_active</code> flag	Don't index it alone; make it part of a composite
Stale statistics	<code>ANALYZE TABLE</code>
The optimiser judges a scan cheaper (returning >~20% of rows)	Often correct — leave it

15.5 Reading `EXPLAIN`

```
EXPLAIN SELECT ...           -- the plan
EXPLAIN ANALYZE SELECT ...   -- the plan + actual execution times (MySQL 8.0.18+, Postgres)
```

MySQL type	Meaning
<code>system</code> / <code>const</code> / <code>eq_ref</code>	● best — one row via a PK/unique lookup
<code>ref</code>	● good — non-unique index lookup
<code>range</code>	● fine — an index range scan
<code>index</code>	● full index scan — better than a table scan, still everything
<code>ALL</code>	● full table scan — the thing you're usually hunting

Also read: `rows` (the estimate — if it's wildly off, statistics are stale), `key` (which index was chosen, `NULL` = none), and `Extra` : `"Using index"` = covering ✓, `"Using filesort"` / `"Using temporary"` = a sort or temp table you may be able to design away.

15.6 The rest of the tuning toolkit

- **Avoid `SELECT *`** — more I/O, breaks covering indexes, and breaks when a column is added.
- **Pagination:** `LIMIT 20 OFFSET 100000` still walks 100,020 rows. Use **keyset pagination**: `WHERE id > :last_seen ORDER BY id LIMIT 20`.
- **Batch, don't loop** — one `INSERT ... VALUES (...), (...), (...)` beats 1,000 round trips; in JPA that's `saveAll` + `hibernate.jdbc.batch_size`.
- **`EXISTS` over `IN`** for large correlated subqueries; **`UNION ALL` over `UNION`**.
- **Denormalise deliberately** for read-heavy reporting — *"a reporting table that duplicates a department name is a legitimate trade, not a mistake."*
- **Partitioning** for very large tables (by date range), and **archiving** old rows.
- **Connection pooling** (HikariCP) — *"a slow endpoint is sometimes pool exhaustion, not a slow query."*

Part 16 — Transactions, ACID and isolation levels

16.1 ACID — with a one-line example each (they ask for examples, not definitions)

	Means	Your example
Atomicity	All or nothing	"A funds transfer is a debit and a credit. If the credit fails, the debit is rolled back — you can't have half of it."
Consistency	The DB moves from one valid state to another; constraints hold	"An order can't reference a customer id that doesn't exist — the FK stops the transaction committing."

	Means	Your example
Isolation	Concurrent transactions don't corrupt each other	"Two people book the last seat at once; isolation decides whether one of them fails cleanly."
Durability	Once committed, it survives a crash	"Committed means written to the write-ahead log and fsynced, so a power cut doesn't lose it."

16.2 ● The three read phenomena and the four isolation levels

Phenomenon	What happens
Dirty read	You read another transaction's uncommitted change; it then rolls back
Non-repeatable read	You read the same row twice and get different values (someone committed an UPDATE between)
Phantom read	You run the same query twice and get different rows (someone committed an INSERT matching your WHERE)

Isolation level	Dirty	Non-repeatable	Phantom	Notes
READ UNCOMMITTED	✓ possible	✓	✓	Practically never used
READ COMMITTED	✗	✓	✓	Default in Postgres, Oracle, SQL Server

Isolation level	Dirty	Non-repeatable	Phantom	Notes
REPEATABLE READ	✗	✗	▲	Default in MySQL/InnoDB — and InnoDB's next-key locks prevent most phantoms too
SERIALIZABLE	✗	✗	✗	As if transactions ran one at a time. Safest, most contention

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
-- Spring:
@Transactional(isolation = Isolation.READ_COMMITTED)
```

● **The follow-up that wins the topic:** *"The default differs — MySQL is REPEATABLE READ , Postgres is READ COMMITTED . That's worth knowing because the same Spring Boot service can behave differently on the two databases."*

16.3 Optimistic vs pessimistic locking — the practical one for a Spring dev

	Pessimistic	Optimistic
Idea	Lock the row up front: SELECT ... FOR UPDATE	No lock; a @Version column detects conflicts at write time
Cost	Blocks other transactions, risks deadlock	Cheap reads; the loser retries
Use when	Contention is high, conflicts are expensive (inventory, payments)	Contention is low — most web CRUD

	Pessimistic	Optimistic
JPA	<code>@Lock(LockModeType.PESSIMISTIC_WRITE)</code>	<code>@Version</code> → <code>OptimisticLockException</code>

"On RoboGebra-style CRUD I'd default to optimistic locking with a `@Version` column — conflicts are rare, and I'd surface a 409 to the client rather than hold a database lock across a user's thinking time."

16.4 Deadlock at the database level

"Two transactions each hold a lock the other wants. InnoDB detects the cycle and kills the cheaper transaction with a deadlock error — so my job is to catch it and retry, and to reduce how often it happens: always acquire locks in the same order, keep transactions short, don't do HTTP calls inside a transaction, and use the narrowest isolation level that's correct."

- This is the database twin of the Java-level deadlock question in [32](#) — **Multithreading**. If you get asked "what's a deadlock", **ask which one they mean** — that's a good look.

Savepoints — worth one line: `SAVEPOINT s1; ... ROLLBACK TO s1;` lets you undo part of a transaction. In Spring that's `Propagation.NESTED`.

Part 17 — Normalization, keys and constraints

17.1 Normal forms — with the one example that shows each

Form	Rule	Violation → fix
1NF	Atomic values; no repeating groups	A <code>phone_numbers</code> column holding <code>'9x, 8y'</code> → a separate <code>phone</code> table

Form	Rule	Violation → fix
2NF	1NF + no partial dependency on part of a composite key	In <code>order_item(order_id, product_id, qty, product_name)</code> , <code>product_name</code> depends only on <code>product_id</code> → move it to <code>product</code>
3NF	2NF + no transitive dependency (non-key → non-key)	In <code>employee(emp_id, dept_id, dept_name)</code> , <code>dept_name</code> depends on <code>dept_id</code> , not on <code>emp_id</code> → move it to <code>dept</code>
BCNF	Stricter 3NF: every determinant is a candidate key	Rare in interviews — one sentence is enough

The line to close on: "3NF is my default for a transactional schema — it removes update anomalies. But I denormalise deliberately for read-heavy paths, reporting tables and dashboards, and I treat that as a documented trade, not an accident."

Denormalisation trade-off: faster reads and fewer joins, at the cost of duplicated data you must keep in sync.

17.2 Keys

Key	Meaning
Super key	Any set of columns that uniquely identifies a row
Candidate key	A minimal super key
Primary key	The candidate key you chose. Unique + NOT NULL , one per table
Alternate key	The candidate keys you didn't choose

Key	Meaning
Unique key	Unique, but allows NULL (and in most engines, multiple NULLs)
Composite key	A PK made of more than one column
Foreign key	References another table's PK/unique key — enforces referential integrity
Surrogate vs natural	Auto-increment/UUID vs a real-world value (email, PAN). Surrogate is usually safer — natural keys change

● **"Primary key vs unique key"** — the answer they want: *"Both enforce uniqueness. A primary key can't be NULL and there's only one; a unique key allows NULLs and you can have several. In InnoDB the PK is also the clustered index, so it determines physical row order — which is why a random UUID PK hurts insert performance and a sequential id doesn't."* That last clause is the mid-level differentiator.

17.3 Constraints and ON DELETE

NOT NULL · UNIQUE · PRIMARY KEY · FOREIGN KEY · CHECK (MySQL enforces it from 8.0.16) · DEFAULT

```
ALTER TABLE employee
  ADD CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES dept(dept_id)
  ON DELETE SET NULL ON UPDATE CASCADE;
```

ON DELETE	Effect on child rows when the parent is deleted
RESTRICT / NO ACTION	Block the delete (the default)
CASCADE	Delete the children too — ▲ <i>"powerful and easy to regret; I prefer explicit deletes in the service layer"</i>
SET NULL	Null out the FK (the column must be nullable)

Part 18 — DDL vs DML, DELETE vs TRUNCATE vs DROP

Category	Commands	Transactional?
DDL — Data Definition	CREATE ALTER DROP TRUNCATE RENAME	Auto-commits in MySQL/ Oracle; transactional in Postgres
DML — Data Manipulation	SELECT INSERT UPDATE DELETE MERGE	✓
DCL — Data Control	GRANT REVOKE	
TCL — Transaction Control	COMMIT ROLLBACK SAVEPOINT	

	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Removes	Selected rows	All rows	Rows + the table itself
WHERE	✓	✗	✗
Rollback	✓	✗ in MySQL/Oracle (auto-commit)	✗ in MySQL/Oracle
Fires triggers	✓	✗	✗
Resets AUTO_INCREMENT	✗	✓	—
Speed	Slow — row by row, logged	Fast — deallocates pages	Fast
Table/structure survives	✓	✓	✗

" **DELETE** is row-by-row and logged so it can be rolled back and fires triggers. **TRUNCATE** deallocates the pages, so it's far faster but it's DDL — no **WHERE** , no triggers, and in MySQL it auto-commits. **DROP** removes the table definition as well."

Part 19 — Views, stored procedures, functions, triggers

Object	What it is	When you'd use it
View	A stored SELECT . No data of its own — resolved at query time	Simplify a repeated join; expose a subset of columns as a security boundary
Materialized view	A view whose results are stored; needs REFRESH	Expensive reporting aggregates. Postgres/Oracle — MySQL has no native support
Stored procedure	Compiled SQL block, can have OUT params, may not return a value	Bulk/batch operations that would otherwise be many round trips
Function	Returns a value, usable inside a SELECT	A reusable calculation
Trigger	Fires automatically on INSERT / UPDATE / DELETE	Audit trails, updated_at maintenance

```
CREATE VIEW v_dept_summary AS
SELECT d.dept_id, d.dept_name, COUNT(e.emp_id) AS headcount, AVG(e.salary)
AS avg_sal
FROM dept d LEFT JOIN employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;
```

"Can you update through a view?" → *"Only a simple one — single table, no aggregate, no **DISTINCT**, no **GROUP BY**. Anything with an aggregate is read-only. Postgres can make complex views updatable with an **INSTEAD OF** trigger."*

Procedure vs function — the table they ask for:

	Procedure	Function
Returns	via OUT params / result sets	a single value (or table)

	Procedure	Function
Callable in <code>SELECT</code>	<code>x</code>	<code>✓</code>
Can do DML	<code>✓</code>	usually restricted
Invoked by	<code>CALL p()</code>	inside an expression

● **The opinion to have ready** — because they *will* ask "would you put business logic in the database?":

"I keep business logic in the application, in the service layer, so it's version-controlled, unit-testable and portable. Stored procedures are hard to diff and hard to test. I'd use one where it genuinely wins — a bulk operation where moving the data to the app is the bottleneck. Triggers I use very sparingly: they're invisible at the call site, so someone debugging the application never thinks to look for them."

Naming that "invisible side effect" cost is what makes this sound like experience.

Part 20 — SQL from the Java / Spring side

● In a fullstack Java round the SQL question often arrives dressed as a Spring question. These are the crossovers.

20.1 The N+1 problem — the most-asked ORM question anywhere

```
List<Dept> depts = deptRepo.findAll();           // 1 query
for (Dept d : depts) d.getEmployees().size();    // + N queries, one per
department
```

Detect it: turn on `spring.jpa.show-sql=true` (or better, use a query counter in tests) and watch one screen of near-identical `SELECT` s go by.

Fix it — four ways, know all four:

Fix	How
<code>JOIN FETCH</code>	<code>@Query("SELECT d FROM Dept d LEFT JOIN FETCH d.employees")</code>
<code>@EntityGraph</code>	<code>@EntityGraph(attributePaths = "employees")</code> on the repository method
Batch fetching	<code>@BatchSize(size = 25)</code> — turns N queries into N/25 <code>IN</code> queries
A projection / DTO query	Select only the columns you need — often the best answer for a read endpoint

▲ **The `JOIN FETCH` + pagination trap:** "*`JOIN FETCH` with `Pageable` makes Hibernate fetch everything and paginate in memory — it logs `HHH000104: firstResult/maxResults specified with collection fetch`. The fix is two queries: page the ids, then fetch the collection for those ids.*" ■ Full depth in [24 — ORM/JPA](#).

20.2 JPQL vs native SQL vs Criteria

	Operates on	Use when
JPQL / HQL	Entities and fields, not tables and columns	Most queries — portable across databases
Native SQL	Real tables	Window functions, <code>WITH RECURSIVE</code> , vendor-specific features, heavy tuning
Criteria API	Type-safe builder	Dynamic queries — a search form with 6 optional filters
Derived queries	Method name	<code>findByDeptIdAndSalaryGreaterThan(...)</code> — simple lookups

```
@Query(value = "SELECT * FROM employee WHERE salary > :min", nativeQuery = true)
List<Employee> findRicherThan(@Param("min") BigDecimal min);
```

"I reach for native SQL when the query is genuinely SQL-shaped — a window function or a recursive CTE — and I accept that I've traded portability for it. And I map it to a DTO projection rather than an entity, so Hibernate doesn't try to manage rows I only want to read."

20.3 ● SQL injection — the security question inside the SQL question

```
// ✗ Vulnerable – string concatenation
String sql = "SELECT * FROM users WHERE email = '" + email + "'";
// email = "x' OR '1'='1" → returns every user

// ✓ Parameterised / prepared statement – the value is never parsed as SQL
jdbcTemplate.query("SELECT * FROM users WHERE email = ?", rowMapper,
    email);

// ✓ JPA named parameters
@Query("SELECT u FROM User u WHERE u.email = :email")
```

"The real fix is parameterised queries, not input sanitising — with a bind parameter the value is sent separately from the statement and can never be interpreted as SQL. Escaping by hand is a losing game. Beyond that: least-privilege database accounts, validate input anyway, and never surface raw SQL errors to the client."

▲ **What binding can't parameterise:** table and column names, and **ORDER BY** directions. If those are dynamic, **whitelist** them against a fixed set — never concatenate.

20.4 @Transactional — the three traps

1. **Self-invocation doesn't work.** A `this.method()` call inside the same bean bypasses the proxy, so the annotation does nothing.
2. **Default rollback is on `RuntimeException` only.** A checked exception commits unless you say `@Transactional(rollbackFor = Exception.class)`.

3. **Don't wrap remote calls.** An HTTP call inside a transaction holds a database connection and any row locks for the whole round trip.

■ Full treatment in [30 §D4](#) and [24](#).

20.5 Connection pooling — one paragraph

"Opening a database connection is expensive, so Spring Boot ships HikariCP and reuses them. The pool size is a ceiling on concurrency — if every request holds a connection while it makes a slow external call, you exhaust the pool and requests queue up looking like a 'slow database' when the database is idle. That's why I keep transactions short and outside I/O."

Part 21 — Predict the output — 23 SQL programs

Run these against the sample data in [Part 1](#). Cover the answer, commit to a number, then check. **This is the format that has cost you interviews before** ([23 — Java Output](#)) — do it the same way here.

****① ****

```
SELECT COUNT(*), COUNT(salary), COUNT(dept_id), COUNT(DISTINCT dept_id)
FROM employee;
```

Answer

****7, 6, 6, 3**** — 7 rows; Farid's salary is NULL; Gita's dept_id is NULL; distinct non-NULL departments are 10, 20, 30.

****② ****

```
SELECT AVG(salary) FROM employee;
```

Answer

****`65833.33`**** — ``395000 / 6``, ****not**** ``/7``. ``AVG`` ignores NULLs. To divide by 7: ``SUM(salary)/COUNT(*)`` → ``56428.57``.

****③****

```
SELECT COUNT(*) FROM dept d LEFT JOIN employee e ON e.dept_id = d.dept_id;
```

Answer

****7**** — 6 matched rows (Gita has no dept, so she isn't reachable from ``dept``) ****+** 1 row for Legal ****** with NULL employee columns.

****④****

```
SELECT * FROM dept WHERE dept_id NOT IN (SELECT dept_id FROM employee);
```

Answer

****Zero rows.**** **●** The subquery contains a NULL (Gita), so ``dept_id <> NULL`` is UNKNOWN and the whole ``AND`` chain can never be TRUE. Expected Legal; got nothing. Fix with ``NOT EXISTS`` or ``WHERE dept_id IS NOT NULL`` inside the subquery.

****⑤****

```
SELECT d.dept_name, COUNT(*) FROM dept d
LEFT JOIN employee e ON e.dept_id = d.dept_id
WHERE e.salary > 50000
GROUP BY d.dept_name;
```

Answer

****Engineering 3, Sales 1.**** Two things happened: the ``WHERE`` on the right table ****silently turned the left join into an inner join**** (Legal vanishes), and Farid's NULL salary fails the predicate (HR vanishes). Esha at exactly 50000 fails ``>``. To keep every department, move the condition into the ``ON``.

****⑥****

```
SELECT emp_name, salary,  
       RANK()      OVER (ORDER BY salary DESC) r,  
       DENSE_RANK() OVER (ORDER BY salary DESC) dr  
FROM employee WHERE salary IS NOT NULL;
```

Answer

| emp_name | salary | r | dr | |---|---|---|---| | Asha | 90000 | 1 | 1 | | Bala | 75000 | 2 | 2 | | Chitra | 75000 | 2 | 2 | | Dev | 60000 | ****4**** | ****3**** | | Esha | 50000 | 5 | 4 | | Gita | 45000 | 6 | 5 | ``RANK`` skips after a tie, ``DENSE_RANK`` doesn't.

****⑦****

```
SELECT salary FROM employee ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

Answer

****75000**** — correct here ****by luck****. Because the top salary is unique, row 2 happens to be the second-highest. If Asha had a twin on 90000, this would return ****90000****. ****Always add `DISTINCT`**** for the "Nth highest" question.

****⑧****

```
SELECT COUNT(*) FROM employee e JOIN dept d ON d.dept_id = e.dept_id;
```

Answer

****6**** — Gita's NULL ``dept_id`` matches nothing (``NULL = 10`` is UNKNOWN), so she's dropped by the inner join.

****⑨****

```
SELECT dept_id, COUNT(*) FROM employee GROUP BY dept_id;
```

Answer

****4 groups: `10→3`, `20→2`, `30→1`, `NULL→1`.**** ● **`GROUP BY`** puts all NULLs in ****one group****, even though **`NULL = NULL`** is UNKNOWN. It's a documented exception — grouping uses "not distinct from", not **`=`**.

****⑩****

```
SELECT SUM(salary) FROM employee WHERE dept_id = 40;
```

Answer

****One row containing `NULL`**** — not zero rows, and not **`0`**. An aggregate with no **`GROUP BY`** always returns exactly one row, and **`SUM`** over an empty set is NULL. Use **`COALESCE(SUM(salary), 0)`**. (But **`SELECT COUNT(\*) ...`** on the same filter returns ****0****, because **`COUNT`** never returns NULL.)

****⑪****

```
SELECT emp_name FROM employee WHERE dept_id <> 10;
```

Answer

****Dev, Esha, Farid**** — 3 rows. ● ****Gita is missing****: **`NULL <> 10`** is UNKNOWN, not TRUE. This is the everyday version of the NULL trap — a **`<>`** filter silently drops NULL rows. Fix: **`WHERE dept\_id <> 10 OR dept\_id IS NULL`**.

****⑫****

```
SELECT dept_id, AVG(salary) FROM employee GROUP BY dept_id HAVING  
AVG(salary) > 60000;
```

Answer

****`10` → 80000`\*\*** only. Sales (20) averages 55000. HR (30) has only Farid with a NULL salary, so **`AVG`** is NULL and **`NULL > 60000`** is UNKNOWN → excluded. Gita's NULL group averages 45000.

****13****

```
SELECT COUNT(*) FROM employee e1, employee e2 WHERE e1.dept_id =
e2.dept_id;
```

Answer

****14.**** Old-style comma join = cross join filtered by `WHERE`. Per department, each employee pairs with every employee in the same department: Engineering $3 \times 3 = 9$, Sales $2 \times 2 = 4$, HR $1 \times 1 = 1 \rightarrow 14$. Gita's NULL matches nothing. ******"I'd write it as an explicit `JOIN ... ON` — comma joins make it far too easy to forget the condition and produce a cartesian product."******

****14****

```
SELECT emp_name, LAST_VALUE(salary) OVER (ORDER BY salary) FROM employee
WHERE salary IS NOT NULL;
```

Answer

******Each row returns its own salary******, not the maximum. With an `ORDER BY` and no explicit frame, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, so "last value" is the current row. Fix with `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

****15****

```
SELECT 'a' = 'A';           -- MySQL (default collation)
```

Answer

******`1` (true) in MySQL****** — the default collation is case-****insensitive****. In ****Postgres** it's false******. This bites when a service is developed on MySQL and deployed on Postgres, or when you assume `WHERE email = ?` is case-sensitive. Use an explicit collation, or normalise to lower case on write.

****16****

```
SELECT COUNT(*) FROM (SELECT DISTINCT salary FROM employee) t;
```

Answer

****6.**** `DISTINCT` treats all NULLs as one value and ****keeps**** it: `{90000, 75000, 60000, 50000, 45000, NULL}`. Contrast with `COUNT(DISTINCT salary)` = ****5****, which drops NULL. Same data, two different answers — that contrast is the whole question.

****17****

```
UPDATE employee SET salary = salary * 1.1; -- how many rows change?
```

Answer

****6 rows change; 7 are "matched".**** Farid's `NULL * 1.1` is NULL, so his value is unchanged — MySQL reports `Rows matched: 7 Changed: 6`. ****And there's no `WHERE`**** — always run the `SELECT` first, and wrap it in a transaction on production. ****Before an UPDATE without a WHERE I'd run `BEGIN`, check the affected count, then `COMMIT`.**

****18****

```
SELECT e.emp_name, m.emp_name AS mgr FROM employee e JOIN employee m ON  
m.emp_id = e.manager_id;
```

Answer

****6 rows**** — everyone except ****Asha****, whose `manager_id` is NULL. To include the CEO, make it a `LEFT JOIN` and `COALESCE` the manager name. This is the standard self-join follow-up.

19 Join cardinality. `A(id)` holds `1, 1, 2` · `B(id)` holds `1, 2, 2`.

```
SELECT COUNT(*) FROM A INNER JOIN B ON A.id = B.id;
SELECT COUNT(*) FROM A LEFT JOIN B ON A.id = B.id;
SELECT COUNT(*) FROM A CROSS JOIN B;
```

Answer

****4, 4, 9.**** Value 1: $2 \times 1 = 2$. Value 2: $1 \times 2 = 2$. Inner = ****4****. Every row on both sides matched, so LEFT is also ****4****. Cross = $3 \times 3 =$ ****9****.

20 Now with NULLs. `A(id)` holds `1, 2, 3, NULL` · `B(id)` holds `1, 1, NULL`.

Answer

| Join | Count | Working | |---|---|---| | ``INNER`` | ****2**** | only value 1 is in both: 1×2 |
 | ``LEFT`` | ****5**** | 2 + unmatched A rows `2, 3, NULL` | | ``RIGHT`` | ****3**** | 2 + B's
 unmatched `NULL` | | ``FULL`` | ****6**** | 2 + 3 + 1 | | ``CROSS`` | ****12**** | 4×3 | The two
 NULLs never meet — `NULL = NULL` is UNKNOWN.

21 Counting on a left join. `A(id)` = `1,1,2,2` · `B(id)` = `1,1,3`.

```
SELECT COUNT(*), COUNT(b.id), COUNT(DISTINCT a.id) FROM A a LEFT JOIN B b
ON a.id = b.id;
```

Answer

****6, 4, 2.**** Six output rows (4 matched + 2 placeholder rows for the unmatched ``2`s`). `COUNT(b.id)` skips those two NULL placeholders → 4. Distinct ``a.id`` values are 1 and 2 → 2. ****Same trap as the department-headcount question.****

22 The double-count. Asha earns 90000 and has 3 orders.

```
SELECT e.emp_name, SUM(e.salary) FROM employee e JOIN orders o ON o.emp_id
= e.emp_id
WHERE e.emp_name = 'Asha' GROUP BY e.emp_id, e.emp_name;
```

Answer

****270000**** — not 90000. The join produces one row per order, so her salary is summed three times. Fix: aggregate `orders` in a subquery first, or use `MAX(e.salary)`. ****This is the real-world version of the `1,1,1,1` puzzle.****

23 Self join. A department has exactly 4 employees.

```
SELECT COUNT(*) FROM employee e1 JOIN employee e2 ON e1.dept_id =
e2.dept_id;           -- ?
SELECT COUNT(*) FROM employee e1 JOIN employee e2 ON e1.dept_id =
e2.dept_id
                                AND e1.emp_id <>
e2.emp_id;           -- ?
SELECT COUNT(*) FROM employee e1 JOIN employee e2 ON e1.dept_id =
e2.dept_id
                                AND e1.emp_id <
e2.emp_id;           -- ?
```

Answer

****16, 12, 6.**** `4x4 = 16` including self-pairs. `<>` removes the 4 self-pairs → 12, but each pair appears in both orders. `<` gives each unordered pair exactly once → `C(4,2) = 6`. ****`<` is the answer they want for "list pairs of colleagues".****

Part 22 — Rapid-fire: 65 questions

#	Question	One-line answer
1	WHERE vs HAVING	WHERE filters rows before grouping; HAVING filters groups after aggregation
2	Can WHERE use an aggregate?	No — it runs before GROUP BY , the aggregate doesn't exist yet
3	Can ORDER BY use a SELECT alias?	Yes — it runs after SELECT . WHERE can't

#	Question	One-line answer
4	Logical execution order	FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
5	COUNT(*) vs COUNT(col)	* counts rows including NULLs; col counts non-NULL values only
6	COUNT(1) vs COUNT(*)	Identical — the optimiser treats them the same
7	Which aggregate never returns NULL?	COUNT — SUM / AVG / MIN / MAX return NULL on an empty set
8	Does AVG ignore NULLs?	Yes — it divides by the non-NULL count
9	INNER vs LEFT JOIN	Inner keeps only matches; left keeps every left row with NULLs where none
10	ON vs WHERE in an outer join	ON decides matching; WHERE filters after — a WHERE on the right table turns it into an inner join
11	Does LEFT JOIN return exactly the left row count?	At least — more if a left row matches several right rows
12	FULL OUTER JOIN in MySQL	Not supported — LEFT ... UNION ... RIGHT
13	Self join	A table joined to itself with two aliases — employee/ manager
14	Cross join row count	left × right
15	Anti-join, 3 ways	LEFT JOIN ... IS NULL , NOT EXISTS , NOT IN (⚠️ NULL-unsafe)

#	Question	One-line answer
16	Why does NOT IN break?	A NULL in the list makes every comparison UNKNOWN → zero rows
17	IN vs EXISTS	IN for small lists; EXISTS short-circuits and is NULL-safe
18	Correlated subquery	References the outer query, so it re-evaluates per outer row
19	UNION vs UNION ALL	UNION deduplicates (implicit sort); UNION ALL doesn't and is faster
20	UNION vs JOIN	Union stacks rows; join combines columns
21	ROW_NUMBER vs RANK vs DENSE_RANK	1,2,3 · 1,2,2,4 (gap) · 1,2,2,3 (no gap)
22	Which for Nth-highest salary?	DENSE_RANK — you're ranking salary levels, not people
23	Window function vs GROUP BY	GROUP BY collapses rows; a window function keeps them
24	Window function in WHERE ?	✗ — wrap it in a subquery/CTE and filter outside
25	Default window frame with ORDER BY	RANGE UNBOUNDED PRECEDING → CURRENT ROW — hence running totals
26	LAG / LEAD	Previous / next row's value — month-over-month deltas
27	NTILE(4)	Splits the window into 4 buckets — quartiles

#	Question	One-line answer
28	CTE	A named WITH subquery — readable, reusable within the statement
29	Recursive CTE	WITH RECURSIVE : anchor + UNION ALL + recursive member — hierarchies
30	CTE vs temp table	CTE = one statement, no index; temp table = session-scoped, indexable
31	View	A stored SELECT with no data of its own
32	Materialized view	Stores results; needs refreshing. Not in MySQL
33	Can you update a view?	Only a simple single-table one — no aggregates or GROUP BY
34	Procedure vs function	Function returns a value and works inside SELECT ; procedure is CALL ed
35	Trigger	Auto-fires on DML — good for audit, bad for hidden business logic
36	DELETE vs TRUNCATE	DML, WHERE , rollback, triggers vs DDL, all rows, fast, auto-commits
37	TRUNCATE vs DROP	Truncate empties, drop removes the table definition too
38	DDL / DML / DCL / TCL	CREATE-ALTER-DROP / SELECT-INSERT-UPDATE-DELETE / GRANT-REVOKE / COMMIT-ROLLBACK

#	Question	One-line answer
39	ACID	Atomicity · Consistency · Isolation · Durability
40	Dirty read	Reading uncommitted data that then rolls back
41	Non-repeatable vs phantom	Same row changes value vs same query returns new rows
42	Default isolation level	MySQL REPEATABLE READ ; Postgres/Oracle/SQL Server READ COMMITTED
43	Optimistic vs pessimistic locking	@Version conflict detection vs SELECT ... FOR UPDATE
44	DB deadlock	Two transactions hold each other's locks; the engine kills one — fix by consistent lock ordering + retry
45	1NF / 2NF / 3NF	Atomic values / no partial dependency on a composite key / no transitive dependency
46	When denormalise?	Read-heavy reporting — a deliberate, documented trade
47	Primary vs unique key	PK: one, NOT NULL , clustered in InnoDB. Unique: many, allows NULLs
48	Surrogate vs natural key	Auto-id/UUID vs a real-world value — natural keys change, so prefer surrogate
49	ON DELETE CASCADE	Deletes child rows — powerful, easy to regret

#	Question	One-line answer
50	What is an index?	A B+ tree that lets you seek instead of scan — costs write speed and space
51	Clustered vs non-clustered	Clustered <i>is</i> the table's physical order (one); non-clustered points back to it
52	Leftmost-prefix rule	Index <code>(a,b)</code> helps <code>WHERE a</code> and <code>a AND b</code> , not <code>b</code> alone
53	Covering index	Contains every column the query needs → index-only scan
54	When is an index ignored?	Function on the column, leading <code>%</code> wildcard, implicit cast, low selectivity, stale stats
55	Sargable	A predicate that can use an index — <code>col >= x</code> , not <code>f(col) = x</code>
56	How do you diagnose a slow query?	<code>EXPLAIN ANALYZE</code> → look for <code>type: ALL</code> , bad row estimates, filesort
57	Why avoid <code>SELECT *</code> ?	Extra I/O, defeats covering indexes, breaks when columns change
58	Why is <code>OFFSET 100000</code> slow?	It still walks all the skipped rows — use keyset pagination on an indexed column
59	Preventing SQL injection	Parameterised/prepared statements — never string concatenation; whitelist dynamic identifiers

#	Question	One-line answer
60	N+1 problem	One query per parent row from the ORM — fix with <code>JOIN FETCH</code> , <code>@EntityGraph</code> , <code>@BatchSize</code> or a DTO projection
61	A has four <code>1</code> s, B has four <code>1</code> s — inner join rows?	16 — every match pairs with every match: 4×4
62	...and left, right, full, cross?	All 16. Nothing is unmatched, so the outer joins add nothing, and cross is also 4×4
63	The cardinality formula	<code>INNER = $\Sigma(\text{countA}(v) \times \text{countB}(v))$</code> over shared values; <code>LEFT = INNER + unmatched A rows</code> ; <code>CROSS = rowsA $\times$ rowsB</code>
64	Do two NULL keys join to each other?	No — <code>NULL = NULL</code> is UNKNOWN. Each NULL row is unmatched on its own side
65	<code>SUM(salary)</code> after joining orders returns 3× too much — why?	The join fanned the employee row out once per order. Aggregate the child table in a subquery first

Part 23 — Drills (answers hidden)

Write the SQL on paper first. Reading these does nothing; typing them is the whole point.

D1. For each department, show the name, the headcount and the highest salary — including departments with nobody in them.

Answer

```
SELECT d.dept_name, COUNT(e.emp_id) AS headcount, MAX(e.salary) AS
top_salary
FROM   dept d
LEFT JOIN employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;
```

`MAX` returns NULL for an empty department — wrap in `COALESCE` if they want 0.

D2. Find the **3rd-highest** salary, handling ties, without using **LIMIT**.

Answer

```
SELECT DISTINCT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rnk FROM
employee
) t WHERE rnk = 3;
-- or, no window functions:
SELECT DISTINCT e.salary FROM employee e
WHERE 2 = (SELECT COUNT(DISTINCT e2.salary) FROM employee e2 WHERE
e2.salary > e.salary);
```

D3. List employees who earn **more than their manager**.

Answer

```
SELECT e.emp_name, e.salary, m.emp_name AS manager, m.salary AS mgr_salary
FROM employee e JOIN employee m ON m.emp_id = e.manager_id
WHERE e.salary > m.salary;
```

D4. For each employee show their salary and the **department average**, on the same row.

Answer

```
SELECT emp_name, salary, dept_id,
    AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
FROM employee;
```

A `GROUP BY` can't do this — it would collapse the rows. That's the point of the question.

D5. Find employees who have **never placed an order**.

Answer

```
SELECT e.* FROM employee e
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.emp_id = e.emp_id);
```

`NOT IN` is the wrong answer here if `orders.emp\_id` is nullable.

D6. Monthly revenue with the **previous month** and the **% change**.

Answer

```
WITH m AS (
    SELECT DATE_FORMAT(order_date, '%Y-%m') AS ym, SUM(amount) AS revenue
    FROM orders GROUP BY ym
)
SELECT ym, revenue,
       LAG(revenue) OVER (ORDER BY ym) AS prev,
       ROUND(100.0*(revenue - LAG(revenue) OVER (ORDER BY ym))
            / NULLIF(LAG(revenue) OVER (ORDER BY ym),0), 2) AS pct
FROM m ORDER BY ym;
```

`NULLIF(...,0)` guards the divide-by-zero — say that out loud.

D7. Delete duplicate users by email, keeping the **most recently created**.

Answer

```
DELETE FROM users WHERE id IN (
    SELECT id FROM (
        SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY created_at
        DESC) rn FROM users
    ) t WHERE rn > 1
);
```

Then `ALTER TABLE users ADD UNIQUE (email);` so it can't recur.

D8. Departments where **every** employee earns more than 50,000.

Answer

```
SELECT d.dept_name
FROM dept d JOIN employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING MIN(e.salary) > 50000;
```

The trick is turning "for all" into ``MIN(...) > x``. The alternative is ``NOT EXISTS (... salary <= 50000)``. ^ Watch NULL salaries — ``MIN`` ignores them, so a department of NULLs would pass; add ``COUNT(*) = COUNT(e.salary)`` if that matters.

D9. Top 2 earners per department, ties included.

Answer

```
SELECT * FROM (
  SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary
    DESC) rnk FROM employee e
) t WHERE rnk <= 2;
```

D10. The full reporting chain for each employee, with their level in the hierarchy.

Answer

Recursive CTE — see [Part 12.2](#122--recursive-cte--the-employee-hierarchy-question).

D11. Count orders by status **as columns**, one row per month.

Answer

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS ym,
       SUM(CASE WHEN status='PAID'      THEN 1 ELSE 0 END) AS paid,
       SUM(CASE WHEN status='PENDING'   THEN 1 ELSE 0 END) AS pending,
       SUM(CASE WHEN status='CANCELLED' THEN 1 ELSE 0 END) AS cancelled
FROM orders GROUP BY ym ORDER BY ym;
```

Conditional aggregation — the portable pivot.

D12. Employees hired in the **last 90 days**, in a way that uses an index on `hire_date`.

Answer

```
SELECT * FROM employee WHERE hire_date >= CURDATE() - INTERVAL 90 DAY;
```

****Not**** ``WHERE DATEDIFF(CURDATE(), hire_date) <= 90`` — a function on the column is not sargable.

D13. This runs in 8 seconds. What do you do?

```
SELECT * FROM orders o JOIN employee e ON e.emp_id = o.emp_id
WHERE YEAR(o.order_date) = 2026 AND e.dept_id = 10;
```

Answer

1. `EXPLAIN ANALYZE` first — confirm it's a full scan before changing anything. 2. `YEAR(o.order_date)` is not sargable → `o.order_date >= '2026-01-01' AND o.order_date < '2027-01-01'`. 3. Index `orders(order_date, emp_id)` and `employee(dept_id, emp_id)`. 4. Drop `SELECT *` — name the columns, which may make an index covering. 5. Re-measure. `"And I'd check it isn't actually an N+1 from the ORM producing 8 seconds of small queries rather than one slow one."`

D14. Second-highest salary **per department**, ties collapsed.

Answer

```
SELECT dept_id, salary FROM (  
  SELECT DISTINCT dept_id, salary,  
    DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) rnk  
  FROM employee WHERE salary IS NOT NULL  
) t WHERE rnk = 2;
```

D15. Why might `SELECT COUNT(*) FROM employee WHERE dept_id != 10` return fewer rows than you expect?

Answer

`NULL`dept_id`` rows are excluded — `NULL != 10` is UNKNOWN, not TRUE. Add `OR dept_id IS NULL`. If you can explain this fluently you're ahead of most candidates.

D16. `A(id) = 1,1,1` and `B(id) = 1,1`. Give the row count for inner, left, right, full and cross — then change B to `1,1,4` and give them again.

Answer

`**B = `1,1`:** inner `3x2` = **6**; left **6**; right **6**; full **6**; cross `3x2` = **6**. Everything matches, so all five are the same. **B = `1,1,4`:** inner still **6** (the `4` matches nothing); left **6** (no unmatched A rows); right **7** (6 + the unmatched `4`); full **7**; cross `3x3` = **9**.`

D17. For `A = 1,1,2,2` and `B = 1,1,3` with a `LEFT JOIN`, what do `COUNT(*)`, `COUNT(b.id)` and `COUNT(DISTINCT a.id)` return?

Answer

`**6, 4, 2.**` The two unmatched `2` rows produce placeholder rows with `NULL`b.id`` — counted by `COUNT(*)`, skipped by `COUNT(b.id)`. Distinct ``a.id`` values are 1 and 2.

D18. This report shows every employee's salary roughly tripled. What happened and how do you fix it?

```
SELECT e.emp_name, SUM(e.salary) AS salary, COUNT(o.order_id) AS orders
FROM employee e JOIN orders o ON o.emp_id = e.emp_id
GROUP BY e.emp_id, e.emp_name;
```

Answer

The join fans each employee out to one row per order, so `SUM(e.salary)` adds the same salary once per order. Fix by aggregating the child table first:

```
SELECT e.emp_name, e.salary, COALESCE(o.cnt, 0) AS orders
FROM employee e
LEFT JOIN (SELECT emp_id, COUNT(*) AS cnt FROM orders GROUP BY emp_id) o
ON o.emp_id = e.emp_id;
```

Or `MAX(e.salary)` instead of `SUM`. ▲ It gets worse with **two** child tables — orders and payments multiply against each other.

Part 24 — One-page cheat sheet

Read this at T-20 minutes. Nothing else.

◆ The five sentences that carry the whole topic

1. **Execution order** — "`FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`. That's why `WHERE` can't use an aggregate and `ORDER BY` can use an alias."
2. **The `LEFT JOIN` pair** — "`COUNT(*)` counts the placeholder row and returns 1 for an empty group; `COUNT(column)` skips NULLs and returns 0. And a `WHERE` on the right-hand table turns a left join back into an inner join — the condition belongs in the `ON`."
3. **Ranking** — "`ROW_NUMBER` 1,2,3 · `RANK` 1,2,2,4 · `DENSE_RANK` 1,2,2,3. For Nth-highest salary you want `DENSE_RANK`, because you're ranking salary levels."

4. **Join cardinality** — "Four 1s joined to four 1s is sixteen rows — and left, right, full and cross all give sixteen too, because nothing is unmatched. Outer joins only add rows for values with no partner on the other side."
5. **NULL** — "NULL means unknown, so **NULL = NULL** is UNKNOWN, and only **TRUE** rows survive a **WHERE**. That's why **NOT IN** with a NULL returns nothing and why **<> 10** drops NULL rows."

▲ Queries to be able to write cold

```
-- Aggregate per group, keeping empty groups
SELECT d.dept_name, COUNT(e.emp_id), COALESCE(ROUND(AVG(e.salary),2),0)
FROM dept d LEFT JOIN employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name HAVING COUNT(e.emp_id) > 0;

-- Nth highest
SELECT DISTINCT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY
salary DESC) r FROM employee) t WHERE r = 2;

-- Top-N per group
SELECT * FROM (SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER
BY salary DESC) r FROM employee e) t WHERE r <= 3;

-- Above own group's average
SELECT * FROM (SELECT e.*, AVG(salary) OVER (PARTITION BY dept_id) a FROM
employee e) t WHERE salary > a;

-- Duplicates
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

-- Anti-join (rows with no match)
SELECT d.* FROM dept d WHERE NOT EXISTS (SELECT 1 FROM employee e WHERE
e.dept_id = d.dept_id);

-- Self join with the manager
SELECT e.emp_name, COALESCE(m.emp_name, '-') FROM employee e LEFT JOIN
employee m ON m.emp_id = e.manager_id;

-- Running total
SELECT order_date, amount, SUM(amount) OVER (ORDER BY order_date ROWS
UNBOUNDED PRECEDING) FROM orders;
```

■ Lines worth having word-perfect

- "I'd use a `LEFT JOIN` here because a department with zero employees is still a valid answer."
- "`COUNT(*)` would count the empty placeholder row — I want `COUNT(e.emp_id)`."
- "`WHERE` filters rows before grouping; `HAVING` filters groups after aggregation."
- "I'd push that filter into `WHERE` rather than `HAVING` so it can use an index."
- "Never wrap the indexed column in a function — I'd rewrite it as a date range so it stays sargable."
- "Before I optimise anything I'd run `EXPLAIN ANALYZE` — measure, don't guess."
- "`NOT EXISTS` rather than `NOT IN`, because `NOT IN` breaks silently if the subquery returns a `NULL`."
- "Parameterised queries, not escaping — the value is sent separately from the statement so it can never be parsed as SQL."

✗ The five mistakes that cost marks

1. Writing `COUNT(*)` in a `LEFT JOIN` aggregate.
2. Putting the right-table filter in `WHERE` instead of `ON`.
3. `ROW_NUMBER` for "second-highest salary" (breaks on ties) — or `LIMIT 1 OFFSET 1` without `DISTINCT`.
 - ...and answering "4" for `A LEFT JOIN B` when both sides hold four 1s. **It's 16.**
4. Forgetting non-aggregated columns in `GROUP BY`.
5. Going silent. **Say the shape first** — "this is a group-by with a having, let me write it" — then write it.

Related files: 30 — Mphasis L2 §G · 26 — Companies §8 · 24 — ORM/JPA & N+1 · 13 — MongoDB · 06 — Spring Boot · 22 — Java Streams (the same problems — group by department, second-highest salary — in Java)

● Altimetrik — Fullstack Java + Angular round (31 Aug 2026)

Attended: Sunday 31 Aug 2026. ✗ Not cleared.

▲ **Read this as a study list, not a post-mortem.** The round was lost on **Spring wiring depth** — component scanning across packages, `@Qualifier` vs `@Primary`, the repository hierarchy, the *classes* inside pagination. Those are now fully answered below and in **06 — Spring Boot**. **Every one of these 22 questions will be asked again somewhere else** — lifecycle hooks are on their 5th round, interceptors on their 4th.

Second time at Altimetrik. Your first attempt (Level 1 Java + an Angular round, 24 questions) is logged in **26 — Companies §Altimetrik**. This round was a different shape: **Spring Boot depth first, Angular second, one Streams coding problem to close.**

Shape of the round: ~22 questions — **2 Java (records + pattern matching)** → **10 Spring/Spring Boot** → **2 microservices** → **1 config/environments** → **6 Angular** → **1 Java Streams live-coding.**

The signal: they went *deep on Spring wiring* (component scanning across packages, `@Qualifier` vs `@Primary`, `CrudRepository` vs `JpaRepository`, pagination classes) — this was not a breadth sweep, it was an "does she actually build with Spring Boot or just use it?" round.

◆ The lesson from this round — in one page

This round was lost on **Spring wiring depth**. Not on effort, and not on Angular. Here is exactly what "depth" meant, so the same gap can't happen twice.

What a BREADTH round asks

"What is @Autowired?"

"What is a repository?"

"How does pagination work?"

"Do you use Spring Boot?"

What THIS round asked ★

"Two beans of the same type – what EXACTLY happens, and what's the precedence between @Qualifier, @Primary and the bean name?" ★

"CrudRepository vs JpaRepository – what does the hierarchy give you, and what does findAll() RETURN?" ★

"Name the CLASSES involved." ★
(Pageable · PageRequest · Sort · Page/PageImpl · Slice/SliceImpl)

"Multiple packages – how does Spring read all the annotations?" ★
(scanBasePackageClasses, @EntityScan, @EnableJpaRepositories, auto-config)

The pattern to recognise ★

A DEPTH round is testing one thing:

"Has she BUILT with this, or has she USED it?" ★

The tell is that the questions go one level BELOW the API you type every day – into the classes behind it, the precedence rules, the return types.

Real-world idea: **anyone can drive a car. This round asked what happens when you press the clutch.**

How to answer a depth question when you're not sure ★

- 1. Give the SHAPE confidently – what you DO know
- 2. Name the CLASS or the RULE if you can reach it
- 3. Say honestly where your certainty ends ★
- 4. Say how you'd check it

"Pagination gives you a Page, which carries the content plus the total – so it runs a second COUNT query, whereas Slice runs one and only knows whether there's a next page. The concrete classes are Pageable and PageRequest, and I believe PageImpl is the implementation – that last one I'd confirm in the source rather than guess." ★

★ **That answer scores far better than a confident wrong class name.** Depth rounds punish bluffing much harder than they punish a bounded "I'd check that."

The three things to drill from this round

- 1. @Qualifier > @Primary > bean name ★ (and by TYPE first → NoUniqueBeanDefinition)
- 2. CrudRepository → PagingAndSortingRepository → JpaRepository ★
JpaRepository.findAll() returns List; CrudRepository returns Iterable ★
- 3. Page (2 queries, has the total ★) vs Slice (1 query, hasNext only) ★

→ All three are now written up in **06 — Spring Boot**.

The 22 questions at a glance

#	Question	Area	Answered below
1	What is a record ? — and why are we using it?	Java 16/17 ★	→
2	Pattern matching	Java 17	→
3	Spring Boot annotations	Spring Boot	→

#	Question	Area	Answered below
4	<code>@Component</code> and <code>@Service</code> annotation	Spring	→
5	"I have multiple packages — I want Spring to read all the annotations"	Spring	→
6	Reading the bean / Spring annotations — <code>@Bean</code>	Spring	→
7	<code>@Autowired</code> , <code>@Qualifier</code> and <code>@Primary</code>	Spring DI	→
8	<code>CrudRepository</code> and <code>JpaRepository</code>	Spring Data	→
9	Pagination methods	Spring Data	→
10	What classes are created in pagination	Spring Data	→
11	Communication in microservices	Microservices	→
12	How do you handle error cascading ?	Microservices	→
13	Two microservices, network failure — A errors, how do you inform B?	Microservices ★	→
14	Dev / QA / Prod — different DBs and properties, how is that handled?	Spring Boot	→

#	Question	Area	Answered below
15	<code>ngOnInit</code> and constructor	Angular ★	→
16	Decorator	Angular	→
17	Interceptor	Angular ★	→
18	Token expired — how do you notify the user?	Angular / Auth	→
19	How would you redirect to a page?	Angular	→
20	Angular core — what is data binding?	Angular ★	→
21	Lazy loading	Angular ★	→
22	Highest-paid employee in each department → department + employee name	Java Streams ●	→

★ = already asked in earlier rounds (repeat). ● = live coding.

Part 1 — Java: the opening question

1. What is a `record`? — and why are we using it?

● **This was the very first question of the round.** They asked "*what is a Record*", then immediately "*why are we using this?*" — i.e. **what problem does it solve that a normal class didn't**. Treat it as one question with two halves and answer both without being prompted.

One line: a **record** (Java 16, previewed in 14/15) is a **transparent carrier for immutable data** — you declare the components and the compiler generates the constructor, the accessors, `equals`, `hashCode` and `toString` for you.

```
public record Employee(int id, String name, int age, String department,
    String city, double salary, String gender) { }
```

- That one line replaces ~80 lines of boilerplate.** The compiler generates:
- a **canonical constructor** `Employee(int, String, int, String, String, double, String)`
 - **accessors named after the components** — `id()`, `name()`, `salary()` — ▲ **not** `getId()` / `getName()`
 - `equals` / `hashCode` based on **all** components (value semantics)
 - `toString` → `Employee[id=1, name=Abraham, age=29, ...]`
 - `private final` fields — the record is **shallowly immutable**

Why are we using it? — the four reasons

Reason	What to say
1. Kills boilerplate	"A DTO used to be a constructor, seven getters, <code>equals</code> , <code>hashCode</code> and <code>toString</code> — all of it noise the reader has to scan past. The record is the data model, in one line."
2. Immutability by default	"Fields are <code>final</code> . That makes it thread-safe , safe to use as a HashMap key , and safe to pass around without defensive copies — no caller can mutate my object behind my back."
3. Correct <code>equals</code> / <code>hashCode</code> , free	"Hand-written <code>equals</code> / <code>hashCode</code> are a classic bug source — someone adds a field and forgets to update them. The record can't drift out of sync."

Reason	What to say
4. It signals <i>intent</i>	"A record says <i>this type is nothing but its data</i> . A class says <i>this type has behaviour</i> . That's semantic information for every future reader — and the compiler enforces it."

◆ **The sentence that lands it:** "Records are Java's answer to the **anaemic data class**. Lombok's `@Value` solved the same problem with annotation processing; records solve it **in the language**, so there's no build-time magic and the semantics are guaranteed."

Where you'd actually use one (give a real example)

DTOs / API request-response objects · value objects (`Money` , `Coordinates` , `PatientId`) · records as map keys · a multi-value return from a method · events published to Kafka · `@ConfigurationProperties` binding.

```
// REST DTO – validation annotations work fine on components
public record CreateEmployeeRequest(
    @NotBlank String name,
    @Min(18) int age,
    @NotBlank String department,
    @Positive double salary) { }

@PostMapping("/employees")
public EmployeeResponse create(@Valid @RequestBody CreateEmployeeRequest
    req) { ... }
```

```
// returning two values without inventing a throwaway class
record SalaryStats(double max, double avg) {}
```

Compact constructor — validation & normalisation

```
public record Employee(int id, String name, double salary) {
    public Employee {                                     // compact form –
no parameter list, no assignment
        if (salary < 0) throw new IllegalArgumentException("salary must be
>= 0");
        name = name.trim();                             // normalise BEFORE
the fields are assigned
    }
    public Employee(int id, String name) { this(id, name, 0); } // extra
constructor must delegate
    public String initials() { return name.substring(0, 1); } // extra
methods are allowed
    public static Employee empty() { return new Employee(0, "", 0); } //
static members allowed
}
```

The rules / restrictions — the depth follow-up

Rule	Detail
Implicitly final	A record cannot be extended , and cannot extend another class (it already extends <code>java.lang.Record</code>)
Can implement interfaces	✓ <code>record Point(int x, int y) implements Comparable<Point></code>
No additional instance fields	All state must be a component — that's what "transparent" means
Static fields/methods	✓ Allowed
Accessors can be overridden	✓ but keep the contract honest
Nested records	✓ Implicitly static
Local records	✓ Declare one inside a method (Java 16+) — great for a stream pipeline's intermediate tuple

▲ The three traps worth volunteering

1. **Shallow immutability.** `record Team(String name, List<Player> players)`
— the *reference* is final, but the **list is still mutable**. Defensive-copy in the compact constructor: `players = List.copyOf(players);`
2. **Accessors are `name()`, not `getName()`**. Frameworks expecting the JavaBean convention can trip. Jackson handles records natively (2.12+), and Spring Boot binds them fine — but an old library might not.
3. **JPA `@Entity` cannot be a record.** JPA needs a no-arg constructor, non-final fields and mutability for proxies/dirty checking. **Use records for DTOs and projections, entities stay classes.** ✓ Spring Data *does* support records as **interface/DTO projections**:

```
java record EmployeeView(String name, double salary) {}  
@Query("select new com.x.EmployeeView(e.name, e.salary) from  
Employee e") List<EmployeeView> findViews();
```

Records + pattern matching = the reason they exist

```
sealed interface Shape permits Circle, Square {}  
record Circle(double radius) implements Shape {}  
record Square(double side) implements Shape {}  
  
double area(Shape s) {  
    return switch (s) {  
        case Circle(double r) -> Math.PI * r * r;           // record  
        DEconstruction pattern  
        case Square(double a) -> a * a;  
    };                                                         // no default  
    needed – sealed ⇒ exhaustive  
}
```

■ **Close on this:** "Records, sealed types and pattern matching were designed together — records make data transparent so patterns can take it apart, and sealed types let the compiler prove the switch is exhaustive. That's Java getting **algebraic data types**, and it's why records are more than a Lombok replacement."

If they meant `this` instead

If the follow-up "why are we using this" was about the `this` keyword: `this` is a reference to the current object — used to disambiguate a field from a parameter (`this.name = name`), to chain constructors (`this(...)`, must be the first statement), to return the current object for a fluent API, and to pass the current object to a collaborator. It doesn't exist in a `static` context, and in a **TypeScript/JS arrow function** `this` is lexical — which is the Angular version of the same question.

Part 2 — Java (continued)

2. Pattern matching

One line: pattern matching lets you **test a type and bind a variable in the same expression**, so the cast disappears.

a) `instanceof` pattern — Java 16, final

```
// before
if (obj instanceof String) { String s = (String) obj; if (s.length() > 5)
{...} }

// after
if (obj instanceof String s && s.length() > 5) { ... } // s is in scope
here
```

The binding variable `s` has **flow scope** — it exists only where the compiler can prove the test passed. That's why `&&` works and `||` doesn't.

b) Pattern matching for `switch` — Java 21, final (preview in 17/19/20)

```
static String describe(Object o) {
    return switch (o) {
        case null        -> "nothing";           // switch can now
handle null explicitly
        case Integer i when i > 100 -> "big int " + i;    // guarded
pattern
        case Integer i    -> "int " + i;
        case String s     -> "string of length " + s.length();
        default           -> "something else";
    };
}
```

- **when** is the **guard** (it was **&&** in the early previews).
- Cases are tested **in order**, so a more specific pattern must come first or you get a *"dominated by a preceding label"* compile error.
- Without an explicit **case null**, a **switch** on a pattern **throws** **NullPointerException** — old switch semantics.

c) Record patterns — Java 21 — destructuring:

```
record Point(int x, int y) {}
record Line(Point a, Point b) {}

if (obj instanceof Line(Point(var x1, var y1), Point(var x2, var y2))) {
    // x1, y1, x2, y2 are all bound — nested destructuring
}
```

d) Where it pays off: **sealed interface Shape permits Circle, Square {}** + a pattern **switch** = **exhaustiveness checked by the compiler**, no **default** needed. Add a new subtype and every switch that doesn't handle it fails to compile. That's the real selling point — say it.

- **Say the version numbers.** **instanceof** pattern = **16**, switch patterns + record patterns = **21**, text blocks = **15**, records + sealed = **16/17**. Getting versions right is a cheap credibility win. Full file: **12 — Java 17 Features**.

Part 3 — Spring Boot (the bulk of the round)

3. Spring Boot annotations

Answer this as **groups**, never as a flat list — a list sounds memorized, groups sound used.

Bootstrapping

| Annotation | What it does |

---|---

| `@SpringBootApplication` | The meta-annotation = `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan` |

| `@EnableAutoConfiguration` | Boot configures beans based on what's on the classpath |

| `@ComponentScan` | Where to look for stereotypes (see Q5) |

| `@Configuration` + `@Bean` | Java config — you construct the bean yourself |

Stereotypes (scanned into the context)

`@Component` · `@Service` · `@Repository` · `@Controller` · `@RestController` (= `@Controller` + `@ResponseBody`)

Injection

`@Autowired` · `@Qualifier` · `@Primary` · `@Value` · `@ConfigurationProperties` · `@Lazy` · `@Scope`

Web layer

`@RequestMapping` · `@GetMapping` / `@PostMapping` / `@PutMapping` / `@PatchMapping` / `@DeleteMapping` · `@PathVariable` · `@RequestParam` · `@RequestBody` · `@ResponseStatus` · `@CrossOrigin`

Validation & errors

`@Valid` / `@Validated` · `@NotNull` `@NotBlank` `@Size` `@Email` · `@ControllerAdvice` / `@RestControllerAdvice` · `@ExceptionHandler`

Data / JPA

`@Entity` · `@Table` · `@Id` · `@GeneratedValue` · `@Column` · `@OneToMany` /

`@ManyToOne` / `@ManyToMany` · `@JoinColumn` · `@Query` · `@Transactional` ·
`@Modifying`

Config & profiles

`@Profile` · `@PropertySource` · `@ConditionalOnProperty` /
`@ConditionalOnMissingBean` / `@ConditionalOnClass`

Testing

`@SpringBootTest` · `@WebMvcTest` · `@DataJpaTest` · `@MockBean` ·
`@TestConfiguration`

Async / scheduling / caching / resilience

`@EnableAsync` + `@Async` · `@EnableScheduling` + `@Scheduled` · `@EnableCaching`
+ `@Cacheable` / `@CacheEvict` · `@CircuitBreaker` / `@Retry` (Resilience4j)

◆ **Close with the mechanism, not more names:** "All of these are just metadata — the work is done by `BeanPostProcessor`s and proxies. That's why `@Transactional` or `@Async` **doesn't work on a self-invocation or a private method**: the call never leaves the object, so it never goes through the proxy."

4. `@Component` vs `@Service` annotation

The honest technical answer first: `@Service`, `@Repository` and `@Controller` are all **`@Component` under the hood** — they're meta-annotated with it. To the component scanner they behave identically: one scanned bean each.

```
@Target(ElementType.TYPE) @Retention(RetentionPolicy.RUNTIME)
@Component                  // ← this is the whole difference, plus
one behaviour below
public @interface Service { ... }
```

So why do they exist?

Annotation	Layer	Anything extra?
<code>@Component</code>	Generic — anything Spring should manage	—

Annotation	Layer	Anything extra?
<code>@Service</code>	Business logic	No extra behaviour — pure intent/readability. AOP pointcuts can target it
<code>@Repository</code>	Data access	✓ Real behaviour: <code>PersistenceExceptionTr</code> <code>anslationPostProcesso</code> <code>r</code> converts vendor exceptions (<code>SQLException</code> , Hibernate's) into Spring's <code>DataAccessException</code> hierarchy
<code>@Controller</code>	Web / MVC	✓ Handler methods are detected by <code>RequestMappingHandlerM</code> <code>apping</code>
<code>@RestController</code>	REST	✓ <code>@Controller</code> + <code>@ResponseBody</code> on every method

◆ **The three sentences that win this question:**

1. "They're all `@Component` — the scanner treats them the same."
2. " `@Repository` is the one with real behaviour — exception translation."
3. " `@Service` is intent. It documents the layer, it gives me a clean AOP pointcut (`@within(org.springframework.stereotype.Service)`), and on a team it stops business logic drifting into the controller."

▲ **Common follow-up:** "Can I put `@Service` on my repository interface?" — It would work (it's a component), but you'd lose exception translation and confuse every reader. With Spring Data you don't annotate the repository interface at all — the proxy is created by `@EnableJpaRepositories` / auto-configuration.

5. Multiple packages — how does Spring find all the annotations?

They asked: "I have multiple packages in Spring, I want all of them read for annotations."

The default rule: `@SpringBootApplication` implies `@ComponentScan` with **no arguments**, which scans **the package of the main class and every sub-package**. So the fix 90% of the time is just: *put the main class in the root package*.

```
com.priyanka.app      ← ApplicationMain.java lives here
├── controller
├── service
├── repository
└── config             ← all scanned automatically
```

When packages are outside that root, four options:

1. `scanBasePackages` on the main class — the usual answer

```
@SpringBootApplication(scanBasePackages = {"com.priyanka.app",
"com.priyanka.common", "com.acme.shared"})
public class ApplicationMain { }
```

2. `@ComponentScan` separately (same effect, more explicit)

```
@SpringBootApplication
@ComponentScan(basePackages = {"com.priyanka.app", "com.acme.shared"})
public class ApplicationMain { }
```

3. `basePackageClasses` — the type-safe version, and the one to name

```
@SpringBootApplication(scanBasePackageClasses = {ApplicationMain.class,
SharedMarker.class})
```

◆ This is **refactor-safe** — renaming the package won't silently break the scan the way a `String` would. Mentioning it is a real differentiator.

4. Filters, when you want only some of them

```
@ComponentScan(basePackages = "com.acme",
    includeFilters = @ComponentScan.Filter(type = FilterType.ANNOTATION,
        classes = MyStereotype.class),
    excludeFilters = @ComponentScan.Filter(type = FilterType.REGEX,
        pattern = "com\\.acme\\.legacy\\.*"))
```

Two things component scan does *not* cover — say these and the answer is complete:

Also outside the main package	The annotation
JPA entities	<code>@EntityScan("com.acme.domain")</code>
Spring Data repositories	<code>@EnableJpaRepositories("com.acme.repo")</code> (or <code>@EnableMongoRepositories</code>)

And for a shared library jar — the *right way*: don't scan someone else's package at all. Ship an **auto-configuration**: a `@AutoConfiguration` class listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (Boot 3; it was `spring.factories` in Boot 2). That's how every starter works, and it's the answer that shows you've built a shared module, not just consumed one.

▲ **Cost of over-scanning:** scanning `com` or a huge tree slows startup and can pull in beans you didn't want. Keep the roots tight.

6. `@Bean` vs stereotype annotations

They asked: "read the bean Spring annotations" — i.e. how a bean gets into the container.

Two routes into the `ApplicationContext`:

	Stereotype (@Component / @Service /...)	@Bean in a @Configuration class
Who constructs it	Spring, by scanning + calling the constructor	You , in the method body
Applies to	Your own classes you can annotate	Third-party classes you can't annotate (<code>ObjectMapper</code> , <code>RestTemplate</code> , <code>DataSource</code>)
Bean name	Class name, decapitalized (<code>orderService</code>)	The method name
Conditional creation	Awkward	Easy — <code>if/else</code> inside the method, <code>@Conditional*</code> on it
Multiple beans of same type	One class = one bean	✓ Several methods → several beans

```

@Configuration
public class AppConfig {

    @Bean                                // bean name = "restTemplate"
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder.setConnectTimeout(Duration.ofSeconds(2))
                       .setReadTimeout(Duration.ofSeconds(5))
                       .build();
    }

    @Bean @Primary
    public ObjectMapper objectMapper() {
        return JsonMapper.builder()
            .addModule(new JavaTimeModule())
            .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
            .build();
    }
}

```

The bean lifecycle (a very common follow-up — know the order):

instantiate → populate dependencies → `BeanNameAware / BeanFactoryAware` → `BeanPostProcessor.postProcessBeforeInitialization` → `@PostConstruct` → `InitializingBean.afterPropertiesSet()` / `initMethod` → **`postProcessAfterInitialization` (this is where the AOP proxy is created)** → *bean is ready* → ... → `@PreDestroy` → `DisposableBean.destroy()`.

Scopes: `singleton` (default, one per context) · `prototype` (new instance per lookup — **and Spring does not destroy it**) · web: `request`, `session`, `application`, `websocket`.

▲ **The classic trap:** a `@Configuration` class is itself CGLIB-proxied, so calling `restTemplate()` from another `@Bean` method returns **the same singleton**, not a new object. Change it to `@Configuration(proxyBeanMethods = false)` and you get a *fresh* object each call. Knowing this distinguishes real usage from tutorial knowledge.

7. `@Autowired`, `@Qualifier` and `@Primary`

How Spring resolves an injection point: by type first. If exactly one bean matches → done. If **more than one** matches →

`NoUniqueBeanDefinitionException`, unless you disambiguate.

```
public interface PaymentService { void pay(BigDecimal amt); }

@Service("card") class CardPaymentService implements PaymentService { }
@Service         class UpiPaymentService implements PaymentService { }
```

Three ways to disambiguate:

1. `@Primary` — the default winner

```
@Service @Primary
class UpiPaymentService implements PaymentService { } // injected
whenever nothing else is specified
```

Global: "when in doubt, use this one."

2. `@Qualifier` — the explicit pick at the injection point

```
@Service
public class CheckoutService {
    private final PaymentService payment;
    public CheckoutService(@Qualifier("card") PaymentService payment)
    { this.payment = payment; }
}
```

Local: "at *this* place, use that one." `@Qualifier` beats `@Primary`.

3. Field name = bean name (the fallback nobody mentions) — if the parameter/field is named `upiPaymentService`, Spring matches it by name. Works, but it's fragile: rename the variable and the wiring changes.

A custom qualifier is the clean version (no magic strings):

```
@Qualifier("card") @Retention(RUNTIME) public @interface CardPayment {}

@Service @CardPayment class CardPaymentService implements PaymentService
{}
...
public CheckoutService(@CardPayment PaymentService payment) { ... }
```

Injecting *all* of them — worth saying, it's how strategy patterns are actually built:

```
@Service
public class PaymentRouter {
    private final Map<String, PaymentService> byName; // key = bean name
    private final List<PaymentService> all; // ordered by
    @Order / Ordered
    public PaymentRouter(Map<String, PaymentService> byName,
        List<PaymentService> all) { ... }
}
```

◆ **Precedence, in one line:** `@Qualifier` > `@Primary` > matching bean name > `NoUniqueBeanDefinitionException`.

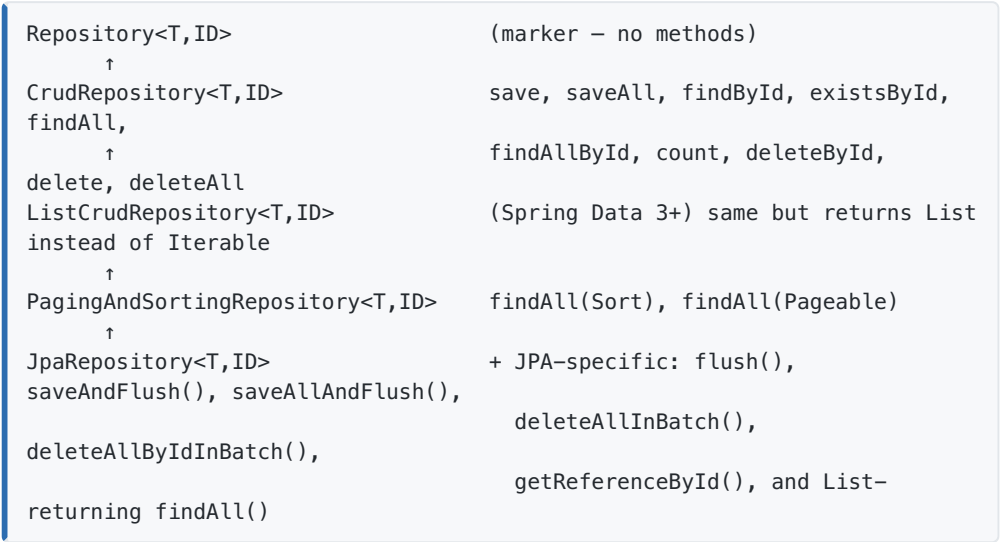
On `@Autowired` itself:

- **Constructor injection is the answer** — since Spring 4.3 `@Autowired` is optional on a single constructor. It gives you `final` fields, guaranteed non-null

- dependencies, and a class you can `new` in a unit test without Spring.
- **Field injection** (`@Autowired` on a field) is discouraged: untestable without reflection, hides a growing dependency list, can't be `final`. IntelliJ literally warns on it.
 - `@Autowired(required = false)` or `Optional<Foo>` / `ObjectProvider<Foo>` for optional deps.
 - **@Resource (JSR-250) matches by name first**, `@Autowired` by type first, `@Inject` (JSR-330) behaves like `@Autowired`. That comparison is a frequent follow-up.

8. CrudRepository vs JpaRepository (and the whole hierarchy)

The hierarchy — draw it, don't list it:



	CrudRepository	JpaRepository
Persistence tech	Technology-agnostic (JPA, Mongo, Redis, Cassandra...)	JPA only
<code>findAll()</code> returns	<code>Iterable<T></code>	✓ <code>List<T></code>

	CrudRepository	JpaRepository
Paging & sorting	✗	✓ (inherited from <code>PagingAndSortingRepository</code>)
Batch delete	✗ (<code>deleteAll</code> = one delete per row)	✓ <code>deleteAllInBatch()</code> — a single SQL statement
Flush control	✗	✓ <code>flush()</code> , <code>saveAndFlush()</code>
Lazy proxy	✗	✓ <code>getReferenceById()</code> (the old <code>getOne()</code>)

What to actually say:

" `JpaRepository` extends `PagingAndSortingRepository` which extends `CrudRepository`, so it's a superset. I use `JpaRepository` on a JPA project because I want `List` returns, `Pageable` and batch deletes. I'd deliberately drop down to `CrudRepository` if I wanted to keep the door open to switching the persistence technology, or to keep the API surface small so nobody calls `deleteAllInBatch()` by accident."

Where do the implementations come from? You never write one.

`@EnableJpaRepositories` (auto-configured by Boot) creates a **JDK dynamic proxy** per interface, backed by `SimpleJpaRepository`;

`RepositoryFactorySupport` resolves each method to either a **derived query** (parsed from the method name) or a declared `@Query`.

The three query styles:


```

public interface EmployeeRepository extends JpaRepository<Employee, Long>
{
    // 1. derived query – parsed from the method name
    List<Employee>
    findByDepartmentAndSalaryGreaterThanOrderBySalaryDesc(String dept, double
    salary);

    // 2. JPQL
    @Query("select e from Employee e where e.department = :dept and
    e.salary > :min")
    List<Employee> search(@Param("dept") String dept, @Param("min") double
    min);

    // 3. native SQL
    @Query(value = "select * from employee where salary > ?1", nativeQuery
    = true)
    List<Employee> rawSearch(double min);

    @Modifying @Transactional // required for update/delete
    @Query("update Employee e set e.salary = e.salary * 1.1 where
    e.department = :d")
    int giveRaise(@Param("d") String dept);
}

```

▲ **@Modifying** needs **clearAutomatically / flushAutomatically** awareness
 — a bulk JPQL update **bypasses the persistence context**, so entities already
 loaded are stale. That's the depth follow-up. More: [24 — ORM / JPA / Hibernate](#).

9. Pagination methods

Why paginate at all (lead with this): `findAll()` on a 2-million-row table loads 2
 million entities into heap and serializes them to the client. Pagination pushes
`LIMIT / OFFSET` into SQL, so the DB returns 20 rows.

The method signature is the whole answer:

```
public interface EmployeeRepository extends JpaRepository<Employee, Long>
{
    Page<Employee> findByDepartment(String department, Pageable
pageable); // + a COUNT query
    Slice<Employee> findBySalaryGreaterThan(double salary, Pageable
pageable); // no COUNT query
    List<Employee> findByCity(String city, Pageable
pageable); // just the window
}
```

Controller:

```
@GetMapping("/employees")
public Page<EmployeeDto> list(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(defaultValue = "salary,desc") String[] sort) {

    Pageable pageable = PageRequest.of(page, size,
Sort.by(Sort.Order.desc("salary")));
    return repo.findAll(pageable).map(mapper::toDto); // Page.map keeps
the metadata
}
```

◆ **Boot resolves it for you.** Just declare `Pageable pageable` as a controller parameter — `PageableHandlerMethodArgumentResolver` binds `?page=0&size=20&sort=salary,desc` automatically. Add `@PageableDefault(size = 20, sort = "salary", direction = DESC)` for defaults. Saying this shows you've shipped it.

Sorting

```
Sort.by("salary").descending().and(Sort.by("name").ascending());
PageRequest.of(0, 10, Sort.by(Sort.Direction.DESC, "salary"));
```

The methods you read off a `Page` :

```
getContent() · getTotalElements() · getTotalPages() · getNumber() (current
page) · getSize() · getNumberOfElements() · hasNext() / hasPrevious() ·
isFirst() / isLast() · nextPageable() · map(Function).
```

▲ **Two traps worth naming — this is where the follow-up goes:**

1. **OFFSET** gets slower the deeper you go. `LIMIT 20 OFFSET 1000000` still makes the DB walk a million rows. For deep paging / infinite scroll use **keyset (cursor) pagination**: `where (salary, id) < (:lastSalary, :lastId) order by salary desc, id desc limit 20`.
2. **Pagination + JOIN FETCH on a collection = HHH000104: firstResult/ maxResults specified with collection fetch; applying in memory** — Hibernate loads *everything* and pages in memory. Fix: page the IDs first, then fetch the collection for those IDs (two queries), or use `@EntityGraph / @BatchSize`.
3. **Always sort by something unique** (add `id` as a tiebreaker) or rows can repeat/disappear across pages.

10. What classes are involved in pagination?

The literal question they asked: "what are the classes created in pagination".

Type	Kind	Role
Pageable	interface	The request : page number, page size, sort. <code>Pageable.unpaged()</code> for "no paging"
PageRequest	class — implements <code>Pageable</code>	The standard implementation. <code>PageRequest.of(page, size) / of(page, size, Sort)</code>
Sort	class	Sort spec — a list of <code>Sort.Order</code> (property + <code>Direction</code> + null handling)
Sort.Order / Sort.Direction	class / enum	One property's direction; <code>ASC / DESC</code>
Page<T>	interface (extends <code>Slice<T></code>)	The response with total count — content + <code>totalElements</code> + <code>totalPages</code>

Type	Kind	Role
<code>PageImpl<T></code>	class — implements <code>Page</code>	The concrete result Spring Data returns (and what you <code>new</code> when paging a list manually)
<code>Slice<T></code>	interface	Content + <code>hasNext()</code> only , no total — one query instead of two
<code>SliceImpl<T></code>	class	Concrete <code>Slice</code>
<code>Window<T></code> / <code>ScrollPosition</code>	Spring Data 3.1+	Keyset/offset scrolling API — the modern deep-paging answer
<code>PagedModel</code> / <code>PagedResourcesAssembler</code>	Spring HATEOAS	Wraps a <code>Page</code> with paging links for a REST API
<code>PageableHandlerMethodArgumentResolver</code>	class	The MVC resolver that builds <code>Pageable</code> from query params

```
Pageable pageable = PageRequest.of(0, 20, Sort.by("salary").descending());
Page<Employee> page = repo.findAll(pageable);

page.getTotalElements();    // 137    ← costs an extra SELECT COUNT(*)
page.getTotalPages();       // 7
page.getNumber();          // 0      ← pages are ZERO-indexed
page.hasNext();             // true
```

◆ **The differentiator answer: `Page` vs `Slice`.**

"`Page` runs **two** queries — the window and a `COUNT(*)` — because it has to report `totalElements`. `Slice` runs **one**: it fetches `size + 1` rows and uses the extra row to answer `hasNext()`. So a numbered pager needs `Page`; an infinite-scroll feed should use `Slice` and skip the count entirely. On a big table that count query is often the slowest part of the request."

▲ **`page` is zero-indexed.** UIs are one-indexed. That off-by-one is a real bug and a favourite follow-up.

11. Communication in microservices

Split it into two axes first — that framing is the answer:

Synchronous (request/response, caller waits)

Option	Notes
REST over HTTP	Default. Simple, debuggable, cache-friendly
RestTemplate	▲ In maintenance mode — say this, it dates you otherwise
WebClient	Reactive, non-blocking, works fine in a blocking app via <code>.block()</code> ; supports timeouts and retries natively
RestClient	Spring 6.1+ — the synchronous, fluent replacement for RestTemplate
OpenFeign	Declarative: an interface + <code>@FeignClient</code> . Integrates with Eureka + Resilience4j
gRPC	Binary, HTTP/2, contract-first with protobuf. Fast + strongly typed; harder to debug
GraphQL	One endpoint, client picks the fields — good for a BFF

```
@FeignClient(name = "order-service", fallback = OrderClientFallback.class)
public interface OrderClient {
    @GetMapping("/orders/{id}")
    OrderDto getOrder(@PathVariable Long id);
}
```

Asynchronous (event/message, caller doesn't wait)

Kafka / RabbitMQ / SQS / SNS — publish an event, consumers react.

- **Decoupling**: the producer doesn't know who consumes.
- **Resilience**: the broker buffers when the consumer is down.
- **Cost**: eventual consistency, ordering and idempotency become *your* problem.

```
@KafkaListener(topics = "order-created", groupId = "billing")
public void onOrderCreated(OrderCreatedEvent event) { ... }
```

The supporting cast (name these — it's what "microservices experience" sounds like)

- **Service discovery** — Eureka / Consul / Kubernetes DNS, so you call `http://order-service` not an IP.
- **API Gateway** — Spring Cloud Gateway: routing, auth, rate limiting, one entry point.
- **Config Server** — centralized configuration (Q14).
- **Resilience4j** — circuit breaker, retry, timeout, bulkhead, rate limiter (Q13).
- **Distributed tracing** — Micrometer Tracing + Zipkin/Jaeger; one trace ID across every hop.
- **Saga** — distributed transactions by compensation (choreography via events, or orchestration via a coordinator), because there is no 2-phase commit across services.

◆ **The one-sentence rule to close on: "Synchronous when I need an answer to serve this request; asynchronous when I just need the other service to eventually know.** Every synchronous call I add is another thing that can take my service down with it — which is exactly the failure they asked about next."

More: [08 — Microservices Basics](#).

12. How do you handle error cascading?

Asked as "how to error case casting" — this is cascading failure: A is slow/down, B blocks on A, B's threads pile up, B goes down, and it walks up the chain.

The mechanism to describe: service A hangs → B's calls to A never return → B's thread pool fills with blocked threads → B stops serving *unrelated* requests → B's callers time out → the whole graph is down because of one leaf. That's a cascading failure, and the cure is **failing fast instead of waiting**.

The five defences, in order of importance:

1. Timeouts — the non-negotiable one.

"A call without a timeout is an unbounded resource leak. The default `RestTemplate` has *no* timeout — it waits forever."

```
@Bean
RestClient restClient(RestClient.Builder b) {
    var f = new SimpleClientHttpRequestFactory();
    f.setConnectTimeout(Duration.ofSeconds(2));
    f.setReadTimeout(Duration.ofSeconds(3));
    return b.requestFactory(f).build();
}
```

2. Circuit breaker — after N% failures the breaker **opens** and calls fail instantly without touching the network. **CLOSED** → **OPEN** → **HALF_OPEN** (a few trial calls) → back to **CLOSED** or **OPEN**.

```
@CircuitBreaker(name = "orderService", fallbackMethod = "orderFallback")
@Retry(name = "orderService")
@TimeLimiter(name = "orderService")
public OrderDto getOrder(Long id) { return orderClient.getOrder(id); }

private OrderDto orderFallback(Long id, Throwable t) {
    log.warn("order-service unavailable, serving cached/degraded response", t);
    return OrderDto.unavailable(id);           // degrade, don't die
}
```

```
resilience4j.circuitbreaker.instances.orderService:
  slidingWindowSize: 20
  failureRateThreshold: 50
  waitDurationInOpenState: 10s
  permittedNumberOfCallsInHalfOpenState: 3
```

3. Retry — with **exponential backoff + jitter**, and only for *idempotent* operations.

▲ "Naive retries make a cascade **worse** — you're adding load to a service that's already failing. Retry only safe operations, cap the attempts, and put the retry *inside* the circuit breaker."

4. Bulkhead — isolate thread pools/semaphores per downstream, so a hang on the reporting service can't consume the threads that serve checkout.

```
(@Bulkhead(name = "..."))
```

5. Fallback / graceful degradation — cached data, a default, a partial response, or a clear "temporarily unavailable" flag in the payload. **The user gets a degraded feature, not a 500 page.**

Plus the plumbing that makes errors legible:

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ServiceUnavailableException.class)
    public ResponseEntity<ProblemDetail>
    handle(ServiceUnavailableException e) {
        var pd =
        ProblemDetail.forStatusAndDetail(HttpStatus.SERVICE_UNAVAILABLE,
        e.getMessage());
        pd.setProperty("traceId", MDC.get("traceId"));
        return ResponseEntity.status(503).body(pd);           // RFC 7807
    }
}
```

Use **ProblemDetail** (RFC 7807, built into Spring 6) and propagate a **trace ID** on every hop, so one failure can be followed across five services.

■ **The sentence that lands it:** "Cascading failure is a **latency** problem before it's an error problem. Slow is worse than down, because down at least fails fast. So: timeout everything, open a breaker when a dependency misbehaves, and degrade the feature instead of the service."

13. Two microservices, a network failure in between — A errors, how do you inform B?

This was the sharpest question of the round. Start by making the question precise, because the right answer depends on which one they mean:

"Two readings — do you mean **A calls B and the call fails**, so how does A find out and cope? Or **A has failed and B needs to know about it**? Let me take both."

Reading A — the call fails: how does the caller cope?

The caller **cannot distinguish** these three cases from a timeout:

- 1. the request never arrived,
- 2. it arrived and B processed it but the response was lost,
- 3. B genuinely errored.

That is the fundamental point, and stating it is what separates a real answer from a textbook one. Consequences:

Concern	What you do
Detect	Connect + read timeouts (never infinite)
Recover	Retry with backoff + jitter — but only if the operation is idempotent
Make it safe to retry	Idempotency key — the caller sends Idempotency-Key: <uuid> , B stores it and returns the original result on a repeat. This is how payment APIs do it
Stop hammering	Circuit breaker — fail fast while B is unhealthy
Keep serving	Fallback : cached value, default, or a partial response
Don't lose the work	Write the intent to an outbox / queue and complete it later

Concern	What you do
See it	Trace ID + metrics + an alert when the breaker opens

Reading B — how do you *reliably* inform the other service?

The honest answer: a synchronous HTTP call is the wrong tool for "tell B something reliably", because the network is exactly the thing that failed. You need to make the message survive the failure:

- 1. **Asynchronous messaging (Kafka / RabbitMQ / SQS).** A publishes an event; the broker persists it; B consumes when it's healthy. The network failure becomes a delay, not a lost message. Consumers must be **idempotent** (dedupe on event ID) because brokers are at-least-once.
- 2. **The Transactional Outbox pattern — the answer that wins this question.**

"The dangerous case is: A commits its DB transaction and then the publish fails, so A's state says one thing and B never hears about it. You can't put a DB commit and a broker publish in one atomic transaction. So A writes the event into an **outbox table in the same local transaction** as the business change. A separate relay (a poller, or CDC with Debezium) reads that table and publishes to the broker, retrying until it succeeds. Atomicity is preserved with a single local transaction, and the message is guaranteed to go out **at least once**."

```
@Transactional
public void placeOrder(Order order) {
    orderRepo.save(order); // business
    state
    outboxRepo.save(new OutboxEvent("OrderPlaced", toJson(order))); //
    same transaction
} // both commit or neither does – the relay publishes later
```

- 3. **Dead Letter Queue** — a message that fails repeatedly goes to a DLQ instead of blocking the partition, and gets alerted on and replayed after a fix.

4. If it must stay synchronous: propagate a meaningful failure — `503` with `Retry-After`, a `ProblemDetail` body with the trace ID, and an error code B can branch on. Never swallow it and return `200`.

5. Saga / compensation — if A succeeded and B can't be reached at all, run the **compensating transaction** (cancel the reservation, refund the payment) so the system converges back to a consistent state. There's no 2PC across services; you get consistency by compensation.

6. Health + observability — Actuator health endpoints, the breaker's state exposed as a metric, tracing across the hop, and an alert on breaker-open so a human knows.

◆ **The 30-second version if they want it short:**

"I'd set timeouts and wrap the call in a circuit breaker with a fallback, so A degrades instead of hanging. For actually *informing* B I wouldn't rely on a synchronous call at all — I'd publish an event through Kafka using the **transactional outbox** pattern, so the event is written in the same DB transaction as the business change and a relay retries until the broker accepts it. B consumes it idempotently when it's back, failures land in a DLQ, and anything already committed that can't complete gets a compensating transaction."

14. Dev / QA / Prod — different properties and a different DB per environment

Asked as: "QA and prod and dev all the properties — one environment's DB data has been changed, how is that handled in Spring Boot? Different properties in different environments."

Level 1 — profile-specific property files (the baseline answer).

```
src/main/resources/  
├─ application.yml           ← common to every environment  
├─ application-dev.yml  
├─ application-qa.yml  
└─ application-prod.yml
```

```
# application.yml
spring:
  application.name: employee-service
  profiles.active: ${SPRING_PROFILES_ACTIVE:dev}
  jpa.open-in-view: false
```

```
# application-qa.yml
spring:
  datasource:
    url: jdbc:postgresql://qa-db:5432/employees
    username: ${DB_USER}
    password: ${DB_PASSWORD}
    jpa.hibernate.ddl-auto: validate
  logging.level.root: INFO
```

Activate it — **never by editing the file**: `--spring.profiles.active=qa` , or `SPRING_PROFILES_ACTIVE=qa` as an env var, or `-Dspring.profiles.active=qa` .

Level 2 — profile-specific beans.

```
@Bean @Profile("dev")           DataSource h2()    { ... }
@Bean @Profile({"qa","prod"})   DataSource pg()   { ... }
@Bean @Profile("!prod")         SeedDataLoader seed() { ... } //
                                never seed prod
```

Also `@ConfigurationProperties` for typed config, and profile groups:
`spring.profiles.group.prod: prod,monitoring,cloud` .

Level 3 — the ordering rule (this is the real answer to "how is it handled").

Spring Boot's **externalized configuration order** — later wins:

```
defaults in code → application.yml → application-{profile}.yaml → OS
environment variables → command-line arguments → config server / Vault.
```

So you never rebuild the artifact per environment. **One jar, many environments** — the environment supplies the values. `SPRING_DATASOURCE_URL` as an env var overrides everything in the jar (relaxed binding maps `SPRING_DATASOURCE_URL` → `spring.datasource.url`).

Level 4 — centralized config for microservices.
Spring Cloud Config Server backed by a git repo (`employee-service-qa.yml`), so config is versioned, auditable and shared across services; `/actuator/refresh` or Spring Cloud Bus re-reads it **without a restart**, and `@RefreshScope` beans get rebuilt. In Kubernetes the equivalent is **ConfigMaps + Secrets**.

Level 5 — secrets and schema, the two things a senior adds:
- **Secrets never live in the repo.** Vault / AWS Secrets Manager / K8s Secrets / env vars. `spring.datasource.password` in git is an instant red flag.
- "The DB data/schema changed" → **schema migrations. Flyway** or **Liquibase**, versioned SQL in source control, applied automatically on startup in order, so dev/QA/prod converge to the same schema. And `spring.jpa.hibernate.ddl-auto`: `update` in dev at most, `validate` in QA/prod, never `update` or `create-drop` — that's the answer to "the DB changed and now things break".

◆ **Summary sentence:** "Same artifact everywhere. Profiles pick the property file, environment variables override anything environment-specific, secrets come from a secret manager, and **Flyway** keeps the schema identical across dev, QA and prod so a change in one environment is a versioned migration, not a surprise."

Part 4 — Angular

15. `ngOnInit` vs constructor

Repeat — lifecycle hooks have now come up in five separate rounds.

	constructor	ngOnInit
Whose	TypeScript/JS — runs when the class is instantiated	Angular — a lifecycle hook (<code>OnInit</code>)
When	At creation, before inputs are set	After the first <code>ngOnChange</code>s , i.e. once <code>@Input</code> s are bound

	constructor	ngOnInit
Use for	Dependency injection only — assigning services to fields	Initialization: API calls, subscriptions, form setup, reading <code>@Input</code> s
<code>@Input()</code> values	x undefined	✓ available
Runs how often	Once	Once
Testable	Hard to control	✓ Called explicitly in tests, easy to control

```

@Component({ selector: 'app-employee', standalone: true, templateUrl: './employee.html' })
export class EmployeeComponent implements OnInit, OnDestroy {
  @Input() employeeId!: number;
  employee?: Employee;
  private destroy$ = new Subject<void>();

  constructor(private employeeService: EmployeeService) {
    // console.log(this.employeeId) → undefined. DI only.
  }

  ngOnInit(): void {
    this.employeeService.get(this.employeeId) // employeeId is set
    now
      .pipe(takeUntil(this.destroy$))
      .subscribe(e => this.employee = e);
  }

  ngOnDestroy(): void { this.destroy$.next(); this.destroy$.complete(); }
}

```

The "why does it matter" sentence: "Putting an HTTP call in the constructor makes the component hard to test and — more importantly — it runs before `@Input` s exist, so `employeeId` is `undefined` and you fetch the wrong thing. Separating construction from initialization is exactly why the hook exists."

Full hook order (worth having ready — it's the guaranteed follow-up):

`constructor` → `ngOnChanges` → `ngOnInit` → `ngDoCheck` →
`ngAfterContentInit` → `ngAfterContentChecked` → `ngAfterViewInit` →

`ngAfterViewChecked` → (repeat the Check hooks on every CD cycle) → `ngOnDestroy`.

◆ **Modern add-on:** with signals, `inject()` in a field initializer replaces constructor injection, and `effect()` / `afterNextRender()` cover cases that used to need `ngAfterViewInit`. `DestroyRef` + `takeUntilDestroyed()` replaces the `destroy$` subject. Mentioning that shows you're current.

16. Decorators

Definition: a decorator is a **function that attaches metadata to a class, property, method or parameter**. Angular's compiler reads that metadata to know how to build and wire things. In TypeScript they're `@`-prefixed; Angular's own are plain functions that store metadata (historically via `reflect-metadata`, now compiled ahead-of-time by the Angular compiler).

The four kinds, with Angular's:

Kind	Angular examples
Class	<code>@Component</code> , <code>@Directive</code> , <code>@Pipe</code> , <code>@Injectable</code> , <code>@NgModule</code>
Property	<code>@Input</code> , <code>@Output</code> , <code>@ViewChild</code> , <code>@ViewChildren</code> , <code>@ContentChild</code> , <code>@HostBinding</code>
Method	<code>@HostListener</code>
Parameter	<code>@Inject</code> , <code>@Optional</code> , <code>@Self</code> , <code>@SkipSelf</code> , <code>@Host</code>

```

@Component({
  selector: 'app-counter',
  standalone: true,
  template: `<button (click)="inc()">{{ count }}</button>`,
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class CounterComponent {
  @Input() start = 0;
  @Output() changed = new EventEmitter<number>();
  @HostBinding('class.active') isActive = false;
  @HostListener('mouseenter') onEnter() { this.isActive = true; }
}

```

A custom decorator — good to have if they push:

```

export function LogTime() {
  return function (target: any, key: string, descriptor:
PropertyDescriptor) {
    const original = descriptor.value;
    descriptor.value = function (...args: any[]) {
      const t0 = performance.now();
      const result = original.apply(this, args);
      console.log(`${key} took ${performance.now() - t0}ms`);
      return result;
    };
    return descriptor;
  };
}

```

▲ Two precision points that read as senior:

- **@Injectable({ providedIn: 'root' })** is what makes a service **tree-shakable** — if nothing injects it, it's dropped from the bundle. That's better than listing it in a module's **providers**.
- Angular's decorators are TypeScript's **legacy/experimental** decorators (**experimentalDecorators**), not the newer ECMAScript stage-3 proposal — and Angular 16+ is moving away from them anyway (**input()**, **output()**, **viewChild()** **signal functions** replace **@Input** / **@Output** / **@ViewChild**).

17. Interceptor

Repeat — interceptors have now been asked in four rounds.

Definition: an `HttpInterceptor` sits in the `HttpClient` pipeline and can inspect or transform **every outgoing request and every incoming response**, in one place, without touching a single service.

Functional interceptor (Angular 15+, the modern form):

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.token;

  // requests are IMMUTABLE – you must clone
  const authReq = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(authReq).pipe(
    retry({ count: 2, delay: 1000 }),
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401) auth.handleExpiredToken(); // see Q18
      return throwError(() => err);
    })
  );
};

bootstrapApplication(AppComponent, {
  providers: [provideHttpClient(withInterceptors([authInterceptor,
    loadingInterceptor]))]
});
```

Class-based (the older form, still very common in real codebases):

```

@Injectables()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler):
    Observable<HttpEvent<any>> {
    return next.handle(req.clone({ setHeaders: { Authorization: `Bearer $
    {this.token}` } })));
  }
}
// NgModule: { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor,
multi: true }

```

What they're used for — the list they want:

| Use | How |

---|---

| **Auth** | Attach the JWT to every request |

| **Error handling** | Central `catchError` → toast, log, redirect on 401/403 |

| **Loading spinner** | Increment a counter on request, decrement on finalize |

| **Logging / timing** | Log method, URL, duration, status |

| **Retry** | `retry({ count, delay })` on transient failures |

| **Caching** | Return a cached `HttpResponse` for GETs |

| **Headers** | Correlation ID, locale, tenant, API version |

| **URL rewriting** | Prefix a relative URL with the environment's API base |

▲ The three details that separate a used-it answer from a read-it one:

1. **HttpRequest** is immutable — you *must* `req.clone()`. Mutating it does nothing.
2. **multi: true** — without it your interceptor *replaces* the array instead of appending.
3. **Order matters and it's asymmetric**: interceptors run in registration order on the way **out**, and in **reverse** order on the way **back**. So put the auth interceptor before the logging one if you want the header logged.

18. The token has expired — how do you notify the user?

Answer as a flow, not a feature. Three signals that a token expired, and a chosen response for each:

1. Detect it reactively — the 401 in an interceptor (the primary mechanism):

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const toast = inject(ToastService);

  return next(addToken(req, auth.token)).pipe(
    catchError((err: HttpResponse) => {
      if (err.status === 401) {
        return auth.refreshToken().pipe(
          // 2. try a
          silent refresh first
          switchMap(newToken => next(addToken(req, newToken))),
          catchError(() => {
            // refresh
            also failed → really expired
            auth.logout();
            toast.warn('Your session has expired. Please sign in again.');
```

2. Detect it proactively — decode the JWT **exp** claim. The token carries **exp** (seconds since epoch), so the client knows when it dies without asking the server:

```
const { exp } = jwtDecode<{ exp: number }>(token);
const msLeft = exp * 1000 - Date.now();
timer(msLeft - 60_000).subscribe(() => this.warnSessionExpiring()); //
warn 1 min before
```

That gives you the **"Your session expires in 60 seconds — [Stay signed in]"** dialog, which is the *good* UX answer.

3. Guard the route — **CanActivate** checks validity before navigation, so an expired user never reaches a protected page and never sees a broken screen:

```
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService); const router = inject(Router);
  return auth.isTokenValid()
    ? true
    : router.createUrlTree(['/login'], { queryParams: { returnUrl:
state.url } });
};
```

Handle the refresh stampede — the follow-up that catches people out:

"If five requests fail with 401 at the same moment, a naive interceptor fires five refresh calls. I gate it with a `BehaviorSubject<string|null> + isRefreshing` flag: the first 401 triggers the refresh, the rest **wait on the subject** with `filter(t => t !== null), take(1), switchMap(retry)`. Same technique as the duplicate-click problem Altimetrik asked me last time."

What the user should actually see (say this — it's the part they asked):

- A **non-blocking toast/snackbar**: *"Your session has expired. Please sign in again."*
- Redirect to `/login` with `returnUrl`, so after re-login they land back where they were — never on the home page.
- **Preserve unsaved work** (draft in a store/ `sessionStorage`) so re-authenticating doesn't lose their form.
- Clear the stale token from storage so nothing retries with it.

◆ **Security second sentence**: "Access tokens should be short-lived (~15 min) with a refresh token in an **httpOnly, Secure, SameSite cookie** rather than `localStorage`, because `localStorage` is readable by any XSS. And the client-side `exp` check is UX only — the **server** is the authority; it must reject the expired token regardless."

19. How do you redirect to a page?

Five ways — name the one you'd use and why.

1. Declarative, in the template:

```
<a routerLink="/employees" [queryParams]="{ page: 1 }"
  routerLinkActive="active">Employees</a>
<a [routerLink]="['/employee', employee.id]">Details</a>
```

2. Programmatically, in a component/service:

```
private router = inject(Router);

this.router.navigate(['/employee', id]); // array
= segments
this.router.navigate(['../sibling'], { relativeTo: this.route }); //
relative
this.router.navigateByUrl('/employees?page=2#top'); //
absolute URL string
this.router.navigate(['/login'], { queryParams: { returnUrl }, replaceUrl:
true });
```

replaceUrl: true replaces the history entry — use it on a login redirect so **Back** doesn't return to the dead page.

3. In the route config — a redirect route:

```
export const routes: Routes = [
  { path: '', redirectTo: '/dashboard', pathMatch: 'full' }, //
  pathMatch is required here
  { path: 'employees', loadComponent: () => import('./employees/
list').then(m => m.ListComponent) },
  { path: '**', component: NotFoundComponent }, //
wildcard LAST
];
```

▲ **pathMatch: 'full' on the empty-path redirect** — with the default **'prefix'**, **''** matches every URL and you get an infinite redirect loop. That's a classic interview trap.

4. From a guard — returning a **UrlTree** (the preferred modern way):

```
return auth.isLoggedIn() ? true : router.createUrlTree(['/login']);
```

Cleaner than calling `router.navigate()` and returning `false`, because it cancels the current navigation atomically.

5. Leaving the app entirely: `window.location.href = '...'` — only for an external URL or an OAuth handoff; it does a full page reload and destroys the SPA state.

◆ **Second sentence:** "I'd add a `CanDeactivate` guard on forms so a redirect can't silently discard unsaved changes, and I subscribe to `Router.events` (`NavigationStart` / `NavigationEnd`) for a global loading bar and analytics."

20. Data binding

Repeat topic. **Definition:** data binding synchronizes the **component class** and the **template**, so you never touch the DOM by hand.

The four types — this is the whole answer, learn the table:

Type	Syntax	Direction	Example
Interpolation	<code>{{ }}</code>	Component → View	<code><h1>{{ employee.name }}</h1></code>
Property binding	<code>[prop]</code>	Component → View	<code> <button [disabled]="!form.valid"></code>
Event binding	<code>(event)</code>	View → Component	<code><button (click)="save()"> <input (input)="onInput(\$event)"></code>
Two-way binding	<code>[(ngModel)]</code> / <code>[(x)]</code>	↔ Both	<code><input [(ngModel)]="name"></code>

Two-way binding is not magic — say this:

```
<!-- these two are exactly equivalent - "banana in a box" is just sugar -->
<input [(ngModel)]="name">
<input [ngModel]="name" (ngModelChange)="name = $event">
```

And it works on **your own** component if you follow the naming convention: an `@Input() x` plus an `@Output() xChange` gives you `[(x)]`.

```
@Input() count = 0;
@Output() countChange = new EventEmitter<number>(); // must be
<inputName>Change
// parent: <app-counter [(count)]="total"></app-counter>
```

Precision points:

- `[disabled]="x"` binds the **DOM property**; `disabled="{{x}}"` sets the **attribute** — different things. For attributes with no DOM property, use `[attr.colspan]`, plus `[class.active]`, `[style.width.px]`.
- **Interpolation is sanitized** — Angular escapes HTML, which is your default XSS protection.
- **✗ Don't call methods in a template** (`{{ getTotal() }}`) — it re-runs on **every** change-detection cycle. Use a pure pipe, a precomputed field, `async` on an observable, or a `computed()` signal.
- `[(ngModel)]` needs `FormsModule` and belongs to **template-driven** forms; reactive forms use `formControlName` instead — don't mix them on one control.

◆ **Modern closer:** "With **signals** (Angular 16+) the read side becomes `{{ count() }}`, `model()` gives a two-way signal input, and change detection can be **zoneless** — Angular knows exactly which signal changed instead of checking the whole tree." Full file: [25 — Angular Binding & Forms](#).

21. Lazy loading

Repeat topic. **Definition:** loading a feature's JavaScript **only when the user navigates to it**, instead of shipping everything in the initial bundle. The Angular CLI splits each lazy route into its own **chunk**.

Standalone components (Angular 15+, current):

```
export const routes: Routes = [
  { path: 'employees',
    loadChildren: () => import('./employees/employees.routes').then(m =>
m.EMPLOYEE_ROUTES) },

  { path: 'profile',
    loadComponent: () => import('./profile/profile.component').then(m =>
m.ProfileComponent) },
];
```

NgModule style (older codebases):

```
{ path: 'admin', loadChildren: () => import('./admin/admin.module').then(m
=> m.AdminModule) }
```

▲ **Never also import the lazy module in `AppModule`** — that eagerly bundles it and silently kills the lazy loading. That's the single most common real-world mistake, and a great thing to volunteer.

Preloading — the nuance that shows depth:

```
provideRouter(routes, withPreloading(PreloadAllModules))
```

"Plain lazy loading trades a fast first paint for a delay on the *first* navigation into the feature. `PreloadAllModules` gets both: boot with the small bundle, then quietly download the rest while the user is idle. For a big app I'd write a **custom preloading strategy** that preloads only routes flagged `data: { preload: true }`, so the admin module isn't downloaded for regular users."

Benefits: smaller initial bundle → faster **FCP/TTI**, less parse/execute on mobile, per-feature caching (deploying one feature invalidates one chunk), and it maps cleanly onto team boundaries.

Disadvantages — always give one, this exact follow-up was asked at Photon:

- A **delay on first navigation** into the feature (mitigate with preloading + a loading indicator).
- **Duplicated dependencies** across chunks if shared code isn't factored into a shared module/library.
- A service **providedIn** a lazy module gets its own instance in that module's injector — not the root singleton you might expect.
- Harder debugging: a chunk can 404 after a redeploy (**ChunkLoadError**) — handle it with an error handler that reloads the page.

Beyond routes:

- **@defer** blocks (Angular 17+): `@defer (on viewport) { <app-heavy /> }`
`@placeholder { ... }` — deferred loading at the **template** level, no route needed.
- **ngx-loadable** /dynamic `import()` for a heavy library (charting, PDF) loaded on demand.
- Images: `<img ngSrc="..." loading="lazy">` with `NgOptimizedImage`.

Part 5 — Live coding

22. Live coding — highest-paid employee in each department

▲ This is the *same* problem Altimetrik asked you in the first round (see [26 §Altimetrik Q1](#)) — and the SQL version was asked at Mphasis. It is now a 3-round repeat. Type it from memory until it's automatic.

The given data:

```
List<Employee> employees = Arrays.asList(
    new Employee(1, "Abraham", 29, "IT", "Mumbai", 20000, "Male"),
    new Employee(2, "Mary", 27, "Sales", "Chennai", 25000, "Female"),
    new Employee(3, "Joe", 28, "IT", "Chennai", 22000, "Male"),
    new Employee(4, "John", 29, "Sales", "Gurgaon", 29000, "Male"),
    new Employee(5, "Liza", 25, "Sales", "Bangalore", 32000, "Female"),
    new Employee(6, "Peter", 27, "IT", "Mumbai", 31500, "Male"),
    new Employee(7, "Harry", 30, "IT", "Kochi", 21000, "Male"));
```

Required output: department → employee **name** (not the whole object).

```
IT    -> Peter
Sales -> Liza
```

✓ **The answer to write (department → name, no **Optional**)**

```
Map<String, String> topEarnerByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.collectingAndThen(

Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary)),
        emp -> emp.map(Employee::getName).orElse("N/A"))));

topEarnerByDept.forEach((dept, name) -> System.out.println(dept + " -> " +
name));
// IT    -> Peter
// Sales -> Liza
```

Why `collectingAndThen` : `maxBy` returns `Optional<Employee>`, because a group *could* be empty in the general contract. `collectingAndThen` applies a finisher to unwrap it — and since we only need the name, we `map(Employee::getName)` in the same step. **Say this out loud while you type it; it's the exact follow-up they asked last time.**

Variants — have these ready, they *will* push

Keep the whole employee object:

```
Map<String, Employee> topByDept = employees.stream()
    .collect(Collectors.toMap(
        Employee::getDepartment,
        Function.identity(),
        BinaryOperator.maxBy(Comparator.comparingDouble(Employee::getSalary))));
```

◆ **This is the cleanest version** — a 3-arg **toMap** with a merge function, no **Optional** anywhere. Naming it as the alternative is a strong move.

Just the max salary per department:

```
Map<String, Double> maxSalary = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment,
        Collectors.summarizingDouble(Employee::getSalary))) //
count/sum/min/avg/max in one pass
    .entrySet().stream()
    .collect(Collectors.toMap(Map.Entry::getKey, e ->
e.getValue().getMax()));

// or simply:
Map<String, Optional<Double>> m = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment,
        Collectors.mapping(Employee::getSalary,
Collectors.maxBy(Double::compare))));
```

Handling ties (two people on the same top salary) — the sharpest follow-up:

```
Map<String, List<String>> allTopEarners = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment))
    .entrySet().stream()
    .collect(Collectors.toMap(Map.Entry::getKey, e -> {
        double max =
e.getValue().stream().mapToDouble(Employee::getSalary).max().orElse(0);
        return e.getValue().stream()
            .filter(emp -> emp.getSalary() == max)
            .map(Employee::getName)
            .toList();
    }));
```

▲ **maxBy** keeps **only one** on a tie (the first encountered). Volunteering that shows you thought past the happy path.

Sorted output / `TreeMap` (grouping's 3-arg form):

```
Map<String, String> sorted = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment, TreeMap::new,
        Collectors.collectingAndThen(
            Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary)),
            e -> e.map(Employee::getName).orElse("N/A"))));
```

The pre-Java-8 version, in case they ask "without streams":

```
Map<String, Employee> top = new HashMap<>();
for (Employee e : employees) {
    Employee current = top.get(e.getDepartment());
    if (current == null || e.getSalary() > current.getSalary())
        top.put(e.getDepartment(), e);
}
```

The SQL version (asked at Mphasis — same problem, different language):

```
SELECT department, name FROM (
    SELECT department, name, salary,
        RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
    FROM employee
) t WHERE rnk = 1;          -- RANK keeps ties, ROW_NUMBER keeps exactly
one
```

Complexity: $O(n)$ single pass, $O(d)$ space where d = number of departments. Say it — it takes two seconds and it's free credibility.

■ More of the same family: [22 — Java Streams: 20 coding problems](#) · [36 — SQL §the salary questions](#).

◆ What this round tells you

1. Altimetrik goes deep on Spring wiring, not Spring trivia. Component scanning across packages, `@Qualifier` vs `@Primary`, the repository hierarchy, the *classes* involved in pagination — none of these can be answered from a "top

50 Spring Boot questions" blog. They're what you know from building. **Study 06**
— **Spring Boot** at the mechanism level.

2. The same problem came back. Highest-paid-employee-per-department was asked at Altimetrik R1, in SQL at Mphasis, and again here. Three rounds. Along with lifecycle hooks (**5 rounds**), interceptors (**4**), data binding/forms (**4**) and lazy loading (**3**). **The bank really is ~25 questions.**

3. Microservices questions are now failure-mode questions. Not "what is a microservice" but "*the network fails — what happens?*" The vocabulary that answers those: **timeout, circuit breaker, retry with backoff, idempotency key, transactional outbox, DLQ, compensating transaction.** Have all seven ready.

3b. They opened on Java 16/17 language features. Records first, pattern matching second — modern-Java questions are now an opener, not a bonus. **12**
— **Java 17 Features** is no longer optional.

4. Angular was asked at breadth, Spring at depth. With a "Fullstack Java + Angular" title, the Java/Spring side is where the round is won or lost. Revise in that order.

▲ **Mark your own fumbles**

Go back through the 22 and tag each one ✓ **answered well** / ▲ **fumbled** / ✗ **blank** — *today, while you still remember*. The ▲ and ✗ rows are your study list for the next round.

Q	Topic	Self-rating
1–2	Records , pattern matching	
3–7	Spring annotations, scanning, DI	
8–10	Spring Data, pagination	
11–13	Microservices communication & failure	
14	Profiles & environments	

Q	Topic	Self-rating
15–21	Angular	
22	Streams coding	

Related files: [26 — Companies \(the recall log\)](#) · [06 — Spring Boot](#) · [08 — Microservices](#) · [24 — ORM/JPA](#) · [04 — Angular](#) · [25 — Binding & Forms](#) · [20 — RxJS](#) · [22 — Java Streams](#) · [12 — Java 17](#) · [36 — SQL](#)

◆ IQVIA — Full Prep Pack (Angular / Ionic Technical Lead)

Interview: coming days. This file is built **directly from the 10 areas they told you to be comfortable with** — nothing more, nothing less. Every area is a Part below, in their order.

▲ **Read this first.** You were rejected at Altimetrik today (**31 Aug** — the 22 questions are logged in **37**). That round was lost on **Spring depth**, not on effort. IQVIA is a *different shape*: the checklist they gave you is **~75% frontend (Angular / TS / RxJS / Ionic)** — which is your actual strongest ground — plus a **Technical Lead** layer that is about judgement, not syntax. **This one plays to you.** Prepare it in the order below and you walk in with an advantage you did not have today.

Know who IQVIA is — 60 seconds that changes the round

IQVIA is a **healthcare data + clinical research company** (formed from the IMS Health / Quintiles merger). Their software handles **clinical trial data, patient data and real-world health evidence**. That single fact should colour half your answers:

Because it's healthcare data...	...say this, unprompted
PHI/PII everywhere	"I'd never log a patient identifier — I log a correlation ID and the record's surrogate key"
HIPAA / GDPR	Consent, data minimisation, right to erasure, data residency, pseudonymisation
FDA-regulated systems (21 CFR Part 11 / GxP)	Audit trails, e-signatures, validated releases — "in a regulated system I can't just hotfix prod; the change goes through a controlled, documented release"

Because it's healthcare data...	...say this, unprompted
Trials run offline / on tablets in clinics	This is <i>exactly</i> why they want Ionic + Capacitor — offline storage, sync, network handling
Long-lived systems	Technical debt, upgrade paths, code quality — the Tech Lead section

■ **Use it once, early:** "I understand this is healthcare data, so I'd treat auditability and PHI handling as functional requirements, not afterthoughts." That one sentence marks you as a lead rather than a senior developer.

✓ **Readiness self-audit — tick these off as you go**

Rate each ✓ **can teach it** / ▲ **can define it** / ✗ **blank**. Study the ✗ and ▲ rows first.

#	Area	Part	Your rating
1	Angular — components → Signals → performance/leaks	Part 1	
2	TypeScript / JavaScript	Part 2	
3	RxJS / NgRx	Part 3	
4	Ionic + Capacitor	Part 4	
5	REST + Security	Part 5	
6	MongoDB + PostgreSQL/SQL	Part 6	
7	Microservices	Part 7	
8	Git + CI/CD + Cloud	Part 8	

#	Area	Part	Your rating
9	Technical Lead questions	Part 9	
10	AI-assisted development	Part 10	

◆ **Areas 9 and 10 are where this job is won.** Every candidate can define a **BehaviorSubject** . Far fewer can describe a production incident they led, or how they review AI-generated code. **Do not leave those two to the night before.**

● **YOUR EVIDENCE — a real file for every one of their 10 areas**



This is the difference between "I know Angular" and "here's where I did it."
Every row below is verified in `/Users/safi/workspace/robogebra-workspace` .
Full detail: **39 — RoboGebra Code Examples**.

Their area	Your evidence	The line to say ★
1. Angular	163 components · 89 services · 27 resolvers · 4 guards · Angular 18.2	"Three Angular apps — the mobile one is on 18.2, the two web apps on 16.2."
change detection	OnPush on 6 components + 55 trackBy	" OnPush selectively on the long progress lists and the AI editor — not globally, because on a mature codebase that ships silent UI bugs wherever an input array is mutated." ★
memory leaks	426 takeUntil · 5 takeUntilDestroyed · 311 async pipe	"Over 400 takeUntil(destroy\$) , and newer code uses takeUntilDestroyed ."

Their area	Your evidence	The line to say ★
2. TypeScript	<code>ApiResponse<T></code> + <code>static fromJSON<U>(data, mapper)</code> · typed <code>ErrorCode</code> enums	"Every paginated response is a generic <code>ApiResponse<T></code> with a static factory that takes a mapper, so the item type survives end to end." ★
3. RxJS	<code>switchMap</code> 246 · <code>BehaviorSubject</code> 122 · <code>catchError</code> 338 · <code>finalize</code> 254 · <code>forkJoin</code> 22	" <code>switchMap</code> in the auth interceptor and every search; <code>forkJoin</code> for the three parallel calls after login."
NgRx	✗ 0 usages — <code>BehaviorSubject</code> services instead	"A deliberate choice — I'd reach for NgRx when state is shared across many unrelated features." ★
4. Ionic + Capacitor	Ionic 8.7.5 · Capacitor 6.2.1 · 19 native plugins	"Push via Firebase, native Razorpay checkout, network detection, and <code>Preference</code> s rather than <code>localStorage</code> because the OS can evict <code>WebView</code> storage." ★★
5. REST + Security	JWT via AWS Cognito + JWKS · one interceptor for token + 401 refresh	"We validate against Cognito's JWKS endpoint, so the API never holds a signing secret." ★
6. MongoDB + SQL	104 <code>@Document</code> · 85 repositories · 27 <code>@Aggregation</code> · compound indexes	"The N+1 fix was one <code>\$match</code> / <code>\$in</code> pipeline instead of fifty-one queries." ★★
7. Microservices	✗ modular monolith — no Eureka/Feign/circuit breaker	"Monolith first. The boundaries are drawn; we haven't paid the operational cost." ★
8. Git + CI/CD + Cloud	Jenkins CI + CD · Docker · LocalStack for local AWS · S3 · Cognito	"LocalStack means a new developer gets S3 and Cognito without real AWS credentials." ★

Their area	Your evidence	The line to say ★
9. Technical Lead	Angular 16 migration across 4 repos - the AsyncConfig pool decision	"The hard part is never Angular — it's the third-party libraries." ★
10. AI-assisted dev	The AI explanation engine sits behind your own service	"The client never calls the model directly — that keeps the key server-side and lets us cache and normalise responses." ★

▲ The four things NOT to claim — each dies on ONE follow-up

✗ SIGNALS → 0 usages in your code ★

✓ "Studied, not shipped. Our app grew from Angular 12 and is still zone-based. I'd migrate signal INPUTS on leaf components first — lowest risk, clearest win — then work upward. What signals really unlock is zoneless change detection, and that's not something you turn on halfway." ★

✗ STANDALONE → 2 components only ★

✓ "New code is standalone-first; legacy NgModules convert when we're already touching them. A big-bang migration on a shipping product was never worth the regression risk." ★

✗ NgRx on RoboGebra → 0 usages (it's EasyVisa) ★

✓ "NgRx was EasyVisa. On RoboGebra we deliberately went simpler."

✗ MICROSERVICES → modular monolith ★

✓ "One deployment, clean domain boundaries. Monolith first."

★ In a Technical Lead interview, saying "we chose not to, and here's the trade-off" scores HIGHER than claiming the buzzword. They are testing judgement, not vocabulary.

■ Your three strongest stories — rehearse these until they're smooth ★

1. THE IONIC MOBILE APP ★ ★ ← your opener; it IS the job description
"We had a working Angular web app and needed to reach mobile without rebuilding. I built it with Ionic and Capacitor, reusing our existing services and models, with about nineteen native plugins. The genuinely new thing was the Ionic page lifecycle — pages stay alive in the navigation stack, so `ngOnInit` runs once and anything that must be fresh goes in `ionViewWillEnter`."
2. THE N+1 FIX ★ ★ ← your "hardest problem" answer
"One query plus N, because `@DBRef` resolves each reference separately. Fifty exercises meant fifty-one round trips. One aggregation pipeline with `$match` and `$in` replaced it. A document database doesn't remove the N+1 problem — it just changes what it looks like."
3. THE ANGULAR 16 MIGRATION ★ ← your LEADERSHIP answer (Tech Lead round)
"Four repositories. I planned it repo by repo and tested each before moving on, so the team was never blocked for long. The hard part is never Angular itself — it's the third-party libraries that haven't released a compatible version yet."

The three habits that decide the round

The pattern across your 10 logged rounds is consistent: **you answer question one and lose on question two**. Fix that mechanically.

1. Every answer is three sentences.

Definition → one concrete thing from your own project → one trade-off or failure mode.

"An interceptor sits in the `HttpClient` pipeline and can transform every request and response. On `RoboGebra` I used one to attach the JWT and centralise 401 handling. The gotcha is that `HttpRequest` is immutable — you must `clone()` — and interceptors run in reverse order on the response."

2. Say "I don't know" cleanly, then bridge.

"I haven't used Saga orchestration in production. I've read how it works — compensating transactions instead of a distributed commit — and the closest thing I've built is X." A clean "I don't know" costs you almost nothing. Bluffing costs you the round.

3. Lead answers, not just developer answers. For a Tech Lead role, half of every technical answer should include *how you'd get a team to do it*: the convention, the lint rule, the code-review checklist, the ADR. Say **"on my team I'd..."**, not just "you can...".

Part 1 — Angular

Their list: components, lifecycle hooks, data binding, DI, services, routing, guards, resolvers, lazy loading, interceptors, forms, decorators, change detection, Signals, performance and memory leaks.

1.1 Components

A component = **a class with a decorator + a template + styles**, the unit of UI.
Key config: `selector`, `template / templateUrl`, `styles`, `standalone`, `changeDetection`, `providers`, `imports` (standalone), `encapsulation`.

Component communication — asked in 4 of your rounds. Answer as a ladder:

Direction	Mechanism
Parent → Child	<code>@Input()</code> (or the new <code>input()</code> signal)
Child → Parent	<code>@Output()</code> <code>EventEmitter</code> (or <code>output()</code>)
Parent → Child (imperative)	<code>@ViewChild</code> to call a method
Content projection	<code><ng-content select="[header]></code>

Direction	Mechanism
Unrelated components	A shared service with a BehaviorSubject / signal
App-wide	NgRx store

```
@Component({
  selector: 'app-patient-card',
  standalone: true,
  imports: [CommonModule],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <h3>{{ patient().name }}</h3>
    <button (click)="select.emit(patient().id)">Select</button>
    <ng-content select="[actions]"></ng-content>`,
})
export class PatientCardComponent {
  patient = input.required<Patient>();           // Angular 17+ signal
  input
  select = output<string>();                       // signal output
}
```

Smart vs presentational — the architecture answer a lead is expected to give: container components inject services and hold state; presentational components take **@Input** s, emit **@Output** s, are **OnPush** and are trivially testable. *"I enforce that split in review, because it's what keeps change detection cheap and components reusable."*

1.2 Lifecycle hooks

Order — memorise it:

```
constructor → ngOnChanges → ngOnInit → ngDoCheck →
ngAfterContentInit → ngAfterContentChecked →
ngAfterViewInit → ngAfterViewChecked →
  (DoCheck / AfterContentChecked / AfterViewChecked repeat every CD
  cycle)
→ ngOnDestroy
```

Hook	Fires	Use it for
<code>constructor</code>	On instantiation	DI only — <code>@Input</code> s are <code>undefined</code> here
<code>ngOnChanges(changes)</code>	Before <code>ngOnInit</code> and on every <code>@Input</code> change	React to input changes; <code>changes.x.firstChange / previousValue</code>
<code>ngOnInit</code>	Once, after first <code>ngOnChanges</code>	API calls, subscriptions, form setup
<code>ngDoCheck</code>	Every CD cycle	Custom dirty checking — expensive, avoid
<code>ngAfterContentInit/Checked</code>	After <code><ng-content></code> projected	<code>@ContentChild</code> access
<code>ngAfterViewInit/Checked</code>	After the view + children render	<code>@ViewChild</code> DOM access, third-party libs (charts)
<code>ngOnDestroy</code>	On destruction	Unsubscribe, clear timers, remove listeners

- ▲ **The trap they love:** changing a bound value in `ngAfterViewInit` throws `ExpressionChangedAfterItHasBeenCheckedError` in dev mode. Fixes: move it to `ngOnInit`, `cdr.detectChanges()`, or `queueMicrotask`. Knowing *why* (dev mode runs a second verification pass) is the senior answer.
- ◆ **Modern note:** signals + `effect()`, `afterNextRender()` / `afterRenderEffect()` and `DestroyRef` are replacing several of these hooks.

1.3 Data binding

Type	Syntax	Direction
Interpolation	<code>{{ value }}</code>	Class → View
Property	<code>[prop]="value"</code>	Class → View
Event	<code>(click)="fn(\$event)"</code>	View → Class
Two-way	<code>[(ngModel)] / [(x)]</code>	Both

`[(x)]` is **sugar** for `[x] + (xChange)` — implement `@Input() x + @Output() xChange` (exact naming) and your own component supports it. Also `[attr.aria-label]`, `[class.active]`, `[style.width.px]`.

✗ **Never call a method in a template** (`{{ getTotal() }}`) — it re-runs on every CD cycle. Use a pure pipe, a precomputed field, `async`, or a `computed()` signal. **This is a performance answer disguised as a binding answer — say it.**

1.4 Dependency injection & services

What it is: you declare what you need in the constructor; Angular's **hierarchical injector** supplies it. That's what makes services swappable and components testable.

```
@Injectable({ providedIn: 'root' })    // tree-shakable app-wide
singleton
export class PatientService {
  private http = inject(HttpClient);    // inject() — Angular 14+, works
  in field initializers
}
```

Provider scopes:

| Where | Effect |

---|---

| `providedIn: 'root'` | One singleton, **tree-shakable** (dropped if unused) |

| `providers: []` on a component | **A new instance per component instance** |

| `providers: []` in a lazy-loaded route | One instance for that lazy injector — *not* the root singleton |

| `providedIn: 'platform' / 'any'` | Across apps on the page / per lazy module |

Injector hierarchy: element injector → parent element injectors → module/environment injector → root → platform → null (then `NullInjectorError: No provider for X`).

Modifiers: `@Optional()` `@Self()` `@SkipSelf()` `@Host()`.

Provider types — the part that shows depth:


```
{ provide: ApiUrl,          useValue: 'https://api.iqvia.internal' }
{ provide: LoggerService, useClass: ProdLoggerService }
{ provide: AuthService,    useExisting: OAuthService }          // alias,
same instance
{ provide: CONFIG, useFactory: (env: Env) => new Config(env), deps:
[Env] }
```

InjectionToken for anything that isn't a class (config objects, primitives) — **new**
InjectionToken<AppConfig>('app.config') .

◆ **Lead framing:** "DI is what lets me swap a real **PatientApiService** for a mock in tests without touching the component, and it's how I'd stub the native layer when the same Ionic code runs in a browser."

1.5 Routing, guards, resolvers

```
export const routes: Routes = [
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' },    // pathMatch
REQUIRED here
  {
    path: 'trials',
    canActivate: [authGuard],
    canMatch: [featureFlagGuard],                             // decides
if the route matches at all
    loadChildren: () => import('./trials/trials.routes').then(m =>
m.TRIAL_ROUTES),
  },
  {
    path: 'patient/:id',
    loadComponent: () => import('./patient/patient.component').then(m =>
m.PatientComponent),
    resolve: { patient: patientResolver },
    canDeactivate: [unsavedChangesGuard],
    data: { roles: ['INVESTIGATOR'] },
  },
  { path: '**', component: NotFoundComponent },                // wildcard
LAST
];
```

The guard types — know all five:

| Guard | Question it answers |

|---|---|

| **CanActivate** | May the user open this route? |

| **CanActivateChild** | ...and its children? |

| **CanDeactivate** | May they leave? (**unsaved-changes prompt**) |

| **CanMatch** | Should this route even match? (**best for role/feature-flag routing — the lazy chunk isn't downloaded if it doesn't match**) |

| **CanLoad** (*deprecated* → use **CanMatch**) | May the lazy bundle load? |

```
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService), router = inject(Router);
  return auth.isAuthenticated()
    ? true
    : router.createUrlTree(['/login'], { queryParams: { returnUrl:
state.url } });
};
```

◆ **Return a `UrlTree`, don't `navigate()` + `return false`** — it cancels the navigation atomically.

Resolvers — fetch data *before* the route activates, so the component never renders empty:

```
export const patientResolver: ResolveFn<Patient> = (route) =>
  inject(PatientService).get(route.paramMap.get('id')!).pipe(
    catchError(() => { inject(Router).navigate(['/not-found']); return
EMPTY; })
  );
// component: patient = this.route.snapshot.data['patient'];
```

▲ **The trade-off to volunteer:** a resolver **blocks navigation** — the old page stays on screen while it loads, which feels like a frozen app on a slow clinic network. *"I use resolvers for small, essential data and a skeleton/loading state for everything else."* That answer alone is worth the question.

Reading params: `paramMap` / `queryParams` as **observables** (component reused when only the id changes) vs `snapshot` (one-shot). Getting that distinction right is a common trap.

1.6 Lazy loading

```
{ path: 'reports', loadChildren: () => import('./reports/reports.routes').then(m => m.ROUTES) }
{ path: 'profile', loadComponent: () => import('./profile.component').then(m => m.ProfileComponent) }
```

- **Preloading:** `provideRouter(routes, withPreloading(PreloadAllModules))`, or a **custom strategy** that preloads only routes flagged `data: { preload: true }`.
- **▲ Never also eagerly import the lazy module/component** — it silently defeats the split.
- **Disadvantages** (always give one): first-navigation delay, duplicated deps across chunks, a service `providedIn` a lazy scope gets its **own instance**, and `ChunkLoadError` after a redeploy (handle it by reloading the page).
- **@defer (v17+)** does the same at *template* level: `@defer (on viewport) { <app-chart/> } @placeholder { ... } @loading { ... }`.

1.7 Interceptors

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const authReq = req.clone({ setHeaders: { Authorization: `Bearer ${auth.token}` } });
  return next(authReq).pipe(
    timeout(30_000),
    retry({ count: 2, delay: (err, n) => timer(500 * 2 ** n) }), // backoff
    catchError((e: HttpResponse) => handle(e)),
  );
};
provideHttpClient(withInterceptors([correlationIdInterceptor, authInterceptor, errorInterceptor]));
```

Uses: auth · central error handling · loading indicator · logging/timing · retry · caching · correlation ID · API base URL.

Three details that prove you've used them: requests are **immutable** →

`clone()` ; class-based needs `multi: true` ; order is **registration order out, reverse order back**.

1.8 Forms

	Template-driven	Reactive
Module	FormsModule	ReactiveFormsModule
Source of truth	The template	The TS class
Structure	ngModel	FormGroup / FormControl / FormArray
Validation	Directives	Functions — unit-testable
Async / dynamic	Painful	Natural
Use for	Tiny forms	Everything real

```
form = this.fb.nonNullable.group({
  email:    ['', [Validators.required, Validators.email],
  [this.uniqueEmailValidator()]],
  password: ['', [Validators.required, Validators.minLength(8)]],
  confirm:  ['',
  doses: this.fb.array<FormGroup>([]), // FormArray
  for dynamic rows
}], { validators: passwordsMatch }); // cross-
field validator

// custom sync validator
export const passwordsMatch: ValidatorFn = (g: AbstractControl) =>
  g.get('password')!.value === g.get('confirm')!.value ? null :
  { mismatch: true };

// async validator (debounced server check)
uniqueEmail(): AsyncValidatorFn {
  return c => timer(300).pipe(switchMap(() =>
    this.api.checkEmail(c.value)),
    map(taken => taken ? { taken: true } :
    null));
}
```

Know: control states (`pristine/dirty` , `touched/untouched` , `valid/invalid` , `pending`), `updateOn: 'blur'|'submit'` (a real performance lever on big forms), `setValue` (all controls, strict) vs `patchValue` (partial), `valueChanges` / `statusChanges` as observables, `markAllAsTouched()` before showing errors on submit, and **typed forms** (Angular 14+).

■ Your resume has **Angular Formly** — mention it here: *"For the clinical-form use case I'd push toward schema-driven forms (Formly / JSON-defined), because the forms change per study and you don't want a release for every field change."* For IQVIA that's an on-the-nose answer.

1.9 Decorators

Metadata functions in four flavours: **class** (`@Component` , `@Directive` , `@Pipe` , `@Injectable` , `@NgModule`), **property** (`@Input` , `@Output` , `@ViewChild` , `@HostBinding`), **method** (`@HostListener`), **parameter** (`@Inject` , `@Optional` , `@Self` , `@SkipSelf`).

Angular uses TypeScript's *legacy* decorators, and v16+ is moving to the **signal-based functions** `input()` , `output()` , `viewChild()` , `model()` instead.

1.10 Change detection — the topic that separates senior from mid

How it works: Angular runs CD **top-down from the root** whenever something *might* have changed. Historically **Zone.js** monkey-patches async APIs (events, `setTimeout` , XHR) and tells Angular "something happened — check everything".

Default vs OnPush :

Default	OnPush
--- --- ---	
When the component is checked	Every cycle, always
Only when: an <code>@Input</code> reference changes · an event fires inside it · an async pipe emits · <code>markForCheck()</code> is called · a signal it reads changes	

```
@Component({ changeDetection: ChangeDetectionStrategy.OnPush, ... })
```

▲ **OnPush** compares **@Input s** by reference. `this.items.push(x)` will not trigger it; `this.items = [...this.items, x]` will. **That is the #1 real-world OnPush bug — say it and you've answered the follow-up before it's asked.**

The escape hatches:

```
private cdr = inject(ChangeDetectorRef);
this.cdr.markForCheck();    // mark this component + ancestors dirty for
the NEXT cycle
this.cdr.detectChanges();  // run CD on this subtree NOW
this.cdr.detach();          // opt out entirely (very hot components) —
you drive it yourself
```

Running work outside Angular (the classic performance fix):

```
this.zone.runOutsideAngular(() => {
  // mousemove / animation loop / chart tick — no CD per frame
  requestAnimationFrame(loop);
});
this.zone.run(() => this.state = done);  // re-enter only when the UI
must update
```

◆ **The modern closer:** "With **signals**, Angular knows exactly which components read the value that changed, so it can do **fine-grained** updates instead of dirty-checking a tree — and that's what makes **zoneless** (`provideZonelessChangeDetection()`) possible."

1.11 Signals

▲ **HONESTY GUARD — read before answering this topic.**

`signal()` / `computed()` / `effect()` in your codebase → 0 usages ★

Learn the section below properly — you must be able to *discuss* signals for a lead role. But **do not imply you've shipped them**. The safe framing, which scores better anyway:

"Signals I've studied but haven't shipped. Our mobile app is on Angular 18 but grew from 12, so change detection is still zone-based. The migration I'd run is signal inputs on leaf components first — lowest risk, clearest win — then work upward. What signals really unlock is zoneless change detection, and that's not something you turn on halfway."

★ Naming a **migration order** is what a technical lead is expected to have. Claiming adoption invites one follow-up you can't answer.

What: a reactive primitive — a value that knows who reads it, so Angular can update precisely what changed.

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `{{ fullName() }} - {{ visits().length }} visits`,
})
export class PatientComponent {
  // inputs / outputs / queries as signals
  patient = input.required<Patient>();
  saved    = output<Patient>();

  first = signal('Ada');
  last  = signal('Lovelace');
  fullName = computed(() => `${this.first()} ${this.last()}`); // lazy +
memoised

  visits = signal<Visit[]>([]);

  constructor() {
    effect(() => console.log('patient changed', this.patient().id)); //
side effects
  }

  rename(n: string) {
    this.first.set(n); // set - replace
    this.visits.update(v => [...v, newVisit]); // update - derive from
current
  }
}
```

Primitive	Purpose
<code>signal(v)</code>	Writable state — <code>.set()</code> , <code>.update()</code> , <code>()</code> to read
<code>computed(fn)</code>	Derived, lazy, memoised ; recomputes only when a dependency changed
<code>effect(fn)</code>	Side effects when dependencies change (logging, localStorage, DOM) — not for deriving state
<code>input()</code> / <code>input.required()</code> / <code>model()</code>	Signal-based <code>@Input</code> / two-way
<code>output()</code>	Signal-era <code>@Output</code>
<code>viewChild()</code> / <code>contentChild()</code>	Signal queries
<code>linkedSignal()</code> (v19)	Writable state that resets from a source
<code>resource()</code> / <code>httpResource()</code> (v19/20, experimental)	Async data as a signal
<code>toSignal()</code> / <code>toObservable()</code>	The RxJS interop bridge

Signals vs RxJS — the question they will ask:

"Signals are for **synchronous state** — 'what is the value right now', with glitch-free derived values. RxJS is for **asynchronous streams over time** — HTTP, websockets, debouncing, cancellation, retry. In practice I use RxJS for the pipeline and `toSignal()` at the edge to feed the template."

```
readonly results = toSignal(
  toObservable(this.query).pipe(
    debounceTime(300), distinctUntilChanged(),
    switchMap(q => this.api.search(q)),
  ), { initialValue: [] });
```

◆ **Benefits to name:** no `async` pipe boilerplate, no manual unsubscribe (signals clean up with the injection context), fine-grained CD, and **zoneless** apps. `effect()` misuse — writing state from an effect — is the anti-pattern to flag.

1.12 Performance

Have **eight** ready; anyone can name two.

- 1. **OnPush everywhere** + immutable updates.
- 2. **trackBy / the new @for (item of items; track item.id)** — without it Angular destroys and rebuilds every DOM node on each change.
- 3. **Lazy load routes**, **@defer** heavy blocks, preload intelligently.
- 4. **Never call functions in templates**; use pure pipes / **computed()**.
- 5. **Virtual scrolling** (**cdk-virtual-scroll-viewport**) for long clinical lists.
- 6. **runOutsideAngular** for high-frequency events; **debounce/throttle** inputs and scroll.
- 7. **Bundle discipline** — **source-map-explorer** / **ng build --stats-json**, budgets in **angular.json** that **fail the CI build**, tree-shakable providers, drop moment.js-style heavy deps.
- 8. **Network** — cache GETs in an interceptor, **shareReplay(1)** for shared data, pagination instead of loading everything, gzip/brotli, HTTP/2.
- 9. **Rendering** — **NgOptimizedImage**, SSR/hydration if it's a public app, **@defer (on viewport)**.
- 10. **Signals / zoneless** for fine-grained updates.

■ **Lead-level answer:** "I don't optimise by instinct — I profile first with the **Angular DevTools profiler** and Chrome Performance, find whether it's change detection, bundle size or the network, and fix that. Then I add a **CI budget** so the regression can't come back."

1.13 Memory leaks — say all five sources

Leak	Fix
Unclosed subscriptions	<code>async pipe · takeUntilDestroyed() · takeUntil(destroy\$) in ngOnDestroy · take(1) / first()</code>
<code>setInterval</code> / <code>setTimeout</code>	<code>clearInterval</code> in <code>ngOnDestroy</code>

Leak	Fix
Manual DOM listeners (<code>addEventListener</code> , third-party libs, charts, maps)	Remove/ <code>destroy()</code> in <code>ngOnDestroy</code>
A long-lived service holding component references (or an ever-growing cache/ <code>BehaviorSubject</code> array)	Clear it; bound the cache
Detached DOM / global registries	Destroy plugin instances; unregister

```
export class LiveVitalsComponent {
  private destroyRef = inject(DestroyRef);
  ngOnInit() {
    this.socket.vitals$
      .pipe(takeUntilDestroyed(this.destroyRef)) // Angular 16+ – no
destroy$ boilerplate
      .subscribe(v => this.vitals.set(v));
  }
}
```

How you'd find one: Chrome DevTools **Memory** tab → heap snapshot, navigate away and back a few times, snapshot again, and compare **Detached HTMLElements** / retained component instances. Saying *how you diagnose* it, not just how you avoid it, is the lead answer.

▲ **Note:** `HttpClient` calls complete on their own, so they don't leak in the same way — but an in-flight request can still call back into a destroyed component. `takeUntilDestroyed` handles both.

■ Deeper: [04 — Angular](#) · [25 — Binding & Forms](#)

Part 2 — TypeScript / JavaScript

Their list: var/let/const, arrow functions, callbacks/promises, event bubbling/capturing, ES6, interfaces/classes, OOP, generics, any/unknown/never/void, modules, destructuring, spread/rest.

2.1 var / let / const

	var	let	const
Scope	Function	Block	Block
Hoisting	Hoisted, initialised undefined	Hoisted into the TDZ → ReferenceError	Same
Redeclare	✓	✗	✗
Reassign	✓	✓	✗ (the binding, not the contents)
On globalThis	✓	✗	✗

```
const user = { name: 'Ada' };
user.name = 'Grace';    // ✓ allowed – const freezes the binding, not the object
// user = {};           // ✗ TypeError
```

The classic output question:

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 0 1 2
(new binding per iteration)
```

- Rule: **const** by default, **let** when you must reassign, **var** never.

2.2 Arrow functions

The point is lexical **this** — an arrow doesn't have its own **this**, it closes over the enclosing scope. That's why callbacks inside a class work without **.bind(this)**.

Also: no **arguments** (use rest **...args**), **not constructible** (**new** throws), no **prototype**, can't be a generator, and **implicit return** without braces (**x => ({a: 1})** needs parens for an object literal).

✗ **Don't use an arrow as an object method** (`this` won't be the object) or as a prototype method you intend to be overridable.

2.3 Callbacks, promises, async/await, the event loop

Callback hell → **Promises** → **async/await** → **RxJS** is the arc to narrate.

```
// Promise states: pending → fulfilled | rejected (settled, immutable after)
const data = await Promise.all([getPatient(id), getVisits(id)]); // fail fast, all must succeed
await Promise.allSettled(tasks); // never rejects – status per item
await Promise.race(tasks); // first to SETTLE (resolve or reject)
await Promise.any(tasks); // first to FULFIL; AggregateError if all reject
```

```
async function load() {
  try { return await this.api.get(); }
  catch (e) { log(e); throw e; }
  finally { this.loading = false; }
}
```

Event loop — the answer that always earns points:

"The call stack runs synchronous code. Async callbacks go to queues. After the stack empties, the loop drains **all microtasks** (promise callbacks, `queueMicrotask`, `MutationObserver`) **before** the next **macrotask** (`setTimeout`, `setInterval`, I/O, UI events). That's why a promise callback beats a `setTimeout(0)`."

```
console.log('1');
setTimeout(() => console.log('2'));
Promise.resolve().then(() => console.log('3'));
console.log('4');
// 1 4 3 2
```

Sequential vs parallel — a real interview differentiator:

```
const a = await getA(); const b = await getB();      //
✗ sequential – 2× latency
const [a, b] = await Promise.all([getA(), getB()]); // ✓ parallel
```

2.4 Event bubbling & capturing

Three phases: **capture (window → target) → target → bubble (target → window)**.

```
el.addEventListener('click', fn);      // bubble phase (default)
el.addEventListener('click', fn, true); // capture phase
```

Method	Does
<code>stopPropagation()</code>	Stops the event travelling further
<code>stopImmediatePropagation()</code>	...and stops other listeners on the same element
<code>preventDefault()</code>	Cancels the default action (form submit, link) — does not stop propagation

Event delegation — one listener on the parent instead of 500 on rows; uses `event.target` and works for dynamically added elements. In Angular this is what makes `(click)` on a `@for` row cheap.

▲ `focus / blur` don't bubble (use `focusin / focusout`); `event.target` = what was clicked, `event.currentTarget` = what the listener is on.

2.5 ES6+ essentials

`let / const` · arrow functions · template literals · **destructuring** · **spread/rest** · default parameters · classes · **modules** · Promises · `Map / Set / WeakMap` · `Symbol` · generators · optional chaining `?.` · nullish coalescing `??` · logical assignment `??=` · `Object.entries/values/fromEntries` · `Array.flat/flatMap/at/includes` · `String.padStart/replaceAll` · top-level `await` · `structuredClone`.

2.6 Destructuring, spread & rest

```
const { name, age = 0, address: { city } = {}, ...rest } = patient; //
rename, default, nested, rest
const [first, , third = 'n/a'] = list;
function f({ id, active = true }: Opts) {} //
destructured params

const merged = { ...defaults, ...overrides }; // spread - later
wins
const copy = [...items, newItem]; // immutable append
(OnPush-safe!)
function sum(...nums: number[]) {} // rest - collects
```

▲ **Spread is a *shallow* copy.** `{...patient}` shares the nested `address` object. Deep copy: `structuredClone(obj)` (modern) or a library. This exact point connects to `OnPush` and `NgRx` immutability — link them out loud.

2.7 Interfaces, classes & OOP in TypeScript

```
interface Identifiable { id: string; }
interface Auditable { createdAt: Date; createdBy: string; }

abstract class BaseEntity implements Identifiable {
  protected constructor(public readonly id: string) {}
  abstract describe(): string; // subclasses must
  implement
}

class Patient extends BaseEntity implements Auditable {
  #ssn: string; // true JS private
  (runtime)
  constructor(id: string, private name: string, // parameter
  properties
    public createdAt = new Date(), public createdBy = 'system',
  ssn = '') {
    super(id); this.#ssn = ssn;
  }
  describe() { return `${this.name} (${this.id})`; } // polymorphism
  get initials() { return this.name[0]; }
}
```

type vs interface — a guaranteed question:

| **interface** | **type** |

|---|---|

| **Declaration merging** (reopens) | ✗ |

| **extends** | Intersections & |

| Objects/classes only | **Unions, tuples, primitives, mapped & conditional types**

|

| Better error messages, preferred for public APIs | Use when you need unions or type computation |

The 4 OOP pillars, in TS terms: encapsulation (**private** / **#**, getters) · inheritance (**extends**) · polymorphism (override / interface implementations) · abstraction (**abstract** , interfaces).

◆ **The senior point: TypeScript is *structurally typed*.** Two unrelated types match if their shapes match — unlike Java/C# nominal typing. And **private** / **readonly** are **compile-time only**; **#field** is enforced at runtime.

Composition over inheritance — say it: in Angular you compose services via DI far more than you extend classes.

2.8 Generics

```
function first<T>(arr: T[]): T | undefined { return arr[0]; }

interface ApiResponse<T> { data: T; total: number; page: number; }

// constraint
function pluck<T, K extends keyof T>(obj: T, key: K): T[K] { return
obj[key]; }

class Repository<T extends { id: string }> {
  private items = new Map<string, T>();
  save(item: T): void { this.items.set(item.id, item); }
  find(id: string): T | undefined { return this.items.get(id); }
}

// conditional + mapped + infer
type Unwrap<T> = T extends Observable<infer U> ? U : T;
type Optional<T> = { [K in keyof T]?: T[K] };
```

Built-in utility types to name: `Partial` `Required` `Readonly` `Pick` `Omit` `Record` `Exclude` `Extract` `NonNullable` `ReturnType` `Parameters` `Awaited`.

■ **Why generics:** "Type safety without duplication — one `ApiResponse<T>` instead of one per endpoint, and the compiler catches the mismatch instead of a runtime error in a clinical form."

2.9 `any` / `unknown` / `never` / `void`

Type	Meaning	Rule
<code>any</code>	Turn the checker off	✗ Avoid — one <code>any</code> leaks through your whole call chain
<code>unknown</code>	Type-safe <code>any</code> — you must narrow before use	✓ Use at every boundary: API responses, <code>JSON.parse</code> , <code>catch (e: unknown)</code>
<code>never</code>	A value that can never occur — a function that never returns	Exhaustiveness checking

Type	Meaning	Rule
<code>void</code>	The function returns nothing meaningful	Return type only
<code>null / undefined</code>	Absence	Turn on <code>strictNullChecks</code>

```
function handle(e: unknown) {
  if (e instanceof HttpResponse) console.log(e.status); // narrowed
  → safe
}

type Status = 'active' | 'closed' | 'pending';
function label(s: Status): string {
  switch (s) {
    case 'active': return 'Active';
    case 'closed': return 'Closed';
    case 'pending': return 'Pending';
    default: const _exhaustive: never = s; return _exhaustive; // add a
status → compile error
  }
}
```

◆ That exhaustiveness trick is the answer to "what's **never** for". Land it.

2.10 Modules

ES Modules (ESM) — the standard: `export` / `export default` / `import` / `import * as` / `dynamic import()` (this is what powers Angular lazy loading). Static, tree-shakable, hoisted, always strict mode, top-level `await`.

CommonJS — Node's `require` / `module.exports`, dynamic, not tree-shakable. Also **AMD** and **UMD** (legacy/browser bundles).

In Angular "module" also means **NgModule** (`declarations` / `imports` / `exports` / `providers`) — root, feature, shared, core, routing — increasingly replaced by **standalone components**. Answer both senses; that's exactly what Tech Mahindra asked.

▲ **Barrel files** (`index.ts`) are convenient but can defeat tree-shaking and create circular imports. Naming that is a lead-level detail.

■ Deeper: [01 — JavaScript](#) · [03 — TypeScript](#) · [19 — JS Variables](#) · [34 — Tech Mahindra round](#)

Part 3 — RxJS / NgRx

Their list: Observable vs Promise, Subject types, hot/cold, switchMap/mergeMap/concatMap/exhaustMap, forkJoin, error handling, unsubscribe/memory leaks, Store/Action/Reducer/Effect/Selector.

3.1 Observable vs Promise — asked in 4 of your rounds

	Promise	Observable
Values	One , ever	Zero to many , over time
Execution	Eager — starts at creation	Lazy — nothing runs until <code>subscribe()</code>
Cancellable	✗	✓ <code>unsubscribe()</code> (Angular aborts the XHR)
Operators	<code>.then</code> chaining only	200+ operators — map, filter, debounce, retry, combine
Retry	Manual	<code>retry()</code> / <code>retryWhen</code>
Multicast	Always one result, shared	Unicast by default; multicast with <code>Subject</code> / <code>share()</code>
Sync possible	Never (always async)	Can emit synchronously
Native	✓	Library (RxJS)

■ **The one-liner:** "A promise is a **single future value you can't cancel**; an observable is a **cancellable stream you compose**. Angular uses observables because a typeahead needs debouncing and cancellation, and a promise can't do either."

Convert: `firstValueFrom(obs$) / lastValueFrom(obs$); from(promise) .`

3.2 Subject types

Type	Behaviour	Use for
Subject	Multicast; late subscribers get nothing	Event bus, "something happened" notifications
BehaviorSubject	Requires an initial value ; replays the latest to new subscribers; <code>.value</code>	★ State in a service — current user, selected trial
ReplaySubject(n, time?)	Replays the last <i>n</i> (optionally within a time window)	Caching recent events, late-joining logs
AsyncSubject	Emits only the last value, and only on <code>complete()</code>	Rare — a one-shot computation result

```
@Injectable({ providedIn: 'root' })
export class SessionService {
  private user$ = new BehaviorSubject<User | null>(null);
  readonly currentUser$ = this.user$.asObservable(); // expose read-only
  — never the Subject
  setUser(u: User) { this.user$.next(u); }
}
```

◆ **Lead detail:** always expose `.asObservable()` (or a `readonly` signal) so consumers can't `next()` into your state. That's an encapsulation answer inside an RxJS question.

3.3 Hot vs cold

Cold = the producer is created *per subscriber* → each subscriber gets its **own independent execution**. `HttpClient` is cold — **subscribe twice, the request fires twice**.

Hot = one shared producer, subscribers share the same values, and late subscribers miss what already happened. `Subject`, DOM events, websockets.

```
const data$ = this.http.get('/api/trials').pipe(shareReplay({ bufferSize: 1, refCount: true }));
// now multiple subscribers share ONE request and late ones get the cached value
```

▲ `shareReplay(1)` without `refCount: true` keeps the source alive forever — that's a real, subtle memory leak. Naming it is a strong senior signal.

3.4 The four flattening operators — know when, not just what

Operator	Behaviour	Canonical use
<code>switchMap</code>	Cancels the previous inner observable	★ Typeahead / search, route param → fetch — you only want the latest
<code>mergeMap</code> (<code>flatMap</code>)	Runs all in parallel , order not guaranteed	Independent parallel work, e.g. uploading N files
<code>concatMap</code>	Queues — one at a time, in order	★ Writes that must not reorder — a sequence of PATCHes, offline sync queue
<code>exhaustMap</code>	Ignores new ones while one is in flight	★ Login / submit button — kills double-submit

```
this.search.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(q => this.api.search(q)),           // cancels the stale request
  takeUntilDestroyed(this.destroyRef),
).subscribe(r => this.results.set(r));
```

▲ **switchMap on a POST is a bug** — cancelling a write leaves the server state unknown. Use **concatMap** or **exhaustMap** for writes. **That sentence is the whole question.**

3.5 Combination & creation

Operator	Behaviour
<code>forkJoin</code>	Waits for all to complete , emits the last of each — like Promise.all . ▲ If any one errors, you get nothing ; if one never completes, it never emits
<code>combineLatest</code>	Emits whenever any source emits (after all have emitted at least once)
<code>zip</code>	Pairs emissions by index
<code>withLatestFrom</code>	Primary stream drives; samples the others
<code>merge</code> / <code>concat</code>	Interleave / run in sequence
<code>startWith</code> , <code>of</code> , <code>from</code> , <code>timer</code> , <code>interval</code> , <code>fromEvent</code> , <code>EMPTY</code> , <code>NEVER</code> , <code>throwError</code> , <code>defer</code>	Creation

```
forkJoin({
  patient: this.api.getPatient(id),
  visits: this.api.getVisits(id).pipe(catchError(() => of([]))), // ★
  isolate the failure
}).subscribe(({ patient, visits }) => ...);
```

◆ **"What if one forkJoin call fails?"** was asked at Photon. The answer is exactly the line above: **catchError on the inner observable** so one non-critical failure doesn't lose the whole page.

3.6 Error handling

```
this.api.get().pipe(
  timeout(30_000),
  retry({ count: 3, delay: (err, n) => n < 3 && err.status >= 500 ?
    timer(1000 * 2 ** n) : throwError(() => err) }),
  catchError(err => {
    this.toast.error('Could not load trials');
    return of([]); // recover with a fallback →
  })
)
stream CONTINUES
// or: return throwError(() => new AppError(err)); // rethrow →
stream DIES
}),
finalize(() => this.loading.set(false)), // runs on complete, error
AND unsubscribe
).subscribe();
```

▲ **An error terminates the observable.** In a long-lived stream (a `valueChanges` typeahead), put the `catchError` inside the `switchMap` — otherwise one failed search kills the whole subscription and the box stops working. **This is the single most common real RxJS bug — say it.**

```
switchMap(q => this.api.search(q).pipe(catchError(() => of([])))) // ✓
inner catch
```

Also: `EMPTY` (complete silently) vs `of([])` (emit a fallback) vs `throwError`. Plus Angular's `ErrorHandler` for a global net.

3.7 Unsubscribing & memory leaks

In order of preference:

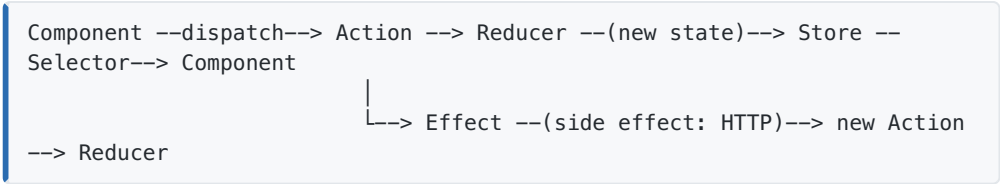
1. `async pipe` — subscribes and unsubscribes for you. *"If I can put it in the template, I do."*
2. `takeUntilDestroyed(destroyRef)` (Angular 16+) — cleanest imperative option.
3. `takeUntil(this.destroy$) + destroy$.next()` in `ngOnDestroy` — the classic. ▲ **`takeUntil` must be the last operator** in the pipe.
4. `take(1) / first()` for one-shot streams.
5. `Subscription / subscription.add()` + `unsubscribe()` in `ngOnDestroy`.

Which streams actually leak? Long-lived ones — `Subject`s in services, `valueChanges`, `router.events`, `fromEvent`, websockets, `interval`. `HttpClient` completes on its own, but the callback can still fire into a dead component.

◆ **Lead answer:** "I make this a review rule, not a memory exercise: `async` pipe by default, and the `rxjs-angular/prefer-takeuntil` ESLint rule fails the build if someone subscribes without `teardown`."

3.8 NgRx — Store / Action / Reducer / Effect / Selector

The flow (draw this):



Piece	Role	Rule
Store	Single immutable state tree	One source of truth
Action	A unique event describing <i>what happened</i>	Name it <code>[Source] Event</code> , e.g. <code>[Trials Page] Load Trials</code> — not a command
Reducer	<code>(state, action) => newState</code>	Pure. Synchronous. Immutable. No HTTP, no <code>Date.now()</code> , no mutation
Effect	Listens to actions, does the side effect, dispatches a result action	Where all async lives
Selector	Queries + derives state; memoised	Components never reach into raw state

```

// actions
export const TrialsActions = createActionGroup({
  source: 'Trials API',
  events: {
    'Load Trials': emptyProps(),
    'Load Trials Success': props<{ trials: Trial[] }>(),
    'Load Trials Failure': props<{ error: string }>(),
  },
});

// reducer
export const trialsReducer = createReducer(
  initialState,
  on(TrialsActions.loadTrials, s => ({ ...s, loading: true, error:
null })),
  on(TrialsActions.loadTrialsSuccess, (s, { trials }) => ({ ...s, loading:
false, trials })),
  on(TrialsActions.loadTrialsFailure, (s, { error }) => ({ ...s, loading:
false, error })),
);

// effect
loadTrials$ = createEffect(() => this.actions$.pipe(
  ofType(TrialsActions.loadTrials),
  switchMap(() => this.api.getTrials().pipe(
    map(trials => TrialsActions.loadTrialsSuccess({ trials })),
    catchError(err => of(TrialsActions.loadTrialsFailure({ error:
err.message }))), // ★ INSIDE
  )),
));

// selectors
export const selectTrialsState =
createFeatureSelector<TrialsState>('trials');
export const selectAllTrials = createSelector(selectTrialsState, s =>
s.trials);
export const selectActive = createSelector(selectAllTrials, ts =>
ts.filter(t => t.active));

```

▲ **The single most-asked NgRx trap:** `catchError` must be **inside** the inner `switchMap`. If it's on the outer `actions$` pipe, the first failure **kills the effect forever** and the feature silently stops working until a page reload. If you say only one thing about NgRx, say this.

Also know: [@ngrx/entity](#) ([EntityAdapter](#) for normalised collections), **Redux DevTools** time-travel, [@ngrx/component-store](#) (local, per-component state — no global boilerplate), **NgRx Signal Store** (the modern signal-based option), and meta-reducers (logging, hydration).

"When would you NOT use NgRx?" — the mature answer they want:

"NgRx is a lot of ceremony. For a small app or state used by one feature, a service with a [BehaviorSubject](#) or a [signal](#) is simpler and I'd argue for it in review. I reach for NgRx when state is **shared across unrelated features, survives navigation, needs debuggable/time-travellable history, or has complex async coordination** — which in a clinical app is real: a study context that a dozen screens read from."

■ Deeper: [20 — RxJS Operators](#) · [21 — NgRx](#)

Part 4 — Ionic + Capacitor

Their list: architecture, Angular vs Ionic lifecycle, navigation, native-device access, Capacitor/plugins, Android/iOS builds, permissions, storage, network handling, mobile performance.

◆ **This is your differentiator.** Most Angular candidates can't answer this section at all, and it's on their list — which means the role involves a mobile/tablet app (very likely **clinical data capture in the field**). Prepare it *properly*.

4.1 Ionic architecture

The layers:

Your Angular app

- └ Ionic Framework – UI components (Web Components built with Stencil)
+ navigation + theming
 - └ Capacitor – the native bridge (JS ↔ native runtime)
 - └ iOS (WKWebView) / Android (Chrome WebView) / Web (PWA)

- **One codebase, three targets** (iOS, Android, web/PWA) — that's the value proposition.
- Ionic components are **standard web components**, so they're framework-agnostic (Angular, React, Vue).
- **Theming via CSS custom properties** — `--ion-color-primary`, dark mode, platform-adaptive styling (`ios` / `md` modes).
- **Cordova vs Capacitor**: Capacitor is the modern successor — **native projects are source-controlled artefacts you can open in Xcode/Android Studio**, plugins are simpler, and it works in the browser as a PWA. Cordova plugins are still usable through Capacitor's compatibility layer.

4.2 Angular vs Ionic lifecycle — the question they will ask

The reason both exist: Ionic **keeps pages in the DOM** when you navigate forward (so the back transition is instant and state is preserved). So `ngOnInit` does **not** run again when you come back — but `ionViewWillEnter` does.

Angular	Ionic	Fires
<code>ngOnInit</code>	—	Once , when the component is created
—	<code>ionViewWillEnter</code>	Every time the page is about to become active ★
—	<code>ionViewDidEnter</code>	After the enter transition finishes (animations, focus, chart init)
—	<code>ionViewWillLeave</code>	Before leaving (pause timers, save draft)
—	<code>ionViewDidLeave</code>	After leaving

Angular	Ionic	Fires
<code>ngOnDestroy</code>	—	When the page is actually destroyed (popped off the stack)

```
export class VisitListPage {
  ngOnInit()      { this.setupForm(); }           // one-time setup
  ionViewWillEnter() { this.refreshVisits(); }      // ★ re-fetch on
every return
  ionViewWillLeave() { this.saveDraft(); }
}
```

◆ **The bug story that proves you've shipped Ionic:** "The classic symptom is *'the list doesn't refresh when I press back'*. The fetch was in `ngOnInit`, which only ran once because the page was cached. Moving it to `ionViewWillEnter` fixes it." Tell it as a story — it's far more convincing than the table.

Also know `ion-router-outlet` vs Angular's `router-outlet` (it's what enables the stack + transitions), and `NavController` `forward` / `back` / `root` **navigation direction**.

4.3 Navigation

- **Angular Router is the primary mechanism** — `routerLink`, `router.navigate`, guards, lazy loading all work as normal.
- **NavController** for direction-aware navigation: `navigateForward` (push animation), `navigateBack`, `navigateRoot` (reset the stack — use after login/logout).
- **Modals / popovers / alerts / action sheets / toasts** are *not* routes — they're controller-driven overlays:

```
const modal = await this.modalCtrl.create({ component: VisitFormPage,
componentProps: { id } });
await modal.present();
const { data, role } = await modal.onWillDismiss();    // get data back
```

- **Tabs** have their own nested outlets and their **own navigation stack per tab**.

- **Hardware back button (Android):**

`platform.backButton.subscribeWithPriority(10, () => ...)` — a real requirement to handle in a form-heavy app.

- **Deep links / universal links** via Capacitor's

`App.addListener('appUrlOpen')`.

4.4 Capacitor, native access & plugins

```
import { Camera, CameraResultType, CameraSource } from '@capacitor/
camera';
import { Geolocation } from '@capacitor/geolocation';
import { Preferences } from '@capacitor/preferences';
import { Network } from '@capacitor/network';
import { Capacitor } from '@capacitor/core';

const photo = await Camera.getPhoto({
  quality: 70, resultType: CameraResultType.Uri, source:
CameraSource.Camera,
});

if (Capacitor.isNativePlatform()) { /* device-only path */ }
```

Core plugins to name: Camera · Geolocation · Filesystem · Preferences · Network · Push Notifications · Local Notifications · Device · App · Haptics · Share · Splash Screen · Status Bar · Browser · Clipboard · Barcode scanning (community).

The CLI flow — know it cold:

```
ionic build                # or: ng build
npx cap add ios android   # once
npx cap sync               # copy web assets + install native deps (copy
+ update)
npx cap open ios           # Xcode
npx cap open android       # Android Studio
npx cap run android -l --external # live reload on a device
```

▲ **npx cap sync** after every web build and every plugin install — "I changed the code but the app didn't update" is almost always a missed sync.

Writing a custom plugin (mention it if they push): implement the Swift/Kotlin side, register it, and call it through a typed TS interface — needed when a device (a clinical instrument, a barcode reader, an SDK) has no community plugin.

4.5 Permissions

```
const status = await Geolocation.checkPermissions();
if (status.location !== 'granted') {
  const req = await Geolocation.requestPermissions();
  if (req.location !== 'granted') { this.showRationale(); return; }
}
```

- Declare them: **AndroidManifest.xml** (`<uses-permission>`) and **Info.plist** (`NSCameraUsageDescription` etc. — **iOS rejects the build/crashes without a usage string**).
- **Android 6+ requests at runtime**; iOS prompts once — **if the user denies, you cannot re-prompt**, you must deep-link them to Settings.
- **Ask in context, not at launch** — request the camera when they tap "add photo", with a rationale screen. That's a UX answer inside a technical question, and leads are expected to give it.
- Android 13+ split notification and media permissions; scoped storage changed file access.

4.6 Storage — pick deliberately, this matters for healthcare

Option	Use	Notes
@capacitor/preferences	Small key/value (settings, flags)	Native UserDefaults / SharedPreferences . Not encrypted
SQLite (@capacitor-community/sqlite)	★ Structured offline data — the clinical-trial answer	Real queries, large datasets, SQLCipher encryption available
IndexedDB / Ionic Storage	Larger web-friendly structured data	Ionic Storage picks the best available driver
Filesystem	Photos, documents, exports	Directory.Data (private) vs Cache
Secure storage / Keychain / Keystore	★ Tokens, credentials, PHI	The only right place for a refresh token on device
x localStorage	Avoid for anything sensitive or important	Can be cleared by the OS; not encrypted; synchronous

◆ The IQVIA-shaped answer: "For an app capturing patient data offline I'd use SQLite with encryption at rest for the records, secure storage / Keychain for tokens, and I'd keep nothing identifying in localStorage or in logs. On Android I'd also set android:allowBackup="false" so PHI doesn't end up in a cloud backup."

4.7 Network handling & offline

```
Network.addListener('networkStatusChange', s =>
  this.online.set(s.connected));
const status = await Network.getStatus();    // { connected,
connectionType }
```

The offline-first pattern — describe it as a design, not an API:

1. Write locally first (SQLite) with a pending flag → the UI is instant and works with no signal.

2. **A sync queue** processes pending records when connectivity returns — `concatMap` so order is preserved, exponential backoff on failure.
3. **Idempotency**: every record carries a **client-generated UUID** so a retry can't create duplicates. ★ *This is the detail that proves you've really built offline sync.*
4. **Conflict resolution**: last-write-wins vs server-wins vs a merge/flag-for-review — **in a clinical system you flag for review rather than silently overwrite.**
5. **UI honesty**: an offline banner, per-record sync status, and never a spinner that hangs forever — every request gets a timeout.

4.8 Mobile performance

- **Virtual scroll** (`ion-virtual-scroll` / CDK) for long lists; `trackBy` / `track`.
- **OnPush + signals** — a mid-range Android tablet has a fraction of a laptop's CPU.
- **Images**: resize/compress *before* upload (`quality: 70`), lazy-load, cache.
- **Startup time**: lazy routes, small initial bundle, hide the splash screen only when the first view is ready, avoid heavy work in `APP_INITIALIZER`.
- **Avoid layout thrash / heavy shadows & blur** in scrolling lists; prefer CSS transforms (GPU) for animation.
- **Debounce inputs**, batch DB writes in a transaction.
- **Test on a real low-end device**, not just the simulator — and profile with Chrome DevTools remote debugging (`chrome://inspect`) for Android, Safari Web Inspector for iOS.
- **Battery/network**: don't poll; use push or a backoff; respect low-power mode.

■ Deeper: [15 — Ionic Level 1](#) · [16 — Codeboard round](#)

Part 5 — REST + Security

Their list: HTTP methods/status codes, authN vs authZ, JWT/access/refresh, interceptors, expired-token handling, CORS, API error handling.

5.1 HTTP methods

Method	Safe?	Idempotent?	Notes
GET	✓	✓	Never changes state; cacheable
POST	✗	✗	Create / non-idempotent action
PUT	✗	✓	Full replace — send the whole resource
PATCH	✗	✗*	Partial update
DELETE	✗	✓	Deleting twice = same end state (2nd returns 404/204)

Idempotent = the same request repeated leaves the same *state* (not the same response). It's what makes **retries safe** — which links straight to Part 7's circuit breakers. **PUT** vs **PATCH** was asked at Mphasis.

5.2 Status codes

Code	Meaning
200 / 201 / 202 / 204	OK / Created (+ Location) / Accepted (async) / No Content
301 / 302 / 304	Moved permanently / Found / Not Modified (ETag caching)
400	Bad request — malformed/validation
401 Unauthorized	★ Actually <i>unauthenticated</i> — who are you? → log in / refresh
403 Forbidden	Authenticated but not allowed — don't retry, don't log out

Code	Meaning
404 / 409 / 410	Not found / Conflict (version, duplicate) / Gone
422	Unprocessable entity — semantic validation failure
429	Too many requests — respect Retry-After
500 / 502 / 503 / 504	Server error / Bad gateway / Unavailable / Gateway timeout

◆ **The distinction they check: 401 vs 403.** 401 → the client should authenticate (refresh the token). 403 → authenticated fine, just not permitted (show "no access", **don't** trigger a token refresh loop). Getting that wrong causes a real, common bug.

5.3 Authentication vs authorization

"**Authentication is *who are you*; authorization is *what may you do*.** AuthN comes first and produces an identity; authZ is a decision about that identity — a role, a permission, an ownership check."

- **AuthN:** username/password, SSO/**SAML**, **OAuth2 / OIDC**, MFA, biometrics on mobile.
- **AuthZ:** **RBAC** (roles) vs **ABAC** (attributes — "*this investigator may only see patients at their own site*", which is exactly the healthcare pattern), scopes/claims.
- ▲ **Frontend authorization is UX only.** Hiding a button is not security — **the API must enforce it.** Say this; leads are expected to.

5.4 JWT, access & refresh tokens

Structure: **header.payload.signature**, base64url-encoded, **signed not encrypted** → anyone can read the payload. **Never put PHI or secrets in a**

JWT.

Claims: `iss` , `sub` , `aud` , `exp` , `iat` , `nbf` , `jti` + your roles/scopes.

	Access token	Refresh token
Lifetime	Short — 5–15 min	Long — hours/days
Sent	On every API call (<code>Authorization: Bearer ...</code>)	Only to the token endpoint
Stored	In memory (safest)	★ httpOnly, Secure, SameSite cookie
Purpose	Access resources	Get a new access token silently

▲ **localStorage** is readable by any XSS. The pragmatic position to state: *"Refresh token in an httpOnly cookie, access token in memory. If a project insists on **localStorage**, then a short expiry, a strict CSP and rigorous output escaping are non-negotiable compensating controls."* Also: **refresh-token rotation** + reuse detection, and a **jti denylist** for logout, because a stateless JWT can't otherwise be revoked. That last point — *"JWTs can't be revoked, which is the real trade-off against sessions"* — is the mature answer.

5.5 Expired-token handling (they listed this explicitly)

The full flow, which is also a great whiteboard answer:

```

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService), router = inject(Router), toast =
    inject(ToastService);

  return next(withToken(req, auth.accessToken)).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status !== 401 || req.url.includes('/refresh')) return
        throwError(() => err);

      return auth.refresh().pipe(                                     // 1. try a silent
        refresh                                                    // 2. replay the
          switchMap(t => next(withToken(req, t))),                  // 3. refresh
          catchError(() => {
            failed → really expired
            auth.logout();
            toast.warn('Your session has expired. Please sign in again.');
```

router.navigate(['/login'], { queryParams: { returnUrl: router.url } }));

```

            return EMPTY;
          }));
    }));
  };

```

Four layers to describe:

1. **Proactive** — decode `exp` and warn *before* it expires: "Your session expires in 60 seconds — [Stay signed in]". In a clinical form, losing 20 minutes of typed data to a silent logout is a serious defect.
2. **Reactive** — the 401 interceptor above.
3. **Route guard** — an expired user never reaches a protected page.
4. **UX** — toast + `returnUrl` redirect + **preserve unsaved work** (draft in the store / SQLite) + clear the stale token.

▲ **The refresh stampede:** five parallel 401s must not fire five refreshes. Gate it with `isRefreshing` + a `BehaviorSubject<string|null>`; the rest wait with `filter(t => !!t)`, `take(1)`, `switchMap(replay)`. **Volunteer this — it's the follow-up.**

5.6 CORS

What it is: a **browser** security mechanism. The **same-origin policy** blocks JS from reading a response from a different origin (scheme + host + port) unless the **server** opts in with `Access-Control-Allow-*` headers.

- **Preflight:** for anything beyond a "simple" request (custom headers like `Authorization`, `PUT` / `PATCH` / `DELETE`, non-form content types), the browser first sends an **OPTIONS** request. The server must answer with `Allow-Origin`, `Allow-Methods`, `Allow-Headers`, `Max-Age`.
- **Credentials:** `withCredentials: true` requires `Access-Control-Allow-Credentials: true` and a **specific origin** — *** is rejected**. That combination is the most common CORS bug.
- **Fixes, ranked:** configure it on the **server/gateway** (correct) → a **dev proxy** (`proxy.conf.json` in the Angular CLI) → `@CrossOrigin` per controller. ✗ Never "allow all origins" in production.
- ■ **The line that shows you understand it:** *"CORS is enforced by the browser, not the server — Postman doesn't hit it, which is why the classic report is 'it works in Postman but not in the app'. And it isn't a security control on the API: the API still has to authenticate and authorize every request."*
- **On mobile:** a Capacitor app runs from `capacitor://localhost` / `https://localhost`, so it's cross-origin too — configure the server, or use `CapacitorHttp` which bypasses the WebView.

5.7 API error handling

On the client:

```
export const errorInterceptor: HttpInterceptorFn = (req, next) =>
  next(req).pipe(catchError((e: HttpErrorResponse) => {
    const msg =
      e.status === 0 ? 'Network unavailable – your work is saved
locally'
      : e.status === 400 ? extractValidation(e)
      : e.status === 403 ? 'You do not have access to this study'
      : e.status === 429 ? 'Too many requests, retrying shortly'
      : e.status >= 500 ? 'Something went wrong on our side'
      : 'Unexpected error';
    inject(ToastService).error(msg);
    inject(LoggerService).log({ url: req.url, status: e.status,
correlationId: req.headers.get('X-Correlation-Id') });
    return throwError(() => e);
  })));
```

▲ **status === 0** means the request never reached the server (offline, DNS, CORS, cert) — distinguish it, because the user-facing message and the retry behaviour are completely different.

On the server: `@RestControllerAdvice` + `@ExceptionHandler` returning `ProblemDetail` (RFC 7807) with a stable machine-readable `type` /error code, a human `detail`, and a **correlation/trace ID**. Never leak stack traces or SQL to the client.

Also name: input validation on **both** sides, output escaping (XSS), parameterised queries (SQL injection), **rate limiting**, HTTPS/HSTS everywhere, **CSP** headers, secrets in a vault not in `environment.ts` (▲ **anything in an Angular bundle is public**), dependency scanning (`npm audit`, Snyk, OWASP Dependency-Check), and the **OWASP Top 10** as the checklist you review against.

■ Deeper: [06 — Spring Boot](#) · [37 Q17–18](#)

Part 6 — MongoDB + PostgreSQL / SQL

Their list: CRUD, indexes, aggregation, embedding vs referencing, joins, WHERE vs HAVING, GROUP BY, subqueries, transactions/ACID, pagination, 2nd/Nth highest salary, highest salary by department.

● **SQL has now appeared in Mphasis, Deloitte's rubric and this list. It is no longer optional.** The complete file is **36 – SQL** – this Part is the condensed, exam-ready version.

6.1 SQL – WHERE vs HAVING & GROUP BY

Logical execution order – memorise it; it explains half the traps:

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT

	WHERE	HAVING
Filters	Individual rows	Groups
Runs	Before GROUP BY	After GROUP BY
Aggregates	✗ Can't use them	✓ That's the point
Column aliases from SELECT	✗ (SELECT hasn't run yet)	✗ in standard SQL (MySQL is lenient)

```
SELECT department, COUNT(*) AS headcount, AVG(salary) AS avg_salary
FROM   employee
WHERE  active = true           -- row filter, BEFORE grouping
GROUP BY department
HAVING COUNT(*) > 5          -- group filter, AFTER grouping
ORDER BY avg_salary DESC;
```

◆ **The performance sentence:** "Put a condition in **WHERE** whenever you can – it removes rows *before* grouping, so there's less to aggregate. **HAVING** is only for conditions on the aggregate itself."

Aggregates + NULL: **COUNT(*)** counts rows including NULLs; **COUNT(col)** skips NULLs; **AVG / SUM** ignore NULLs (so **AVG** ≠ **SUM/COUNT(*)** when NULLs exist). Every non-aggregated **SELECT** column must be in **GROUP BY** (Postgres enforces it strictly).

6.2 Joins

Join	Returns
INNER	Only matching rows on both sides
LEFT	All left rows + matches (NULLs where none)
RIGHT	Mirror of LEFT
FULL OUTER	Everything from both sides
CROSS	Cartesian product
Self join	A table joined to itself (employee → manager)

```
-- departments and their headcount, INCLUDING empty departments
SELECT d.name, COUNT(e.id) AS headcount      -- ★ COUNT(e.id), not
COUNT(*)
FROM   department d
LEFT   JOIN employee e ON e.dept_id = d.id
GROUP BY d.name;
```

▲ **Two traps that get asked:** (1) `COUNT(*)` on a `LEFT JOIN` returns **1** for an empty department, not 0 — count the joined column. (2) A condition on the right table belongs in `ON`, not `WHERE` — putting it in `WHERE` turns the outer join back into an inner join.

Anti-join: `LEFT JOIN ... WHERE b.id IS NULL`, or `NOT EXISTS`. ▲ `NOT IN` with a **NULL in the subquery returns *no rows*** — three-valued logic. That's a favourite.

6.3 The salary questions — they named these explicitly

Second / Nth highest salary — know at least three ways and their trade-offs:

```

-- 1. Window function – best, handles ties explicitly
SELECT DISTINCT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM
employee
) t WHERE rnk = 2;

-- 2. Correlated subquery – portable, no window functions needed
SELECT MAX(salary) FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);

-- 3. LIMIT / OFFSET – simple, but N-1 offset and ties are wrong
SELECT DISTINCT salary FROM employee ORDER BY salary DESC LIMIT 1 OFFSET
1;

-- 4. NOT IN
SELECT MAX(salary) FROM employee WHERE salary NOT IN (SELECT MAX(salary)
FROM employee);

```

◆ **The differentiator: RANK vs DENSE_RANK vs ROW_NUMBER**. With salaries 100, 100, 90: **ROW_NUMBER** → 1,2,3; **RANK** → 1,1,3 (gap); **DENSE_RANK** → 1,1,2 (no gap). **"Second highest" means DENSE_RANK = 2 if you mean the second distinct salary, ROW_NUMBER = 2 if you mean the second person.** Ask which they mean — that question *is* the senior answer.

Highest salary per department — asked 3 times already, and it's on their list:

```

SELECT department, name, salary FROM (
    SELECT department, name, salary,
        RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
    FROM employee
) t WHERE rnk = 1;
-- RANK keeps ties; ROW_NUMBER
picks exactly one

```

```

-- the pre-window-function version
SELECT e.department, e.name, e.salary
FROM   employee e
JOIN   (SELECT department, MAX(salary) AS mx FROM employee GROUP BY
department) m
ON     e.department = m.department AND e.salary = m.mx;

```

And the Java Streams version (they asked this at Altimetrik today — have both ready):


```
Map<String, String> top =
employees.stream().collect(groupingBy(Employee::getDepartment,
collectingAndThen(maxBy(comparingDouble(Employee::getSalary)),
e -> e.map(Employee::getName).orElse("N/A"))));
```

Employees earning above their department's average — the other classic:

```
SELECT e.* FROM employee e
WHERE e.salary > (SELECT AVG(salary) FROM employee x WHERE x.department =
e.department);
```

6.4 Subqueries

- **Scalar** (returns one value) · **row** · **table** (in **FROM**, a derived table) · **correlated** (references the outer query — runs per outer row, so it's the slow one).
- **IN** vs **EXISTS** vs **JOIN**: **EXISTS** short-circuits on the first match and is usually best for "does a related row exist"; **JOIN** when you need columns from the other table; **IN** is fine for a small static list — but watch the **NOT IN + NULL** bomb.
- **CTEs** (**WITH x AS (...)**) for readability, and **recursive CTEs** for hierarchies (employee → manager chain, or a study → site → subject tree).

6.5 Indexes

What: usually a **B-tree** giving $O(\log n)$ lookup instead of a full scan.
Create: `CREATE INDEX idx_emp_dept_salary ON employee(dept_id, salary DESC);`

Concept	Point to make
Composite index & leftmost prefix	<code>(a, b, c)</code> serves <code>a</code> , <code>a, b</code> , <code>a, b, c</code> — not <code>b</code> alone . Column order matters
Covering index	The index contains every column the query needs → index-only scan , no table lookup (Postgres: <code>INCLUDE</code>)

Concept	Point to make
Sargability	<code>WHERE YEAR(created) = 2026</code> can't use the index; <code>WHERE created >= '2026-01-01' AND created < '2027-01-01'</code> can. Same for a leading <code>LIKE '%X'</code>
Selectivity	An index on a boolean/gender column is usually useless
Cost	Indexes slow down writes and consume storage — every insert maintains every index
Postgres extras	Partial (<code>WHERE active</code>), expression, GIN for JSONB/full-text, BRIN for huge append-only tables
Read the plan	<code>EXPLAIN ANALYZE</code> — Seq Scan vs Index Scan, the row estimate vs actual, and where the time really goes

■ "Make this query faster" is really: **EXPLAIN ANALYZE** first, then index/rewrite, then re-measure. Never guess.

6.6 Transactions & ACID

ACID: Atomicity (all or nothing) · **Consistency** (constraints hold before and after) · **Isolation** (concurrent transactions don't corrupt each other) · **Durability** (committed = survives a crash).

Isolation levels & the anomalies they prevent:

Level	Dirty read	Non-repeatable read	Phantom
Read Uncommitted	✓ possible	✓	✓
Read Committed (Postgres default)	✗	✓	✓
Repeatable Read (MySQL InnoDB default)	✗	✗	✓*
Serializable	✗	✗	✗

In Spring: `@Transactional` — and the three traps worth naming: **self-invocation bypasses the proxy**; it only rolls back on **unchecked** exceptions by default (`rollbackFor = Exception.class` for checked); and `readOnly = true` is a real optimisation hint. **Optimistic locking** (`@Version`) vs **pessimistic** (`SELECT ... FOR UPDATE`) — in a clinical system, optimistic with a "this record changed, review the conflict" prompt is usually right.

6.7 Pagination (SQL side)

```
SELECT * FROM employee ORDER BY salary DESC, id DESC LIMIT 20 OFFSET 40;
-- offset paging
SELECT * FROM employee WHERE (salary, id) < (:lastSalary, :lastId)
-- ★ keyset paging
ORDER BY salary DESC, id DESC LIMIT 20;
```

▲ **Always add a unique tiebreaker to ORDER BY** or rows repeat/vanish between pages. **OFFSET degrades linearly** — at page 50,000 the DB still walks every skipped row, so deep paging / infinite scroll wants **keyset**. In Spring Data: `Page` (2 queries incl. `COUNT`) vs `Slice` (1 query) — see [37 Q9–10](#).

6.8 MongoDB

CRUD:

```
db.patients.insertOne({ _id, name, siteId, visits: [] });
db.patients.find({ siteId: 'S1', age: { $gte: 18 } }).sort({ name:
1 }).skip(20).limit(10);
db.patients.updateOne({ _id }, { $set: { status: 'active' }, $push:
{ visits: visit } });
db.patients.deleteOne({ _id });
```

Operators to know: `$eq $ne $gt $gte $in $nin $and $or $exists $regex` ·
update: `$set $unset $inc $push $pull $addToSet` · `upsert: true` .

Aggregation pipeline — the equivalent of GROUP BY, and they will ask for it:

```

db.employees.aggregate([
  { $match: { active: true } },                                // ★
  filter FIRST (uses indexes)
  { $group: { _id: '$department', maxSalary: { $max: '$salary' },
              avgSalary: { $avg: '$salary' }, count: { $sum: 1 } } },
  { $sort: { maxSalary: -1 } },
  { $project: { _id: 0, department: '$_id', maxSalary: 1, count: 1 } },
  { $limit: 10 },
]);

```

Stages: `$match` `$group` `$project` `$sort` `$limit` `$skip` `$unwind` `$lookup` (the closest thing to a join) `$facet` `$addFields` `$count` `$out`.

◆ **Say this:** "`$match` and `$sort` go as early as possible so the pipeline can use indexes — once you've `$group` ed, the index is gone."

Embedding vs referencing — a genuine design question, answer it as trade-offs:

	Embed (subdocuments)	Reference (store an ObjectId)
Read	✓ One query, no join	Needs <code>\$lookup</code> or a second query
Write	Whole document rewritten	Update independently
Use when	1-to-few · data read together · doesn't grow unbounded · not shared	1-to-many/many-to-many · unbounded growth · shared across documents · updated independently
Example	A patient's address, a visit's vitals	Patient → Study; Study → Sites

▲ **The constraint that decides it: the 16 MB document limit** and unbounded arrays. *"I'd embed the vitals inside a visit because they're always read together and bounded, but reference the study, because thousands of patients share it and it's updated independently."*

Also know: indexes (single, compound — **same leftmost-prefix rule**, text, TTL for expiring data), the `_id / ObjectId`, schema-on-read + **schema validation rules**, replica sets (HA) vs sharding (horizontal scale), and **multi-document ACID transactions since 4.0** (so "Mongo has no transactions" is now a wrong answer).

SQL vs Mongo — the answer a lead gives: "It's not about which is better, it's about the access pattern. Postgres for relational integrity, joins across many entities, and strong constraints — which is most clinical data. Mongo for flexible/evolving documents read as a unit, e.g. per-study forms whose shape differs. And Postgres `JSONB` covers a lot of the middle ground, so 'we need flexible fields' isn't automatically a reason to leave the relational database."

■ Deeper: [36 — SQL \(complete\)](#) · [13 — MongoDB](#) · [24 — ORM/JPA](#)

Part 7 — Microservices

Their list: REST vs messaging, service discovery, API gateway, timeout/retry/circuit breaker, Resilience4j, Kafka basics, network failures, Saga, distributed logging/tracing, database-per-service.

These are the **exact** questions Altimetrik asked today (Q11–13). You now have the answers — this is a re-run you can win. Full versions in [37 Part 3](#).

7.1 REST vs messaging

	Synchronous REST	Asynchronous messaging
Coupling	Caller knows the callee, waits	Producer doesn't know consumers
Failure	Callee down → caller fails/blocks	Broker buffers; consumer catches up
Consistency	Immediate	Eventual

	Synchronous REST	Asynchronous messaging
Complexity	Low	Ordering, idempotency, duplicates are yours
Use when	You need the answer to serve this request	The other service just needs to eventually know

Sync options: **RestClient** (Spring 6.1+) / **WebClient** (reactive) / **OpenFeign** (declarative) / gRPC. ▲ **RestTemplate** is in maintenance mode — don't name it as your default.

7.2 Service discovery & API gateway

- **Discovery:** services register with **Eureka / Consul** (or Kubernetes DNS + Services) so you call **http://patient-service** instead of an IP. Client-side (Ribbon/Spring Cloud LoadBalancer) vs server-side (a load balancer) discovery.
- **API Gateway** (Spring Cloud Gateway, Kong, AWS API Gateway): **one entry point** — routing, authN/authZ, rate limiting, TLS termination, request aggregation, CORS, logging. Keeps cross-cutting concerns out of every service.
- **BFF (Backend for Frontend)** — worth naming for a mobile-heavy role: a gateway shaped for the mobile client so the tablet makes one call instead of six on a bad clinic connection.

7.3 Timeout / retry / circuit breaker / Resilience4j

The failure they're testing: a cascading failure. A is slow → B's threads all block on A → B dies → B's callers die. **Slow is worse than down**, because down at least fails fast.

```
@CircuitBreaker(name = "patientSvc", fallbackMethod = "fallback")
@Retry(name = "patientSvc")
@TimeLimiter(name = "patientSvc")
@Bulkhead(name = "patientSvc")
public PatientDto get(String id) { return client.get(id); }

private PatientDto fallback(String id, Throwable t) {
    return cache.get(id).orElse(PatientDto.unavailable(id));    //
    degrade, don't die
}
```

```
resilience4j.circuitbreaker.instances.patientSvc:
  slidingWindowSize: 20
  failureRateThreshold: 50
  waitDurationInOpenState: 10s
  permittedNumberOfCallsInHalfOpenState: 3
```

Circuit breaker states: CLOSED → (failure rate exceeded) → OPEN → (wait) → HALF_OPEN → CLOSED or back to OPEN.

Resilience4j modules — name all six: CircuitBreaker · Retry · RateLimiter · Bulkhead (isolate thread pools so one slow dependency can't consume them all) · TimeLimiter · Cache. *(It replaced Hystrix, which is retired.)*

▲ **Retries must be bounded, backed off with jitter, and only on idempotent operations** — naive retries make a cascade worse by adding load to a service that's already failing.

7.4 Kafka basics

Model: producers write to a **topic**, which is split into **partitions**; each partition is an **ordered, append-only log**. Consumers join a **consumer group**; each partition is read by exactly **one consumer in the group**, which is how you scale. **Offsets** track position, so a consumer can replay.

Concept	Point
Partition key	Same key → same partition → ordering guaranteed per key (e.g. per patient), not globally
Delivery	At-least-once by default → consumers must be idempotent (dedupe on event ID)
Retention	Time/size based — Kafka keeps messages after consumption, so you can replay
Consumer group	Adding consumers beyond the partition count adds nothing
DLQ	Poison messages go to a dead-letter topic instead of blocking the partition
Kafka vs RabbitMQ	Kafka = a durable, replayable log for high-throughput event streaming; Rabbit = a broker for routing/work queues with per-message ack

```
@KafkaListener(topics = "visit-recorded", groupId = "reporting")
public void on(VisitRecorded e) { ... }           // must be idempotent
```

7.5 Network failures — the sharpest question, asked today

The core insight to state: a caller **cannot tell** "the request never arrived" from "it was processed but the response was lost" from "it genuinely failed". Everything follows from that:

Need	Answer
Detect	Connect + read timeouts — never infinite
Recover	Retry with backoff + jitter
Make retries safe	★ Idempotency key — the caller sends a UUID; the server returns the original result on a repeat
Stop the bleeding	Circuit breaker + fallback

Need	Answer
Don't lose the work	★ Transactional outbox
Poison messages	DLQ
Undo what can't complete	Compensating transaction (Saga)
See it	Correlation/trace ID, breaker-state metrics, alerts

Transactional outbox — the answer that wins the question:

"You can't atomically commit a DB transaction *and* publish to a broker. So I write the event into an **outbox table in the same local transaction** as the business change; a relay (a poller or CDC via Debezium) publishes it and retries until the broker accepts. Atomicity comes from one local transaction, and delivery is at-least-once — so the consumer must be idempotent."

7.6 Saga

Why: there's no two-phase commit across services, so a business transaction spanning services is a **sequence of local transactions, each with a compensating action**.

Style	How	Trade-off
Choreography	Services react to each other's events	Decoupled, but the flow is implicit and hard to follow
Orchestration	A coordinator drives each step	Explicit and debuggable, but the orchestrator is a new component to own

Give a concrete compensation, not a definition — that's the follow-up that catches people: *"Enrol subject → reserve a kit → schedule the first visit. If scheduling fails, the compensation is 'release the kit reservation' and 'mark the enrolment pending' — not a rollback, because the kit reservation was already committed in another service's database."*

7.7 Distributed logging & tracing

- **Correlation/trace ID** generated at the gateway, propagated on every hop (`traceparent` /W3C Trace Context), and logged in **every** log line — via MDC in Spring, and an Angular interceptor that sends `X-Correlation-Id` so a **user-reported bug can be traced from the tap to the SQL query**.
- **Micrometer Tracing (formerly Sleuth) + Zipkin/Jaeger** for traces; **ELK / OpenSearch / Splunk / CloudWatch** for centralised logs; **Prometheus + Grafana** for metrics; **Actuator** for `/health` , `/metrics` , `/info` .
- **Structured (JSON) logs** — greppable and queryable, not free text.
- **▲ Healthcare: never log PHI**. Log identifiers and correlation IDs, not names, dates of birth or diagnoses. **Say this at IQVIA**.
- **The three pillars**: logs (what happened) · metrics (how much/how fast) · traces (where the time went).

7.8 Database per service

The rule: each service **owns its data**; no other service touches its tables. That's what makes independent deployment and independent scaling real.

Consequences to name (this is the whole answer):

- **✗ No cross-service joins** → **API composition** (call both and join in code) or **CQRS** (a read model built from events).
- **✗ No distributed transactions** → **Saga**.
- **▲ Data duplication is expected and fine** — the order service keeps a copy of the customer name it needs; it's eventually consistent.
- **✓ Each service can pick the right store** (Postgres here, Mongo there).
- **The pragmatic caveat a lead adds**: *"A shared database is the classic distributed monolith — but I've also seen teams split too early. I'd start with a modular monolith and clean module boundaries, and extract a service when there's a real reason: independent scaling, independent release cadence, or a separate team."* That answer signals judgement, which is what a lead interview is for.

■ Deeper: **08 — Microservices** · **37 Q11–13** · **30 — Mphasis L2**

Part 8 — Git + CI/CD + Cloud

Their list: merge/rebase, branching/conflicts, PR/code review, CI/CD basics, AWS fundamentals, deployment, logs, environment configuration.

8.1 Merge vs rebase

	merge	rebase
History	Preserves it; adds a merge commit	Rewrites — replays your commits on top
Result	True but noisy graph	Linear, readable
Safety	Always safe	▲ Never rebase a branch others have pulled

```
git checkout feature && git rebase main      # replay my work on top of
latest main
git rebase --continue / --abort
git merge --no-ff feature                    # keep the feature grouping
visible
git pull --rebase                            # avoid pointless merge
commits on pull
git cherry-pick <sha>                        # one commit onto another
branch
git revert <sha>                             # ✓ undo on a shared
branch (new commit)
git reset --hard <sha>                      # ▲ local only – rewrites
history
git stash / git stash pop
git log --oneline --graph --all
git bisect start                             # binary-search for the
commit that broke it
```

■ **The team policy answer (what a lead is really being asked): "Rebase locally to keep my own branch clean, merge to integrate.** The rule I set is: never rebase anything that's been pushed and shared, and use `--force-with-lease`

rather than `--force` if I must update my own PR branch. On main we squash-merge so each PR is one revertable commit, with the ticket ID in the message."

8.2 Branching & conflict resolution

- **Strategies: GitHub Flow** (main + short-lived feature branches + PR — my default) · **Git Flow** (develop/release/hotfix — heavier, but fits **regulated release trains**, which may well be IQVIA's world) · **trunk-based** with feature flags.
- **Protection:** no direct pushes to main, required reviews, required green CI, linear history, signed commits.
- **Conflicts:** they happen where two branches changed the same lines. `git status` → open the `<<<<<< / ===== / >>>>>>` markers → **understand both sides** (`git log -p`, ask the other author) → resolve → `git add` → `git rebase --continue`. Use `git mergetool`, `git diff --ours/--theirs`, and `rerere` for repeats.
- **How you prevent them:** small PRs, short-lived branches, rebase/pull often, agree file ownership, and don't let a branch live for three weeks. **That prevention answer is the lead-level part.**

8.3 PR & code review process

What I'd put in place (say it as a process, with a why):

1. **Small PRs** — under ~400 lines. Review quality falls off a cliff beyond that.
2. **A PR template:** what changed, why, how it was tested, screenshots, ticket link, risk/rollback.
3. **CI gates before a human looks:** lint, format, unit tests + coverage threshold, build, bundle budget, dependency/security scan.
4. **At least one approval;** a second for security-, data- or migration-touching changes.
5. **Review for:** correctness · edge cases and error paths · tests · readability/naming · security (authZ, injection, PHI in logs) · performance · public API design.

Formatting is the linter's job, not the reviewer's.

- 6. **Comment style:** ask questions rather than issue commands; label **blocking** vs **nit**; approve with minor comments instead of blocking on taste.
- 7. **SLA:** review within one working day, so PRs don't rot and conflict.
- 8. **Definition of done:** merged, deployed to a test environment, ticket updated.

8.4 CI/CD

Pipeline stages:

```
commit → lint + unit tests → build → SAST/dependency scan → package
(Docker) →
deploy to DEV → integration/e2e tests → deploy to QA (approval) → UAT →
deploy to PROD (approval) → smoke tests → monitor
```

Concretely for your stack: `npm ci` → `ng lint` → `ng test --watch=false --code-coverage` → `ng build --configuration=production` → `mvn verify` → Docker image tagged with the commit SHA → push to a registry → deploy.

Term	Meaning
CI	Every commit is merged and verified automatically
Continuous Delivery	Every green build is <i>releasable</i> ; deployment is a button
Continuous Deployment	Every green build goes to prod automatically
Blue/green	Two environments, switch traffic — instant rollback
Canary	5% of traffic first, watch the metrics, then ramp
Rolling	Replace instances gradually (the Kubernetes default)
Feature flag	Deploy the code dark, enable it separately — decouples deploy from release

Tools: GitHub Actions / GitLab CI / Jenkins / Azure DevOps · Docker · Kubernetes/ECS · Terraform · SonarQube · Snyk/Dependabot.

▲ **Regulated-industry nuance worth raising at IQVIA:** "In a validated (GxP) system, continuous deployment straight to production usually isn't allowed — releases are controlled and documented. So I'd aim for **continuous delivery with an approval gate**, full traceability from ticket → commit → build → deployed artefact, and automated evidence capture rather than manual sign-off screenshots." Very few candidates will say this.

8.5 AWS fundamentals

Service	One line
EC2	Virtual machines
S3	Object storage — static Angular builds, documents, exports; versioning + lifecycle + SSE-KMS encryption
CloudFront	CDN in front of S3 — ★ the standard way to host an Angular SPA
RDS / Aurora	Managed Postgres/MySQL — backups, Multi-AZ, read replicas
DocumentDB / DynamoDB	Managed Mongo-compatible / managed NoSQL key-value
Lambda	Serverless functions — event-driven, no servers to run
API Gateway	Managed REST entry point — throttling, auth, keys
ECS / EKS / Fargate	Containers — orchestrated / Kubernetes / serverless containers
ELB (ALB/NLB)	Load balancing + TLS termination
IAM	Users, roles, policies — least privilege ; use roles, never long-lived keys
CloudWatch	Logs, metrics, alarms, dashboards

Service	One line
SQS / SNS / EventBridge / MSK	Queue / pub-sub / event bus / managed Kafka
Secrets Manager / Parameter Store / KMS	Secrets and encryption keys
VPC, subnets, security groups	Network isolation — the DB is in a private subnet, never public
Route 53 / ACM	DNS / TLS certificates
Regions & AZs	★ Data residency — for GDPR, EU patient data stays in an EU region

A typical deployment for your stack: Angular/Ionic build → **S3 + CloudFront** (with the SPA fallback rewriting 404 → `index.html`); Spring Boot in a Docker image → **ECS Fargate** behind an **ALB**; **RDS Postgres** in a private subnet; **Secrets Manager** for credentials; logs and alarms in **CloudWatch**.

8.6 Logs & environment configuration

- **Environments:** dev → QA → staging/UAT → prod, ideally identical in shape.
- **One artefact, many environments** — the build is promoted, not rebuilt. Configuration comes from **environment variables / Parameter Store / Secrets Manager / Spring profiles**, never from a rebuilt bundle.
- **Angular caveat:** `environment.ts` is compiled into a public bundle — ▲ it is not a place for secrets, only public endpoints. Runtime config via a fetched `config.json` is often better, because it lets one build serve every environment.
- **Spring:** `application-{profile}.yaml` + `SPRING_PROFILES_ACTIVE` ; env vars and CLI args override the file (relaxed binding maps `SPRING_DATASOURCE_URL` → `spring.datasource.url`).
- **Schema:** Flyway/Liquibase migrations in source control; `ddl-auto: validate` in QA/prod, never `update` .

- **Logging:** structured JSON, levels used properly (**ERROR** = someone must act), **correlation IDs**, centralised aggregation, retention policy — and **no PHI/PII/tokens in logs, ever**. Alerting on error rate and latency, not on log volume.

■ Deeper: [07 — AWS Basics](#) · [37 Q14](#)

Part 9 — Technical Lead questions

Their list: code reviews, mentoring, architecture decisions, sprint estimation/ planning, production incidents, technical debt, disagreements, stakeholder communication, code quality.

● **This Part is the difference between "good senior developer" and "hire as a lead". Every answer here uses STAR: Situation → Task → Action → Result — and the *Result must have a number in it*.**

▲ **The blanks marked <...> are for you to fill from RoboGebra / Subsea / Backoffice Migration / EasyVisa. Write your real numbers in before the interview — a specific number is what makes a story believable. **Do this today; don't improvise it in the room.****

9.1 The universal answer shape

Every lead question has the same skeleton:

1. My principle (one sentence) → **2. How I apply it in practice** (the mechanism) → **3. A specific example from my work** (STAR + number) → **4. What I'd do differently / how I'd scale it.**

Practise saying **step 3 out loud** for each of the nine topics. That's the whole preparation.

9.2 Code reviews

Principle: "Code review is for **correctness, shared understanding and raising the floor** — not for enforcing my personal style. Anything a machine can check should be checked by a machine."

Practice:

- Automate style: Prettier + ESLint + Checkstyle/Spotless run in CI, so reviews are about **logic, not commas**.
- A checklist: does it do what the ticket says · edge cases and error paths · tests for the new behaviour · naming/readability · security (authZ, injection, **PHI in logs**) · performance (N+1, unbounded queries, CD churn) · public API/contract changes · migration/rollback.
- **Small PRs (<400 lines)** and a **one-working-day review SLA**.
- **Comment style:** label **[blocking]** vs **[nit]**; ask *"what happens if this is null?"* rather than *"add a null check"*; approve with nits rather than blocking on taste.
- **Praise in review too** — it's how you set the standard positively and keep juniors submitting.

■ **STAR:** "On **<project>**, PRs were sitting for **<3–4 days>** and arriving as **<1000+>** -line changes, so review was rubber-stamping. I introduced a PR template, a 400-line guideline, and moved formatting into CI. Review turnaround dropped to **<under a day>** and we caught **<N>** real defects in the first month that would have reached QA."

9.3 Mentoring developers

Principle: "My job is to make the person independent, not to solve their ticket."

Practice: pair on the first task of a new area · **explain the why, not just the fix** · give them the whole vertical slice (API → UI → tests), not just the CSS · review their PRs with questions rather than corrections · a weekly 1:1 that isn't a status meeting · let them lead a design discussion and back them publicly · a "you own this module" assignment to build confidence.

■ **STAR:** "A junior on `<project>` kept getting `<change-detection / subscription-leak>` bugs. Instead of fixing them in review, I ran a 30-minute session on `<OnPush and immutable updates>`, added the pattern to our team guide, and had them present it back to the team. Their PRs stopped hitting that class of bug, and in `<N>` months they owned the `<feature>` module end to end."

▲ Also prepare **"a mentee who wasn't improving"** — the answer is: be specific about the gap, agree a concrete goal with a timeline, follow up weekly, involve their manager early, and be honest rather than kind-in-the-short-term.

9.4 Architecture decisions

Principle: "I decide against **requirements and constraints**, not against what's fashionable — and I write the decision down."

Practice:

- Name the **non-functional drivers**: users, data volume, latency, offline, compliance, team skills, timeline, cost.
- **2–3 options with trade-offs**, then a recommendation. Never one option.
- **ADRs (Architecture Decision Records)** in the repo: context, options, decision, consequences. *"So in a year, the person asking 'why is it like this' has an answer — that matters more in a long-lived regulated product than anywhere else."*
- **Prototype/spike** when the risk is real; decide reversible things quickly and irreversible things carefully ("one-way vs two-way doors").
- Prefer the **boring** technology unless there's a reason.

■ **STAR (use a real one):** "On `<project>` we had to decide between `<NgRx and a service-with-signals>` for state. I listed the drivers — `<team size, shared state across features, debuggability>` — prototyped both on one feature, and chose `<X>` because `<reason>`. I wrote it up as an ADR. `<Result: onboarding time / bug rate / delivery>`."

Have one for each: monolith vs microservices · NgRx vs simpler state · SQL vs NoSQL · Ionic vs native · library vs build-it-yourself.

9.5 Sprint estimation & planning

Practice: story points for **relative complexity + uncertainty** (not hours) · **planning poker** so the whole team's information surfaces · **break anything above ~8 points down** — a big number means we don't understand it yet · include testing, review, deployment and the unknowns in the estimate · track **velocity over 3+ sprints** and plan from the average, not from the best sprint · leave **~20% capacity** for support, bugs and tech debt · a clear **Definition of Ready** and **Definition of Done**.

"How do you handle a sprint you're going to miss?" — a favourite:

"**Raise it as soon as I know, not at the review.** I re-forecast, present options to the PO — cut scope, extend, add help, or ship behind a flag — and let them choose. What I don't do is quietly let the team burn out or drop quality to make a date, because that just moves the cost into the next sprint."

"**An estimate you got badly wrong?**" — have one, own it, and say what you changed: "We estimated *<the migration>* at *<2 sprints>* and it took *<4>* because *<the legacy API had undocumented behaviour>*. Since then I insist on a timeboxed spike before estimating anything that touches a system nobody on the team has worked in."

9.6 Production incidents

The structure — say it as a sequence, it reads as experience:

1. **Assess impact** — who's affected, how badly, is data at risk. **In a clinical system, patient-data integrity outranks uptime.**
2. **Communicate early** — a status message to stakeholders before they ask, and a single incident channel.
3. **Mitigate before diagnosing** — **roll back / flip the feature flag / scale up** first. "Restore service, then find the cause — the cause can wait, the users can't."
4. **Diagnose** — logs by correlation ID, traces, metrics, recent deploys ("what changed?" is the highest-yield first question).
5. **Fix and verify.**

6. **Blameless post-mortem** — timeline, root cause (5 whys), and **action items with owners and dates**.

7. **Prevent** — the monitoring/alert/test that would have caught it, added as a ticket that actually gets scheduled.

■ **STAR:** "On `<project>`,
`<symptom – e.g. the app started timing out for all users after a release>`. I `<rolled back within N minutes>` to restore service, then traced it to `<cause – e.g. an N+1 query introduced by a lazy association>`. The fix was `<what>`, and I added `<a query-count assertion / an alert on p95 latency>` so the same class of problem is caught before prod. Downtime was `<N>` minutes and it hasn't recurred."

◆ If you genuinely haven't led a prod incident, say so and give the closest real thing — a severe QA/UAT defect you triaged under pressure — then describe the process above as what you'd run. **Honest + structured beats invented.**

9.7 Technical debt

Principle: "Debt is a **trade-off, not a sin**. Sometimes taking it is right. What's not acceptable is taking it **silently**."

Practice: keep a **visible debt register** with impact (what it costs us per sprint) rather than "this is ugly" · **~20% of each sprint** allocated to it, agreed with the PO up front · **the boy-scout rule** for small things · tie the pitch to **business language** — "*this is why `<feature>` takes 3 days instead of 1 and why we had `<N>` regressions*" · **strangler-fig** incremental replacement rather than a big-bang rewrite · **▲ resist rewrites** — they're usually a slower, riskier version of refactoring.

■ **STAR:** "`<Legacy area>` had `<no tests / duplicated logic>`, so every change there caused regressions. Rather than a rewrite, I `<added characterisation tests, then extracted the shared logic module by module>` while we delivered features around it. Over `<N>` sprints, `<bug rate / change lead time>` improved by `<X>`."

9.8 Disagreements

Principle: "Disagree on the merits, decide on the data, then commit fully either way."

Practice: understand their position first and say it back in your own words · move from opinion to **evidence** — a benchmark, a spike, a prototype, a bug count · separate **reversible** decisions (just try it) from **expensive** ones (spend the time) · if it's still tied, **escalate to a decision-maker rather than letting it drift** · once decided, **support it publicly even if it wasn't your choice** · revisit later with evidence if you were right.

■ **STAR:** "A colleague wanted **<approach A>** ; I preferred **** . Rather than argue in review, we timeboxed a **<1-day>** spike on both against our real **<data volume / device>** . **<A>** turned out **<measurably better/worse>** on **<metric>** , so we went with **<choice>** — including when that meant dropping my own preference. The point is the decision got made on evidence in a day instead of a week of debate."

▲ Also prepare **"a disagreement with your manager or a stakeholder"** — same structure, plus: raise it privately, present impact and options rather than resistance, and if they still choose differently, **document the risk and execute properly**.

9.9 Stakeholder communication

Principle: "Translate. Stakeholders don't need the architecture; they need **impact, options, and a date they can trust**."

Practice: lead with the **conclusion**, then the detail · talk in **business terms** (users affected, revenue/compliance risk, delivery date), never in framework names · give **options with trade-offs** rather than a flat "no" — *"we can have it by Friday without **<X>**, or fully by Wednesday"* · **surface bad news early** — the cost of a delay doubles once it's a surprise · write decisions down and confirm in one line after a call · manage expectations with ranges, not false precision.

■ **STAR:** "On `<project>`, `<stakeholder>` wanted `<feature>` in `<timeframe>`. Instead of saying it was impossible, I broke it into `<a core slice that met the real deadline>` and `<the rest>`, explained the risk of the compressed option in terms of `<QA time / defect risk>`, and let them choose. We delivered `<the slice>` on time and `<the rest>` the following sprint."

9.10 Maintaining code quality

The full stack of mechanisms — this is a "what do you actually put in place" question:

| Layer | Mechanism |

---|---

| **Standards** | A written team guide (naming, folder structure, state rules, error handling) — short and actually maintained |

| **Automation** | Prettier + ESLint (+ `rxjs` / `@angular-eslint` rules), Checkstyle/Spotless, **pre-commit hooks** — style is never a review topic |

| **Tests** | Unit (Jest/Karma), component, integration, e2e (Cypress/Playwright); **a coverage threshold that fails the build**; tests required with every bug fix |

| **CI gates** | Build, tests, coverage, **bundle budget**, SonarQube quality gate, dependency & SAST scan |

| **Review** | Small PRs, checklist, required approval, one-day SLA |

| **Architecture** | ADRs, module boundaries, lint rules that ban cross-feature imports |

| **Observability** | Error tracking (Sentry), so quality is measured in production, not just in CI |

| **Culture** | The boy-scout rule, blameless post-mortems, and **the lead writing code too** — you can't set a standard you don't practise |

◆ **The closing sentence for this whole Part:** "Quality isn't a phase or a person — it's the set of defaults the team works inside. My job as lead is to make the right thing the easy thing, and to make sure the team owns the standard rather than me policing it."

Part 10 — AI-assisted development

Their list: how you use Claude, Cursor and GitHub Copilot; where they improve productivity; how you review AI-generated code; security/privacy concerns; testing generated code; examples from actual development.

◆ **This is a modern-lead question, and most candidates answer it badly in one of two ways:** either "I don't really use it" (reads as behind) or "it writes most of my code" (reads as reckless). **The winning position is: enthusiastic user, rigorous reviewer, clear on the boundaries — especially with healthcare data.** Prepare this like a technical topic, because they listed it like one.

10.1 How you use them — be specific per tool

Tool	Shape	What it's genuinely good for
GitHub Copilot	Inline autocomplete in the editor	Boilerplate as you type — DTOs, mappers, test scaffolding, repetitive <code>*.spec.ts</code> , obvious next lines
Cursor	An AI-native editor with codebase context	Multi-file changes, "rename this concept everywhere", explaining an unfamiliar area, refactors across files
Claude / Claude Code	Conversational + agentic, long context	Design discussion and trade-offs, reviewing my own diff before I raise the PR, writing tests from a spec, explaining a stack trace, generating documentation/ADRs, migration plans

■ **The framing sentence:** "I treat them as a **very fast junior pair-programmer with excellent recall and no accountability**. They accelerate the parts of the job

that are typing; they don't do the part that's deciding. The responsibility for anything that lands in a PR with my name on it is entirely mine."

10.2 Where they genuinely improve productivity

High value:

- **Boilerplate** — Angular components/services/specs, reducers/effects/selectors, DTOs, mappers, form scaffolding.
- **Tests** — first-draft unit tests and, crucially, **edge cases I hadn't thought of** ("what other cases should this cover?").
- **Understanding unfamiliar code** — "explain this module", "where is this state mutated?" — this is the biggest real win when joining a legacy codebase.
- **Translation work** — JS→TS, template-driven→reactive forms, RxJS→signals, an NgModule app→standalone, SQL→JPQL.
- **Regex, config, YAML, migrations, shell** — things that are quick to verify but slow to write.
- **Documentation, ADRs, PR descriptions, release notes.**
- **Rubber-ducking a design** — arguing with it surfaces holes in my own thinking.

Low value / actively risky:

- **Novel business logic** — clinical rules, eligibility criteria, anything domain-specific it can't know.
- **Anything security-critical** (auth flows, crypto, permission checks) — plausible-looking and subtly wrong is the worst combination.
- **Architecture decisions** — it doesn't know our team, constraints, deadline or regulator.
- **Anything where I couldn't tell whether the answer is wrong** — that's the real boundary.

10.3 How you review AI-generated code — say this as a checklist

"The rule I hold myself and my team to: I must be able to explain every line I commit. If I can't, it doesn't go in."

1. **Read it fully** — never accept a suggestion I haven't read end to end.
2. **Check the API is real** — models hallucinate methods, options and package names. Verify against the actual docs/version. ▲ **"Slopsquatting"**: a hallucinated package name can be registered by an attacker — never install a dependency an AI suggested without checking it exists and is legitimate.
3. **Check the version matches ours** — AI answers skew to older Angular/RxJS idioms (`HttpModule`, deprecated operators, `subscribe(next, error)` overloads).
4. **Edge cases and error paths** — generated code is usually the happy path only: nulls, empties, failures, cancellation, concurrency.
5. **Security** — authorization checks, injection, secrets, PHI in logs, unsafe `innerHTML`.
6. **Performance** — N+1 queries, missing `trackBy`, subscriptions without teardown, unbounded queries.
7. **Consistency** — does it match our conventions and reuse our existing utilities, or has it invented a parallel way of doing the same thing? **This is the one that quietly rots a codebase.**
8. **Tests** — I write or verify the tests myself; I don't let the same generator both write the code and certify it.
9. **Licensing/provenance** — for anything that looks like a large verbatim block.

10.4 Security & privacy — the section that matters most at IQVIA

● **Lead with the healthcare framing; it's exactly the judgement they're screening for.**

- **Never paste PHI/PII, patient data, credentials, keys or customer data into any AI tool.** If I need help with a query, I use **synthetic or anonymised** data and a schema, not real rows.
- **Know the deployment model:** an enterprise/business tier with **no training on your data**, or a self-hosted/VPC deployment, is a completely different risk profile from a personal consumer account. **Use the approved tool, on the approved account, and follow the company's AI policy** — in a regulated company that policy exists and is not optional.
- **Proprietary code is confidential too** — the same rule applies to source, not just data.
- **Supply chain:** verify suggested dependencies; keep `npm audit` /Snyk in CI.
- **Auditability:** in a GxP/validated system, generated code goes through exactly the same controlled review, test and release process as any other code — **there is no fast lane**. AI changes who types the code, not who's accountable for it.
- **Bias/correctness:** never let generated logic make a clinical or eligibility decision without human verification and a test.

■ **The sentence to have ready:** "My rule is simple: **the AI can see the shape of the problem, never the patient's data**. Schema yes, synthetic data yes, real PHI never — and I'd make that an explicit team rule, not an assumption."

10.5 Testing generated code

- **Tests are the safety net that makes AI-assisted development responsible**
— the faster code is produced, the more the tests matter.

- **I don't let the same prompt write both** the implementation and the test that certifies it, at least not without reading both critically — it will happily test the behaviour it *implemented*, including the bug.
- Good pattern: **write (or specify) the test first**, then let AI implement against it — the test is the spec, so correctness is verifiable rather than assumed.
- Use AI *for* testing: "what edge cases am I missing?", generating table-driven cases, building fixtures and mocks, and drafting e2e scenarios from acceptance criteria.
- **Coverage isn't correctness** — generated tests can be assertion-free or tautological (`expect(component).toBeTruthy()` and nothing else). Review that each test would actually **fail** if the behaviour broke.
- Everything still passes the same CI gates: lint, tests, coverage threshold, SonarQube, security scan.

10.6 Examples from actual development — ▲ prepare two, from your real work

They asked for examples. Have two ready, 60 seconds each, in this shape: *the task* → *what I asked the tool* → *what it produced* → *what I had to fix* → *the outcome*. **The "what I had to fix" part is the most important sentence** — it proves you reviewed rather than pasted.

Example skeleton A — speed:

"On <project> I had to <convert N template-driven forms to reactive / write specs for a service layer>. I used <Copilot/Claude> to generate the first pass. It got the structure right but <used a deprecated validator API / missed the cross-field validation / didn't handle the async case>, which I rewrote. It turned roughly <a day> of typing into <an hour> of typing plus <an hour> of careful review — and the review is where the value of my time actually was."

Example skeleton B — understanding:

"When I joined <the Backoffice migration>, I used <Cursor/Claude> to explain <an unfamiliar legacy module> and map where <state was mutated / a config was read>. It gave me a map in minutes that would have taken me <a day> of grepping. I then verified the key claims by reading the code myself, because it was <confidently wrong about one thing – X>."

◆ **Close the whole section on the lead angle:** "As a lead I'd want a team position on this, not individual improvisation: an approved tool list, a written rule about what data can go in, AI-assisted code reviewed to the same standard as any other, and no relaxation of the CI gates. Used that way it's a genuine multiplier — especially for onboarding and for test coverage, which are usually the two things a team is short of."

Study plan

If you have ~5 days (adjust proportionally — the *order* is what matters):

Day	Focus	Deliverable by the end of the day
1	Part 9 (Tech Lead) + Part 10 (AI)	Nine STAR stories written down with real numbers. Do this first — it's the part you can't cram, and it's what differentiates you
2	Part 1 (Angular) — CD, Signals, performance, leaks, guards/resolvers	Explain OnPush + the immutable-update trap out loud, from memory
3	Part 3 (RxJS/NgRx) + Part 2 (TS/JS)	Draw the NgRx flow diagram from memory; say the four flattening operators with their use cases

Day	Focus	Deliverable by the end of the day
4	Part 4 (Ionic/Capacitor) + Part 5 (REST/Security)	The <code>ionViewWillEnter</code> bug story; the token-refresh interceptor written from scratch
5	Part 6 (SQL/Mongo) + Part 7 (Microservices) + Part 8 (Git/CI/AWS)	Write the top-salary-per-department query and the Streams version without looking

Every day, before you start: 15 minutes on [26 — Companies \\$Most-repeated](#).
Every day, in the evening: answer 5 questions **out loud, timed at 90 seconds each**. Reading is not preparing — you lose rounds on delivery, not on knowledge.

One-page cheat sheet — read this 20 minutes before

Angular

`constructor` = DI only → `ngOnInit` = inputs are ready · `OnPush` fires on **input reference change** / **event** / **async pipe** / **signal** — ▲ `push()` won't trigger it, `[...arr, x]` will · `takeUntilDestroyed()` · `trackBy` / `track` · guards return a `UrlTree` · `CanMatch` for role-based lazy routes · resolvers **block navigation** · lazy: never eagerly import the same module · `signal` / `computed` (**lazy+memoised**) / `effect` (**side effects only**) · never call methods in templates.

TS/JS

`const` freezes the **binding**, not the object · arrow = **lexical** `this` · **microtasks before macrotasks** · `unknown` = narrow before use; `never` = **exhaustiveness check** · spread is a **shallow** copy · `type` for unions, `interface` for objects (declaration merging) · TS is **structurally** typed.

RxJS/NgRx

Promise = one, eager, uncancellable · Observable = many, lazy, cancellable ·

`BehaviorSubject` for state, expose `.asObservable()` · `switchMap` search / `concatMap` ordered writes / `exhaustMap` submit / `mergeMap` parallel · ▲ never `switchMap` a POST · `forkJoin` = `Promise.all`, `catchError` **inside** so one failure doesn't kill the page · **NgRx**: `catchError` **INSIDE** the effect's inner pipe or the effect dies forever · `shareReplay({refCount:true})` .

Ionic

`ngOnInit` runs once, `ionViewWillEnter` runs every time you come back (pages are cached — that's the "list doesn't refresh" bug) · `npx cap sync` after every build/plugin · SQLite (encrypted) for offline data, **Keychain/Keystore for tokens**, never `localStorage` for PHI · offline = local write + sync queue + **client-generated UUID for idempotency** + conflict policy · request permissions in context.

REST/Security

401 = who are you (refresh) · **403 = not allowed (don't refresh)** · `PUT` full/ idempotent vs `PATCH` partial · JWT is **signed, not encrypted** — **no PHI inside** · access token short + in memory, refresh token in an **httpOnly cookie** · **gate the refresh stampede** · CORS is enforced by the **browser** (works in Postman ≠ works in the app) · `status === 0` = never reached the server.

SQL/Mongo

`FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY→LIMIT` · `WHERE` filters rows, `HAVING` filters groups · `COUNT(e.id)` not `COUNT(*)` on a LEFT JOIN · `DENSE_RANK` for Nth distinct salary, `RANK() ... PARTITION BY dept` = 1 for top per department · **leftmost prefix**, sargability, `EXPLAIN ANALYZE` · ▲ `NOT IN` + NULL = no rows · Mongo: `$match` first, **embed if read together & bounded, reference if shared or unbounded**, 16 MB limit.

Microservices

Sync when I need the answer now, async when B just needs to eventually know · timeout → **circuit breaker (CLOSED/OPEN/HALF_OPEN)** → retry+backoff+jitter → bulkhead → fallback · retries only if **idempotent** → **idempotency key** · **transactional outbox** + idempotent consumer + **DLQ** · Saga = compensating transactions (give a concrete one) · Kafka: partition key ⇒ ordering per key, at-least-once · **trace ID on every hop, never log PHI**.

Lead

Principle → mechanism → **STAR with a number** → what I'd change · incidents: **mitigate first, diagnose second, blameless post-mortem** · debt: visible, quantified in business terms, ~20% per sprint, **strangler not rewrite** · disagreement: **timeboxed spike beats a long argument**, then commit either way · AI: fast junior pair-programmer, **I must be able to explain every line I commit, and the model never sees PHI.**

◆ Questions to ask them

Pick 3–4 — asking good questions is itself part of the assessment for a lead role.

1. "What does the **team** look like — how many engineers, and would I be leading or joining as a senior contributor first?"
2. "Is the product **Angular web, Ionic mobile, or both**? Do the clinical users work offline in the field?" *(shows you've read the stack for what it implies)*
3. "How much of the work is **new development** versus modernising an existing system?"
4. "What's the **release process** like — is this a validated/GxP system with controlled releases, and how does that shape CI/CD?" ★
5. "How does the team handle **PHI in non-production environments** — synthetic data, anonymisation?" ★
6. "What does the team currently use for **state management**, and is there an appetite to move toward signals?"
7. "Where does the team feel its **biggest technical debt** is right now?"
8. "Does the team have a position on **AI-assisted development** — approved tools, data rules?" ★ *(brings their own item back to them, as a lead would)*
9. "What would you want the person in this role to have achieved in their **first 90 days**?"
10. "What are the next steps and the timeline?"

★ = the ones that specifically mark you as thinking like a lead in a **regulated healthcare** company.

◆ Before you walk in

You have now been through 10 rounds and logged 134 questions. That is a real asset — most candidates walk out and forget by evening. The Altimetrik result today is one data point, not a verdict: the topics they asked are all now written down and answered, and **this checklist is 75% the ground you're strongest on.**

Three things to do the morning of:

1. Read the **cheat sheet** above and **26 §Most-repeated** — nothing else.
2. Say your **60-second self-introduction** out loud once → **00 — Self Introduction.**
3. Pick your **three strongest STAR stories** from Part 9 and have them ready to reach for.

In the room: three-sentence answers · a clean "I don't know" when you don't · **"on my team I'd..."** for anything lead-shaped · and one sentence early that shows you understand this is healthcare data.

After the round — the same day — log every question into 26. That habit is why this file exists.

Related files: [26 — Companies \(recall log\)](#) · [37 — Altimetrik 31 Aug](#) · [04 — Angular](#) · [03 — TypeScript](#) · [01 — JavaScript](#) · [20 — RxJS](#) · [21 — NgRx](#) · [15 — Ionic](#) · [25 — Binding & Forms](#) · [36 — SQL](#) · [13 — MongoDB](#) · [08 — Microservices](#) · [07 — AWS](#) · [06 — Spring Boot](#) · [00 — Self Introduction](#)

● RoboGebra Code — Real Examples for Every Interview Topic

Why this file exists: the strongest possible answer to any interview question is *"here's the concept — and here's where I actually used it."*

This file maps **every major interview topic to real code in your own workspace** at `/Users/safi/workspace/robogebra-workspace` .

Every topic file in this pack now carries a ● **In your RoboGebra code** block pointing back here.

The workspace at a glance

```
robogebra-workspace/
├─ robogebra-portal/           Java 17 + Spring Boot 3.2.0 + MongoDB  ←
the MAIN backend
├─ robogebra-admin-portal/    Java + Spring Boot                               ←
admin backend
├─ robogebra-mobile/         Angular 18.2 + Ionic 8.7 + Capacitor 6.2 ←
the flagship app
├─ robogebra-web/            Angular 16.2 + Material 16 + PrimeNG 16
├─ robogebra-backoffice/     Angular 16.2 + PrimeNG 16 (TypeScript 4.9)
├─ robogebra-crm/
● Node + Express 4 + Mongoose 8           ← the ODM example
├─ robogebra-crm-web/        React 18 + Vite + TanStack Query + Radix
└─ robogebra-website/       marketing site
```

The version table — memorise this ★

Project	Stack	Versions
robogebra-portal	Java / Spring Boot	Java 17, Spring Boot 3.2.0, Spring Security, WebFlux, Quartz, Thymeleaf
	Data	Spring Data MongoDB (not JPA ▲)

Project	Stack	Versions
	Integrations	AWS S3 + Cognito, Firebase Admin 9.3, Razorpay 1.4.3, iText/html2pdf, java-jwt 4.4
robogebra-mobile	Angular / Ionic	Angular 18.2 , Ionic 8.7.5 , Capacitor 6.2.1 , RxJS 7.8, TypeScript 5.4
robogebra-web	Angular	Angular 16.2 , Material 16, PrimeNG 16, RxJS 7.8, TypeScript 5.1
robogebra-backoffice	Angular	Angular 16.2 , PrimeNG 16.9, TypeScript 4.9
robogebra-crm	Node	Express 4.21, Mongoose 8.8 , jsonwebtoken 9
robogebra-crm-web	React	React 18.3, Vite, TanStack Query 5, react-hook-form 7, Radix UI

▲ **The trap to never fall into:** `spring-data-jpa` appears in the portal's `pom.xml`, but there is **no JPA starter**, **no DataSource** and **no @Entity** — the portal uses **MongoDB**. Say **ODM**, not **ORM**. (See 24 — [ORM/JPA](#) section A.)

Codebase scale — good numbers to quote

robogebra-portal (Java)	robogebra-mobile (Angular/Ionic)
110 × @RestController	163 components
207 × @Service	89 services
85 × @Repository	27 route resolvers
104 × @Document (MongoDB)	3 HTTP interceptors
130 × @Transactional	4 route guards
92 × @Valid	
27 × @Aggregation pipelines	RxJS operator usage:
26 × Java record	map 496 · filter 485 · takeUntil 426
16 × @Cacheable	catchError 338 · finalize 254
12 × @Async	switchMap 246 · BehaviorSubject 122

Part 1 — Java & Spring Boot topics

@Async + thread pools → 32 — Multithreading, 06 — Spring Boot

File: `robogebra-portal/src/main/java/com/robogebra/cms/config/AsyncConfig.java`

This is a **textbook-perfect** answer to "have you configured a thread pool?" — it does every single thing the interview answer says you should:

```
@Bean(NOTIFICATION_TASK_EXECUTOR)
public ThreadPoolTaskExecutor notificationTaskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(4);
    executor.setMaxPoolSize(8);
    executor.setQueueCapacity(500); // BOUNDED queue ★
    executor.setThreadNamePrefix("push-"); // named → readable
    thread dumps ★
    executor.setRejectedExecutionHandler(new
    ThreadPoolExecutor.CallerRunsPolicy()); // backpressure ★
    executor.setWaitForTasksToCompleteOnShutdown(true); // graceful
    shutdown ★
    executor.setAwaitTerminationSeconds(30);
    return executor;
}
```

■ How to tell this story (45 seconds)

"On RoboGebra we had one shared `@Async` pool, and push notification fan-out was starving it — a single announcement touches every teacher on the platform, and each device is a blocking FCM round trip, so PostHog analytics events would queue behind it. I gave push its own `ThreadPoolTaskExecutor` with a bounded 500-item queue and a `CallerRunsPolicy` rejection handler, so a backlog slows the submitter down instead of growing an unbounded queue until the heap gives out — a late push beats an OOM. I also had to declare Boot's own `applicationTaskExecutor` by hand, because `TaskExecutorConfiguration` is `@ConditionalOnMissingBean(Executor.class)` — the moment you declare any

executor, Boot's auto-configured one backs out and every unqualified `@Async` silently resolves to your pool."

★ That last sentence about `@ConditionalOnMissingBean` is a **genuinely senior** observation. It directly demonstrates the auto-configuration answer in [06 — Spring Boot](#).

Related concepts it proves you know:

- ✓ core vs max pool size
- ✓ rejection policies
- ✓ graceful shutdown
- ✓ config back-off ★
- ✓ why one shared pool is a risk (head-of-line blocking)
- ✓ bounded queue = backpressure
- ✓ named threads for thread dumps
- ✓ `@ConditionalOnMissingBean` / auto-

`@Cacheable` → [06 — Spring Boot](#), [17 — Design Patterns \(Proxy\)](#)

File: `robogebra-portal/.../domain/exercisesolution/service/ExerciseSolutionService.java` (16 usages across the codebase)

```
@Cacheable(value = CacheNames.EXERCISE_SOLUTION)
public ExerciseSolutionDTO getSolution(String exerciseItemId) { ... }

@Cacheable(value = CacheNames.EXERCISE_SOLUTION, key = "'admin-' +
#exerciseItemId")
public ExerciseSolutionDTO getAdminSolution(String exerciseItemId) { ... }
```

■ How to use it

"Solution content is expensive to assemble and almost never changes, so it's cached with `@Cacheable`. Note the custom `key` on the admin variant — two methods caching into the same cache region need distinct keys or the admin view would serve the student's cached payload."

★ Follow-up you should volunteer: "`@Cacheable` is proxy-based like `@Transactional`, so calling it from inside the same bean bypasses the cache entirely." → [17 — Proxy pattern](#).

`@Transactional` → 06, 24

130 usages across the portal.

★ **The interesting nuance for MongoDB:** transactions in MongoDB require a **replica set** (Mongo 4.0+). That is a great point to raise — it shows you know `@Transactional` isn't free everywhere.

"We're on MongoDB, so `@Transactional` needs a replica set to work at all — it's not the same free lunch as a relational database. Most of our write paths are single-document, which Mongo makes atomic by itself, so we reach for a multi-document transaction only where we genuinely span collections."

Java records → 12 — Java 17

26 files use `public record`, e.g. `common/exception/QuizConflictException.java`, `domain/payment/service/RazorPayService.java`.

"We're on Java 17, so DTOs and small value objects are records — the payment service uses them for request/response payloads. What we don't do is make an entity a record, because the Mongo mapper (like Hibernate) needs a no-arg constructor and mutable fields."

Global exception handling → 06 — Spring Boot

Files:

- `common/exception/handler/ExceptionHandlerAdvice.java`
- `security/FilterChainExceptionHandler.java` ★

★ The second file is the interesting one, and a **great** thing to bring up:

"`@ControllerAdvice` only catches exceptions thrown inside the MVC dispatch. Anything thrown in a security filter happens before the `DispatcherServlet`, so the advice never sees it and the client gets a bare 500. We have a dedicated `FilterChainExceptionHandler` in the filter chain for exactly that gap."

That is a distinction most candidates have never thought about.

Bean validation → 06 — Spring Boot

92 usages of `@Valid` across the controllers, with `spring-boot-starter-validation` and `jakarta.validation-api 3.0.2`.

▲ Note the `jakarta`, not `javax` — Boot 3. → 12 — Java 17 migration section.

Spring Security + JWT → 06 — Spring Boot

Files: `config/SecurityConfig.java`, `security/` package, `java-jwt 4.4.0` + `jwks-rsa`, AWS **Cognito** for identity.

"Auth is JWT-based and stateless. We validate against Cognito's JWKS endpoint rather than a shared secret — `jwks-rsa` fetches and caches the public keys, so the API can verify a token without ever holding a signing key."

Part 2 — MongoDB / ODM topics

`@Document` + `MongoRepository` → 13 — MongoDB, 24 — ORM vs ODM

104 `@Document` classes, **85** repositories.

File: `domain/exerciseitem/repository/ExerciseItemRepository.java`

```

@Repository
public interface ExerciseItemRepository extends
MongoRepository<ExerciseItemEntity, String> {

    // derived query – Spring writes it from the METHOD NAME
    Optional<ExerciseItemEntity> findByExercise_IdAndIndex(String
exerciseId, String index);

    // an aggregation PIPELINE where a derived query isn't enough ★
    @Aggregation(pipeline = {
        "{ '$match': { 'exercise.$id': { '$in': ?0 } } }",
        "{ '$sort': { 'displayOrder': 1 } }"
    })
    List<ExerciseItemEntity> findAllByExerciseIds(List<ObjectId>
exerciseIds);
}

```

■ The N+1 story — in Mongo ★

This is your **best** N+1 answer, because it's real and it's yours:

"The classic N+1 shows up in Mongo too. Loading a chapter's exercises and then fetching each exercise's items one by one is one query plus N. I replaced that with a single `@Aggregation` pipeline that `$match` es on `exercise.$id: { $in: [...] }` and sorts by `displayOrder` — one round trip instead of N. That's the same fix as a JPA `JOIN FETCH`, just expressed as a pipeline."

27 `@Aggregation` pipelines in the codebase — quote that number.

Mongoose (Node) → 13 — MongoDB

Files: `robogebra-crm/src/models/` — `user.model.js`, `institute.model.js`, `transaction.model.js`, `coupon.model.js`, `affiliate.model.js`, `payout.model.js`, `webhook.model.js`, `counter.model.js`

Stack: Express 4.21 + **Mongoose 8.8** + jsonwebtoken 9, layered as `routes` → `controllers` → `services` → `models` with `middlewares/` and `validations/`.

"The CRM is a Node service using Mongoose. It's the same ODM idea as Spring Data MongoDB but with schema definitions in JavaScript, and it's where I'd point if someone asks whether I've worked outside Java."

★ Note `counter.model.js` — that's the classic Mongo **auto-increment sequence** pattern, since Mongo has no `AUTO_INCREMENT`.

Part 3 — Angular topics

HTTP interceptors → [04 — Angular](#), [38 — IQVIA](#)

File: `robogebra-mobile/src/app/core/interceptors/header-authorization.interceptor.ts`

This one file demonstrates **six** interview topics at once:


```

@Inject()
export class HeaderAuthorizationInterceptor implements HttpInterceptor,
OnDestroy {

    private readonly cancelRequests$: Subject<void> = new Subject<void>();

    intercept(req: HttpRequest<any>, next: HttpHandler):
Observable<HttpEvent<any>> {
        if (this.isAuthEndpoint(req.url)) {           // 1. don't attach a
token to /auth
            return this.invokeHttpCall(req, next);
        }

        let updatedHeader: HttpHeaders = req.headers;
        if (!req.headers.has('Content-Type') && !(req.body instanceof
FormData)) {
            updatedHeader = updatedHeader.set("Content-Type",
'application/json'); // 2. FormData guard ★
        }

        return this.authService.getAuthDetailsObservable().pipe(
            switchMap((authDetails: AuthDetails) =>
                // 3. switchMap
                if (authDetails != null) {
                    updatedHeader = updatedHeader.set("Authorization",
`Bearer ${authDetails.token}`);
                }
                const cloned = req.clone({ headers:
updatedHeader }); // 4. IMMUTABLE clone ★
                return this.invokeHttpCall(cloned, next);
            })
        );
    }

    private invokeHttpCall(req, next): Observable<HttpEvent<any>> {
        return next.handle(req).pipe(

            takeUntil(this.cancelRequests$), //
            / 5. mass-cancel on logout ★
            catchError((error: HttpResponse) =>
                this.handleUnAuthResponse(error, req, next)) // 6. 401 → refresh
            );
    }
}

```

■ The six things to say about it

1. `HttpRequest` is IMMUTABLE → you MUST `req.clone()`; mutating it does nothing ★
2. `FormData` guard – setting `Content-Type` manually BREAKS multipart file uploads,
because the browser must generate the boundary itself ★ ★
3. `switchMap` because the token itself arrives asynchronously (from storage)
4. The auth endpoints are skipped, or login would need a token to get a token
5. `takeUntil(cancelRequests$)` lets logout cancel every in-flight request at once ★
6. `catchError` centralises 401 → refresh-token → retry, in ONE place instead of
in every service

★ Point 2 is the one that makes interviewers sit up — it's a real bug most people have shipped.

Route guards → 04 — Angular

File: `robogebra-mobile/src/app/core/guard/auth-guard.service.ts`

```

canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot):
Observable<boolean> {
  return this.authService.isLoggedInObservable().pipe(
    tap(isLoggedIn => {
      if (!isLoggedIn) {
        this.authService.setRedirectUrl(state.url); // ★
        remember where they wanted to go
        this.router.navigate(['/'] + AppRoutes.LOGIN);
      }
    }),
    map(isLoggedIn => {
      if (!isLoggedIn) return false;

      if (this.authService.isLoggedInAsParent()) { // ★ ROLE-
based routing
        const targetPath = state.url.split('?')[0].replace(/^\//,
'');
        const isAllowed = PARENT_ALLOWED_ROUTES.some(r =>
          targetPath === r || targetPath.startsWith(r + '/'));
        if (!isAllowed) {
          this.router.navigate(['/'] +
AppRoutes.PARENT_DASHBOARD));
          return false;
        }
      }
      return true;
    })
  );
}

```

■ Two details worth volunteering

*"The guard returns an **Observable<boolean>**, not a plain boolean, because login state itself is async — the router waits for it to emit. And it stores the requested URL before redirecting, so after login the user lands where they originally wanted instead of on a generic home page. On top of authentication we do role-based routing: a parent account is only allowed into the parent routes and is bounced to the dashboard otherwise."*

Other guards: `login-guard.service.ts` (keeps a logged-in user off `/login`), `public-preview-guard.service.ts`, `anonymous-solution-guard.service.ts`.

RxJS in production → 20 — RxJS Operators

Real usage counts in `robogebra-mobile` (quote these — they prove it isn't textbook knowledge):

```
map 496 · filter 485 · takeUntil 426 · catchError 338 · finalize 254
switchMap 246 · BehaviorSubject 122 · forkJoin 22 · combineLatest 21
debounceTime 13 · distinctUntilChanged 9 · shareReplay 3 ·
takeUntilDestroyed 5
```

`debounceTime` + `switchMap` — the type-ahead search ★

File: `features/pages/study-materials/study-materials.component.ts:288`

```
this.loadVideosSubject$.pipe(
  takeUntil(this.destroy$),
  debounceTime(500),                                // ★ wait for the user to
stop typing
  tap(() => (this.isLoading = true)),
  filter(() => {                                     // ★ don't search on 1-2
characters
    const query = this.searchTerm?.trim() || '';
    return query.length === 0 || query.length >= 3;
  }),
  switchMap(() => this.studyMaterialService.search(query,
tags, ...)) // ★ CANCELS the previous request
)
```

"This is the canonical search pipeline. `debounceTime(500)` stops us firing a request per keystroke, the `filter` avoids a pointless search on one or two characters, and `switchMap` cancels the previous in-flight request — which is what prevents the out-of-order-response bug where a slow query for 'ma' lands after a fast one for 'maths' and overwrites the correct results."

★ That last clause is the **real reason** for `switchMap`, and it's what separates a memorised answer from an experienced one.

`forkJoin` — parallel startup calls

File: `app.component.ts:166`

```

this.isLoggedIn$.pipe(
  filter(Boolean),
  filter(() => !this.authService.isLoggedInAsParent()),
  switchMap(() => forkJoin({                                // ★ three calls IN
    PARALLEL
      userSummary:    this.userService.fetchUserSummary(),
      userProfile:    this.userService.fetchUserProfile(false),
      userPreference:
this.userService.fetchUserPreference(false)
    })),
  takeUntil(this.destroy$)
).subscribe({ next: async ({ userSummary, userProfile, userPreference })
=> { ... } });

```

*"Three independent calls after login. Sequentially that's the sum of the latencies; **forkJoin** makes it the slowest of the three. Note it's the object form of **forkJoin**, so the result destructures by name rather than by array index — much harder to get wrong."*

takeUntil(this.destroy\$) — 426 usages ★

The memory-leak answer, everywhere in the codebase:

```

private readonly destroy$ = new Subject<void>();

ngOnDestroy(): void {
  this.destroy$.next();
  this.destroy$.complete();
}

```

*"Every manual subscription is unsubscribed with **takeUntil(this.destroy\$)** — there are over 400 of them. **takeUntil** must be the last operator in the pipe, otherwise operators after it keep their own subscription alive. On Angular 16+ we've started using **takeUntilDestroyed()** instead, which removes the boilerplate."*

★ The "**takeUntil** must be last" rule is a classic follow-up.

File: `core/services/user-summary.service.ts` (122 **BehaviorSubject** usages overall)

```
@Injectable({ providedIn: 'root' }) // ★
singleton, tree-shakable
export class UserSummaryService {
  private userSummarySubject$ = new
  BehaviorSubject<UserSummaryGroupDTO>(null);

  getUserSummarySubject(): Observable<UserSummaryGroupDTO> {
    return this.userSummarySubject$.asObservable(); // ★ expose
    read-only
  }

  getUserSummary(): UserSummaryGroupDTO {
    return this.userSummarySubject$.getValue(); // ★
    synchronous read
  }
}
```

■ The "why not NgRx?" answer ★

"We use **BehaviorSubject** -backed services rather than NgRx.

BehaviorSubject gives the two things we actually needed — a current value on subscribe, and a synchronous `getValue()` for callers that can't subscribe, like the institute-code HTTP interceptor. NgRx would have added actions, reducers, effects and selectors for state that only a handful of components read. I'd reach for NgRx when state is shared across many unrelated features and I need time-travel debugging or strict traceability."

★ Note the pattern of **exposing** `asObservable()` and keeping the subject private — that stops any component calling `.next()` and mutating global state from anywhere. Say that.

ChangeDetectionStrategy.OnPush → 04 — Angular, 38 — IQVIA

Used in 6 components, on exactly the right ones — the heavy list and modal views:

```
features/pages/learning-progress/chapter-progress-list/    ← a long list
features/pages/learning-progress/chapter-detail-modal/
features/pages/settings/payments/
features/pages/books/exercise-solution/.../solution-step-ai-assistant-
editor/
shared/video-splash-screen/
```

"*OnPush* is applied selectively rather than globally — on the long progress lists and the AI-assistant editor, where default change detection was re-rendering on every unrelated event. With *OnPush* the component only re-checks when an *@Input* reference changes, an event fires inside it, or an *async* pipe emits — which is also why we pass new object references instead of mutating in place."

★ *trackBy* appears **55 times** — mention it alongside *OnPush* as the other half of list performance.

Route resolvers → 04 — Angular

27 resolvers in `core/resolvers/`: `book-list.resolver.ts`, `chapter.resolver.ts`, `exercise-context.resolver.ts`, `chapter-access-ruler.resolver.ts`, `children.resolver.ts` ...

"We pre-fetch route data with resolvers so a page never renders in a half-loaded state. The access-ruler resolvers are the interesting ones — they decide what a user is allowed to see for that chapter or exercise before the route activates, so the component never has to render a 'you don't have access' flash."

Standalone components & modern Angular → 04, 38 — IQVIA

Be **honest and precise** here — it's a strength, not a weakness:

```
robogebra-mobile is Angular 18.2, but still largely NgModule-based:
standalone: true           → 2 components (migration started)
inject()                   → 30 usages    (modern DI, adopted)
takeUntilDestroyed()       → 5 usages    (modern teardown, adopted)
signal() / computed()      → 0           ▲ not adopted yet
```

"We're on Angular 18 but the app is still mostly NgModule-based — it grew from Angular 12 and a big-bang standalone migration was never worth the regression risk. We've adopted the pieces that are cheap and safe: `inject()` in newer code, `takeUntilDestroyed()` for teardown, and standalone for new components. Signals I've studied but haven't shipped — the migration path we'd take is signal inputs on leaf components first, since those are the lowest-risk."

★ **This is the right answer.** Claiming full signals adoption invites one follow-up you can't back up. Naming the *migration strategy* is what a lead is expected to have.

Angular forms → 25 — Angular Binding & Forms

```
Validators.* → 67 usages
FormBuilder → 2 usages    ▲ mostly template-driven / manual FormGroup
```

"Most forms are reactive with explicit `FormGroup`s and custom validators — 67 validator usages across the app. The parent-info form is the interesting one: it does an async availability check with `debounceTime` and `distinctUntilChanged` before validating."

File: `features/pages/settings/profile/parent-info/parent-info.component.ts:365`

Part 4 — Ionic & Capacitor topics

Capacitor plugins → 15 — Ionic Level 1, 38 — IQVIA

19 Capacitor plugins in production. This is your strongest Ionic evidence — quote the list:

@capacitor/app	app state, deep links, back button
@capacitor/browser	in-app browser
@capacitor/device	device info
@capacitor/haptics	tactile feedback
@capacitor/keyboard	keyboard events + resize behaviour
@capacitor/network	online/offline detection ★
@capacitor/preferences	native key-value storage ★
@capacitor/push-notifications	+ @capacitor-firebase/messaging (FCM) ★
@capacitor/screen-orientation	
@capacitor/splash-screen	
@capacitor/status-bar	
@capacitor-community/firebase-analytics	
@capawesome/capacitor-app-update	in-app update prompts ★
capacitor-native-settings	deep-link into OS settings
capacitor-razorpay	native payments ★

■ The story to tell

"The mobile app is Ionic 8 with Capacitor 6 and about nineteen native plugins. The ones with real complexity were push notifications — Capacitor Firebase messaging on the client and Firebase Admin on the Spring side, with a dedicated bounded thread pool for the fan-out — and native Razorpay for payments, because the web checkout flow doesn't survive an app-store review. We also use `@capacitor/network` for offline detection and `@capacitor/preferences` rather than `localStorage`, since WebView storage can be evicted by the OS."

★ That last point — **Preferences over localStorage** because the OS can evict **WebView storage** — is a genuinely good, specific Ionic answer.

★ `@capawesome/capacitor-app-update` is another strong detail: it shows you thought about the fact that **users don't update apps**, unlike a web deploy.

Part 5 — React (the CRM web)

robogebra-crm-web → useful if anyone asks "only Angular?"

```
React 18.3 + Vite + TypeScript 5.8
TanStack Query 5      ← server-state caching (the React answer to a state
library)
react-hook-form 7      ← forms
react-router-dom 6     ← routing
Radix UI + shadcn/ui  ← headless component primitives
```

"The CRM front end is React with Vite, TanStack Query and react-hook-form. I'm primarily an Angular developer, but I've worked across both — the mental model transfers directly: TanStack Query is doing what an NgRx entity store plus effects would do, and react-hook-form maps onto reactive forms."

Part 6 — Cross-cutting: what this codebase proves you can discuss

Interview topic	Your evidence
Thread pools, @Async , backpressure	AsyncConfig — bounded queue + CallerRunsPolicy ★
Spring auto-configuration	the @ConditionalOnMissingBean(Executor.class) back-off story ★
Caching	16 × @Cacheable with custom keys
N+1 and query optimisation	27 @Aggregation pipelines replacing per-item lookups ★
ORM vs ODM	Spring Data MongoDB + Mongoose — and knowing the difference ★

Interview topic	Your evidence
Exception handling	<code>@ControllerAdvice</code> plus a filter-chain handler for pre-MVC errors ★
Security / JWT	Cognito + JWKS validation, stateless
RxJS depth	426 <code>takeUntil</code> , 246 <code>switchMap</code> , the <code>debounce+switchMap</code> search ★
HTTP interceptors	token attach, FormData guard, mass-cancel, 401 refresh ★
Route guards + resolvers	4 guards, 27 resolvers, role-based routing
Change detection / performance	selective <code>OnPush</code> + 55 × <code>trackBy</code>
State management	<code>BehaviorSubject</code> services, and <i>why not NgRx</i> ★
Ionic / Capacitor	19 plugins, push, native payments, offline ★
Migration judgement	Angular 18 on an NgModule codebase — a staged plan, not a big bang ★
Polyglot	Java, TypeScript, Node/Express, React

▲ Three things to be careful about

1. RoboGebra uses MONGODB, not JPA.
The unused `spring-data-jpa` dependency in the pom is a TRAP.
Say "Spring Data MongoDB — an ODM". ★
2. Signals are NOT used (0 usages). Don't claim them.
Say what you've studied and what your migration plan would be. ★
3. Standalone components are barely adopted (2). The app is NgModule-based.
Frame it as a deliberate risk decision, not as being behind. ★

■ The 60-second "tell me about your architecture" answer

"RoboGebra is an AI-assisted maths learning platform. The backend is Java 17 on Spring Boot 3.2 with MongoDB — around 110 REST controllers and 200 services, using Spring Data MongoDB with aggregation pipelines where derived queries aren't enough. Auth is stateless JWT validated against AWS Cognito's JWKS; we use S3 for assets, Firebase for push, Razorpay for payments, and Quartz for scheduled jobs.

*On the front end there are three Angular applications plus a React CRM. The flagship is the Ionic 8 / Capacitor 6 mobile app on Angular 18, with about nineteen native plugins. State is managed with **BehaviorSubject** -backed services rather than NgRx, since the sharing is localised; route guards and resolvers handle access control and pre-fetching, and there's a single HTTP interceptor for token attachment and 401 refresh.*

*The parts I'd call out as engineering rather than plumbing: separating the push-notification thread pool with a bounded queue and caller-runs backpressure, and replacing per-item lookups with **\$match** / **\$in** aggregation pipelines to kill an N+1."*

Related files: [05 Core Java](#) · [06 Spring Boot](#) · [13 MongoDB](#) · [24 ORM/JPA](#) · [32 Multithreading](#) · [04 Angular](#) · [15 Ionic](#) · [20 RxJS](#) · [18 RoboGebra Technical Versions](#)

Java 21 — Version-Specific Features (Easy Version)

Why this file exists: Java 21 is the **current LTS** most companies are migrating to, and "what's new in 21?" is now a standard follow-up to any Java 17 question. **Virtual threads** in particular come up constantly.

● **Your honest position:** RoboGebra is on **Java 17 / Spring Boot 3.2**. So this is "*studied, not shipped*" — and the answer that scores is **what you'd use it for and what the migration would cost**, not a claim you've run it in production. There's a ready-made script for that at the end.

Every topic follows the same shape — **Easiest way to remember** → **old way vs new way** → **real-world example** → **diagram** → **interview trap** → **Interview-ready answer** → **Easy memory box**.

Contents

1. [The LTS timeline — where 21 sits](#)
 2. [Virtual Threads ★★ the headline](#)
 3. [Pattern Matching for `switch` ★](#)
 4. [Record Patterns ★](#)
 5. [Sequenced Collections ★](#)
 6. [Structured Concurrency & Scoped Values \(preview\)](#)
 7. [Generational ZGC](#)
 8. [What came after 21](#)
 9. [Java 17 vs 21 — the answer table](#)
 10. [Your answer scripts](#)
 11. [Quick Revision Sheet](#)
-

The LTS timeline — where 21 sits

8 — 11 — 17 — 21 — 25 ← LTS ★
2014 2018 2021 2023 2025

★ Java 21 (Sept 2023) is the LTS most enterprises are on or moving to.
Java 25 (Sept 2025) is the newest LTS.

Spring Boot 3.x → requires Java 17 minimum, runs happily on 21 ★
Virtual threads → Spring Boot 3.2 supports them with ONE property ★

★ **The line to say:** "17 was the boilerplate release — records, sealed types, pattern matching. 21 is the concurrency release — virtual threads are the biggest change to how Java handles I/O since NIO."

Easy memory

LTS: 8 → 11 → 17 → 21 ★ → 25
Java 17 = less BOILERPLATE ★ | Java 21 = CONCURRENCY ★
Spring Boot 3.2 + Java 21 → virtual threads with one property ★

1. Virtual Threads — the headline feature ★★

Deep dive with the thread-pool context: [32 — Multithreading, Part 16.](#)

The easiest way to remember:

A PLATFORM thread = 1 Java thread : 1 OS thread → ~1 MB → thousands max
A VIRTUAL thread = scheduled by the JVM → ~hundreds of bytes →
MILLIONS ★

And when it BLOCKS on I/O it UNMOUNTS from its carrier thread
instead of holding an OS thread hostage. ★ ★

The problem it solves

THREAD-PER-REQUEST with platform threads:

10,000 concurrent requests → 10,000 OS threads → ~10 GB → dead ▲
So Tomcat caps at ~200 threads, and request 201 QUEUES

The "solution" before 21 was REACTIVE (WebFlux):

- ✓ scales beautifully
- ✗ but your code becomes Mono/Flux chains, stack traces become unreadable,
- and one blocking JDBC call ruins the whole thing ▲

Java 21's answer: keep the SIMPLE blocking code, make the threads cheap ★
★

The code

```
// one virtual thread, directly
Thread.startVirtualThread(() -> doWork());

// a virtual thread PER TASK – the normal way ★
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (var task : tasks) executor.submit(task);
} // close() waits for all of them – try-with-resources ★

// Spring Boot 3.2 – literally one property ★ ★
// application.properties
spring.threads.virtual.enabled=true
```

PLATFORM THREADS

req1 → thread1 → waiting
req2 → thread2 → waiting
...
waiting)
req201 → no thread left ▲

VIRTUAL THREADS

req1 ┘
req2 ┘ → a few CARRIER threads ▲
req3 ┘ (each UNMOUNTS while

req10000 → fine ✓

Real-world idea: **a restaurant.**

PLATFORM → each waiter stands at ONE table until the meal ends
20 waiters = 20 tables, maximum.

VIRTUAL → a waiter takes the order, walks away, and comes back when
the food is ready ▲ – 20 waiters serve 200 tables ★

▲ The three caveats — say these, they're what separate real knowledge

1. DON'T POOL virtual threads ★ ★

They're cheap to CREATE — pooling defeats the entire purpose.
`newVirtualThreadPoolExecutor()`, never `newFixedThreadPool`.

2. They do NOT speed up CPU-BOUND work ★

The gain is from UNMOUNTING while blocked on I/O.
CPU work still needs a core. Use platform threads / parallel streams
there.

3. PINNING ★

In Java 21, a virtual thread inside a `synchronized` block could be
PINNED to its carrier — it couldn't unmount, so the benefit was
lost.

→ `ReentrantLock` was preferred in hot paths.

(Later JDKs improved this, but on 21 it's the correct caveat to
state.)

★ Also worth knowing: **thread-local variables become expensive** with millions
of threads — which is exactly why **Scoped Values** were introduced alongside
them.

Interview-ready answer

Virtual threads, finalised in Java 21, are lightweight threads scheduled by the JVM
rather than the operating system. A platform thread maps one-to-one onto an OS
thread and costs around a megabyte of stack, so a server caps out in the low
thousands; a virtual thread costs a few hundred bytes, so millions are viable. The
key behaviour is that when a virtual thread blocks on I/O it unmounts from its
carrier thread instead of holding an OS thread, which makes the simple thread-
per-request model scale like reactive code while keeping ordinary blocking code
that's readable and debuggable. In Spring Boot 3.2 it's a single property. The
caveats are that you should never pool them, they don't help CPU-bound work,

and on 21 a **synchronized** block could pin a virtual thread to its carrier, so **ReentrantLock** was preferred in hot paths.

Easy memory

Virtual thread = JVM-scheduled, ~hundreds of bytes, MILLIONS possible ★
UNMOUNTS on blocking I/O ★ ★ (a waiter who walks away)

```
Executors.newVirtualThreadPerTaskExecutor() |  
spring.threads.virtual.enabled=true ★
```

▲ DON'T POOL them ★ | no help for CPU-bound ★ | synchronized can PIN ★
Blocking code that scales like reactive – without the Mono/Flux ★

2. Pattern Matching for **switch** (final in 21) ★

The easiest way to remember:

switch can now match on TYPE, not just on a value. ★

```
// BEFORE – the if/else instanceof ladder  
static String describe(Object o) {  
    if (o instanceof Integer i)      return "int " + i;  
    else if (o instanceof String s)  return "string of length " +  
s.length();  
    else if (o == null)              return "null";  
    else                            return "something else";  
}  
  
// JAVA 21 ★  
static String describe(Object o) {  
    return switch (o) {  
        case Integer i      -> "int " + i;  
        case String s       -> "string of length " + s.length();  
        case null           -> "null";                // ★ null is a CASE  
now  
        default             -> "something else";  
    };  
}
```

Three things that are new

- | | |
|-----------------------|---|
| 1. TYPE PATTERNS | case Integer i -> ★ |
| 2. `case null` | a switch no longer throws NPE on null ★ |
| 3. GUARDS with `when` | case Integer i when i > 100 -> ★ |

```
return switch (shape) {  
    case Circle c when c.radius() > 10 -> "big circle";    // ★ a guarded  
    pattern  
    case Circle c                                -> "small circle";  
    case Square s                                -> "square";  
};
```

★ The killer combination — sealed + switch = compiler-checked exhaustiveness

```
sealed interface Shape permits Circle, Square, Triangle { }  
  
String area(Shape s) {  
    return switch (s) {  
        case Circle c    -> "πr2";  
        case Square sq   -> "side2";  
        case Triangle t  -> "½bh";  
    }; //  
★ NO default needed – the compiler knows the full list  
}
```

Add a fourth shape to the `permits` list

↓

EVERY switch that doesn't handle it FAILS TO COMPILE ★ ★

↓

You find it at BUILD time, not at 2 a.m. in production

▲ **The default trap:** if you add a **default** branch, you *lose* that exhaustiveness check — the new type silently falls into **default**. **For a sealed type, omit default on purpose.**

Real-world idea: a **postal sorting machine** that knows every valid destination. Add a new city to the list and it refuses to run until someone tells it which bin that city goes in.

Easy memory

switch on TYPE ★ | case null ★ | `when` guards ★

SEALED + switch = compiler-checked EXHAUSTIVENESS ★ ★
→ a new subtype BREAKS THE BUILD, not production

▲ Adding `default` DESTROYS the exhaustiveness check ▲ – omit it deliberately

Versions: instanceof pattern = 16 · switch patterns + record patterns = 21 ★

3. Record Patterns (final in 21) ★

The easiest way to remember:

Record patterns DESTRUCTURE a record right in the pattern – you get the components as variables, without calling the accessors. ★

```
record Point(int x, int y) { }  
record Line(Point start, Point end) { }
```

```
// BEFORE  
if (o instanceof Point p) {  
    int x = p.x();           // manual extraction  
    int y = p.y();  
    System.out.println(x + y);  
}  
  
// JAVA 21 ★  
if (o instanceof Point(int x, int y)) {           // ★ destructured  
    System.out.println(x + y);  
}
```

It NESTS — which is where it gets powerful ★

```
// pull all four numbers out of a nested structure, in one pattern ★  
if (o instanceof Line(Point(var x1, var y1), Point(var x2, var y2))) {  
    double length = Math.hypot(x2 - x1, y2 - y1);  
}
```

And combined with `switch`:

```
String describe(Object o) {  
    return switch (o) {  
        case Point(int x, int y) when x == y -> "on the diagonal";    //  
★    pattern + guard  
        case Point(int x, int y)                -> "point " + x + ", " + y;  
        case Line(Point s, Point e)              -> "line from " + s + " to "  
+ e;  
        default                                  -> "unknown";  
    };  
}
```

Real-world idea: **unpacking a parcel in one move**. Instead of opening the box, taking out the inner box, and then taking out the item, you state the shape you expect and everything lands in your hand already labelled.

★ The three features are designed to work **together**:

```
RECORDS (16) → the data shape  
SEALED (17) → the closed set of shapes  
PATTERNS (21) → matching on them EXHAUSTIVELY ★
```

That trio is Java's answer to what other languages call
ALGEBRAIC DATA TYPES – say that phrase and it lands ★

Easy memory

```
if (o instanceof Point(int x, int y)) ★ – destructure in the pattern  
NESTS: Line(Point(var x1, var y1), Point(var x2, var y2)) ★  
Works inside switch, with `when` guards ★
```

records + sealed + patterns = ALGEBRAIC DATA TYPES ★ (say this)

4. Sequenced Collections — the one nobody knows ★

The easiest way to remember:

Before 21, "get the first element" was a DIFFERENT method for every collection.
Java 21 gave them all ONE interface. ★

The problem — genuinely embarrassing

```
list.get(0); // List
deque.getFirst(); // Deque
sortedSet.first(); // SortedSet
linkedHashSet.iterator().next(); // LinkedHashSet – no method at all!

// and the LAST element:
list.get(list.size() - 1); // List
deque.getLast(); // Deque
sortedSet.last(); // SortedSet
// LinkedHashSet – you had to iterate the WHOLE thing ▲
```

Four collections that all have a clear order... and four different APIs ★

The Java 21 fix

```
interface SequencedCollection<E> extends Collection<E> {
    void addFirst(E e);    void addLast(E e);
    E getFirst();          E getLast(); // ★ the same
    everywhere
    E removeFirst();       E removeLast();
    SequencedCollection<E> reversed(); // ★ a REVERSED VIEW
}
```

```
List<String> list = new ArrayList<>(List.of("a", "b", "c"));
list.getFirst(); // "a" ★ works on List now
list.getLast(); // "c" ★
list.reversed(); // [c, b, a] – a VIEW, not a copy ★

LinkedHashSet<String> set = new LinkedHashSet<>(List.of("x", "y"));
set.getFirst(); // "x" ★ finally

LinkedHashMap<String,Integer> map = new LinkedHashMap<>();
map.firstEntry(); map.lastEntry(); map.reversed(); // ★
SequencedMap
```

New interfaces:

SequencedCollection ★ → List, Deque, LinkedHashSet, SortedSet
SequencedSet
SequencedMap ★ → LinkedHashMap, SortedMap

★ **reversed()** returns a **VIEW, not a copy** — so it's O(1) and changes write through. Say that; it's the detail that shows you read the API rather than the headline.

Real-world idea: **four remote controls that all had a different "power" button.** Java 21 didn't add a new feature — it put the button in the same place on all of them.

▲ Why it matters practically:

LinkedHashSet is exactly what you use for "unique, in insertion order" ★
→ before 21, getting its last element meant iterating the whole set ▲

Easy memory

SequencedCollection ★ → getFirst() getLast() addFirst() addLast()
reversed()
SequencedMap ★ → firstEntry() lastEntry() reversed()

The pain it removes: list.get(0) vs deque.getFirst() vs sortedSet.first()
reversed() is a VIEW, not a copy ★ (O(1), writes through)
Biggest win: LinkedHashSet finally has getFirst/getLast ★

5. Structured Concurrency & Scoped Values (preview)

▲ **Both were PREVIEW in Java 21** — mention them as "the direction of travel", not as something you'd ship.

Structured Concurrency

The idea: if a task splits into subtasks, they should live and DIE together —
like a block scope, but for threads. ★

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Supplier<User> user = scope.fork(() -> fetchUser(id));    // ★
    subtasks
        Supplier<Order> order = scope.fork(() -> fetchOrder(id));

    scope.join();                // wait for both
    scope.throwIfFailed();        // ★ if EITHER failed, the other is
    CANCELLED

    return new Dashboard(user.get(), order.get());
} // nothing outlives the block ★
```

The problem it fixes ★

With an `ExecutorService`, if one of two parallel calls fails, the other keeps running – wasting work and leaking a thread. Nothing ties them together, and the stack trace doesn't show the relationship ▲

Structured concurrency makes the **RELATIONSHIP** explicit – one fails, the sibling is cancelled, and the scope can't be left open ★

Real-world idea: **a group booking**. If the flight can't be confirmed, you don't want the hotel quietly booking itself anyway.

Scoped Values

A safer, cheaper replacement for `ThreadLocal` ★ – immutable, and automatically bounded to a scope instead of leaking in a pooled thread.

```
private static final ScopedValue<User> CURRENT_USER =
    ScopedValue.newInstance();

ScopedValue.where(CURRENT_USER, user).run(() -> handleRequest());
// inside: CURRENT_USER.get() ★ – and it's gone when the block ends
```

★ **Why it exists:** with **millions of virtual threads**, a `ThreadLocal` per thread is expensive and easy to leak — you must `remove()` it in a `finally`. Scoped values are immutable and self-clearing.

Easy memory

STRUCTURED CONCURRENCY (preview) ★ – subtasks live and DIE together
StructuredTaskScope.ShutdownOnFailure → one fails, the sibling is CANCELLED
(a group booking – no hotel without the flight)

SCOPED VALUES (preview) ★ – an immutable, self-clearing ThreadLocal
Exists BECAUSE virtual threads make ThreadLocal expensive and leaky ★

▲ Both were PREVIEW in 21 – say "the direction of travel", not "we use it"

6. Generational ZGC

ZGC = the ultra-low-pause collector (sub-millisecond, even on huge heaps).
Java 21 made it GENERATIONAL – so it also exploits "most objects die young". ★

Before: ZGC scanned the whole heap every cycle → low pause, high CPU cost
After: young objects are collected separately ★ → same tiny pauses, much less CPU and memory overhead

-XX:+UseZGC -XX:+ZGenerational

★ Enough to say: *"G1 is still the default; ZGC is for latency-critical services with large heaps, and Java 21 made it generational, which removed most of its overhead."*

Easy memory

G1 = the DEFAULT ★ | ZGC = sub-millisecond pauses, big heaps
Java 21 → GENERATIONAL ZGC ★ (young/old split → far less overhead)

What came after 21

▲ Be careful and brief here — knowing the *shape* is enough, and over-claiming is risky.

Java 22 (Mar 2024)	unnamed variables & patterns FINAL (`_`) Foreign Function & Memory API final
Java 23 (Sep 2024)	★ STRING TEMPLATES WITHDRAWN – previewed in 21/22, then REMOVED for redesign
Java 24 (Mar 2025)	further preview work
Java 25 (Sep 2025)	★ the next LTS after 21

★ The String Templates story is a genuinely good thing to mention:

```
// this was PREVIEW in 21 – and it is NOT in the language today ▲  
String s = STR."Hello \{name}";
```

"String templates were previewed in 21 but withdrawn in 23 for redesign — so I wouldn't build on them. It's a good example of why I check whether a feature is preview or final before adopting it."

★ That sentence shows you **verify** rather than repeat blog posts — and it protects you from confidently describing a feature that no longer exists.

Easy memory

```
22 → unnamed variables `_` final | 23 → STRING TEMPLATES WITHDRAWN  
★ | 25 → next LTS  
"I check preview vs final before adopting" ★
```

Java 17 vs 21 — the comparison they want

	Java 17 (2021 LTS)	Java 21 (2023 LTS)
Theme	less boilerplate ★	concurrency ★
Records	✓ (16)	✓

	Java 17 (2021 LTS)	Java 21 (2023 LTS)
Sealed classes	✓ (17)	✓
Pattern matching <code>instanceof</code>	✓ (16)	✓
Pattern matching <code>switch</code>	preview	✓ final ★
Record patterns	✗	✓ final ★
Virtual threads	✗	✓ final ★★
Sequenced collections	✗	✓ ★
Structured concurrency	✗	preview
Scoped values	✗	preview
Generational ZGC	✗	✓

The one-sentence answer ★

"Java 17 was the boilerplate release — records, sealed types, pattern matching for `instanceof`. Java 21 is the concurrency release: virtual threads make thread-per-request cheap again, and it finalised pattern matching for `switch` and record patterns, which together with records and sealed types give you exhaustive, compiler-checked domain modelling. The small one people forget is sequenced collections — `getFirst()` and `getLast()` finally work the same way everywhere."

Your Answer Scripts

■ "Are you on Java 21?" — the honest answer ★

"Our backend is Java 17 on Spring Boot 3.2, so 21 is something I've studied rather than shipped. What I'd move for is virtual threads — we have a push-notification fan-out where each device is a blocking FCM call, and I currently manage that with a dedicated bounded thread pool. On 21 that's a virtual-thread executor and the

*pool tuning largely goes away. Spring Boot 3.2 already supports it with a single property, so the migration cost is low — the real work is auditing for **synchronized** blocks in hot paths, because on 21 those can pin a virtual thread to its carrier."*

★ **That answer is strong** because it names a *real* problem in your own code that Java 21 would solve, and it names the migration risk.

■ "What would you use virtual threads for?"

"I/O-bound, thread-per-request work — REST endpoints that call other services or a database, and fan-out work like notifications. Not CPU-bound work, because unmounting only helps while you're blocked. And I wouldn't pool them; the whole point is that creating one is cheap."

■ "Should we migrate from 17 to 21?"

*"The upgrade itself is usually cheap — 21 is largely source-compatible with 17 and Spring Boot 3 already supports both. I'd frame it by what you get: virtual threads if you're I/O-bound and hitting thread-pool limits, and pattern matching plus record patterns if you have domain modelling that's currently a chain of **instanceof**. If neither of those is a real pain, 17 is still supported and there's no urgency — I'd rather move for a reason than for the version number."*

★ **That is a technical-lead answer** — it treats a version bump as a cost/benefit decision, not a default.

Quick Revision Sheet

LTS: 8 → 11 → 17 → 21 ★ → 25

Java 17 = less BOILERPLATE | Java 21 = CONCURRENCY ★

VIRTUAL THREADS ★ ★ (the headline)

JVM-scheduled, ~hundreds of bytes, MILLIONS possible

UNMOUNT on blocking I/O ★ → thread-per-request is cheap again

`Executors.newVirtualThreadPerTaskExecutor()`

Spring Boot 3.2: `spring.threads.virtual.enabled=true` ★

▲ DON'T POOL ★ | no CPU-bound gain ★ | synchronized can PIN ★

PATTERN MATCHING for switch ★ (final in 21)

`case Integer i -> · case null -> · case X x when guard ->`

SEALED + switch = compiler-checked EXHAUSTIVENESS ★ ★

▲ adding `default` DESTROYS that check ▲

RECORD PATTERNS ★ (final in 21)

`if (o instanceof Point(int x, int y))` – and it NESTS

records + sealed + patterns = ALGEBRAIC DATA TYPES ★

SEQUENCED COLLECTIONS ★ (the one nobody knows)

`getFirst() getLast() addFirst() addLast() reversed()`

`SequencedCollection · SequencedSet · SequencedMap`

`reversed()` is a VIEW, not a copy ★

Biggest win: `LinkedHashSet` finally has `getFirst/getLast` ★

PREVIEW in 21

Structured concurrency ★ – subtasks live and DIE together

Scoped values ★ – immutable `ThreadLocal` replacement (needed BECAUSE of millions of virtual threads)

ALSO Generational ZGC ★ (G1 is still the default)

AFTER 22 unnamed ` \_ ` final · 23 STRING TEMPLATES WITHDRAWN ★ · 25 next LTS

● YOUR POSITION ★

"We're on Java 17 / Spring Boot 3.2 – 21 is studied, not shipped.

What I'd move for is virtual threads: our push fan-out is blocking FCM calls managed with a bounded pool, and on 21 that's a virtual-thread executor. The migration risk is `synchronized` in hot paths – pinning."

Related files: [12 — Java 17 Features](#) · [32 — Multithreading \(virtual threads in context\)](#) · [05 — Core Java](#) · [33 — Interface vs Abstract Class \(sealed types\)](#) · [39 — RoboGebra Code Examples](#)

● Capgemini — Java + Spring + Angular round (Sept 2026)

Attended: early Sept 2026 (edit the exact date here — the file is named 3sep as a placeholder). ● **Result pending.**

You walked out with all 10 questions but did NOT write the last program. That program is Q10 below, solved four ways with the Stream API — **type it, don't read it.** It is a twist on "first non-repeating character", which Altimetrik already asked you (26 §Altimetrik Q3). That family is now on its 2nd round.

Shape of the round: ~10 questions — 2 Java core → 4 Spring / architecture → 3 Angular → 1 Java Streams live-coding.

The signal: this was a **breadth sweep**, not a depth round. Nothing here goes below the API you use daily — but the last question was live coding, and **that is where the round is decided.**

The 10 questions at a glance

#	Question exactly as asked	Area	Answered below
1	Java 8 new features	Java ★	→
2	HashMap and LinkedHashMap	Collections ★	→
3	Setter injection and constructor injection	Spring DI ★	→
4	@Component and @Bean	Spring ★	→
5	How do microservices load values?	Spring Boot config	→

#	Question exactly as asked	Area	Answered below
6	Monolithic and microservices	Architecture ★	→
7	Components in Angular	Angular ★	→
8	How do you create a new component?	Angular CLI	→
9	ngOnChanges and ngDoCheck	Angular ★	→
10	● String name = "Priyanka" — second non-repeating character from the END , using Stream API	Java Streams ●	→

★ = already asked in an earlier round (repeat). ● = live coding. **7 of the 10 are repeats** — the bank really is ~25 questions.

Part 1 — Java core

1. Java 8 new features

One line: Java 8 is the release where Java became **functional** — lambdas let you pass behaviour as a value, and everything else (streams, method references, **Optional**, default methods) exists to make that useful.

The list — say it in five groups, not as 15 bullets

Group	What's in it
1. Lambdas & functional interfaces ★	<code>(a, b) -> a + b</code> · <code>@FunctionalInterface</code> (exactly one abstract method) · the built-in set: <code>Function</code> , <code>Predicate</code> , <code>Consumer</code> , <code>Supplier</code> , <code>BiFunction</code> , <code>UnaryOperator</code>
2. Stream API ★	<code>filter</code> / <code>map</code> / <code>sorted</code> / <code>distinct</code> / <code>limit</code> (intermediate, lazy) → <code>collect</code> / <code>forEach</code> / <code>reduce</code> / <code>count</code> / <code>anyMatch</code> (terminal, eager) · <code>Collectors.groupingBy</code> / <code>toMap</code> / <code>joining</code> / <code>partitioningBy</code> · <code>parallelStream()</code>
3. Interface changes	default methods (add a method without breaking every implementer — this is how <code>Collection.stream()</code> was added) and static methods in interfaces
4. Optional	A container that is either present or empty — <code>of</code> / <code>ofNullable</code> / <code>empty</code> , <code>map</code> , <code>filter</code> , <code>orElse</code> / <code>orElseGet</code> / <code>orElseThrow</code> . A return type, not a field type
5. New Date/Time API (<code>java.time</code>) ★	<code>LocalDate</code> , <code>LocalTime</code> , <code>LocalDateTime</code> , <code>ZonedDateTime</code> , <code>Duration</code> , <code>Period</code> , <code>DateTimeFormatter</code> — immutable and thread-safe , unlike <code>Date</code> / <code>Calendar</code> / <code>SimpleDateFormat</code>

The smaller ones worth naming if they push: method references

(`Employee::getName`), `CompletableFuture` (async composition), `StringJoiner` ,
`Arrays.parallelSort` , `Map` helpers (`getOrDefault` , `computeIfAbsent` , `merge` ,
`forEach`), repeatable annotations, and **PermGen** → **Metaspace** (a JVM change, not a language one).


```
// the whole release in one snippet
List<String> names = employees.stream()           // Stream
API                                             // lambda +
    .filter(e -> e.getSalary() > 50_000)         // method
Predicate                                     // method
    .map(Employee::getName)                     // method
reference                                     // method
    .sorted()
    .collect(Collectors.toList());

Optional<Employee> top = employees.stream()       // Optional
    .max(Comparator.comparingDouble(Employee::getSalary));

String name = top.map(Employee::getName).orElse("none"); // no null
check

LocalDate joined = LocalDate.of(2021, 6, 1);      // java.time
long years = ChronoUnit.YEARS.between(joined, LocalDate.now());
```

Why it matters — the sentence that upgrades the answer

- "Before 8, you said *how* to loop. After 8, you say *what* you want and the library decides how — which is what makes `parallelStream()` a one-word change instead of a rewrite. **Default methods are the quiet hero**: without them, adding `stream()` to `Collection` would have broken every implementation of `List` on earth."

▲ The three traps

1. **Streams are lazy.** `list.stream().filter(...)` with no terminal operation does **nothing** — not one element is touched.
2. **A stream is single-use.** Re-using one → `IllegalStateException: stream has already been operated upon or closed`.
3. **`parallelStream()` is not free.** It uses the shared **common ForkJoinPool**; for small collections, or inside a web request already running on a thread pool, it is usually **slower**. Say "*I reach for it only for large, CPU-bound, stateless work*".

◆ Easy memory box — Java 8 = "L.S.I.O.D."

Lambdas · Streams · Interface default/static methods · Optional · Date-Time API.

- Deeper: [05 — Core Java](#) · [22 — Java Streams: 20 coding problems](#) · follow-up they often ask next: [12 — Java 17](#) and [40 — Java 21](#).

2. HashMap vs LinkedHashMap (and TreeMap)

They said "*HashMap and LinkedHashMap*" — answer it as **one family with one difference**, then let them pull you deeper.

One line: all three are **Map** s; they differ **only in iteration order** and what that order costs you.

HashMap	→ NO order guarantee	→ $O(1)$ get/put	→ fastest, least memory
LinkedHashMap	→ INSERTION order (or access order)	→ $O(1)$	→ + 2 pointers per entry
TreeMap	→ SORTED by key	→ $O(\log n)$	→ Red-Black tree, key must be Comparable

The comparison table

	HashMap	LinkedHashMap	TreeMap
Order	None (looks random, and changes on resize)	Insertion order by default; access order if <code>accessOrder=true</code>	Sorted by natural order or a Comparator
Structure	Array of buckets + linked list → tree at 8	HashMap + a doubly-linked list through all entries	Red-Black tree
get / put	$O(1)$ average	$O(1)$ average	$O(\log n)$

	HashMap	LinkedHashMap	TreeMap
null key	✓ one	✓ one	✗ NullPointerException
Memory	Lowest	+2 references (before / after) per entry	Node overhead
Thread-safe	✗	✗	✗ (use ConcurrentHashMap / ConcurrentSkipListMap)

How HashMap actually works (the follow-up, every time)

```

put("Priyanka", 1)
├── hash = key.hashCode() ^ (h >>> 16)    // spread the high bits
├── index = hash & (n - 1)                // n is a POWER OF 2 → cheap
modulo
├── bucket[index]
│   ├── empty        → put the node here
│   ├── same key     → replace the value (equals() decides)
│   └── collision     → append to the linked list
│                       └── list length ≥ 8 AND table ≥ 64 → TREEIFY
(Red-Black tree, O(log n))
                                (if table < 64 it RESIZES instead of
treeifying)

Default capacity 16 · load factor 0.75 → resize (double) at 12 entries ·
resize rehashes everything

```

LinkedHashMap — the one thing that makes it special

It is literally **HashMap** **plus** a doubly-linked list stitched through the entries, so iteration is a walk down that list:

```
Map<Character, Integer> m = new LinkedHashMap<>();
m.put('P', 1); m.put('r', 2); m.put('i', 3);
// iteration → P, r, i ALWAYS. A HashMap gives no such promise.
```

And the killer use case — an LRU cache in 6 lines:

```
class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int max;
    LruCache(int max) { super(16, 0.75f, true); this.max = max; } //
    true = ACCESS order
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> eldest)
    {
        return size() > max; //
    }
    evict least-recently-used
}
```

■ **The sentence that lands it:** "I use **HashMap** by default. I switch to **LinkedHashMap** the moment **order is part of the answer** — building a response the UI renders in order, or counting characters where I have to report the *first* one. And **Q10 in this round is exactly that case:** a **HashMap** would have scrambled the character order and broken the answer."

▲ The traps worth volunteering

1. **A mutable key is a lost entry.** Put an object in, change a field used by `hashCode()`, and `get()` looks in the wrong bucket. Keys should be **immutable** — **String**, boxed types, records.
 2. **equals / hashCode come as a pair.** Override one without the other and lookups silently fail.
 3. **HashMap under concurrency** — in Java 7 a resize could create an **infinite loop**; since 8 it "only" loses data. Either way: **ConcurrentHashMap**.
 4. **Treeify needs Comparable-ish keys.** Without a natural order it falls back to comparing class names/identity hashes — still $O(\log n)$, but the point is it isn't magic.
- Full depth (asked at Virtusa R2 and Altimetrik): **31 — HashMap Internals**.

Part 2 — Spring & architecture

3. Setter injection vs constructor injection

One line: constructor injection = the dependency is mandatory; setter injection = the dependency is optional or replaceable. Spring's own docs recommend **constructor injection** for almost everything.

```
// ✓ CONSTRUCTOR INJECTION – the default you should be using
@Service
public class OrderService {
    private final PaymentClient payment;          // final = can never be
    null, never reassigned
    private final OrderRepository repo;

    // @Autowired is OPTIONAL since Spring 4.3 when there is exactly ONE
    constructor
    public OrderService(PaymentClient payment, OrderRepository repo) {
        this.payment = payment;
        this.repo = repo;
    }
}

// SETTER INJECTION – for genuinely optional dependencies
@Service
public class ReportService {
    private Notifier notifier = new NoOpNotifier();          // sensible
    default

    @Autowired(required = false)
    public void setNotifier(Notifier notifier) { this.notifier =
    notifier; }
}
```

The comparison

	Constructor ✓	Setter
Dependency is...	Mandatory	Optional / changeable
Fields can be <code>final</code>	✓ → immutable, thread-safe	✗

	Constructor ✓	Setter
Missing bean fails...	At startup (fail fast) ★	At startup too, unless <code>required=false</code> — but the object exists half-built
Object is always valid	✓ You cannot construct it wrong	✗ Valid only <i>after</i> every setter has run
Testing without Spring	✓ <code>new OrderService(mockA, mockB)</code>	Needs <code>new</code> + n setter calls
Circular dependency	▲ Fails loudly at startup ★	Silently "works" — it hides a design smell
Too many dependencies	Painfully obvious → tells you the class does too much ★	Easy to hide 12 of them

The third one they may name: field injection

```
@Autowired private PaymentClient payment;    // ✗ don't
```

Why it's discouraged: the field can't be `final`; the dependency is invisible from the outside (nothing in the public API says the class needs it); you cannot construct the object in a plain unit test without reflection; and it makes it effortless to accumulate ten dependencies without noticing. IntelliJ literally warns "*Field injection is not recommended.*"

The circular-dependency answer (the follow-up)

A needs B, B needs A

Constructor injection → `BeanCurrentlyInCreationException` at STARTUP.
Spring cannot build either one first.

★ Good – it made you look.

Setter injection → Spring creates both, then wires them. It "works",
and the bad design survives to production.

Since Spring Boot 2.6 circular references are `DISALLOWED` by default
(`spring.main.allow-circular-references=true` re-enables them – a plaster,
not a fix).

The real fix: extract the shared logic into a third bean, or use an event.

■ **Interview-ready answer:** "I use **constructor injection everywhere** — it lets me make the fields `final`, so the object can never exist in a half-built state, and a missing bean blows up at startup instead of as a `NullPointerException` in production. Since Spring 4.3 I don't even need `@Autowired` when there's one constructor. I keep **setter injection for genuinely optional dependencies** with a sensible default. And I treat a **circular dependency as a design signal**, not something to configure around."

■ More Spring wiring depth (`@Qualifier` vs `@Primary`, scanning across packages): [06 — Spring Boot](#) · [37 Q3–Q7](#).

4. `@Component` VS `@Bean`

One line: `@Component` = "Spring, build this class for me" (you own the class).

`@Bean` = "I'll build the object, you manage it" (you don't own the class).

```
// @Component – on YOUR OWN class, found by component scanning
@Service // @Service IS a @Component
public class OrderService { ... }

// @Bean – on a METHOD inside a @Configuration class, for a class you
// can't annotate
@Configuration
public class AppConfig {

    @Bean // bean name = the METHOD name
    → "restClient"
    public RestClient restClient(RestClient.Builder builder) {
        return builder.baseUrl("https://api.example.com")
            .defaultHeader("X-App", "robogebra")
            .build();
    }

    @Bean
    @ConditionalOnMissingBean // conditional creation –
    impossible with @Component
    public ObjectMapper objectMapper() {
        return new ObjectMapper().registerModule(new JavaTimeModule())
            .disable(SerializationFeature.WRITE_DATES
                _AS_TIMESTAMPS);
    }
}
```

	@Component (and @Service / @Repository / @Controller)	@Bean
Applied to	A class	A method
Who constructs it	Spring (via the constructor)	You , in the method body
Discovered by	Component scanning (@ComponentScan)	Declared inside a @Configuration class
Bean name	Class name, decapitalised → orderService	Method name → restClient
Third-party classes	✗ You can't add an annotation to a JAR	✓ The whole reason @Bean exists

	<code>@Component</code> (and <code>@Service</code> / <code>@Repository</code> / <code>@Controller</code>)	<code>@Bean</code>
Conditional / multiple instances	Awkward	✓ Easy — <code>if</code> s, <code>@Conditional</code> , two methods returning the same type

When to use which — the decision line

Is it a class in MY source tree?

YES → `@Component` / `@Service` / `@Repository` / `@Controller`

NO → `@Bean` in a `@Configuration` class

Do I need to configure it, or create several of the same type?

YES → `@Bean` (a method can take arguments, branch, read properties)

The stereotype family (they usually ask this next)

`@Service`, `@Repository` and `@Controller` are all **meta-annotated with `@Component`** — Spring treats them identically for scanning. The difference is intent, *except `@Repository`, which is the only one with real behaviour*: it enables **persistence exception translation** (a vendor `SQLException` becomes Spring's `DataAccessException`).

▲ The `proxyBeanMethods` trap ★

```
@Configuration                                // proxyBeanMethods = true by default →
CGLIB proxy
public class AppConfig {
    @Bean public A a() { return new A(); }
    @Bean public B b() { return new B(a()); }    // a() does NOT run twice
    —
}                                                // the proxy returns the
SAME singleton ★
```

With `@Configuration(proxyBeanMethods = false)` (or `@Component` instead of `@Configuration`) that call is a **plain Java call** and you get **two different A instances**. Volunteer this — it's the one detail that shows you've been bitten by it.

5. How do microservices load values?

The question is **externalised configuration**: *how does one build of the service get the right URLs, credentials and toggles in dev, QA and prod?*

One line: one artefact, many environments — the jar never changes, only what is injected into it.

The four ways to read a value

```
// 1. @Value – a single value, with a default after the colon
@Value("${payment.timeout-ms:5000}")
private int timeoutMs;

// 2. @ConfigurationProperties – TYPE-SAFE binding of a whole block ✓
preferred
@ConfigurationProperties(prefix = "payment")
@Validated
public record PaymentProperties(@NotBlank String baseUrl,
                               @Positive int timeoutMs,
                               boolean retryEnabled) { }

// 3. Environment – programmatic lookup
@Autowired Environment env;
String url = env.getProperty("payment.base-url");

// 4. @Profile – a whole BEAN only exists in some environments
@Bean @Profile("!prod")
public DataSeeder devSeeder() { return new DataSeeder(); }
```

```
# application.yml – shared defaults
payment:
  base-url: http://localhost:8081
  timeout-ms: 5000
---
# application-prod.yml – overrides for prod only
payment:
  base-url: https://payments.internal
  timeout-ms: 2000
```

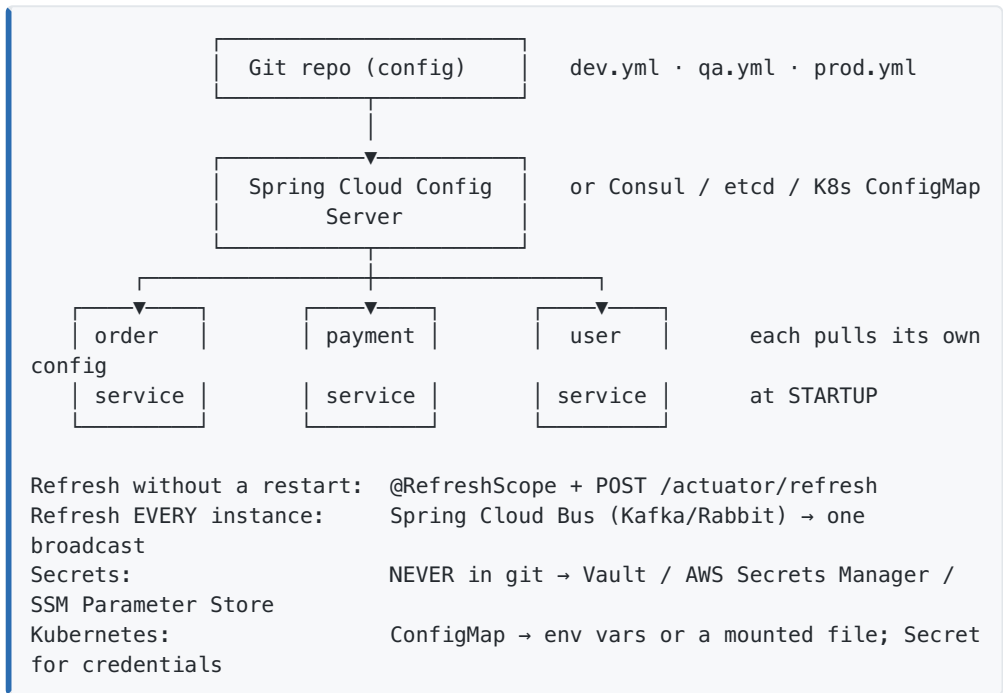
Activate with `--spring.profiles.active=prod` or
`SPRING_PROFILES_ACTIVE=prod`.

The precedence order — say it as "later wins" ★

```
HIGHEST — command-line args      --payment.timeout-ms=2000
      |— OS environment variables  PAYMENT_TIMEOUT_MS=2000    ←
relaxed binding
      |— application-{profile}.yml (outside the jar)
      |— application.yml           (outside the jar)
      |— application-{profile}.yml (inside the jar)
      |— application.yml           (inside the jar)
LOWEST — @Value / @ConfigurationProperties defaults
```

Relaxed binding is the detail worth naming: `payment.timeout-ms` in YAML is the same property as `PAYMENT_TIMEOUT_MS` as an env var. That is what makes Docker and Kubernetes work without touching the jar.

At microservice scale — the part that makes it a *microservices* answer



■ **Interview-ready answer:** "The build artefact is identical in every environment — what changes is what's injected. Defaults live in `application.yml`, environment differences in `application-{profile}.yml`, and anything that differs per deployment comes in as an **environment variable**, which wins over the file because of the precedence order. I bind config with `@ConfigurationProperties` rather than scattered `@Value`s, so it's type-safe and validated at startup. At scale you centralise it in a **Config Server backed by git** — you get versioned, auditable config and `@RefreshScope` for changing a value without a redeploy. **Secrets never go in git or in `application.yml`** — those come from Vault or AWS Secrets Manager."

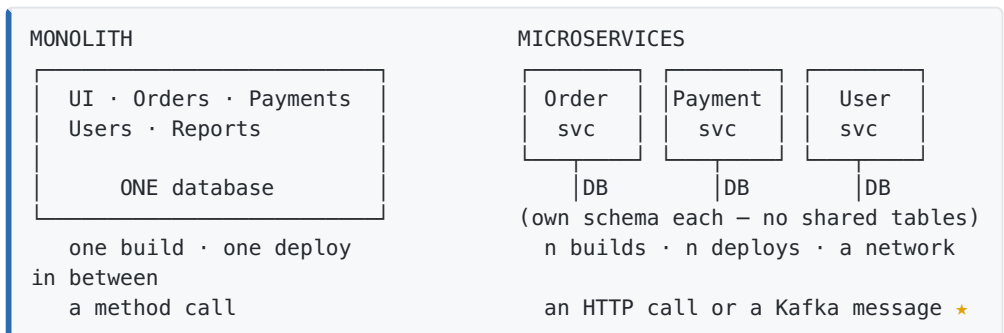
▲ Traps

- `@Value` in a constructor works; `@Value` on a field read from a constructor is still `null` — field injection happens *after* construction.

- `@ConfigurationProperties` needs to be registered — `@EnableConfigurationProperties`, `@ConfigurationPropertiesScan`, or `@Component` on the class.
 - Angular's `environment.ts` is compiled into the bundle and is **PUBLIC**.
Never put a secret there. (Repeat of the same lesson in [38 — IQVIA.](#))
- **06 — Spring Boot · 37 Q14** (dev/QA/prod, asked 4 days earlier — **the same question, twice in one week**).

6. Monolithic vs microservices

One line: a monolith is one deployable unit; microservices are many independently deployable units, each owning its own data.



	Monolith	Microservices
Deployment	One artefact	One per service
Data	One schema, real ACID transactions ★	Database per service → eventual consistency, Saga instead of <code>@Transactional</code>
Calls between modules	In-process — fast, always succeeds	Over a network — can be slow, can fail, can arrive twice ★
Scaling	Scale the whole app	Scale only the hot service

	Monolith	Microservices
A bad release	Takes everything down	Blast radius is one service (if you added a circuit breaker)
Team fit	Small team, one codebase	Many teams that must deploy independently ★
Tech choice	One stack	Per service
Debugging	A stack trace	Distributed tracing (correlation ID across services)
Ops cost	Low	High — CI/CD per service, gateway, discovery, monitoring, on-call

The answer that separates you from a blog post ★

■ "Microservices are an **organisational** solution before they're a technical one — you split when independent teams need to deploy independently. The cost is real: every in-process method call becomes a network call that can be slow, fail, or arrive twice, and a database transaction becomes a **Saga with compensating actions**. So the sequence I believe in is **modular monolith first** — clean module boundaries, one deployment — and you extract a service when there's a concrete reason: a different scaling profile, a different release cadence, or a different team owning it."

▲ The one you must get right about your own project ●

RoboGebra is a MODULAR MONOLITH — say so.

There is **no Eureka, no Feign, no circuit breaker and no Kafka** in your codebase. Claiming "microservices" survives exactly one follow-up ("*how do your services discover each other?*") and then costs you the round.

The honest version is the stronger answer: *"RoboGebra is a Spring Boot modular monolith on MongoDB — the modules are separated by package and by service boundary, but it deploys as one unit, because the team is small and the*

release cadence is shared. I've designed against microservice patterns and I can talk through them, but I'd be overstating it to say I run a fleet in production."

The anti-pattern to name

The distributed monolith: services split across the network but still deployed together, still sharing a database, still requiring a coordinated release. **All of the cost, none of the benefit.** Naming this makes you sound like someone who has seen it.

■ **08 — Microservices Basics** · failure vocabulary (timeout · circuit breaker · retry+backoff · idempotency key · outbox · DLQ · compensating transaction): **37 Q11–Q13**.

Part 3 — Angular

7. Components in Angular

One line: a component is **one piece of the screen plus the logic that drives it** — a TypeScript class with an `@Component` decorator that binds a **template** (what the user sees) to **state** (what the class holds).

```

@Component({
  selector: 'app-student-card',          // the tag: <app-student-card>
  templateUrl: './student-card.component.html',
  styleUrls: ['./student-card.component.scss'],
  // standalone: true,                    // Angular 14+; NgModule-based in
  your project
})
export class StudentCardComponent implements OnInit, OnChanges {
  @Input() student!: Student;            // data IN from the
  parent
  @Output() selected = new EventEmitter<string>(); // event OUT to the
  parent

  constructor(private studentService: StudentService) {} // DI only –
  inputs are undefined here

  ngOnInit(): void { /* inputs are ready – safe to call the API */ }

  onClick(): void { this.selected.emit(this.student.id); }
}

```

The four files the CLI creates

```

student-card/
├── student-card.component.ts    the class – state + behaviour
├── student-card.component.html  the template – what renders
├── student-card.component.scss  styles – SCOPED to this component by
default ★
└── student-card.component.spec.ts the unit test

```

The `@Component` metadata worth naming

Property	What it does
<code>selector</code>	The tag / attribute / class the component matches
<code>template</code> / <code>templateUrl</code>	Inline HTML vs an external file
<code>styles</code> / <code>styleUrls</code>	Scoped CSS
<code>providers</code>	A new instance of a service per component instance ★

Property	What it does
<code>changeDetection</code>	<code>Default</code> vs <code>OnPush</code> (only re-check on input <i>reference</i> change, event, or async pipe)
<code>encapsulation</code>	<code>Emulated</code> (default — Angular adds <code>_ngcontent</code> attributes) · <code>None</code> (leaks globally ▲) · <code>ShadowDom</code>
<code>animations</code> , <code>host</code> , <code>imports</code> (standalone)	

An app is a tree of components



Component communication (asked in **4 separate rounds** now):

`@Input()` parent→child · `@Output()` + `EventEmitter` child→parent · a **shared service with a `BehaviorSubject`** for unrelated components · `@ViewChild` for direct access · route params / `NgRx` store for app-wide state.

▲ **Honesty guard for your projects:** you use the `NgModule` style with `declarations` , not standalone components (**2 usages**), and you have **no signals** (0 usages). Say *"I know standalone and signals are the direction Angular is going, and I've read into them — the codebase I work in is NgModule-based on Angular 16"*. That reads as current; claiming daily use does not survive a follow-up.

■ **04 — Angular** · **39 — RoboGebra Code Examples**.

8. How do you create a new component?

The answer they want first — the CLI:

```
ng generate component student-card
ng g c student-card           # short form
```

What the CLI does for you (say all four — this is the actual answer):

1. creates the folder `src/app/student-card/`
2. creates 4 files `.ts · .html · .scss · .spec.ts`
3. adds `@Component` with selector `'app-student-card'` and the correct file paths
4. REGISTERS it → adds `StudentCardComponent` to ``declarations`` in the nearest `NgModule` ★
(or, with `--standalone`, makes it standalone instead)

Then you use it by putting its selector in a parent template: `<app-student-card [student]="s" (selected)="onSelect($event)"></app-student-card>`

The flags worth knowing

Flag	Why
<code>--dry-run / -d</code>	Show what it would create without creating it — always safe
<code>--skip-tests</code>	No <code>.spec.ts</code>
<code>--inline-template / -t</code> , <code>--inline-style / -s</code>	Small components in one file
<code>--flat</code>	No folder
<code>--module=admin</code>	Register it in a specific module (matters once you have lazy-loaded feature modules)
<code>--standalone</code>	Angular 14+; default from Angular 17
<code>--change-detection=OnPush</code>	Sets <code>OnPush</code> from the start
<code>--prefix=admin</code>	<code><admin-student-card></code> instead of <code>app-</code>
<code>ng g c features/students/student-card</code>	Path = folder structure

Doing it by hand (they sometimes ask "and without the CLI?")

Create the `.ts` with an `@Component` decorator, create the template, then **add the class to `declarations`** in the `NgModule` — and to `exports` if another module needs it. **Forgetting that registration is the #1 cause of** `'app-student-card' is not a known element`.

▲ The classic error and its three causes: `'app-x' is not a known element`
→
(1) not declared in any module · (2) declared in module A but used in module B **without being `exported`** · (3) a typo in the selector. For a standalone component the cause is different: it's missing from the consumer's `imports` array.

■ 04 — Angular · 25 — Binding & Forms.

9. `ngOnChanges` VS `ngDoCheck`

★ Second round running — Photon asked this exact pair (26 §Photon Q6).

One line: `ngOnChanges` fires only when an `@Input()` 's *reference* changes. `ngDoCheck` fires on *every single* change-detection cycle — which is why one is cheap and the other is dangerous.

	<code>ngOnChanges</code>	<code>ngDoCheck</code>
When	Before <code>ngOnInit</code> , then every time an <code>@Input</code> reference changes	Every change-detection run — a click, a timer, an HTTP response, anywhere in the app
Fires with no <code>@Input</code> s?	✗ Never	✓ Always
Argument	<code>changes:</code> <code>SimpleChanges</code> — <code>previousValue</code> , <code>currentValue</code> , <code>firstChange()</code>	None — you do the comparing

	ngOnChanges	ngDoCheck
Cost	Cheap	▲ Can run hundreds of times a second
Use it for	React to a new input value	Detect a change Angular cannot see : a mutated object/array

```
export class StudentCardComponent implements OnChanges, DoCheck {
  @Input() student!: Student;
  private lastName = '';

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['student'] && !changes['student'].firstChange) {
      this.reload(changes['student'].currentValue); // new REFERENCE
    }
  }

  ngDoCheck(): void {
    if (this.student?.name !== this.lastName) {      // catches a
    MUTATION
      this.lastName = this.student.name;             // ngOnChanges never
    fired for this
    }
  }
}
```

The whole point — why `ngDoCheck` exists ★

```
Parent does: this.student = { ...this.student, name: 'New' }; // NEW
object
→ reference changed → ngOnChanges FIRES ✓

Parent does: this.student.name = 'New'; // SAME
object, mutated
→ reference identical → ngOnChanges does NOT fire ✗
→ the template still updates (default CD re-reads bindings)
  but your ngOnChanges logic never ran
→ ngDoCheck is where you catch it ★
```

Angular compares inputs with `===` (reference equality), not deep equality.

That single fact answers the whole question.

The hook order

```
constructor
  ↓
ngOnChanges      ← first, and again on every input change
  ↓
ngOnInit         ← once
  ↓
ngDoCheck        ← after every ngOnChanges, and on every CD cycle after
that
  ↓
ngAfterContentInit → ngAfterContentChecked
  ↓
ngAfterViewInit   → ngAfterViewChecked
  ↓
ngOnDestroy
```

▲ The rules that keep `ngDoCheck` safe

1. **Never do heavy work in it** — no API calls, no deep clones, no `JSON.stringify` of a big object. It runs on every cycle.
2. **Never mutate state that's bound to the template** in it → `ExpressionChangedAfterItHasBeenCheckedError` in dev mode.
3. Use `KeyValueDiffers` (objects) / `IterableDiffers` (arrays) instead of hand-rolled comparison — that's what Angular's own `NgClass` / `NgForOf` use.
4. **The better fix is usually not to need it:** treat inputs as **immutable** — replace the object instead of mutating it — and then `ngOnChanges` + `OnPush` handle everything, faster.

■ **The closing line:** "In practice I try never to need `ngDoCheck`. If I'm reaching for it, it usually means a parent is mutating an object it passed down — and the cleaner fix is to make the update immutable, which also lets me turn on `OnPush`."

■ **04 — Angular · 26 \$Photon Q6 · change detection depth: 38 — IQVIA.**

Part 4 — The live-coding question

10. ● Live coding — second non-repeating character from the END

This is the one you didn't write. Here it is in the **simplest possible way — 3 steps, Stream API**. No tricks. **Type it until you can write it without looking.** It is a twist on "*first non-repeating character*", which **Altimetrik already asked** — the family is now on its 2nd round, so learn it as a pattern, not as one answer.

The question, restated

```
String name = "Priyanka";  
// find the SECOND non-repeating character counting from the END
```

First — work it out on paper (10 seconds)

```
"Priyanka" → P r i y a n k a  
  
How many times each one comes:  
P=1   r=1   i=1   y=1   a=2   n=1   k=1  
  
Characters that come only ONE time:  P r i y n k  
Now count from the END:              k = 1st  
                                      n = 2nd ★  
  
ANSWER = n
```

Say this out loud before you type anything. It takes ten seconds and it makes the code obvious.

✓ THE PROGRAM — the simple one (write this)

Three steps. That's all it is.

Step 1 — count each character. Step 2 — keep the ones that came 1 time.

Step 3 — pick the 2nd from the last.

```
import java.util.*;
import java.util.stream.*;

public class Main {
    public static void main(String[] args) {

        String name = "Priyanka";

        // STEP 1 — count how many times each character comes
        Map<Character, Long> count = name.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(c -> c, LinkedHashMap::new,
Collectors.counting()));
        // {P=1, r=1, i=1, y=1, a=2, n=1, k=1}

        // STEP 2 — keep only the characters whose count is 1
        List<Character> single = count.entrySet().stream()
            .filter(e -> e.getValue() == 1)
            .map(e -> e.getKey())
            .collect(Collectors.toList());
        // [P, r, i, y, n, k]

        // STEP 3 — take the 2nd one from the END
        System.out.println(single.get(single.size() - 2));
    }
}
```

Output:

```
n
```

Line-by-line, in plain words

Line	What it does, simply
<code>name.chars()</code>	Break the word into its characters

Line	What it does, simply
<code>.mapToObj(c -> (char) c)</code>	<code>chars()</code> gives numbers — turn them back into letters
<code>Collectors.groupingBy(c -> c, ..., counting())</code>	Group the same letters together and count them → <code>{P=1, r=1, ... a=2, ...}</code>
<code>LinkedHashMap::new</code>	Keep them in the order they appeared ★ (a normal <code>HashMap</code> mixes up the order — then "2nd from the end" means nothing)
<code>.filter(e -> e.getValue() == 1)</code>	Keep only the letters that came one time
<code>.map(e -> e.getKey())</code>	I want the letter , not the count
<code>.collect(Collectors.toList())</code>	Put them in a list → <code>[P, r, i, y, n, k]</code>
<code>single.size() - 2</code>	Last position is <code>size - 1</code> , so 2nd from the end is <code>size - 2</code> ★

◆ Easy memory box — 3 steps:

① **count** → ② **keep count == 1** → ③ **take `size - 2`**.

That's the whole program. If you remember only one thing, remember `size - 2`.

● The one mistake to avoid

```
Collectors.groupingBy(c -> c, Collectors.counting())           // ✗ plain
HashMap — order is mixed up
Collectors.groupingBy(c -> c, LinkedHashMap::new, counting()) // ✓ keeps
P, r, i, y, n, k order
```

LinkedHashMap is the whole trick. Without it the list can come out as `[r, P, k, i, y, n]` and your answer is wrong. If they ask "why *LinkedHashMap*?" — "because the question asks about position, and `HashMap` has no order."

If they ask for more

"Do it without the list"

Same idea, but reverse the word first and just skip one:

```
Map<Character, Long> count = name.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, Collectors.counting()));

String reverse = new StringBuilder(name).reverse().toString();    //
"aknayirP"

char answer = reverse.chars()
    .mapToObj(c -> (char) c)
    .filter(c -> count.get(c) == 1)           // k, n, y, i, r, P
    .skip(1)                                // skip k (that's the 1st)
    .findFirst()                            // n ★
    .get();

System.out.println(answer);    // n
```

Simple idea: *2nd from the end of the word = 2nd from the start of the reversed word.*

"Now make it the 3rd one" (they usually do)

Change one number — make it a method:

```

static char nthFromEnd(String s, int n) {
    Map<Character, Long> count = s.chars().mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(c -> c,
Collectors.counting()));

    return new StringBuilder(s).reverse().toString().chars()
        .mapToObj(c -> (char) c)
        .filter(c -> count.get(c) == 1)
        .skip(n - 1) // ★ only this line changes
        .findFirst()
        .get();
}

nthFromEnd("Priyanka", 1); // k
nthFromEnd("Priyanka", 2); // n ★ the asked answer
nthFromEnd("Priyanka", 3); // y

```

"Now do it without the Stream API"

```

static char secondFromEnd(String s) {
    Map<Character, Integer> count = new LinkedHashMap<>();
    for (char c : s.toCharArray()) {
        count.put(c, count.getOrDefault(c, 0) + 1); // count
    }

    int found = 0;
    for (int i = s.length() - 1; i >= 0; i--) { // go backwards
        char c = s.charAt(i);
        if (count.get(c) == 1) {
            found++;
            if (found == 2) return c; // the 2nd one →
        }
    }
    return ' ';
}

```

The questions they ask after

They ask	You say (short and simple)
How fast is it?	" O(n) — one pass to count, one pass to find. I go through the word twice, not more."

They ask	You say (short and simple)
Why count first?	"I can't tell if the first letter repeats until I've seen the last letter. So counting has to finish before I can pick."
Why <code>LinkedHashMap</code> ?	"Because the question is about position , and a normal <code>HashMap</code> doesn't keep order."
Is it case sensitive?	"Yes — <code>P</code> and <code>p</code> would be different. I'd ask you if that's wanted; if not I'd do <code>name.toLowerCase()</code> first." (Here it doesn't change the answer.)
What if there is no 2nd one?	" <code>single.size() - 2</code> would fail, so I'd check <code>if (single.size() >= 2)</code> first, or return an <code>Optional</code> ."
First non-repeating instead?	"Same program — just take <code>single.get(0)</code> ." That's the Altimetrik version of this question.

✗ The one version NOT to write

```
name.chars().filter(c -> name.chars().filter(x -> x == c).count() == 1) // ▲ slow
```

It works, but it **goes through the whole word again for every single letter** — that's $O(n^2)$. The 3-step version above goes through it twice. If it slips out, just say "that's slow, let me count once first" and fix it.

◆ The whole family is the same 3 steps. Count → keep `count == 1` → pick the position they asked for:

Question	Change only step 3
First non-repeating	<code>single.get(0)</code>
Last non-repeating	<code>single.get(single.size() - 1)</code>
2nd from the end ★	<code>single.get(single.size() - 2)</code>

Question	Change only step 3
2nd from the start	<code>single.get(1)</code>
First repeating	change step 2 to <code>e.getValue() > 1</code> , then <code>get(0)</code>

■ More of this family: [22 — Java Streams: 20 coding problems](#) · [09 — Coding Problems](#) · [26 §Altimetrik Q3](#).

◆ What this round tells you

1. It was a breadth sweep — and 7 of the 10 were repeats. Java 8, HashMap, DI, `@Component` / `@Bean`, monolith-vs-microservices, Angular components, `ngOnChanges` / `ngDoCheck` — every one is already in your log. **The bank is ~25 questions and you have now seen it eleven times.**

2. The same question came back four days later. Altimetrik (31 Aug) asked "dev/QA/prod — different DBs and properties, how is that handled?"; Capgemini asked "how do microservices load values?" — **the same answer**. Likewise `@Component` vs `@Bean` at both. Re-reading [37](#) was worth marks in this round.

3. The round is decided by the coding question. Nine theory answers you already knew, then one program. **That is the pattern across Altimetrik, Photon, Codeboard and now Capgemini.** Theory gets you to the coding question; the coding question decides the outcome. From now on, **type one Streams problem a day** from [22](#) — reading them is not the same skill.

4. Say the answer out loud before you type it. For Q10 the paper work is ten seconds (`P r i y n k` → `k` → `n`) and it makes the code obvious. Going straight to the keyboard is what makes these questions feel hard.

▲ Mark your own fumbles — do this TODAY

Q	Topic	✓ answered well / ▲ fumbled / ✗ blank
1	Java 8 features	
2	HashMap vs LinkedHashMap	
3	Setter vs constructor injection	
4	@Component vs @Bean	
5	How microservices load values	
6	Monolith vs microservices	
7	Angular components	
8	Creating a component	
9	ngOnChanges vs ngDoCheck	
10	● Second non-repeating from the end	✗ not written — now solved above; type it

Related files: 26 — Companies (the recall log) · 05 — Core Java · 31 — HashMap Internals · 06 — Spring Boot · 08 — Microservices · 04 — Angular · 22 — Java Streams · 09 — Coding Problems · 37 — Altimetrik (31 Aug)

Java Coding + Streams – Final Easy-Memory Sheet

#	Question	Logic Snippet	What to Think	◆ Easy Memory
1	Reverse String	<code>for(i=len-1; i>=0; i--)</code>	Read characters from last to first	BACKWARD → ADD
2	Reverse Words	<code>split(" ")</code> + backward loop	Split into words, read backwards	SPLIT → BACKWARD → JOIN
3	Palindrome	<code>str.equals(reverse)</code>	Reverse string and compare	REVERSE → COMPARE
4	Anagram	<code>chars().sorted().toArray()</code>	Sort both strings and compare	SORT BOTH → EQUALS
5	First Non-Repeating	<code>indexOf(c) == lastIndexOf(c)</code>	Same first/last index means appears once	UNIQUE → FIRST
6	First Non-Repeating from End	<code>reverse → filter → findFirst</code>	Reverse first, then find unique	REVERSE → UNIQUE → FIRST
7	Second Non-Repeating from End	<code>reverse → filter → skip(1) → findFirst</code>	Skip first unique character	REVERSE → UNIQUE → SKIP 1 → FIRST
8	Missing Number	<code>expected – actual</code>	Expected sum minus array sum	EXPECTED – ACTUAL
9	Second Largest	<code>distinct().sorted(reverse).skip(1)</code>	Remove duplicate → descending → skip highest	DISTINCT → DESC → SKIP 1 → FIRST

#	Question	Logic Snippet	What to Think	◆ Easy Memory
10	Find Duplicates	<code>filter(n -> ! seen.add(n))</code>	<code>add()</code> false means already exists	!seen.add = DUPLICATE
11	Two Sum	<code>need = target - nums[i]</code>	Search previously seen need	TARGET - CURRENT = NEED
12	Character Frequency	<code>groupingBy(c -> c, counting())</code>	Put same characters together and count	GROUP → COUNT
13	Maximum / Minimum	<code>mapToInt().m ax()/min()</code>	Convert to numbers then max/min	NUMBER → MAX/MIN
14	Sort Objects	<code>sorted(Compa rator.compar ing(...))</code>	Tell Java which field to compare	SORT → FIELD
15	List → Map	<code>toMap(KEY, VALUE)</code>	Choose map key and value	KEY → VALUE
16	Sum / Average	<code>mapToInt().s um()/ average()</code>	Convert to numeric stream and calculate	NUMBER → SUM/AVG
17	Element Frequency	<code>groupingBy(n -> n, counting())</code>	Same numbers together + count	GROUP → COUNT
18	Remove Duplicates	<code>distinct()</code>	Keep unique values	REMOVE = DISTINCT
19	Reverse Using Stream	<code>range → reverse index → joining</code>	Generate indexes and read backwards	RANGE → BACKWARD → JOIN
20	Group Objects	<code>groupingBy(E mployee::get Department)</code>	Same department together	GROUP BY FIELD

#	Question	Logic Snippet	What to Think	◆ Easy Memory
21	Even / Odd	<code>partitioningBy(n -> n%2==0)</code>	Condition creates true/false groups	2 GROUPS → PARTITION
22	Longest String	<code>max(comparingInt(String::length))</code>	Compare strings by length	MAX → LENGTH
23	All Match	<code>allMatch(condition)</code>	Every element must pass	ALL → allMatch
24	Any Match	<code>anyMatch(condition)</code>	At least one must pass	ANY → anyMatch
25	None Match	<code>noneMatch(condition)</code>	No element should pass	NONE → noneMatch
26	Find First	<code>filter(...).findFirst()</code>	Filter then take first	FILTER → FIRST
27	Find Any	<code>filter(...).findAny()</code>	Filter then take any	FILTER → ANY
28	Stream → List	<code>collect(Collectors.toList())</code>	Collect result into List	COLLECT → LIST
29	Stream → Set	<code>collect(Collectors.toSet())</code>	Collect result into Set	COLLECT → SET
30	Join Strings	<code>Collectors.joining(", ")</code>	Put delimiter between strings	STRINGS → JOIN
31	Flatten Lists	<code>flatMap(Collection::stream)</code>	Open inner lists into one stream	NESTED → FLAT
32	Highest Paid / Department	<code>toMap(dept, employee, keepHigher)</code>	Same dept? Keep employee with higher salary	DEPT → COMPARE → KEEP HIGHER

The Logic Snippets You Actually Need to Remember

1. Reverse String — Normal

```
String reverse = "";

for (int i = str.length() - 1; i >= 0; i--) {
    reverse += str.charAt(i);
}
```

Think:

```
LAST → BACKWARD → ADD
```

2. Reverse Words — Normal

```
String[] words = str.split(" ");
String result = "";

for (int i = words.length - 1; i >= 0; i--) {
    result += words[i] + " ";
}
```

Think:

```
SPLIT
  ↓
GO BACKWARD
  ↓
ADD WORDS
```

3. Palindrome — Normal

Use the same reverse logic:

```
String reverse = "";

for (int i = str.length() - 1; i >= 0; i--) {
    reverse += str.charAt(i);
}

boolean result = str.equals(reverse);
```

Think:

```
REVERSE → SAME?
      ↓
    TRUE
```

4. Anagram

```
int[] a = str1.chars().sorted().toArray();
int[] b = str2.chars().sorted().toArray();

boolean result = Arrays.equals(a, b);
```

Think:

```
listen → sort → eilnst
silent → sort → eilnst
      ↓
    SAME
```

◆ **SORT BOTH → EQUALS**

5. First Non-Repeating

```
Character result = str.chars()
    .mapToObj(c -> (char) c)
    .filter(c -> str.indexOf(c) == str.lastIndexOf(c))
    .findFirst()
    .orElse(null);
```

Important logic:

```
str.indexOf(c) == str.lastIndexOf(c)
```

If first position and last position are the same, the character occurs only once.

◆ UNIQUE → FIRST

6. First Non-Repeating from END

```
String reverse =  
    new StringBuilder(str).reverse().toString();  
  
Character result = reverse.chars()  
    .mapToObj(c -> (char) c)  
    .filter(c -> str.indexOf(c) == str.lastIndexOf(c))  
    .findFirst()  
    .orElse(null);
```



```
REVERSE → UNIQUE → FIRST
```

7. Second Non-Repeating from END

Same program — add only:

```
.skip(1)
```

```
Character result = reverse.chars()  
    .mapToObj(c -> (char) c)  
    .filter(c -> str.indexOf(c) == str.lastIndexOf(c))  
    .skip(1)  
    .findFirst()  
    .orElse(null);
```



```
FIRST → findFirst  
SECOND → skip(1) → findFirst  
THIRD → skip(2) → findFirst
```

Very useful connection.

8. Missing Number

```
int n = nums.length;

int expected = n * (n + 1) / 2;
int actual = Arrays.stream(nums).sum();

int missing = expected - actual;
```



What SHOULD be there
-
What IS there
=
MISSING

9. Second Largest

```
Integer result = list.stream()
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .orElse(null);
```

Think:

```
[10,40,20,40,30]
```

```
DISTINCT
```

```
↓
```

```
10,40,20,30
```

```
DESC
```

```
↓
```

```
40,30,20,10
```

```
SKIP 1
```

```
↓
```

```
30
```

◆ **DISTINCT → DESC → SKIP 1 → FIRST**

10. Find Duplicates

```
Set<Integer> seen = new HashSet<>();  
  
Set<Integer> duplicate = list.stream()  
    .filter(n -> !seen.add(n))  
    .collect(Collectors.toSet());
```

Key:

```
seen.add(n)
```

```
true → first time  
false → already exists
```

Therefore:

```
!seen.add(n)
```

means duplicate.

◆ **FALSE ADD = DUPLICATE**

11. Two Sum

```
Map<Integer, Integer> map = new HashMap<>();

for (int i = 0; i < nums.length; i++) {

    int need = target - nums[i];

    if (map.containsKey(need))
        return new int[]{ map.get(need), i };

    map.put(nums[i], i);
}
```

Don't memorize the whole code first.

Memorize:

```
TARGET = 9

Current = 2
Need = 9 - 2 = 7

Current = 7
Need = 9 - 7 = 2

Have we already seen 2?
YES
→ ANSWER
```

◆ **NEED = TARGET – CURRENT**

12 & 17 Are the SAME FAMILY

Character Frequency

```
str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(
        c -> c,
        Collectors.counting()
    ));
```

Number Frequency

```
list.stream()
    .collect(Collectors.groupingBy(
        n -> n,
        Collectors.counting()
    ));
```

Both are:

```
SAME VALUES TOGETHER
      ↓
    GROUP
      ↓
    COUNT
```

◆ **FREQUENCY = GROUP + COUNT**

13 & 16 Are the SAME FAMILY ★

Start with:

```
list.stream()
    .mapToInt(Integer::intValue)
```

Then choose:

```
.sum()
```

or:

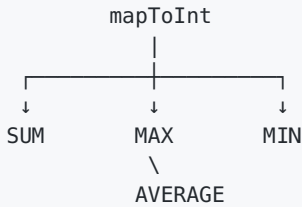
```
.max().orElse(0)
```

or:

```
.min().orElse(0)
```

or:

```
.average().orElse(0)
```



So don't memorize four programs.

Remember:

```
NUMBER CALCULATION = mapToInt
```

14. Sort Objects

```
employees.stream()
    .sorted(
        Comparator.comparing(Employee::getSalary)
    )
    .collect(Collectors.toList());
```

Change only the field:

```
Employee::getSalary
Employee::getName
Employee::getAge
```

◆ **SORT OBJECT → COMPARING(FIELD)**

15. List → Map

```
employees.stream()
    .collect(Collectors.toMap(
        Employee::getId,
        Employee::getName
    ));
```

Think:

ID		NAME
↓		↓
KEY		VALUE
1	→	Priya
2	→	Arun

◆ toMap(KEY, VALUE)

18. Remove Duplicates

```
list.stream()
    .distinct()
    .collect(Collectors.toList());
```

Don't confuse:

```
FIND duplicates
→ !seen.add()

REMOVE duplicates
→ distinct()
```

This distinction is worth memorizing.

19. Reverse String Using Stream

```
String result = IntStream.range(0, str.length())
    .mapToObj(i -> str.charAt(str.length() - 1 - i))
    .map(String::valueOf)
    .collect(Collectors.joining());
```

Key:

```
str.length() - 1 - i
```

means read from the end.

◆ **RANGE → BACKWARD → JOIN**

20. Group Objects

```
employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment
    ));
```

Result:

```
IT    → [Priya, Divya]
HR    → [Arun]
Sales → [Karthik]
```

◆ **GROUP OBJECT → groupingBy(FIELD)**

21. Even / Odd

```
list.stream()
    .collect(Collectors.partitioningBy(
        n -> n % 2 == 0
    ));
```

Think:

Condition
 $n \% 2 == 0$

↓

TRUE

FALSE

↓

↓

EVEN

ODD

◆ **TWO GROUPS** → **partitioningBy**

22. Longest String

```
list.stream()
    .max(
        Comparator.comparingInt(String::length)
    )
    .orElse(null);
```

Think:

```
Java      → 4
Angular   → 7
SpringBoot → 10
           ↑
          MAX
```

◆ **MAX** → **LENGTH**

23, 24, 25 — Remember Together

```
list.stream().allMatch(n -> n > 5);
list.stream().anyMatch(n -> n > 5);
list.stream().noneMatch(n -> n > 5);
```

◆

```
ALL? → allMatch  
ANY? → anyMatch  
NONE? → noneMatch
```

That's one family, not three programs.

26 & 27 — Remember Together

First:

```
list.stream()  
    .filter(n -> n > 20)  
    .findFirst()  
    .orElse(null);
```

Any:

```
list.stream()  
    .filter(n -> n > 20)  
    .findAny()  
    .orElse(null);
```



```
CONDITION  
  ↓  
FILTER  
  ↓  
FIRST / ANY
```

28 & 29 — Remember Together

List:

```
stream.collect(Collectors.toList());
```

Set:

```
stream.collect(Collectors.toSet());
```



```
Want LIST? → toList
Want SET? → toSet
```

30. Join Strings

```
list.stream()
    .collect(Collectors.joining(", "));
```

Java
Spring
Angular

↓ joining(", ")

Java, Spring, Angular

◆ STRINGS → JOINING

31. Flatten List of Lists

```
list.stream()
    .flatMap(Collection::stream)
    .collect(Collectors.toList());
```

Think:

[[1,2], [3,4], [5,6]]

↓ flatMap

[1,2,3,4,5,6]

◆ NESTED → OPEN → ONE LIST

32. Highest-Paid Employee per Department ★

Use the simpler `toMap` version:

```
Map<String, Employee> result = employees.stream()
    .collect(Collectors.toMap(
        Employee::getDepartment,
        e -> e,
        (e1, e2) ->
            e1.getSalary() > e2.getSalary() ? e1 : e2
    ));
```

Don't look at this as one big code.

Break it into 3 things:

```
Employee::getDepartment
```

means:

```
KEY = Department
```

Then:

```
e -> e
```

means:

```
VALUE = Employee
```

Then:

```
(e1, e2) ->
    e1.getSalary() > e2.getSalary() ? e1 : e2
```

means:

Same department?

↓

Compare both salaries

↓

Keep higher employee

Example:

Priya → IT → 75,000

Divya → IT → 85,000

Same key = IT

↓

Priya vs Divya

↓

85,000 is higher

↓

IT → Divya

◆ **DEPT → EMPLOYEE → COMPARE → KEEP HIGHER**

★ **FINAL MEMORY — Memorize Only This**

Before the interview, revise this block:

Reverse String

→ BACKWARD → ADD

Reverse Words

→ SPLIT → BACKWARD → JOIN

Palindrome

→ REVERSE → COMPARE

Anagram

→ SORT BOTH → EQUALS

First Non-Repeat

→ UNIQUE → FIRST

From End

→ REVERSE → UNIQUE → FIRST

Second From End

→ REVERSE → UNIQUE → SKIP 1 → FIRST

Missing Number

→ EXPECTED - ACTUAL

Second Highest

→ DISTINCT → DESC → SKIP 1 → FIRST

Find Duplicate

→ !seen.add()

Two Sum

→ TARGET - CURRENT = NEED

Frequency

→ GROUP + COUNT

Max/Min/Sum/Avg

→ mapToInt

Sort Object

→ comparing(FIELD)

List → Map

→ toMap(KEY, VALUE)

Remove Duplicate

→ distinct()

Group Object

→ groupingBy(FIELD)

Even/Odd

→ partitioningBy

Longest String

→ MAX → LENGTH

Condition

→ allMatch / anyMatch / noneMatch

Find

→ FILTER → FIRST / ANY

Stream → Collection

→ toList / toSet

Join Strings

→ joining

Nested Lists

→ flatMap

Highest Salary / Dept

→ DEPT → EMPLOYEE → COMPARE → KEEP HIGHER

The most important connections

Instead of 32 questions, mentally reduce them to about 9 families:

STRING → Reverse / Palindrome / Anagram

FIND → filter / findFirst

UNIQUE → distinct / seen.add

NUMBER → mapToInt

SORT → sorted / Comparator

GROUP → groupingBy / counting

MAP → toMap

CONDITION → match / partition

NESTED → flatMap

Related files: [22 — Java Streams: 20 Coding Problems](#) · [41 — Capgemini](#) · [37 — Altimetrik](#) · [09 — Coding Problems](#)

Java 17 — Rapid-Recall Flashcards

Cover the **A:** line, say your answer out loud, then check. Full detail in [12-java17-features.md](#).

▲ = high-yield in every round. Target: name 5 Java 17 features + where you'd use them, cold.

● **Your stack:** Java 17 · Spring Boot 3.2 · **26 record files** in the portal — records are used for DTOs and value objects, **never** for a persistence model (the Mongo mapper needs a no-arg constructor and mutable fields, same as Hibernate) ★

Q1 ▲ Why is Java 17 important?

A: It's an **LTS** release (Sept 2021), like 8/11/21. Bundles records, sealed classes, pattern matching, switch expressions, text blocks into a stable, supported version companies standardize on.

Q2 What is an LTS release?

A: Long-Term Support — years of security patches/updates, safe for production. Enterprises run 8 → 11 → 17 → 21. Non-LTS (12–16, 18–20) are short-lived preview releases.

Q3 ▲ Java 8 vs Java 17 headline differences?

A: 8 = lambdas/streams/Optional, switch statements, verbose POJOs, instanceof+cast. 17 adds **records, sealed classes, pattern matching, switch expressions, text blocks** (+ JPMS from 9).

Q4 ▲ What is a record?

A: Immutable data-carrier class. Compiler auto-generates constructor, accessors (`e.name()` — no "get"), `equals/hashCode/toString`. Ideal for **DTOs**. `public record Employee(Long id, String name){}`.

Q5 Record rules/limits?

A: Fields final; record implicitly final; can't extend a class (can implement interfaces). Supports static members + a **compact constructor** for validation.

Q6 Record vs Lombok @Data?

A: Record = native, immutable by design, no library. Lombok = 3rd-party processor for mutable classes. JPA entities need no-arg ctor + mutability → still regular class/Lombok.

Q7 ▲ What are sealed classes?

A: Restrict **which classes may extend/implement** via `permits` → controlled, known hierarchy. Each permitted subclass must be `final`, `sealed`, or `non-sealed`. Pairs with pattern-matching switch (compiler knows all cases).

Q8 ▲ What are switch expressions?

A: `switch` returns a value via arrow syntax — no fall-through, no `break`; use `yield` to return from a block. Exhaustive checks. `case MONDAY, FRIDAY -> "Busy";` .

Q9 ▲ Pattern matching for instanceof?

A: Combines type check + cast: `if (obj instanceof String s) { s.length(); }` — `s` already cast, no boilerplate.

Q10 Pattern matching for switch?

A: Match on type in a switch (preview 17, final 21): `case Circle c -> ...` . With sealed types it's exhaustive → no `default` needed.

Q11 ▲ What are text blocks?

A: Multi-line string literals with `"""` — no `\n` / `+` . Great for JSON, SQL, HTML.

Q12 What is `var` (Java 10)?

A: Local variable type inference — compiler infers type. Only for **locals with an initializer**. Readability, not dynamic typing. `var list = new ArrayList<String>();` .

Q13 What is JPMS (Java 9)?

A: Module system — group packages into modules with explicit dependencies/exported APIs via `module-info.java` . Better encapsulation + smaller runtimes (`jlink`). Most apps still use the classpath.

Q14 Collection factory methods (Java 9)?

A: Immutable one-liners: `List.of(...)` , `Map.of(...)` , `Set.of(...)` .

Q15 Stream/Optional improvements after 8?

A: `Stream.takeWhile/dropWhile/ofNullable` ; `Optional.ifPresentOrElse/or/stream` ; `Collectors.teeing` ; `String.strip/isEmpty/lines/repeat` .

Q16 Helpful NullPointerExceptions (Java 14)?

A: NPE message names the exact null variable — e.g. `Cannot invoke "String.length()" because "name" is null` .

Q17 New HTTP Client (Java 11)?

A: Built-in `java.net.http.HttpClient` — HTTP/2, async via `CompletableFuture`, WebSockets. Replaces old `URLConnection`.

Q18 Single-file Java program (Java 11)?

A: `java Hello.java` runs source directly — no separate `javac` . Good for scripts/learning.

Q19 ▲ Which Java 17 features in a Spring Boot microservice?

A: **Records** for DTOs/request-response · **text blocks** for inline JSON/SQL · **switch expressions** for status→action mapping · **sealed + pattern matching** for closed domain-event sets · **var** for concise locals.

Q20 ▲ Migrate Java 8 → 17?

A: 1) Bump JDK + build tool source/target to 17. 2) Check removed APIs (`javax` → `jakarta` in Spring Boot 3.x, Nashorn, `SecurityManager`). 3) Update deps (Spring Boot 3.x needs 17). 4) Run full test suite. 5) Adopt new features gradually — migrate first, modernize second.

Q21 ★ Honesty framing line:

A: "I've worked extensively with Java 8 — streams, lambdas, Optional, functional interfaces. I've been upskilling on Java 17 — records, sealed classes, pattern matching, switch expressions — and I'm comfortable using them."

MongoDB — Rapid-Recall Flashcards

Cover the **A:** line, say your answer out loud, then check. Full detail in 13-mongodb.md.

▲ = high-yield in every round · ● = from YOUR code. Aim to clear all cards in <10 min as a daily habit.

Q1 ▲ What is MongoDB?

A: A NoSQL, document-oriented DB. Stores flexible JSON-like documents (BSON) in collections. Schema-less, horizontally scalable.

Q2 Document vs Collection?

A: Document = a BSON record ($\approx$ row, can be nested). Collection = group of documents ($\approx$ table, no fixed schema).

Q3 What is BSON?

A: Binary JSON — MongoDB's storage format. Adds types (ObjectId, Date, binary, int/long/decimal); optimized for speed/space.

Q4 What is `_id` ?

A: Auto-added unique primary key. Default = 12-byte ObjectId (has a timestamp); can override with your own value.

Q5 ▲ MySQL vs MongoDB — key differences?

A: Tables/rows vs collections/documents · fixed schema vs flexible · joins/FKs vs embed/reference · vertical vs horizontal (sharding) scaling · SQL vs JSON query language.

Q6 ▲ When choose MongoDB over MySQL?

A: Mongo → flexible/evolving schema, nested data, high write throughput, horizontal scale, catalogs/logs. MySQL → strong relational integrity, complex joins, transactional consistency. Often use **both** (polyglot persistence).

Q7 ACID vs BASE?

A: ACID (relational) = Atomicity, Consistency, Isolation, Durability. BASE (NoSQL)

= Basically Available, Soft state, Eventual consistency. Mongo = ACID per single doc always; multi-doc ACID since v4.0.

Q8 CRUD methods?

A: insertOne/insertMany · find/findOne · updateOne/updateMany (`$set` , `$inc`) · deleteOne/deleteMany.

Q9 Common query operators?

A: Comparison `$eq` `$ne` `$gt` `$gte` `$lt` `$lte` `$in` `$nin` · Logical `$and` `$or` `$not` `$nor` · Element `$exists` `$type` · Array `$all` `$elemMatch` `$size` · Update `$set` `$unset` `$inc` `$push` `$pull` `$addToSet` .

Q10 ▲ Embedding vs Referencing?

A: Embed = nested data in parent → one-to-few, read together, bounded size.
Reference = store other doc's `_id` → one-to-many/many-to-many, large or independently-changing data (needs 2nd lookup / `$lookup`).

Q11 Rule of thumb embed vs reference?

A: Embed when accessed together + bounded ("contains"). Reference when large/shared/unbounded or queried on its own. Mind the **16 MB** document limit.

Q12 Does schema-less mean no design?

A: No — design documents around **query patterns** ("model for how you read").
Flexibility aids evolution; a bad model still hurts.

Q13 ▲ What are indexes?

A: B-tree structures that avoid full collection scans. `createIndex({name:1})` ascending, `{role:1, exp:-1}` compound.

Q14 Types of indexes?

A: Single-field, compound, multikey (arrays), text, geospatial, hashed (sharding), TTL (auto-expire).

Q15 How to analyze query performance?

A: `explain("executionStats")` . Look for `IXSCAN` (good, index used) vs `COLLSCAN` (bad, full scan).

Q16 ▲ What is the aggregation pipeline?

A: Stages that transform docs like a Unix pipe (≈ SQL GROUP BY + functions).
e.g. `$match` → `$group` → `$sort` → `$limit`.

Q17 Common aggregation stages?

A: `$match` `$group` `$project` `$sort` `$limit` `$skip` `$unwind` (flatten arrays)
`$lookup` (join) `$count` `$addFields`.

Q18 Replica set?

A: Group of servers with same data: one primary (writes) + secondaries (copies).
Primary fails → automatic election promotes a secondary → high availability.

Q19 Sharding?

A: Horizontal scaling — split data across shards by a **shard key**; `mongos` router directs queries. For huge datasets / high write throughput.

Q20 Replication vs Sharding?

A: Replication = copies of **same** data → availability + read scaling. Sharding = **splits** data → write scaling + large datasets.

Q21 ▲ Spring Data MongoDB setup?

A: Add `spring-boot-starter-data-mongodb`, set `spring.data.mongodb.uri`,
annotate entities `@Document(collection="...")` with `@Id`, use
`MongoRepository`.

Q22 ▲ What is MongoRepository?

A: Spring Data interface = CRUD + derived query methods (`findByRole`,
`findBySkillsContaining`, `findByExpGreaterThan`). Like `JpaRepository`, for
Mongo.

Q23 ▲ MongoRepository vs MongoTemplate?

A: Repository = high-level derived queries, quick standard CRUD. Template =
low-level, fine-grained control for complex queries, aggregations, bulk ops,
dynamic criteria.

Q24 Aggregations in Spring Data?

A: Via `MongoTemplate` + `Aggregation` builder:

```
Aggregation.newAggregation(match(...), group(...).sum(...).as(...))  
then mongoTemplate.aggregate(...).
```


Q25 Map a MySQL feature to MongoDB?

A: Find the read pattern. Hierarchical + read together (e.g. dashboard w/ nested progress) → embed in one doc = single fast read vs many joins. Shared + large → reference.

Q26 ★★ Honest framing line — REWRITTEN (the old one was wrong):

A: **"Yes — MongoDB is the database on my current project.** RoboGebra's Spring Boot backend uses Spring Data MongoDB: around a hundred `@Document` classes, eighty-five repositories, and twenty-seven `@Aggregation` pipelines. I've also used **Mongoose** on the Node CRM, so I've worked the document model from both Java and JavaScript."

▲ The previous version of this card said *"my production DB experience is MySQL/Postgres/Neo4j"* and treated Mongo as a gap. **That was wrong and it was costing marks.** See [13-mongodb.md](#).

● Your-code cards — the ones that actually win the round ★

Q27 ★★ Tell me about a hard MongoDB problem you solved.

A: **An N+1.** Loading a chapter's exercises then fetching each exercise's items one at a time = $1 + N$ queries — and `@DBRef` makes it easy to fall into, because Spring Data resolves each reference with its own query. Fifty exercises meant fifty-one round trips. I replaced it with one `@Aggregation` pipeline: `$match` on `exercise.$id` with `$in`, then `$sort` by `displayOrder`. **"A document database doesn't remove the N+1 problem — it just changes what it looks like."** ★

Q28 ★ How do you decide embed vs reference?

A: Read together + **bounded** → embed. Shared / large / **unbounded** → reference. And decide **per field, not per entity** ★ — our

`ExerciseSolutionEntity` uses `@DBRef` for the shared exercise item (it changes independently) but **embeds** the AI step output (meaningless outside its solution, always read with it). Hard limit: **16 MB per document**, so never embed an unbounded array.

Q29 ★ What's the cost of @DBRef ?

A: An **extra query per reference** — that's the N+1 trap. It's convenient for modelling, expensive at read time. Where it hurts, replace it with an aggregation pipeline.

Q30 ★ Do you use indexes? Give a real one.

A: Yes — `@CompoundIndex(name = "user_exercise_index", def = "{ 'user.$id': 1, 'exercise.$id': 1 }")` on quiz attempts, because the hot query is always *"this user's attempts at this exercise"*. **Field order matters:** with `user` first the same index also serves *"all attempts by this user"* — but it would **not** serve a query on `exercise` alone (the **prefix rule** ★).

Q31 ★ @Transactional on MongoDB — anything different?

A: Yes. Multi-document transactions need a **replica set** (Mongo 4.0+) and carry real overhead — it's not the free lunch it is on a relational DB. Single-document writes are **always atomic**, so most of our write paths don't need a transaction at all.

Q32 ★ Spring Data MongoDB vs Spring Data JPA — the difference that matters?

A: **No persistence context, no dirty checking, no lazy loading.** In JPA an entity loaded inside a transaction saves itself at commit; in Mongo **you must call `save()` explicitly**. The repository *programming model* is identical — which is why moving between them is easy.

Q33 What does ?0 mean in an @Aggregation pipeline?

A: It binds the **first method parameter** into the pipeline. `"{ '$match': { 'exercise.$id': { '$in': ?0 } } }"` takes the `List<ObjectId>` argument.

Q34 ★ Mongoose — what do you know?

A: The Node ODM, used on our CRM (Mongoose 8.8 + Express). Schema-level **validators**, reusable **plugins**, and **pre / post hooks** — password hashing happens in a `pre('save')`. `.populate()` is the rough equivalent of `@DBRef`. Also `counter.model.js` — the **auto-increment sequence pattern**, since Mongo has no `AUTO_INCREMENT` ★.

Q35 ★ MongoDB numbers you can quote:

A: 104 `@Document` · 85 `MongoRepository` interfaces · 27 `@Aggregation` pipelines

· 130 `@Transactional` · `@CompoundIndex` on the hot paths · plus 10+ Mongoose schemas on the Node CRM.

★ **The one-liner for the whole topic:** *"MongoDB is my current production database, in two languages. The interesting part isn't the CRUD — it's knowing that N+1 and index prefixes still apply, they just look different."*